Книга Ghidra

Русский перевод книги The Ghidra Book: The Definitive Guide.

Больше крутых материалов в моём телеграм-канале @grogrigs

Глава 1. Введение в дизассемблирование

Возможно, вам интересно, чего ожидать от книги, посвященной Ghidra. Хотя в центре внимания, очевидно, находится Ghidra, эта книга не задумывалась как "Руководство пользователя Ghidra". Вместо этого мы собираемся использовать Ghidra как инструмент, позволяющий обсуждать методы обратной разработки, которые пригодятся при анализе самого разного программного обеспечения - от уязвимых приложений до вредоносных программ. Там, где это уместно, мы будем приводить подробные последовательности действий в Ghidra для выполнения конкретных операций, связанных с рассматриваемой задачей. В результате знакомство с возможностями Ghidra пойдет довольно обходным путем: от базовых действий, которые понадобятся при первом изучении файла, до продвинутого применения и настройки Ghidra для более сложных задач обратной разработки. Мы не пытаемся охватить все возможности Ghidra. Однако мы подробно рассмотрим те из них, которые окажутся наиболее полезными при решении ваших задач обратной разработки. Эта книга поможет сделать Ghidra самым мощным средством в вашем наборе инструментов.

Прежде чем переходить к каким-либо особенностям Ghidra, мы рассмотрим основы процесса дизассемблирования и кратко познакомимся с другими инструментами, доступными для обратной разработки скомпилированного кода. Хотя эти инструменты могут не обладать всем диапазоном возможностей Ghidra, каждый из них покрывает отдельные подмножества функциональности Ghidra и дает ценное представление о конкретных возможностях Ghidra. Остальная часть этой главы посвящена высокоуровневому пониманию процесса дизассемблирования.

Теория дизассемблирования

Любой, кто хотя бы немного изучал языки программирования, вероятно, слышал о разных поколениях языков, но для тех, кто, возможно, в тот момент задремал, кратко напомним эту классификацию.

Языки первого поколения - это самый низкий уровень представления языка. Обычно они состоят из единиц и нулей либо из сокращенной записи, например шестнадцатеричной, и понятны только специалистам, привыкшим читать двоичные данные. На этом уровне трудно отличить данные от инструкций, потому что все содержимое выглядит одинаково. Языки первого поколения также могут называться машинными языками, а в некоторых случаях байт-кодом; программы на машинном языке часто называют бинарными файлами.

Языки второго поколения, также называемые языками ассемблера, отделены от машинного языка фактически одной таблицей соответствий. Обычно они сопоставляют конкретные битовые шаблоны, или коды операций (opcodes), с короткими, но запоминающимися последовательностями символов, называемыми мнемониками. Эти мнемоники помогают программистам помнить инструкции, с которыми они связаны. Ассемблер - это инструмент, который программисты используют для перевода программ на языке ассемблера в машинный язык, пригодный для исполнения. Помимо мнемоник инструкций, полноценный язык ассемблера обычно включает директивы для ассемблера, помогающие задавать размещение кода и данных в памяти итогового бинарного файла.

Языки третьего поколения делают еще один шаг к выразительным возможностям естественных языков: в них появляются ключевые слова и конструкции, из которых программисты строят программы. Обычно языки третьего поколения не зависят от платформы, хотя написанные на них программы могут стать платформенно-зависимыми из-за использования возможностей, уникальных для конкретной операционной системы. Среди часто приводимых примеров языков

третьего поколения - FORTRAN, C и Java. Для перевода таких программ на язык ассемблера, непосредственно в машинный язык или в близкую форму наподобие байт-кода программисты обычно используют компиляторы.

Языки четвертого поколения существуют, но к теме этой книги отношения не имеют, поэтому мы не будем их рассматривать.

Что такое дизассемблирование

В традиционной модели разработки программ компиляторы, ассемблеры и компоновщики используются по отдельности или совместно для создания исполняемых программ. Чтобы пройти этот путь в обратном направлении, то есть выполнить обратную разработку программ, мы применяем инструменты, обращающие результаты ассемблирования и компиляции. Неудивительно, что такие инструменты называются дизассемблерами и декомпиляторами, и они делают примерно то, что следует из их названий. Дизассемблер обращает процесс ассемблирования, поэтому на выходе от него следует ожидать язык ассемблера, а на входе - машинный язык. Декомпиляторы стремятся получить представление на языке высокого уровня, принимая на вход ассемблерный или даже машинный код.

Обещание "восстановить исходный код" всегда привлекательно на конкурентном рынке программного обеспечения, поэтому создание пригодных к практическому применению декомпиляторов остается активной областью исследований в информатике. Ниже перечислены лишь некоторые причины, по которым декомпиляция сложна.

Процесс компиляции сопровождается потерей информации. На уровне машинного языка нет имен переменных и функций, а сведения о типе переменной приходится определять по тому, как используются данные, а не по явному объявлению типа. Увидев перенос 32 бит данных, нужно провести небольшое расследование, чтобы понять, представляют ли эти 32 бита целое число, 32-разрядное число с плавающей точкой или 32-разрядный указатель.

Компиляция имеет отношение "многие ко многим". Это означает, что одну исходную программу можно перевести на язык ассемблера множеством способов, а один машинный код можно множеством способов перевести обратно в исходный текст. Поэтому файл, который сначала скомпилировали, а затем сразу декомпилировали, обычно дает исходный файл, сильно отличающийся от оригинала.

Декомпиляторы зависят от языка и библиотек. Обработка бинарного файла, созданного компилятором Delphi, декомпилятором, рассчитанным на получение кода на C, может дать весьма странный результат. Точно так же обработка скомпилированного Windows-бинарника декомпилятором, ничего не знающим о программном API Windows, вряд ли принесет что-то полезное.

Для точной декомпиляции бинарного файла нужна почти идеальная возможность дизассемблирования. Любые ошибки или пропуски на этапе дизассемблирования почти наверняка перейдут в декомпилированный код. Дизассемблированный код можно проверить на корректность по соответствующим справочникам процессора; однако для проверки корректности вывода декомпилятора не существует канонических справочников.

В Ghidra есть встроенный декомпилятор; ему посвящена глава 19.

Зачем нужно дизассемблирование

Назначение средств дизассемблирования часто состоит в том, чтобы помочь понять программы, когда исходный код недоступен. Типичные ситуации, в которых используется дизассемблирование, включают следующее:

• анализ вредоносных программ;
• анализ закрытого программного обеспечения на наличие уязвимостей;
• анализ закрытого программного обеспечения для обеспечения совместимости;
• анализ кода, сгенерированного компилятором, для проверки производительности или корректности компилятора;
• отображение инструкций программы во время отладки.

В следующих разделах каждая из этих ситуаций объясняется подробнее.

Анализ вредоносных программ

Если вы не имеете дело со скриптовыми вредоносными программами, их авторы редко оказывают вам любезность и предоставляют исходный код своих творений. Без исходного кода набор вариантов для точного выяснения поведения вредоносной программы оказывается очень ограниченным. Две основные техники анализа вредоносных программ - динамический анализ и статический анализ. Динамический анализ предполагает выполнение вредоносной программы в тщательно контролируемой среде (песочнице) с записью всех наблюдаемых аспектов ее поведения с помощью любого числа системных средств инструментирования. В отличие от него, статический анализ пытается понять поведение программы простым чтением программного кода, который в случае вредоносной программы обычно состоит только из листинга дизассемблирования и, возможно, листинга декомпилятора.

Анализ уязвимостей

Для упрощения разделим весь процесс аудита безопасности на три этапа: обнаружение уязвимости, анализ уязвимости и разработка эксплойта. Эти же этапы применимы независимо от того, есть ли у вас исходный код; однако объем усилий существенно возрастает, когда в распоряжении имеется только бинарный файл. Первый этап процесса - обнаружить в программе потенциально эксплуатируемое состояние. Часто это делают динамическими методами, например фаззингом[1], но это можно выполнить и статическим анализом - обычно ценой гораздо больших усилий. После обнаружения проблемы часто требуется дальнейший анализ, чтобы определить, можно ли вообще ее эксплуатировать и, если да, то при каких условиях.

Выявление переменных, которыми можно управлять в интересах атакующего, - важный ранний шаг в обнаружении уязвимостей. Листинги дизассемблирования дают уровень детализации, необходимый для точного понимания того, как компилятор решил разместить переменные программы. Например, полезно знать, что объявленный программистом массив символов из 70 байт при размещении был округлен компилятором до 80 байт. Только листинги дизассемблирования позволяют точно установить, в каком порядке компилятор расположил переменные, объявленные глобально или внутри функций. Понимание взаимного расположения переменных в памяти часто необходимо при разработке эксплойтов. В конечном счете совместное применение дизассемблера и отладчика позволяет разработать эксплойт.

Совместимость программ

Если программа распространяется только в двоичном виде, конкурентам очень трудно создать совместимое с ней программное обеспечение или предоставить заменяющие ее плагины. Распространенный пример - код драйвера оборудования, выпущенный только для одной платформы. Если производитель медленно добавляет поддержку других платформ или, что еще хуже, отказывается ее добавлять, для создания подходящих драйверов оборудования может потребоваться значительная работа по обратной разработке. В подобных случаях статический анализ кода остается почти единственным выходом, причем нередко приходится изучать не только драйвер, но и встроенное программное обеспечение.

Проверка компилятора

Поскольку назначение компилятора или ассемблера состоит в создании машинного языка, для проверки соответствия результата проектным требованиям часто нужны качественные средства дизассемблирования. Аналитики также могут искать дополнительные возможности оптимизации выдаваемого компилятором кода, а с точки зрения безопасности - выяснять, не был ли сам компилятор скомпрометирован настолько, что начал внедрять потайные входы в создаваемые программы.

Представление кода при отладке

Вероятно, чаще всего дизассемблеры используются для создания листингов внутри отладчиков. К сожалению, встроенные в отладчики дизассемблеры обычно довольно примитивны. Как правило, они не способны выполнять пакетное дизассемблирование, а иногда отказываются дизассемблировать код, если не могут определить границы функции. Поэтому отладчик лучше применять вместе с качественным дизассемблером: это дает больше контекста и позволяет лучше понимать происходящее во время отладки.

Как работает дизассемблирование

Теперь, когда назначение дизассемблирования вам хорошо известно, пора разобраться в самом процессе. Представьте типичную непростую задачу дизассемблера: "Возьми эти 100 Кбайт, отличи код от данных, переведи код на язык ассемблера для показа пользователю и, пожалуйста, ничего не пропусти по пути". К этому можно добавить сколько угодно особых пожеланий: найти функции, распознать таблицы переходов, определить локальные переменные - и тем самым еще сильнее усложнить работу дизассемблера.

Чтобы выполнить все наши требования, любому дизассемблеру приходится выбирать различные алгоритмы при обработке переданных ему файлов. Качество полученного листинга дизассемблирования непосредственно зависит от качества использованных алгоритмов и правильности их реализации.

В этом разделе мы обсудим два основных алгоритма, применяемых сегодня для дизассемблирования машинного кода. Представляя эти алгоритмы, мы также отметим их недостатки, чтобы подготовить вас к ситуациям, в которых дизассемблер, кажется, дает сбой. Понимание ограничений дизассемблера позволит вмешаться вручную и повысить общее качество результата дизассемблирования.

Простейший алгоритм дизассемблирования

Для начала разработаем простой алгоритм, который принимает машинный язык на вход и выдает язык ассемблера на выходе. Это поможет понять трудности, допущения и компромиссы, лежащие в основе автоматического дизассемблирования.

1. Первым делом в процессе дизассемблирования нужно определить область кода, которую предстоит дизассемблировать. Это не обязательно так просто, как кажется. Инструкции обычно перемешаны с данными, и важно отличать одно от другого. В наиболее распространенном случае, при дизассемблировании исполняемого файла, файл соответствует распространенному формату исполняемых файлов, например формату переносимых исполняемых файлов `Portable Executable` (PE), применяемому в Windows, или формату исполняемых и компонуемых файлов (ELF), распространенному во многих Unix-подобных системах. Обычно в таких форматах предусмотрены механизмы - часто в виде иерархических заголовков файла, - позволяющие найти разделы файла, содержащие код, и точки входа[2] в этот код.
2. Получив адрес инструкции, нужно прочитать значение или значения, находящиеся по этому адресу или смещению в файле, и найти в таблице соответствующую двоичному коду операции мнемонику языка ассемблера. В зависимости от сложности дизассемблируемого набора инструкций это может быть как простая операция, так и последовательность дополнительных действий: распознавание префиксов, изменяющих поведение инструкции, и определение требуемых операндов. В наборах с инструкциями переменной длины, таких как Intel x86, для полного разбора одной инструкции может потребоваться прочитать дополнительные байты.
3. После извлечения инструкции и декодирования всех требуемых операндов ее эквивалент на языке ассемблера форматируется и выводится как часть листинга дизассемблирования. Может существовать возможность выбрать один из нескольких синтаксисов вывода языка ассемблера. Например, два преобладающих формата для ассемблера x86 - формат Intel и формат AT&T.
4. После вывода инструкции нужно перейти к следующей инструкции и повторять предыдущий процесс, пока не будут дизассемблированы все инструкции в файле.

Синтаксис ассемблера x86: AT&T и Intel

Для исходного кода на ассемблере используются два основных синтаксиса: AT&T и Intel. Хотя оба относятся к языкам второго поколения, их синтаксис сильно различается - от доступа к переменным, константам и регистрам до переопределения сегментов и размера инструкций, косвенной адресации и смещений. Синтаксис AT&T отличается использованием символа `%` перед именами всех регистров, символа `$` перед литеральными константами, также называемыми непосредственными операндами, и порядком операндов, при котором исходный операнд находится слева, а операнд назначения - справа. В синтаксисе AT&T инструкция добавления 4 к регистру `EAX` будет выглядеть как `add $0x4,%eax`. GNU Assembler (`as`) и многие другие инструменты GNU, включая `gcc` и `gdb`, по умолчанию используют синтаксис AT&T.

Синтаксис Intel отличается от AT&T тем, что не требует префиксов для регистров или литералов, а порядок операндов в нем обратный: исходный операнд находится справа, а операнд назначения - слева. Та же инструкция `add` в синтаксисе Intel будет выглядеть как `add eax,0x4`. Среди ассемблеров, использующих синтаксис Intel, - Microsoft Assembler (MASM) и Netwide Assembler (NASM).

Существуют различные алгоритмы для определения того, где начинать дизассемблирование, как выбрать следующую инструкцию для дизассемблирования, как отличать код от данных и как определить, что последняя инструкция уже дизассемблирована. Два преобладающих алгоритма дизассемблирования - линейный проход и рекурсивный спуск.

Дизассемблирование линейным проходом

Алгоритм дизассемблирования линейным проходом использует очень прямолинейный подход к поиску инструкций для дизассемблирования: там, где заканчивается одна инструкция, начинается другая. В результате самые трудные решения - где начать и когда остановиться. Обычно предполагают, что все содержимое разделов программы, помеченных как код и обычно заданных заголовками файла программы, представляет собой инструкции машинного языка. Дизассемблирование начинается с первого байта в разделе кода и линейно проходит по разделу, дизассемблируя одну инструкцию за другой, пока не будет достигнут конец раздела. Никаких попыток понять поток управления программы путем распознавания нелинейных инструкций, таких как ветвления, не предпринимается.

Во время дизассемблирования можно поддерживать указатель, отмечающий начало текущей дизассемблируемой инструкции. В ходе процесса вычисляется длина каждой инструкции, и она используется для определения расположения следующей инструкции, которую нужно дизассемблировать. Наборы инструкций фиксированной длины, например MIPS, несколько легче дизассемблировать, поскольку поиск последующих инструкций в них прямолинеен.

Главное преимущество алгоритма линейного прохода состоит в том, что он обеспечивает полное покрытие разделов кода программы. Один из основных недостатков метода линейного прохода - он не учитывает данные, которые могут быть перемешаны с кодом. Это видно в листинге 1-1, где показан вывод функции, дизассемблированной дизассемблером линейного прохода.

```
40123f:   55                      push ebp
401240:   8b ec                   mov ebp,esp
401242:   33 c0                   xor eax,eax
401244:   8b 55 08                mov edx,DWORD PTR [ebp+8]
401247:   83 fa 0c                cmp edx,0xc
40124a:   0f 87 90 00 00 00       ja 0x4012e0
401250:   ff 24 95 57 12 40 00    jmp DWORD PTR [edx*4+0x401257]
401257:   e0 12                   loopne 0x40126b
401259:   40                      inc eax
40125a:   00 8b 12 40 00 90       add BYTE PTR [ebx-0x6fffbfee],cl
401260:   12 40 00                adc al,BYTE PTR [eax]
401263:   95                      xchg ebp,eax
401264:   12 40 00                adc al,BYTE PTR [eax]
401267:   9a 12 40 00 a2 12 40    call 0x4012:0xa2004012
40126e:   00 aa 12 40 00 b2       add BYTE PTR [edx-0x4dffbfee],ch
401274:   12 40 00                adc al,BYTE PTR [eax]
401277:   ba 12 40 00 c2          mov edx,0xc2004012
40127c:   12 40 00                adc al,BYTE PTR [eax]
40127f:   ca 12 40                lret 0x4012
401282:   00 d2                   add dl,dl
401284:   12 40 00                adc al,BYTE PTR [eax]
401287:   da 12                   ficom DWORD PTR [edx]
401289:   40                      inc eax
40128a:   00 8b 45 0c eb 50       add BYTE PTR [ebx+0x50eb0c45],cl
401290:   8b 45 10                mov eax,DWORD PTR [ebp+16]
401293:   eb 4b                   jmp 0x4012e0
```

Листинг 1-1. Дизассемблирование линейным проходом

Эта функция содержит оператор `switch`, и использованный в данном случае компилятор решил реализовать `switch` с помощью таблицы переходов для определения целевых адресов меток `case`. Более того, компилятор решил встроить таблицу переходов в саму функцию. Оператор `jmp`

ссылается на таблицу адресов. К сожалению, дизассемблер воспринимает таблицу адресов как последовательность инструкций и ошибочно создает приведенное выше представление на языке ассемблера.

Если рассматривать последовательные 4-байтовые группы в таблице переходов как значения с прямым порядком байтов[3], видно, что каждая из них представляет указатель на расположенный рядом адрес, который фактически является назначением одного из разных переходов (`004012e0`, `0040128b`, `00401290`, ...). Следовательно, инструкция `loopne` на самом деле вовсе не инструкция. Напротив, она указывает на сбой алгоритма линейного прохода: он не смог правильно отличить встроенные данные от кода.

Линейный проход используется движками дизассемблирования, встроенными в GNU Debugger (`gdb`), отладчик Microsoft WinDbg и утилиту `objdump`.

Дизассемблирование рекурсивным спуском

Алгоритм дизассемблирования рекурсивным спуском использует другой подход к поиску инструкций: он опирается на понятие потока управления, которое определяет, следует ли дизассемблировать инструкцию, исходя из того, ссылается ли на нее другая инструкция. Чтобы понять рекурсивный спуск, полезно классифицировать инструкции по тому, как они влияют на указатель инструкций.

Инструкции последовательного потока

Инструкции последовательного потока передают выполнение инструкции, которая непосредственно следует за ними. Примеры таких инструкций включают простые арифметические инструкции, например `add`; инструкции передачи данных между регистром и памятью, например `mov`; и операции со стеком, например `push` и `pop`. Для таких инструкций дизассемблирование идет так же, как при линейном проходе.

Инструкции условного ветвления

Инструкции условного ветвления, такие как x86 `jnz`, предлагают два возможных пути выполнения. Если условие истинно, ветвление выполняется, и указатель инструкций должен измениться, отражая цель перехода. Однако если условие ложно, выполнение продолжается линейно, и для дизассемблирования следующей инструкции можно использовать методику линейного прохода. Поскольку в статическом контексте обычно невозможно определить результат условной проверки, алгоритм рекурсивного спуска дизассемблирует оба пути, откладывая дизассемблирование целевой инструкции ветвления путем добавления ее адреса в список адресов, которые нужно дизассемблировать позднее.

Инструкции безусловного ветвления

Безусловные ветвления не следуют модели линейного потока, поэтому алгоритм рекурсивного спуска обрабатывает их иначе. Как и в случае с инструкциями последовательного потока, выполнение может перейти только к одной инструкции; однако эта инструкция не обязана непосредственно следовать за инструкцией ветвления. Фактически, как видно в листинге 1-1, нет вообще никакого требования, чтобы за безусловным переходом сразу следовала инструкция. Поэтому нет причин немедленно дизассемблировать байты, идущие после безусловного ветвления.

Дизассемблер рекурсивного спуска пытается определить цель безусловного перехода и продолжает дизассемблирование с целевого адреса. К сожалению, некоторые безусловные ветвления могут создавать проблемы для дизассемблеров рекурсивного спуска. Когда цель инструкции перехода зависит от значения времени выполнения, определить назначение перехода статическим анализом может быть невозможно. Инструкция x86 `jmp rax` демонстрирует эту проблему. Регистр `rax` содержит значение только тогда, когда программа фактически выполняется. Поскольку при статическом анализе в регистре нет значения, у нас нет способа определить цель инструкции перехода и, следовательно, нет способа понять, где продолжать процесс дизассемблирования.

Инструкции вызова функций

Инструкции вызова функций работают похоже на инструкции безусловного перехода, включая неспособность дизассемблера определить цель инструкций вроде `call rax`, но с дополнительным ожиданием: после завершения функции выполнение обычно возвращается к инструкции, непосредственно следующей за инструкцией вызова. В этом отношении они похожи на инструкции условного ветвления, потому что создают два пути выполнения. Целевой адрес инструкции вызова добавляется в список для отложенного дизассемблирования, а инструкция, непосредственно следующая за вызовом, дизассемблируется способом, похожим на линейный проход.

Рекурсивный спуск может дать сбой, если программы ведут себя при возврате из вызванных функций не так, как ожидается. Например, код функции может намеренно изменить адрес возврата этой функции так, что после завершения управление вернется в место, отличное от ожидаемого дизассемблером. Простой пример показан в следующем неверном листинге, где функция `badfunc` перед возвратом вызывающему коду просто добавляет 1 к адресу возврата:

```
badfunc proc near
48 FF 04 24  inc qword ptr [rsp] ; увеличивает сохраненный адрес возврата
C3           retn
badfunc endp
; ------------------------------------
label:
E8 F6 FF FF FF   call badfunc
05 48 89 45 F8   add eax, F8458948h
```

В результате управление фактически не переходит к инструкции `add`, следующей за вызовом `badfunc`. Правильное дизассемблирование выглядит так:

```
badfunc proc near
48 FF 04 24  inc qword ptr [rsp]
C3           retn
badfunc endp
; ------------------------------------
label:
E8 F6 FF FF FF  call badfunc
05              db 5        ;ранее первый байт инструкции add
48 89 45 F8     mov [rbp-8], rax
```

Этот листинг яснее показывает поток выполнения программы, в котором функция `badfunc` фактически возвращает управление к инструкции `mov`. Важно понимать, что дизассемблер линейного прохода также не сможет правильно дизассемблировать этот код, хотя и по несколько иным причинам.

Инструкции возврата

В некоторых случаях у алгоритма рекурсивного спуска заканчиваются пути, по которым можно следовать. Инструкция возврата из функции, например x86 `ret`, не дает сведений о том, какая инструкция будет выполнена следующей. Если бы программа действительно выполнялась, адрес был бы взят с вершины стека времени выполнения, и выполнение продолжилось бы с этого адреса. У дизассемблеров нет доступа к стеку. Вместо этого дизассемблирование резко останавливается. Именно в этот момент дизассемблер рекурсивного спуска обращается к списку адресов, которые он откладывал для последующего дизассемблирования. Адрес удаляется из этого списка, и процесс дизассемблирования продолжается с него. Это и есть рекурсивный процесс, давший название алгоритму дизассемблирования.

Одно из главных преимуществ алгоритма рекурсивного спуска - его лучшая способность отличать код от данных. Поскольку этот алгоритм основан на потоке управления, он гораздо реже ошибочно дизассемблирует значения данных как код. Главный недостаток рекурсивного спуска - неспособность следовать по непрямым путям кода, таким как переходы или вызовы, которые используют таблицы указателей для поиска целевого адреса. Однако при добавлении некоторых эвристик для выявления указателей на код дизассемблеры рекурсивного спуска могут обеспечивать очень полное покрытие кода и превосходное распознавание кода и данных. В листинге 1-2 показан вывод дизассемблера рекурсивного спуска Ghidra, примененного к тому же оператору `switch`, который ранее был показан в листинге 1-1.

```
0040123f  PUSH   EBP
00401240  MOV    EBP,ESP
00401242  XOR    EAX,EAX
00401244  MOV    EDX,dword ptr [EBP + param_1]
00401247  CMP    EDX,0xc
0040124a  JA     switchD_00401250::caseD_0
        switchD_00401250::switchD
00401250  JMP    dword ptr [EDX*0x4 + ->switchD_00401250::caseD_0] = 004012e0
        switchD_00401250::switchdataD_00401257
00401257  addr   switchD_00401250::caseD_0
0040125b  addr   switchD_00401250::caseD_1
0040125f  addr   switchD_00401250::caseD_2
00401263  addr   switchD_00401250::caseD_3
00401267  addr   switchD_00401250::caseD_4
0040126b  addr   switchD_00401250::caseD_5
0040126f  addr   switchD_00401250::caseD_6
00401273  addr   switchD_00401250::caseD_7
00401277  addr   switchD_00401250::caseD_8
0040127b  addr   switchD_00401250::caseD_9
0040127f  addr   switchD_00401250::caseD_a
00401283  addr   switchD_00401250::caseD_b
```

```
00401287  addr   switchD_00401250::caseD_c
        switchD_00401250::caseD_1
0040128b  MOV    EAX,dword ptr [EBP + param_2]
0040128e  JMP    switchD_00401250::caseD_00040128E
```

Листинг 1-2. Дизассемблирование рекурсивным спуском

Обратите внимание, что этот участок бинарного файла распознан как оператор `switch` и отформатирован соответствующим образом. Понимание процесса рекурсивного спуска поможет нам распознавать ситуации, в которых Ghidra может создавать неоптимальные дизассемблированные представления, и позволит вырабатывать стратегии улучшения вывода Ghidra.

Итоги

Нужно ли глубокое понимание алгоритмов дизассемблирования при использовании дизассемблера? Нет. Полезно ли оно? Да! Борьба с инструментами - последнее, на что хочется тратить время во время обратной разработки. Одно из многочисленных преимуществ Ghidra состоит в том, что, будучи интерактивным дизассемблером, она дает много возможностей направлять и переопределять ее решения. В результате часто получается дизассемблирование, которое одновременно является подробным и точным.

В следующей главе мы рассмотрим различные существующие инструменты, полезные во многих ситуациях обратной разработки. Хотя они не связаны с Ghidra напрямую, многие из них повлияли на Ghidra и помогают объяснить большое разнообразие информационных представлений, доступных в пользовательском интерфейсе Ghidra.

Сноски

[1] [назад] Фаззинг - метод обнаружения уязвимостей, при котором для программы создают большое количество различных входных данных в надежде, что один из вариантов вызовет обнаруживаемый сбой, который затем удастся проанализировать и использовать.
[2] [назад] Точка входа программы - это просто адрес инструкции, которой операционная система передает управление после загрузки программы в память.
[3] [назад] x86 - архитектура с прямым порядком байтов, то есть младший байт многобайтового значения данных хранится первым, по меньшему адресу памяти, чем каждый последующий байт этого элемента данных. Данные с обратным порядком байтов хранятся наоборот: старший байт значения данных размещается по меньшему адресу памяти, чем каждый последующий байт. Процессоры могут классифицироваться как использующие прямой порядок байтов, обратный порядок байтов или, в некоторых случаях, оба порядка.

Глава 2. Инструменты обратной разработки и дизассемблирования

Разобравшись с некоторыми основами дизассемблирования, но прежде чем погружаться в особенности Ghidra, полезно понять, какие еще инструменты применяются для обратной разработки бинарных файлов. Многие из этих инструментов появились раньше Ghidra и по-прежнему полезны как для быстрого взгляда на файлы, так и для перепроверки работы, которую выполняет Ghidra. Как мы увидим, Ghidra объединяет многие возможности таких инструментов в своем пользовательском интерфейсе, предоставляя единую интегрированную среду для обратной разработки.

Инструменты классификации

Когда впервые сталкиваешься с неизвестным файлом, часто полезно ответить на простые вопросы вроде: "Что это такое?" Первое практическое правило при попытке ответить на этот вопрос - никогда не полагаться на расширение файла, определяя, чем файл является на самом деле. Это же второе, третье и четвертое практические правила. Как только вы станете сторонником идеи, что расширения файлов ничего не значат, вам, возможно, стоит познакомиться с одной или несколькими следующими утилитами.

file

Команда `file` - стандартная утилита, входящая в большинство операционных систем в стиле *nix, а также в подсистему Windows для Linux (WSL)[1]. Эта команда также доступна пользователям Windows после установки Cygwin или MinGW[2]. Команда `file` пытается определить тип файла, исследуя конкретные поля внутри него. В некоторых случаях `file` распознает распространенные строки, такие как `#!/bin/sh` (shell-скрипт) и `<html>` (HTML-документ).

Файлы, содержащие не-ASCII-содержимое, представляют несколько большую сложность. В таких случаях `file` пытается определить, похоже ли содержимое на структуру известного формата файла. Во многих случаях она ищет конкретные значения меток, часто называемые магическими числами[3], которые, как известно, уникальны для определенных типов файлов. Следующие шестнадцатеричные листинги показывают несколько примеров магических чисел, используемых для идентификации некоторых распространенных типов файлов.

Исполняемый файл Windows PE:

```
00000000 4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00 MZ..............
00000010 B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00 ........@.......
```

Файл изображения JPEG:

```
00000000 FF D8 FF E0 00 10 4A 46 49 46 00 01 01 01 00 60 ......JFIF.....`
00000010 00 60 00 00 FF DB 00 43 00 0A 07 07 08 07 06 0A .`.....C........
```

Файл Java `.class`:

```
00000000 CA FE BA BE 00 00 00 32 00 98 0A 00 2E 00 3E 08 .......2......>.
00000010 00 3F 09 00 40 00 41 08 00 42 0A 00 43 00 44 0A .?..@.A..B..C.D.
```

Команда `file` способна идентифицировать множество форматов файлов, включая несколько типов ASCII-текстовых файлов, а также различные исполняемые форматы и форматы файлов данных. Проверки магических чисел, выполняемые `file`, управляются правилами, содержащимися в magic-файле. Стандартный magic-файл зависит от операционной системы, но типичные расположения включают `/usr/share/file/magic`, `/usr/share/misc/magic` и `/etc/magic`. Дополнительные сведения о magic-файлах см. в документации к `file`.

В некоторых случаях `file` умеет различать варианты внутри заданного типа файла. Следующий листинг демонстрирует способность `file` идентифицировать не только несколько вариантов ELF-бинарников, но и сведения о том, как бинарный файл был скомпонован - статически или динамически, - а также был ли он очищен от символов.

```
ghidrabook# file ch2_ex_*
  ch2_ex_x64:        ELF 64-bit LSB shared object, x86-64, version 1 (SYSV),
                     dynamically linked, interpreter /lib64/l, for GNU/Linux
                     3.2.0, not stripped
  ch2_ex_x64_dbg:    ELF 64-bit LSB shared object, x86-64, version 1 (SYSV),
                     dynamically linked, interpreter /lib64/l, for GNU/Linux
                     3.2.0, with debug_info, not stripped
  ch2_ex_x64_static: ELF 64-bit LSB executable, x86-64, version 1 (GNU/Linux),
                     statically linked, for GNU/Linux 3.2.0, not stripped
  ch2_ex_x64_strip:  ELF 64-bit LSB shared object, x86-64, version 1 (SYSV),
                     dynamically linked, interpreter /lib64/l, for GNU/Linux
                     3.2.0, stripped
  ch2_ex_x86:        ELF 32-bit LSB shared object, Intel 80386, version 1
                     (SYSV), dynamically linked, interpreter /lib/ld-, for
                     GNU/Linux 3.2.0, not stripped
  ch2_ex_x86_dbg:    ELF 32-bit LSB shared object, Intel 80386, version 1
                     (SYSV), dynamically linked, interpreter /lib/ld-, for
                     GNU/Linux 3.2.0, with debug_info, not stripped
  ch2_ex_x86_static: ELF 32-bit LSB executable, Intel 80386, version 1
                     (GNU/Linux), statically linked, for GNU/Linux 3.2.0,
                     not stripped
  ch2_ex_x86_strip:  ELF 32-bit LSB shared object, Intel 80386, version 1
                     (SYSV), dynamically linked, interpreter /lib/ld-, for
                     GNU/Linux 3.2.0, stripped
  ch2_ex_Win32:      PE32 executable (console) Intel 80386, for MS Windows
  ch2_ex_x64:        PE32+ executable (console) x86-64, for MS Windows
```

Среда WSL

Подсистема Windows для Linux предоставляет командную среду GNU/Linux непосредственно внутри Windows без необходимости создавать виртуальную машину. Во время установки WSL пользователи выбирают дистрибутив Linux и затем могут запускать его в WSL. Это дает доступ к распространенному свободному программному обеспечению командной строки (`grep`, `awk`), компиляторам (`gcc`, `g++`), интерпретаторам (Perl, Python, Ruby), сетевым утилитам (`nc`, `ssh`) и многим другим средствам. После установки WSL многие программы, написанные для использования в Linux, можно компилировать и выполнять в системах Windows.

Утилита `file` и похожие утилиты не безошибочны. Файл вполне может быть определен неверно просто потому, что случайно содержит распознавательные признаки конкретного формата. Это легко проверить самостоятельно: с помощью шестнадцатеричного редактора измените первые 4

байта любого файла на последовательность магического числа Java: `CA FE BA BE`. Утилита `file` ошибочно определит недавно измененный файл как скомпилированные данные класса Java. Точно так же текстовый файл, содержащий только два символа `MZ`, будет определен как исполняемый файл MS-DOS. Хороший подход в любой работе по обратной разработке - никогда полностью не доверять выводу какого-либо инструмента, пока вы не сопоставили этот вывод с несколькими инструментами и ручным анализом.

Очистка бинарных исполняемых файлов от символов

Очистка бинарного файла, или stripping, - это процесс удаления символов из бинарного файла. Бинарные объектные файлы содержат символы как результат компиляции. Некоторые из этих символов используются во время компоновки для разрешения ссылок между файлами при создании итогового исполняемого файла или библиотеки. В других случаях символы могут присутствовать, чтобы предоставить дополнительные сведения для отладчиков. После компоновки многие символы больше не требуются. Параметры, переданные компоновщику, могут заставить его удалить ненужные символы во время сборки. Кроме того, для удаления символов из существующих бинарных файлов можно использовать утилиту `strip`. Хотя очищенный бинарный файл будет меньше своего неочищенного аналога, поведение очищенного бинарного файла останется неизменным.

PE Tools

PE Tools - набор инструментов, полезных для анализа как работающих процессов, так и исполняемых файлов в системах Windows[4]. На рисунке 2-1 показан основной интерфейс PE Tools: он отображает список активных процессов и предоставляет доступ ко всем утилитам PE Tools.

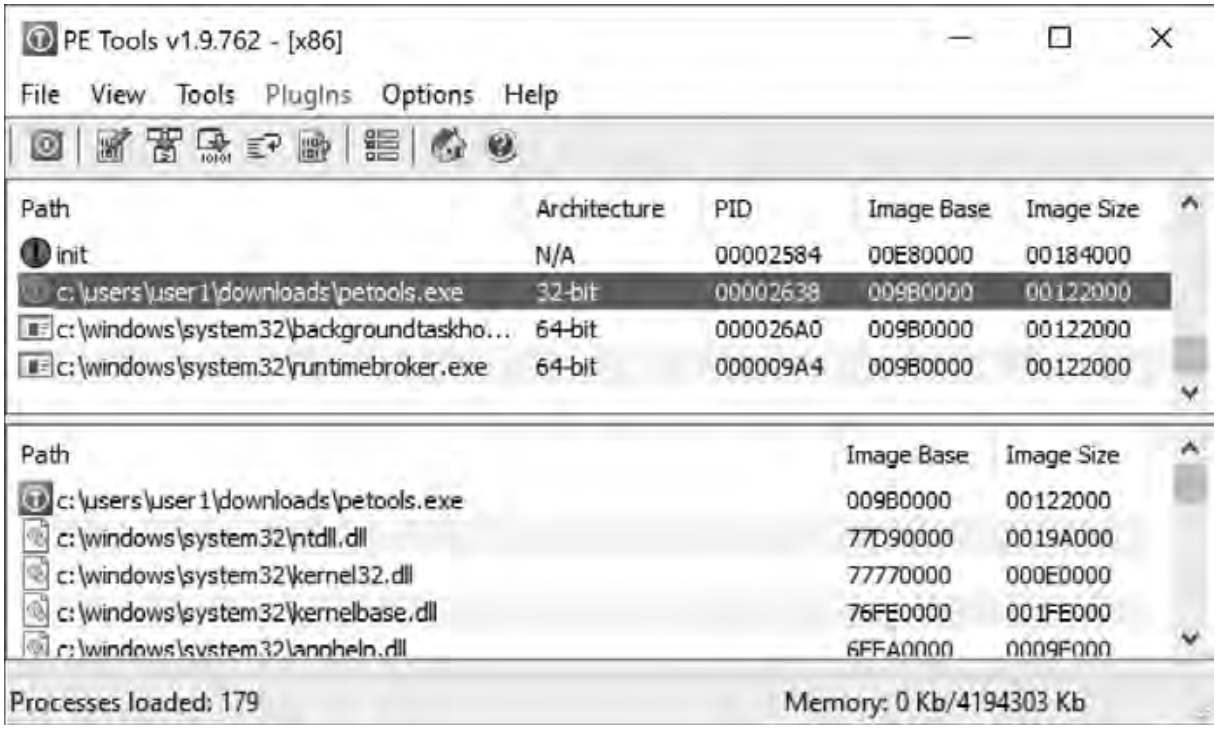


Рисунок 2-1. Утилита PE Tools

Из списка процессов пользователи могут выгрузить образ памяти процесса в файл или воспользоваться утилитой PE Sniffer, чтобы определить, каким компилятором был создан исполняемый файл или обрабатывался ли он какими-либо известными утилитами обфускации. Меню Tools предлагает похожие варианты анализа файлов на диске. Пользователи могут

просматривать поля PE-заголовка файла с помощью встроенной утилиты PE Editor, которая также позволяет легко изменять любые значения заголовка. Изменение PE-заголовков часто требуется при попытке восстановить корректный PE-файл из его обфусцированной версии.

Обфускация бинарных файлов

Обфускация - это любая попытка скрыть истинный смысл чего-либо. Применительно к исполняемым файлам обфускация означает любую попытку скрыть настоящее поведение программы. Программисты могут применять обфускацию по разным причинам. Среди часто приводимых примеров - защита проприетарных алгоритмов и сокрытие вредоносного намерения. Почти все виды вредоносных программ используют обфускацию, пытаясь затруднить анализ. Широко доступны инструменты, помогающие авторам программ создавать обфусцированные программы. Инструменты и методы обфускации, а также их влияние на процесс обратной разработки подробнее обсуждаются в главе 21.

PEiD

PEiD - еще один инструмент Windows, основные задачи которого состоят в определении компилятора, использованного для сборки конкретного Windows PE-бинарника, и в выявлении любых инструментов, использованных для обфускации Windows PE-бинарника[5]. На рисунке 2-2 показано применение PEiD для определения инструмента - в данном случае ASPack, - использованного для обфускации варианта червя Gaobot[6].

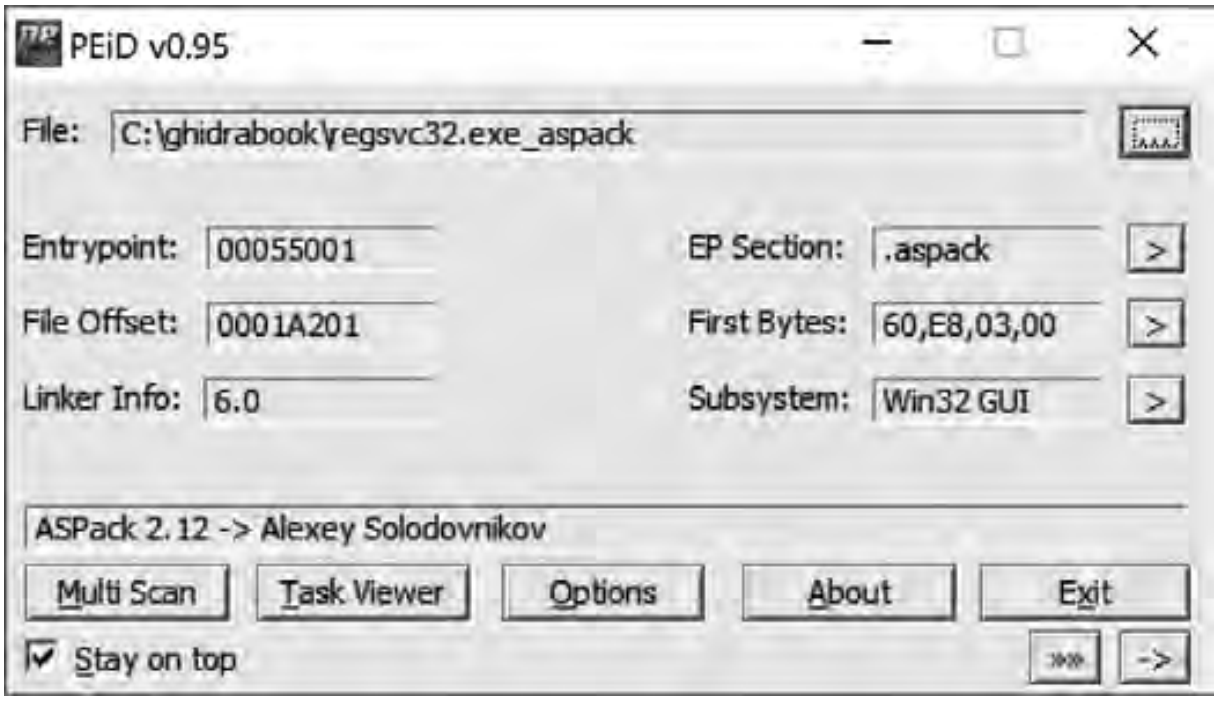


Рисунок 2-2. Утилита PEiD

Многие дополнительные возможности PEiD пересекаются с возможностями PE Tools, включая способность кратко описывать PE-заголовки файла, собирать сведения о работающих процессах и выполнять базовое дизассемблирование.

Обзорные инструменты

Поскольку наша цель - выполнять обратную разработку бинарных программных файлов, после первоначальной классификации файла нам понадобятся более сложные инструменты для извлечения подробных сведений. Инструменты, обсуждаемые в этом разделе, по необходимости гораздо лучше осведомлены о форматах файлов, которые они обрабатывают. В большинстве случаев такие инструменты понимают вполне конкретный формат файла и используются для разбора входных файлов с целью извлечь очень конкретную информацию.

nm

Когда исходные файлы компилируются в объектные файлы, компиляторы должны встроить сведения о расположении любых глобальных, то есть внешних, символов, чтобы компоновщик мог разрешить ссылки на эти символы при объединении объектных файлов в исполняемый файл. Если компоновщику не указано удалить символы из итогового исполняемого файла, он обычно переносит символы из объектных файлов в получившийся исполняемый файл. Согласно man-странице, утилита `nm` "перечисляет символы из объектных файлов".

Когда `nm` используется для изучения промежуточного объектного файла - файла `.o`, а не исполняемого файла, - стандартный вывод дает имена всех функций и глобальных переменных, объявленных в файле. Пример вывода утилиты `nm` показан ниже:

```
ghidrabook# gcc -c ch2_nm_example.c
ghidrabook# nm ch2_nm_example.o
                 U exit
                 U fwrite
  000000000000002e t get_max
                 U _GLOBAL_OFFSET_TABLE_
                 U __isoc99_scanf
  00000000000000a6 T main
  0000000000000000 D my_initialized_global
  0000000000000004 C my_uninitialized_global
                 U printf
                 U puts
                 U rand
                 U srand
                 U __stack_chk_fail
                 U stderr
                 U time
  0000000000000000 T usage
ghidrabook#
```

Здесь видно, что `nm` перечисляет каждый символ вместе со сведениями о нем. Буквенные коды используются для указания типа перечисляемого символа. В этом примере встречаются следующие буквенные коды:

`U` - неопределенный символ, обычно ссылка на внешний символ.

`T` - символ, определенный в секции text, обычно имя функции.

`t` - локальный символ, определенный в секции text. В программе на C это обычно соответствует статической функции.

`D` - инициализированное значение данных.

`C` - неинициализированное значение данных.

Примечание

> Буквенные коды в верхнем регистре используются для глобальных символов, тогда как буквенные коды в нижнем регистре используются для локальных символов. Дополнительные сведения, включая полное объяснение буквенных кодов, можно найти на man-странице `nm`.

Несколько больше сведений отображается, когда `nm` используется для вывода символов из исполняемого файла. Во время компоновки символы, когда это возможно, разрешаются в виртуальные адреса, в результате чего при запуске `nm` становится доступно больше информации. Ниже показан сокращенный пример вывода `nm`, примененной к исполняемому файлу:

```
ghidrabook# gcc -o ch2_nm_example ch2_nm_example.c
ghidrabook# nm ch2_nm_example
  ...
                 U fwrite@@GLIBC_2.2.5
  0000000000000938 t get_max
  0000000000201f78 d _GLOBAL_OFFSET_TABLE_
                 w __gmon_start__
  0000000000000c5c r __GNU_EH_FRAME_HDR
  0000000000000730 T _init
  0000000000201d80 t __init_array_end
  0000000000201d78 t __init_array_start
  0000000000000b60 R _IO_stdin_used
                 U __isoc99_scanf@@GLIBC_2.7
                 w _ITM_deregisterTMCloneTable
                 w _ITM_registerTMCloneTable
  0000000000000b50 T __libc_csu_fini
  0000000000000ae0 T __libc_csu_init
                 U __libc_start_main@@GLIBC_2.2.5
  00000000000009b0 T main
  0000000000202010 D my_initialized_global
  000000000020202c B my_uninitialized_global
                 U printf@@GLIBC_2.2.5
                 U puts@@GLIBC_2.2.5
                 U rand@@GLIBC_2.2.5
  0000000000000870 t register_tm_clones
                 U srand@@GLIBC_2.2.5
                 U __stack_chk_fail@@GLIBC_2.4
  0000000000000800 T _start
  0000000000202020 B stderr@@GLIBC_2.2.5
                 U time@@GLIBC_2.2.5
  0000000000202018 D __TMC_END__
  000000000000090a T usage
ghidrabook#
```

К этому моменту некоторым символам, например `main`, назначены виртуальные адреса; новые символы, такие как `__libc_csu_init`, появились в результате процесса компоновки; у некоторых, например `my_unitialized_global`, изменился тип символа; а другие остаются неопределенными, поскольку продолжают ссылаться на внешние символы. В данном случае исследуемый бинарный файл скомпонован динамически, а неопределенные символы определены в разделяемой библиотеке C.

ldd

Когда создается исполняемый файл, нужно разрешить расположение всех библиотечных функций, на которые он ссылается. У компоновщика есть два метода разрешения вызовов библиотечных функций: статическая компоновка и динамическая компоновка. Аргументы командной строки, переданные компоновщику, определяют, какой из двух методов используется. Исполняемый файл может быть скомпонован статически, динамически или обоими способами[7].

При запросе статической компоновки компоновщик объединяет объектные файлы приложения с копией требуемой библиотеки, создавая исполняемый файл. Во время выполнения не нужно

искать библиотечный код, потому что он уже содержится внутри исполняемого файла. Преимущества статической компоновки состоят в том, что (1) она приводит к немного более быстрым вызовам функций и (2) упрощает распространение бинарных файлов, поскольку не нужно делать предположений о наличии библиотечного кода в системах пользователей. Недостатки статической компоновки включают (1) более крупные итоговые исполняемые файлы и (2) большую сложность обновления программ при изменении библиотечных компонентов. Программы сложнее обновлять, потому что их нужно перекомпоновывать каждый раз при изменении библиотеки. С точки зрения обратной разработки статическая компоновка несколько усложняет ситуацию. Если перед нами стоит задача анализа статически скомпонованного бинарного файла, нет простого способа ответить на вопросы "Какие библиотеки скомпонованы в этот бинарный файл?" и "Какие из этих функций являются библиотечными?" В главе 13 обсуждаются трудности, возникающие при обратной разработке статически скомпонованного кода.

Динамическая компоновка отличается от статической тем, что компоновщику не нужно создавать копию каких-либо требуемых библиотек. Вместо этого компоновщик просто вставляет ссылки на все требуемые библиотеки, часто файлы `.so` или `.dll`, в итоговый исполняемый файл, что обычно приводит к гораздо меньшим исполняемым файлам. При динамической компоновке обновлять библиотечный код гораздо проще. Поскольку поддерживается одна копия библиотеки и на эту копию ссылаются многие бинарные файлы, замена единственной устаревшей библиотеки новой версией приводит к тому, что любой новый процесс, основанный на бинарном файле, динамически ссылающемся на эту библиотеку, использует обновленную версию. Один из недостатков динамической компоновки - более сложный процесс загрузки. Все необходимые библиотеки нужно найти и загрузить в память, в отличие от загрузки одного статически скомпонованного файла, который уже содержит весь библиотечный код. Другой недостаток динамической компоновки состоит в том, что поставщикам приходится распространять не только собственный исполняемый файл, но и все библиотечные файлы, от которых он зависит. Попытка выполнить программу в системе, где отсутствуют все требуемые библиотечные файлы, завершится ошибкой.

Следующий вывод демонстрирует создание динамически и статически скомпонованных версий программы, размер получившихся бинарных файлов и то, как `file` идентифицирует эти бинарные файлы:

```
ghidrabook# gcc -o ch2_example_dynamic ch2_example.c
ghidrabook# gcc -o ch2_example_static ch2_example.c -static
ghidrabook# ls -l ch2_example_*
  -rwxrwxr-x 1 ghidrabook ghidrabook  12944 Nov  7 10:07 ch2_example_dynamic
  -rwxrwxr-x 1 ghidrabook ghidrabook 963504 Nov  7 10:07 ch2_example_static
ghidrabook# file ch2_example_*
  ch2_example_dynamic: ELF 64-bit LSB executable, x86-64, version 1 (SYSV),
  dynamically linked, interpreter /lib64/l, for GNU/Linux 3.2.0,
  BuildID[sha1]=e56ed40012accb3734bde7f8bca3cc2c368455c3, not stripped
  ch2_example_static:  ELF 64-bit LSB executable, x86-64, version 1 (GNU/Linux),
  statically linked, for GNU/Linux 3.2.0,
  BuildID[sha1]=430996c6db103e4fe76aea7d578e636712b2b4b0, not stripped
ghidrabook#
```

Чтобы динамическая компоновка работала правильно, динамически скомпонованные бинарные файлы должны указывать, от каких библиотек они зависят, вместе с конкретными ресурсами, требуемыми от каждой из этих библиотек. В результате, в отличие от статически скомпонованных бинарных файлов, довольно просто определить библиотеки, от которых зависит динамически скомпонованный бинарный файл. Утилита `ldd` - "список динамических зависимостей" - применяется для перечисления динамических библиотек, требуемых любому исполняемому файлу. В следующем примере `ldd` используется для определения библиотек, от которых зависит веб-сервер Apache:

```
ghidrabook# ldd /usr/sbin/apache2
  linux-vdso.so.1 =>  (0x00007fffc1c8d000)
  libpcre.so.3 => /lib/x86_64-linux-gnu/libpcre.so.3 (0x00007fbeb7410000)
  libaprutil-1.so.0 => /usr/lib/x86_64-linux-gnu/libaprutil-1.so.0 (
    0x00007fbeb71e0000)
  libapr-1.so.0 => /usr/lib/x86_64-linux-gnu/libapr-1.so.0 (0x00007fbeb6fa0000)
  libpthread.so.0 => /lib/x86_64-linux-gnu/libpthread.so.0 (0x00007fbeb6d70000)
  libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007fbeb69a0000)
  libcrypt.so.1 => /lib/x86_64-linux-gnu/libcrypt.so.1 (0x00007fbeb6760000)
  libexpat.so.1 => /lib/x86_64-linux-gnu/libexpat.so.1 (0x00007fbeb6520000)
  libuuid.so.1 => /lib/x86_64-linux-gnu/libuuid.so.1 (0x00007fbeb6310000)
  libdl.so.2 => /lib/x86_64-linux-gnu/libdl.so.2 (0x00007fbeb6100000)
  /lib64/ld-linux-x86-64.so.2 (0x00007fbeb7a00000)
ghidrabook#
```

Утилита `ldd` доступна в системах Linux и BSD. В системах macOS похожая функциональность доступна через утилиту `otool` с параметром `-L`: `otool -L filename`. В системах Windows утилиту `dumpbin`, входящую в набор инструментов Visual Studio, можно использовать для перечисления зависимых библиотек: `dumpbin /dependents filename`.

> **Осторожнее с инструментами!**
>
> Хотя `ldd` может казаться простым инструментом, man-страница `ldd` утверждает, что "никогда не следует применять `ldd` к недоверенному исполняемому файлу, поскольку это может привести к выполнению произвольного кода". Хотя в большинстве случаев это маловероятно, это напоминает, что запуск даже внешне простых инструментов обратной разработки программ (SRE) может иметь непредвиденные последствия при изучении недоверенных входных файлов. Хотя, будем надеяться, очевидно, что выполнение недоверенных бинарных файлов вряд ли безопасно, разумно принимать меры предосторожности даже при статическом анализе недоверенных бинарных файлов и считать, что компьютер, на котором выполняются задачи SRE, вместе с любыми данными на нем или другими подключенными к нему узлами может быть скомпрометирован в результате SRE-деятельности.

objdump

В то время как `ldd` довольно специализирована, `objdump` чрезвычайно универсальна. Назначение `objdump` - "отображать информацию из объектных файлов"[8]. Это достаточно широкая цель, и для ее достижения `objdump` реагирует более чем на 30 параметров командной строки, предназначенных для извлечения различных фрагментов информации из объектных файлов. Инструмент `objdump` можно использовать для отображения следующих данных, связанных с объектными файлами, и многого другого.

Заголовки секций - сводные сведения по каждой секции в файле программы.

Частные заголовки - сведения о расположении программы в памяти и другая информация, требуемая загрузчиком времени выполнения, включая список требуемых библиотек, такой как

создаваемый ldd.

Отладочная информация - любые отладочные сведения, встроенные в файл программы.

Информация о символах - сведения из таблицы символов, выгруженные способом, похожим на утилиту nm.

Листинг дизассемблирования - инструмент objdump выполняет дизассемблирование линейным проходом секций файла, помеченных как код. При дизассемблировании кода x86 objdump может создавать синтаксис AT&T или Intel, а результат дизассемблирования можно сохранить как текстовый файл. Такой текстовый файл называется статическим, или "мертвым", листингом дизассемблирования; хотя такие файлы, безусловно, можно использовать для обратной разработки, по ним трудно эффективно перемещаться и еще труднее последовательно и без ошибок изменять.

Инструмент objdump доступен как часть набора GNU binutils и встречается в Linux, FreeBSD и Windows через WSL или Cygwin[9]. Обратите внимание, что objdump опирается на библиотеку Binary File Descriptor (libbfd), компонент binutils, для доступа к объектным файлам и потому способен разбирать форматы файлов, поддерживаемые libbfd, среди которых ELF и PE. Для ELF-специфического разбора также доступна утилита readelf. Утилита readelf предлагает большинство тех же возможностей, что и objdump, а главное различие между ними состоит в том, что readelf не полагается на libbfd.

otool

Утилиту otool проще всего описать как вариант, похожий на objdump, для macOS; она полезна для разбора сведений о бинарных файлах macOS Mach-O. Следующий листинг демонстрирует, как otool отображает зависимости Mach-O-бинарника от динамических библиотек, выполняя тем самым функцию, похожую на ldd:

```
ghidrabook# file osx_example
  osx_example: Mach-O 64-bit executable x86_64
ghidrabook# otool -L osx_example
  osx_example:
    /usr/lib/libstdc++.6.dylib (compatibility version 7.0.0, current version 7.4.
    0)
    /usr/lib/libgcc_s.1.dylib (compatibility version 1.0.0, current version 1.0.0)
    /usr/lib/libSystem.B.dylib (compatibility version 1.0.0, current version 1281.
    0.0)
```

Утилиту otool можно использовать для отображения сведений, связанных с заголовками файла и таблицами символов, а также для дизассемблирования секции кода файла. Дополнительные сведения о возможностях otool см. на соответствующей man-странице.

dumpbin

Утилита командной строки dumpbin входит в набор инструментов Microsoft Visual Studio. Как и otool и objdump, dumpbin способна отображать широкий диапазон сведений, связанных с файлами Windows PE. Следующий листинг показывает, как dumpbin отображает динамические зависимости программы Windows Notepad способом, похожим на ldd:

```
$ dumpbin /dependents C:\Windows\System32\notepad.exe
Microsoft (R) COFF/PE Dumper
Copyright (C) Microsoft Corporation.  All rights reserved.
Dump of file notepad.exe
File Type: EXECUTABLE IMAGE
  Image has the following delay load dependencies:
```

```
    ADVAPI32.dll
    COMDLG32.dll
    PROPSYS.dll
    SHELL32.dll
    WINSPOOL.DRV
    urlmon.dll
  Image has the following dependencies:
    GDI32.dll
    USER32.dll
    msvcrt.dll
    ...
```

Дополнительные параметры `dumpbin` позволяют извлекать сведения из различных секций PE-бинарника, включая символы, имена импортируемых функций, имена экспортируемых функций и дизассемблированный код. Дополнительная информация об использовании `dumpbin` доступна на сайте Microsoft[10].

c++filt

Языки, допускающие перегрузку функций, должны иметь механизм различения множества перегруженных версий функции, поскольку у каждой версии одно и то же имя. Следующий пример C++ показывает прототипы нескольких перегруженных версий функции с именем `demo`:

```
void demo(void);
void demo(int x);
void demo(double x);
void demo(int x, double y);
void demo(double x, int y);
void demo(char* str);
```

Как правило, в объектном файле невозможно иметь две функции с одним и тем же именем. Чтобы разрешить перегрузку, компиляторы выводят уникальные имена для перегруженных функций, включая сведения, описывающие последовательность типов аргументов функции. Процесс получения уникальных имен для функций с одинаковыми именами называется искажением имен[11]. Если с помощью `nm` выгрузить символы из скомпилированной версии приведенного выше кода C++, можно увидеть примерно следующее - вывод отфильтрован, чтобы сосредоточиться на версиях `demo`:

```
ghidrabook# g++ -o ch2_cpp_example ch2_cpp_example.cc
ghidrabook# nm ch2_cpp_example | grep demo
  000000000000060b T _Z4demod
  0000000000000626 T _Z4demodi
  0000000000000601 T _Z4demoi
  0000000000000617 T _Z4demoid
  0000000000000635 T _Z4demoPc
  00000000000005fa T _Z4demov
```

Стандарт C++ не определяет стандартную схему искажения имен, оставляя разработчикам компиляторов возможность создавать собственные. Чтобы расшифровать показанные здесь искаженные варианты `demo`, нужен инструмент, который понимает схему искажения имен нашего компилятора - в данном случае `g++`. Именно для этого предназначена `c++filt`. Эта утилита воспринимает каждое входное слово так, будто оно является искаженным именем, а затем пытается определить компилятор, использованный для генерации этого имени. Если имя похоже на корректное искаженное имя, она выводит деманглированную версию имени. Когда `c++filt` не распознает слово как искаженное имя, она просто выводит это слово без изменений.

Если передать результаты `nm` из предыдущего примера через `c++filt`, можно восстановить деманглированные имена функций, как показано ниже:

```
ghidrabook# nm ch2_cpp_example | grep demo | c++filt
  000000000000060b T demo(double)
  0000000000000626 T demo(double, int)
  0000000000000601 T demo(int)
  0000000000000617 T demo(int, double)
  0000000000000635 T demo(char*)
  00000000000005fa T demo()
```

Важно отметить, что искаженные имена содержат дополнительные сведения о функциях, которые `nm` обычно не предоставляет. Эта информация может быть чрезвычайно полезна в ситуациях обратной разработки, а в более сложных случаях такие дополнительные сведения могут включать данные об именах классов или соглашениях о вызове функций.

Инструменты глубокого изучения

До сих пор мы обсуждали инструменты, выполняющие беглый анализ файлов на основе минимальных знаний об их внутренней структуре. Мы также видели инструменты, способные извлекать конкретные фрагменты данных из файлов на основе очень подробного знания структуры файла. В этом разделе мы обсудим инструменты, предназначенные для извлечения определенных типов информации независимо от типа анализируемого файла.

strings

Иногда полезно задавать более общие вопросы о содержимом файла - вопросы, которые не обязательно требуют каких-либо специальных знаний о структуре файла. Один из таких вопросов: "Содержит ли этот файл встроенные строки?" Конечно, сначала нужно ответить на вопрос: "Что именно считается строкой?" Свободно определим строку как последовательную цепочку печатных символов. Это определение часто дополняют минимальной длиной и конкретным набором символов. Таким образом, можно задать поиск всех последовательностей не менее чем из четырех подряд идущих печатных ASCII-символов и вывести результаты в консоль. Поиск таких строк обычно никак не ограничивается структурой файла. Искать строки в ELF-бинарнике можно так же легко, как в документе `Microsoft Word`.

Утилита `strings` специально предназначена для извлечения строкового содержимого из файлов, часто без учета формата этих файлов. Использование `strings` с настройками по умолчанию - 7-битные ASCII-последовательности длиной не менее четырех символов - может дать примерно такой результат:

```
ghidrabook# strings ch2_example
  /lib64/ld-linux-x86-64.so.2
  libc.so.6
  exit
  srand
  __isoc99_scanf
  puts
  time
  __stack_chk_fail
  printf
  stderr
  fwrite
  __libc_start_main
  GLIBC_2.7
  GLIBC_2.4
  GLIBC_2.2.5
```

```
_ITM_deregisterTMCloneTable
__gmon_start__
_ITM_registerTMCloneTable
usage: ch4_example [max]
Простая игра в угадывание!
Угадайте число от 1 до %d.
Некорректный ввод, выход!
Поздравляем, вы угадали за %d попыток!
Sorry too low, please try again
Sorry too high, please try again
GCC: (Ubuntu 7.4.0-1ubuntu1~18.04.1) 7.4.0
...
```

К сожалению, хотя мы видим строки, похожие на те, которые программа может выводить, другие строки выглядят как имена функций и библиотек. Нужно быть осторожными и не делать поспешных выводов о поведении программы. Аналитики часто попадают в ловушку, пытаясь вывести поведение программы по выводу `strings`. Помните: присутствие строки внутри бинарного файла никак не означает, что эта строка когда-либо каким-либо образом используется этим бинарным файлом.

Несколько заключительных замечаний об использовании `strings`:

- По умолчанию `strings` не указывает, где именно внутри файла расположена строка. Используйте аргумент командной строки `-t`, чтобы `strings` печатала сведения о смещении в файле для каждой найденной строки.
- Многие файлы используют альтернативные наборы символов. Используйте аргумент командной строки `-e`, чтобы `strings` искала широкие символы, например 16-битный Unicode.

Почему `strings` изменилась?

Исторически при использовании `strings` с исполняемыми файлами она по умолчанию искала символьные последовательности только в загружаемых инициализированных секциях данных бинарного файла. Для этого `strings` должна была разбирать бинарный файл, чтобы найти такие секции, используя библиотеки вроде `libbfd`. При разборе недоверенных бинарных файлов уязвимости в библиотеках потенциально могли приводить к выполнению произвольного кода[12]. В результате поведение `strings` по умолчанию было изменено: теперь она исследует весь бинарный файл без разбора загружаемых инициализированных секций данных, что синонимично использованию флага `-a`. Историческое поведение можно вызвать с помощью флага `-d`.

Дизассемблеры

Как упоминалось ранее, существуют инструменты, создающие дизассемблированные представления бинарных объектных файлов в стиле статических листингов. Бинарные файлы PE, ELF и Mach-O можно дизассемблировать с помощью `dumpbin`, `objdump` и `otool` соответственно. Однако ни один из этих инструментов не может работать с произвольными блоками двоичных данных. Иногда вы столкнетесь с бинарным файлом, который не соответствует широко используемому формату; в таком случае понадобятся инструменты, способные начинать процесс дизассемблирования с указанных пользователем смещений.

Два примера таких потоковых дизассемблеров для набора инструкций x86 - `ndisasm` и diStorm[13]. Утилита `ndisasm` входит в NASM[14]. Следующий пример иллюстрирует использование `ndisasm` для дизассемблирования фрагмента шеллкода, созданного с помощью Metasploit Framework[15]:

```
ghidrabook#  msfvenom -p linux/x64/shell_find_port -f raw > findport
ghidrabook#  ndisasm -b 64 findport
  00000000  4831FF            xor rdi,rdi
  00000003  4831DB            xor rbx,rbx
  00000006  B314              mov bl,0x14
  00000008  4829DC            sub rsp,rbx
  0000000B  488D1424          lea rdx,[rsp]
  0000000F  488D742404        lea rsi,[rsp+0x4]
  00000014  6A34              push byte +0x34
  00000016  58                pop rax
  00000017  0F05              syscall
  00000019  48FFC7            inc rdi
  0000001C  66817E024A67      cmp word [rsi+0x2],0x674a
  00000022  75F0              jnz 0x14
  00000024  48FFCF            dec rdi
  00000027  6A02              push byte +0x2
  00000029  5E                pop rsi
  0000002A  6A21              push byte +0x21
  0000002C  58                pop rax
  0000002D  0F05              syscall
  0000002F  48FFCE            dec rsi
  00000032  79F6              jns 0x2a
  00000034  4889F3            mov rbx,rsi
  00000037  BB412F7368        mov ebx,0x68732f41
  0000003C  B82F62696E        mov eax,0x6e69622f
  00000041  48C1EB08          shr rbx,byte 0x8
  00000045  48C1E320          shl rbx,byte 0x20
  00000049  4809D8            or rax,rbx
  0000004C  50                push rax
  0000004D  4889E7            mov rdi,rsp
  00000050  4831F6            xor rsi,rsi
  00000053  4889F2            mov rdx,rsi
  00000056  6A3B              push byte +0x3b
  00000058  58                pop rax
  00000059  0F05              syscall
ghidrabook#
```

Гибкость потокового дизассемблирования полезна во многих ситуациях. Один сценарий связан с анализом компьютерных сетевых атак, при которых сетевые пакеты могут содержать шеллкод. Потоковые дизассемблеры можно использовать для дизассемблирования тех частей пакета, которые содержат шеллкод, чтобы проанализировать поведение вредоносной полезной нагрузки. Другая ситуация связана с анализом образов ROM, для которых не удается найти справочник по размещению. Одни части ROM будут содержать данные, а другие - код. Потоковые дизассемблеры можно использовать для дизассемблирования только тех участков образа, которые считаются кодом.

Итоги

Инструменты, обсуждавшиеся в этой главе, не обязательно лучшие в своих категориях. Однако они представляют инструменты, обычно доступные каждому, кто хочет выполнять обратную разработку бинарных файлов. Что важнее, они представляют типы инструментов, которые во многом мотивировали разработку Ghidra. В последующих главах мы иногда будем выделять автономные инструменты, предоставляющие функциональность, похожую на встроенную в Ghidra. Знание этих инструментов значительно улучшит ваше понимание пользовательского интерфейса Ghidra и многочисленных информационных представлений, которые предлагает Ghidra.

Сноски

[1] [назад] См. https://docs.microsoft.com/en-us/windows/wsl/about/.
[2] [назад] Для Cygwin см. http://www.cygwin.com/. Для MinGW см. http://www.mingw.org/.
[3] [назад] Магическое число - специальное значение метки, требуемое некоторыми спецификациями форматов файлов; его присутствие указывает на соответствие таким спецификациям. В некоторых случаях магические числа выбирают из шуточных соображений. Метка `MZ` в заголовках исполняемых файлов MS-DOS представляет инициалы Марка Збиковски, одного из первоначальных архитекторов MS-DOS, а шестнадцатеричное значение `0xcafebabe`, широко известное магическое число, связанное с файлами Java `.class`, было выбрано потому, что это легко запоминающаяся последовательность шестнадцатеричных цифр.
[4] [назад] См. https://github.com/petoolse/petools/.
[5] [назад] См. https://github.com/wolfram77web/app-peid/.
[6] [назад] См. https://www.trendmicro.com/vinfo/us/threat-encyclopedia/malware/GAOBOT/.
[7] [назад] Подробнее о компоновке см. John R. Levine, *Linkers and Loaders* (Morgan Kaufmann, 1999).
[8] [назад] См. http://www.sourceware.org/binutils/docs/binutils/objdump.html.
[9] [назад] См. http://www.gnu.org/software/binutils/.
[10] [назад] См. https://docs.microsoft.com/en-us/cpp/build/reference/dumpbin-command-line/.
[11] [назад] Обзор искажения имен см. http://en.wikipedia.org/wiki/Name_mangling.
[12] [назад] См. CVE-2014-8485 и https://lcamtuf.blogspot.com/2014/10/psa-dont-run-strings-on-untrusted-files.html.
[13] [назад] См. https://github.com/gdabah/distorm/.
[14] [назад] См. http://www.nasm.us/.
[15] [назад] См. https://metasploit.com/.

Глава 3. Знакомство с Ghidra

Ghidra - свободно доступный набор инструментов SRE с открытым исходным кодом, разработанный National Security Agency (NSA). Платформенно-независимая среда Ghidra включает интерактивный дизассемблер и декомпилятор, а также множество связанных инструментов, которые совместно помогают анализировать код. Она поддерживает широкий набор архитектур наборов инструкций и бинарных форматов и может работать как в автономных, так и в совместных конфигурациях SRE. Возможно, лучшая особенность Ghidra состоит в том, что она позволяет настраивать рабочую среду, разрабатывать собственные плагины и сценарии для улучшения процесса SRE и делиться своими нововведениями с сообществом Ghidra в целом.

Лицензии Ghidra

Ghidra распространяется бесплатно и лицензируется по Apache License версии 2.0. Эта лицензия предоставляет отдельным пользователям большую свободу в использовании Ghidra, но имеет и связанные с ней ограничения. Всем, кто загружает, использует или редактирует Ghidra, рекомендуется прочитать пользовательское соглашение Ghidra (`docs/UserAgreement.html`), а также лицензионные файлы в каталогах `GPL` и `licenses`, чтобы убедиться в соблюдении всех лицензионных соглашений, поскольку сторонние компоненты внутри Ghidra имеют собственные лицензии. Если вы когда-нибудь забудете что-либо из этого абзаца, Ghidra заботливо отображает лицензионную информацию каждый раз при запуске Ghidra или при выборе `About Ghidra` в меню `Help`.

Версии Ghidra

Ghidra доступна для Windows, Linux и macOS. Хотя Ghidra очень гибко настраивается, большинство новых пользователей, вероятно, загрузят Ghidra и решат начать с самой текущей версии Ghidra Core, включающей традиционную функциональность обратной разработки. Эта книга сосредоточена на функциональности Ghidra Core для необщих проектов. Кроме того, мы уделим время обсуждению общих проектов и Ghidra в режиме без графического интерфейса, а также конфигураций Developer, Function ID и Experimental.

Ресурсы поддержки Ghidra

Работа с новым программным набором может пугать, особенно когда цель состоит в решении сложной практической задачи с помощью обратной разработки. Как пользователь Ghidra или потенциальный разработчик, вы можете задаваться вопросом, куда обратиться за помощью при возникновении вопросов, связанных с Ghidra. Если мы достаточно хорошо выполним свою работу, этой книги хватит во многих ситуациях. Однако когда вам понадобится дополнительная помощь, можно обратиться к следующим ресурсам.

Официальная справочная документация. Ghidra содержит подробную справочную систему, которую можно активировать через меню или нажатием `F1`. Справочная система предоставляет иерархическое меню, а также возможность поиска. Хотя меню `Help` предлагает различные представления, сейчас оно не поддерживает вопросительные запросы вроде "Как это сделать?"

Файлы README. В некоторых случаях меню `Ghidra Help` направит вас к дополнительному содержимому по конкретной теме, например к файлу README. В документацию включено много файлов README, дополняющих конкретные плагины, расширяющих темы меню `Help`, например `support/analyzeHeadlessREADME.html`, помогающих с разными вариантами установки, например `docs/InstallationGuide.html`, и содействующих вашему развитию как разработчика, например `Extensions/Eclipse/GhidraDev/GhidraDev_README.html`, если вы решите пойти по этому пути и, возможно, разработать поддержку вопросительных запросов вроде "Как это сделать?"

Сайт Ghidra. Домашняя страница проекта Ghidra (https://www.ghidra-sre.org/) предоставляет потенциальным пользователям, текущим пользователям, разработчикам и участникам проекта варианты для дальнейшего изучения Ghidra. Наряду с подробной информацией о загрузке, связанной с каждым выпуском Ghidra, полезное видео с руководством по установке проводит вас через процесс установки.

Каталог Ghidra `docs`**.** Установка Ghidra включает каталог с полезной документацией, связанной с Ghidra, включая пригодное для печати руководство по меню и горячим клавишам (`docs/CheatSheet.html`), которое может значительно облегчить первое знакомство с Ghidra, и многое другое. Учебные материалы, охватывающие начальные, средние и продвинутые возможности Ghidra, находятся в `docs/GhidraClass`.

Загрузка Ghidra

Получить бесплатную копию Ghidra можно простым процессом из трех шагов:

1. Перейдите на https://ghidra-sre.org/.
2. Нажмите большую красную кнопку Download Ghidra.
3. Сохраните файл в нужном месте на своем компьютере.

Как и во многих простых трехшаговых процессах, есть пара моментов, где некоторые независимые пользователи могут слегка отклониться от рекомендованного пути. Следующие варианты предназначены для тех, кто хочет чего-то иного, чем традиционный стартовый набор:

- Если вы хотите установить другой выпуск, просто нажмите кнопку Releases, и у вас появится возможность загрузить другие выпущенные версии. Хотя отдельная функциональность может различаться, основы Ghidra должны остаться прежними.
- Если вы хотите выполнить установку на сервер для поддержки совместной работы, подождите до главы 11, чтобы узнать, как внести это важное изменение в установку, или можете забежать вперед и попробовать сделать это, используя сведения в каталоге `server`. В худшем случае легко откатиться и начать заново с простого трехшагового процесса, запустив локальный экземпляр Ghidra.
- По-настоящему смелые духом могут захотеть собрать Ghidra из исходного кода. Исходный код Ghidra доступен на GitHub по адресу https://github.com/NationalSecurityAgency/ghidra/.

Продолжим с традиционным процессом установки.

Установка Ghidra

Итак, что сделала магическая красная кнопка загрузки, когда вы нажали ее и выбрали место назначения на компьютере? Если все прошло по плану, теперь в выбранном каталоге должен быть ZIP-файл. Для исходного выпуска Ghidra имя ZIP-файла было `ghidra_9.0_PUBLIC_20190228.zip`. Соглашение об именовании можно разобрать. Сначала `9.0` - номер версии. Затем `PUBLIC` - тип выпуска; существуют и другие типы выпусков, например `BETA_DEV`. Наконец, указана дата выпуска, за которой следует расширение файла `.zip`.

На самом деле этот ZIP-файл представляет собой коллекцию из более чем 3400 файлов, составляющих фреймворк Ghidra. Если вас устраивает место, куда вы сохранили файл, распаковка, например щелчком правой кнопкой мыши и выбором Extract All в Windows, предоставит доступ к иерархическому каталогу Ghidra. Обратите внимание, что Ghidra должна компилировать некоторые свои внутренние файлы данных, поэтому пользователю Ghidra обычно нужен доступ на запись ко всем подкаталогам программы Ghidra.

Структура каталога Ghidra

Знакомство с содержимым установки Ghidra ни в коем случае не является обязательным требованием перед началом работы с Ghidra. Однако, поскольку сейчас наше внимание направлено на извлеченную загрузку, сначала посмотрим на базовую структуру. Понимание структуры каталогов Ghidra станет важнее по мере перехода к более продвинутым возможностям Ghidra, рассматриваемым в последующих главах. Ниже приведено краткое описание каждого подкаталога внутри установки Ghidra. На рисунке 3-1 показана структура каталогов Ghidra.

Рисунок 3-1. Структура каталогов Ghidra

`docs`. Содержит общую вспомогательную документацию о Ghidra и о том, как ее использовать. В этом каталоге есть два подкаталога, которые стоит упомянуть. Во-первых, подкаталог `GhidraClass` предоставляет учебные материалы, помогающие изучать Ghidra. Во-вторых, подкаталог `languages` описывает язык спецификации процессоров Ghidra - SLEIGH. SLEIGH подробно обсуждается в главе 18.

`Extensions`. Содержит полезные заранее собранные расширения, а также важное содержимое и сведения для написания расширений Ghidra. Этот каталог подробнее рассматривается в главах 15, 17 и 18.

`Ghidra`. Содержит код Ghidra. Вы узнаете больше о ресурсах и содержимом этого каталога, когда мы начнем настраивать Ghidra в главе 12 и создавать новые возможности в главах 13-18.

`GPL`. Некоторые компоненты, составляющие часть фреймворка Ghidra, были разработаны не командой Ghidra, а представляют собой другой код, распространяемый по GNU General Public

License (GPL). Каталог `GPL` содержит файлы, связанные с этим содержимым, включая лицензионную информацию.

`licenses`. Содержит файлы, описывающие надлежащее и законное использование различных сторонних компонентов Ghidra.

`server`. Поддерживает установку сервера Ghidra, который облегчает совместную SRE. Этот каталог подробно обсуждается в главе 11.

`support`. Служит универсальным местом для различных специализированных возможностей и функций Ghidra. В качестве бонуса именно здесь находится значок Ghidra (`ghidra.ico`), если вы хотите дополнительно настроить рабочую среду, например создать ярлык для сценария запуска Ghidra. Этот каталог обсуждается по мере необходимости на протяжении книги, когда мы вводим различные возможности Ghidra.

Запуск Ghidra

Наряду с подкаталогами, файлы в корневом каталоге позволяют начать путь SRE с Ghidra. В этом каталоге находится еще один лицензионный файл (`LICENSE.txt`), но, что важнее, здесь вы найдете сценарии, которые фактически запускают Ghidra. При первом двойном щелчке по `ghidraRun.bat` или запуске эквивалентного сценария `ghidraRun` из командной строки в Linux или macOS вам нужно будет согласиться с лицензионным соглашением конечного пользователя (EULA), показанным на рисунке 3-2, чтобы подтвердить намерение использовать Ghidra в соответствии с пользовательским соглашением Ghidra. После согласия вы больше не увидите это окно при последующих запусках, но сможете в любое время просмотреть его содержимое через меню `Help`.

Кроме того, у вас могут запросить путь к установленной Java. Если Java у вас не установлена, см. руководство по установке в подкаталоге `docs`, где в разделе `Java Notes` приведена вспомогательная документация. Ghidra требует Java Development Kit (JDK) версии 11 или выше[1].

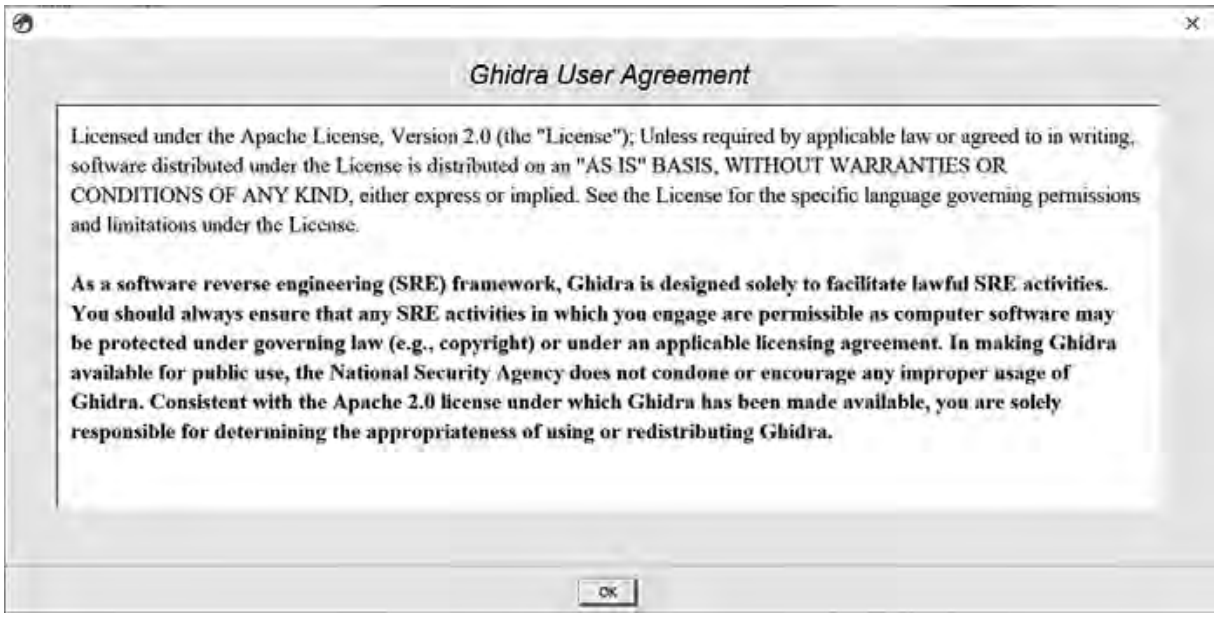


Рисунок 3-2. Пользовательское соглашение Ghidra

Итоги

После успешного открытия Ghidra вы готовы перейти к ее использованию для выполнения чего-то полезного. В течение следующих нескольких глав вы узнаете, как использовать Ghidra для базового анализа файлов, познакомитесь с `CodeBrowser` и многими распространенными окнами отображения Ghidra, а также увидите, как настраивать эти представления и управлять ими, чтобы лучше понимать поведение программы.

Сноски

[1] [назад] JDK доступен по адресу https://adoptopenjdk.net/releases.html?variant=openjdk11&jvmVariant=hotspot/.

Глава 4. Начало работы с Ghidra

Пора наконец перейти к реальному использованию Ghidra. Остальная часть этой книги посвящена различным возможностям Ghidra и тому, как применять их с наибольшей пользой для ваших задач обратной разработки. В этой главе мы сначала рассмотрим варианты, которые появляются при запуске Ghidra, затем опишем, что происходит при открытии одного бинарного файла для анализа. Наконец, мы дадим краткий обзор пользовательского интерфейса, чтобы подготовить основу для последующих глав.

Запуск Ghidra

Каждый раз при запуске Ghidra вас ненадолго встречает заставка, на которой отображаются логотип Ghidra, сведения о сборке, номера версий Ghidra и Java, а также лицензионная информация. Если вы хотите внимательно прочитать заставку и узнать больше о своих версиях, ее можно вывести в любой момент, выбрав `Help > About Ghidra` в окне `Ghidra Project`.

После исчезновения заставки Ghidra отображает окно `Ghidra Project` за диалогом `Tip of the Day`, как показано на рисунке 4-1. Советы можно листать, нажимая кнопку `Next Tip`. Когда будете готовы начать работу, закройте диалог `Tip of the Day`.

Если вы предпочитаете не видеть ежедневные советы, можете снять флажок `Show Tips on Startup` в нижней части диалога. Если вы снимете флажок, а затем поймете, что скучаете по диалогу `Tip of the Day`, его легко вернуть через меню `Ghidra Help`.

Рисунок 4-1. Запуск Ghidra

Если закрыть диалог `Tip of the Day` или снять флажок и перезапустить Ghidra, появится окно `Ghidra Project`. Ghidra использует проектную среду, позволяющую управлять инструментами и данными, связанными с файлом или группой файлов, пока вы с ними работаете. Это начальное знакомство сосредоточено на одном файле как компоненте необщего проекта. Более сложные возможности проектов обсуждаются в главе 11.

Создание нового проекта

Если вы запускаете Ghidra впервые, нужно создать проект. Если вы уже запускали Ghidra раньше, активным будет проект, с которым вы работали последним. Выбор `File > New Project` позволяет задать характеристики среды, связанной с проектом. Первый шаг при создании нового проекта - выбрать между необщим проектом и общим проектом. В этой главе мы начнем с необщего проекта. После этого выбора появится диалог, показанный на рисунке 4-2. Для необщих проектов требуется указать каталог и имя проекта.

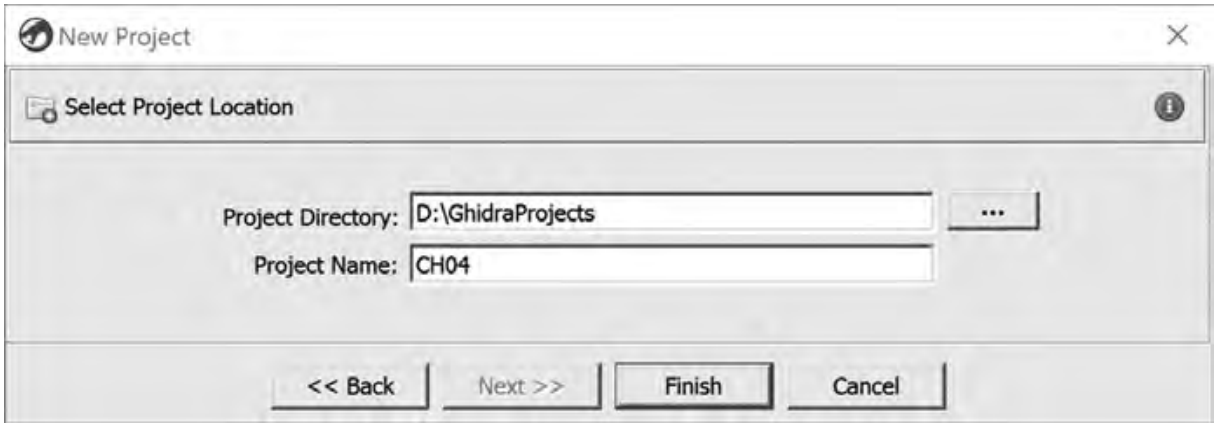


Рисунок 4-2. Создание проекта Ghidra

После ввода сведений о расположении проекта нажмите `Finish`, чтобы завершить процесс создания проекта. Вы вернетесь в окно `Project` с выбранным только что созданным проектом, как показано на рисунке 4-3.

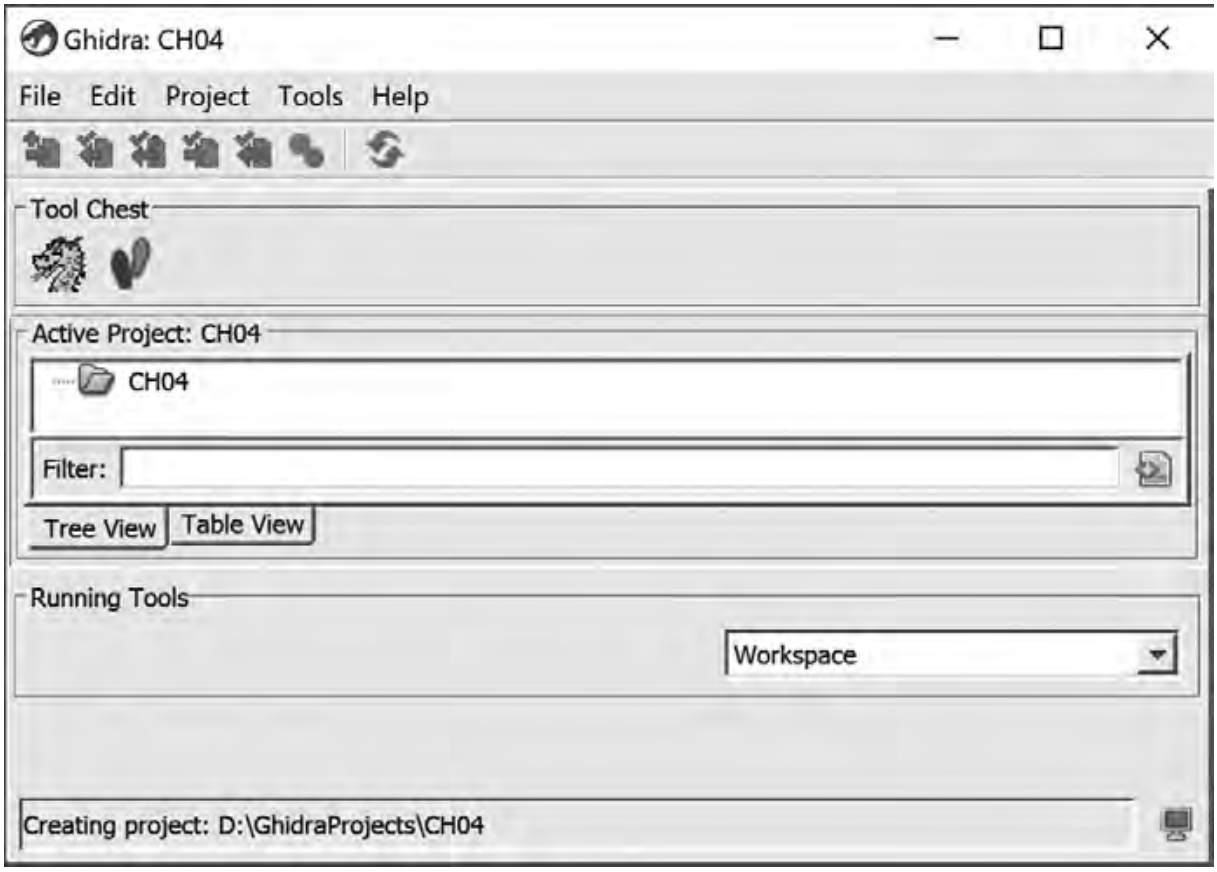


Рисунок 4-3. Окно `Ghidra Project`

Загрузка файлов в Ghidra

Чтобы выполнить какую-либо полезную работу, нужно добавить в новый проект хотя бы один файл. Открыть файл можно либо выбрав `File > Import File` и перейдя к файлу, который нужно импортировать, либо перетащив файл непосредственно в папку в окне `Project`. После выбора файла появится диалог Import, показанный на рисунке 4-4.

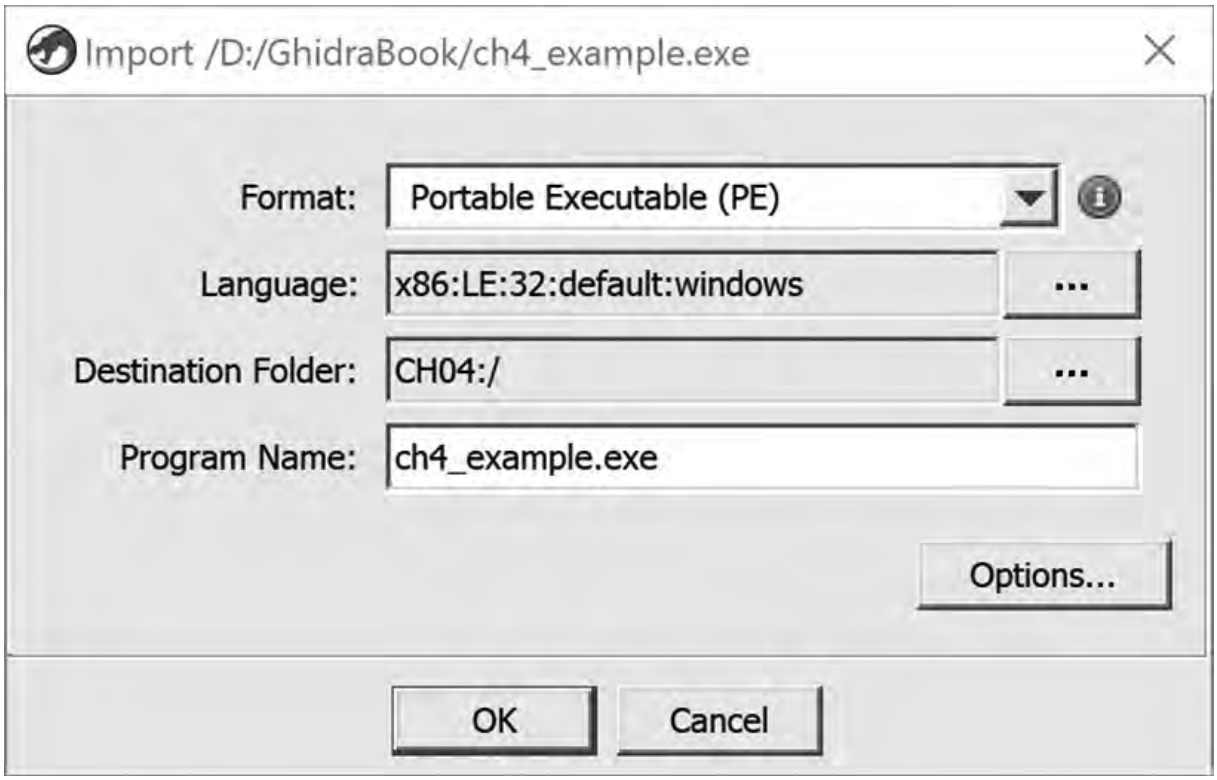


Рисунок 4-4. Диалог Ghidra Import

Ghidra формирует список возможных типов файла и предоставляет его в раскрывающемся списке `Format` в верхней части диалога. Нажатие кнопки `Information` справа от поля `Format` покажет список поддерживаемых форматов, которые описаны в главе 17. Раскрывающийся список `Format` предоставляет подмножество загрузчиков Ghidra, лучше всего подходящих для обработки выбранного файла. В этом примере в списке `Format` доступны два варианта: `Portable Executable` (PE) и `Raw Binary`. Вариант `Raw Binary` присутствует всегда, поскольку это стандартный способ Ghidra загружать файлы, которые она не распознает; он предоставляет самый низкоуровневый вариант загрузки любого файла. Когда предлагается выбор из нескольких загрузчиков, неплохая стратегия - принять варианты по умолчанию, если у вас нет конкретной информации, противоречащей определению Ghidra.

Поле `Language` позволяет указать, какой процессорный модуль должен использоваться в процессе дизассемблирования. Спецификация языка/компилятора Ghidra может состоять из типа процессора, спецификации порядка байтов (LE/BE), значения разрядности (16/32/64), варианта процессора и идентификатора компилятора, например `ARM:LE:32:v7:default`. Дополнительные сведения см. во врезке "Спецификации языка/компилятора" в главе 13, а также в разделе "Файлы определения языков" на странице 396. В большинстве случаев Ghidra выбирает правильный процессор на основе сведений, прочитанных из заголовков исполняемого файла.

Поле `Destination Folder` позволяет выбрать папку проекта, в которой будет отображаться вновь импортированный файл. По умолчанию отображается верхняя папка проекта, но для организации

импортированных программ внутри проекта можно добавлять подпапки. Можно выбрать кнопки расширения справа от полей `Language` и `Destination Folder`, чтобы просмотреть другие варианты для каждого из них. Также можно отредактировать текст в поле `Program Name`. Пусть вас не сбивает с толку изменение терминологии: `Program Name` - это имя, которое Ghidra использует для ссылки на импортированный бинарный файл внутри проекта, в том числе для отображения в окне проекта. По умолчанию оно совпадает с именем импортированного файла, но его можно изменить на что-то более описательное, например "Вредоносная программа со Starship Enterprise".

Помимо четырех полей, показанных на рисунке 4-4, через кнопку `Options` можно получить доступ к другим параметрам управления процессом загрузки. Эти параметры зависят от выбранного формата и процессора. Параметры для `ch4_example.exe`, PE-файла для x86, показаны на рисунке 4-5 с выбранными значениями по умолчанию. Хотя обычно хорошим подходом является движение вперед с параметрами по умолчанию, по мере накопления опыта можно выбирать и другие варианты. Например, можно включить параметр `Load External Libraries`, если вы хотите, чтобы все зависимые библиотеки также импортировались в проект.

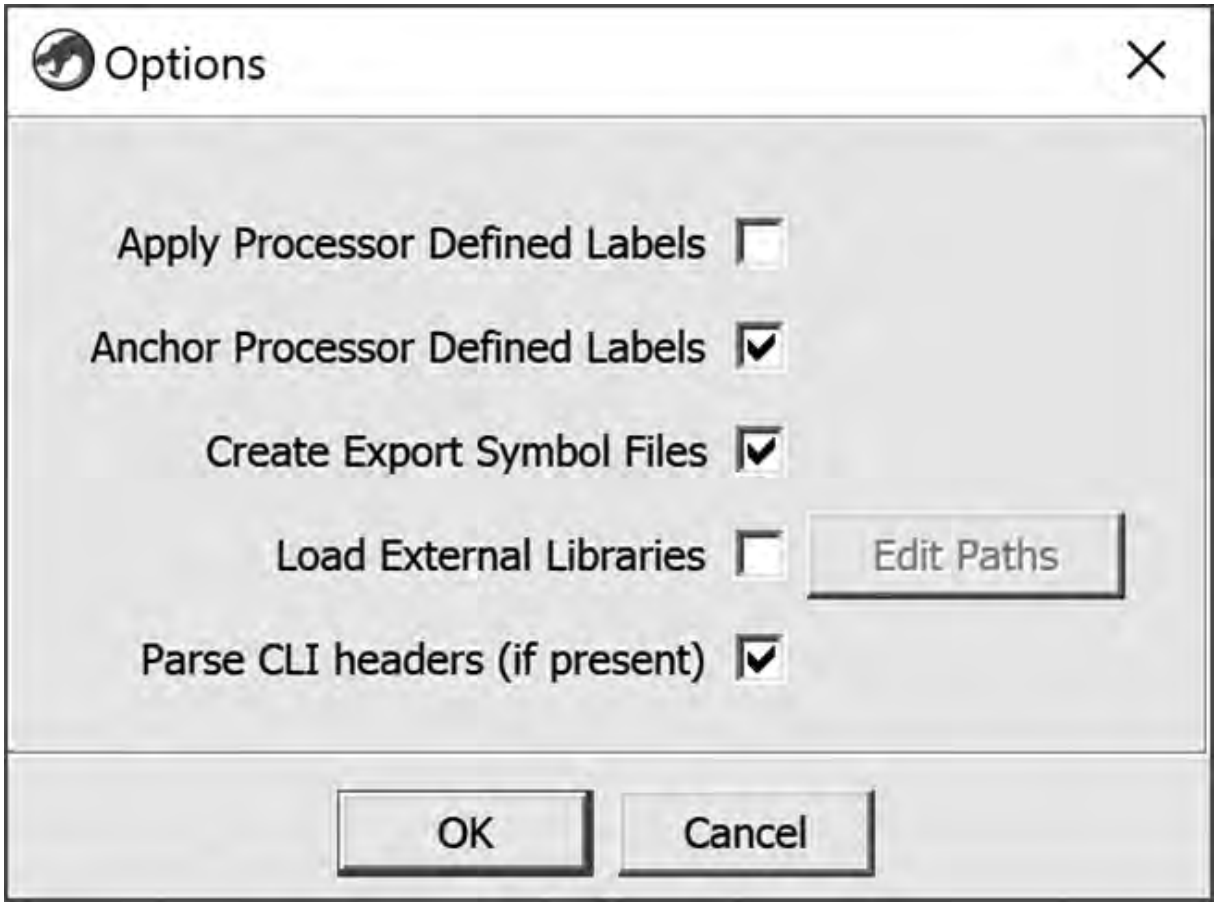


Рисунок 4-5. Параметры загрузки PE-файла в Ghidra

Параметры импорта используются для более тонкого управления процессом загрузки файла. Они применимы не ко всем типам входных файлов, и в большинстве случаев можно полагаться на варианты по умолчанию. Дополнительная информация о параметрах доступна в `Ghidra Help`. Подробнее процесс импорта и загрузчики Ghidra рассматриваются в главе 17.

Когда параметры загрузки вас устраивают и вы нажимаете `OK`, чтобы закрыть диалоги, появляется окно `Import Results Summary`, показанное на рисунке 4-6. Оно дает возможность просмотреть выбранные параметры импорта вместе с базовой информацией, которую загрузчик извлек из выбранного файла. В разделе "`Importing Files`" на странице 262 мы обсуждаем способы изменить некоторые результаты импорта до анализа, если у вас есть дополнительная информация,

не отраженная в окне `Import Results Summary`.

Рисунок 4-6. Окно `Ghidra Import Results Summary`

Использование загрузчика Raw Binary

Иногда `Raw Binary` будет единственной записью в раскрывающемся списке `Format`. Так Ghidra сообщает, что ни один из ее загрузчиков не распознал выбранный файл. Примеры ситуаций, в которых может потребоваться загрузчик `Raw Binary`, включают анализ пользовательских образов прошивок и полезных нагрузок эксплойтов, которые могли быть извлечены из захватов сетевых пакетов или журнальных файлов. В таких случаях Ghidra не может распознать какие-либо сведения заголовка файла, направляющие процесс загрузки, поэтому вам нужно вмешаться и выполнить задачи, которые загрузчики часто выполняют автоматически: указать процессор, разрядность, а в некоторых случаях конкретный компилятор.

Например, если вы знаете, что бинарный файл содержит код x86, в диалоге `Language` доступно много вариантов, как показано на рисунке 4-7. Часто требуется небольшое исследование, а иногда и метод проб и ошибок, чтобы сузить выбор языка до варианта, который будет работать с вашим бинарным файлом. Любая информация об устройстве, для запуска на котором был предназначен файл, окажется полезной. Если вы уверены, что файл не предназначен для системы Windows, для параметра `Compiler` следует выбрать `gcc` или `default`, если он доступен.

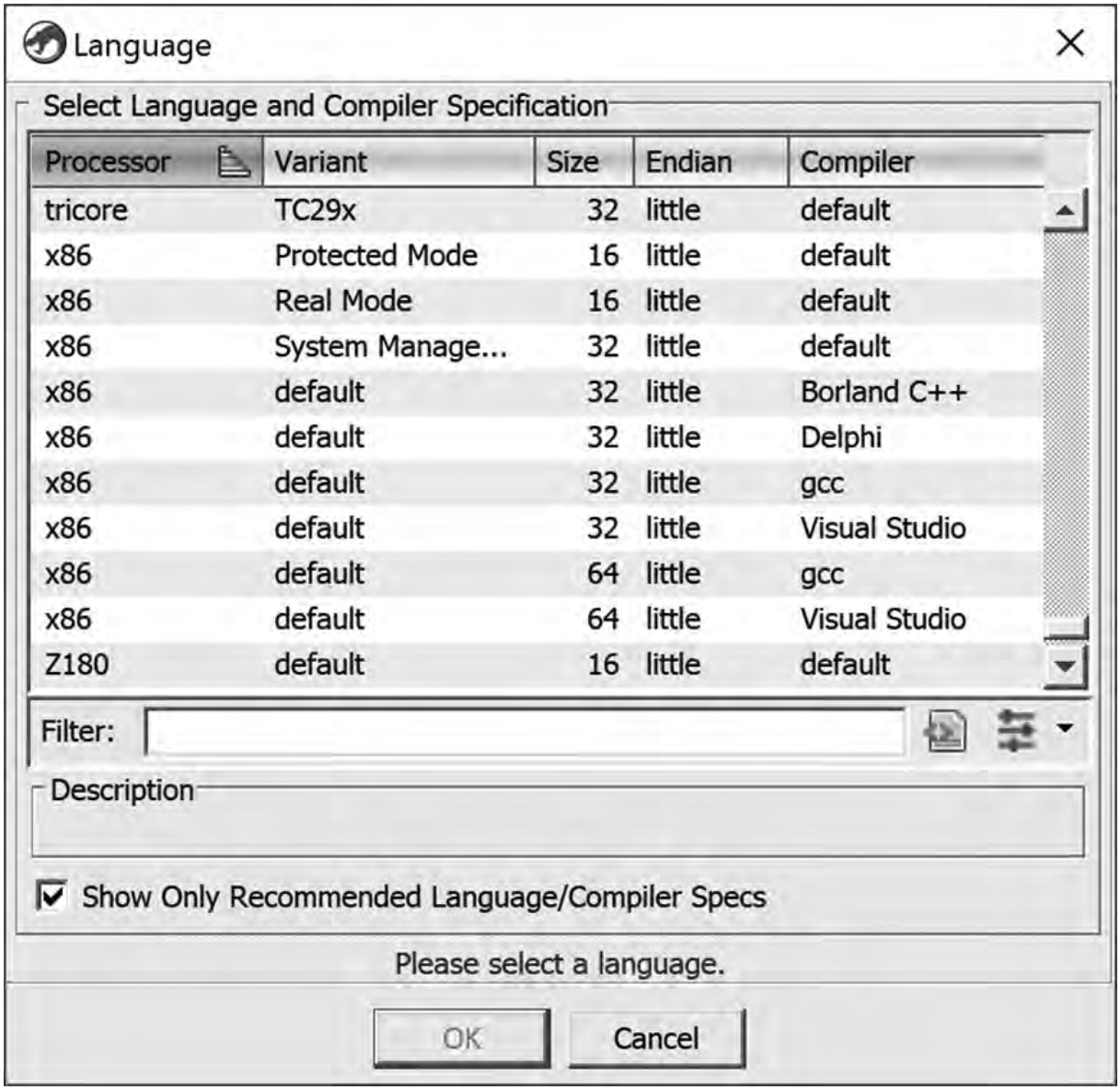


Рисунок 4-7. Варианты выбора языка и компилятора

Если бинарный файл не содержит сведений заголовка, с которыми Ghidra может работать, Ghidra также не распознает схему размещения файла в памяти. Если вам известны базовый адрес, смещение в файле или длина файла, можно ввести эти значения в соответствующие поля параметров загрузчика, показанные на рисунке 4-8, или продолжить загрузку файла, не вводя эти дополнительные сведения. Эти сведения можно предоставить или скорректировать в любой момент до или после анализа через окно Memory Map, обсуждаемое в разделе "Окно Memory Map" на странице 85.

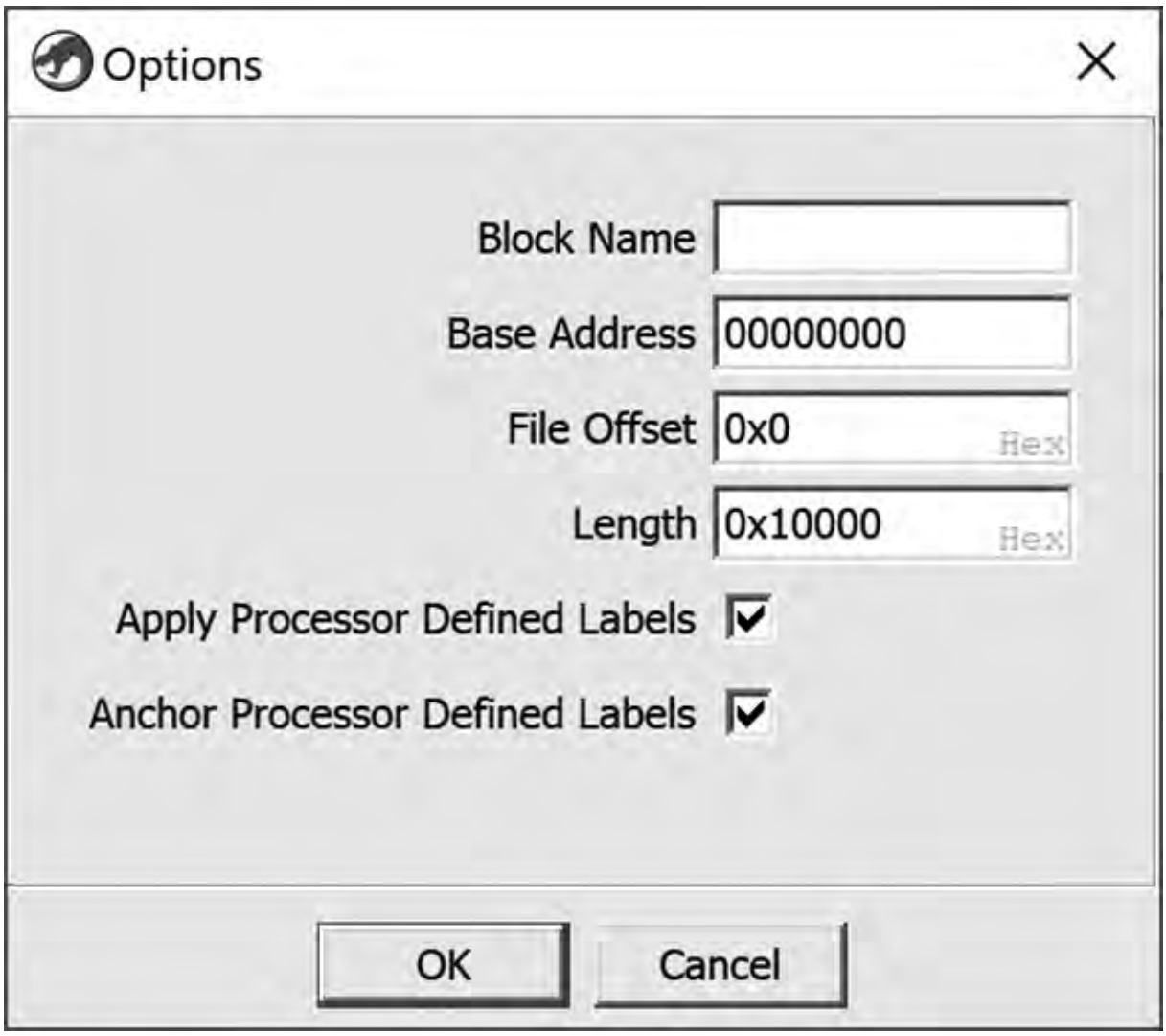


Рисунок 4-8. Параметры загрузчика Ghidra `Raw Binary`

Глава 17 предоставляет более подробное обсуждение ручной загрузки и организации нераспознанных бинарных файлов.

Анализ файлов с помощью Ghidra

По своей сути Ghidra фактически является приложением базы данных, управляемым библиотекой плагинов, каждый из которых имеет собственную функциональность. Все данные проекта хранятся в специальной базе данных, которая растет и развивается по мере того, как пользователь добавляет информацию в проект. Различные представления, предлагаемые Ghidra, - это просто виды базы данных, раскрывающие информацию в форматах, полезных для специалиста по обратной разработке программ. Любые изменения, которые пользователи вносят в базу данных, отражаются в представлениях и сохраняются в базе данных, но эти изменения никак не влияют на исходный исполняемый файл. Сила Ghidra заключается в инструментах, которые она содержит для анализа данных в базе и управления ими.

`CodeBrowser` объединяет многие инструменты, доступные в Ghidra, и обладает уникальной функциональностью, помогающей держать окна упорядоченными, добавлять и удалять инструменты, перестраивать содержимое и документировать процесс. По умолчанию `CodeBrowser` открывается с окнами Program Tree, `Symbol Tree`, `Data Type Manager`, `Listing`, `Decompiler` и Console. Эти и другие представления вводятся в главе 5.

Описанный выше процесс можно использовать для создания проектов и наполнения их файлами, но настоящая работа анализа еще не началась. Когда вы дважды щелкаете файл в окне `Ghidra Project`, появляется окно `CodeBrowser`, показанное на рисунке 4-9. Если это первый выбор одного из импортированных файлов, появится возможность разрешить Ghidra выполнить автоанализ файла. Пример автоанализа с использованием диалога `Analysis Options` показан на рисунке 4-10. В большинстве случаев, связанных с бинарными файлами с распространенных платформ и созданными широко доступными компиляторами, автоанализ, вероятно, является правильным первым выбором. Процесс автоанализа можно остановить в любой момент, нажав красную кнопку остановки в правом нижнем углу окна `CodeBrowser`. Эта кнопка видна только во время автоанализа.

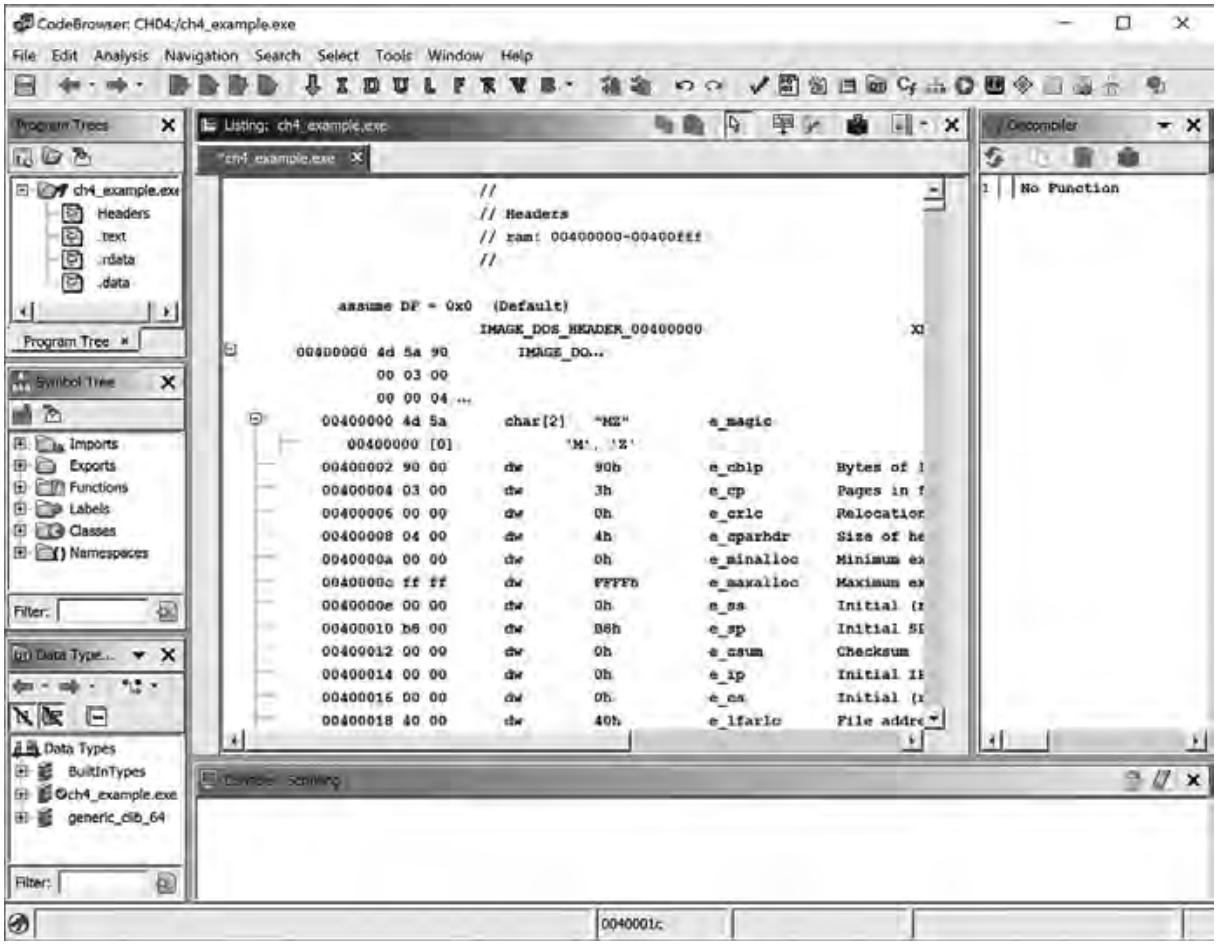


Рисунок 4-9. Окно `Ghidra CodeBrowser`

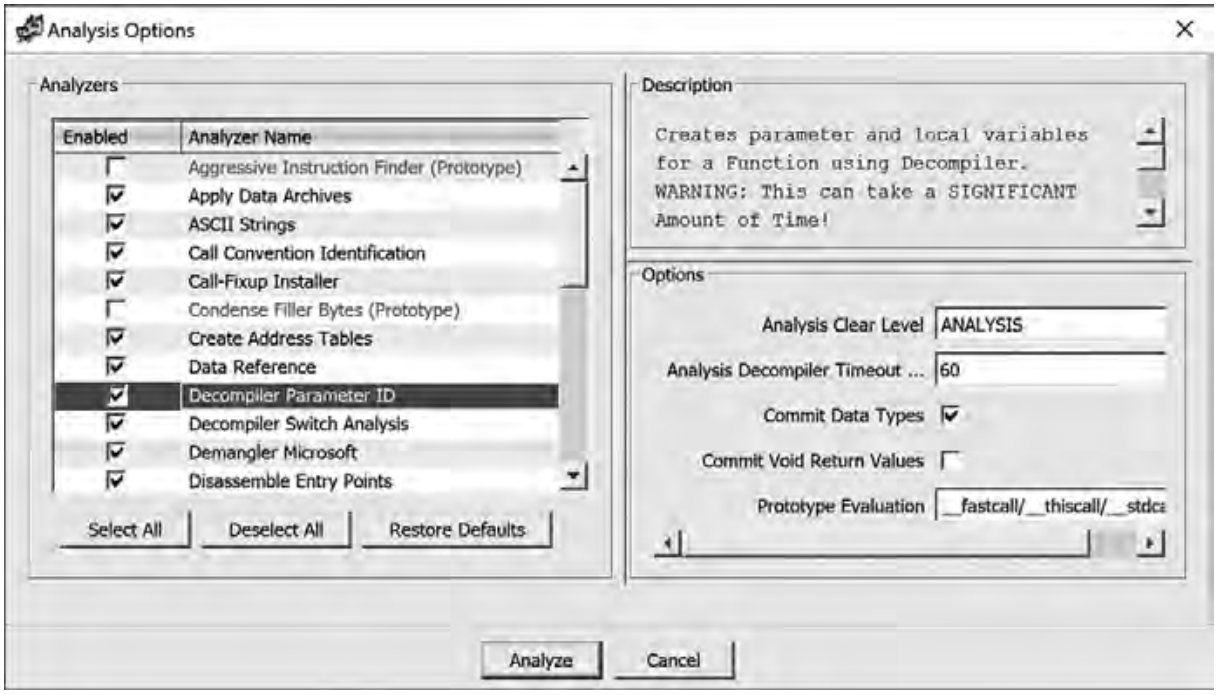


Рисунок 4-10. Диалог `Analysis Options`

Помните: если вас не устраивает автоанализ Ghidra, всегда можно отказаться от работы, закрыв `CodeBrowser` и выбрав не сохранять изменения; после этого можно снова открыть файл и попробовать другую комбинацию параметров автоанализа. Наиболее распространенные причины

изменения параметров автоанализа связаны с необычно структурированными файлами, например обфусцированными бинарными файлами, или бинарными файлами, созданными компиляторами либо в операционных системах, которые могут быть неизвестны Ghidra.

Обратите внимание: если вы открываете чрезвычайно большой бинарный файл, возможно 10 Мбайт или больше, Ghidra может выполнять автоанализ от минут до часов. В таких случаях можно отключить некоторые из более требовательных анализаторов или задать для них тайм-аут анализа, например `Decompiler Switch Analysis`, `Decompiler Parameter ID` и Stack. Как показано на рисунке 4-10, выделение анализатора отображает его описание, которое может включать полезные предупреждения о том, сколько времени может занять запуск анализатора. Кроме того, вы увидите рамку `Options`, дающую возможность управлять некоторыми аспектами поведения отдельных анализаторов. Любой анализ, который вы решите отключить или который завершится по тайм-ауту, всегда можно запустить позже с помощью вариантов, доступных в меню `Ghidra Analysis`.

Предупреждения автоанализа

Как только загрузчик начинает анализировать файл, во время анализа он может столкнуться с проблемами, которые сочтет достаточно важными, чтобы предупредить вас. Один пример возникает с PE-файлами, собранными без связанного файла Program Database (PDB). В таких случаях после завершения анализа появится диалог `Auto Analysis Summary`, включающий сообщение со сводкой обнаруженных проблем (см. рисунок 4-11).

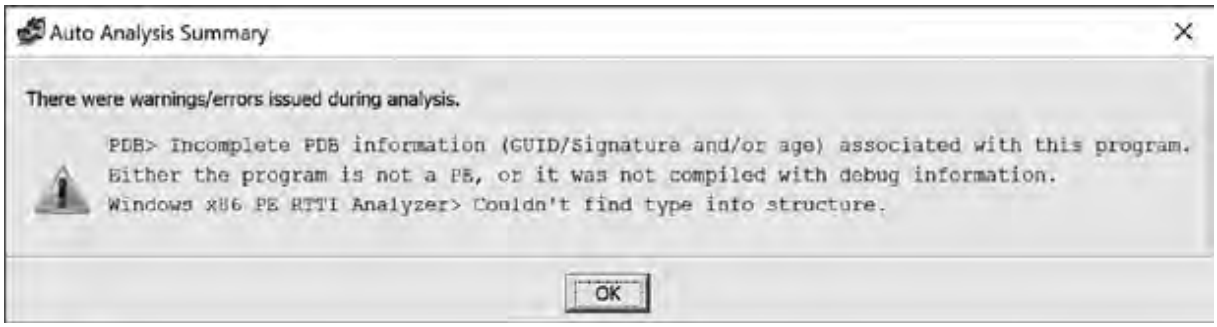


Рисунок 4-11. Диалог `Auto Analysis Summary`

В большинстве случаев сообщения носят информационный характер. В некоторых случаях они являются инструктивными и предлагают способы решить проблему, возможно установив необязательную стороннюю утилиту, которую Ghidra сможет использовать в будущем.

После автоанализа файла в Ghidra можно увидеть, что сведения из сводки импорта дополнены новой информацией о файле, как показано на рисунке 4-12.

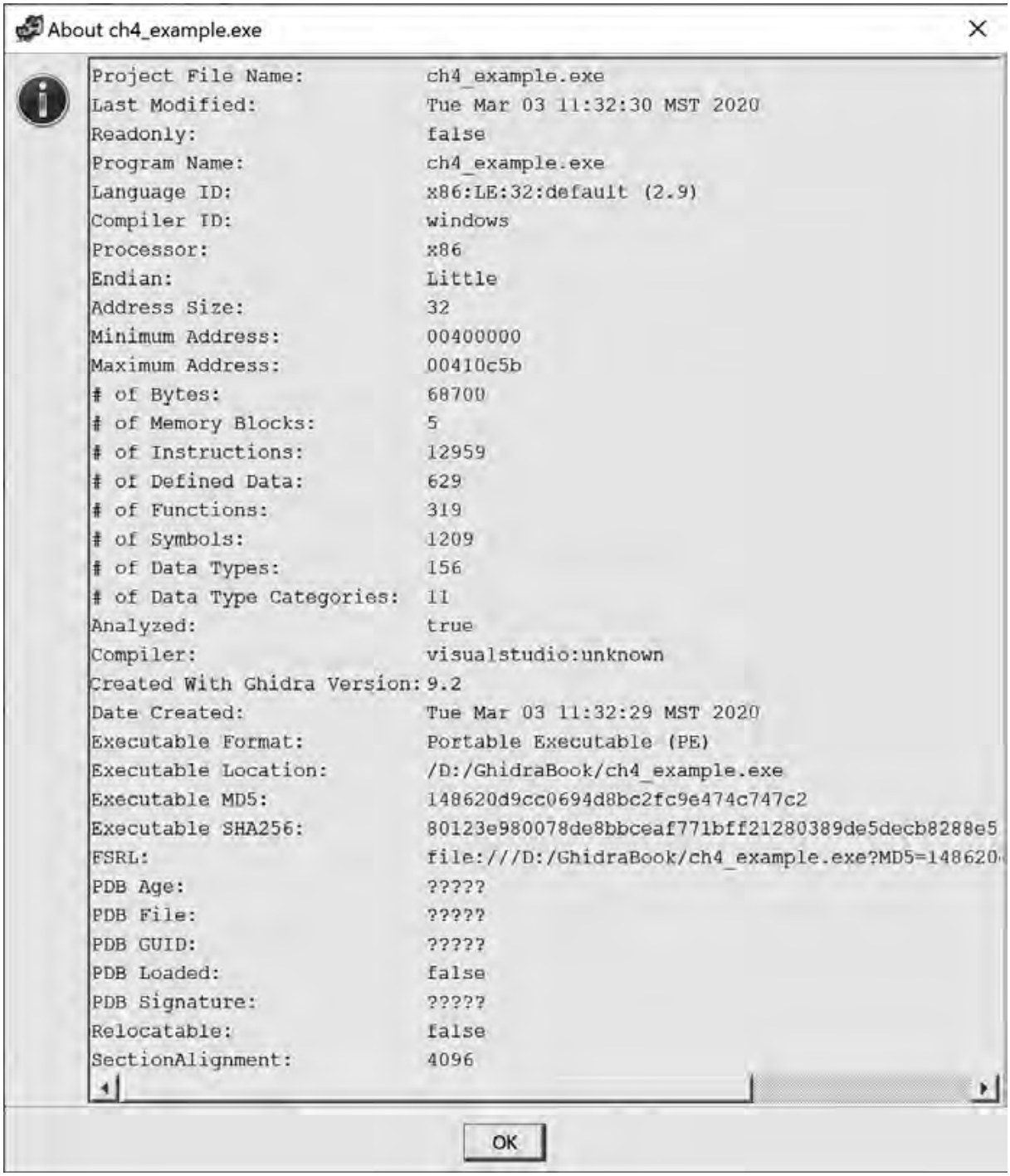


Рисунок 4-12. Представление `Ghidra Help > About` *ch4_example.exe со сведениями из сводки импорта*

Результаты автоанализа

Автоанализ Ghidra выполняется путем запуска каждого выбранного анализатора по вновь загруженному бинарному файлу. Диалог `Analysis Options`, а также `Ghidra Help` предлагают описания каждого анализатора. Анализаторы по умолчанию выбраны потому, что исторически пользователи Ghidra считали их наиболее полезными для широкого диапазона типов файлов. В следующих разделах мы обсудим некоторые наиболее полезные сведения, извлекаемые из бинарного файла во время его первоначальной загрузки и последующего автоанализа.

Идентификация компилятора

Определение компилятора, использованного для сборки фрагмента программного обеспечения, может помочь понять соглашения о вызове функций, используемые в бинарном файле, а также определить, с какими библиотеками бинарный файл может быть скомпонован. Если компилятор можно определить при загрузке файла, автоанализ Ghidra включит знания о поведении, специфичном для этого компилятора. Различия, которые можно наблюдать при использовании разных компиляторов и разных параметров времени компиляции, являются темой главы 20.

Идентификация аргументов функций и локальных переменных

Внутри каждой выявленной функции, определенной по записям таблицы символов и адресам, являющимся целями инструкций вызова, Ghidra выполняет подробный анализ поведения регистра указателя стека, чтобы как распознавать обращения к переменным, расположенным в стеке, так и понимать структуру стекового кадра функции. Имена для таких переменных автоматически создаются на основе их использования: либо как локальных переменных внутри функции, либо как размещенных в стеке аргументов, переданных в функцию в рамках процесса вызова функции. Стековые кадры подробнее обсуждаются в главе 6.

Информация о типах данных

Ghidra использует свои знания о распространенных библиотечных функциях и связанных с ними параметрах, чтобы идентифицировать функции, типы данных и структуры данных, используемые внутри каждой функции. Эта информация добавляется в окна `Symbol Tree` и `Data Type Manager`, а также в окно `Listing`. Такой процесс экономит огромное количество времени, предоставляя сведения, которые иначе пришлось бы вручную извлекать из различных справочников по интерфейсам прикладного программирования (API) и применять самостоятельно. Подробные сведения о том, как Ghidra обрабатывает библиотечные функции и связанные с ними типы данных, обсуждаются в главе 8.

Поведение рабочего стола во время первоначального анализа

Во время первоначального анализа недавно открытого файла на рабочем столе `CodeBrowser` происходит огромное количество действий. Понять ход анализа можно, наблюдая за обновлениями анализа в правом нижнем углу окна `CodeBrowser`. Это также позволяет следить за прогрессом анализа. Если вы не эксперт по скорочтению, можно открыть связанный файл журнала Ghidra и просмотреть действия в более спокойном темпе. Файл журнала можно открыть из окна `Ghidra Project`, выбрав `Help > Show Log`. Обратите внимание, что пункт меню Show Log доступен только в меню `Ghidra Project > Help`, а не в меню `CodeBrowser > Help`.

Следующий вывод взят из файла журнала, созданного Ghidra во время автоанализа `ch4_example.exe`, и является типичным для сообщений, формируемых в процессе автоанализа. Сообщения образуют повествование о процессе анализа и дают представление о последовательности операций, выполняемых Ghidra, а также о времени, потребовавшемся на каждую задачу во время анализа:

```
2019-09-23 15:38:26 INFO  (AutoAnalysisManager) ----------------------------
    ASCII Strings                             0.016 secs
    Apply Data Archives                       1.105 secs
    Call Convention Identification            0.018 secs
    Call-Fixup Installer                      0.000 secs
    Create Address Tables                     0.012 secs
    Create Function                           0.000 secs
    Data Reference                            0.014 secs
    Decompiler Parameter ID                   2.866 secs
    Decompiler Switch Analysis                2.693 secs
    Demangler                                 0.004 secs
    Disassemble Entry Points                  0.016 secs
    Embedded Media                            0.031 secs
    External Entry References                 0.000 secs
    Function ID                               0.312 secs
    Function Start Search                     0.051 secs
    Function Start Search After Code          0.006 secs
    Function Start Search After Data          0.005 secs
    Non-Returning Functions - Discovered      0.062 secs
    Non-Returning Functions - Known           0.000 secs
    PDB                                       0.000 secs
    Reference                                 0.025 secs
    Scalar Operand References                 0.074 secs
    Shared Return Calls                       0.000 secs
    Stack                                     0.063 secs
    Subroutine References                     0.016 secs
    Windows x86 PE Exception Handling         0.000 secs
    Windows x86 PE RTTI Analyzer              0.000 secs
    WindowsResourceReference                  0.100 secs
    X86 Function Callee Purge                 0.001 secs
    x86 Constant Reference Analyzer           0.509 secs
-------------------------------------------------
     Total Time   7 secs
-------------------------------------------------
2019-09-23 15:38:26 DEBUG (ToolTaskManager)   task finish (8.128 secs)
2019-09-23 15:38:26 DEBUG (ToolTaskManager)   Queue - Auto Analysis
2019-09-23 15:38:26 DEBUG (ToolTaskManager)   (0.0 secs)
2019-09-23 15:38:26 DEBUG (ToolTaskManager)   task Complete (8.253 secs)
```

Даже до завершения автоанализа можно начать перемещаться по различным представлениям данных. Когда автоанализ завершен, безопасно вносить любые желаемые изменения в файл проекта.

Сохранение работы и выход

Когда нужно сделать перерыв в анализе, разумно сохранить работу. В окне `CodeBrowser` это легко сделать любым из следующих способов:

• использовать один из вариантов `Save` в меню `CodeBrowser > File`;
• нажать значок `Save` на панели инструментов `CodeBrowser`;
• закрыть окно `CodeBrowser`;
• сохранить проект в окне Ghidra;
• выйти из Ghidra через меню Ghidra File.

В каждом случае будет предложено сохранить все измененные файлы. Более подробная информация об изменении внешнего вида и функциональности `CodeBrowser` и других инструментов Ghidra обсуждается в главе 12.

Советы и приемы для рабочего стола Ghidra

Ghidra отображает огромное количество информации, и ее рабочий стол может стать загроможденным. Вот несколько быстрых советов, которые помогут использовать рабочий стол максимально эффективно:

- чем больше места на экране вы выделите Ghidra, тем счастливее будете. Используйте этот факт, чтобы оправдать покупку монитора королевского размера или четырех;
- не забывайте использовать меню `Window` в `CodeBrowser` как средство открытия новых представлений или восстановления представлений данных, которые вы случайно закрыли. Многие окна также можно открыть с помощью кнопок инструментов на панели `CodeBrowser`;
- когда вы открываете новое окно, оно может появиться перед существующим окном. В таком случае ищите вкладки в верхней или нижней части окон, позволяющие переключаться между ними;
- любое окно можно закрыть, открыть заново по мере необходимости и перетащить в новое место на рабочем столе `CodeBrowser`;
- внешним видом представлений можно управлять через `Edit > Tool Options`, найдя соответствующие параметры `Display`.

Хотя эти указания лишь верхушка айсберга, они должны помочь, когда вы начнете перемещаться по рабочему столу `Ghidra CodeBrowser`. Дополнительные советы и приемы `CodeBrowser`, включая сочетания клавиш и параметры панели инструментов, обсуждаются в главе 5.

Итоги

Знакомство с рабочим столом `CodeBrowser` значительно улучшит ваш опыт работы с Ghidra. Обратная разработка бинарного кода достаточно сложна и без борьбы с инструментами. Параметры, выбранные во время первоначальной загрузки, и связанный анализ, выполняемый Ghidra, задают основу для всего анализа, который вы будете делать позже. На этом этапе вас может устроить работа, которую Ghidra выполнила от вашего имени, и для простых бинарных файлов этого может быть достаточно. С другой стороны, если вам интересно, как получить дополнительный контроль над процессом обратной разработки, теперь вы готовы глубже погрузиться в функциональность многочисленных представлений данных Ghidra. В следующих главах вы познакомитесь с каждым из основных представлений, обстоятельствами, при которых каждое из них полезно, и тем, как овладеть инструментами и представлениями, чтобы оптимизировать рабочий процесс.

Глава 5. Представления данных Ghidra

К этому моменту вы уже должны достаточно уверенно создавать проекты, загружать в них бинарные файлы и запускать автоанализ. После завершения начальной фазы анализа Ghidra пора брать управление в свои руки. Как обсуждалось в главе 4, при запуске Ghidra ваше приключение начинается в окне `Ghidra Project`. Когда вы открываете файл внутри одного из своих проектов, открывается второе окно. Это `Ghidra CodeBrowser`, и именно оно станет основной базой для значительной части ваших усилий SRE. Вы уже использовали `CodeBrowser` для автоанализа файла; теперь мы глубже рассмотрим меню `CodeBrowser`, его окна и базовые параметры, чтобы расширить ваше понимание возможностей Ghidra и позволить создать среду SRE-анализа, соответствующую вашему личному рабочему процессу. Начнем с основных представлений данных Ghidra.

CodeBrowser

Окно `CodeBrowser` можно открыть из окна `Ghidra Project`, выбрав `Tools > Run Tool > CodeBrowser`. Хотя `CodeBrowser` обычно открывается при выборе файла для анализа, здесь мы открываем пустой экземпляр, чтобы продемонстрировать функциональность и параметры настройки без влияния содержимого, связанного с конкретным файлом, как показано на рисунке 5-1. В стандартной конфигурации `CodeBrowser` имеет шесть подокон. Прежде чем переходить к деталям каждого из этих представлений, немного посмотрим на меню `CodeBrowser` и связанную с ним функциональность.

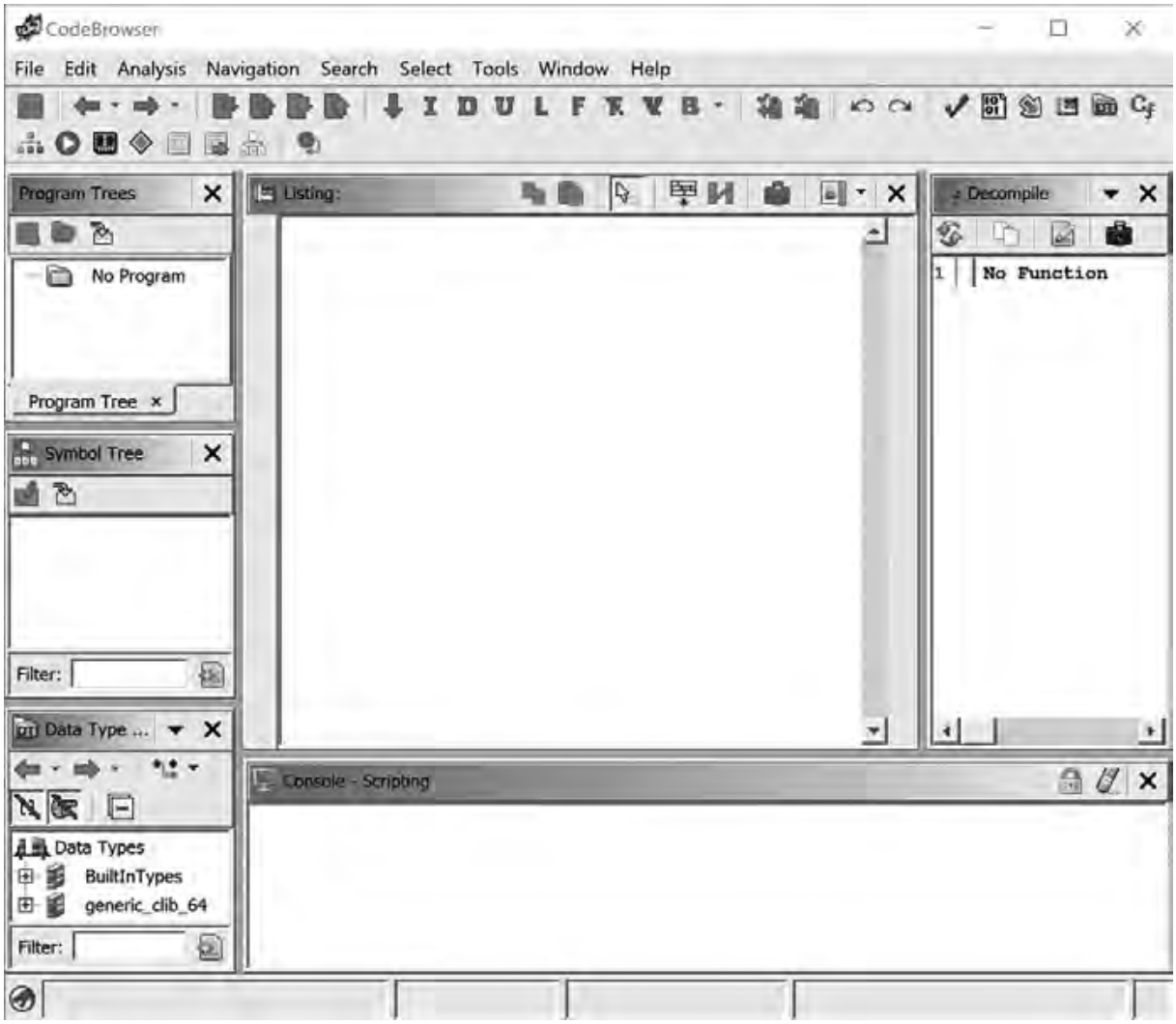


Рисунок 5-1. Незаполненное окно `CodeBrowser`

В верхней части окна `CodeBrowser` находится главное меню, а непосредственно под ним - панель инструментов. Панель инструментов предоставляет ярлыки в один щелчок для некоторых наиболее часто используемых пунктов меню. Поскольку сейчас у нас не загружен файл, в этом разделе мы сосредоточимся на пунктах меню, не связанных с загруженным файлом. Другие действия меню будут продемонстрированы и объяснены в контексте их применимости к процессу SRE.

File. Предоставляет базовую функциональность, ожидаемую в большинстве меню работы с файлами, включая варианты `Open/Close`, `Import/Export`, `Save` и Print. Кроме того, некоторые параметры специфичны для Ghidra, например Tool options, позволяющие сохранять инструмент `CodeBrowser` и управлять им, и `Parse C Source`, который может помочь процессу декомпиляции, извлекая сведения о типах данных из заголовочных файлов C. См. раздел "Разбор заголовочных файлов C" на странице 269.

Edit. Включает одну команду, применимую вне отдельных подокон: команду `Edit > Tool Options`, которая открывает новое окно и позволяет управлять параметрами и настройками, связанными с многочисленными инструментами, доступными из `CodeBrowser`. Параметры, связанные с консолью, показаны на рисунке 5-2. Кнопка `Restore Defaults`, то есть возврат к настройкам по умолчанию, всегда доступна в правом нижнем углу.

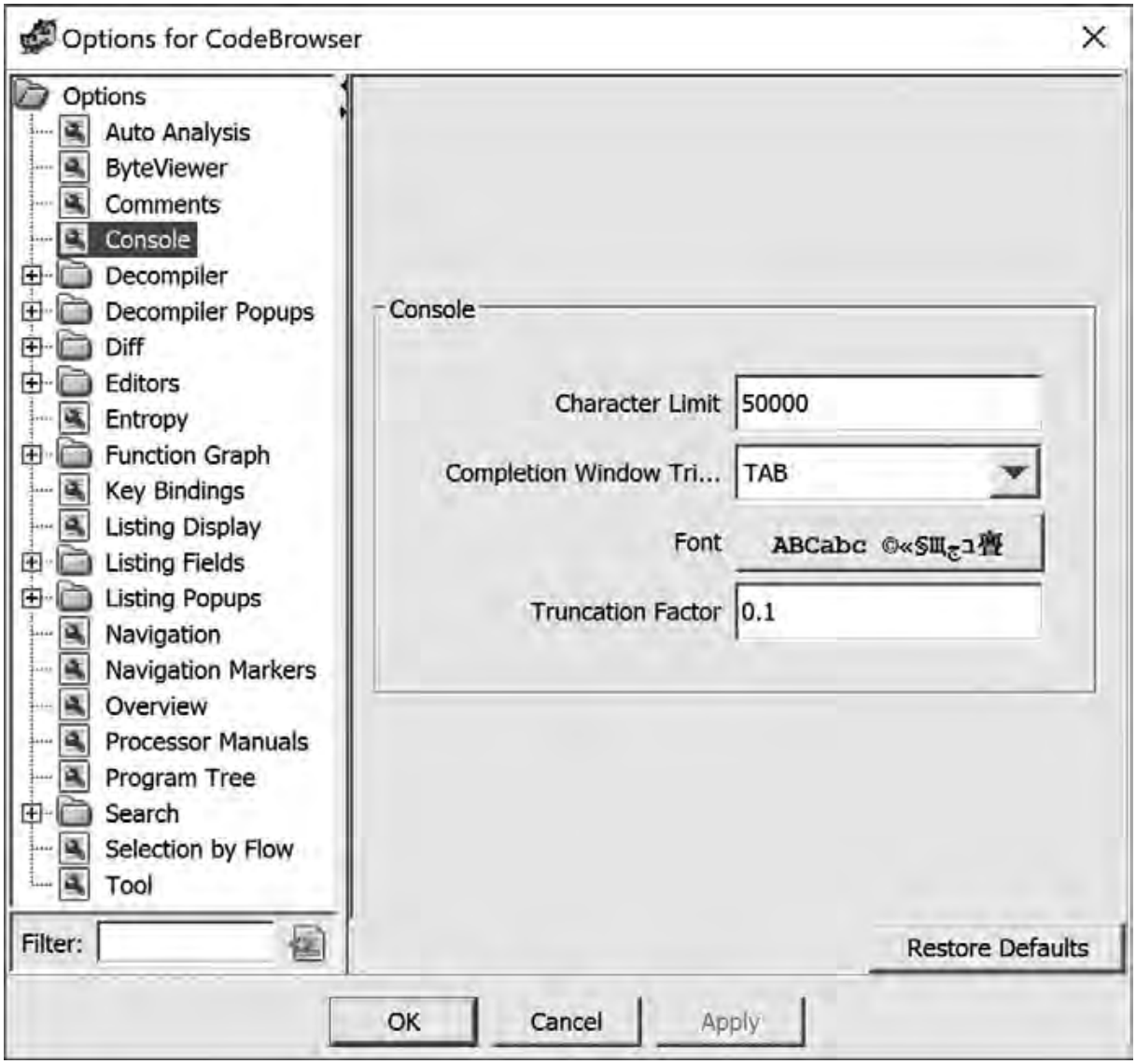


Рисунок 5-2. Параметры редактирования консоли `CodeBrowser`

`Analysis`**.** Позволяет повторно анализировать бинарный файл или выборочно выполнять отдельные задачи анализа. Базовые параметры анализа были введены в разделе "Анализ файлов с помощью Ghidra" на странице 48.

Navigation. Облегчает перемещение внутри файлов. Это меню предоставляет базовую клавиатурную функциональность, поддерживаемую многими приложениями, и добавляет специальные варианты навигации для бинарных файлов. Хотя меню дает один способ перемещения по файлу, после приобретения опыта с множеством доступных вариантов навигации вы, вероятно, будете использовать параметры панели инструментов или сочетания клавиш, указанные справа от каждого пункта меню.

Search. Предоставляет возможности поиска по памяти, тексту программы, строкам, таблицам адресов, прямым ссылкам, шаблонам инструкций и многому другому. Базовая функциональность поиска вводится в разделе "Поиск" на странице 114. Более специализированные концепции поиска будут представлены в контексте многочисленных примеров в последующих главах.

Select. Предоставляет возможность выделить часть файла для рассмотрения в рамках конкретной задачи. Выделения могут основываться на подпрограммах, функциях, потоках управления или просто на подсвечивании нужного участка файла.

Tools. Включает несколько интересных функций, позволяющих разместить на рабочем столе дополнительные ресурсы SRE. Одна из самых полезных - вариант `Processor Manual`, который открывает руководство процессора, связанное с текущим файлом. Если попытаться открыть отсутствующее руководство процессора, будет предложен способ добавить это руководство, как показано на рисунке 5-3.

Рисунок 5-3. Сообщение об отсутствующем руководстве процессора

`Window`**.** Позволяет настроить рабочую среду Ghidra под ваш рабочий процесс. Большую часть этой главы мы посвятим знакомству со стандартными окнами Ghidra и исследованию как их, так и некоторых других окон, которые окажутся полезными.

`Help`**.** Предоставляет богатые, хорошо организованные и очень подробные возможности. Окно `Help` поддерживает поиск, разные представления, избранное, увеличение и уменьшение масштаба, а также параметры печати и настройки страницы.

Окна CodeBrowser

Развернутое меню `Window` видно в центральной части рисунка 5-4. По умолчанию при запуске `CodeBrowser` открываются шесть доступных окон: `Program Trees`, `Symbol Tree`, `Data Type Manager`, `Listing`, Console и `Decompiler`. Имя каждого окна отображается в левом верхнем углу соответствующего окна. Каждое из этих окон присутствует как пункт в меню `Window`, а для некоторых также есть связанные значки на панели инструментов непосредственно под меню. Например, на рисунке 5-4 стрелками выделены пункт панели инструментов и пункт меню для открытия и доступа к окну `Decompiler`.

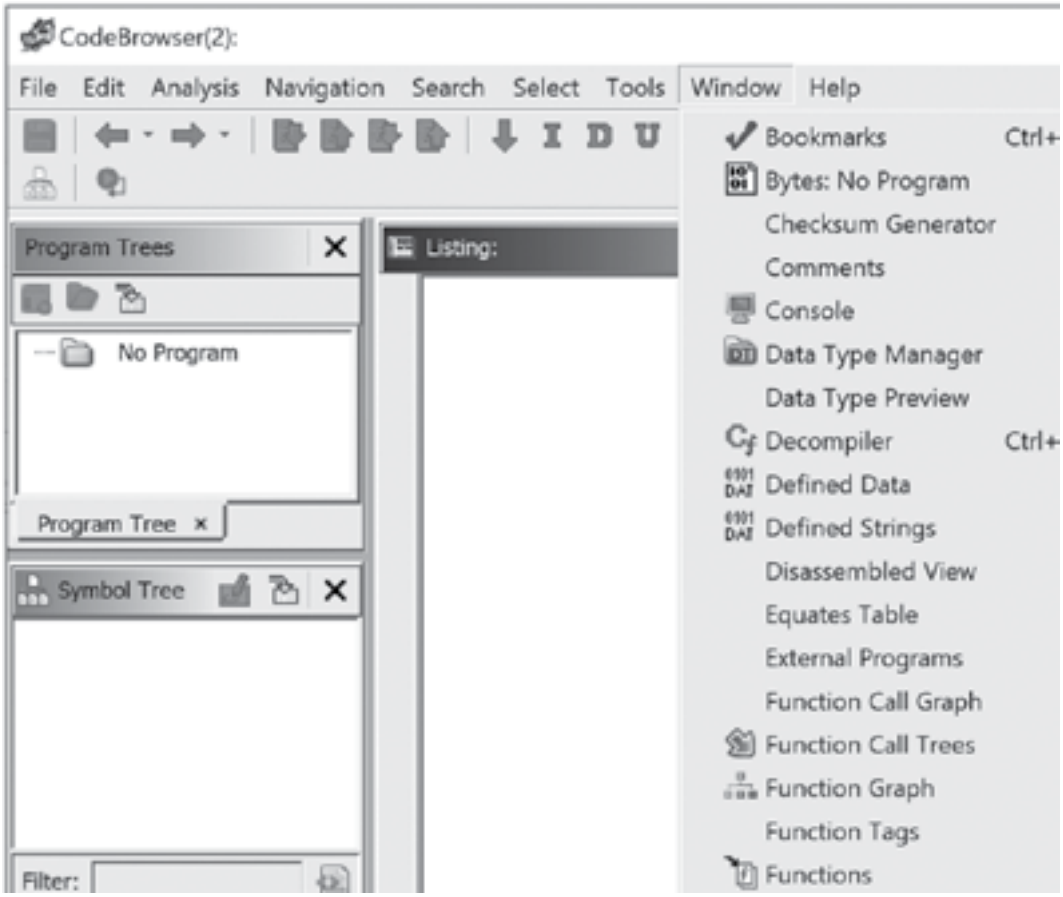


Рисунок 5-4. Окно `CodeBrowser` *с выделенными вариантами отображения окна* `Decompiler`

Давайте погрузимся в шесть стандартных окон, чтобы понять их фундаментальную важность в процессе SRE.

Горячие клавиши, кнопки и панели - ого!

Почти все часто используемые действия в Ghidra имеют связанный пункт меню, горячую клавишу и кнопку на панели инструментов. Если их нет, вы можете их создать. Панель инструментов Ghidra очень гибко настраивается, как и сопоставление горячих клавиш с действиями меню. См. `CodeBrowser Edit > Tool Options > Key Bindings` или просто наведите указатель на команду и нажмите `F4`. Как будто этого недостаточно, Ghidra также предлагает хорошие контекстно-зависимые действия меню в ответ на щелчки правой кнопкой мыши. Хотя такие контекстные меню не предлагают исчерпывающий список допустимых действий в заданном месте, они служат хорошими напоминаниями о наиболее распространенных действиях, которые вы будете выполнять. Эта гибкость позволяет выполнять действие самым удобным для вас способом и настраивать среду по мере того, как вы узнаете, как Ghidra может работать на вас.

Окна внутри и снаружи

Когда вы начнете исследовать различные окна Ghidra, вы заметите, что по умолчанию одни окна открываются внутри рабочего стола `CodeBrowser`, а другие - как новые плавающие окна за пределами рабочего стола `CodeBrowser`. Потратим минуту на обсуждение этих "внутренних" и "внешних" окон в контексте среды Ghidra.

"Внешние" окна плавают за пределами среды `CodeBrowser` и могут быть связанными или независимыми. Эти окна позволяют изучать их содержимое бок о бок с `CodeBrowser`. Примеры таких окон - Function Graph, Comments и Memory Map.

Далее существуют три отдельных класса "внутренних" окон:

- окна, которые по умолчанию открываются в CodeBrowser, например Symbol Tree и Listing;
- окна, которые складываются вместе со стандартным окном CodeBrowser, например Bytes;
- окна, которые создают пространство вместе с другими окнами CodeBrowser или разделяют его, например Equates и External Programs.

Когда вы открываете окно, разделяющее пространство с другим открытым окном, оно появляется перед существующим окном. Все окна, разделяющие одно пространство, снабжаются вкладками для быстрого перехода между окнами. Если вы хотите одновременно видеть два окна, разделяющие одно пространство, можно щелкнуть заголовок окна и перетащить его за пределы окна CodeBrowser.

Но будьте осторожны. Вернуть окна обратно в окно CodeBrowser не так просто, как вынести их наружу. Подробнее см. раздел "Перестановка окон" на странице 242.

Где моё окно?

В Ghidra много окон, и бывает трудно отслеживать, где они находятся в конкретный момент. Это становится еще сложнее, когда вы открываете больше окон, а другие исчезают за ними в CodeBrowser или на рабочем столе. В Ghidra есть уникальная функция, помогающая находить такие пропавшие окна. Щелчок по соответствующему значку панели инструментов или пункту меню выведет выбранное окно на передний план, но этого может оказаться недостаточно. Если продолжать щелкать значок панели инструментов для окна, пропавшее окно попытается привлечь ваше внимание: будет вибрировать, менять размер шрифта или цвета, масштабироваться, вращаться и выполнять другие заметные движения, которые наверняка бросятся в глаза и помогут его найти. Если вам скучно, можете помахать ему в ответ.

Окно Listing

Окно Listing, также известное как окно Disassembly, будет вашим основным инструментом для просмотра, изменения и анализа дизассемблированных представлений, созданных Ghidra. Текстовое представление показывает полный листинг дизассемблирования программы и служит главным способом просмотра областей данных бинарного файла.

Представление CodeBrowser для ch5_example1.exe показано в стандартной конфигурации на рисунке 5-5. Поле слева от окна Listing предоставляет важные сведения о файле, а также о вашем положении внутри файла. Справа от окна Listing, непосредственно справа от вертикальной полосы прокрутки, есть дополнительная область маркеров, которая также предоставляет важную информацию и возможности навигации. Полоса прокрутки показывает ваше положение в файле и может использоваться для навигации. Непосредственно справа от полосы прокрутки находятся некоторые информационные представления, включая закладки, которые дают дополнительное понимание файла.

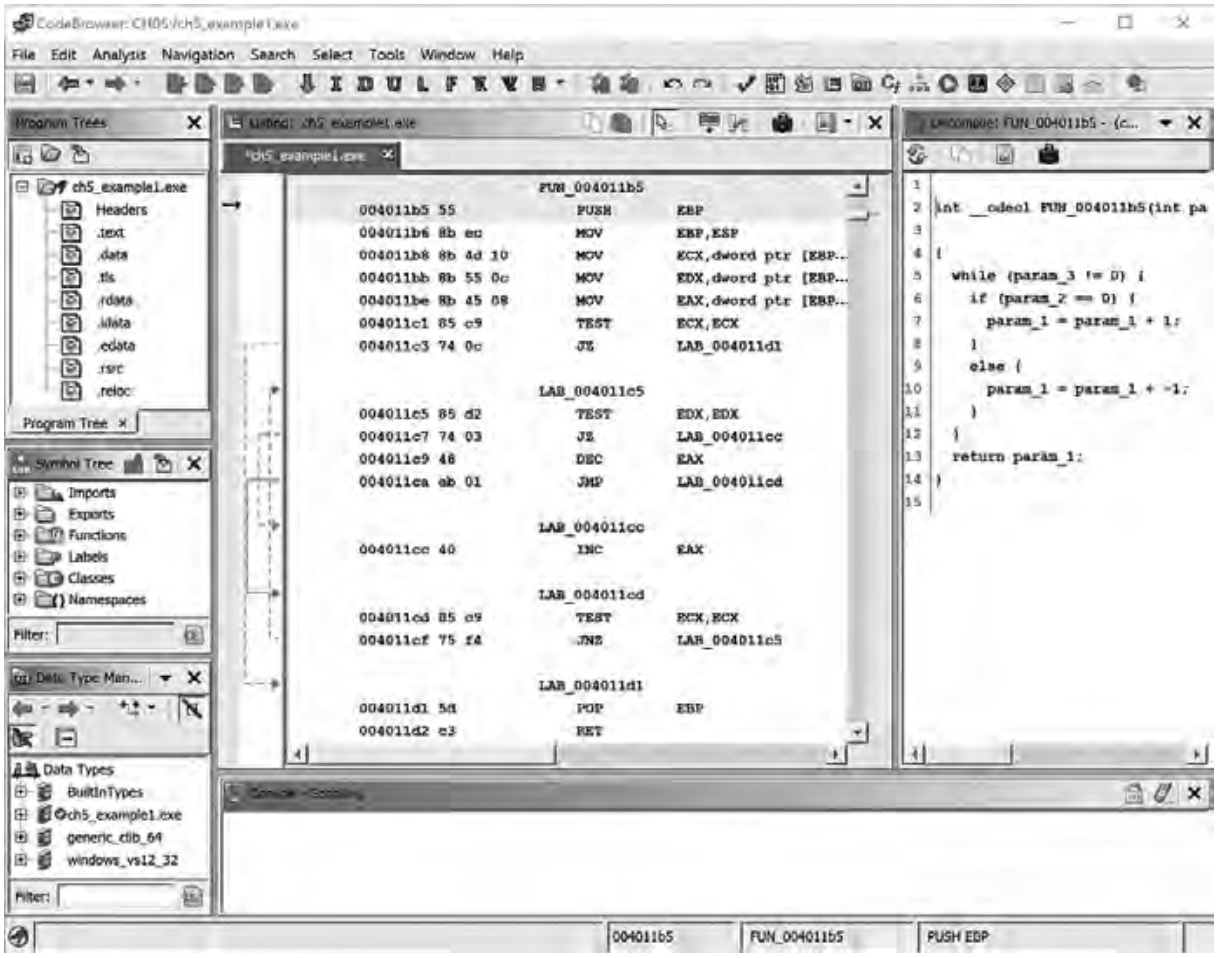


Рисунок 5-5. Стандартное окно `CodeBrowser` *с загруженным ch5_example1.exe*

Ваши любимые панели

После автоанализа файла можно использовать информационные боковые панели, чтобы перемещаться по файлу и продолжать его анализ. По умолчанию отображается только панель Navigation. Можно добавить или скрыть панели `Overview` и Entropy с помощью кнопки инструмента `Toggle Overview Margin` в правом верхнем углу окна `Listing` (см. рисунок 5-6). Независимо от того, какие панели отображаются, навигационный маркер слева от всех панелей напоминает, где вы находитесь в файле. Щелчок левой кнопкой мыши в любом месте любой из панелей переместит вас в это место файла и обновит содержимое окна `Listing`.

Теперь, когда вы знаете, как управлять появлением и исчезновением панелей, исследуем, что показывает каждая панель и как ее можно использовать в процессе SRE.

Область Navigation Marker. Позволяет перемещаться по файлу, но у нее есть и другая очень важная функция: если щелкнуть область Navigation Marker правой кнопкой мыши, вы увидите классы маркеров и закладок, которые могут быть связаны с файлом. Выбирая и снимая выбор типов маркеров, можно управлять тем, что отображается на панели Navigation. Это позволяет легко перемещаться по конкретным типам маркеров, например по подсветкам.

Панель `Overview`**.** Предоставляет важную визуальную информацию о содержимом файла. Горизонтальные полосы на панели `Overview` представляют цветные области программы. Хотя Ghidra предоставляет цвета по умолчанию, связанные с распространенными категориями, такими как функции, внешние ссылки, данные и инструкции, цветовой схемой можно управлять через меню `Edit > Tool Options`. По умолчанию, если навести

указатель на область, можно увидеть подробную информацию о ней, включая тип области и связанный адрес, если он применим.

Панель Entropy. Предоставляет уникальную функциональность Ghidra: она "стереотипизирует" содержимое файла на основе окружающего его содержимого. Если внутри области очень мало вариации, ей назначается низкое значение энтропии. Если степень случайности высока, соответствующее значение энтропии также высоко. Наведение указателя мыши на горизонтальную полосу на панели Entropy покажет значение энтропии от 0.0 до 8.0, тип, например `.text`, а также связанный адрес в файле. Гибко настраиваемую панель Entropy можно использовать, чтобы помочь определить наиболее вероятное содержимое полосы. Дополнительные сведения об этой возможности и лежащей в ее основе математике можно найти в меню `Ghidra Help`.

На рисунке 5-6 приведена разбивка кнопок инструментов, специфичных для окна `Listing`. На рисунке 5-7 мы развернули окно `Listing` и увеличили его, чтобы рассмотреть, что в нем показано. Дизассемблированный текст представлен линейно, а крайний левый столбец по умолчанию отображает виртуальные адреса.

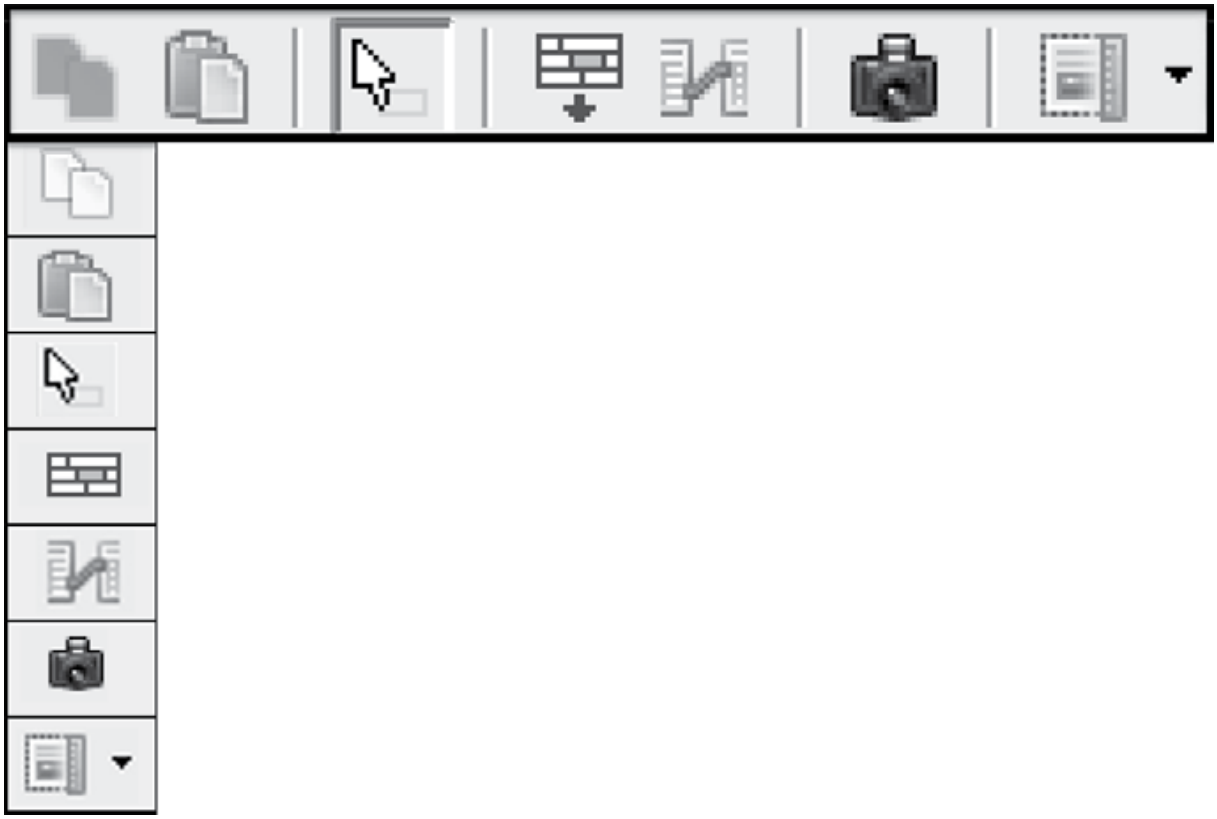


Рисунок 5-6. Кнопки инструментов окна `Listing`

Кнопка `Copy` и кнопка `Paste` предоставляют функциональность, доступную в ряде окон Ghidra; она меняется в зависимости от окна, в котором эти кнопки появляются, а также от содержимого, выбранного при активации операции. В некоторых случаях, когда содержимое несовместимо с операцией, вы увидите сообщение об ошибке. Кнопка `Toggle Mouse Hover Popups` позволяет выбрать, должны ли всплывающие подсказки при наведении мыши отображать информацию. `Browser Field Formatter` позволяет форматировать окно `Listing` (см. главу 12). `Open Diff View` позволяет сравнивать два файла (см. главу 23). `Snapshot` создает и открывает отсоединенную копию окна `Listing`. `Toggle Overview Margin Displays` позволяет выбрать, отображаются ли панели энтропии и обзора.

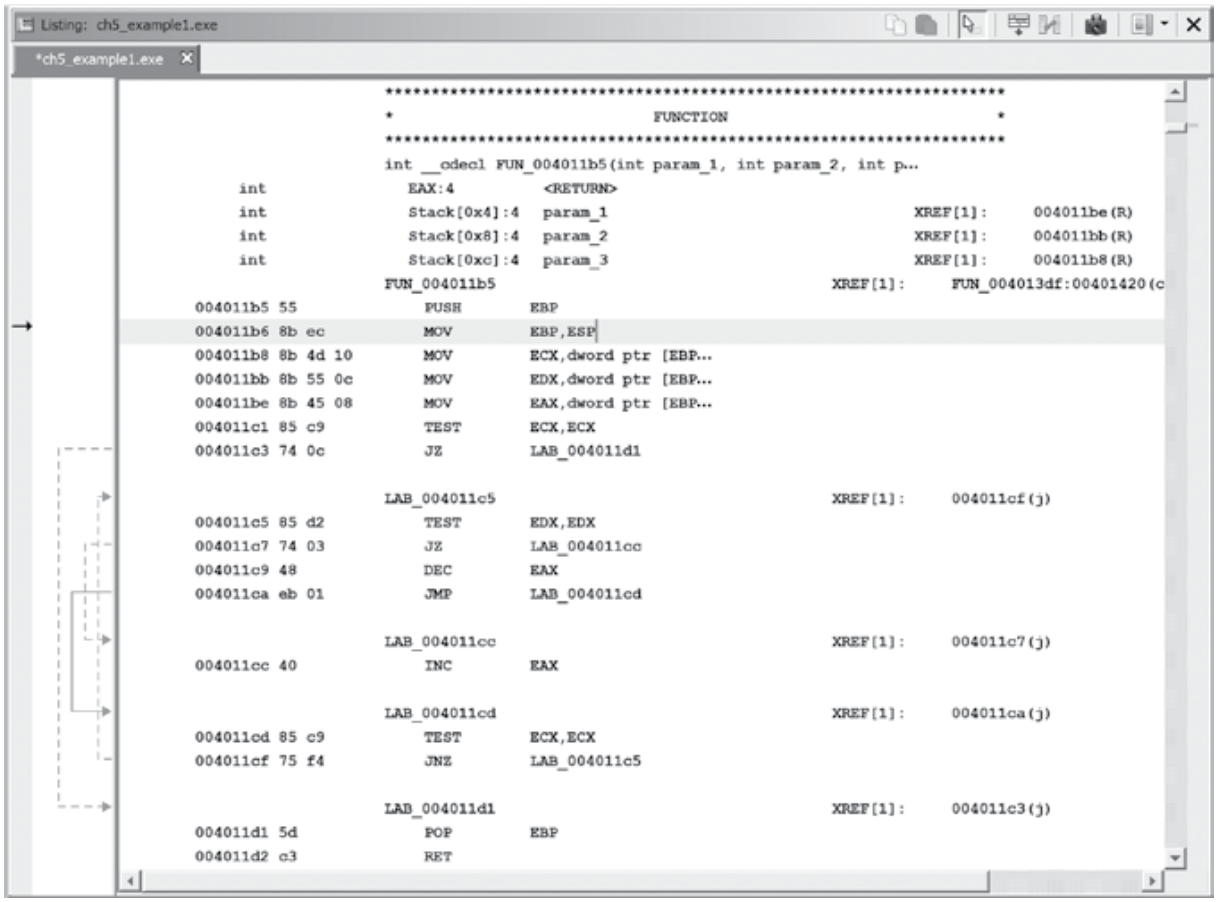


Рисунок 5-7. Окно `Listing` *с подписанными примерами артефактов*

Внутри окна `Listing` есть несколько элементов, заслуживающих внимания. Серая полоса в крайней левой части окна - это маркерное поле. Оно используется для указания текущего положения в файле и включает точечные маркеры и маркеры областей, которые описаны в `Ghidra Help`. В этом примере текущее положение в файле (`004011b6`) отмечено в маркерном поле маленькой черной стрелкой.

Область непосредственно справа от маркерного поля используется для графического изображения нелинейного потока внутри функции[1]. Когда исходный или целевой адрес инструкции потока управления виден в окне `Listing`, появляются связанные стрелки потока. Сплошные стрелки обозначают безусловные переходы, а пунктирные стрелки - условные переходы. Наведение указателя на линию потока открывает подсказку, отображающую начальный и конечный адрес потока, а также тип потока. Когда переход, условный или безусловный, передает управление на более ранний адрес в программе, это часто указывает на цикл. На рисунке 5-7 это демонстрируется стрелкой потока от адреса `004011cf` к `004011c5`. Можно легко перейти к источнику или назначению любого перехода, дважды щелкнув связанную стрелку потока.

Объявления в верхней части рисунка 5-7 показывают лучшую оценку Ghidra относительно структуры стекового кадра функции[2]. Ghidra вычисляет структуру стекового кадра функции, то есть локальные переменные, выполняя подробный анализ поведения указателя стека и любого указателя стекового кадра, используемого внутри функции. Представления стека подробнее обсуждаются в главе 6.

В листингах обычно есть многочисленные перекрестные ссылки на данные и код, обозначенные `XREF`, которые видны в правой части рисунка 5-7. Перекрестная ссылка создается каждый раз, когда одно место в дизассемблированном представлении ссылается на другое место в нем же. Например, инструкция по адресу A, переходящая к инструкции по адресу B, приведет к созданию перекрестной ссылки из A в B. Наведение указателя на адрес ссылки вызывает всплывающее окно

ссылки с местом, откуда ссылка выполнена. Всплывающее окно ссылки имеет такую же структуру, как окно `Listing`, но желтый фон, похожий на всплывающую подсказку. Это окно позволяет просматривать содержимое, но не позволяет переходить по ссылкам. Перекрестные ссылки являются темой главы 9.

Создание дополнительных окон дизассемблирования

Если вы когда-нибудь захотите одновременно видеть листинги двух функций, достаточно открыть еще одно окно дизассемблирования с помощью значка Snapshot на панели инструментов `Listing` (см. рисунок 5-6). Первое открытое окно дизассемблирования имеет префикс `Listing:` перед именем файла. Все последующие окна дизассемблирования называются `[Listing: <filename>]`, чтобы показать, что они отсоединены от основного представления. Снимки отсоединены, поэтому по ним можно свободно перемещаться, не влияя на другие окна.

Настройка окон `Listing`

Листинг дизассемблирования можно разложить на ряд составляющих полей, включая такие сведения, как поле мнемоники, поле адреса и поле комментария. Листинги, которые мы уже видели, составлены из стандартного набора полей, предоставляющих важную информацию о файле. Однако иногда стандартное представление не дает той информации, которую вы хотели бы видеть. Здесь на сцену выходит Browser Field Formatter.

Browser Field Formatter дает возможность настраивать более 30 полей, чтобы обеспечить полный контроль над внешним видом окон `Listing`. Browser Field Formatter можно активировать, нажав кнопку на панели инструментов `Listing` (см. рисунок 5-6). Это открывает мощное подменю и редактор макета, показанный на рисунке 5-8, в верхней части листинга. Browser Field Formatter позволяет управлять внешним видом разрывов адресов, блочных комментариев, функций, переменных, инструкций, данных, структур и массивов. В каждой из этих категорий есть поля, которые можно регулировать, подстраивать и контролировать, создавая идеальный для себя формат листинга. Мы в основном придерживаемся стандартных форматов листингов, но вам стоит изучить Browser Field Formatter и определить, улучшают ли какие-либо параметры ваше понимание содержимого окна `Listing`.

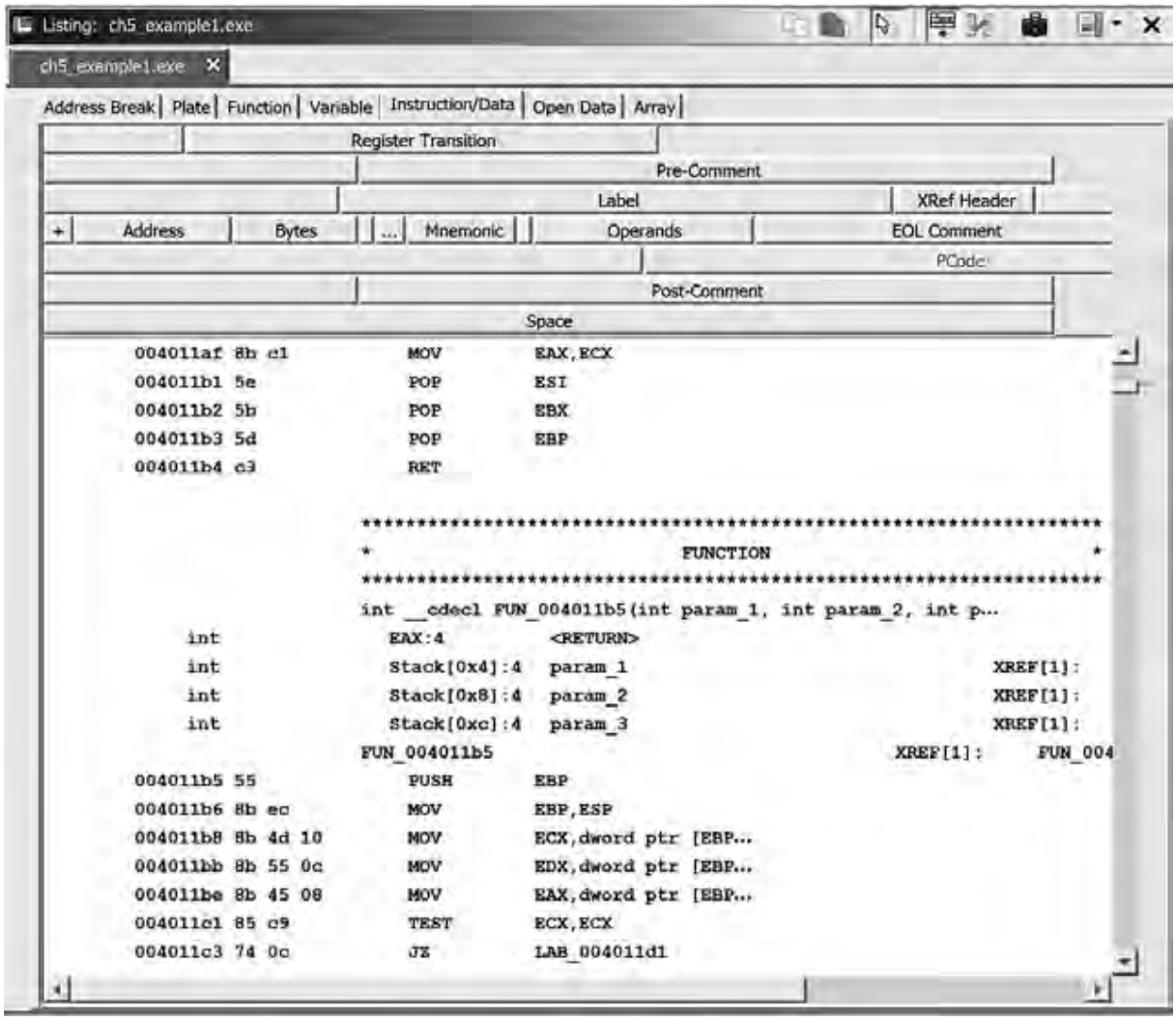


Рисунок 5-8. Окно `Listing` *с активированным Browser Field Formatter*

Представление Ghidra Function Graph

Хотя ассемблерные листинги интересны и информативны, поток программы может быть проще понять через графовое представление. Окно Function Graph, связанное с `CodeBrowser`, можно открыть, выбрав `Window > Function Graph` или щелкнув связанный значок на панели инструментов `CodeBrowser`. Окно Function Graph, соответствующее функции на рисунке 5-7, показано на рисунке 5-9. Графовые представления чем-то напоминают блок-схемы программ: функция разбивается на базовые блоки, чтобы можно было визуализировать поток управления функции от одного блока к другому[3].

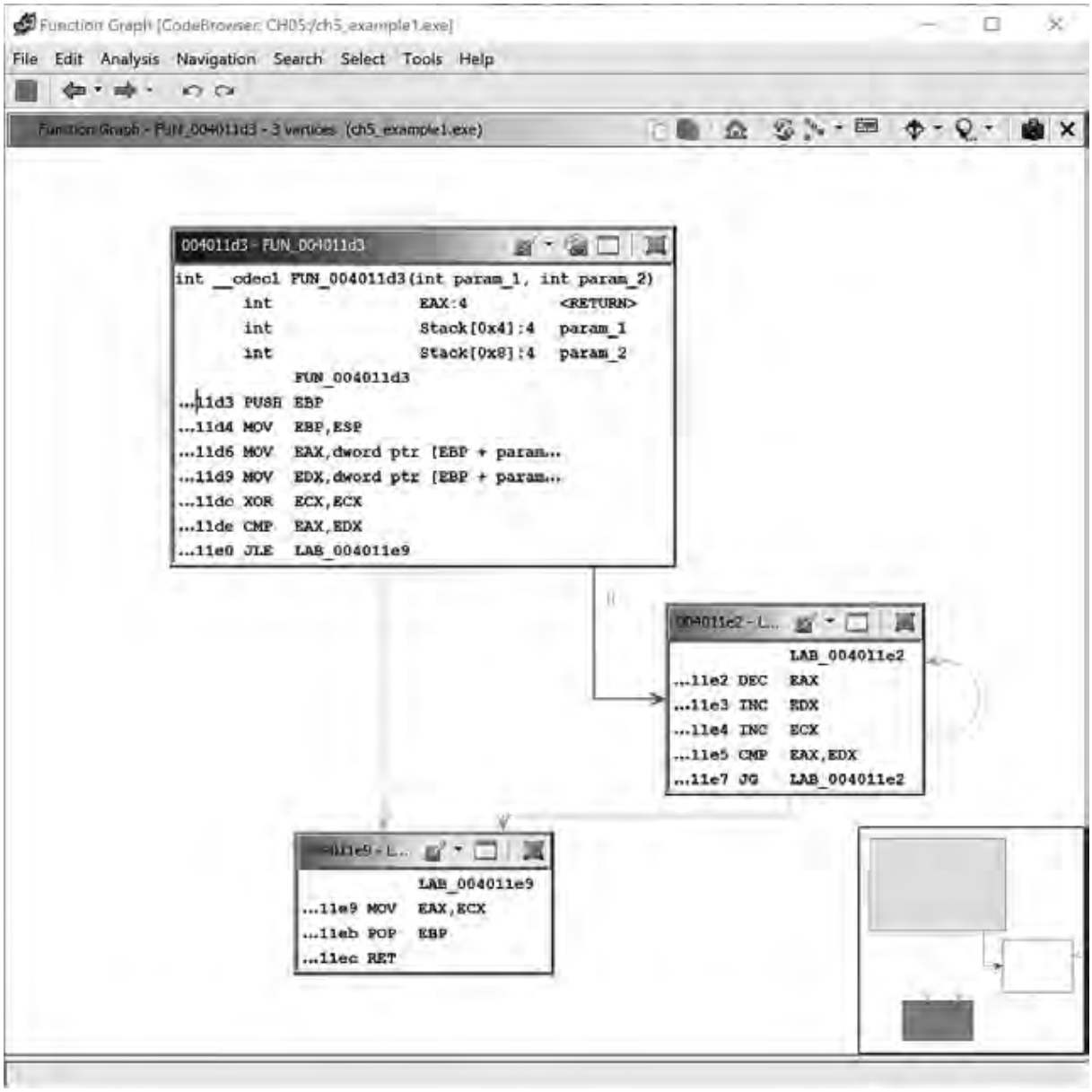


Рисунок 5-9. Графовое представление листинга с рисунка 5-7

На экране Ghidra использует стрелки разных цветов, чтобы различать разные типы потоков между блоками функции. Кроме того, при наведении мыши потоки анимируются, показывая направление. Базовые блоки, завершающиеся условным переходом, создают два возможных потока: стрелка Yes edge, то есть да, проверяемое условие выполнено, по умолчанию зеленая, а стрелка No edge, то есть нет, проверяемое условие не выполнено, по умолчанию красная. Базовые блоки, завершающиеся только одним потенциальным последующим блоком, используют Normal edge, по умолчанию синюю, чтобы указать следующий выполняемый блок. Можно щелкнуть любую стрелку, чтобы увидеть связанный переход от одного блока к другому. Поскольку инструменты графа и листинга по умолчанию синхронизированы, ваше положение в файле обычно остается согласованным при переключении между представлениями листинга и графа и при навигации внутри них. Исключения обсуждаются в главе 10, а также в `Ghidra Help`.

В режиме графа Ghidra отображает одну функцию за раз. Ghidra облегчает перемещение по графу с помощью традиционных приемов взаимодействия с изображениями, таких как панорамирование и масштабирование. Большие или сложные функции могут сделать граф чрезвычайно загроможденным, и по нему станет трудно перемещаться; именно здесь помогает Satellite View. По умолчанию Satellite View располагается в правом нижнем углу окна графа и может быть ценным

средством для понимания текущего положения (см. рисунок 5-9).

Спутниковая навигация

Satellite View всегда отображает полную блочную структуру графа вместе с выделенной рамкой, указывающей область графа, которая сейчас видна в окне дизассемблирования. Щелчок по любому блоку в Satellite View центрирует граф вокруг этого блока. Выделенная рамка действует как линза, и ее можно перетаскивать по обзорному окну, чтобы быстро переместить представление графа в любое место графа. Помимо средства навигации по окну Function Graph, это волшебное окно обладает и другими свойствами, которые могут работать как на вас, так и против вас при изучении файлов.

Это окно занимает ценное место в Function Graph и может скрывать важные блоки и содержимое именно тогда, когда вы хотите их видеть. Есть два способа исправить ситуацию. Можно щелкнуть Satellite View правой кнопкой мыши и снять флажок Dock Satellite View. Это переместит Satellite View со всей его функциональностью за пределы окна Function Graph. Повторное включение этого параметра в любой момент вернет его в исходное положение внутри окна Function Graph.

Второй вариант - скрыть Satellite View, если она не нужна для навигации. Это еще один флажок, доступный в контекстном меню по правому щелчку. Когда вы скрываете Satellite View, в правом нижнем углу окна Function Graph появляется маленький значок. Щелчок по этому значку восстановит Satellite View.

Когда Satellite View видима, она может заставлять основное представление работать медленнее, чем хотелось бы. Скрытие Satellite View может помочь сделать его более отзывчивым.

Помимо навигации с помощью Satellite View, представление внутри окна Function Graph можно изменять множеством способов под свои потребности.

Панорамирование. Во-первых, помимо использования Satellite View для быстрого перемещения графа, можно изменить положение графа, щелкнув фон и перетащив его, чтобы изменить представление графа.

Масштабирование. Можно увеличивать и уменьшать масштаб традиционными клавиатурными способами, такими как `ctrl`/`command`, прокруткой мыши или связанными сочетаниями клавиш. Если слишком сильно уменьшить масштаб, можно пройти порог отрисовки, при котором содержимое блоков больше не отображается. Каждый блок становится просто цветным прямоугольником. В некоторых случаях, особенно при работе бок о бок с окном `Listing`, это может быть преимуществом, поскольку повышает скорость отрисовки графа функции.

Перестановка блоков. Отдельные блоки внутри графа можно перетаскивать в новые положения, щелкнув строку заголовка нужного блока и переместив его на новое место. Все связи между блоками сохраняются при перемещении блоков. Если в какой-то момент вы захотите вернуться к стандартной компоновке графа, это можно сделать, выбрав значок Refresh на панели инструментов Function Graph.

Группировка и сворачивание блоков. Блоки можно группировать по отдельности или вместе с другими блоками и сворачивать, чтобы уменьшить загромождение представления. Группировка приводит к сворачиванию блока. Сворачивание блоков - простой способ отслеживать блоки, которые вы уже проанализировали. Любой блок можно свернуть, выбрав значок Group в крайней правой части панели инструментов блока. Если выбрать этот параметр при нескольких выделенных блоках, они будут свернуты, а список связанных блоков отобразится в сложенном окне. Некоторые нюансы, связанные с формированием и расформированием групп, а также с выполнением действий над вновь созданными группами, объясняются в `Ghidra Help`.

Инструменты устанавливают связи

Инструменты могут работать вместе или независимо. Мы видели, как окно `Listing` и окно Function Graph совместно используют данные и как события, происходящие в одном окне, влияют на другое. Если выбрать конкретный блок в окне Function Graph, соответствующий код будет подсвечен в окне `Listing`. И наоборот, перемещение между функциями в окне `Listing` приводит к обновлению окна Function Graph. Это одна из многих связей инструментов, которая возникает автоматически и является двунаправленной. В Ghidra также есть возможности для однонаправленных связей, а также возможность вручную соединять и разъединять инструменты с использованием модели "производитель/потребитель", связанной с событиями инструментов. В этой книге мы сосредоточимся на двунаправленных автоматических связях инструментов, предоставляемых Ghidra.

Настройка отображения графа

Чтобы помочь в анализе, Ghidra предоставляет строку меню в верхней части каждого узла в представлении Function Graph, позволяющую управлять отображением конкретного узла. Можно управлять цветом фона и текста узла, перейти к XREF, просмотреть полнооконный листинг узла графа и использовать функциональность группировки для объединения и сворачивания узлов. Обратите внимание, что изменение фона блока в Function Graph также изменяет фон в окне `Listing`. Некоторые из этих возможностей могут быть не нужны, если вы активно используете окно `Listing` вместе с окном Function Graph, но параметры настройки могут оказаться полезными и, безусловно, заслуживают изучения. Эти параметры подробнее обсуждаются в главе 10.

Поскольку графовое представление открывается в окне, внешнем по отношению к `CodeBrowser`, два представления можно видеть рядом. Поскольку между окнами есть связь, изменение положения в одном окне перемещает маркер положения в другом. Хотя многие пользователи склонны предпочитать одно представление другому для визуализации потока программы, выбирать только одно не обязательно. Также помните, что контроль над графовым и текстовым представлениями простирается далеко за пределы этих примеров. Дополнительные графовые возможности Ghidra рассматриваются в главе 10, а дополнительные сведения об управлении параметрами представлений Ghidra доступны в `Ghidra Help`.

В следующих пяти главах мы в основном будем использовать для примеров представление листинга, дополняя его графовым представлением в случаях, когда оно существенно повышает ясность. В главе 6 мы сосредоточимся на понимании дизассемблированного представления Ghidra, а в главе 7 рассмотрим особенности управления представлением листинга, чтобы очищать и аннотировать дизассемблированный код.

Перемещение

Помимо традиционных способов навигации по файлу, таких как стрелка вверх, стрелка вниз, Page Up, Page Down и так далее, Ghidra предоставляет навигационные инструменты, специфичные для процесса SRE. Значки на панели Navigation, показанной на рисунке 5-10, упрощают перемещение по программе. Познакомимся со значками, которые служат специалисту по обратной разработке.

Рисунок 5-10. Панель Navigation в `CodeBrowser`

Слева находится значок Direction. Эта стрелка переключается между направлением вверх и вниз и управляет направлением для всех остальных значков навигации. Следующие восемь значков продвигают вас по различным целям, показанным на рисунке 5-11.

Рисунок 5-11. Определения панели Navigation

Вместо того чтобы просто переводить вас к следующим данным в листинге, выбор варианта Data пропускает соседние данные и переносит к началу следующих несмежных данных. Instruction и Undefined демонстрируют такое же поведение. Стрелка раскрывающегося списка в крайней правой части панели Navigation отображает список, позволяющий выбрать конкретные типы закладок для быстрой навигации. Хотя эти навигационные ярлыки используются главным образом с окном `Listing`, они работают во всех окнах, соединенных с окном `Listing`. Навигация внутри любого из этих окон приводит к синхронной навигации во всех соединенных окнах.

Окно Program Trees

Вернемся к обсуждению стандартных окон `CodeBrowser` и кратко рассмотрим окно `Program Trees`, показанное на рисунке 5-12.

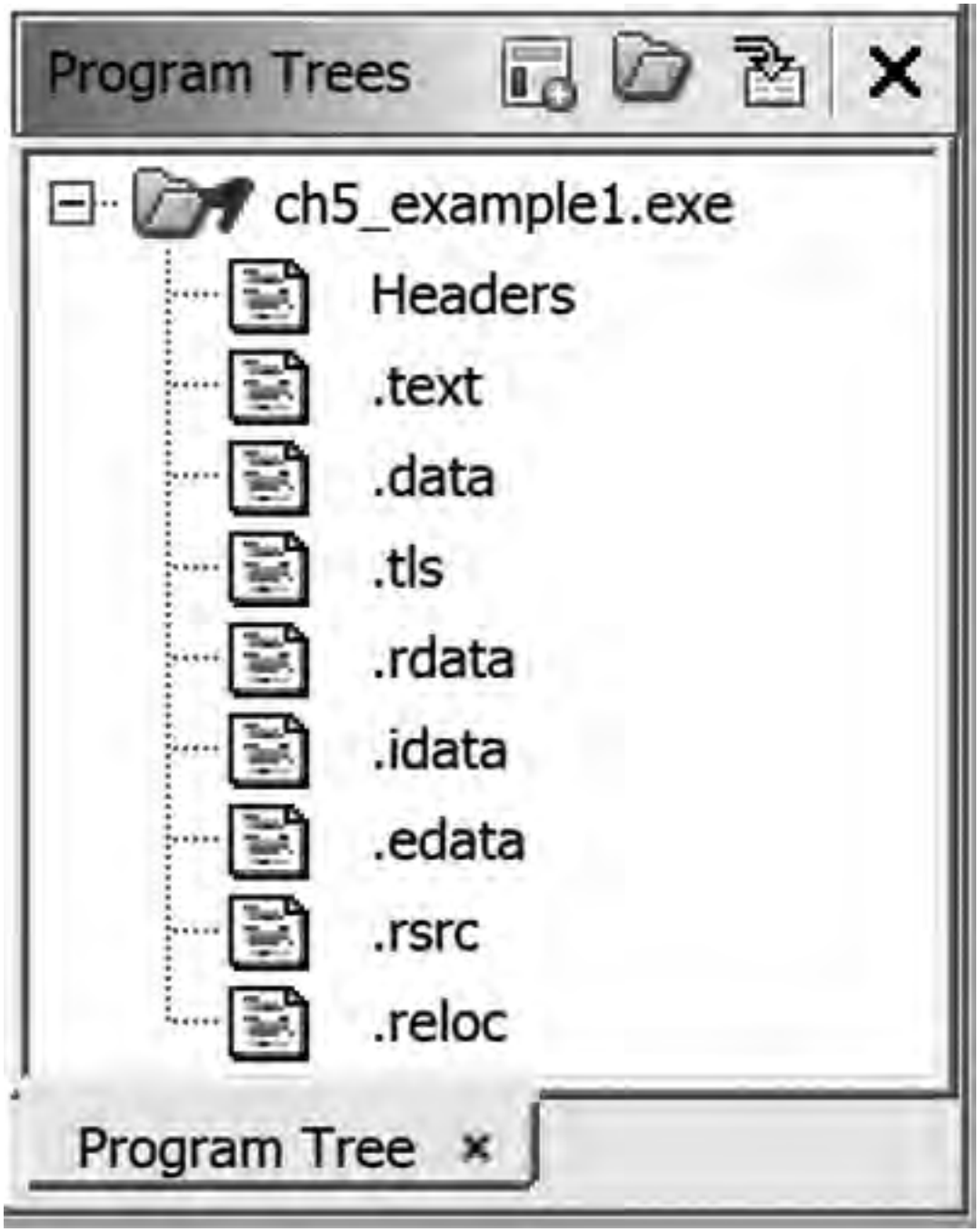


Рисунок 5-12. Окно `Program Trees`

Это окно показывает программу, организованную по папкам и фрагментам, и дает возможность уточнять организацию, которая создается во время автоанализа. Фрагмент - это термин Ghidra для непрерывного диапазона адресов. Фрагменты не могут перекрываться. Более традиционное название фрагмента - секция программы, например `.text`, `.data` и `.bss`. Операции, связанные с деревом программы, включают следующее:

• создать папку или фрагмент;
• развернуть, свернуть или объединить папки;
• добавить или удалить папки и фрагменты;
• определить содержимое в окне `Listing` и переместить его во фрагмент;
• сортировать по имени или адресу;
• выбирать адреса;
• копировать, вырезать и вставлять фрагменты или папки;

• изменять порядок папок.

Окно Program Trees является соединенным окном, поэтому щелчок по фрагменту в этом окне переводит вас к соответствующему месту в окне Listing. Дополнительные сведения об окне Program Trees можно найти в Ghidra Help.

Окно Symbol Tree

Когда вы импортируете файл в проект Ghidra, для загрузки содержимого файла выбирается модуль загрузчика Ghidra. Если в бинарном файле присутствует таблица символов, загрузчик способен извлечь сведения из нее, обсуждавшиеся в главе 2, для отображения в окне Symbol Tree, показанном на рисунке 5-13. Окно Symbol Tree включает imports, exports, functions, labels, classes и namespaces, связанные с программой. Каждая из этих категорий и связанные с ними типы символов обсуждаются в следующих разделах.

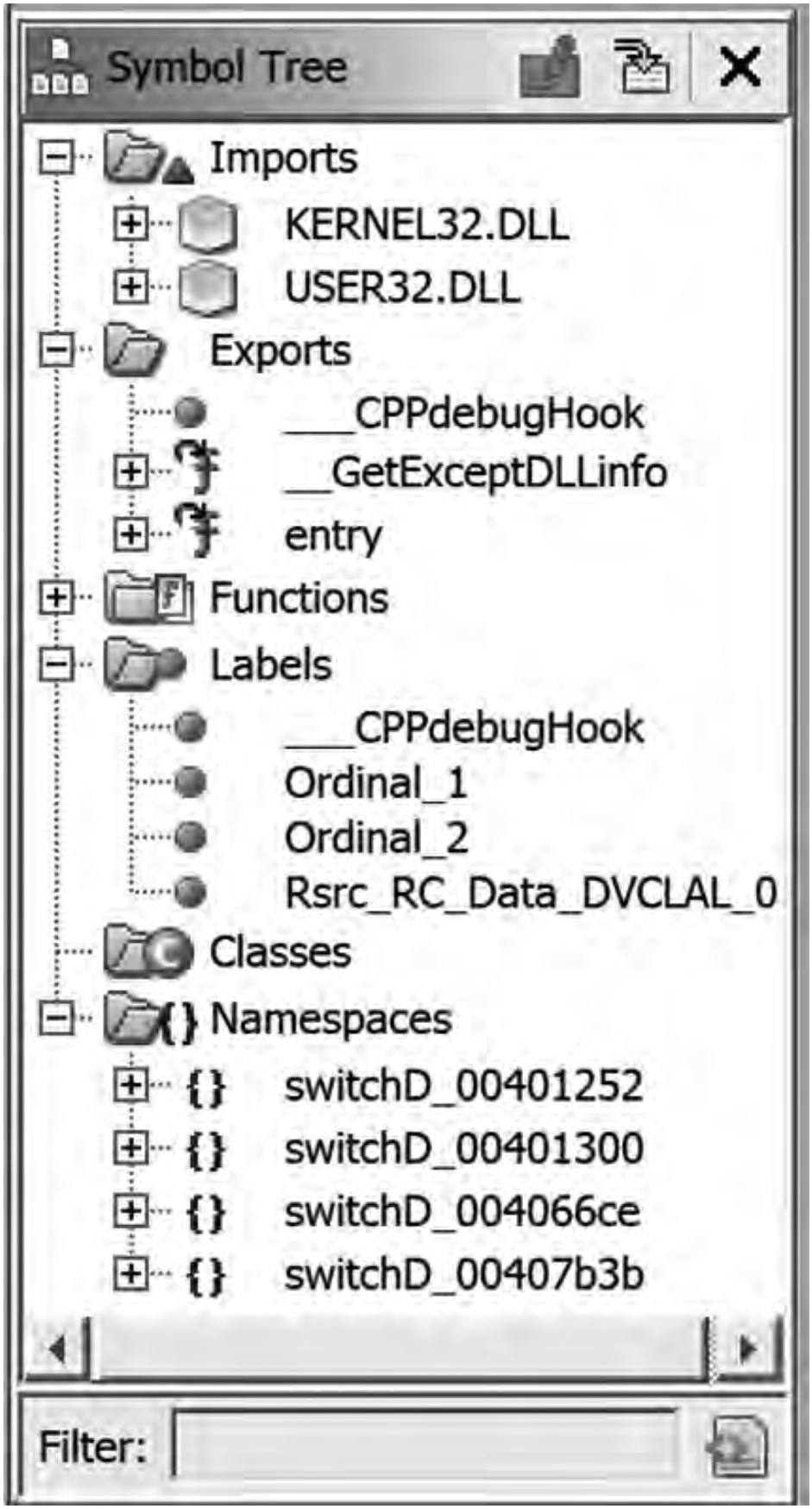


Рисунок 5-13. Окно `CodeBrowser` `Symbol Tree`

Все шесть папок `Symbol Tree` можно контролировать с помощью фильтра в нижней части окна `Symbol Tree`. Эта функциональность станет ценнее по мере знакомства с анализируемым файлом.

Кроме того, вы увидите, что окно `Symbol Tree` предлагает функциональность, похожую на инструменты командной строки, такие как `objdump (-T)`, `readelf (-s)` и `dumpbin (/EXPORTS)`.

Imports

Папка `Imports` в окне `Symbol Tree` перечисляет все функции, импортируемые анализируемым бинарным файлом. Она имеет смысл только тогда, когда бинарный файл использует разделяемые библиотеки: у статически скомпонованных бинарных файлов нет внешних зависимостей, а значит, нет и импортов. Папка `Imports` перечисляет импортируемые библиотеки с записями для каждого элемента, функции или данных, импортированного из этой библиотеки. Щелчок по любому символу в представлении `Symbol Tree` переводит все соединенные представления к выбранному символу. В нашем примере Windows-бинарника щелчок по `GetModuleHandleA` в папке `Imports` перевел бы окно дизассемблирования к записи таблицы адресов импорта для `GetModuleHandleA`, которая в этом примере находится по адресу `0040e108`, как показано на рисунке 5-14.

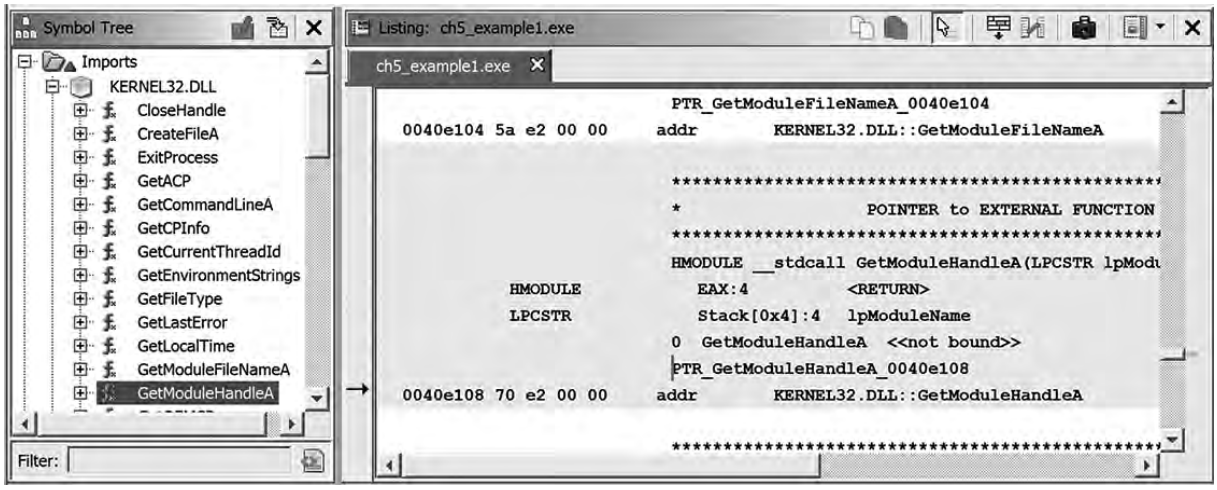


Рисунок 5-14. Запись таблицы адресов импорта и связанное место в окне `Listing`

Важный момент, который нужно помнить о категории `Imports`: она отображает только символы, названные в таблице импортов бинарного файла. Символы, которые бинарный файл решает загрузить самостоятельно с помощью механизма вроде `dlopen`/`dlsym` или `LoadLibrary`/`GetProcAddress`, не будут перечислены в окне `Symbol Tree`.

Exports

Папка Exports перечисляет точки входа в файл. Они включают точку входа выполнения программы, заданную в секции заголовка, а также любые функции и переменные, которые файл экспортирует для использования другими файлами. Экспортируемые функции часто встречаются в разделяемых библиотеках, таких как файлы Windows DLL. Экспортируемые записи перечисляются по имени, и соответствующий виртуальный адрес будет подсвечен в окне `Listing` при выборе экспорта. Для исполняемых файлов папка Exports всегда содержит как минимум одну запись: точку входа выполнения программы. Ghidra может назвать этот символ `entry` или `_start`, в зависимости от типа бинарного файла.

Functions

Папка `Functions` содержит список каждой функции, которую Ghidra выявила в бинарном файле. Наведение указателя на имя функции в окне `Symbol Tree` создает всплывающее окно с подробной информацией о функции, как показано на рисунке 5-15. В рамках процесса загрузки загрузчик использует различные алгоритмы, включая анализ структуры файла и сопоставление последовательностей байтов, чтобы вывести компилятор, использованный для создания файла. Во время фазы анализа анализатор Function ID использует сведения об идентификации компилятора для хеш-сопоставления тел функций, чтобы выявить присутствие тел библиотечных функций, которые могли быть скомпонованы в бинарный файл. Когда найдено совпадение хеша, Ghidra извлекает имя совпавшей функции из базы данных хешей, содержащейся в файлах Ghidra `.fidbf`, и добавляет это имя как символ функции. Хеш-сопоставление особенно полезно для бинарных файлов, очищенных от символов, поскольку предоставляет средство восстановления символов, не зависящее от наличия таблицы символов. Эта функциональность подробнее обсуждается в разделе "Function IDs" на странице 272.

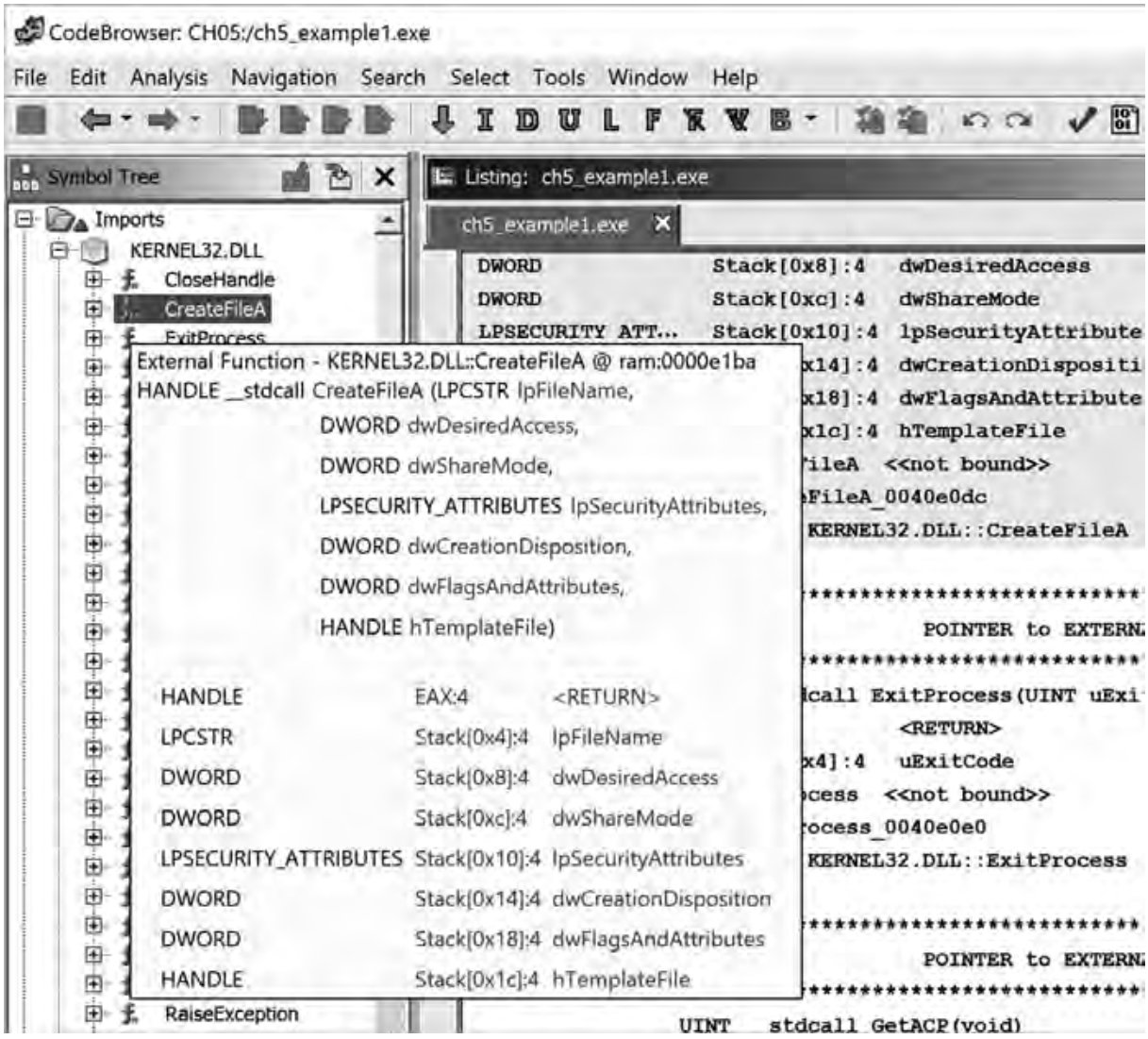


Рисунок 5-15. Всплывающее окно папки `Functions` *в* `Symbol Tree`

Labels

Папка Labels - эквивалент папки `Functions` для данных. Любые символы данных, содержащиеся в таблице символов бинарного файла, будут перечислены в папке Labels. Кроме того, каждый раз, когда вы добавляете новое имя метки к адресу данных, эта метка добавляется в папку Labels.

Classes

Папка `Classes` содержит запись для каждого класса, выявленного Ghidra во время фазы анализа. Под каждым классом Ghidra перечисляет выявленные данные и методы, которые могут помочь понять поведение класса. Классы C++ и структуры, которые Ghidra использует для заполнения папки classes, подробнее обсуждаются в главе 8.

Namespaces

В папке Namespaces Ghidra может создавать новые пространства имен, чтобы обеспечить организацию и гарантировать, что назначенные имена не конфликтуют в бинарном файле. Например, пространство имен может быть создано для каждой выявленной внешней библиотеки или для каждого оператора `switch`, использующего таблицы переходов, что позволяет повторно использовать метки таблиц переходов в других операторах `switch` без конфликтов.

Окно Data Type Manager

Окно `Data Type Manager` позволяет находить, организовывать и применять типы данных к файлу с использованием системы архивов типов данных. Архивы представляют накопленные Ghidra знания о предопределенных типах данных, извлеченные из заголовочных файлов, входящих в большинство популярных компиляторов. Обрабатывая заголовочные файлы, Ghidra понимает типы данных, ожидаемые распространенными библиотечными функциями, и может соответствующим образом аннотировать листинги дизассемблирования и декомпилятора. Аналогично из этих заголовочных файлов Ghidra понимает как размер, так и структуру сложных структур данных. Вся эта информация собирается в архивные файлы и применяется каждый раз при анализе бинарного файла.

Если вернуться к рисунку 5-4, видно, что корень дерева `BuiltInTypes`, содержащего примитивные типы вроде `int`, которые нельзя изменить, переименовать или переместить внутри архива типов данных, отображается в окне `Data Type Manager` в левом нижнем углу окна `CodeBrowser` даже без загруженной программы. Помимо встроенных типов, Ghidra поддерживает создание пользовательских типов данных, включая структуры, объединения, перечисления и typedefs. Она также поддерживает массивы и указатели как производные типы данных.

Каждый открытый файл имеет связанную запись в окне `Data Type Manager`, как ранее было показано на рисунке 5-5. Папка имеет то же имя, что и текущий файл, а записи внутри папки специфичны для текущего файла.

Окно `Data Type Manager` отображает узлы для каждого открытого архива типов данных. Архивы могут открываться автоматически, например когда программа ссылается на архив, или вручную пользователем. Типы данных и `Data Type Manager` подробнее рассматриваются в главах 8 и 13.

Окно Console

Окно Console в нижней части окна `CodeBrowser` служит областью вывода Ghidra для плагинов и сценариев, включая те, которые вы разрабатываете самостоятельно, и является местом, где следует искать сведения о задачах, выполняемых Ghidra во время работы с файлом. Разработка сценариев и плагинов вводится в главах 14 и 15.

Окно Decompiler

Окно `Decompiler` позволяет одновременно просматривать и изменять ассемблерное и C-представление бинарного файла через соединенные окна. C-представление, создаваемое декомпилятором Ghidra, не всегда идеально, но оно может быть очень полезным для понимания бинарного файла. Базовая функциональность декомпилятора включает восстановление выражений, переменных, параметров функций и полей структур. Декомпилятор также часто способен восстанавливать блочную структуру функции, которая обычно скрывается в языке ассемблера, поскольку он не является блочно-структурированным и широко использует операторы `goto` или их эквиваленты, чтобы выглядеть блочно-структурированным.

Окно `Decompiler` отображает C-представление функции, выбранной в окне `Listing`, как показано на рисунке 5-16. В зависимости от вашего опыта работы с языком ассемблера декомпилированный код может быть гораздо проще для понимания, чем код в окне `Listing`. Даже начинающие программисты должны суметь определить бесконечный цикл в декомпилированной функции. Условие цикла `while` зависит от значения `param_3`, которое внутри цикла не изменяется.

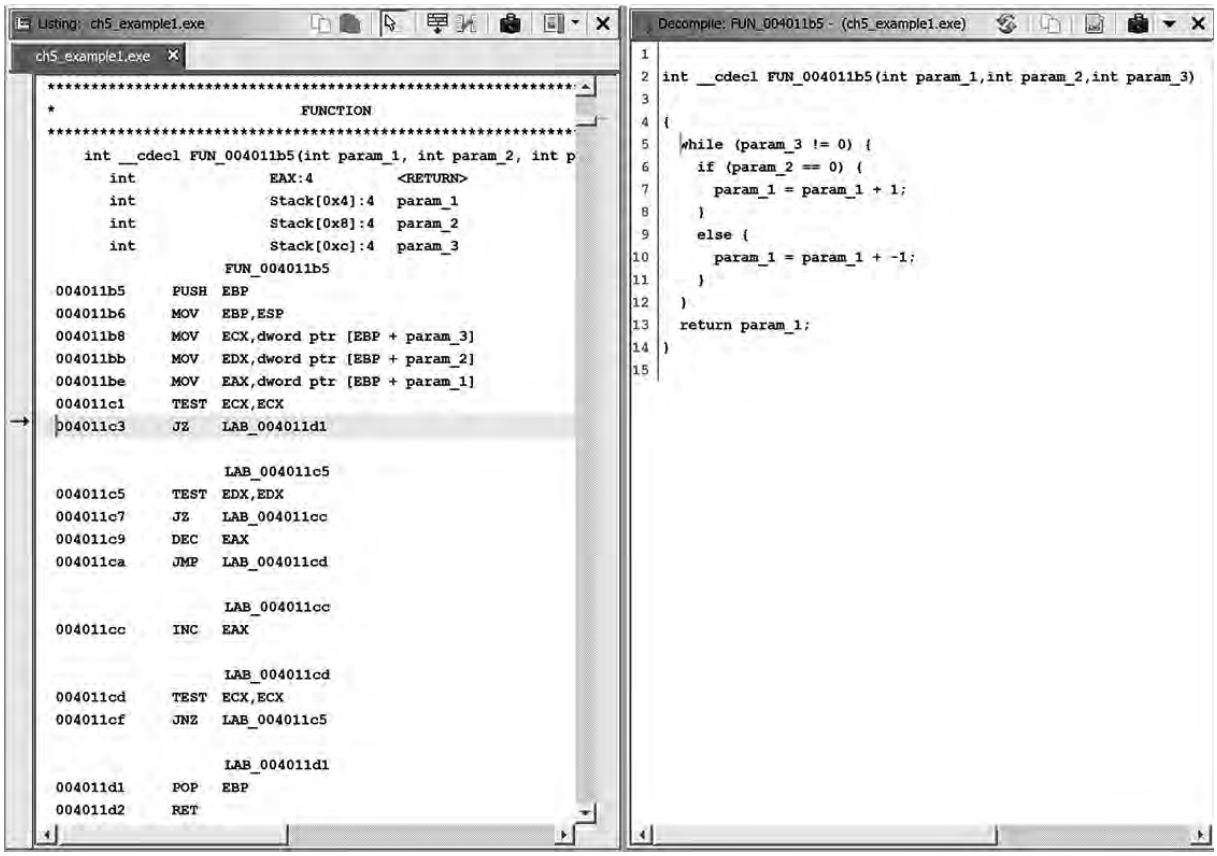


Рисунок 5-16. Окна `Listing` *и* `Decompiler`

Значки окна `Decompiler` показаны на рисунке 5-17. Можно использовать значок Snapshot, чтобы открыть дополнительные, отсоединенные окна `Decompiler`, если вы хотите сравнить

декомпилированные версии нескольких функций или продолжить просмотр конкретной функции, перемещаясь в другое место в окне `Listing`. Значок Export позволяет сохранить декомпилированную функцию в файл C.

Внутри окна `Decompiler` через щелчок правой кнопкой мыши доступны контекстные меню, позволяющие выполнять действия, связанные с выделенным элементом. Параметры, связанные с одним из параметров функции, `param_1`, показаны на рисунке 5-18.

Рисунок 5-17. Панель инструментов окна `Decompiler`

Кнопка Re-decompile повторно декомпилирует листинг при выборе. Кнопка Copy копирует выбранное содержимое из окна `Decompiler` в буфер обмена Ghidra. Кнопка Export экспортирует декомпилированную функцию и позволяет выбрать место назначения файла. Кнопка Snapshot создает и открывает отсоединенную копию окна `Decompiler`. Кнопка Debug Function Decompilation запускает декомпилятор и сохраняет всю связанную информацию в XML-файл.

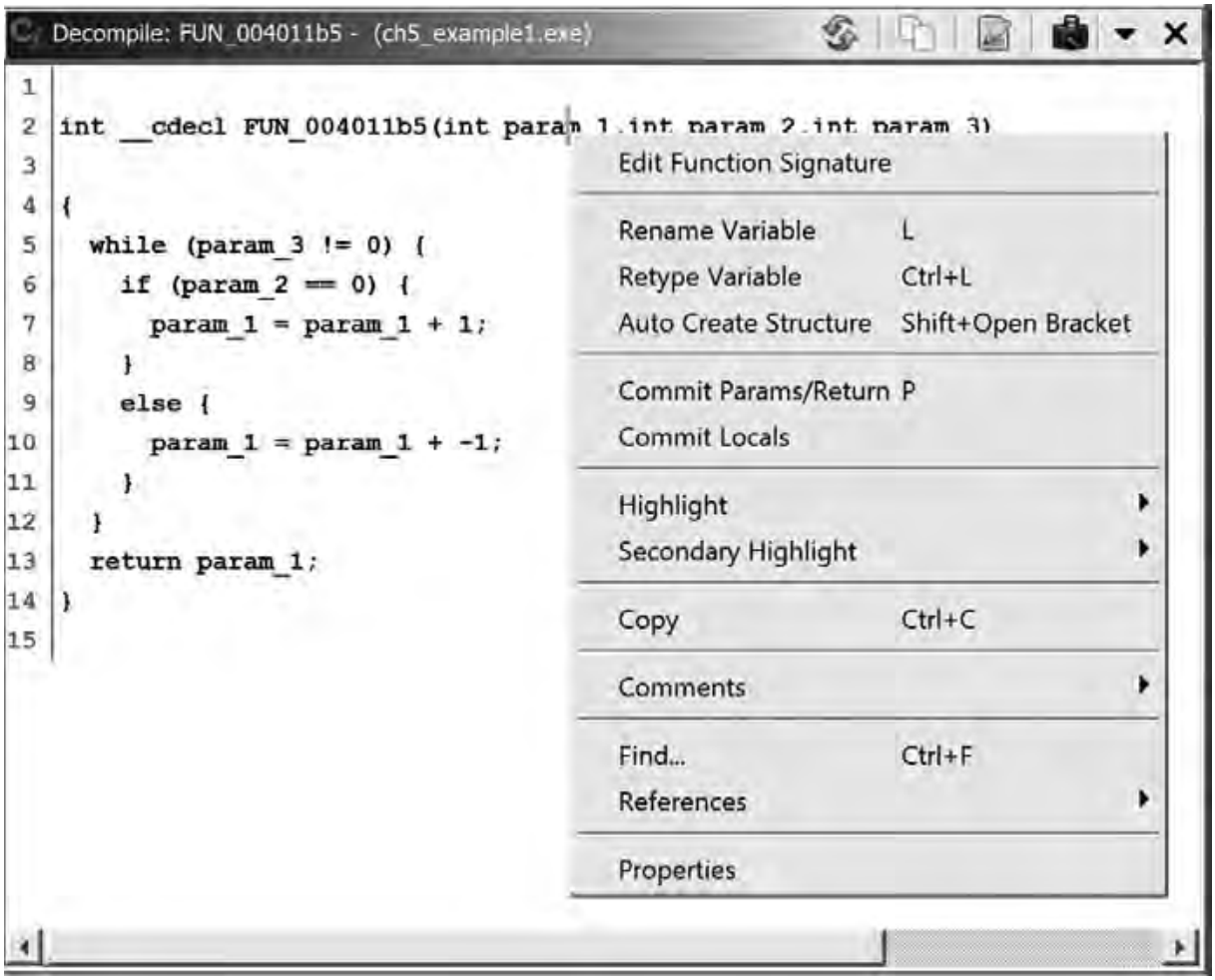


Рисунок 5-18. Параметры окна `Decompiler` *для параметров функции*

Декомпиляция - чрезвычайно сложный процесс, и теория декомпиляторов остается активной областью исследований. В отличие от дизассемблирования, точность которого можно проверить по справочникам производителей, не существует справочников, предоставляющих канонические переводы языка ассемблера обратно в C или, если уж на то пошло, C в ассемблер. Фактически, хотя декомпилятор Ghidra всегда создает исходный код C, может оказаться, что бинарный файл, который анализирует декомпилятор, изначально был написан на языке, отличном от C, и многие C-ориентированные допущения декомпилятора могут вообще не выполняться.

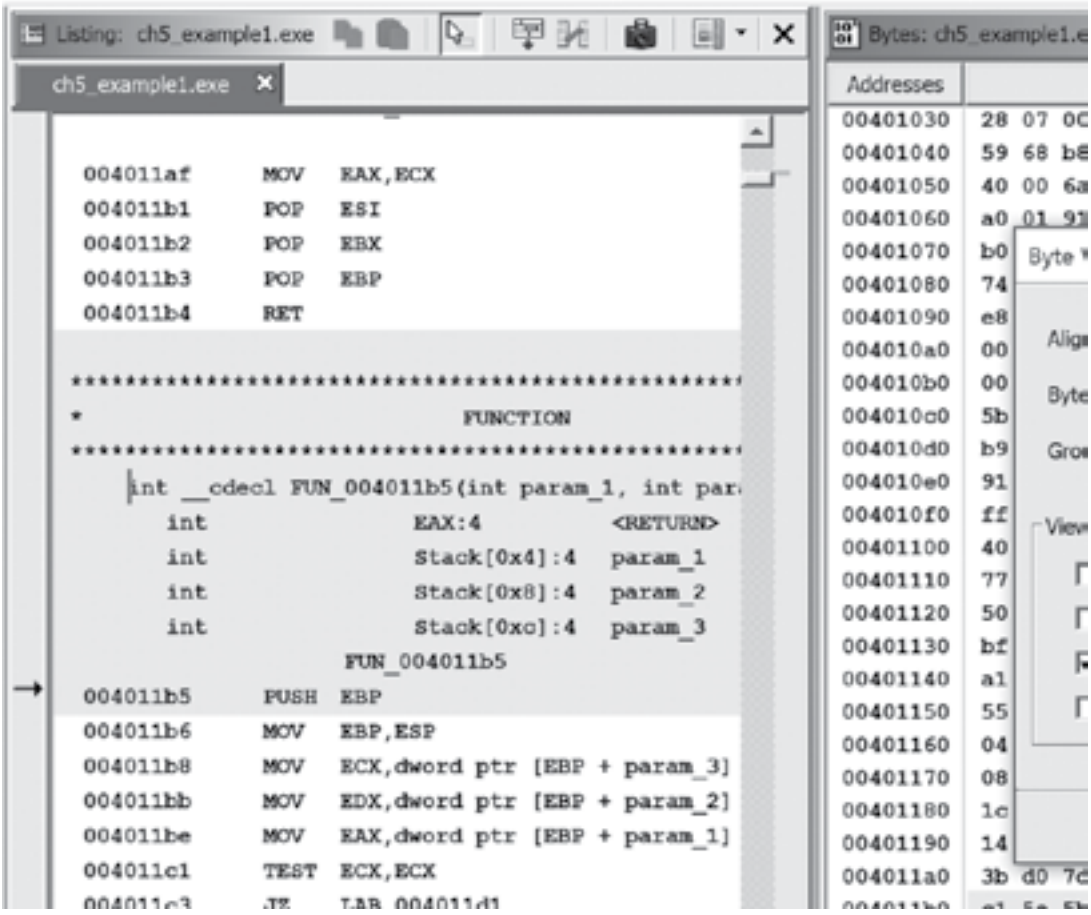
Как и большинство сложных плагинов, декомпилятор имеет свои особенности, а качество его вывода в значительной степени зависит от качества входных данных. Многие проблемы и неровности в окне `Decompiler` можно проследить до проблем в лежащем в основе дизассемблированном представлении, поэтому, если декомпилированный код не имеет смысла, возможно, придется потратить время на улучшение качества дизассемблирования. В большинстве случаев это означает аннотирование дизассемблированного представления более точной информацией о типах данных, что обсуждается в главах 8 и 13. Мы продолжим исследовать возможности декомпилятора в последующих главах и подробно обсудим его в главе 19.

Другие окна Ghidra

Помимо шести стандартных окон, можно открывать другие окна, чтобы поддерживать процесс SRE альтернативными или специализированными представлениями файла. Список доступных окон отображается из меню `Window`, ранее показанного на рисунке 5-4. Полезность этих представлений зависит как от характеристик анализируемого бинарного файла, так и от вашего владения Ghidra. Некоторые из этих окон достаточно специализированы, чтобы требовать более подробного рассмотрения в последующих главах, но несколько распространенных окон мы представим здесь.

Окно Bytes

Окно `Bytes` предоставляет сырой взгляд на содержимое файла на уровне байтов. По умолчанию окно `Bytes` открывается в правой верхней части `CodeBrowser` и предоставляет стандартное отображение шестнадцатеричного дампа содержимого программы по 16 байтов в строке. Это окно одновременно служит шестнадцатеричным редактором и может быть настроено на отображение разных форматов с помощью инструмента `Settings` на панели инструментов окна `Bytes`. Во многих случаях может быть полезно добавить ASCII-представление в окно `Bytes`, как показано на рисунке 5-19. На этом рисунке также показаны диалог `Byte Viewer Options` и значки панели инструментов для редактирования или создания снимка байтового представления.


Listing: ch5_example1.exe
ch5_example1.exe
004011af MOV EAX,ECX
004011b1 POP ESI
004011b2 POP EBX
004011b3 POP EBP
004011b4 RET
* FUNCTION
int __cdecl FUN_004011b5(int param_1, int par
int EAX:4 <RETURN>
int Stack[0x4]:4 param_1
int Stack[0x8]:4 param_2
int Stack[0xc]:4 param_3
FUN_004011b5
004011b5 PUSH EBP
004011b6 MOV EBP,ESP
004011b8 MOV ECX,dword ptr [EBP + param_3]
004011bb MOV EDX,dword ptr [EBP + param_2]
004011be MOV EAX,dword ptr [EBP + param_1]
004011c1 TEST ECX,ECX
Bytes: ch5_example1.e
Addresses
00401030 28 07 0C
00401040 59 68 bE
00401050 40 00 6a
00401060 a0 01 91
00401070 b0
00401080 74
00401090 e8
004010a0 00
004010b0 00
004010c0 5b
004010d0 b9
004010e0 91
004010f0 ff
00401100 40
00401110 77
00401120 50
00401130 bf
00401140 a1
00401150 55
00401160 04
00401170 08
00401180 1c
00401190 14
004011a0 3b d0 7d

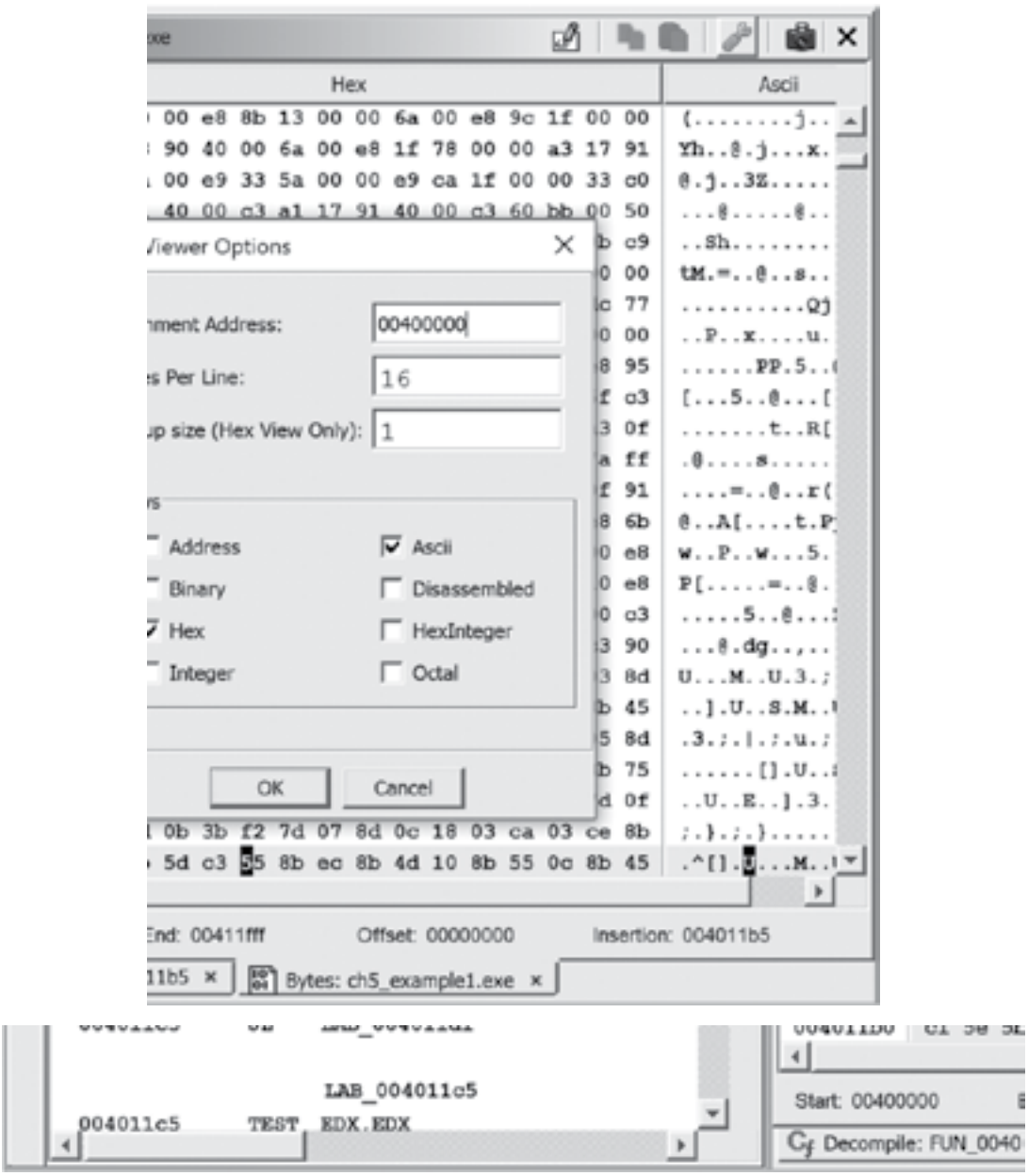


Рисунок 5-19. Синхронизированные шестнадцатеричное и дизассемблированное представления с выделенными значками Toggle и Snapshot

Как и в случае с окном `Listing`, несколько окон `Bytes` можно открыть одновременно с помощью значка Snapshot (см. рисунок 5-19) на панели инструментов окна `Bytes`. По умолчанию первое окно `Bytes` имеет связь с окном `Listing`, поэтому прокрутка в одном окне и щелчок по элементу заставляют другое окно прокрутиться к тому же месту, то есть к тому же виртуальному адресу. Последующие окна `Bytes` отсоединены, что позволяет прокручивать их независимо. Когда окно отсоединено, имя окна появляется в квадратных скобках в строке заголовка.

Чтобы превратить окно `Bytes` в шестнадцатеричный или ASCII-редактор, просто переключите значок карандаша, выделенный на рисунке 5-19. Курсор станет красным, показывая, что можно редактировать, хотя вы не сможете редактировать адреса, содержащие существующий элемент кода, например инструкцию. Когда закончите редактирование, переключите значок снова, и вы вернетесь в режим только чтения. Обратите внимание, что любые изменения не будут отражены в отсоединенных окнах `Bytes`.

Если столбец Hex отображает вопросительные знаки вместо шестнадцатеричных значений, Ghidra сообщает, что не уверена, какие значения могут занимать заданный диапазон виртуальных

адресов. Так бывает, когда программа содержит секцию `bss`[4], которая обычно не занимает места внутри файла, но расширяется загрузчиком для размещения требований программы к статическому хранилищу.

Окно Defined Data

Окно Defined Data отображает строковое представление данных, определенных в текущей программе, представлении или выделении, вместе со связанным адресом, типом и размером, как показано на рисунке 5-20. Как и в большинстве табличных окон, можно сортировать по любому столбцу в порядке возрастания или убывания, щелкнув заголовок столбца. Двойной щелчок по любой строке в окне Defined Data заставляет окно `Listing` перейти к адресу выбранного элемента.

При использовании с перекрестными ссылками, обсуждаемыми в главе 9, окно Defined Data предоставляет средство быстро заметить интересный элемент и всего несколькими щелчками отследить назад любое место в программе, которое ссылается на этот элемент. Например, можно увидеть строку `"SOFTWARE\Microsoft\Windows\Current Version\Run"` и задуматься, почему приложение ссылается именно на этот ключ реестра Windows, а затем обнаружить, что программа задает этот ключ реестра, чтобы автоматически запускаться при загрузке Windows.

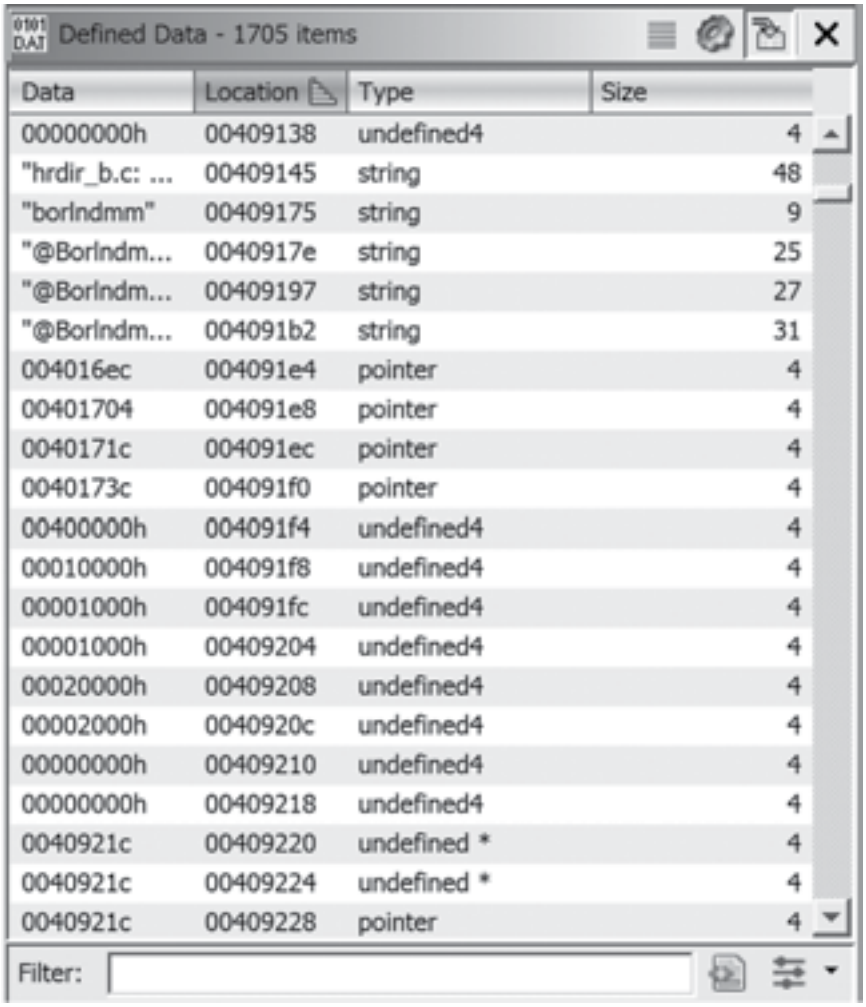


Рисунок 5-20. Окно Defined Data с выделенным значком `Filter`

Окно Defined Data обладает обширными возможностями фильтрации. Помимо панели `Filter` в нижней части окна, значок `Filter` в правом верхнем углу, выделенный на рисунке 5-20, позволяет управлять дополнительными параметрами фильтра типов данных, как показано на рисунке 5-21.

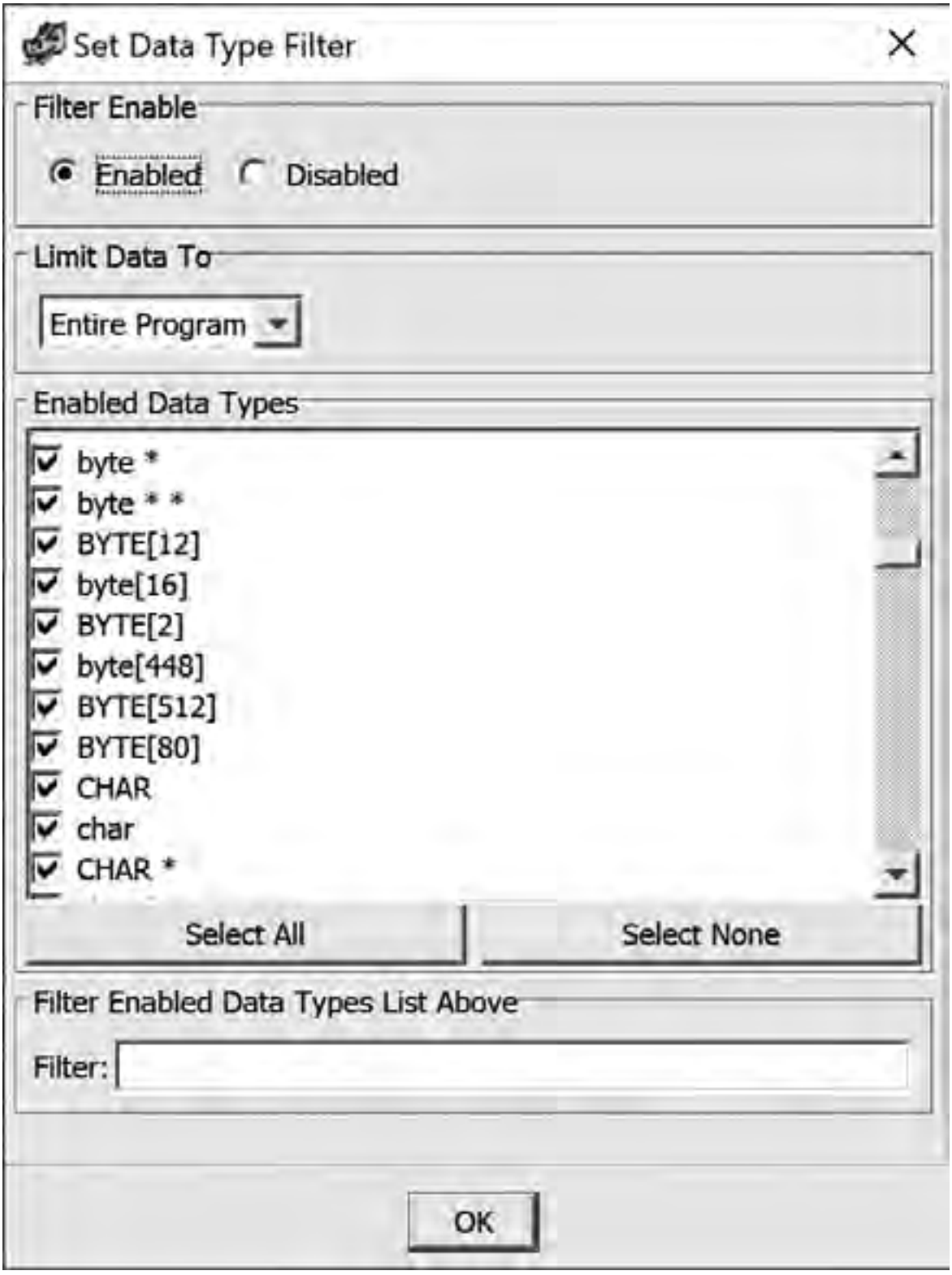


Рисунок 5-21. Параметры фильтра типов Defined Data

Каждый раз, когда вы закрываете диалог `Set Data Type Filter` нажатием `OK`, Ghidra заново формирует содержимое окна Defined Data в соответствии с новыми настройками.

Окно Defined Strings

Окно `Defined Strings` отображает строки, определенные в бинарном файле. Пример этого окна показан на рисунке 5-22. Помимо стандартных столбцов, показанных на рисунке, можно добавить столбцы, щелкнув правой кнопкой мыши строку заголовков столбцов. Возможно, один из самых интересных доступных столбцов - флаг Has Encoding Error, который может указывать на проблему с набором символов или неверную идентификацию строки. Помимо этого окна, в Ghidra доступна значительная функциональность поиска строк. Она обсуждается в главе 6.

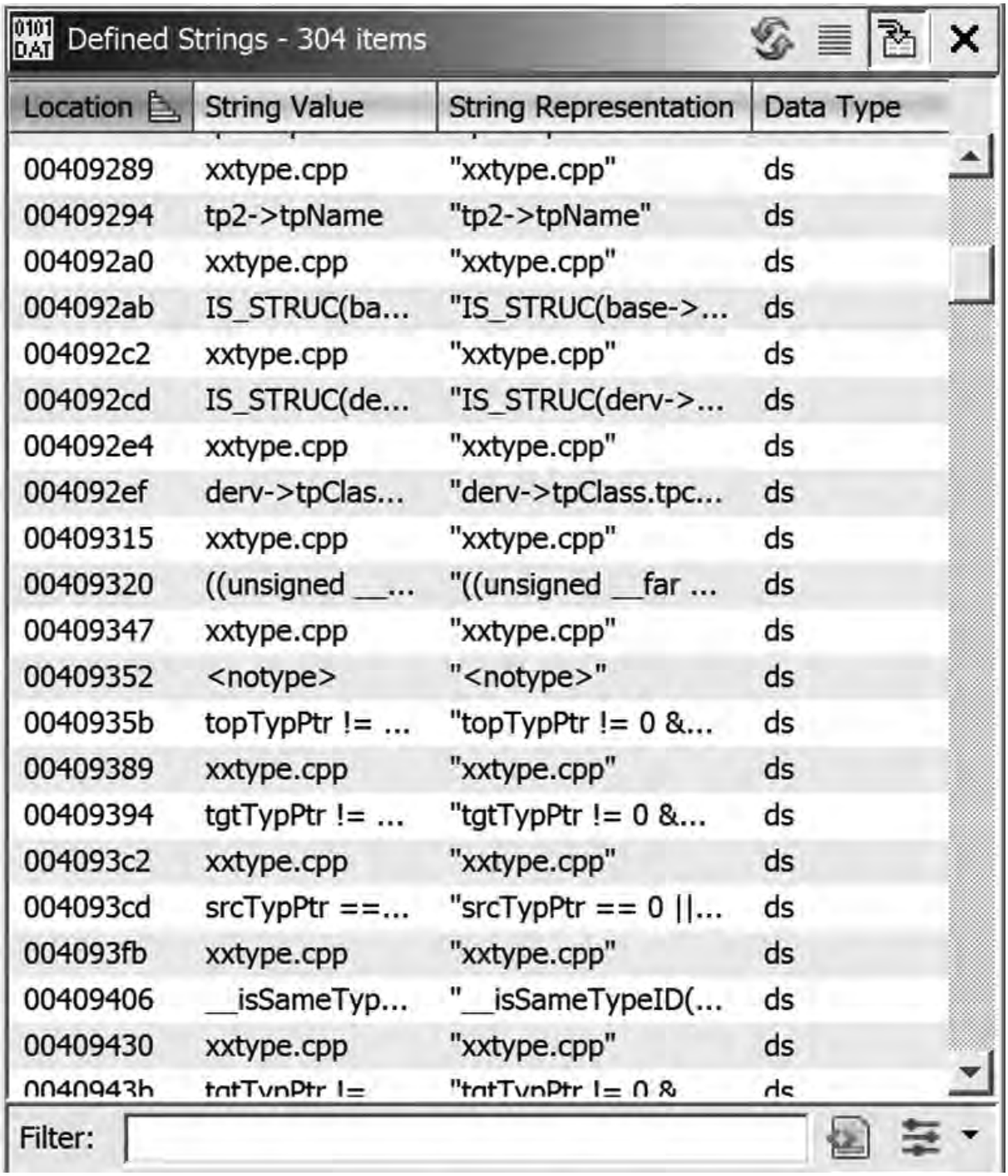
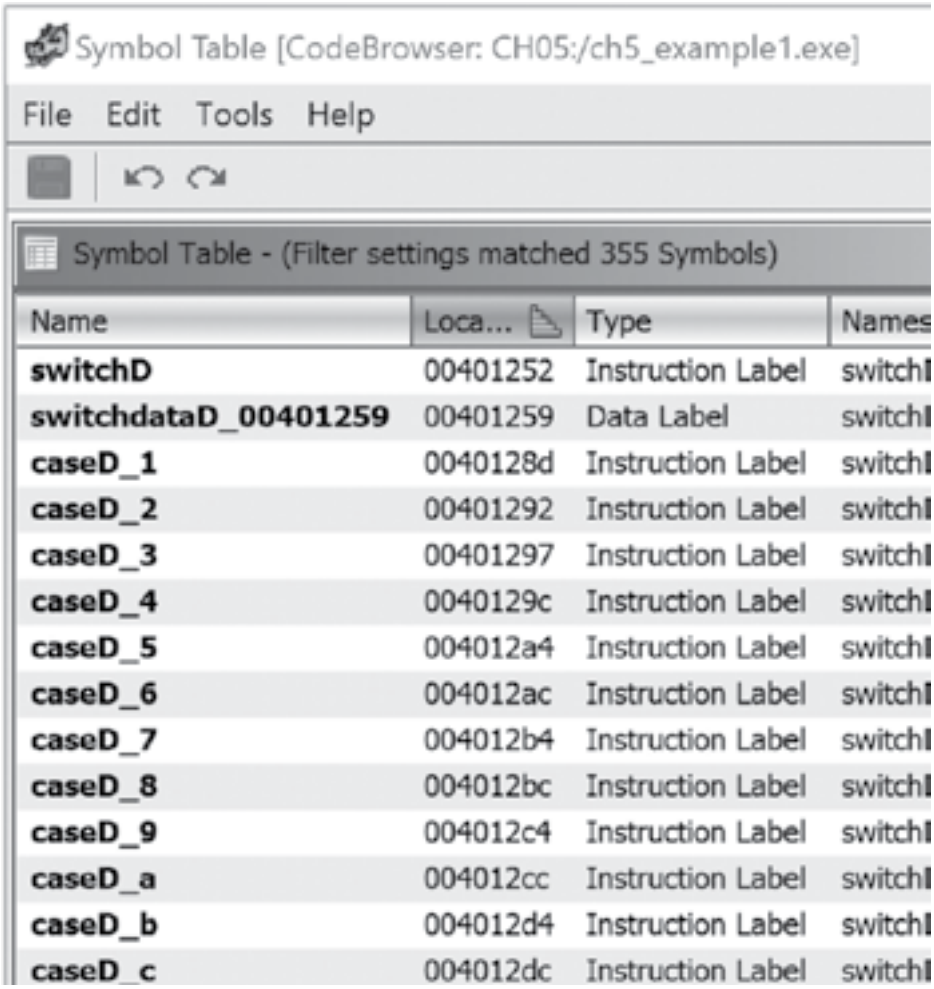


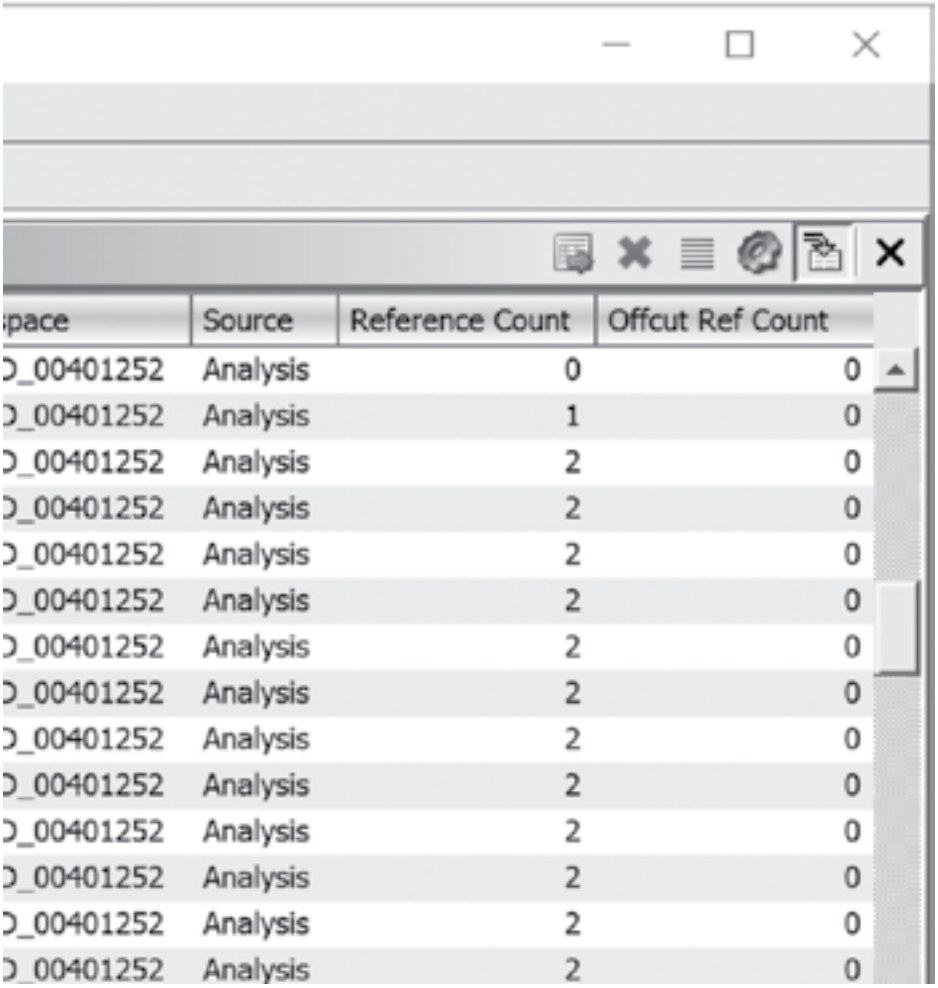
Рисунок 5-22. Окно `Defined Strings`

Окна Symbol Table и Symbol References

Окно `Symbol Table` предоставляет сводный список всех глобальных имен внутри бинарного файла. По умолчанию отображаются восемь столбцов, как показано на рисунке 5-23. Окно гибко настраивается: можно добавлять и удалять столбцы в представлении, а также сортировать по любому столбцу в порядке возрастания или убывания. Первые два стандартных столбца - Name и Location. Имя - это не более чем символическое описание, данное символу, определенному по некоторому адресу.

`Symbol Table` соединена с окном `Listing`, но предоставляет возможность управлять взаимодействием с окном `Listing`. Выделенный значок справа на рисунке 5-23 - переключатель, определяющий, вызывает ли одиночный щелчок по месту в окне `Symbol Table` связанное перемещение в окне `Listing`. Независимо от выбора переключателя, двойной щелчок по любой записи Location в `Symbol Table` немедленно переместит представление `Listing` к выбранной записи. Это дает полезный инструмент для быстрого перехода к известным местам в листинге программы.





pace	Source	Reference Count	Offcut Ref Count
D_00401252	Analysis	0	0
D_00401252	Analysis	1	0
D_00401252	Analysis	2	0
D_00401252	Analysis	2	0
D_00401252	Analysis	2	0
D_00401252	Analysis	2	0
D_00401252	Analysis	2	0
D_00401252	Analysis	2	0
D_00401252	Analysis	2	0
D_00401252	Analysis	2	0
D_00401252	Analysis	2	0
D_00401252	Analysis	2	0
D_00401252	Analysis	2	0
D_00401252	Analysis	2	0
D_00401252	Analysis	2	0

caseD_0	004012e2	Instruction Label	switchD_00401252	Analysis
FUN_004012e4	004012e4	Function	Global	Default
switchD	00401300	Instruction Label	switchD_00401300	Analysis
switchdataD_00401307	00401307	Data Label	switchD_00401300	Analysis
caseD_1	00401368	Instruction Label	switchD_00401300	Analysis
caseD_2	0040136d	Instruction Label	switchD_00401300	Analysis
caseD_3	00401372	Instruction Label	switchD_00401300	Analysis
caseD_4	00401377	Instruction Label	switchD_00401300	Analysis
caseD_5	0040137f	Instruction Label	switchD_00401300	Analysis
caseD_6	00401387	Instruction Label	switchD_00401300	Analysis
caseD_7	0040138f	Instruction Label	switchD_00401300	Analysis
caseD_8	00401397	Instruction Label	switchD_00401300	Analysis
caseD_0	004013dd	Instruction Label	switchD_00401300	Analysis
FUN_004013df	004013df	Function	Global	Default

Filter:

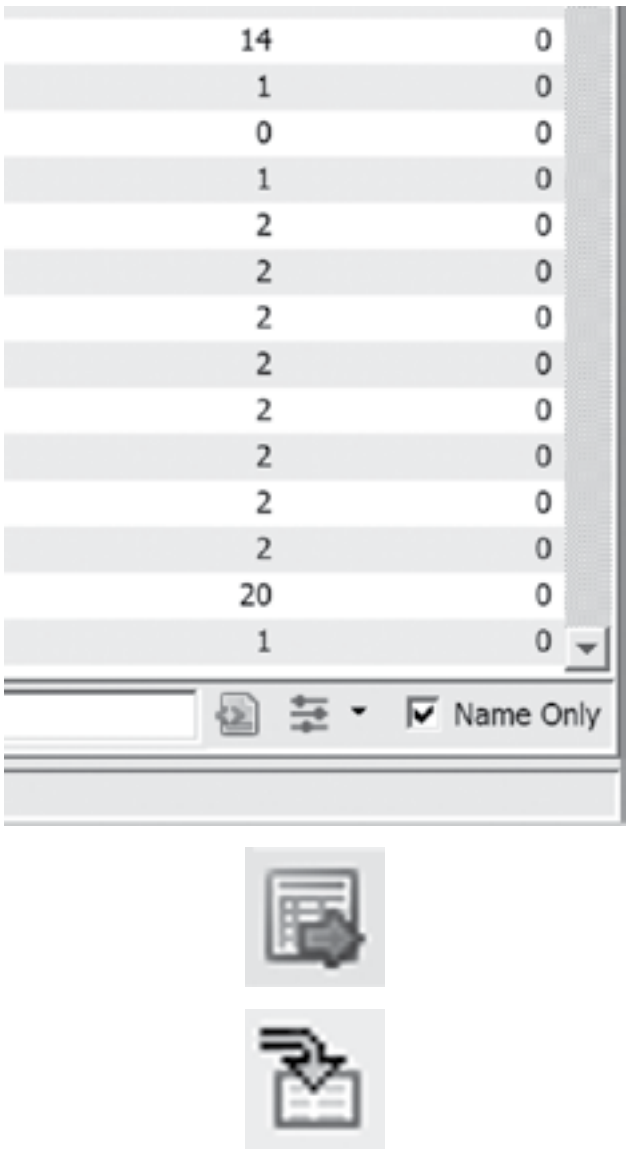


Рисунок 5-23. Окно `Symbol Table` *с выделенными значками* `Display Symbol References` *и* `Navigation Toggle`

В окне `Symbol Table` доступны обширные возможности фильтрации, и к параметрам фильтрации можно обращаться несколькими способами. Значок шестеренки на панели инструментов открывает диалог `Symbol Table Filter`. Диалог с установленным флажком `Use Advanced Filters` показан на рисунке 5-24. Помимо этого диалога, можно использовать параметры `Filter` в нижней части окна. Подробные обсуждения параметров фильтрации таблицы символов доступны в `Ghidra Help`.

Выделенный значок слева на рисунке 5-23 - это значок `Display Symbol References`. Щелчок по этому значку добавляет окно `Symbol References` к окну `Symbol Table`. По умолчанию эти две таблицы появляются рядом. Чтобы повысить читаемость, можно перетащить окно `Symbol References` под окно `Symbol Table`, как показано на рисунке 5-25. Связь между этими двумя таблицами однонаправленная: таблица `Symbol References` обновляется при выборе в `Symbol Table`.

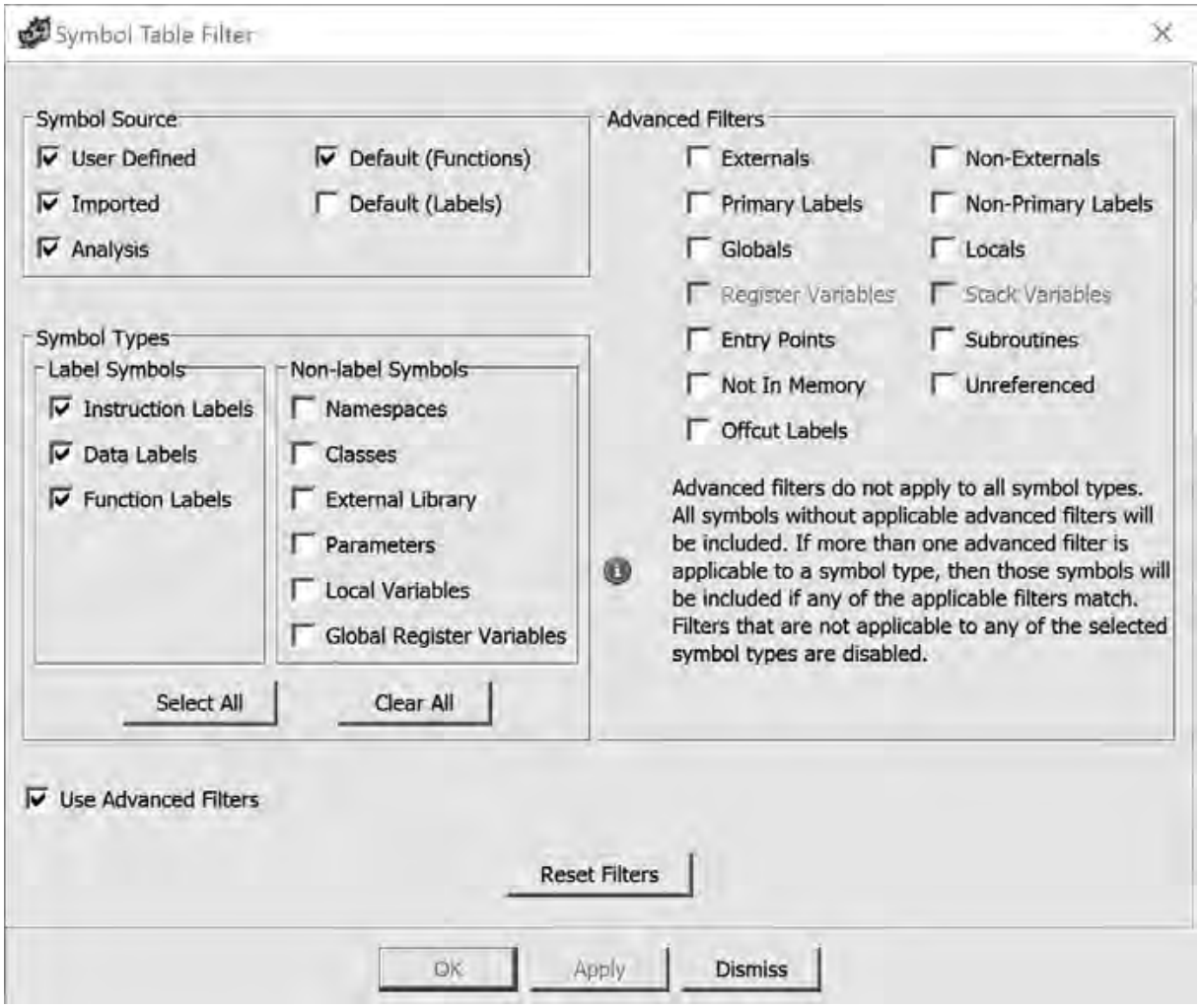


Рисунок 5-24. Диалог `Symbol Table Filter`

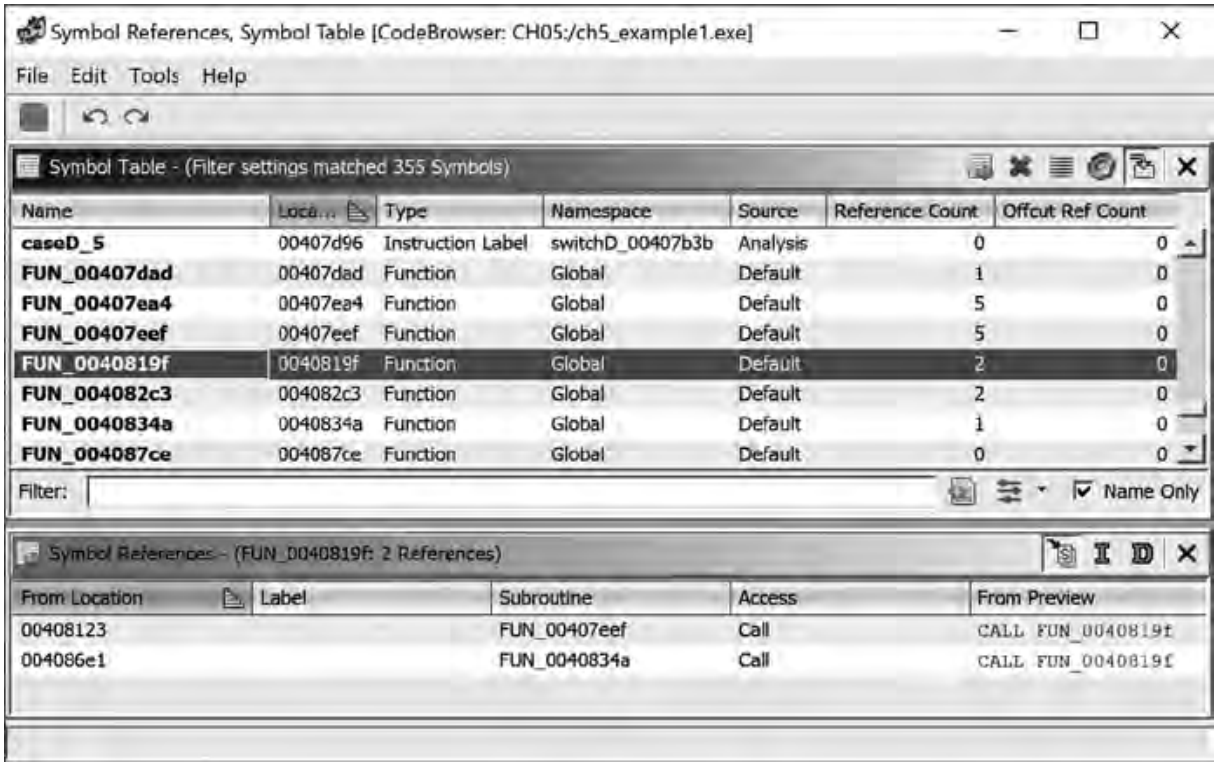


Рисунок 5-25. `Symbol Table` *с* `Symbol References`

Как и окно Symbol Table, окно Symbol References имеет те же элементы управления организацией столбцов. Кроме того, содержимое окна Symbol References управляется тремя значками (S, I и D) в правом верхнем углу панели инструментов Symbol References. Эти параметры взаимоисключающие, то есть одновременно может быть выбран только один.

Значок S**.** Когда выбран этот значок, окно Symbol References отображает все ссылки на символ, выбранный в Symbol Table. На рисунке 5-25 показано окно Symbol References с выбранным этим параметром.

Значок I**.** Когда выбран этот значок, окно Symbol References отображает все ссылки инструкций из функции, выбранной в Symbol Table. Этот список будет пуст, если вы не выбрали точку входа функции.

Значок D**.** Когда выбран этот значок, окно Symbol References отображает все ссылки на данные из функции, выбранной в Symbol Table. Этот список будет пуст, если вы не выбрали точку входа функции или если функция не ссылается ни на какие символы данных.

Окно Memory Map

Окно Memory Map отображает сводный список блоков памяти, присутствующих в программе, как показано на рисунке 5-26. Обратите внимание, что то, что Ghidra называет блоками памяти, часто называется секциями при обсуждении структуры бинарных файлов. Информация, представленная в окне, включает имя блока памяти, или секции, начальный и конечный адреса, длину, флаги разрешений, тип блока, флаг инициализации, а также место для имени исходного файла и пользовательских комментариев. Начальный и конечный адреса представляют диапазон виртуальных адресов, в который секции программы будут отображены во время выполнения.

Memory Map - Image Base: 00400000

Memory Blocks

Name	Start	End	Length	R	W	X	Volatile	Type	Initialized	Byte Source	Source	Comment
Headers	00400000	004005ff	0x600	☑	☐	☐	☐	Default	☑	File: ch5_example1.exe: 0x0		
.text	00401000	004089ff	0x7a00	☑	☐	☑	☐	Default	☑	File: ch5_example1.exe: 0x600		
.text	00408a00	00408fff	0x600	☑	☐	☑	☐	Default	☐			
.data	00409000	0040b3ff	0x2400	☑	☑	☐	☐	Default	☑	File: ch5_example1.exe: 0x8000		
.data	0040b400	0040bfff	0xc00	☑	☑	☐	☐	Default	☐			
.tls	0040c000	0040c1ff	0x200	☑	☑	☐	☐	Default	☑	File: ch5_example1.exe: 0xa400		
.tls	0040c200	0040cfff	0xe00	☑	☑	☐	☐	Default	☐			
.rdata	0040d000	0040d1ff	0x200	☑	☐	☐	☐	Default	☑	File: ch5_example1.exe: 0xa600		
.rdata	0040d200	0040dfff	0xe00	☑	☐	☐	☐	Default	☐			
.idata	0040e000	0040e5ff	0x600	☑	☐	☐	☐	Default	☑	File: ch5_example1.exe: 0xa800		
.idata	0040e600	0040efff	0xa00	☑	☐	☐	☐	Default	☐			
.edata	0040f000	0040f1ff	0x200	☑	☐	☐	☐	Default	☑	File: ch5_example1.exe: 0xae00		
.edata	0040f200	0040ffff	0xe00	☑	☐	☐	☐	Default	☐			
.rsrc	00410000	004101ff	0x200	☑	☐	☐	☐	Default	☑	File: ch5_example1.exe: 0xb000		
.rsrc	00410200	00410fff	0xe00	☑	☐	☐	☐	Default	☐			
.reloc	00411000	004117ff	0x800	☑	☐	☐	☐	Default	☑	File: ch5_example1.exe: 0xb200		
.reloc	00411800	00411fff	0x800	☑	☐	☐	☐	Default	☐			

Рисунок 5-26. Окно Memory Map

Двойной щелчок по любому начальному или конечному адресу в окне перемещает окно Listing и все другие соединенные окна к указанному адресу. Панель инструментов окна Memory Map предоставляет параметры для добавления и удаления блоков, перемещения блоков, разделения и объединения блоков, редактирования адресов и установки новой базы образа. Эти возможности особенно полезны при обратной разработке файлов с нестандартными форматами, поскольку структура сегментов бинарного файла могла быть не обнаружена загрузчиком Ghidra.

Аналоги окна Memory Map в командной строке включают `objdump (-h)`, `readelf (-S)` и `dumpbin (/HEADERS)`.

Окно Function Call Graph

В любой программе функция может как вызывать другие функции, так и вызываться другими функциями. Окно Function Call Graph показывает непосредственных соседей заданной функции. Для наших целей будем называть Y соседом X, если Y напрямую вызывает X или если X напрямую вызывает Y. Когда вы открываете окно Function Call Graph, Ghidra определяет соседей функции, в которой расположен курсор, и создает соответствующее представление. Это представление показывает функцию в контексте ее использования в файле программы, но это лишь часть общей картины.

На рисунке 5-27 показана функция с именем `FUN_0040198c`, которая вызывается из `FUN_00401edc` и, в свою очередь, вызывает шесть других функций. Двойной щелчок по любой функции в окне немедленно перемещает окно `Listing` и другие соединенные окна к выбранной функции. Перекрестные ссылки Ghidra (XREFs) являются механизмами, лежащими в основе формирования окна Function Call Graph. XREFs подробнее рассматриваются в главе 9.

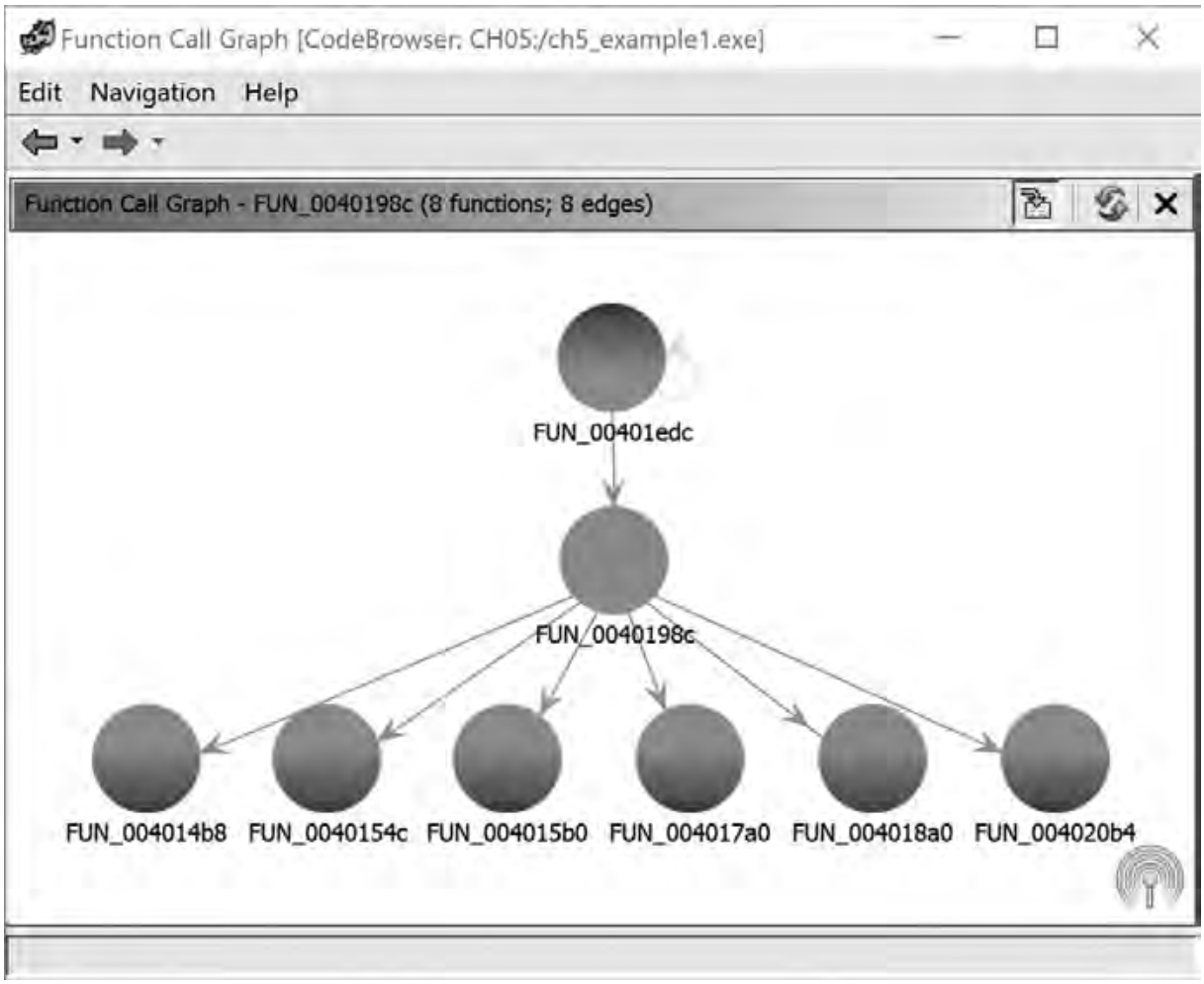


Рисунок 5-27. Окно Function Call Graph

Итоги

На первый взгляд количество представлений, которые предлагает Ghidra, может казаться ошеломляющим. Возможно, проще всего придерживаться стандартных представлений, пока вы не почувствуете себя достаточно уверенно, чтобы начать исследовать дополнительные варианты отображения. В любом случае вы совершенно не обязаны использовать все, что предлагает Ghidra. Не каждое окно будет полезно в каждом сценарии обратной разработки.

Один из лучших способов познакомиться с представлениями Ghidra - просто походить по различным подокнам с вкладками, которые Ghidra заполняет данными о вашем бинарном файле, а также открыть несколько других доступных окон. Эффективность и результативность ваших сеансов обратной разработки будут расти по мере повышения удобства работы с Ghidra.

Кто вызывает?

Хотя окно Function Call Graph полезно, иногда нужна большая картина или по крайней мере картина побольше. Окно Function Call Trees (`Window > Function Call Trees`) позволяет видеть все вызовы к выбранной функции и из нее. Окно Function Call Trees, показанное на рисунке 5-28, имеет две секции: одну для входящих вызовов и одну для исходящих вызовов. Как входящие, так и исходящие вызовы можно разворачивать и сворачивать по желанию.

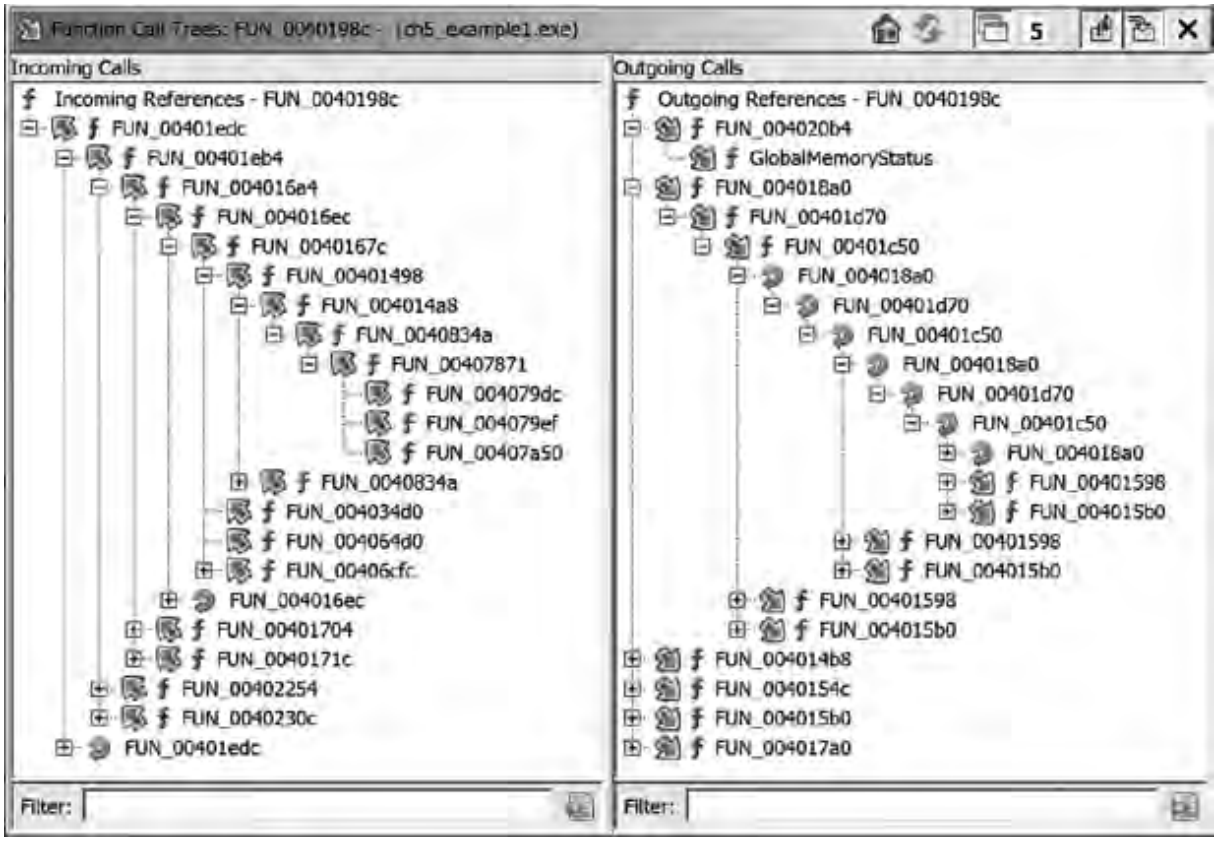


Рисунок 5-28. Представление Function Call Trees

Если открыть окно Function Call Tree с выбранной функцией `entry`, можно увидеть иерархическое представление вызовов функций программы.

Ghidra - очень сложный инструмент. Помимо окон, рассмотренных в этой главе, вы можете столкнуться с дополнительными диалогами по мере того, как будете стремиться овладеть Ghidra. Мы будем вводить ключевые диалоги по мере их актуальности на протяжении остальной части

книги.

К этому моменту вы должны начать чувствовать себя увереннее с интерфейсом Ghidra и рабочим столом `CodeBrowser`. В следующей главе мы сосредоточимся на многочисленных способах, которыми можно управлять дизассемблированным представлением, чтобы лучше понимать его поведение и в целом упростить себе жизнь с Ghidra.

Сноски

[1] [назад] Ghidra использует термин flow ("поток"), чтобы обозначить, как выполнение может продолжаться от заданной инструкции. Нормальный, также называемый обычным, поток указывает на стандартное последовательное выполнение инструкций. Jump flow указывает, что текущая инструкция переходит или может перейти в непоследовательное место. Call flow указывает, что текущая инструкция вызывает подпрограмму.
[2] [назад] Стековый кадр, или запись активации, - это блок памяти, выделенный в стеке времени выполнения программы, который содержит как параметры, переданные в функцию, так и локальные переменные, объявленные внутри функции. Стековые кадры выделяются при входе в функцию и освобождаются при выходе из нее. Стековые кадры подробнее обсуждаются в главе 6.
[3] [назад] Базовый блок - это максимальная последовательность инструкций, которая выполняется без ветвления от начала до конца. У каждого базового блока есть одна точка входа, первая инструкция блока, и одна точка выхода, последняя инструкция блока. Первая инструкция в базовом блоке часто является целью инструкции ветвления, а последняя инструкция в базовом блоке часто является инструкцией ветвления.
[4] [назад] Секция `bss` создается компилятором для размещения всех неинициализированных статических переменных программы. Поскольку этим переменным не назначено начальное значение, нет необходимости выделять для них место в файловом образе программы, поэтому размер секции указывается в одном из заголовков программы. Когда программа выполняется, загрузчик выделяет требуемое место и инициализирует весь блок нулями.

Глава 6. Понимание дизассемблированного представления Ghidra

В этой главе мы рассмотрим важные базовые навыки, которые помогут лучше понимать дизассемблированное представление Ghidra. Мы начнем с базовых приемов навигации, позволяющих перемещаться по ассемблерному коду и изучать встречающиеся артефакты. Переходя от функции к функции, вы обнаружите, что нужно расшифровывать прототип каждой функции, используя только подсказки, доступные в дизассемблированном представлении. Поэтому мы обсудим приемы, помогающие понять, сколько параметров получает функция и как можно расшифровать типы данных каждого встреченного параметра. Поскольку значительная часть работы функции связана с локальными переменными, поддерживаемыми самой функцией, мы также обсудим, как функции используют стек для хранения локальных переменных и как с помощью Ghidra точно понять, каким образом функция использует любое стековое пространство, которое она может зарезервировать для себя. Анализируете ли вы код в отладчике, изучаете вредоносную программу или разрабатываете эксплойты, понимание того, как расшифровывать размещенные в стеке переменные функции, является важнейшим навыком для понимания поведения любой программы. Наконец, мы рассмотрим возможности поиска, которые предоставляет Ghidra, и то, как они могут способствовать пониманию дизассемблированного представления.

Навигация по дизассемблированному представлению

В главах 4 и 5 мы показали, что на базовом уровне Ghidra объединяет возможности многих распространенных инструментов обратной разработки в своем интегрированном представлении `CodeBrowser`. Умение перемещаться по этому представлению - один из ключевых навыков, необходимых для освоения Ghidra. Статические листинги дизассемблирования, такие как предоставляемые инструментами вроде `objdump`, не имеют встроенных навигационных возможностей, кроме прокрутки листинга вверх и вниз. Даже с лучшими текстовыми редакторами, предлагающими встроенный поиск в стиле `grep`, по таким статическим листингам очень трудно перемещаться. Ghidra, напротив, предоставляет исключительные возможности навигации. Помимо довольно стандартных функций поиска, знакомых по текстовым редакторам или текстовым процессорам, Ghidra создает и отображает полный список перекрестных ссылок, ведущих себя как гиперссылки на веб-странице. В итоге в большинстве случаев для перехода к интересующим местам достаточно двойного щелчка.

Имена и метки

Когда программа дизассемблируется, каждому месту в программе назначается виртуальный адрес. В результате мы можем перейти куда угодно внутри программы, указав виртуальный адрес места, которое хотим посетить. К сожалению, держать каталог адресов в голове - задача нетривиальная. Именно это побудило ранних программистов назначать символические имена тем местам программы, на которые они хотели ссылаться, существенно облегчая себе жизнь. Назначение символических имен адресам программы было похоже на назначение мнемонических имен инструкций кодам операций программы: программы становились легче для чтения и написания, потому что идентификаторы становились проще для запоминания. Ghidra продолжает эту традицию, создавая метки для виртуальных адресов и позволяя пользователю изменять и расширять набор меток.

Мы уже видели использование имен в связи с окном `Symbol Tree`. Напомним: двойной щелчок по имени заставлял представление `Listing` и окно `Symbol References` перейти к указанному месту. Хотя между терминами name и label есть различия в использовании, например функции имеют имена и появляются в отдельной ветви Ghidra `Symbol Tree`, отличной от labels, в контексте навигации эти термины в значительной степени взаимозаменяемы, потому что оба представляют цели навигации.

Ghidra создает символические имена во время фазы автоанализа, используя существующее имя из бинарного файла, если оно доступно, или автоматически генерируя имя на основе того, как место используется внутри бинарного файла. Помимо своего символического назначения, любая метка, отображаемая в окне дизассемблирования, является потенциальной целью навигации, похожей на гиперссылку на веб-странице. Два основных отличия этих меток от стандартных гиперссылок состоят в том, что метки никак не подсвечиваются, чтобы показать, что по ним можно перейти, и что Ghidra обычно требует двойного щелчка для перехода, а не одиночного щелчка, как традиционная гиперссылка.

Навигация в Ghidra

В листинге, показанном на рисунке 6-1, каждый из символов, отмеченных сплошной стрелкой, представляет именованную цель навигации. Двойной щелчок по любому из них в окне `Listing` заставит Ghidra переместить представление `Listing` и все соединенные окна к выбранному месту.

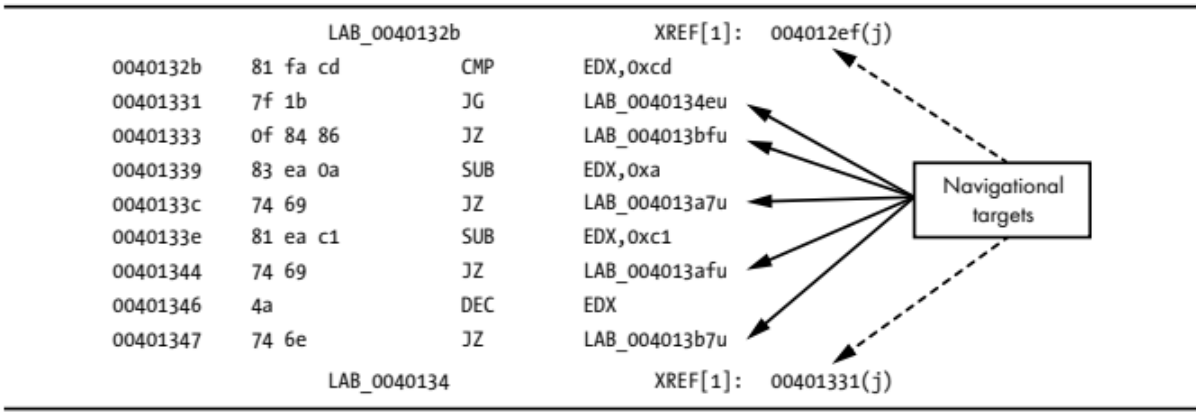


Рисунок 6-1. Листинг с целями навигации

Для целей навигации Ghidra рассматривает как цели навигации еще два типа отображаемых сущностей. Во-первых, перекрестные ссылки, обозначенные пунктирными стрелками на рисунке 6-1, рассматриваются как цели навигации. Двойной щелчок по нижнему адресу перекрестной ссылки переместит представление к месту, откуда ссылка выполнена, в данном случае `00401331`. Перекрестные ссылки подробнее рассматриваются в главе 9. Наведение указателя на любой из этих навигируемых объектов отображает всплывающее окно, показывающее код назначения.

Во-вторых, другой тип отображаемой сущности, получающий особое отношение в навигационном смысле, - это сущность, использующая шестнадцатеричные значения. Если отображаемая последовательность шестнадцатеричных значений представляет корректный виртуальный адрес внутри бинарного файла, соответствующий виртуальный адрес будет показан справа, как на рисунке 6-2. Двойной щелчок по отображаемому значению переместит окно дизассемблирования к соответствующему виртуальному адресу. На рисунке 6-2 двойной щелчок по любому из значений, обозначенных сплошной стрелкой, выполнит переход, потому что каждое из них является корректным виртуальным адресом внутри этого конкретного бинарного файла. Двойной щелчок по любым другим значениям не даст эффекта.

Вы приглашены на соглашение об именовании!

Ghidra предоставляет пользователю большую гибкость при назначении меток, но некоторые шаблоны имеют специальное значение и зарезервированы для Ghidra. К ним относятся следующие префиксы, когда за ними следует подчеркивание и адрес: `EXT`, `FUN`, `SUB`, `LAB`, `DAT`, `OFF` и `UNK`. Создавая метку, избегайте этих шаблонов. Кроме того, в метках не допускаются пробелы и непечатаемые символы. С положительной стороны, метки могут иметь длину до 2000 символов. Считайте внимательно, если думаете, что рискуете превысить этот предел.

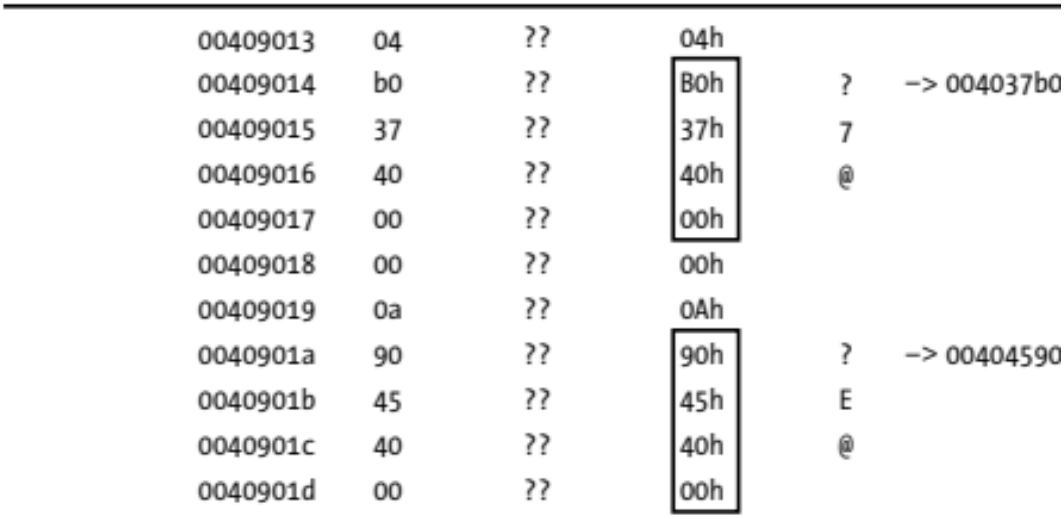


Рисунок 6-2. Листинг с шестнадцатеричными целями навигации

Go To

Когда вы знаете адрес или имя, к которому хотите перейти, например переходите к `main` в ELF-бинарнике, чтобы начать анализ, можно прокручивать листинг в поисках адреса, прокручивать папку `Functions` в окне `Symbol Tree`, чтобы найти нужное имя, или использовать функции поиска Ghidra, которые обсуждаются далее в этой главе. В конечном счете самый простой способ попасть к известному адресу или имени - использовать диалог Go To, показанный на рисунке 6-3. Он доступен через `Navigation > Go To` или с помощью горячей клавиши `G`, пока окно дизассемблирования активно.

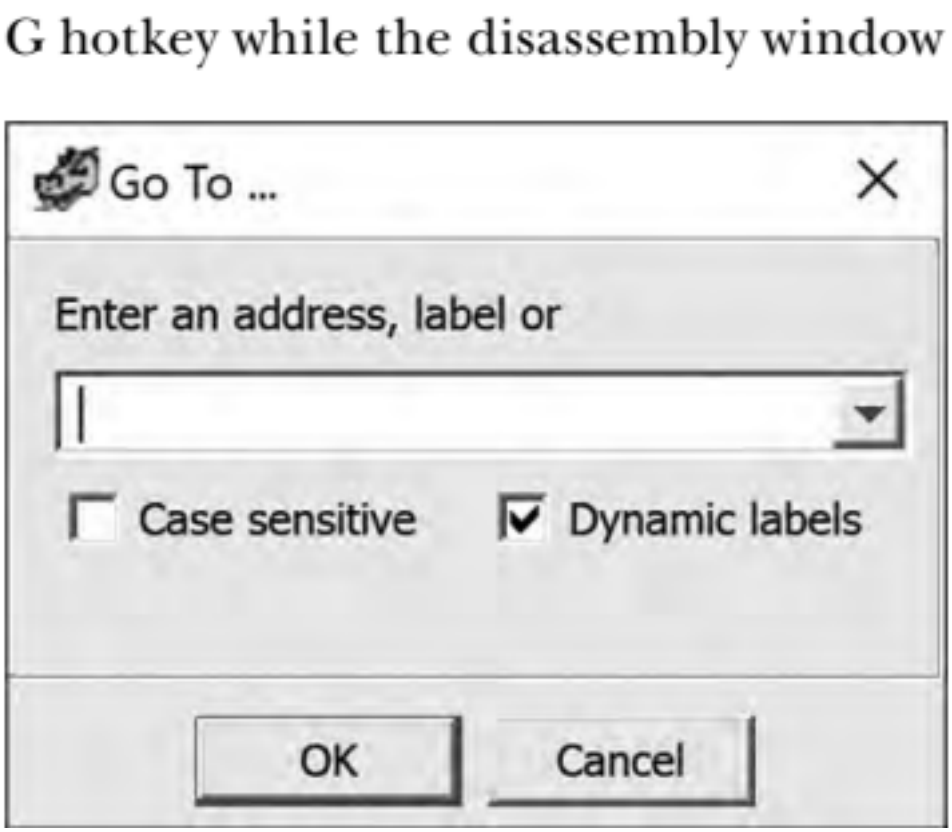


Рисунок 6-3. Диалог Go To

Переход к любому месту в бинарном файле так же прост, как указание корректного адреса, то есть чувствительного к регистру имени символа или шестнадцатеричного значения, и нажатие `OK`; после этого представление сразу перейдет к нужному месту. Значения, введенные в диалог, при последующем использовании доступны через раскрывающийся список истории, что упрощает возврат к ранее запрошенным местам.

История навигации

В качестве последней навигационной возможности Ghidra поддерживает переход вперед и назад на основе порядка, в котором вы перемещались по дизассемблированному представлению. Каждый раз, когда вы переходите к новому месту внутри дизассемблированного представления, ваше текущее место добавляется в список истории. По этому списку можно перемещаться из окна Go To или с помощью значков стрелок влево и вправо на панели инструментов `CodeBrowser`.

В окне Go To, показанном на рисунке 6-3, стрелка справа от текстового поля открывает список выбора, позволяющий выбрать из предыдущих мест, введенных в диалог Go To. Кнопки панели инструментов `CodeBrowser`, видимые ближе к левому верхнему углу на рисунке 6-4, предоставляют знакомое по браузерам поведение перехода вперед и назад. С каждой кнопкой связан подробный раскрывающийся список истории, который дает мгновенный доступ к любому месту в истории навигации без необходимости повторять весь путь по списку. Пример раскрывающегося списка, связанного со стрелкой назад, показан на рисунке 6-4.

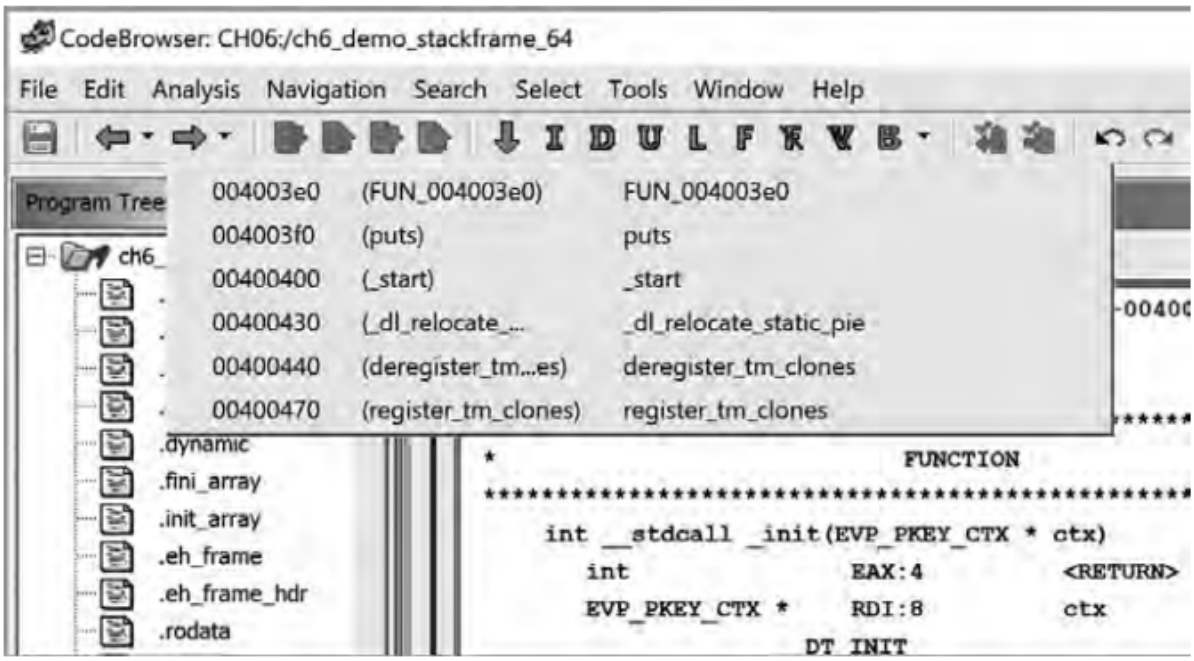


Рисунок 6-4. Стрелки навигации вперед и назад со списком адресов

`alt` + стрелка влево (`option` + стрелка влево на Mac) для навигации назад - одна из самых полезных горячих клавиш, которые стоит запомнить. Навигация назад чрезвычайно удобна, когда вы прошли по цепочке вызовов функций на несколько уровней вглубь и решили вернуться к исходному положению внутри дизассемблированного представления. `alt` + стрелка вправо (`option` + стрелка вправо на Mac) перемещает окно дизассемблирования вперед по списку истории.

Хотя теперь у нас есть гораздо более ясное представление о навигации по дизассемблированному представлению в Ghidra, мы все еще не придали смысл различным местам назначения, которые посетили. В следующем разделе исследуется, почему функции в целом и стековые кадры в частности являются такими важными целями навигации для специалиста по обратной разработке.

Стековые кадры

Поскольку Ghidra - низкоуровневый инструмент анализа, многие его возможности и представления предполагают, что пользователь в некоторой степени знаком с низкоуровневыми деталями скомпилированных языков, сосредоточенными на особенностях генерации машинного языка и управления памятью, используемой программой высокого уровня. Ghidra уделяет особое внимание тому, как компиляторы обрабатывают объявления локальных переменных и обращения к ним. Возможно, вы заметили, что в начале большинства листингов функций значительное количество строк посвящено локальным переменным. Эти строки являются результатом подробного анализа стека, который Ghidra выполняет для каждой функции с помощью анализатора Stack. Такой анализ необходим, потому что компиляторы размещают локальные переменные функции, а в некоторых обстоятельствах и входящие аргументы функции, в блоках памяти, выделенных в стеке. В этом разделе мы рассмотрим, как компиляторы обрабатывают локальные переменные и аргументы функций, чтобы помочь вам лучше понимать детали представления `Listing` в Ghidra.

Механика вызова функций

Вызов функции может требовать память для информации, передаваемой в функцию в виде параметров, или аргументов, и для временного пространства хранения во время выполнения функции. Значения параметров или соответствующие им адреса памяти нужно сохранить там, где функция сможет их найти. Временное пространство часто выделяется программистом через объявление локальных переменных, которые можно использовать внутри функции, но нельзя обращаться к ним после завершения функции. Стековые кадры, также известные как записи активации, - это блоки памяти, выделенные внутри стека времени выполнения программы и посвященные конкретному вызову функции.

Компиляторы используют стековые кадры, чтобы сделать выделение и освобождение параметров функций и локальных переменных прозрачным для программиста. Для соглашений о вызове, передающих параметры в стеке, компилятор вставляет код, помещающий параметры функции в стековый кадр до передачи управления самой функции, после чего компилятор вставляет код для выделения достаточного объема памяти под локальные переменные функции. В некоторых случаях адрес, по которому функция должна вернуться, также сохраняется внутри нового стекового кадра. Стековые кадры также делают возможной рекурсию[1], поскольку каждый рекурсивный вызов функции получает собственный стековый кадр, аккуратно отделяя каждый вызов от предыдущего.

При вызове функции выполняются следующие операции:

1. Вызывающая функция помещает все параметры, требуемые вызываемой функцией, в места, заданные соглашением о вызове, используемым вызываемой функцией. Указатель стека программы может измениться, если параметры передаются в стеке времени выполнения.
2. Вызывающая функция передает управление вызываемой функции инструкцией вроде x86 `CALL`, ARM `BL` или MIPS `JAL`. Адрес возврата сохраняется в стеке программы или в регистре процессора.
3. При необходимости вызываемая функция настраивает указатель кадра и сохраняет все значения регистров, которые вызывающая функция ожидает получить неизменными[2].
4. Вызываемая функция выделяет пространство для любых локальных переменных, которые ей могут потребоваться. Часто это делается путем изменения указателя стека программы для резервирования пространства в стеке времени выполнения.
5. Вызываемая функция выполняет свои операции, потенциально обращаясь к переданным ей параметрам и создавая результат. Если функция возвращает результат, он часто помещается в конкретный регистр или регистры, которые вызывающая функция может проверить после возврата функции.
6. Когда функция завершает свои операции, любое стековое пространство, зарезервированное для локальных переменных, освобождается. Часто это делается обращением действий, выполненных на шаге 4.
7. Регистры, значения которых были сохранены на шаге 3 от имени вызывающей функции, восстанавливаются к исходным значениям.
8. Вызываемая функция возвращает управление вызывающей функции. Типичные инструкции для этого включают x86 `RET`, ARM `POP` и MIPS `JR`. В зависимости от используемого соглашения о вызове эта операция также может удалить один или несколько параметров из стека программы.
9. После того как вызывающая функция снова получает управление, ей может понадобиться удалить параметры из стека программы, восстановив указатель стека программы к значению, которое он имел до шага 1.

Шаги 3 и 4 настолько часто выполняются при входе в функцию, что вместе называются прологом функции. Аналогично шаги 6-8 составляют эпилог функции. Все эти операции, кроме шага 5, являются частью накладных расходов, связанных с вызовом функции, которые могут быть неочевидны в исходном коде программы высокого уровня, но вполне наблюдаемы в языке

ассемблера.

> **Они правда исчезли?**
>
> Когда мы говорим об "удалении" элементов из стека, а также об удалении целых стековых кадров, мы имеем в виду, что указатель стека изменяется так, что указывает на данные ниже в стеке, и удаленное содержимое больше недоступно через операцию `POP`. Пока это содержимое не перезаписано операцией `PUSH`, оно все еще там. С точки зрения программирования это считается удалением. С точки зрения цифровой криминалистики нужно просто поискать немного внимательнее, чтобы найти содержимое. С точки зрения инициализации переменных это означает, что любые неинициализированные локальные переменные внутри стекового кадра могут содержать устаревшие значения, оставшиеся в памяти от последнего использования конкретного диапазона байтов стека.

Соглашения о вызовах

При передаче аргументов от вызывающей стороны к вызываемой вызывающая функция должна сохранять параметры ровно там, где вызываемая функция ожидает их найти; иначе могут возникнуть серьезные проблемы. Соглашение о вызове точно определяет, куда вызывающая функция должна помещать любые параметры, требуемые функцией: в конкретные регистры, в стек программы или и в регистры, и в стек. Когда параметры передаются в стеке программы, соглашение о вызове также определяет, кто отвечает за их удаление из стека после завершения вызываемой функции: вызывающая сторона или вызываемая.

Независимо от архитектуры, для которой выполняется обратная разработка, понять код вокруг вызова функции будет трудно, если не понимать используемые соглашения о вызовах. В следующих разделах мы рассмотрим некоторые распространенные соглашения о вызовах, встречающиеся в скомпилированном коде C и C++.

Аргументы в стеке и регистрах

Аргументы функций могут передаваться в регистрах процессора, в стеке программы или в комбинации того и другого. Когда аргументы помещаются в стек, вызывающая функция выполняет запись в память, часто `PUSH`, чтобы поместить аргумент в стек, а вызываемая функция затем должна выполнить чтение из памяти, чтобы получить доступ к аргументу. Чтобы ускорить процесс вызова функции, некоторые соглашения о вызовах передают аргументы в регистрах процессора. Когда аргумент передается в регистре, нет необходимости выполнять операции записи и чтения памяти, поскольку аргумент сразу доступен вызываемой функции в назначенном регистре. Единственный недостаток регистровых соглашений о вызовах состоит в том, что процессоры имеют конечное число регистров, а списки аргументов функций могут быть произвольно длинными, поэтому соглашения должны корректно обрабатывать функции, требующие больше аргументов, чем доступно регистров. Избыточные аргументы, которые "выливаются" из доступных регистров, обычно помещаются в стек.

Соглашение о вызове C

Соглашение о вызове C - стандартное соглашение о вызове, используемое большинством компиляторов C при генерации вызовов функций. Ключевое слово `_cdecl` можно использовать в прототипе функции, чтобы принудительно применить это соглашение о вызове в программах C/C++. Соглашение о вызове `cdecl` задает, что вызывающая функция помещает любые размещенные в стеке параметры функции в стек в порядке справа налево и что вызывающая функция, а не вызываемая, удаляет параметры из стека после завершения вызываемой функции. Для 32-разрядных x86-бинарников `cdecl` передает все аргументы в стеке программы. Для 64-разрядных x86-бинарников `cdecl` различается по операционным системам; в Linux до шести аргументов помещаются в регистры `RDI`, `RSI`, `RDX`, `RCX`, `R8` и `R9` именно в таком порядке, а все дополнительные аргументы переходят в стек. Для ARM-бинарников `cdecl` передает первые четыре аргумента в регистрах `R0`-`R3`, а пятый и последующие аргументы переходят в стек.

Когда размещенные в стеке аргументы помещаются в стек в порядке справа налево, самый левый аргумент всегда оказывается на вершине стека при вызове функции. Это упрощает поиск первого аргумента независимо от того, сколько аргументов было передано.

Это также упрощает для вызываемой функции поиск ожидаемых аргументов независимо от числа параметров, которых функция ожидает, и делает соглашение о вызове `cdecl` идеально подходящим для функций, которые могут принимать переменное число аргументов, например `printf`.

Требование, чтобы вызывающая функция удаляла параметры из стека, означает, что вы часто будете видеть инструкции, которые корректируют указатель стека программы сразу после возврата из вызываемой функции. В случае функций, способных принимать переменное число аргументов, вызывающая сторона точно знает, сколько аргументов она передала функции, и легко может выполнить правильную корректировку, тогда как вызываемая функция заранее не знает, сколько параметров она получит.

В следующих примерах мы рассмотрим вызовы функций в 32-разрядном x86-бинарном файле, каждая из которых использует свое соглашение о вызове. Первая функция имеет следующий прототип:

```
void demo_cdecl(int w, int x, int y, int z);
```

По умолчанию эта функция будет использовать соглашение о вызове `cdecl`, ожидая, что четыре параметра будут помещены в стек в порядке справа налево, и требуя, чтобы вызывающая сторона очистила параметры из стека. Для следующего вызова функции на C:

```
demo_cdecl(1, 2, 3, 4);    // вызов demo_cdecl (на C)
```

компилятор мог бы сгенерировать такой код:

```
PUSH   4              ; поместить параметр z
PUSH   3              ; поместить параметр y
PUSH   2              ; поместить параметр x
PUSH   1              ; поместить параметр w
CALL   demo_cdecl     ; вызвать функцию
ADD    ESP, 16        ; скорректировать ESP до прежнего значения
```

Четыре операции PUSH изменяют указатель стека программы (ESP) на 16 байт (`4 * sizeof(int)` на 32-разрядной архитектуре), и это изменение отменяется сразу после возврата из `demo_cdecl`. Следующий прием, который использовался в некоторых версиях компиляторов GNU (`gcc` и `g++`), также соблюдает соглашение о вызове `cdecl`, но устраняет необходимость для вызывающей стороны явно очищать параметры из стека после каждого вызова `demo_cdecl`:

```
MOV    [ESP+12], 4    ; переместить параметр z в четвертую позицию стека
MOV    [ESP+8], 3     ; переместить параметр y в третью позицию стека
MOV    [ESP+4], 2     ; переместить параметр x во вторую позицию стека
MOV    [ESP], 1       ; переместить параметр w на вершину стека
CALL   demo_cdecl     ; вызвать функцию
```

В этом примере при размещении параметров для `demo_cdecl` в стеке указатель стека программы не изменяется. Обратите внимание, что оба способа приводят к тому, что в момент вызова функции указатель стека указывает на самый левый стековый аргумент.

Стандартное соглашение о вызове

В 32-разрядных DLL Windows Microsoft широко использует соглашение о вызове, которое она назвала стандартным соглашением о вызове. В исходном коде его можно потребовать с помощью модификатора `_stdcall` в объявлении функции, как показано здесь:

```
void _stdcall demo_stdcall(int w, int x, int y);
```

Чтобы избежать путаницы вокруг слова "стандартное", далее в книге мы будем называть это соглашение о вызове соглашением `stdcall`.

Соглашение о вызове `stdcall` также требует, чтобы все размещаемые в стеке параметры функции помещались в стек программы в порядке справа налево, но за очистку любых размещенных в стеке аргументов после завершения функции отвечает вызываемая функция. Это возможно только для функций, принимающих фиксированное число параметров; функции с переменным числом аргументов, такие как `printf`, не могут использовать соглашение о вызове `stdcall`.

Функция `demo_stdcall` ожидает три целочисленных параметра, занимающих в стеке всего 12 байт (`3 * sizeof(int)` на 32-разрядной архитектуре). x86-компилятор может использовать специальную форму инструкции RET, чтобы одновременно снять адрес возврата с вершины стека и прибавить к указателю стека значение, очищающее размещенные в стеке аргументы функции. В случае `demo_stdcall` для возврата к вызывающей стороне мы могли бы увидеть следующую инструкцию:

```
RET 12    ; вернуться и очистить 12 байт из стека
```

Использование `stdcall` устраняет необходимость очищать параметры из стека после каждого вызова функции, что дает немного меньшие и немного более быстрые программы. По соглашению Microsoft использует `stdcall` для всех функций с фиксированными аргументами, экспортируемых из 32-разрядных файлов разделяемых библиотек (DLL). Это важный момент, о котором нужно помнить, если вы пытаетесь создать прототипы функций или бинарно-совместимые замены для каких-либо компонентов разделяемой библиотеки.

Соглашение fastcall для x86

Компиляторы Microsoft C/C++ и GNU gcc/g++ (версии 3.4 и новее) распознают соглашение `fastcall`, вариант соглашения `stdcall`, в котором первые два параметра помещаются соответственно в регистры ECX и EDX. Все оставшиеся параметры помещаются в стек в порядке справа налево, а вызываемые функции отвечают за удаление параметров из стека при возврате к вызывающей стороне. Следующее объявление демонстрирует использование модификатора `fastcall`:

```
void fastcall demo_fastcall(int w, int x, int y, int z);
```

Для следующего вызова функции на C:

```
demo_fastcall(1, 2, 3, 4);      // вызов demo_fastcall (на C)
```

компилятор мог бы сгенерировать такой код:

```
PUSH   4              ; переместить параметр z во вторую позицию стека
PUSH   3              ; переместить параметр y на вершину стека
MOV    EDX, 2         ; переместить параметр x в EDX
MOV    ECX, 1         ; переместить параметр w в ECX
CALL   demo_fastcall  ; вызвать функцию
```

После возврата из `demo_fastcall` корректировка стека не требуется, поскольку `demo_fastcall` отвечает за очистку параметров y и z из стека при возврате к вызывающей стороне. Важно понимать, что, поскольку два аргумента передаются в регистрах, вызываемой функции нужно очистить из стека только 8 байт, хотя всего у функции четыре аргумента.

Соглашения о вызовах C++

Нестатические функции-члены в классах C++ должны получать доступ к указателю на объект, который используется для вызова функции (указателю `this`).[3] Адрес объекта, используемого для вызова функции, должен быть предоставлен вызывающей стороной как параметр, но стандарт языка C++ не задает, как именно он должен передаваться, поэтому неудивительно, что разные компиляторы используют разные приемы.

В x86 компилятор Microsoft C++ использует соглашение о вызове `thiscall`, которое передает `this` в регистре ECX/RCX и требует, чтобы нестатическая функция-член очищала параметры из стека, как в `stdcall`. Компилятор GNU g++ рассматривает `this` как подразумеваемый первый параметр любой нестатической функции-члена и во всех остальных отношениях ведет себя так, будто используется соглашение `cdecl`. Поэтому для 32-разрядного кода, скомпилированного g++, `this` помещается на вершину стека перед вызовом нестатической функции-члена, а вызывающая сторона отвечает за удаление параметров (их всегда будет как минимум один) из стека после возврата функции. Дополнительные характеристики скомпилированных программ C++ обсуждаются в главах 8 и 20.

Другие соглашения о вызовах

Полное рассмотрение каждого соглашения о вызове потребовало бы отдельной книги. Соглашения о вызовах часто зависят от операционной системы, языка, компилятора и/или процессора, и вам может потребоваться самостоятельное исследование, если вы столкнетесь с кодом, созданным менее распространенными компиляторами. Однако несколько дополнительных ситуаций заслуживают отдельного упоминания: оптимизированный код, пользовательский код на языке ассемблера и системные вызовы.

Когда функции экспортируются для использования другими программистами, например библиотечные функции, важно, чтобы они придерживались хорошо известных соглашений о вызовах, тогда программисты смогут легко взаимодействовать с этими функциями.

С другой стороны, если функция предназначена только для внутреннего использования программой, то применяемое этой функцией соглашение о вызове должно быть известно только внутри программы. В таких случаях оптимизирующие компиляторы могут выбирать альтернативные соглашения о вызове, чтобы генерировать более быстрый код. Например, использование параметра `/GL` с Microsoft C/C++ предписывает выполнять "оптимизацию всей программы", что может привести к оптимизированному использованию регистров на границах функций, а ключевое слово `regparm` в GNU `gcc/g++` позволяет программисту указать, что до трех аргументов должны передаваться в регистрах.

Когда программисты берут на себя труд писать на языке ассемблера, они получают полный контроль над тем, как параметры будут передаваться любым созданным ими функциям. Если они не хотят сделать свои функции доступными другим программистам, программисты на ассемблере свободны передавать параметры любым подходящим для них способом. Поэтому будьте особенно внимательны при анализе пользовательского ассемблерного кода, например процедур обфускации и шеллкода.

Системный вызов - это особый тип вызова функции, используемый для запроса службы операционной системы. Системные вызовы обычно вызывают переход состояния из пользовательского режима в режим ядра, чтобы ядро операционной системы обслужило запрос пользователя. Способ инициирования системных вызовов различается между операционными системами и процессорами. Например, 32-разрядные системные вызовы Linux x86 могут инициироваться инструкцией `INT 0x80` или инструкцией `sysenter`, тогда как другие x86-операционные системы могут использовать только инструкцию `sysenter` или другие номера прерываний, а 64-разрядный x86-код использует инструкцию `syscall`. Во многих x86-системах (Linux является исключением) параметры системных вызовов помещаются в стек времени выполнения, а номер системного вызова помещается в регистр `EAX` непосредственно перед инициированием системного вызова. Системные вызовы Linux принимают параметры в определенных регистрах и иногда в памяти, когда параметров больше, чем доступных регистров.

Дополнительные соображения о стековом кадре

На любом процессоре регистры являются конечным ресурсом, которым должны совместно и согласованно пользоваться все функции внутри программы. Когда функция (`func1`) выполняется, ее картина мира такова, что она полностью контролирует все регистры процессора. Когда `func1` вызывает другую функцию (`func2`), `func2` может захотеть принять такой же взгляд и использовать все доступные регистры процессора в соответствии со своими нуждами, но если `func2` произвольно изменит регистры, она может уничтожить значения, от которых зависит `func1`.

Чтобы решить эту проблему, все компиляторы следуют четко определенным правилам выделения и использования регистров. Эти правила обычно называют двоичным интерфейсом приложений платформы (ABI). ABI делит регистры на две категории: сохраняемые вызывающей стороной и сохраняемые вызываемой стороной. Когда одна функция вызывает другую, вызывающей стороне нужно сохранить только регистры из категории сохраняемых вызывающей стороной, чтобы предотвратить потерю значений. Любые регистры из категории сохраняемых вызываемой стороной должны быть сохранены вызываемой функцией до того, как этой функции будет разрешено использовать какой-либо из этих регистров для собственных целей. Обычно это происходит как часть последовательности пролога функции, а сохраненные значения вызывающей стороны восстанавливаются в эпилоге функции непосредственно перед возвратом. Регистры, сохраняемые

вызывающей стороной, называют затираемыми регистрами, поскольку вызываемая функция может свободно изменять их содержимое, не сохраняя предварительно ни один из них. И наоборот, регистры, сохраняемые вызываемой стороной, называют незатираемыми регистрами.

ABI System V для 32-разрядных процессоров Intel указывает, что к регистрам, сохраняемым вызывающей стороной, относятся `EAX`, `ECX` и `EDX`, а к регистрам, сохраняемым вызываемой стороной, относятся `EBX`, `EDI`, `ESI`, `EBP` и `ESP`.[4] В скомпилированном коде вы можете заметить, что компиляторы часто предпочитают использовать внутри функции регистры, сохраняемые вызывающей стороной, поскольку освобождены от обязанности сохранять и восстанавливать их содержимое при входе в функцию и выходе из нее.

Размещение локальных переменных

В отличие от соглашений о вызовах, которые задают, как параметры передаются в функцию, никаких соглашений, задающих расположение локальных переменных функции в памяти, нет. При компиляции функции компилятор должен вычислить объем пространства, требуемый локальным переменным функции, вместе с пространством, необходимым для сохранения любых незатираемых регистров, и определить, можно ли выделить эти переменные в регистрах процессора или их нужно разместить в стеке программы. Точный способ такого распределения не важен ни вызывающей функции, ни функциям, которые, в свою очередь, могут быть вызваны, и обычно невозможно определить расположение локальных переменных функции, опираясь только на изучение исходного кода функции. В отношении стековых кадров можно утверждать одно: компилятор должен выделить хотя бы один регистр, чтобы помнить местоположение вновь выделенного стекового кадра функции. Самый очевидный выбор для этого регистра - указатель стека, который по определению указывает на стек и, следовательно, на стековый кадр текущей функции.

Примеры стековых кадров

Когда вы выполняете любую сложную задачу, например обратную разработку бинарного файла, всегда стремитесь эффективно использовать свое время. Когда речь идет о понимании поведения дизассемблированной функции, чем меньше времени вы тратите на изучение типовых последовательностей кода, тем больше времени у вас остается на трудные последовательности. Прологи и эпилоги функций - отличные примеры типовых последовательностей кода, и важно уметь распознавать их, понимать их и быстро переходить к более интересному коду, требующему большего размышления.

Ghidra кратко излагает свое понимание прологов функций в списке локальных переменных в начале листинга каждой функции, и хотя это может сделать код более читаемым, это никак не сокращает объем дизассемблированного кода, который вам нужно прочитать. В следующих примерах мы обсудим два распространенных типа стековых кадров и рассмотрим код, необходимый для их создания, чтобы, встретив похожий код в реальных программах, вы могли быстро пройти его и перейти к сути функции.

Рассмотрим следующую функцию, скомпилированную на 32-разрядном компьютере на базе x86:

```
void helper(int j, int k);    // прототип функции

void demo_stackframe(int a, int b, int c) {
    int x;
    char buffer[64];
    int y;
    int z;
    // тело функции не особенно важно,
    // кроме следующего вызова функции
    helper(z, y);
}
```

Локальным переменным `demo_stackframe` требуется 76 байт (три 4-байтовых целых числа и 64-байтовый буфер). Эта функция могла бы использовать либо `stdcall`, либо `cdecl`, и стековый кадр выглядел бы одинаково.

Пример 1: доступ к локальным переменным через указатель стека

На рисунке 6-5 показан один возможный стековый кадр для вызова `demo_stackframe`. В этом примере компилятор решил использовать указатель стека всякий раз, когда он обращается к переменной, содержащейся в стековом кадре, оставив все остальные регистры доступными для других целей. Если какая-либо инструкция приводит к изменению значения указателя стека, компилятор должен позаботиться о том, чтобы учесть это изменение во всех последующих обращениях к локальным переменным.

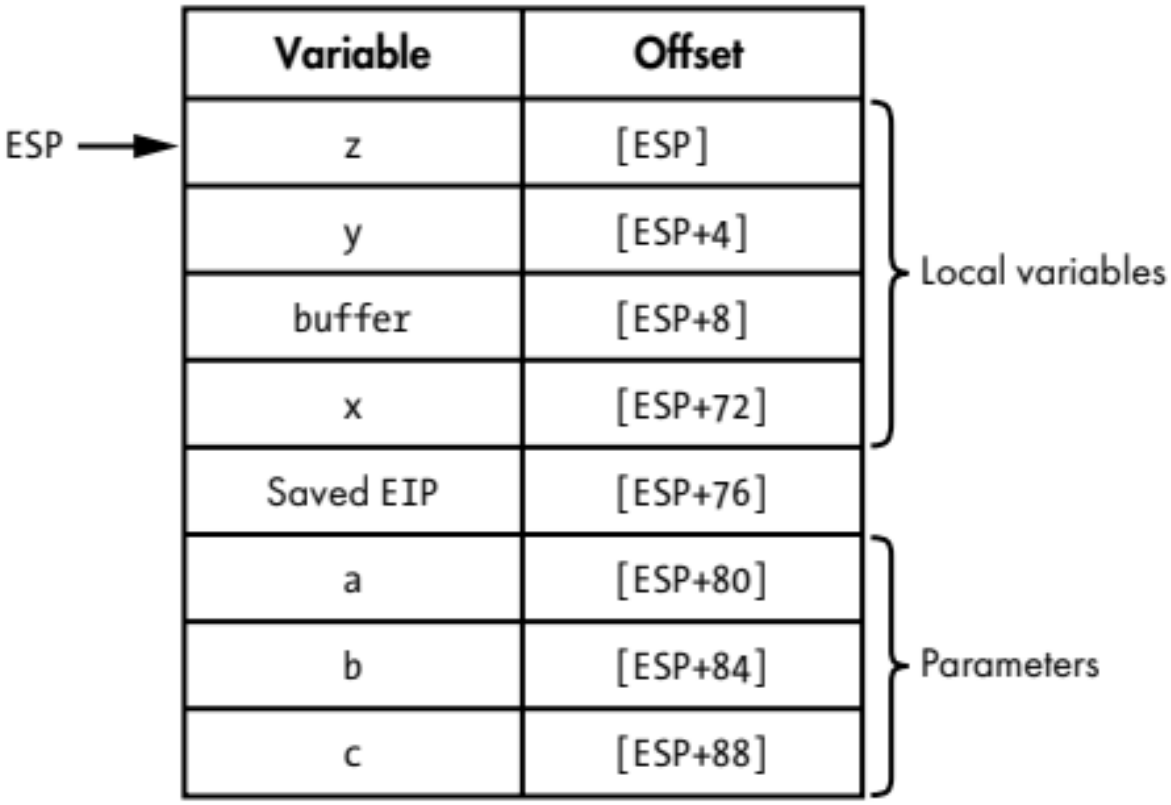


Figure 6-5: Sample stack frame for a function compiled on a 32-bit x86 computer

Рисунок 6-5. Пример стекового кадра для функции, скомпилированной на 32-разрядном x86-компьютере

Пространство для этого кадра создается при входе в `demo_stackframe` однострочным прологом:

```
SUB    ESP, 76        ; выделить достаточно места для всех локальных переменных
```

Столбец Offset на рисунке 6-5 указывает режим адресации x86 (в данном случае база + смещение), необходимый для обращения к каждой из локальных переменных и параметров в стековом кадре. В этом случае `ESP` используется как базовый регистр, а каждое смещение является относительным смещением от `ESP` до начала переменной внутри стекового кадра. Однако смещения, показанные на рисунке 6-5, корректны только пока значение, хранящееся в `ESP`, не изменяется. К сожалению, указатель стека изменяется часто, и компилятор должен постоянно адаптироваться, чтобы при обращении к любым переменным внутри стекового кадра использовались правильные смещения. Рассмотрим вызов `helper`, выполняемый в функции `demo_stackframe`; его код показан здесь:

```
PUSH   dword [ESP+4]  ; поместить y
PUSH   dword [ESP+4]  ; поместить z
CALL   helper
ADD    ESP, 8         ; cdecl требует, чтобы вызывающая сторона очистила параметры
```

Первая инструкция `PUSH` корректно помещает локальную переменную y согласно смещению на рисунке 6-5. На первый взгляд может показаться, что вторая инструкция `PUSH` ошибочно обращается к локальной переменной y второй раз. Однако поскольку все переменные в стековом кадре адресуются относительно `ESP`, а первая инструкция `PUSH` изменяет `ESP`, все смещения на рисунке 6-5 должны быть временно скорректированы. Поэтому после первой инструкции `PUSH` новое смещение для локальной переменной z становится `[ESP+4]`. При изучении функций, которые обращаются к переменным стекового кадра с помощью указателя стека, нужно внимательно

отслеживать любые изменения указателя стека и соответствующим образом корректировать все будущие смещения переменных.

Когда `demo_stackframe` завершилась, ей нужно вернуться к вызывающей стороне. В конечном счете инструкция `RET` снимет нужный адрес возврата с вершины стека в регистр указателя инструкций (в данном случае `EIP`). Прежде чем адрес возврата может быть снят, локальные переменные нужно удалить с вершины стека, чтобы указатель стека правильно указывал на сохраненный адрес возврата при выполнении инструкции `RET`. Для этой конкретной функции (если предполагается использование соглашения о вызове `cdecl`) эпилог становится следующим:

```
ADD    ESP, 76        ; скорректировать ESP, чтобы он указывал на сохраненный
    адрес возврата
RET                   ; вернуться к вызывающей стороне
```

Пример 2: дайте указателю стека передышку

Ценой выделения второго регистра для поиска переменных внутри стекового кадра указателю стека можно позволить свободно изменяться без необходимости заново вычислять смещения для каждой переменной внутри кадра. Конечно, компилятор должен обязаться не изменять этот второй регистр; иначе ему придется столкнуться с теми же проблемами, что были описаны в предыдущем примере. В такой ситуации компилятор должен сначала выбрать регистр для этой цели, а затем сгенерировать код, инициализирующий этот регистр при входе в функцию.

Любой регистр, выбранный для этой цели, называется указателем кадра. В предыдущем примере `ESP` использовался как указатель кадра, и мы можем сказать, что это был стековый кадр на базе `ESP`. ABI большинства архитектур предлагает, какой регистр следует использовать как указатель кадра. Указатель кадра всегда считается незатираемым регистром, поскольку вызывающая функция уже может использовать его для той же цели. В x86-программах регистр `EBP`/`RBP` (расширенный указатель базы) обычно выделяется для использования как указатель кадра. По умолчанию большинство компиляторов генерируют код, использующий в качестве указателя кадра регистр, отличный от указателя стека, хотя обычно существуют параметры, позволяющие указать, что вместо этого должен использоваться указатель стека. Например, GNU `gcc`/`g++` предлагает параметр компилятора `-fomit-frame-pointer`, который генерирует функции, не использующие второй регистр в качестве указателя кадра.

Чтобы увидеть, как будет выглядеть стековый кадр для `demo_stackframe` при использовании выделенного указателя кадра, нужно рассмотреть новый код пролога:

```
PUSH   EBP            ; сохранить значение EBP вызывающей стороны, поскольку это
    незатираемый регистр
MOV    EBP, ESP       ; заставить EBP указывать на сохраненное значение регистра
SUB    ESP, 76        ; выделить пространство для локальных переменных
```

Инструкция `PUSH` сохраняет значение `EBP`, которое сейчас используется вызывающей стороной, поскольку `EBP` является незатираемым регистром. Значение `EBP` вызывающей стороны должно быть восстановлено перед возвратом. Если от имени вызывающей стороны нужно сохранить какие-либо другие регистры (например, `ESI` или `EDI`), компиляторы могут сохранить их одновременно с сохранением `EBP` или отложить их сохранение до выделения локальных переменных. Поэтому стандартного места для хранения сохраненных регистров внутри стекового кадра нет.

После сохранения `EBP` его можно изменить так, чтобы он указывал на текущее положение стека, с помощью инструкции `MOV`, которая копирует текущее значение указателя стека (единственного регистра, который в этот момент гарантированно указывает внутрь стека) в `EBP`. Наконец, как и в стековом кадре на базе `ESP`, выделяется пространство для локальных переменных. Получившаяся

схема стекового кадра показана на рисунке 6-6.

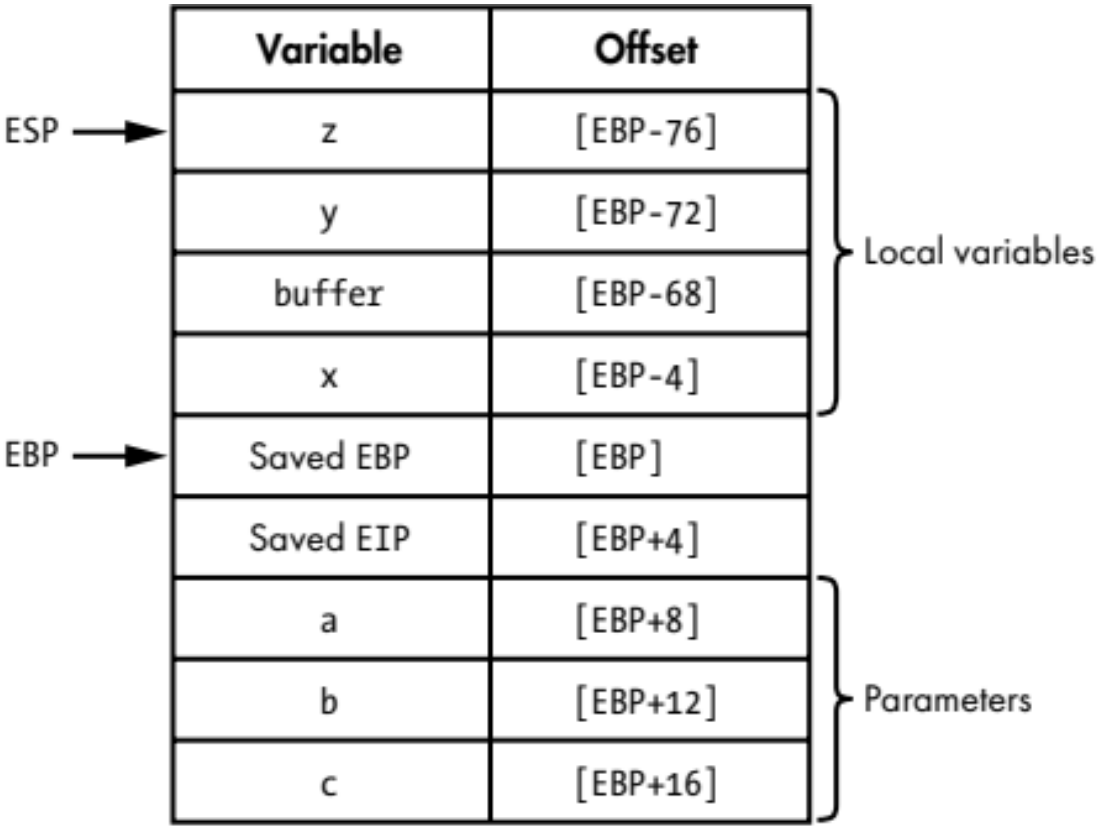


Figure 6-6: An EBP-based stack frame

Рисунок 6-6. Стековый кадр на базе EBP

При выделенном указателе кадра все смещения переменных теперь можно вычислять относительно регистра указателя кадра, как показано на рисунке 6-6. Чаще всего (хотя и не обязательно) положительные смещения используются для доступа к любым размещенным в стеке аргументам функции, тогда как отрицательные смещения используются для доступа к локальным переменным. Когда применяется выделенный указатель кадра, указатель стека может свободно изменяться, не влияя на смещение до любых переменных внутри кадра. Теперь вызов функции `helper` можно реализовать следующим образом:

```
PUSH   dword [ebp-72] ; PUSH y
PUSH   dword [ebp-76] ; PUSH z
CALL   helper
ADD    ESP, 8         ; cdecl требует, чтобы вызывающая сторона очистила параметры
```

То, что указатель стека изменился после первой инструкции `PUSH`, никак не влияет на обращение к локальной переменной `z` в следующей инструкции `PUSH`.

В эпилоге функции, использующей указатель кадра, указатель кадра вызывающей стороны должен быть восстановлен перед возвратом. Если указатель кадра восстанавливается с помощью инструкции POP, локальные переменные должны быть очищены из стека до того, как старое значение указателя кадра можно будет снять со стека, но это становится простым благодаря тому, что текущий указатель кадра указывает на место в стеке, где хранится сохраненное значение указателя кадра. В 32-разрядных x86-программах, использующих EBP как указатель кадра, следующий код представляет типичный эпилог:

```
MOV    ESP, EBP       ; очистить локальные переменные, сбросив ESP
POP    EBP            ; восстановить значение EBP вызывающей стороны
RET                   ; снять адрес возврата, чтобы вернуться к вызывающей стороне
```

Эта операция настолько распространена, что архитектура x86 предлагает инструкцию LEAVE для выполнения той же задачи:

```
LEAVE                 ; копирует EBP в ESP, А ЗАТЕМ снимает значение в EBP
RET                   ; снять адрес возврата, чтобы вернуться к вызывающей стороне
```

Хотя имена регистров и используемых инструкций в других процессорных архитектурах, разумеется, будут отличаться, базовый процесс построения стековых кадров останется тем же. Независимо от архитектуры, вам нужно познакомиться с типичными последовательностями пролога и эпилога, чтобы быстро переходить к анализу более интересного кода внутри функций.

Представления стека в Ghidra

Стековые кадры - понятие времени выполнения; стековый кадр не может существовать без стека и без выполняющейся программы. Хотя это верно, это не означает, что при статическом анализе с такими инструментами, как Ghidra, следует игнорировать понятие стекового кадра. Весь код, необходимый для настройки стековых кадров каждой функции, присутствует в бинарном файле. Благодаря тщательному анализу этого кода мы можем получить подробное понимание структуры стекового кадра любой функции, даже если функция не выполняется. Фактически часть наиболее сложного анализа Ghidra выполняется специально для определения структуры стековых кадров каждой функции, которую она дизассемблирует.

Анализ стековых кадров в Ghidra

Во время начального анализа Ghidra прилагает большие усилия, чтобы отслеживать поведение указателя стека на протяжении функции, отмечая каждую операцию PUSH или POP, а также любые арифметические операции, которые могут изменить указатель стека, например прибавление или вычитание постоянных значений. Цель этого анализа - определить точный размер области локальных переменных, выделенной стековому кадру функции, определить, используется ли в данной функции выделенный указатель кадра (например, распознав последовательность PUSH EBP/MOV EBP, ESP), и распознать все обращения к памяти, относящиеся к переменным внутри стекового кадра функции.

Например, если Ghidra заметила бы инструкцию

```
MOV    EAX, [EBP+8]
```

в теле `demo_stackframe`, она поняла бы, что первый аргумент функции (в данном случае a) загружается в регистр EAX (см. рисунок 6-6). Ghidra может различать обращения к памяти, которые получают доступ к аргументам функции (тем, что находятся ниже сохраненного адреса возврата), и обращения к локальным переменным (тем, что находятся выше сохраненного адреса возврата).

Ghidra делает дополнительный шаг и определяет, к каким местам памяти внутри стекового кадра есть прямые обращения. Например, хотя стековый кадр на рисунке 6-6 имеет размер 96 байт, есть только семь переменных, обращения к которым мы, вероятно, увидим (четыре локальные и три параметра). В результате вы можете сосредоточить внимание на семи вещах, которые Ghidra определила как важные, и меньше думать обо всех байтах, которым Ghidra не дала имен. В процессе идентификации и именования отдельных элементов внутри стекового кадра Ghidra также распознает пространственные отношения переменных друг к другу. Это может быть чрезвычайно полезно в некоторых сценариях, например при разработке эксплойтов, когда Ghidra упрощает точное определение того, какие переменные могут быть перезаписаны в результате переполнения буфера. Декомпилятор Ghidra (рассматривается в главе 19) также сильно опирается на анализ стековых кадров и использует его результаты, чтобы вывести, сколько аргументов принимает функция и какие объявления локальных переменных необходимы в декомпилированном коде.

Стековые кадры в представлении Listing

Понимание поведения функции часто сводится к пониманию типов данных, которыми эта функция манипулирует. Когда вы читаете дизассемблированный листинг, одна из первых возможностей понять данные, которыми манипулирует функция, - посмотреть разбор стекового кадра функции. Ghidra предлагает два представления стекового кадра любой функции: сводное представление и подробное представление. Чтобы понять эти два представления, мы будем ссылаться на следующую версию `demo_stackframe`, которую скомпилировали с помощью `gcc`:

```
void demo_stackframe(int i, int j, int k) {
    int x = k;
    char buffer[64];
    int y = j;
    int z = 10;
    buffer[0] = 'A';
    helper(z, y);
}
```

Поскольку локальные переменные существуют только во время выполнения функции, любая локальная переменная, которая не используется в функции осмысленным образом, по существу бесполезна. С точки зрения высокого уровня следующий код является функционально эквивалентной (можно сказать, оптимизированной) версией `demo_stackframe`:

```
void demo_stackframe_2(int b) {
    helper(10, b);
}
```

(Так что, хотя эта функция делает вид, будто выполняет много работы, на самом деле она просто старается выглядеть занятой, чтобы произвести впечатление на начальника.)

В исходной версии `demo_stackframe` локальные переменные `x` и `y` инициализируются соответственно из параметров `k` и `j`. Локальная переменная `z` инициализируется литеральным значением `10`, а первый символ в 64-байтовом локальном массиве с именем `buffer` инициализируется символом `'A'`. Соответствующий дизассемблированный листинг Ghidra для этой функции при использовании стандартного автоматического анализа показан на рисунке 6-7.

Рисунок 6-7. Дизассемблирование функции demo_stackframe

В этом листинге есть много моментов, которые нужно разобрать, поскольку мы начинаем знакомиться с нотацией дизассемблирования Ghidra. В этом обсуждении мы сосредоточимся на двух разделах дизассемблирования, которые дают нам особенно полезную информацию. Начнем с увеличенного рассмотрения сводки стека, показанной в следующем листинге. (Вы всегда можете вернуться к рисунку 6-7, чтобы увидеть этот сводный стековый кадр в контексте.) Чтобы упростить обсуждение, термины локальная переменная и аргумент используются для различения двух типов переменных. Термин переменная используется, когда речь идет об обоих типах вместе.

```
undefined   AL:1            <RETURN>
undefined   Stack[0x4]:1    param_1
undefined4  Stack[0x8]:4    param_2
undefined4  Stack[0xc]:4    param_3
undefined4  Stack[-0x10]:4  local_10
undefined4  Stack[-0x14]:4  local_14
undefined4  Stack[-0x18]:4  local_18
undefined1  Stack[-0x58]:1  local_58
```

Ghidra предоставляет сводное представление стека, где перечисляет каждую переменную, к которой есть прямое обращение внутри стекового кадра, вместе с важной информацией о каждой

из них. Осмысленные имена (в третьем столбце), которые Ghidra назначает каждой переменной, дают информацию о переменных, когда вы видите их в дизассемблированном листинге: имена аргументов, переданных функции, начинаются с полезного префикса `param_`, а имена локальных переменных начинаются с `local_`. В результате легко различать эти два типа переменных.

Префиксы имен переменных сочетаются с информацией о позиции или местоположении переменной. Для аргументов, таких как `param_3`, число в имени соответствует позиции аргумента в списке параметров функции. Для локальных переменных, таких как `local_10`, число является шестнадцатеричным смещением, представляющим местоположение переменной внутри стекового кадра. Местоположение также можно найти в среднем столбце листинга, слева от имен. Этот столбец состоит из двух компонентов, разделенных двоеточием: оценки Ghidra размера переменной в байтах и местоположения переменной внутри стекового кадра, представленного как смещение этой переменной от начального значения указателя стека при входе в функцию.

Табличное представление этого стекового кадра показано на рисунке 6-8. Как обсуждалось ранее, параметры находятся ниже сохраненного адреса возврата и поэтому имеют положительное смещение от адреса возврата. Локальные переменные находятся выше сохраненного адреса возврата и поэтому имеют отрицательное смещение. Порядок локальных переменных в стеке не совпадает с порядком, в котором они были объявлены в исходном коде, показанном ранее в этой главе, поскольку компилятор свободен располагать локальные переменные в стеке исходя из множества внутренних факторов, таких как выравнивание по байтам и размещение массивов относительно других локальных переменных.

Address	Desc	Name
-0x68	helper parameters	
-0x64		
-0x58	buffer	local_58
-0x18	z	local_18
-0x14	y	local_14
-0x10	x	local_10
-0x04	Saved EBP	
0x00	Saved RET	
0x04	i	param_1
0x08	j	param_2
0x0c	k	param_3

Рисунок 6-8. Пример изображения стекового кадра

Анализ стекового кадра с помощью декомпилятора

Помните функциональный эквивалент кода, который мы определили?

```
void demo_stackframe_2(int j) {
    helper(10, j);
}
```

Код, который декомпилятор сгенерировал для этой функции, показан на рисунке 6-9. Сгенерированный декомпилятором Ghidra код очень похож на наш оптимизированный код, поскольку декомпилятор включает только исполняемый эквивалент исходной функции. (Исключение - включение `param_1`.)

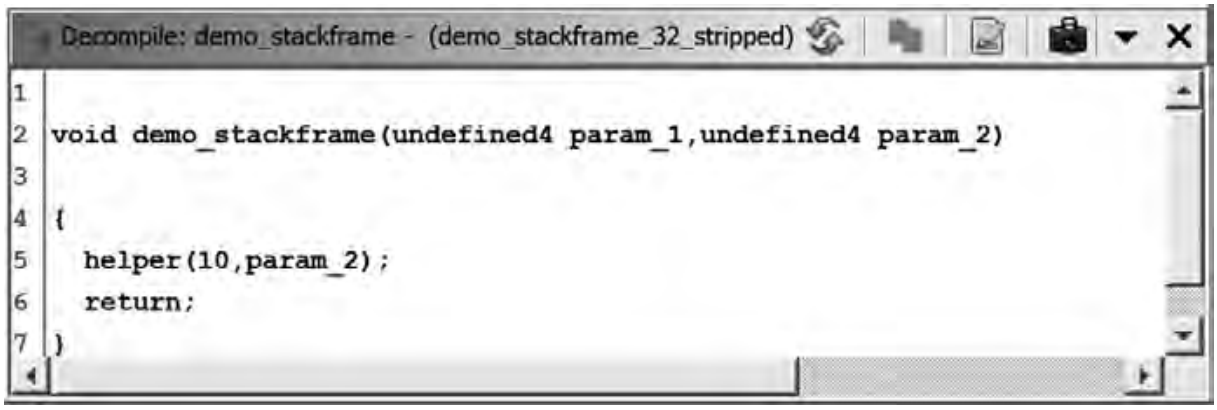


Рисунок 6-9. Окно `Decompiler` *для demo_stackframe (с анализатором* `Decompiler Parameter ID`*)*

Вы могли заметить, что функция `demo_stackframe` принимала три целочисленных параметра, но только два из них (`param_1` и `param_2`) учтены в листинге декомпилятора. Какого не хватает и почему? Оказывается, дизассемблер Ghidra и декомпилятор Ghidra подходят к именам немного по-разному. Хотя оба именуют все параметры вплоть до последнего, к которому есть обращение, декомпилятор именует только параметры вплоть до последнего, который используется осмысленным образом. Один из анализаторов, который Ghidra может выполнить для вас, называется `Decompiler Parameter ID`. В большинстве случаев этот анализатор не включен по умолчанию (он включается только для файлов `Windows PE` размером меньше 2 МБ). Когда анализатор `Decompiler Parameter ID` включен, Ghidra использует сведения о параметрах, полученные декомпилятором, чтобы именовать параметры функции в дизассемблированном листинге. Следующий листинг показывает переменные в дизассемблированном листинге `demo_stackframe`, когда анализатор `Decompiler Parameter ID` включен:

```
undefined   AL:1            <RETURN>
undefined   Stack[0x4]:4    param_1
undefined4  Stack[0x8]:4    param_2
undefined4  Stack[-0x10]:4  local_10
undefined4  Stack[-0x14]:4  local_14
undefined4  Stack[-0x18]:4  local_18
undefined1  Stack[-0x58]:1  local_58
```

Обратите внимание, что `param_3` больше не появляется в списке аргументов функции, поскольку декомпилятор определил, что он не используется внутри функции осмысленным образом. Этот конкретный стековый кадр подробнее обсуждается в главе 8. Если вам когда-либо понадобится, чтобы Ghidra выполнила анализ `Decompiler Parameter ID` после открытия бинарного файла с отключенным анализатором, вы всегда можете выбрать `Analysis > One Shot > Decompiler Parameter ID`, чтобы запустить анализатор уже после первоначального анализа.

Локальные переменные как операнды

Перенесем внимание на собственно дизассемблированную часть следующего листинга:

```
08048473 55           PUSH   EBP
08048474 89 e5        MOV    EBP,ESP
08048476 83 ec 58     SUB    ESP,0x58
08048479 8b 45 10     MOV    EAX,dword ptr [EBP + param_3]
0804847c 89 45 f4     MOV    dword ptr [EBP + local_10],EAX
0804847f 8b 45 0c     MOV    EAX,dword ptr [EBP + param_2]
08048482 89 45 f0     MOV    dword ptr [EBP + local_14],EAX
08048485 c7 45 ec     MOV    dword ptr [EBP + local_18],0xa
         0a 00 00 00
0804848c c6 45 ac 41  MOV    byte ptr [EBP + local_58],0x41
08048490 83 ec 08     SUB    ESP,0x8
08048493 ff 75 f0     PUSH   dword ptr [EBP + local_14]
08048496 ff 75 ec     PUSH   dword ptr [EBP + local_18]
```

Функция использует распространенный пролог функции для стекового кадра на базе `EBP`. Компилятор выделяет в стековом кадре 88 байт (`0x58` равно 88) пространства локальных переменных. Это немного больше оценочных 76 байт и показывает, что компиляторы иногда дополняют пространство локальных переменных лишними байтами, чтобы поддержать определенное выравнивание памяти внутри стекового кадра.

Важное отличие между дизассемблированным листингом Ghidra и анализом стекового кадра, который мы выполнили ранее, состоит в том, что в дизассемблированном листинге вы не видите обращений к памяти, похожих на `[EBP-12]` (которые, например, можно увидеть в `objdump`). Вместо этого Ghidra заменила все постоянные смещения символическими именами, соответствующими символам в представлении стека и их относительным смещениям от начального местоположения указателя стека функции. Это соответствует цели Ghidra - создавать дизассемблирование более высокого уровня. С символическими именами просто легче работать, чем с числовыми константами. Кроме того, это дает нам имя, которое можно изменить так, чтобы оно соответствовало нашему пониманию назначения переменной, когда оно станет известно. Для справки Ghidra действительно отображает сырую форму текущей инструкции, без каких-либо меток, в самом нижнем правом углу окна `CodeBrowser`.

В этом примере, поскольку у нас есть исходный код для сравнения, мы можем сопоставить сгенерированные Ghidra имена переменных с именами, использованными в исходном коде, используя разные подсказки, доступные в дизассемблировании:

1. Во-первых, `demo_stackframe` принимает три параметра, `i`, `j` и `k`, которые соответствуют переменным `param_1`, `param_2` и `param_3` соответственно.
2. Локальная переменная `x` (`local_10`) инициализируется из параметра `k` (`param_3`).
3. Аналогично, локальная переменная `y` (`local_14`) инициализируется из параметра `j` (`param_2`).
4. Локальная переменная `z` (`local_18`) инициализируется значением `10`.
5. Первый символ `buffer[0]` (`local_58`) в 64-байтовом символьном массиве инициализируется значением `A` (ASCII `0x41`).
6. Два аргумента для вызова `helper` помещаются в стек. 8-байтовая корректировка стека, предшествующая этим двум операциям `PUSH`, вместе с двумя `PUSH` дает итоговое изменение стека на 16 байт. В результате стек сохраняет любое 16-байтовое выравнивание, достигнутое ранее в программе.

Редактор стекового кадра Ghidra

Помимо сводного представления стека, Ghidra предлагает подробный редактор стекового кадра, в котором учтен каждый байт, выделенный стековому кадру. Окно Stack Frame Editor открывается щелчком правой кнопкой мыши и выбором Function > Edit Stack Frame в контекстном меню, когда в сводном представлении стека Ghidra для функции выбрана функция или стековая переменная. Получившееся окно для функции `demo_stackframe` показано на рисунке 6-10.

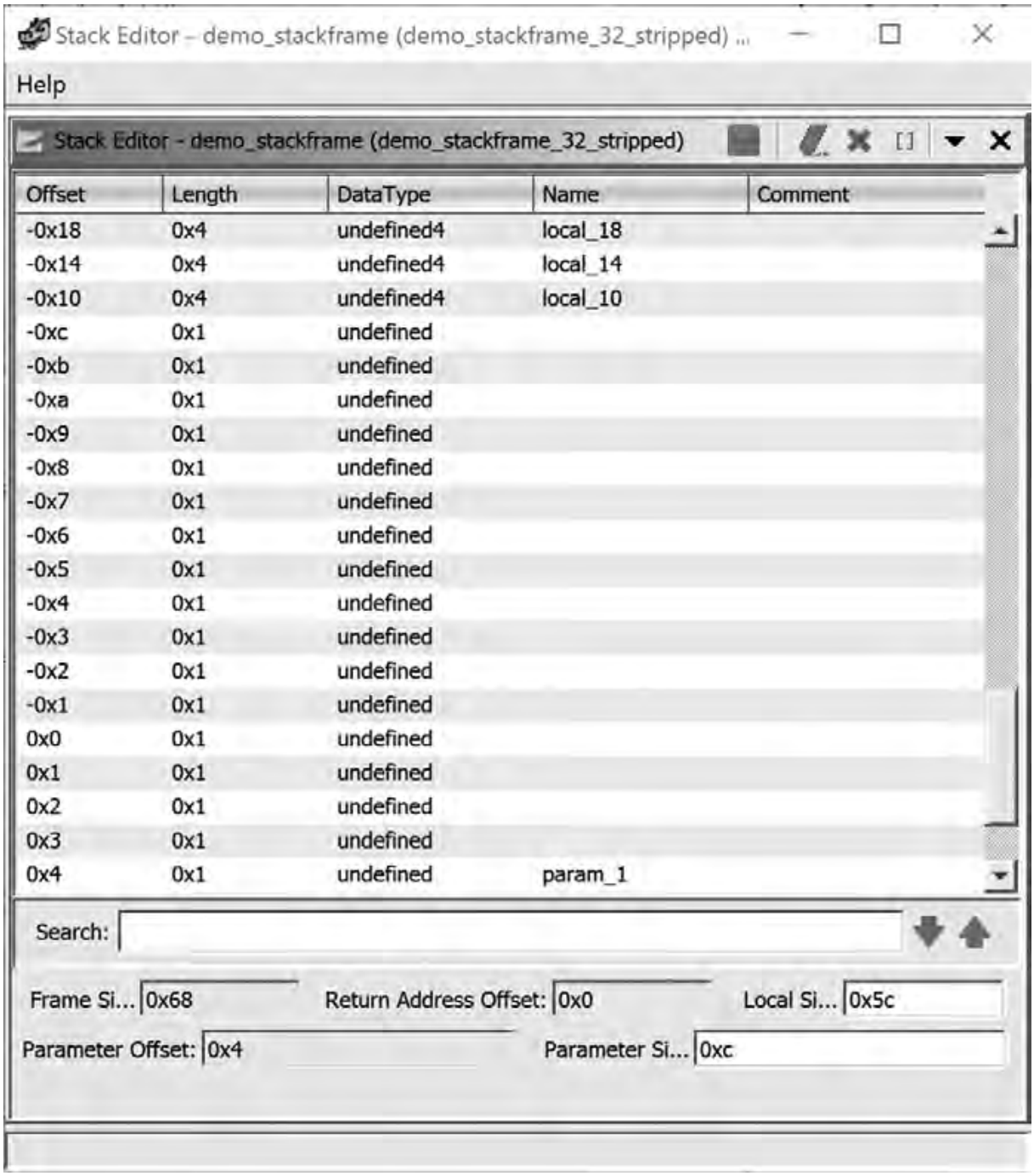


Рисунок 6-10. Пример сводного представления стека

Поскольку подробное представление учитывает каждый байт стекового кадра, оно занимает значительно больше места, чем сводное представление. Часть стекового кадра, показанная на рисунке 6-10, охватывает всего 29 байт, что является лишь небольшой частью всего стекового кадра. Кроме того, в предыдущем листинге `local_10`, `local_14` и `local_18` имеют прямые обращения в дизассемблированном листинге, где их содержимое инициализировалось с помощью

dword-записей (4-байтовых записей). На основе того, что перемещались 32 бита данных, Ghidra может вывести, что каждая из этих переменных является 4-байтовой величиной, и помечает каждую как undefined4 (4-байтовая переменная неизвестного типа).

Поскольку это Stack Frame Editor, мы можем использовать это окно, чтобы редактировать поля, менять форматы отображения и добавлять дополнительную информацию, если это помогает нашему процессу. Например, мы могли бы добавить имя для сохраненного адреса возврата по адресу 0x0.

Параметры на базе регистров

Соглашения о вызовах ARM используют до четырех регистров для передачи параметров функциям без использования стека. Некоторые соглашения x86-64 используют до шести регистров, а некоторые соглашения MIPS - до восьми. Параметры на базе регистров немного сложнее идентифицировать, чем параметры на базе стека.

Рассмотрим следующие два фрагмента языка ассемблера:

```
stackargs:              ; пример 32-разрядной функции x86
    PUSH EBP            ; сохранить ebp из сохраняемых вызываемой стороной
    MOV  EBP, ESP       ; настроить указатель кадра

    MOV  EAX, [EBP + 8]  ; получить размещенный в стеке аргумент
    MOV  CL, byte [EAX]  ; разыменовать полученный аргумент-указатель
    ...
    RET

regargs:                ; пример функции x86-64
    PUSH RBP            ; сохранить rbp из сохраняемых вызываемой стороной
    MOV  RBP, RSP       ; настроить указатель кадра

    MOV  CL, byte [RDI]  ; разыменовать аргумент-указатель
    ...
    RET
```

В первой функции выполняется доступ к области стека ниже сохраненного адреса возврата, и мы заключаем, что функция ожидает как минимум один аргумент. Ghidra, как и большинство высококлассных дизассемблеров, выполняет анализ указателя стека и указателя кадра, чтобы идентифицировать инструкции, обращающиеся к элементам стекового кадра функции.

Во второй функции RDI используется до того, как был инициализирован. Единственный логичный вывод состоит в том, что RDI должен был быть инициализирован в вызывающей стороне; в таком случае RDI используется для передачи информации из вызывающей стороны в функцию regargs (то есть является параметром). В терминах анализа программ RDI является живым на входе в regargs. Чтобы определить число параметров на базе регистров, которых ожидает функция, идентифицируйте все регистры, которые, по-видимому, живы внутри функции: их содержимое читается и используется до того, как регистр был записан (инициализирован) внутри функции.

К сожалению, такой анализ потока данных обычно выходит за пределы возможностей большинства дизассемблеров, включая Ghidra. Декомпиляторы, с другой стороны, должны выполнять такой тип анализа и в целом довольно хорошо идентифицируют использование параметров на базе регистров. Анализатор Ghidra Decompiler Parameter ID (Edit > Options for <prog> > Properties > Analyzers) может обновить дизассемблированный листинг на основе анализа параметров, выполненного декомпилятором.

Представление редактора стека дает подробный взгляд на внутреннюю работу компиляторов. На рисунке 6-10 ясно, что компилятор вставил 8 дополнительных байт между сохраненным указателем

кадра `-0x4` и локальной переменной x (`local_10`). Эти байты занимают смещения от `-0x5` до `-0xc` в стековом кадре. Если вы сами не являетесь автором компилятора или не готовы глубоко погрузиться в исходный код GNU `gcc`, вам остается только строить предположения о том, почему эти дополнительные байты выделены именно так. В большинстве случаев мы можем отнести лишние байты к заполнению для выравнивания, и обычно присутствие этих дополнительных байтов не влияет на поведение программы. В главе 8 мы вернемся к представлению редактора стека и его использованию при работе с более сложными типами данных, такими как массивы и структуры.

Поиск

Как было показано в начале главы, Ghidra упрощает навигацию по дизассемблированию, чтобы находить известные вам артефакты и обнаруживать новые. Она также проектирует многие свои представления данных так, чтобы они сводили конкретные типы информации (имена, строки, импорты и т. д.), благодаря чему их тоже легко находить. Однако эффективный анализ дизассемблированного листинга часто требует возможности искать новые подсказки, которые помогут анализу дизассемблирования. К счастью для нас, в Ghidra есть меню Search, позволяющее выполнять поиски для нахождения интересующих элементов. Стандартные пункты меню поиска показаны на рисунке 6-11. В этом разделе мы исследуем способы поиска по дизассемблированию с использованием как текстового поиска, так и поиска байтов, предоставляемых `CodeBrowser`.

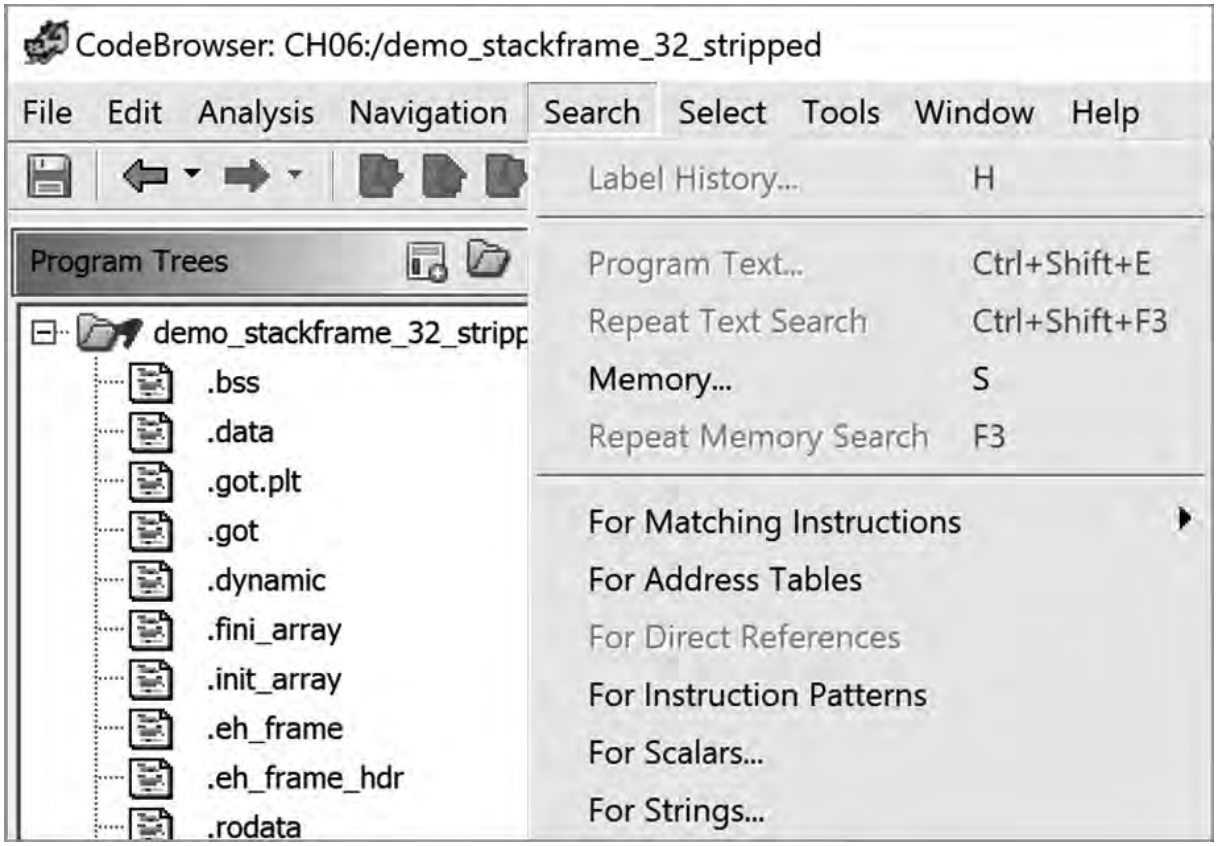


Рисунок 6-11. Пункты меню Search в Ghidra

Поиск текста программы

Текстовые поиски Ghidra фактически являются поисками подстрок в представлении дизассемблированного листинга. Текстовые поиски запускаются через Search > Program Text, что открывает диалог, показанный на рисунке 6-12. Доступны два типа поиска: вся база данных программы, которая простирается за пределы того, что вы видите в окне `CodeBrowser`, и отображение листинга внутри `CodeBrowser`. Помимо типа поиска, несколько самоочевидных параметров позволяют выбрать, как и что искать.

Чтобы перемещаться между совпадениями, используйте кнопки Next и Previous в нижней части диалога Search Program Text или выберите Search All, чтобы открыть результаты поиска в новом окне, позволяющем легко перейти к любому совпадению.

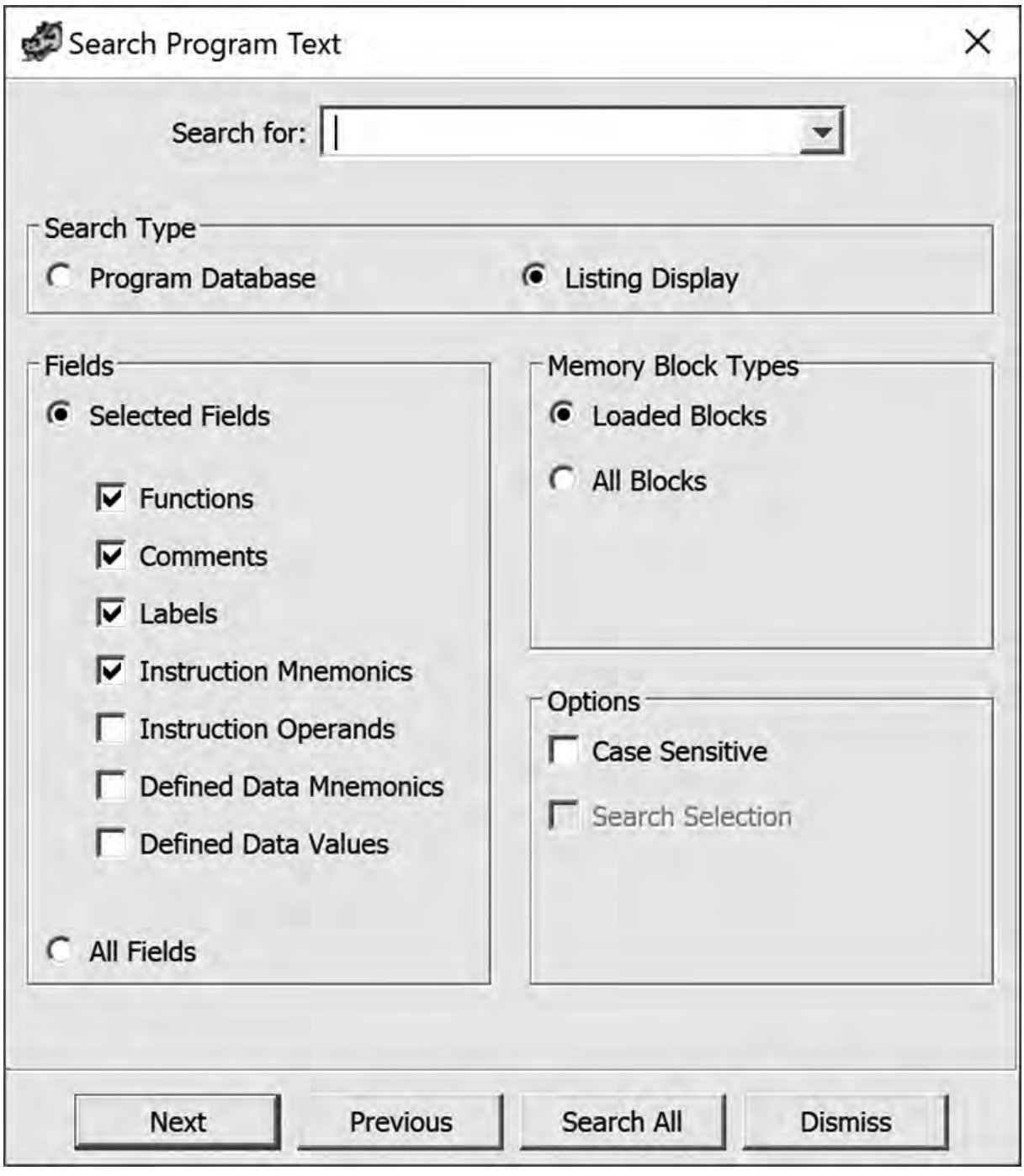


Рисунок 6-12. Диалог Search Program Text

Поиск в памяти

Если вам нужно искать конкретное бинарное содержимое, например известную последовательность байтов, то текстовые поиски не являются ответом. Вместо этого нужно использовать функциональность поиска в памяти Ghidra. Поиск в памяти можно запустить через `Search > Memory` или связанной горячей клавишей `S`. На рисунке 6-13 показан диалог `Search Memory`. Чтобы найти последовательность шестнадцатеричных байтов, строку поиска следует задать как разделенный пробелами список двухзначных, не зависящих от регистра шестнадцатеричных значений, например `c9 c3`, как показано на рисунке 6-13. Если вы не уверены в шестнадцатеричной последовательности, можно использовать подстановочные символы (`*` или `?`).

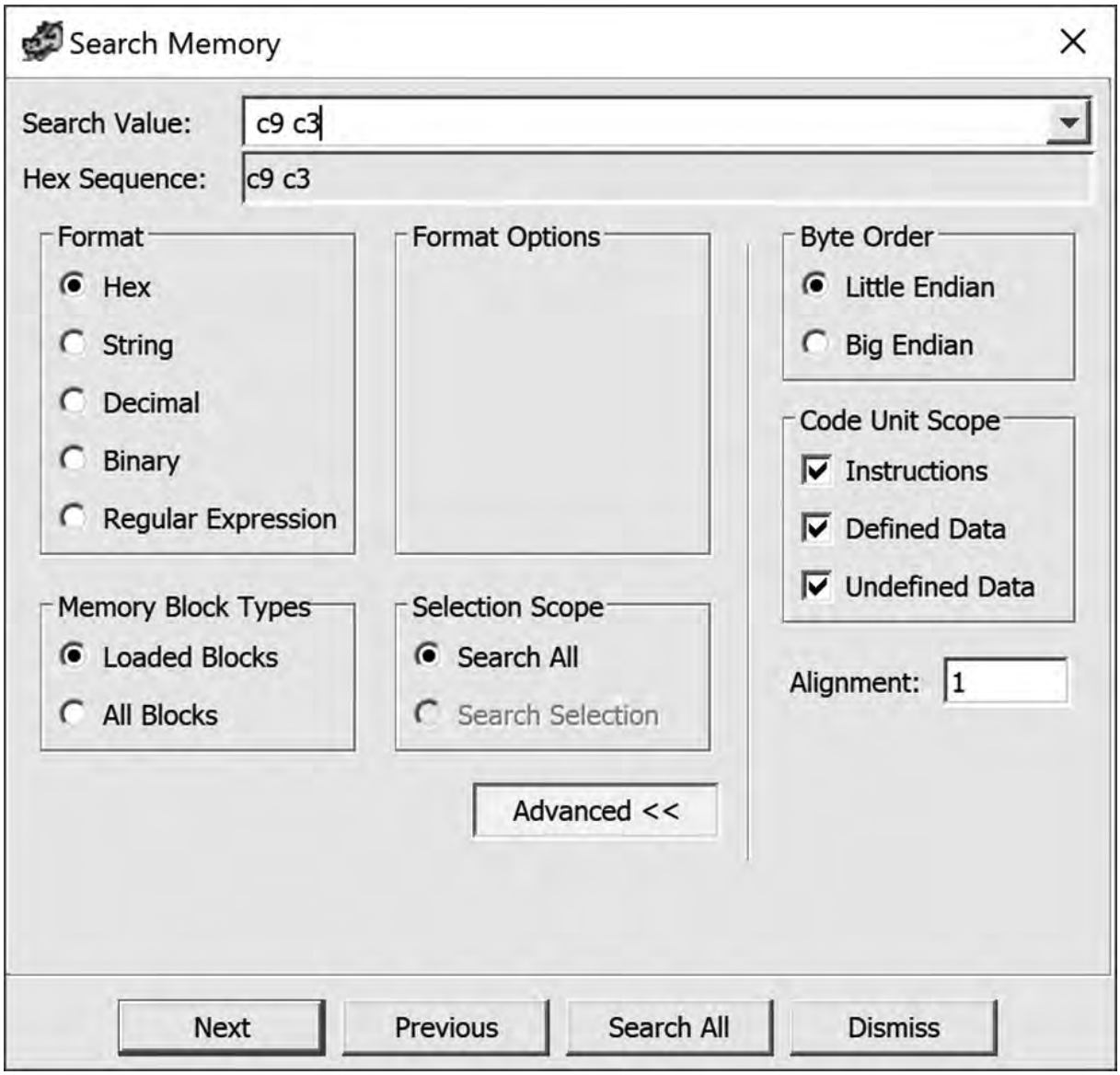


Рисунок 6-13. Диалог Search Memory

Нарекаю тебя...

Окна поиска относятся к тем типам окон в Ghidra, которые можно переименовывать как угодно; это поможет отслеживать окна поиска во время экспериментов. Чтобы переименовать окно, просто щелкните правой кнопкой мыши по строке заголовка и задайте

имя, имеющее для вас смысл. Удобный прием - включить в имя строку поиска вместе с мнемоникой, которая поможет вспомнить выбранные настройки.

Результаты Search Memory для байтов `c9 c3`, запущенного с параметром Search All, показаны на рисунке 6-14. Вы можете сортировать по любому столбцу, переименовать окно или применить фильтр. Это окно также предлагает некоторые параметры по щелчку правой кнопкой мыши, включая возможность удалять строки и манипулировать выделениями.

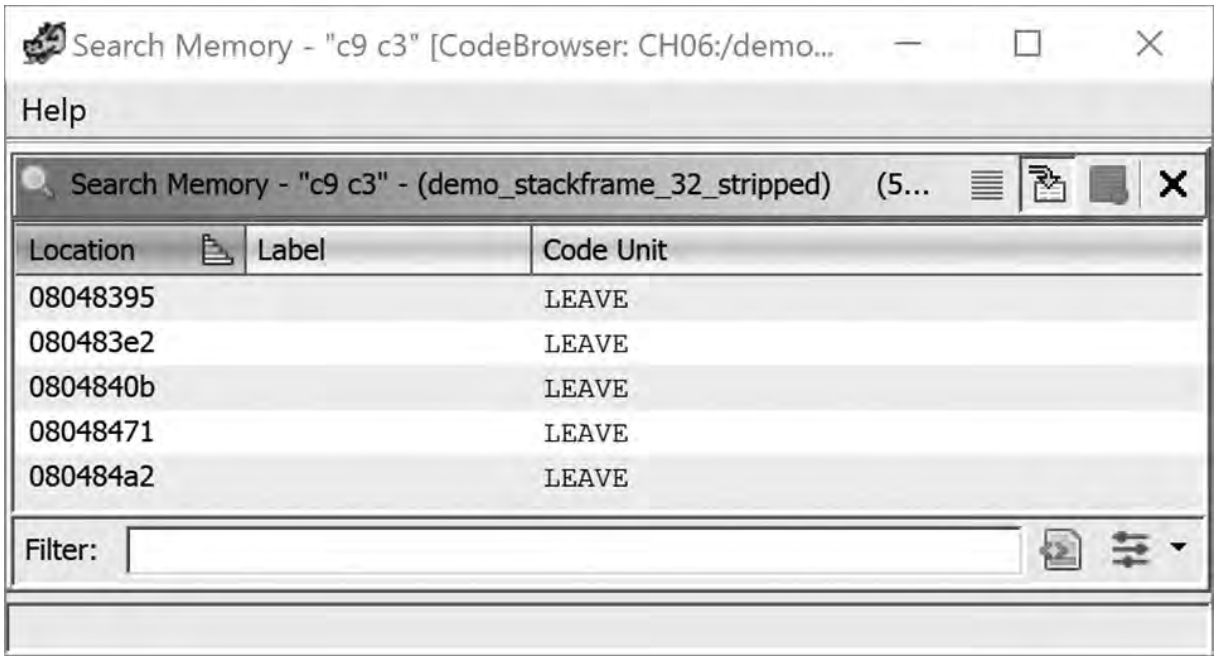


Рисунок 6-14. Результаты Search Memory

Искомые значения также можно вводить в строковом, десятичном, двоичном форматах и в формате регулярных выражений. Строковый, десятичный и двоичный форматы предоставляют уместные для своего контекста параметры формата. Регулярные выражения позволяют искать конкретный шаблон, но только в направлении вперед из-за ограничений их обработки. Ghidra использует встроенную грамматику регулярных выражений Java, подробно описанную в `Ghidra Help`.

Резюме

Цель этой главы состояла в том, чтобы дать вам минимально необходимые навыки для эффективной интерпретации дизассемблированных листингов Ghidra и навигации по ним. Подавляющее большинство ваших взаимодействий с Ghidra будет включать операции, которые мы уже обсудили. Однако способность выполнять базовую навигацию, понимать важные конструкции дизассемблирования вроде стека и искать по дизассемблированию - лишь верхушка айсберга для специалиста по обратной разработке.

Когда эти навыки надежно освоены, логичный следующий шаг - научиться использовать Ghidra под ваши конкретные нужды. В следующей главе мы начнем рассматривать, как вносить самые базовые изменения в дизассемблированный листинг, чтобы добавлять новые знания, основанные на нашем понимании содержимого и поведения бинарного файла.

Сноски

[1] [назад] Рекурсия возникает, когда функция прямо или косвенно вызывает саму себя. Каждый раз, когда функция рекурсивно вызывает себя, она создает новый стековый кадр. Если нет четко определенного условия остановки или если условие остановки не достигается за разумное число рекурсивных вызовов, неконтролируемая рекурсия может поглотить все доступное стековое пространство и аварийно завершить программу.
[2] [назад] Указатель кадра - это регистр, указывающий на место внутри стекового кадра. Переменные внутри стекового кадра обычно адресуются по относительному расстоянию от места, на которое указывает указатель кадра.
[3] [назад] Класс C++ может определять два типа функций-членов: статические и нестатические. Нестатические функции-члены используются для изменения атрибутов конкретных объектов и поэтому должны иметь некоторый способ точно знать, с каким объектом они работают (указатель `this`). Статические функции-члены принадлежат классу в целом и используются для изменения атрибутов, общих для всех экземпляров класса. Они не требуют и не получают указатель `this`.
[4] [назад] См. https://wiki.osdev.org/System_V_ABI .

Глава 7. Манипулирование дизассемблированием

После навигации манипулирование дизассемблированием является следующей по значимости возможностью Ghidra. Ghidra позволяет легко изменять дизассемблированные представления, чтобы добавлять новую информацию или переформатировать листинг под ваши конкретные нужды, а благодаря базовой структуре Ghidra изменения, которые вы вносите в дизассемблирование, легко распространяются на все связанные представления Ghidra, поддерживая согласованную картину вашей программы. Ghidra автоматически обрабатывает операции вроде контекстно-зависимого поиска и замены, когда это уместно, и превращает переформатирование инструкций в данные, а данных - в инструкции, в простую работу. И, пожалуй, лучшая возможность состоит в том, что почти все сделанное можно отменить!

Манипулирование именами и метками

К этому моменту мы встретили две категории идентификаторов в дизассемблированных представлениях Ghidra: метки (идентификаторы, связанные с местоположениями) и имена (идентификаторы, связанные с переменными стекового кадра). В основном мы будем называть и то и другое именами, поскольку Ghidra тоже проводит это различие несколько свободно. (Если быть совсем точным, у меток на самом деле есть связанные имена, адреса, истории и т. д. Имя метки - это то, как мы обычно ссылаемся на метку.) Более конкретные термины мы используем тогда, когда различие имеет критическое значение.

Напомним: имена стековых переменных имеют один из двух префиксов в зависимости от того, является ли переменная параметром (`param_`) или локальной переменной (`local_`), а местоположения получают имена/метки с полезными префиксами во время автоматического анализа (например, `LAB_`, `DAT_`, `FUN_`, `EXT_`, `OFF_` и `UNK_`). В большинстве случаев Ghidra автоматически генерирует имена и метки на основе своей лучшей догадки об использовании связанной переменной или адреса, но вам нужно самостоятельно анализировать программу, чтобы понять назначение местоположения или переменной.

Когда вы начинаете анализировать любую программу, один из первых и самых распространенных способов манипулировать дизассемблированным листингом - заменить стандартные имена более осмысленными. К счастью, Ghidra позволяет легко изменить любое имя и разумно распространяет изменения имени по всей программе. Чтобы открыть диалог изменения имени, выберите имя щелчком мыши, а затем используйте горячую клавишу `L` или пункт Edit Label в контекстном меню по щелчку правой кнопкой мыши. Далее процесс для стековых переменных (имен) и именованных местоположений (меток) различается, как подробно описано в следующих разделах.

Как жаль, что я это сделал

Часть мастерства в обратной разработке программного обеспечения - умение исследовать, экспериментировать и, когда нужно, откатываться назад и повторять путь заново. Мощная возможность Undo в Ghidra дает вам гибкость для отмены (и повторения) действий в процессе SRE. Доступно несколько способов получить эту волшебную силу: соответствующие значки со стрелками на панели инструментов `CodeBrowser`, как показано на рисунке 7-1; пункт Edit > Undo в меню `CodeBrowser`; а также горячие клавиши `ctrl-Z` для отмены и `ctrl-shift-Z` для повтора.

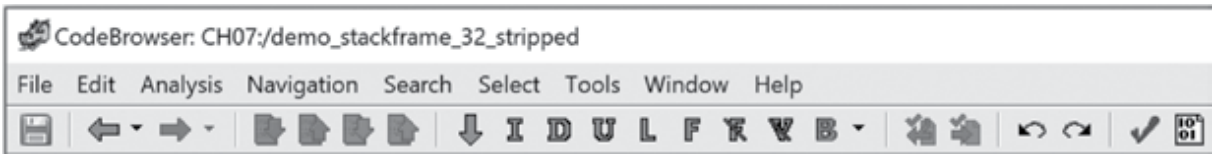


Рисунок 7-1. Значки Undo и Redo на панели инструментов `CodeBrowser`

Переименование параметров и локальных переменных

Имена, связанные со стековыми переменными, не связаны с конкретным виртуальным адресом. Как и в большинстве языков программирования, такие имена ограничены областью видимости функции, которой принадлежит данный стековый кадр. Поэтому каждая функция в программе может иметь собственную стековую переменную с именем `param_1`, но ни одна функция не может иметь более одной переменной с именем `param_1`, как показано на рисунке 7-2.

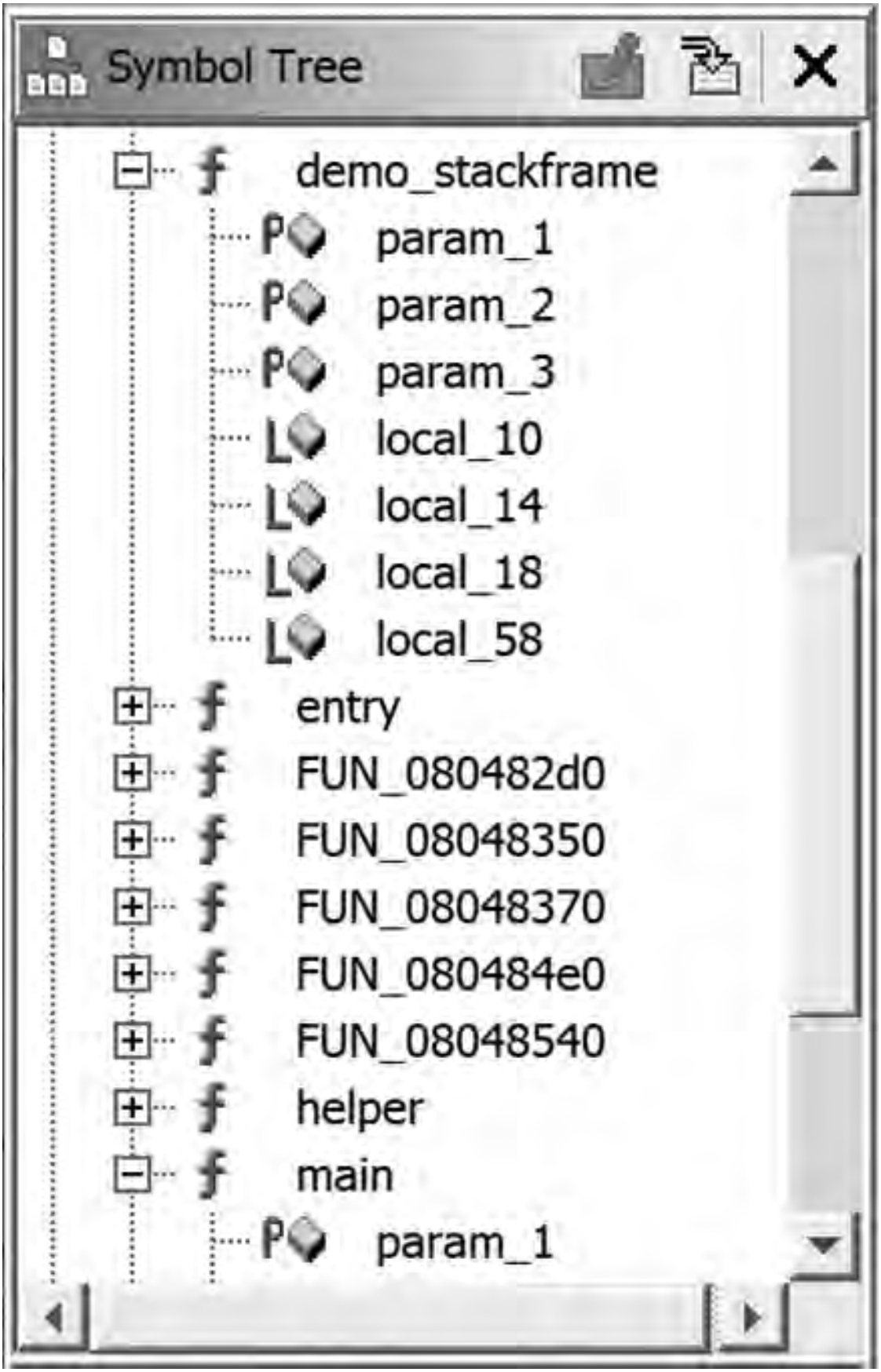


Рисунок 7-2. `Symbol Tree`*, показывающее повторное использование имен параметров (*`param_1`*)*

Когда вы переименовываете переменную в окне `Listing`, появляется информативный диалог, показанный на рисунке 7-3. Тип сущности, которую вы изменяете (переменная, функция и т. д.),

отображается в строке заголовка окна, а текущее имя (то, которое вот-вот будет изменено) отображается в редактируемом текстовом поле и в строке заголовка.

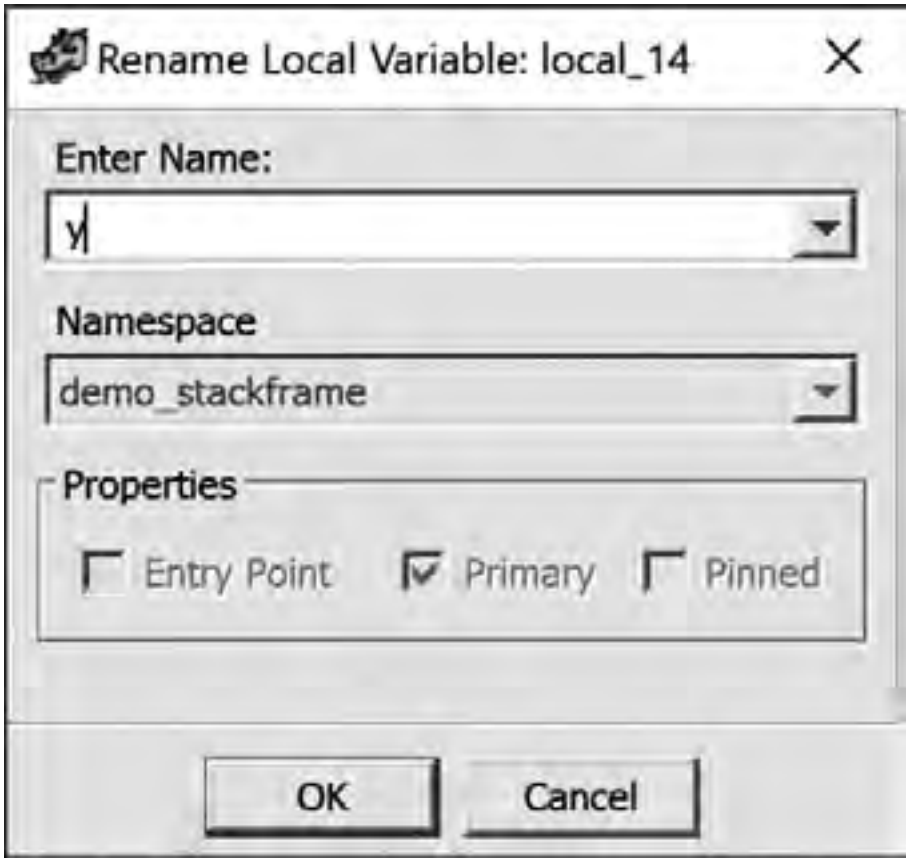


Рисунок 7-3. Переименование стековой переменной (`local_14` *в* `y`*)*

После указания нового имени Ghidra изменяет каждое вхождение старого имени в текущей функции. Следующий листинг показывает результат переименования `local_14` в `y` в `demo_stackframe`:

```
      ****************************************************************
      *                          FUNCTION                            *
      ****************************************************************
      undefined demo_stackframe(undefined param_1, undefined4
         undefined     AL:1              <RETURN>
         undefined     Stack[0x4]:1      param_1
         undefined4    Stack[0x8]:4      param_2
         undefined4    Stack[0xc]:4      param_3
         undefined4    Stack[-0x10]:4    local_10
         undefined4    Stack[-0x14]:4    y
         undefined4    Stack[-0x18]:4    local_18
         undefined1    Stack[-0x58]:1    local_58
      demo_stackframe
08048473 55           PUSH   EBP
08048474 89 e5        MOV    EBP,ESP
08048476 83 ec 58     SUB    ESP,0x58
08048479 8b 45 10     MOV    EAX,dword ptr [EBP + param_3]
0804847c 89 45 f4     MOV    dword ptr [EBP + local_10],EAX
0804847f 8b 45 0c     MOV    EAX,dword ptr [EBP + param_2]
08048482 89 45 f0     MOV    dword ptr [EBP + y],EAX
08048485 c7 45 ec     MOV    dword ptr [EBP + local_18],0xa
         0a 00 00 00
0804848c c6 45 ac 41  MOV    byte ptr [EBP + local_58],0x41
08048490 83 ec 08     SUB    ESP,0x8
08048493 ff 75 f0     PUSH   dword ptr [EBP + y]
08048496 ff 75 ec     PUSH   dword ptr [EBP + local_18]
08048499 e8 88 ff     CALL   helper
         ff ff
```

```
0804849e 83 c4 10     ADD    ESP,0x10
080484a1 90           NOP
080484a2 c9           LEAVE
080484a3 c3           RET
```

Эти изменения также отражаются в `Symbol Tree`, как показано на рисунке 7-4.

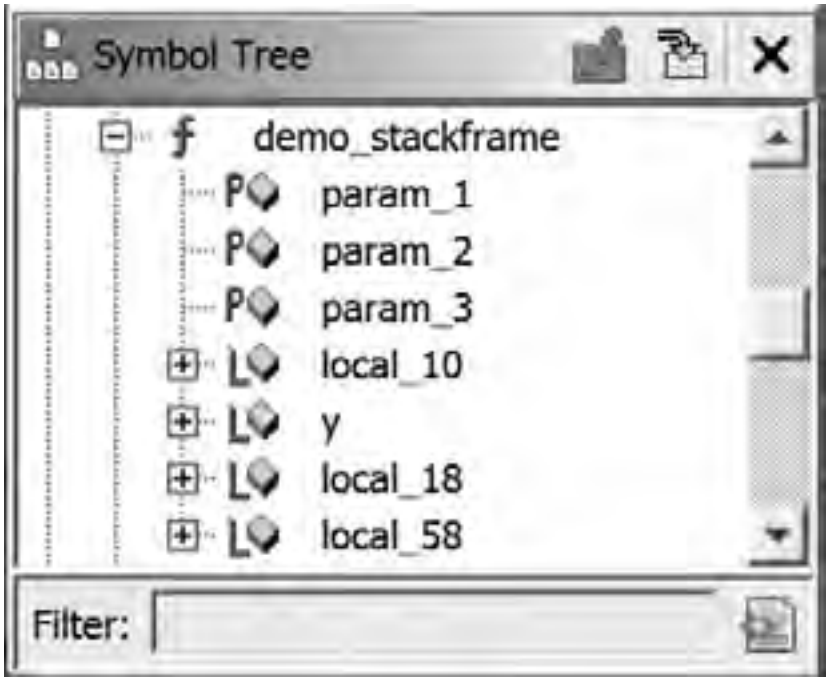


Рисунок 7-4. Представление `Symbol Tree` *для переименованной стековой переменной* `y`

Запрещенные имена

Некоторые интересные правила ограничивают то, как можно называть переменные внутри функции. Вот несколько наиболее важных правил для параметров:

- Нельзя использовать в имени префикс `param_`, за которым следует целое число, даже если получившееся имя не конфликтует с существующим именем параметра.
- Можно использовать префикс `param_`, за которым следуют другие символы.
- Можно использовать префикс `Param_`, за которым следует целое число, поскольку имена чувствительны к регистру (но это может быть не лучшей идеей).
- Можно восстановить имя параметра до исходного имени, назначенного Ghidra, введя `param_`, за которым следует целое значение. Если вы используете исходное целое значение, Ghidra вернет имя без возражений. Если вы используете любое целое значение, отличное от исходного, Ghidra предупредит: "Переименование не удалось - стандартные имена использовать нельзя". В этот момент щелчок Cancel в диалоге Rename Parameter восстановит исходное имя.
- Можно иметь два параметра с именами `param_1` (названный Ghidra) и `Param_1` (названный вами). Имена чувствительны к регистру, но повторно использовать их может быть нежелательно.

Локальные переменные также чувствительны к регистру, и можно использовать префикс `local_` с нечисловым суффиксом.

Для всех типов переменных нельзя использовать имя переменной, которое уже используется в этой области видимости (например, в той же функции). Такая попытка будет отклонена с указанием причины в диалоге.

Наконец, если вы окончательно запутались в своих метках, можно посмотреть историю меток для переменной, нажав горячую клавишу `H`, выбрав Show All History и введя текущее

имя (или прежнее имя) переменной в текстовое поле. (Этот вариант также доступен через Search > Label History в главном меню.)

Переименование меток

Метка - это стандартное или назначенное пользователем имя, связанное с местоположением. Как и для стековых переменных, диалог изменения имени открывается горячей клавишей `L` или контекстным пунктом Edit Label. Когда вы изменяете имя местоположения, вы также можете изменить его пространство имен и свойства, как показано на рисунке 7-6.

Где следует менять имя?

Имена переменных можно изменять из окон `Listing`, `Symbol Tree` и `Decompiler`; результат будет одинаковым независимо от способа, но диалог, открываемый из окна `Listing`, показывает больше информации. Все правила, связанные с именованием переменных, применяются при использовании любого из этих способов.

Многие примеры имен параметров в этой книге были изменены в окне `Listing` с помощью диалога, показанного слева на рисунке 7-5. Чтобы изменить имя в `Symbol Tree`, щелкните имя правой кнопкой мыши и выберите Rename в контекстном меню. В окне `Decompiler` используйте горячую клавишу `L` или контекстный пункт Rename Variable; соответствующий диалог показан справа на рисунке 7-5. Хотя эти два диалога предоставляют одну и ту же функциональность, правый диалог не включает сведения о пространстве имен или свойствах, связанных с параметром.

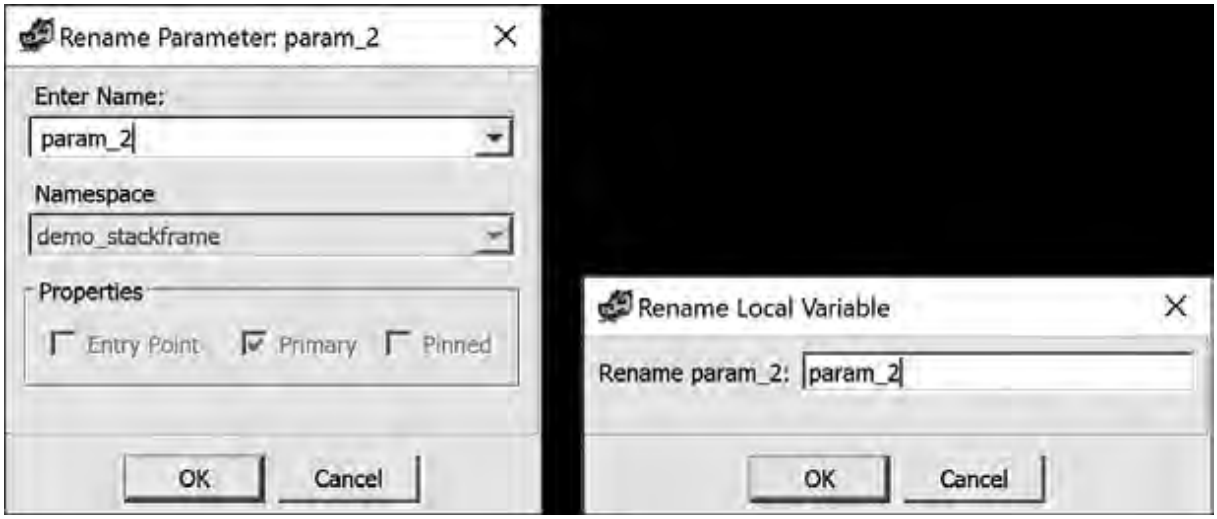


Рисунок 7-5. Переименование переменной из окна `Listing` *или* `Symbol Tree` *(слева) или из окна* `Decompiler` *(справа)*

В Ghidra пространство имен - это просто именованная область видимости. Внутри пространства имен все символы уникальны. Глобальное пространство имен содержит все символы внутри бинарного файла. Пространства имен функций вложены в глобальное пространство имен. Внутри пространства имен функции все имена переменных и метки уникальны. Сами функции могут содержать вложенные пространства имен, например пространство имен, связанное с оператором `switch` (что позволяет повторно использовать метки `case` в отдельных пространствах имен; например, когда функция содержит два оператора `switch`, в каждом из которых есть `case 10`).

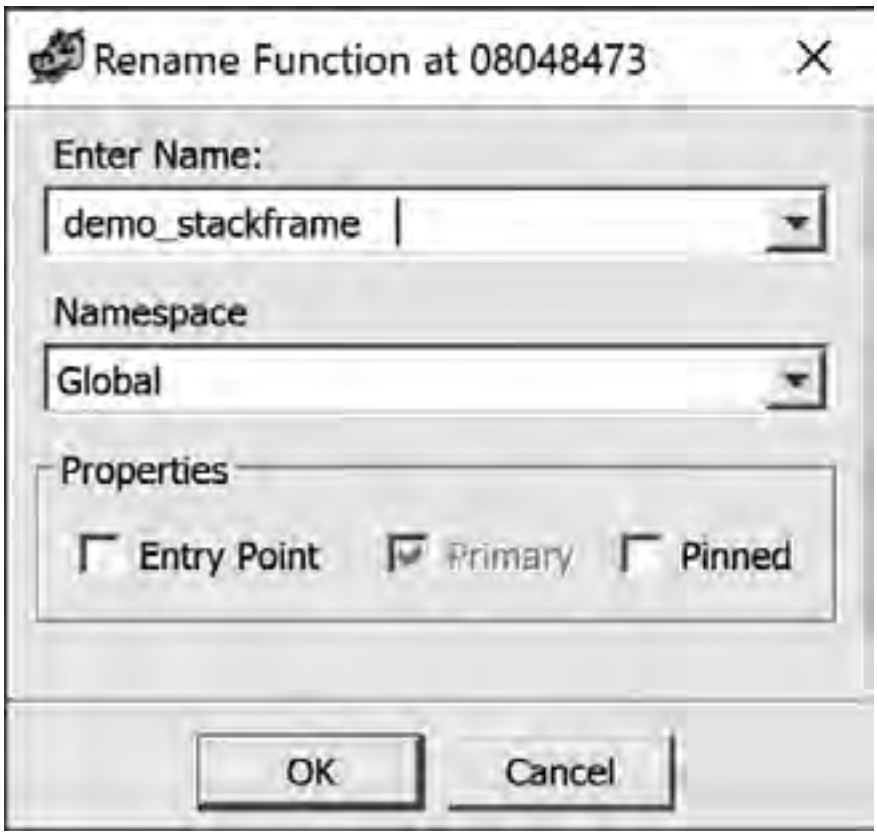


Рисунок 7-6. Переименование функции

Этот расширенный диалог показывает тип сущности и виртуальный адрес местоположения в строке заголовка. В разделе Properties можно определить адрес как точку входа или закрепить адрес (см. "Редактирование меток" на странице 126). Как упоминалось в главе 6, Ghidra ограничивает имена максимум 2000 символами, поэтому смело используйте осмысленные имена или даже встраивайте повествование об адресе (без пробелов). Окно `Listing` покажет только часть имени, если длина чрезмерна, но окно `Decompiler` показывает его целиком.

Добавление новой метки

Хотя Ghidra генерирует много стандартных меток, вы также можете добавлять новые метки и связывать их с любым адресом в листинге. Их можно использовать для аннотирования дизассемблирования, хотя во многих случаях комментарии (обсуждаются далее в этой главе) являются более подходящим механизмом. Чтобы добавить новую метку, откройте диалог Add Label (горячая клавиша `L`), показанный на рисунке 7-7, для адреса, связанного с текущим положением курсора. Раскрывающийся список для имени включает список имен, которые вы использовали недавно, а раскрывающийся список Namespace позволяет выбрать подходящую область видимости метки.

Рисунок 7-7. Диалог Add Label

Вы можете столкнуться с конфликтами, если попытаетесь использовать один из зарезервированных префиксов Ghidra при вводе имени. Если вы настаиваете на использовании зарезервированного префикса, Ghidra отклонит новую метку, если сочтет, что может возникнуть конфликт имен. Это происходит только тогда, когда Ghidra определяет, что ваш суффикс похож на адрес (по нашему опыту, это означает четыре или более шестнадцатеричных цифры). Например, Ghidra разрешит `FUN_zone` и `FUN_123`, но отклонит `FUN_12345`. Кроме того, если вы попытаетесь добавить метку по тому же адресу, что и функция со стандартной меткой (например, `FUN_08048473`), Ghidra переименует функцию, а не добавит вторую метку в этом местоположении.

Редактирование меток

Чтобы отредактировать метку, используйте горячую клавишу `L` или контекстный пункт Edit Label. При редактировании метки отображается тот же диалог, что и при добавлении метки, за исключением того, что поля диалога будут инициализированы текущими значениями существующей метки. Обратите внимание, что редактирование меток может повлиять на другие метки, которые имеют тот же адрес, независимо от того, используют ли они то же пространство имен. Например, если вы определите метку как точку входа, Ghidra определит все метки, связанные с этим местоположением, как точки входа.

Префиксы с FUN_

Когда Ghidra создает метки во время автоматического анализа, она использует осмысленные префиксы, за которыми следует адрес, чтобы сообщить вам, чего ожидать в этом местоположении. Эти префиксы перечислены ниже с очень общими описаниями. Дополнительные сведения о значении каждого префикса можно найти в `Ghidra Help`.

`LAB_address` Код - автоматически сгенерированная метка (обычно цель перехода внутри функции)

`DAT_address` Данные - автоматически сгенерированное имя глобальной переменной

`FUN_address` Функция - автоматически сгенерированное имя функции

`SUB_address` Цель вызова или эквивалентной операции - вероятно, не функция

`EXT_address` Внешняя точка входа - вероятно, чужая функция

`OFF_address` Отщепленный участок внутри существующих данных или кода - вероятно, ошибка дизассемблирования

`UNK_address` Неизвестно - назначение данных здесь невозможно определить

С метками функций связаны следующие конкретные варианты поведения:

- Если удалить стандартную метку функции (например, `FUN_08048473`) в окне `Listing`, префикс `FUN_` будет заменен префиксом `SUB_` (в этом случае получится `SUB_08048473`).
- Добавление новой метки к адресу, у которого есть стандартная метка `FUN_`, изменяет имя функции, а не создает новую метку.
- Метки чувствительны к регистру, поэтому можно использовать `Fun_` или `fun_` как допустимый префикс, если ваше желание - создать запутанное дизассемблирование.

Флажок Primary на рисунке 7-7 указывает, что именно эта метка будет отображаться при отображении адреса. По умолчанию этот флажок отключен для основной метки, поэтому снять выбор с основного имени нельзя. Это необходимо, чтобы всегда было имя для отображения. Если бы в качестве основной была выбрана другая метка, ее флажок был бы отключен, а флажки для других меток по тому же адресу были бы включены.

Хотя до сих пор мы связывали метки с адресами, в действительности метки чаще всего связаны с содержимым, которое просто имеет адрес. Например, метка `main` обычно обозначает начало блока кода, являющегося главной функцией программы. Ghidra назначает адрес этому местоположению на основе информации из заголовка файла. Если бы мы переместили все содержимое бинарного файла в новый диапазон адресов, мы ожидали бы, что метка `main` продолжит правильно связываться с новым адресом `main` и соответствующим неизмененным байтовым содержимым. Когда метка закреплена (pinned), связь метки с содержимым по ее адресу разрывается. Если после этого переместить содержимое бинарного файла в новый диапазон адресов, любые закрепленные метки не переместятся вместе с ним, а останутся зафиксированными на адресе, к которому вы их закрепили. Самое распространенное применение закрепленных меток - именование векторов сброса и областей ввода-вывода, отображенных в память, которые существуют по конкретным адресам, назначенным разработчиками процессора/системы.

Удаление метки

Чтобы удалить метку в позиции курсора, можно использовать контекстный пункт по щелчку правой кнопкой мыши (или горячую клавишу `delete`). Имейте в виду, что не все метки можно удалить. Во-первых, невозможно удалить стандартную метку, сгенерированную Ghidra. Во-вторых, если вы переименовали стандартную метку, а позже решили удалить новую метку, Ghidra заменит удаляемое имя исходной назначенной стандартной меткой (это прямое следствие предыдущего утверждения). Более тонкие детали, связанные с удалением меток, обсуждаются в `Ghidra Help`.

Это баг или возможность?

Во время экспериментов с именами функций вы можете заметить, что Ghidra совершенно спокойно позволяет дать двум функциям одно и то же имя. Это может вызвать

воспоминания о перегруженных функциях, которые можно различать по переданным им параметрам. Возможность Ghidra идет дальше: вы можете дать двум функциям совершенно одинаковое имя, даже если это приводит к дублирующимся прототипам функций внутри одного пространства имен. Это возможно потому, что метка не является уникальным идентификатором (первичным ключом в смысле базы данных) и поэтому не идентифицирует функцию однозначно, даже если рассматривать ее вместе со связанными параметрами. Дублирующиеся имена можно использовать для пометки функций; например, чтобы классифицировать их для дальнейшего анализа или исключить из рассмотрения. Помните, что все имена сохраняются в истории функции (горячая клавиша `H`) и могут быть легко возвращены.

Навигация по меткам

Метки связаны с местоположениями, к которым можно перейти, поэтому двойной щелчок по ссылке на метку переместит вас к этой метке. Хотя это подробнее обсуждается в главе 9, помните, что вы можете добавлять метки к любому местоположению, к которому хотите переходить в дизассемблировании. Хотя такая же функциональность описана в разделе "Аннотации" на странице 132, иногда метка (особенно с ее лимитом в 2000 символов) является самым быстрым способом достичь той же цели.

Комментарии

Встраивание комментариев в дизассемблированные листинги и листинги декомпилятора - особенно полезный способ оставлять себе заметки о вашем прогрессе и находках во время анализа программы. Ghidra предлагает пять категорий комментариев, каждая из которых подходит для своей цели. Начнем с комментариев, которые можно добавлять непосредственно в дизассемблирование в окне `Listing`.

Хотя к диалогу Set Comment (показанному на рисунке 7-8) можно перейти через контекстное меню по щелчку правой кнопкой мыши, самый быстрый способ - использовать горячую клавишу для комментариев, клавишу точки с запятой (`;`). (Это логичный выбор, поскольку точка с запятой является индикатором комментария во многих вариантах ассемблера.)

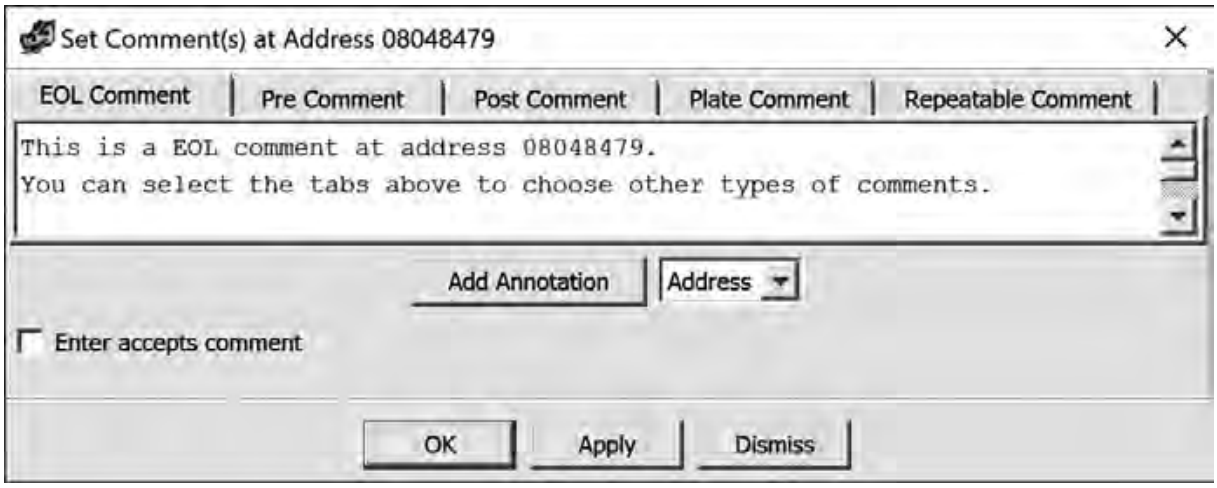


Рисунок 7-8. Диалог Set Comment

Диалог Set Comment открывается в привязке к конкретному адресу: 08048479 на рисунке 7-8, как показано в строке заголовка. Любое содержимое, введенное на одной из пяти вкладок категорий комментариев (EOL, Pre, Post, Plate и Repeatable Comments), связывается с этим адресом.

По умолчанию вы вводите содержимое в текстовое поле, включая переводы строки, создаете комментарий длиной в одну или несколько строк, а затем щелкаете Apply или OK. (Apply позволяет увидеть комментарий в контексте и оставляет диалог Set Comment открытым для дальнейшего редактирования.) Чтобы сэкономить время при вводе коротких комментариев, установите флажок Enter accepts comment в нижнем левом углу диалога. (Вы всегда можете временно снять этот флажок, если пишете особенно информативный блочный комментарий.)

Чтобы удалить комментарий, очистите текст комментария в диалоге Set Comment или используйте горячую клавишу delete, когда курсор находится на комментарии в окне Listing. Щелчок правой кнопкой мыши и пункт Comments -> Show History for Comment можно использовать, чтобы вызвать комментарии, связанные с конкретным адресом, и при необходимости восстановить их.

Комментарии в конце строки

Пожалуй, самый часто используемый тип комментария - комментарий в конце строки (EOL), размещаемый в конце существующих строк окна Listing. Чтобы добавить такой комментарий, откройте диалог Set Comment горячей клавишей точки с запятой и выберите вкладку EOL Comment. По умолчанию EOL-комментарии отображаются синим текстом и будут занимать несколько строк, если вы введете несколько строк в текстовое поле комментария. Каждая строка будет иметь отступ так, чтобы выровняться по правой стороне дизассемблирования, а существующее содержимое будет сдвинуто вниз, чтобы освободить место для новых комментариев. Вы можете редактировать свои комментарии в любое время, снова открыв диалог Set Comment. Самый быстрый способ удалить комментарий - щелкнуть комментарий в окне Listing и нажать delete.

Сама Ghidra добавляет много EOL-комментариев во время автоматического анализа. Например, когда вы загружаете PE-файл, Ghidra вставляет описательные EOL-комментарии для описания полей в разделе IMAGE_DOS_HEADER, включая комментарий "магическое число". Ghidra может сделать это только тогда, когда у нее есть эта информация, связанная с конкретным типом данных. Такая информация обычно содержится в библиотеках типов, которые отображаются в окне Data Type Manager и подробно обсуждаются в главе 8 и главе 13. Среди всех типов комментариев EOL-комментарии наиболее настраиваемы через параметры Edit > Tool Options > Listing Fields для каждого типа комментария.

> **Эти три кнопки**
>
> Из трех кнопок в нижней части диалога Set Comment (рисунок 7-8) кнопки OK и Apply ведут себя так, как вы ожидаете. Щелчок OK закрывает диалог и фиксирует изменения. При щелчке Apply листинг обновляется, чтобы вы могли изучить изменения и подтвердить их или продолжить редактирование комментария.
>
> Однако Dismiss - это не то же самое, что Cancel, который вышел бы из диалога без какого-либо влияния на ваш листинг! Уникальный термин соответствует уникальному поведению. Щелчок кнопки Dismiss немедленно закрывает окно, если вы не изменяли никаких комментариев, но позволяет решить, хотите ли вы сохранить изменения, если комментарии были изменены. Закрытие окна с помощью X в правом верхнем углу демонстрирует то же поведение. Эта функциональность Dismiss будет встречаться и в других местах Ghidra.

Предварительные и последующие комментарии

Предварительные и последующие комментарии - это полно-строчные комментарии, которые появляются либо непосредственно перед заданной строкой дизассемблирования, либо сразу после нее. Следующий листинг показывает многострочный предварительный комментарий и усеченный однострочный последующий комментарий, связанные с адресом `08048476`. При наведении на усеченный комментарий отображается полный комментарий. По умолчанию предварительные комментарии отображаются фиолетовым, а последующие - синим, чтобы их было легко связать с правильным адресом в листинге.

```
08048473  PUSH    EBP
08048474  MOV     EBP,ESP
        ******** Предварительный комментарий - это многострочный комментарий.
        ******** Следующая инструкция выделяет 88 байт пространства
        ******** локальных переменных в стековом кадре.
08048476  SUB     ESP,0x58
        ******** Последующий комментарий - теперь, когда мы выделили пространство.
    ..
08048479  MOV     EAX,dword ptr [EBP + param_3]
```

Plate-комментарии

Plate-комментарии позволяют группировать комментарии для отображения в любом месте окна `Listing`. Plate-комментарий центрируется и помещается внутри прямоугольника, ограниченного звездочками. Многие листинги, которые мы рассматривали, включают простой plate-комментарий со словом `FUNCTION` внутри рамки, как показано на рисунке 7-9. Этот пример включает связанное окно `Decompiler` справа, чтобы вы могли увидеть, что в стандартном представлении plate-комментарий вставлен в окно `Listing`, но соответствующего комментария в окне `Decompiler` нет.

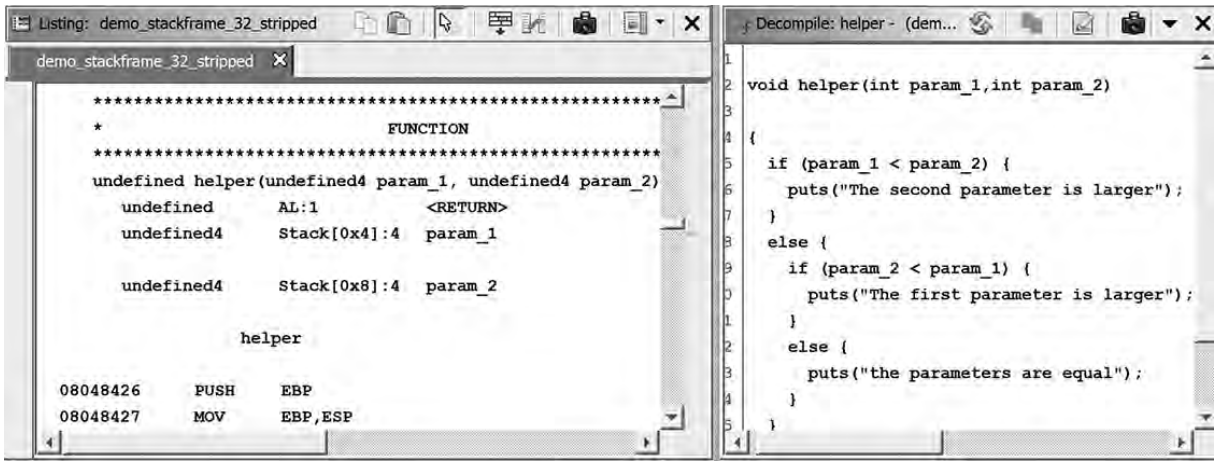


Рисунок 7-9. Пример plate-комментария

Когда вы открываете диалог комментария с выбранным первым адресом функции, у вас есть возможность заменить этот общий plate-комментарий собственным, более информативным, как показано на рисунке 7-10. Помимо замены стандартного plate-комментария, Ghidra добавляет ваш комментарий как C-стиль комментарий в верхнюю часть окна `Decompiler`. Если бы курсор находился в верхней части окна `Decompiler` при создании plate-комментария, результат был бы тем же.

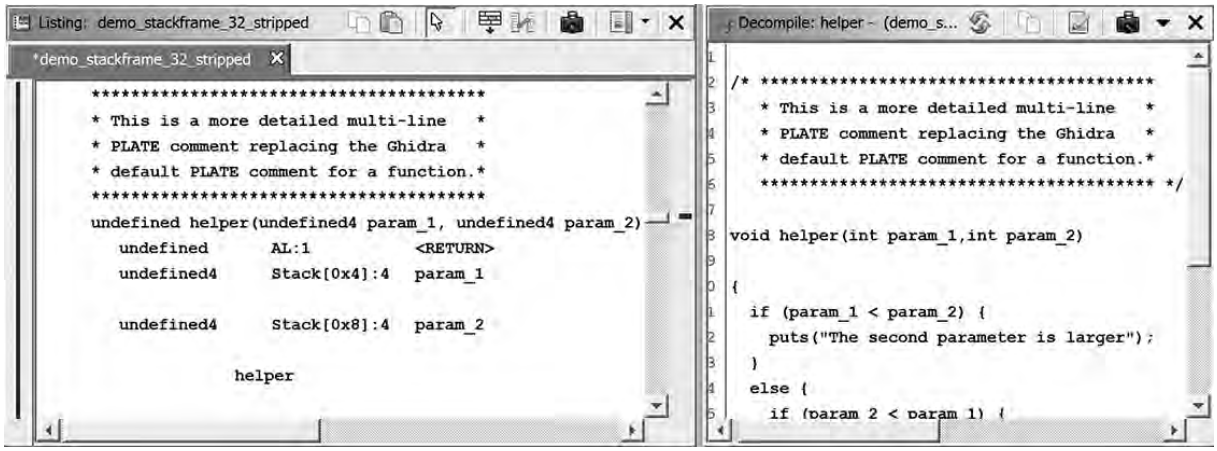


Рисунок 7-10. Пример пользовательского plate-комментария

Примечание

Только plate-комментарии и pre-комментарии по умолчанию отображаются в окне `Decompiler`, хотя это можно изменить с помощью параметров `Edit > Tool Options > Decompiler > Display`.

Повторяемые комментарии

Повторяемый комментарий вводится один раз, но может автоматически появляться во многих местах дизассемблирования. Поведение повторяемых комментариев связано с понятием перекрестных ссылок, которые подробно обсуждаются в главе 9. В основном повторяемый комментарий, введенный в цели перекрестной ссылки, отражается в источнике перекрестной ссылки. В результате один повторяемый комментарий может быть отражен во многих местах дизассемблирования (поскольку перекрестные ссылки могут быть отношением многие-к-одному). В дизассемблированном листинге стандартный цвет для повторяемых комментариев - оранжевый, а для отраженных комментариев - серый, благодаря чему их легко отличить от других типов комментариев. Следующий листинг демонстрирует использование повторяемого комментария.

```
08048432  JGE    LAB_08048446                  Повторяемый комментарий по адресу
    08048446
08048434  SUB    ESP,0xc
08048437  PUSH   s_The_second_parameter_is_larger
0804843c  CALL   puts
08048441  ADD    ESP,0x10
08048444  JMP    LAB_08048470
        LAB_08048446
08048446  MOV    EAX,dword ptr [EBP + param_2] Повторяемый комментарий по адресу
    08048446
```

В листинге повторяемый комментарий установлен по адресу `08048446` и повторяется по адресу `08048432`, поскольку инструкция по адресу `08048432` ссылается на адрес `08048446` как на цель перехода (следовательно, существует перекрестная ссылка из `08048432` в `08048446`).

Когда EOL-комментарий и повторяемый комментарий имеют один и тот же адрес, в листинге виден только EOL-комментарий. Оба комментария можно просмотреть и отредактировать в диалоге Set Comment. Если удалить EOL-комментарий, повторяемый комментарий станет видимым в листинге.

Комментарии к параметрам и локальным переменным

Чтобы связать комментарий со стековой переменной, выберите стековую переменную и используйте горячую клавишу точки с запятой. На рисунке 7-11 показано получившееся минимальное окно комментария. Комментарий будет отображаться рядом со стековой переменной в усеченном формате, похожем на EOL-комментарий. При наведении на комментарий он отображается полностью. Цвет комментария соответствует стандартному цвету типа переменной, а не синему цвету EOL-комментариев по умолчанию.

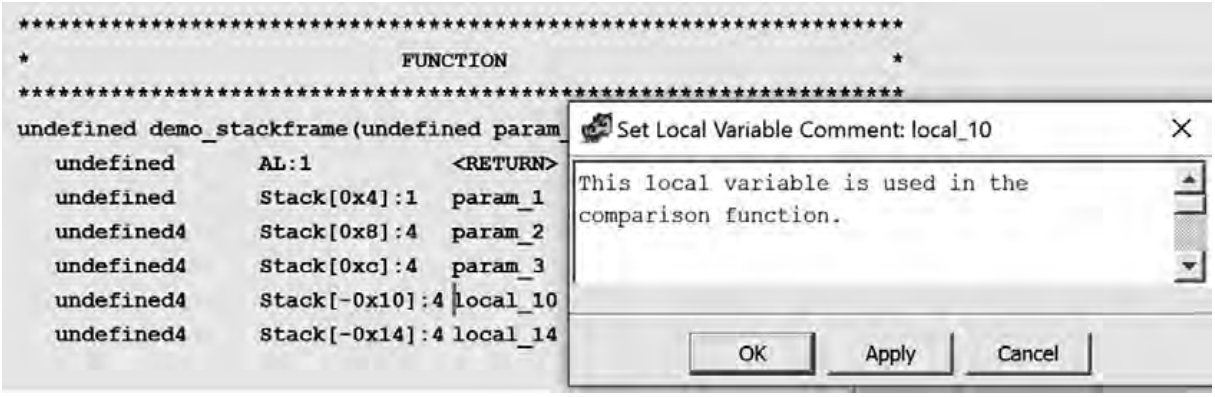


Рисунок 7-11. Комментарий стековой переменной

Аннотации

Ghidra предоставляет мощную возможность аннотировать комментарии ссылками на программы, URL-адреса, адреса и символы в диалоге Set Comment. Информация о символах в комментариях будет автоматически обновляться при изменении имен символов. Когда вы используете аннотацию для запуска указанного исполняемого файла, можно задать необязательные параметры, чтобы получить еще больше контроля (да, нам это тоже кажется опасным).

Например, аннотация в plate-комментарии на рисунке 7-12 предоставляет гиперссылку на адрес в листинге. Дополнительная информация о возможностях аннотаций приведена в `Ghidra Help`.

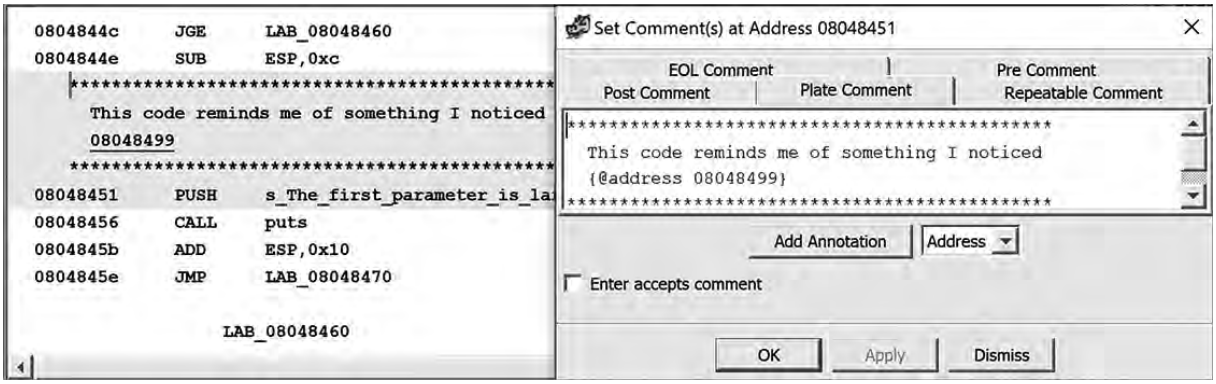


Рисунок 7-12. Пример аннотации адреса

Базовые преобразования кода

Во многих случаях вас полностью устроят дизассемблированные листинги, которые генерирует Ghidra. Однако в некоторых случаях это будет не так. По мере того как типы файлов, которые вы анализируете, все дальше отклоняются от обычных исполняемых файлов, созданных распространенными компиляторами, вам может потребоваться взять под больший контроль процессы анализа и отображения дизассемблирования. Это будет особенно верно, если вы анализируете обфусцированный код или файлы, использующие пользовательский формат файла (неизвестный Ghidra).

Ghidra облегчает следующие преобразования кода (среди прочих):

• Изменение параметров отображения кода
• Форматирование операндов инструкций
• Манипулирование функциями
• Преобразование данных в код
• Преобразование кода в данные

В целом, если бинарный файл очень сложен или если Ghidra не знакома с последовательностями кода, генерируемыми компилятором, использованным для построения бинарного файла, то Ghidra столкнется с большим числом проблем на этапе анализа, и вам потребуется вручную корректировать дизассемблированный код.

Изменение параметров отображения кода

Ghidra позволяет очень тонко управлять форматированием строк в окне `Listing`. Компоновка управляется с помощью Browser Field Formatter (представлен в главе 5). Выбор значка Browser Field Formatter открывает вкладочное отображение всех полей, связанных с вашим листингом, как показано на рисунке 5-8. Вы можете добавлять, удалять и переставлять поля с помощью простого интерфейса перетаскивания, который позволяет сразу наблюдать изменения в листинге. Тесная связь между элементом в поле листинга и связанным Browser Field Formatter очень полезна. Каждый раз, когда вы перемещаете курсор в новое место в окне `Listing`, Browser Field Formatter перемещает подходящую вкладку и связанное поле, чтобы вы могли сразу определить параметры, связанные с конкретным элементом. Дополнительное обсуждение Browser Field Formatter см. в разделе "Специальные возможности редактирования инструментов" на странице 247.

Чтобы управлять внешним видом отдельных элементов в окне `Listing`, можно выбрать `Edit > Tool Options`, как описано в главе 4. Уникальные подменю для каждого поля в окне `Listing` позволяют точно настроить каждое поле по вашему вкусу. Хотя возможности, связанные с каждым полем, различаются, в целом можно управлять цветами отображения, связанными стандартными значениями, конфигурациями и форматами. Например, пользователи, которые любят ассемблерный код и читают его в свободное время, могут настроить стандартные параметры в области `EOL Comments Field`, показанной на рисунке 7-13, чтобы включить параметр `Show Semicolon at Start of Each Line` и видеть ассемблерные комментарии в привычном формате.

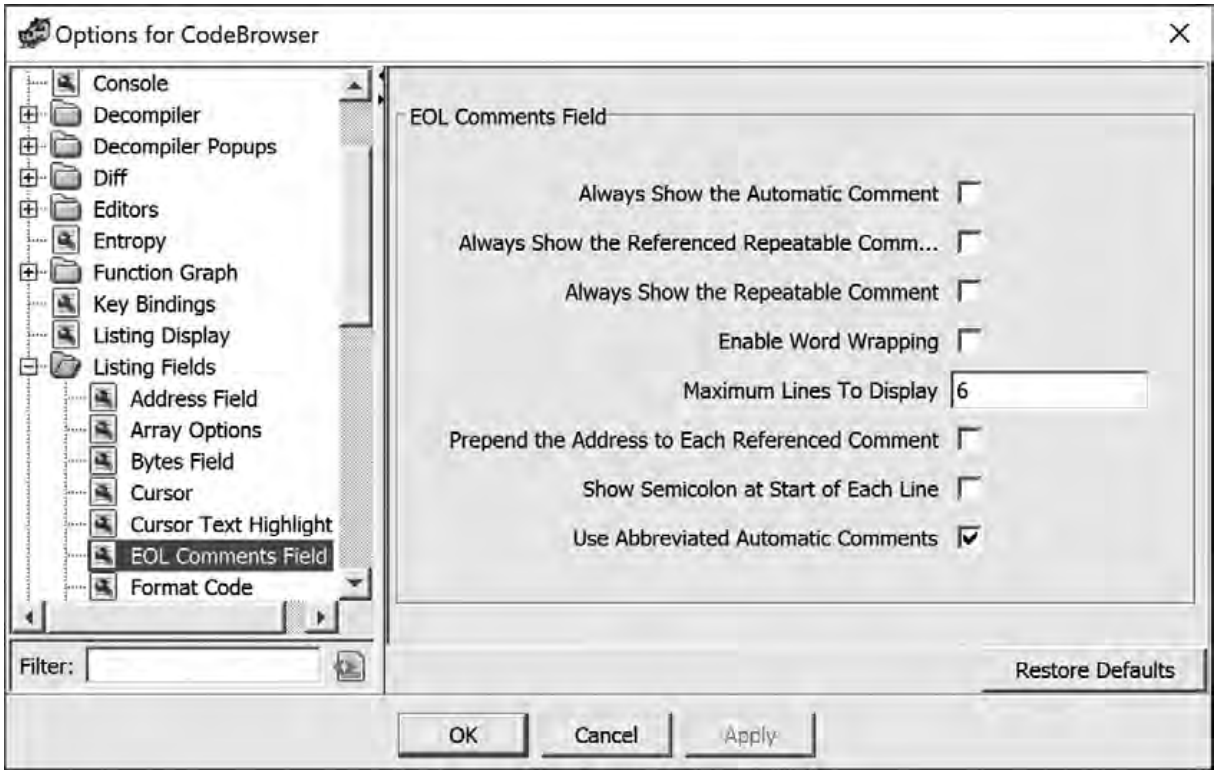


Рисунок 7-13. Меню `Tool Options` *для* `EOL Comments Field`

Чтобы раскрасить фон отдельных строк или более крупных выделений в окне `Listing`, выберите пункт Colors через контекстное меню по щелчку правой кнопкой мыши и выберите цвет. Диапазон доступных цветов широк, а для недавно использованных цветов предусмотрен быстрый выбор. Через то же меню можно также очистить цвет фона для строки, выделения или всего файла.

> **Примечание**
>
> Параметры очистки не появляются, если в листинге сейчас не заданы цвета.

Форматирование операндов инструкций

Во время процесса автоматического анализа Ghidra принимает множество решений о том, как форматировать операнды, связанные с каждой инструкцией, особенно разные целочисленные константы, используемые широким набором типов инструкций. Помимо прочего, эти константы могут представлять относительные смещения в инструкциях перехода или вызова, абсолютные адреса глобальных переменных, значения для использования в арифметических операциях или определенные программистом константы. Чтобы сделать дизассемблирование более читаемым, Ghidra старается использовать символические имена вместо чисел всякий раз, когда это возможно.

В некоторых случаях решения о форматировании принимаются на основе контекста дизассемблируемой инструкции (например, инструкции вызова); в других случаях решение основано на используемых данных (например, обращение к глобальной переменной или смещение внутри стекового кадра или структуры). Часто точный контекст, в котором используется константа, может быть неразличим для Ghidra. Когда это происходит, константа обычно форматируется как шестнадцатеричное значение.

Если вы не относитесь к тем немногим людям в мире, которые едят, спят и дышат шестнадцатеричными числами, то возможности форматирования операндов Ghidra вам понравятся. Предположим, что в дизассемблированном листинге у вас есть следующее:

```
08048485  MOV    dword ptr [EBP + local_18],0xa
0804848c  MOV    byte ptr [EBP + local_58],0x41
```

Щелчок правой кнопкой мыши по шестнадцатеричной константе `0x41` открывает контекстно-зависимое меню, показанное на рисунке 7-14. (См. рисунок 6-7 для этого примера в контексте.) Константу можно переформатировать в разные числовые представления, отображенные в правой части рисунка, или как символьную константу (поскольку это значение также попадает в печатаемый диапазон ASCII). Это может быть очень полезной возможностью, поскольку вы можете не осознавать, сколько представлений может быть связано с данной константой. Во всех случаях меню показывает точный текст, который заменит текст операнда, если выбрать конкретный вариант.

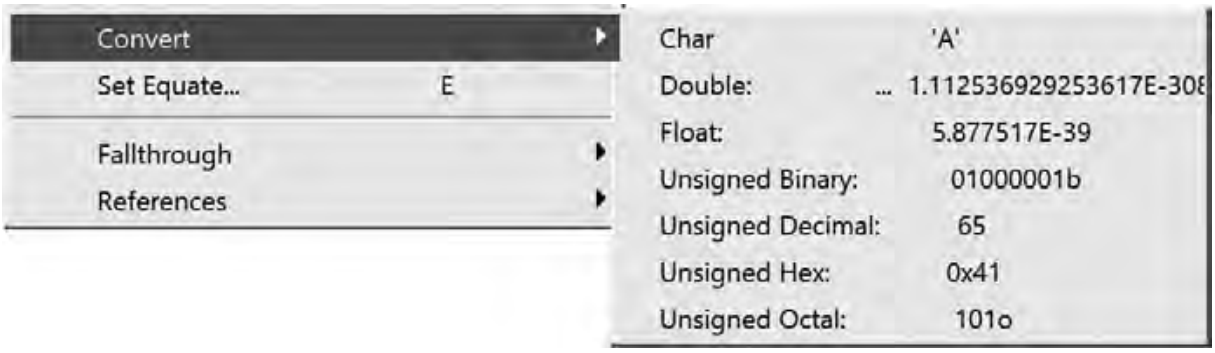


Рисунок 7-14. Параметры форматирования для констант

Во многих случаях программисты используют именованные константы в исходном коде. Такие константы могут быть результатом операторов `#define` (или их эквивалента) либо принадлежать набору перечислимых констант. К сожалению, к тому моменту, когда компилятор закончит работу с исходным кодом, уже невозможно определить, использовал ли исходный код символическую константу или литеральную числовую константу. К счастью, Ghidra поддерживает большой каталог именованных констант, связанных со многими распространенными библиотеками, такими как стандартная библиотека C или Windows API. Этот каталог доступен через пункт Set Equate (горячая клавиша `E`) в контекстно-зависимом меню, связанном с любым значением константы. Выбор этого пункта для константы `0xa` открывает диалог Set Equate (рисунок 7-15).

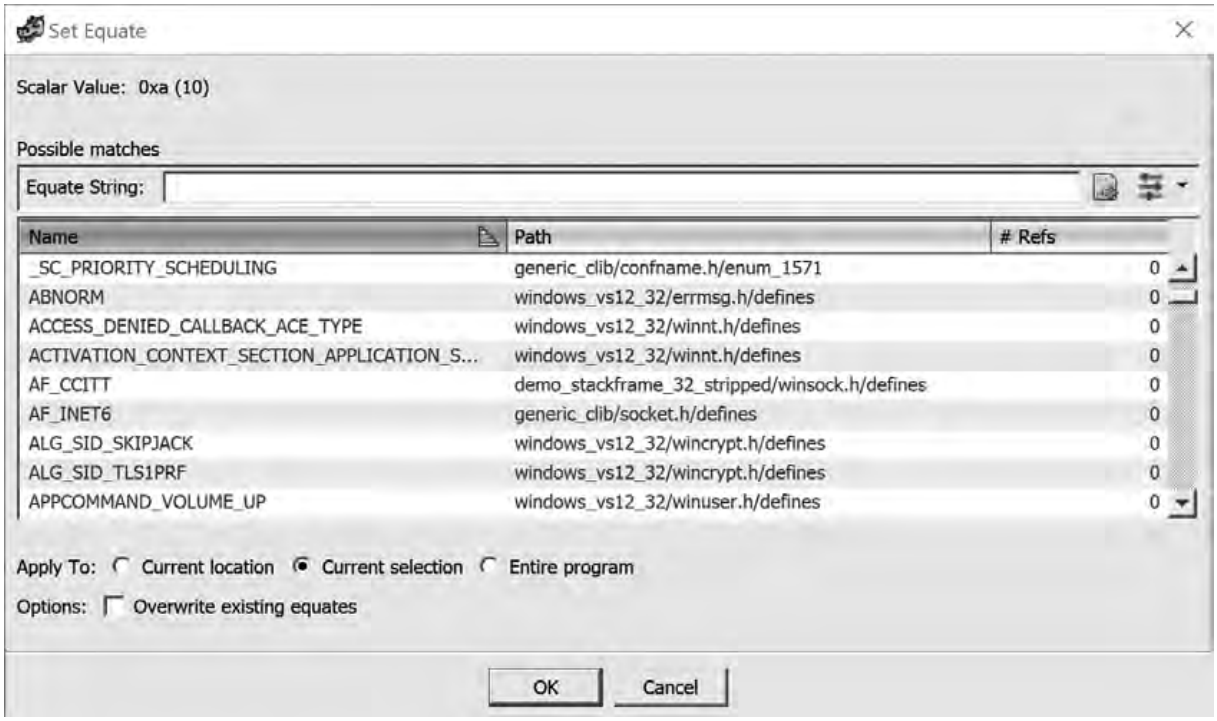


Рисунок 7-15. Диалог Set Equate

Диалог заполняется из внутреннего списка констант Ghidra после фильтрации по значению константы, которую мы пытаемся отформатировать. В этом случае мы можем прокрутить список, чтобы увидеть все константы, которые, как известно Ghidra, приравниваются к значению `0xA`. Если бы мы определили, что значение используется вместе с созданием сетевого соединения в стиле X.25, мы могли бы выбрать `AF_CCITT` и получить следующую строку дизассемблирования:

```
08048485  MOV    dword ptr [EBP + local_18],AF_CCITT
```

Список стандартных констант полезен для определения того, может ли конкретная константа быть связана с известным именем, и может сэкономить много времени при чтении документации API в поисках потенциальных совпадений.

Манипулирование функциями

Ghidra предоставляет возможность манипулировать функциями в дизассемблировании (например, исправлять то, какой код Ghidra считает принадлежащим функциям, или изменять атрибуты функций), что особенно полезно, когда вы не согласны с результатами автоматического анализа. В некоторых случаях, например когда Ghidra не удается обнаружить вызов функции, функции могут быть не распознаны, поскольку может не существовать очевидного способа добраться до них. В других случаях Ghidra может неправильно определить конец функции, и вам потребуется исправить дизассемблирование. Ghidra может испытывать трудности с поиском конца функции, если компилятор разбил функцию на несколько диапазонов адресов или если в процессе оптимизации кода компилятор объединил общие завершающие последовательности двух или более функций, чтобы сэкономить место.

Создание новых функций

Новые функции можно создавать из существующих инструкций, которые еще не принадлежат функции. Функции создаются щелчком правой кнопкой мыши по первой инструкции, которую нужно включить в новую функцию, и выбором Create Function (или горячей клавишей `F`). Если вы выбрали диапазон, он станет телом функции. Если нет, Ghidra проследует по потоку управления, чтобы попытаться определить границы тела функции.

Удаление функций

Существующие функции можно удалить, поместив курсор внутри сигнатуры функции и используя горячую клавишу `delete`. Вы можете захотеть удалить функцию, если считаете, что Ghidra ошиблась в автоматическом анализе или что вы ошиблись при создании функции. Обратите внимание: хотя функция и связанные с ней атрибуты перестанут существовать, базовое байтовое содержимое не изменится, поэтому при желании функцию можно создать заново.

Редактирование атрибутов функции

Ghidra связывает с каждой распознанной функцией несколько атрибутов, которые можно просмотреть, выбрав пункт `Window` > `Functions` в меню `CodeBrowser`. (Хотя по умолчанию отображается только пять атрибутов, можно добавить любой из 16 дополнительных атрибутов, щелкнув правой кнопкой мыши заголовок столбца.) Чтобы редактировать атрибуты, откройте диалог Edit Function из контекстного меню по щелчку правой кнопкой мыши, когда курсор находится в области между plate-комментарием функции и последней локальной переменной, перечисленной перед началом дизассемблированного кода функции. Пример диалога Edit Function показан на рисунке 7-16.

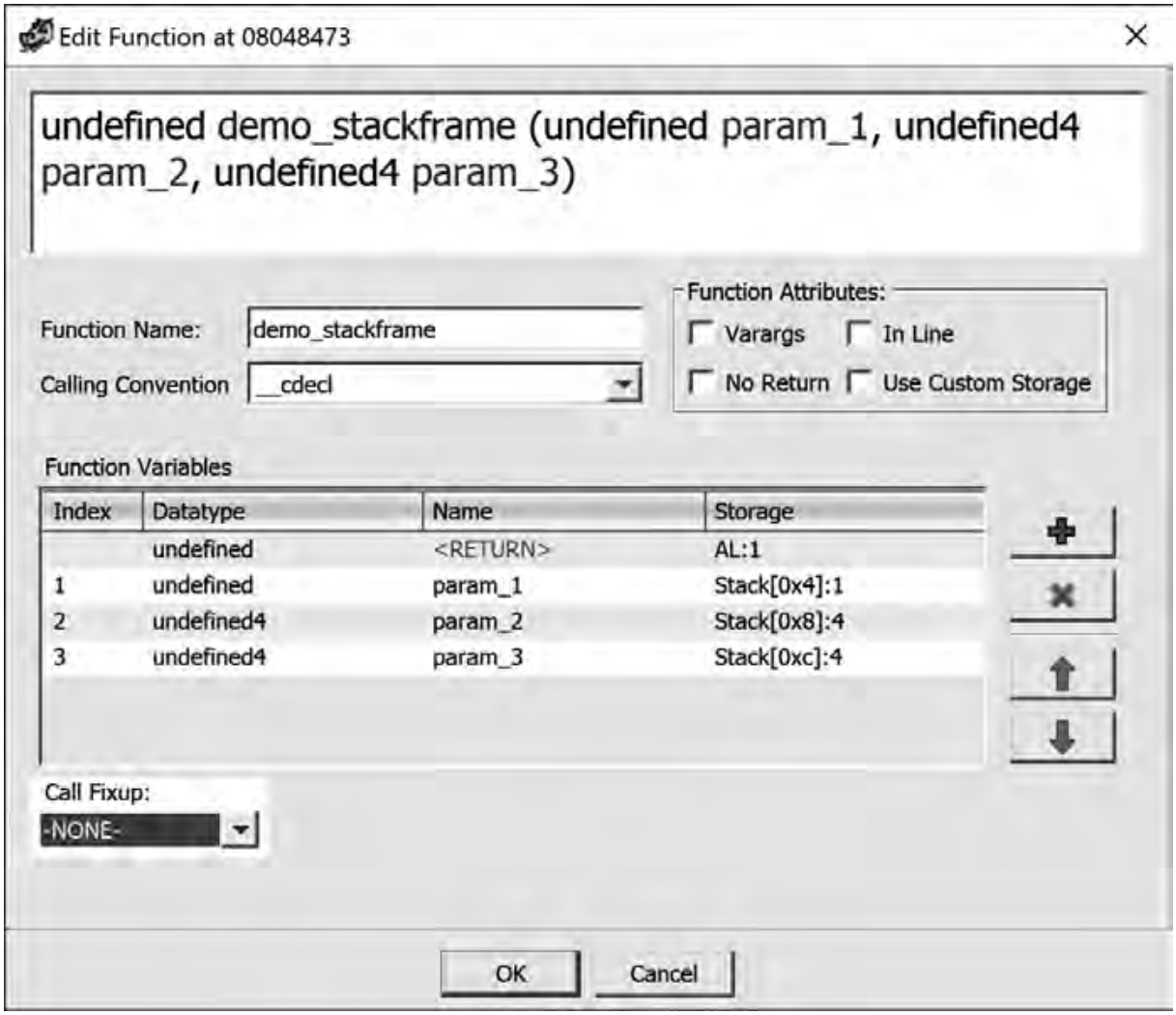


Рисунок 7-16. Диалог Edit Function

Каждый атрибут, который можно изменить через этот диалог, объясняется ниже:

Function Name

Имя можно изменить в текстовом поле в верхней части диалога или в поле Function Name.

Function Attributes

В этой области можно включить пять необязательных атрибутов функции. Первые четыре атрибута, Varargs, In Line, No Return и Use Custom Storage, являются флажками, которые по умолчанию сняты. Пятый необязательный атрибут, Call Fixup, находится в нижней левой части диалога, по умолчанию имеет значение none и предоставляет раскрывающееся меню, из которого можно выбрать значение. Если вы изменяете какой-либо из атрибутов функции, Ghidra автоматически распространяет обновленный прототип функции во все места, где он может отображаться в дизассемблировании.

Параметр Varargs указывает, что функция принимает переменное число аргументов (например, `printf`). Varargs также включается, если вы редактируете (в текстовом поле в верхней части рисунка 7-16) список параметров функции так, что последний аргумент имеет многоточие (`...`). Параметр In Line не влияет на анализ дизассемблирования, кроме включения ключевого слова `inline` в прототип функции. (Помните: если функция действительно была встроена компилятором, вы не увидите эту функцию как отдельную сущность в дизассемблировании, потому что ее тело было бы встроено в тело вызывающих ее функций.) Параметр No Return используется, когда известно, что функция никогда не возвращается (например, если она использует `exit` или

непрозрачный предикат для перехода к другой функции). Когда функция помечена как No Return, Ghidra не будет предполагать, что байты после вызова этой функции достижимы, если у нее нет других доказательств их достижимости, например инструкции перехода, нацеленной на эти байты. Параметр Use Custom Storage позволяет переопределить анализ Ghidra для местоположений и размеров хранения параметров и возвращаемого значения.

Calling Convention

Раскрывающийся список Calling Convention позволяет изменить соглашение о вызове, используемое функцией. Изменение соглашения о вызове может изменить анализ указателя стека Ghidra, поэтому важно указать его правильно.

Function Variables

Область Function Variables позволяет редактировать переменные функции с подсказками. По мере изменения данных в четырех столбцах, связанных с переменными, Ghidra будет предоставлять информацию, помогающую изменить вещи корректно. Например, попытки изменить Storage для `param_1` приведут к сообщению "Включите Use Custom Storage, чтобы разрешить редактирование хранилища параметров и возвращаемого значения". Четыре значка справа позволяют добавлять, удалять переменные и перемещаться по ним.

Преобразование данных в код (и наоборот)

Во время этапа автоматического анализа байты данных могут быть ошибочно классифицированы как байты кода и дизассемблированы в инструкции, или байты кода могут быть ошибочно классифицированы как байты данных и отформатированы как значения данных. Это происходит по многим причинам, в том числе потому, что некоторые компиляторы встраивают данные в секцию кода программ, а некоторые байты кода никогда напрямую не упоминаются как код, и поэтому Ghidra решает их не дизассемблировать. В частности, обфусцированные программы часто намеренно размывают различие между кодом и данными. (См. главу 21.)

Первый вариант для переформатирования чего-либо - удалить его текущее форматирование (код или данные). Можно сделать функции, код или данные неопределенными, щелкнув правой кнопкой мыши элемент, который нужно сделать неопределенным, и выбрав `Clear Code Bytes` (горячая клавиша `C`). Снятие определения с элемента приводит к тому, что лежащие в основе байты переформатируются как список сырых байтовых значений. Большие области можно сделать неопределенными, выбрав диапазон адресов операцией щелчка и перетаскивания перед выполнением операции снятия определения. В качестве примера рассмотрим этот простой листинг функции:

```
004013e0  PUSH   EBP
004013e1  MOV    EBP,ESP
004013e3  POP    EBP
004013e4  RET
```

Снятие определения с этой функции дало бы показанную здесь последовательность некатегоризованных байтов, которую можно переформатировать практически любым способом:

```
004013e0      ??       55h    U
004013e1      ??       89h
004013e2      ??       E5h
004013e3      ??       5Dh    ]
004013e4      ??       C3h
```

Чтобы дизассемблировать последовательность неопределенных байтов, щелкните правой кнопкой мыши первый байт, который нужно дизассемблировать, и выберите Disassemble. Это заставит Ghidra запустить алгоритм рекурсивного спуска с этой точки. Большие области можно преобразовывать в код, выбрав диапазон адресов щелчком и перетаскиванием перед выполнением операции преобразования в код.

Преобразование кода в данные немного сложнее. Во-первых, нельзя напрямую преобразовать код в данные с помощью контекстного меню, если сначала не снять определение с инструкций, которые нужно преобразовать в данные, а затем не отформатировать байты подходящим образом. Базовое форматирование данных обсуждается в следующем разделе.

Базовые преобразования данных

Чтобы понять поведение программы, правильно отформатированные данные могут быть так же важны, как правильно отформатированный код. Ghidra берет информацию из множества источников и использует алгоритмический подход, чтобы определить наиболее подходящий способ форматирования данных внутри дизассемблирования. Например:

- Типы и/или размеры данных могут быть выведены из того, как используются регистры. Инструкция, загружающая 32-разрядный регистр из памяти, подразумевает, что связанное место памяти содержит 4-байтовый тип данных (хотя мы можем не суметь отличить 4-байтовое целое от 4-байтового указателя).
- Прототипы функций можно использовать для назначения типов данных параметрам функций. Ghidra поддерживает большую библиотеку прототипов функций именно для этой цели. Анализ выполняется над параметрами, переданными функциям, в попытке связать параметр с местоположением памяти. Если такую связь удается обнаружить, тип данных можно применить к связанному местоположению памяти. Рассмотрим функцию, единственный параметр которой является указателем на `CRITICAL_SECTION` (тип данных Windows API). Если Ghidra может определить адрес, переданный при вызове этой функции, этот адрес может быть помечен как объект `CRITICAL_SECTION`.
- Анализ последовательности байтов может выявить вероятные типы данных. Именно это происходит, когда бинарный файл сканируется на строковое содержимое. Когда встречаются длинные последовательности ASCII-символов, разумно предположить, что они представляют символьные массивы.

В следующих нескольких разделах мы обсудим некоторые базовые преобразования, которые можно выполнять над данными внутри ваших дизассемблированных представлений.

Задание типов данных

Ghidra предлагает спецификаторы размера и типа данных. Чаще всего встречаются спецификаторы byte, word, dword и qword, представляющие соответственно 1-, 2-, 4- и 8-байтовые данные. Типы данных можно задавать или изменять, щелкнув правой кнопкой мыши любую строку дизассемблирования, содержащую данные (а не инструкцию), и выбрав подменю Set Data Type, показанное на рисунке 7-17.

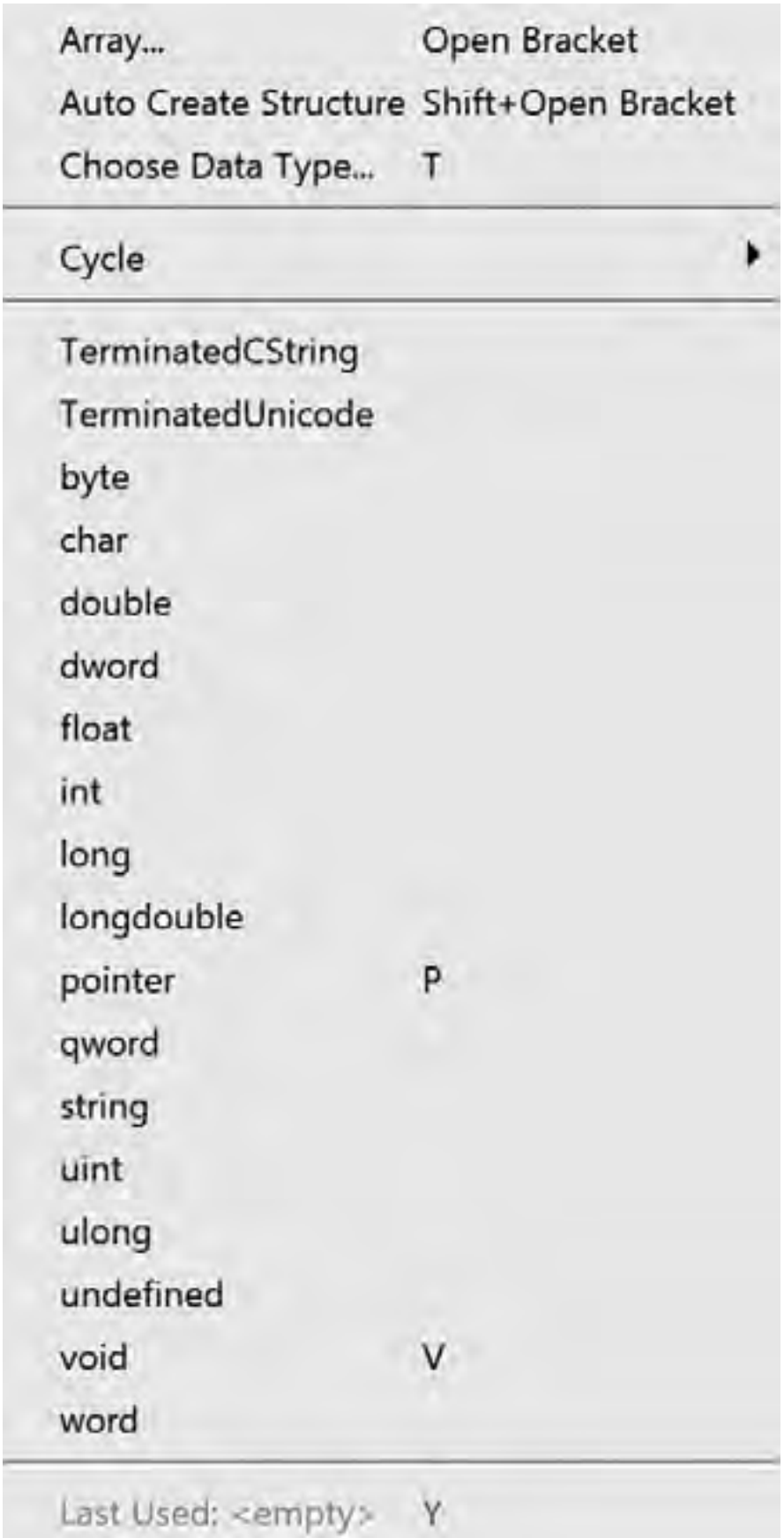


Рисунок 7-17. Подменю Data

Этот список позволяет сразу изменить форматирование и размер данных текущего выбранного элемента, выбрав тип данных. Пункт Cycle позволяет быстро циклически переключаться по группе

связанных типов данных, например числовых, символьных и типов с плавающей точкой, как показано (со связанными горячими клавишами) на рисунке 7-18. Например, повторное нажатие `F` будет переключать вас между `float` и `double`, поскольку это единственные элементы в этой циклической группе.

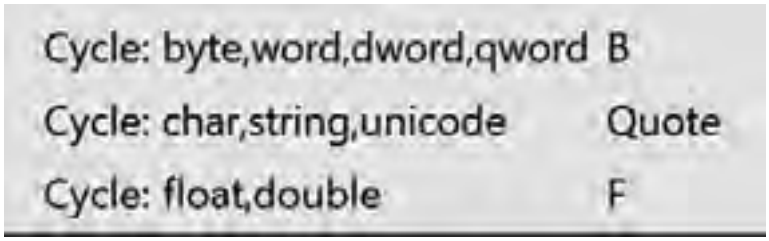


Рисунок 7-18. Группы циклического переключения

Переключение между типами данных приводит к тому, что элементы данных растут, уменьшаются или остаются того же размера. Если размер элемента остается прежним, единственное наблюдаемое изменение состоит в способе форматирования данных. Если вы уменьшаете размер элемента, например с `ddw` (4 байта) до `db` (1 байт), любые лишние байты (в этом случае 3) становятся неопределенными. Если вы увеличиваете размер элемента, Ghidra предупредит о любом конфликте и проведет вас через его разрешение. Пример с заданием размерности массива показан на рисунке 7-19.

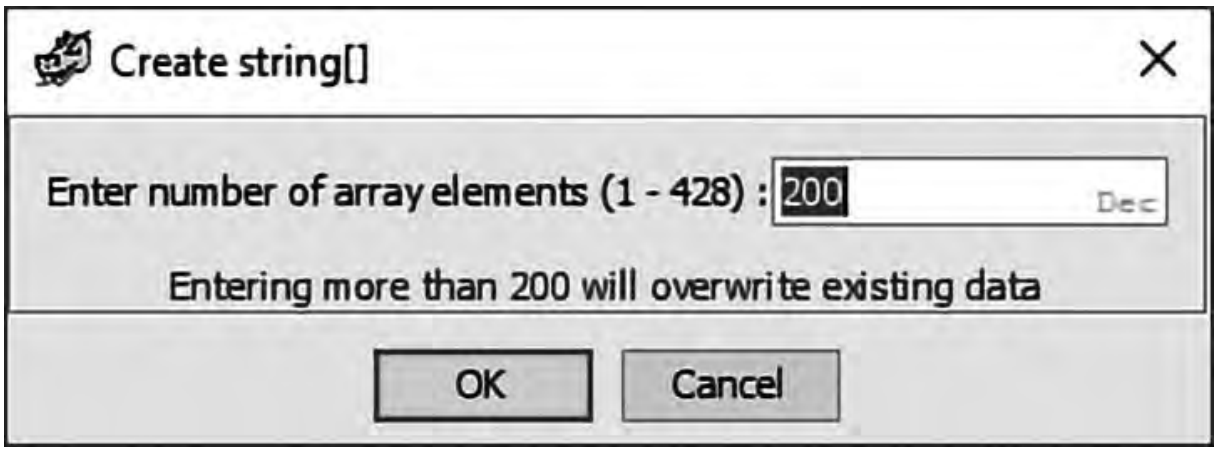


Рисунок 7-19. Пример объявления массива и предупреждения

Работа со строками

Выбор Search > For Strings вызывает диалог, показанный на рисунке 7-20, где можно задать и контролировать критерии поиска для конкретного поиска строк. Хотя большинство полей в этом окне не требуют пояснений, уникальная возможность Ghidra - способность связать с поиском модель слов. Модель слов можно использовать, чтобы определить, считается ли конкретная строка словом в данном контексте. Модели слов обсуждаются в главе 13.

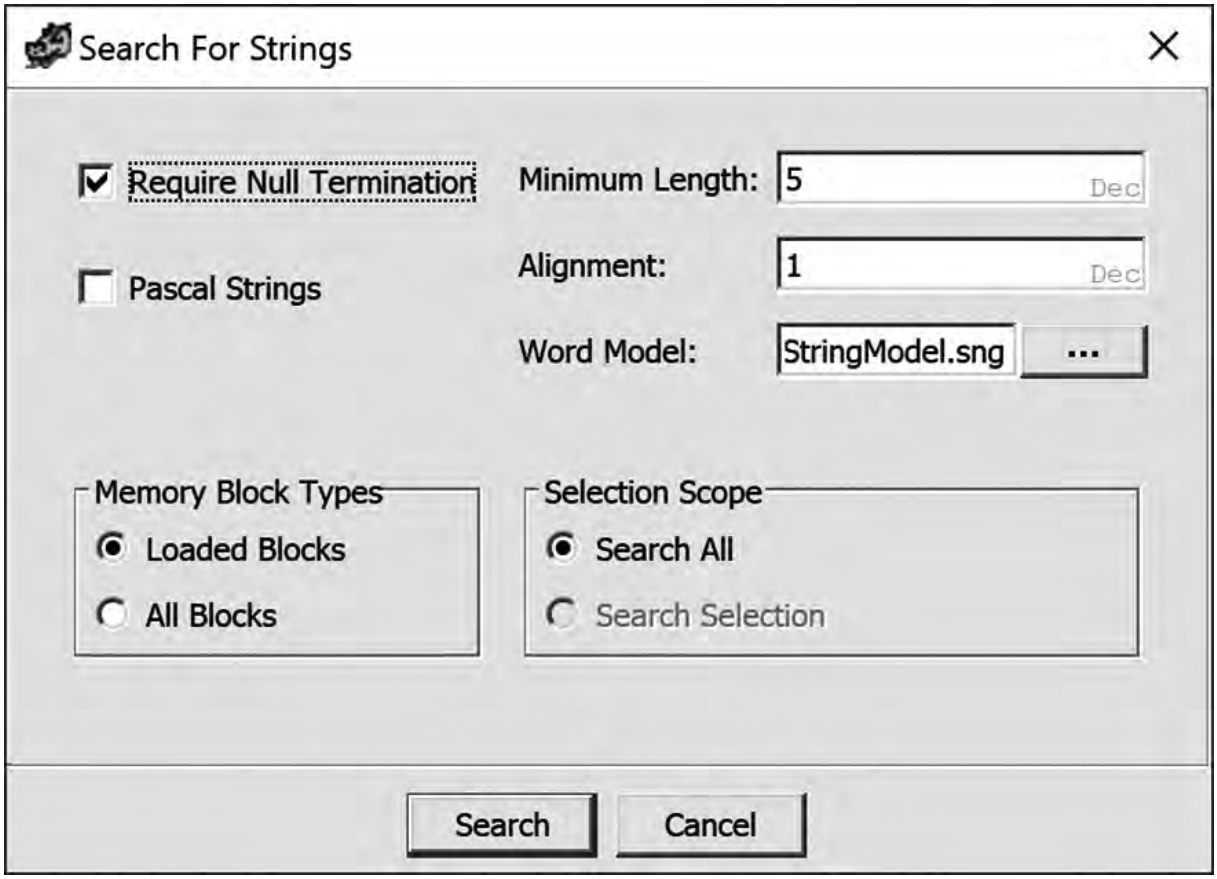


Рисунок 7-20. Диалог Search For Strings

После выполнения поиска результаты отображаются в окне String Search (рисунок 7-21). Последующие поиски будут открываться во вкладках внутри того же окна, а строка заголовка окна будет включать временные метки для каждого поиска, чтобы их было легко упорядочить.

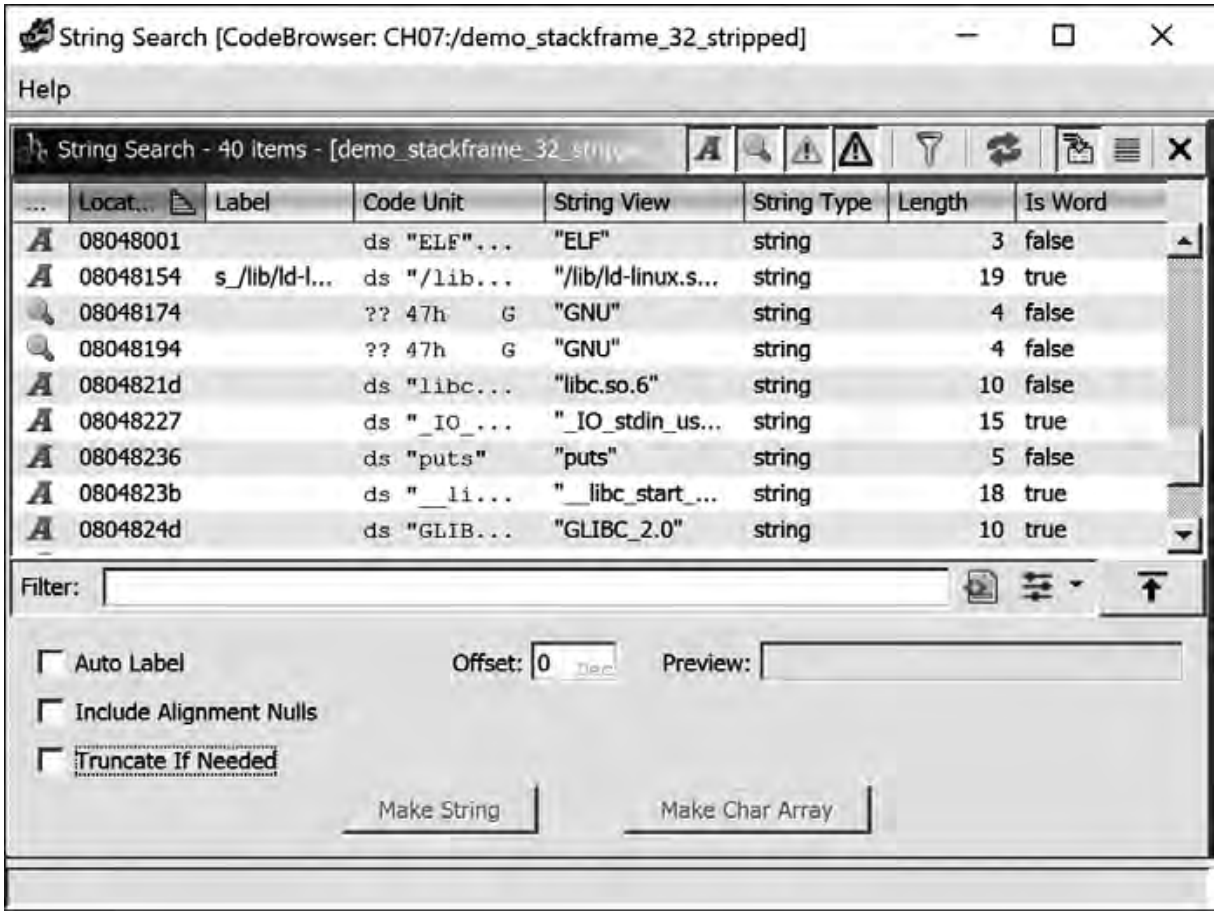


Рисунок 7-21. Окно String Search с результатами поиска

Самый левый столбец окна String Search содержит значки, которые указывают статус определения строки (от неопределенной до конфликтующей). Значения значков показаны на рисунке 7-22. Чтобы показать или скрыть строки в любой из категорий, переключайте соответствующие значки в строке заголовка.

Icon	Definition
	The string is already defined (and thus appears in the Defined Strings window). Such strings are usually the target of a cross-reference.
	The string is not defined. The string is not the target of a cross-reference, and the bytes generally appear as individual hex values.
	Part of the string has been defined. Usually this is a string that has a defined string as a substring.
	The string conflicts (overlaps) with something already defined such as existing instructions or data.

Figure 7-22: String toggle icon definitions

Рисунок 7-22. Определения значков переключения строк

Использование значков позволяет легко идентифицировать элементы в листинге, которые еще не определены как строки, и сделать из этих записей строку или символьный массив, выбрав их и щелкнув кнопку Make String или Make Char Array соответственно. Эти вновь определенные сущности будут отображаться в окне `Defined Strings`, которое обсуждается в разделе "Окно `Defined Strings`" на странице 81.

Определение массивов

Один из недостатков дизассемблированных листингов, полученных из языков более высокого уровня, состоит в том, что они дают очень мало подсказок о размере массивов. В дизассемблированном листинге массив может занимать огромное количество места, если каждый элемент массива указан в собственной строке. Следующий листинг показывает последовательность элементов в секции данных. Тот факт, что только первый элемент в листинге упоминается какими-либо инструкциями, предполагает, что это может быть первый элемент массива. Вместо прямого обращения дополнительные элементы внутри массивов часто адресуются с помощью вычислений индекса относительно начала массива.

```
         DAT_004195a4                            XREF[1]:  main:00411727(W)
004195a4    undefined4    ??
004195a8      ??          ??
004195a9      ??          ??
004195aa      ??          ??
004195ab      ??          ??
004195ac      ??          ??
004195ad      ??          ??
004195ae      ??          ??
004195af      ??          ??
004195b0      ??          ??
004195b1      ??          ??
004195b2      ??          ??
004195b3      ??          ??
004195b4      ??          ??
004195b5      ??          ??
004195b6      ??          ??
```

Ghidra может сгруппировать последовательные определения данных в одно определение массива. Чтобы создать массив, выберите первый элемент массива и используйте пункт Data > Create Array в контекстном меню (горячая клавиша `[`). Вам будет предложено указать число элементов в массиве, либо можно принять стандартное значение, которое предлагает Ghidra. (Если вы выбрали диапазон данных, а не одно значение, Ghidra использует ваш выбор как границы массива.) По умолчанию тип и размер данных, связанные с элементами массива, основаны на типе данных первого элемента в выделении. Массив представлен в свернутом формате, но его можно развернуть, чтобы увидеть отдельные элементы. Число элементов, отображаемых в строке, управляется в `Edit > Tool Options` окна `CodeBrowser`. Массивы подробнее обсуждаются в главе 8.

Резюме

Вместе с предыдущей главой эта глава охватывает самые распространенные операции, которые когда-либо понадобятся пользователям Ghidra. Манипулирование дизассемблированием позволяет объединять ваши знания со знаниями, которые Ghidra передала во время этапа анализа, чтобы получать ценную информацию. Как и в исходном коде, эффективное использование имен, назначение типов данных и подробные комментарии не только помогут вам помнить, что вы уже проанализировали, но и значительно помогут другим, кто будет пользоваться вашей работой. В следующей главе мы рассмотрим, как работать с более сложными структурами данных, такими как C `struct`, и изучим некоторые низкоуровневые детали скомпилированного C++.

Глава 8. Типы данных и структуры данных

Понимание типов данных и структур данных, которые вы встречаете при анализе бинарного файла, является фундаментом обратной разработки. Данные, передаваемые в функцию, - ключ к обратной разработке сигнатуры функции (числа, типа и последовательности параметров, требуемых функцией). Помимо этого, типы данных и структуры данных, объявленные и используемые внутри функций, дают дополнительные подсказки о том, что делает каждая функция. Это усиливает важность глубокого понимания того, как типы данных и структуры данных представлены и обрабатываются на уровне языка ассемблера.

В этой главе мы уделим значительное время этим темам, настолько критичным для успеха обратной разработки. Мы покажем, как распознавать структуры данных, используемые в дизассемблировании, и как моделировать эти структуры в Ghidra. Затем покажем, как богатая коллекция шаблонов структур Ghidra может сэкономить вам время при анализе. Поскольку классы C++ являются сложным расширением структур C, глава завершается обсуждением обратной разработки скомпилированных программ C++. Итак, начнем обсуждение манипулирования и определения простых и сложных типов данных и структур, встречающихся в скомпилированных программах.

Осмысление данных

Как специалист по обратной разработке, вы хотите придать смысл данным, которые видите в дизассемблировании. Самый простой способ связать конкретный тип данных с переменной - наблюдать использование переменной как параметра функции, о которой нам что-то известно. Во время этапа анализа Ghidra прилагает все усилия, чтобы аннотировать типы данных, когда их можно вывести на основе использования переменной с функцией, для которой у Ghidra есть прототип.

Для импортированных библиотечных функций Ghidra часто уже знает прототип функции. В таких случаях прототип можно легко просмотреть, наведя указатель мыши на имя функции в окне `Listing` или в окне `Symbol Tree`. Когда Ghidra ничего не знает о последовательности параметров функции, она, как минимум, должна знать имя библиотеки, из которой функция была импортирована (см. папку `Imports` в окне `Symbol Tree`). Когда это происходит, лучшими ресурсами для изучения сигнатуры и поведения функции являются любые связанные man-страницы или другая доступная документация API. Когда все остальное не помогает, вспомните поговорку: "Google - ваш друг".

Самые доступные плоды в понимании поведения бинарных программ лежат в каталогизации библиотечных функций, которые вызывает программа. Программа на C, вызывающая функцию `connect`, создает сетевое соединение. Программа Windows, вызывающая `RegOpenKey`, обращается к реестру Windows. Однако требуется дополнительный анализ, чтобы понять, как и почему эти функции вызываются.

Чтобы выяснить, как вызывается функция, нужно узнать о параметрах, связанных с функцией. Рассмотрим программу на C, которая вызывает функцию `connect` в рамках получения HTML-страницы. Чтобы вызвать `connect`, программе нужно знать IP-адрес и порт назначения сервера, на котором размещена страница; эти сведения предоставляет библиотечная функция `getaddrinfo`. Ghidra распознает ее как библиотечную функцию и добавляет комментарий к вызову, чтобы предоставить нам дополнительную информацию в окне `Listing`, как показано здесь:

```
00010a30  CALL  getaddrinfo    int getaddrinfo(char * __name, c...
```

Получить больше информации об этом вызове можно несколькими способами. Наведение указателя мыши на сокращенный комментарий справа от инструкции показывает, что Ghidra предоставила полный прототип функции, чтобы помочь понять параметры, передаваемые при вызове функции. Наведение на имя функции в `Symbol Tree` отображает прототип функции и переменные во всплывающем окне. Кроме того, выбор Edit Function в меню по щелчку правой кнопкой мыши предоставляет ту же информацию в редактируемом формате, как показано на рисунке 8-1. Если вам нужна еще более подробная информация, можно затем использовать окно `Data Type Manager`, чтобы найти сведения о конкретных параметрах, например о типе данных `addrinfo`. Если бы вы щелкнули `getaddrinfo` в предыдущем листинге, то увидели бы, что содержимое, показанное на рисунке 8-1, воспроизводится внутри листинга. (Это находится внутри thunk-функции, которая обсуждается в разделе "Thunk" на странице 212.)

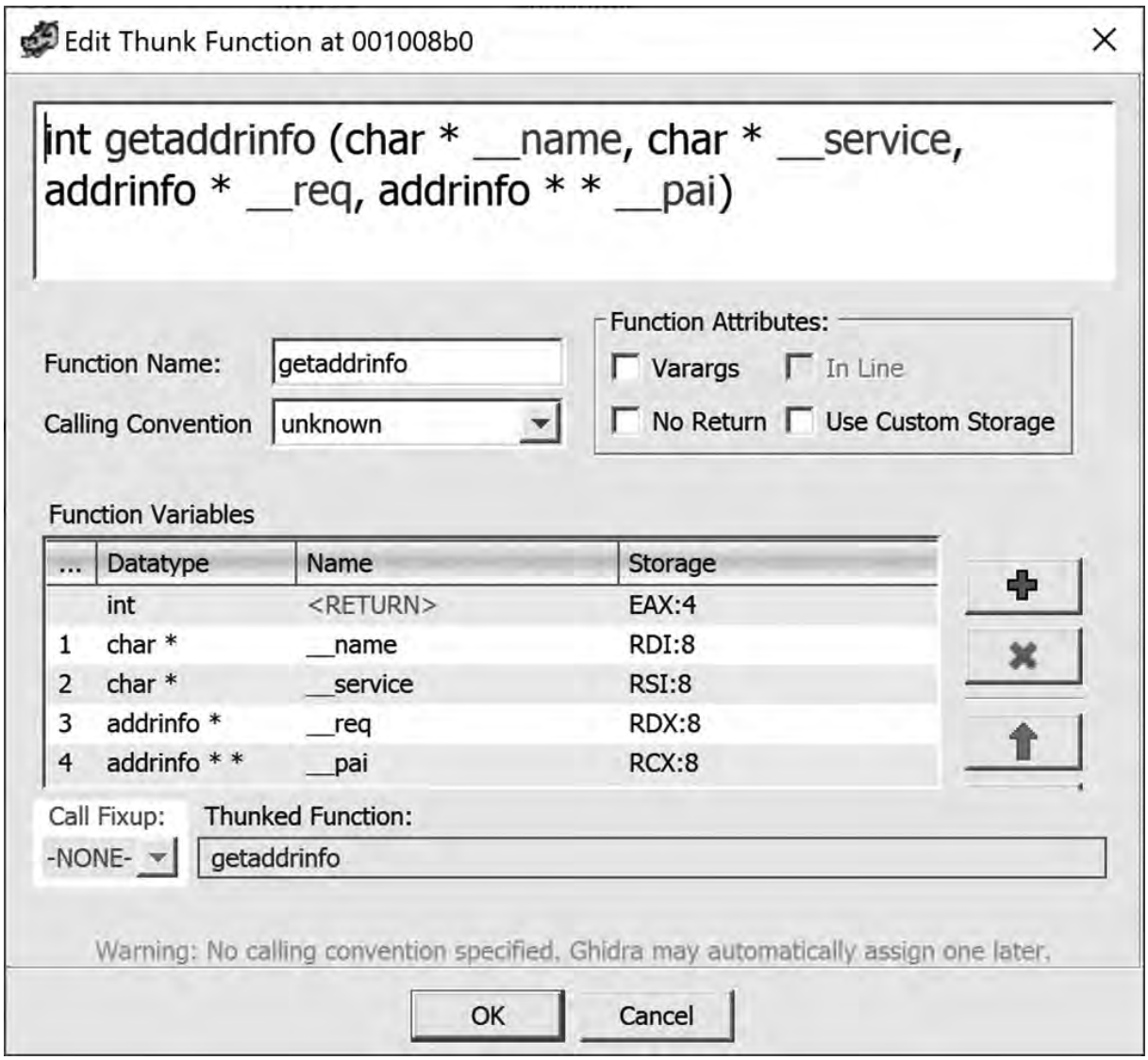


Рисунок 8-1. Окно Edit Function для функции `getaddrinfo`

Наконец, вам не обязательно переходить через окна `Symbol Tree` и `Data Type Manager`, чтобы сделать эти наблюдения, поскольку декомпилятор уже применил эту информацию в окне `Decompiler`. Если вы посмотрите на окно `Decompiler`, то увидите, что Ghidra уже встроила имена членов для полей, содержащихся в структуре (`addrinfo`), используя сведения из загруженных библиотек типов. Для того же примера в следующем фрагменте кода из декомпилятора видно, что имена членов `ai_family` и `ai_socktype` помогают понять, что `local_48` является структурой, используемой при получении информации, необходимой для `connect`. В этом случае присваивание `ai_family` указывает, что используется IPv4-адрес (`2` соответствует символической константе `AF_INET`), а `ai_socktype` указывает на использование потокового сокета (`1` соответствует символической константе `SOCK_STREAM`):

```
local_48.ai_family = 2;
local_48.ai_socktype = 1;
local_10 = getaddrinfo(param_1,"www",&local_48,&local_18);
```

Распознавание использования структур данных

Хотя примитивные типы данных часто помещаются в регистры процессора или операнды инструкций, составные типы данных, такие как массивы и структуры, обычно требуют более сложных последовательностей инструкций для доступа к отдельным элементам данных, которые они содержат. Прежде чем обсуждать возможности Ghidra для повышения читаемости кода, использующего сложные типы данных, нужно рассмотреть, как выглядит такой код.

Доступ к элементам массива

Массивы являются самыми простыми составными структурами данных с точки зрения расположения в памяти. Традиционно массивы - это непрерывные блоки памяти, содержащие последовательные элементы одного и того же типа данных (однородная коллекция). Размер массива равен произведению числа элементов в массиве и размера каждого элемента. В нотации C минимальное число байтов, потребляемых объявлением целочисленного массива

```
int array_demo[100];
```

вычисляется как

```
int bytes = 100 * sizeof(int); // или 100 * sizeof(array_demo[0])
```

К отдельным элементам массива можно обращаться, задав значение индекса, которое может быть переменной или константой, как показано в этих допустимых обращениях к массиву:

```
array_demo[20] = 15;              // фиксированный индекс в массиве
for (int i = 0; i < 100; i++) {
    array_demo[i] = i;            // изменяющийся индекс в массиве
}
```

Предположим для примера, что `sizeof(int)` равен 4 байтам. Тогда первое обращение к массиву обращается к целочисленному значению, расположенному на 80 байт от начала массива, тогда как второе обращение к массиву обращается к целым числам по смещениям 0, 4, 8, ... 96 байт от начала массива. Смещение для первого обращения к массиву можно вычислить во время компиляции как `20 * 4`. В большинстве случаев смещение для второго обращения к массиву должно вычисляться во время выполнения, потому что значение счетчика цикла `i` не фиксировано во время компиляции. Поэтому произведение `i * 4` вычисляется на каждом проходе цикла, чтобы определить точное смещение внутри массива.

В конечном счете способ обращения к элементу массива зависит не только от типа используемого индекса, но и от того, где массив выделен в пространстве памяти программы.

Глобально выделенные массивы

Когда массив выделен в глобальной области данных программы (например, в секции `.data` или `.bss`), компилятор знает базовый адрес массива во время компиляции, что позволяет ему вычислять фиксированные адреса для любого элемента массива, к которому обращаются с использованием фиксированного индекса. Рассмотрим следующую простую программу, которая обращается к глобальному массиву с использованием как фиксированных, так и переменных индексов.

```
int global_array[3];

int main(int argc, char **argv) {
    int idx = atoi(argv[1]); // для простоты без проверки границ
    global_array[0] = 10;
    global_array[1] = 20;
    global_array[2] = 30;
    global_array[idx] = 40;
}
```

> **Чего на самом деле ожидает C?**
>
> Для простоты мы сказали, что C ожидает целочисленный индекс как переменную или константу. В действительности подойдет любое выражение, которое можно вычислить или интерпретировать как целое число. Общее правило таково: "везде, где можно использовать целое число, можно использовать выражение, вычисляющееся в целое число". Разумеется, это не ограничивается только целыми числами. C вполне готов вычислить любое предоставленное выражение и попытаться заставить его работать с ожидаемым типом переменной. А что, если значения выходят за границы массива? Конечно, вы получаете основу для многочисленных эксплуатируемых уязвимостей! Значения будут читаться из полученной области памяти за границами или записываться в нее, либо программа просто аварийно завершится, если вычисленный целевой адрес недопустим внутри программы.

Если дизассемблировать версию соответствующего бинарного файла, очищенную от символов, функция `main` содержит следующий код:

```
          ...
00100657  CALL   atoi
0010065c  MOV    dword ptr [RBP + local_c],EAX
0010065f  MOV    dword ptr [DAT_00301018],10
00100669  MOV    dword ptr [DAT_0030101c],20
00100673  MOV    dword ptr [DAT_00301020],30
0010067d  MOV    EAX,dword ptr [RBP + local_c]
00100680  CDQE
00100682  LEA    RDX,[RAX*4]
0010068a  LEA    RAX,[DAT_00301018]
00100691  MOV    dword ptr [RDX + RAX*1]=>DAT_00301018,40
          ...
```

Хотя у этой программы только одна глобальная переменная (глобальный массив), строки дизассемблирования, похоже, указывают на три глобальные переменные: `DAT_00301018`, `DAT_0030101c` и `DAT_00301020` соответственно. Однако инструкция `LEA` загружает адрес глобальной переменной, виденной ранее. В этом контексте, в сочетании с вычислением смещения (`RAX*4`) и масштабированным обращением к памяти, `DAT_00301018`, скорее всего, является базовым адресом глобального массива. Аннотированный операнд `=>DAT_00301018` предоставляет нам базу массива,

в который будет записано `40`.

Что такое бинарник, очищенный от символов?

Когда компиляторы генерируют объектные файлы, они должны включать достаточно информации, чтобы компоновщик мог выполнить свою работу. Одна из задач компоновщика - разрешать ссылки между объектными файлами, например вызов функции, тело которой находится в другом файле, используя информацию из сгенерированного компилятором символа. Во многих случаях компоновщик объединяет всю информацию таблицы символов из объектных файлов и включает сводную информацию в получившийся исполняемый файл. Эта информация не нужна для корректного выполнения исполняемого файла, но она очень полезна с точки зрения обратной разработки, поскольку Ghidra (и другие инструменты, такие как отладчики) может использовать информацию таблицы символов для восстановления имен и размеров функций и глобальных переменных.

Удаление символов из бинарного файла, или очистка от символов, означает удаление частей исполняемого файла, которые не являются существенными для работы бинарного файла во время выполнения. Это можно сделать с помощью утилиты командной строки `strip`, постобрабатывающей исполняемый файл, или с помощью параметров сборки компилятора и/или компоновщика (`-s` для `gcc`/`ld`), чтобы они сами создали бинарник, очищенный от символов. Помимо информации таблицы символов, `strip` может удалить любую информацию отладочных символов, такую как имена локальных переменных и информация о типах, встроенную в бинарный файл при сборке. Без информации о символах инструменты обратной разработки должны иметь алгоритмы как для идентификации, так и для именования функций и данных.

Исходя из имен, назначенных Ghidra, мы знаем, что глобальный массив начинается с 12 байтов по адресу `00301018`. Во время компиляции компилятор использовал фиксированные индексы (0, 1, 2), чтобы вычислить фактические адреса соответствующих элементов массива (`00301018`, `0030101c` и `00301020`), к которым обращаются через глобальные переменные. Исходя из значений, перемещаемых в эти местоположения, можно предположить, что мы перемещаем в этот массив 32-разрядные целочисленные (`dword`) значения. Если перейти к связанным данным в листинге, мы увидим такое содержимое:

```
        DAT_00301018
00301018      ??       ??
00301019      ??       ??
0030101a      ??       ??
0030101b      ??       ??
        DAT_0030101c
0030101c      ??       ??
0030101d      ??       ??
0030101e      ??       ??
0030101f      ??       ??
        DAT_00301020
00301020      ??       ??
00301021      ??       ??
00301022      ??       ??
00301023      ??       ??
```

Вопросительные знаки указывают, что этот массив, вероятно, выделен в секции `.bss` программы и что в образе файла отсутствуют значения инициализации.

Распознать массив в дизассемблировании легче, когда к нему обращаются с использованием переменных индексов. Когда для доступа к глобальным массивам используются постоянные индексы, соответствующие элементы массива выглядят в дизассемблировании как глобальные переменные. Однако использование переменных значений индекса раскрывает базовый адрес массива и размер отдельных элементов, поскольку смещение внутри массива должно вычисляться

с использованием индекса. (Такие операции масштабирования требуются, чтобы преобразовать целочисленный индекс массива из C в байтовое смещение для правильного элемента массива на языке ассемблера.)

Используя операции форматирования типов и массивов Ghidra, обсуждавшиеся в предыдущей главе (Data > Create Array), можно отформатировать `DAT_000301018` как трехэлементный целочисленный массив, получив строки дизассемблирования с именованным массивом, к которому обращаются по индексам, а не по смещениям:

```
00100660  MOV    dword ptr [INT_ARRAY_00301018],10
0010066a  MOV    dword ptr [INT_ARRAY_00301018[1]],20
00100674  MOV    dword ptr [INT_ARRAY_00301018[2]],30
```

Стандартное имя массива, назначенное Ghidra, `INT_ARRAY_00301018`, включает тип массива, а также начальный адрес массива.

> **Обновление информации о символах в комментариях**
>
> Когда вы начинаете идентифицировать типы данных, менять имена символов и т. д., можно убедиться, что ценные комментарии, которые вы добавили в листинг, не устареют и не станут трудными для понимания, используя аннотации комментариев, которые обновляются автоматически. Пункт аннотации Symbol позволяет включать ссылки на символы, которые будут обновляться по мере того, как вы изменяете символы, чтобы точно отразить свои находки. (См. "Аннотации" на странице 132.)

Посмотрим на окно `Decompiler` до (рисунок 8-2) и после (рисунок 8-3) создания массива. На рисунке 8-2 важное предупреждение во второй строке является еще одной подсказкой, что перед вами может быть массив, а присваивание целочисленных значений поддерживает предположение, что тип массива - целочисленный.

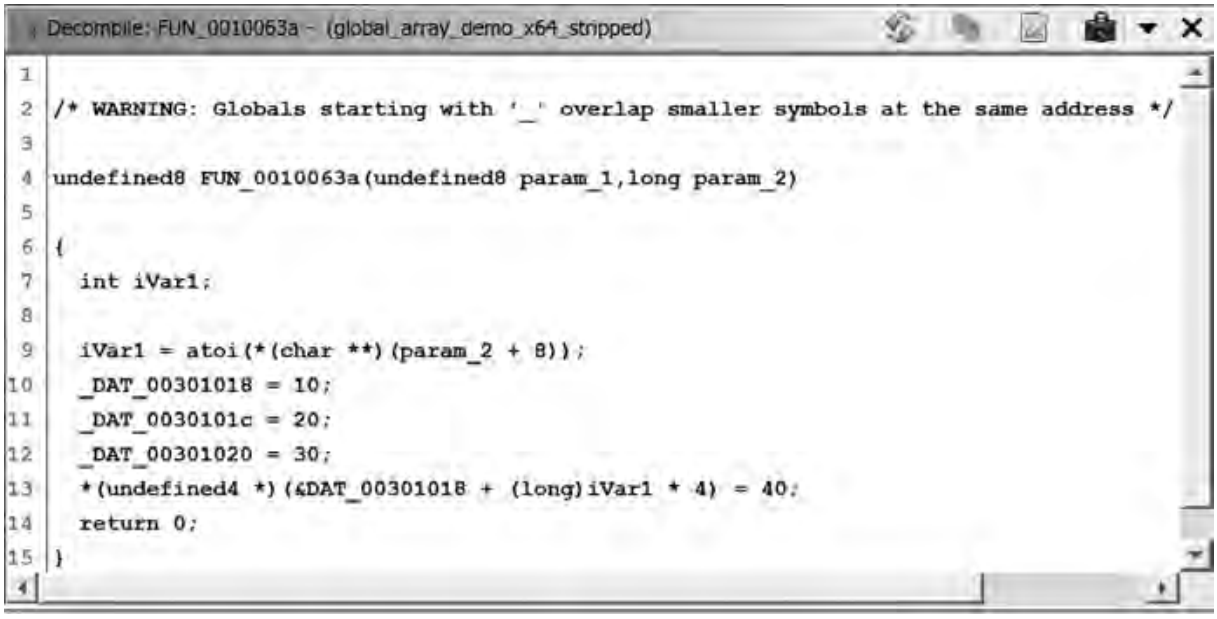


Рисунок 8-2. Представление `Decompiler`*, указывающее на потенциальный массив*

После создания целочисленного массива код в окне `Decompiler` обновляется, чтобы использовать новую переменную массива, как показано на рисунке 8-3.

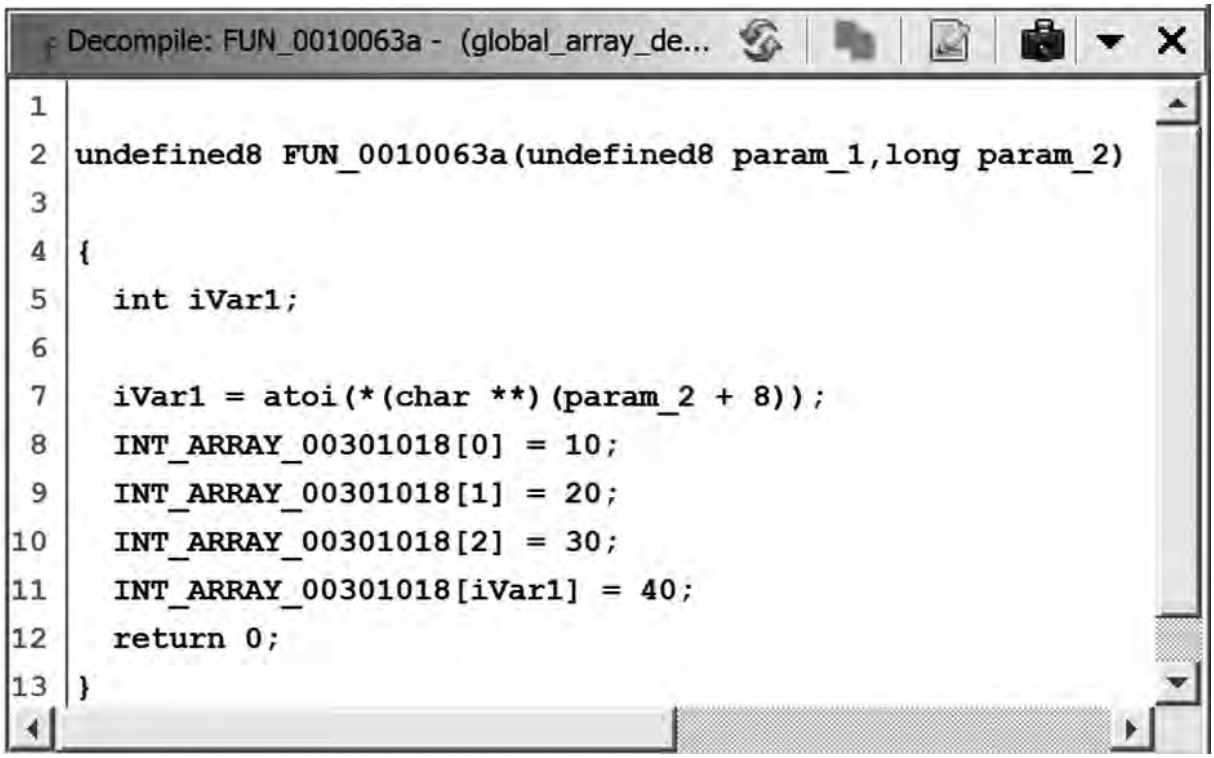


Рисунок 8-3. Представление `Decompiler` *после объявления типа массива*

Массивы, выделенные в стеке

Компилятор не может знать абсолютный адрес массива, выделенного в стеке как локальная переменная функции, во время компиляции, поэтому даже обращения, использующие постоянные индексы, требуют некоторых вычислений во время выполнения. Несмотря на различия, компиляторы часто обрабатывают массивы, выделенные в стеке, почти так же, как глобально выделенные массивы.

Следующая программа - вариант предыдущего примера, использующий массив, выделенный в стеке, а не глобальный массив:

```
int main(int argc, char **argv) {
   int stack_array[3];
   int idx = atoi(argv[1]); // проверка границ для простоты опущена
   stack_array[0] = 10;
   stack_array[1] = 20;
   stack_array[2] = 30;
   stack_array[idx] = 40;
}
```

Адрес, по которому будет выделен `stack_array`, неизвестен во время компиляции, поэтому компилятор не может заранее вычислить адрес `stack_array[2]`, как он сделал для `global_array[2]`. Однако компилятор может вычислить относительное местоположение любого элемента внутри массива. Например, элемент `stack_array[2]` начинается со смещения `2 * sizeof(int)` от начала массива, и компилятор прекрасно знает это во время компиляции. Если компилятор решит выделить `stack_array` по смещению `EBP-0x18` внутри стекового кадра, он может вычислить `EBP-0x18+2*sizeof(int)`, что во время компиляции сводится к `EBP-0x10` и избавляет от необходимости дополнительной арифметики во время выполнения для доступа к `stack_array[2]`. Это становится очевидным в следующем листинге:

```
      undefined main()
         undefined     AL:1           <RETURN>
         undefined4    Stack[-0xc]:4  local_c
         undefined4    Stack[-0x10]:4 local_10
         undefined4    Stack[-0x14]:4 local_14
         undefined4    Stack[-0x18]:4 local_18
         undefined4    Stack[-0x1c]:4 local_1c
         undefined8    Stack[-0x28]:8 local_28
0010063a  PUSH   RBP
0010063b  MOV    RBP,RSP
0010063e  SUB    RSP,0x20
00100642  MOV    dword ptr [RBP + local_1c],EDI
00100645  MOV    qword ptr [RBP + local_28],RSI
00100649  MOV    RAX,qword ptr [RBP + local_28]
0010064d  ADD    RAX,0x8
00100651  MOV    RAX,qword ptr [RAX]
00100654  MOV    RDI,RAX
00100657  MOV    EAX,0x0
0010065c  CALL   atoi
00100661  MOV    dword ptr [RBP + local_c],EAX
00100664  MOV    dword ptr [RBP + local_18],10
0010066b  MOV    dword ptr [RBP + local_14],20
00100672  MOV    dword ptr [RBP + local_10],30
00100679  MOV    EAX,dword ptr [RBP + local_c]
0010067c  CDQE
0010067e  MOV    dword ptr [RBP + RAX*0x4 + -0x10],40
00100686  MOV    EAX,0x0
0010068b  LEAVE
0010068c  RET
```

Обнаружить этот массив еще сложнее, чем глобальный массив. Эта функция выглядит так, будто у нее шесть несвязанных переменных (`local_c`, `local_10`, `local_14`, `local_18`, `local_1c` и `local_28`), а не массив из трех целых чисел и целочисленная индексная переменная. Две из этих локальных переменных (`local_1c` и `local_28`) - это два параметра функции, `argc` и `argv`, сохраненные для дальнейшего использования.

Использование постоянных значений индекса обычно скрывает присутствие массива, выделенного в стеке, потому что вы видите только присваивания отдельным локальным переменным. Только умножение намекает на существование массива, отдельные элементы которого имеют размер 4 байта. Разберем эту инструкцию подробнее: `RBP` содержит адрес базового указателя стекового кадра; `RAX*4` - это индекс массива (преобразованный `atoi` и сохраненный в `local_c`), умноженный на размер элемента массива; `-0x10` - это смещение к началу массива от `RBP`.

Процесс преобразования локальных переменных в массив немного отличается от создания массива в секции данных листинга. Поскольку информация о структуре стека связана с первым адресом функции, нельзя выбрать подмножество стековых переменных. Вместо этого поместите курсор на переменную в начале массива, `local_18`, выберите Set Data Type, затем пункт Array в контекстном меню по щелчку правой кнопкой мыши, а затем укажите число элементов в массиве. Ghidra покажет предупреждение о конфликте с локальными переменными, которые мы втягиваем в определение массива, как показано на рисунке 8-4.

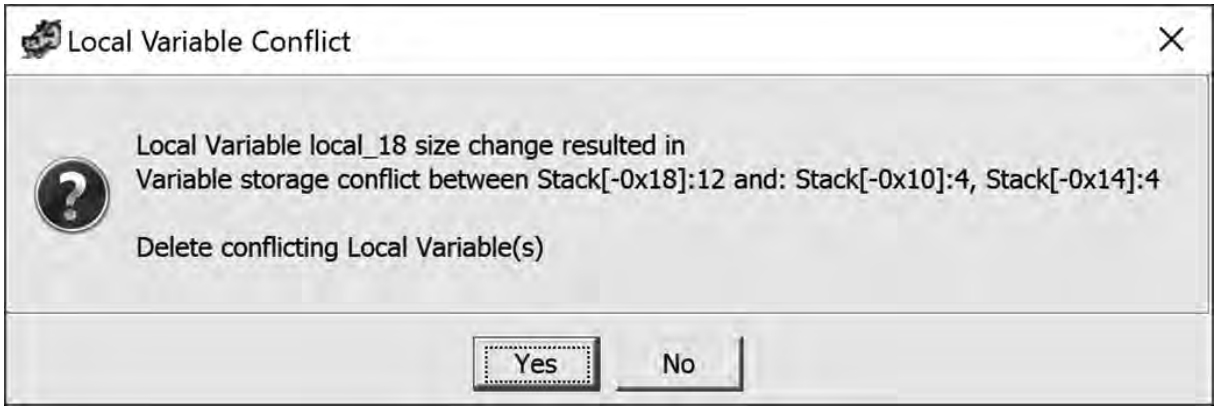


Рисунок 8-4. Предупреждение о потенциальном конфликте при определении стекового массива

Если вы продолжите, несмотря на потенциальный конфликт, вы увидите массив в окне `Listing`, как показано здесь:

```
          ...
00100664  MOV    dword ptr [RBP + local_18[0]],10
0010066b  MOV    dword ptr [RBP + local_18[1]],20
00100672  MOV    dword ptr [RBP + local_18[2]],30
          ...
```

Даже после определения массива листинг декомпилятора на рисунке 8-5 не похож на исходный код. Декомпилятор опустил статические присваивания массиву, потому что считает, что они не вносят вклад в результат функции. Вызов `atoi` и получившееся присваивание остаются, потому что Ghidra не может вычислить побочные эффекты вызова `atoi`, но Ghidra ошибочно принимает сохраненный результат `atoi` за четвертый элемент массива (`local_c` в дизассемблировании и `iVar1` в листинге декомпилятора).

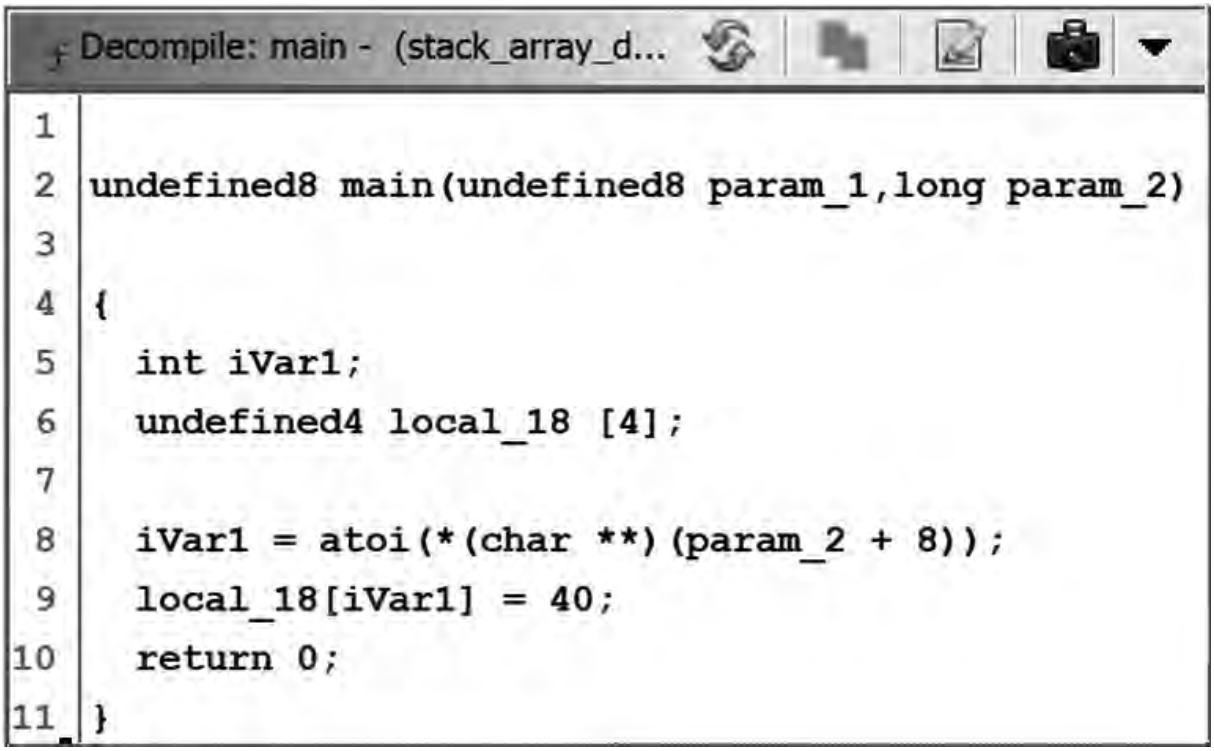


Рисунок 8-5. Представление `Decompiler` *функции со всеми стековыми переменными после определения массива*

Массивы, выделенные в куче

Массивы, выделенные в куче, выделяются с помощью функции динамического выделения памяти, такой как `malloc` (C) или `new` (C++). С точки зрения компилятора основное отличие при работе с массивом, выделенным в куче, состоит в том, что компилятор должен генерировать все обращения внутрь массива на основе адреса, возвращенного функцией выделения памяти. Следующая программа на C выделяет небольшой массив в куче программы:

```
int main(int argc, char **argv) {
   int *heap_array = (int*)malloc(3 * sizeof(int));
   int idx = atoi(argv[1]); // проверка границ для простоты опущена
   heap_array[0] = 10;
   heap_array[1] = 20;
   heap_array[2] = 30;
   heap_array[idx] = 40;
}
```

Соответствующее дизассемблирование немного сложнее, чем в двух предыдущих примерах:

```
      undefined main()
         undefined     AL:1              <RETURN>
         undefined8    Stack[-0x10]:8    heap_array
         undefined4    Stack[-0x14]:4    local_14
         undefined4    Stack[-0x1c]:4    local_1c
         undefined8    Stack[-0x28]:8    local_28
0010068a  PUSH   RBP
0010068b  MOV    RBP,RSP
0010068e  SUB    RSP,0x20
00100692  MOV    dword ptr [RBP + local_1c],EDI
00100695  MOV    qword ptr [RBP + local_28],RSI
00100699  MOV    EDI,0xc
0010069e  CALL   malloc
001006a3  MOV    qword ptr [RBP + heap_array],RAX
001006a7  MOV    RAX,qword ptr [RBP + local_28]
001006ab  ADD    RAX,0x8
001006af  MOV    RAX,qword ptr [RAX]
001006b2  MOV    RDI,RAX
001006b5  CALL   atoi
001006ba  MOV    dword ptr [RBP + local_14],EAX
001006bd  MOV    RAX,qword ptr [RBP + heap_array]
001006c1  MOV    dword ptr [RAX],10
001006c7  MOV    RAX,qword ptr [RBP + heap_array]
001006cb  ADD    RAX,0x4
001006cf  MOV    dword ptr [RAX],20
001006d5  MOV    RAX,qword ptr [RBP + heap_array]
001006d9  ADD    RAX,0x8
001006dd  MOV    dword ptr [RAX],30
001006e3  MOV    EAX,dword ptr [RBP + local_14]
001006e6  CDQE
001006e8  LEA    RDX,[RAX*0x4]
001006f0  MOV    RAX,qword ptr [RBP + heap_array]
001006f4  ADD    RAX,RDX
001006f7  MOV    dword ptr [RAX],40
001006fd  MOV    EAX,0x0
00100702  LEAVE
00100703  RET
```

Начальный адрес массива (возвращенный из `malloc` в регистре `RAX`) сохраняется в локальной переменной `heap_array`. В этом примере, в отличие от предыдущих, каждое обращение к массиву начинается с чтения содержимого `heap_array`, чтобы получить базовый адрес массива. Обращения к `heap_array[0]`, `heap_array[1]` и `heap_array[2]` требуют смещений соответственно 0, 4 и 8 байт. Обращение к массиву с переменным индексом `heap_array[idx]` реализуется несколькими инструкциями: они вычисляют смещение внутри массива путем умножения индекса массива на размер элемента массива и добавляют результат к базовому адресу массива.

У массивов, выделенных в куче, есть одна особенно приятная особенность: число элементов, выделенных массиву, можно вычислить из общего размера массива и размера каждого элемента. Параметр, переданный функции выделения памяти (`12`, переданный в `malloc`), сообщает число байтов, выделенных массиву. Деление этого значения на размер элемента (4 байта в этом примере, что видно по смещениям, идущим с шагом 4, и по масштабному множителю) сообщает нам число элементов в массиве. В этом примере был выделен трехэлементный массив.

Декомпилятор тоже смог распознать массив, как видно на рисунке 8-6. (Имя указателя на массив, `puVar2`, указывает, что это указатель на беззнаковое целое, используя префикс `pu`.)

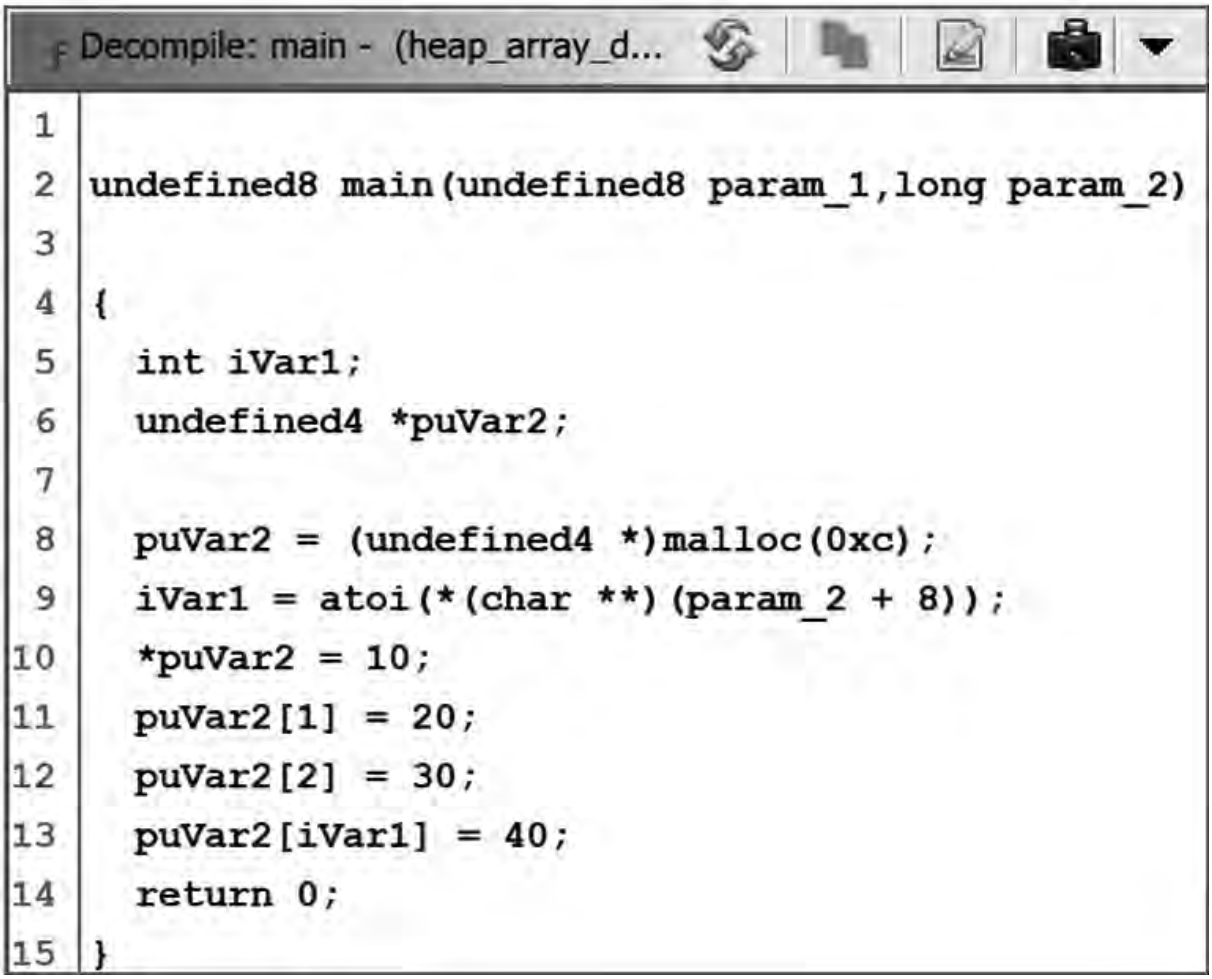


Рисунок 8-6. Представление `Decompiler` *функции с массивом в куче*

В этой функции, в отличие от функции с массивом, выделенным в стеке, листинг декомпилятора показывает присваивания массиву с постоянными индексами, хотя обычно он исключил бы их, потому что массив не используется в других операциях и не возвращается из функции. Этот случай отличается тем, что присваивания манипулируют не просто стековыми переменными: стековая переменная фактически является указателем на память, которую `malloc` запросила из кучи. Запись через эту переменную записывает не в локальную стековую переменную, а использует стековую переменную, чтобы найти выделенную память. Программа может потерять указатель (адрес начала массива в куче) при выходе из функции, но значения останутся в памяти. (Этот конкретный пример на самом деле демонстрирует утечку памяти. Хотя это не лучшая практика программирования, он позволяет показать понятие массива в куче.)

В заключение: массивы легче всего распознать, когда переменная используется как индекс массива. Операция доступа к массиву, требующая масштабировать индекс на размер элемента массива перед добавлением получившегося смещения к базовому адресу массива, выделяется в

дизассемблированном листинге.

Доступ к членам структуры

Структуры в стиле C, здесь в общем виде называемые структурами, группируют коллекции (часто неоднородных) элементов данных в составной тип данных. В исходном коде поля данных внутри структуры адресуются по имени, а не по индексу. К сожалению, эти информативные имена полей компилятор преобразует в числовые смещения, поэтому к моменту, когда вы смотрите на дизассемблирование, доступ к полям структуры выглядит удивительно похожим на доступ к элементам массива с использованием постоянных индексов.

Следующее определение структуры, содержащее пять неоднородных полей, будет использоваться в следующих примерах:

```
struct ch8_struct {     //Размер  Минимальное смещение  Смещение по умолчанию
    int    field1;      //  4       0             0
    short  field2;      //  2       4             4
    char   field3;      //  1       6             6
    int    field4;      //  4       7             8
    double field5;      //  8       11            16
};                      // Минимальный общий размер: 19 Размер по умолчанию: 24
```

Когда компилятор встречает определение структуры, он поддерживает текущую сумму числа байтов, потребляемых полями структуры, чтобы определить смещение каждого поля внутри структуры. Сумма пространства, требуемого для выделения каждого поля внутри структуры, определяет минимальное пространство, необходимое для структуры. Однако никогда не следует предполагать, что компилятор использует минимально необходимое пространство для выделения структуры. По умолчанию компиляторы выравнивают поля структуры по адресам памяти, которые позволяют наиболее эффективно читать и записывать эти поля. Например, 4-байтовые целочисленные поля будут выровнены по смещениям, делящимся на четыре, а 8-байтовые `double` - по смещениям, делящимся на восемь. В зависимости от состава структуры компилятор может вставлять байты заполнения, чтобы удовлетворить требованиям выравнивания, а значит фактический размер структуры будет больше суммы ее составных полей. Стандартные смещения и получившийся размер структуры для примерной структуры видны в столбце "Смещение по умолчанию" в комментариях в предыдущем определении структуры, и они дают 24, а не минимальные 19.

Структуры можно упаковать в минимально необходимое пространство с помощью параметров компилятора, запрашивающих конкретные выравнивания членов. Microsoft C/C++ и GNU `gcc/g++` распознают прагму `pack` для управления выравниванием полей структуры. Компиляторы GNU дополнительно распознают атрибут `packed` для управления выравниванием структуры на уровне отдельной структуры. Запрос 1-байтового выравнивания для полей структуры заставляет компиляторы сжать структуру в минимально необходимое пространство. Смещения и размер примерной структуры приведены в столбце "Минимальное смещение". (Обратите внимание, что некоторые процессоры работают лучше, когда данные выровнены согласно своему типу, тогда как другие процессоры могут генерировать исключения, если данные не выровнены по конкретным границам.)

Помня эти факты, посмотрим, как структуры обрабатываются в скомпилированном коде. Как и в случае с массивами, доступ к членам структуры выполняется путем сложения базового адреса структуры со смещением нужного члена. Однако, тогда как смещения массива могут вычисляться во время выполнения из предоставленного значения индекса (поскольку каждый элемент массива имеет один и тот же размер), смещения структуры должны вычисляться во время компиляции и появятся в скомпилированном коде как фиксированные смещения внутри структуры, выглядя почти

идентично ссылкам на массив, которые используют постоянные индексы.

Создание структур в Ghidra сложнее, чем создание массивов, поэтому мы рассмотрим его в следующем разделе, после того как покажем несколько примеров дизассемблированных и декомпилированных структур.

Глобально выделенные структуры

Как и у глобально выделенных массивов, адреса глобально выделенных структур известны во время компиляции. Это позволяет компилятору вычислить адрес каждого члена структуры во время компиляции и устраняет необходимость выполнять какие-либо вычисления во время выполнения. Рассмотрим следующую программу, которая обращается к глобально выделенной структуре:

```
struct ch8_struct global_struct;

int main() {
    global_struct.field1 = 10;
    global_struct.field2 = 20;
    global_struct.field3 = 30;
    global_struct.field4 = 40;
    global_struct.field5 = 50.0;
}
```

Если эта программа скомпилирована со стандартными параметрами выравнивания структуры, при дизассемблировании можно ожидать примерно следующее:

```
     undefined main()
        undefined     AL:1              <RETURN>
001005fa  PUSH   RBP
001005fb  MOV    RBP,RSP
001005fe  MOV    dword ptr [DAT_00301020],10
00100608  MOV    word ptr [DAT_00301024],20
00100611  MOV    byte ptr [DAT_00301026],30
00100618  MOV    dword ptr [DAT_00301028],40
00100622  MOVSD  XMM0,qword ptr [DAT_001006c8]
0010062a  MOVSD  qword ptr [DAT_00301030],XMM0
00100632  MOV    EAX,0x0
00100637  POP    RBP
00100638  RET
```

Это дизассемблирование вообще не содержит вычислений для доступа к членам структуры, и при отсутствии исходного кода было бы невозможно с какой-либо уверенностью утверждать, что структура вообще используется. Поскольку компилятор выполнил все вычисления смещений во время компиляции, эта программа выглядит так, будто обращается к пяти глобальным переменным, а не к пяти полям внутри одной структуры. Вы должны заметить сходство с предыдущим примером глобально выделенных массивов, использующих постоянные значения индекса.

На рисунке 8-2 равномерные смещения в сочетании со значениями позволили нам предположить (точно), что мы имеем дело с массивом. В этом примере мы вправе заключить, что не имеем дело с массивом, потому что размеры переменных неоднородны (`dword`, `word`, `byte`, `dword` и `qword` соответственно), но нам не хватает достаточных доказательств, чтобы утверждать, что мы имеем дело со `struct`.

Структуры, выделенные в стеке

Как и массивы, выделенные в стеке, структуры, выделенные в стеке, трудно распознать только по расположению стека, и декомпилятор не дает дополнительного понимания. Изменение предыдущей программы так, чтобы использовать структуру, выделенную в стеке и объявленную в `main`, дает следующее дизассемблирование:

```
       undefined main()
          undefined     AL:1              <RETURN>
          undefined8    Stack[-0x18]:8    local_18
          undefined4    Stack[-0x20]:4    local_20
          undefined1    Stack[-0x22]:1    local_22
          undefined2    Stack[-0x24]:2    local_24
          undefined4    Stack[-0x28]:4    local_28
  001005fa  PUSH   RBP
  001005fb  MOV    RBP,RSP
  001005fe  MOV    dword ptr [RBP + local_28],10
  00100605  MOV    word ptr [RBP + local_24],20
  0010060b  MOV    byte ptr [RBP + local_22],30
  0010060f  MOV    dword ptr [RBP + local_20],40
  00100616  MOVSD  XMM0,qword ptr [DAT_001006b8]
  0010061e  MOVSD  qword ptr [RBP + local_18],XMM0
  00100623  MOV    EAX,0x0
  00100628  POP    RBP
  00100629  RET
```

И снова для доступа к полям структуры не выполняется никаких вычислений, поскольку компилятор может определить относительные смещения каждого поля внутри стекового кадра во время компиляции, и перед нами остается та же потенциально вводящая в заблуждение картина: будто используются пять отдельных переменных, а не одна переменная, случайно содержащая пять отдельных полей. В действительности `local_28` должна быть началом 24-байтовой структуры, а каждая из остальных переменных должна быть каким-то образом отформатирована, чтобы отражать тот факт, что они являются полями внутри структуры.

Структуры, выделенные в куче

Структуры, выделенные в куче, раскрывают гораздо больше о размере структуры и расположении ее полей. Когда структура выделена в куче программы, у компилятора нет выбора, кроме как генерировать код для вычисления правильного адреса поля всякий раз, когда к полю выполняется обращение, потому что адрес структуры неизвестен во время компиляции. Для глобально выделенных структур компилятор может вычислить фиксированный начальный адрес. Для структур, выделенных в стеке, компилятор может вычислить фиксированную связь между началом структуры и указателем кадра для охватывающего стекового кадра. Когда структура выделена в куче, единственная ссылка на структуру, доступная компилятору, - это указатель на начальный адрес структуры.

Чтобы продемонстрировать структуры, выделенные в куче, мы изменим примерную программу так, чтобы объявить указатель внутри `main` и назначить ему адрес блока памяти, достаточно большого для хранения структуры:

```
int main() {
    struct ch8_struct *heap_struct;
    heap_struct = (struct ch8_struct*)malloc(sizeof(struct ch8_struct));
    heap_struct->field1 = 10;
    heap_struct->field2 = 20;
    heap_struct->field3 = 30;
    heap_struct->field4 = 40;
    heap_struct->field5 = 50.0;
}
```

Вот соответствующее дизассемблирование:

```
     undefined main()
        undefined     AL:1              <RETURN>
        undefined8    Stack[-0x10]:8    heap_struct
0010064a  PUSH   RBP
0010064b  MOV    RBP,RSP
0010064e  SUB    RSP,16
00100652  MOV    EDI,24
00100657  CALL   malloc
0010065c  MOV    qword ptr [RBP + heap_struct],RAX
00100660  MOV    RAX,qword ptr [RBP + heap_struct]
00100664  MOV    dword ptr [RAX],10
0010066a  MOV    RAX,qword ptr [RBP + heap_struct]
0010066e  MOV    word ptr [RAX + 4],20
00100674  MOV    RAX,qword ptr [RBP + heap_struct]
00100678  MOV    byte ptr [RAX + 6],30
0010067c  MOV    RAX,qword ptr [RBP + heap_struct]
00100680  MOV    dword ptr [RAX + 8],40
00100687  MOV    RAX,qword ptr [RBP + heap_struct]
0010068b  MOVSD  XMM0,qword ptr [DAT_00100728]
00100693  MOVSD  qword ptr [RAX + 16],XMM0
00100698  MOV    EAX,0x0
0010069d  LEAVE
0010069e  RET
```

В этом примере мы можем различить точный размер и расположение структуры. Размер структуры можно вывести как 24 байта на основе объема памяти, запрошенного у `malloc`. Структура содержит следующие поля по указанным смещениям:

- 4-байтовое (`dword`) поле по смещению 0
- 2-байтовое (`word`) поле по смещению 4
- 1-байтовое поле по смещению 6
- 4-байтовое (`dword`) поле по смещению 8
- 8-байтовое (`qword`) поле по смещению 16

На основе использования инструкций с плавающей точкой (`MOVSD`) можно дополнительно вывести, что поле `qword` на самом деле является `double`.

Та же программа, скомпилированная с упаковкой структур по 1-байтовому выравниванию, дает следующее дизассемблирование:

```
0010064a  PUSH    RBP
0010064e  SUB    RSP,16
00100652  MOV    EDI,19
00100657  CALL   malloc
0010065c  MOV    qword ptr [RBP + local_10],RAX
00100660  MOV    RAX,qword ptr [RBP + local_10]
00100664  MOV    dword ptr [RAX],10
0010066a  MOV    RAX,qword ptr [RBP + local_10]
0010066e  MOV    word ptr [RAX + 4],20
00100674  MOV    RAX,qword ptr [RBP + local_10]
00100678  MOV    byte ptr [RAX + 6],30
0010067c  MOV    RAX,qword ptr [RBP + local_10]
```

```
00100680  MOV    dword ptr [RAX + 7],40
00100687  MOV    RAX,qword ptr [RBP + local_10]
0010068b  MOVSD  XMM0,qword ptr [DAT_00100728]
00100693  MOVSD  qword ptr [RAX + 11],XMM0
00100698  MOV    EAX,0x0
0010069d  LEAVE
0010069e  RET
```

Единственные изменения - меньший размер структуры (теперь 19 байт) и скорректированные смещения, учитывающие новое выравнивание каждого поля структуры.

Независимо от выравнивания, использованного при компиляции программы, поиск структур, выделенных и обрабатываемых в куче программы, является самым быстрым способом определить размер и расположение данной структуры данных. Однако помните, что многие функции не окажут вам услугу немедленно обратиться к каждому члену структуры, чтобы помочь понять расположение структуры. Вместо этого вам может понадобиться отслеживать использование указателя на структуру и отмечать смещения, используемые каждый раз, когда этот указатель разыменовывается, а затем постепенно собирать полную схему структуры. В разделе "Пример 3: автоматизированное создание структуры" на странице 437 вы увидите, как декомпилятор может автоматизировать этот процесс.

Массивы структур

Некоторые программисты говорят, что красота составных структур данных в том, что они позволяют строить сколь угодно сложные структуры, вкладывая меньшие структуры в большие: например, массивы структур, структуры внутри структур и структуры, содержащие массивы как члены. Предыдущие обсуждения массивов и структур точно так же применимы к таким вложенным типам. В качестве примера рассмотрим следующую простую программу, в которой `heap_struct` указывает на массив из пяти элементов `ch8_struct`:

```
int main() {
    int idx = 1;
    struct ch8_struct *heap_struct;
    heap_struct = (struct ch8_struct*)malloc(sizeof(struct ch8_struct) * 5);
    heap_struct[idx].field1 = 10;
}
```

Под капотом доступ к `field1` включает умножение значения индекса на размер элемента массива (в данном случае размер структуры), а затем добавление смещения к нужному полю. Соответствующее дизассемблирование показано здесь:

```
     undefined main()
        undefined     AL:1              <RETURN>
        undefined4    Stack[-0xc]:4     idx
        undefined4    Stack[-0x18]:8    heap_struct
0010064a  PUSH   RBP
0010064b  MOV    RBP,RSP
0010064e  SUB    RSP,16
00100652  MOV    dword ptr [RBP + idx],1
00100659  MOV    EDI,120
0010065e  CALL   malloc
00100663  MOV    qword ptr [RBP + heap_struct],RAX
00100667  MOV    EAX,dword ptr [RBP + idx]
0010066a  MOVSXD RDX,EAX
0010066d  MOV    RAX,RDX
00100670  ADD    RAX,RAX
00100673  ADD    RAX,RDX
00100676  SHL    RAX,3
0010067a  MOV    RDX,RAX
0010067d  MOV    RAX,qword ptr [RBP + heap_struct]
00100681  ADD    RAX,RDX
00100684  MOV    dword ptr [RAX],10
```

```
0010068a  MOV    EAX,0
0010068f  LEAVE
00100690  RET
```

Функция выделяет 120 байт в куче. Индекс массива в `RAX` умножается на 24 с помощью последовательности операций, заканчивающейся `SHL RAX, 3`, перед добавлением к начальному адресу массива. (Если вам не сразу очевидно, что последовательность операций эквивалентна умножению на 24, не беспокойтесь. Такие последовательности кода обсуждаются в главе 20.) Поскольку `field1` является первым членом `struct`, дополнительное смещение для создания итогового адреса присваивания в `field1` не требуется.

Из этих фактов можно вывести размер элемента массива (24), число элементов массива (`120 / 24 = 5`) и то, что внутри каждого элемента массива по смещению 0 есть 4-байтовое (`dword`) поле. Этот короткий листинг не дает достаточно информации, чтобы делать какие-либо выводы о том, как оставшиеся 20 байтов внутри каждой структуры распределены по дополнительным полям. Размер массива можно еще легче вывести по той же формуле из листинга декомпилятора на рисунке 8-7 (`0x18` в шестнадцатеричном виде равно 24 в десятичном).

Decompile: main - (heap_struct_array_demo_x64_stripped)

```
1
2  undefined8 main(void)
3
4  {
5    void *pvVar1;
6
7    pvVar1 = malloc(120);
8    *(undefined4 *)((long)pvVar1 + 0x18) = 10;
9    return 0;
10 }
```

Рисунок 8-7. Представление `Decompiler` *функции с массивом структур, выделенным в куче*

Создание структур с помощью Ghidra

В предыдущей главе вы видели, как использовать возможности агрегирования массивов Ghidra, чтобы сворачивать длинные списки объявлений данных в одну строку дизассемблирования, представляющую массив. В следующих нескольких разделах рассматриваются средства Ghidra для повышения читаемости кода, который манипулирует структурами. Наша цель - уйти от загадочных ссылок на структуры вроде `[EDX + 10h]` к чему-то более читаемому, например `[EDX + ch8_struct.field_e]`.

Каждый раз, когда вы обнаруживаете, что программа манипулирует структурой данных, нужно решить, хотите ли вы включить имена полей структуры в свое дизассемблирование или можете

разобраться со всеми числовыми смещениями, разбросанными по листингу. В некоторых случаях Ghidra может распознать использование структуры, определенной как часть стандартной библиотеки C или Windows API, и использовать знание точной схемы структуры, чтобы преобразовать числовые смещения в символические имена полей. Это идеальный случай, поскольку он оставляет вам намного меньше работы. Мы вернемся к этому сценарию после того, как вы немного лучше поймете, как Ghidra в целом работает с определениями структур.

Создание новой структуры

Когда у Ghidra нет знания о расположении структуры, структуру можно создать, выбрав данные и используя контекстное меню по щелчку правой кнопкой мыши. Когда вы выбираете Data > Create Structure (или используете горячую клавишу `shift-[`), вы увидите окно Create Structure, показанное на рисунке 8-8. Поскольку вы выделили блок данных (который может быть определенным или неопределенным), Ghidra попытается идентифицировать существующие структуры, имеющие совпадающий формат или тот же размер. Вы можете выбрать одну из существующих структур в окне или создать новую структуру. В этом примере мы используем примерный код с глобально выделенной структурой, обсуждавшийся ранее, и создаем новую структуру с именем `ch8_struct`. Как только вы щелкнете `OK`, структура станет официальным типом в окне `Data Type Manager`, а информация будет распространена в другие окна `CodeBrowser`.

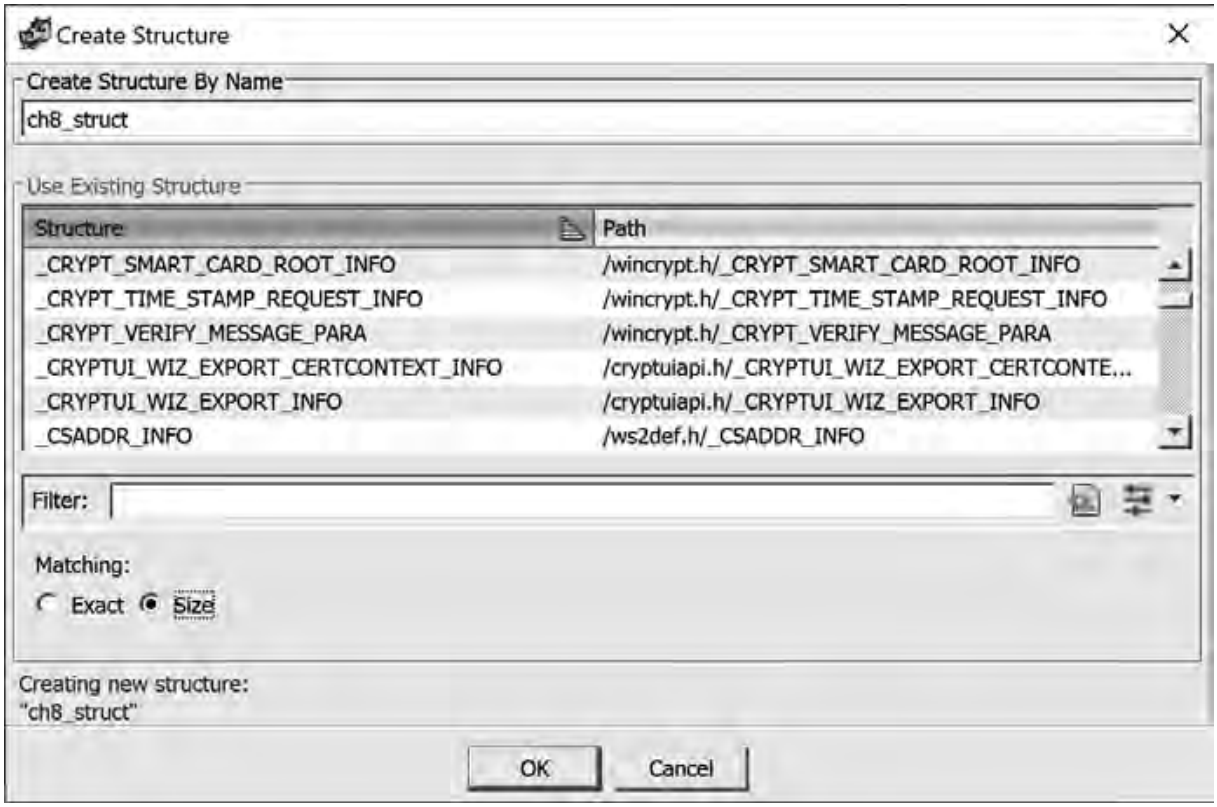


Рисунок 8-8. Окно Create Structure

Посмотрим на эффект этого создания в связанных окнах `CodeBrowser`, начиная с окна `Listing`. Как показано ранее в главе, дизассемблированный листинг дает мало подсказок о том, что перед вами может быть структура, потому что код изменяет ряд, казалось бы, несвязанных глобальных переменных:

```
001005fa  PUSH   RBP
001005fb  MOV    RBP,RSP
001005fe  MOV    dword ptr [DAT_00301020],10
00100608  MOV    word ptr [DAT_00301024],20
00100611  MOV    byte ptr [DAT_00301026],30
00100618  MOV    dword ptr [DAT_00301028],40
00100622  MOVSD  XMM0,qword ptr [DAT_001006c8]
0010062a  MOVSD  qword ptr [DAT_00301030],XMM0
00100632  MOV    EAX,0
00100637  POP    RBP
00100638  RET
```

Когда вы переходите к связанным элементам данных, выбираете диапазон (с `00301020` по `00301037`) и создаете связанную структуру, вы видите, что отдельные элементы данных в структуре теперь связаны со структурой с именем `ch8_struct_00301020`, и каждый элемент структуры имеет имя `field_`, объединенное с его смещением от первого элемента структуры.

```
00401035  POP    EBP
001005fb  MOV    RBP,RSP
001005fe  MOV    dword ptr [ch8_struct_00301020],10
00100608  MOV    word ptr [ch8_struct_00301020.field_0x4],20
00100611  MOV    byte ptr [ch8_struct_00301020.field_0x6],30
00100618  MOV    dword ptr [ch8_struct_00301020.field_0x8],40
00100622  MOVSD  XMM0,qword ptr [DAT_001006c8]
0010062a  MOVSD  qword ptr [ch8_struct_00301020.field_0x10],XMM0
00100632  MOV    EAX,0
00100637  POP    RBP
00100638  RET
```

Это лишь одно из окон, которое меняется при создании структуры. Вспомните, что окно `Decompiler` давало полезное предупреждение о том, что мы можем работать со структурой или массивом. После создания структуры предупреждение исчезает, а декомпилированный код больше напоминает исходный C-код, как показано на рисунке 8-9.

Decompile: main - (global_struct_demo_x64_stripped)

```
1
2  undefined8 main(void)
3
4  {
5    ch8_struct_00301020._0_4_ = 10;
6    ch8_struct_00301020._4_2_ = 20;
7    ch8_struct_00301020._6_1_ = 30;
8    ch8_struct_00301020._8_4_ = 40;
9    ch8_struct_00301020._16_8_ = 0x4049000000000000;
10   return 0;
11 }
```

Рисунок 8-9. Представление `Decompiler` *после создания* `struct`

Состояние union

`union` - это конструкция, похожая на структуру. Главное различие между структурами и объединениями состоит в том, что поля структуры имеют уникальные смещения и собственное выделенное пространство памяти, тогда как все поля `union` перекрывают друг друга, начиная со смещения 0. В результате все поля `union` совместно используют одно и то же пространство памяти. Окно Union Editor в Ghidra выглядит похоже на окно Structure Editor, и функциональность в основном та же.

Новая структура теперь также появляется как запись в окне `Data Type Manager` в `CodeBrowser`. На рисунке 8-10 показана новая запись в окне `Data Type Manager` и связанное окно, показывающее все использования `ch8_struct`.

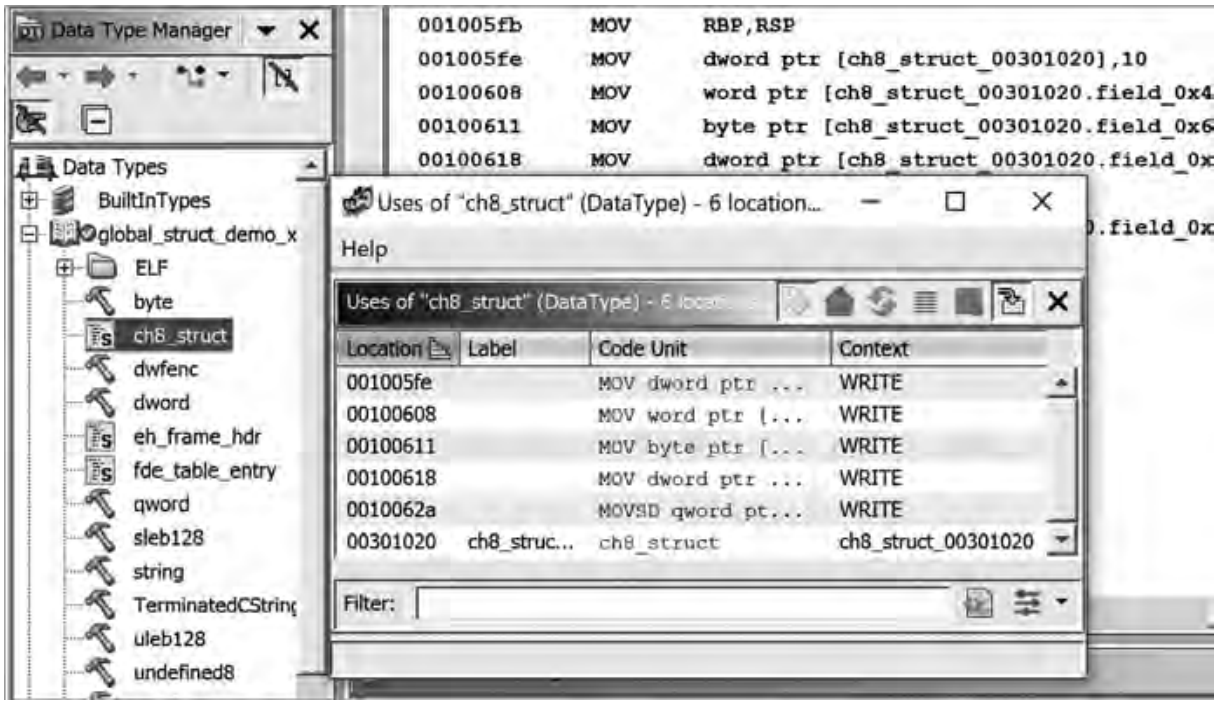


Рисунок 8-10. Новая объявленная структура в окнах `Data Type Manager` *и References*

Редактирование членов структуры

На этом этапе Ghidra представляет вновь созданную структуру как непрерывную коллекцию неопределенных байтов с перекрестными ссылками на каждом смещении, к которому обращается примерная программа, а не как коллекцию определенных типов данных (которые вы идентифицировали по размеру каждого элемента и способу его использования). Чтобы определить тип каждого поля, можно отредактировать структуру из окна `Listing`, щелкнув правой кнопкой мыши и выбрав соответствующий пункт Data. Кроме того, можно отредактировать структуру из `Data Type Manager`, дважды щелкнув структуру.

Если дважды щелкнуть вновь созданную структуру в окне `Data Type Manager` (показано на рисунке 8-10), откроется окно Structure Editor (показано на рисунке 8-11), где будут видны 24 элемента неопределенного типа, каждый длиной 1. Чтобы определить число, размеры и типы отдельных элементов внутри структуры, можно изучить дизассемблирование, а можно позволить листингу декомпилятора, показанному ранее на рисунке 8-9, дать ответы.

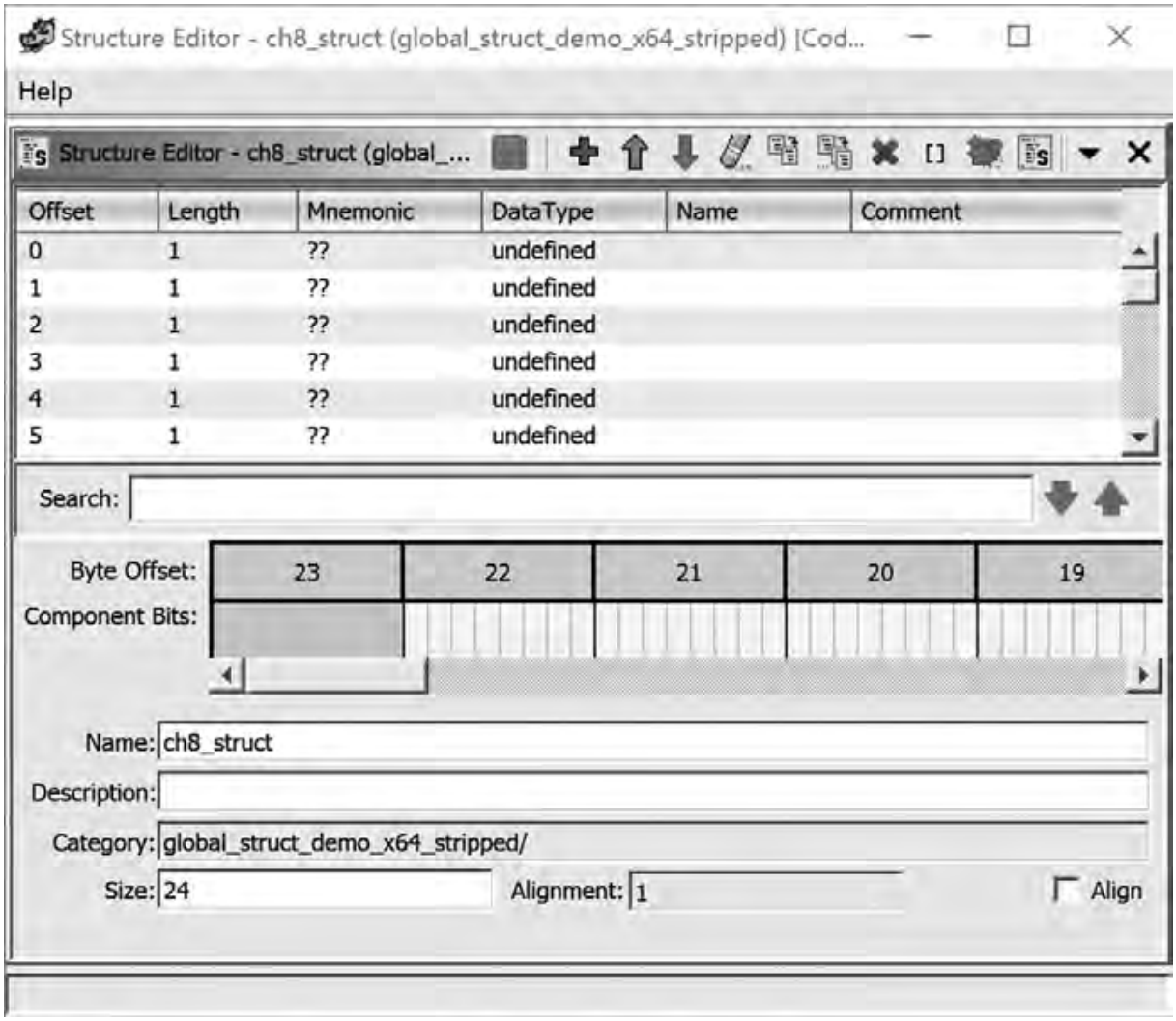


Рисунок 8-11. Окно Structure Editor

Исходный листинг декомпилятора, связанный с нашей вновь созданной структурой, показывает, что внутри одной и той же структуры `ch8_struct_00301020` используются пять элементов, с именами полей, содержащими два целых числа. Первое целое представляет смещение от базового адреса структуры. Второе показывает число используемых байтов, что является хорошим индикатором размера элемента. Используя эту информацию (и несколько осмысленных имен полей), можно обновить окно `Structure Editor`, как показано на рисунке 8-12. Полоса прокрутки `Byte Offset/Component Bits` в `Structure Editor` предоставляет визуальное представление структуры. Когда структура редактируется, окно `Decompiler` (слева на рисунке 8-12), окно `Listing` и другие связанные окна также обновляются.

Поскольку `field_c` является символом, декомпилятор преобразовал целое число 30 в ASCII-символ, представленный числом 30 (`0x1e`), который является непечатаемым управляющим символом (RS). В `Structure Editor` байты заполнения (обозначенные мнемоникой `??`) включены для правильного выравнивания полей, а смещения каждого поля и общий размер структуры (24 байта) совпадают со значениями, виденными в предыдущих примерах.

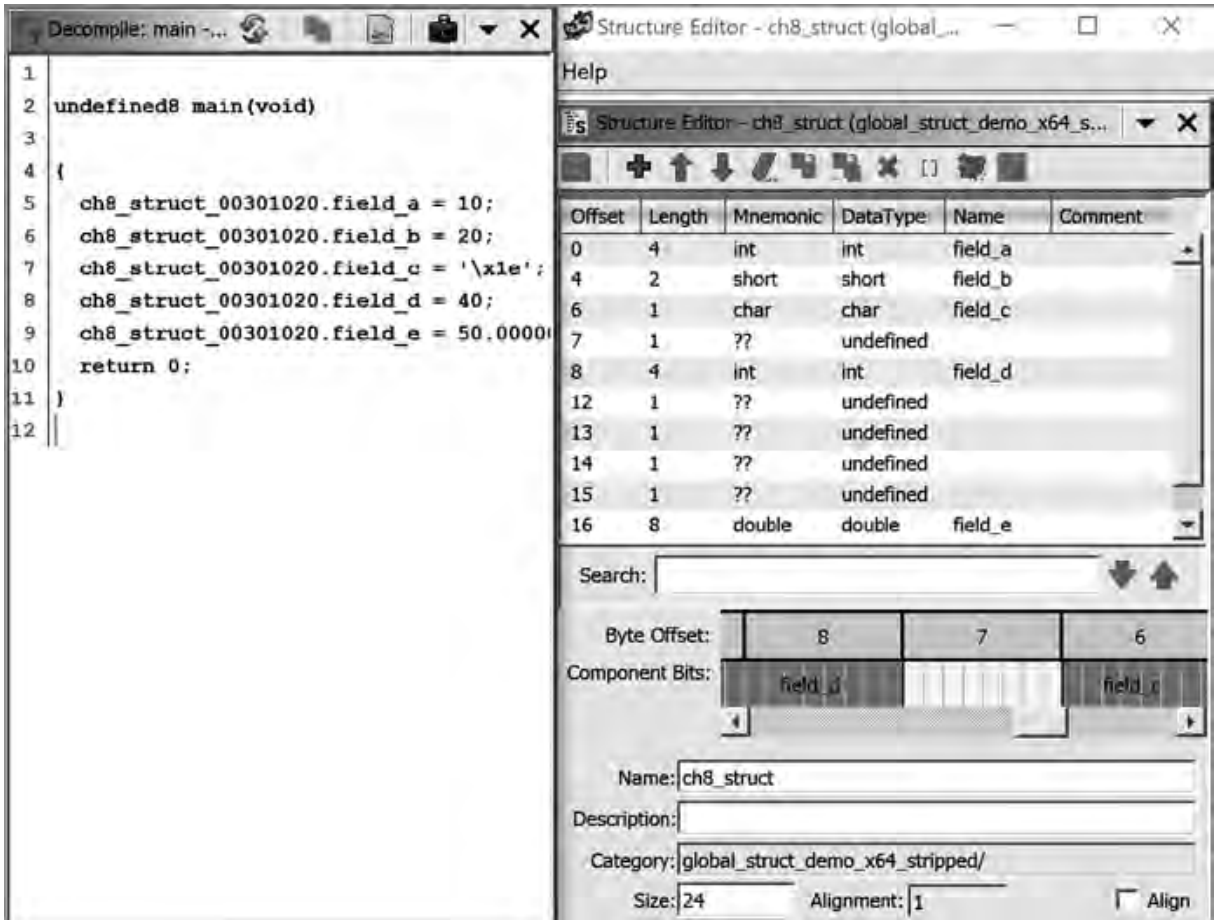


Рисунок 8-12. Окна `Decompiler` *и Structure Editor после редактирования структуры*

Применение схем структур

Вы видели, как использовать существующие определения структур и создавать новые, чтобы связывать существующую память с конкретной схемой памяти. Вы также видели, как эта связь распространяется по окнам `CodeBrowser`, делая содержимое понятнее. Расплывчатые обращения к памяти вроде `[EBX+8]` становятся более читаемыми при преобразовании числовых смещений структуры в символические ссылки вроде `[EBX+ch8_struct.field_d]`, особенно потому, что символическим ссылкам можно дать осмысленные имена. Использование Ghidra иерархической нотации ясно показывает, к какому именно типу структуры и к какому именно полю внутри этой структуры выполняется доступ.

Библиотека известных схем структур Ghidra заполнена информацией, собранной путем разбора распространенных заголовочных файлов C. Схема структуры определяет ее общий размер, имя и размер каждого поля и начальное смещение каждого поля внутри структуры. Схемы структур можно использовать даже без связанного содержимого в секции данных, что особенно полезно при работе с указателями на структуры.

Каждый раз, когда вы встречаете обращение к памяти вида `[reg+N]` (например, `[RAX+0x12]`), где `reg` - имя регистра, а `N` - небольшая константа, `reg` используется как указатель, а `N` представляет смещение внутри памяти, на которую указывает `reg`. Это распространенный шаблон доступа к членам структуры, где `reg` указывает на начало структуры, а `N` выбирает поле по смещению `N` структуры. При некоторых обстоятельствах Ghidra с вашей помощью может очистить такой тип обращения к памяти так, чтобы он отражал и тип структуры, на которую указывает регистр, и

конкретное поле внутри этой структуры, к которому выполняется обращение.

Рассмотрим 32-разрядную версию примера из начала главы, где мы запрашивали HTTP-страницу с сервера. Запрос выполняется функцией с именем `get_page`. В этой версии бинарного файла Ghidra утверждает, что функция получает три параметра, размещенных в стеке. Эти параметры отображаются в окне `Listing` следующим образом:

```
undefined get_page(undefined4 param_1, undefined param_2...
   undefined     AL:1              <RETURN>
   undefined4    Stack[0x4]:4      param_1
   undefined     Stack[0x8]:1      param_2
   undefined4    Stack[0xc]:4      param_3
```

Окно `Decompiler` показывает, что `param_3` используется с некоторыми смещениями в вызове `connect`:

```
iVar1=connect(local_14,*(sockaddr **)(param_3+20),*(socklen_t*)(param_3+16));
```

Проследив последовательность вызовов и возвращаемые значения вызываемых функций, можно заключить, что `param_3` является указателем на `struct addrinfo`, и изменить тип `param_3` на `addrinfo*` (используя `ctrl-L` из окна `Listing` или `Decompiler`). Декомпилированная инструкция с использованием `param_3` будет заменена гораздо более информативной инструкцией, показанной здесь:

```
iVar1 = connect(local_14, param_3->ai_addr, param_3->ai_addrlen);
```

Вы видите, что арифметика указателей заменена ссылками на поля структуры. Арифметика указателей в исходном коде редко говорит сама за себя. Любые усилия, потраченные на обновление типов данных для переменных программы, окупятся. Вы сэкономите коллегам время, которое потребовалось бы, чтобы самостоятельно вывести тип `param_3`, и поблагодарите себя, вернувшись после двух недель на пляже, за то, что не нужно заново анализировать код, чтобы снова узнать тип той переменной, который вы забыли обновить.

Краткое введение в обратную разработку C++

Классы C++ являются объектно-ориентированными расширениями структур C, поэтому несколько логично завершить обсуждение структур данных обзором особенностей скомпилированного кода C++. Подробное рассмотрение C++ выходит за рамки этой книги. Здесь мы попытаемся охватить главные моменты и несколько различий между компилятором C++ Microsoft и GNU `g++`.

Помните, что прочное фундаментальное понимание языка C++ сильно поможет вам понимать скомпилированный C++. Объектно-ориентированные концепции, такие как наследование и полиморфизм, достаточно трудны для освоения на уровне исходного кода. Попытка погрузиться в эти концепции на уровне ассемблера без их понимания на уровне исходного кода может оказаться упражнением в разочаровании.

Указатель this

Указатель `this` доступен во всех нестатических функциях-членах C++. Всякий раз, когда такая функция вызывается, `this` инициализируется так, чтобы указывать на объект, использованный для вызова функции. Рассмотрим следующие вызовы функций в C++:

```
// object1, object2 и *p_obj имеют один и тот же тип.
object1.member_func();
object2.member_func();
p_obj->member_func();
```

В трех вызовах member_func указатель this принимает значения &object1, &object2 и p_obj соответственно.

Проще всего рассматривать this как скрытый первый параметр, передаваемый во все нестатические функции-члены. Как обсуждалось в главе 6, компилятор Microsoft C++ использует соглашение о вызове thiscall и передает this в регистре ECX (x86) или RCX (x86-x64). Компилятор GNU g++ обращается с this точно так же, как если бы это был первый (самый левый) параметр нестатических функций-членов. В 32-разрядном Linux x86 адрес объекта, используемого для вызова функции, помещается на вершину стека перед вызовом функции. В Linux x86-64 this передается в первом регистровом параметре, RDI.

С точки зрения обратной разработки перемещение адреса в регистр ECX непосредственно перед вызовом функции является вероятным индикатором двух вещей. Во-первых, файл был скомпилирован компилятором C++ Microsoft. Во-вторых, функция, возможно, является функцией-членом. Когда один и тот же адрес передается в две или более функции, можно заключить, что все эти функции принадлежат одной и той же иерархии классов.

Внутри функции использование ECX до его инициализации подразумевает, что вызывающая сторона должна была инициализировать ECX (вспомните обсуждение живости из раздела "Параметры на базе регистров" на странице 113), и является возможным признаком того, что функция является функцией-членом (хотя функция может просто использовать соглашение о вызове fastcall). Более того, когда функция-член передает this дополнительным функциям, можно вывести, что эти функции тоже являются членами того же класса.

Для кода, скомпилированного GNU g++, вызовы функций-членов выделяются несколько слабее, потому что this выглядит очень похоже на любой другой первый параметр. Однако любую функцию, которая не принимает указатель в качестве первого аргумента, определенно можно исключить как функцию-член.

Виртуальные функции и vftable

Виртуальные функции обеспечивают полиморфное поведение в программах C++. Для каждого класса (или подкласса через наследование), содержащего виртуальные функции, компилятор генерирует таблицу, содержащую указатели на каждую виртуальную функцию класса. Такие таблицы называются vftable (также vtable). Каждый экземпляр класса, содержащего виртуальные функции, получает дополнительный элемент данных, который указывает на vftable класса. Указатель vftable выделяется как первый элемент данных внутри экземпляра класса, и когда объект создается во время выполнения, его функция-конструктор устанавливает указатель vftable так, чтобы он указывал на соответствующую vftable. Когда этот объект вызывает виртуальную функцию, правильная функция выбирается путем поиска в vftable объекта. Таким образом, vftable - это базовый механизм, обеспечивающий разрешение вызовов виртуальных функций во время выполнения.

Несколько примеров помогут прояснить использование vftable. Рассмотрим следующие определения классов C++:

```
class BaseClass {
    public:
        BaseClass();
        virtual void vfunc1() = 0;
        virtual void vfunc2();
        virtual void vfunc3();
        virtual void vfunc4();
    private:
        int x;
        int y;
};

class SubClass : public BaseClass {
    public:
        SubClass();
        virtual void vfunc1();
        virtual void vfunc3();
        virtual void vfunc5();
    private:
        int z;
};
```

В этом случае `SubClass` наследуется от `BaseClass`. `BaseClass` содержит четыре виртуальные функции, тогда как `SubClass` содержит пять (четыре из `BaseClass`, две из которых он переопределяет, плюс новую `vfunc5`). Внутри `BaseClass` функция `vfunc1` является чисто виртуальной функцией, что обозначается `= 0` в ее объявлении. Чисто виртуальные функции не имеют реализации в объявляющем их классе и должны быть переопределены в подклассе, прежде чем класс будет считаться конкретным. Иными словами, функции с именем `BaseClass::vfunc1` нет, и пока подкласс не предоставит реализацию, объекты нельзя создавать. `SubClass` предоставляет такую реализацию, поэтому объекты `SubClass` можно создавать. В объектно-ориентированных терминах `BaseClass::vfunc1` является абстрактной функцией, что делает `BaseClass` абстрактным базовым классом (то есть неполным классом, который нельзя напрямую создать, поскольку ему не хватает реализации по крайней мере одной функции).

На первый взгляд `BaseClass` содержит два элемента данных, а `SubClass` - три элемента данных. Однако вспомните, что любой класс, содержащий виртуальные функции явно или потому, что они унаследованы, также содержит указатель vftable. В результате скомпилированная реализация `BaseClass` имеет три элемента данных, тогда как созданные объекты `SubClass` имеют четыре элемента данных. В каждом случае первый элемент данных - указатель vftable. Внутри `SubClass` указатель vftable фактически наследуется от `BaseClass`, а не вводится специально для `SubClass`. Это видно в упрощенной схеме памяти на рисунке 8-13, где один объект `SubClass` был динамически выделен. Во время создания объекта указатель vftable нового объекта инициализируется так, чтобы указывать на правильную vftable (в этом случае `SubClass`).

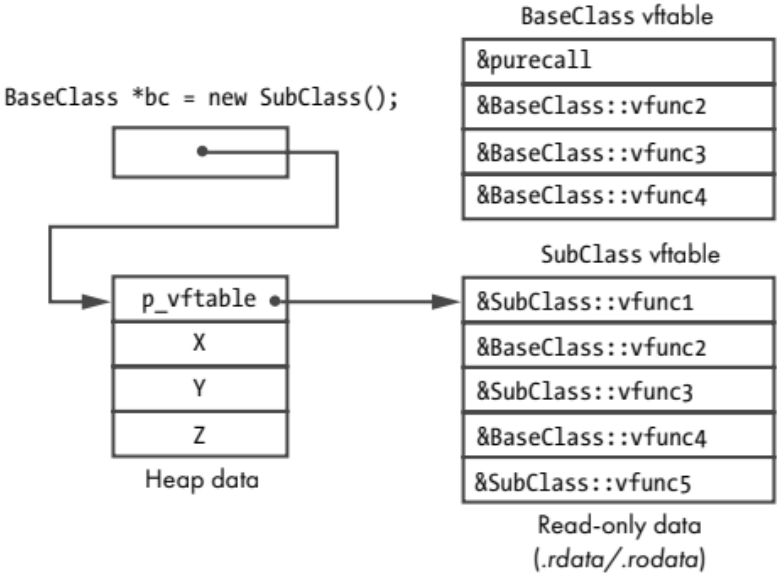


Figure 8-13: A simple vftable layout

The vftable for `SubClass` contains two pointers to functions belonging to `BaseClass` (`BaseClass::vfunc2` and `BaseClass::vfunc4`) because `SubCl` does not override either of these functions and instead inherits them f `BaseClass`. The vftable for `BaseClass` shows how pure virtual functions ar handled. Because there is no implementation for the pure virtual func `BaseClass::vfunc1`, no address is available to store in the `BaseClass` vftabl for `vfunc1`. In such cases, compilers insert the address of an error-hand function, dubbed `purecall` in Microsoft libraries and `__cxa_pure_virtual`

Рисунок 8-13. Простая схема vftable

Vftable для `SubClass` содержит два указателя на функции, принадлежащие `BaseClass` (`BaseClass::vfunc2` и `BaseClass::vfunc4`), потому что `SubClass` не переопределяет ни одну из этих функций, а наследует их от `BaseClass`. Vftable для `BaseClass` показывает, как обрабатываются чисто виртуальные функции. Поскольку реализации для чисто виртуальной функции `BaseClass::vfunc1` нет, нет и адреса, который можно сохранить в слот vftable `BaseClass` для `vfunc1`. В таких случаях компиляторы вставляют адрес функции обработки ошибки, называемой `purecall` в библиотеках Microsoft и `__cxa_pure_virtual` в библиотеках GNU. Теоретически эти функции никогда не должны вызываться, но если это все же происходит, они вызывают аварийное завершение программы.

При манипулировании классами в Ghidra необходимо учитывать указатель vftable. Поскольку классы C++ являются расширениями структур C, можно использовать возможности определения структур Ghidra для задания схемы классов C++. В полиморфных классах нужно включать указатель vftable как первое поле внутри класса, а также учитывать указатель vftable в общем размере объекта. Это наиболее заметно при наблюдении динамического выделения объекта с использованием оператора `new`, где значение размера, переданное в `new`, включает пространство, необходимое всем явно объявленным полям класса (и любых суперклассов), а также любое пространство, требуемое для указателя vftable.

В следующем примере объект SubClass создается динамически, а его адрес сохраняется в указателе BaseClass. Затем указатель передается функции (call_vfunc), которая использует указатель для вызова vfunc3:

```
void call_vfunc(BaseClass *bc) {
    bc->vfunc3();
}

int main() {
    BaseClass *bc = new Subclass();
    call_vfunc(bc);
}
```

Поскольку vfunc3 является виртуальной функцией, а bc указывает на объект SubClass, компилятор должен гарантировать, что будет вызвана SubClass::vfunc3. Следующее дизассемблирование 32-разрядной версии call_vfunc, скомпилированной Microsoft C++, демонстрирует, как разрешается вызов виртуальной функции:

```
      undefined __cdecl call_vfunc(int * bc)
         undefined     AL:1              <RETURN>
         int *         Stack[0x4]:4      bc
004010a0  PUSH   EBP
004010a1  MOV    EBP,ESP
004010a3  MOV    EAX,dword ptr [EBP + bc]
004010a6  MOV    EDX,dword ptr [EAX]
004010a8  MOV    ECX,dword ptr [EBP + bc]
004010ab  MOV    EAX,dword ptr [EDX + 8]
004010ae  CALL   EAX
004010b0  POP    EBP
004010b1  RET
```

Указатель vftable (адрес vftable SubClass) считывается из структуры и сохраняется в EDX. Затем указатель this перемещается в ECX. После этого vftable индексируется, чтобы считать третий указатель (в данном случае адрес SubClass::vfunc3) в регистр EAX. Наконец, вызывается виртуальная функция.

Операция индексирования vftable выглядит очень похоже на операцию ссылки на структуру. Фактически она ничем не отличается, и можно определить новые структуры для класса и его vftable (щелкнув правой кнопкой мыши в окне Data Type Manager), а затем использовать определенные структуры (см. рисунок 8-14), чтобы сделать дизассемблирование и декомпиляцию более читаемыми.

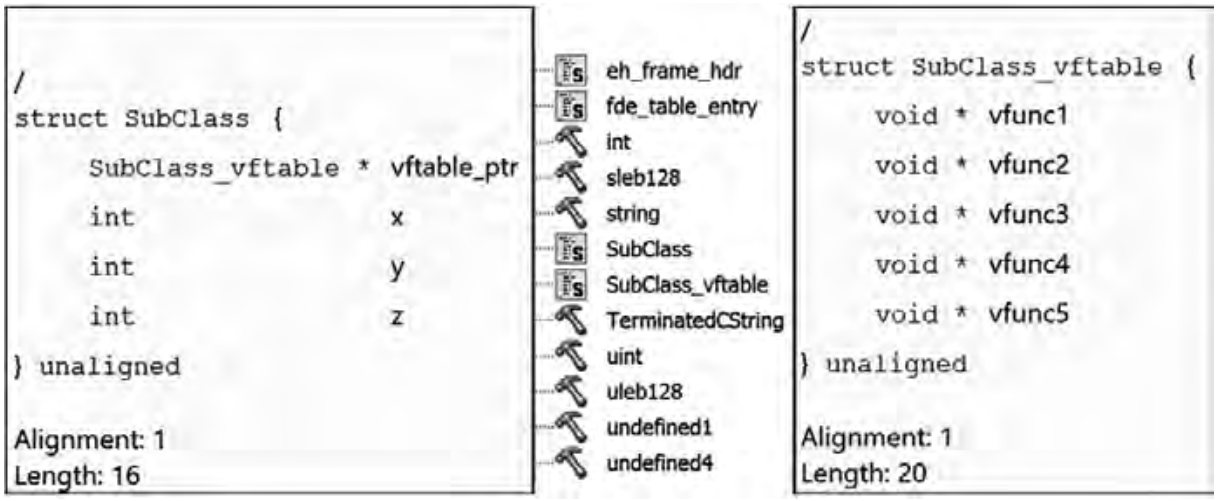


Рисунок 8-14. Окно Data Manager с новыми SubClass *и* SubClass_vftable

Окно Decompiler со ссылками на новые структуры показано на рисунке 8-15.

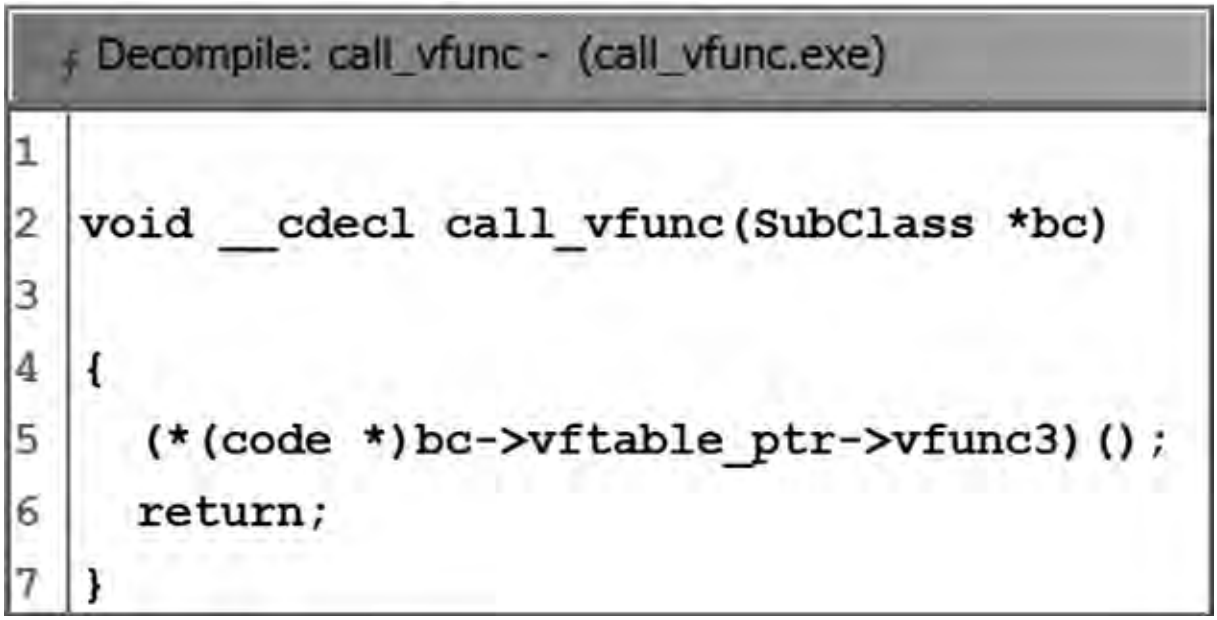


Рисунок 8-15. Окно `Decompiler`*, отражающее определенные структуры для* `SubClass`

Vftable класса напрямую упоминается только в двух обстоятельствах: внутри конструктора(ов) класса и деструктора. Когда вы находите vftable, можно использовать возможности перекрестных ссылок данных Ghidra (см. главу 9), чтобы быстро найти все конструкторы и деструкторы связанного класса.

Жизненный цикл объекта

Понимание механизма, с помощью которого объекты создаются и уничтожаются, может помочь выявлять иерархии объектов и отношения вложенных объектов, а также быстро идентифицировать функции-конструкторы и функции-деструкторы классов.

> **Что такое конструктор?**
>
> Функция-конструктор класса - это функция инициализации, вызываемая при создании нового объекта этого класса. Конструкторы дают возможность инициализировать переменные внутри класса. Обратная конструктору функция, деструктор, вызывается, когда объект выходит из области видимости или когда динамически выделенный объект явно удаляется. Функции-деструкторы выполняют действия очистки, например освобождают ресурсы вроде открытых файловых дескрипторов и динамически выделенной памяти. Правильно написанные деструкторы уменьшают вероятность утечек памяти.

Класс хранения объекта определяет, когда вызывается его конструктор.[1] Для глобальных и статически выделенных объектов (статический класс хранения) конструкторы вызываются во время запуска программы до входа в функцию `main` программы. Конструкторы объектов, выделенных в стеке (автоматический класс хранения), вызываются, когда объект входит в область видимости внутри функции, в которой он объявлен. Во многих случаях это произойдет сразу при входе в функцию, где он объявлен. Однако, когда объект объявлен внутри вложенного блочного оператора, его конструктор не вызывается до входа в этот блок, если вход в него вообще происходит. Когда объект динамически выделяется в куче программы, его создание является двухшаговым процессом: оператор `new` вызывается для выделения памяти объекта, а затем конструктор вызывается для инициализации объекта. Microsoft C++ гарантирует, что результат `new` не равен

null, перед вызовом конструктора, но GNU g++ этого не делает.

Когда конструктор выполняется, происходит следующая последовательность действий:

1. Если у класса есть суперкласс, вызывается конструктор суперкласса.
2. Если у класса есть виртуальные функции, указатель vftable инициализируется так, чтобы указывать на vftable класса. Это может перезаписать указатель vftable, который был инициализирован в конструкторе суперкласса, что как раз является желаемым поведением.
3. Если у класса есть какие-либо элементы данных, которые сами являются объектами, вызывается конструктор для каждого из этих элементов данных.
4. Наконец, выполняется конструктор класса. Это код конструктора C++, заданный программистом класса.

С точки зрения программиста конструкторы не задают тип возврата и не позволяют возвращать значение. Некоторые компиляторы фактически возвращают this как результат, который они могут далее использовать в вызывающей стороне, но это деталь реализации компилятора, и программисты C++ не могут получить доступ к возвращенному значению.

Деструкторы, как следует из названия, вызываются в конце времени жизни объекта. Для глобальных и статических объектов деструкторы вызываются кодом очистки, который выполняется после завершения функции main. Деструкторы объектов, выделенных в стеке, вызываются по мере выхода объектов из области видимости. Деструкторы объектов, выделенных в куче, вызываются через оператор delete непосредственно перед освобождением памяти, выделенной объекту.

> **Что такое оператор new?**
>
> Оператор new используется для динамического выделения памяти в C++ почти так же, как malloc используется в C. Он используется для выделения памяти из кучи и позволяет программе запрашивать пространство по мере необходимости во время выполнения. Оператор new встроен в язык C++, тогда как malloc является лишь функцией стандартной библиотеки. Помните, что C является подмножеством C++, поэтому в программе C++ можно увидеть и то и другое. Самое заметное различие между malloc и new состоит в том, что вызовы new для объектных типов приводят к неявному вызову конструктора объекта, тогда как память, возвращенная malloc, не инициализируется до того, как становится доступной вызывающей стороне.

Действия, выполняемые деструкторами, имитируют действия конструкторов, за исключением того, что выполняются примерно в обратном порядке:

1. Если у класса есть виртуальные функции, указатель vftable для объекта восстанавливается так, чтобы указывать на vftable соответствующего класса. Это требуется на случай, если подкласс перезаписал указатель vftable в рамках процесса своего создания.
2. Выполняется код деструктора, заданный программистом.
3. Если у класса есть какие-либо элементы данных, которые сами являются объектами, выполняется деструктор для каждого из этих членов.
4. Наконец, если у объекта есть суперкласс, вызывается деструктор суперкласса.

Понимая, когда вызываются конструкторы и деструкторы суперклассов, можно проследить иерархию наследования объекта через цепочку вызовов связанных функций его суперклассов.

> **Кажется, вы перегружены**
>
> Перегруженные функции - это функции с одинаковым именем, но разными параметрами. C++ требует, чтобы каждая версия перегруженной функции отличалась от любой другой версии последовательностью и/или количеством типов параметров, которые принимает функция. Иными словами, хотя они имеют одно имя функции, каждый прототип функции

должен быть уникален, а каждое тело перегруженной функции можно однозначно идентифицировать внутри дизассемблированного бинарного файла. Это не следует путать с функциями вроде `printf`, которые принимают переменное число аргументов, но связаны с одним телом функции.

Искажение имен

Искажение имен, также называемое декорированием имен, - это механизм, который компиляторы C++ используют, чтобы различать перегруженные версии функции. Чтобы генерировать уникальные внутренние имена для перегруженных функций, компиляторы декорируют имя функции дополнительными символами, которые кодируют различные сведения о функции: пространство имен, которому принадлежит функция (или владеющий ей класс), если оно есть; класс, которому принадлежит функция, если он есть; и последовательность параметров (тип и порядок), требуемую для вызова функции.

Искажение имен является деталью реализации компилятора для программ C++ и, как таковое, не является частью спецификации языка C++. Неудивительно, что поставщики компиляторов разработали собственные, часто несовместимые соглашения об искажении имен. К счастью, Ghidra понимает соглашения об искажении имен, применяемые компилятором C++ Microsoft и GNU `g++` версии 3 (и новее), а также некоторыми другими компиляторами. Ghidra предоставляет имена вида `FUN_address` вместо искаженного имени. Искаженные имена действительно несут ценную информацию о сигнатуре каждой функции, и Ghidra включает эту информацию в окно `Symbol Table`, а также распространяет ее в дизассемблирование и другие связанные окна. (Чтобы определить сигнатуру функции без искаженного имени, вам, возможно, пришлось бы провести трудоемкий анализ данных, входящих в функцию и выходящих из нее.)

Идентификация типов во время выполнения

C++ предоставляет операторы для определения (`typeid`) и проверки (`dynamic_cast`) типа данных объекта во время выполнения. Чтобы поддерживать эти операции, компиляторы C++ должны встраивать в бинарный файл программы специфичную для типа информацию для каждого полиморфного класса. Когда операция `typeid` или `dynamic_cast` выполняется во время выполнения, библиотечные процедуры обращаются к специфичной для типа информации, чтобы определить точный тип времени выполнения полиморфного объекта, на который дана ссылка. К сожалению, как и в случае с искажением имен, Runtime Type Identification (RTTI) является деталью реализации компилятора, а не вопросом языка, и стандартного способа, которым компиляторы реализуют возможности RTTI, нет.

Мы кратко рассмотрим сходства и различия между реализациями RTTI компилятора C++ Microsoft и GNU `g++`. В частности, опишем, как находить информацию RTTI и как из нее узнавать имя класса, к которому эта информация относится. Читателям, желающим более подробного обсуждения реализации RTTI Microsoft, следует обратиться к ссылкам, перечисленным в конце этой главы. В частности, эти материалы подробно описывают, как обходить иерархию наследования класса, включая трассировку этой иерархии при использовании множественного наследования.

Рассмотрим следующую простую программу, использующую полиморфизм:

```
class abstract_class {
    public:
        virtual int vfunc() = 0;
};

class concrete_class : public abstract_class {
    public:
        concrete_class(){};
        int vfunc();
};

int concrete_class::vfunc() {return 0;}

void print_type(abstract_class *p) {
    cout << typeid(*p).name() << endl;
}

int main() {
    abstract_class *sc = new concrete_class();
    print_type(sc);
}
```

Функция `print_type` печатает тип объекта, на который указывает указатель `p`. В этом случае она должна напечатать "`concrete_class`", поскольку объект `concrete_class` создается в функции `main`. Как `print_type` и, точнее, `typeid` узнает, на объект какого типа указывает `p`?

Ответ удивительно прост. Поскольку каждый полиморфный объект содержит указатель на vftable, компиляторы используют этот факт, размещая информацию о типе класса рядом с vftable класса. Конкретно компилятор помещает непосредственно перед vftable класса указатель на структуру, содержащую информацию о классе, которому принадлежит vftable. В коде GNU g++ этот указатель указывает на структуру `type_info`, содержащую указатель на имя класса. В коде Microsoft C++ указатель указывает на структуру Microsoft `RTTICompleteObjectLocator`, которая, в свою очередь, содержит указатель на структуру `TypeDescriptor`. Структура `TypeDescriptor` содержит символьный массив, задающий имя полиморфного класса.

Информация RTTI требуется только в программах C++, использующих оператор `typeid` или `dynamic_cast`. Большинство компиляторов предоставляют параметры для отключения генерации RTTI в бинарных файлах, которым она не нужна; поэтому вас не должно удивлять, если вы встретите скомпилированные бинарные файлы без информации RTTI, даже если vftable присутствуют.

Для программ C++, собранных компилятором C++ Microsoft, Ghidra содержит анализатор RTTI, включенный по умолчанию и способный идентифицировать структуры RTTI Microsoft, аннотировать эти структуры (если они присутствуют) в дизассемблированном листинге и использовать имена классов, восстановленные из этих структур RTTI, в папке `Classes` окна `Symbol Tree`. У Ghidra нет анализатора RTTI для бинарных файлов, не относящихся к Windows. Когда Ghidra встречает бинарный файл не для Windows, из которого не удалены символы, если она понимает схему искажения имен, примененную в бинарном файле, то Ghidra использует доступную информацию имен для заполнения папки `Classes` окна `Symbol Tree`. Если из бинарного файла не для Windows были удалены символы, Ghidra не сможет автоматически восстановить имена классов или идентифицировать vftable либо информацию RTTI.

Отношения наследования

Разобрать отношения наследования можно с помощью конкретной реализации RTTI компилятора, но RTTI может отсутствовать, когда программа не использует операторы `typeid` или `dynamic_cast`. Если информации RTTI нет, какие приемы можно использовать, чтобы определить отношения наследования между классами C++?

Самый простой способ определить иерархию наследования - наблюдать цепочку вызовов конструкторов суперклассов, которые вызываются при создании объекта. Самое большое препятствие для этого приема - использование встраиваемых конструкторов. В C/C++ функция, объявленная как `inline`, обычно рассматривается компилятором как макрос, и код функции разворачивается вместо явного вызова функции. Встраиваемые функции скрывают факт использования функции, поскольку инструкция вызова на языке ассемблера не будет сгенерирована. Из-за этого трудно понять, что конструктор суперкласса на самом деле был вызван.

Анализ и сравнение vftable также может раскрывать отношения наследования. Например, сравнивая vftable, показанные на рисунке 8-14, мы замечаем, что vftable для `SubClass` содержит два тех же указателя, которые появляются в vftable для `BaseClass`, и заключаем, что `BaseClass` и `SubClass` должны быть как-то связаны. Чтобы понять, какой из них является базовым классом, а какой подклассом, можно применять следующие правила, по отдельности или в сочетании:

- Когда две vftable содержат одинаковое число записей, два соответствующих класса могут участвовать в отношении наследования.
- Когда vftable класса X содержит больше записей, чем vftable класса Y, класс X может быть подклассом класса Y.
- Когда vftable класса X содержит записи, которые также встречаются в vftable класса Y, должно существовать одно из следующих отношений: X является подклассом Y, Y является подклассом X, или X и Y оба являются подклассами общего суперкласса Z.
- Когда vftable класса X содержит записи, которые также встречаются в vftable класса Y, и vftable класса X содержит по крайней мере одну запись `purecall`, которой нет в соответствующей записи vftable класса Y, тогда класс Y, вероятно, является подклассом класса X.

Хотя предыдущий список отнюдь не является исчерпывающим, мы можем использовать эти правила, чтобы вывести отношение между `BaseClass` и `SubClass` на рисунке 8-14. В этом случае применимы три последних правила, но именно последнее правило на основе одного лишь анализа vftable приводит нас к выводу, что `SubClass` наследуется от `BaseClass`.

Ссылки по обратной разработке C++

Существует несколько отличных материалов по обратной разработке скомпилированного C++.[2] Хотя многие детали в каждой из этих статей относятся конкретно к программам, скомпилированным компилятором C++ Microsoft, многие концепции в равной степени применимы к программам, скомпилированным другими компиляторами C++.

Резюме

Вы можете ожидать встречи со сложными типами данных почти во всех программах, кроме самых тривиальных. Понимание того, как выполняется доступ к данным внутри структур данных, и умение распознавать подсказки о расположении этих структур данных являются важнейшими навыками обратной разработки. Ghidra предоставляет широкий набор возможностей, специально предназначенных для работы со структурами данных. Знакомство с этими возможностями существенно повысит вашу способность понимать, какими данными манипулирует программа, и тратить больше времени на понимание того, как и почему эти данные обрабатываются. В следующей главе мы продолжим обсуждение базовых возможностей Ghidra и подробно рассмотрим перекрестные ссылки.

Сноски

[1] [назад] Класс хранения переменной грубо определяет ее время жизни во время выполнения программы. Два самых распространенных класса хранения в C - static и automatic. Пространство хранения статических переменных существует на протяжении всей программы. Автоматические переменные связаны с вызовами функций и существуют только в течение конкретного вызова функции.
[2] [назад] См. статью Игоря Скочинского "Reversing Microsoft Visual C++ Part II: `Classes`, Methods and RTTI", доступную по адресу http://www.openrce.org/articles/full_view/23, и работу Paul Vincent Sabanal и Mark Vincent Yason "Reversing C++", доступную по адресу http://www.blackhat.com/presentations/bh-dc-07/Sabanal_Yason/Paper/bh-dc-07-Sabanal_Yason-WP.pdf.

Глава 9. Перекрестные ссылки

Два распространенных вопроса, которые возникают при обратной разработке бинарного файла: "Откуда вызывается эта функция?" и "Какие функции обращаются к этим данным?" Эти и другие похожие вопросы стремятся идентифицировать и каталогизировать ссылки к различным ресурсам программы и от них. Следующие два примера показывают полезность таких вопросов.

Пример 1

Просматривая большое число ASCII-строк в конкретном бинарном файле, вы видите строку, которая кажется особенно подозрительной: "Заплатите в течение 72 часов, иначе ключ восстановления будет уничтожен, а ваши данные навсегда останутся зашифрованными." Сама по себе эта строка является лишь косвенным свидетельством. Она никоим образом не подтверждает, что бинарный файл обладает возможностью или намерением выполнить атаку шифровальщика-вымогателя. Ответ на вопрос "Где эта строка упоминается в бинарном файле?" помог бы быстро отследить местоположение(я) программы, где используется эта строка. Эта информация, в свою очередь, должна помочь найти любой связанный код шифровальщика-вымогателя, использующий строку, или показать, что в данном контексте строка безвредна.

Пример 2

Вы нашли функцию, содержащую стековый буфер, который можно переполнить, что, возможно, приведет к эксплуатации программы, и хотите определить, действительно ли это возможно. Если вы хотите разработать и продемонстрировать эксплойт, функция бесполезна для вас, пока вы не сможете заставить ее выполниться. Это приводит к вопросу "Какие функции вызывают эту уязвимую функцию?", а также к дополнительным вопросам о природе данных, которые эти функции могут передавать уязвимой функции. Эту линию рассуждений нужно продолжать по мере движения назад по потенциальным цепочкам вызовов, пока вы не найдете такую, на которую можете повлиять, чтобы показать эксплуатируемость переполнения.

Основы ссылок

Ghidra может помочь анализировать оба этих случая (и многие другие) с помощью широких механизмов отображения и доступа к информации о ссылках. В этой главе мы обсудим типы ссылок, которые предоставляет Ghidra, инструменты для доступа к информации о ссылках и способы интерпретации этой информации. В главе 10 мы используем графовые возможности Ghidra, чтобы рассмотреть визуальные представления отношений ссылок.

Все ссылки подчиняются одним и тем же общим правилам движения. С каждой ссылкой связано понятие направления. Все ссылки выполняются от одного адреса к другому адресу. Если вы знакомы с теорией графов, можно думать об адресах как об узлах (или вершинах) ориентированного графа, а о ссылках - как о ребрах, которые идентифицируют направленные соединения между узлами. Рисунок 9-1 дает краткое напоминание базовой терминологии графов. В этом простом графе три узла - A, B и C - соединены двумя ориентированными ребрами.

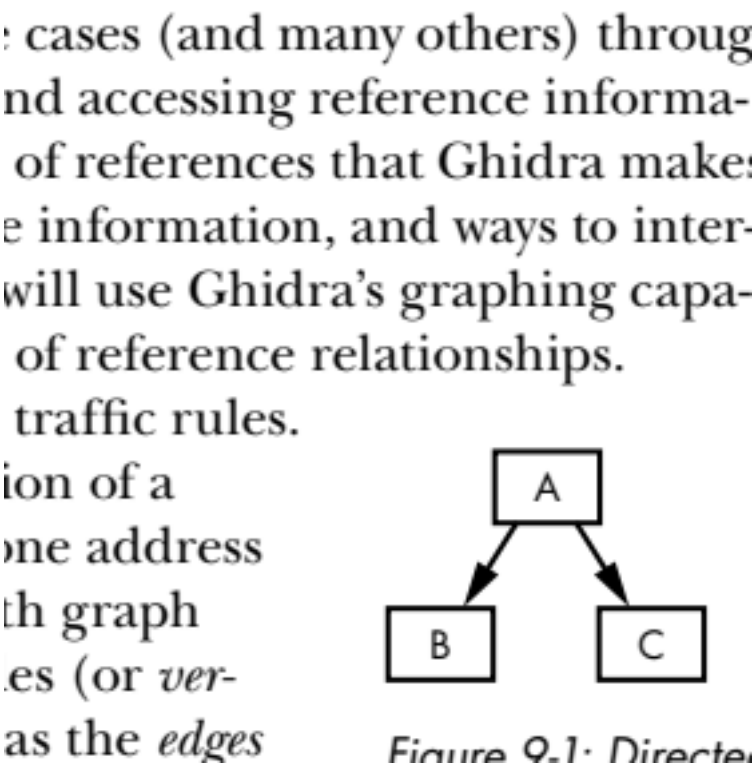


Рисунок 9-1. Ориентированный граф с тремя узлами и двумя ребрами

Ориентированные ребра представлены стрелками, указывающими допустимое направление движения по ребру. На рисунке 9-1 движение от A к B возможно, но движение от B к A - нет, подобно улице с односторонним движением. Если бы стрелки были двунаправленными, движение в любом направлении было бы допустимо.

У Ghidra есть две базовые категории ссылок: прямые ссылки и обратные ссылки, у каждой из которых также есть подкатегории. Обратные ссылки являются менее сложным из двух типов и, вероятно, чаще всего используются в обратной разработке. Обратные ссылки, также называемые перекрестными ссылками, предоставляют средство навигации между местоположениями в листинге, такими как код и данные.

Перекрестные ссылки (обратные ссылки)

Обратные ссылки внутри Ghidra часто называют просто XREF, что является мнемоникой для термина "перекрестная ссылка". В этом тексте мы используем термин XREF только при обращении к конкретной последовательности символов (`XREF`) в листинге, пункте меню или диалоге Ghidra. Во всех остальных случаях мы придерживаемся более общего термина перекрестная ссылка, когда говорим об обратных ссылках. Начнем с конкретных примеров XREF в Ghidra, а затем перейдем к более полному примеру.

Пример 1: базовые XREF

Начнем с изучения некоторых XREF, которые мы встретили в `demo_stackframe` (см. главу 6), и используем следующий листинг, чтобы понять связанный формат и смысл:

```
*************************************************************
*                          FUNCTION                         *
*************************************************************
undefined demo_stackframe(undefined param_1, undefined4...
   undefined   AL:1            <RETURN>
   undefined   Stack[0x4]:4    param_1
   undefined4  Stack[0x8]:4    param_2   XREF[1]:  0804847f(R)
   undefined4  Stack[0xc]:4    param_3   XREF[1]:  08048479(R)
   undefined4  Stack[-0x10]:4  local_10  XREF[1]:  0804847c(W)
   undefined4  Stack[-0x14]:4  local_14  XREF[2]:  08048482(W),
                                                   08048493(R)
   undefined4  Stack[-0x18]:4  local_18  XREF[2]:  08048485(W),
                                                   08048496(R)
   undefined1  Stack[-0x58]:1  local_58  XREF[1]:  0804848c(W)
demo_stackframe                           XREF[4]:  Entry Point(*),
                                                   main:080484be(c),
                                                   080485e4,
                                                   08048690(*)
```

Ghidra не только указывает, что существует перекрестная ссылка, с помощью индикатора `XREF`, но и показывает число перекрестных ссылок индексным значением после `XREF`. Эта часть перекрестной ссылки (например, `XREF[2]:`) называется заголовком XREF. Рассматривая заголовки в листинге, мы видим, что у большинства перекрестных ссылок только один ссылающийся адрес, но у некоторых их больше.

После заголовка идет адрес, связанный с перекрестной ссылкой, и это объект, по которому можно перейти. После адреса находится индикатор типа в скобках. Для перекрестных ссылок данных (как в этом примере) допустимые типы: `R` (переменная читается по соответствующему адресу XREF), `W` (в переменную выполняется запись) и `*` (адрес местоположения берется как указатель). Итого: перекрестные ссылки данных идентифицируются в листинге там, где данные объявлены, а связанные записи XREF предоставляют ссылки на местоположения, где эти данные упоминаются.

Листинг также содержит перекрестную ссылку кода. Перекрестные ссылки кода - очень важное понятие, поскольку они позволяют Ghidra строить графы функций и графы вызовов функций, которые являются предметом главы 10. Перекрестная ссылка кода используется, чтобы указать, что инструкция передает или может передать управление другой инструкции. Способ, которым инструкции передают управление, называется потоком (flow). Потоки могут быть любого из трех базовых типов: последовательный, переход или вызов. Потоки перехода и вызова можно дополнительно разделить по тому, является ли целевой адрес ближним или дальним.

Последовательный поток - самый простой тип потока, поскольку он представляет линейное выполнение от одной инструкции к следующей. Это стандартный поток выполнения для всех неветвящихся инструкций, таких как `ADD`. Для последовательных потоков нет специальных индикаторов отображения, кроме порядка, в котором инструкции перечислены в дизассемблированном листинге: если инструкция A имеет последовательный поток к инструкции B, то инструкция B будет следовать сразу после инструкции A в дизассемблированном листинге.

> **Форматирование XREF**
>
> Как и у большинства элементов, которые вы встречаете в окне `Listing`, можно управлять атрибутами отображения перекрестных ссылок. Выбор `Edit > Tool Options` открывает редактируемые параметры `CodeBrowser`. Поскольку XREF является частью окна `Listing`, поле `XREFs Field` можно найти в папке `Listing Fields`. При выборе оно откроет диалог, показанный на рисунке 9-2 (здесь со стандартными параметрами). Если изменить `Maximum Number of XREFs to Display` на 2, заголовок для всех перекрестных ссылок,

превышающих это число, будет отображаться как `XREF[more]`. Параметр отображения нелокальных пространств имен позволяет быстро идентифицировать все перекрестные ссылки, находящиеся не внутри тела текущей функции. Все параметры объясняются в `Ghidra Help`.

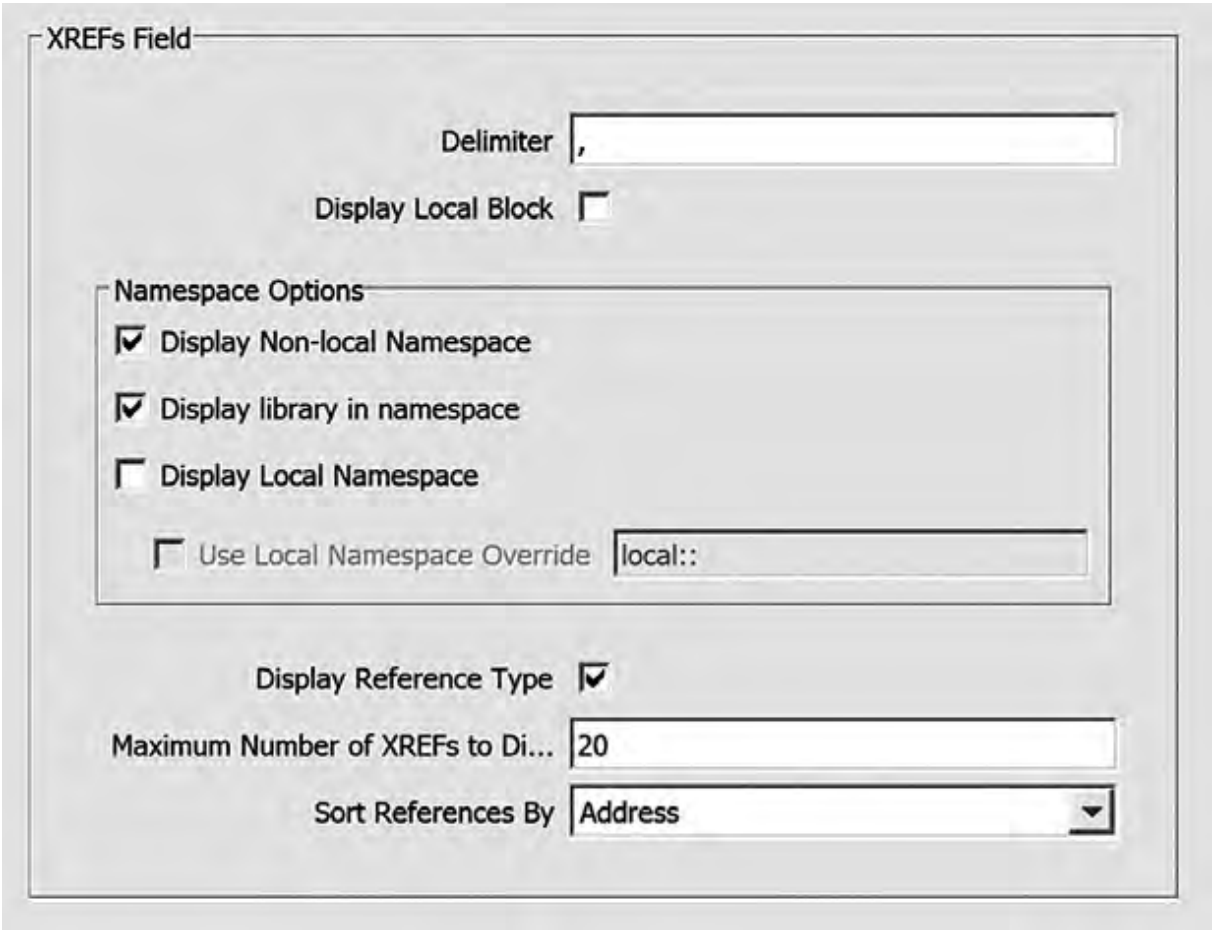


Рисунок 9-2. Окно редактирования `XREFs Field` *со стандартными параметрами*

Пример 2: XREF переходов и вызовов

Быстро рассмотрим новый пример, содержащий перекрестные ссылки кода, демонстрирующие переходы и вызовы. Как и у перекрестных ссылок данных, у перекрестных ссылок кода тоже есть связанная запись XREF в окне `Listing`. Следующий листинг показывает информацию, связанную с функцией `main`:

```
*************************************************************
*                          FUNCTION                         *
*************************************************************
undefined4 __stdcall main(void)
   undefined4  EAX:4           <RETURN>
   undefined4  Stack[-0x8]:4   ptr       XREF[3]:  00401014(W),
                                                   0040101b(R),
                                                   00401026(R)
main                                     XREF[1]:  entry:0040121e(c)
```

Вы можете ясно идентифицировать три XREF, связанные со стековой переменной, а также XREF, связанную с самой функцией. Расшифруем смысл XREF `entry:0040121e(c)`. Адрес (или, в этом случае, идентификатор) перед двоеточием указывает ссылающуюся (или исходную) сущность. В этом случае управление передается из `entry`. Справа от двоеточия находится конкретный адрес

внутри `entry`, являющийся источником перекрестной ссылки. Суффикс `(c)` указывает, что это CALL к `main`. Проще говоря, перекрестная ссылка говорит: "`main` вызывается с адреса `0040121e` внутри `entry`."

Если дважды щелкнуть адрес перекрестной ссылки, чтобы перейти по ссылке, мы попадем к указанному адресу внутри `entry`, где сможем изучить вызов. Хотя XREF является однонаправленной ссылкой, мы можем быстро вернуться к `main`, дважды щелкнув имя функции (`main`) или использовав стрелку навигации назад на панели инструментов `CodeBrowser`:

```
0040121e  CALL   main
```

В следующем листинге суффикс `(j)` у XREF указывает, что это помеченное местоположение является целью JUMP:

```
004011fe  JZ     LAB_00401207
00401200  PUSH   EAX
00401201  CALL   __amsg_exit
00401206  POP    ECX
        LAB_00401207                            XREF[1]: 004011fe(j)
00401207  MOV    EAX,[DAT_0040acf0]
```

Как и в предыдущем примере, можно дважды щелкнуть адрес XREF, чтобы перейти к инструкции, которая передала управление. Вернуться можно двойным щелчком по связанной метке.

Пример ссылок

Пройдем пример от исходного кода к дизассемблированию, чтобы продемонстрировать многие типы перекрестных ссылок. Следующая программа, `simple_flows.c`, содержит различные операции, которые задействуют возможности перекрестных ссылок Ghidra, как отмечено в тексте комментариев:

```
int read_it;            // целочисленная переменная, читаемая в main
int write_it;           // целочисленная переменная, записываемая 3 раза в main
int ref_it;             // целочисленная переменная, адрес которой берется в main
void callflow() {}      // функция, вызываемая дважды из main

int main() {
    int *ptr = &ref_it; // дает ссылку данных (*) типа "указатель"
    *ptr = read_it;     // дает ссылку данных (R) типа "чтение"
    write_it = *ptr;    // дает ссылку данных (W) типа "запись"
    callflow();         // дает ссылку кода (c) типа "вызов"
    if (read_it == 3) { // дает ссылку кода (j) типа "переход"
        write_it = 2;   // дает ссылку данных (W) типа "запись"
    }
    else {              // дает ссылку кода (j) типа "переход"
        write_it = 1;   // дает ссылку данных (W) типа "запись"
    }
    callflow();         // дает ссылку кода (c) типа "вызов"
}
```

Перекрестные ссылки кода

Листинг 9-1 показывает дизассемблирование предыдущей программы.

```
      undefined4 __stdcall main(void)
         undefined4 EAX:4 <RETURN>
         undefined4 Stack[-0x8]:4 ptr          XREF[3]:  00401014(W),
                                                         0040101b(R),
                                                         00401026(R)
      main                                     XREF[1]:  entry:0040121e(c)
00401010  PUSH   EBP
```

```
00401011  MOV    EBP,ESP
00401013  PUSH   ECX
00401014  MOV    dword ptr [EBP + ptr],ref_it
0040101b  MOV    EAX,dword ptr [EBP + ptr]
0040101e  MOV    ECX,dword ptr [read_it]
00401024  MOV    dword ptr [EAX]=>ref_it,ECX
00401026  MOV    EDX,dword ptr [EBP + ptr]
00401029  MOV    EAX=>ref_it,dword ptr [EDX]
0040102b  MOV    [write_it],EAX
00401030  CALL   callflow
00401035  CMP    dword ptr [read_it],3
0040103c  JNZ    LAB_0040104a
0040103e  MOV    dword ptr [write_it],2
00401048  JMP    LAB_00401054
        LAB_0040104a                            XREF[1]: 0040103c(j)
0040104a  MOV   dword ptr [write_it],1
        LAB_00401054                            XREF[1]:  00401048(j)
00401054  CALL   callflow
00401059  XOR    EAX,EAX
0040105b  MOV    ESP,EBP
0040105d  POP    EBP
0040105e  RET
```

Листинг 9-1. Дизассемблирование `main` *в* `simple_flows.exe`

Каждая инструкция, кроме `JMP` и `RET`, имеет связанный последовательный поток к своему непосредственному преемнику. Инструкциям, используемым для вызова функций, таким как x86-инструкция `CALL`, назначается поток вызова, указывающий передачу управления целевой функции. Потоки вызовов отмечаются XREF у целевой функции (адреса назначения потока). Дизассемблирование функции `callflow`, упомянутой в листинге 9-1, показано в листинге 9-2.

```
      undefined __stdcall callflow(void)
         undefined AL:1 <RETURN>
      callflow                                XREF[4]:  0040010c(*),
                                                        004001e4(*),
                                                        main:00401030(c),
                                                        main:00401054(c)
00401000  PUSH   EBP
00401001  MOV    EBP,ESP
00401003  POP    EBP
00401004  RET
```

Листинг 9-2. Дизассемблирование функции `callflow`

Лишние XREF?

Время от времени в листинге встречается что-то, что кажется аномальным. В листинге 9-2 есть две указательные XREF, `0040010c(*)` и `004001e4(*)`, которые нелегко объяснить. Мы сразу поняли две XREF, которые можно было проследить назад к вызовам `callflow` в `main`. Что это за две другие XREF?

Оказывается, это интересный артефакт именно этого кода. Эта программа была скомпилирована для Windows, что дает PE-файл, а две аномальные XREF ведут нас к PE-заголовку в секции Headers листинга. Два адреса ссылок (включая связанные байты) показаны здесь:

```
0040010c  00 10 00 00 ibo32     callflow              BaseOfCode
                 .  .  .
004001e4  00 10 00 00 ibo32     callflow              VirtualAddress
```

Почему эта функция упоминается в PE-заголовке? Быстрый поиск в Google может помочь понять происходящее: `callflow` просто оказывается самым первым элементом в секции `.text`, а два PE-поля косвенно ссылаются на начало секции `.text`, отсюда и неожиданные XREF, связанные с функцией `callflow`.

В этом примере мы видим, что `callflow` вызывается дважды из `main`: один раз с адреса `00401030` и еще раз с адреса `00401054`. Перекрестные ссылки, возникающие из вызовов функций, различаются суффиксом `(c)`. Исходное местоположение, отображаемое в перекрестных ссылках, указывает и адрес, с которого выполняется вызов, и функцию, содержащую этот вызов.

Поток перехода назначается каждой инструкции безусловного и условного ветвления. Условным ветвлениям также назначаются последовательные потоки, чтобы учитывать поток управления, когда ветвление не выполняется; у безусловных ветвлений нет связанного последовательного потока, потому что ветвление выполняется всегда. Потоки перехода связаны с перекрестными ссылками стиля jump, отображаемыми у цели `JNZ` в листинге 9-1. Как и у перекрестных ссылок стиля call, перекрестные ссылки jump отображают адрес ссылающегося местоположения (источник перехода). Перекрестные ссылки jump различаются суффиксом `(j)`.

Базовые блоки

В анализе программ базовый блок - это максимальная последовательность инструкций, которая выполняется без ветвления от начала до конца. Поэтому каждый базовый блок имеет одну точку входа (первую инструкцию блока) и одну точку выхода (последнюю инструкцию блока). Первая инструкция базового блока часто является целью инструкции ветвления, а последняя инструкция часто является инструкцией ветвления. Первая инструкция может быть целью нескольких перекрестных ссылок кода. Кроме первой инструкции, никакая другая инструкция внутри базового блока не может быть целью перекрестной ссылки кода. Последняя инструкция базового блока может быть источником нескольких перекрестных ссылок кода, например условного перехода, или может перетекать в инструкцию, являющуюся целью нескольких перекрестных ссылок кода (которая по определению должна начинать новый базовый блок).

Перекрестные ссылки данных

Перекрестные ссылки данных используются для отслеживания того, как данные доступны внутри бинарного файла. Три наиболее часто встречающихся типа перекрестных ссылок данных указывают, когда местоположение читается, когда в местоположение выполняется запись и когда берется адрес местоположения. Глобальные переменные из предыдущей примерной программы показаны в листинге 9-3, поскольку они дают несколько примеров перекрестных ссылок данных.

```
         read_it                                 XREF[2]:  main:0040101e(R),
                                                           main:00401035(R)
0040b720 undefined4    ??
         write_it                                XREF[3]:  main:0040102b(W),
                                                           main:0040103e(W),
                                                           main:0040104a(W)
0040b724    ??         ??
0040b725    ??         ??
0040b726    ??         ??
0040b727    ??         ??
         ref_it                                  XREF[3]:  main:00401014(*),
                                                           main:00401024(W),
                                                           main:00401029(R)
0040b728 undefined4    ??
```

Листинг 9-3. Глобальные переменные, упоминаемые в `simple_flows.c`

Перекрестная ссылка чтения указывает, что содержимое местоположения памяти читается. Перекрестные ссылки чтения могут исходить только из адреса инструкции, но могут ссылаться на любое местоположение программы. Глобальная переменная `read_it` читается дважды в листинге 9-1. Связанные комментарии перекрестных ссылок, показанные в этом листинге, точно указывают,

какие местоположения в `main` ссылаются на `read_it`, и распознаются как перекрестные ссылки чтения по суффиксу `(R)`. Чтение, выполненное из `read_it` в листинге 9-1, является 32-разрядным чтением в регистр `ECX`, что заставляет Ghidra форматировать `read_it` как `undefined4` (4-байтовое значение неопределенного типа). Ghidra часто пытается вывести размер элемента данных на основе того, как этот элемент обрабатывается кодом по всему бинарному файлу.

Глобальная переменная `write_it` упоминается три раза в листинге 9-1. Связанные перекрестные ссылки записи генерируются и отображаются как комментарии для переменной `write_it`, указывая местоположения программы, которые изменяют содержимое переменной. Перекрестные ссылки записи используют суффикс `(W)`. В этом случае Ghidra не отформатировала `write_it` как 4-байтовую переменную, хотя, кажется, для этого достаточно информации. Как и перекрестные ссылки чтения, перекрестные ссылки записи могут исходить только из инструкции программы, но могут ссылаться на любое местоположение программы. Обычно перекрестная ссылка записи, нацеленная на байт инструкции программы, указывает на сам модифицирующийся код и часто встречается в процедурах деобфускации вредоносных программ.

Третий тип перекрестной ссылки данных, указательная перекрестная ссылка, указывает, что используется адрес местоположения (а не содержимое местоположения). Адрес глобальной переменной `ref_it` берется в листинге 9-1, что приводит к указательной перекрестной ссылке у `ref_it` в листинге 9-3, на что указывает суффикс `(*)`. Указательные перекрестные ссылки обычно являются результатом получения адресов либо в коде, либо в данных. Как вы видели в главе 8, операции доступа к массивам обычно реализуются путем прибавления смещения к начальному адресу массива, и первый адрес большинства глобальных массивов часто можно распознать по наличию указательной перекрестной ссылки. По этой причине большинство строковых литералов (строки являются массивами символов в C/C++) являются целями указательных перекрестных ссылок.

В отличие от перекрестных ссылок чтения и записи, которые могут исходить только из местоположений инструкций, указательные перекрестные ссылки могут исходить как из местоположений инструкций, так и из местоположений данных. Пример указателей, которые могут исходить из секции данных программы, - любая таблица адресов (например, vftable, которая приводит к генерации указательной перекрестной ссылки от каждой записи таблицы к соответствующей виртуальной функции). Посмотрим это в контексте на примере `SubClass` из главы 8. Дизассемблирование vftable для `SubClass` показано здесь:

```
         SubClass::vftable           XREF[1]:  SubClass_Constructor:00401062(*)
00408148 void * SubClass::vfunc1 vfunc1
0040814c void * BaseClass::vfunc2 vfunc2
00408150 void * SubClass::vfunc3 vfunc3
00408154 void * BaseClass::vfunc4 vfunc4
00408158 void * SubClass::vfunc5 vfunc5
```

Здесь вы видите, что элемент данных по адресу `0040814c` является указателем на `BaseClass::vfunc2`. Переход к `BaseClass::vfunc2` показывает следующий листинг:

```
      *************************************************************
      *                          FUNCTION                         *
      *************************************************************
      undefined __stdcall vfunc2(void)
         undefined AL:1 <RETURN>
         undefined4 Stack[-0x8]:4 local_8      XREF[1]:  00401024(W)
      BaseClass::vfunc2                        XREF[2]:  00408138(*),
                                                         0040814c(*)
00401020  PUSH   EBP
00401021  MOV    EBP,ESP
00401023  PUSH   ECX
00401024  MOV    dword ptr [EBP + local_8],ECX
00401027  MOV    ESP,EBP
00401029  POP    EBP
0040102a  RET
```

В отличие от большинства функций, у этой функции нет перекрестных ссылок кода. Вместо этого мы видим две указательные перекрестные ссылки, указывающие, что адрес функции получается в двух местоположениях. Вторая XREF ссылается назад на запись vftable `SubClass`, обсуждавшуюся ранее. Переход по первой XREF привел бы нас к vftable для `BaseClass`, которая также содержит указатель на эту виртуальную функцию.

Этот пример демонстрирует, что виртуальные функции C++ редко вызываются напрямую и обычно не являются целью перекрестной ссылки вызова. Из-за того, как создаются vftable, все виртуальные функции C++ будут упомянуты как минимум одной записью vftable и всегда будут целью как минимум одной указательной перекрестной ссылки. (Помните, что переопределение виртуальной функции не является обязательным.)

Когда бинарный файл содержит достаточно информации, Ghidra может находить vftable за вас. Любые vftable, найденные Ghidra, перечисляются как запись под записью соответствующего класса внутри папки `Classes` окна `Symbol Tree`. Щелчок по vftable в окне `Symbol Tree` переносит вас к местоположению vftable в секции данных программы.

Окна управления ссылками

К этому моменту вы, вероятно, заметили, что аннотации XREF довольно часто встречаются в окне `Listing`. Это не случайно, поскольку ссылки, образованные перекрестными ссылками, являются связующим материалом, удерживающим программу вместе. Перекрестные ссылки рассказывают историю внутрифункциональных и межфункциональных зависимостей, и большинство успешных работ по обратной разработке требует всестороннего понимания их поведения. Следующие разделы выходят за рамки базовой пользы перекрестных ссылок для отображения и навигации и вводят несколько вариантов управления перекрестными ссылками в Ghidra.

Окно XRefs

Можно использовать заголовки XREF, чтобы узнать больше о конкретной перекрестной ссылке, как показано в следующем листинге:

```
undefined4 Stack[-0x10]:4 local_10    XREF[1]:  0804847c(W)
undefined4 Stack[-0x14]:4 local_14    XREF[2]:  08048482(W),
                                                08048493(R)
```

Двойной щелчок по заголовку `XREF[2]` откроет связанное окно XRefs, показанное на рисунке 9-3, с более подробным списком перекрестных ссылок. По умолчанию окно показывает местоположение, метку (если применимо), ссылающееся дизассемблирование и тип ссылки.

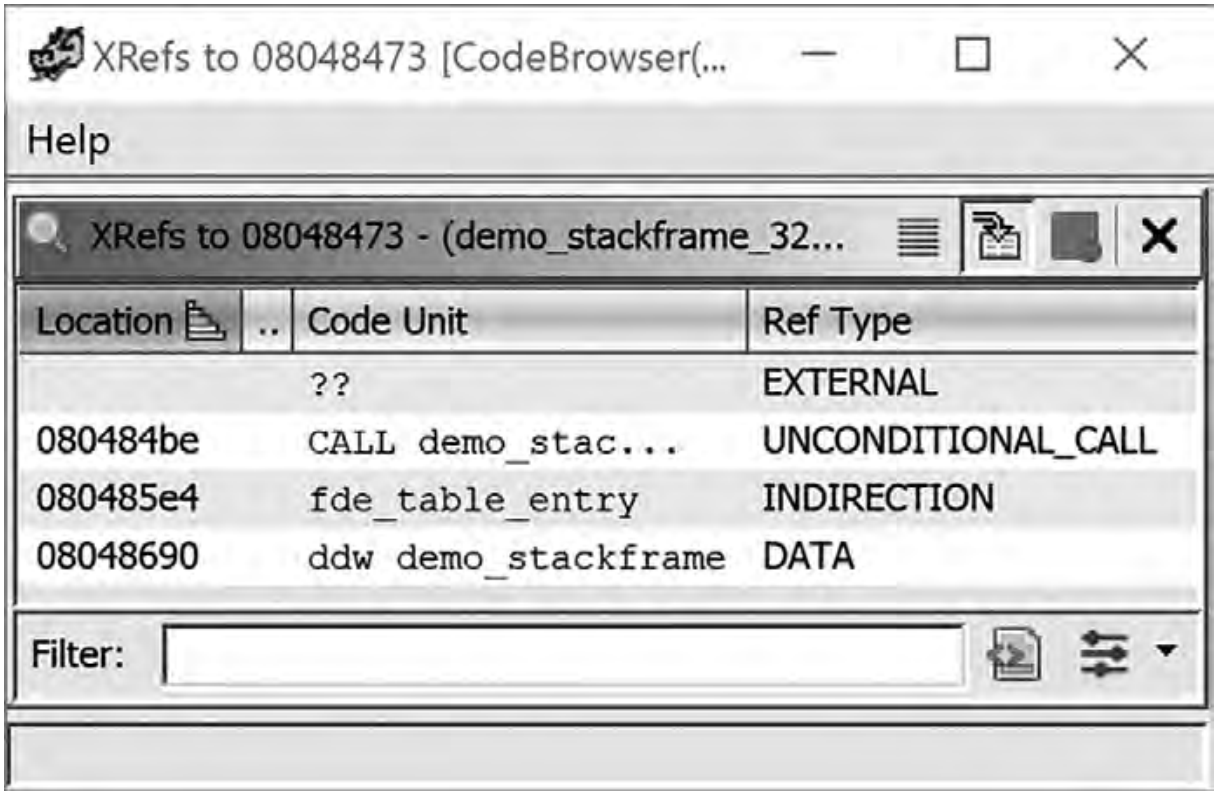


Рисунок 9-3. Окно XRefs

References To

Еще одно окно, которое может быть полезно для понимания потока программы, - окно References To. Щелчок правой кнопкой мыши по любому адресу в окне `Listing` и выбор References > Show Reference to Address открывает окно, показанное на рисунке 9-4.

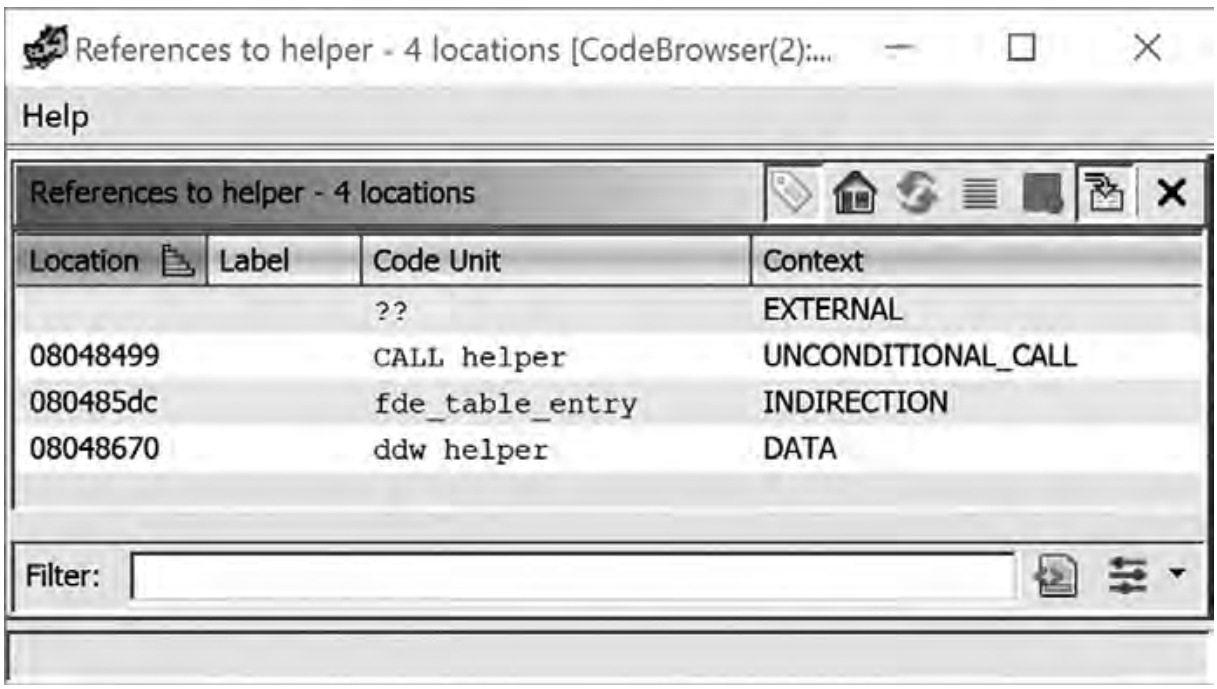


Рисунок 9-4. Окно References To

В этом примере мы выбрали начальный адрес функции `helper`. Внутри этого окна можно перейти к связанному местоположению, щелкнув любую запись в окне.

Symbol References

Еще одно представление ссылок, которое было введено в разделе "Окна `Symbol Table` и `Symbol References`" на странице 82, - это комбинация окон `Symbol Table` и Symbol Reference. По умолчанию, когда вы выбираете `Window` > `Symbol References`, вы получаете два связанных окна. Одно отображает каждый символ во всей таблице символов. Другое отображает связанные ссылки на символы. Выбор любой записи в окне `Symbol Table` (функции, vftable и т. д.) приводит к отображению связанных ссылок на символ в окне `Symbol References`.

Списки ссылок можно использовать для быстрого определения каждого местоположения, из которого вызывается конкретная функция. Например, многие считают функцию C `strcpy` опасной, поскольку она копирует исходный массив символов, вплоть до связанного завершающего нулевого символа включительно, в массив назначения без каких-либо проверок, что массив назначения достаточно велик, чтобы вместить все символы из источника. Можно найти любой один вызов `strcpy` в листинге и использовать вышеупомянутый способ для открытия окна References To, но если вы не хотите тратить время на поиск того, где `strcpy` используется в бинарном файле, можно открыть окно `Symbol References` и быстро найти `strcpy` и все связанные ссылки.

Продвинутое манипулирование ссылками

В начале этой главы мы приравняли термин обратная ссылка к перекрестной ссылке и кратко упомянули, что у Ghidra также есть прямые ссылки, которых существует два типа. Выведенные прямые ссылки обычно добавляются в листинг автоматически и взаимно-однозначно соответствуют обратным ссылкам, хотя по выведенным прямым ссылкам перемещаются в противоположном направлении. Иными словами, мы обходим обратные ссылки от целевого адреса назад к исходному адресу, а выведенные прямые ссылки - от исходного адреса вперед к целевому адресу.

Второй тип - явная прямая ссылка. Существует несколько типов явных прямых ссылок, и управление ими гораздо сложнее, чем другими перекрестными ссылками. Типы явных прямых ссылок включают ссылки памяти, внешние ссылки, стековые ссылки и регистровые ссылки. Помимо просмотра ссылок, Ghidra позволяет добавлять и редактировать различные типы ссылок.

Вам может понадобиться добавить собственные перекрестные ссылки, когда статический анализ Ghidra не может определить цели перехода или вызова, вычисляемые во время выполнения, но вам известна цель из другого анализа. В следующем коде, который мы в последний раз видели в главе 8, вызывается виртуальная функция.

```
0001072e  PUSH   EBP
0001072f  MOV    EBP,ESP
00010731  SUB    ESP,8
00010734  MOV    EAX,dword ptr [EBP + param_1]
00010737  MOV    EAX,dword ptr [EAX]
00010739  ADD    EAX,8
0001073c  MOV    EAX,dword ptr [EAX]
0001073e  SUB    ESP,12
00010741  PUSH   dword ptr [EBP + param_1]
00010744  CALL   EAX
00010746  ADD    ESP,16
00010749  NOP
0001074a  LEAVE
```

```
0001074b  RET
```

Значение, хранящееся в EAX, зависит от значения указателя, переданного в param_1. В результате у Ghidra недостаточно информации, чтобы создать перекрестную ссылку, связывающую 00010744 (адрес инструкции CALL) с целью вызова. Ручное добавление перекрестной ссылки (например, на SubClass::vfunc3), помимо прочего, включило бы целевые функции в граф вызовов, тем самым улучшив анализ программы в Ghidra. Щелчок правой кнопкой мыши по вызову и выбор References > Add Reference from открывает диалог, показанный на рисунке 9-5. Этот диалог также доступен через пункт References > Add/Edit.

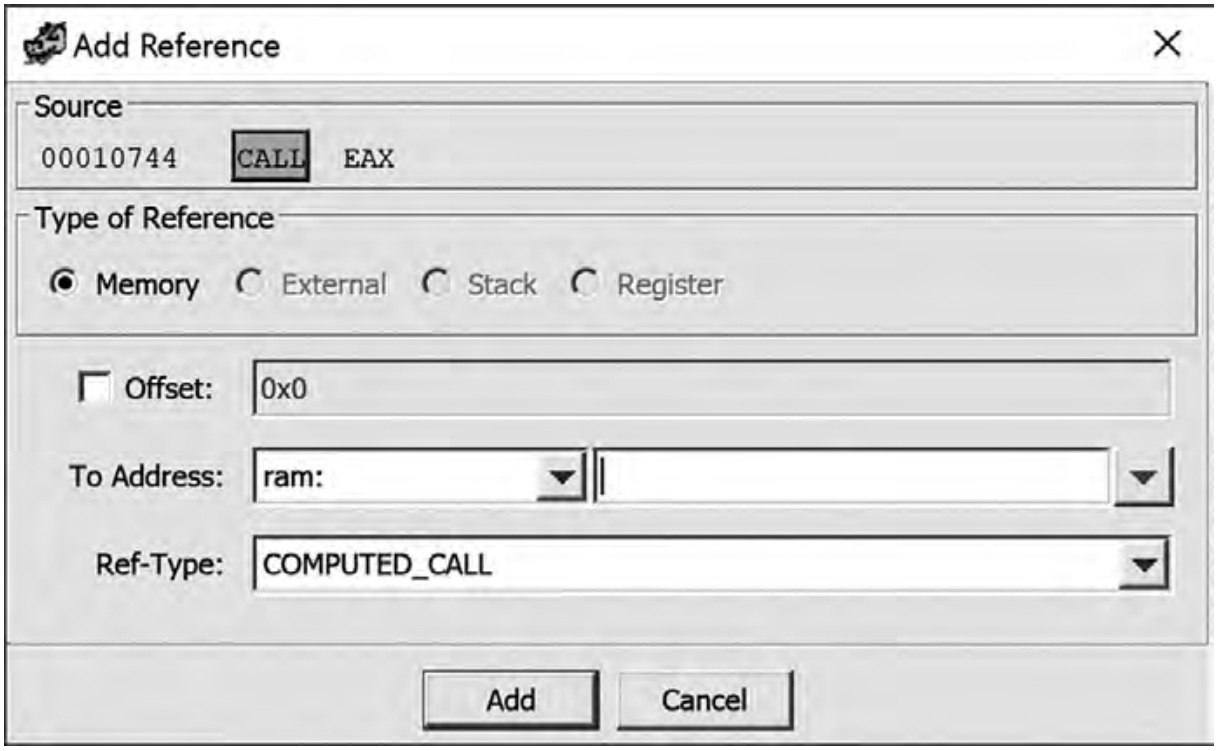


Рисунок 9-5. Диалог Add Reference

Укажите адрес целевой функции в параметре To Address и убедитесь, что для Ref-Type выбран правильный параметр. Когда вы закроете диалог кнопкой Add, Ghidra создаст ссылку, и новая перекрестная ссылка (c) появится по целевому адресу. Дополнительную информацию о прямых ссылках, включая остальные типы ссылок, а также манипулирование ссылками, можно найти в Ghidra Help.

Резюме

Ссылки - мощные инструменты, помогающие понять, как артефакты внутри бинарного файла связаны друг с другом. Мы подробно обсудили перекрестные ссылки и ввели некоторые другие возможности, связанные со ссылками, к которым еще вернемся в следующих главах. В следующей главе мы рассмотрим визуальные представления ссылок и то, как получающиеся графы могут помочь лучше понять потоки управления внутри функций и отношения между функциями в наших бинарных файлах.

Глава 10. Графы

Визуальное представление данных с помощью графов, как мы делали в предыдущей главе (см. рисунок 9-1), предоставляет краткий и ясный механизм для демонстрации множества соединений между узлами внутри графа, а также помогает распознавать шаблоны, которые иначе было бы трудно обнаружить при работе с графом как с абстрактным типом данных. Графовые представления Ghidra предлагают новый взгляд (в дополнение к листингам дизассемблирования и декомпилятора) на содержимое бинарного файла. Они позволяют быстро увидеть поток управления в функции и отношения между функциями в файле, представляя функции и другие типы блоков как узлы, а потоки и перекрестные ссылки - как ребра (линии, соединяющие узлы). После достаточной практики вы можете обнаружить, что распространенные управляющие конструкции, такие как операторы `switch` и вложенные структуры `if`/`else`, легче распознавать в графической форме, чем в длинном текстовом листинге. В главе 5 мы кратко представили окна Function Graph и Function Call Graph. В этой главе мы глубже погрузимся в графовые возможности Ghidra.

Поскольку перекрестные ссылки связывают один адрес с другим, они являются естественной отправной точкой для построения графов наших бинарных файлов. Ограничиваясь последовательными потоками и конкретными типами перекрестных ссылок, можно вывести ряд полезных графов для анализа бинарных файлов. Хотя потоки и перекрестные ссылки служат ребрами в наших графах, смысл узлов может различаться. В зависимости от типа графа, который мы хотим сгенерировать, узлы могут содержать одну или несколько инструкций или целые функции. Начнем обсуждение графов с того, как Ghidra организует код в блоки, а затем перейдем к типам графов, доступных в Ghidra.

Базовые блоки

В компьютерной программе базовый блок - это группа из одной или нескольких инструкций с единственным входом в начале блока и единственным выходом из конца блока. За исключением последней инструкции, каждая инструкция внутри базового блока передает управление ровно одной следующей инструкции внутри блока. Аналогично, за исключением первой инструкции, каждая инструкция в базовом блоке получает управление ровно от одной предыдущей инструкции внутри блока. В разделе "Перекрестные ссылки (обратные ссылки)" на странице 185 мы определили это как последовательный поток. Время от времени вы можете заметить вызов функции в середине базового блока и подумать: "Разве это не именно тот тип инструкции, вроде перехода, который должен завершать блок?" Для целей определения базовых блоков факт, что вызовы функций передают управление за пределы текущего блока, обычно игнорируется, если только не известно, что вызываемая функция не возвращается обычным образом.

После выполнения первой инструкции базового блока гарантируется, что остальная часть блока выполнится до конца. Это может существенно влиять на инструментирование программы во время выполнения, поскольку больше не нужно ставить точку останова на каждую инструкцию программы или даже выполнять программу по одной инструкции, чтобы записать, какие инструкции были выполнены. Вместо этого точки останова можно ставить на первую инструкцию каждого базового блока, и при срабатывании каждой точки останова можно считать, что каждая инструкция в связанном блоке будет выполнена. Перенесем внимание на возможности Function Graph в Ghidra, чтобы получить другой взгляд на блоки.

Графы функций

Окно Function Graph, представленное в главе 5, отображает одну функцию в графическом формате. Следующая программа состоит из одной функции, которая состоит из одного базового блока, поэтому это полезная отправная точка для демонстрации графов функций Ghidra:

```
int global_array[3];

int main() {
    int idx = 2;
    global_array[0] = 10;
    global_array[1] = 20;
    global_array[2] = 30;
    global_array[idx] = 40;
}
```

Когда вы открываете окно Function Graph (`Window` > Function Graph) при выбранной `main`, вам показывается граф функции только с одним базовым блоком, как на рисунке 10-1.

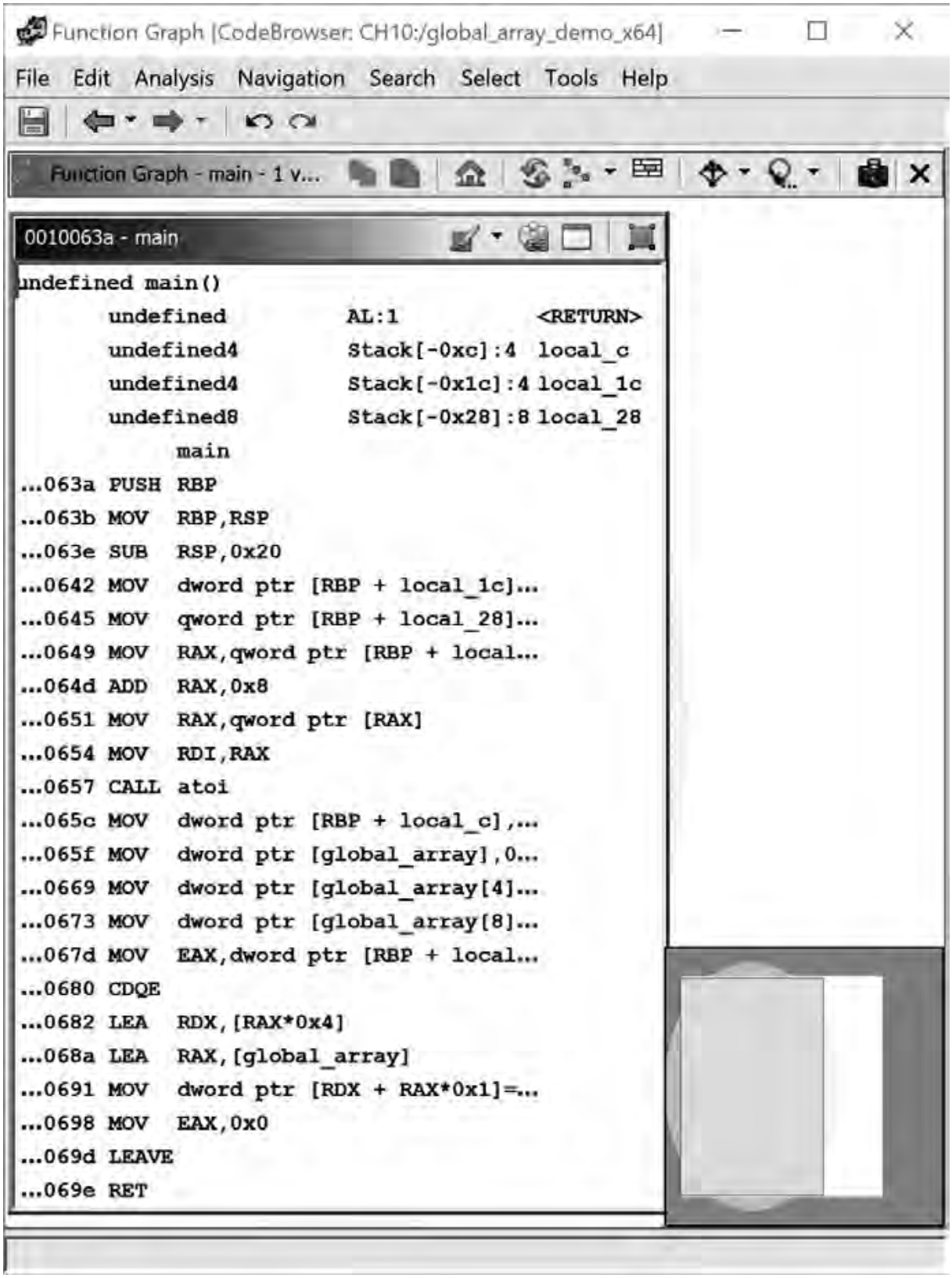


Рисунок 10-1. Окно Function Graph с одним блоком и обзором-satellite в нижнем правом углу

Окно Function Graph и окно `Listing` имеют полезную двунаправленную связь. Если смотреть на окна рядом, одновременный листинг и графическое представление могут помочь лучше понять поток управления функции. Изменения, которые вы вносите в окне Function Graph (например, переименование функций, переменных и т. д.), немедленно отражаются в окне `Listing`. Изменения, которые вы вносите в окне `Listing`, также отражаются в окне Function Graph, хотя,

возможно, придется обновить окно, чтобы увидеть изменение.

Артикуляция

По мере усложнения функций число блоков в каждой из них, скорее всего, будет расти. Когда вы впервые генерируете граф функции, ребра, соединяющие блоки, артикулированы. Это означает, что они аккуратно изгибаются под углом 90 градусов, чтобы не скрываться за узлами. В результате получается аккуратная сеточная раскладка, где все компоненты всех ребер либо горизонтальны, либо вертикальны. Если вы решите изменить раскладку графа, перетаскивая узлы, ребра могут потерять артикуляцию и вернуться к прямым линиям, проходящим за другими узлами графа. Рисунок 10-2 демонстрирует контраст между артикулированным представлением слева и неартикулированной версией справа. В любое время можно вернуться к исходной раскладке, обновив окно Function Graph.

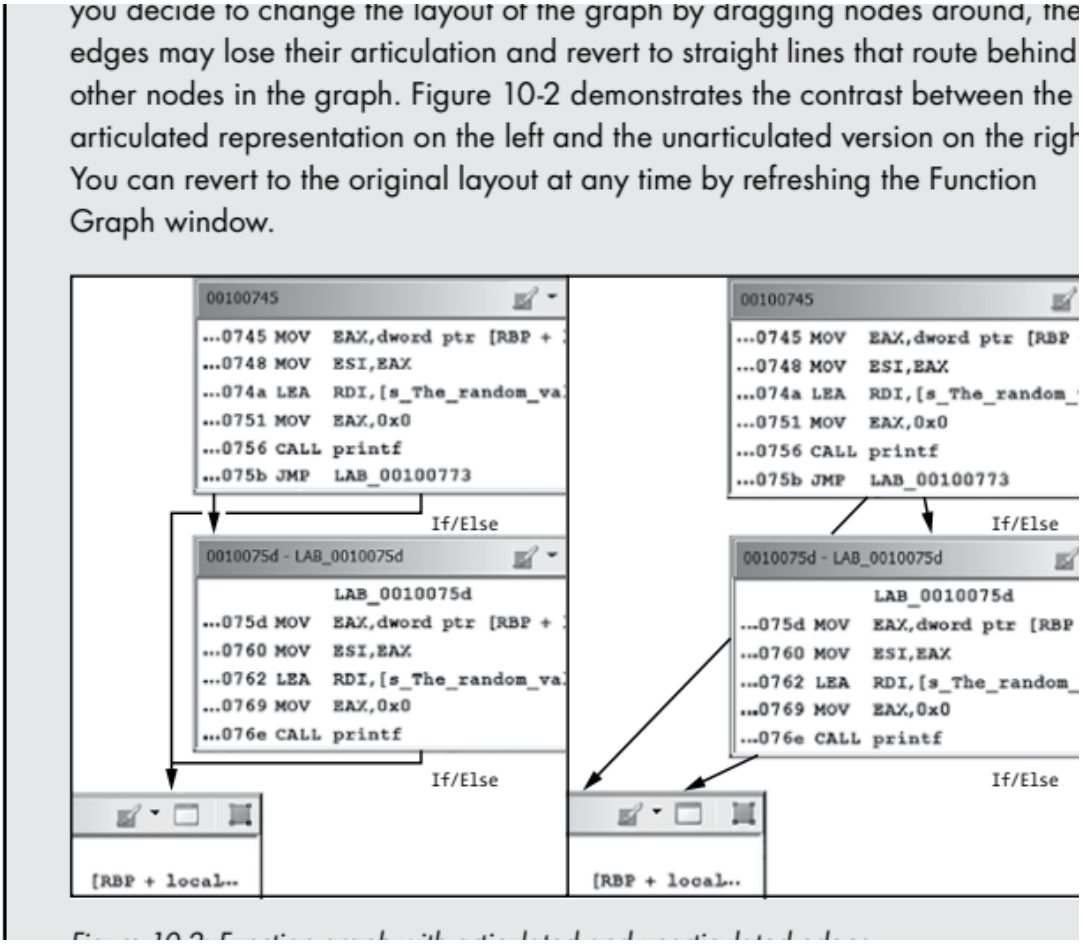


Рисунок 10-2. Граф функции с артикулированными и неартикулированными ребрами

Если щелкнуть любую строку текста в окне Function Graph, курсор в окне `Listing` переместится к соответствующему месту в дизассемблированном листинге. Если дважды щелкнуть данные в графе функции, окно `Listing` перейдет к связанным данным в секции данных листинга, а окно Function Graph сохранит фокус на функции. (Хотя Ghidra сейчас не предоставляет графовую визуализацию данных или отношений между компонентами данных, она позволяет одновременно смотреть данные в представлении листинга и связанный код в представлении графа.)

Рассмотрим короткий пример, чтобы продемонстрировать связь между окном `Listing` и окном Function Graph. Предположим, вы видите переменную `global_array` на рисунке 10-1 и хотите узнать больше о ее типе. Когда вы переходите к ней двойным щелчком по имени в графовом представлении, можно увидеть, что Ghidra классифицировала `global_array` как массив

неопределенных байтов (undefined1), к которому обращаются по индексам четвертого и восьмого элементов. Если изменить определение массива в секции данных окна Listing с undefined1[12] на int[3] (соответственно показано в верхней и нижней половинах рисунка 10-3), можно сразу увидеть эффект объявления в дизассемблировании в окне Function Graph (а также в окне Decompiler): значения индексов меняются на 1 и 2, отражая новый 4-байтовый размер каждого элемента массива.

ration on the disassembly in the Function Graph window (as well as th
Decompiler window): the index values change to 1 and 2 to reflect the
4-byte size of each array element.

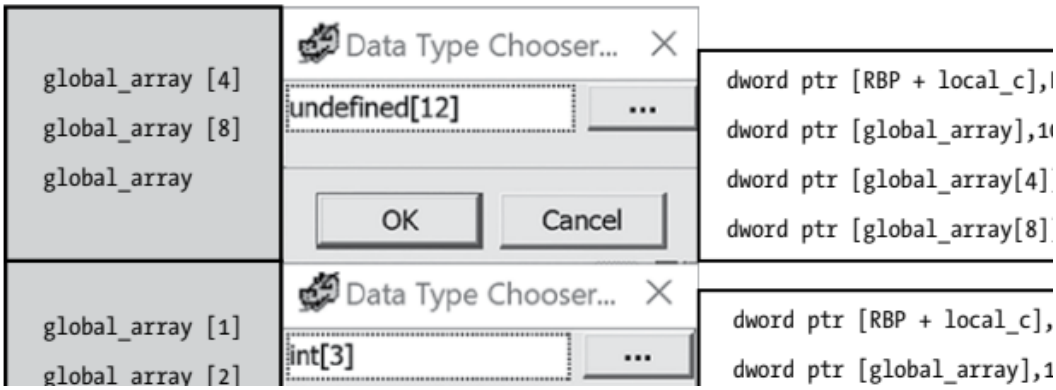


Рисунок 10-3. Влияние изменения объявления массива на окна Function Graph и Listing

Навигация в окне Listing гибка, если вы не щелкаете другую функцию. Можно прокручивать все содержимое окна Listing, щелкать и вносить изменения в секции данных, вносить изменения внутри функции и т. д. Если щелкнуть внутри другой функции, графовое представление обновится и отобразит граф новой выбранной функции.

На рисунке 10-4 выделены меню и панели инструментов, доступные в окне Function Graph.

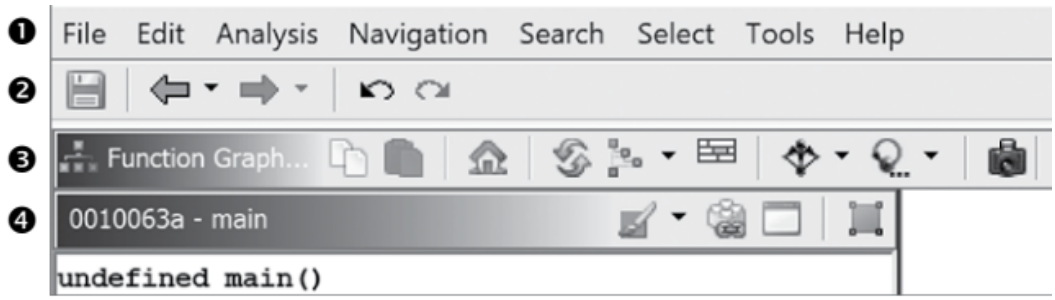


Рисунок 10-4. Панели инструментов Function Graph

Граф функции на самом деле не более чем графическое представление окна Listing, изолированное до одной функции, поэтому неудивительно, что все меню CodeBrowser (за исключением меню Window) доступны в окне Function Graph. Доступное подмножество панели инструментов CodeBrowser включает возможность сохранить текущее состояние открытого файла, отменять и повторять действия, а также перемещаться вперед и назад внутри текущей навигационной цепочки. Важно отметить, что, поскольку окна связаны, это может вывести вас из текущей функции (и вернуть обратно), что изменит содержимое окна Function Graph.

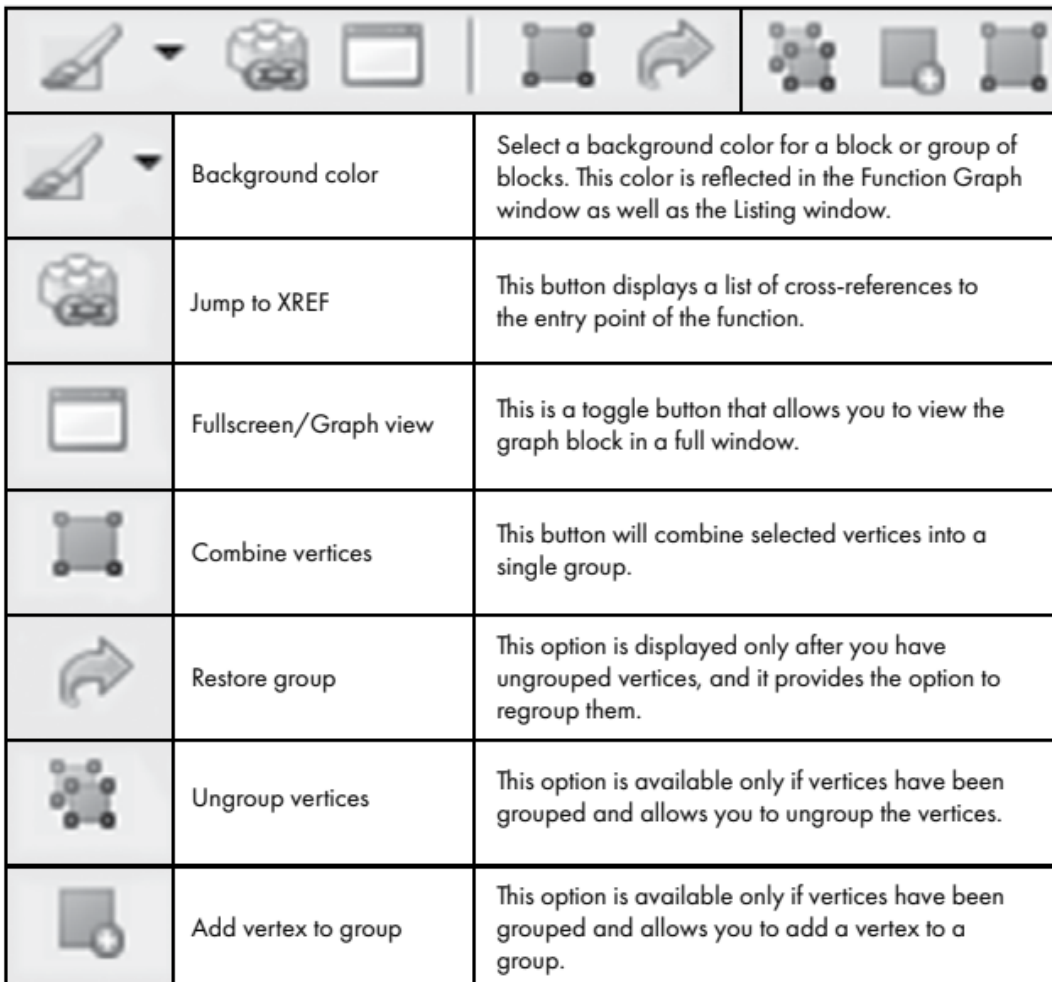
Значки панели инструментов Function Graph и их стандартное поведение описаны на рисунке 10-5.

Что такое порог взаимодействия?

При работе с окном Function Graph, особенно со сложной функцией, вы можете уменьшить масштаб, потому что не видите всего, что хотите увидеть. Когда отдельные узлы становятся слишком маленькими для осмысленного взаимодействия с ними, вы прошли порог взаимодействия. Тени у каждого узла в Function Graph используются для обозначения этого состояния. Виртуальные адреса могут показывать только младшие значения, а само число узлов в отображении графа может стать трудноуправляемым. Попытка выбрать содержимое внутри узла приводит к выбору всего блока. Не отчаивайтесь, если сложность функции вытолкнула вас за этот порог. Можно щелкнуть любой узел, чтобы привести его в фокус, или дважды щелкнуть узел, чтобы приблизить его.

Рисунок 10-5. Операции панели инструментов Function Graph

У каждого базового блока также есть панель инструментов, позволяющая изменять блок и группировать его с другими блоками, объединяя несколько блоков (вершин) в один блок (см. рисунок 10-6 с объяснением значков панели инструментов и их стандартного поведения). Эта возможность чрезвычайно полезна для снижения сложности графов, возникающей из-за сильно вложенных функций. Например, можно свернуть все блоки, вложенные в оператор цикла, в один узел графа после того, как вы поняли поведение цикла и меньше нуждаетесь в просмотре кода внутри него. В зависимости от числа вложенных блоков, которые вы сгруппируете, читаемость графа может существенно улучшиться. Чтобы сгруппировать узлы, нужно выбрать все узлы, которые будут принадлежать группе, используя `ctrl`-щелчок для выбора всех узлов-участников, а затем щелкнуть инструмент Combine vertices у узла, который вы считаете корневым для группы. Restore group - особенно полезная кнопка, позволяющая быстро заглянуть внутрь группы, а затем снова свернуть ее.



Background color	Select a background color for a block or group of blocks. This color is reflected in the Function Graph window as well as the Listing window.
Jump to XREF	This button displays a list of cross-references to the entry point of the function.
Fullscreen/Graph view	This is a toggle button that allows you to view the graph block in a full window.
Combine vertices	This button will combine selected vertices into a single group.
Restore group	This option is displayed only after you have ungrouped vertices, and it provides the option to regroup them.
Ungroup vertices	This option is available only if vertices have been grouped and allows you to ungroup the vertices.
Add vertex to group	This option is available only if vertices have been grouped and allows you to add a vertex to a group.

Рисунок 10-6. Панель инструментов базового блока Function Graph

Чтобы увидеть некоторые другие возможности, связанные с графом функции, нужно рассмотреть примеры с более чем одним базовым блоком. Следующая программа используется в последующих примерах:

```
int do_random() {
    int r;
    srand(time(0));
    r = rand();
    if (r % 2 == 0) {
       printf("Случайное значение %d четное\n", r);
    }
    else {
       printf("Случайное значение %d нечетное\n", r);
    }
    return r;
}

int main() {
    do_random();
}
```

Функция `do_random` содержит управляющие конструкции (`if`/`else`), которые приводят к графу с четырьмя базовыми блоками, обозначенными на рисунке 10-8. Просмотр функции с более чем одним блоком делает более очевидным, что граф функции является графом потока управления, где ребра указывают возможные потоки от одного блока к другому. Обратите внимание, что раскладка Ghidra для графов функций называется `Nested Code Layout` и близко напоминает поток С-кода. Это упрощает просмотр графического представления окон `Listing` и `Decompiler` в

контексте более крупной программы. Чтобы сохранить это представление, мы настоятельно рекомендуем изменить параметры графа так, чтобы ребра обходили вершины (`Edit > Tool Options > Function Graph > Nested Code Layout > Route Edges Around Vertices`). По умолчанию у Ghidra есть неудачная склонность проводить ребра за узлами, что часто может создавать вводящую в заблуждение картину отношений между узлами.

> **Этот граф устарел**
>
> Хотя некоторые изменения в листинге немедленно отражаются в окне Function Graph, в других случаях граф может устареть (не синхронизироваться с представлением листинга). Когда это происходит, Ghidra отображает сообщение, показанное на рисунке 10-7, в нижней части окна графа.

Рисунок 10-7. Предупреждение об устаревшем графе

> Значок recycle слева от сообщения позволяет обновить граф без возврата к исходной раскладке. (Конечно, можно также выбрать обновление с повторной раскладкой.)

В графе, показанном на рисунке 10-8, `BLOCK-1` является единственной точкой входа в функцию. Этот блок, как и все базовые блоки, демонстрирует последовательный поток от инструкции к инструкции внутри блока. Ни один из трех вызовов функций внутри блока (к `time`, `srand` и `rand`) не "разрывает" базовый блок, поскольку Ghidra предполагает, что все они вернутся для продолжения последовательного выполнения оставшихся инструкций. В `BLOCK-2` выполняется вход, если условие `JNZ` в конце `BLOCK-1` вычисляется как ложное, то есть случайное значение четное. В `BLOCK-3` выполняется вход, если условие `JNZ` вычисляется как истинное, указывая, что случайное значение нечетное. Последний блок, `BLOCK-4`, достигается после завершения `BLOCK-2` или `BLOCK-3`. Обратите внимание, что щелчок по ребру приводит его в фокус и делает его толще остальных ребер. На рисунке ребро, соединяющее `BLOCK-1` и `BLOCK-3`, является активным и отображается жирным.

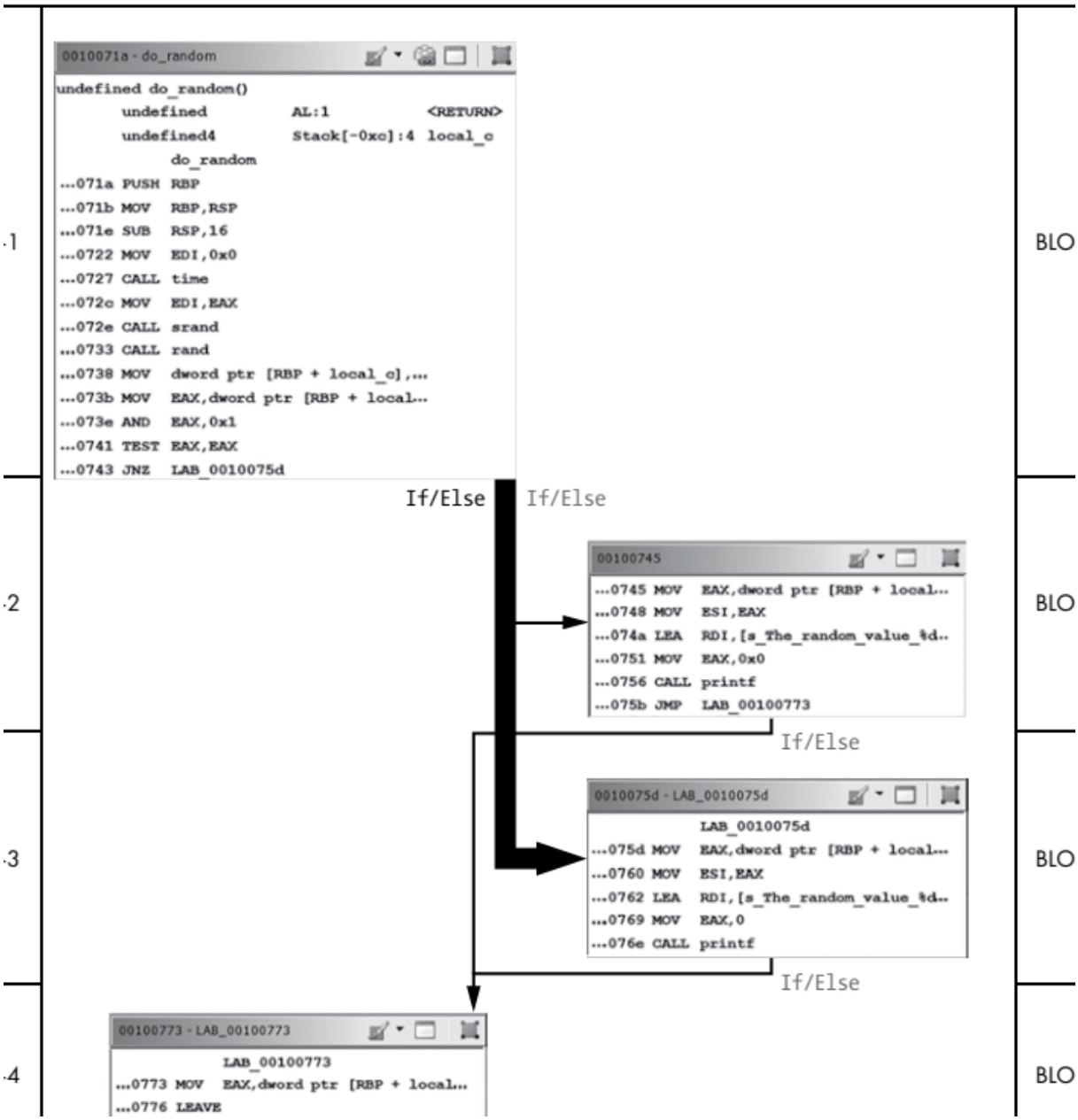


Рисунок 10-8. Граф функции с выбранной темной линией, показывающей поток при выполнении условия

Если у вас есть особенно длинный базовый блок и вы хотите разбить его на меньшие блоки или хотите визуально изолировать участок кода для дальнейшего анализа, можно разделить базовый блок внутри графа функции, добавив в блок новые метки. Использование горячей клавиши `L` для вставки новой метки в строке `0010072e` в `BLOCK-1` перед вызовом `srand` приводит к добавлению пятого блока в граф функции на рисунке 10-9. Новое добавленное ребро представляет поток и не связано с перекрестной ссылкой.

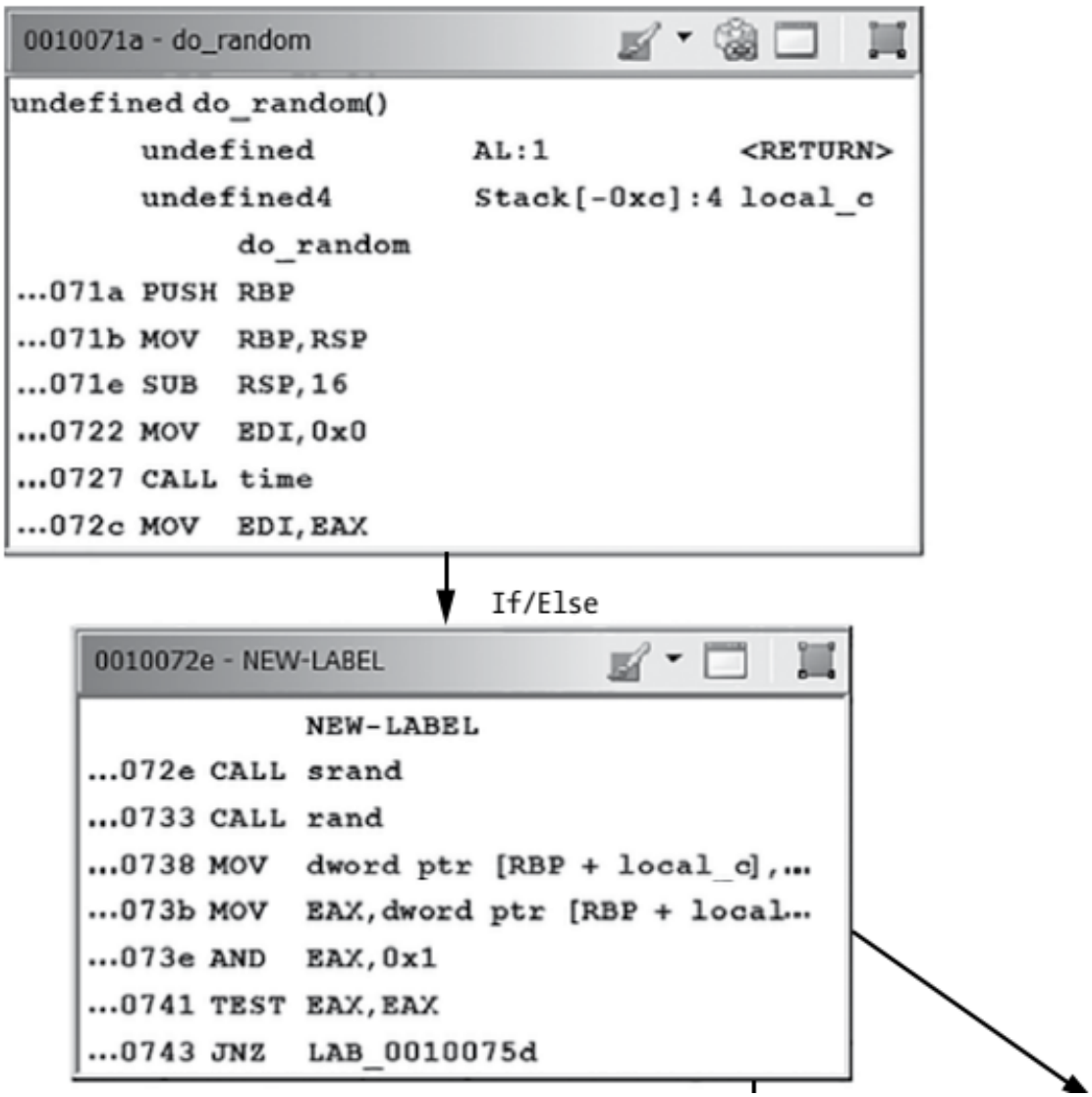


Рисунок 10-9. Граф функции с новой меткой, вводящей новый базовый блок

Взаимодействие с графами функций

Хотя это нелегко показать в книге, окно Function Graph включает цвет, анимацию и информационные всплывающие окна при взаимодействии с различными компонентами графа.

Ребра

Ребра окрашиваются на основе природы перехода, представленного ребром. Цветами можно управлять через окно `Edit > Tool Options`, как показано на рисунке 10-10. По умолчанию зеленое ребро указывает условный переход, когда условие истинно (переход выполнен), красное ребро указывает сквозной переход (переход не выполнен), а синее ребро указывает безусловный переход. Щелчок по отдельному ребру или набору ребер увеличивает толщину ребра и меняет цвет на подсвеченный оттенок того же цвета.

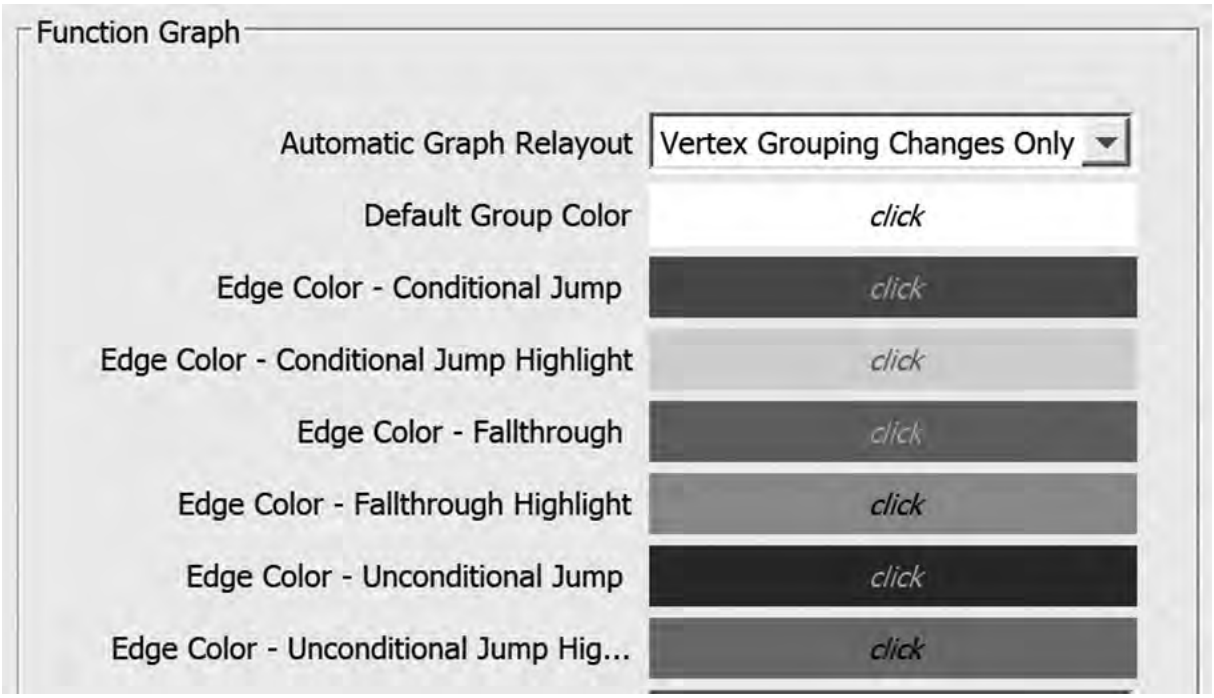


Рисунок 10-10. Параметры настройки цветов Function Graph

Узлы

Содержимое каждого узла - это дизассемблированный листинг соответствующего базового блока. Способ взаимодействия с кодом листинга идентичен взаимодействию с кодом в окне `Listing`. Например, наведение на имена открывает всплывающее окно, отображающее дизассемблирование в именованном местоположении. Когда вы наводите указатель на узел, Ghidra использует анимацию подсветки пути на связанных ребрах, чтобы указать направление потока управления, соответствующее текущим параметрам подсветки пути. Эту функциональность можно отключить в `Edit > Tool Options`.

Satellite

Satellite (малый обзор графа) имеет желтый ореол вокруг блока, который сейчас находится в фокусе, как и окно Function Graph. Для простой идентификации блок входа функции (содержащий адрес точки входа функции) зеленый в satellite, а любые блоки возврата (блоки, содержащие `ret` или эквивалент) красные. Даже если изменить цвет фона связанного блока в графе, цвета входа и выхода в satellite не меняются. Все остальные блоки будут повторять цвет, назначенный им в окне Function Graph.

Графы вызовов функций

Граф вызовов функций полезен для быстрого понимания иерархии вызовов функций, выполняемых внутри программы. Графы вызовов функций похожи на графы функций, но каждый блок представляет все тело функции, а каждое ребро представляет перекрестную ссылку вызова от одной функции к другой.

Чтобы обсудить графы вызовов функций, мы используем следующую тривиальную программу, создающую простую иерархию вызовов функций:

```
#include <stdio.h>

void depth_2_1() {
    printf("inside depth_2_1\n");
}

void depth_2_2() {
    fprintf(stderr, "inside depth_2_2\n");
}

void depth_1() {
    depth_2_1();
    depth_2_2();
    printf("inside depth_1\n");
}

int main() {
    depth_1();
}
```

После компиляции динамически связанной версии этой программы с помощью GNU `gcc` и загрузки бинарного файла в Ghidra можно сгенерировать граф вызовов функций через `Window` > Function Call Graph. По умолчанию это создает граф вызовов функций, центрированный вокруг функции, которая сейчас выбрана. Граф вызовов функций при выбранной `main` показан на рисунке 10-11. (В этих примерах обзор-satellite скрыт для ясности. Чтобы снова показать обзор-satellite, используйте значок в нижнем правом углу рисунка 10-11.)

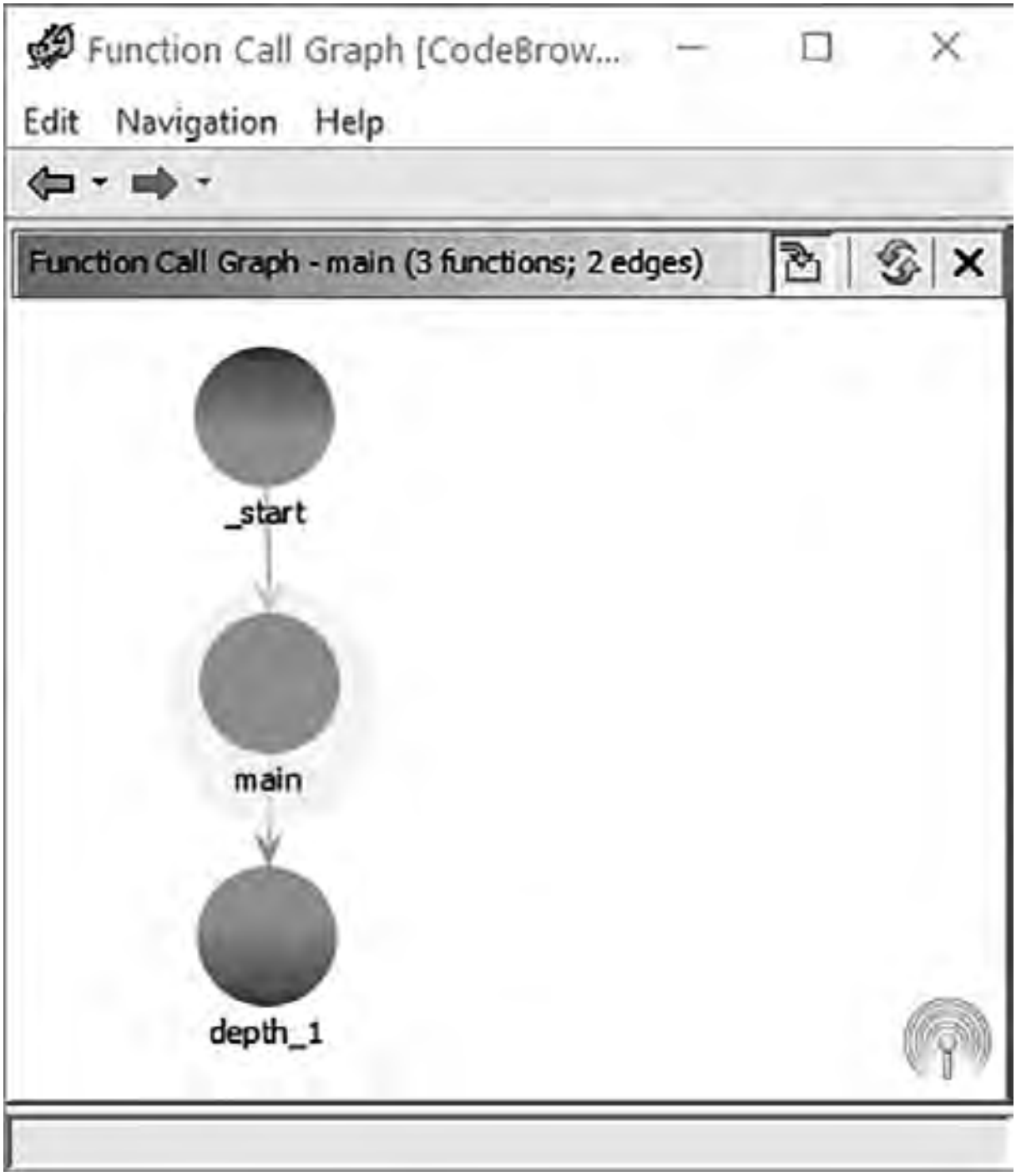


Рисунок 10-11. Простой граф вызовов функций с фокусом на `main`

Строка `main (3 functions; 2 edges)` в заголовке графа сообщает, в какой функции мы находимся, а также число отображенных функций и ребер. Наведение на узел в графе отображает значки + и/или - сверху и/или снизу узла, как показано на рисунке 10-12.

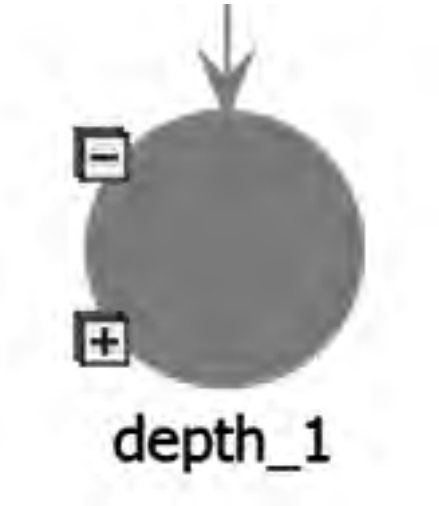


Рисунок 10-12. Узел Function Call Graph со значками разворачивания и сворачивания

Значок + сверху или снизу означает, что можно отобразить дополнительные входящие или исходящие функции. И наоборот, значок - предоставляет возможность свернуть узлы. Например, щелчок по символу - в нижней части функции `depth_1`, когда она развернута, изменит граф вызовов функций с показанного на рисунке 10-13 на граф на рисунке 10-11.

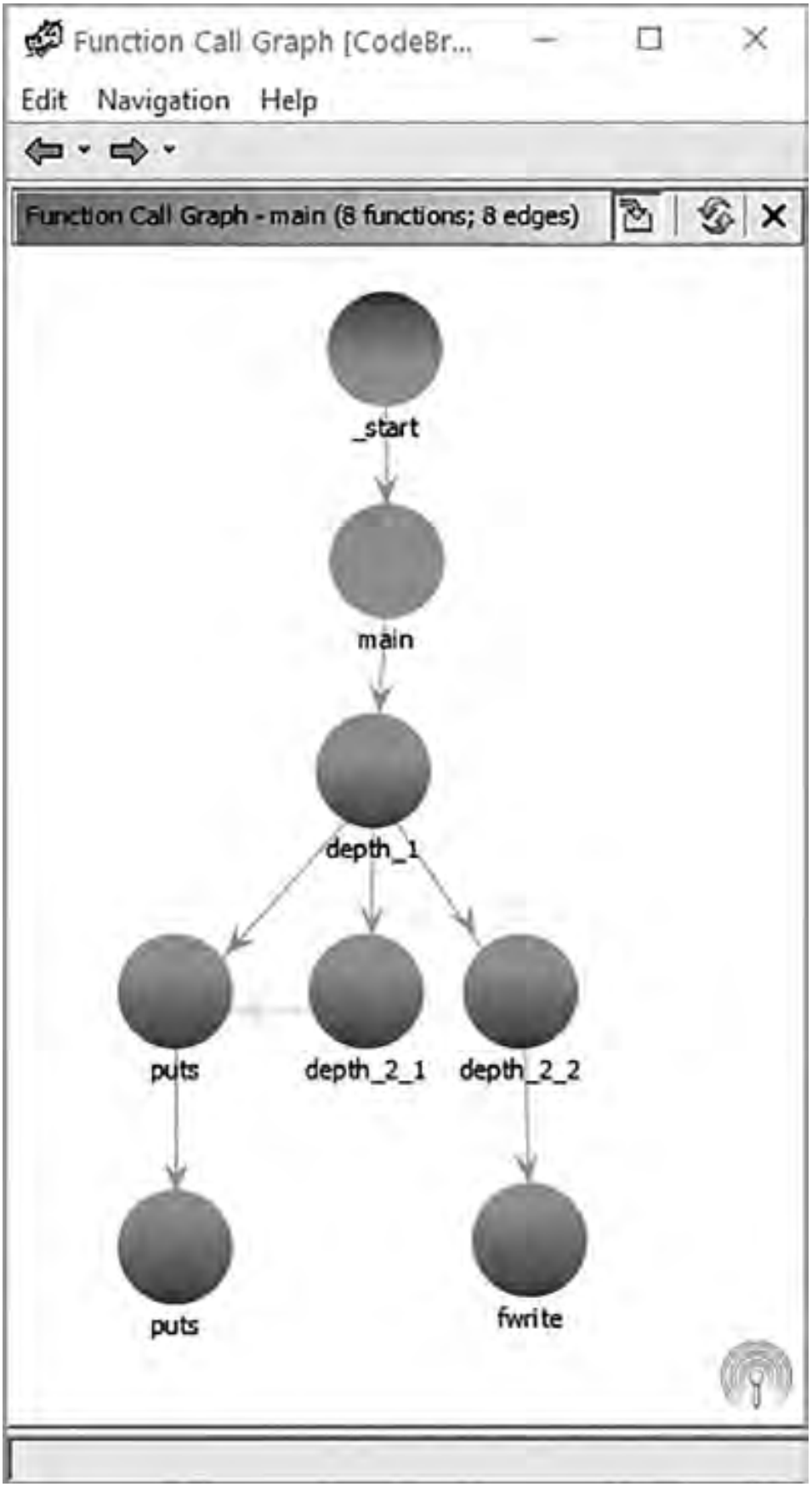


Рисунок 10-13. Граф вызовов функций, развернутый от `main`

Контекстное меню по щелчку правой кнопкой мыши, связанное с каждым узлом, предоставляет варианты одновременного разворачивания или сворачивания всех исходящих ребер для всех

узлов на одном горизонтальном уровне. Это эквивалентно одновременному щелчку по значку + или - на всех узлах одного ранга. Наконец, двойной щелчок по узлу в графе центрирует граф на выбранном узле и полностью разворачивает все входящие и исходящие ребра. Параметр, который по умолчанию отключен, но многим кажется полезным, дает возможность уменьшать масштаб и снова увеличивать его. Этот параметр можно включить через `Edit > Tool Options`, установив флажок `Scroll Wheel Pans`. Ghidra поддерживает короткую историю графов в кэше при смене фокуса, чтобы сохранить состояние графа при возврате. Это позволяет разворачивать и сворачивать узлы, уходить из графа, а затем возвращаться и находить граф ровно в том виде, в котором вы его оставили, чтобы продолжить анализ.

На рисунке 10-14 показана та же программа, но с фокусом на `_start`, а не `main`, и большинство узлов полностью развернуто, чтобы показать полный масштаб графа. Помимо нашей функции `main` и связанных подпрограмм, можно увидеть оберточный код, вставленный компилятором. Этот код отвечает за инициализацию и завершение работы библиотек, а также за правильную настройку окружения перед передачей управления функции `main`. (Внимательные читатели могут заметить, что компилятор заменил вызовы `printf` и `fprintf` на `puts` и `fwrite` соответственно, поскольку они более эффективны при печати статических строк.)

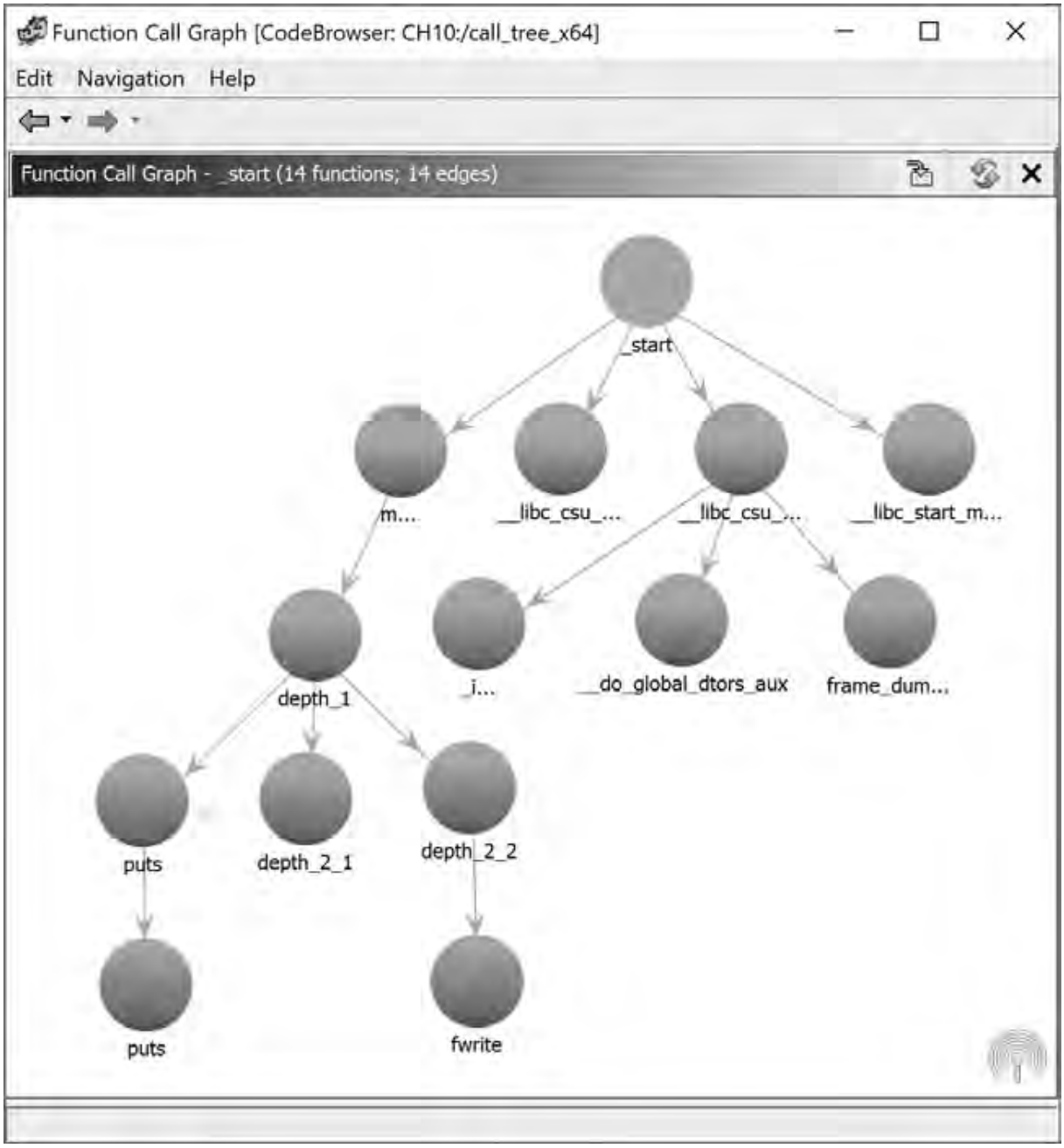


Рисунок 10-14. Развернутый граф вызовов функций от `_start`

Thunk

Вы можете заметить, что граф на рисунке 10-14 показывает несколько (по-видимому, рекурсивных) вызовов `puts`. Добро пожаловать в волшебный мир thunk-функций. Thunk-функция - это прием компилятора, облегчающий вызовы функций, чей адрес неизвестен во время компиляции (например, динамически связанной библиотечной функции). Ghidra называет функцию, адрес которой неизвестен, thunked-функцией. Компилятор заменяет все вызовы thunked-функций, которые делает программа, вызовом заглушки thunk-функции, вставленной компилятором в исполняемый файл. Заглушка thunk-функции обычно выполняет поиск в таблице, чтобы узнать адрес thunked-функции во время выполнения, прежде чем передать управление thunked-функции. Таблица, к которой обращается thunk-заглушка, заполняется во время выполнения после того, как становятся известны адреса связанных thunked-функций. В исполняемых файлах Windows эта таблица обычно называется таблицей импорта. В ELF-бинарных файлах эта таблица

обычно называется глобальной таблицей смещений (или GOT).

Если мы перейдем к `puts` из функции `depth_1` в окне `Listing`, окажемся в следующем листинге:

```
  *************************************************************
  *                         THUNK FUNCTION                    *
  *************************************************************
           thunk int puts(char * __s)
              Thunked-Function: <EXTERNAL>::puts
    int               EAX:4          <RETURN>
    char *            RDI:8          __s
              puts@@GLIBC_2.2.5
              puts     XREF[2]: puts:00100590(T),
                                puts:00100590(c), 00300fc8(*)
00302008                  ??         ??
00302009                  ??         ??
0030200a                  ??         ??
```

Этот листинг thunk-функции появляется в секции программы, которую Ghidra называет EXTERNAL. Листинги thunked-функций Ghidra вроде этого являются следствием того, как внешние библиотеки динамически загружаются и связываются с процессами во время выполнения; это означает, что библиотеки обычно недоступны во время статического анализа. Хотя листинг дает указание на вызываемую функцию и библиотеку, код функции напрямую недоступен (если библиотека также не загружена в Ghidra, что легко сделать на странице параметров во время процесса импорта).

Здесь мы также наблюдаем новый тип XREF. Суффикс `(T)` у первой XREF указывает, что эта XREF является ссылкой на thunked-функцию.

Теперь вернемся к статически связанной версии программы `call_tree`. Начальный граф, сгенерированный от функции `main`, идентичен динамически связанной версии, показанной на рисунке 10-11. Однако, чтобы получить представление о потенциальной сложности графов статически связанных бинарных файлов, исследуем два развертывания, которые кажутся относительно безобидными. На рисунке 10-15 показаны исходящие вызовы из функции `puts`. Строка заголовка показывает `puts(9 functions; 11 edges)`. Обратите внимание, что суммарные значения в заголовке могут быть неточными, пока программа не будет полностью проанализирована.

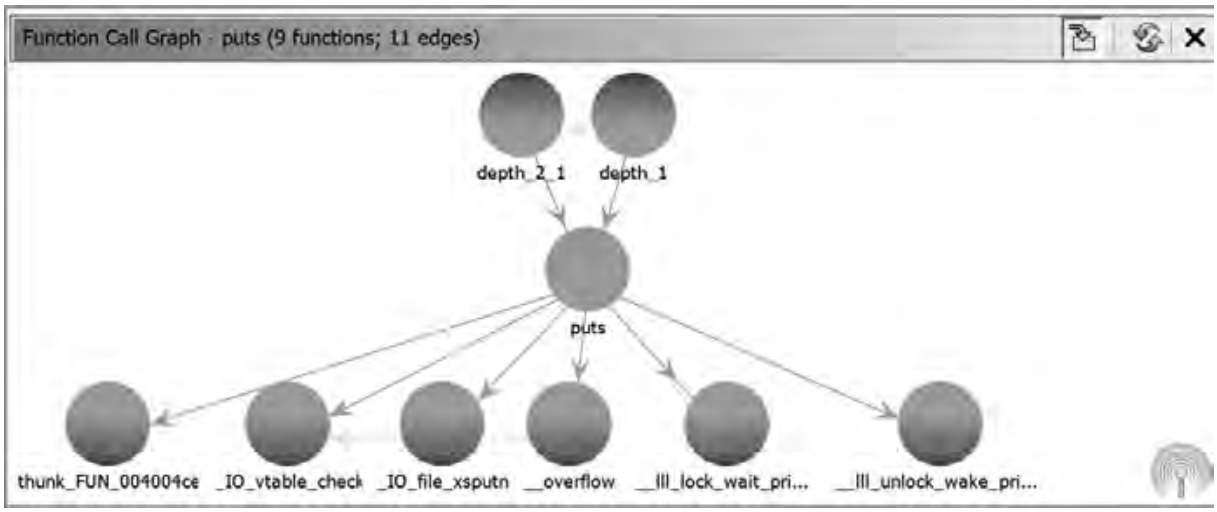


Рисунок 10-15. Граф вызовов функций в статически связанном бинарном файле

Когда мы переносим фокус на `_lll_lock_wait_private`, нам показывается подавляющий граф с 70 узлами и более чем 200 ребрами, часть которого показана на рисунке 10-16.

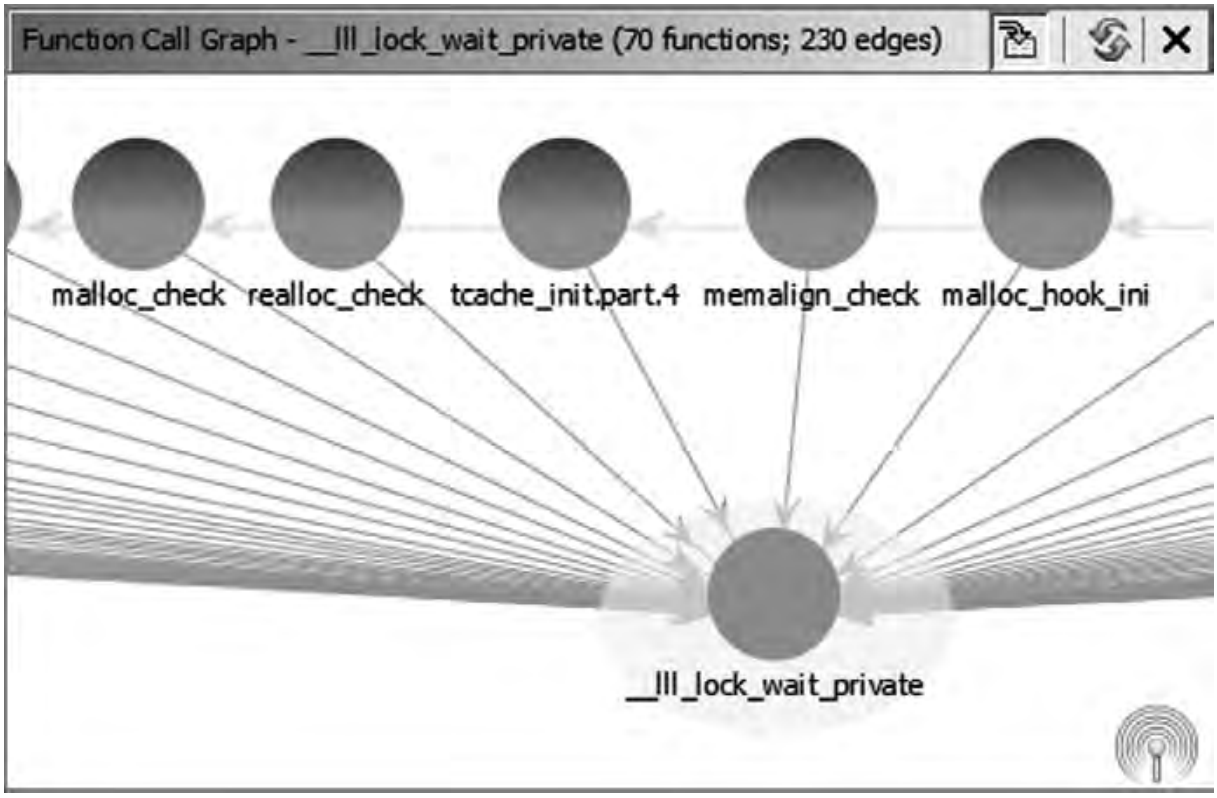


Рисунок 10-16. Развернутый граф вызовов функций в статически связанном бинарном файле

Хотя статически связанные бинарные файлы сложны и работа со связанными графами может быть трудной, две возможности делают это терпимым. Во-первых, обычно можно найти `main` с помощью горячей клавиши `G` или переходом от символа входа программы. Во-вторых, после того как вы нашли `main` в листинге, можно открыть и легко контролировать, что отображается в связанном графе вызовов функций.

Деревья

Ghidra представляет многие иерархические концепции, связанные с конкретным бинарным файлом, как древовидную структуру. Хотя это не всегда деревья в чисто теоретико-графовом смысле, эти структуры дают возможность разворачивать и сворачивать узлы и видеть иерархические отношения между узлами разных типов. Когда мы обсуждали окно `CodeBrowser` в главе 5, вы познакомились с `Program Trees`, `Symbol Tree`, Function Call Tree и `Data Type Manager` (который также представлен как дерево). Эти древовидные представления можно использовать одновременно с другими представлениями, чтобы получить дополнительное понимание анализируемого бинарного файла.

Резюме

Графы - мощный инструмент, помогающий анализировать любой бинарный файл. Если вы привыкли смотреть дизассемблированные представления в чисто текстовом формате, может потребоваться некоторое время, чтобы привыкнуть к графовому отображению. В Ghidra обычно достаточно понять, что вся информация, доступная в текстовом отображении, остается доступной и в графовом отображении; однако она может быть отформатирована несколько иначе. Например, перекрестные ссылки становятся ребрами, соединяющими блоки в графовом отображении.

Какой граф смотреть, зависит от того, что вы хотите узнать о бинарном файле. Если вы хотите знать, как достигается конкретная функция, вас, вероятно, интересует граф вызовов функций. Если вы хотите знать, как достигается конкретная инструкция, вас, вероятно, больше интересует граф функции. Оба могут дать ценное понимание того, как работает программа.

Теперь, когда вы увидели базовую функциональность, доступную при запуске Ghidra как автономного экземпляра с вами в роли единственного специалиста по обратной разработке, пора исследовать варианты использования Ghidra как инструмента совместной работы. В следующей главе мы рассмотрим Ghidra Server и среду, которую он предоставляет для поддержки совместной обратной разработки.

Глава 11. Совместная SRE

К этому моменту вы должны уверенно ориентироваться в окружении проекта Ghidra и во множестве доступных инструментов и окон. Вы знаете, как создать проект, импортировать файлы, перемещаться и манипулировать дизассемблированием. Вы понимаете типы данных, структуры данных и перекрестные ссылки Ghidra. Но понимаете ли вы масштаб? Бинарный файл размером 200 МБ, скорее всего, породит дизассемблирование длиной в миллионы строк и с сотнями тысяч функций. Даже с самым большим монитором в портретной ориентации, который вы сможете найти, вы одновременно увидите лишь несколько сотен строк этого дизассемблирования.

Один способ взяться за такую монументальную задачу - назначить для нее команду людей, но это вводит дополнительную проблему: как синхронизировать усилия всех участников, чтобы люди не наступали друг другу на изменения? Пора расширить обсуждение использования Ghidra и охватить совместную работу команды над общим проектом. Одна только поддержка совместной обратной разработки делает Ghidra уникальной среди инструментов анализа программного обеспечения. В этой главе мы представляем сервер совместной работы Ghidra, входящий в стандартный дистрибутив Ghidra. Мы обсудим его установку, настройку и использование, чтобы помочь направить больше глаз на ваши самые сложные задачи RE.

Командная работа

SRE - сложный процесс, и мало кто является экспертом во всех его тонкостях. Возможность дать аналитикам с разными наборами навыков одновременно анализировать один бинарный файл может резко сократить время, необходимое для получения желаемых результатов. Звезда навигации по потокам управления в сложной программе может содрогаться от необходимости анализировать и документировать связанные структуры данных. Эксперт по анализу вредоносных программ может плохо подходить для поиска уязвимостей, а любой, кого поджимает время, с меньшей вероятностью потратит это время на вставку неизбежного множества комментариев, которые, безусловно, будут полезны в будущем, но в краткосрочной перспективе могут помешать анализировать дополнительный код. Пять коллег могут захотеть по отдельности анализировать один и тот же бинарный файл, но понимать, что есть определенные шаги процесса, которые всем им придется выполнить. Одному человеку может понадобиться передать задание коллеге для экспертного ввода или на время отпуска. Иногда просто полезно иметь несколько пар глаз, смотрящих на одно и то же, для проверки здравого смысла. Какой бы ни была мотивация, возможность общего проекта в Ghidra поддерживает совместную SRE во многих формах.

Настройка Ghidra Server

Совместная работа в Ghidra обеспечивается общим экземпляром Ghidra Server. Если вы системный администратор, отвечающий за настройку Ghidra Server, вам предстоит сделать много выборов, например развернуть его на физическом сервере или в виртуальной среде ради простоты миграции и повторяемой установки. Развертывание, которое мы используем в этой главе для демонстрации совместных возможностей Ghidra, подходит только для разработки и экспериментов. Если вы настраиваете Ghidra Server для промышленного использования, следует внимательно прочитать документацию Ghidra Server и определить подходящую конфигурацию для вашей среды и конкретного сценария. (Об установке Ghidra Server, всех вариантах установки и связанных подходах можно было бы написать отдельную книгу, но это не эта книга.)

Хотя Ghidra Server можно настроить на всех платформах, поддерживающих Ghidra, мы опишем запуск экземпляра Ghidra Server в окружении Linux и будем предполагать некоторое знакомство с командной строкой Linux и системным администрированием. Мы внесем несколько небольших изменений в конфигурационный файл Ghidra Server (указанный в `server/server.conf`), чтобы облегчить концепции, которые хотим продемонстрировать в этой главе, и не слишком полагаться на интерфейс командной строки Linux после завершения начальной установки, настройки, администрирования и контроля доступа. Изменения включают перенос стандартного каталога репозиториев Ghidra в выбранное нами место, как рекомендовано в документации Ghidra Server, а также настройку параметров управления пользователями и контроля доступа.

Следующие шаги проведут вас через скриптовый процесс создания окружения и начального набора пользователей Ghidra на хосте Ubuntu.

1. Определите несколько переменных окружения, используемых во всем скрипте, включая версию Ghidra, которую вы устанавливаете:

```
# задать некоторые переменные окружения
OWNER=ghidrasrv
SVRROOT=/opt/${OWNER}
REPODIR=/opt/ghidra-repos
GHIDRA_URL=https://ghidra-sre.org/ghidra_version.zip
GHIDRA_ZIP=/tmp/ghidra.zip
```

2. Установите два пакета (`unzip` и OpenJDK), необходимые для завершения установки и запуска сервера:

```
sudo apt update && sudo apt install -y openjdk-version-jdk unzip
```

3. Создайте непривилегированного пользователя для запуска сервера и каталог для размещения общих репозиториев Ghidra вне каталога, в который будет установлен Ghidra Server. Разделение исполняемых файлов сервера и репозиториев рекомендовано в руководстве по конфигурации сервера и упрощает будущие обновления сервера. Инструмент администрирования Ghidra Server (`svrAdmin`) будет использовать домашний каталог пользователя администратора сервера.

> **Особые предложения Ghidra Server**
>
> Ваш Ghidra Server рад предложить следующие варианты установки.
>
> `Platforms` - физическое железо, виртуальные машины, контейнеры и многое другое.
>
> `Operating systems` - разные варианты Windows, Linux и macOS. Что-нибудь на любой вкус.
>
> `Authentication methods` - выбирайте, как друзья и коллеги смогут получить доступ: от "открыто для широкой публики" до "только PKI" и всего промежуточного.
>
> `Preparation` - можно установить из контейнера, скрипта, `.bat`-файлов или подробных инструкций, а можно сделать собственный рецепт, стуча по консоли, пока не произойдет что-нибудь захватывающее.
>
> Если вы не видите того, что вам нравится, не беспокойтесь: это лишь часть доступного. Варианты Ghidra Server способны удовлетворить даже самых взыскательных гостей и предоставить им установку Ghidra их мечты. Благодарим за посещение вашего Ghidra Server. Дополнительные сведения см. в расширенном серверном меню, `server/svrREADME.html`, доступном в ближайшем каталоге Ghidra.

```
sudo useradd -r -m -d /home/${OWNER} -s /usr/sbin/nologin -U ${OWNER}
sudo mkdir ${REPODIR}
sudo chown ${OWNER}.${OWNER} ${REPODIR}
```

4. Скачайте Ghidra, распакуйте его и переместите в корневой каталог сервера. Убедитесь, что при скачивании Ghidra вы берете последний публичный выпуск (дата выпуска есть в имени файла `.zip`):

```
wget ${GHIDRA_URL} -O ${GHIDRA_ZIP}
mkdir /tmp/ghidra && cd /tmp/ghidra && unzip ${GHIDRA_ZIP}
sudo mv ghidra_* ${SVRROOT}
cd /tmp && rm -f ${GHIDRA_ZIP} && rmdir ghidra
```

5. Создайте резервную копию исходного файла конфигурации сервера и измените местоположение, где будут сохраняться репозитории:

```
cd ${SVRROOT}/server && cp server.conf server.conf.orig
REPOVAR=ghidra.repositories.dir
sed -i "s@^$REPOVAR=.*\$@$REPOVAR=$REPODIR@g" server.conf
```

6. Добавьте параметр `-u` к параметрам запуска Ghidra Server, чтобы пользователи могли указывать имя пользователя при подключении, вместо принудительного использования локального имени пользователя. Этот вариант позволяет для демонстрации входить как несколько разных пользователей с одной машины и входить в одну учетную запись с нескольких машин. (Некоторые версии Ghidra ожидают путь репозитория как последний параметр командной строки, поэтому мы изменили `parameter.2` на `parameter.3`, а затем добавили новую строку `parameter.2=-u` перед обновленной строкой.)

```
PARM=wrapper.app.parameter.
sed -i "s/^${PARM}2=/${PARM}3=/" server.conf
sed -i "/^${PARM}3=/i ${PARM}2=-u" server.conf
```

7. Измените владельца процесса Ghidra Server и каталога Ghidra Server на пользователя `ghidrasvr`. (Поскольку это был лишь демонстрационный сервер, мы оставили все остальные параметры неизменными. Настоятельно рекомендуется прочитать `server/svrREADME.html`, чтобы определить конфигурации, подходящие для промышленного развертывания.)

```
ACCT=wrapper.app.account
sed -i "s/^.*$ACCT=.*/$ACCT=$OWNER/" server.conf
sudo chown -R ${OWNER}.${OWNER} ${SVRROOT}
```

8. Наконец, установите Ghidra Server как службу и добавьте пользователей, которым разрешено подключаться к серверу:

```
sudo ./svrInstall
sudo ${SVRROOT}/server/svrAdmin -add user1
sudo ${SVRROOT}/server/svrAdmin -add user2
sudo ${SVRROOT}/server/svrAdmin -add user3
```

Хотя более подробное обсуждение контроля доступа будет далее в главе, важно упомянуть его здесь, поскольку пользователи должны существовать в системе аутентификации, которую использует ваш экземпляр Ghidra Server. Это происходит на самом Ghidra Server. По умолчанию каждый пользователь должен войти из клиента Ghidra в течение 24 часов, используя стандартный пароль `changeme` (который должен быть изменен при первом входе). Если пользователь не активирует учетную запись в течение 24 часов, учетная запись блокируется и должна быть сброшена. Ghidra предоставляет системному администратору Ghidra Server несколько вариантов аутентификации: от простых паролей до инфраструктуры открытых ключей (PKI). Мы выбрали локальный пароль Ghidra (это значение по умолчанию).

Если вы хотите установить собственный Ghidra Server или просто нуждаетесь в более подробном описании различных вариантов установки, см. `server/svrREADME.html` в каталоге Ghidra.

Репозиторий проекта

Одно преимущество совместной командной работы состоит в том, что несколько людей могут одновременно работать над одним бинарным файлом. Один недостаток совместной

командной работы состоит в том, что несколько людей могут одновременно работать над одним бинарным файлом. Всякий раз, когда несколько пользователей взаимодействуют с одним и тем же содержимым, существует возможность возникновения состояния гонки. В состоянии гонки порядок, в котором выполняются операции (например, сохранение обновленного файла), может повлиять на итоговый результат. У Ghidra есть репозиторий проекта и система версионирования, чтобы управлять тем, какие изменения фиксируются, когда и кем.

Репозиторий Ghidra берет файлы на редактирование и возвращает их, отслеживает историю версий и позволяет видеть, что сейчас взято на редактирование. Когда вы берете файл на редактирование, вы получаете копию файла. Когда вы завершаете работу с файлом и возвращаете его, создается новая версия файла, которая становится частью наследия файла; если кто-то еще тоже вернул новую версию файла, репозиторий помогает разрешить любые конфликты. Мы продемонстрируем взаимодействия с репозиторием далее в главе.

Общие проекты

До этого момента мы создавали и использовали только автономные проекты Ghidra, подходящие для одного аналитика, работающего на одном компьютере. Теперь, когда вы настроили Ghidra Server и предоставили себе доступ к нему, пройдем процесс создания общего проекта. Общий проект можно сделать доступным для любых пользователей, которым разрешено подключаться к вашему Ghidra Server, и он облегчает совместный, одновременный доступ к проекту.

Создание общего проекта

Когда вы создаете новый проект (`File > New Project`) и выбираете `Shared Project`, нужно указать сведения о сервере, связанные с вашим Ghidra Server, как показано слева на рисунке 11-1. Предоставляется стандартный номер порта (`13100`), но необходимо указать имя хоста или IP-адрес сервера, а также, в зависимости от конфигурации Ghidra Server, возможно, пройти аутентификацию.

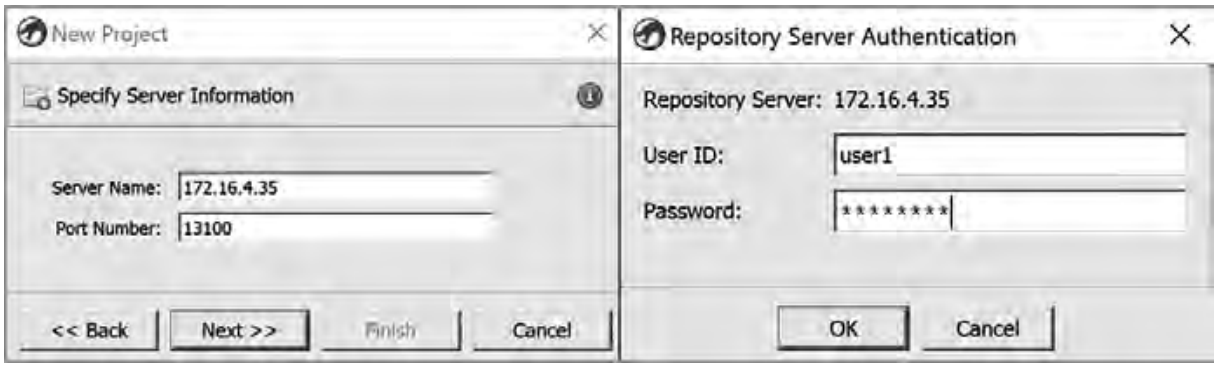


Рисунок 11-1. Вход в репозиторий Ghidra Server

Справа на рисунке мы входим как один из пользователей, созданных нашим установочным скриптом (`user1`). Если это первый вход под этим пользователем, вам нужно изменить пароль с `changeme`, как обсуждалось ранее.

Затем выберите существующий репозиторий или создайте новый, введя новое имя репозитория, как показано на рисунке 11-2. Для этого примера мы создадим новый репозиторий с именем `CH11`.

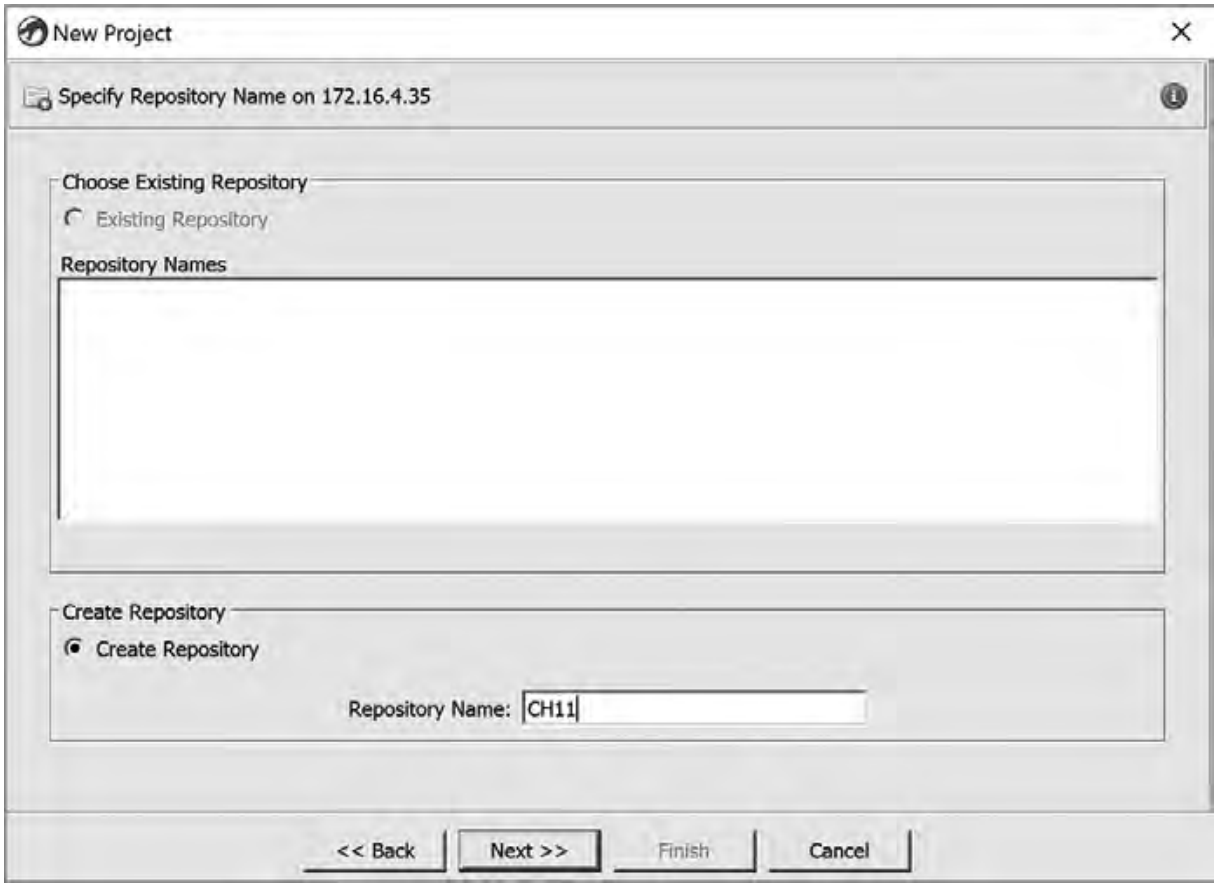


Рисунок 11-2. Диалог New Project

Щелчок Next создает новый репозиторий и новый проект и переносит вас к уже знакомому окну `Project` (рисунок 11-3).

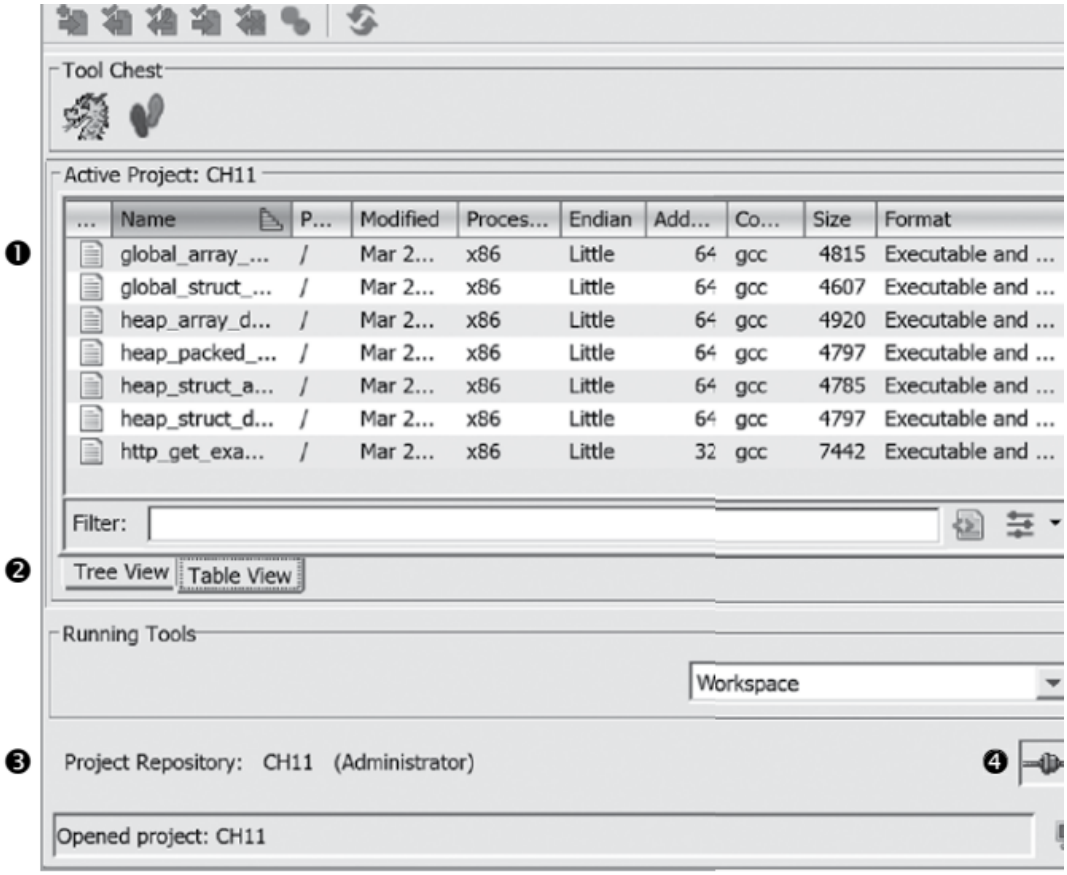


Figure 11-3: Project window for shared project showing Table View

We have imported some files ❶ and are displaying them using a table instead of the default tree structure for project files. The *Table View*, which

Рисунок 11-3. Окно `Project` *для общего проекта с Table View*

Мы импортировали несколько файлов и отображаем их в виде таблицы вместо стандартной древовидной структуры файлов проекта. Table View, один из вариантов раскладки вкладок, предоставляет гораздо больше информации о каждом файле проекта. Окно `Project` показывает имя репозитория проекта (`CH11`), вашу роль в проекте (`Administrator`) и значок справа, предоставляющий сведения о подключении к серверу. В этом случае наведение на значок отображает сообщение "Connected as user1 to 172.16.4.35." Если бы вы не были подключены, значок был бы разорванной ссылкой, а не подключенной ссылкой, показанной на изображении.

Управление проектами

После создания проекта и назначения администратора авторизованные пользователи могут войти на сервер и работать с проектом. Успешный вход приводит вас в окно `Ghidra Project`, где будет доступ к разрешенным проектам.

Меню окна Project

Теперь, когда мы создали и подключились к Ghidra Server, варианты, доступные в окне `Project`, становятся более осмысленными: некоторые ранее недоступные параметры теперь получают новый контекст. Здесь, а также в главе 12, мы обсудим отдельные компоненты меню и то, как их можно использовать для улучшения процесса анализа.

File

Меню File показано на рисунке 11-4. Первые пять пунктов меню File - довольно стандартные файловые операции, и их поведение соответствует тому, что вы ожидаете от приложений с меню. Мы подробно обсудим заметные пункты, отмеченные цифрами.

Кто здесь главный?

Администратор сервера отвечает за создание учетных записей Ghidra Server и настройку протокола аутентификации для подключений к серверу. Администрирование сервера по своей природе ориентировано на командную строку, и от администратора сервера не требуется самому быть пользователем Ghidra. На стороне клиента любой авторизованный пользователь может создавать репозитории на Ghidra Server, автоматически становясь администратором каждого созданного им репозитория. Это дает ему полный контроль над репозиторием, включая то, кто может иметь к нему доступ и какой тип доступа может иметь каждый пользователь. После создания администраторы могут предоставлять доступ дополнительным авторизованным пользователям через окно `Project` в Ghidra.

Я не хочу делиться

Использование установки Ghidra Server для необщих проектов также имеет преимущества. Ваше первое знакомство с Ghidra было сосредоточено на установке Ghidra на одном компьютере и использовании этого компьютера для доступа к проектам и файлам (которые все хранились на этом компьютере). Это означает, что вся работа анализа зависит от этого компьютера. Ghidra Server облегчает доступ к файлам из разных точек и с широкого набора устройств. Можно требовать аутентификацию перед доступом к файлам, и можно преобразовать проекты из необщих в общие при желании. Одно ограничение состоит в том, что нужно быть подключенным к Ghidra Server, чтобы брать файлы на редактирование или возвращать их.

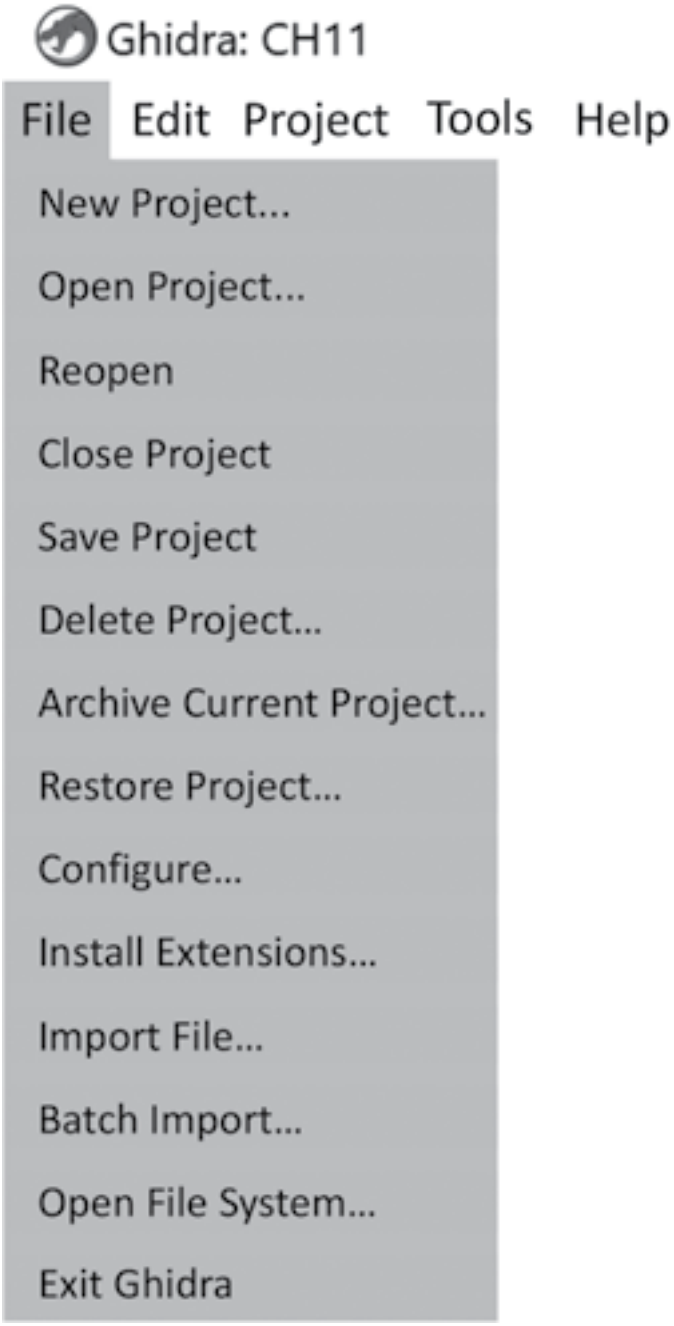


Рисунок 11-4. Меню File

Удаление проектов

Удаление проекта - постоянное действие в Ghidra, которое нельзя отменить. К счастью, оно требует усилий и подтверждения. Во-первых, нельзя удалить активный проект. Это снижает опасность случайного удаления. Чтобы удалить проект, необходимо выполнить три шага:

1. Выберите `File > Delete Project` в меню.
2. Перейдите к проекту, который нужно удалить, или введите его имя.
3. Подтвердите, что хотите удалить проект, в появившемся окне подтверждения.

Удаление проекта удаляет все связанные с ним файлы. Поэтому может быть разумно сначала архивировать проект через пункт `Archive Current Project`.

Архивирование проектов

Архивирование проекта позволяет сохранить снимок проекта, связанных файлов и связанных конфигураций инструментов. Причины архивировать проект включают следующее:

• Вы собираетесь удалить проект, но хотите сохранить копию "на всякий случай".
• Вы хотите упаковать проект для миграции на другой сервер.
• Вы хотите версию, которую можно легко переносить между версиями Ghidra.
• Вы хотите создать резервную копию проекта.

Чтобы архивировать проект, выполните следующие шаги:

1. Закройте окно `CodeBrowser` и все связанные инструменты.
2. Выберите `File > Archive Current Project` в меню.
3. Выберите местоположение и имя архивного файла на локальной машине.

Если вы выберете имя существующего файла, у вас будет возможность изменить имя или перезаписать существующий файл. Архивированные файлы можно легко восстановить через пункт `Restore Project`.

Batch Import

Пункт Batch Import позволяет импортировать коллекцию файлов в проект одной операцией. Когда вы выбираете File > Batch Import, Ghidra показывает окно выбора файлов, похожее на показанное на рисунке 11-5. Это окно позволяет перейти к каталогу, содержащему файлы, которые вы хотите импортировать.

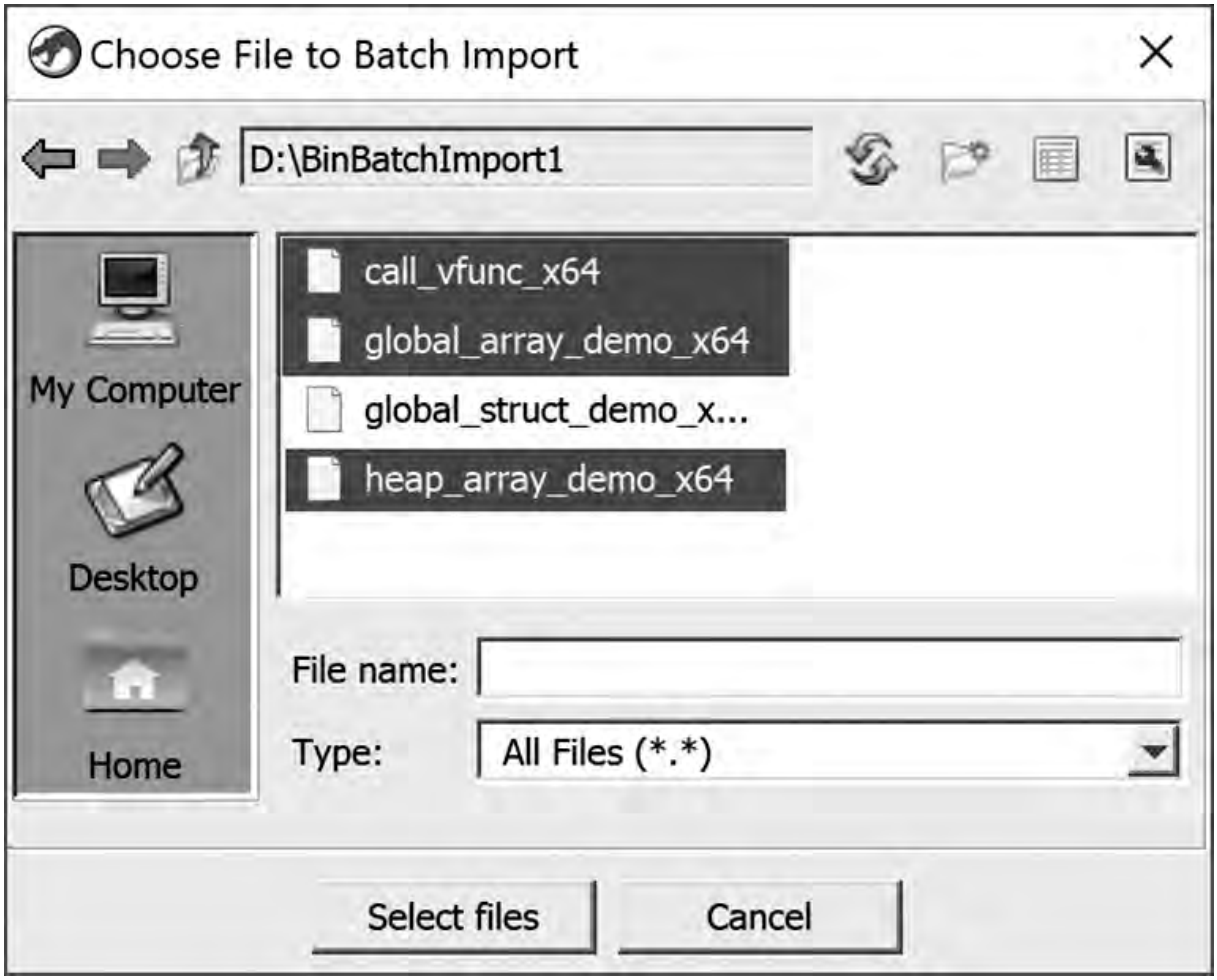


Рисунок 11-5. Окно выбора пакетного импорта файлов с выбранными файлами

Можно выбрать файл (или файлы) из одного каталога либо весь каталог для добавления в список пакетного импорта. После выделения файлов и щелчка `Select files` вы переходите в окно Batch Import, где показаны файлы, уже выбранные для импорта. На рисунке 11-6 файлы из каталога `BinBatchImport1` были загружены как отдельные файлы, а каталог `BinBatchImport2` был добавлен как каталог с пятью файлами, что показано справа от имени каталога. Можно добавлять/удалять файлы, чтобы уточнить список импорта, и управлять несколькими параметрами, включая глубину рекурсии для поиска файлов в каталоге.

Чтобы определить подходящий предел глубины в окне Batch Import или просто исследовать файловую систему, используйте пункт меню Open File System. Этот пункт открывает выбранный контейнер файловой системы (`.zip`-файл, `.tar`-файл, каталог и т. д.) в отдельном окне. (Лучше определить глубину заранее, потому что вам понадобится второй экземпляр Ghidra, чтобы работать с обоими окнами одновременно. Каждое окно блокирует доступ к другому в одном экземпляре.)

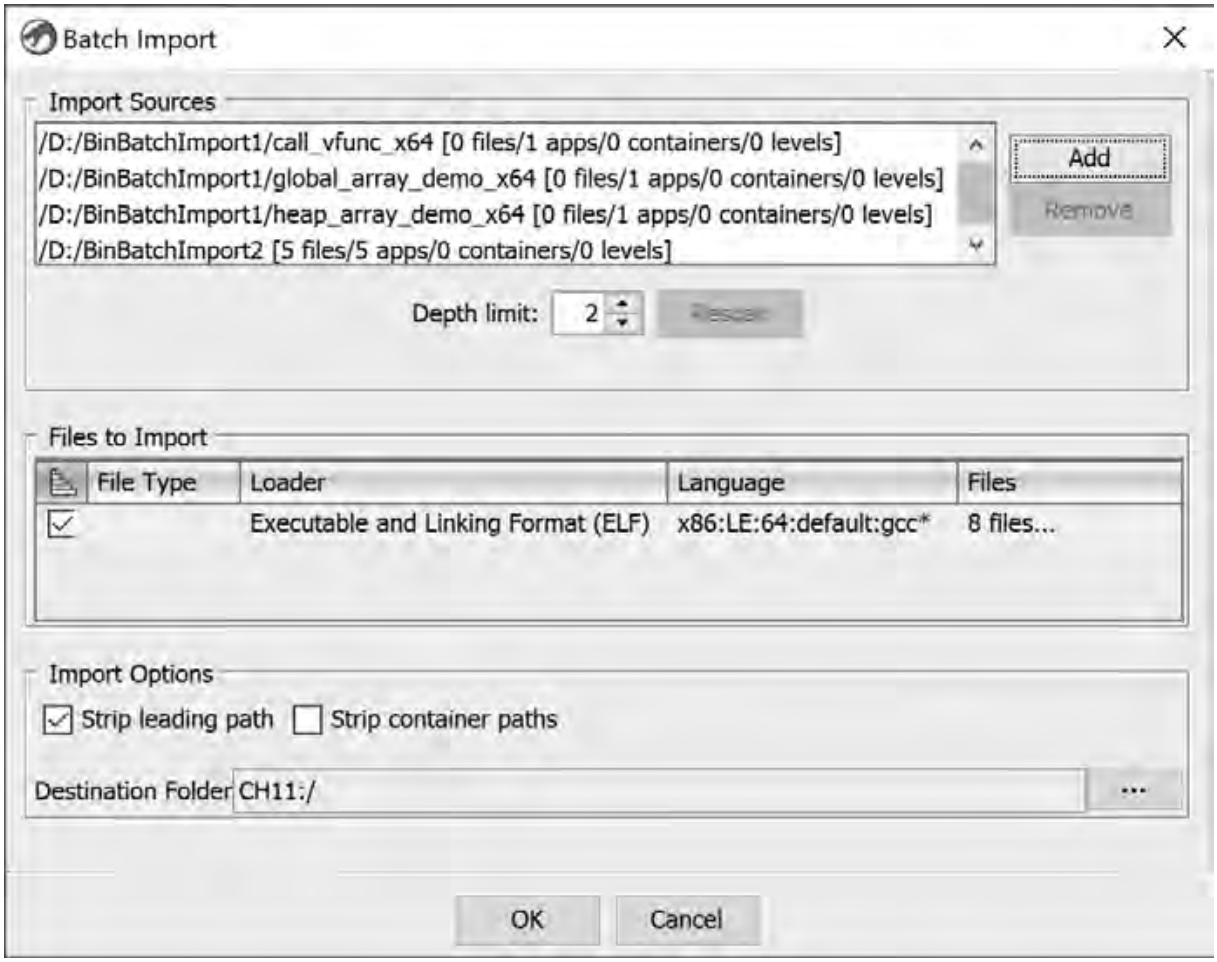


Рисунок 11-6. Диалог подтверждения пакетного импорта файлов

Edit

Меню Edit показано на рисунке 11-7. Пункты `Tool Options` и `Plugin Path` обсуждаются в главе 12, но параметры PKI связаны с настройкой Ghidra Server и заслуживают обсуждения в этой главе.

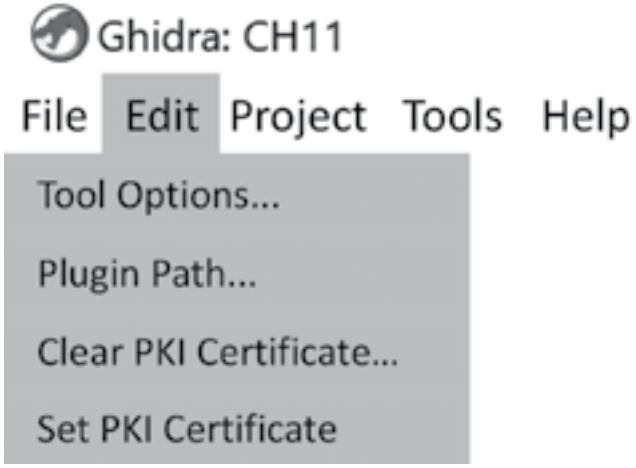


Рисунок 11-7. Меню Edit

Сертификаты PKI

Как упоминалось в начале этой главы, при настройке Ghidra Server можно выбрать один из нескольких методов аутентификации. Мы настроили простой сервер, использующий имя пользователя и пароль для аутентификации. Сертификаты PKI сложнее. Хотя реализации PKI могут различаться, следующий пример представляет разумный процесс клиентской аутентификации PKI для Ghidra Server.

`User1` хочет пройти аутентификацию, чтобы работать над своим проектом Ghidra Server. У нее есть клиентский сертификат, включающий ее имя пользователя и открытый криптографический ключ. У нее также есть закрытый криптографический ключ, соответствующий открытому ключу в сертификате, который она надежно прячет для важных случаев вроде этого. Ее сертификат был криптографически подписан центром сертификации (CA), которому доверяет Ghidra Server.

`User1` предъявляет серверу свой сертификат, из которого сервер может извлечь открытый ключ и имя пользователя. Сервер выполняет проверки, подтверждающие действительность сертификата (например, что он не находится в списке отозванных сертификатов, находится в действительном диапазоне дат, имеет действительную подпись от доверенного CA и, возможно, другие проверки). Если все проверки проходят, сервер подтверждает действительный сертификат и связывает личность `User1` с открытым ключом. Теперь `User1` должна доказать, что у нее есть соответствующий закрытый ключ, чтобы Ghidra Server мог проверить его по извлеченному открытому ключу. Пока закрытый ключ действительно находится только у `User1`, Ghidra Server корректно проверяет ее сертификат, а сервер подтверждает, что `User1` фактически обладает закрытым ключом, поэтому `User1` считается аутентифицированной.

Процесс управления центрами сертификации PKI описан в файле README Ghidra Server (`server/svrREADME.html`). Пункты меню Set PKI Certificate и Clear PKI Certificate позволяют пользователю связать себя с файлом ключей (`*.pfx`, `*.pks`, `*.p12`) или удалить такую связь. При установке PKI-сертификата пользователю будет показано окно навигации по файлам для выбора подходящего хранилища ключей. Сертификат можно очистить в любое время через пункт Clear PKI Certificate. Если вы решите включить PKI-аутентификацию, утилиту Java `keytool` можно использовать для управления ключами, сертификатами и Java-хранилищами ключей.

Project

Меню `Project`, показанное на рисунке 11-8, предоставляет средства для управления действиями на уровне проекта, включая просмотр и копирование из других проектов, изменение пароля и управление доступом пользователей к проектам, которые вы администрируете.

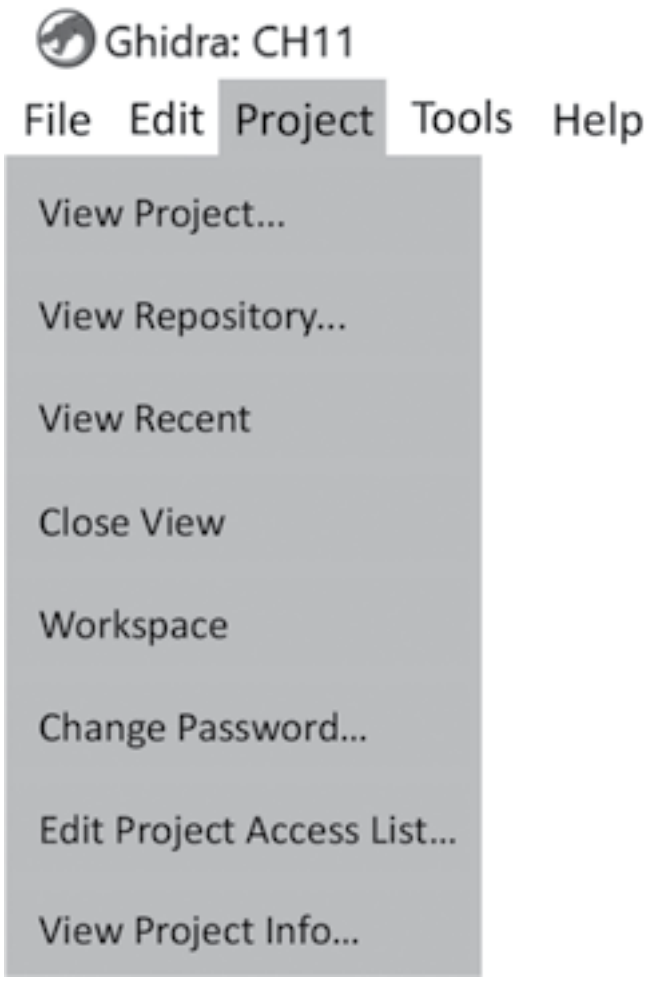


Рисунок 11-8. Меню `Project`

Просмотр проектов и репозиториев

Первые четыре пункта связаны с просмотром проектов и репозиториев. Первые два, `View Project` и `View Repository`, открывают версии проекта (локального) или репозитория (удаленного сервера) только для чтения в новом окне рядом с окном `Active Project`. На рисунке 11-9 локальный проект `ExtraFiles` открыт рядом с активным проектом. Можно исследовать проект только для чтения или перетащить любой файл или каталог из окна `Read-Only Project Data` в окно `Active Project`. На рисунке 11-9 три выбранных файла (с расширением `NEW`) были скопированы из окна `Project Data` в активный проект `CH11`.

Следующий пункт, View Recent, предоставляет список недавних проектов, который может ускорить процесс поиска проекта или репозитория. Close View закрывает представление только для чтения (хотя в некоторых версиях Ghidra этот пункт, похоже, неактивен). Более простая надежная альтернатива - щелкнуть X в нижней части вкладки проекта, которую вы хотите закрыть, как видно в правом нижнем углу рисунка 11-9.

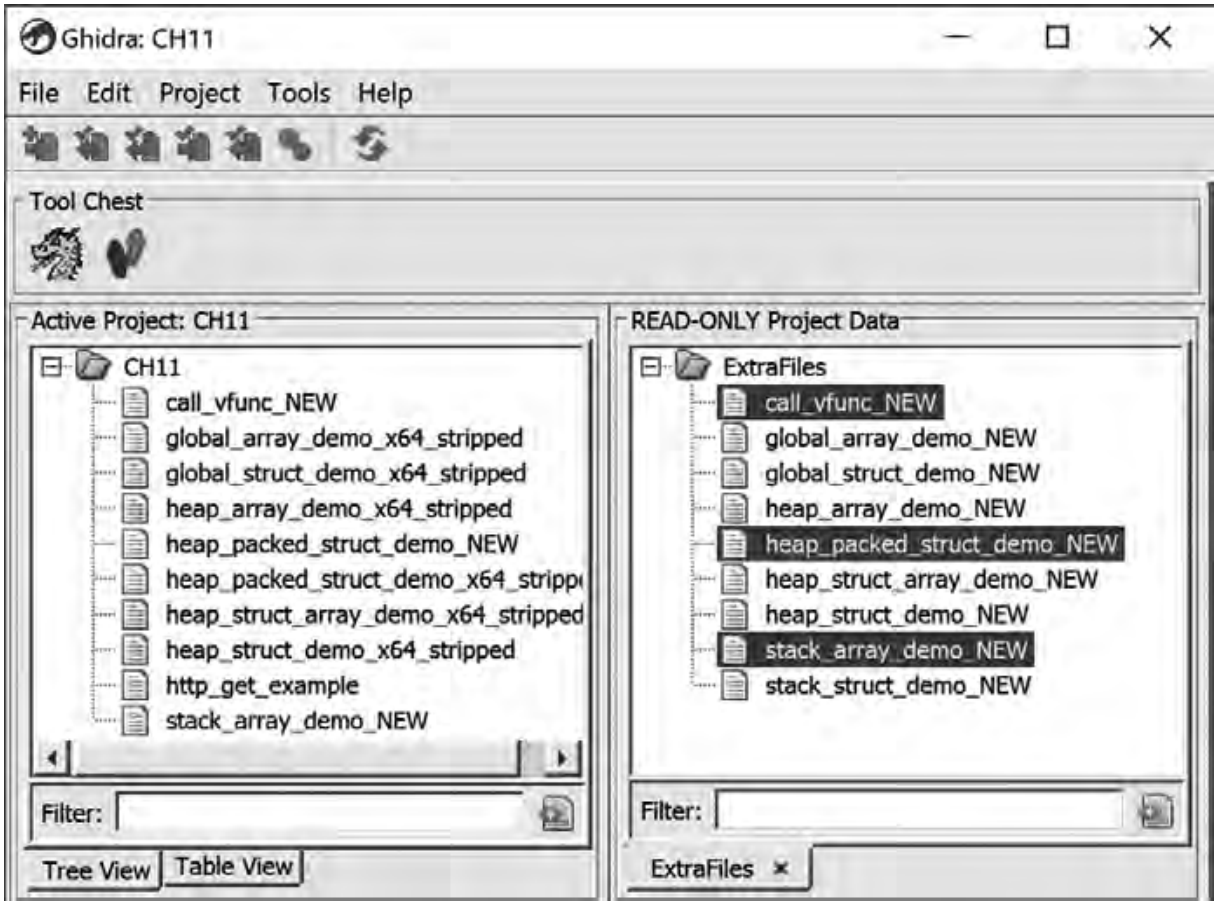


Рисунок 11-9. Использование окна `Project` *для просмотра другого проекта*

Изменение паролей и доступа к проекту

Пункт Change Password доступен только пользователям общих проектов при условии, что Ghidra Server настроен с методом аутентификации, позволяющим смену пароля. Это двухшаговый процесс с начальным диалогом подтверждения, как показано на рисунке 11-10, за которым следует тот же диалог смены пароля, который используется для начального обязательного изменения пароля.

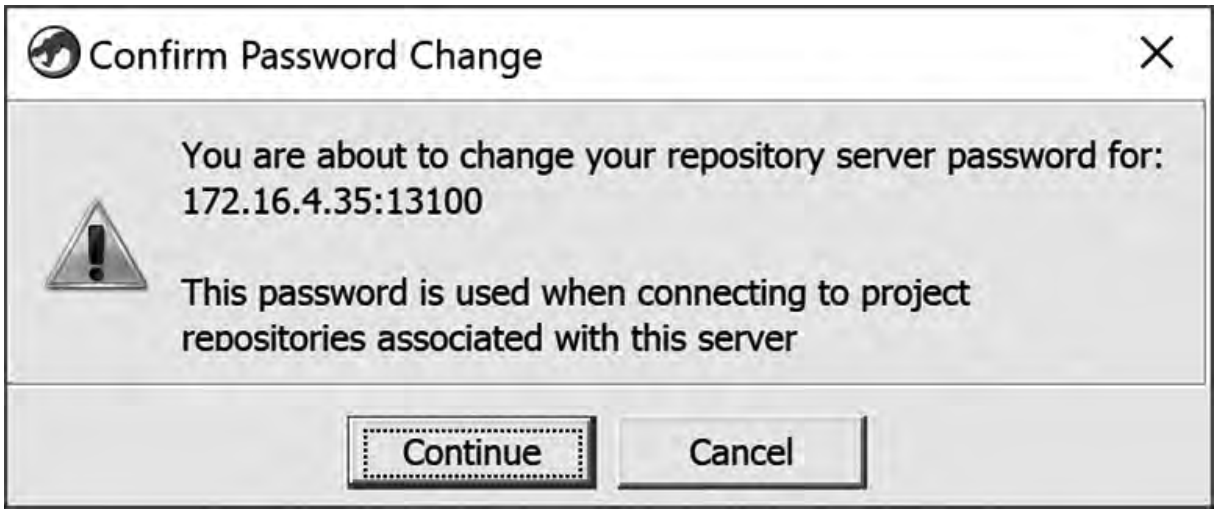


Рисунок 11-10. Начальный диалог подтверждения смены пароля

Хотя каждый пользователь может управлять собственным паролем, общие проекты также предлагают возможность управлять тем, кто может получить доступ к проекту и какие разрешения предоставлены каждому пользователю. Как упоминалось ранее в главе, системный администратор Ghidra Server имеет некоторый контроль над доступом. В частности, администратор может назначить администратора репозитория и создавать и удалять учетные записи пользователей.

На стороне клиента, если вы являетесь администратором, вы также можете управлять доступом через пункт `Edit Project Access List` в меню `Project`. При его выборе появится диалог, показанный на рисунке 11-11, позволяющий добавлять и удалять пользователей проекта и управлять связанными разрешениями. Каждый пользователь может быть помещен ровно в один класс привилегий: от наименее привилегированного (Read Only слева) до наиболее привилегированного (Admin справа).

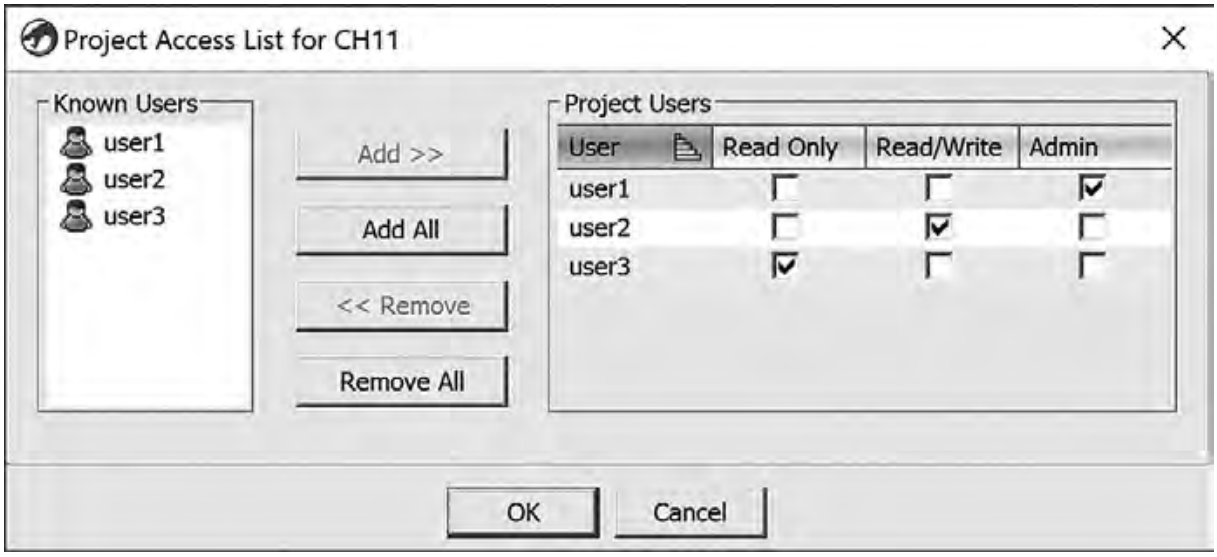


Рисунок 11-11. Окно контроля доступа

Просмотр информации о проекте

Последний пункт меню - `View Project Info`. Параметры, доступные в появившемся диалоге, зависят от того, размещен ли проект на Ghidra Server. Примеры диалогов информации о проекте для проекта без сервера (слева) и с сервером (справа) показаны на рисунке 11-12. Хотя отображаемая информация довольно прямолинейна, кнопки в нижней части каждого окна позволяют преобразовать необщий проект в общий проект (кнопкой Convert to Shared) или изменить информацию проекта.

Щелчок по кнопке Convert to Shared открывает диалог, который просит указать сведения о сервере и ввести ID пользователя и пароль администратора проекта. Последующие шаги позволяют указать репозиторий, добавить пользователей, задать их разрешения и подтвердить, что вы хотите преобразовать проект. Обратите внимание: эту операцию нельзя отменить, и она удаляет всю существующую локальную историю версий.

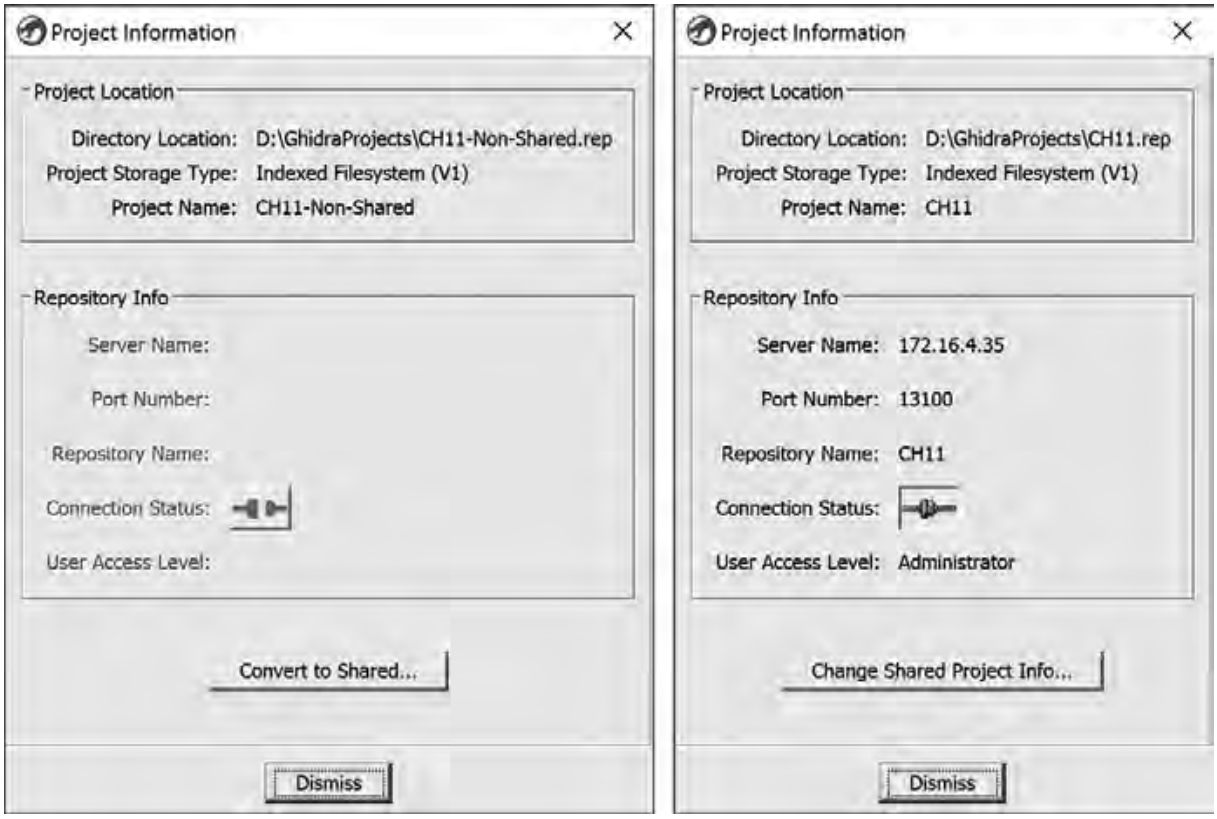


Рисунок 11-12. Окна информации о проекте для необщего и общего проектов

Репозиторий проекта

На этом этапе вы можете задаваться вопросом, как проекты могут быть общими, сохраняя целостность проекта. Этот раздел охватывает процесс, который Ghidra использует, чтобы обеспечить сохранение работы всех участников в общем проекте, над которым команда может работать одновременно. Прежде чем слишком глубоко погружаться в процесс, исследуем типы файлов, связанные с общим проектом Ghidra. Начнем с обсуждения отношений между проектом и репозиторием.

Репозиторий является ключевым посредником процесса версионирования. Когда вы создаете новый необщий проект, создаются файл проекта (`.gpr`) и каталог репозитория с расширением `.rep`, чтобы обеспечить контроль версий. Дополнительные файлы создаются для управления блокировками, версиями и т. д., но понимание назначения каждого файла не критично для успешного использования Ghidra. Для необщих проектов все файлы находятся на вашем компьютере в каталогах, которые вы указываете при создании проекта (см. главу 4).

Когда вы создаете общий проект, у вас есть возможность создать новый репозиторий или выбрать из существующих репозиториев, как обсуждалось ранее в главе (см. рисунок 11-2). Если одновременно создаются новый проект и новый репозиторий, между проектом и его репозиторием существует отношение один-к-одному, и вы становитесь администратором проекта. Если вы выбираете существующий репозиторий, вы создаете новый проект, администратором которого не являетесь (если только вы не владели репозиторием). В любом случае файл `.gpr` и каталог `.rep` имеют одно и то же базовое имя. Если репозиторий назван `RepoExample`, файл проекта будет называться `RepoExample.gpr`, а папка репозитория - `RepoExample.rep`. (Несмотря на расширение, репозиторий является каталогом, а не файлом.)

Итого: если вы создаете репозиторий, вы являетесь администратором проекта и можете выбирать, кто еще имеет доступ к вашему репозиторию. Если вы используете существующий репозиторий, вы пользователь с правами и привилегиями, назначенными администратором проекта. Что же происходит, когда несколько пользователей хотят вносить изменения в один и тот же проект? Здесь вступает в игру контроль версий.

Контроль версий и отслеживание версий

Ghidra включает две очень разные системы версионирования. В этой главе мы обсуждаем контроль версий, и, надеемся, эта концепция скоро станет достаточно ясной. У Ghidra также есть возможность отслеживания версий. Отслеживание версий используется для определения различий (и сходств) между двумя бинарными файлами. В сообществе SRE этот процесс обычно известен как сравнение бинарных файлов. Цели могут включать определение обновлений в разных версиях бинарного файла, идентификацию функций, используемых семейством вредоносных программ, определение сигнатур и т. д. Эта функциональность может быть важна, поскольку связанный исходный код может быть недоступен для сравнения на уровне исходников. Отслеживание версий Ghidra подробнее обсуждается в главе 23.

Контроль версий

Контроль версий - важная концепция в любой системе, где изменения могут вноситься несколькими пользователями или желательна записанная история изменений. Контроль версий позволяет управлять обновлениями системы, эффективно контролируя состояния гонки. В окне `Project` есть панель инструментов контроля версий (рисунок 11-13). Для многих операций необходимо, чтобы соответствующий файл(ы) был закрыт перед завершением действия.

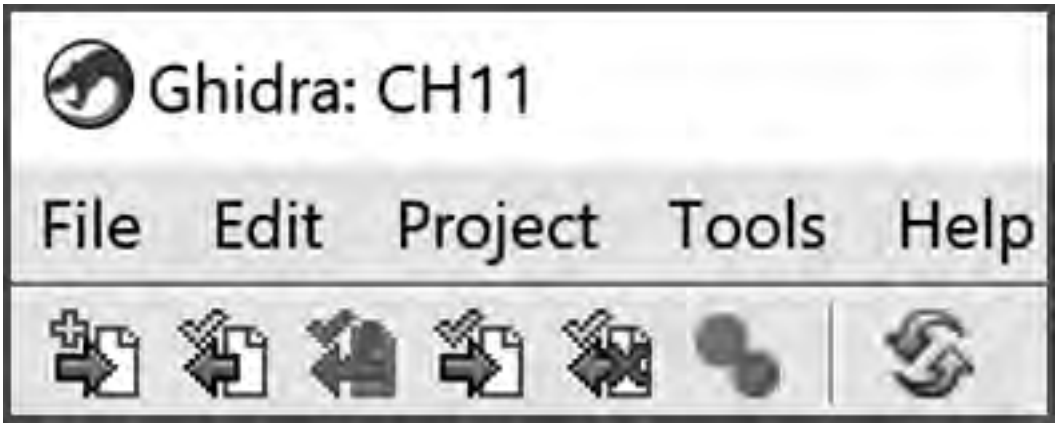


Рисунок 11-13. Панель инструментов контроля версий окна `Ghidra Project`

Значки включаются для допустимых операций контроля версий на основе выбранного файла(ов). Базовые действия, составляющие рабочий процесс контроля версий, показаны на рисунке 11-14. (Мы включили столбец с грубыми эквивалентами Git для всех поклонников Git.)

Icon	Action	Special option(s)	Similar `git` commands
	Add file to version control	Keep File Checked Out	`git add` `git commit`
	Check out file	None	`git clone (ish)`
	Update checked-out file with latest version	Keep File Checked Out Create *.keep* file	`git pull`
	Check in file	Keep File Checked Out Create *.keep* file	`git commit` `git push`
	Undo checkout	Save Copy with *.keep* extension	`git checkout`
	Find my checkouts recursively	None	`git status`

Figure 11-14: Ghidra version control toolbar actions

In addition to using the toolbar icons, you can perform the version co
trol actions via the right-click context menu.

Merging Files

When a collaborative team member decides to check in changes they have
made to the project, one of two conditions will be true:

No conflict In this case, no new versions of the file have been checke
in since the user checked out the file. Since no potential conflict exists

Рисунок 11-14. Действия панели инструментов контроля версий Ghidra

Помимо использования значков панели инструментов, действия контроля версий можно выполнять через контекстное меню по щелчку правой кнопкой мыши.

Слияние файлов

Когда участник совместной команды решает вернуть изменения, внесенные им в проект, истинным будет одно из двух условий:

`Нет конфликта` - в этом случае с момента, когда пользователь взял файл на редактирование, не было возвращено новых версий файла. Поскольку потенциального конфликта нет (нет зафиксированных конфликтующих изменений, о которых пользователь еще не знает), возвращаемый файл станет новой версией файла. Старая версия будет сохранена архивным образом, а номер версии увеличен, чтобы можно было отслеживать непрерывную цепочку версий.

`Потенциальный конфликт` - в этом случае другой пользователь зафиксировал новые изменения, пока файл был взят на редактирование первым пользователем. Порядок, в котором файлы

возвращаются, может повлиять на итоговую "текущую версию". В этом случае Ghidra начинает процесс слияния. Если представленные изменения не создают конфликтов, Ghidra продолжает автоматический процесс слияния. Если конфликты обнаружены, каждый должен быть вручную разрешен пользователем.

В качестве примера конфликта предположим, что `user1` и `user2` оба взяли один и тот же файл на редактирование, и `user2` изменяет имя `FUN_00123456` на `hash_something` и возвращает это изменение. Тем временем `user1` анализирует ту же функцию и переименовывает ее в `compute_hash`. Когда `user1` наконец возвращает свои изменения (после `user2`), он будет уведомлен о конфликте именования и должен будет выбрать правильное имя функции, между `hash_something` и `compute_hash`, прежде чем операция возврата изменений сможет быть завершена. Дополнительную информацию об этом процессе можно найти в `Ghidra Help`.

Комментарии контроля версий

Когда вы добавляете или изменяете файл под контролем версий, следует добавить комментарий, объясняющий, что вы сделали. Каждое действие контроля версий отображает диалог с полем комментария и специальными параметрами. Рисунок 11-15 показывает диалог комментария Add File to Version Control.

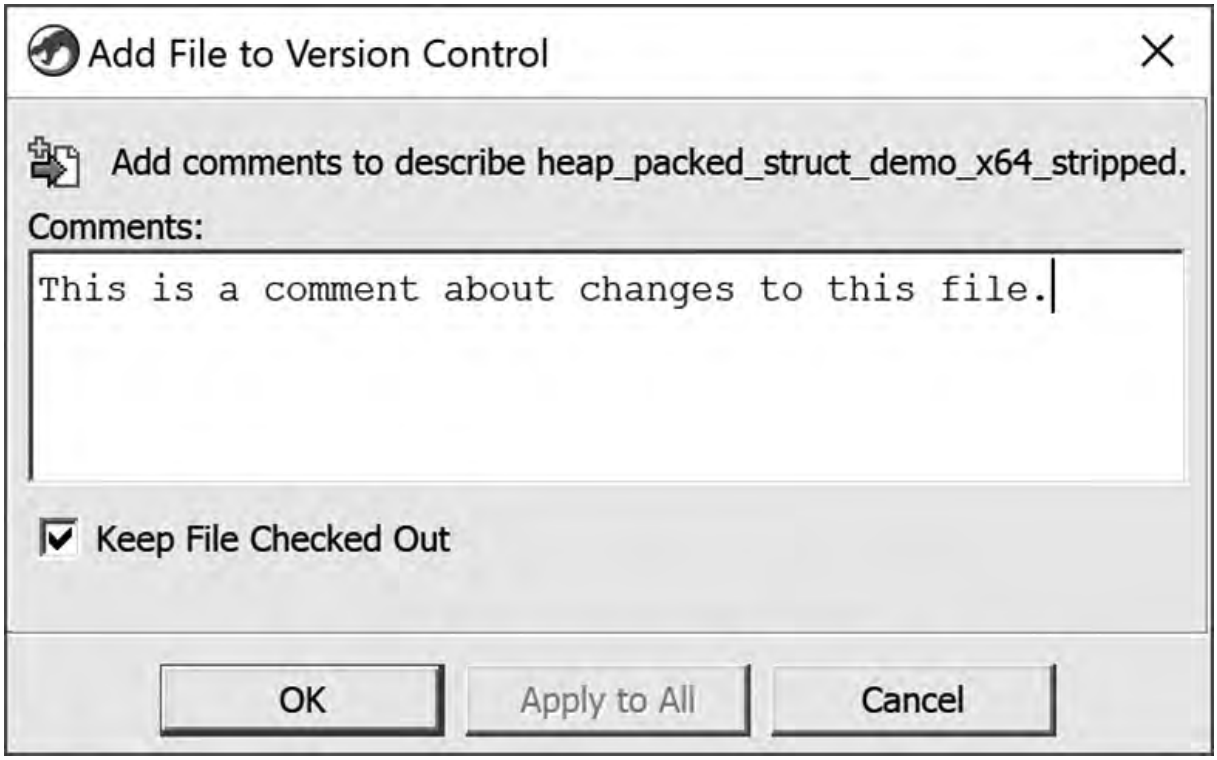


Рисунок 11-15. Диалог комментария Add File to Version Control в Ghidra

В строке заголовка отображается выполняемое действие, дополненное ниже связанным значком и описанием того, что следует вводить в текстовое поле Comments. Если выбрано более одного файла, любые комментарии будут связаны только с первым файлом, если не щелкнуть кнопку Apply to All. Ниже текстового поля Comments находятся конкретные параметры, связанные с выполняемым действием, которые пользователь может включить или отключить. См. третий столбец рисунка 11-14 для специальных параметров каждого действия.

Пример сценария

С общими проектами связано много тонкостей, параметров и перегруженной терминологии. Чтобы прояснить некоторые концепции, связанные с Ghidra Server и общими проектами, пройдем пример, демонстрирующий обсуждаемые понятия, начиная с понятия проекта.

Проект - это локальная сущность, живущая на клиентской машине (как локальный репозиторий Git). Общие проекты также связаны с репозиторием на Ghidra Server (как удаленный репозиторий Git), и именно в этом репозитории хранятся результаты всей совместной аналитической работы. Файлы становятся общими после того, как они импортированы и добавлены в контроль версий, а до этого они являются приватными. Поэтому пользователь может импортировать файлы в проект, после чего они приватны, а затем выбрать добавление их в контроль версий, после чего они становятся общими.

> **Помогите! Мой файл захвачен!**
>
> У Ghidra есть специальный термин (и связанный значок `Project Data Tree`) для ситуации, которая часто может возникать в окружении общего проекта. Если в вашем проекте есть приватный файл (импортированный, но еще не добавленный в контроль версий), а другой пользователь добавляет в репозиторий файл с тем же именем, ваш файл будет захвачен. Это настолько частое явление, что Ghidra предоставляет пункт контекстного меню по щелчку правой кнопкой мыши для обработки ситуации. Нужно закрыть захваченный файл, а затем выбрать пункт Undo Hijack в контекстном меню. Это даст возможность принять файл из репозитория и сохранить копию собственного файла, если нужно. Другие варианты разрешения захвата включают переименование файла, перемещение его в другой проект и удаление.

На самом деле разрешения проекта - это разрешения репозитория. Если вы создаете проект с использованием существующего репозитория, вы фактически говорите: "Этот локальный проект удаленно поддерживается тем репозиторием на сервере" (как `git clone`). Пройдем последовательность действий общего проекта и посмотрим, как они влияют на окружение общего проекта.

1. `user1` создает новый общий проект (и связанный новый репозиторий) с именем `CH11-Example`, добавляет `user2` и `user3` и назначает им разрешения (см. рисунок 11-16).

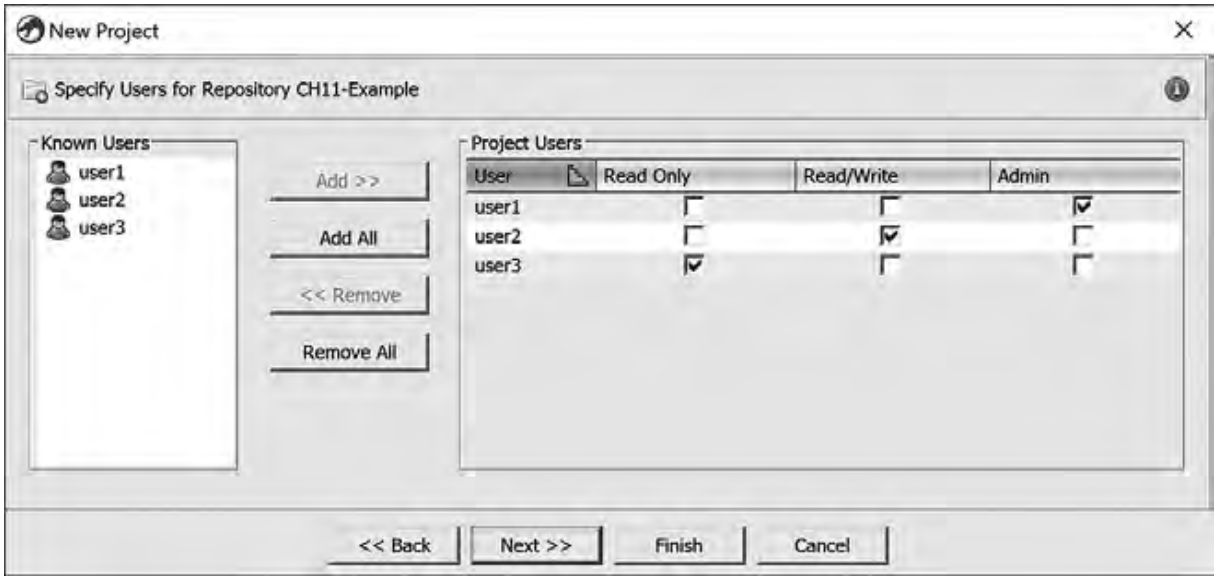


Рисунок 11-16. Пример сценария, шаг 1

2. user2 создает новый общий проект, связанный с существующим репозиторием CH11-Example (то есть user2 клонирует CH11-Example). Обратите внимание, что проект имеет не то же имя, что проект user1, но репозиторий (удаленный) тот же. Кроме того, разрешения user2 для репозитория показаны в нижней части окна (см. рисунок 11-17).

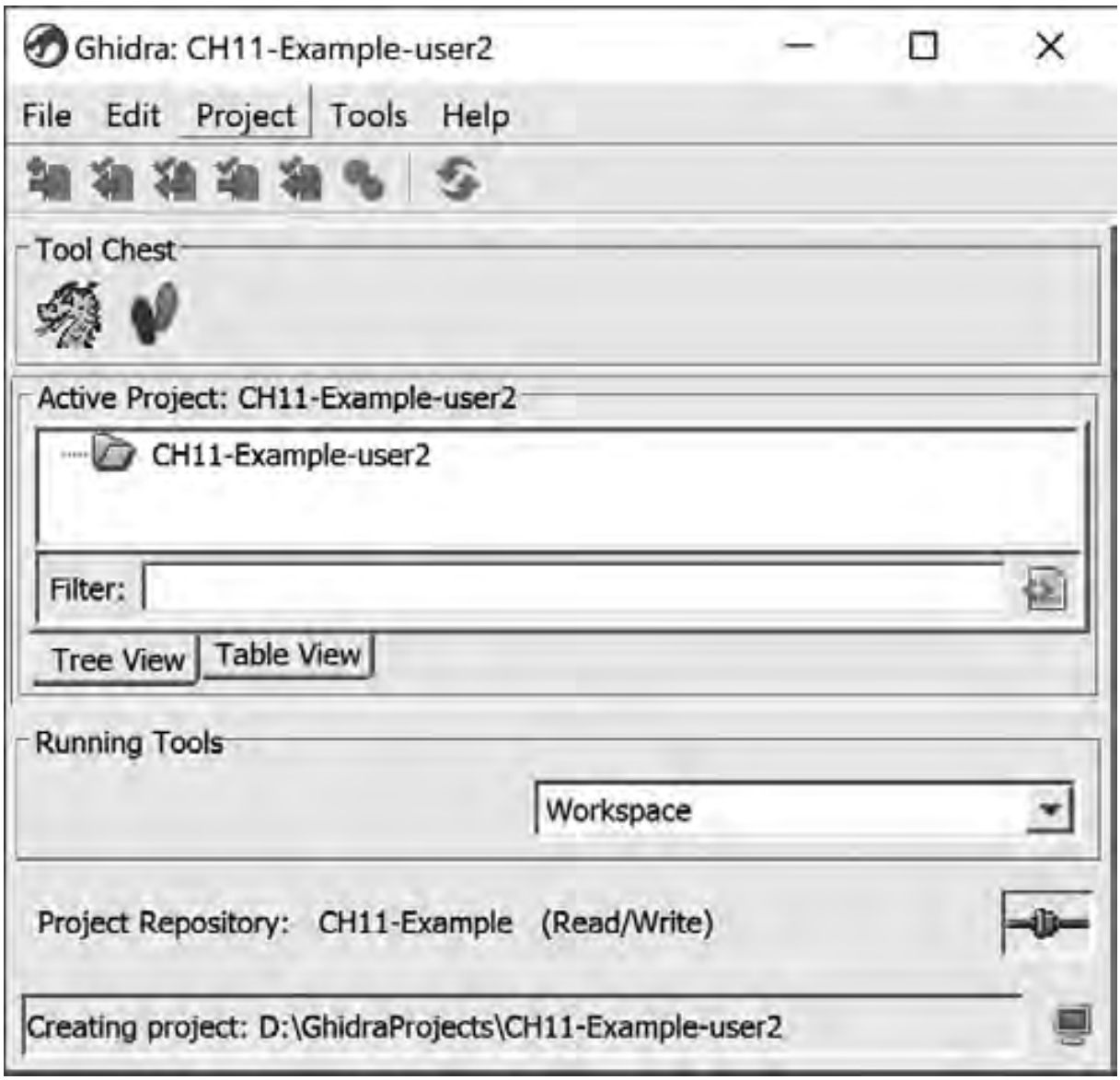


Рисунок 11-17. Пример сценария, шаг 2

3. user1 импортирует файл и добавляет его в контроль версий, после чего user2 тоже может его видеть (примерно эквивалент `git add`/`commit`/`push`). Это показано на рисунке 11-18.

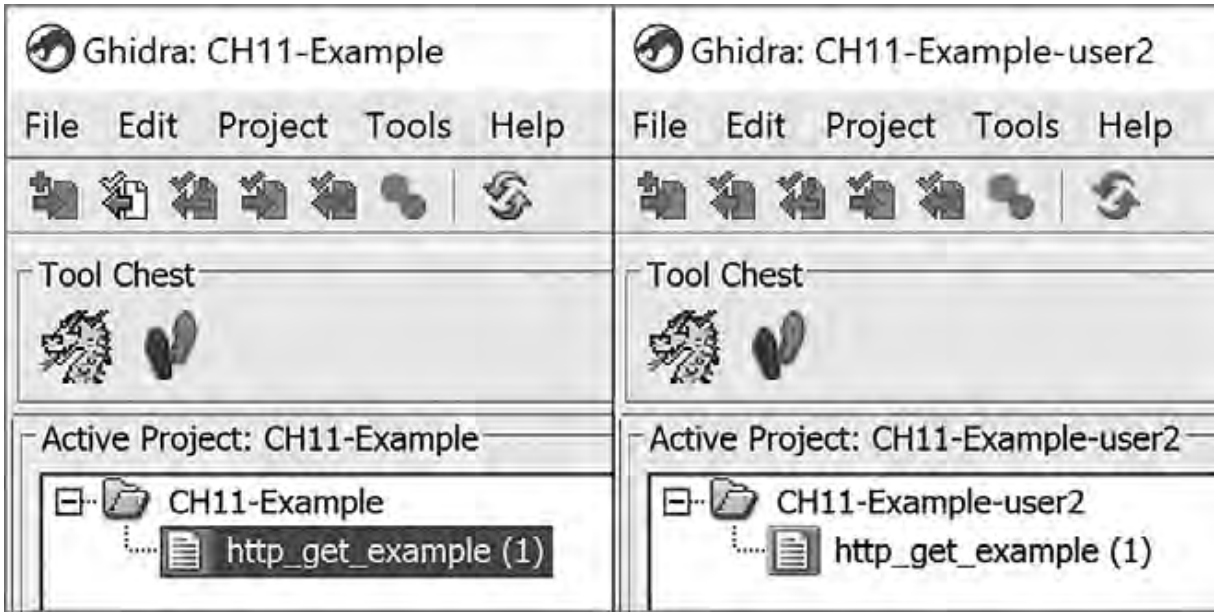


Рисунок 11-18. Пример сценария, шаг 3

4. Затем `user1` и `user2` каждый импортируют один и тот же файл в свои проекты, но не добавляют их в контроль версий. Это приватные файлы (см. рисунок 11-19).

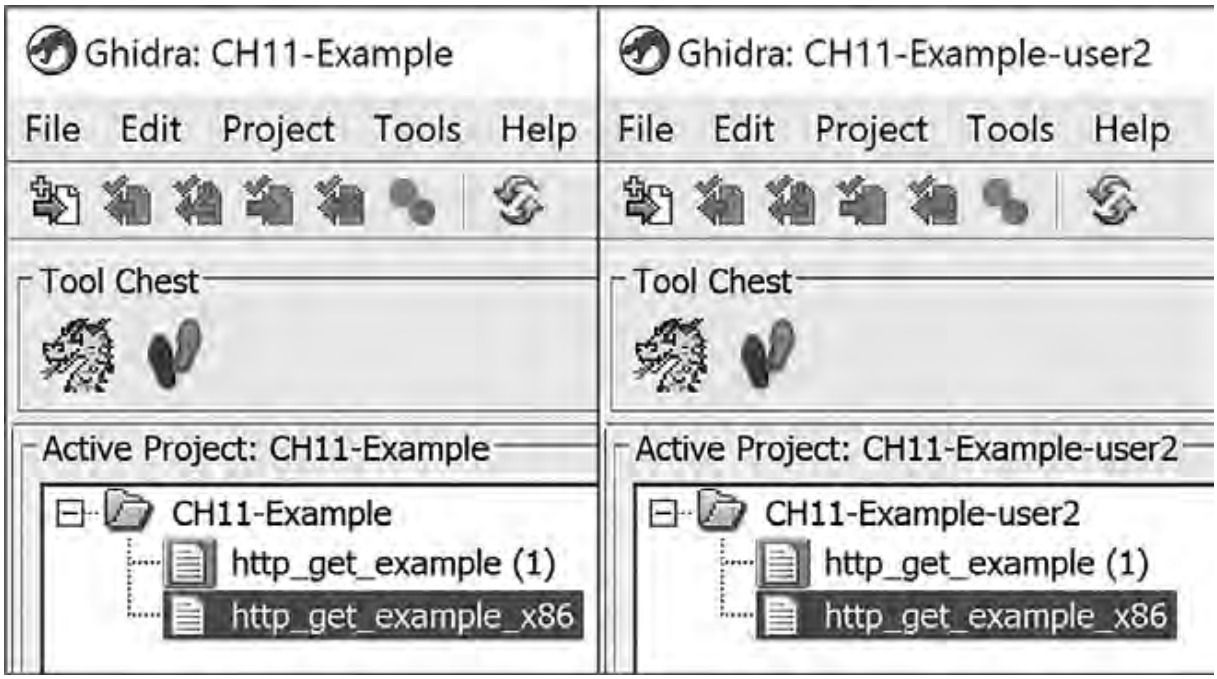


Рисунок 11-19. Пример сценария, шаг 4

5. `user2` добавляет этот второй файл в контроль версий (что возвращает его в репозиторий). В результате файл больше не приватный. Теперь `user1` видит его как захваченный файл (см. рисунок 11-20).

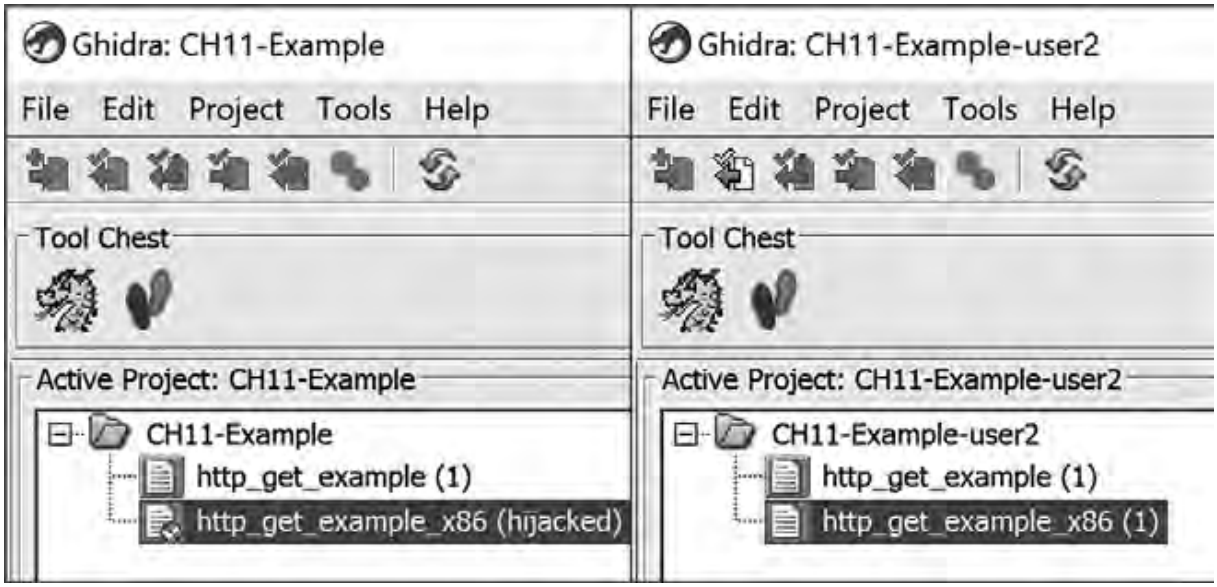


Рисунок 11-20. Пример сценария, шаг 5

6. `user1` выбирает Undo Hijack в контекстном меню по щелчку правой кнопкой мыши и получает возможность заменить свой файл версией из репозитория и сохранить копию собственного файла, если нужно. Она выбирает принять версию репозитория и сохранить копию собственного файла (которую она переместила в другой проект, и теперь он имеет расширение `.keep`). Теперь все снова хорошо. В этом случае `user1` видит состояние второго файла таким, каким оно было, когда `user2` добавил его в контроль версий (см. рисунок 11-21).

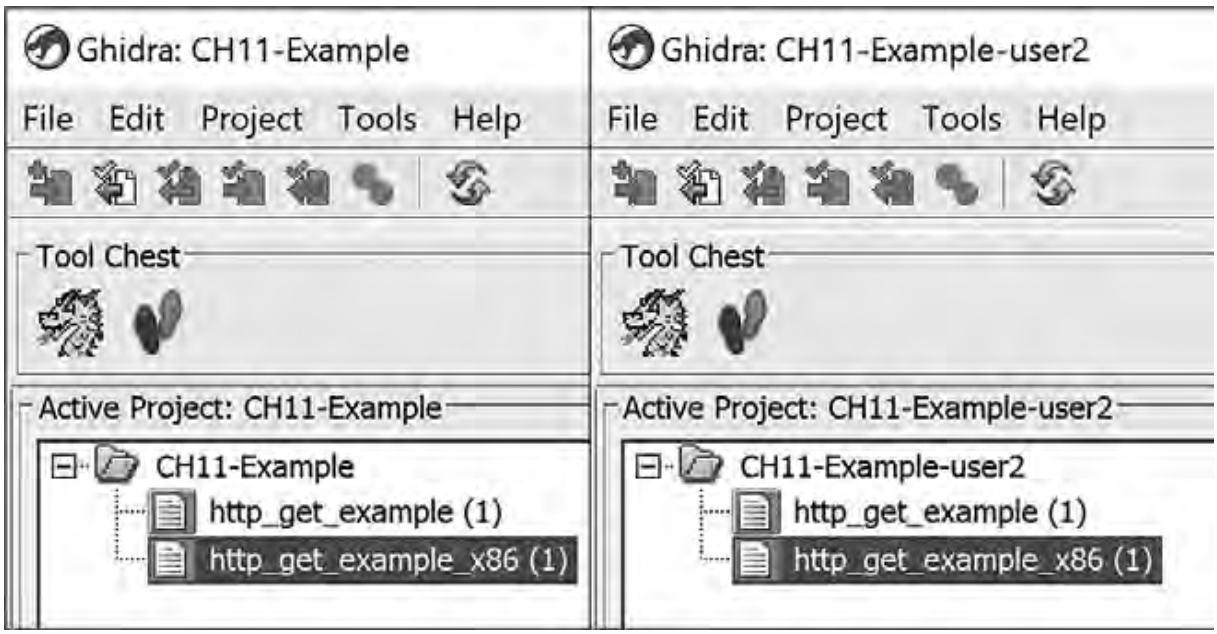


Рисунок 11-21. Пример сценария, шаг 6

7. `user1` берет второй файл на редактирование, анализирует его, а затем возвращает. Теперь и `user1`, и `user2` видят проанализированную версию файла (версия 2), как показано на рисунке 11-22.

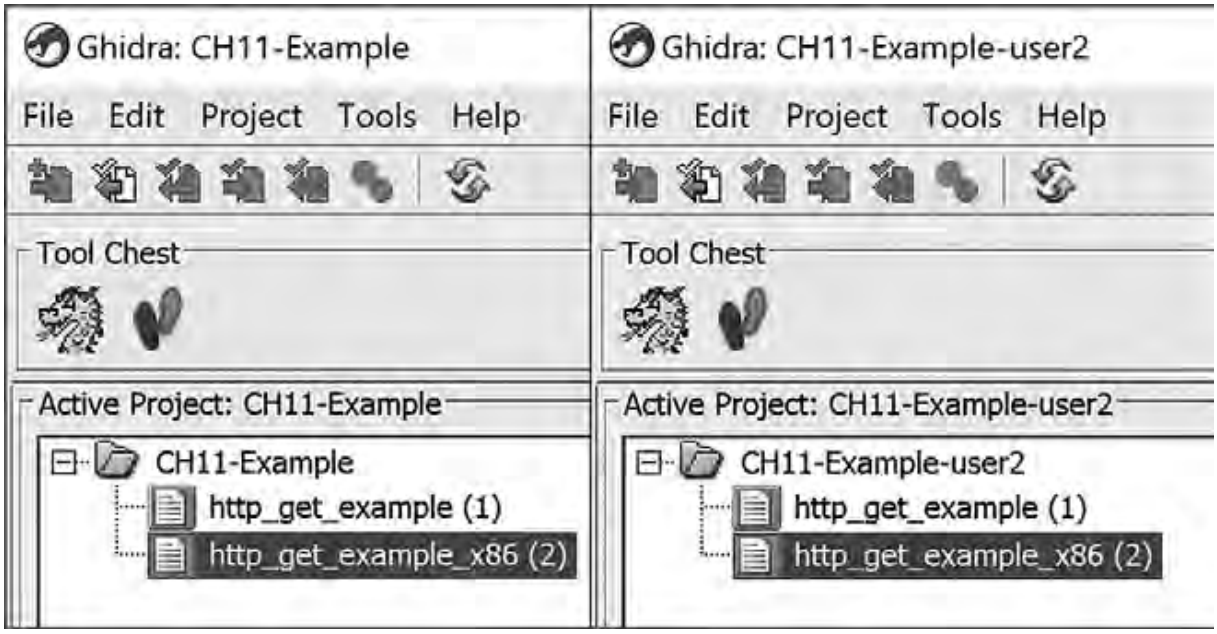


Рисунок 11-22. Пример сценария, шаг 7

8. `user3` создает проект и связывает его с тем же репозиторием (см. рисунок 11-23). Теперь `user3` видит все файлы и может вносить локальные изменения (включая добавление приватных файлов), но не имеет возможности фиксировать изменения в репозитории, потому что ей не были предоставлены права записи. (Проект отмечен как "Read Only" в нижней части окна.)

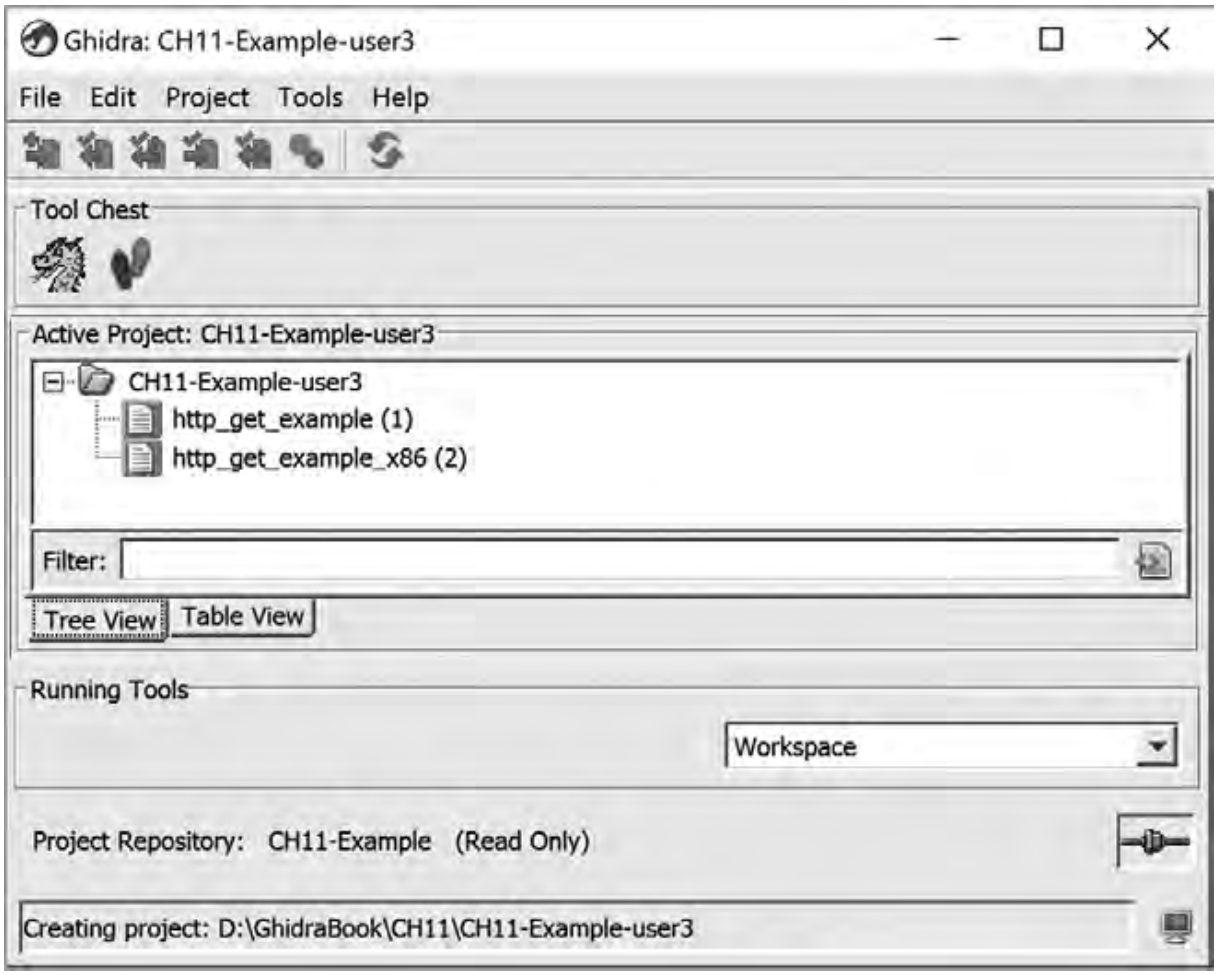


Рисунок 11-23. Пример сценария, шаг 8

9. `user2` возвращает все свои файлы перед уходом с работы. Это важно, потому что она знает, что хочет продолжить работу над проектом, используя домашний компьютер. Поскольку проекта нет на ее домашнем компьютере, ей нужно войти в Ghidra Server и создать новый проект с использованием существующего репозитория. Это создаст проект на ее домашнем компьютере, где она сможет продолжить работу. (Если бы она не вернула все свои файлы перед уходом с работы, у нее дома не было бы доступа к последней работе.)
10. Остальные пользователи уходят домой уверенными, что их совместный Ghidra Server работает как задумано.

Резюме

Не всем потребуется Ghidra Server или общие проекты для поддержки совместной обратной разработки, но многие связанные возможности можно также применять к необщим проектам. Оставшиеся главы сосредоточены на необщих проектах, со ссылками на общие проекты и Ghidra Server там, где это уместно. Независимо от конфигурации вашей установки Ghidra, есть хороший шанс, что стандартные конфигурации, инструменты и представления могут быть неидеальны для вашего рабочего процесса. Следующая глава посвящена конфигурациям Ghidra, инструментам и рабочим областям, а также тому, как заставить их работать для вас лучше.

Глава 12. Настройка Ghidra

После некоторого времени работы с Ghidra у вас могут появиться предпочтительные настройки, которые вы захотите использовать как значения по умолчанию каждый раз при открытии нового проекта или применить ко всем файлам в конкретном проекте. К этому моменту вас может смущать, почему одни измененные параметры сохраняются от сеанса к сеансу, а другие приходится заново настраивать при каждой загрузке нового проекта или файла. В этой главе мы рассмотрим способы настройки внешнего вида и поведения Ghidra по умолчанию, чтобы они лучше соответствовали вашим задачам обратной разработки.

Чтобы понимать область действия некоторых настроек, полезно разобраться в нечетком различии между терминами плагин и инструмент. В общем смысле верно следующее:

Плагин. Плагин - это программный компонент (например, `Byte Viewer`, окно `Listing` и т. д.), который добавляет функциональность в Ghidra. Плагины часто представлены как окна, но многие плагины выполняют свою работу в фоновом режиме (например, анализаторы).

Инструмент. Инструмент может быть одним плагином или набором плагинов, которые работают совместно. Обычно они представлены как полезный графический пользовательский интерфейс (GUI), помогающий пользователям выполнять задачи. Инструмент, с которым мы много работали, `CodeBrowser`, является окном, служащим GUI-каркасом. Function Graph также является инструментом.

Не паникуйте, если эти определения соблюдаются не строго. Во многих случаях различать эти два понятия просто не нужно. Например, некоторые меню, такие как меню `Tool Options`, обсуждаемое далее в этой главе, включают параметры, которые можно применять и к инструментам, и к плагинам, несмотря на использование слова `Tool`. В этом контексте, как и во многих других, различие не важно, поскольку оба типа объектов обрабатываются одинаково. Вы сможете успешно пройти процесс настройки даже тогда, когда употребление терминов меняется.

Помимо настроек Ghidra, в конце главы мы также обсудим рабочие области Ghidra. Рабочие области связывают инструмент с конфигурацией и дают возможность создавать и использовать персональный виртуальный рабочий стол.

CodeBrowser

В главах 4 и 5 мы представили `CodeBrowser` и многие связанные с ним окна. Некоторые базовые варианты настройки мы уже рассматривали; теперь пройдем более подробный пример настройки `CodeBrowser`, прежде чем перейти к окну `Ghidra Project` и рабочим областям.

Перестановка окон

Следующие шесть базовых операций позволяют управлять тем, где отдельные окна появляются относительно окна `CodeBrowser`:

Open. Окна обычно открываются через меню `Window` в `CodeBrowser`. У каждого окна есть значения по умолчанию, определяющие, где оно откроется.

Close. Окна можно закрывать, нажимая X в правом верхнем углу окна. Если вы снова откроете закрытое окно, оно появится в том же месте, а не в исходном положении по умолчанию.

Move. Перемещайте окна перетаскиванием.

Stack. Используйте перетаскивание, чтобы складывать окна в стек и выводить их из стека.

Resize. При наведении на границу между двумя окнами появляется стрелка, которая позволяет увеличивать и уменьшать окна, прилегающие к этой границе.

Undock. Вы можете отсоединить инструмент от окна `CodeBrowser`, но повторное закрепление устроено не так прямолинейно, как хотелось бы, как показано на рисунке 12-1.

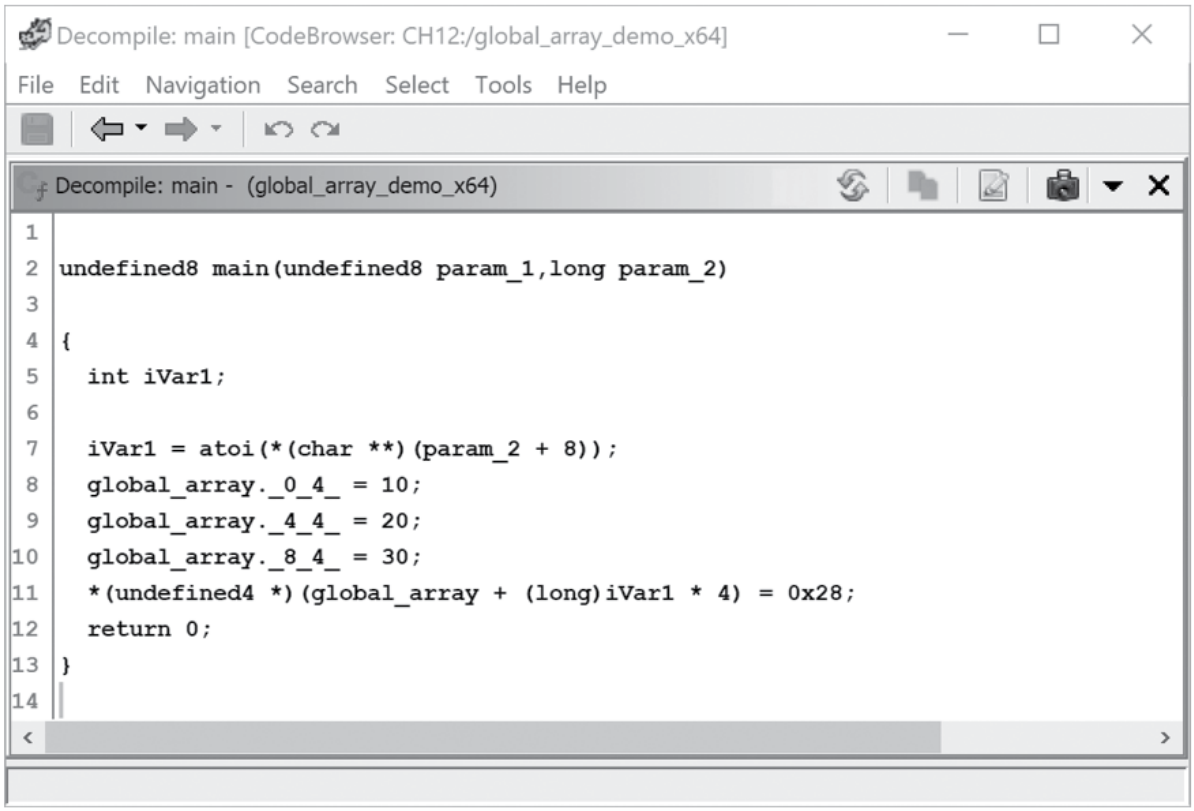


Рисунок 12-1. Повторное закрепление окна `Decompiler`

Чтобы снова закрепить окно, нельзя щелкать по строке заголовка, потому что в этом случае вы просто будете перетаскивать окно поверх `CodeBrowser`. Вместо этого щелкните по внутренней строке заголовка, чтобы повторно закрепить окно или сложить его в стек. Теперь, когда мы можем переставлять окна, настроим сами окна с помощью меню `Edit -> Tool Options`.

Редактирование Tool Options

Когда вы выбираете `Edit -> Tool Options`, открывается окно параметров `CodeBrowser`, показанное на рисунке 12-2. Это окно позволяет управлять параметрами, связанными с отдельными компонентами `CodeBrowser`.

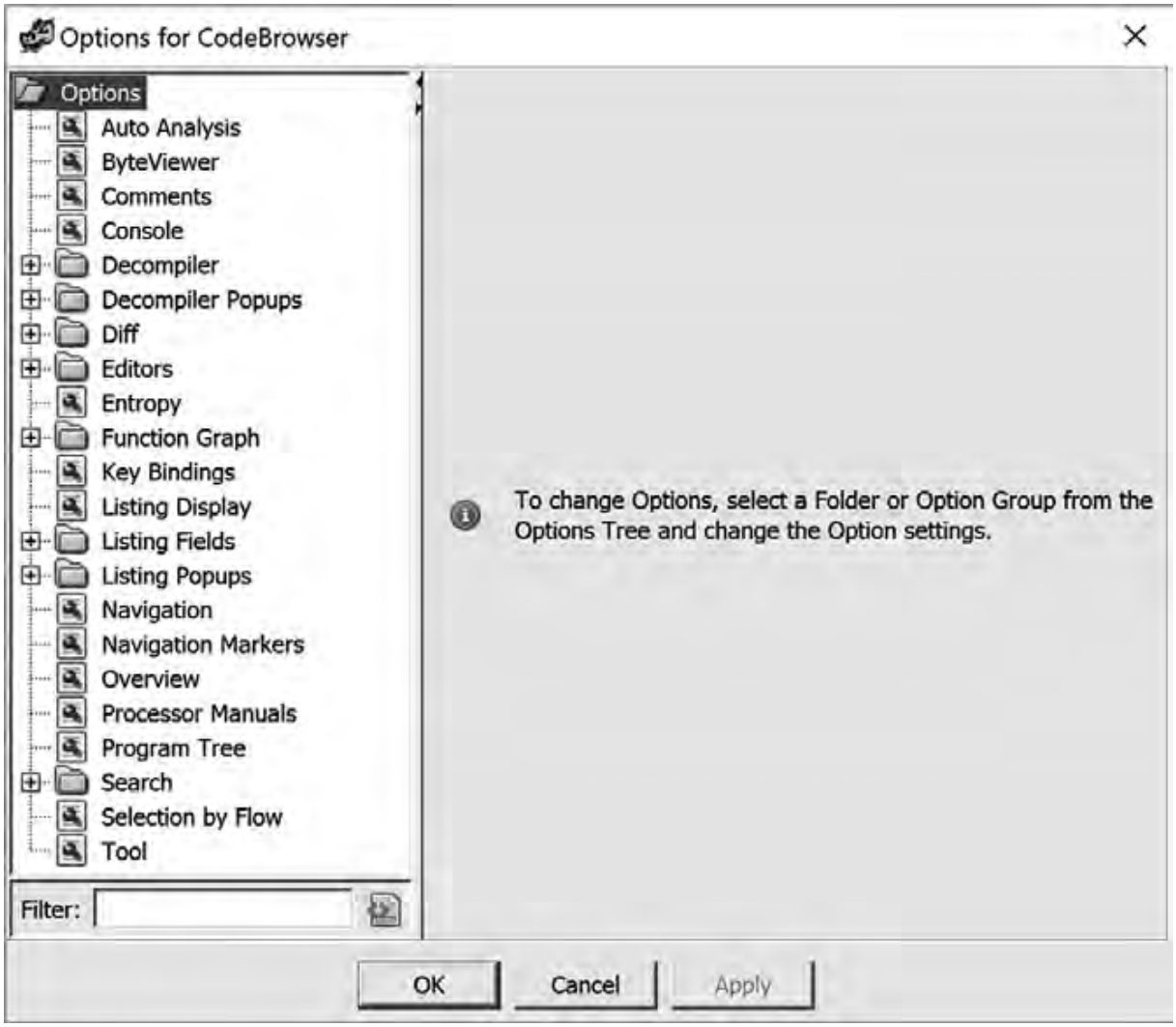


Рисунок 12-2. Окно `Edit -> Tool Options` *в* `CodeBrowser` *по умолчанию*

Доступные параметры определяются разработчиками каждого компонента, а заметная разница между наборами параметров отражает специфическую природу отдельных инструментов. Поскольку описание каждого доступного параметра инструмента заняло бы отдельную книгу, мы рассмотрим несколько изменений, влияющих на инструменты, которые обсуждались в предыдущих главах, а также несколько параметров, похожих для многих инструментов.

Хотя это может быть не очевидно при печати в оттенках серого, многие инструменты используют цвет для обозначения атрибутов, и соответствующая цветовая палитра настраивается. Щелчок по цвету по умолчанию в окне `Options` открывает стандартный диалог `Color Editor`, как показано на панелях параметров `Byte Viewer` на рисунке 12-3. Это дает возможность управлять цветом множества элементов внутри `CodeBrowser`.

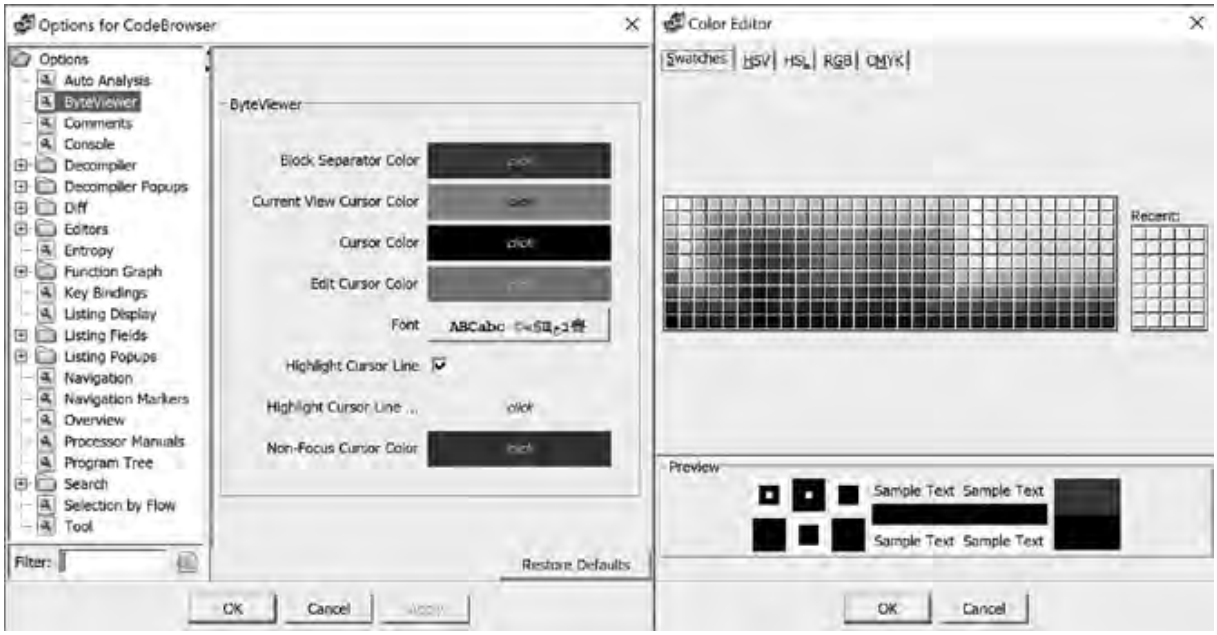


Рисунок 12-3. Диалог `Color Editor` *в* `Edit -> Tool Options`

На рисунке 12-3 можно выбрать цвета для шести элементов окна `Byte Viewer`: `Block Separator`, `Current View Cursor`, `Cursor`, `Edit Cursor`, `Highlight Cursor Line` и `Non-Focus Cursor`. Помимо настройки цвета в окне `Byte Viewer`, можно выбрать шрифт и включить подсветку строки курсора. Удобно, что панель параметров любого инструмента `CodeBrowser` содержит параметр `Restore Defaults` в правом нижнем углу. Это позволяет использовать специальные цветовые схемы на некоторых этапах анализа, а затем возвращать цветовую схему инструмента к значениям по умолчанию.

Помимо косметических изменений, многие инструменты дают возможность задавать параметры в редактируемых настройках. Мы уже намекали на такую возможность, когда вводили новую функциональность в предыдущих главах, например возможность управлять тем, какие анализаторы включаются в автоматический анализ. В целом, всякий раз, когда у чего-либо есть значение по умолчанию, существует способ заменить его чем-то другим.

Параметры некоторых общих инструментов также доступны и изменяемы через окно `Options`. Например, привязки клавиш используются для задания соответствий между действиями Ghidra и последовательностями горячих клавиш, и в окне `CodeBrowser` по умолчанию есть более 550 действий, для которых можно создать или переназначить привязку горячей клавиши с помощью окна `Options`. Переназначение горячих клавиш полезно во многих ситуациях: чтобы сделать дополнительные команды доступными через горячие клавиши, изменить последовательности по умолчанию на более запоминаемые, изменить последовательности, которые могут конфликтовать с операционной системой или терминальным приложением, и даже переназначить все горячие клавиши так, чтобы они совпадали с горячими клавишами других дизассемблеров.

С каждой привязкой клавиш связаны три поля, как показано на рисунке 12-4. Первое поле - `Action Name`. В одних случаях имя действия соответствует команде меню (например, `Analysis -> Auto Analyze`). В других случаях это параметр, связанный с командой меню (например, `Aggressive Instruction Finder` внутри `Analysis Options`).

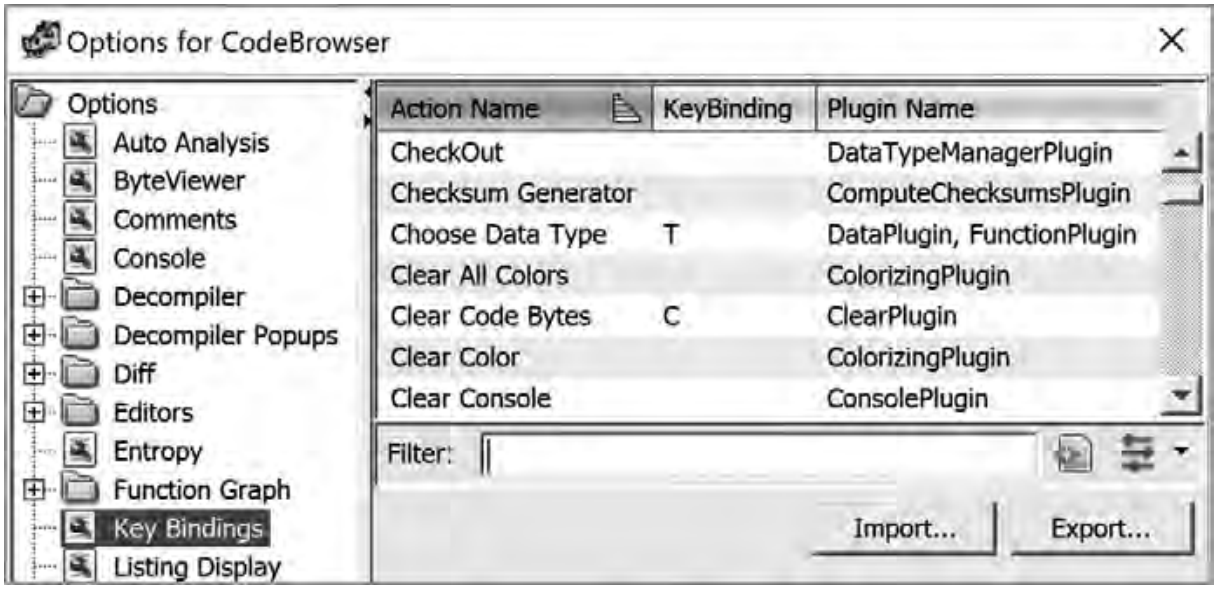


Рисунок 12-4. Параметр `Key Bindings` *в* `Edit -> Tool Options`

Второй столбец - собственно привязка клавиш (горячая клавиша), связанная с действием. В последнем столбце находится имя плагина, в котором реализовано действие. Это еще один пример синонимического употребления терминов, когда различие между ними не важно. Большинство записей в столбце `Plugin Name` действительно являются плагинами, но инструменты, такие как `Configure tool`, также включены в список. В этом контексте привязку клавиш тоже можно применить к любому из них.

Не все действия имеют связанные горячие клавиши, но их легко назначить: выберите действие и введите нужную горячую клавишу в текстовое поле. Если горячая клавиша уже связана с другим действием, отображается список всех остальных использований этой горячей клавиши. Когда вы используете горячую клавишу, у которой несколько привязок, Ghidra покажет список возможных действий, и вам потребуется выбрать подходящее.

Редактирование инструмента

Внизу окна `Edit -> Tool Options` находится параметр `Tool`. Значение `Tool` меняется в зависимости от инструмента, из меню которого был открыт диалог `Options`. Обычно это либо `CodeBrowser`, либо окно `Project`. На рисунке 12-5 показаны параметры конфигурации по умолчанию для инструмента `CodeBrowser`. Самая заметная подсказка о том, что мы смотрим страницу параметров именно `CodeBrowser`, находится в строке заголовка диалога `Options`.

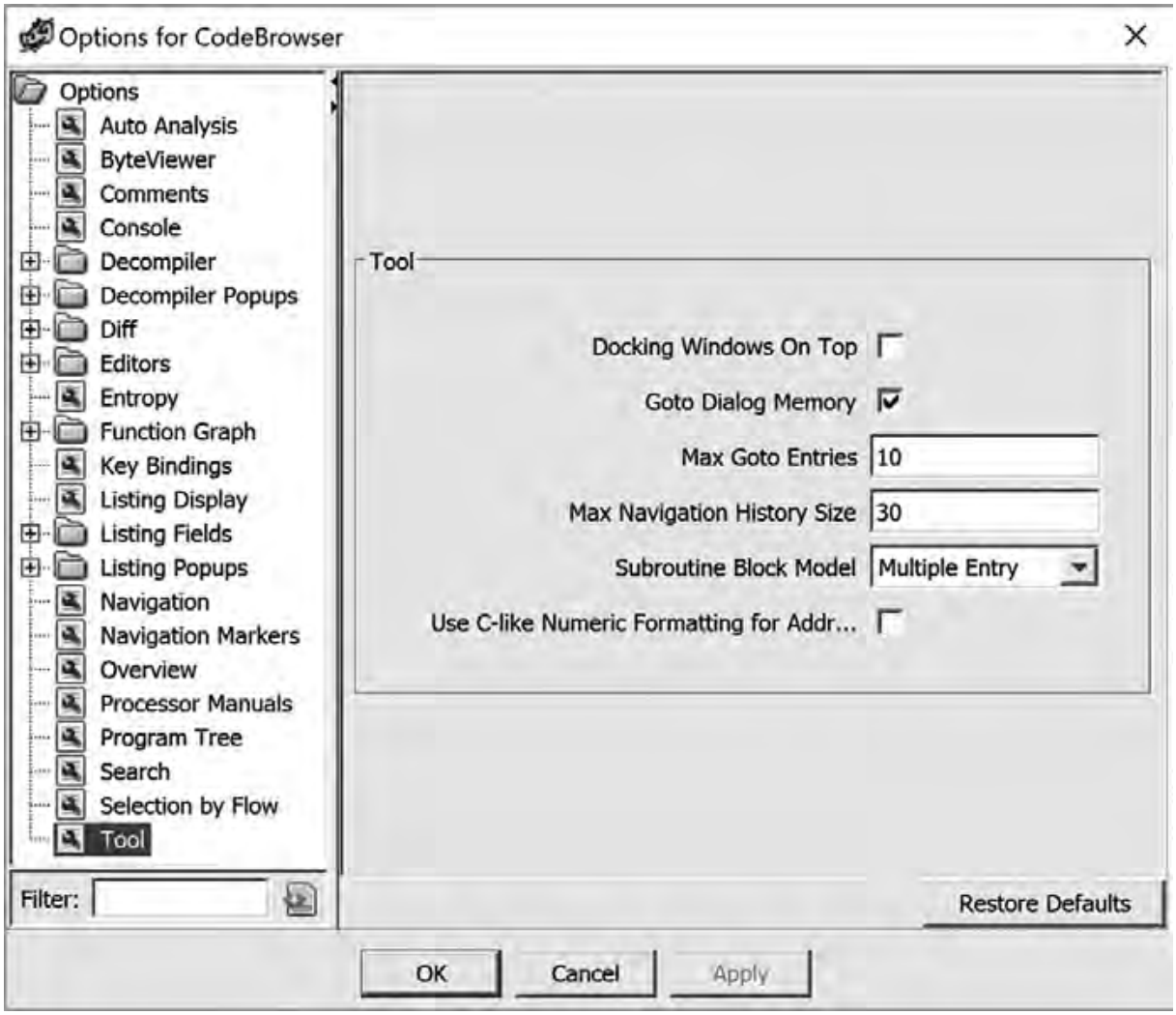


Рисунок 12-5. Использование `Edit -> Tool Options -> Tool` *для редактирования параметров* `CodeBrowser`

Специальные возможности редактирования инструмента

Некоторые инструменты имеют возможности редактирования, встроенные прямо в их отдельные окна, так что вы можете сразу видеть влияние параметров на связанное содержимое. Самый обширный набор встроенных возможностей редактирования доступен в окне `Listing`. Окно `Listing` содержит текстовое содержимое дизассемблирования и хорошо настраивается с помощью `Browser Field Formatter`, представленного в разделе "Изменение параметров отображения кода" на странице 133. На рисунке 12-6 показано окно `Listing` с открытым `Browser Field Formatter` по умолчанию.

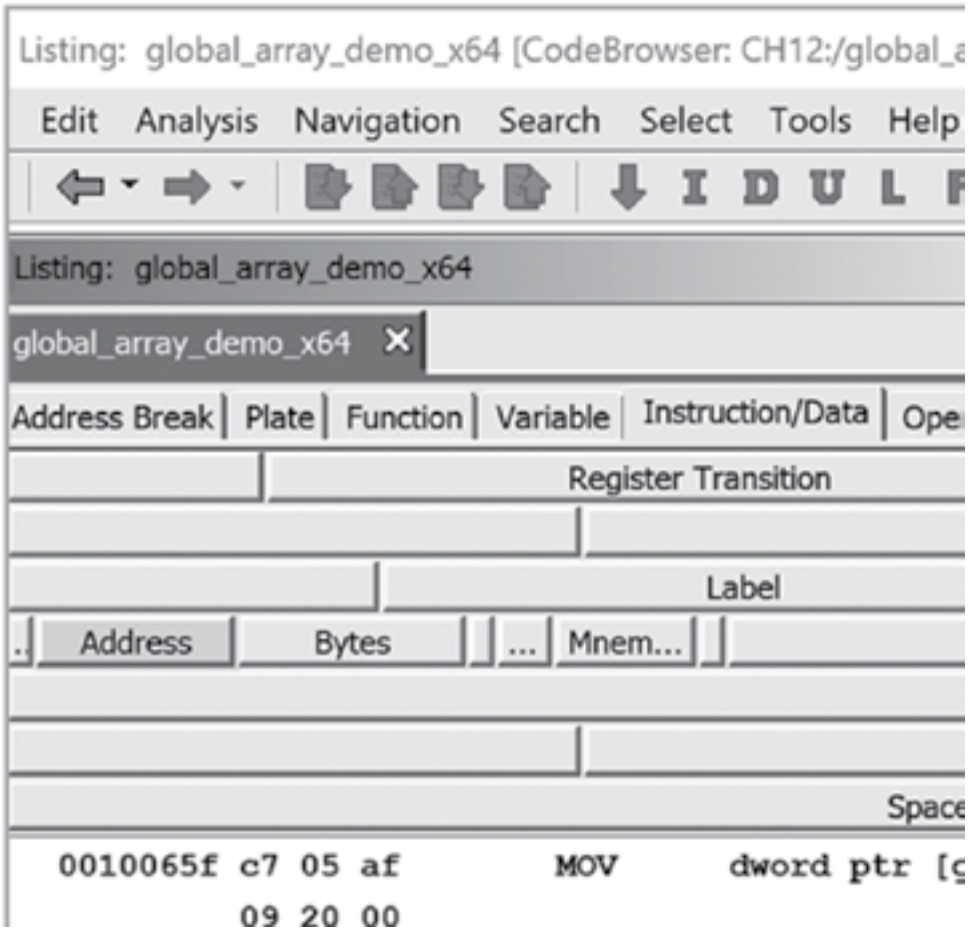


Рисунок 12-6. Окно `Listing` *с открытым Browser Field Formatter по умолчанию*

В верхней части форматтера отображается ряд вкладок, представляющих различные типы полей, присутствующие в дизассемблировании. В данном случае мы смотрим на инструкции, поэтому выбрана вкладка Instruction/Data. Остальная часть форматтера показывает полосы для каждого отдельного поля, связанного с адресом в разделе Instruction/Data. В этом случае курсор находится на адресе внутри окна `Listing`, поэтому поле Address подсвечено.

Browser Field Formatter можно использовать для изменения внешнего вида листинга. Возможности широки, и у каждого поля есть собственные связанные параметры. Мы рассмотрим только некоторые простые возможности, многие из которых похожи на редактирование внешнего вида окон в `CodeBrowser`. Можно переставлять поля, перетаскивая их в новые места; увеличивать или уменьшать ширину поля; а также добавлять, удалять, включать или отключать отдельные поля.

На рисунке 12-7 показано то же содержимое листинга после удаления поля `Bytes`. Во многих изображениях листинга в предыдущих главах мы убирали поле `Bytes`, чтобы уплотнить листинг и показать больше содержимого в доступном пространстве.

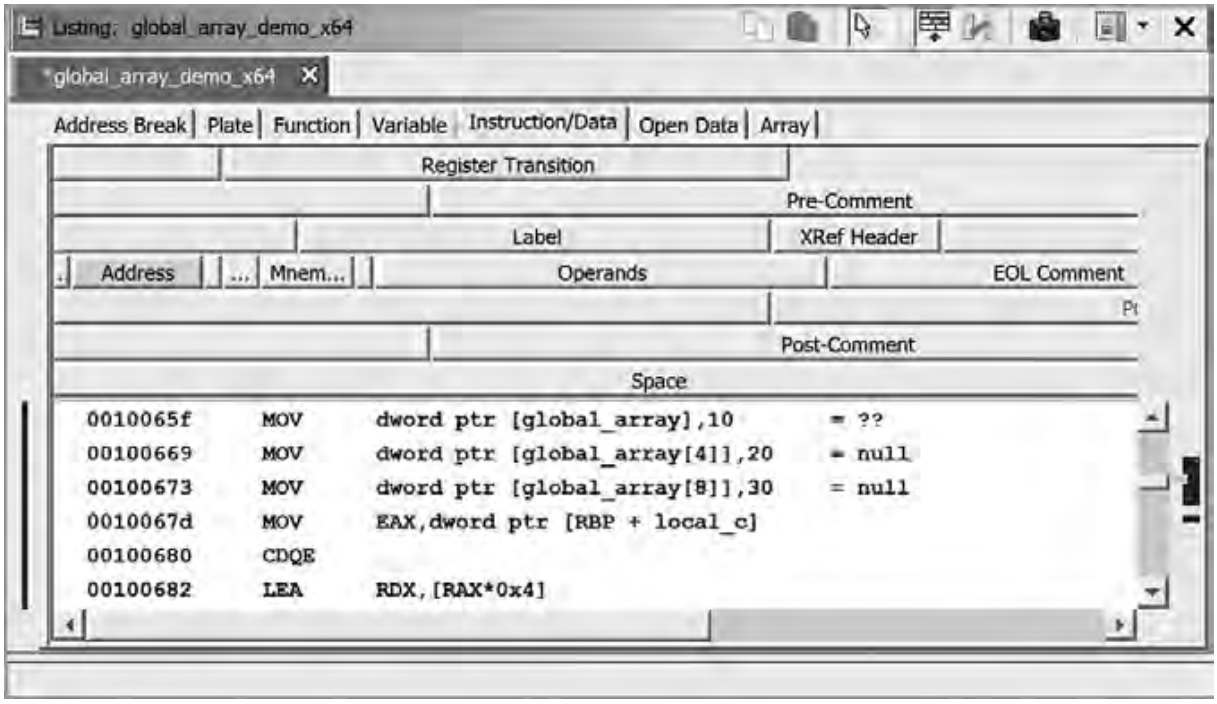


Рисунок 12-7. Окно `Listing` *с настроенными выборами* `Browser Field Formatter`

Сохранение макета CodeBrowser

При закрытии `CodeBrowser` можно сохранить любые изменения макета, связанные с файлом. Вместо этого можно выйти без сохранения; тогда появится предупреждение, чтобы убедиться, что вы понимаете последствия. Если в окне `CodeBrowser` использовать `File -> Save Tool`, текущий внешний вид `CodeBrowser` будет связан с текущим файлом в активном проекте. При следующем открытии файла Ghidra будет использовать сохраненный макет `CodeBrowser`. Когда одновременно открыто несколько экземпляров `CodeBrowser` и некоторые из них (или все) изменены, это может привести к конфликтующим конфигурациям инструмента. Тогда Ghidra отобразит новый диалог `Save Tool`, показанный на рисунке 12-8.

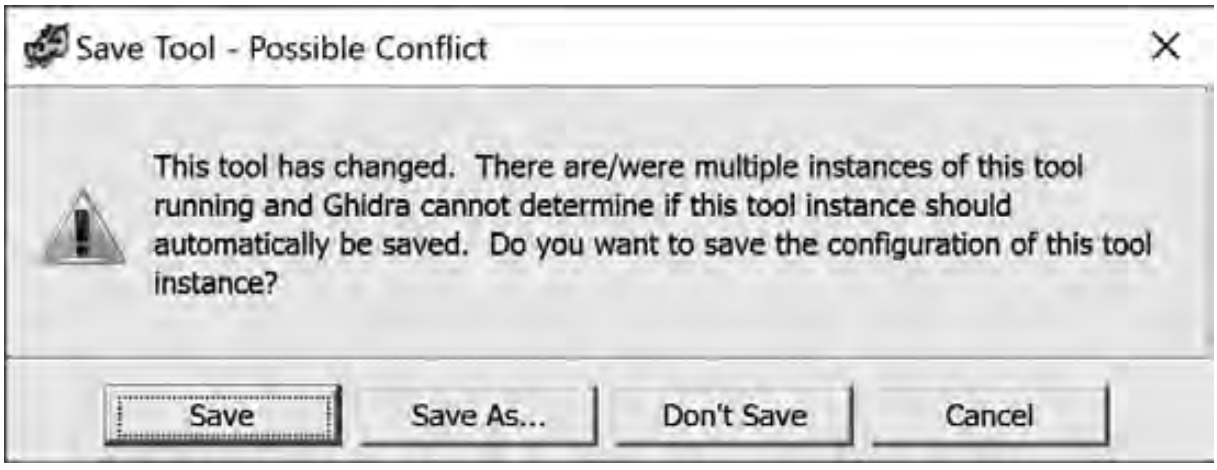


Рисунок 12-8. Диалог Ghidra `Save Tool - Possible Conflict`

Позднее в этой главе мы покажем, как использовать эту и похожую функциональность настройки для создания нового мощного набора инструментов, подстроенного под ваши индивидуальные задачи и вкусы в обратной разработке.

Окно Ghidra Project

Переключим передачи (или, точнее, окна) и вернемся к окну `Ghidra Project`, показанному на рисунке 12-9. Главное меню обсуждалось в предыдущей главе. Прежде чем говорить о настройках окна `Project`, рассмотрим две области окна, которые мы еще не обсуждали.

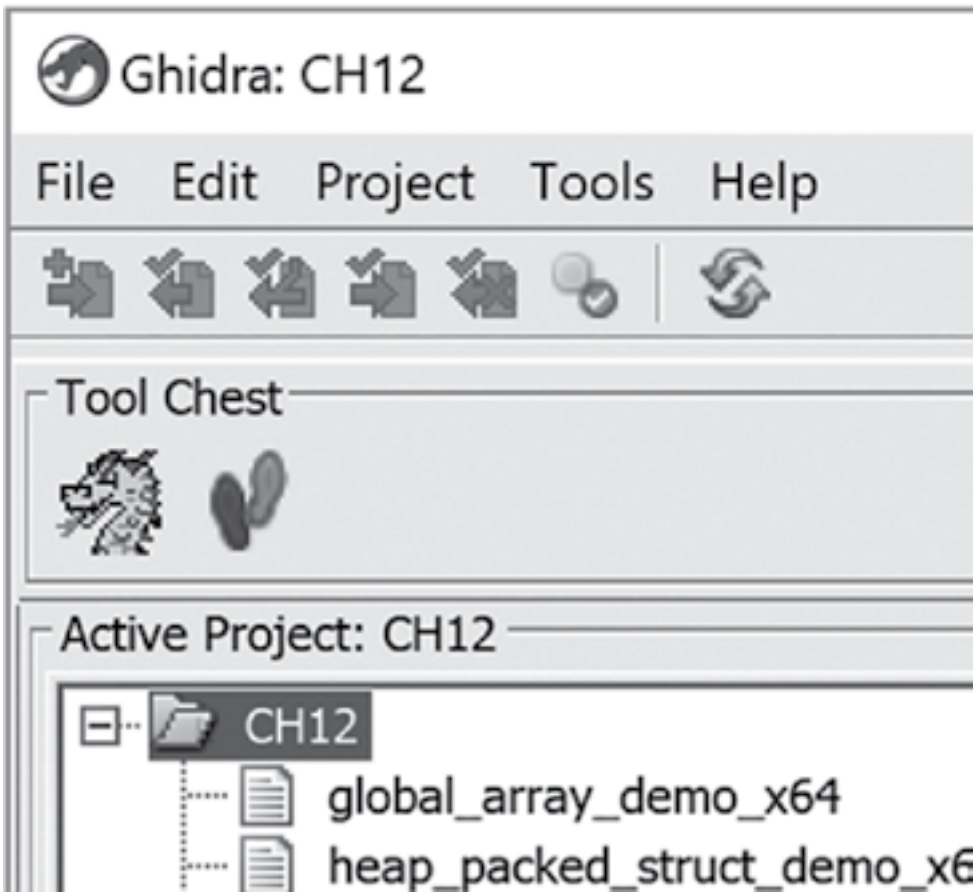


Рисунок 12-9. Окно `Ghidra Project`

`Tool Chest` отображает значки всех инструментов, способных работать с бинарными файлами, которые вы импортировали в свои проекты. По умолчанию доступны два инструмента. Значок дракона является значком по умолчанию для `CodeBrowser`, а значок следов связан с инструментом контроля версий Ghidra. Чуть позже в этой главе мы покажем, как дополнить `Tool Chest`, изменяя и импортируя инструменты, а также создавая собственные.

`Running Tools` содержит значки для каждого запущенного экземпляра инструмента. В этом примере мы открыли каждый файл проекта в отдельном окне `CodeBrowser`. В результате сейчас работают четыре экземпляра `CodeBrowser`. Щелчок по любому значку `Running Tools` выводит связанный инструмент на передний план рабочего стола.

Вернемся к меню окна `Ghidra Project` и рассмотрим некоторые параметры настройки окна. Начнем с изучения четырех действий `Edit -> Tool Options` для проекта Ghidra, показанных на рисунке 12-10. Два параметра совпадают с `CodeBrowser`: `Key Bindings` и `Tool`.

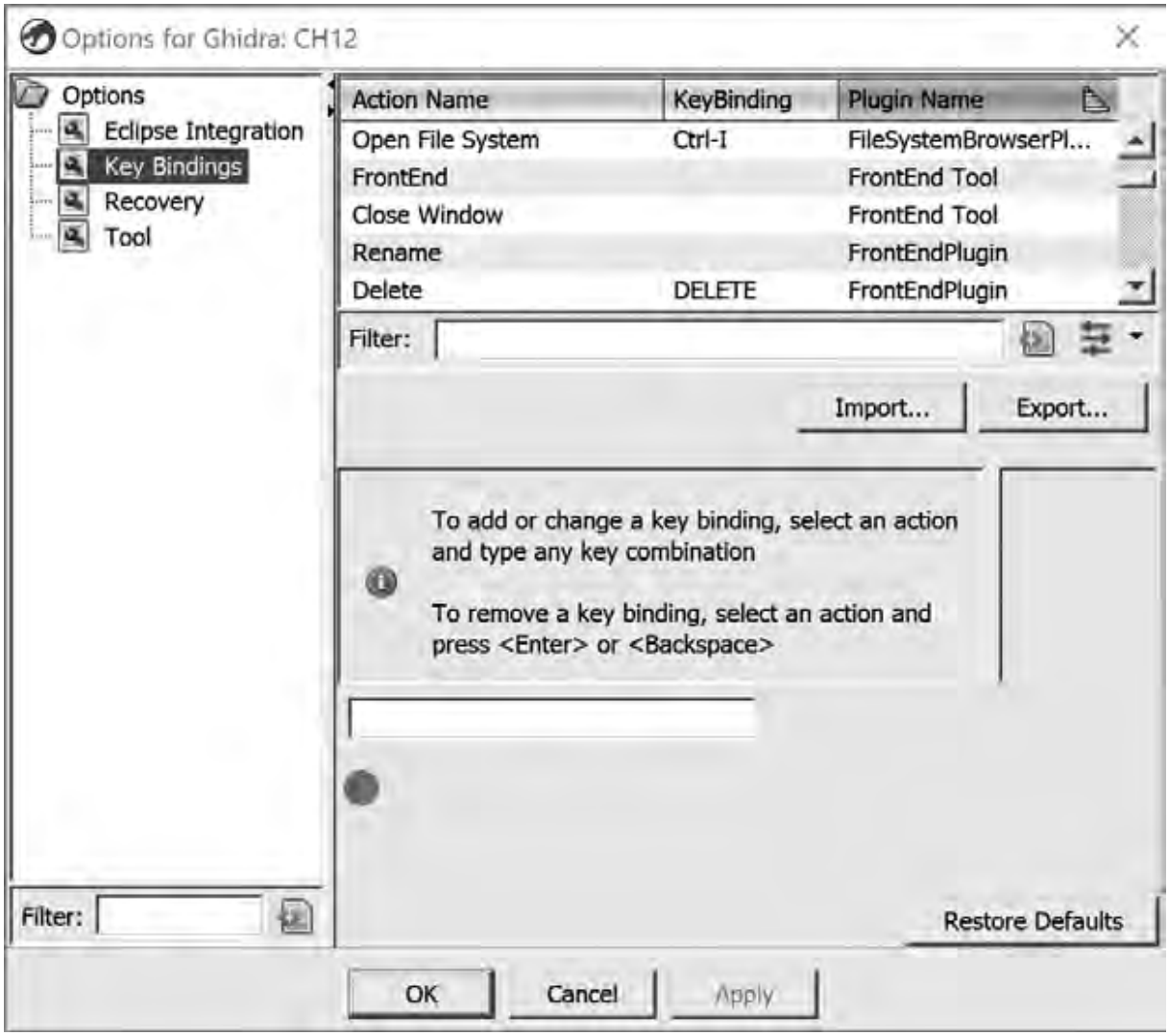


Рисунок 12-10. Окно `Ghidra Project` *(оно же* `Ghidra Frontend`*) через* `Edit -> Tool Options`

На рисунке 12-10 выбран параметр `Key Bindings`. В инструменте `Ghidra Project` значительно меньше действий, чем в `CodeBrowser`, и, следовательно, меньше вариантов привязки клавиш. Если вы повторяете пример у себя, вы можете заметить, что большинство действий связано с FrontEndPlugin. Инструмент `Ghidra Project` также называется `Ghidra Frontend`, и эти термины взаимозаменяемо используются во всей среде Ghidra, включая `Ghidra Help`.

`Eclipse Integration` является темой главы 15, поэтому пока отложим обсуждение этого конкретного параметра. `Recovery` просто позволяет задать частоту снимков. Значение по умолчанию - 5 минут. Установка этого значения в 0 отключает снимки.

Последний параметр, `Tool`, может быть довольно интересным для экспериментов. Как упоминалось ранее в главе, общий термин инструмент в этом контексте относится к активному инструменту. В данном случае это инструмент `Ghidra Project`. Связанные параметры показаны на рисунке 12-11, и мы сосредоточимся на `Swing Look And Feel` и `Use Inverted Colors`, которые меняют внешний вид окон Ghidra.

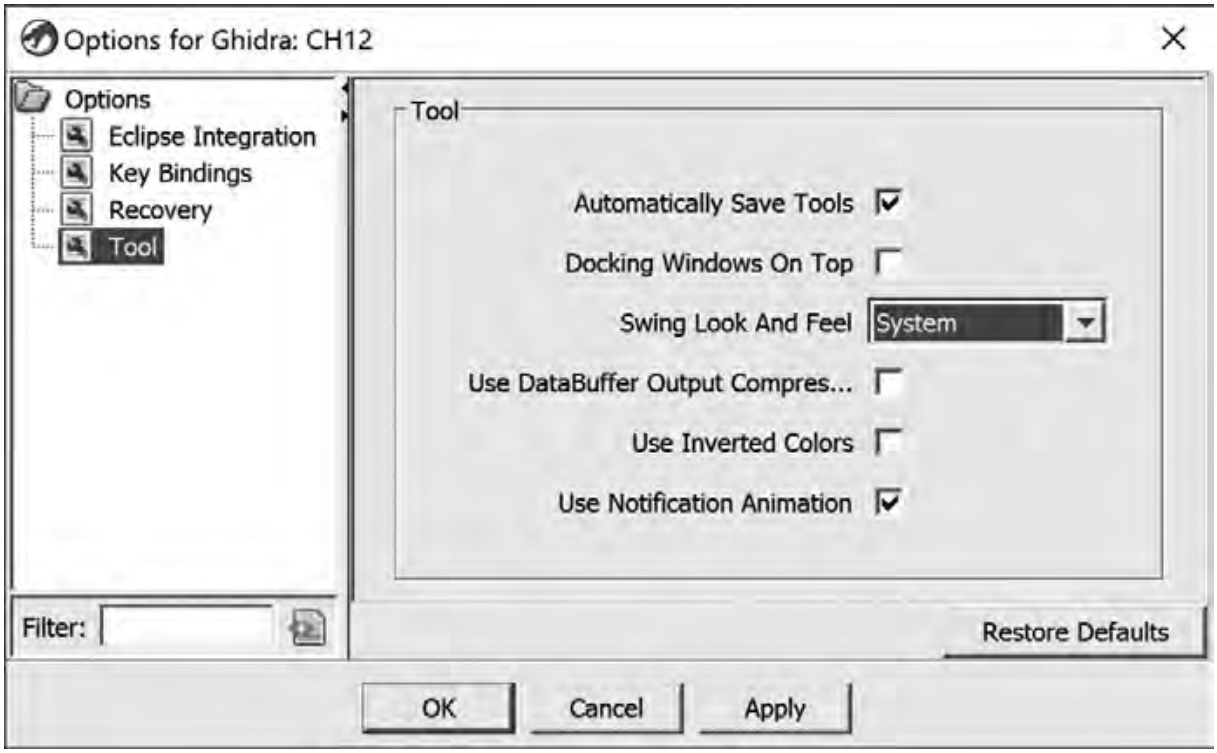


Рисунок 12-11. Параметры редактирования tool `Ghidra Project`

Сочетание `Use Inverted Colors` с выбором `Metal` для `Swing Look And Feel` дает темную тему, популярную у многих специалистов по обратной разработке. Изменения вступят в силу после перезапуска Ghidra, и новые стили будут использоваться для всех окон Ghidra, включая `CodeBrowser` и `Decompiler`. Фрагмент получившегося окна `CodeBrowser` показан на рисунке 12-12.

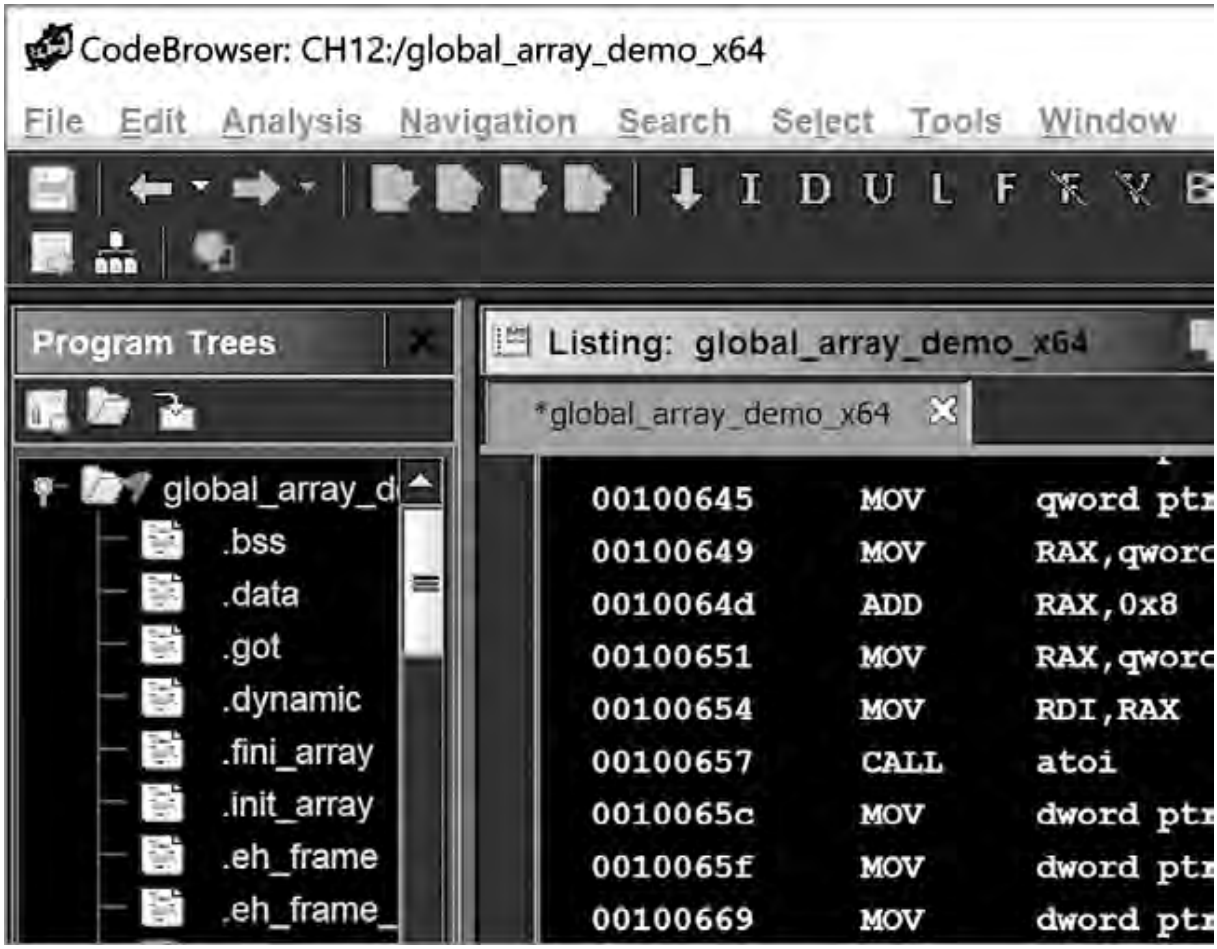


Рисунок 12-12. Часть окна `CodeBrowser` *с темной темой*

Теперь, когда вы знаете, как изменить внешний вид Ghidra, чтобы он лучше соответствовал вашей индивидуальности, вернемся к меню File и выясним, что означает конфигурация в этом контексте. Параметр File -> Configure отображает три категории коллекций плагинов Ghidra, как показано на рисунке 12-13. У каждой категории свое назначение.

Ghidra Core содержит набор плагинов, которые мы использовали в конфигурации Ghidra по умолчанию. Они предоставляют базовую функциональность, необходимую для обратной разработки. Категория Developer предоставляет плагины, помогающие в процессе разработки новых плагинов. Это хорошая отправная точка, если вы хотите больше узнать о разработке для Ghidra. Последняя группа плагинов - Experimental. Эти плагины не прошли тщательного тестирования и могут дестабилизировать ваш экземпляр Ghidra, поэтому используйте их с осторожностью.

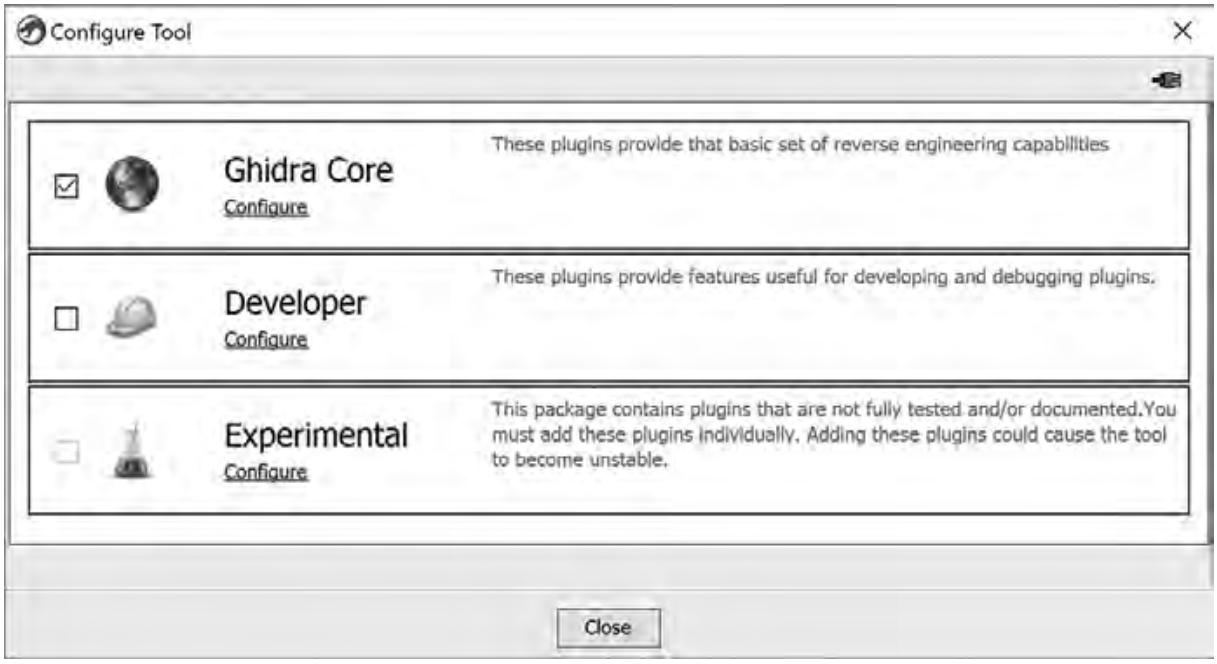


Рисунок 12-13. Параметры конфигурации проекта Ghidra

Хотя в составе установки Ghidra по умолчанию включена только Ghidra Core, вы можете установить флажок рядом с другими вариантами, чтобы включить и их. Используйте параметр Configure под категорией, чтобы выбрать (или снять выбор) отдельные плагины, появляющиеся в списке категории. На рисунке 12-14 показан список плагинов Ghidra Core, включая описание и категорию для каждого. Если щелкнуть плагин Ghidra в этом меню, окно в нижней части экрана предоставит дополнительную информацию о плагине.

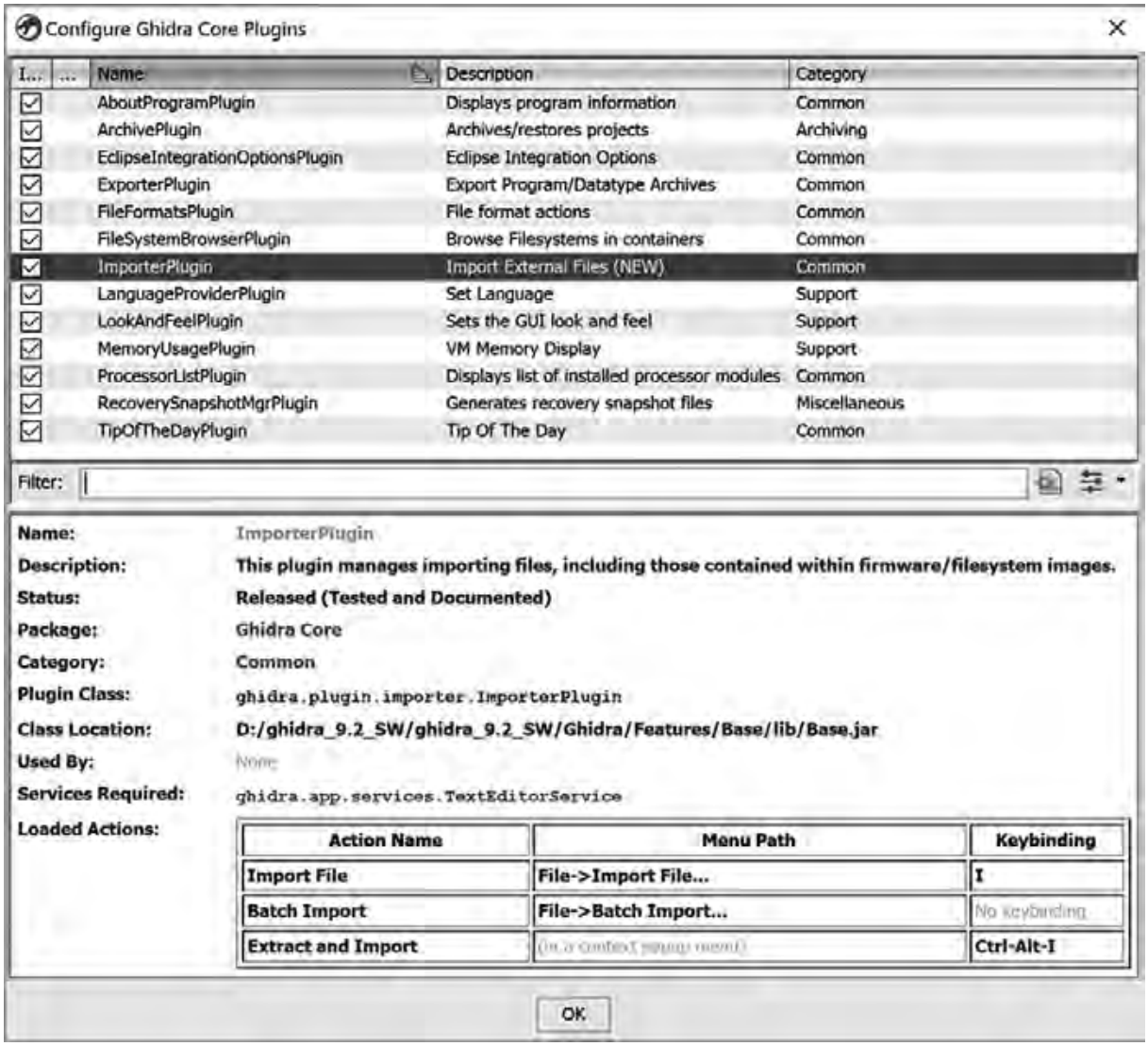


Рисунок 12-14. Окно конфигурации Ghidra Core с выбранным ImporterPlugin

Для конфигурации Ghidra доступны еще два пункта меню `Ghidra Project`. Первый - `File -> Install Extensions`; его мы обсудим в главе 15. Другой параметр, `Edit -> Plugin Path`, позволяет добавлять, изменять и удалять новые пользовательские пути плагинов, которые сообщают Ghidra, где искать дополнительные Java-классы помимо установленных значений по умолчанию. Через этот параметр можно включить дополнительные плагины и классы в ваш экземпляр Ghidra. После редактирования пути плагинов требуется перезапустить Ghidra, чтобы увидеть результат.

Теперь, когда вы увидели возможность изменять параметры плагинов, можно перейти к расширению использования плагинов. Пункт меню `Tools` позволяет выполнять операции, связанные с инструментами, включая создание новых инструментов, если ни один из существующих не подходит вашим потребностям точно. В этом случае мы будем создавать и использовать инструменты, представляющие собой коллекции существующих плагинов, а не писать плагины с нуля.

Инструменты

Большинство параметров инструментов находится в меню Tools окна `Ghidra Project`, показанном на рисунке 12-15. До сих пор вы использовали и изменяли инструмент по умолчанию, `CodeBrowser`, как основной инструмент анализа. Теперь мы покажем, как создавать пользовательские инструменты в Ghidra.

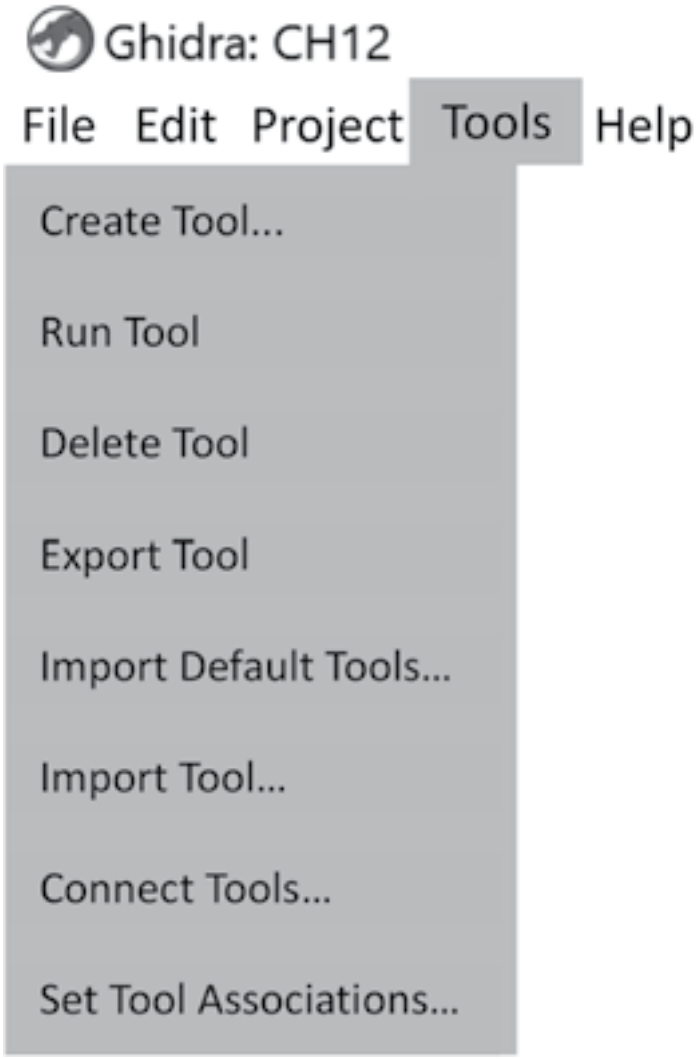


Рисунок 12-15. Параметры меню Ghidra Tools

Пункты меню Tools позволяют создать пустой инструмент, который затем можно заполнить плагинами; запустить инструмент из `Tool Chest`; удалить инструмент из `Tool Chest`; экспортировать инструмент для совместного использования; импортировать инструменты по умолчанию в `Tool Chest`; импортировать инструмент в `Tool Chest`; создавать связи между инструментами (см. главу 5); а также связывать типы файлов с конкретными инструментами.

Если вы экспериментировали с изменением инструмента `CodeBrowser`, вас могло раздражать, что инструмент по умолчанию изменяется для последующих открываемых файлов. Рассмотрим специализированный случай: вы хотите исследовать файл с большим количеством вызовов функций, по которому сложно перемещаться. В главе 10 мы показали использование графов вызовов функций и графов функций, помогающих понять поток управления программы. Оба этих графа открываются в собственных окнах, что может создавать сложности, если у вас открыто много файлов. Решим эти сложности с помощью специализированного инструмента под названием `ExamineControlFlow`, который можно использовать для анализа потока управления в программе.

Когда вы выбираете пункт меню `Tools -> Create Tool...`, перед вами появляются два окна, показанные на рисунке 12-16 одно над другим. Верхнее окно на рисунке представляет параметры плагинов, похожие на те, что вы видели на рисунке 12-13, но с одной дополнительной категорией, Function ID, которую мы обсудим в главе 13. Нижнее окно на рисунке - пустое окно разработки инструмента без названия, которое можно настроить для создания вашего инструмента `ExamineControlFlow`.

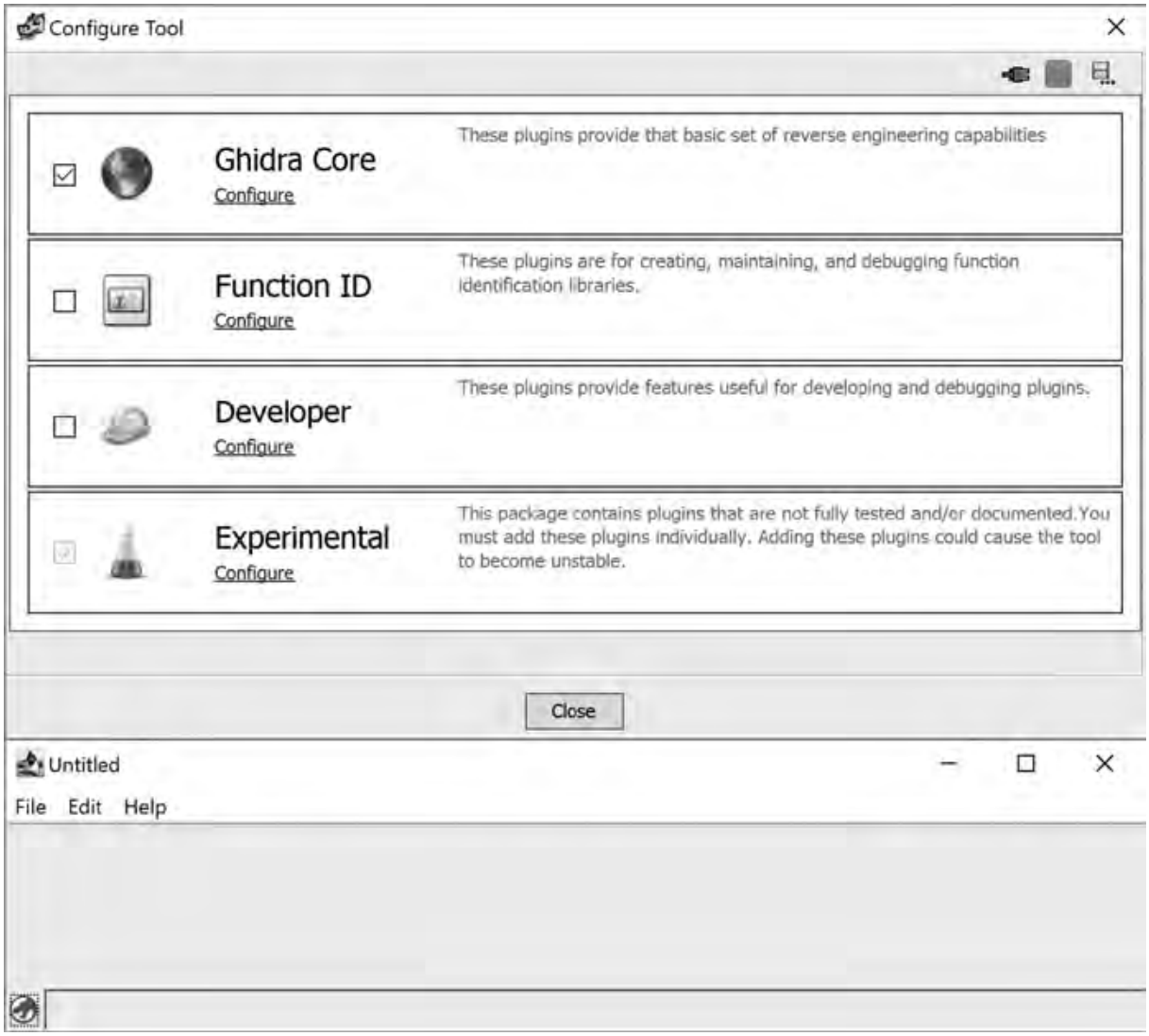


Рисунок 12-16. Окно Ghidra Configure Tool

Новый инструмент можно составить из плагинов категории Ghidra Core. Когда вы выбираете категорию Ghidra Core, окно разработки инструмента заполняется параметрами из Ghidra Core, как показано на рисунке 12-17. Получившееся окно имеет много общего с `CodeBrowser`. Это логично, поскольку `CodeBrowser` также основан на Ghidra Core.

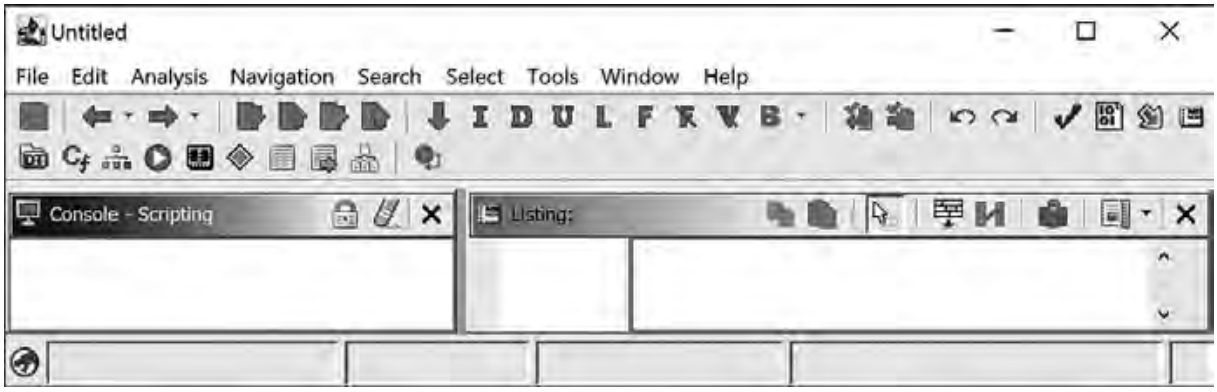


Рисунок 12-17. Новый инструмент без названия до конфигурации

Вам потребуется удалить некоторые плагины, которые вы не хотите видеть в новом инструменте, а затем указать окна, которые вам нужны. Щелкните параметр Configure под Ghidra Core и удалите следующие ненужные плагины. Можно удалить и многие другие, но ради краткости мы этого не делаем:

• Console
• DataTypeManagerPlugin
• EclipseIntegrationPlugin
• ProgramTreePlugin

Каждый из них связан с другими плагинами, поэтому при удалении каждого из них из нового инструмента Ghidra будет показывать предупреждение со списком дополнительных плагинов, которые также будут удалены. Вы можете в любое время добавить плагины обратно, выбрав File -> Configure в новом инструменте. Пример предупреждения, связанного с удалением DataTypeManagerPlugin, показан на рисунке 12-18.

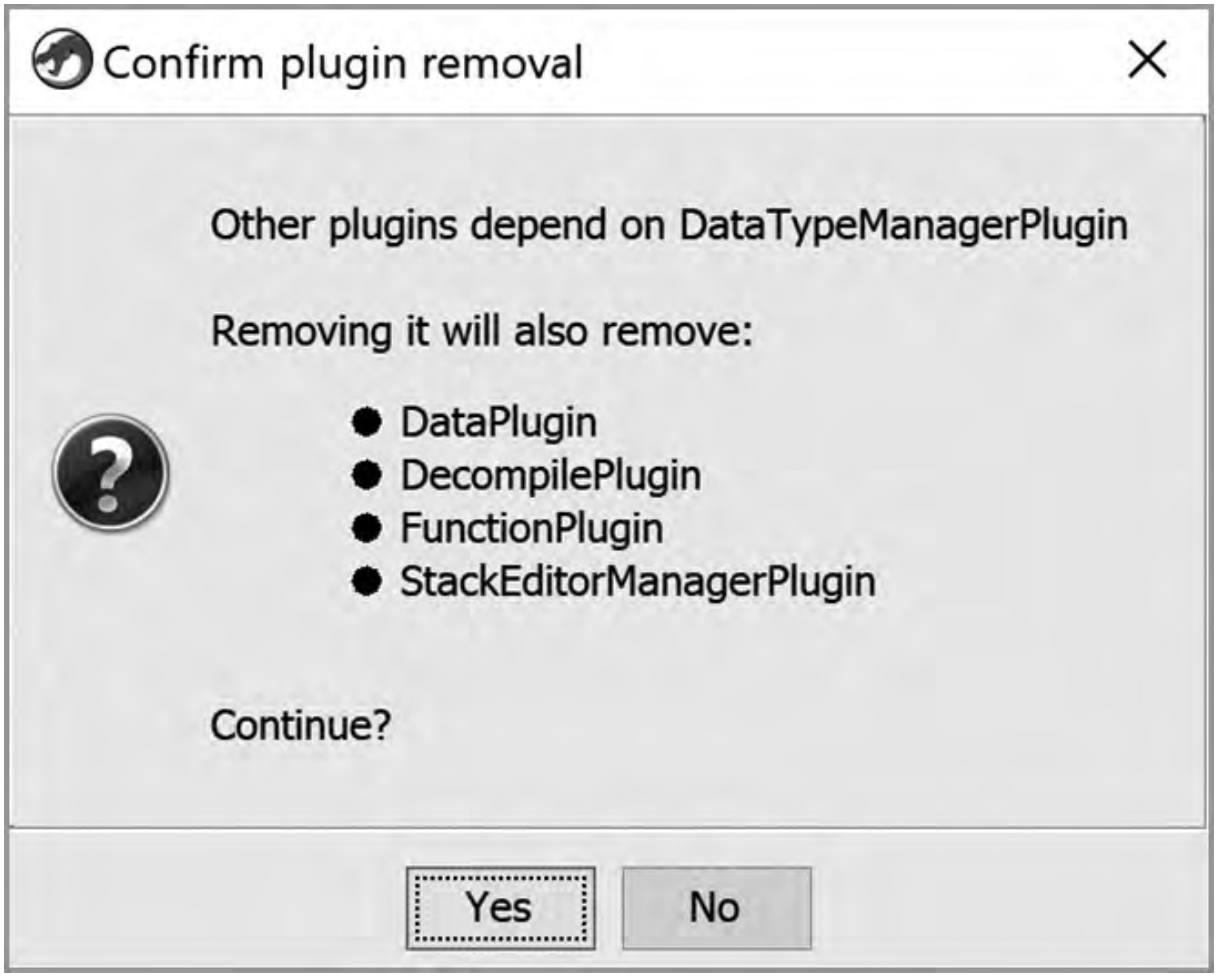


Рисунок 12-18. Предупреждение о зависимости плагина для DataTypeManagerPlugin

Вы также можете управлять макетом нового инструмента. В данном случае вы хотите видеть окна `Listing`, Function Call Graph и Function Graph в одном инструменте. Используя приемы, описанные в предыдущих главах, откройте нужные окна через меню `Window` в новом инструменте, а затем перетащите их в желаемые места. Новый инструмент без названия показан на рисунке 12-19.

Рисунок 12-19. Новый инструмент без названия

Поскольку вы планируете часто использовать этот инструмент и делиться им с коллегами, его следует сохранить, выбрав `File -> Save Tool As`. Появятся параметры, позволяющие задать имя инструмента и связать с ним значок (см. рисунок 12-20). Можно выбрать один из предоставленных значков или выбрать собственный файл изображения в поддерживаемом формате, например `.jpg`, `.png`, `.gif` и т. д.

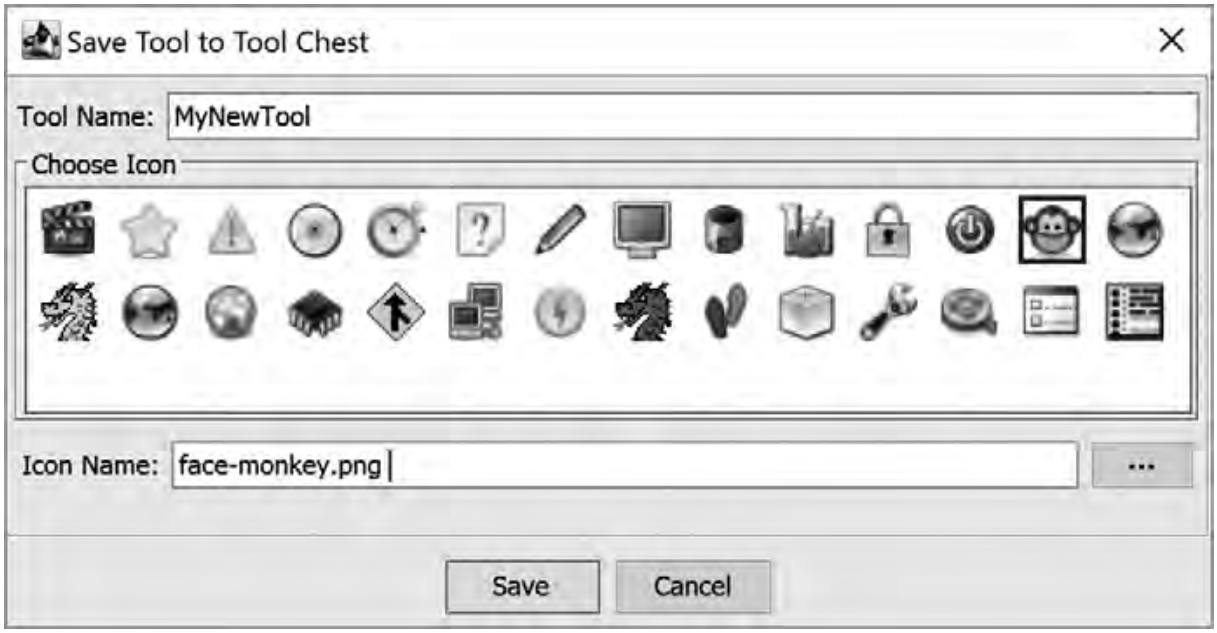


Рисунок 12-20. Параметры значков для новых инструментов

Этот новый инструмент (и другие инструменты, которые вы создадите) становится частью `Tool Chest` и будет отображаться как вариант в ваших проектах, как показано на рисунке 12-21.

Рисунок 12-21. Новый проект с новыми параметрами инструментов, отображенными в `Tool Chest`

Чтобы поделиться новым инструментом с другими, экспортируйте его с помощью `Tools -> Export Tools`. Ghidra попросит выбрать папку для сохранения инструмента, а затем создаст файл `.tool`, содержащий спецификацию вашего инструмента. Чтобы импортировать инструмент, используйте параметр `Tools -> Import Tool`.

Хотя двойной щелчок по файлу в окне `Ghidra Project` по умолчанию открывает файл в `CodeBrowser`, вы можете выбрать любой инструмент в `Tool Chest`, щелкнув файл правой кнопкой мыши и затем выбрав инструмент из контекстного меню. Как вариант, можно перетащить имя файла и бросить его на инструмент.

Чем больше вы используете Ghidra, тем отчетливее становится, что не существует универсального интерфейса Ghidra, который точно предоставляет инструменты, нужные для каждой задачи RE. Как специалист по обратной разработке, вы выбираете подход к анализу конкретного файла в значительной степени исходя из самого файла, цели анализа и продвижения к этой цели.

Мы посвятили большую часть этой главы и предыдущих глав описанию того, как можно изменить внешний вид Ghidra, а также набор доступных инструментов, чтобы удовлетворить ваши потребности. Последний шаг в настройке Ghidra - возможность сохранять созданные конфигурации, чтобы выбирать правильную конфигурацию в зависимости от выполняемого проекта анализа. Это достигается созданием и сохранением рабочих областей Ghidra.

Рабочие области

Рабочую область Ghidra можно рассматривать как виртуальный рабочий стол, включающий инструменты в их текущей конфигурации и связанные файлы. Представьте, что вы анализируете бинарный файл. Пока вы смотрите на файл, вы замечаете характеристики, похожие на другой файл, который анализировали на прошлой неделе. Вы хотите сравнить два файла, чтобы выявить сходства между двумя функциями, но при этом хотите продолжать анализировать текущий файл. Это две разные задачи, у которых есть общий файл.

Один способ одновременно двигаться по этим двум направлениям - создать рабочую область, связанную с каждой задачей анализа. Текущий анализ можно сохранить, выбрав `Project -> Workspace -> Add` в окне `Ghidra Project` и задав новой рабочей области имя. В этом примере назовем рабочую область `FileAnalysis`. Затем можно открыть другой инструмент из `Tool Chest` и, возможно, сравнить два файла с помощью специализированного инструмента, использующего `Diff View` (см. главу 23), а затем создать вторую рабочую область (`FileComparison`) тем же способом. Теперь можно легко переключаться между рабочими областями, выбирая рабочую область в раскрывающемся меню, показанном на рисунке 12-22, или используя параметр Switch в меню `Project` -> Workspace, который циклически переключает доступные рабочие области.

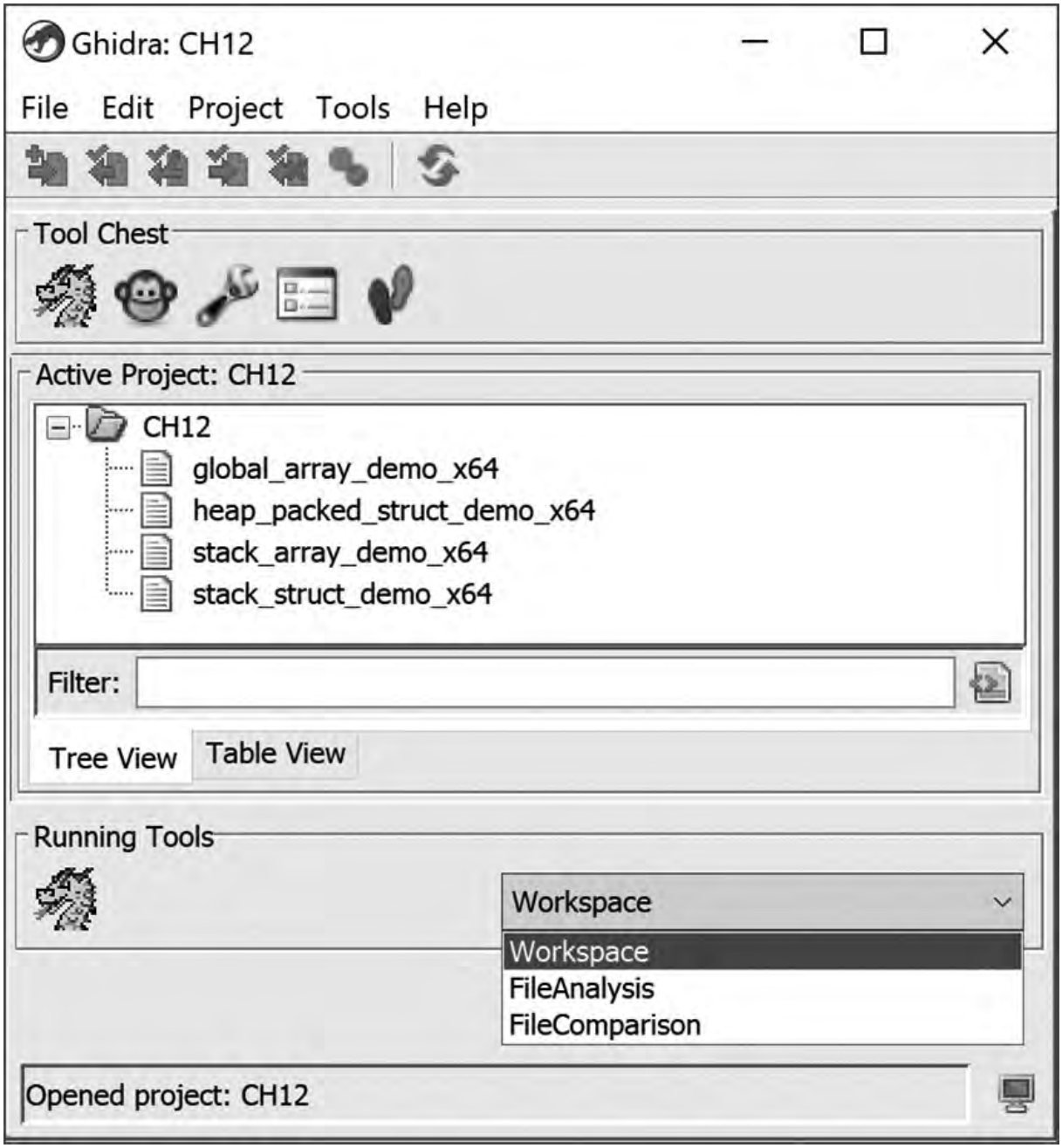


Рисунок 12-22. Параметры рабочей области в окне `Ghidra Project`

Резюме

Когда вы только начинаете работать с Ghidra, вас может полностью устраивать ее поведение по умолчанию и стандартный макет `CodeBrowser`. Однако по мере того как вы лучше осваиваете базовые возможности Ghidra, вы наверняка найдете способы настроить Ghidra под свой рабочий процесс обратной разработки. Невозможно в одной главе полностью охватить каждый возможный параметр, который предлагает Ghidra, но мы представили и разобрали на примерах те возможности настройки, которые, скорее всего, понадобятся вам на каком-то этапе опыта SRE. Поиск дополнительных полезных инструментов и параметров остается предметом самостоятельного исследования для любознательных читателей.

Глава 13. Расширение мировоззрения Ghidra

Одна из вещей, которых мы ожидаем от качественного инструмента обратной разработки, - полностью автоматическая идентификация и аннотирование как можно большей части бинарного файла. В идеальном случае идентифицируются 100 процентов инструкций, и они группируются в 100 процентов исходных функций, из которых состоит бинарный файл. У каждой из этих функций было бы имя и полный прототип, а все данные, которыми манипулируют функции, также были бы идентифицированы, включая полное понимание исходных типов данных, использованных программистами. Именно это и является целью Ghidra: начиная с начального импорта бинарного файла и продолжая автоматическим анализом, после чего все, что Ghidra не смогла сделать сама, становится упражнением для пользователя.

В этой главе мы рассмотрим методы, которые Ghidra использует для идентификации различных конструкций внутри бинарных файлов, и обсудим, как можно расширить ее способность делать это. Начнем с процессов начальной загрузки и анализа. Выбор, который вы делаете на этих этапах, помогает определить, какие ресурсы Ghidra задействует для анализируемого файла. Это возможность предоставить Ghidra сведения, которые она могла не обнаружить автоматически, чтобы этапы анализа Ghidra могли принимать более информированные решения. Затем мы рассмотрим, как Ghidra использует модели слов, типы данных и алгоритмы идентификации функций, а также как каждый из этих механизмов можно расширить, подстраивая его производительность под ваше конкретное применение RE.

Импорт файлов

Во время импорта диалог, показанный на рисунке 13-1, представляет начальный анализ Ghidra относительно идентичности файла, который будет направлять процесс загрузки. Вы можете переопределить любое из полей или продолжить с рекомендациями, которые сделала Ghidra. Дополнительные параметры, доступные через кнопку `Options...`, зависят от типа загружаемого файла. На рисунке 13-1 показаны параметры для PE-файла, а на рисунке 13-2 - параметры загрузки ELF-бинарного файла.

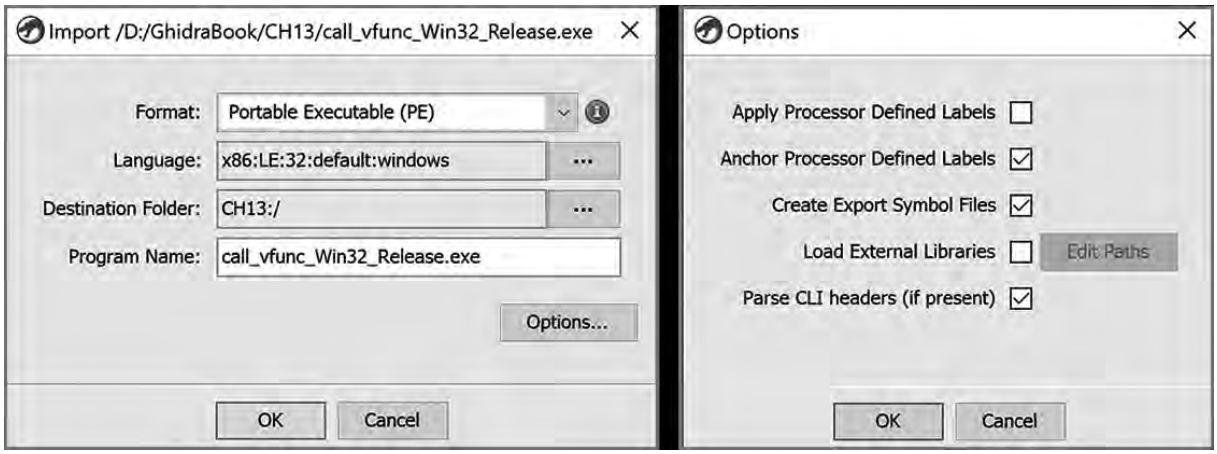


Рисунок 13-1. Диалог импорта и параметры для PE-файла

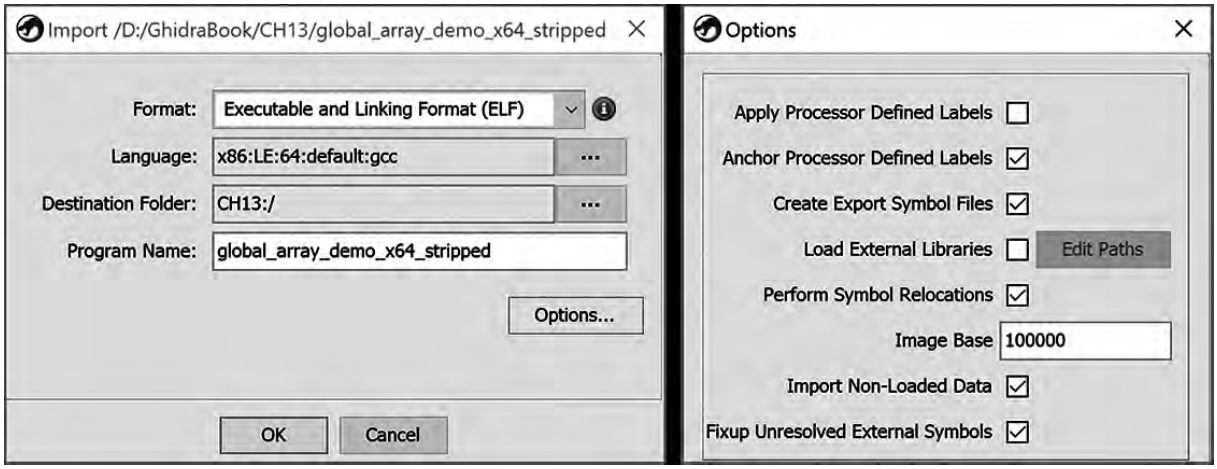


Рисунок 13-2. Диалог импорта и параметры для ELF-бинарного файла

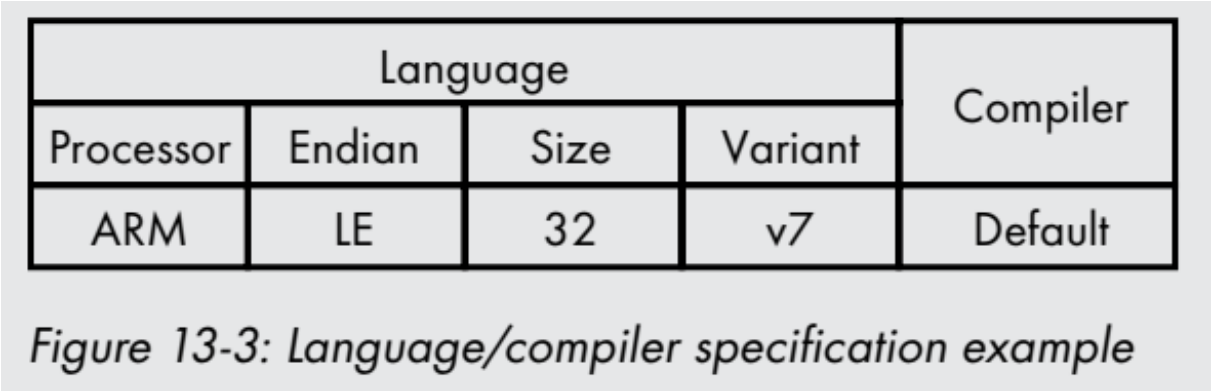
Параметр `Format` задает, какой загрузчик Ghidra будет использовать для импорта файла. Ghidra полагается на подробное знание загрузчика о конкретном формате файла, чтобы идентифицировать характеристики файла и выбрать правильные плагины для анализа. Хорошо написанный загрузчик распознает определенное содержимое или структурные особенности, чтобы определить тип файла, архитектуру и, желательно, компилятор, использованный для создания бинарного файла. Информация о компиляторе может улучшить идентификацию функций. Чтобы снять отпечаток компилятора, загрузчик исследует структуру бинарного файла в поиске характеристик, специфичных для компилятора (например, количество, имена, положение и порядок секций программы), или ищет в бинарном файле последовательности байтов, специфичные для компилятора (например, блоки кода или строки). Например, в бинарных файлах, скомпилированных с помощью `gcc`, нередко встречаются строки версии, такие как `GCC: (Ubuntu 7.3.0-27ubuntu1~18.04) 7.3.0`.

Спецификации языка и компилятора

Поле `Language` на рисунках 13-1 и 13-2 точно определяет, как Ghidra будет интерпретировать любые байты, распознанные как машинный код внутри загружаемого файла. Спецификация языка и компилятора состоит из трех-пяти подполь, разделенных двоеточиями:

- Поле имени процессора задает тип процессора, для которого был собран бинарный файл. Оно направляет Ghidra к конкретному подкаталогу внутри `Ghidra/Processors`.
- Поле порядка байтов указывает порядок байтов процессора бинарного файла: от младшего к старшему (`LE`) или от старшего к младшему (`BE`).
- Поле размера архитектуры (разрядности) обычно совпадает с размером указателя для выбранного процессора (16/32/64 бита).
- Поле варианта/режима процессора используется для выбора конкретной модели выбранного процессора или определения конкретного режима работы. Например, при выборе процессора x86 можно выбрать режимы System Management Mode, Real Mode, Protected Mode или default. Для процессора ARM можно выбрать модели `v4`, `v4T`, `v5`, `v5T`, `v6`, `Cortex`, `v7`, `v8`, `v8T` и другие.
- Если известно, поле компилятора задает компилятор или, в некоторых случаях, соглашение о вызове, использованные для компиляции бинарного файла. Допустимые имена включают `windows`, `gcc`, `borlandcpp`, `borlanddelphi` и `default`.

Рисунок 13-3 разбивает идентификатор языка `ARM:LE:32:v7:default` на составляющие подполя. Одна из важнейших задач загрузчика - вывести правильную спецификацию языка и компилятора.



Language				Compiler
Processor	Endian	Size	Variant	
ARM	LE	32	v7	Default

Figure 13-3: Language/compiler specification example

Рисунок 13-3. Пример спецификации языка и компилятора

Когда Ghidra завершает процесс загрузки, отображается окно `Import Results Summary`, как показано на рисунке 13-4.

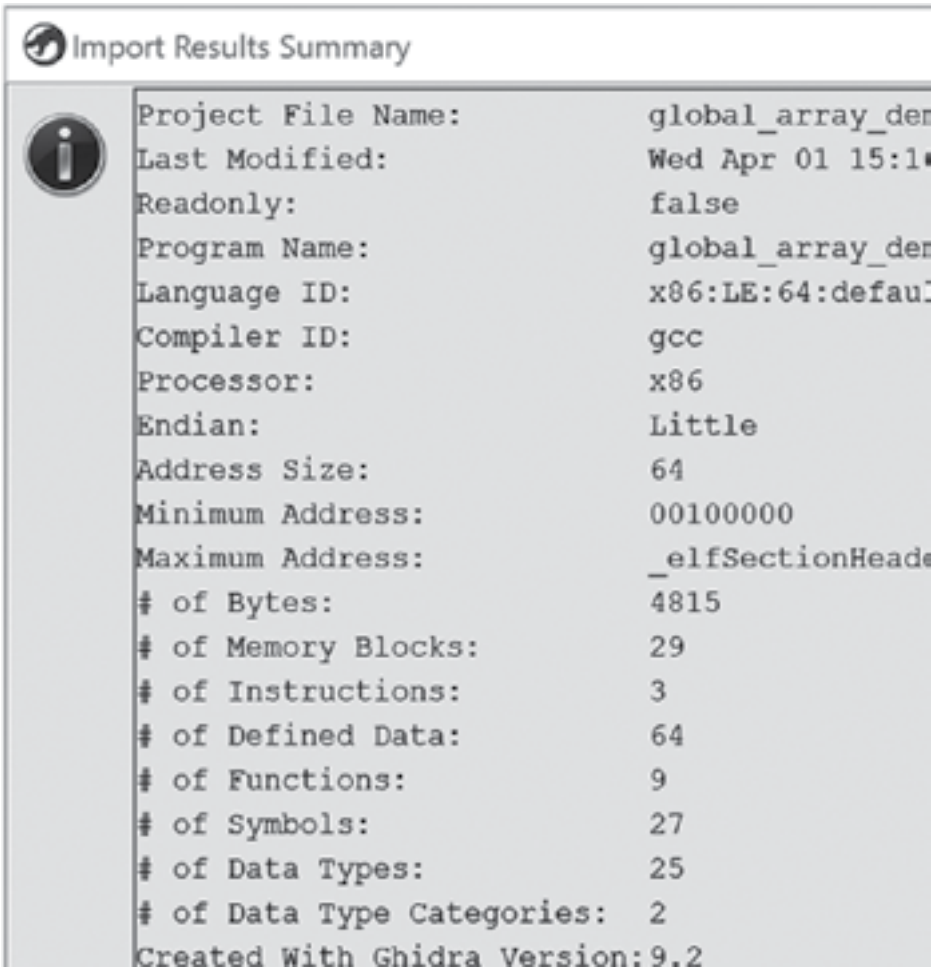


Рисунок 13-4. Окно `Import Results Summary` *для ELF-бинарного файла*

В этой сводке указана `ELF Required Library`, `lib.so.6`. Обратите внимание, что эта библиотека не была бы перечислена как требование, если бы файл был статически слинкован. Когда исполняемый файл зависит от нескольких общих библиотек, может быть перечислено больше одного библиотечного файла. Понимание того, от каких библиотек зависит программа, может направить вас к ресурсам, которые могут понадобиться при анализе программы. Например, если в списке обязательных библиотек появляется `libssl.so` или `libcrypto.so`, вам может понадобиться найти документацию OpenSSL и, возможно, исходный код. Далее в этой главе мы обсудим, как Ghidra может использовать исходный код. После успешного импорта файла можно выполнить

автоматический анализ.

Анализаторы

Автоматический анализ выполняется набором совместно работающих средств анализа (анализаторов), которые активируются либо вручную (например, при открытии нового файла), либо автоматически, когда обнаружено изменение, способное повлиять на результирующее дизассемблирование. Анализаторы запускаются последовательно в приоритетном порядке на основе типа анализатора, потому что изменения, которые делает один анализатор, могут повлиять на последующие анализаторы. Например, анализаторы стека не могут смотреть на функции, пока анализатор функций не просмотрел все вызовы и не создал функции. Эту иерархию мы подробнее исследуем в главе 15, когда будем строить анализатор.

Когда вы открываете новый файл в `CodeBrowser` и выбираете автоматический анализ, Ghidra показывает список анализаторов, которые можно запустить для этого файла. Список стандартных и необязательных анализаторов зависит от информации о файле, предоставленной загрузчиком (она также показывается пользователю как часть сводки импорта, как на рисунке 13-4). Например, Windows x86 PE RTTI Analyzer был бы малополезен при анализе ELF- или ARM-бинарного файла. Выбор анализаторов по умолчанию можно изменить через меню `Edit -> Tool Options`.

Некоторые анализаторы также доступны как однократные варианты через меню Analysis -> One Shot в `CodeBrowser`. Анализатор появляется в списке, если он поддерживает однократное использование и применим к типу анализируемого файла. Однократный анализ полезен для запуска анализаторов, которые не были выбраны во время начального автоматического анализа, или для повторного запуска анализатора после обнаружения новой информации, от которой может выиграть дополнительный анализ. Например, если во время начального анализа вы получили сообщение об отсутствующем PDB, можно найти PDB-файл и затем запустить PDB analyzer.

Параметр `Analyze All Open` в меню `CodeBrowser -> Analysis` анализирует все открытые файлы проекта сразу, используя список анализаторов, выбранных в `Edit -> Tool Options`. Если все открытые файлы проекта имеют одну и ту же архитектуру (спецификацию языка и компилятора), будут проанализированы все файлы. Любые файлы, не совпадающие с архитектурой текущего файла, не будут включены в анализ. Это гарантирует, что анализаторы соответствуют типу анализируемого файла.

Многие инструменты `CodeBrowser`, включая анализаторы, полагаются на различные артефакты, чтобы идентифицировать важные конструкции в файле. К счастью, мы можем расширять многие из этих артефактов, улучшая возможности Ghidra. Начнем с обсуждения файлов моделей слов и того, как они используются для идентификации специальных строк и типов строк в результатах поиска.

Модели слов

Модель слов предоставляет способ идентифицировать специальные строки и типы строк, которые вам интересно искать, например известные идентификаторы, адреса электронной почты, пути к каталогам, расширения файлов и т. д. Когда поиск строк связан с моделью слов, окно результатов String Search будет включать столбец IsWord, указывающий, является ли найденная строка словом согласно модели слов. Определение интересующих строк как допустимых слов и последующая фильтрация по допустимым словам - хороший способ приоритизировать строки для дальнейшего изучения.

На высоком уровне модель слов использует обучающие наборы допустимых строк, чтобы определить, что "если триграмма X (последовательность из трех символов) встречается в последовательности Y длиной Z, то существует вероятность P, что Y является словом". Полученная вероятность косвенно используется как порог для решения, следует ли считать строку допустимым словом во время анализа.

`StringModel.sng`, показанный на рисунке 13-5, является файлом модели слов по умолчанию для поиска строк в Ghidra.

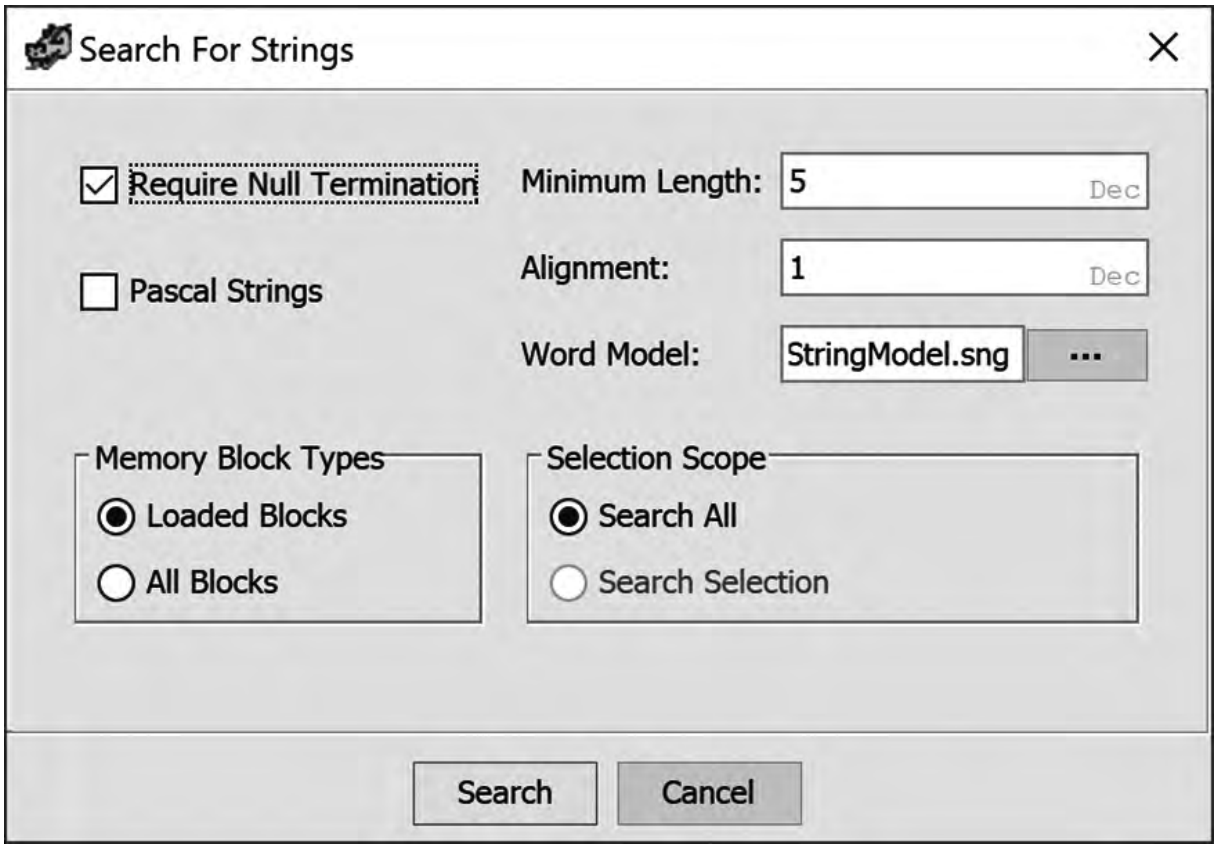


Рисунок 13-5. Диалог Search for Strings

Следующий фрагмент из файла `StringModel.sng` показывает формат допустимого файла модели слов:

```
# Тип модели: нижний регистр
# Файл обучения: contractions.txt
# Файл обучения: uniqueStrings_012615_minLen8.edited.txt
# Файл обучения: connectives
# Файл обучения: propernames
# Файл обучения: web2
# Файл обучения: web2a
# Файл обучения: words
# [^] обозначает начало строки
# [$] обозначает конец строки
# [SP] обозначает пробел
# [HT] обозначает горизонтальную табуляцию
[HT]    [HT]    [HT]    17
[HT]    [HT]    [SP]    8
[HT]    [HT]    (   1
[HT]    [HT]    ;   1
[HT]    [HT]    \   25
[HT]    [HT]    a   2
[HT]    [HT]    b   1
[HT]    [HT]    c   1
```

Первые 12 строк файла - комментарии метаданных о модели. В этом примере тип модели - `lowercase`, что, вероятно, означает, что модель не различает верхний и нижний регистр. Перечислены имена обучающих файлов, использованных для этой модели. Имена обычно указывают на содержимое: `contractions.txt`, скорее всего, является файлом допустимых сокращений, таких как `can't`. Четыре строки описывают нотацию для некоторых непечатаемых ASCII-символов, используемых в триграммах. Затем начинается собственно список триграмм, где каждая строка содержит три символа триграммы и значение, используемое при определении вероятности того, что триграмма является частью слова.

Вы можете дополнить или заменить модель слов по умолчанию, отредактировав `StringModel.sng` или создав новые файлы моделей и сохранив их в `Ghidra/Features/Base/data/stringngrams`, а затем выбрав новый файл в поле Word Model диалога Search for Strings. Есть много причин изменять модели слов, например включение строк, специфичных для известных семейств вредоносных программ, или обнаружение слов на языках, отличных от английского. В конечном счете модели слов дают мощный способ управлять типами строк, которые Ghidra распознает как более приоритетные, помечая их в окне Strings.

Похожим образом мы можем редактировать и расширять типы данных, которые распознает Ghidra.

Типы данных

`Data Type Manager` позволяет управлять всеми типами данных, связанными с файлом. Ghidra позволяет повторно использовать определения типов данных, сохраняя их в файлах архивов типов данных. Каждый корневой узел в окне `Data Type Manager` является архивом типов данных. На рисунке 13-6 показано окно `Data Type Manager` с тремя архивами типов данных, выбранными загрузчиком Ghidra.

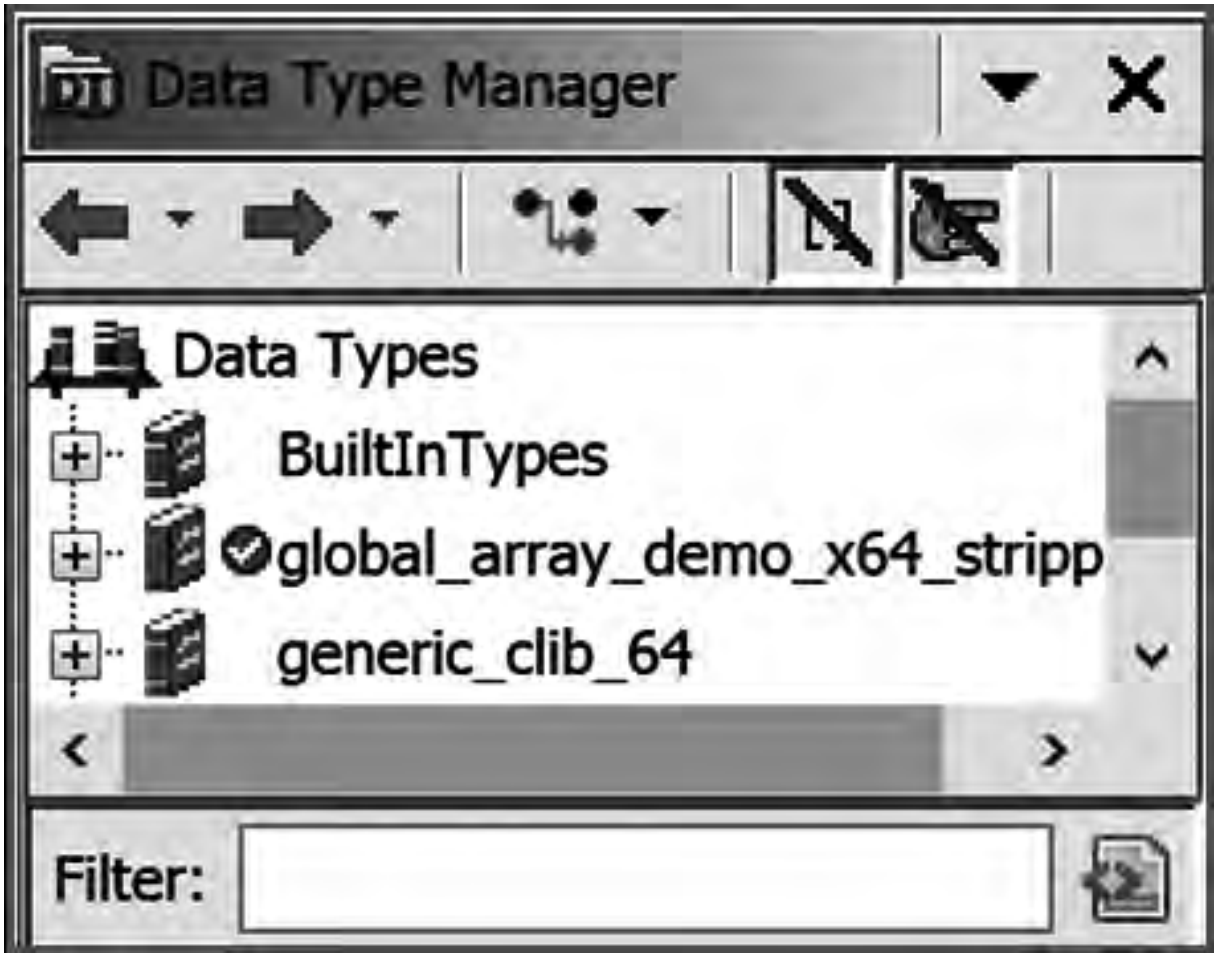


Рисунок 13-6. Окно `Data Type Manager`

Архив `BuiltInTypes` отображается всегда. Этот архив включает все (и только те) типы, которые смоделированы внутри Ghidra Java-классами, реализующими интерфейс `ghidra.program.model.data.BuiltInDataType`. Ghidra ищет каждый такой класс в своем classpath, чтобы заполнить этот архив.

Второй архив специфичен для анализируемого файла и имеет то же имя, что и файл. В данном случае архив связан с файлом `global_array_demo_x64`. Галочка рядом с архивом указывает, что он связан с активным файлом. Изначально Ghidra заполняет этот архив типами данных, специфичными для формата файла (например, типами данных, связанными с PE или ELF). Во время автоматического анализа Ghidra копирует дополнительные типы из других архивов в этот архив, когда распознает их использование в программе. Иными словами, этот архив содержит подмножество всех типов данных, известных `Data Type Manager`, которые используются в текущей программе. Этот архив также является домом для любых пользовательских типов данных, которые вы решите создать в Ghidra, как обсуждалось в разделе "Создание структур с помощью Ghidra" на странице 166.

Третий архив предоставляет 64-битные прототипы функций ANSI C и типы данных библиотеки C. Этот конкретный архив содержит информацию, извлеченную из заголовков стандартной библиотеки C 64-битной Linux-системы, и является одним из нескольких платформо-специфичных архивов в установке Ghidra по умолчанию. Он присутствует потому, что конкретный бинарный файл имеет зависимость от библиотеки `libc.so.6`, как указано на рисунке 13-4. В установке Ghidra по умолчанию есть еще четыре платформо-специфичных архива данных, расположенные в каталоге `Ghidra/Features/Base/data/typeinfo` внутри подкаталога, специфичного для платформы. Имена файлов указывают поддерживаемые платформы: `generic_clib.gdt`, `generic_clib_64.gdt`,

`mac_osx.gdt`, `windows_vs12_32.gdt` и `windows_vs12_64.gdt`. Расширение `.gdt` используется для всех архивов типов данных Ghidra.

Помимо архивов, которые загрузчик Ghidra выбирает автоматически, вы можете добавлять собственные архивы типов данных как узлы в окне `Data Type Manager`. Для демонстрации на рисунке 13-7 показано окно `Data Type Manager` после добавления всех файлов `.gdt` по умолчанию в список Data Types. Правая часть рисунка показывает меню для манипулирования архивами и типами данных. Дополнительные архивы загружаются через пункт Open File Archive, который открывает браузер файлов для выбора интересующего архива.

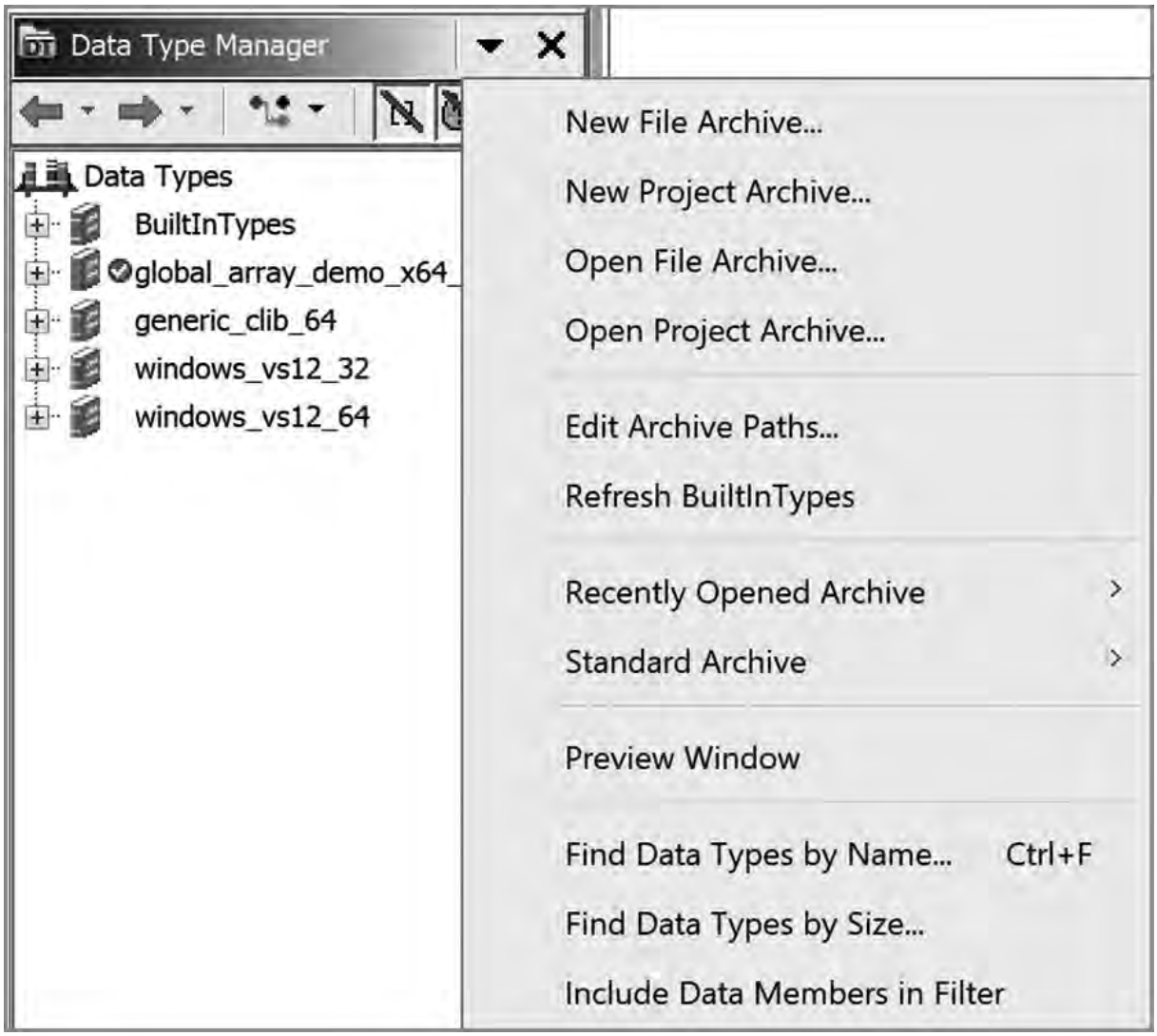


Рисунок 13-7. `Data Type Manager` *со всеми стандартными архивами и раскрытым меню options*

Чтобы добавить новые встроенные типы в архив `BuiltInTypes`, добавьте соответствующие `.class`-файлы в classpath Ghidra. Если вы добавляете типы во время работы Ghidra, необходимо выполнить Refresh BuiltInTypes (см. рисунок 13-7), чтобы они появились. Операция обновления заставляет Ghidra повторно просканировать classpath и найти любые новые классы `BuiltInDataType`. Любознательный читатель может найти многочисленные примеры встроенных типов в исходном дистрибутиве Ghidra по пути `Ghidra/Framework/SoftwareModeling/src/main/java/ghidra/program/model/data`.

Создание новых архивов типов данных

Невозможно заранее предусмотреть каждый тип данных, который может встретиться при анализе бинарных файлов. Архивы, включенные в дистрибутив Ghidra, содержат типы данных, извлеченные из наиболее часто используемых библиотек в системах Windows (Windows SDK) и Unix (библиотека C). Когда Ghidra не содержит информации о типах данных, используемых в анализируемой программе, она дает возможность создавать новые архивы типов данных, заполнять их разными способами и делиться ими с другими. В следующих разделах мы обсудим три способа, которыми вы, вероятно, будете создавать новые архивы типов данных.

Разбор заголовочных файлов C

Один из наиболее распространенных источников информации о типах данных - заголовочные файлы C. Предположим, что у вас есть нужные заголовочные файлы или вы потратили время на их самостоятельное создание. Тогда можно создать собственный архив типов данных с помощью плагина C-Parser, извлекая информацию из существующего заголовочного файла C. Например, если вы часто анализируете бинарные файлы, которые линкуются с криптографической библиотекой OpenSSL, можно скачать исходный код OpenSSL и попросить Ghidra разобрать включенные заголовочные файлы, чтобы создать архив типов данных и сигнатур функций OpenSSL.

Этот процесс далеко не так прямолинеен, как может показаться. Заголовочные файлы часто заполнены макросами, предназначенными для влияния на поведение компилятора в зависимости от используемого компилятора, операционной системы и целевой архитектуры. Например, C-структура

```
struct parse_demo {
    uint32_t int_member;
    char    *ptr_member;
};
```

занимает 8 байт при компиляции на 32-битной системе и 16 байт при компиляции на 64-битной системе. Эта вариативность создает проблему для Ghidra, которая пытается действовать как универсальный препроцессор, и именно вам нужно провести Ghidra через процесс разбора, чтобы создать полезный архив. Когда придет время использовать ваш архив с Ghidra, вы должны убедиться, что архив был создан способом, совместимым с анализируемым бинарным файлом, то есть не загружайте 64-битные архивы для анализа 32-битного файла.

Чтобы разобрать один или несколько заголовочных файлов C, выберите `File -> Parse C Source` в `CodeBrowser`, чтобы открыть диалог, показанный на рисунке 13-8. Раздел исходных файлов для разбора предоставляет упорядоченный список заголовочных файлов, которые плагин должен разобрать. Порядок важен, потому что типы данных и директивы препроцессора из одного файла становятся доступными для следующего файла.

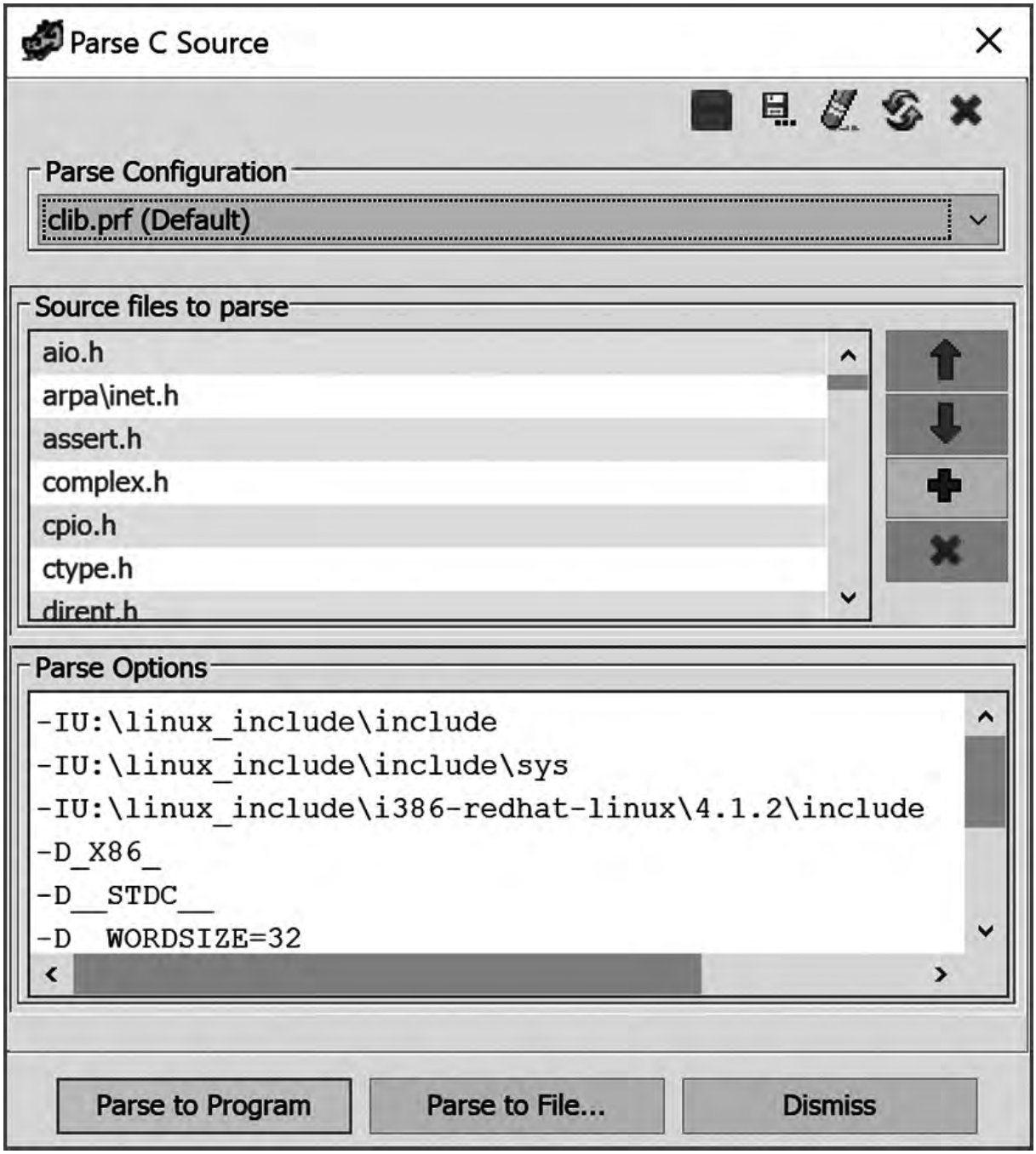


Рисунок 13-8. Диалог `Parse C Source`

Поле `Parse Options` предоставляет список параметров, похожих на параметры командной строки компилятора, которые влияют на поведение плагина C-Parser. Парсер распознает только параметры `-I` (каталог включаемых файлов) и `-D` (определить макрос), понятные большинству компиляторов. Ghidra предлагает несколько конфигураций препроцессора в виде файлов `.prf`, которые можно выбрать, чтобы получить разумные значения по умолчанию для распространенных сочетаний операционных систем и компиляторов. Вы также можете настроить любую из доступных конфигураций или создать собственную с нуля и сохранить ее в собственный `.prf` для будущего использования. Распространенное изменение параметров парсера - корректно задать архитектуру, на которую должен ориентироваться C-Parser, поскольку все поставляемые конфигурации нацелены на x86. Например, в конфигурации, ориентированной на Linux, можно заменить `-D_X86_` на `-D__ARMEL__`, если вы анализируете ARM-бинарные файлы с порядком байтов от младшего к старшему.

Вывод плагина можно объединить с текущим активным файлом кнопкой Parse to Program или сохранить в отдельный файл архива типов данных Ghidra (`.gdt`) кнопкой Parse to File. Дополнительная информация о C-Parser доступна в `Ghidra Help`.

Создание нового файлового архива

В качестве альтернативы разбору заголовочных файлов C вы можете захотеть сохранить пользовательские типы данных, которые создаете при анализе файла, в архиве, которым можно поделиться с другими пользователями Ghidra или использовать в других проектах Ghidra. Параметр New File Archive в `Data Type Manager` (см. рисунок 13-7) просит выбрать имя файла и место сохранения, а затем создает новый пустой архив, отображаемый в окне `Data Type Manager`. Вы можете добавлять новые типы в архив, используя приемы, описанные в разделе "Создание структур с помощью Ghidra" на странице 166. После создания архива им можно делиться с другими пользователями Ghidra или использовать его в других проектах Ghidra.

Создание нового архива проекта

Архив данных проекта существует только внутри проекта, в котором был создан. Это может быть полезно, если вы ожидаете повторно использовать пользовательские типы данных в нескольких файлах внутри проекта, но не ожидаете применять эти типы данных за пределами проекта. В `Data Type Manager` параметр `New Project Archive` (см. рисунок 13-7) просит выбрать папку внутри проекта для нового архива, а затем создает новый пустой архив, отображаемый в окне `Data Type Manager`. Как и в другие архивы типов данных, в него можно добавлять новые типы по мере необходимости.

Function IDs

Когда вы приступаете к обратной разработке любого бинарного файла, последнее, на что хочется тратить время, - обратная разработка библиотечных функций, поведение которых можно куда легче понять, просто прочитав справочную страницу, исходный код или немного поискав в интернете. К сожалению, статически слинкованные бинарные файлы размывают границу между кодом приложения и библиотечным кодом: целые библиотеки объединяются с кодом приложения в один монолитный исполняемый файл. К счастью, в Ghidra есть инструменты для распознавания и пометки библиотечного кода независимо от того, был ли код взят из библиотечного архива или просто является результатом повторного использования кода в нескольких бинарных файлах. Это позволяет сосредоточиться на уникальном коде приложения. Анализатор Function ID распознает многие распространенные библиотечные функции с помощью сигнатур функций, включенных в Ghidra, а базы сигнатур функций можно расширять с помощью плагина Function ID.

Анализатор Function ID работает с базами данных Function ID (`FidDbs`), которые используют иерархию хеш-значений для характеристики функций. Для каждой функции вычисляется полный хеш, предназначенный быть устойчивым к изменениям, которые может внести компоновщик, и специфический хеш, помогающий различать варианты функций. Главное различие между ними в том, что специфический хеш может включать конкретные значения константных операндов на основе эвристики, тогда как полный хеш этого не делает. Сочетание двух хешей вместе с информацией о любых связанных родительских и дочерних функциях образует отпечаток для каждой библиотечной функции, который записывается в FidDb. Анализатор Function ID выводит тот же тип отпечатка для каждой функции в анализируемом бинарном файле и сравнивает его со всеми известными отпечатками в релевантных FidDbs. Когда находится совпадение, Ghidra восстанавливает исходное имя функции из FidDb, применяет соответствующую метку к анализируемой функции, добавляет функцию в окно `Symbol Tree` и обновляет комментарий-заголовок функции. Ниже приведен пример комментария-заголовка для функции `_malloc`:

```
*************************************************************
* Библиотечная функция - SingleMatch                        *
* Имя: _malloc                                              *
* Библиотека: Visual Studio 2005 Release                    *
*************************************************************
```

Информация о функциях в FidDb хранится иерархически и включает имя, версию и вариант. Поле варианта используется для кодирования информации, такой как настройки компилятора, которые влияют на хеши, но не являются частью номера версии.

Анализатор Function ID предлагает несколько параметров, доступных при выборе анализатора в диалоге Auto Analysis, чтобы управлять его поведением, как показано на рисунке 13-9. `Instruction count threshold` - настраиваемый порог, предназначенный для уменьшения ложных срабатываний от случайных совпадений с маленькими функциями. Ложные срабатывания возникают, когда функция ошибочно сопоставляется с библиотечной функцией. Ложные отрицания возникают, когда функция не сопоставляется с библиотечной функцией, хотя должна была бы. Порог примерно представляет минимальное число инструкций, которое функция, ее родители и потомки должны содержать в сумме, чтобы рассматриваться для совпадения. Дополнительную информацию об оценках совпадения см. в разделе `Scoring and Disambiguation` в `Ghidra Help`.

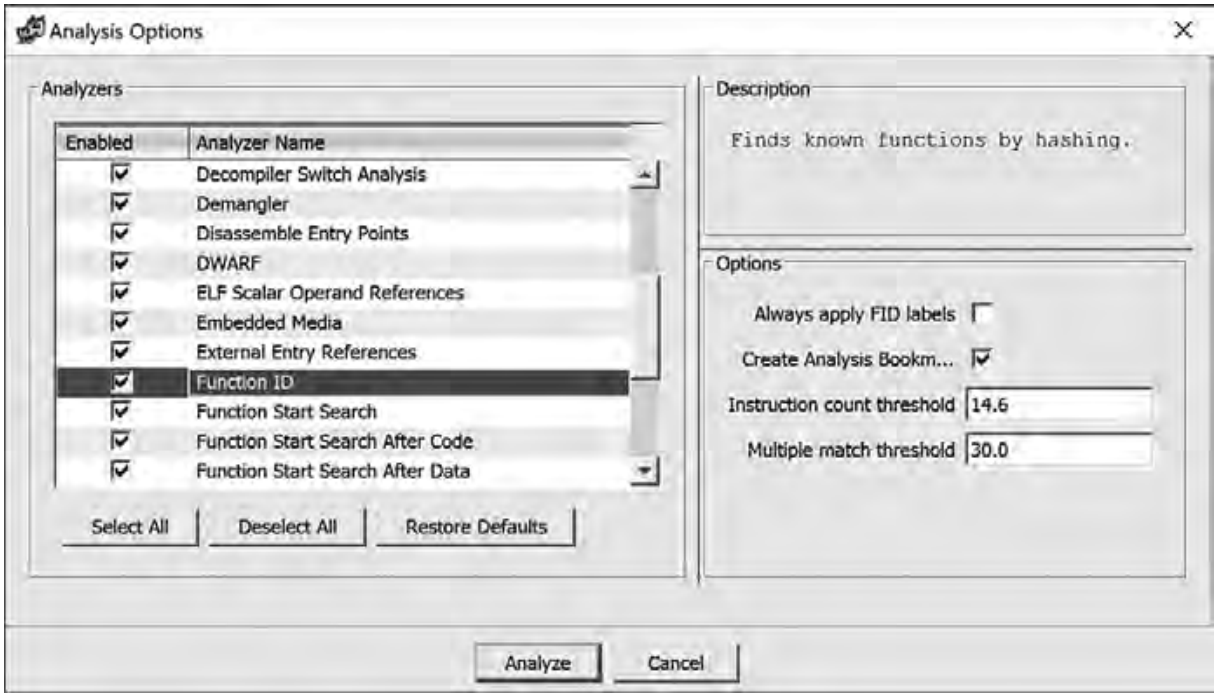


Рисунок 13-9. Параметры автоматического анализа

Поскольку фактическая функциональность внутри бинарного файла обычно содержится в функциях, возможность расширять сигнатуры функций имеет первостепенное значение для уменьшения дублирования усилий, и эту работу облегчает плагин Function ID.

Плагин Function ID

Плагин Function ID (не путайте с анализатором Function ID) позволяет создавать, изменять и контролировать связи для FidDb. Этот плагин не включен в установке Ghidra по умолчанию. Чтобы включить его, выберите File -> Configure в окне `CodeBrowser`, затем установите флажок Function ID. Выберите Configure в описании Function ID и выберите `FidPlugin`, чтобы увидеть дополнительную информацию о действиях, связанных с плагином, как показано на рисунке 13-10.

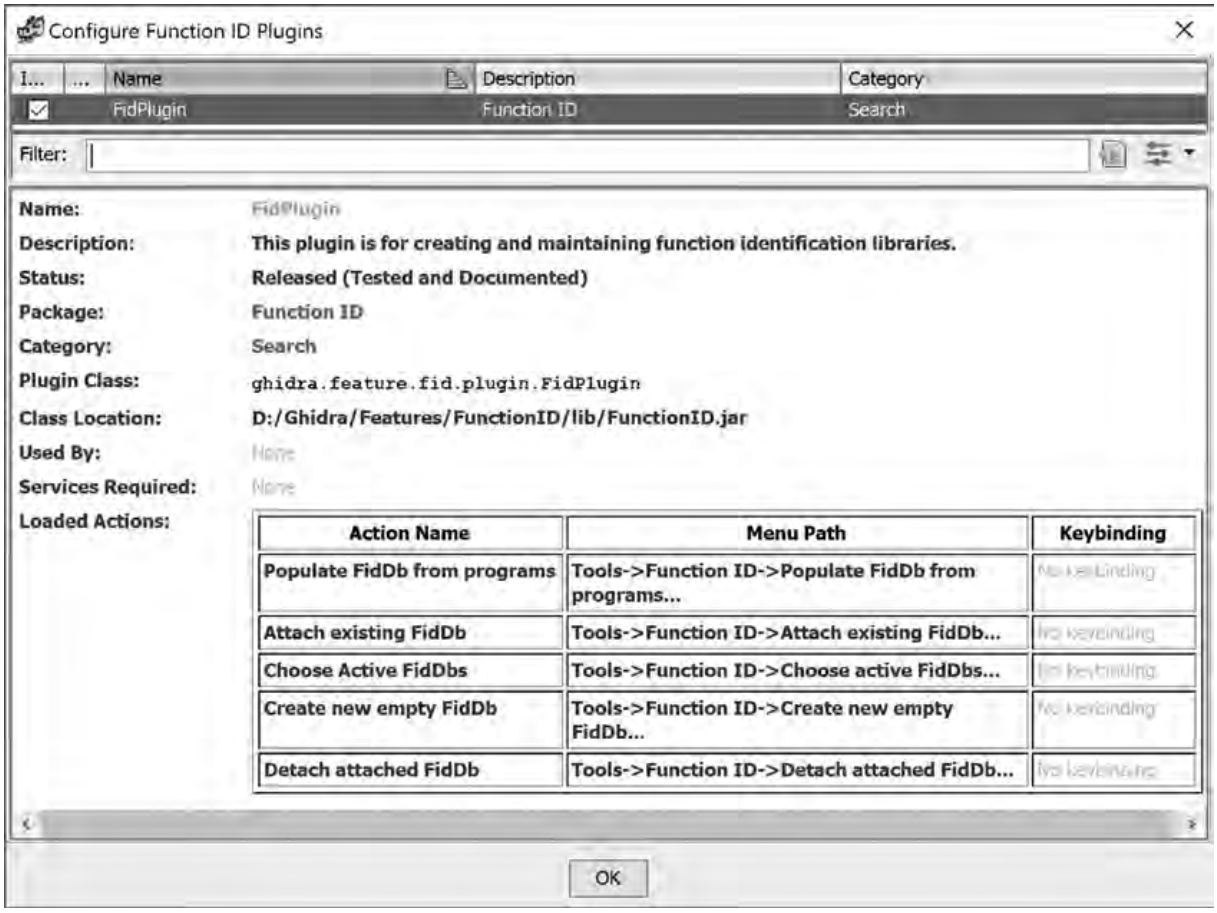


Рисунок 13-10. Сведения о FidPlugin

После включения плагин Function ID управляется через меню Tools -> Function ID в `CodeBrowser`, как показано на рисунке 13-11.

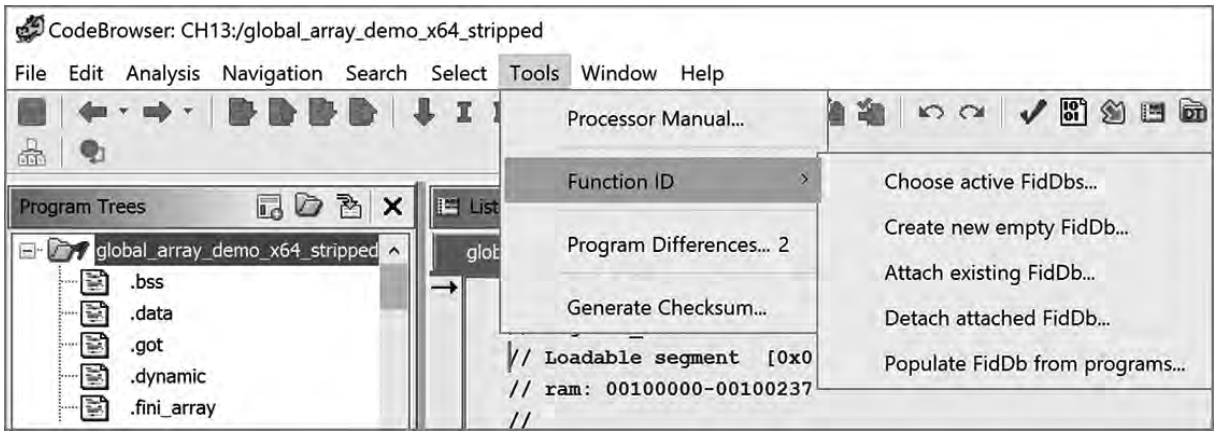


Рисунок 13-11. Подменю Function ID в `CodeBrowser`

Прежде чем пройти пример использования плагина Function ID для расширения сигнатур Ghidra, кратко обсудим пять новых пунктов меню:

Choose active FidDbs. Отображает список активных баз данных Function ID. Каждую можно выбрать или снять выбор с помощью связанного флажка.

`Create new empty FidDb`. Позволяет создать и назвать новую базу данных Function ID. Новая FidDb будет перечислена при выборе `Choose active FidDbs`.

Attach existing FidDb. Отображает диалог выбора файла, позволяющий добавить существующую FidDb в список активных FidDb. После добавления FidDb можно выбрать Choose active FidDbs и увидеть добавленную FidDb в списке.

Detach existing FidDb. Может применяться только к FidDb, которые были подключены вручную. Операция удаляет связь между выбранной FidDb и текущим экземпляром Ghidra.

`Populate FidDb from programs`. Создает новые отпечатки функций для добавления в существующую FidDb. Диалог на рисунке 13-12 используется для управления этим процессом; его использование мы обсудим чуть позже.

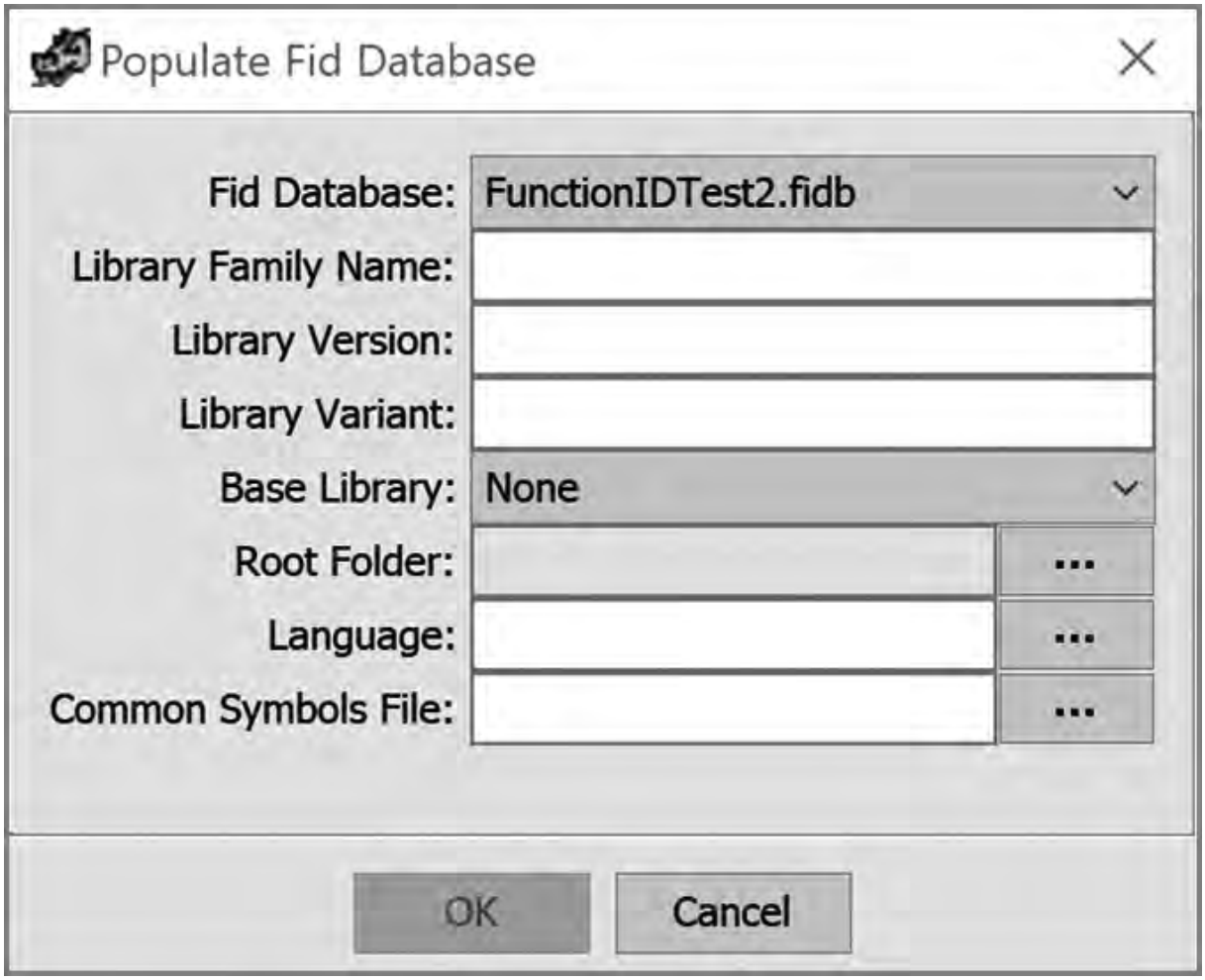


Рисунок 13-12. Диалог Populate Fid Database

Пример плагина Function ID: UPX

Когда мы выполняем автоматический анализ бинарных файлов, содержащих очень мало функций помимо библиотечных функций, которые Ghidra распознает, наша задача обратной разработки несколько упрощается. Мы можем сосредоточиться на подмножестве функций, которые Ghidra не распознает, предполагая, что именно там находится новая интересная функциональность. Наша задача становится намного сложнее, когда Ghidra вообще не может идентифицировать функции. Когда мы, люди-аналитики, распознаем функции и расширяем способность Ghidra распознавать те же функции в будущем, мы сокращаем будущий объем работы. Пройдем демонстрацию того, насколько мощным может быть такое расширение.

Предположим, мы загружаем 64-битный ELF-бинарный файл Linux в Ghidra и выполняем автоматический анализ. Получившиеся записи `Symbol Tree` показаны на рисунке 13-13. Мы используем `Symbol Tree`, чтобы перейти к точке входа и изучить код. Начальный анализ приводит нас к мысли, что бинарный файл упакован Ultimate Packer for eXecutables (UPX) и что функции, которые мы видим, были добавлены упаковщиком UPX для распаковки бинарного файла во время выполнения. Мы подтверждаем эту гипотезу, сравнивая байты, которые видим в `entry`, с опубликованными байтами функции точки входа UPX. В качестве альтернативы можно создать собственный бинарный файл, упакованный UPX, для сравнения. Теперь добавим эту информацию в нашу FidDb, чтобы не выполнять тот же анализ снова, если когда-нибудь встретим другой 64-битный ELF-бинарный файл Linux, упакованный UPX.

Рисунок 13-13. Предполагаемые функции упаковщика UPX для `upx_demo1_x64_static.upx`

Функции, добавляемые в FidDb, должны иметь осмысленные имена. Поэтому мы изменяем имена функций в нашем примере так, чтобы указать, что они являются частью упаковщика UPX, как показано на рисунке 13-14, а затем добавляем эти функции в новую базу данных Function ID, чтобы Ghidra могла корректно помечать функции в будущем.

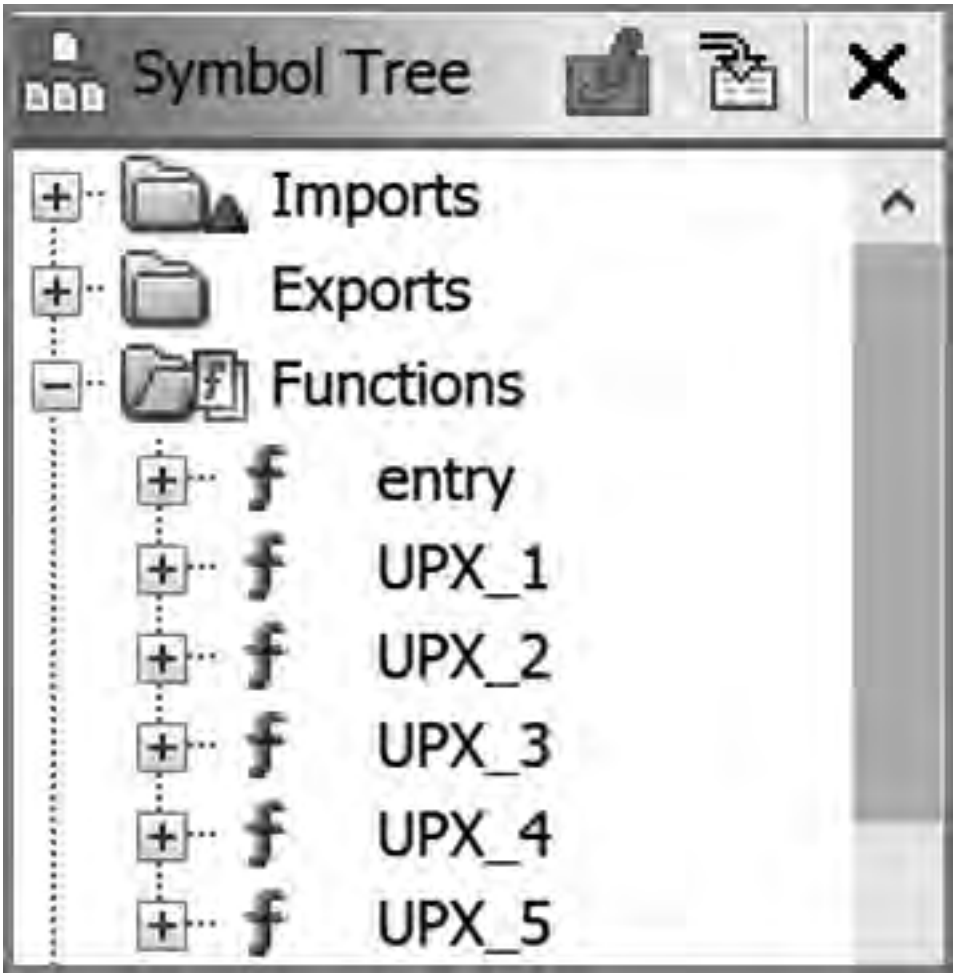


Рисунок 13-14. Помеченные функции упаковщика UPX для `upx_demo1_x64_static.upx`

Мы создаем новую FidDb, выбирая `Tools -> Function ID -> Create new empty FidDb`, а затем называем новую FidDb `UPX.fidb`. Затем заполняем новую базу данных информацией, извлеченной из обновленного бинарного файла, выбрав `Tools -> Function ID -> Populate FidDb from programs`. Введите информацию о FidDb в появившийся диалог, как показано на рисунке 13-15.

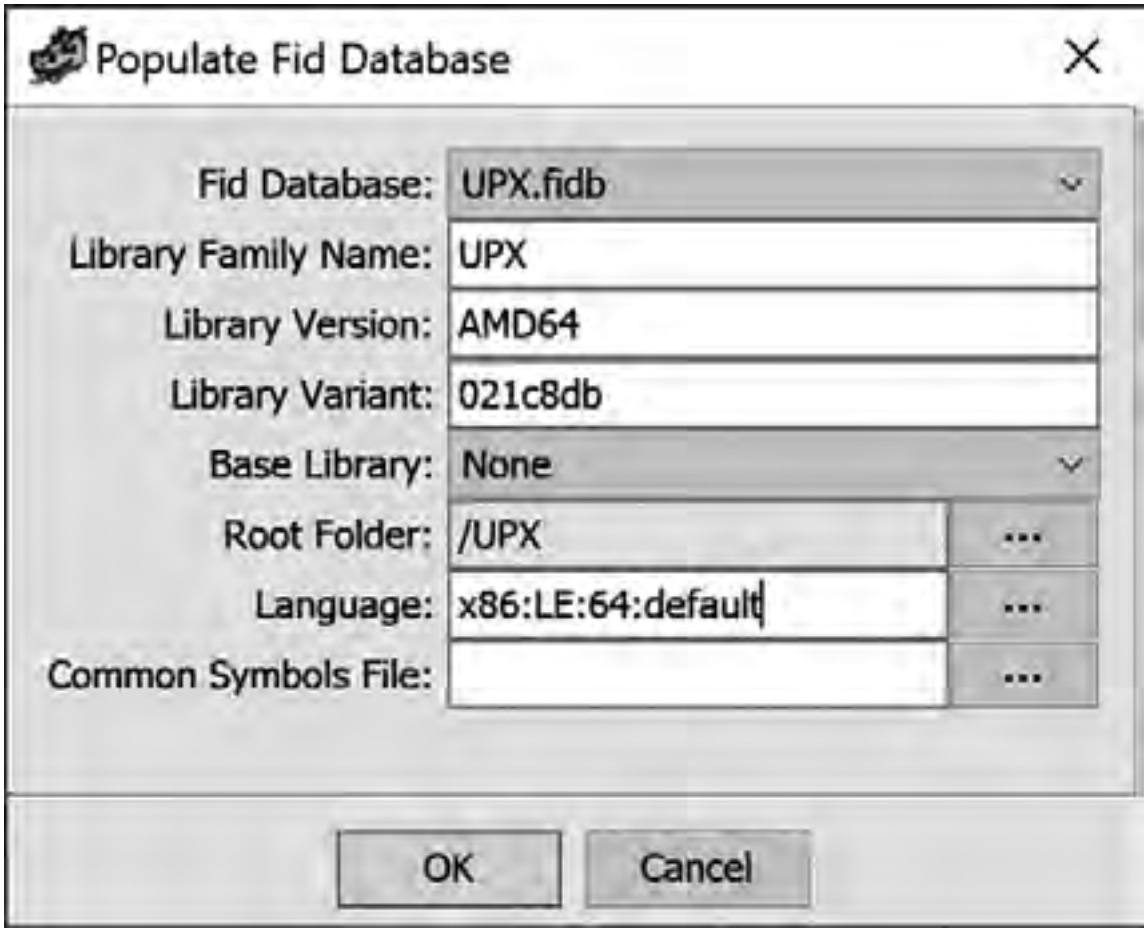


Рисунок 13-15. Диалог Populate Fid Database

Назначение каждого поля и введенные нами значения описаны ниже:

Fid Database. `UPX.fidb` - имя нашей новой FidDb. Раскрывающийся список позволяет выбрать одну из всех созданных вами FidDbs.

`Library Family Name`. Выберите имя, описывающее библиотеку, из которой извлекаются данные функций. В нашем случае мы ввели `UPX`.

Library Version. Это может быть номер версии, имя платформы или сочетание того и другого. Поскольку UPX доступен для многих платформ, мы выбрали версию библиотеки на основе архитектуры бинарного файла.

Library Variant. Это поле можно использовать для любой дополнительной информации, отличающей эту библиотеку от других с той же версией. В этом примере мы использовали commit ID для этой версии UPX из репозитория UPX на GitHub (https://github.com/upx/).

Base Library. Здесь можно сослаться на другую FidDb, которую Ghidra будет использовать для установления отношений родитель/потомок. Мы не использовали базовую библиотеку, поскольку UPX полностью самодостаточен.

Root Folder. Это поле задает папку проекта Ghidra. Все файлы в выбранной папке будут обработаны во время процесса приема функций. В этом случае мы выбрали `/UPX` из раскрывающегося меню.

`Language`. Содержит идентификатор языка Ghidra, связанный с новой FidDb. Чтобы файл из корневой папки был обработан, его идентификатор языка должен совпадать с этим значением. Эта запись заполняется из окна `Import Results Summary` для бинарного файла, но может быть изменена с помощью кнопки справа от текстового поля.

Common Symbols File. Это поле задает файл со списком функций, которые следует исключить из процесса приема. В данном случае это поле не используется.

Когда мы нажимаем `OK`, начинается процесс приема. После его завершения мы видим результаты заполнения FidDb (рисунок 13-16).

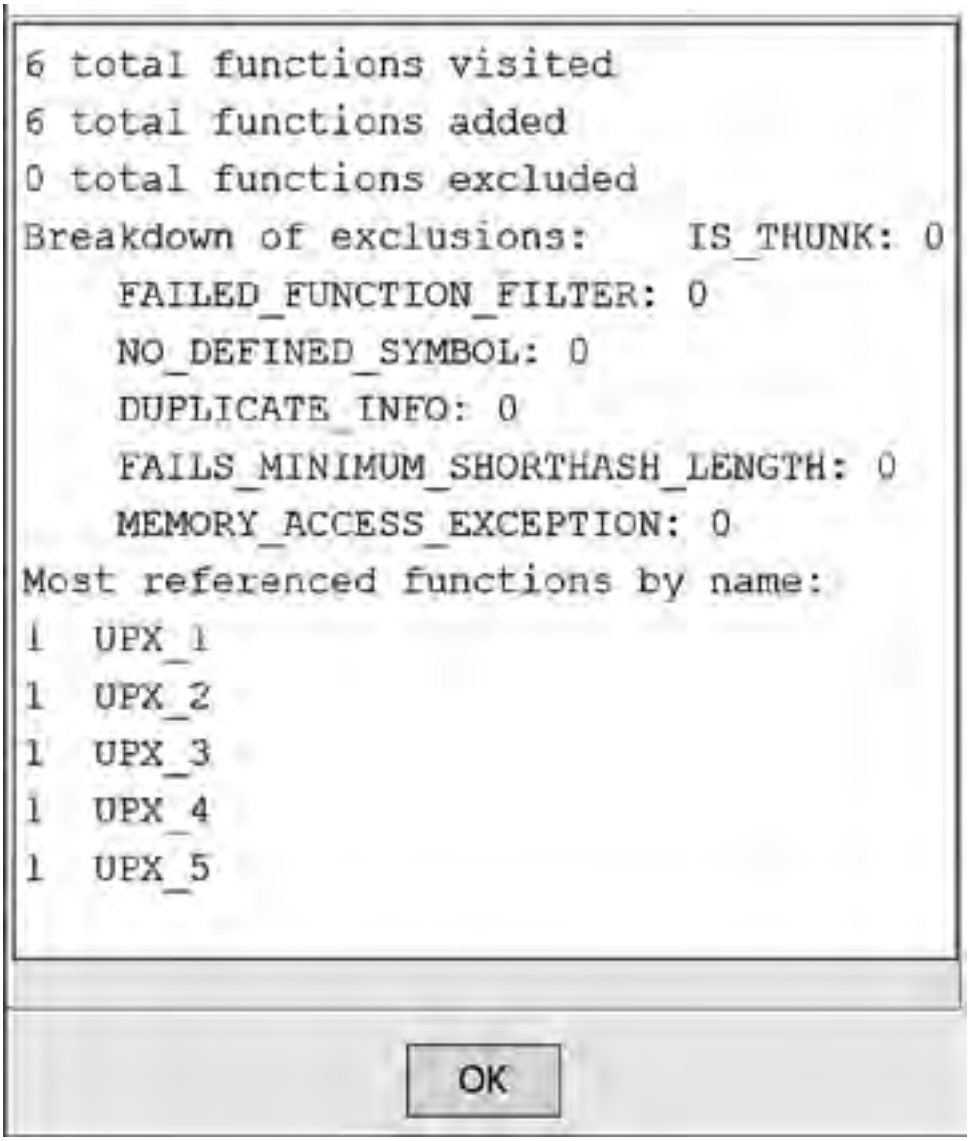


Рисунок 13-16. Окно результатов заполнения UPX FidDb

После создания новой FidDb Ghidra может использовать ее для идентификации функций в любом анализируемом бинарном файле. Мы демонстрируем это, загрузив новый 64-битный ELF-бинарный файл Linux, упакованный UPX, `upx_demo2_x64_static.upx`, и выполнив автоматический анализ без анализатора Function ID. Получившийся `Symbol Tree`, показанный на рисунке 13-17, содержит пять неопознанных функций, как мы и ожидаем.

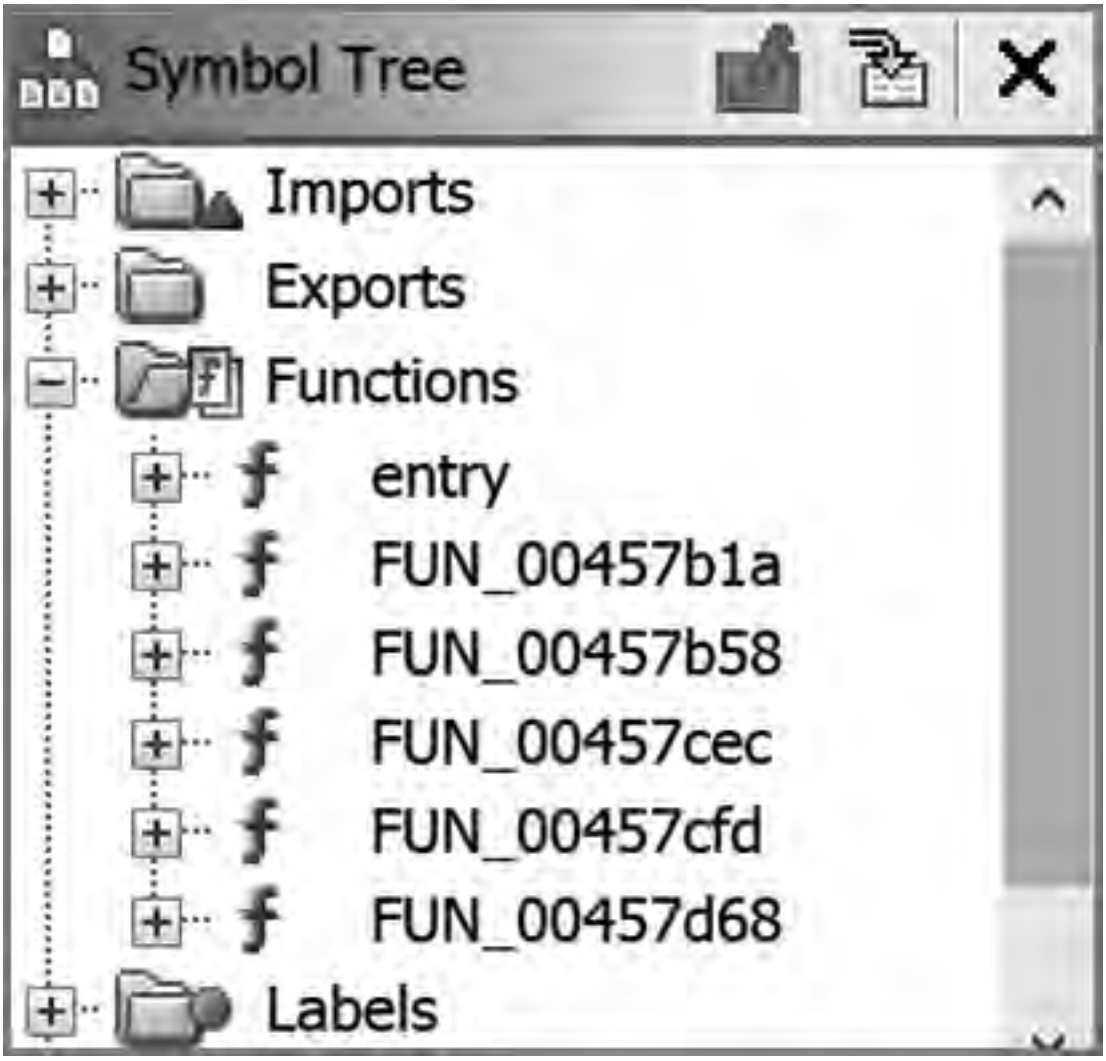


Рисунок 13-17. Запись `Symbol Tree` *для* `upx_demo2_x64_static.upx` *до анализатора Function ID*

Запуск Function ID как однократного анализатора (Analysis -> One Shot -> Function ID) приводит к `Symbol Tree`, показанному на рисунке 13-18, где уже присутствуют имена функций UPX.

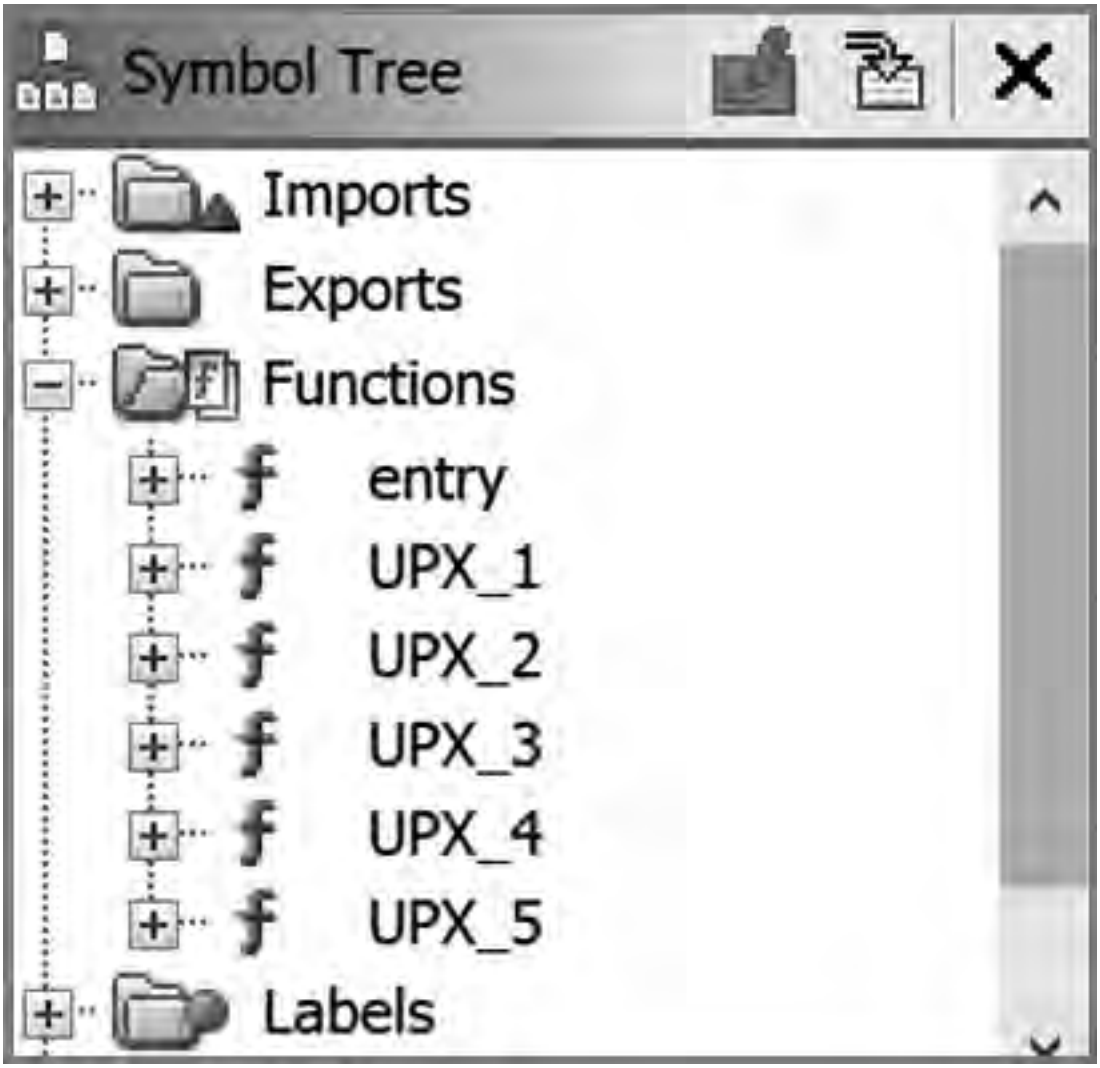


Рисунок 13-18. Запись `Symbol Tree` *для* `upx_demo2_x64_static.upx` *после анализатора Function ID*

Анализатор также обновляет окно `Listing` новыми именами функций и комментариями-заголовками, например комментарием-заголовком для `UPX_1`, показанным ниже. Этот комментарий-заголовок содержит информацию, которую мы предоставили при создании FidDb:

```
*************************************************************
* Библиотечная функция - одиночное совпадение               *
* Имя: UPX_1                                                *
* Библиотека: UPX AMD64 021c8db                             *
*************************************************************
                      undefined UPX_1()
undefined         AL:1           <RETURN>
    UPX_1                            XREF[1]:     UPX_2:00457c08(c)
00457b1a 48 8d 04 2f  LEA    RAX,[RDI + RBP*0x1]
00457b1e 83 f9 05     CMP    ECX,0x5
```

Создание новых FidDb - это только начало расширения возможностей Ghidra по идентификации функций. Можно анализировать параметры, связанные с функцией, и сохранять их в архиве типов данных. Затем, когда `Function ID` корректно идентифицирует функцию, можно перетащить соответствующую запись `Data Type Manager` на функцию в окне `Listing`, и прототип функции будет обновлен подходящими параметрами.

Пример плагина Function ID: профилирование статической библиотеки

Когда вы выполняете обратную разработку статически слинкованного бинарного файла, одна из первых желаемых вещей - FidDb, совпадающая с функциями, слинкованными в этот бинарный файл, чтобы Ghidra могла идентифицировать библиотечный код и избавить вас от его анализа. Следующий пример отвечает на два важных вопроса: (1) как узнать, есть ли у вас такая FidDb, и (2) что делать, если ее нет? Ответ на первый вопрос прост: Ghidra поставляется как минимум с дюжиной FidDb в виде файлов .fidbf, и все они связаны с библиотечным кодом Visual Studio. Если бинарный файл не является бинарным файлом Windows и вы еще не создали или не импортировали никаких FidDb, потребуется создать собственную FidDb с помощью плагина Ghidra Function ID, что отвечает на второй вопрос.

Самое важное при заполнении новой FidDb - понимать, что вам нужен входной источник с высокой вероятностью совпадения с любыми бинарными файлами, к которым вы планируете применять FidDb. В примере с UPX у нас был бинарный файл, содержащий код, который, как подсказывала интуиция, мы могли бы встретить снова в будущем. В обычном случае статической линковки у нас есть бинарный файл, и мы просто хотим сопоставить как можно больше кода в этом бинарном файле.

Есть разные способы распознать, что вы имеете дело со статически слинкованным бинарным файлом. В Ghidra посмотрите на папку Imports внутри Symbol Tree. Эта папка будет пустой для полностью статически слинкованного бинарного файла, которому не нужны импортированные функции. Частично статически слинкованный бинарный файл может иметь некоторые импорты, поэтому можно искать строки авторских прав или строковые данные версии известных библиотек в окне Defined Strings.

В командной строке можно использовать простые утилиты, такие как file и strings:

```
$ file upx_demo2_x64_static_stripped
  upx_demo2_x64_static_stripped: ELF 64-bit LSB executable, x86-64,
  version 1 (GNU/Linux), statically linked, for GNU/Linux 3.2.0,
  BuildID[sha1]=54e3569c298166521438938cc2b7a4dda7ab7f5c, stripped
$ strings upx_demo2_x64_static_stripped | grep GCC
  GCC: (Ubuntu 7.4.0-1ubuntu1~18.04.1) 7.4.0
```

Вывод file сообщает, что бинарный файл статически слинкован, очищен от всех символов и взят из Linux-системы. Очищенный бинарный файл не содержит знакомых имен, которые могли бы подсказать поведение функций. Фильтрация вывода strings с помощью grep GCC определяет компилятор, GCC 7.4.0, а также Linux-дистрибутив, Ubuntu 18.04.1, использованный для сборки бинарного файла. Ту же информацию можно найти через функциональность Search -> Program Text в CodeBrowser, используя GCC как квалификатор. Вероятно, этот бинарный файл был слинкован с libc.a, архивом стандартных функций библиотеки C, используемым в статически слинкованных бинарных файлах на Unix-подобных системах. Поэтому мы берем копию libc.a из Ubuntu 18.04.1 и используем ее как отправную точку для восстановления символов в нашем очищенном бинарном файле. Дополнительные строки в бинарном файле могли бы привести нас к выбору дополнительных статических библиотек для анализа Function ID; однако в этом примере мы ограничимся libc.a.

Чтобы использовать libc.a для заполнения FidDb, Ghidra должна идентифицировать содержащиеся в нем инструкции и функции. Формат архивного файла, откуда и расширение .a, определяет контейнер для других файлов, чаще всего объектных файлов (.o), которые компилятор может извлечь и слинковать в исполняемый файл. Процесс импорта контейнерных файлов в Ghidra отличается от процесса импорта одиночных бинарных файлов, поэтому при импорте libc.a через File -> Import, как мы обычно делаем для одиночного файла, Ghidra предлагает альтернативные режимы импорта, как показано на рисунке 13-19. Эти варианты также доступны из меню File.

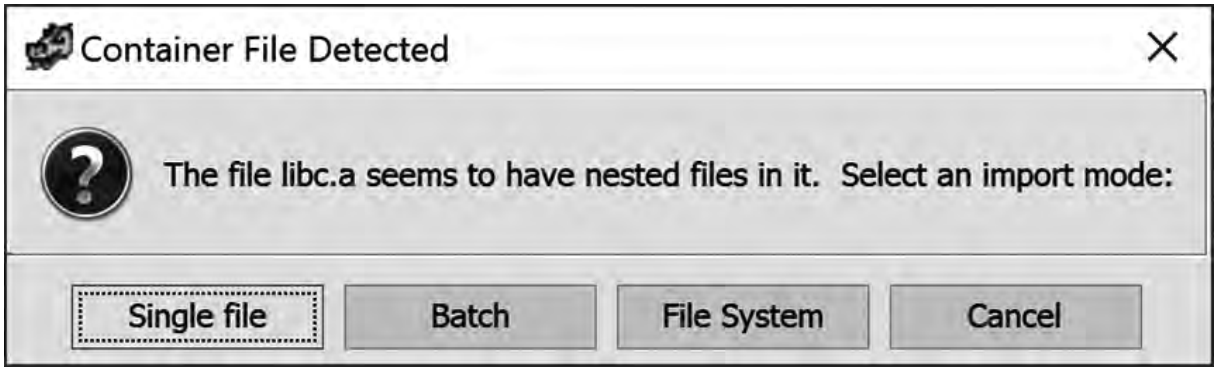


Рисунок 13-19. Импорт контейнерного файла

Режим Single File просит Ghidra импортировать контейнер так, будто это одиночный файл. Поскольку контейнер не является исполняемым файлом, Ghidra, вероятно, предложит формат `Raw Binary` для импорта и выполнит минимальный автоматический анализ. В режиме File System Ghidra открывает окно файлового браузера (см. рисунок 13-20), чтобы показать содержимое контейнерного файла. В этом режиме можно выбрать любую комбинацию файлов из контейнера для импорта в Ghidra, используя параметры контекстных меню.

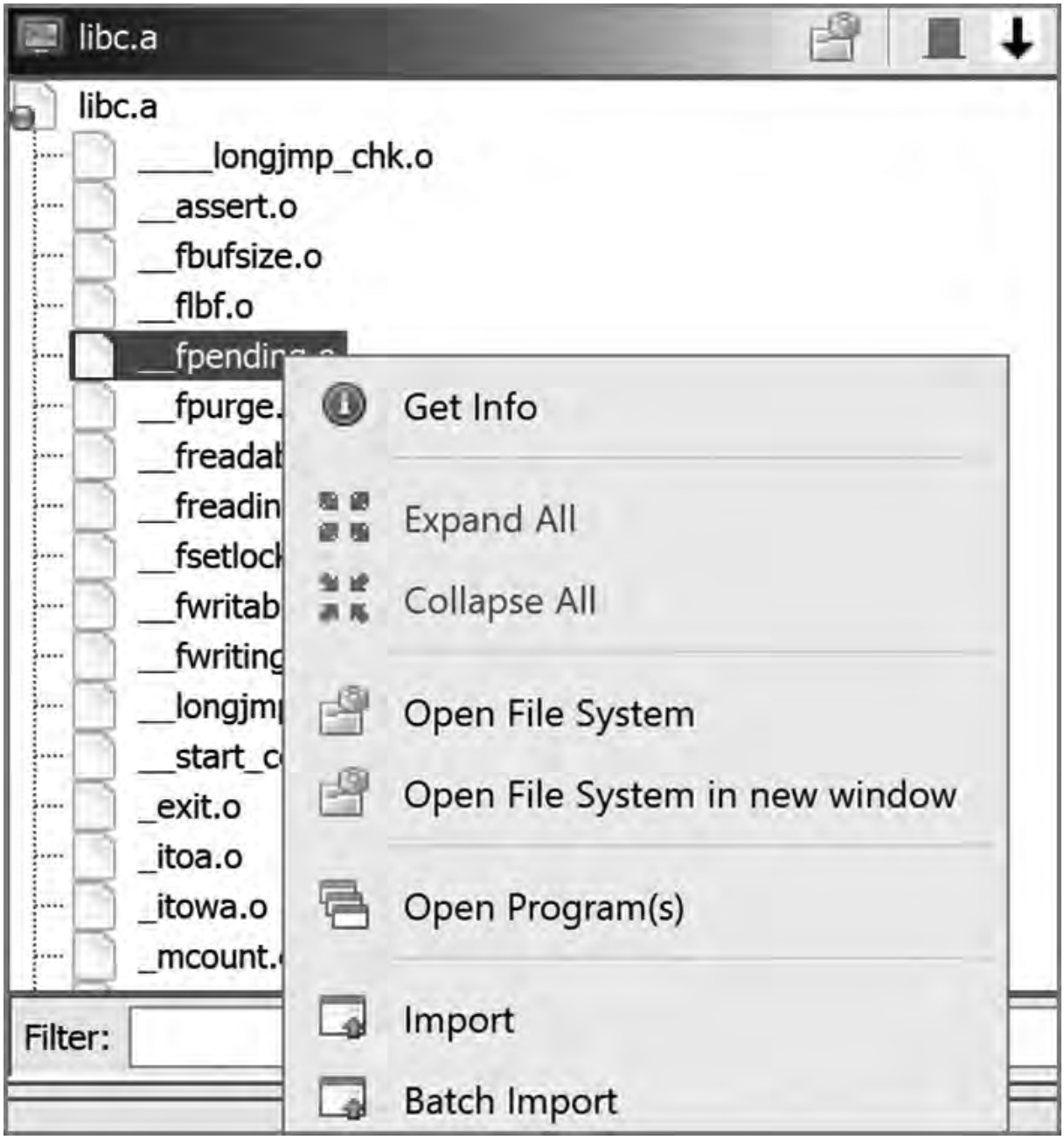


Рисунок 13-20. Режим импорта File System

В режиме Batch Ghidra автоматически импортирует файлы в контейнере, не останавливаясь для показа информации по каждому отдельному файлу. После начальной обработки содержимого контейнера Ghidra отображает диалог Batch Import, показанный на рисунке 13-21. Перед нажатием `OK` можно просмотреть информацию о каждом импортируемом файле, добавить дополнительные файлы в пакетный импорт, задать параметры импорта и выбрать папку назначения внутри проекта Ghidra. Рисунок 13-21 показывает, что мы собираемся импортировать 1690 файлов из архива `libc.a` в корневой каталог проекта `CH13`.

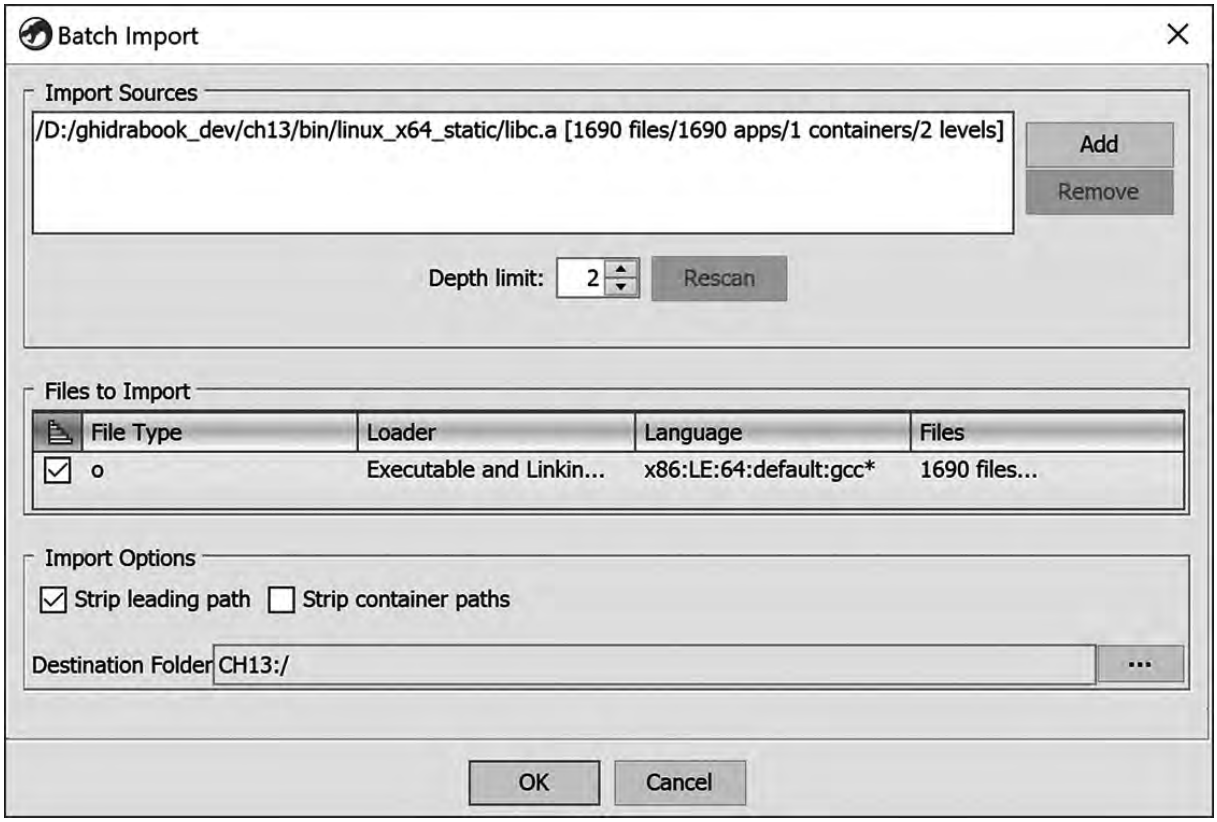


Рисунок 13-21. Диалог Batch Import в Ghidra

Нажмите `OK`, чтобы запустить процесс импорта, который может занять некоторое время. После завершения импорта вы сможете просматривать недавно импортированные файлы в окне `Ghidra Project`. Поскольку `libc.a` является контейнерным файлом, он появится как папка в окне `Project`, и можно перемещаться по ее содержимому, открывая и анализируя любой из файлов внутри папки.

На этом этапе мы наконец можем захватить отпечатки каждой функции `libc` в FidDb и использовать эту FidDb для анализа Function ID нашего примера статически слинкованного бинарного файла. Этот процесс повторяет пример UPX, начиная с создания новой пустой FidDb, которая затем будет заполнена из программ. Программами в этом случае будет все содержимое недавно импортированной папки `libc.a`. Здесь мы сталкиваемся с существенной проблемой.

Когда мы выбираем файлы для заполнения новой FidDb, нужно убедиться, что каждый файл был должным образом проанализирован Ghidra для идентификации функций и связанных с ними инструкций, то есть входных данных для процесса хеширования Function ID. До сих пор мы видели, что Ghidra анализирует программы только тогда, когда мы открываем их в `CodeBrowser`, но с `libc.a` перед нами стоит устрашающая задача анализа 1690 отдельных файлов внутри архива `libc.a`. Открывать и анализировать их по одному - плохое использование времени. Даже выбор открыть все файлы при импорте и использование `Analyze All Open` в Ghidra все равно потребуют времени на обработку всех 1690 файлов и, вероятно, ручного вмешательства для настройки параметров инструмента и распределения ресурсов, чтобы вместить задачу такого размера в экземпляр Ghidra.

Если эта проблема кажется громоздкой, вы правы. Это не та задача, которую следует решать вручную через Ghidra с графическим интерфейсом. Это четко определенная повторяющаяся задача, которая не должна требовать человеческого вмешательства. К счастью, следующие три главы вводят методы, которые можно использовать для автоматизации этой и других задач. Когда мы дойдем до раздела "Автоматизированное создание FidDb" на странице 359, мы вернемся к этой конкретной задаче и покажем, насколько легко пакетную обработку можно выполнить с помощью режима работы Ghidra без графического интерфейса.

Независимо от метода, которым мы обрабатываем `libc.a`, после завершения достаточно вернуться к плагину Function ID и заполнить новую FidDb, получив следующие результаты:

```
FidDb Populate Results
2905 total functions visited
2638 total functions added
267 total functions excluded
Breakdown of exclusions:    FAILS_MINIMUM_SHORTHASH_LENGTH: 234
    DUPLICATE_INFO: 9
    FAILED_FUNCTION_FILTER: 0
    IS_THUNK: 16
    NO_DEFINED_SYMBOL: 8
    MEMORY_ACCESS_EXCEPTION: 0
Most referenced functions by name:
749  __stack_chk_fail
431  free
304  malloc
...
```

Новая FidDb теперь доступна для использования и позволяет анализатору Function ID сопоставить многие функции, содержащиеся в `upx_demo2_x64_static_stripped`, значительно сокращая нашу нагрузку обратной разработки для этого конкретного бинарного файла.

Резюме

Эта глава показала несколько способов расширения Ghidra: разбор исходных файлов C, расширение моделей слов и извлечение отпечатков функций с помощью плагина Function ID. Когда бинарный файл содержит статически слинкованный код или код, повторно использованный из ранее проанализированных бинарных файлов, сопоставление этих функций с FidDb Ghidra может избавить от ручного продирания через гору кода. Предсказуемо, существует так много библиотек статической линковки, что Ghidra не может включить файлы FidDb, покрывающие каждый возможный вариант использования. Возможность создавать собственные файлы FidDb при необходимости позволяет собрать коллекцию FidDb, настроенную под ваши конкретные потребности. В главах 14 и 15 мы вводим мощные возможности сценариев Ghidra, чтобы еще сильнее расширить функциональность Ghidra.

Глава 14. Базовые скрипты Ghidra

Ни одно приложение не может удовлетворить каждую потребность каждого пользователя. Просто невозможно заранее предвидеть каждый потенциальный сценарий использования, который может возникнуть. Модель открытого исходного кода Ghidra облегчает запросы новых возможностей и инновационные вклады разработчиков. Однако иногда нужно немедленно решить текущую проблему и нельзя ждать, пока кто-то другой реализует новую функциональность. Чтобы поддержать непредвиденные сценарии использования и программное управление действиями Ghidra, Ghidra включает встроенные возможности сценариев.

Варианты использования сценариев бесконечны: от простых однострочников до полноценных программ, автоматизирующих типовые задачи или выполняющих сложный анализ. В этой главе мы сосредоточимся на базовых сценариях, предоставляемых через интерфейс `CodeBrowser`. Мы представим внутреннюю среду сценариев, обсудим разработку сценариев на Java и Python, а затем в главе 15 перейдем к другим встроенным вариантам сценариев.

Script Manager

`Ghidra Script Manager` доступен через меню `CodeBrowser`. Выбор `Window -> Script Manager` открывает окно, показанное на рисунке 14-1. Окно также можно открыть значком `Script Manager` на панели инструментов `CodeBrowser` - зеленым кругом со стрелкой внутри, который также показан в левом верхнем углу окна `Script Manager`.

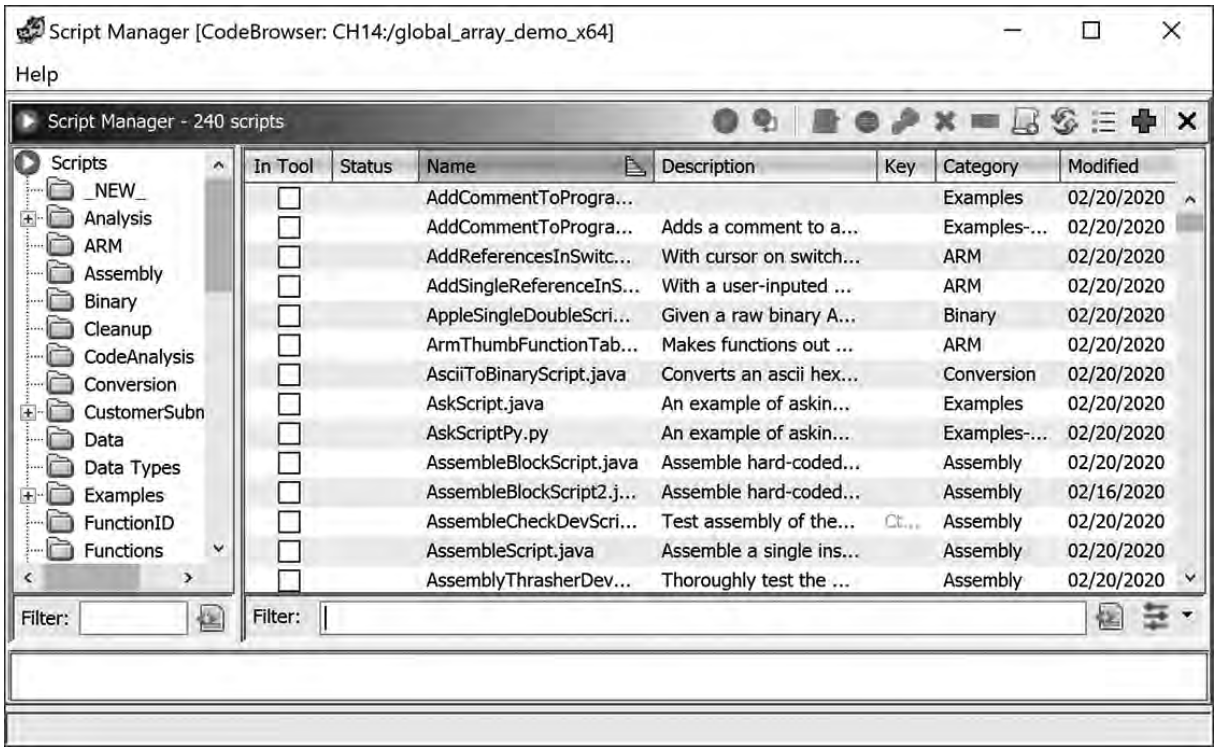


Рисунок 14-1. Окно `Script Manager`

Окно Script Manager

В новой установке `Ghidra Script Manager` загружается с более чем 240 сценариями, организованными в дерево категорий, как видно в левой части рисунка 14-1. Некоторые папки содержат подпапки для еще более детальной классификации сценариев. Можно раскрывать и сворачивать папки, чтобы видеть организацию сценариев. Выбор отдельной папки или подпапки ограничивает отображение сценариями внутри выбранной папки. Чтобы заполнить это окно, Ghidra находит и индексирует все сценарии в подкаталогах с именем `ghidra_scripts` внутри папки дистрибутива Ghidra. Ghidra также ищет каталог `ghidra_scripts` внутри вашего домашнего каталога и индексирует все найденные там сценарии.

Набор сценариев по умолчанию покрывает широкий диапазон функциональности. Некоторые сценарии предназначены для демонстрации фундаментальных концепций написания сценариев. Столбцы таблицы списка сценариев предоставляют дополнительные сведения о назначении каждого сценария. Как и в большинстве таблиц Ghidra, можно управлять тем, какие столбцы отображаются, а также порядком сортировки отдельных столбцов. По умолчанию отображаются все доступные поля таблицы, кроме Created и Path. Шесть информационных столбцов дают следующие сведения о сценарии:

Status. Указывает состояние сценария. Обычно поле пустое, но может содержать красный значок, указывающий на ошибку в сценарии. Если вы связали со сценарием значок панели инструментов, этот значок появится в этом столбце.

Name. Содержит имя файла сценария, включая расширение.

Description. Описание, извлеченное из комментария с метаданными внутри сценария. Это поле может быть довольно длинным, но все содержимое можно прочитать, наведя курсор на поле. Подробнее это поле обсуждается в разделе "Разработка сценариев" на странице 289.

Key. Указывает, назначена ли клавишная привязка для запуска сценария.

Category. Задает путь, по которому сценарий будет перечислен в тематической иерархии `Script Manager`. Это логическая иерархия, а не иерархия каталогов файловой системы.

Modified. Дата последнего сохранения сценария. Для сценариев по умолчанию это дата установки экземпляра Ghidra.

Поле фильтра в левой части окна выполняет поиск по категориям сценариев. Фильтр справа ищет по именам и описаниям сценариев. Наконец, внизу находится дополнительное окно, изначально пустое. Это окно отображает метаданные о выбранном сценарии в удобном для обработки формате, включающем поля, извлеченные из метаданных внутри сценария. Формат и смысл полей метаданных обсуждаются в разделе "Написание сценариев Java (не JavaScript!)" на странице 289.

Хотя `Script Manager` предоставляет значительный объем информации, главная сила этого окна находится в предоставляемой им панели инструментов. Обзор панели инструментов приведен на рисунке 14-2.

Панель инструментов Script Manager

В `Script Manager` нет меню, помогающих управлять сценариями. Вместо этого все действия управления сценариями связаны с инструментами на панели инструментов `Script Manager` (рисунок 14-2).

	Run script	Runs a selected script. The script will be recompiled if necessary, and will display the error icon in the status field and an error message in the console if the script does not compile.
	Rerun last script	Runs the last script run. This option is also available in the CodeBrowser window after a script is run.
	Edit script	Opens the selected script in a window for editing.
	Edit script with Eclipse	Opens a script in Eclipse for editing. See Chapter 15.
	Assign key binding	Allows assignment of a shortcut key combination for the script.
	Delete script	Permanently deletes a user-created script. System-provided scripts cannot be deleted using this method.
	Rename script	Allows you to rename a user-created script. System-provided scripts cannot be renamed using this method.
	Create new script	Opens an empty script template in a basic editor window.
	Refresh script list	Rescans the script directories and regenerates the list of scripts.
	Script directories	Provides a list of script directories that you can select/deselect. You also have the option to add new directories to the list.
	Help	Opens the Javadoc for the `GhidraScript` class. The first time this is selected, Ghidra automatically builds the Javadoc content.

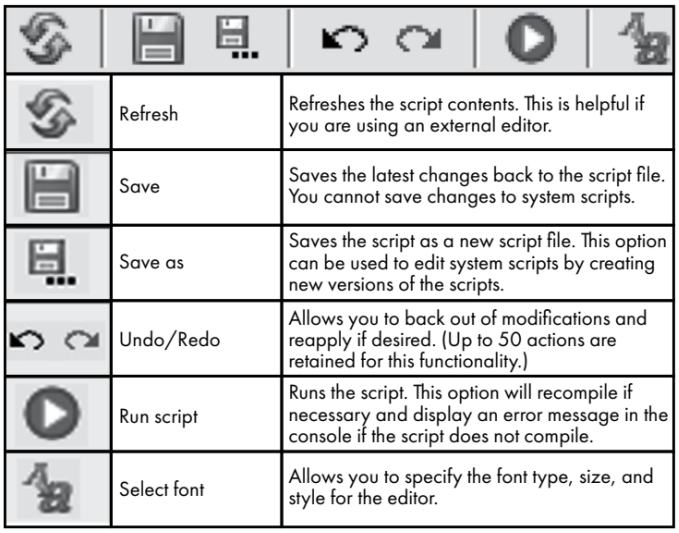
Figure 14-2: Script Manager toolbar

Рисунок 14-2. Панель инструментов `Script Manager`

Панель инструментов `Script Manager` включает запуск выбранного сценария, повторный запуск последнего сценария, редактирование сценария, редактирование сценария в Eclipse, назначение клавишной привязки, переименование созданного пользователем сценария, удаление созданного пользователем сценария, создание нового сценария, обновление списка сценариев, управление каталогами сценариев и открытие справки/Javadoc для класса `GhidraScript`.

Хотя большинство пунктов меню достаточно ясны из описаний на рисунке 14-2, параметры Edit заслуживают дополнительного обсуждения. Редактирование в Eclipse рассматривается в главе 15, поскольку оно облегчает более продвинутые возможности сценариев. Параметр Edit Script открывает примитивное окно текстового редактора со своей панелью инструментов, показанной на рисунке 14-3. Связанные действия предоставляют базовую функциональность для редактирования файлов. Получив редактор, можно перейти к написанию настоящих сценариев.

Figure 14-2: Script Manager toolbar



Refresh	Refreshes the script contents. This is helpful if you are using an external editor.
Save	Saves the latest changes back to the script file. You cannot save changes to system scripts.
Save as	Saves the script as a new script file. This option can be used to edit system scripts by creating new versions of the scripts.
Undo/Redo	Allows you to back out of modifications and reapply if desired. (Up to 50 actions are retained for this functionality.)
Run script	Runs the script. This option will recompile if necessary and display an error message in the console if the script does not compile.
Select font	Allows you to specify the font type, size, and style for the editor.

Figure 14-3: Edit Script toolbar

Рисунок 14-3. Панель инструментов Edit Script

Панель инструментов Edit Script предоставляет обновление содержимого сценария, сохранение изменений обратно в файл сценария, сохранение как новый файл сценария, отмену и повтор действий, запуск сценария и выбор гарнитуры, размера и стиля шрифта для редактора. Обновление полезно, если вы используете внешний редактор. Системные сценарии нельзя изменять напрямую через `Save`, но `Save As` позволяет создать новую версию такого сценария.

Разработка сценариев

В Ghidra существует несколько методов разработки сценариев. В этой главе мы сосредоточимся на написании сценариев с использованием Java и Python, поскольку именно эти языки используются существующими сценариями в окне `Script Manager`. Большинство из более чем 240 системных сценариев написано на Java, поэтому начнем с редактирования и разработки сценариев на Java.

Написание сценариев Java (не JavaScript!)

В Ghidra сценарий, написанный на Java, фактически является полной спецификацией класса, предназначенного для незаметной компиляции, динамической загрузки в ваш работающий экземпляр Ghidra, вызова и, наконец, выгрузки. Класс должен расширять класс `Ghidra.app.script.GhidraScript`, реализовывать метод `run()` и быть аннотирован комментариями, предоставляющими метаданные сценария в формате Javadoc. Мы покажем структуру файла сценария, опишем требования к метаданным, посмотрим на некоторые системные сценарии, а затем перейдем к редактированию существующих сценариев и созданию собственных.

На рисунке 14-4 показан редактор сценариев, открытый при выборе Create New Script (см. рисунок 14-2) для создания нового сценария Java. Мы назвали новый сценарий `CH14_NewScript`.

Рисунок 14-4. Новый пустой сценарий

В верхней части файла находятся комментарии и теги метаданных, используемые для создания ожидаемой информации Javadoc. Эта информация также используется для заполнения полей в окне `Script Manager` (см. рисунок 14-1). Любые комментарии, начинающиеся с `//` перед объявлениями класса, поля или метода, станут частью Javadoc Description для сценария. Дополнительные комментарии можно размещать внутри сценария, и они не будут включены в описание. Кроме того, внутри комментариев метаданных поддерживаются следующие теги:

@author. Предоставляет информацию об авторе сценария. Информация указывается по усмотрению автора и может включать любые релевантные сведения, например имя, контактную

информацию, дату создания и т. д.

@category. Определяет, где сценарий появляется в дереве категорий. Это единственный обязательный тег, и он должен присутствовать во всех сценариях Ghidra. Символ точки действует как разделитель пути для имен категорий, например `@category Ghidrabook.CH14`.

@keybinding. Документирует сочетание клавиш для доступа к сценарию из окна `CodeBrowser`, например `@keybinding K`.

@menupath. Определяет путь меню для сценария, разделенный точками, а также предоставляет способ запускать сценарий из меню `CodeBrowser`, например `@menupath File.Run.ThisScript`.

@toolbar. Связывает значок со сценарием. Этот значок отображается как кнопка панели инструментов в окне `CodeBrowser` и может использоваться для запуска сценария. Если Ghidra не может найти изображение в каталоге сценария или установке Ghidra, будет использовано изображение по умолчанию, например `@toolbar myImage.png`.

Когда вы сталкиваетесь с новым API, таким как Ghidra API, может пройти некоторое время, прежде чем вам станет комфортно писать сценарии без постоянного обращения к доступной документации API. Java особенно чувствительна к проблемам пути классов и правильному включению необходимых вспомогательных пакетов. Вариант, экономящий время и здравый смысл, - редактировать существующую программу, а не создавать новую. Мы используем этот подход при представлении простого примера сценария.

Пример редактирования сценария: поиск по регулярному выражению

Предположим, вам поручили разработать сценарий, который принимает регулярное выражение от пользователя и выводит совпадающие строки в консоль. Кроме того, этот сценарий должен появляться в `Script Manager` для конкретного проекта. Хотя Ghidra предлагает множество способов выполнить эту задачу, вас попросили создать сценарий. Чтобы найти сценарий с похожей функциональностью в качестве основы, вы просматриваете категории в `Script Manager`, проверяете содержимое категорий Strings и Search, затем фильтруете по термину `strings` и находите другие варианты. Использование фильтров дает более полный список сценариев, связанных со строками, для рассмотрения. В этом примере вы отредактируете первый сценарий в списке, который разделяет часть функциональности с тем, что должен делать ваш сценарий, - `CountAndSaveStrings.java`.

Откройте сценарий в редакторе, чтобы подтвердить, что он является хорошей отправной точкой для новой функциональности: щелкните нужный сценарий правой кнопкой мыши и выберите `Edit with basic editor`; затем сохраните этот сценарий под новым именем `FindStringsByRegex.java` с помощью `Save As`. Ghidra не позволяет редактировать системные сценарии, поставляемые как часть установки Ghidra, внутри окна `Script Manager`, хотя это можно делать в Eclipse и других редакторах. Вы также могли бы отредактировать файл до использования `Save As`, поскольку Ghidra не даст случайно записать измененное содержимое в существующий сценарий `CountAndSaveStrings.java`.

Исходный `CountAndSaveStrings.java` содержит следующие метаданные:

```
/* ###
 * IP: GHIDRA
 *
 * Лицензировано по Apache License версии 2.0 ("Лицензия");
 * использовать этот файл можно только при соблюдении условий Лицензии.
 * Копию Лицензии можно получить по адресу
 * http://www.apache.org/licenses/LICENSE-2.0
 * Если применимое законодательство или письменное соглашение не требует иного,
 * программное обеспечение по этой Лицензии распространяется "КАК ЕСТЬ",
 * БЕЗ КАКИХ-ЛИБО ГАРАНТИЙ ИЛИ УСЛОВИЙ, явных или подразумеваемых.
 * Конкретные разрешения и ограничения см. в тексте Лицензии.
 */
//Подсчитывает количество определенных строк в текущем выделении,
//или в текущей программе, если выделения нет,
//и сохраняет результаты в файл.
//@category CustomerSubmission.Strings
```

Мы можем оставить, изменить или удалить лицензионное соглашение для сценария без влияния на выполнение сценария или связанную Javadoc. Мы изменим описание сценария, чтобы информация, отображаемая в Javadoc и `Script Manager`, точно описывала сценарий. Автор сценария включил только один из пяти доступных тегов, поэтому мы добавим заполнители для незаполненных тегов и пересмотрим описание следующим образом:

```
// Подсчитывает количество определенных строк, соответствующих регулярному
// выражению в текущем выделении или в текущей программе, если выделения нет,
// и выводит количество совпадающих строк в консоль.
//
//@author Ghidrabook
//@category Ghidrabook.CH14
//@keybinding
//@menupath
//@toolbar
```

Тег категории `Ghidrabook.CH14` будет добавлен в древовидное отображение `Script Manager`, как показано на рисунке 14-5.

Следующая часть исходного сценария содержит инструкции импорта Java. Из длинного списка импортов, который Ghidra включает при создании нового сценария, как показано на рисунке 14-4, для поиска строк необходимы только следующие импорты, поэтому мы сохраним тот же список, что и в исходном `CountAndSaveStrings.java`:

```
import ghidra.app.script.GhidraScript;
import ghidra.program.model.listing.*;
import ghidra.program.util.ProgramSelection;
import java.io.*;
```

Сохраните новый сценарий, затем выберите его в `Script Manager`, чтобы увидеть содержимое, показанное на рисунке 14-5. Наша новая категория включена в дерево сценариев, а метаданные сценария отображаются в информационном окне и таблице сценариев. Таблица содержит только один сценарий, `Ghidrabook.CH14`, поскольку это единственный сценарий в выбранной категории.

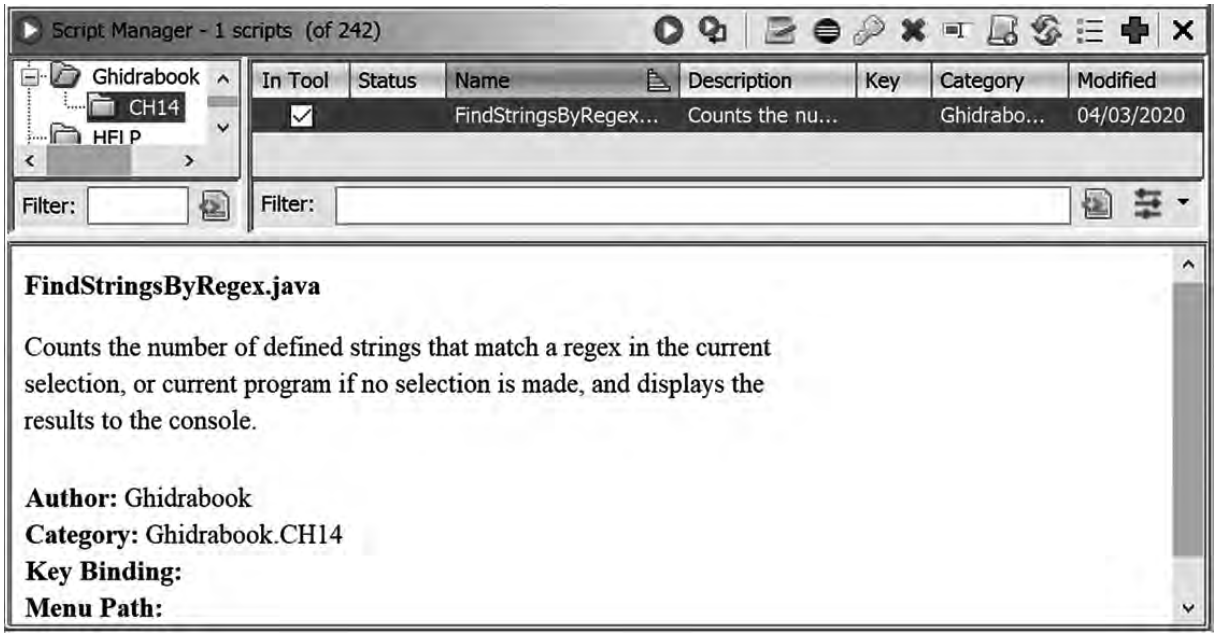


Рисунок 14-5. Информация о новом сценарии в окне `Script Manager`

Поскольку эта книга не является учебником по Java, мы суммируем изменения, внесенные в сценарий, а не объясняем синтаксис и функциональность Java. Следующий список описывает поведение `CountAndSaveStrings.java`:

1. Получить содержимое листинга программы для поиска.
2. Получить файл, в который сохранять результаты.
3. Открыть файл.
4. Итерировать по листингу программы: подсчитать количество подходящих строк и записать каждую подходящую строку в файл.
5. Закрыть файл.
6. Записать количество подходящих строк в консоль.

Желаемая функциональность нашего измененного сценария описана ниже:

1. Получить содержимое листинга программы для поиска.
2. Запросить у пользователя регулярное выражение (regex) для поиска.
3. Итерировать по листингу программы: подсчитать количество подходящих строк и записать каждую подходящую строку в консоль.
4. Записать количество подходящих строк в консоль.

Наш новый сценарий будет значительно короче исходного, поскольку нет необходимости взаимодействовать с файловой системой и выполнять связанную проверку ошибок. Наша реализация выглядит так:

```
public class FindStringsByRegex extends GhidraScript {
   @Override
   public void run() throws Exception {
      String regex =
         askString("Введите регулярное выражение",
         "Введите регулярное выражение, совпадения с которым нужно найти:");
      Listing listing = currentProgram.getListing();
      DataIterator dataIt;
      if (currentSelection != null) {
          dataIt = listing.getDefinedData(currentSelection, true);
      }
      else {
         dataIt = listing.getDefinedData(true);
      }
      Data data;
```

```
        String type;
        int counter = 0;
        while (dataIt.hasNext() && !monitor.isCancelled()) {
           data = dataIt.next();
           type = data.getDataType().getName().toLowerCase();
           if (type.contains("unicode") || type.contains("string")) {
              String s = data.getDefaultValueRepresentation();
              if (s.matches(regex)) {
                 counter++;
                 println(s);
              }
           }
        }
        println("Найдено совпадающих строк: " + counter);
     }
  }
```

Все сценарии Java, которые вы пишете для Ghidra, должны расширять (наследоваться от) существующего класса `Ghidra.app.script.GhidraScript`. После сохранения финальной версии сценария выберите его в `Script Manager` и выполните. При выполнении сценария мы видим приглашение, показанное на рисунке 14-6. Этот рисунок включает регулярное выражение, которое мы будем искать для проверки сценария.

Рисунок 14-6. Приглашение нового сценария для ввода regex

Консоль `CodeBrowser` отображает следующее содержимое после завершения выполнения нашего нового сценария:

```
FindStringsByRegex.java> Running...
FindStringsByRegex.java> "Fatal error: glibc detected an invalid stdio handle\n"
FindStringsByRegex.java> "Unknown error "
FindStringsByRegex.java> "internal error"
FindStringsByRegex.java> "relocation error"
FindStringsByRegex.java> "symbol lookup error"
FindStringsByRegex.java> "Fatal error: length accounting in
    _dl_exception_create_format\n"
FindStringsByRegex.java> "Fatal error: invalid format in exception string\n"
FindStringsByRegex.java> "error while loading shared libraries"
FindStringsByRegex.java> "Unknown error"
FindStringsByRegex.java> "version lookup error"
FindStringsByRegex.java> "sdlerror.o"
FindStringsByRegex.java> "dl-error.o"
FindStringsByRegex.java> "fatal_error"
FindStringsByRegex.java> "strerror.o"
FindStringsByRegex.java> "strerror"
FindStringsByRegex.java> "__strerror_r"
FindStringsByRegex.java> "_dl_signal_error"
FindStringsByRegex.java> "__dlerror"
FindStringsByRegex.java> "_dlerror_run"
FindStringsByRegex.java> "_dl_catch_error"
FindStringsByRegex.java> Найдено совпадающих строк: 20
FindStringsByRegex.java> Finished!
```

Этот простой пример демонстрирует низкий порог входа в обширные возможности сценариев Java в Ghidra. Существующие сценарии можно легко изменять, а новые сценарии можно строить с нуля с помощью `Script Manager`. Более сложные возможности сценариев Java мы представим в главах 15 и 16, но Java - лишь один из вариантов сценариев, предоставляемых Ghidra. Ghidra также позволяет создавать сценарии на Python.

Сценарии Python

Из более чем 240 сценариев в `Script Manager` лишь небольшая часть написана на Python. Сценарии Python легко найти, отфильтровав по расширению `.py` в `Script Manager`. Большинство сценариев Python можно найти в категории `Examples.Python` в дереве; они включают отказ от ответственности, похожий на показанный на рисунке 14-7.

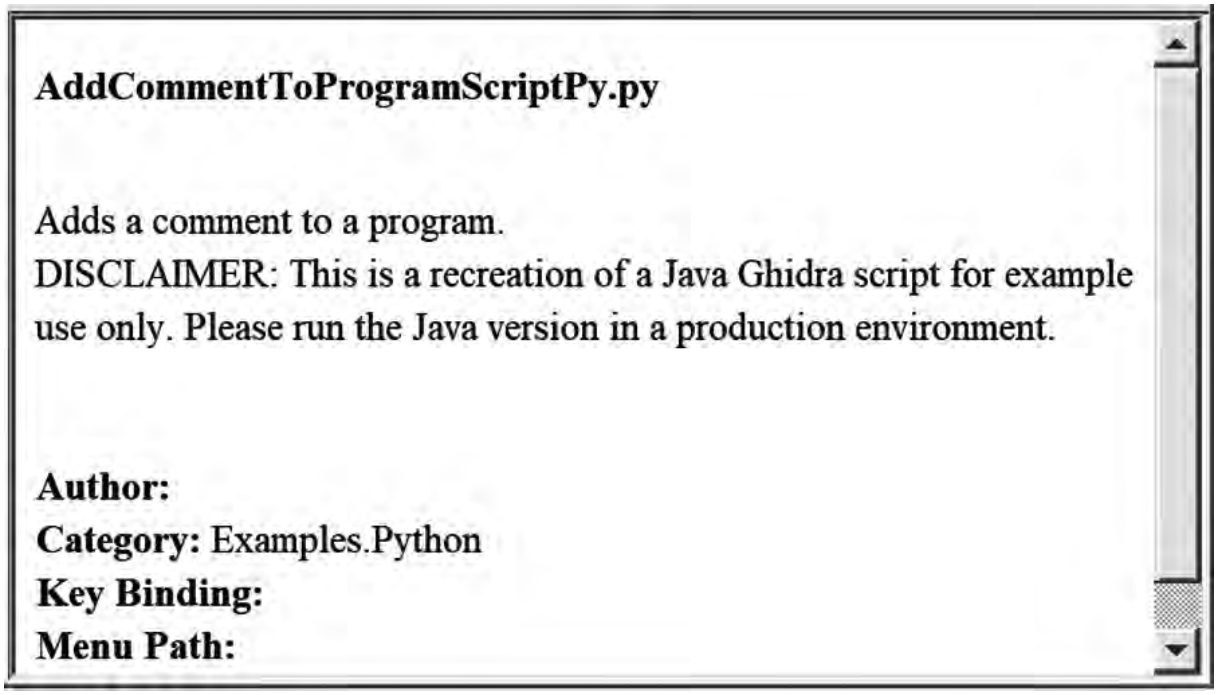


Рисунок 14-7. Пример сценария Python с отказом от ответственности

Из примеров в этом каталоге следующие три являются хорошей отправной точкой, если вы предпочитаете использовать Python:

ghidra_basic.py. Этот сценарий включает примеры базовых сценариев Python, связанных с Ghidra.

python_basics.py. Это очень базовое введение во многие команды Python, которые вы можете захотеть использовать.

jython_basic.py. Этот сценарий расширяет базовые команды Python, демонстрируя содержимое, специфичное для Jython.

Возможности Ghidra, продемонстрированные в этих примерах, едва затрагивают поверхность доступных API Ghidra. Скорее всего, вам все равно придется потратить некоторое время на чтение библиотеки примеров Java Ghidra, прежде чем вы будете готовы обращаться к полному Java API Ghidra из своих сценариев Python.

Помимо запуска сценариев Python, Ghidra предоставляет `Python Interpreter`, позволяющий использовать Python/Jython для прямого доступа к объектам Java, связанным с Ghidra, как показано на рисунке 14-8.

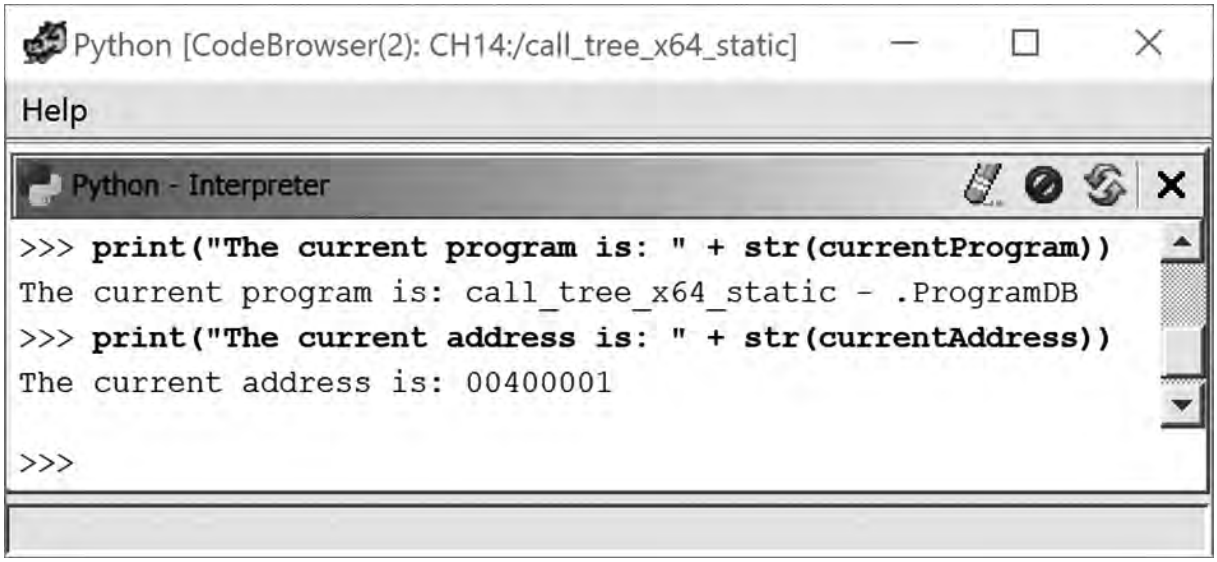


Рисунок 14-8. Пример print в `Python Interpreter`

`Python Interpreter` доступен через `CodeBrowser` выбором `Windows -> Python`. Дополнительную информацию об использовании интерпретатора см. в `Ghidra Help`. Чтобы получить информацию API при использовании Python и `Python Interpreter`, выберите `Help -> Ghidra API Help` в левом верхнем углу окна Interpreter, показанного на рисунке 14-8; это откроет содержимое Javadoc для класса `GhidraScript`. В качестве альтернативы Python имеет встроенную функцию `help()`, которая в Ghidra была изменена для прямого доступа к Javadoc Ghidra. Чтобы использовать функцию, введите `help(object)` в интерпретаторе, как показано на рисунке 14-9. Например, `help(currentProgram)` отображает содержимое Javadoc Ghidra, описывающее класс Ghidra API `ProgramDB`.

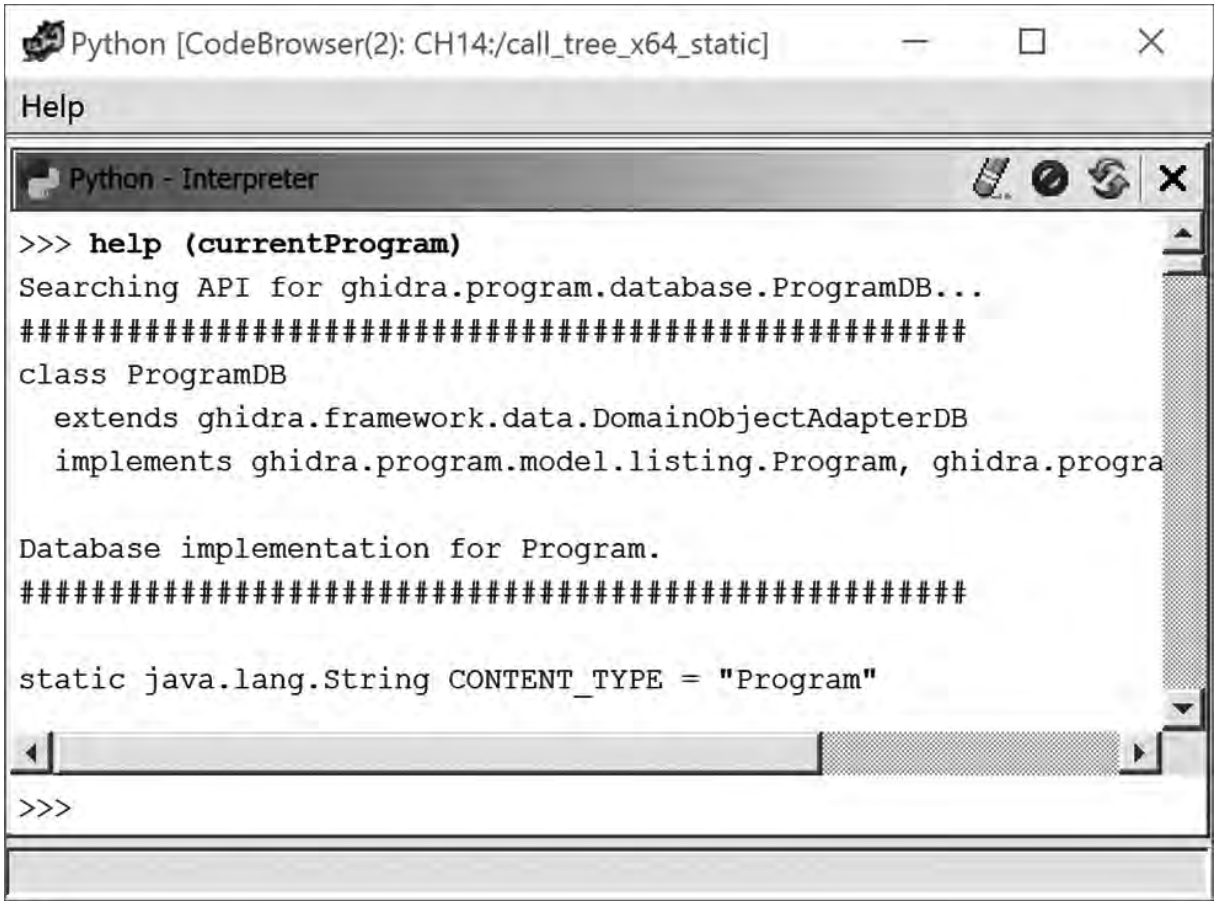


Рисунок 14-9. Пример `Help` *в* `Python Interpreter`

Будущее Python в Ghidra

Python популярен для создания сценариев благодаря простоте и многочисленным доступным библиотекам. Хотя большинство сценариев в релизе Ghidra написано на Java, сообщество обратной разработки вокруг открытого исходного кода, вероятно, будет использовать Python как основной язык сценариев внутри Ghidra. Ghidra полагается на Jython для поддержки Python, что дает преимущество прямого доступа к объектам Java Ghidra. Jython совместим с Python 2, конкретно 2.7.1, но не с Python 3. Хотя жизненный цикл Python 2 завершился в январе 2020 года, сценарии Python 2 продолжат работать внутри Ghidra, а любые новые сценарии Ghidra на Python 2 следует писать так, чтобы они были как можно более переносимыми на Python 3.

Поддержка других языков

Наконец, Ghidra может поддерживать сценарии на языках, отличных от Java и Python, что позволяет переносить существующие сценарии из вашего набора инструментов обратной разработки в рабочий процесс Ghidra. Эта функциональность подробнее обсуждается в `Ghidra Help`.

Введение в Ghidra API

К этому моменту у вас есть вся информация, необходимая для редактирования и запуска сценариев Ghidra. Теперь пора использовать Ghidra API, чтобы расширить возможности сценариев и напрямую взаимодействовать с артефактами Ghidra. Ghidra предоставляет API в двух довольно разных стилях.

Program API определяет объектную иерархию глубиной во много уровней, на вершине которой находится класс `Program`. Этот API может меняться от одной версии Ghidra к другой. Flat API "уплощает" Program API, раскрывая все уровни этого API из одного класса, `FlatProgramAPI`. Flat API часто является самым удобным способом доступа ко многим конструкциям Ghidra. Кроме того, он менее склонен меняться от одной версии Ghidra к следующей.

В оставшейся части главы мы выделим некоторые наиболее полезные возможности Flat API. Когда потребуется, также предоставим сведения о конкретных классах из Program API. Для обсуждения мы используем Java, поскольку это родной язык Ghidra.

Ghidra API содержит множество пакетов, классов и связанных функций для взаимодействия с вашими проектами Ghidra и связанными файлами. Все они подробно описаны в документации в стиле Javadoc, поставляемой с Ghidra, к которой можно получить доступ, щелкнув красный плюс в окне `Script Manager`. Эта документация вместе с примерными сценариями, поставляемыми с Ghidra, является вашим основным справочником по API и их использованию. Самый распространенный способ понять, как что-то сделать, - просматривать классы Ghidra в поиске класса, который, судя по имени, делает то, что нужно. По мере накопления опыта работы с Ghidra ваше понимание соглашений об именовании и организации файлов поможет быстрее находить подходящие классы.

Ghidra следует архитектуре Java Swing "модель-делегат", в которой значения и характеристики данных хранятся в объектах модели и отображаются объектами-делегатами пользовательского интерфейса, такими как представления дерева, списка и таблицы. Делегаты обрабатывают события, например щелчки мышью, чтобы обновлять данные и представления. В подавляющем большинстве случаев ваши сценарии будут сосредоточены на данных, инкапсулированных в классах модели, используемых для представления различных программных конструкций и конструкций обратной разработки.

Оставшаяся часть этого раздела сосредоточена на часто используемых классах модели, их отношениях друг с другом и полезных API для взаимодействия с ними. Мы не пытаемся покрыть весь Ghidra API, и доступно гораздо больше функций и классов. Авторитетная документация для всего Ghidra API - это Javadoc, поставляемый с Ghidra, и в конечном счете исходный код Java, из которого собрана Ghidra.

Интерфейс Address

Интерфейс `Address` описывает модель адреса внутри адресного пространства. Все адреса представлены смещением размером до 64 бит. Сегментированные адреса могут дополнительно уточняться значением сегмента. Во многих случаях смещение адреса эквивалентно виртуальному адресу внутри листинга программы. Метод `getOffset` извлекает значение смещения `long` из экземпляра `Address`. Многие функции Ghidra API требуют объекты `Address` как аргументы или возвращают объект `Address` как результат.

Интерфейс Symbol

Интерфейс `Symbol` определяет свойства, общие для всех символов. Как минимум символ состоит из имени и адреса. Эти атрибуты можно получить следующими функциями-членами:

```
Address getAddress()
```

Возвращает адрес `Symbol`.

```
String getName()
```

Возвращает имя `Symbol`.

Интерфейс Reference

`Reference` моделирует отношение перекрестной ссылки (как описано в главе 9) между исходным адресом и адресом назначения и характеризуется типом ссылки. Полезные функции, связанные с `Reference`, включают следующие:

```
public Address getFromAddress()
```

Возвращает исходный адрес для этой ссылки.

```
public Address getToAddress()
```

Возвращает адрес назначения для этой ссылки.

```
public RefType getReferenceType()
```

Возвращает объект `RefType`, описывающий природу связи между исходным адресом и адресом назначения.

Класс GhidraScript

Хотя этот класс не моделирует конкретный атрибут в бинарном файле, каждый сценарий, который вы пишете, должен быть подклассом `GhidraScript`, который, в свою очередь, является подклассом `FlatProgramAPI`. В результате ваши сценарии мгновенно получают доступ ко всему Flat API, и ваша единственная обязанность - предоставить реализацию:

```
protected abstract void run() throws Exception;
```

которая, будем надеяться, заставит сценарий делать что-то интересное. Остальная часть класса `GhidraScript` дает доступ к наиболее распространенным ресурсам для взаимодействия с пользователем Ghidra и анализируемой программой. Некоторые наиболее полезные функции и элементы данных этого класса, включая часть унаследованных от `FlatProgramAPI`, суммированы в следующих разделах.

Полезные элементы данных

Класс `GhidraScript` предоставляет удобный доступ к ряду объектов, часто используемых в сценариях:

```
protected Program currentProgram;
```

Это текущая открытая программа. Класс `Program` обсуждается позднее. Этот элемент данных, вероятно, является вашим входом к извлечению более интересной информации, например списков

инструкций и символов.

```
protected Address currentAddress;
```

Это адрес текущего положения курсора. Класс `Address` обсуждается позднее.

```
protected ProgramLocation currentLocation;
```

Объект `ProgramLocation`, описывающий текущее положение курсора, включая адрес, строку курсора, столбец и другую информацию.

```
protected ProgramSelection currentSelection;
```

Объект `ProgramSelection`, представляющий диапазон адресов, выбранный в Ghidra с графическим интерфейсом.

```
protected TaskMonitor monitor;
```

Класс `TaskMonitor` обновляет состояние длительных задач и проверяет, не отменил ли пользователь длительную задачу (`monitor.isCancelled()`). Любые длительные циклы, которые вы пишете, должны включать вызов `monitor.isCancelled` как дополнительное условие завершения, чтобы распознать попытку пользователя отменить сценарий.

Функции пользовательского интерфейса

Класс `GhidraScript` предоставляет удобные функции для базовых операций пользовательского интерфейса, от простого вывода сообщений до более интерактивных элементов диалогов. Некоторые из наиболее распространенных функций пользовательского интерфейса описаны ниже:

```
public void println(String message)
```

Печатает `message` с последующим переводом строки в окно консоли Ghidra. Эта функция полезна для вывода сообщений состояния или результатов сценариев ненавязчивым способом.

```
public void printf(String message, Object... args)
```

Использует `message` как строку формата Java и печатает результирующую строку отформатированных `args` в окно консоли Ghidra.

```
public void popup(final String message)
```

Отображает `message` во всплывающем диалоге, требующем от пользователя нажать `OK`, прежде чем выполнение сценария продолжится. Это более навязчивый способ показывать пользователю сообщения состояния.

```
public String askString(String title, String message)
```

Одна из множества доступных функций запроса. `askString` отображает диалог ввода текста, используя `message` как приглашение, и возвращает текст, введенный пользователем.

```
public boolean askYesNo(String title, String question)
```

Использует диалог, чтобы задать пользователю вопрос с ответом да или нет. Возвращает `true` для "да" и `false` для "нет".

```
public Address askAddress(String title, String message)
```

Отображает диалог, используя `message` как приглашение, и разбирает ввод пользователя в объект `Address`.

```
public int askInt(String title, String message)
```

Отображает диалог, используя `message` как приглашение, и разбирает ввод пользователя в `int`.

```
public File askFile(final String title, final String approveButtonText)
```

Отображает системный диалог выбора файла и возвращает объект Java `File`, представляющий выбранный пользователем файл.

```
public File askDirectory(final String title, final String approveButtonText)
```

Отображает системный диалог выбора файла и возвращает объект Java `File`, представляющий выбранный пользователем каталог.

```
public boolean goTo(Address address)
```

Перемещает все подключенные окна дизассемблирования Ghidra к `address`. Перегруженные версии этой функции принимают аргумент `Symbol` или `Function` и соответствующим образом перемещают отображения.

Функции, связанные с адресами

Для процессора адрес обычно является просто числом, которое указывает на место в памяти. Ghidra моделирует адреса с помощью класса `Address`. `GhidraScript` предоставляет функцию-обертку, которая легко преобразует числа в объекты Ghidra `Address`:

```
public Address toAddr(long offset)
```

Удобная функция для создания объекта `Address` в адресном пространстве по умолчанию.

Чтение памяти программы

Класс `Memory` представляет непрерывные диапазоны байтовых значений, например содержимое исполняемого файла, загруженного в Ghidra. Внутри объекта `Memory` каждое байтовое значение связано с адресом, хотя адреса могут быть помечены как неинициализированные и не иметь значения для извлечения. Ghidra выбрасывает `MemoryAccessException`, если вы пытаетесь обратиться к месту внутри объекта памяти с недопустимым адресом. Полное описание доступных функций API см. в документации класса `Memory`. Следующие удобные функции раскрывают часть класса `Memory` через Flat API:

```
public byte getByte(Address addr)
```

Возвращает одиночное байтовое значение, извлеченное из `addr`. Тип данных `byte` в Java является знаковым типом, поэтому это значение будет в диапазоне -128..127.

```
public byte[] getBytes(Address addr, int length)
```

Возвращает `length` байтов из памяти, начиная с `addr`.

```
public int getInt(Address addr)
```

Возвращает 4-байтовое значение, начиная с `addr`, как Java `int`. Эта функция учитывает порядок байтов и базовую архитектуру бинарного файла при восстановлении значения `int`.

```
public long getLong(Address addr)
```

Возвращает 8-байтовое значение, начиная с `addr`, как Java `long`. Эта функция учитывает порядок байтов и базовую архитектуру бинарного файла при восстановлении значения `long`.

Функции поиска в программе

Возможности поиска Ghidra находятся в разных классах Program API в зависимости от типа искомого объекта. Класс `Memory` содержит функциональность поиска сырых байтов. Единицы кода, такие как `Data` и `Instruction`, текст комментариев и связанные итераторы получаются из класса `Listing`. Символы/метки и связанные итераторы доступны через класс `SymbolTable`. Следующие удобные функции раскрывают часть доступной функциональности поиска через Flat API:

```
public Data getFirstData()
```

Возвращает первый элемент данных в программе.

```
public Data getDataAfter(Data data)
```

Возвращает следующий элемент данных после `data` или `null`, если такого элемента данных нет.

```
public Data getDataAt(Address address)
```

Возвращает элемент данных по `address` или `null`, если такого элемента данных нет.

```
public Instruction getFirstInstruction()
```

Возвращает первую инструкцию в программе.

```
public Instruction getInstructionAfter(Instruction instruction)
```

Возвращает следующий элемент инструкции после `instruction` или `null`, если такой инструкции нет.

```
public Instruction getInstructionAt(Address address)
```

Возвращает инструкцию по `address` или `null`, если такой инструкции нет.

```
public Address find(String text)
```

Ищет текстовую строку внутри окна `Listing`. Компоненты `Listing` просматриваются в следующем порядке:

1. Комментарии-заголовки
2. Предваряющие комментарии
3. Метки
4. Мнемоники и операнды единиц кода
5. Комментарии конца строки
6. Повторяемые комментарии
7. Завершающие комментарии

Успешный поиск возвращает адрес, содержащий совпадение. Обратите внимание, что из-за порядка поиска возвращенный адрес может не представлять первое вхождение `text` в листинге дизассемблирования, если рассматривать листинг в строго возрастающем порядке адресов.

```
public Address find(Address start, byte[] values);
```

Ищет в памяти, начиная с `addr`, заданную последовательность байтовых значений. Когда `addr` равен `null`, поиск начинается с самого низкого допустимого адреса в бинарном файле. Успешный поиск возвращает адрес первого байта в совпадающей последовательности.

```
public Address findBytes(Address start, String byteString)
```

Ищет в памяти, начиная с `addr`, заданный `byteString`, который может содержать регулярные выражения. Когда `addr` равен `null`, поиск начинается с самого низкого допустимого адреса в бинарном файле. Успешный поиск возвращает адрес первого байта в совпадающей последовательности.

Манипулирование метками и символами

Необходимость манипулировать именованными местоположениями довольно часто возникает в сценариях. Для работы с именованными местоположениями в базе данных Ghidra доступны следующие функции:

```
public Symbol getSymbolAt(Address address)
```

Возвращает `Symbol`, связанный с указанным адресом, или `null`, если местоположение не имеет `Symbol`.

```
public Symbol createLabel(Address address, String name, boolean makePrimary)
```

Назначает указанное `name` указанному `address`. Ghidra позволяет назначать несколько имен одному адресу. Если `makePrimary` равно `true`, новое имя станет основным именем, связанным с `address`.

```
public List<Symbol> getSymbols(String name, Namespace namespace)
```

Возвращает список всех символов с именем `name` в `namespace`. Когда `namespace` равен `null`, ищется глобальное пространство имен. Если результат пуст, именованный символ не существует. Если результат содержит только один элемент, имя уникально.

Работа с функциями

Многие сценарии предназначены для анализа функций внутри программы. Следующие функции можно использовать для доступа к информации о функциях программы:

```
public final Function getFirstFunction()
```

Возвращает первый объект `Function` в программе.

```
public Function getGlobalFunctions(String name)
```

Возвращает первый объект `Function` для именованной функции или `null`, если такой функции нет.

```
public Function getFunctionAt(Address entryPoint)
```

Возвращает объект `Function` для функции по `entryPoint` или `null`, если такой функции нет.

```
public Function getFunctionAfter(Function function)
```

Возвращает объект `Function` для следующей за `function` функции или `null`, если такой функции нет.

```
public Function getFunctionAfter(Address address)
```

Возвращает объект `Function` для функции, начинающейся после `address`, или `null`, если такой функции нет.

Работа с перекрестными ссылками

Перекрестные ссылки были рассмотрены в главе 9. В Ghidra Program API объект верхнего уровня `Program` содержит `ReferenceManager`, который, что неудивительно, управляет ссылками внутри программы. Как и для многих других конструкций программы, Flat API предлагает удобные функции для доступа к перекрестным ссылкам; некоторые из них приведены ниже:

```
public Reference[] getReferencesFrom(Address address)
```

Возвращает массив всех объектов `Reference`, исходящих из `address`.

```
public Reference[] getReferencesTo(Address address)
```

Возвращает массив всех объектов `Reference`, завершающихся в `address`.

Функции манипулирования программой

При автоматизации задач анализа вы можете захотеть добавить новую информацию в программу. Flat API предоставляет множество функций для изменения содержимого программы, включая следующие:

```
public final void clearListing(Address address)
```

Удаляет любую инструкцию или данные, определенные по `address`.

```
public void removeFunctionAt(Address address)
```

Удаляет функцию по `address`.

```
public boolean disassemble(Address address)
```

Выполняет дизассемблирование рекурсивным спуском, начиная с `address`. Возвращает `true`, если операция успешна.

```
public Data createByte(Address address)
```

Преобразует элемент по указанному адресу в байт данных. Также доступны `createWord`, `createDword`, `createQword` и другие функции создания данных.

```
public boolean setEOLComment(Address address, String comment)
```

Добавляет комментарий конца строки по указанному адресу. Дополнительные функции, связанные с комментариями, включают `setPlateComment`, `setPreComment` и `setPostComment`.

```
public Function createFunction(Address entryPoint, String name)
```

Создает функцию с указанным `name` по `entryPoint`. Ghidra пытается автоматически определить конец функции, находя инструкцию возврата функции.

```
public Data createAsciiString(Address address)
```

Создает строку ASCII с завершающим нулем по `address`.

```
public Data createAsciiString(Address address, int length)
```

Создает строку ASCII указанной длины по `address`. Если `length` равен нулю или меньше, Ghidra пытается автоматически найти нулевой завершитель строки.

```
public Data createUnicodeString(Address address)
```

Создает строку Unicode с завершающим нулем по `address`.

Класс Program

Класс `Program` представляет корень иерархии Program API и самый внешний слой модели данных бинарного файла. Вы часто будете использовать объект `Program` (обычно `currentProgram`) для доступа к бинарной модели. Часто используемые функции-члены класса `Program` включают следующие:

```
public Listing getListing()
```

Извлекает объект `Listing` для текущей программы.

```
public FunctionManager getFunctionManager()
```

Извлекает FunctionManager программы, который предоставляет доступ ко всем функциям, идентифицированным внутри бинарного файла. Этот класс предоставляет функциональность для отображения Address обратно в содержащую его Function (Function getFunctionContaining(Address addr)). Кроме того, он предоставляет FunctionIterator, полезный, когда нужно обработать каждую функцию в программе.

```
public SymbolTable getSymbolTable()
```

Извлекает объект SymbolTable программы. Используя SymbolTable, можно работать с отдельными символами или итерировать по каждому символу в программе.

```
public Memory getMemory()
```

Извлекает объект Memory, связанный с этой программой, что позволяет работать с сырым байтовым содержимым программы.

```
public ReferenceManager getReferenceManager()
```

Извлекает объект ReferenceManager программы. ReferenceManager можно использовать для добавления и удаления ссылок, а также для получения итераторов для многих типов ссылок.

```
public Address getMinAddress()
```

Возвращает самый низкий допустимый адрес внутри программы. Чаще всего это базовый адрес памяти бинарного файла.

```
public Address getMaxAddress()
```

Возвращает самый высокий допустимый адрес внутри программы.

```
public LanguageID getLanguageID()
```

Возвращает объектное представление спецификации языка бинарного файла. Саму спецификацию языка затем можно получить с помощью функции getIdAsString().

Интерфейс Function

Интерфейс Function определяет требуемое поведение Program API для объектов функций. Функции-члены предоставляют доступ к различным атрибутам, обычно связанным с функциями, и включают следующие:

```
public String getPrototypeString(boolean formalSignature,
                                  boolean includeCallingConvention)
```

Возвращает прототип объекта Function как строку. Два аргумента влияют на формат возвращаемой строки прототипа.

```
public AddressSetView getBody()
```

Возвращает набор адресов, содержащий тело кода функции. Набор адресов состоит из одного или нескольких диапазонов адресов и допускает ситуации, когда код функции распределен по нескольким несмежным диапазонам памяти. Получите AddressIterator, чтобы посетить все адреса в наборе, или AddressRangeIterator, чтобы итерировать по каждому диапазону. Обратите внимание, что нужно использовать объект Listing, чтобы получить фактические инструкции, содержащиеся в теле функции (см. getInstructions).

```
public StackFrame getStackFrame()
```

Возвращает стековый кадр, связанный с функцией. Результат можно использовать для извлечения подробной информации о размещении локальных переменных функции и аргументов на стеке.

Интерфейс Instruction

Интерфейс `Instruction` определяет требуемое поведение Program API для объектов инструкций. Функции-члены предоставляют доступ к различным атрибутам, обычно связанным с инструкциями, и включают следующие:

```
public String getMnemonicString()
```

Возвращает мнемонику инструкции.

```
public String getComment(int commentType)
```

Возвращает комментарий `commentType`, связанный с инструкцией, или `null`, если комментарий данного типа не связан с инструкцией. `commentType` может быть одним из `EOL_COMMENT`, `PRE_COMMENT`, `POST_COMMENT` или `REPEATABLE_COMMENT`.

```
public int getNumOperands()
```

Возвращает количество операндов, связанных с этой инструкцией.

```
public int getOperandType(int opIndex)
```

Возвращает битовую маску флагов типа операнда, определенных в классе `OperandType`.

```
public String toString()
```

Возвращает строковое представление инструкции.

Примеры сценариев Ghidra

В оставшейся части главы мы представим несколько довольно распространенных ситуаций, в которых сценарий можно использовать для ответа на вопрос о программе. Ради краткости показано только тело функции `run` каждого сценария.

Пример 1: перечисление функций

Многие сценарии работают с отдельными функциями. Примеры включают генерацию дерева вызовов, укорененного в конкретной функции, генерацию графа потока управления функции и анализ стековых кадров каждой функции в программе. Листинг 14-1 итерирует по каждой функции в программе и печатает базовую информацию о каждой функции, включая начальный и конечный адреса функции, размер аргументов функции и размер локальных переменных функции. Весь вывод отправляется в окно консоли.

```
// ch14_1_flat.java
void run() throws Exception {
   int ptrSize = currentProgram.getDefaultPointerSize();
   Function func = getFirstFunction();
   while (func != null && !monitor.isCancelled()) {
      String name = func.getName();
      long addr = func.getBody().getMinAddress().getOffset();
      long end = func.getBody().getMaxAddress().getOffset();
      StackFrame frame = func.getStackFrame();
      int locals = frame.getLocalSize();
      int args = frame.getParameterSize();
      printf("Функция: %s, начало: %x, конец: %x\n", name, addr, end);
      printf("  Область локальных переменных: %d байт\n", locals);
      printf("  Аргументы используют %d байт (%d apr.)\n", args, args / ptrSize);
      func = getFunctionAfter(func);
   }
}
```

Листинг 14-1. Сценарий перечисления функций

Сценарий использует Flat API Ghidra, чтобы итерировать по всем функциям, начиная с первой и переходя к каждой следующей. Получается ссылка на стековый кадр каждой функции, затем извлекаются размер локальных переменных и аргументов на стеке. Перед продолжением итерации печатается сводка для каждой функции.

Пример 2: перечисление инструкций

Внутри заданной функции может понадобиться перечислить каждую инструкцию. Листинг 14-2 подсчитывает количество инструкций, содержащихся в функции, определенной текущим положением курсора:

```
// ch14_2_flat.java
public void run() throws Exception {
   Listing plist = currentProgram.getListing();
   Function func = getFunctionContaining(currentAddress);
   if (func != null) {
      InstructionIterator iter = plist.getInstructions(func.getBody(), true);
      int count = 0;
      while (iter.hasNext() && !monitor.isCancelled()) {
         count++;
         Instruction ins = iter.next();
      }
      popup(String.format("%s содержит %d инструкций\n",
                          func.getName(), count));
   }
   else {
      popup(String.format("Функция не найдена по адресу %x",
                          currentAddress.getOffset()));
   }
}
```

Листинг 14-2. Сценарий перечисления инструкций

Функция начинается с получения ссылки на функцию, содержащую курсор. Если функция найдена, следующий шаг - использовать объект `Listing` программы, чтобы получить `InstructionIterator` по функции. Цикл итерации подсчитывает количество извлеченных инструкций, а итог сообщается пользователю через всплывающий диалог сообщения.

Пример 3: перечисление перекрестных ссылок

Итерация по перекрестным ссылкам может быть запутанной из-за количества функций, доступных для доступа к данным перекрестных ссылок, и из-за того, что перекрестные ссылки кода являются двунаправленными. Чтобы получить нужные данные, необходимо обратиться к правильному типу перекрестной ссылки для вашей ситуации.

В первом примере перекрестной ссылки, показанном в листинге 14-3, мы извлекаем список всех вызовов функций, сделанных внутри функции, итерируя по каждой инструкции в функции, чтобы определить, вызывает ли инструкция другую функцию. Один способ сделать это - разобрать результаты функции `getMnemonicString` в поиске инструкций вызова. Это было бы не очень переносимым или эффективным решением, поскольку инструкция, используемая для вызова функции, различается между типами процессоров, а для определения, какая именно функция вызывается, потребовался бы дополнительный разбор. Перекрестные ссылки избегают этих трудностей, поскольку они независимы от процессора и напрямую сообщают целевую перекрестную ссылку.

```
// ch14_3_flat.java
void run() throws Exception {
   Listing plist = currentProgram.getListing();
   Function func = getFunctionContaining(currentAddress);
   if (func != null) {
      String fname = func.getName();
      InstructionIterator iter = plist.getInstructions(func.getBody(), true);
      while (iter.hasNext() && !monitor.isCancelled()) {
         Instruction ins = iter.next();
         Address addr = ins.getMinAddress();
         Reference refs[] = ins.getReferencesFrom();
         for (int i = 0; i < refs.length; i++) {
            if (refs[i].getReferenceType().isCall()) {
               Address tgt = refs[i].getToAddress();
               Symbol sym = getSymbolAt(tgt);
               String sname = sym.getName();
               long offset = addr.getOffset();
               printf("%s вызывает %s по адресу 0x%x\n", fname, sname, offset);
            }
         }
      }
   }
}
```

Листинг 14-3. Перечисление вызовов функций

Опасные функции

Функции C `strcpy` и `sprintf` считаются опасными, потому что позволяют неограниченное копирование в буферы назначения. Хотя каждая из них может безопасно использоваться программистами, проверяющими размеры буферов источника и назначения, такие проверки слишком часто забываются программистами, не осознающими опасность этих функций. Например, функция `strcpy` объявлена так:

```
char *strcpy(char *dest, const char *source);
```

Функция `strcpy` копирует все символы до первого нулевого завершающего символа включительно, встреченного в исходном буфере, в заданный буфер назначения (`dest`). Основная проблема в том, что во время выполнения невозможно определить размер любого массива, и `strcpy` не может определить, достаточна ли емкость буфера назначения, чтобы вместить все данные, копируемые из `source`. Такие непроверенные операции копирования являются одной из главных причин уязвимостей переполнения буфера.

Мы начинаем с получения ссылки на функцию, содержащую курсор. Затем итерируем по каждой инструкции в функции, а для каждой инструкции итерируем по каждой перекрестной ссылке, исходящей из инструкции. Нас интересуют только перекрестные ссылки, которые вызывают другие функции, поэтому нужно проверить возвращаемое значение `getReferenceType`, чтобы определить, является ли `isCall` истинным.

Пример 4: поиск вызовов функций

Перекрестные ссылки также полезны для определения каждой инструкции, которая ссылается на конкретное местоположение. В листинге 14-4 мы итерируем по всем перекрестным ссылкам к конкретному символу, в отличие от ссылок от символа в предыдущем примере.

```
// ch14_4_flat.java
public void list_calls(Function tgtfunc) {
   String fname = tgtfunc.getName();
   Address addr = tgtfunc.getEntryPoint();
```

```
    Reference refs[] = getReferencesTo(addr);
    for (int i = 0; i < refs.length; i++) {
       if (refs[i].getReferenceType().isCall()) {
          Address src = refs[i].getFromAddress();
          Function func = getFunctionContaining(src);
          if (func.isThunk()) {
             continue;
          }
          String caller = func.getName();
          long offset = src.getOffset();
           printf("%s вызывается из 0x%x в %s\n", fname, offset, caller);
       }
    }
 }
 public void getFunctions(String name, List<Function> list) {
    SymbolTable symtab = currentProgram.getSymbolTable();
    SymbolIterator si = symtab.getSymbolIterator();
    while (si.hasNext()) {
       Symbol s = si.next();
       if (s.getSymbolType() != SymbolType.FUNCTION || s.isExternal()) {
          continue;
       }
       if (s.getName().equals(name)) {
          list.add(getFunctionAt(s.getAddress()));
       }
    }
 }
 public void run() throws Exception {
    List<Function> funcs = new ArrayList<Function>();
    getFunctions("strcpy", funcs);
    getFunctions("sprintf", funcs);
    funcs.forEach((f) -> list_calls(f));
 }
```

Листинг 14-4. Перечисление вызывающих сторон функции[1]

В этом примере мы написали вспомогательную функцию `getFunctions`, чтобы собрать объекты `Function`, связанные с интересующими функциями. Для каждой интересующей функции мы вызываем вторую вспомогательную функцию, `list_calls`, чтобы обработать все перекрестные ссылки к функции. Если тип ссылки определяется как перекрестная ссылка типа вызова, извлекается вызывающая функция, и ее имя отображается пользователю. Помимо прочего, этот подход можно использовать для создания недорогого анализатора безопасности, подсвечивающего все вызовы функций вроде `strcpy` и `sprintf`.

Пример 5: эмуляция поведения языка ассемблера

Есть ряд причин, по которым вам может понадобиться написать сценарий, эмулирующий поведение анализируемой программы. Например, изучаемая программа может быть самомодифицирующейся, как многие вредоносные программы, или программа может содержать закодированные данные, которые декодируются при необходимости во время выполнения. Не запуская программу и не извлекая измененные данные из памяти работающего процесса, как понять поведение программы?

Если процесс декодирования не слишком сложен, вы можете быстро написать сценарий, выполняющий те же действия, которые выполняет программа при запуске. Использование сценария для декодирования данных таким образом устраняет необходимость запускать программу, когда вы не знаете, что она делает, или не имеете доступа к платформе, на которой ее можно выполнить. Например, без среды выполнения MIPS вы не можете выполнить MIPS-бинарный файл и наблюдать декодирование данных, которое он может выполнять. Однако можно написать сценарий Ghidra, имитирующий поведение бинарного файла и вносящий необходимые изменения внутри вашего проекта Ghidra, без какой-либо среды выполнения MIPS.

Следующий код x86 был извлечен из бинарного файла DEFCON Capture the Flag:

```
08049ede  MOV    dword ptr [EBP + local_8],0x0
        LAB_08049ee5
08049ee5  CMP    dword ptr [EBP + local_8],0x3c1
08049eec  JA     LAB_08049f0d
08049eee  MOV    EDX,dword ptr [EBP + local_8]
08049ef1  ADD    EDX,DAT_0804b880
08049ef7  MOV    EAX,dword ptr [EBP + local_8]
08049efa  ADD    EAX,DAT_0804b880
08049eff  MOV    AL,byte ptr [EAX]=>DAT_0804b880
08049f01  XOR    EAX,0x4b
08049f04  MOV    byte ptr [EDX],AL=>DAT_0804b880
08049f06  LEA    EAX=>local_8,[EBP + -0x4]
08049f09  INC    dword ptr [EAX]=>local_8
08049f0b  JMP    LAB_08049ee5
```

Этот код декодирует закрытый ключ, встроенный в бинарный файл программы. Используя сценарий в листинге 14-5, мы можем извлечь закрытый ключ без запуска программы.

```
// ch14_5_flat.java
public void run() throws Exception {
   int local_8 = 0;
   while (local_8 <= 0x3C1) {
      long edx = local_8;
      edx = edx + 0x804B880;
      long eax = local_8;
      eax = eax + 0x804B880;
      int al = getByte(toAddr(eax));
      al = al ^ 0x4B;
      setByte(toAddr(edx), (byte)al);
      local_8++;
   }
}
```

Листинг 14-5. Эмуляция языка ассемблера с помощью сценария Ghidra

Листинг 14-5 является довольно буквальным переводом предыдущей последовательности языка ассемблера, созданным по следующим механическим правилам:

- Для каждой стековой переменной и регистра, использованных в коде ассемблера, объявите переменную сценария подходящего типа.
- Для каждого оператора языка ассемблера напишите оператор, имитирующий его поведение.
- Эмулируйте чтение и запись стековых переменных чтением и записью соответствующей переменной, объявленной в сценарии.
- Эмулируйте чтение из нестекового местоположения с помощью функции `getByte`, `getWord`, `getDword` или `getQword` в зависимости от объема читаемых данных (1, 2, 4 или 8 байтов).
- Эмулируйте запись в нестековое местоположение с помощью функции `setByte`, `setWord`, `setDword` или `setQword` в зависимости от объема записываемых данных.
- Если код содержит цикл, для которого условие завершения не сразу очевидно, начните с бесконечного цикла, например `while(true){...}`, а затем вставьте оператор `break`, когда встретите операторы, вызывающие завершение цикла.
- Когда код ассемблера вызывает функции, все усложняется. Чтобы правильно симулировать поведение кода ассемблера, нужно имитировать поведение вызванной функции, включая предоставление возвращаемого значения, имеющего смысл в контексте симулируемого кода.

По мере роста сложности кода ассемблера становится все труднее писать сценарий, эмулирующий все аспекты последовательности языка ассемблера, но вам не обязательно полностью понимать, как работает эмулируемый код. Переводите одну-две инструкции за раз. Если каждая инструкция была корректно переведена, сценарий в целом должен правильно имитировать полную функциональность исходного кода ассемблера. После завершения сценарий можно использовать, чтобы лучше понять нижележащий ассемблерный код. Вы снова увидите этот подход и более

универсальную функциональность эмуляции в главе 21, когда мы будем обсуждать анализ обфусцированных бинарных файлов.

Например, когда мы переводим пример алгоритма и некоторое время размышляем о том, как он работает, можно сократить сценарий эмуляции следующим образом:

```
public void run() throws Exception {
   for (int local_8 = 0; local_8 <= 0x3C1; local_8++) {
      Address addr = toAddr(0x804B880 + local_8);
      setByte(addr, (byte)(getByte(addr) ^ 0x4B));
   }
}
```

После выполнения сценария декодированный закрытый ключ можно увидеть начиная с адреса `0x804B880`. Если вы не хотите изменять базу данных Ghidra при эмуляции кода, замените вызов функции `setByte` вызовом `printf`, который выведет результаты в консоль `CodeBrowser`, или запишите данные в дисковый файл для бинарных данных. Не забывайте, что помимо Java API Ghidra у вас есть доступ ко всем стандартным классам Java API, а также к любым другим пакетам Java, которые вы выбрали установить в своей системе.

Резюме

Сценарии предоставляют мощный способ автоматизировать повторяющиеся задачи и расширять возможности Ghidra. Эта глава представила функциональность Ghidra для редактирования и создания новых сценариев с использованием как Java, так и Python. Встроенная возможность строить, компилировать и запускать сценарии на Java внутри среды `CodeBrowser` позволяет расширять возможности Ghidra без необходимости глубокого понимания внутренних тонкостей среды разработки Ghidra. Главы 15 и 16 вводят интеграцию с Eclipse и возможность запускать Ghidra в режиме без графического интерфейса.

Сноски

[1] [назад] Предоставлено Kenshoto, организаторами CTF на DEFCON 15. DEFCON Capture the Flag - ежегодное соревнование по хакингу, проводимое на DEFCON (http://www.defcon.org/).

Глава 15. Eclipse и GhidraDev

Сценарии, распространяемые с Ghidra, и сценарии, которые мы создали в главе 14, относительно просты. Требуемый код был минимальным, что значительно упростило этапы разработки и тестирования. Базовый редактор сценариев, предоставляемый `Ghidra Script Manager`, подходит для быстрых черновых работ, но ему не хватает развитых возможностей для управления сложными проектами. Для более существенных задач Ghidra предоставляет плагин, облегчающий разработку с использованием среды разработки Eclipse. В этой главе мы рассмотрим Eclipse и роль, которую он может сыграть в разработке более продвинутых сценариев Ghidra. Мы также покажем, как Eclipse можно использовать для создания новых модулей Ghidra, и вернемся к этой теме в последующих главах, когда расширим набор загрузчиков Ghidra и обсудим внутреннюю работу процессорных модулей Ghidra.

Eclipse

Eclipse - это интегрированная среда разработки (IDE), используемая многими Java-разработчиками, что делает ее естественным выбором для разработки под Ghidra. Хотя Eclipse и Ghidra можно запускать на одной машине без какого-либо взаимодействия между ними, интеграция этих двух приложений может значительно упростить разработку для Ghidra. Без интеграции Eclipse был бы просто еще одним вариантом редактирования сценариев вне среды Ghidra. Интегрируя Eclipse с Ghidra, вы внезапно получаете богатую IDE, включающую специфичную для Ghidra функциональность, ресурсы и шаблоны, которые облегчают ваш процесс разработки для Ghidra. Интеграция Eclipse и Ghidra не требует значительных усилий; нужно лишь предоставить каждому приложению некоторые сведения о другом, чтобы их можно было использовать вместе.

Интеграция Eclipse

Чтобы Ghidra могла работать с Eclipse, в Eclipse должен быть установлен плагин GhidraDev. Интегрировать два приложения можно как из Ghidra, так и из Eclipse. Инструкции для обоих подходов включены в документ `GhidraDev_README.html`, находящийся в каталоге `Extensions/Eclipse/GhidraDev` вашей установки Ghidra.

Хотя письменная документация проводит вас через детали процесса, самая простая отправная точка - выбрать действие Ghidra, которому требуется Eclipse, например `Edit Script with Eclipse` (см. рисунок 14-2). Если вы выберете этот вариант и ранее не интегрировали Eclipse и Ghidra, вам будет предложено указать сведения о каталогах, необходимые для установления соединения. В зависимости от вашей конфигурации может потребоваться предоставить путь к каталогу установки Eclipse, каталогу рабочей области Eclipse, каталогу установки Ghidra, каталогу подключаемых модулей Eclipse и, возможно, номер порта, используемый для связи с Eclipse при редактировании сценариев.

Документация Ghidra поможет преодолеть любые препятствия, встреченные во время процесса интеграции. По-настоящему смелые могут исследовать плагины интеграции в каталоге `Ghidra/Features/Base/src/main/java/ghidra/app/plugin/core/eclipse` в репозитории исходного кода Ghidra.

Запуск Eclipse

После успешной интеграции Ghidra и Eclipse их можно использовать для написания сценариев и плагинов Ghidra. При первом запуске Eclipse после интеграции с Ghidra вы, скорее всего, увидите диалог, показанный на рисунке 15-1, с запросом установить путь связи между вашим экземпляром Ghidra и экземпляром Eclipse GhidraDev.

Рисунок 15-1. Диалог GhidraDevUser Consent

Двигаясь дальше, вы увидите экран приветствия Eclipse IDE, как показано на рисунке 15-2. У этого экземпляра Eclipse есть новое дополнение в строке меню: GhidraDev. Это меню мы будем использовать для создания более сложных сценариев и инструментов Ghidra.

Рисунок 15-2. Экран приветствия Eclipse IDE

Начальная страница для Ghidra Eclipse, рабочая область Welcome to the Eclipse IDE for Java Developers, включает ссылки на многочисленные учебные материалы, документацию и информацию об Eclipse IDE и Java, которые должны предоставить необходимую базовую поддержку пользователям, новым для Eclipse, а также необязательное напоминание опытным пользователям. Чтобы двигаться дальше с Ghidra, мы сосредоточимся на том, как меню GhidraDev можно использовать для расширения существующих возможностей Ghidra, построения новых возможностей и настройки Ghidra для улучшения рабочего процесса обратной разработки.

Редактирование сценариев с помощью Eclipse

После установки плагина GhidraDev в Eclipse вы готовы создавать новые сценарии или редактировать существующие с помощью Eclipse IDE. Переходя от использования `Ghidra Script Manager` для создания и редактирования сценариев к использованию Eclipse, стоит помнить: хотя Eclipse можно запустить из `Script Manager`, сделать это можно только для редактирования существующего сценария (см. рисунок 14-2). Если вы хотите редактировать новый сценарий с помощью Eclipse, сначала нужно запустить Eclipse, а затем использовать меню GhidraDev для создания нового сценария. Независимо от того, запускаете ли вы Eclipse сами или попадаете в Eclipse через `Ghidra Script Manager`, в оставшейся части этой главы мы используем Eclipse, а не базовый редактор `Script Manager`, для создания и изменения сценариев и модулей для Ghidra.

Чтобы отредактировать первый сценарий, созданный в разделе "Пример редактирования сценария: поиск по регулярному выражению" на странице 290, выберите `File -> Open File` в меню Eclipse и перейдите к сценарию `FindStringByRegex.java`. Это откроет сценарий в Eclipse IDE, и вы сможете начать использовать богатый набор параметров редактирования Eclipse. На рисунке 15-3 показаны первые несколько строк сценария со свернутыми комментариями и импортами. Сворачивание строк - стандартная возможность Eclipse IDE, которая может вызвать некоторую путаницу, если вы переключаетесь между базовым редактором, предоставленным Ghidra, и Eclipse.

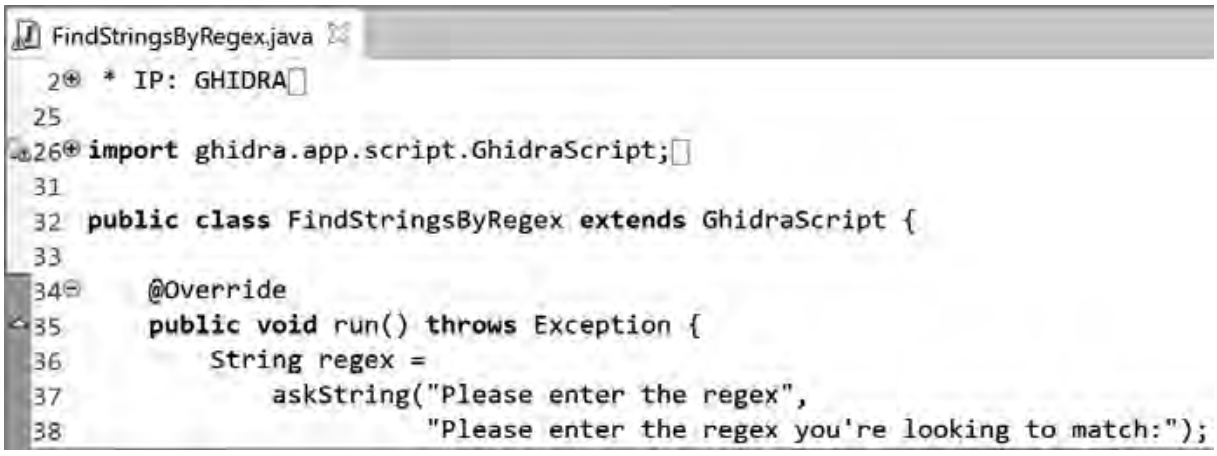


Рисунок 15-3. Представление FindStringsByRegex в редакторе Eclipse

По умолчанию отображается только одна строка комментариев. Можно щелкнуть значок, чтобы раскрыть содержимое (значок + слева от строки 2) и показать все комментарии, а также свернуть содержимое (значок - слева от строки 34), если нужно. То же верно для строки 26 с инструкциями импорта. Наведение на значок любого свернутого раздела отображает скрытое содержимое во всплывающем окне.

Прежде чем начать строить примеры, расширяющие возможности Ghidra, нужно больше понять о меню GhidraDev и Eclipse IDE. Переключим внимание обратно на меню GhidraDev и исследуем различные параметры и то, как их можно использовать в контексте.

Меню GhidraDev

Раскрытое меню GhidraDev показано на рисунке 15-4 и включает пять параметров, которые можно использовать для управления средой разработки и работы с файлами. В этой главе мы сосредоточимся на разработке на Java, хотя Python является вариантом в нескольких окнах.

Рисунок 15-4. Параметры меню GhidraDev

GhidraDev -> New

Меню GhidraDev -> New предоставляет три пункта подменю, как показано на рисунке 15-5. Все три пункта запускают мастера, которые проводят через связанный процесс создания. Начнем с самого простого варианта - создания нового сценария Ghidra. Это альтернативный путь создания сценариев к тому, что обсуждался в главе 14.

Рисунок 15-5. Подменю GhidraDev -> New

Создание сценария

Создание нового сценария через GhidraDev -> New -> Ghidra Script приводит к диалогу, позволяющему ввести информацию о новом сценарии. Пример диалога, заполненного содержимым, показан на рисунке 15-6. Помимо информации о каталоге и файле, диалог собирает те же метаданные, которые мы вручную вводили в файлы сценариев в базовом редакторе `Script Manager`.

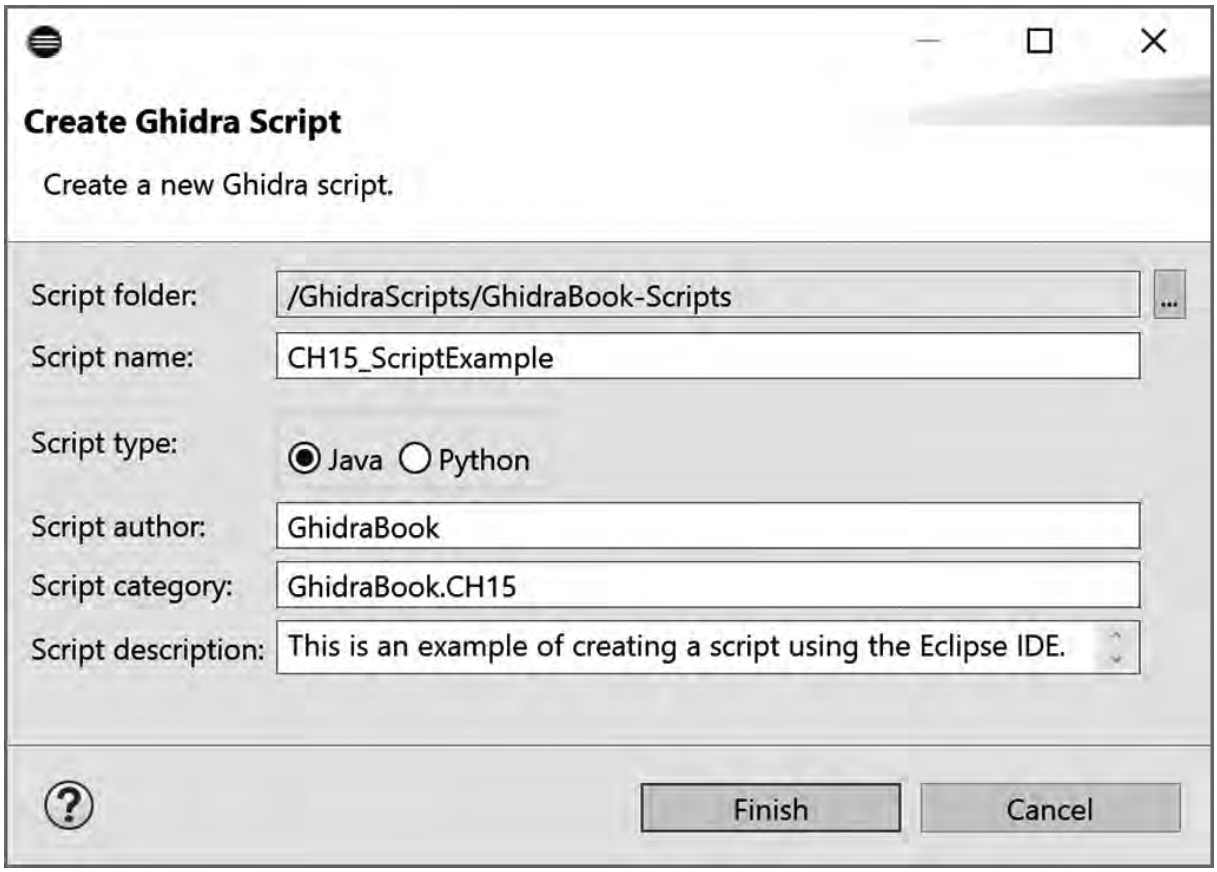


Рисунок 15-6. Диалог Create Ghidra Script

Кнопка `Finish` в нижней части диалога создает шаблон сценария, показанный на рисунке 15-7. Метаданные, введенные на рисунке 15-6, включены в раздел комментариев в верхней части сценария. Это содержимое имеет тот же формат, что и метаданные, которые мы видели в главе 14 (см. верхнюю часть рисунка 14-4). Когда вы редактируете этот сценарий в Eclipse, тег задачи (значок буфера обмена, видимый слева от строки 14 на рисунке 15-7), связанный с каждым элементом `TODO` в сценарии, указывает места, где еще нужно выполнить работу. Теги задач можно удалять и вставлять по желанию.

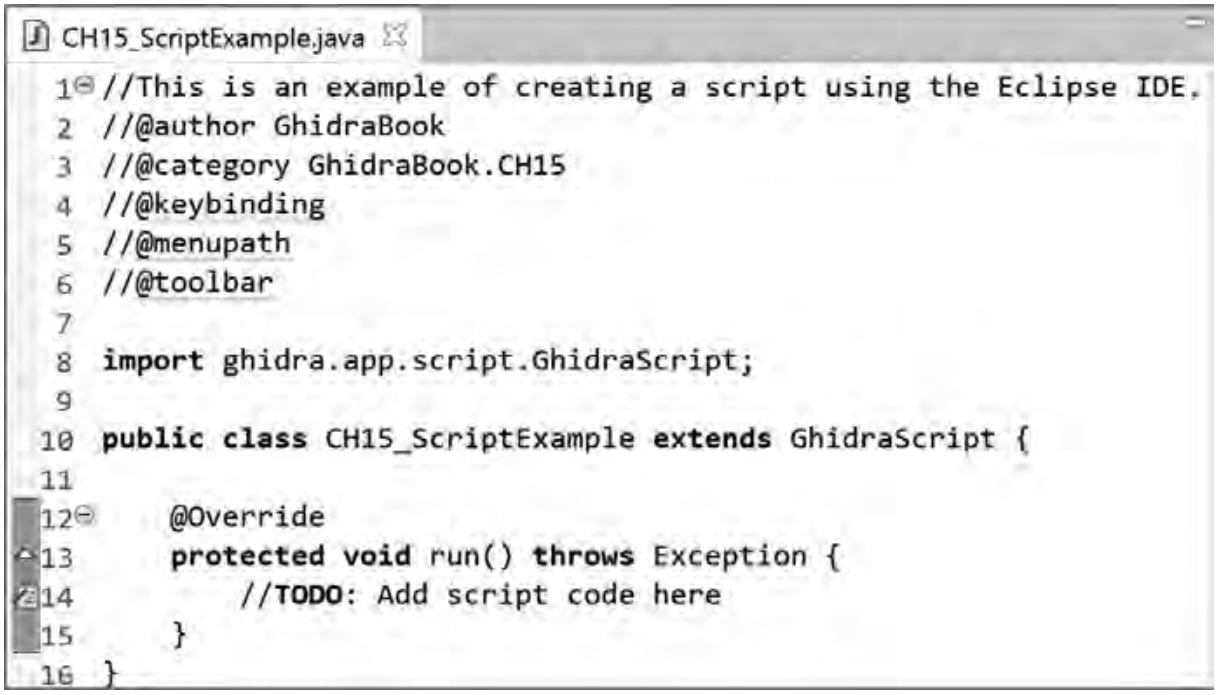


Рисунок 15-7. Каркас сценария GhidraDev -> New Script

Eclipse не загружает ваш сценарий заранее списком инструкций импорта, как это делает базовый редактор Ghidra (см. рисунок 14-4). Не беспокойтесь. Eclipse помогает управлять инструкциями `import`, сообщая, когда вы используете что-то, требующее связанной инструкции импорта. Например, если заменить комментарий TODO на рисунке 15-7 объявлением Java `ArrayList`, Eclipse добавляет тег ошибки к строке и подчеркивает `ArrayList` красным. Наведение на тег ошибки или `ArrayList` отображает всплывающее окно с предложенными быстрыми исправлениями для решения проблемы, как показано на рисунке 15-8.

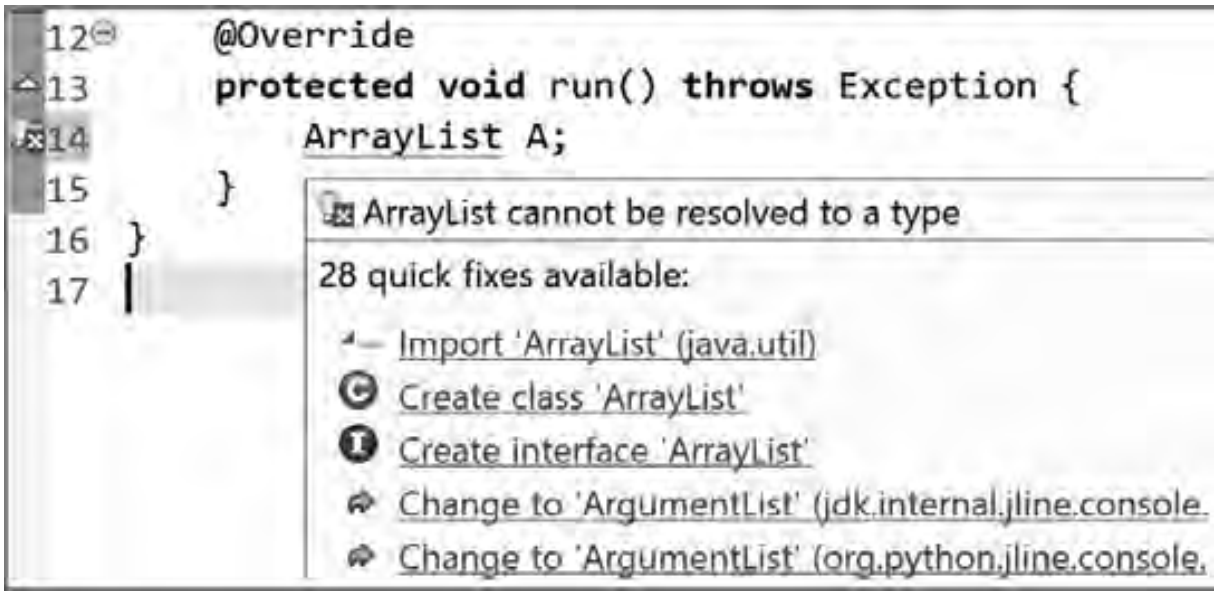


Рисунок 15-8. Параметры Eclipse Quick Fix

Выбор первого варианта в списке предложений инструктирует Eclipse добавить выбранную инструкцию импорта в сценарий, как показано на рисунке 15-9. Хотя список потенциальных инструкций импорта, загружаемый при создании нового сценария в `CodeBrowser Script Manager`, был полезен, в Eclipse он не так необходим.

```
8  import java.util.ArrayList;
9
10 import ghidra.app.script.GhidraScript;
11
12 public class CH15_ScriptExample extends GhidraScript {
13
14     @Override
15     protected void run() throws Exception {
16         ArrayList A;
17     }
```

Рисунок 15-9. Eclipse после применения импорта Quick Fix

Создание проекта сценариев

Второй вариант в меню `GhidraDev -> New` создает новый проект сценариев, как показано на рисунке 15-10. Мы называем первый проект сценариев `CH15_ProjectExample_linked` и помещаем его в каталог по умолчанию, настроенный для Eclipse. Флажок `Create run configuration` позволяет создать конфигурацию запуска, которая предоставляет Eclipse необходимую информацию (аргументы командной строки, пути каталогов и т. д.) для запуска Ghidra и позволяет использовать Eclipse для запуска и отладки сценария в Ghidra. Оставьте этот флажок в выбранном состоянии по умолчанию. Нажмите `Finish`, чтобы завершить создание сценария с использованием формата по умолчанию, который связывает проект сценариев с вашим домашним каталогом.

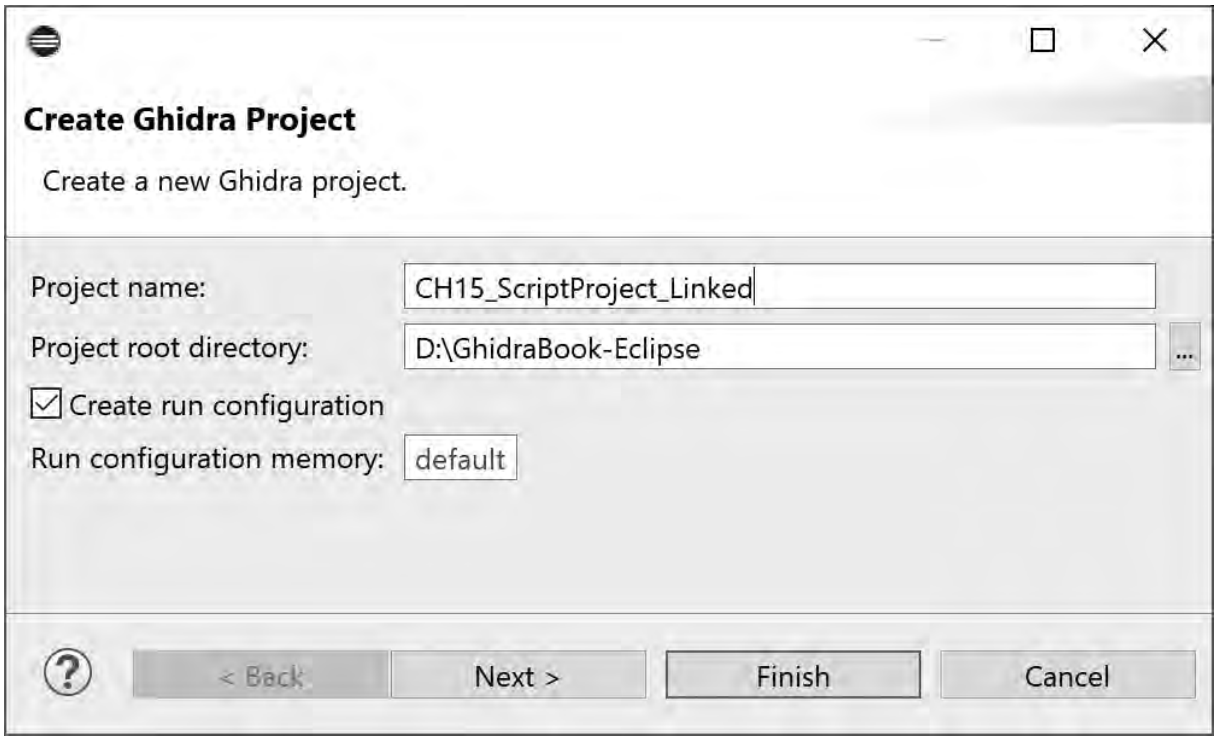


Рисунок 15-10. Диалог Eclipse Ghidra Script Project

Мы создадим второй проект сценариев `CH15_ProjectExample` и на этот раз выберем кнопку Next. Выбор Next дает диалог с двумя параметрами связывания, установленными по умолчанию, отсюда расширение `_linked` в имени нашего первого проекта. Первый параметр создает ссылку на ваш

домашний каталог сценариев. Второй позволяет создать ссылку на каталоги сценариев установки Ghidra. Ссылка в данном случае означает, что папки, представляющие ваш домашний каталог сценариев и/или собственные каталоги сценариев Ghidra, будут добавлены в новый проект, делая любые сценарии в этих каталогах легко доступными во время работы над проектом.

Результаты выбора или отмены этих параметров и последующего нажатия `Finish` станут ясны позднее в главе, когда мы обсудим Eclipse Package Explorer. Для второго проекта сценариев снимите первый флажок ссылки, как показано на рисунке 15-11.

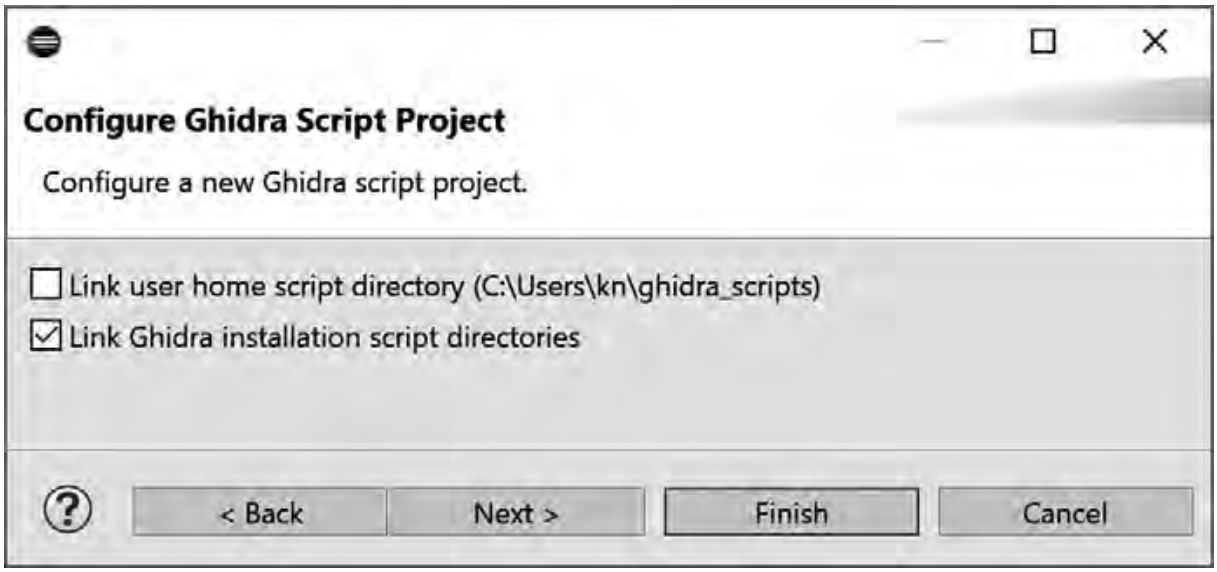


Рисунок 15-11. Параметры конфигурации Eclipse для проектов сценариев

Создание проекта модуля

Последний вариант в меню GhidraDev -> New создает проект модуля Ghidra. Не путайте его с модулем Ghidra (например, анализатором, загрузчиком и т. д.): проект модуля Ghidra агрегирует код для нового модуля Ghidra со связанными файлами справки, документацией и другими ресурсами, например значками. Кроме того, он дает некоторый контроль над тем, как ваш новый модуль взаимодействует с другими модулями внутри Ghidra. Мы демонстрируем модули Ghidra в контексте в этой и будущих главах.

Оба типа модулей Ghidra отличаются от модулей Java, введенных в Java 9 как способ инкапсулировать пакеты и другие ресурсы и предоставить возможность держать пакеты закрытыми или избирательно делиться отдельными пакетами с выбранными модулями, фактически позволяя модулям управлять совместным использованием сервисов.

Выбор New -> Ghidra Module Project отображает диалог, показанный на рисунке 15-12, который должен быть знаком, потому что он точно такой же, как диалог Script Project. Мы называем новый проект `CH15_ModuleExample`, чтобы его было легко распознать в Package Explorer.

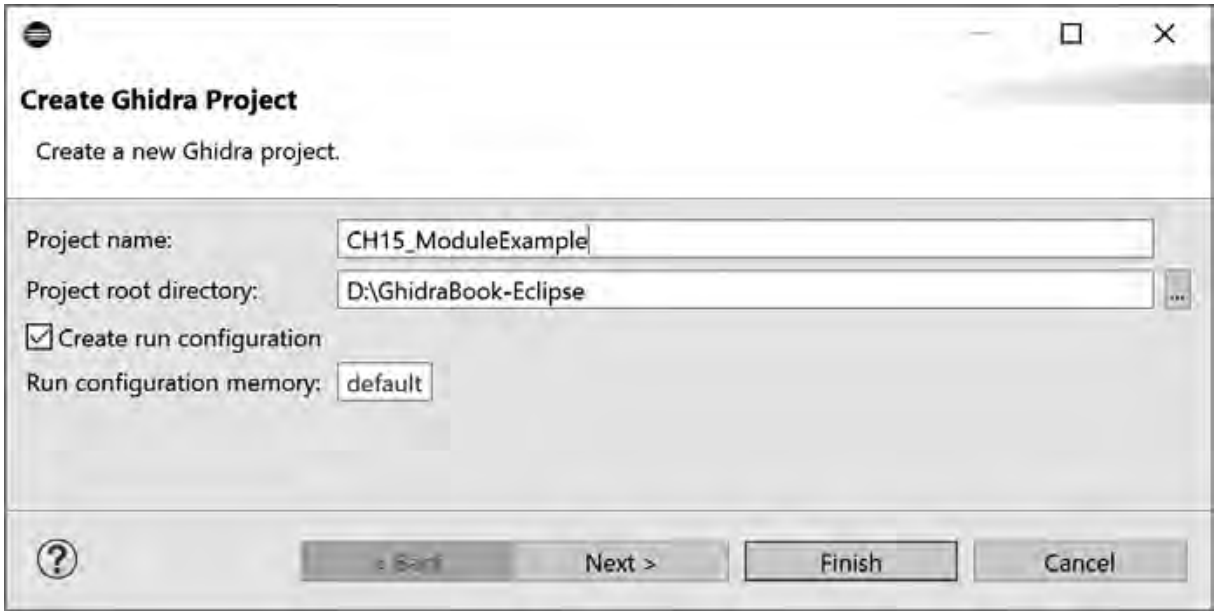


Рисунок 15-12. Диалог Eclipse Module Project

Нажатие Next на этом этапе позволяет основать модуль на существующих шаблонах Ghidra, как показано на рисунке 15-13. По умолчанию выбраны все параметры. Можно изменить это, включив ни один, некоторые или все шаблоны в зависимости от целей разработки. Любые выбранные параметры будут сгруппированы вместе в проекте внутри Package Explorer. В нашем случае мы сняли выбор со всех параметров.

Хотя большинство выборов создаст связанный шаблон исходного кода с тегами задач, есть два исключения. Во-первых, если не выбрать ни один шаблон модуля, файла шаблона не будет. Кроме того, процессорный модуль не создает файл шаблона, но генерирует другое вспомогательное содержимое. Процессорные модули обсуждаются в главе 18.

Рисунок 15-13. Параметры шаблонов для проектов модулей Ghidra

Теперь, когда вы знаете, как создавать сценарии Ghidra, проекты сценариев и проекты модулей, переключим внимание на Eclipse Package Explorer, чтобы лучше понять, как работать с нашими новыми созданиями.[1]

Навигация по Package Explorer

Package Explorer в Eclipse - это вход к файлам Ghidra, которые нужны для завершения вашего расширения Ghidra. Здесь мы представим иерархическую организацию, а затем углубимся в примеры проектов и модулей Ghidra, созданные через меню GhidraDev. Рисунок 15-14 показывает пример окна Eclipse Package Explorer, содержащего элементы, которые мы создали ранее в этой главе, а также несколько других, созданных для демонстрации влияния различных параметров на результирующее содержимое Package Explorer.

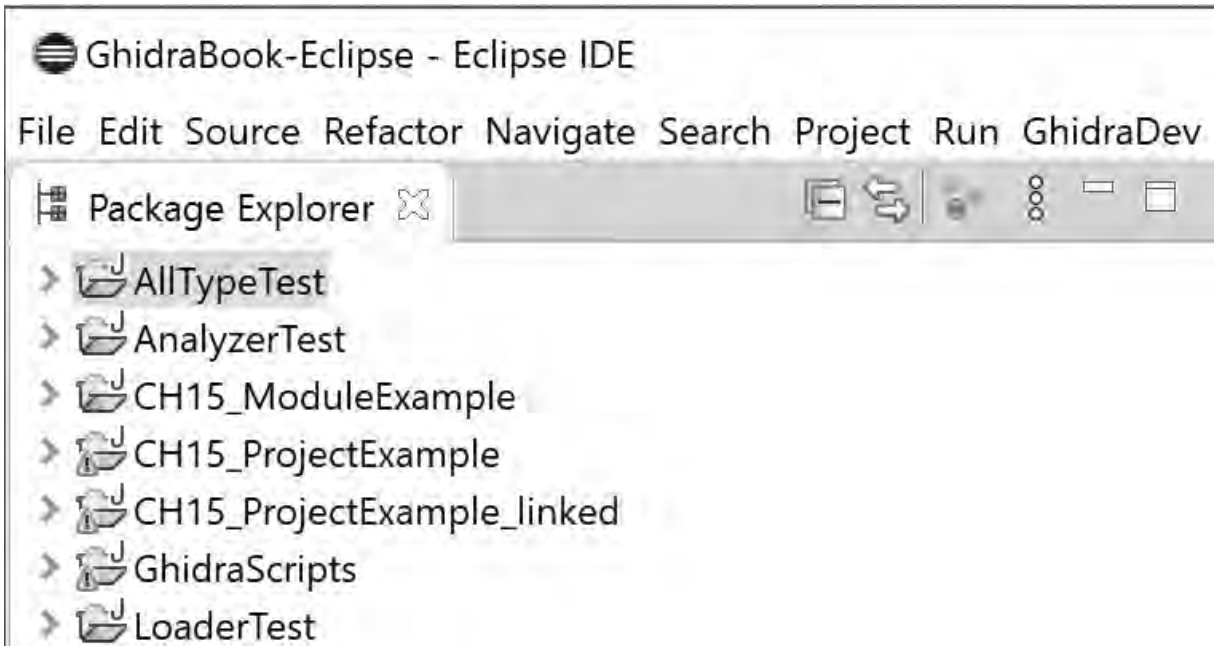


Рисунок 15-14. Package Explorer, заполненный примерными модулями и проектом

Начнем с двух проектов сценариев. `CH15_ProjectExample_linked` - проект сценариев, созданный с включенными обоими параметрами связывания (см. рисунок 15-11). Сразу под ним мы видим похожий проект, `CH15_ProjectExample`, но в этом случае ни один параметр связывания не был включен. Частично раскрытая запись Package Explorer для `CH15_ProjectExample` показана на рисунке 15-15.

Рисунок 15-15. Записи проекта сценариев в Package Explorer без ссылок

В этот проект сценариев включены следующие четыре компонента:

JUnit4. Фреймворк модульного тестирования с открытым исходным кодом для Java. Дополнительная информация доступна по адресу https://junit.org/junit4/index.html.

JRE System Library. Системная библиотека среды выполнения Java.

Referenced Libraries. Библиотеки, на которые есть ссылки и которые не являются частью `JRE System Library`, но являются частью нашей установки Ghidra.

Ghidra. Каталог текущей установки Ghidra. Мы раскрыли этот каталог, чтобы вы могли увидеть знакомую структуру файлов, представленную в главе 3 (см. рисунок 3-1) и используемую на протяжении книги.

Сравните содержимое рисунка 15-15 с раскрытым содержимым `CH15_ProjectExample_linked`, показанным на рисунке 15-16. Для этого проекта сценариев мы выбрали оба параметра связывания. Связывание домашнего каталога сценариев пользователя приводит к записи Home scripts в иерархии проекта и дает простой доступ к сценариям, ранее написанным нами для использования как примеры или для изменения.

Рисунок 15-16. Записи проекта сценариев в Package Explorer со ссылками

Связывание каталогов сценариев установки Ghidra приводит к появлению всех папок на рисунке 15-16, которые начинаются с `Ghidra` и заканчиваются `scripts`. Каждая соответствует каталогу сценариев внутри каталога `Ghidra/Features` вашей установки Ghidra. Раскрытие любой из этих папок предоставляет доступ к исходному коду каждого сценария, включенного в установку Ghidra.

Как и домашние сценарии, они могут служить примерами для изменения или основой для создания новых сценариев. Хотя вам не разрешено перезаписывать эти сценарии из базового редактора `Ghidra Script Manager`, их можно редактировать в Eclipse и других редакторах вне среды `Ghidra Project`. Когда вы закончите создавать или редактировать новый сценарий, можно сохранить его в соответствующий каталог сценариев внутри проекта сценариев, и он будет доступен при следующем открытии `Ghidra Script Manager`.[2]

Теперь, когда мы посмотрели на сценарии внутри Eclipse Package Explorer, посмотрим, как представлен созданный нами проект модуля Ghidra. Частично раскрытое содержимое нашего проекта в Package Explorer показано на рисунке 15-17.

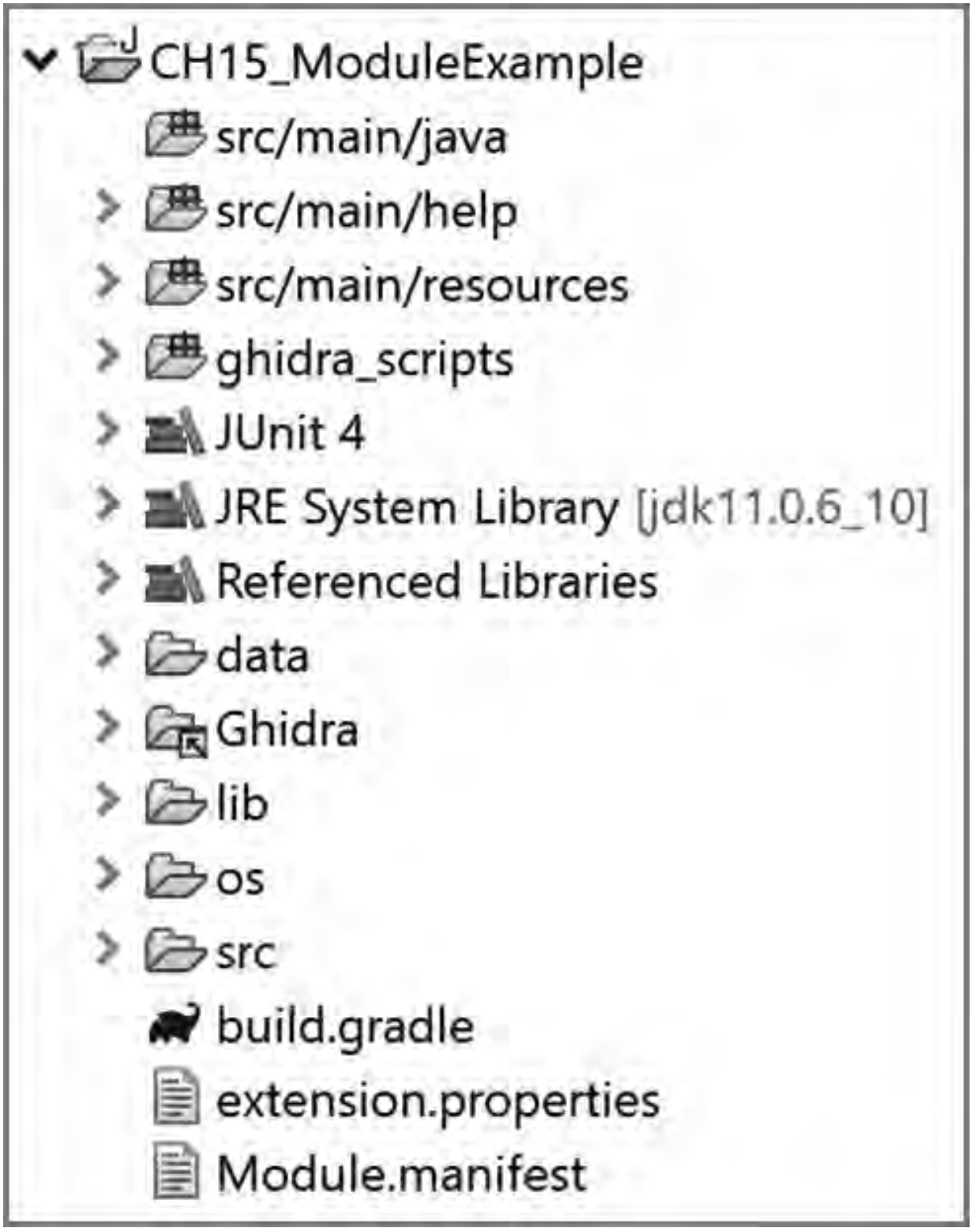


Рисунок 15-17. Иерархия Package Explorer для CH15_ModuleExampleModule

Проекты модулей включают следующие новые элементы:

src/main/java. Местоположение для исходного кода. Если вы создали тип модуля, для которого доступен шаблон, связанные файлы `.java` помещаются в этот каталог.

src/main/help. Когда вы создаете или расширяете содержимое, у вас есть возможность добавить полезную информацию в `Ghidra Help`, используя файлы и информацию в этом каталоге.

src/main/resources. Как и многие другие записи в каталоге `src/main`, раскрытие этого содержимого приведет к файлу `README.txt`, который предоставляет дополнительную информацию о назначении каталога и том, как его следует использовать. Например, файл `src/main/resources/images/README.txt` сообщает, что это место, где должны храниться любые файлы изображений или значков, связанные с модулем.

ghidra_scripts. Здесь хранятся сценарии Ghidra, специфичные для этого модуля.

data. Эта папка содержит любые независимые файлы данных, используемые с модулем. Хотя использование с другими типами модулей не запрещено, эта папка главным образом используется с процессорными модулями и обсуждается в главе 18.

lib. Любые файлы `.jar`, требуемые модулю, должны храниться в этой папке.

os. Внутри этой папки есть подкаталоги для `linux64`, `osx64` и `win64`, предназначенные для любых родных бинарных файлов, от которых может зависеть модуль.

src. Этот каталог используется для тестовых случаев модульного тестирования.

build.gradle. Gradle - система сборки с открытым исходным кодом. Этот файл используется для сборки вашего расширения Ghidra.

extension.properties. Этот файл хранит метаданные о расширении.

Module.manifest. В этот файл можно вводить информацию о модуле, например сведения о конфигурации.

Вы могли заметить на рисунке 15-14, что мы создали дополнительные тестовые модули (`AnalyzerTest`, `AllTypeTest` и `LoaderTest`). Каждый был создан с использованием разной комбинации параметров Module Template (см. рисунок 15-13), что приводит к созданию разного набора файлов для каждого проекта. При использовании этих шаблонов как отправной точки для ваших проектов полезно знать, какой объем работы Eclipse и Ghidra уже выполнили за вас и сколько еще предстоит сделать.

Начнем с каталога `AnalyzerTest`, созданного для демонстрации шаблона анализатора. Раскройте каталог `src/main/java`, чтобы найти файл `AnalyzerTestAnalyzer.java`. Имя является конкатенацией имени модуля (`AnalyzerTest`) с типом шаблона (`Analyzer`). Дважды щелкните этот файл, чтобы открыть его в редакторе и увидеть код, показанный на рисунке 15-19. Как и шаблоны сценариев ранее в главе, Eclipse IDE предоставляет теги задач со связанными комментариями, которые направляют нас при построении анализатора, а также параметры для раскрытия и сворачивания содержимого. Модуль `LoaderTest` содержит шаблон для построения загрузчика, который подробнее обсуждается в главе 17. Оставшийся модуль, `AllTypeTest`, является модулем по умолчанию, получающимся при обходе параметров шаблонов модулей. Он заполняет каталог `src/main/java` всеми шаблонами, как показано на рисунке 15-20.

```
 2⊕ * IP: GHIDRA
16 package analyzertest;
17
18⊕ import ghidra.app.services.AbstractAnalyzer;
26
27⊖ /**
28  * TODO: Provide class-level documentation that describes what this analyzer does.
29  */
30 public class AnalyzerTestAnalyzer extends AbstractAnalyzer {
31
32⊕     public AnalyzerTestAnalyzer() {
38
40⊕     public boolean getDefaultEnablement(Program program) {
46
48⊕     public boolean canAnalyze(Program program) {
55
57⊕     public void registerOptions(Options options, Program program) {
64
66⊕     public boolean added(Program program, AddressSetView set, TaskMonitor monitor, MessageLog log)
74 }
```

Рисунок 15-19. Шаблон анализатора по умолчанию для модуля (комментарии, импорты и функции свернуты)

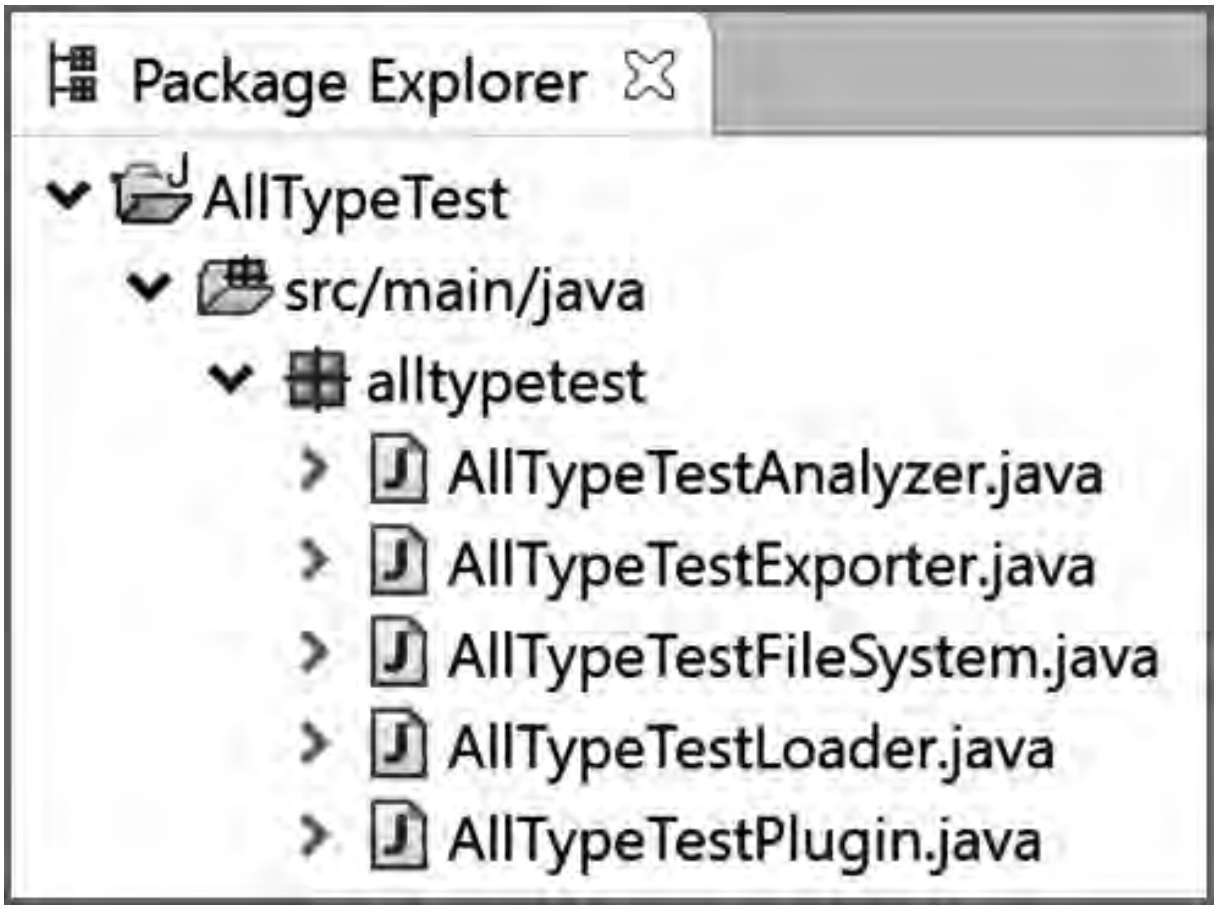


Рисунок 15-20. Пример содержимого исходного кода модуля по умолчанию

Теперь, когда мы увидели, насколько полезными могут быть Ghidra и Eclipse при создании новых модулей, используем эту информацию, чтобы построить новый анализатор.

Снова строим тот же сценарий?

В главе 14 мы представили игрушечный пример в среде `Ghidra Script Manager`, где изменили существующий сценарий `CountAndSaveStrings` и использовали его для построения нового сценария `FindStringsByRegex`. Следующие шаги выполняют ту же задачу внутри Eclipse IDE:

1. Найдите `CountAndSaveStrings.java` в Eclipse (`ctrl-shift-R`).
2. Дважды щелкните, чтобы открыть файл в редакторе Eclipse.
3. Замените существующий класс и комментарии новым классом и комментариями.
4. Сохраните файл (`EclipseFindStringByRegex.java`) в рекомендованном каталоге `ghidra_scripts`.
5. Запустите новый сценарий из окна `Script Manager` в Ghidra.

Можно запустить Ghidra вручную, чтобы получить доступ к окну `Script Manager`. В качестве альтернативы можно выбрать параметр Run As в Eclipse IDE, который покажет диалог на рисунке 15-18. Первый параметр запускает Ghidra. Второй параметр запускает версию Ghidra без GUI, что является темой главы 16.

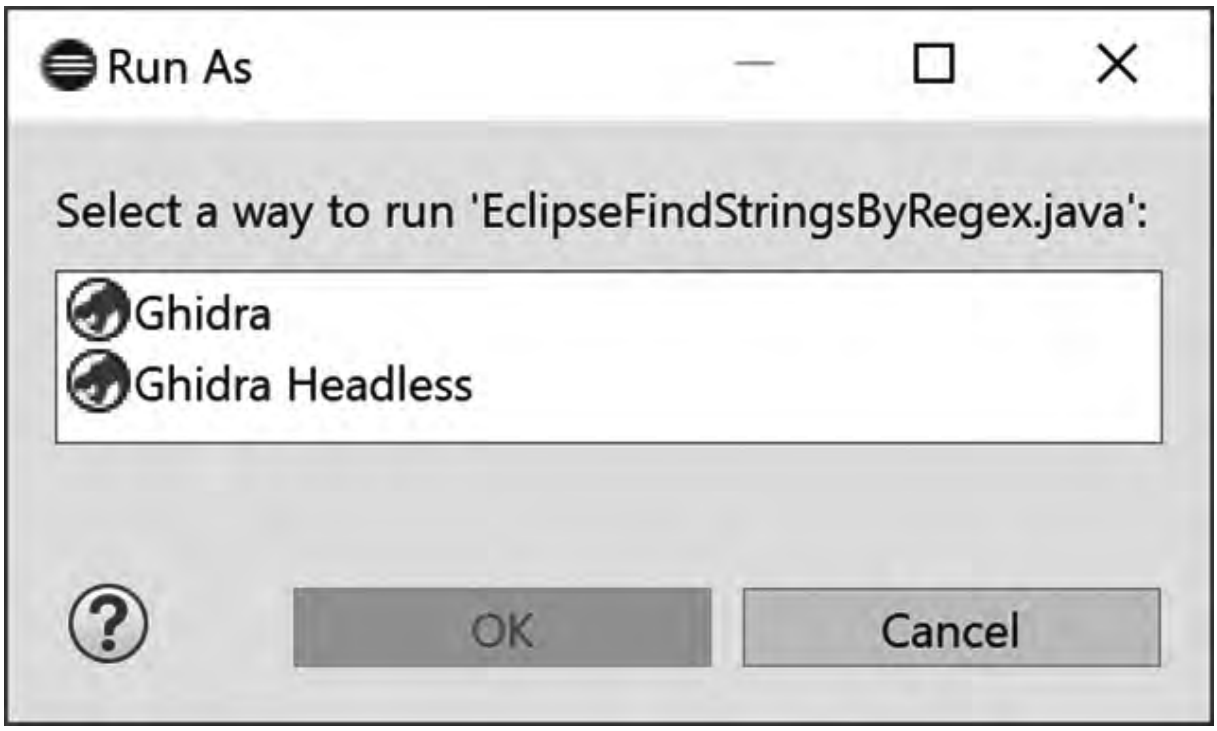


Рисунок 15-18. Параметры Eclipse Run As

После запуска Ghidra можно запустить сценарий из `Script Manager` и редактировать его с помощью Eclipse.

Пример: проект модуля анализатора Ghidra

Когда основы интеграции Eclipse позади, пройдем построение простого анализатора Ghidra для идентификации потенциальных ROP-гаджетов в листинге. Мы используем упрощенный процесс разработки ПО, поскольку это лишь простой демонстрационный проект. Наш процесс включает следующие шаги:

1. Определить проблему.
2. Создать модуль Eclipse.
3. Построить анализатор.
4. Добавить анализатор в нашу установку Ghidra.
5. Протестировать анализатор из нашей установки Ghidra.

Шаг 1: определить проблему

Наша задача - спроектировать и разработать анализатор инструкций, который будет идентифицировать простые ROP-гаджеты внутри бинарного файла. Анализатор нужно добавить в Ghidra, и он должен быть доступен как выбираемый анализатор в меню Ghidra Analyzer.

Что такое ROP-гаджет и зачем он нам?

Для тех, кто не знаком с разработкой эксплойтов, ROP означает программирование, ориентированное на возвраты. Одна мера смягчения рисков безопасности ПО, предназначенная для противодействия внедрению сырого шеллкода, заключается в том, чтобы гарантировать, что ни одна область памяти, доступная для записи, одновременно не является исполняемой. Такие меры часто называют Non-eXecutable (NX) или Data Execution Prevention (DEP), потому что становится невозможно внедрить шеллкод в память (она должна быть доступной для записи), а затем передать управление этому шеллкоду (она должна быть исполняемой).

Техники ROP стремятся захватить стек программы, часто через стековое переполнение буфера, чтобы разместить тщательно созданную последовательность адресов возврата и данных в стеке. В какой-то момент после переполнения программа начинает использовать адреса возврата, предоставленные атакующим, вместо адресов возврата, помещенных на стек нормальным выполнением программы. Адреса возврата, которые атакующий помещает в стек, указывают на местоположения памяти программы, уже содержащие код в результате обычных операций загрузки программы и библиотек. Поскольку исходный автор эксплуатируемой программы не проектировал программу так, чтобы она выполняла работу атакующего, атакующему часто нужно выбирать и связывать небольшие части этого существующего кода.

ROP-гаджет - один такой фрагмент кода, а механизм выстраивания последовательности часто полагается на то, что гаджет завершается инструкцией возврата, отсюда "ориентированное на возвраты", которая извлекает адрес из теперь контролируемого атакующим стека, чтобы передать управление следующему гаджету. Гаджет часто выполняет очень простую задачу, например загружает регистр из стека. Следующий простой гаджет можно использовать для инициализации RAX в системе x86-64:

```
POP RAX  ; извлечь следующий элемент контролируемого атакующим стека в RAX
RET      ; передать управление по адресу в следующем элементе стека
```

Поскольку каждая эксплуатируемая программа отличается, атакующие не могут полагаться на то, что в любом заданном бинарном файле будет присутствовать конкретный набор гаджетов. Автоматизированные искатели гаджетов - это инструменты, которые ищут в бинарном файле последовательности инструкций, которые можно использовать как гаджеты, и представляют эти гаджеты атакующему, который должен решить, какие из них полезны для построения атаки. Самые сложные искатели гаджетов выводят семантику гаджета и автоматически выстраивают последовательность гаджетов для выполнения заданного действия, избавляя атакующего от необходимости делать это вручную.

Шаг 2: создать модуль Eclipse

Мы используем GhidraDev -> New -> Ghidra Module Project, чтобы создать модуль `SimpleROP` с использованием шаблона модуля анализатора. Это создает файл `SimpleROPAnalyzer.java` в папке `src/main/java` внутри модуля `SimpleROP`. Получившееся представление Package Explorer показано на рисунке 15-21.

Рисунок 15-21. Записи src/main в Package Explorer для SimpleROP

Шаг 3: построить анализатор

Часть сгенерированного кода `SimpleROPAnalyzer.java` показана на рисунке 15-22. Функции были свернуты, чтобы мы могли видеть все предоставленные методы анализатора. Eclipse будет рекомендовать импорты, если они понадобятся во время разработки кода, поэтому можно сразу перейти к программированию задач, которые нужно выполнить, и добавлять рекомендованные инструкции импорта, когда Eclipse обнаружит их необходимость.

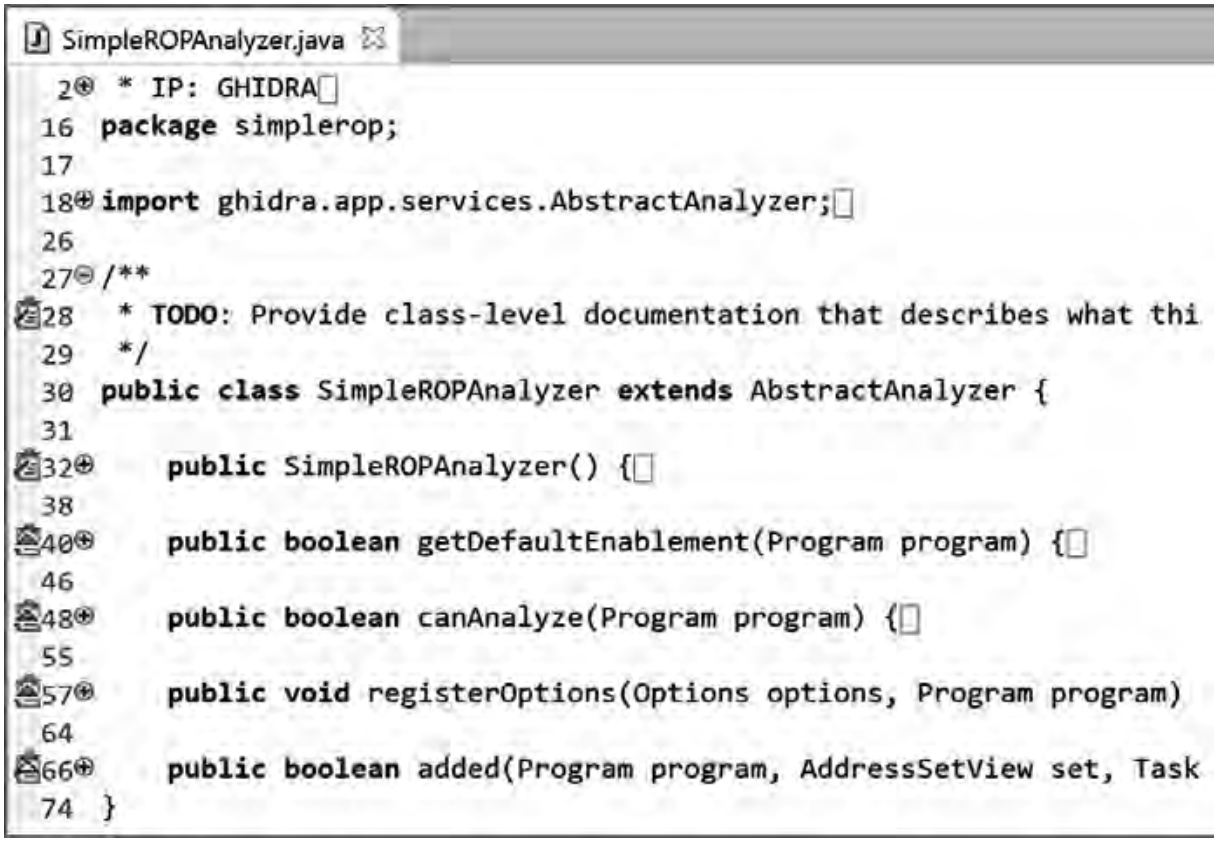


Рисунок 15-22. Шаблон SimpleROPAnalyzer

Шесть тегов задач слева от номеров строк на рисунке 15-22 указывают, с чего начать разработку. Мы раскроем связанные разделы по мере выполнения каждой задачи и включим содержимое до и после, связанное с каждой задачей. Обратите внимание, что часть содержимого будет перенесена или переформатирована ради удобочитаемости, а комментарии минимизированы для экономии места.

Для функциональности мы будем полагаться на следующие объявления уровня класса:

```
private int gadgetCount = 0;         // Подсчитывает количество гаджетов
private BufferedWriter outFile;      // Выходной файл
// Список "интересных" инструкций
private List<String> usefulInstructions = Arrays.asList(
    "NOP", "POP", "PUSH", "MOV", "ADD", "SUB", "MUL", "DIV", "XOR");
// Список "интересных" инструкций без операндов
private List<String> require0Operands = Arrays.asList("NOP");
// Список "интересных" инструкций с одним операндом
private List<String> require1RegOperand = Arrays.asList("POP", "PUSH");
// Список "интересных" инструкций, у которых первый
// параметр должен быть регистром
private List<String> requireFirstRegOperand = Arrays.asList(
     "MOV", "ADD", "SUB", "MUL", "DIV", "XOR");
// Список "стартовых" инструкций с НУЛЕМ операндов
private List<String> startInstr0Params = Arrays.asList("RET");
// Список "стартовых" инструкций с ОДНИМ регистровым операндом
private List<String> startInstr1RegParam = Arrays.asList("JMP", "CALL");
```

Комментарии, связанные с каждым объявлением, описывают назначение каждой переменной. Различные переменные `List` содержат инструкции, из которых будут составляться наши гаджеты, и классифицируют эти инструкции по количеству и типу операндов, которые им требуются, а также по тому, является ли инструкция допустимой начальной инструкцией для одного из наших гаджетов. Поскольку наш алгоритм построения гаджета движется назад по памяти, "start" здесь фактически означает начальную точку для алгоритма. Во время выполнения эти же начальные инструкции были бы последними инструкциями, выполненными в данном гаджете.

Шаг 3-1: документировать класс

Когда мы раскрываем первый тег задачи, видим следующее описание задачи:

```
/**
 * TODO: Добавить документацию уровня класса, описывающую,
 * что делает этот анализатор.
 */
```

Замените существующие комментарии TODO комментариями, описывающими, что делает анализатор:

```
/**
 * Этот анализатор ищет ROP-гаджеты в бинарном файле.
 * Адрес и содержимое каждого гаджета записываются
 * в файл inputfilename_gadgets.txt в домашнем каталоге пользователя.
 */
```

Шаг 3-2: назвать и описать анализатор

Раскрытие следующего тега задачи предоставляет комментарий TODO и строку кода, которую нужно изменить. В Eclipse IDE код, подлежащий изменению, отображается пурпурным шрифтом и имеет имя, указывающее на связанную задачу. Вторая задача содержит следующее:

```
// TODO: Назвать анализатор и дать ему описание.
public SimpleROPAnalyzer() {
   super("Мой анализатор",
         "Здесь находится описание анализатора",
          AnalyzerType.BYTE_ANALYZER);
}
```

Две строки нужно заменить осмысленным содержимым. Кроме того, нужно указать тип анализатора. Чтобы облегчить разрешение зависимостей между анализаторами, Ghidra группирует анализаторы в следующие категории: байт, данные, функция, модификаторы функций, сигнатуры функций и инструкция. В этом случае мы строим анализатор инструкций. Получившийся код выглядит так:

```
public SimpleROPAnalyzer() {
   super("SimpleROP",
         "Искать ROP-гаджеты в бинарном файле",
          AnalyzerType.INSTRUCTION_ANALYZER);
}
```

Шаг 3-3: определить, должен ли анализатор быть анализатором по умолчанию

Третья задача просит вернуть `true`, если анализатор должен быть включен по умолчанию:

```
public boolean getDefaultEnablement(Program program) {
   // TODO: Вернуть истину, если анализатор должен быть включен по умолчанию
   return false;
}
```

Мы не хотим, чтобы этот анализатор был включен по умолчанию; следовательно, никаких изменений кода не требуется.

Шаг 3-4: определить, подходит ли вход для анализатора

Четвертая задача требует определить, совместим ли анализатор с содержимым программы:

```
public boolean canAnalyze(Program program) {
   // TODO: Проверить 'program', чтобы определить, должен ли этот
   // анализатор ее анализировать.
   // Вернуть истину, если может.
   return false;
}
```

Поскольку этот анализатор предназначен только для демонстрации, мы предполагаем, что входной файл совместим с нашим анализом, и просто возвращаем `true`. На практике мы добавили бы код для проверки совместимости файла анализа до использования анализатора. Например, можно было бы возвращать `true` только после определения, что файл является x86-бинарным файлом. Рабочие примеры такой проверки можно найти в большинстве анализаторов, включенных в установку Ghidra (`Ghidra/Features/Base/lib/Base-src/Ghidra/app/analyzers`), доступных через каталог вашего модуля внутри Eclipse:

```
public boolean canAnalyze(Program program) {
   return true;
}
```

Шаг 3-5: зарегистрировать параметры анализатора

Пятая задача дает возможность указать любые специальные параметры, которые мы хотим представить пользователям анализатора:

```
public void registerOptions(Options options, Program program) {
   // TODO: Если у анализатора есть пользовательские параметры, зарегистрировать
    их здесь
   options.registerOption("Здесь находится имя параметра", false, null,
                          "Здесь находится описание параметра");
}
```

Поскольку анализатор только демонстрационный, мы не будем добавлять параметры. Параметры могли бы включать выборы, управляемые пользователем, например выбор выходного файла, необязательное аннотирование листинга и т. д. Параметры каждого анализатора отображаются в окне Analyzer при выборе отдельного анализатора:

```
public void registerOptions(Options options, Program program) {
}
```

Шаг 3-6: выполнить анализ

Шестая задача выделяет функцию, вызываемую при запуске нашего анализатора:

```
public boolean added(Program program, AddressSetView set, TaskMonitor
                     monitor, MessageLog log) throws CancelledException {
   // TODO: Выполнить анализ, когда в 'program' что-то добавляется.
   // Вернуть истину, если анализ выполнен успешно.
   return false;
}
```

Это часть модуля, которая выполняет работу. В этом модуле используются четыре метода, каждый из которых подробно приведен ниже:

```
//**************************************************************************
//  Этот метод вызывается при запуске анализатора.
//**************************************************************************
public boolean added(Program program, AddressSetView set, TaskMonitor
                  monitor, MessageLog log) throws CancelledException {
   gadgetCount = 0;
   String outFileName = System.getProperty("user.home") + "/" +
                        program.getName() + "_gadgets.txt";
   monitor.setMessage("Поиск ROP-гаджетов");
   try {
      outFile = new BufferedWriter(new FileWriter(outFileName));
   } catch (IOException e) {/* pass */}
   // пройти по каждой инструкции в бинарном файле
   Listing code = program.getListing();
   InstructionIterator instructions = code.getInstructions(set, true);
   while (instructions.hasNext() && !monitor.isCancelled()) {
      Instruction inst = instructions.next();
      if (isStartInstruction(inst)) {
         // Найдена "стартовая" инструкция. Это будет последняя
         // инструкция в потенциальном ROP-гаджете, поэтому попробуем
         // построить гаджет отсюда
         ArrayList<Instruction> gadgetInstructions =
            new ArrayList<Instruction>();
         gadgetInstructions.add(inst);
         Instruction prevInstr = inst.getPrevious();
         buildGadget(program, monitor, prevInstr, gadgetInstructions);
      }
   }
   try {
      outFile.close();
   } catch (IOException e) {/* pass */}
   return true;
}
//**************************************************************************
//  Этот метод вызывается рекурсивно, пока не найдет инструкцию,
//  которую мы не хотим включать в ROP-гаджет.
//**************************************************************************
private void buildGadget(Program program, TaskMonitor monitor,
                         Instruction inst,
                         ArrayList<Instruction> gadgetInstructions) {
   if (inst == null || !isUsefulInstruction(inst) ||
      monitor.isCancelled()) {
      return;
   }
   gadgetInstructions.add(inst);
   buildGadget(program, monitor, inst.getPrevious(), gadgetInstructions);
   gadgetCount += 1;
   for (int ii = gadgetInstructions.size() - 1; ii >= 0; ii--) {
      try {
         Instruction insn = gadgetInstructions.get(ii);
         if (ii == gadgetInstructions.size() - 1) {
            outFile.write(insn.getMinAddress() + ";");
         }
         outFile.write(insn.toString() + ";");
      } catch (IOException e) {/* pass */}
   }
   try {
      outFile.write("\n");
   } catch (IOException e) {/* pass */}
   // Сообщать счетчик монитору на каждом сотом гаджете
   if (gadgetCount % 100 == 0) {
      monitor.setMessage("Найдено ROP-гаджетов: " + gadgetCount);
   }
   gadgetInstructions.remove(gadgetInstructions.size() - 1);
}
//**************************************************************************
//  Этот метод определяет, полезна ли инструкция в контексте
//  ROP-гаджета
```

```
//***************************************************************************
private boolean isUsefulInstruction(Instruction inst) {
   if (!usefulInstructions.contains(inst.getMnemonicString())) {
      return false;
   }
   if (require0Operands.contains(inst.getMnemonicString())) {
      return true;
   }
   if (require1RegOperand.contains(inst.getMnemonicString()) &&
      inst.getNumOperands() == 1) {
      Object[] opObjects0 = inst.getOpObjects(0);
      for (int ii = 0; ii < opObjects0.length; ii++) {
         if (opObjects0[ii] instanceof Register) {
            return true;
         }
      }
   }
   if (requireFirstRegOperand.contains(inst.getMnemonicString()) &&
      inst.getNumOperands() >= 1) {
      Object[] opObjects0 = inst.getOpObjects(0);
      for (int ii = 0; ii < opObjects0.length; ii++) {
         if (opObjects0[ii] instanceof Register) {
            return true;
         }
      }
   }
   return false;
}
//***************************************************************************
//  Этот метод определяет, является ли инструкция "стартом"
//  потенциального ROP-гаджета
//***************************************************************************
private boolean isStartInstruction(Instruction inst) {
   if (startInstr0Params.contains(inst.getMnemonicString())) {
      return true;
   }
   if (startInstr1RegParam.contains(inst.getMnemonicString()) &&
      inst.getNumOperands() >= 1) {
      Object[] opObjects0 = inst.getOpObjects(0);
      for (int ii = 0; ii < opObjects0.length; ii++) {
         if (opObjects0[ii] instanceof Register) {
            return true;
         }
      }
   }
   return false;
}
```

Ghidra вызывает метод `added` анализатора, чтобы начать анализ. Наш алгоритм проверяет каждую инструкцию в бинарном файле, чтобы определить, является ли инструкция допустимой начальной точкой для построителя гаджетов. Каждый раз, когда находится допустимая начальная инструкция, вызывается наша функция создания гаджета `buildGadget`. Создание гаджета - это рекурсивный проход назад по списку инструкций, который продолжается, пока инструкция считается для нас полезной. Наконец, каждый гаджет печатается после завершения путем итерации по его инструкциям.

Шаг 4: протестировать анализатор внутри Eclipse

Во время процесса разработки принято часто тестировать и изменять код. Пока вы строите анализатор, его функциональность можно тестировать внутри Eclipse с помощью параметра Run As и выбора Ghidra. Это открывает Ghidra с временно установленной текущей версией модуля. Если при тестировании модуля результаты не те, которых вы ожидали, можно отредактировать файл в Eclipse и протестировать повторно. Когда результат устраивает, переходите к шагу 5. Использование этого метода для тестирования кода внутри Eclipse может сильно сэкономить время во время процесса разработки.

Шаг 5: добавить анализатор в установку Ghidra

Чтобы добавить анализатор в установку Ghidra, нужно экспортировать модуль из Eclipse, а затем установить расширение в Ghidra. Экспорт выполняется выбором GhidraDev -> Export -> Ghidra Module Extension, выбором модуля и нажатием Next. В следующем окне выберите параметр Gradle Wrapper, показанный на рисунке 15-23, если у вас нет локальной установки Gradle; обратите внимание, что требуется подключение к интернету, чтобы wrapper мог обратиться к `gradle.org`. Нажмите `Finish`, чтобы завершить процесс экспорта. Если вы впервые экспортируете модуль, каталог `dist` будет добавлен в модуль внутри Eclipse, а файл `.zip` экспортированного содержимого будет сохранен в папке.

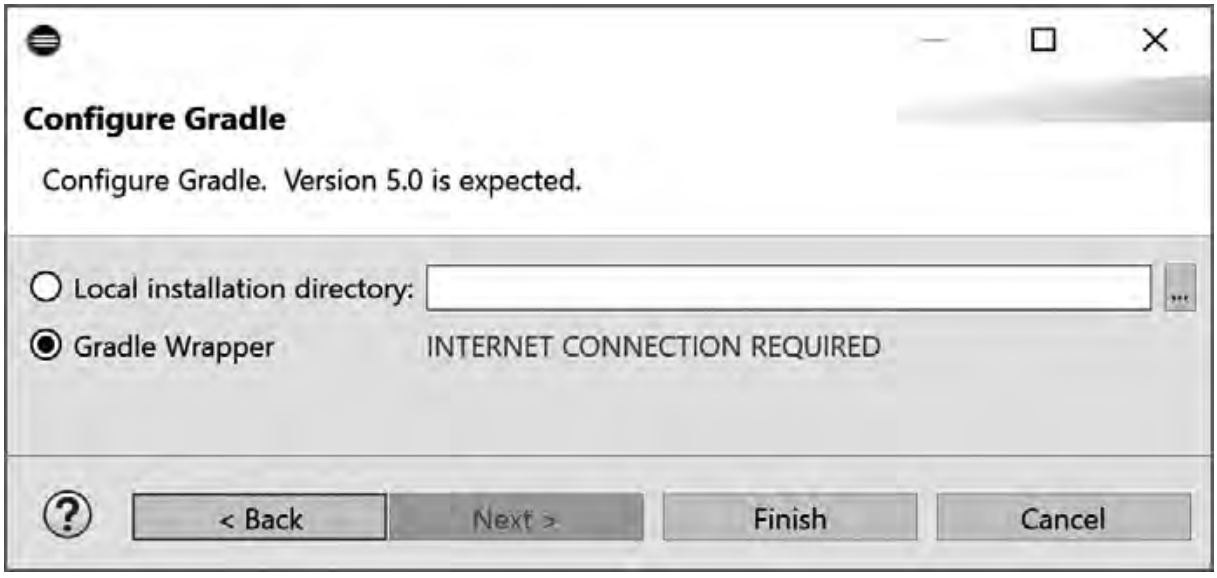


Рисунок 15-23. Диалог Configure Gradle

В окне `Ghidra Project` добавьте новый анализатор, выбрав File -> Install Extensions. Будет отображено окно, похожее на показанное на рисунке 15-24, со всеми существующими расширениями, которые еще не установлены.

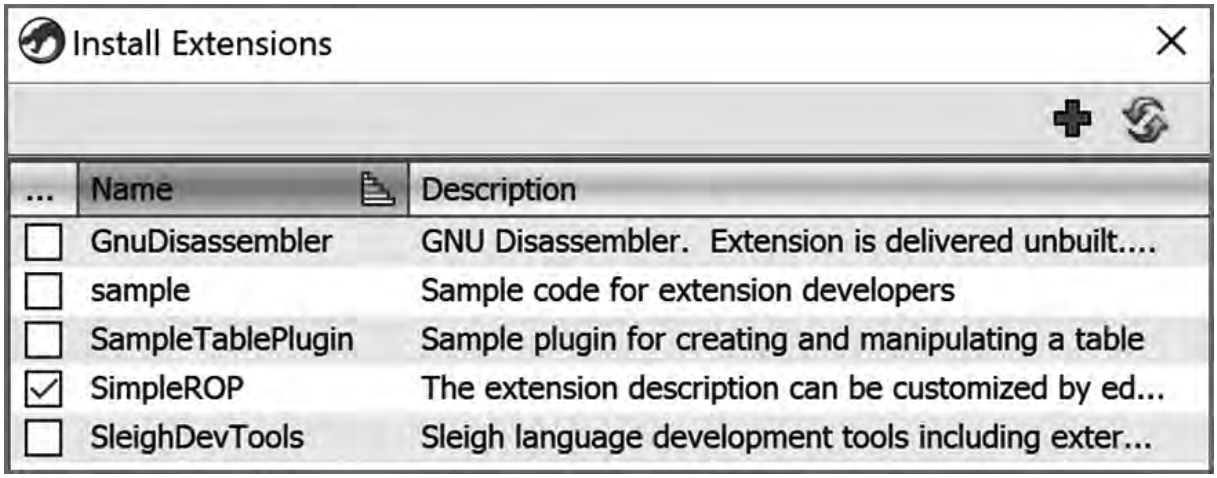


Рисунок 15-24. Окно Install Extensions

Добавьте новый анализатор `SimpleROP`, выбрав значок + в правом верхнем углу и перейдя к недавно созданному файлу `.zip` в связанном каталоге `dist`. Когда анализатор появится в списке, можно выбрать его и нажать `OK` (не показано). Перезапустите Ghidra, чтобы использовать новую функциональность из меню `Analysis`.

Шаг 6: протестировать анализатор внутри Ghidra

Как и с ограниченным планом разработки, мы использовали план тестирования ограниченного объема только для демонстрации функциональности. `SimpleROP` прошел приемочное тестирование, поскольку анализатор соответствовал следующим критериям:

1. (`Pass`) `SimpleROP` появляется в `Analysis Options` в меню `CodeBrowser -> Analysis`.
2. (`Pass`) Описание `SimpleROP` появляется в окне описания `Analysis Options` при выборе.

Тестовые случаи 1 и 2 прошли, как показано на рисунке 15-25. Если бы мы решили зарегистрировать и запрограммировать связанные параметры на шаге 3-5, они были бы отображены в панели `Options` справа.

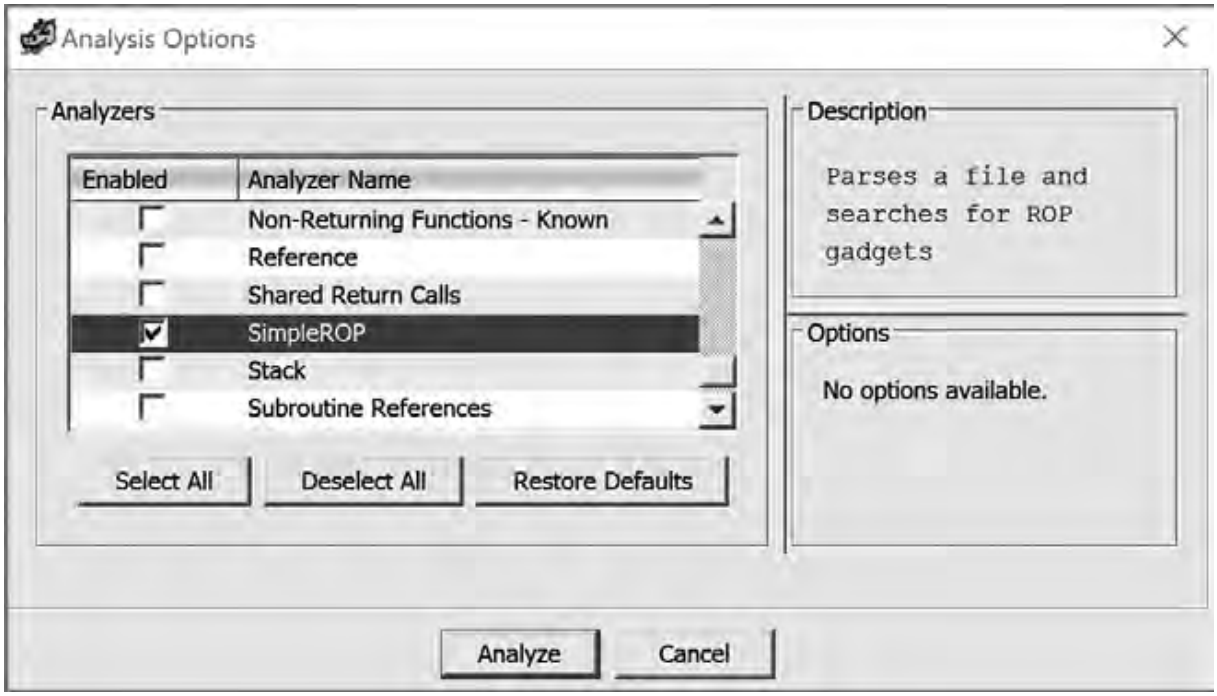


Рисунок 15-25. Окно `Analysis Options`

3. (`Pass`) `SimpleROP` выполняется при выборе.

В этом случае мы запускали `SimpleROP` на проанализированном файле и как часть автоматического анализа. Запуск `SimpleROP` на непроанализированном файле не дал бы результатов, поскольку расширения `INSTRUCTION_ANALYZER` требуют, чтобы инструкции были предварительно идентифицированы, что является частью автоматического анализа по умолчанию. Когда `SimpleROP` запускается как часть автоматического анализа, он приоритизируется корректно благодаря типу анализатора, назначенному на шаге 3-2. Рисунок 15-26 показывает подтверждение Ghidra Log, что анализатор `SimpleROP` был выполнен.

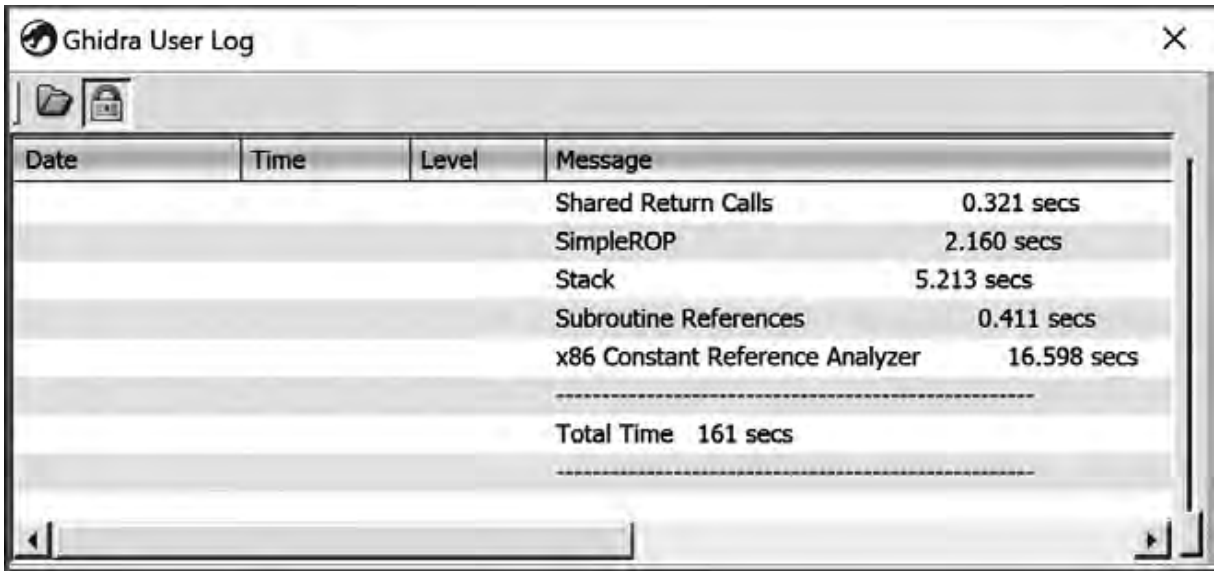


Рисунок 15-26. Окно Ghidra User Log с подтверждением анализа

4. (`Pass`) `SimpleROP` записывает каждый гаджет в файл с именем `fileZZZ_gadgets.txt` при анализе `fileZZZ`.

Следующий фрагмент из файла `call_tree_x64_static_gadgets.txt` показывает, что многие гаджеты взяты из части листинга `call_tree_x64_static`, показанной на рисунке 15-27:

```
00400412;ADD RSP,0x8;RET;
004004ce;NOP;RET;
00400679;ADD RSP,0x8;POP RBX;POP RBP;POP R12;POP R13;POP R14;POP R15;RET;
0040067d;POP RBX;POP RBP;POP R12;POP R13;POP R14;POP R15;RET;
0040067e;POP RBP;POP R12;POP R13;POP R14;POP R15;RET;
0040067f;POP R12;POP R13;POP R14;POP R15;RET;
00400681;POP R13;POP R14;POP R15;RET;
00400683;POP R14;POP R15;RET;
00400685;POP R15;RET;
00400a8b;POP RBP;MOV EDI,0x6babd0;JMP RAX;
00400a8c;MOV EDI,0x6babd0;JMP RAX;
00400a98;POP RBP;RET;
```

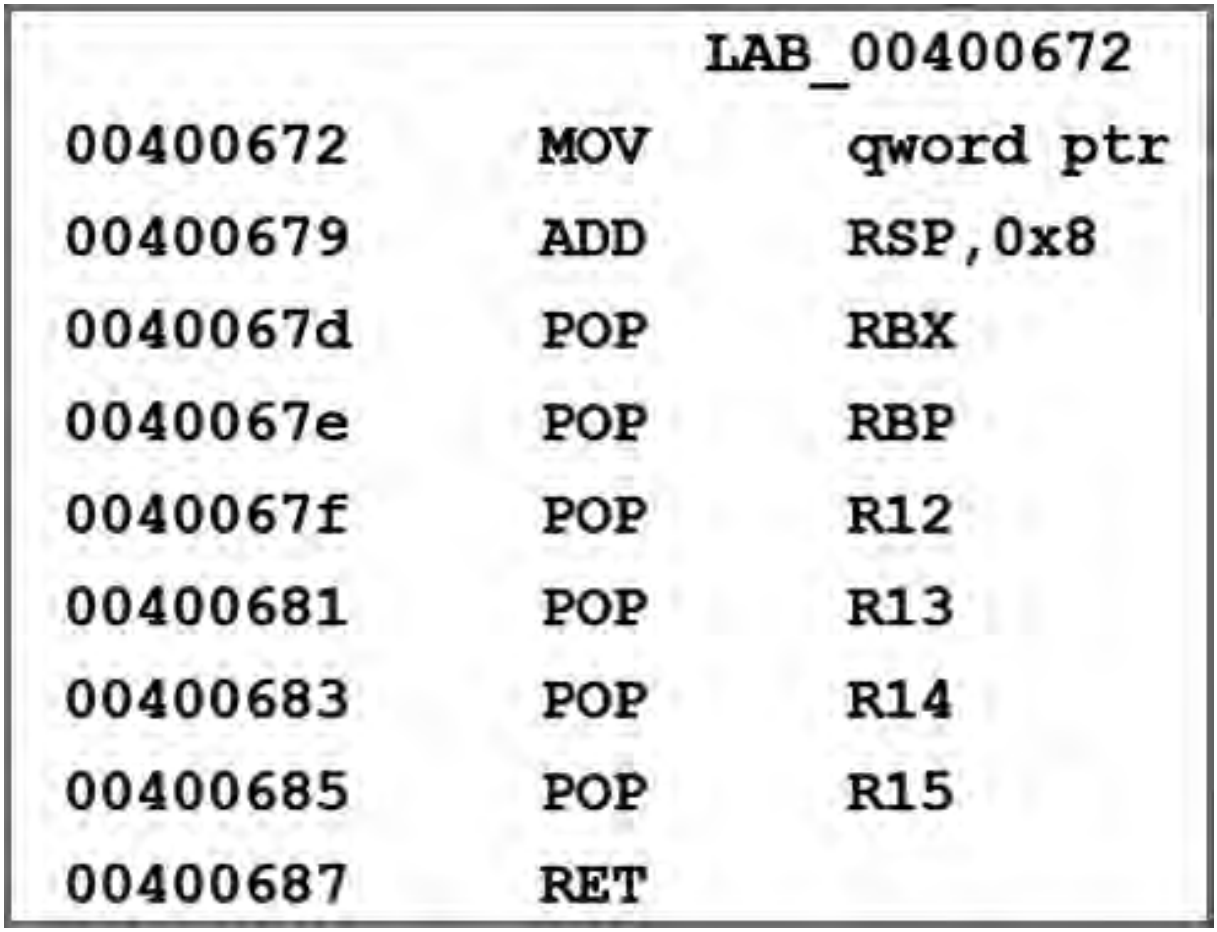


Рисунок 15-27. Листинг `CodeBrowser` *для* `call_tree_x64_static`

Резюме

В главе 14 мы представили сценарии как способ расширения возможностей Ghidra. В этой главе мы представили модули расширений Ghidra вместе с возможностями интеграции Eclipse в Ghidra. Хотя Eclipse не является единственным вариантом редактирования расширений Ghidra, интеграция Ghidra и Eclipse IDE предоставляет невероятно мощную среду для разработчиков, расширяющих возможности Ghidra. Мастера разработки и шаблоны снижают порог создания расширений, поскольку дают программистам направляемый подход к изменению существующего содержимого и построению новых расширений. В главе 16 мы рассмотрим Ghidra без графического интерфейса - вариант, появившийся на рисунке 15-18. Последующие главы опираются на интеграцию Ghidra и Eclipse IDE, чтобы еще больше расширить возможности Ghidra и предоставить прочную основу для превращения Ghidra в оптимальный инструмент для вашего рабочего процесса обратной разработки.

Сноски

[1] [назад] Package Explorer существует уже давно, тогда как модули являются более поздним дополнением к Java. В конфигурации по умолчанию Package Explorer можно воспринимать как обозреватель проекта для Java-проекта, который вы создали или импортировали.
[2] [назад] Расположение сценария внутри Eclipse Package Explorer и каталога `Ghidra/Features` не обязательно совпадает с организацией сценариев внутри `Script Manager`. Это ожидаемо, поскольку сценарии внутри каталога `Ghidra/Features` организованы по папкам, имеющим общую функциональность. Организация сценариев внутри `Ghidra Script Manager` основана на метаданных категории внутри каждого сценария.

Глава 16. Ghidra в режиме без графического интерфейса

В предыдущих главах мы сосредоточились на исследовании одного файла внутри одного проекта при помощи Ghidra с графическим интерфейсом. Помимо графического интерфейса, у Ghidra есть интерфейс командной строки, называемый анализатором без графического интерфейса Ghidra. Этот анализатор предоставляет часть тех же возможностей, что и Ghidra с графическим интерфейсом, включая способность работать с проектами и файлами, но лучше подходит для пакетной обработки и сценарного управления Ghidra. В этой главе мы обсудим режим Ghidra без графического интерфейса и то, как он может помочь выполнять повторяющиеся задачи над большим количеством файлов. Мы начнем со знакомого примера, а затем расширим обсуждение до более сложных параметров.

Начало работы

Начнем с воспоминания о нашем первом использовании Ghidra в главе 4. Мы успешно выполнили следующие шаги:

1. Запустить Ghidra.
2. Создать новый проект Ghidra.
3. Определить местоположение проекта.
4. Импортировать файл в проект.
5. Автоматически проанализировать файл.
6. Сохранить и выйти.

Повторим эти задачи с помощью интерфейса командной строки Ghidra, анализатора без графического интерфейса. Анализатор без графического интерфейса (`analyzeHeadless` или `analyzeHeadless.bat`), а также полезный файл `analyzeHeadlessREADME.html` находятся в каталоге `support` вашей установки Ghidra. Чтобы упростить пути к файлам, мы временно поместили файл `global_array_demo_x64` в тот же каталог. Сначала определим команды и параметры, необходимые для каждой отдельной задачи, а затем соберем их вместе, чтобы достичь цели. В предыдущих главах это не имело большого значения, но при работе из командной строки различия между тремя платформами Ghidra становятся заметнее. В наших примерах мы используем установку Windows и отмечаем существенные отличия на других платформах.

Прямой или обратный слэш?

Существенное различие между платформами операционных систем, поддерживающими Ghidra, - способ задания путей файловой системы. Хотя синтаксис согласован, разные платформы используют разные разделители каталогов. Windows использует обратный слэш, а Linux и macOS - прямой слэш. В Windows путь выглядит так:

```
D:\Ghidra Projects\ch16\demo_stackframe_32
```

А в Linux и macOS так:

```
/Ghidra Projects/ch16/demo_stackframe_32
```

Для пользователей Windows этот синтаксис может быть еще более запутанным, поскольку прямые слэши используются в URL и ключах командной строки, а также в документации Ghidra. Операционные системы распознают эту проблему и пытаются принимать оба варианта, но не всегда предсказуемо. В примерах этой главы мы используем соглашение Windows, чтобы читатели могли насладиться обратной совместимостью с DOS.

Шаг 1: запустить Ghidra

Этот шаг выполняется командой `analyzeHeadless`. Все дополнительные шаги будут выполнены с помощью параметров и опций, связанных с этой командой. Запуск `analyzeHeadless` без параметров отображает сообщение об использовании с синтаксисом и опциями команды, как показано на рисунке 16-1. Чтобы запустить Ghidra, нужно добавить к команде некоторые из этих параметров.

```
Headless Analyzer Usage: analyzeHeadless
          <project_location> <project_name>[/<folder_path>]
            | ghidra://<server>[:<port>]/<repository_name>[/<folder_path>]
          [[-import [<directory>|<file>]+] | [-process [<project_file>]]]
          [-preScript <ScriptName>]
          [-postScript <ScriptName>]
          [-scriptPath "<path1>[;<path2>...]"]
          [-propertiesPath "<path1>[;<path2>...]"]
          [-scriptlog <path to script log file>]
          [-log <path to log file>]
          [-overwrite]
          [-recursive]
          [-readOnly]
          [-deleteProject]
          [-noanalysis]
          [-processor <languageID>]
          [-cspec <compilerSpecID>]
          [-analysisTimeoutPerFile <timeout in seconds>]
          [-keystore <KeystorePath>]
          [-connect <userID>]
          [-p]
          [-commit ["<comment>"]]
          [-okToDelete]
          [-max-cpu <max cpu cores to use>]
          [-loader <desired loader name>]

Please refer to 'analyzeHeadlessREADME.html' for detailed usage examples and notes.
```

Рисунок 16-1. Синтаксис анализатора без графического интерфейса

Шаги 2 и 3: создать новый проект Ghidra в указанном местоположении

В режиме без графического интерфейса Ghidra создает проект за вас, если проект еще не существует. Если проект уже существует в указанном местоположении, Ghidra открывает существующий проект. Поэтому требуются два параметра: местоположение проекта и имя проекта. Следующая команда создает проект с именем `CH16` в нашем каталоге `D:\Ghidra Projects`:

```
analyzeHeadless D:\Ghidra Projects CH16
```

Это минимальный запуск Ghidra без графического интерфейса для открытия проекта, и он не делает ничего сверх этого. Более того, ответное сообщение от Ghidra говорит ровно это:

```
Nothing to do...must specify -import, -process, or prescript and/or postscript.
```

Шаг 4: импортировать файл в проект

Чтобы импортировать файл, Ghidra требует опцию `-import` и имя импортируемого файла. Мы импортируем `global_array_demo_x64`, который уже использовали раньше. Как упоминалось, ради простоты в этом начальном примере мы поместили файл в каталог `support`. В качестве альтернативы можно было бы указать полный путь к файлу в командной строке. Добавим опцию `-import` к нашей команде:

```
analyzeHeadless D:\Ghidra Projects CH16 -import global_array_demo_x64
```

Шаги 5 и 6: автоматически проанализировать файл, сохранить и выйти

В режиме без графического интерфейса автоматический анализ и сохранение выполняются по умолчанию, поэтому команда из шага 4 делает все, что нам нужно. Чтобы не анализировать файл, требуется опция `-noanalysis`, а для управления тем, как сохраняются проект и связанные файлы, доступны дополнительные опции.

Вот завершенная команда для выполнения всех шести целей:

```
analyzeHeadless D:\Ghidra Projects CH16 -import global_array_demo_x64
```

Как и со многими консольными командами, вы можете спросить себя: "Как убедиться, что что-то произошло?" Первый признак успеха или отказа - сообщения, отображаемые в консоли. Информационные сообщения с префиксом `INFO` дают отчеты о ходе работы, пока анализатор без графического интерфейса начинает работу. Сообщения об ошибках начинаются с префикса `ERROR`. Листинг 16-1 включает подмножество сообщений, включая сообщение об ошибке.

```
INFO  HEADLESS Script Paths:
   C:\Users\Ghidrabook\ghidra_scripts
   D:\ghidra_PUBLIC\Ghidra\Extensions\SimpleROP\ghidra_scripts
   D:\ghidra_PUBLIC\Ghidra\Features\Base\ghidra_scripts
   D:\ghidra_PUBLIC\Ghidra\Features\BytePatterns\ghidra_scripts
   D:\ghidra_PUBLIC\Ghidra\Features\Decompiler\ghidra_scripts
   D:\ghidra_PUBLIC\Ghidra\Features\FileFormats\ghidra_scripts
   D:\ghidra_PUBLIC\Ghidra\Features\FunctionID\ghidra_scripts
   D:\ghidra_PUBLIC\Ghidra\Features\GnuDemangler\ghidra_scripts
   D:\ghidra_PUBLIC\Ghidra\Features\Python\ghidra_scripts
   D:\ghidra_PUBLIC\Ghidra\Features\VersionTracking\ghidra_scripts
   D:\ghidra_PUBLIC\Ghidra\Processors\8051\ghidra_scripts
   D:\ghidra_PUBLIC\Ghidra\Processors\DATA\ghidra_scripts
   D:\ghidra_PUBLIC\Ghidra\Processors\PIC\ghidra_scripts(HeadlessAnalyzer)
INFO  HEADLESS: execution starts (HeadlessAnalyzer)
INFO  Opening existing project: D:\Ghidra Projects\CH16 (HeadlessAnalyzer)
ERROR Abort due to Headless analyzer error:
   ghidra.framework.store.LockException:
```

```
Unable to lock project! D:\Ghidra Projects\CH16 (HeadlessAnalyzer)
java.io.IOException: ghidra.framework.store.LockException:
Unable to lock project! D:\Ghidra Projects\CH16
...
```

Листинг 16-1. Анализатор без графического интерфейса с условием ошибки

Пути сценариев, используемые в режиме без графического интерфейса, перечислены в начале. Позднее в главе мы покажем, как использовать дополнительные сценарии с командами режима без графического интерфейса. Расширение, созданное нами в предыдущей главе, `SimpleROP`, включено в путь сценариев, потому что каждое расширение добавляет новый путь в путь сценариев. `LockException` - пожалуй, самая распространенная ошибка, связанная с анализатором без графического интерфейса. Анализатор без графического интерфейса завершается отказом, если вы пытаетесь запустить его на проекте, который уже открыт в другом экземпляре Ghidra. Когда это происходит, анализатор без графического интерфейса не может заблокировать проект для собственного исключительного использования, и команда завершается неудачей.

Чтобы исправить ошибку, закройте любой работающий экземпляр Ghidra, где открыт проект `CH16`, и запустите команду снова. Рисунок 16-2 показывает хвост вывода для успешного выполнения нашей команды, похожий на всплывающие окна, которые мы видим при анализе файлов в Ghidra с графическим интерфейсом.

```
INFO  ----------------------------------------------------
    ASCII Strings                                  0.883 secs
    Apply Data Archives                            0.590 secs
    Call Convention Identification                 0.137 secs
    Call-Fixup Installer                           0.004 secs
    Create Address Tables                          0.012 secs
    Create Function                                0.005 secs
    DWARF                                          0.020 secs
    Data Reference                                 0.010 secs
    Decompiler Switch Analysis                     0.473 secs
    Demangler GNU                                  0.050 secs
    Disassemble Entry Points                       0.105 secs
    ELF Scalar Operand References                  0.010 secs
    Embedded Media                                 0.014 secs
    External Entry References                      0.001 secs
    Function ID                                    0.051 secs
    Function Start Search                          0.043 secs
    Function Start Search After Code               0.002 secs
    Function Start Search After Data               0.001 secs
    GCC Exception Handlers                         0.076 secs
    Non-Returning Functions - Discovered           0.013 secs
    Non-Returning Functions - Known                0.004 secs
    Reference                                      0.025 secs
    Shared Return Calls                            0.005 secs
    Stack                                          0.054 secs
    Subroutine References                          0.007 secs
    Subroutine References - One Time               0.000 secs
    x86 Constant Reference Analyzer                0.093 secs
----------------------------------------------------------
     Total Time   2 secs
```

Рисунок 16-2. Результаты анализатора без графического интерфейса, отображенные в консоли

Чтобы проверить результаты в Ghidra с графическим интерфейсом, откройте проект и подтвердите, что файл был загружен, как показано на рисунке 16-3, а затем откройте файл в `CodeBrowser`, чтобы подтвердить анализ.

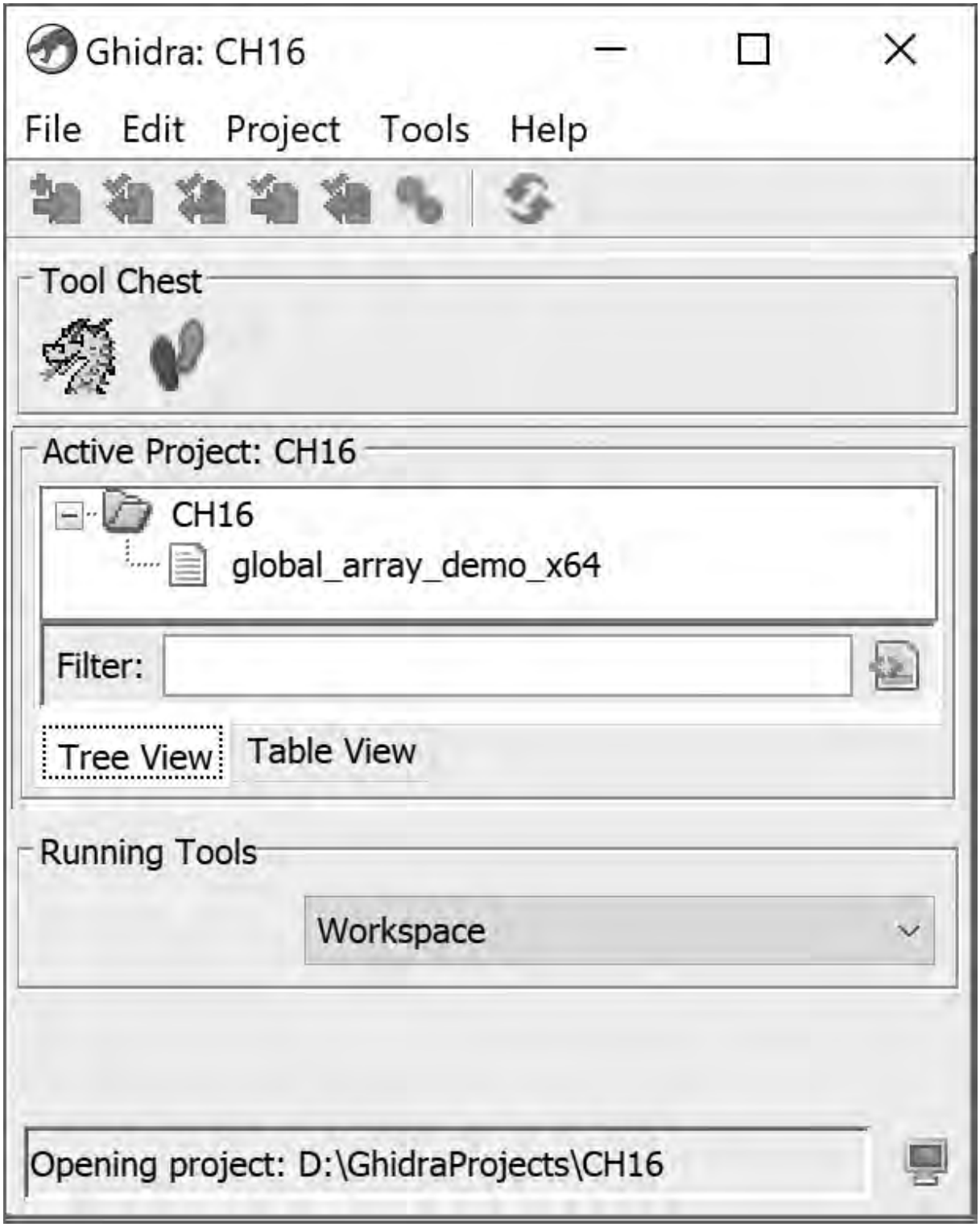


Рисунок 16-3. Подтверждение в Ghidra с графическим интерфейсом, что проект создан и файл загружен

Теперь, когда мы повторили предыдущий анализ с помощью Ghidra в режиме без графического интерфейса, исследуем ситуации, где этот режим имеет преимущество перед графическим интерфейсом. Чтобы создать проект и загрузить и проанализировать все файлы, показанные на рисунке 16-4, с помощью Ghidra с графическим интерфейсом, мы могли бы создать проект и затем загрузить каждый файл отдельно или выбрать файлы для операции пакетного импорта, как обсуждалось в разделе "Пакетный импорт" на странице 226. Ghidra в режиме без графического интерфейса позволяет назвать каталог и проанализировать все содержащиеся в нем файлы.

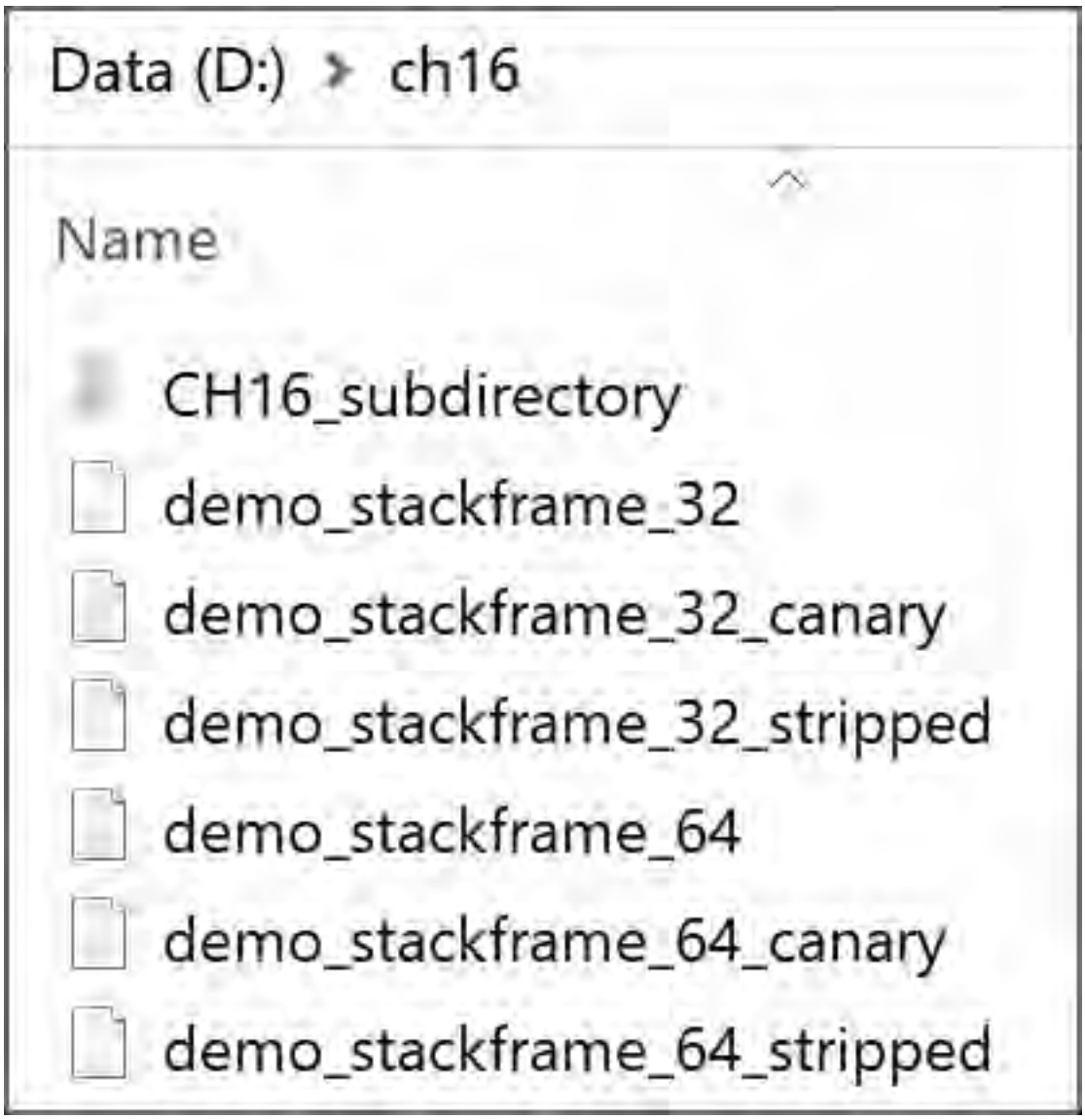


Рисунок 16-4. Входной каталог для примеров Ghidra в режиме без графического интерфейса

Следующая команда говорит анализатору без графического интерфейса открыть или создать проект с именем `CH16` в каталоге `D:\Ghidra Projects` и импортировать и анализировать все файлы в каталоге `D:\ch16`:

```
analyzeHeadless D:\Ghidra Projects CH16 -import D:\ch16
```

После выполнения команды можно загрузить новый проект в Ghidra с графическим интерфейсом и увидеть связанные файлы, как показано на рисунке 16-5. Подкаталог `D:\ch16\CH16_subdirectory` не появляется в проекте, как и файлы внутри подкаталога. Мы вернемся к этому при обсуждении дополнительных опций и параметров, которые можно использовать с Ghidra в режиме без графического интерфейса в следующем разделе.

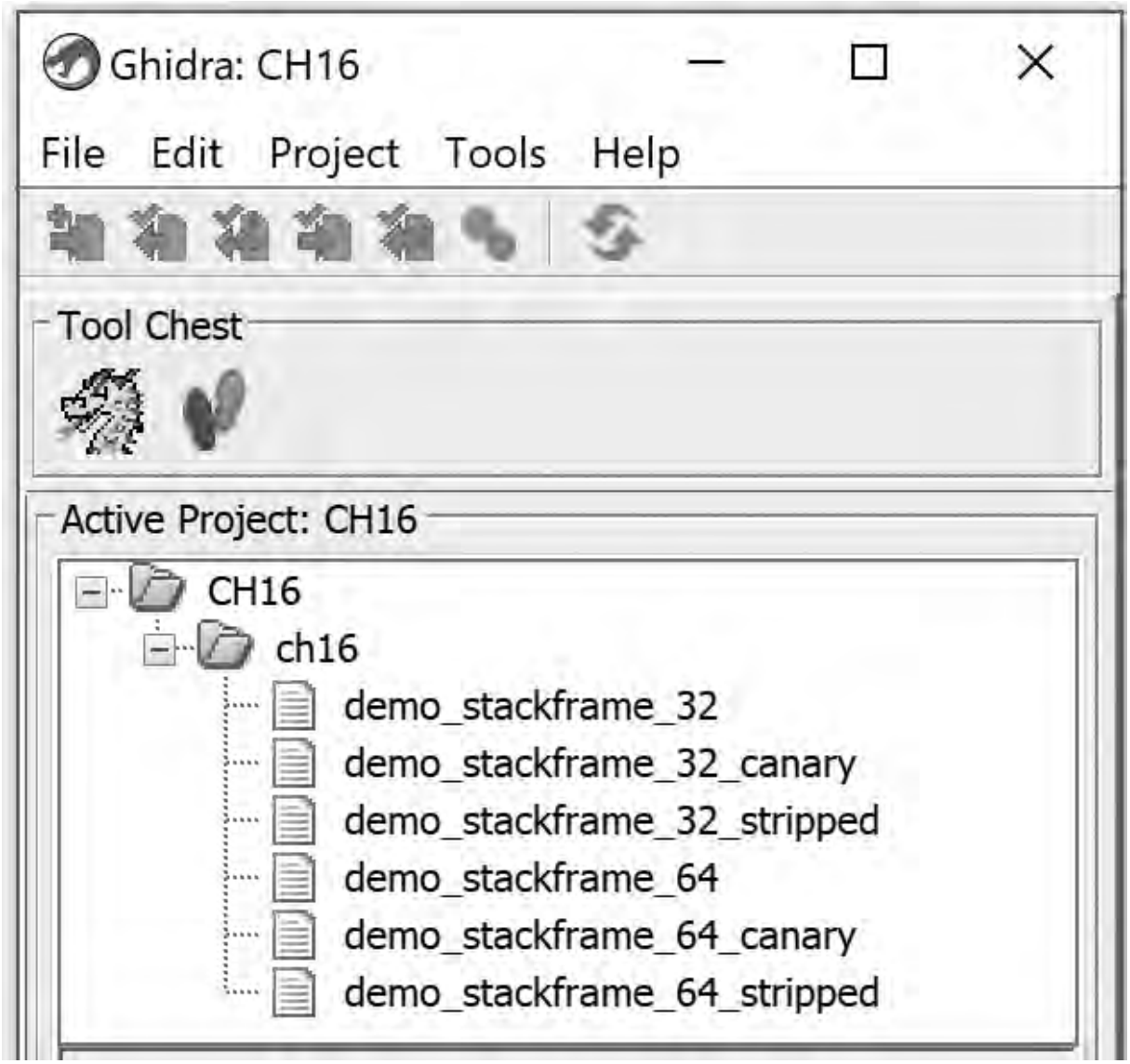


Рисунок 16-5. Проект, полученный при указании каталога для Ghidra в режиме без графического интерфейса

Опции и параметры

Простые примеры использования Ghidra в режиме без графического интерфейса для создания проекта, загрузки и анализа одиночного файла и использования пакетной обработки для импорта всего каталога - это только начало возможного. Хотя мы не сможем обсудить все возможности Ghidra в этом режиме, дадим краткое введение в каждую из доступных на данный момент опций.

Общие опции

Ниже приведены краткие описания со связанными примерами дополнительных опций, которые можно использовать для более точного управления тем, что происходит в наших простых примерах. Перенесенные строки имеют отступ. Там, где встречаются распространенные условия ошибок, они обсуждаются. Специализированные условия ошибок оставлены читателю как упражнение в уюте файла `Ghidra Help`.

`-log logfilepath`

Многое может пойти не так или, наоборот, правильно при выполнении из командной строки. К счастью, плагины Ghidra дают непрерывную обратную связь о происходящем во время работы Ghidra. Хотя в Ghidra с графическим интерфейсом эта обратная связь менее жизненно важна, потому что есть визуальные подсказки, в режиме без графического интерфейса она важна.

По умолчанию файл журнала записывается в `.ghidra/.ghidra_<VER>_PUBLIC/application.log` в домашнем каталоге пользователя. Можно выбрать новое местоположение, добавив опцию `-log` в командную строку. Чтобы создать каталог `CH16-logs` и записать файл журнала в `CH16-Logfile`, используйте следующую команду:

```
analyzeHeadless D:\Ghidra Projects CH16 -import global_array_demo_x64
  -log D:\Ghidra Projects\CH16-logs\CH16-Logfile
```

`-noanalysis`

Эта опция инструктирует Ghidra не анализировать файлы, импортируемые из командной строки. Открытие файла `global_array_demo_x64` в Ghidra с графическим интерфейсом после выполнения следующего оператора представило бы загруженную, но не проанализированную версию файла внутри проекта `CH16`:

```
analyzeHeadless D:\Ghidra Projects CH16 -import global_array_demo_x64
  -noanalysis
```

`-overwrite`

В листинге 16-1 мы видели условие ошибки, когда Ghidra попыталась открыть проект, который уже был открыт. Вторая распространенная ошибка возникает, когда Ghidra пытается импортировать файл в проект, а файл уже был импортирован. Чтобы импортировать новую версию файла или перезаписать существующий файл независимо от содержимого, используйте опцию `-overwrite`. Без этой опции повторный запуск следующей команды режима без графического интерфейса привел бы к ошибке при втором выполнении. С этой опцией команду можно запускать сколько угодно раз:

```
analyzeHeadless D:\Ghidra Projects CH16 -import global_array_demo_x64
  -overwrite
```

`-readOnly`

Чтобы импортировать файл без сохранения файла в проект, используйте опцию `-readOnly`. Если используется эта опция, опция `-overwrite` будет проигнорирована, если присутствует. Эта опция также имеет смысл при использовании с `-process`, а не с командой `-import`. Опция `-process` рассматривается позднее в главе.

```
analyzeHeadless D:\Ghidra Projects CH16 -import global_array_demo_x64
  -readOnly
```

`-deleteProject`

Эта опция инструктирует Ghidra не сохранять проект, создаваемый с опцией `-import`. Ее можно использовать с любыми другими опциями, но она предполагается, даже если опущена, при использовании `-readOnly`. Недавно созданный проект удаляется после завершения анализа. Эта опция не удалит существующий проект:

```
analyzeHeadless D:\Ghidra Projects CH16 -import global_array_demo_x64
  -deleteProject
```

`-recursive`

По умолчанию Ghidra не рекурсирует в подкаталоги, когда ее просят обработать весь каталог. Используйте эту опцию, если хотите, чтобы Ghidra выполняла рекурсивную обработку каталогов, то есть обрабатывала любые подкаталоги, найденные по пути. Чтобы продемонстрировать эту функциональность, мы снова укажем Ghidra на тот же каталог `ch16`, который обработали ранее, но на этот раз используем опцию `-recursive`:

```
analyzeHeadless D:\Ghidra Projects CH16 -import D:\ch16 -recursive
```

Открытие проекта `CH16` после запуска этой команды дает структуру проекта, показанную на рисунке 16-6. В отличие от рисунка 16-5, `CH16_subdirectory` включен в проект вместе со связанным файлом, а иерархия каталогов сохранена внутри иерархии проекта.

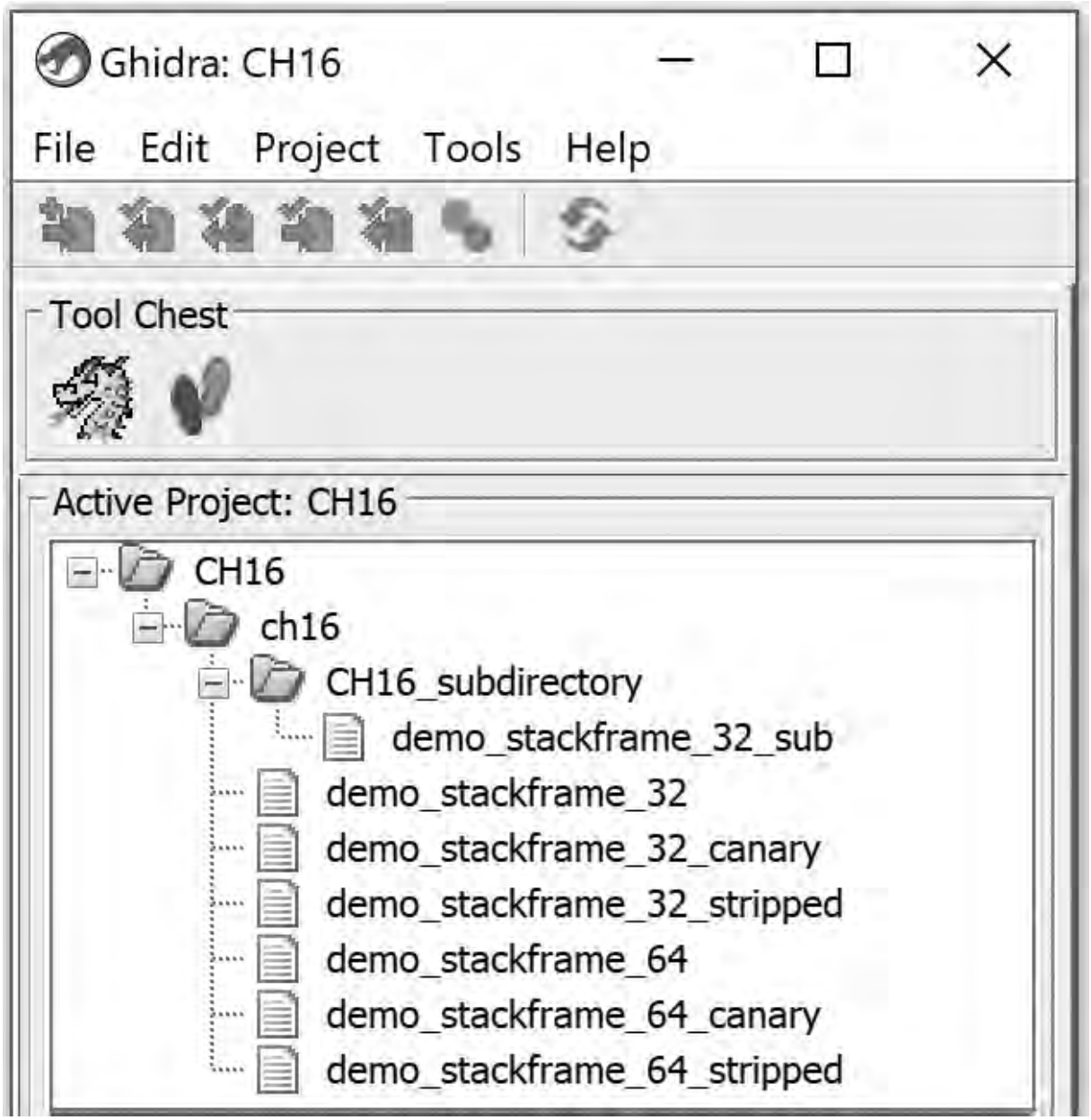


Рисунок 16-6. Проект Ghidra в режиме без графического интерфейса, полученный с опцией `-recursive`

`-analysisTimeoutPerFile seconds`

Анализируя файлы или наблюдая, как Ghidra их анализирует, вы могли заметить несколько факторов, влияющих на время анализа, например размер файла, статически ли он слинкован и параметры анализа декомпилятора. Независимо от содержимого файла и опций, нельзя заранее точно знать, сколько времени займет анализ.

В режиме без графического интерфейса, особенно при обработке большого количества файлов, опция `-analysisTimeoutPerFile` позволяет гарантировать, что задача завершится за разумное время. С этой опцией задается тайм-аут в секундах, и анализ будет прерван при истечении времени. Например, наша существующая команда Ghidra в режиме без графического интерфейса занимает чуть больше одной секунды на анализ в нашей системе (см. рисунок 16-2). Если бы время выполнения сценария было действительно ограничено, следующая команда режима без графического интерфейса остановила бы анализ через одну секунду:

```
analyzeHeadless D:\Ghidra Projects CH16 -import global_array_demo_x64
  -analysisTimeoutPerFile 1
```

Это привело бы к отображению в консоли, показанному на рисунке 16-8.

```
    Non-Returning Functions - Known          0.004 secs
    Reference                                0.025 secs
    Shared Return Calls                      0.004 secs
    Subroutine References                    0.008 secs
    Subroutine References - One Time         0.000 secs
    x86 Constant Reference Analyzer          0.092 secs
-------------------------------------------------------
     Total Time   1 secs
-------------------------------------------------------
 (AutoAnalysisManager)
ERROR REPORT: Analysis timed out at 1 seconds. Processing not completed
 (HeadlessAnalyzer)
```

Рисунок 16-8. Предупреждение консоли о том, что время анализа истекло

Подстановочные знаки!

Подстановочные знаки дают простой способ выбирать несколько файлов для Ghidra в режиме без графического интерфейса, не перечисляя каждый отдельно. Коротко: звездочка (*) соответствует любой последовательности символов, а вопросительный знак (?) соответствует одному символу. Чтобы загрузить и проанализировать только 32-битные файлы с рисунка 16-7, используйте подстановочный знак следующим образом:

```
analyzeHeadless D:\Ghidra Projects CH16 -import D:\ch16\demo_stackframe_32*
```

Это создает проект `CH16` и загружает и анализирует все 32-битные файлы в каталоге `ch16`. Получившийся проект показан на рисунке 16-7. Подробную информацию об использовании подстановочных знаков для указания файлов для импорта и обработки см. в `analyzeHeadlessREADME.html`. Вы также увидите подстановочные знаки в будущих примерах сценариев для Ghidra в режиме без графического интерфейса.

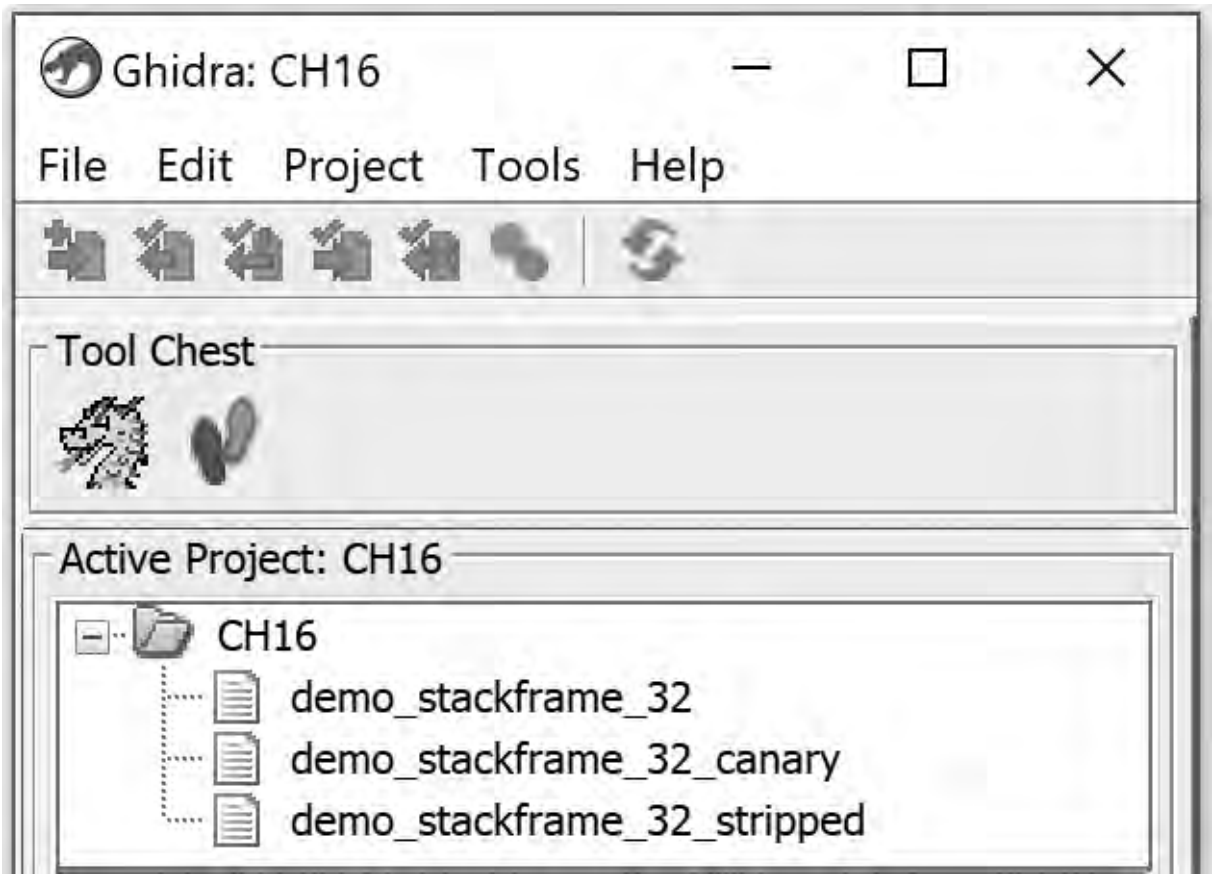


Рисунок 16-7. Файлы проекта, полученные из подстановочного знака `demo_stackframe_32*`

`-processor languageID` **и** `-cspec compilerSpecID`

Как показано в предыдущих примерах, Ghidra обычно довольно хорошо идентифицирует информацию о файле и делает рекомендации по импорту. Пример окна с рекомендациями для конкретного файла показан на рисунке 16-9. Это окно отображается каждый раз, когда вы используете графический интерфейс для импорта файла в проект.

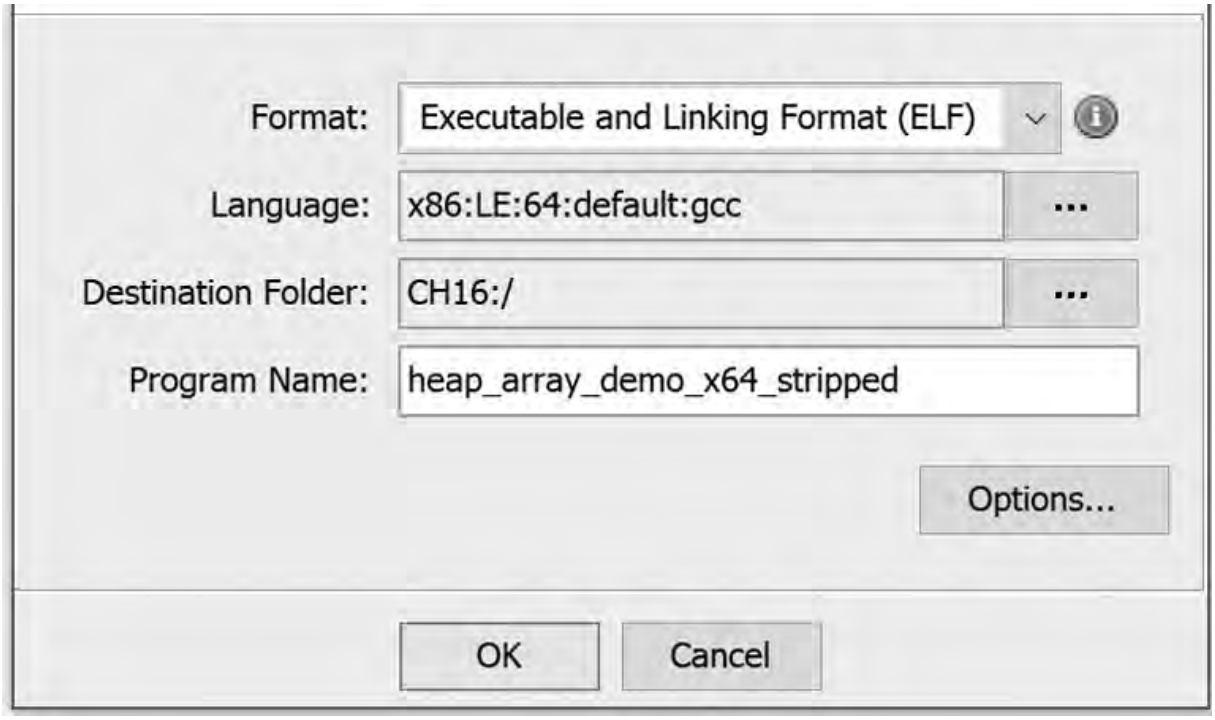


Рисунок 16-9. Диалог подтверждения импорта в Ghidra с графическим интерфейсом

Если вы считаете, что лучше понимаете подходящие язык или компилятор, можно раскрыть поле справа от `Language specification`. Это покажет окно, приведенное на рисунке 16-10, где можно выбрать спецификацию языка и компилятора.

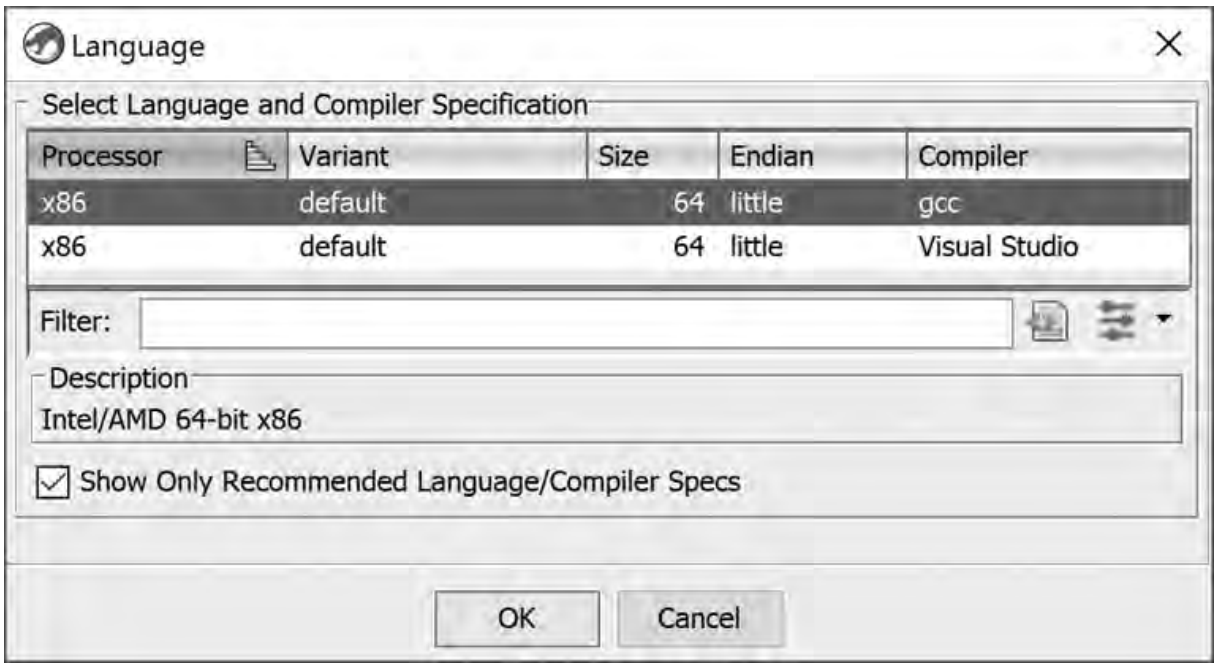


Рисунок 16-10. Окно выбора спецификации языка/компилятора Ghidra

Чтобы сделать то же в Ghidra в режиме без графического интерфейса, используйте опции `-cspec` и/или `-processor`, как показано ниже. Нельзя использовать опцию `-cspec` без опции `-processor`. Можно использовать опцию `-processor` без опции `-cspec`; в этом случае Ghidra выберет компилятор по умолчанию, связанный с процессором.

```
analyzeHeadless D:\Ghidra Projects CH16 -import global_array_demo_x64
  -processor "x86:LE:64:default" -cspec "gcc"
```

`-loader loadername`

Опция `-loader` может быть самой сложной из опций Ghidra в режиме без графического интерфейса. Аргумент `loadername` задает один из модулей загрузчиков Ghidra (обсуждаются в главе 17), который будет использован для импорта нового файла в именованный проект. Примеры имен загрузчиков включают `PeLoader`, `ElfLoader` и `MachoLoader`. Каждый модуль загрузчика может распознавать собственные дополнительные аргументы командной строки. Эти дополнительные аргументы обсуждаются в `support/analyzeHeadlessREADME.html`.

`-max-cpu number`

Эта опция позволяет поставить верхний предел на количество процессорных ядер (CPU), используемых для обработки команды Ghidra в режиме без графического интерфейса. Опция требует целочисленное значение как аргумент. Если значение меньше 1, максимальное количество ядер устанавливается в 1.

```
analyzeHeadless D:\Ghidra Projects CH16 -import global_array_demo_x64
  -max-cpu 5
```

Серверные опции

Некоторые команды используются только при взаимодействии с Ghidra Server. Поскольку это не фокус этой книги, мы упомянем эти команды лишь кратко. Дополнительную информацию можно найти в `analyzeHeadlessREADME.html`.

`ghidra://server[:port]/repository_name[/folder_path]`

Предыдущие примеры все задавали местоположение проекта или имя проекта. Эта альтернатива позволяет указать репозиторий Ghidra Server и необязательный путь к папке.

`-p`

С Ghidra Server эта опция принудительно вызывает приглашение пароля через консоль.

`-connect [userID]`

Эта опция предоставляет `userID`, чтобы переопределить `userID` по умолчанию при подключении к Ghidra Server.

`-keystore path`

Эта опция позволяет указать закрытый файл хранилища ключей при использовании аутентификации PKI или SSH.

`-commit ["comment"]`

Хотя фиксация включена по умолчанию, эта опция позволяет связать комментарий с фиксацией.

Опции сценариев

Пожалуй, самые мощные применения Ghidra в режиме без графического интерфейса связаны со сценарными возможностями Ghidra. Главы 14 и 15 показали, как сценарии можно создавать и использовать с Ghidra с графическим интерфейсом. После представления опций сценариев мы продемонстрируем, насколько мощной может быть Ghidra в режиме без графического интерфейса в сценарном контексте.

`-process [project_file]`

Эта опция обрабатывает выбранные файлы, в отличие от их импорта. Если файл не указан, будут обработаны все файлы в папке проекта. Все указанные файлы также будут проанализированы, если не использовать опцию `-noanalysis`. Ghidra принимает два подстановочных символа (`*` и `?`) для опции `-process`, чтобы упростить выбор нескольких файлов. Для этой опции, в отличие от `-import`, вы называете импортированные файлы проекта Ghidra, а не локальные файлы файловой системы, поэтому нужно заключать имена файлов, содержащие эти подстановочные символы, в кавычки, чтобы оболочка не раскрыла их преждевременно.

`-scriptPath "path1[;path2...]"`

По умолчанию Ghidra в режиме без графического интерфейса включает много путей сценариев по умолчанию, а также пути сценариев для импортированных расширений, как видно в листинге 16-1. Чтобы расширить список путей, в которых Ghidra ищет доступные сценарии, используйте опцию `-scriptPath`, требующую аргумент со списком путей в кавычках. Внутри кавычек несколько путей должны быть разделены точкой с запятой. В компонентах пути распознаются два специальных префиксных обозначения: `$GHIDRA_HOME` и `$USER_HOME`. `$GHIDRA_HOME` относится к каталогу установки Ghidra, а `$USER_HOME` - к домашнему каталогу пользователя. Обратите внимание, что это не переменные окружения, и командная оболочка может потребовать экранировать начальный `$`, чтобы он был передан Ghidra. Следующий пример добавляет каталог `D:\GhidraScripts` в путь сценариев:

```
analyzeHeadless D:\Ghidra Projects CH16 -import global_array_demo_x64
  -scriptPath "D:\GhidraScripts"
```

После запуска команды новый каталог сценариев, `D:\GhidraScripts`, включается в путь сценариев:

```
INFO  HEADLESS Script Paths:
  D:\GhidraScripts
  C:\Users\Ghidrabook\ghidra_scripts
  D:\ghidra_PUBLIC\Ghidra\Extensions\SimpleROP\ghidra_scripts
  D:\ghidra_PUBLIC\Ghidra\Features\Base\ghidra_scripts
  D:\ghidra_PUBLIC\Ghidra\Features\BytePatterns\ghidra_scripts
  D:\ghidra_PUBLIC\Ghidra\Features\Decompiler\ghidra_scripts
  D:\ghidra_PUBLIC\Ghidra\Features\FileFormats\ghidra_scripts
  D:\ghidra_PUBLIC\Ghidra\Features\FunctionID\ghidra_scripts
  D:\ghidra_PUBLIC\Ghidra\Features\GnuDemangler\ghidra_scripts
  D:\ghidra_PUBLIC\Ghidra\Features\Python\ghidra_scripts
  D:\ghidra_PUBLIC\Ghidra\Features\VersionTracking\ghidra_scripts
  D:\ghidra_PUBLIC\Ghidra\Processors\8051\ghidra_scripts
  D:\ghidra_PUBLIC\Ghidra\Processors\DATA\ghidra_scripts
  D:\ghidra_PUBLIC\Ghidra\Processors\PIC\ghidra_scripts (HeadlessAnalyzer)
INFO  HEADLESS: execution starts (HeadlessAnalyzer)
```

`-preScript`

Эта опция задает сценарий, который запускается до анализа. Сценарий может содержать необязательный список аргументов.

`-postScript`

Эта опция задает сценарий, который запускается после анализа. Сценарий может содержать необязательный список аргументов.

`-propertiesPath`

Эта опция задает путь к любым файлам свойств, связанным со сценарием. Файлы свойств предоставляют входные данные для сценариев, запускаемых в режиме без графического интерфейса. Примеры сценариев и связанных файлов свойств включены в документацию анализатора без графического интерфейса.

`-okToDelete`

Поскольку сценарии могут делать все, что задумали их создатели, сценарий может удалить или попытаться удалить файлы внутри проекта Ghidra. Чтобы предотвратить это как нежелательный побочный эффект, Ghidra в режиме без графического интерфейса не позволит удаление файлов сценарием, если опция `-okToDelete` не включена при вызове сценария. Примечание: этот параметр не требуется при запуске в режиме `-import`.

Написание сценариев

Теперь, когда вы понимаете базовые компоненты команды Ghidra в режиме без графического интерфейса, построим несколько сценариев для запуска из командной строки.

HeadlessSimpleROP

Вспомните анализатор `SimpleROP`, который мы написали в главе 15. Мы написали модуль с помощью Eclipse IDE, а затем импортировали расширение в Ghidra, чтобы запускать его на любом импортированном файле. Теперь мы хотим указать `SimpleROP` на каталог и заставить его идентифицировать ROP-гаджеты в каждом файле или выбранных файлах в каталоге. Помимо выходного файла `SimpleROP` с ROP-гаджетами для каждого существующего бинарного файла, мы также хотим сводный файл, перечисляющий каждый файл и количество идентифицированных ROP-гаджетов в каждом.

Для такой работы запуск `SimpleROP` через Ghidra с графическим интерфейсом внес бы временные накладные расходы для действий вроде открытия и закрытия `CodeBrowser`, чтобы показать каждый файл в окне `Listing`, и т. д. Чтобы достичь нашей новой цели, нам не нужно видеть файлы в окне `CodeBrowser`. Почему бы не написать сценарий для поиска гаджетов полностью независимо от графического интерфейса? Это именно тот вариант использования, который подходит для Ghidra в режиме без графического интерфейса.

Хотя мы могли бы изменить функциональность `SimpleROP` для достижения цели, мы не хотим потерять полезность существующего расширения Ghidra, которое другие пользователи могут найти полезным. Мы понимаем, что представили его только в предыдущей главе, но оно могло стать вирусным. Вместо этого используем часть кода из `SimpleROP` как основу для создания нового сценария `HeadlessSimpleROP`, который находит все ROP-гаджеты в `<filename>`, создает и записывает их в `<filename>_gadgets.txt`, затем добавляет `<path>/<filename>` и количество ROP-гаджетов в сводный файл `HeadlessSimpleROP` с именем `gadget_summary.txt`. Вся другая требуемая функциональность, например разбор каталогов и файлов, будет предоставлена Ghidra в режиме без графического интерфейса с помощью опций, обсужденных ранее в этой главе.

Чтобы упростить разработку, мы создаем новый сценарий с помощью подхода Eclipse -> GhidraDev, представленного в главе 15, затем копируем исходный код `SimpleROPAnalyzer.java` в новый шаблон сценария и редактируем код по необходимости. Наконец, запустим сценарий с помощью опции `-postScript`, чтобы он вызывался после фазы анализа для каждого открытого файла.

Создание шаблона сценария HeadlessSimpleROP

Начните с создания шаблона. В меню GhidraDev выберите New -> GhidraScript и заполните информацию, показанную в диалоге на рисунке 16-11. Хотя сценарий можно поместить в любую папку, мы поместим его в папку `ghidra_scripts` внутри существующего модуля `SimpleROP` в Eclipse.

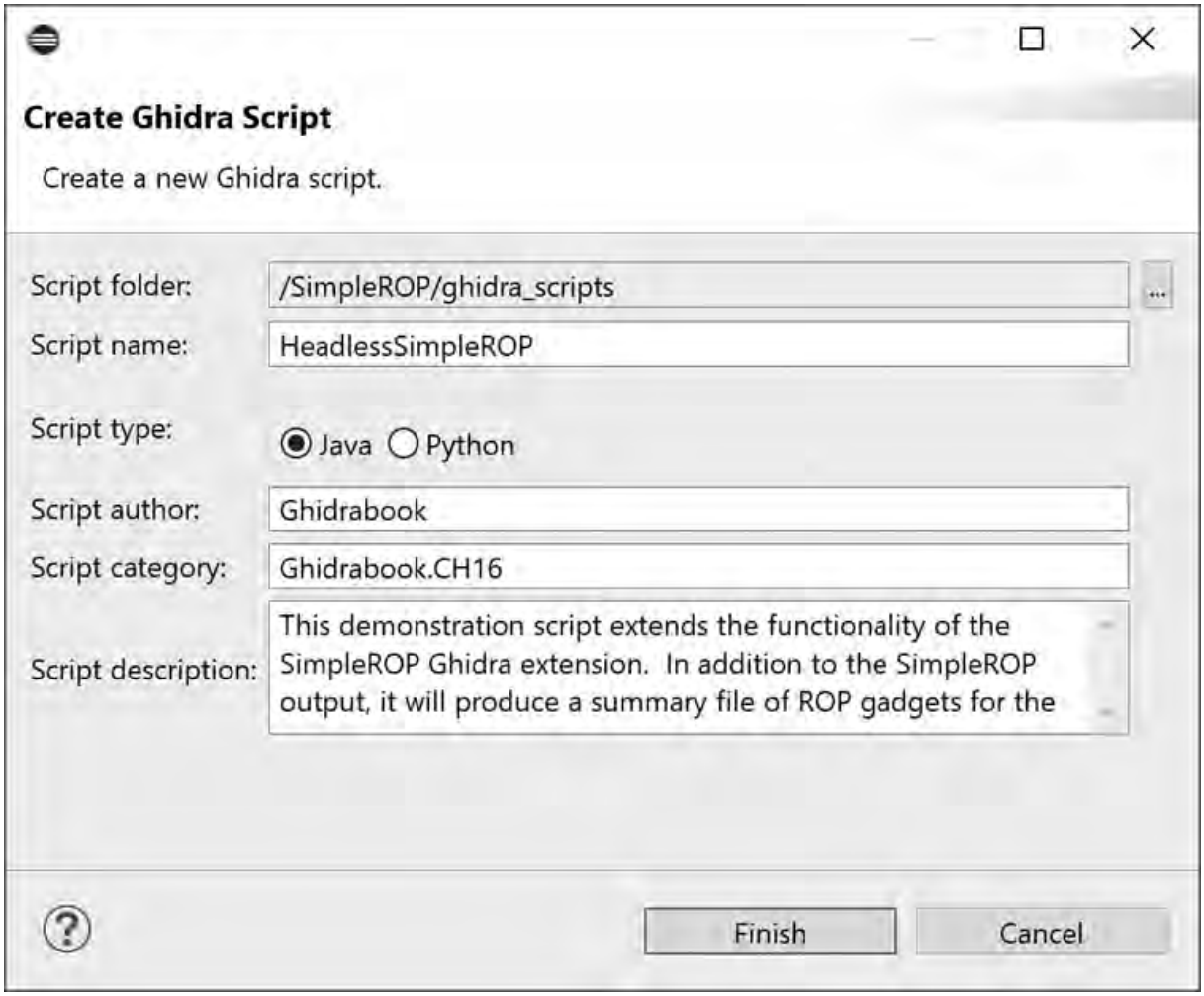
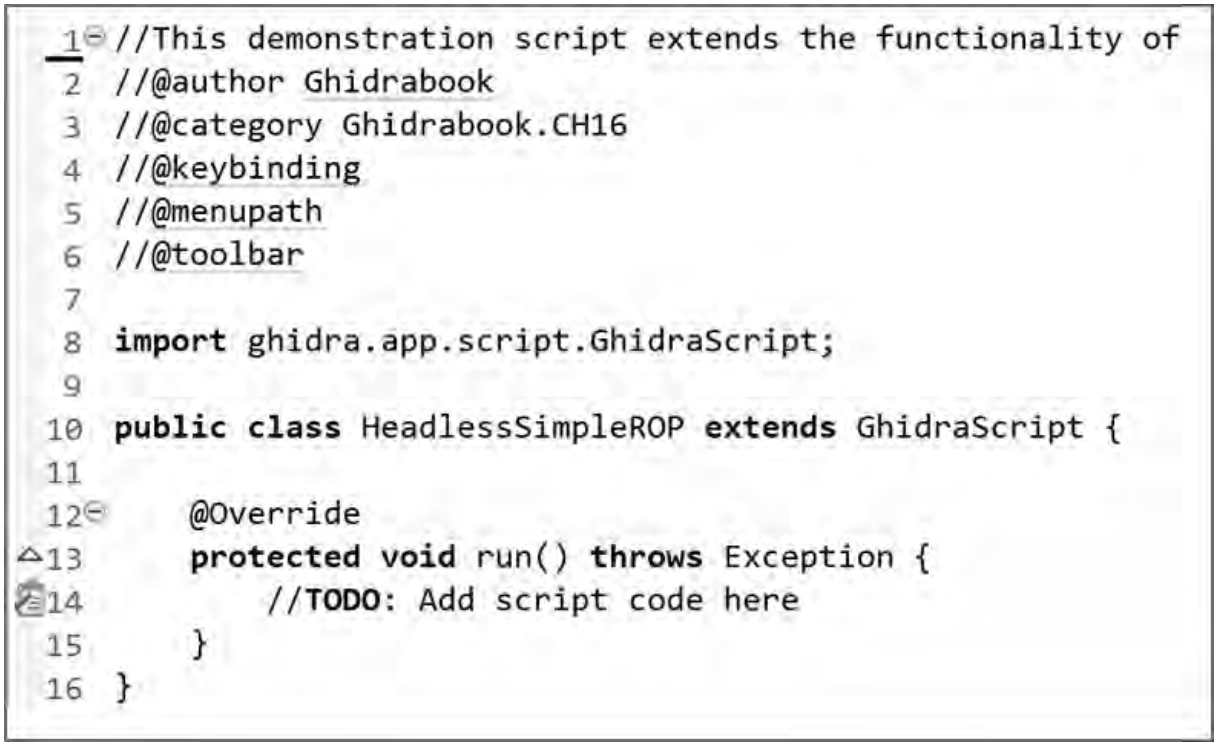


Рисунок 16-11. Диалог Create Ghidra Script

Нажмите `Finish`, чтобы увидеть новый шаблон сценария с метаданными, как показано на рисунке 16-12. Тег задачи на строке 14 показывает, с чего начать.



```
1  //This demonstration script extends the functionality of
2  //@author Ghidrabook
3  //@category Ghidrabook.CH16
4  //@keybinding
5  //@menupath
6  //@toolbar
7
8  import ghidra.app.script.GhidraScript;
9
10 public class HeadlessSimpleROP extends GhidraScript {
11
12     @Override
13     protected void run() throws Exception {
14         //TODO: Add script code here
15     }
16 }
```

Рисунок 16-12. Новый шаблон сценария HeadlessSimpleROP

Чтобы преобразовать анализатор `SimpleROP` в сценарий `HeadlessSimpleROP`, нужно сделать следующее:

1. Удалить ненужные инструкции импорта.
2. Удалить публичные методы анализатора.
3. Продублировать функциональность метода `added`, который вызывается при вызове `SimpleROPAnalyzer`, с помощью метода `run`, который вызывается при вызове сценария `HeadlessSimpleROP`.
4. Добавить функциональность для добавления имени файла и количества найденных гаджетов в сводный файл `gadget_summary.txt`.

Мы поместим сценарий `HeadlessSimpleROP` в каталог `D:\GhidraScripts` и используем анализатор без графического интерфейса для демонстрации его функциональности. В следующих разделах мы запустим серию тестов, вызывая сценарий `HeadlessSimpleROP` с использованием элементов в структуре каталогов, показанной на рисунке 16-6. Эти тесты также демонстрируют некоторые опции, связанные с Ghidra в режиме без графического интерфейса.

Тестовый сценарий 1: загрузить, проанализировать и обработать один файл

В следующем листинге мы используем Ghidra в режиме без графического интерфейса для импорта, анализа и вызова нашего сценария, чтобы сгенерировать отчет о гаджетах для одного файла. Символ ^ является символом продолжения строки в командной оболочке Windows:

```
analyzeHeadless D:\Ghidra Projects CH16_ROP ^
     -import D:\ch16\demo_stackframe_32 ^
     -scriptPath D:\GhidraScripts ^
     -postScript HeadlessSimpleROP.java
```

При выполнении анализатор Ghidra без графического интерфейса создает проект `CH16_ROP` в каталоге `Ghidra Projects`, затем импортирует файл `demo_stackframe_32`, который также будет загружен и проанализирован. Каталог, где находится наш сценарий, указывается с помощью

scriptPath. Наконец, после анализа наш сценарий запускается на импортированном и проанализированном файле.

После завершения команды мы проверяем содержимое файлов gadget_summary.txt и demo_stackframe_32_gadgets.txt, чтобы определить, сработал ли сценарий правильно. demo_stackframe_32_gadgets.txt содержит 16 потенциальных ROP-гаджетов:

```
080482c6;ADD ESP,0x8;POP EBX;RET;
080482c9;POP EBX;RET;
08048343;MOV EBX,dword ptr [ESP];RET;
08048360;MOV EBX,dword ptr [ESP];RET;
08048518;SUB ESP,0x4;PUSH EBP;PUSH dword ptr [ESP + 0x2c];PUSH dword ptr [ESP +
    0x2c];
         CALL dword ptr [EBX + EDI*0x4 + 0xffffff0c];
0804851b;PUSH EBP;PUSH dword ptr [ESP + 0x2c];PUSH dword ptr [ESP + 0x2c];
         CALL dword ptr [EBX + EDI*0x4 + 0xffffff0c];
0804851c;PUSH dword ptr [ESP + 0x2c];PUSH dword ptr [ESP + 0x2c];
         CALL dword ptr [EBX + EDI*0x4 + 0xffffff0c];
08048520;PUSH dword ptr [ESP + 0x2c];CALL dword ptr [EBX + EDI*0x4 + 0xffffff0c];
08048535;ADD ESP,0xc;POP EBX;POP ESI;POP EDI;POP EBP;RET;
08048538;POP EBX;POP ESI;POP EDI;POP EBP;RET;
08048539;POP ESI;POP EDI;POP EBP;RET;
0804853a;POP EDI;POP EBP;RET;
0804853b;POP EBP;RET;
0804854d;ADD EBX,0x1ab3;ADD ESP,0x8;POP EBX;RET;
08048553;ADD ESP,0x8;POP EBX;RET;
08048556;POP EBX;RET;
```

Вот связанная запись в gadget_summary.txt:

```
demo_stackframe_32: Found 16 potential gadgets
```

Тестовый сценарий 2: загрузить, проанализировать и обработать все файлы в каталоге

В этом тесте мы импортируем весь каталог, а не файл, с помощью оператора import:

```
analyzeHeadless D:\Ghidra Projects CH16_ROP ^
      -import D:\ch16 ^
      -scriptPath D:\GhidraScripts ^
      -postScript HeadlessSimpleROP.java
```

Когда анализатор без графического интерфейса завершает работу, в gadget_summary.txt находится следующее содержимое:

```
demo_stackframe_32: Found 16 potential gadgets
demo_stackframe_32_canary: Found 16 potential gadgets
demo_stackframe_32_stripped: Found 16 potential gadgets
demo_stackframe_64: Found 24 potential gadgets
demo_stackframe_64_canary: Found 24 potential gadgets
demo_stackframe_64_stripped: Found 24 potential gadgets
```

Это шесть файлов в корневом каталоге, показанном на рисунке 16-6. Помимо сводного файла гаджетов, мы также создали отдельные файлы гаджетов, перечисляющие потенциальные ROP-гаджеты, связанные с каждым файлом. В оставшихся примерах нас будет интересовать только сводный файл гаджетов.

Тестовый сценарий 3: загрузить, проанализировать и обработать все файлы в каталоге рекурсивно

В этом тесте мы добавляем опцию `-recursive`. Это расширяет операцию импорта, заставляя рекурсивно посещать все файлы во всех подкаталогах внутри каталога `ch16`:

```
analyzeHeadless D:\Ghidra Projects CH16_ROP ^
      -import D:\ch16  ^
      -scriptPath D:\GhidraScripts ^
      -postScript HeadlessSimpleROP.java ^
      -recursive
```

Когда анализатор без графического интерфейса завершает работу, в `gadget_summary.txt` находится следующее содержимое; файл подкаталога появляется в начале списка:

```
demo_stackframe_32_sub: Found 16 potential gadgets
demo_stackframe_32: Found 16 potential gadgets
demo_stackframe_32_canary: Found 16 potential gadgets
demo_stackframe_32_stripped: Found 16 potential gadgets
demo_stackframe_64: Found 24 potential gadgets
demo_stackframe_64_canary: Found 24 potential gadgets
demo_stackframe_64_stripped: Found 24 potential gadgets
```

Тестовый сценарий 4: загрузить, проанализировать и обработать все 32-битные файлы в каталоге

В этом тесте мы используем * как подстановочный знак оболочки, чтобы ограничить содержимое импорта файлами с 32-битным обозначением:

```
analyzeHeadless D:\Ghidra Projects CH16ROP ^
      -import D:\ch16\demo_stackframe_32* ^
      -recursive ^
      -postScript HeadlessSimpleROP.java ^
      -scriptPath D:\GhidraScripts
```

Получившийся файл `gadget_summary` содержит следующее:

```
demo_stackframe_32: Found 16 potential gadgets
demo_stackframe_32_canary: Found 16 potential gadgets
demo_stackframe_32_stripped: Found 16 potential gadgets
```

Если заранее известно, что вас интересуют только сгенерированные файлы гаджетов, используйте опцию `-readOnly`. Эта опция инструктирует Ghidra не сохранять импортированные файлы в проект, указанный в команде, и полезна для предотвращения захламления проекта при пакетной обработке множества файлов.

Автоматизированное создание FidDb

В главе 13 мы начали создавать базу данных Function ID (`FidDb`), заполненную отпечатками функций, взятыми из статической версии `libc`. Используя графический интерфейс и режим пакетного импорта файлов Ghidra, мы импортировали 1 690 объектных файлов из архива `libc.a`. Однако мы столкнулись с препятствием при анализе файлов, потому что графический интерфейс имеет минимальную поддержку пакетного анализа. Теперь, когда вы знакомы с Ghidra в режиме без графического интерфейса, мы можем использовать ее для завершения новой FidDb.

Пакетный импорт и анализ

Импорт и анализ 1 690 файлов из архива когда-то казался пугающей задачей, но предыдущие примеры показали все, что нужно знать, чтобы быстро справиться с этой задачей. Здесь мы рассмотрим два случая и приведем примеры командной строки для каждого.

Если `libc.a` еще не был импортирован в проект Ghidra, мы извлекаем содержимое `libc.a` в каталог, а затем используем Ghidra в режиме без графического интерфейса для обработки всего каталога:

```
$ mkdir libc.a && cd libc.a
$ ar x path\to\archive && cd ..
$ analyzeHeadless D:\Ghidra Projects CH16 -import libc.a ^
        -processor x86:LE:64:default -cspec gcc -loader ElfLoader ^
        -recursive
```

Команда приводит к тысячам строк вывода, пока Ghidra сообщает о ходе работы по 1 690 файлам, которые она обрабатывает, но после завершения команды у вас будет новая папка `libc.a` в проекте, содержащая 1 690 проанализированных файлов.

Если мы использовали графический интерфейс для пакетного импорта `libc.a`, но не обработали ни один из 1 690 импортированных файлов, следующая командная строка выполнит анализ:

```
$ analyzeHeadless D:\Ghidra Projects CH16\libc.a -process
```

Когда весь статический архив эффективно импортирован и проанализирован, мы можем использовать возможности плагина Function ID, чтобы создать и заполнить FidDb, как подробно описано в главе 13.

Резюме

Хотя Ghidra с графическим интерфейсом остается самым прямолинейным и полнофункциональным вариантом, запуск Ghidra в режиме без графического интерфейса предлагает огромную гибкость при создании сложных инструментов, построенных вокруг автоматизированного анализа Ghidra. К этому моменту мы покрыли все наиболее часто используемые возможности Ghidra и исследовали способы заставить Ghidra работать на вас. Пора переходить к более продвинутым возможностям.

В течение нескольких следующих глав мы рассмотрим подходы к некоторым более сложным проблемам, возникающим при обратной разработке бинарных файлов, включая работу с неизвестными форматами файлов и неизвестными архитектурами процессоров путем построения сложных расширений Ghidra. Мы также уделим время исследованию декомпилятора Ghidra и обсудим некоторые способы, которыми компиляторы могут различаться при генерации кода, чтобы улучшить вашу беглость при чтении листингов дизассемблирования.

Глава 17. Загрузчики Ghidra

За исключением короткого примера с загрузчиком `Raw Binary` в главе 4, Ghidra до сих пор определяла тип файла и без проблем загружала и анализировала все файлы, которые мы ей давали. Так будет не всегда. В какой-то момент вы, вероятно, столкнетесь с диалогом, похожим на показанный на рисунке 17-1. Этот конкретный файл является шеллкодом, который Ghidra не может распознать, потому что у него нет определенной структуры, осмысленного расширения файла или магического числа.

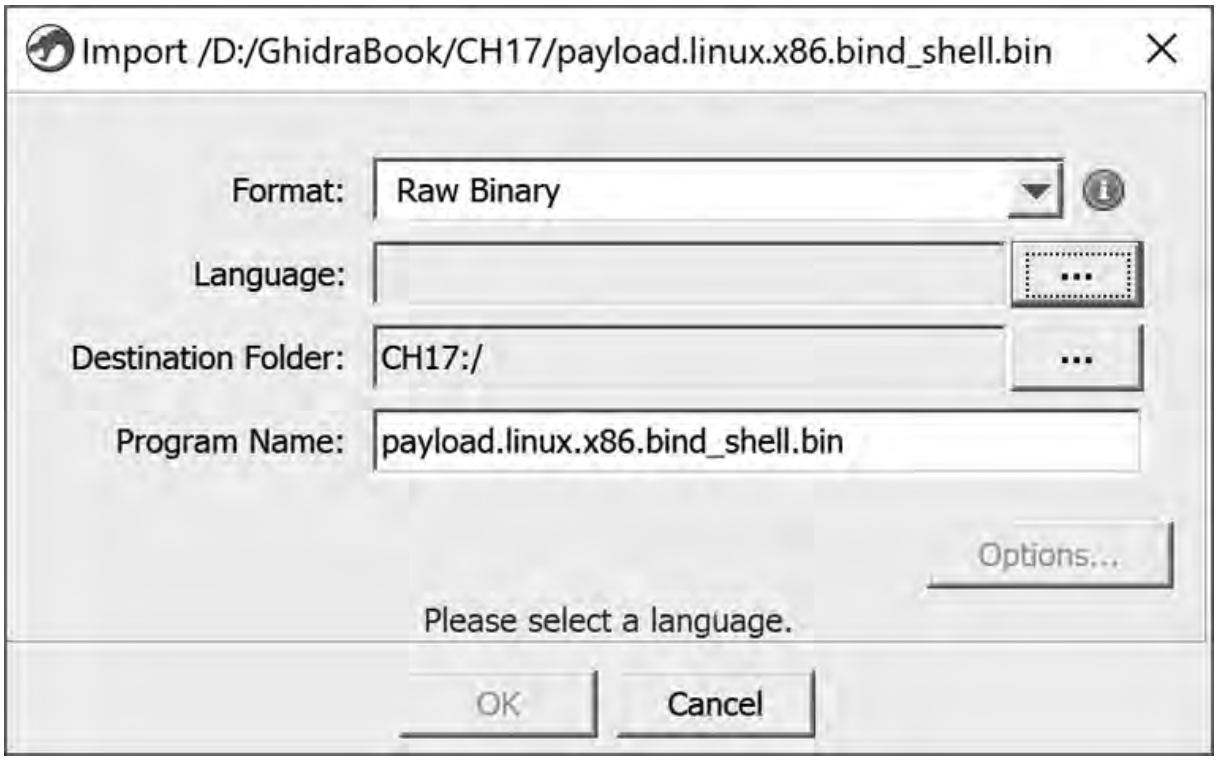


Рисунок 17-1. Пример загрузчика `Raw Binary`

Что произошло, когда мы попытались импортировать этот файл? Начнем с высокоуровневого обзора процесса Ghidra для загрузки файла:

1. В окне `Ghidra Project` пользователь указывает файл для загрузки в проект.
2. Ghidra Importer опрашивает все загрузчики Ghidra, и каждый загрузчик пытается идентифицировать файл. Затем каждый отвечает списком спецификаций загрузки для заполнения диалога Import, если он может загрузить файл. Пустой список означает: "Я не могу загрузить этот файл".
3. Importer собирает ответы от всех загрузчиков, строит список загрузчиков, распознавших файл, и представляет пользователю заполненный диалог Import.
4. Пользователь выбирает загрузчик и связанную информацию для загрузки файла.
5. Importer вызывает выбранный пользователем загрузчик, который затем загружает файл.

Для файла на рисунке 17-1 ни один загрузчик, специфичный для формата, не ответил "да". В результате задача была передана единственному загрузчику, готовому принять любой файл в любое время, - загрузчику `Raw Binary`. Этот загрузчик почти ничего не делает, перекладывая бремя анализа на специалиста по обратной разработке. Если вы когда-нибудь будете анализировать похожие файлы, которые все выглядят как имеющие "сырой" формат, возможно, пришло время построить специализированный загрузчик, который поможет с частью или всем процессом загрузки. Чтобы создать новый загрузчик, который Ghidra сможет использовать для загрузки файла в новом

формате, нужно выполнить несколько задач.

В этой главе мы сначала проведем вас через анализ файла, формат которого Ghidra не распознает. Это поможет понять процесс анализа неизвестного файла и одновременно убедительно покажет, зачем строить загрузчик; именно этим мы займемся во второй половине главы.

Анализ неизвестного файла

Ghidra включает модули загрузчиков для распознавания многих распространенных исполняемых и архивных форматов файлов, но Ghidra не может вместить постоянно растущее количество форматов файлов для хранения исполняемого кода. Бинарные образы могут содержать исполняемые файлы, отформатированные для конкретных операционных систем, образы ROM, извлеченные из встроенных систем, образы прошивок, извлеченные из flash-обновлений, или просто сырые блоки машинного языка, возможно извлеченные из захватов сетевых пакетов. Формат таких образов может диктоваться операционной системой (исполняемые файлы), целевым процессором и архитектурой системы (образы ROM) или вообще ничем (эксплойтный шеллкод, встроенный в данные прикладного уровня).

Предполагая, что доступен процессорный модуль для дизассемблирования кода, содержащегося в неизвестном бинарном файле, ваша задача - правильно расположить образ файла внутри Ghidra, прежде чем сообщить Ghidra, какие части бинарного файла представляют код, а какие - данные. Для большинства типов процессоров результат загрузки файла в сыром формате - это просто список содержимого файла, сложенный в один сегмент, начиная с нулевого адреса, как показано в листинге 17-1.

```
00000000 4d       ??         4Dh    M
00000001 5a       ??         5Ah    Z
00000002 90       ??         90h
00000003 00       ??         00h
00000004 03       ??         03h
00000005 00       ??         00h
00000006 00       ??         00h
00000007 00       ??         00h
```

Листинг 17-1. Начальные строки непроанализированного PE-файла, загруженного с помощью загрузчика `Raw Binary`

В некоторых случаях, в зависимости от развитости выбранного процессорного модуля, происходит некоторое дизассемблирование. Например, выбранный процессор для встроенного микроконтроллера может делать конкретные предположения о раскладке памяти образов ROM, или анализатор со знанием распространенных последовательностей кода, связанных с конкретным процессором, может оптимистично форматировать любые совпадения как код.

Когда вы сталкиваетесь с нераспознанным файлом, вооружитесь всей информацией о файле, которую сможете получить. Полезные ресурсы могут включать заметки о том, как и где файл был получен, справочники по процессору, справочники по операционной системе, документацию по проектированию системы и любую информацию о раскладке памяти, полученную через отладку или аппаратно-ассистированный анализ, например логические анализаторы.

В следующем разделе ради примера предположим, что Ghidra не распознает формат файлов Windows PE. PE - хорошо известный формат файлов, знакомый многим читателям. Что еще важнее, документы, подробно описывающие структуру PE-файлов, широко доступны, что делает разбор произвольного PE-файла относительно простой задачей.

Ручная загрузка файла Windows PE

Когда можно найти документацию по формату конкретного файла, жизнь становится значительно проще при попытке использовать Ghidra для осмысления бинарного файла. Листинг 17-1 показывает первые несколько строк непроанализированного PE-файла, загруженного в Ghidra с помощью загрузчика `Raw Binary` и `x86:LE:32:default:windows` как спецификации языка/компилятора.[1] Спецификация PE утверждает, что допустимый PE-файл начинается со структуры заголовка MS-DOS, начиная с 2-байтовой сигнатуры `4Dh 5Ah` (`MZ`), которую мы видим в первых двух строках листинга 17-1. 4-байтовое значение по смещению `0x3C` в файле содержит смещение к следующему заголовку, который нужно найти: заголовку PE.[2]

Две стратегии для разбиения полей заголовка MS-DOS: (1) определить значения данных подходящего размера для каждого поля в заголовке MS-DOS и (2) использовать функциональность `Data Type Manager` в Ghidra, чтобы определить и применить структуру `IMAGE_DOS_HEADER` согласно спецификации PE-файла. Мы рассмотрим сложности, связанные с вариантом 1, в примере позднее в главе. В этом случае вариант 2 требует значительно меньше усилий.

При использовании загрузчика `Raw Binary` Ghidra не загружает `Data Type Manager` с типами данных Windows, поэтому мы сами можем загрузить архив, содержащий типы MS-DOS, `windows_vs12_32.gdt`. Найдите `IMAGE_DOS_HEADER`, либо перейдя к нему внутри архива, либо выбрав `ctrl-F`, чтобы найти его в окне `Data Type Manager`; затем перетащите заголовок на начало файла. Также можно поставить курсор на первый адрес в листинге и выбрать Data -> Choose Data Type (или горячую клавишу `T`) из контекстного меню правой кнопки мыши и ввести или найти тип данных в появившемся диалоге Data Type Chooser. Любой из этих вариантов дает следующий листинг, с описательными комментариями конца строки для каждого поля:

```
00000000 4d 5a      WORD      5A4Dh     e_magic
00000002 90 00      WORD      90h       e_cblp
00000004 03 00      WORD      3h        e_cp
00000006 00 00      WORD      0h        e_crlc
00000008 04 00      WORD      4h        e_cparhdr
0000000a 00 00      WORD      0h        e_minalloc
0000000c ff ff      WORD      FFFFh     e_maxalloc
0000000e 00 00      WORD      0h        e_ss
00000010 b8 00      WORD      B8h       e_sp
00000012 00 00      WORD      0h        e_csum
00000014 00 00      WORD      0h        e_ip
00000016 00 00      WORD      0h        e_cs
00000018 40 00      WORD      40h       e_lfarlc
0000001a 00 00      WORD      0h        e_ovno
0000001c 00 00 00   WORD[4]             e_res
         00 00 00
         00 00
00000024 00 00      WORD      0h        e_oemid
00000026 00 00      WORD      0h        e_oeminfo
00000028 00 00 00   WORD[10]            e_res2
         00 00 00
         00 00 00
0000003c d8 00 00   LONG      D8h       e_lfanew
```

Поле `e_lfanew` в последней строке предыдущего листинга имеет значение `D8h`; это указывает, что заголовок PE должен находиться по смещению `D8h` (216 байт) внутри бинарного файла. Просмотр байтов по смещению `D8h` должен показать магическое число заголовка PE, `50h 45h` (`PE`), а это означает, что по смещению `D8h` внутри бинарного файла нужно применить структуру `IMAGE_NT_HEADERS`. Ниже приведена часть получившегося развернутого листинга Ghidra:

```
000000d8    IMAGE_NT_HEADERS
   000000d8       DWORD           4550h     Signature
   000000dc    IMAGE_FILE_HEADER            FileHeader
      000000dc    WORD            14Ch      Machine
      000000de    WORD            5h        NumberOfSections
      000000e0    DWORD           40FDFD    TimeDateStamp
      000000e4    DWORD           0h        PointerToSymbolTable
      000000e8    DWORD           0h        NumberOfSymbols
      000000ec    WORD            E0h       SizeOfOptionalHeader
      000000ee    WORD            10Fh      Characteristics
   000000f0    IMAGE_OPTIONAL_HEADER32      OptionalHeader
      000000f0    WORD            10Bh      Magic
      000000f2    BYTE            '\u0006'  MajorLinkerVersion
      000000f3    BYTE            '\0'      MinorLinkerVersion
      000000f4    DWORD           21000h    SizeOfCode
      000000f8    DWORD           A000h     SizeOfInitializedData
      000000fc    DWORD           0h        SizeOfUninitializedData
      00000100    DWORD           14E0h     AddressOfEntryPoint
      00000104    DWORD           1000h     BaseOfCode
      00000108    DWORD           1000h     BaseOfData
      0000010c    DWORD           400000h   ImageBase
      00000110    DWORD           1000h     SectionAlignment
      00000114    DWORD           1000h     FileAlignment
```

К этому моменту мы выявили несколько интересных фрагментов информации, которые помогут дальше уточнить раскладку бинарного файла. Во-первых, поле `Machine` в заголовке PE указывает тип целевого процессора, для которого был собран файл. Значение `14Ch` означает, что файл предназначен для процессоров x86. Если бы тип машины был другим, например `1C0h` (ARM), нам пришлось бы закрыть `CodeBrowser`, щелкнуть файл правой кнопкой мыши в окне `Project`, выбрать `Set Language` и выбрать правильную настройку языка.

Поле `ImageBase` указывает базовый виртуальный адрес загруженного образа файла. Используя эту информацию, мы можем включить сведения о виртуальных адресах в `CodeBrowser`. Через пункт меню `Window -> Memory Map` нам показывается список блоков памяти (рисунок 17-2), составляющих текущую программу. В этом случае один блок памяти содержит все содержимое программы. Загрузчик `Raw Binary` не имеет средств для определения подходящих адресов памяти для какого-либо содержимого программы, поэтому помещает все содержимое в один блок памяти, начинающийся с нулевого адреса.

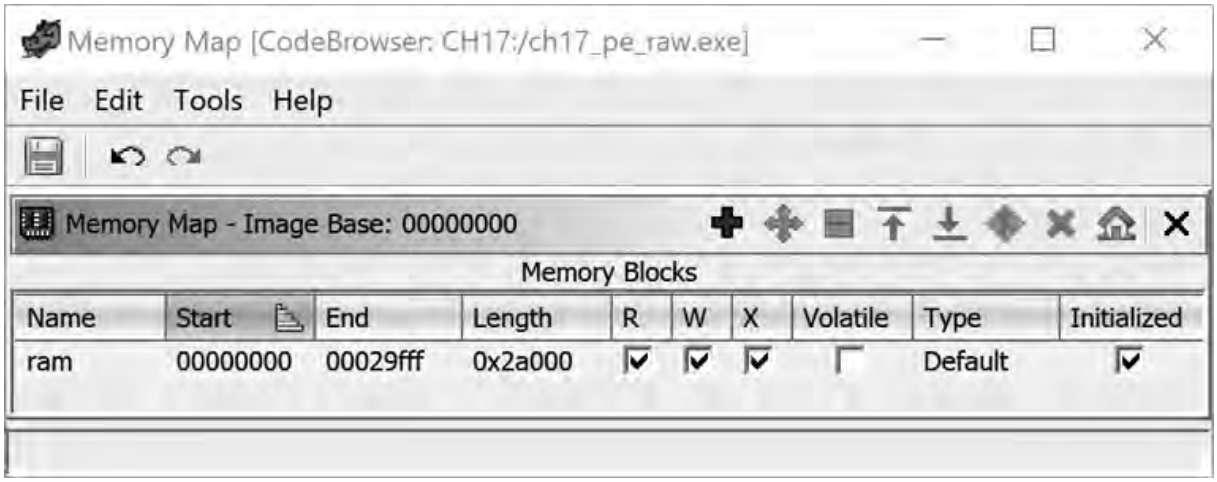


Рисунок 17-2. Окно Memory Map

Кнопки инструментов окна Memory Map, показанные на рисунке 17-3, используются для работы с блоками памяти. Чтобы правильно отобразить образ в память, первое, что нужно сделать, - установить базовый адрес, указанный в заголовке PE.

Рисунок 17-3. Инструменты окна Memory Map

Add Block открывает диалог Add Memory Block и позволяет добавить информацию, необходимую для создания нового блока памяти. Move Block при выбранном блоке памяти позволяет изменить начальный или конечный адрес блока, фактически перемещая его. Split Block разделяет выбранный блок памяти на два блока памяти. Expand Up добавляет дополнительные байты перед блоком памяти; Expand Down добавляет байты после него. Merge Blocks объединяет два или больше выбранных блоков памяти. Delete Block удаляет выбранные блоки памяти. `Set Image Base` позволяет выбрать или изменить базовый адрес программы.

Поле `ImageBase` говорит нам, что правильный базовый адрес для этого бинарного файла - `00400000`. Можно использовать `Set Image Base`, чтобы изменить базу образа с значения по умолчанию на это значение. После нажатия `OK` все окна Ghidra будут обновлены и отразят новую раскладку памяти программы, как показано на рисунке 17-4. Будьте осторожны при использовании этого варианта после того, как уже определено несколько блоков памяти: он сдвинет каждый блок памяти на то же расстояние, что и базовый блок памяти.

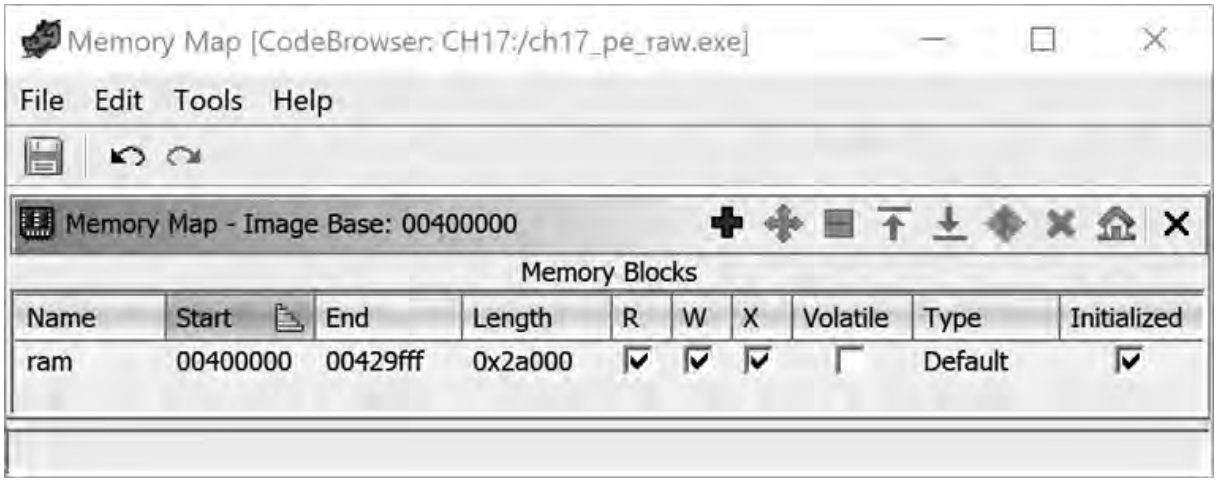


Рисунок 17-4. Memory Map после установки базы образа

Поле `AddressOfEntryPoint` задает относительный виртуальный адрес (RVA) точки входа программы. В спецификации PE-файла RVA - это относительное смещение от базового виртуального адреса программы, а точка входа программы - адрес первой инструкции внутри файла программы, которая будет выполнена. В этом случае RVA точки входа `14E0h` означает, что программа начнет выполнение по виртуальному адресу `4014E0h` (`400000h + 14E0h`). Это первое указание на то, где следует искать код внутри программы. Однако прежде чем мы сможем это сделать, нужно правильно отобразить остаток программы на подходящие виртуальные адреса.

Формат PE использует секции для описания отображения содержимого файла в диапазоны памяти. Разбор заголовков секций для каждой секции в файле позволяет завершить базовую раскладку виртуальной памяти программы. Поле `NumberOfSections` указывает количество секций, содержащихся в PE-файле (в этом случае пять). Согласно спецификации PE, массив структур заголовков секций следует сразу за структурой `IMAGE_NT_HEADERS`. Отдельные элементы массива - это структуры `IMAGE_SECTION_HEADER`, которые мы определяем в редакторе структур Ghidra и применяем (в этом случае пять раз) к байтам после `IMAGE_NT_HEADERS`. В качестве альтернативы можно выбрать первый байт первого заголовка секции и установить его тип в `IMAGE_SECTION_HEADER[n]`, где `n` в этом примере равно 5, чтобы свернуть весь массив в одну строку отображения Ghidra.

Поля `FileAlignment` и `SectionAlignment` указывают, как данные каждой секции выровнены внутри файла и как те же данные будут выровнены при отображении в память. В нашем примере оба поля настроены на выравнивание по смещениям в `1000h` байт. В формате PE нет требования, чтобы эти два числа совпадали. Однако то, что они совпадают, облегчает нам жизнь: смещения к содержимому внутри файла на диске идентичны смещениям к соответствующим байтам в загруженном в память образе файла. Понимание того, как выровнены секции, важно, чтобы избежать ошибок при ручном создании секций для нашей программы.[3]

После структурирования каждого заголовка секции у нас достаточно информации, чтобы создать дополнительные сегменты внутри программы. Применение шаблона `IMAGE_SECTION_HEADER` к байтам, сразу следующим за структурой `IMAGE_NT_HEADERS`, дает первый заголовок секции в нашем листинге Ghidra:

```
004001d0    IMAGE_SECTION_HEADER
   004001d0       BYTE[8]         ".text"   Name
   004001d8    _union_226                   Misc
      004001d8    DWORD           20A80h    PhysicalAddress
      004001d8    DWORD           20A80h    VirtualSize
   004001dc       DWORD           1000h     VirtualAddress
   004001e0       DWORD           21000h    SizeOfRawData
   004001e4       DWORD           1000h     PointerToRawData
   004001e8       DWORD           0h        PointerToRelocations
   004001ec       DWORD           0h        PointerToLinenumbers
   004001f0       WORD            0h        NumberOfRelocations
   004001f2       WORD            0h        NumberOfLinenumbers
```

Поле `Name` сообщает, что этот заголовок описывает секцию `.text`. Все оставшиеся поля потенциально полезны для форматирования листинга, но мы сосредоточимся на трех, описывающих раскладку секции. Поле `PointerToRawData` (`1000h`) указывает файловое смещение, по которому можно найти содержимое секции. Обратите внимание, что это значение кратно значению выравнивания файла, `1000h`. Секции внутри PE-файла расположены в порядке возрастания файловых смещений и виртуальных адресов. Поскольку эта секция начинается по файловому смещению `1000h`, первые `1000h` байт файла содержат данные заголовков файла и заполнение. Поэтому, хотя байты заголовков, строго говоря, не образуют секцию, мы можем подчеркнуть их логическую связь, сгруппировав их в блок памяти в листинге Ghidra.

Ghidra предлагает два способа создать новые блоки памяти, оба доступны через окно Memory Map с рисунка 17-2. Инструмент Add Block (см. рисунок 17-3) открывает диалог на рисунке 17-5, который используется для добавления новых блоков памяти, не перекрывающихся ни с одним существующим блоком памяти. Диалог запрашивает имя нового блока памяти, его начальный адрес и длину. Блок может быть инициализирован постоянным значением (например, заполнен нулями), инициализирован содержимым из текущего файла (вы указываете файловое смещение, откуда берется содержимое) или оставлен неинициализированным.

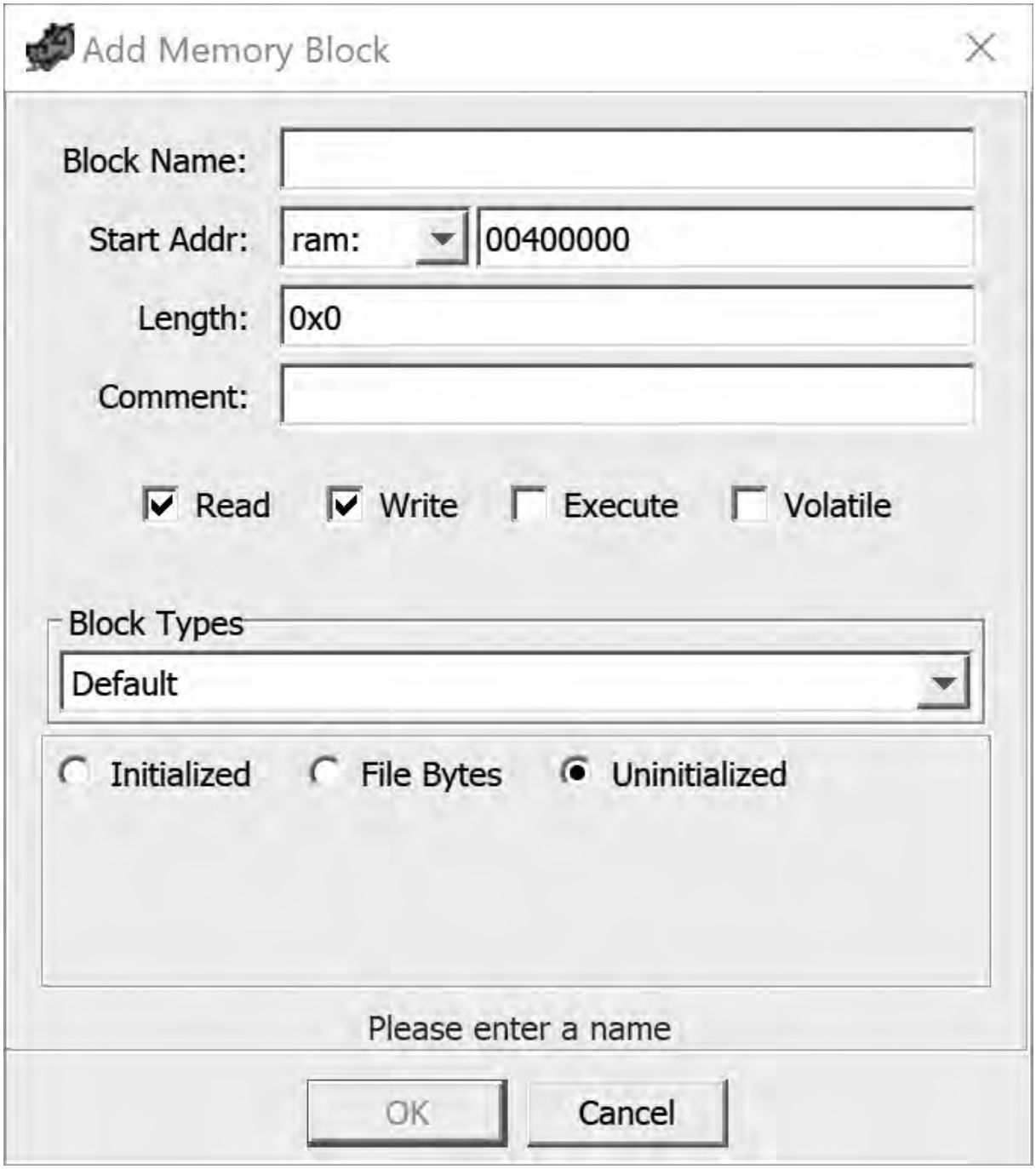


Рисунок 17-5. Диалог Add Memory Block

Второй способ создать новый блок - разделить существующий блок. Чтобы разделить блок в Ghidra, сначала выберите блок для разделения в окне Memory Map, а затем используйте инструмент Split Block (см. рисунок 17-3), чтобы открыть диалог на рисунке 17-6. Мы только начинаем, поэтому у нас есть только один блок для разделения. Начнем с разделения файла в начале секции `.text`, чтобы отделить заголовки программы от начала существующего блока. Когда мы вводим длину (`1000h`) нашего разделяемого блока (секции заголовков), Ghidra автоматически вычисляет оставшиеся поля адреса и длины. Остается только задать имя нового блока, создаваемого в точке разделения. Здесь мы используем имя из первого заголовка секции: `.text`.

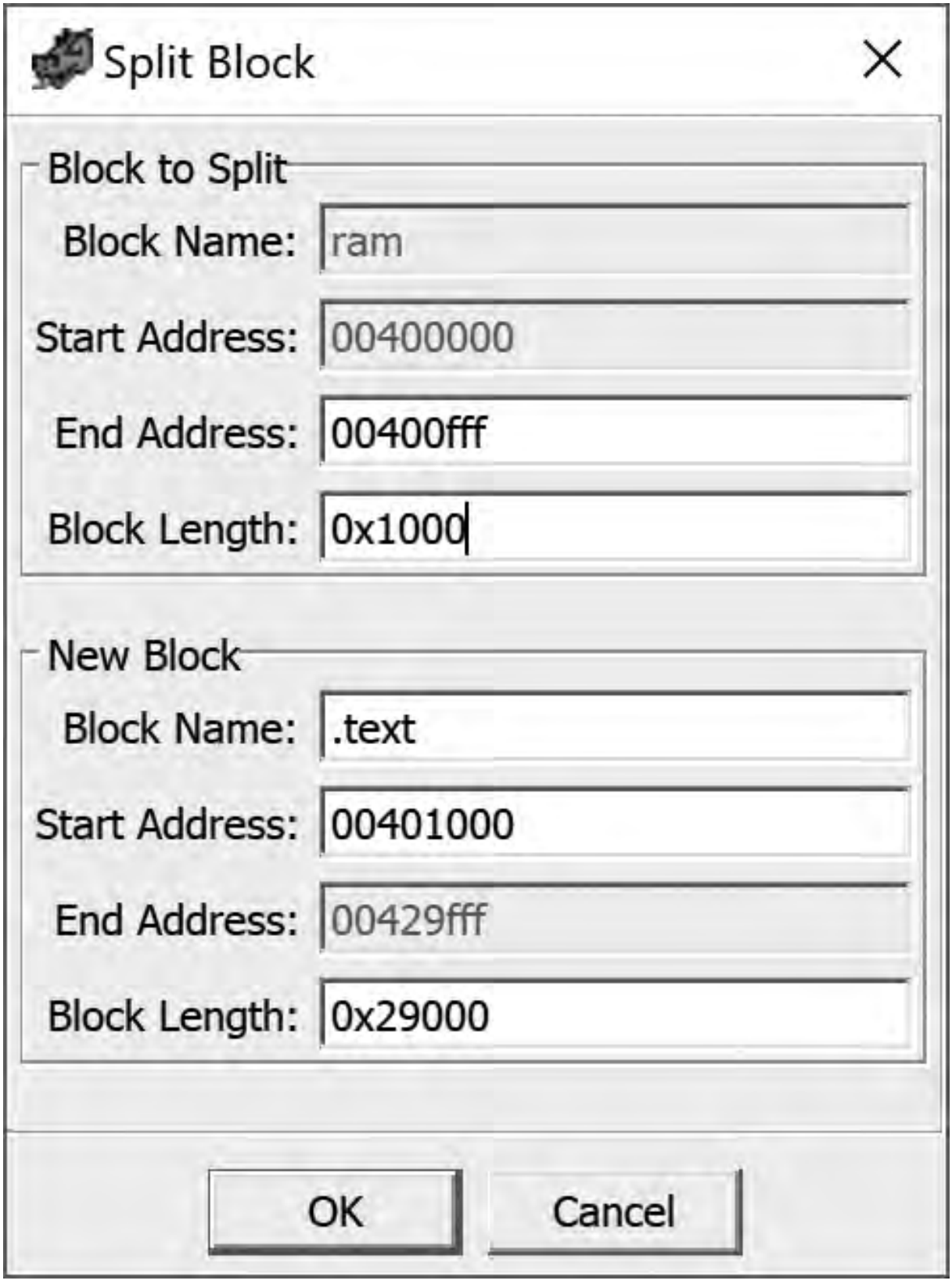


Рисунок 17-6. Диалог Split Block

Теперь в карте памяти два блока. Первый блок содержит заголовки программы правильного размера. Второй блок содержит секцию `.text`, правильно названную, но пока неправильного размера. Эта ситуация отражена на рисунке 17-7, где видно, что размер секции `.text` равен `0x29000` байт.

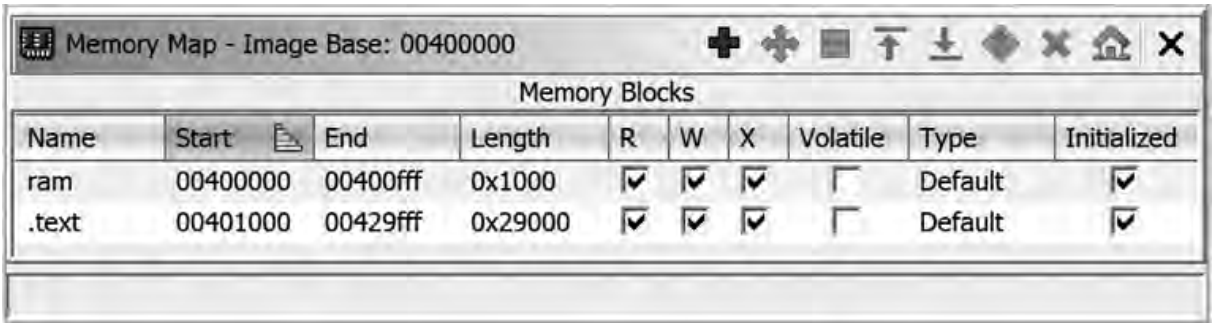


Рисунок 17-7. Memory Map после разделения блока

Возвращаясь к заголовку секции `.text`, мы видим, что поле `VirtualAddress` (`1000h`) является RVA, который задает смещение в памяти (от `ImageBase`), с которого начинается содержимое секции, а поле `SizeOfRawData` (`21000h`) указывает, сколько байт данных присутствует в файле. Иными словами, этот конкретный заголовок секции сообщает, что секция `.text` создается отображением `21000h` байт из файловых смещений `1000h-21FFFh` на виртуальные адреса `401000h-421FFFh`.

Поскольку мы разделили исходный блок памяти в начале секции `.text`, вновь созданная секция `.text` временно содержит все оставшиеся секции, так как ее текущий размер `0x29000` больше правильного размера `0x21000`. Сверяясь с оставшимися заголовками секций и повторно разделяя последний блок памяти, мы продвигаемся к правильной итоговой карте памяти программы. Однако возникает проблема, когда мы доходим до следующей пары заголовков секций:

```
00400220    IMAGE_SECTION_HEADER
   00400220       BYTE[8]         ".data"   Name
   00400228    _union_226                   Misc
      00400228    DWORD           5624h     PhysicalAddress
      00400228    DWORD           5624h     VirtualSize
   0040022c       DWORD           24000h    VirtualAddress
   00400230       DWORD           4000h     SizeOfRawData
   00400234       DWORD           24000h    PointerToRawData
   00400238       DWORD           0h        PointerToRelocations
   0040023c       DWORD           0h        PointerToLinenumbers
   00400240       WORD            0h        NumberOfRelocations
   00400242       WORD            0h        NumberOfLinenumbers
   00400244       DWORD           C0000040h Characteristics
00400248    IMAGE_SECTION_HEADER
   00400248       BYTE[8] ".idata" Name
   00400250    _union_226 Misc
      00400250    DWORD           75Ch      PhysicalAddress
      00400250    DWORD           75Ch      VirtualSize
   00400254       DWORD           2A000h    VirtualAddress
   00400258       DWORD           1000h     SizeOfRawData
   0040025c       DWORD           28000h    PointerToRawData
   00400260       DWORD           0h        PointerToRelocations
   00400264       DWORD           0h        PointerToLinenumbers
   00400268       WORD            0h        NumberOfRelocations
   0040026a       WORD            0h        NumberOfLinenumbers
   0040026c       DWORD           C0000040h Characteristics
```

Виртуальный размер секции `.data` больше ее размера в файле. Что это значит и как это влияет на нашу карту памяти? Компилятор сделал вывод, что программе требуется `5624h` байт статических данных времени выполнения, но предоставляет только `4000h` байт для инициализации этих данных. Оставшиеся `1624h` байт данных времени выполнения не будут инициализированы содержимым из исполняемого файла, поскольку они выделены для неинициализированных глобальных переменных. Такие переменные нередко выделяются внутри специальной секции программы с именем `.bss`.

Чтобы завершить карту памяти, нужно выбрать подходящий размер для секции `.data` и также убедиться, что последующие секции отображены правильно. Секция `.data` отображает `4000h` байт файловых данных из файлового смещения `24000h` на адрес памяти `424000h` (`ImageBase + VirtualAddress`). Следующая секция (`.idata`) отображает `1000h` байт из файлового смещения `28000h` на адрес памяти `42A000h`. Если вы внимательно следите, то могли заметить, что секция `.data`, похоже, занимает в памяти `6000h` байт (`42A000h-424000h`), и это действительно так. Причина в том, что секции `.data` требуется `5624h` байт, но это не четное кратное `1000h`, поэтому секция будет дополнена до `6000h` байт, чтобы секция `.idata` правильно соблюдала требование выравнивания секций, указанное в заголовке PE. Чтобы завершить карту памяти, нужно выполнить следующие действия:

1. Разделить секцию `.data`, используя длину `4000h`. Получившаяся секция `.idata` пока будет начинаться с `428000h`.
2. Переместить секцию `.idata` на адрес `42A000h`, щелкнув значок Move Block (рисунок 17-3) и установив начальный адрес в `42A000h`.
3. Отделить и, если необходимо, переместить любые оставшиеся секции, чтобы получить итоговую раскладку программы.
4. При желании расширить любые секции, чей виртуальный размер выравнивается до более высокой границы, чем их размер в файле. В нашем примере виртуальный размер секции `.data`, `5624h`, выравнивается до `6000h`, а ее размер в файле, `4000h`, выравнивается до `4000h`. После того как мы освободили место, переместив секцию `.idata` в ее правильное расположение, мы расширим секцию `.data` с `4000h` до `6000h` байт.

Чтобы расширить секцию `.data`, выделите секцию `.data` в окне Memory Map, а затем выберите инструмент Expand Down (см. рисунок 17-3), чтобы изменить конечный адрес или длину секции. Диалог Expand Block Down показан на рисунке 17-8. Эта операция добавит к имени секции расширение `.exp`.

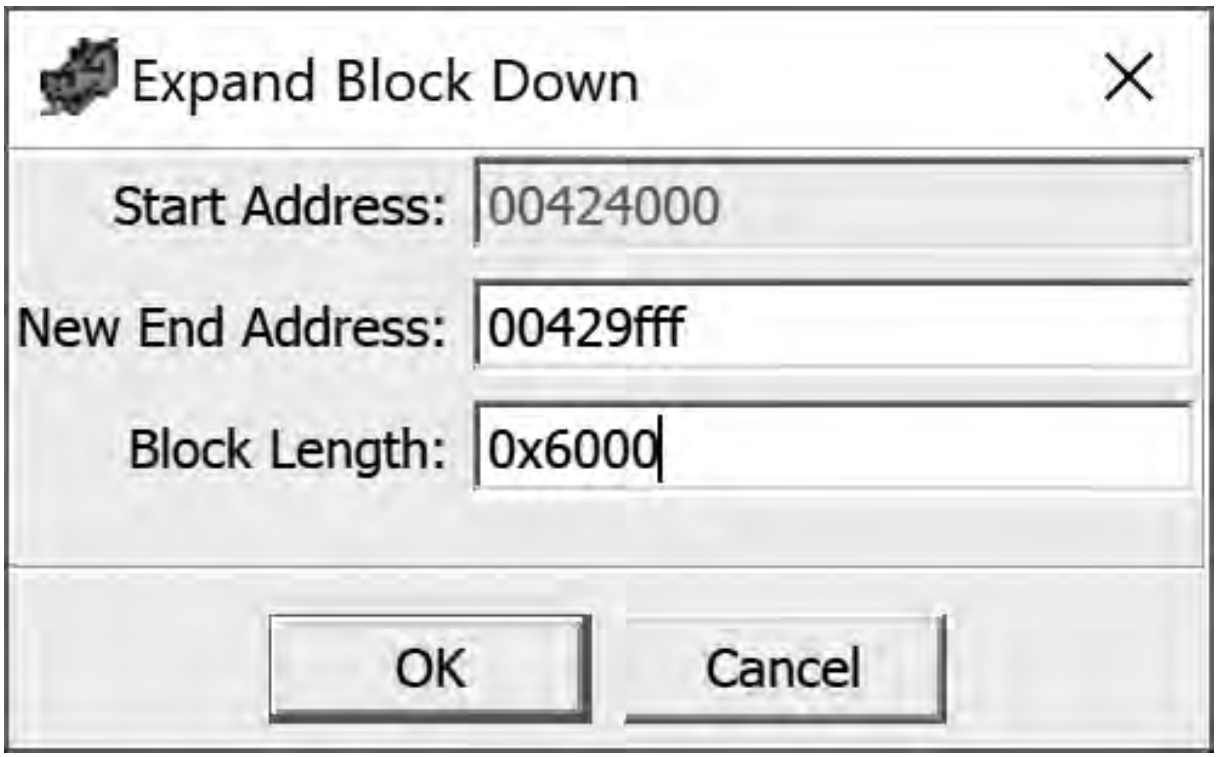
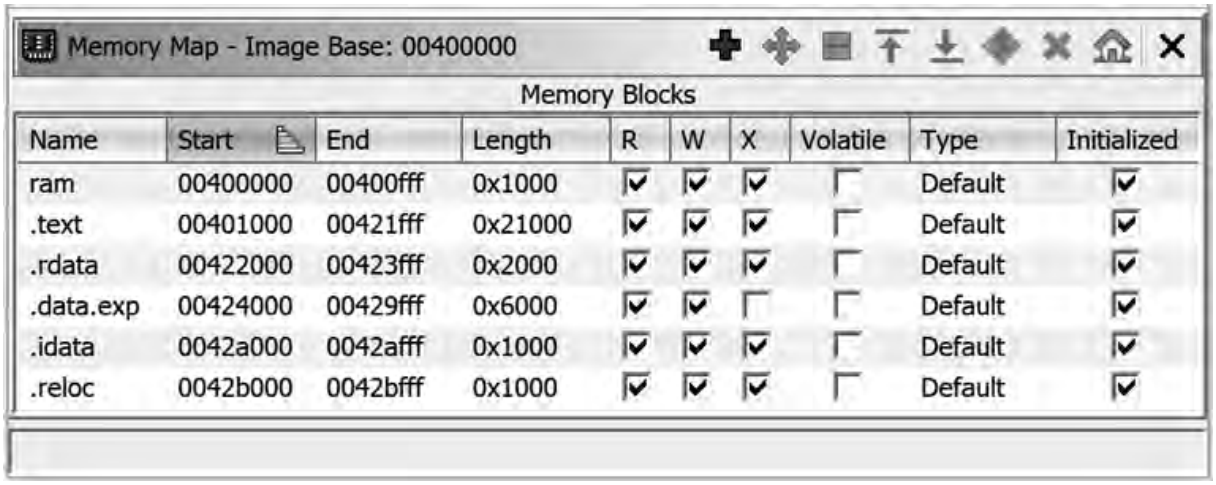


Рисунок 17-8. Диалог Expand Block Down

Наша итоговая карта памяти, полученная после серии перемещений, разделений и расширений блоков, показана на рисунке 17-9. Помимо столбцов с именем секции, начальным и конечным

адресами и длиной, для каждой секции в виде флажков показаны разрешения на чтение (R), запись (W) и выполнение (X). Для PE-файлов эти значения задаются битами в поле `Characteristics` каждого заголовка секции. Обратитесь к спецификации PE за сведениями о разборе поля `Characteristics`, чтобы правильно установить разрешения для каждой секции.



Name	Start	End	Length	R	W	X	Volatile	Type	Initialized
ram	00400000	00400fff	0x1000	☑	☑	☑	☐	Default	☑
.text	00401000	00421fff	0x21000	☑	☑	☑	☐	Default	☑
.rdata	00422000	00423fff	0x2000	☑	☑	☑	☐	Default	☑
.data.exp	00424000	00429fff	0x6000	☑	☑	☐	☐	Default	☑
.idata	0042a000	0042afff	0x1000	☑	☑	☑	☐	Default	☑
.reloc	0042b000	0042bfff	0x1000	☑	☑	☑	☐	Default	☑

Рисунок 17-9. Итоговая Memory Map после создания всех секций

Когда все секции программы правильно отображены, нужно найти байты, которые с высокой вероятностью являются кодом. `AddressOfEntryPoint` (RVA `14E0h`, или виртуальный адрес `4014E0h`) ведет нас к точке входа программы, о которой известно, что это код. Перейдя в это место, мы видим следующий листинг сырых байтов:

```
004014e0  ??     55h    U
004014e1  ??     8Bh
004014e2  ??     ECh
...
```

Использование контекстного меню для дизассемблирования (горячая клавиша `D`) с адреса `004014e0` запускает процесс рекурсивного спуска, ход которого можно отслеживать в нижнем правом углу `CodeBrowser`, и приводит к тому, что приведенные выше байты переформатируются как код:

```
     FUN_004014e0
004014e0  PUSH   EBP
004014e1  MOV    EBP,ESP
004014e3  PUSH   -0x1
004014e5  PUSH   DAT_004221b8
004014ea  PUSH   LAB_004065f0
004014ef  MOV    EAX,FS:[0x0]
004014f5  PUSH   EAX
```

К этому моменту мы надеялись бы, что у нас достаточно кода для всестороннего анализа бинарного файла. Если бы у нас было меньше подсказок относительно раскладки памяти бинарного файла или разделения кода и данных внутри файла, нам пришлось бы полагаться на другие источники информации, чтобы направлять анализ. Некоторые возможные подходы к определению правильной раскладки памяти и поиску кода включают следующее:

- Использовать справочные руководства по процессору, чтобы понять, где могут находиться векторы сброса.
- Искать строки в бинарном файле, которые могут подсказать архитектуру, операционную систему или компилятор, использованный для сборки бинарного файла.
- Искать распространенные последовательности кода, например прологи функций, связанные с процессором, для которого был собран бинарный файл.
- Выполнить статистический анализ частей бинарного файла, чтобы найти области, статистически похожие на известные бинарные файлы.
- Искать повторяющиеся последовательности данных, которые могут быть таблицами адресов, например множество нетривиальных 32-битных целых чисел с одинаковыми верхними 12 битами.[4] Это могут быть указатели, дающие подсказки относительно раскладки памяти бинарного файла.

Завершая обсуждение загрузки сырых бинарных файлов, обратите внимание: вам пришлось бы повторять каждый шаг из этого раздела всякий раз, когда вы открываете бинарный файл того же формата, который остается неизвестным для Ghidra. По пути вы могли бы автоматизировать некоторые действия, написав скрипты, выполняющие часть разбора заголовков и создания сегментов. Именно в этом и состоит назначение модуля загрузчика Ghidra. В следующем разделе мы напишем простой модуль загрузчика, чтобы познакомиться с архитектурой модулей загрузчиков Ghidra, прежде чем перейти к более сложным модулям загрузчиков, выполняющим распространенные задачи, связанные с загрузкой файлов, которые придерживаются структурированного формата.

Пример 1: SimpleShellcode Loader Module

В начале этой главы мы попытались загрузить файл шеллкода в Ghidra и были направлены к загрузчику `Raw Binary`. В главе 15 мы использовали Eclipse и GhidraDev для создания модуля анализатора, а затем добавили его как расширение Ghidra. Вспомните, что один из вариантов модулей, предоставляемых Ghidra, - создание модуля загрузчика. В этой главе мы построим простой модуль загрузчика как расширение Ghidra для загрузки шеллкода. Как и в примере из главы 15, мы используем упрощенный процесс разработки ПО, поскольку это простой демонстрационный проект. Процесс включает следующие шаги:

1. Определить проблему.
2. Создать модуль Eclipse.
3. Построить загрузчик.
4. Добавить загрузчик в нашу установку Ghidra.
5. Протестировать загрузчик из нашей установки Ghidra.

Что такое шеллкод и зачем он нам?

Если быть педантичным, шеллкод - это сырой машинный код, единственная цель которого - породить пользовательский процесс командной оболочки, например `/bin/sh`, чаще всего путем прямого взаимодействия с ядром операционной системы с помощью системных вызовов. Использование системных вызовов устраняет любые зависимости от библиотек пользовательского пространства, таких как `libc`.

Термин raw в этом случае не следует путать с загрузчиком Ghidra `Raw Binary`. Сырой машинный код - это код без упаковки в виде файловых заголовков, и он довольно компактен по сравнению со скомпилированным исполняемым файлом, выполняющим те же действия. Компактный шеллкод для x86-64 в Linux может занимать всего 30 байт, но скомпилированная версия следующей программы на C, которая также порождает оболочку, все равно превышает 6000 байт даже после удаления символов:

```
#include <stdlib.h>
int main(int argc, char **argv, char **envp) {
   execve("/bin/sh", NULL, NULL);
}
```

Недостаток шеллкода в том, что его нельзя запускать напрямую из командной строки. Вместо этого он обычно внедряется в существующий процесс, после чего выполняется действие для передачи управления шеллкоду. Злоумышленники могут попытаться поместить шеллкод в пространство памяти процесса вместе с другими входными данными, потребляемыми процессом, а затем вызвать уязвимость перехвата потока управления, позволяющую перенаправить выполнение процесса на внедренный шеллкод. Поскольку шеллкод часто встроен в другие входные данные, предназначенные для процесса, его можно наблюдать в сетевом трафике, предназначенном для уязвимого серверного процесса, или внутри файла, который должен быть открыт уязвимым приложением просмотра.

Со временем термин "шеллкод" стал использоваться в общем смысле для описания любого сырого машинного кода, включенного в эксплойт, независимо от того, порождает ли выполнение этого машинного кода пользовательскую оболочку на целевой системе.

Шаг 0: сделать шаг назад

Прежде чем даже начать определять проблему, нужно понять (a) что Ghidra сейчас делает с файлом шеллкода и (b) что мы хотели бы, чтобы Ghidra делала с файлом шеллкода. По сути, нам нужно загрузить и проанализировать файл шеллкода как сырой бинарный файл, а затем использовать найденную информацию для разработки загрузчика шеллкода и, возможно, анализатора. К счастью для нас, большинство шеллкодов далеко не так сложно, как PE-файл. Сделаем глубокий вдох и погрузимся в мир шеллкода.

Начнем с анализа файла шеллкода, который мы пытались загрузить в начале главы. Мы загрузили файл и были направлены к загрузчику `Raw Binary` как к единственному варианту, как было показано ранее на рисунке 17-1. Рекомендации по языку не было, поскольку загрузчик `Raw Binary` просто "унаследовал" наш файл, потому что ни один из других загрузчиков не захотел его принять. Выберем относительно распространенную спецификацию языка/компилятора `x86:LE:32:default:gcc`, как показано на рисунке 17-10.

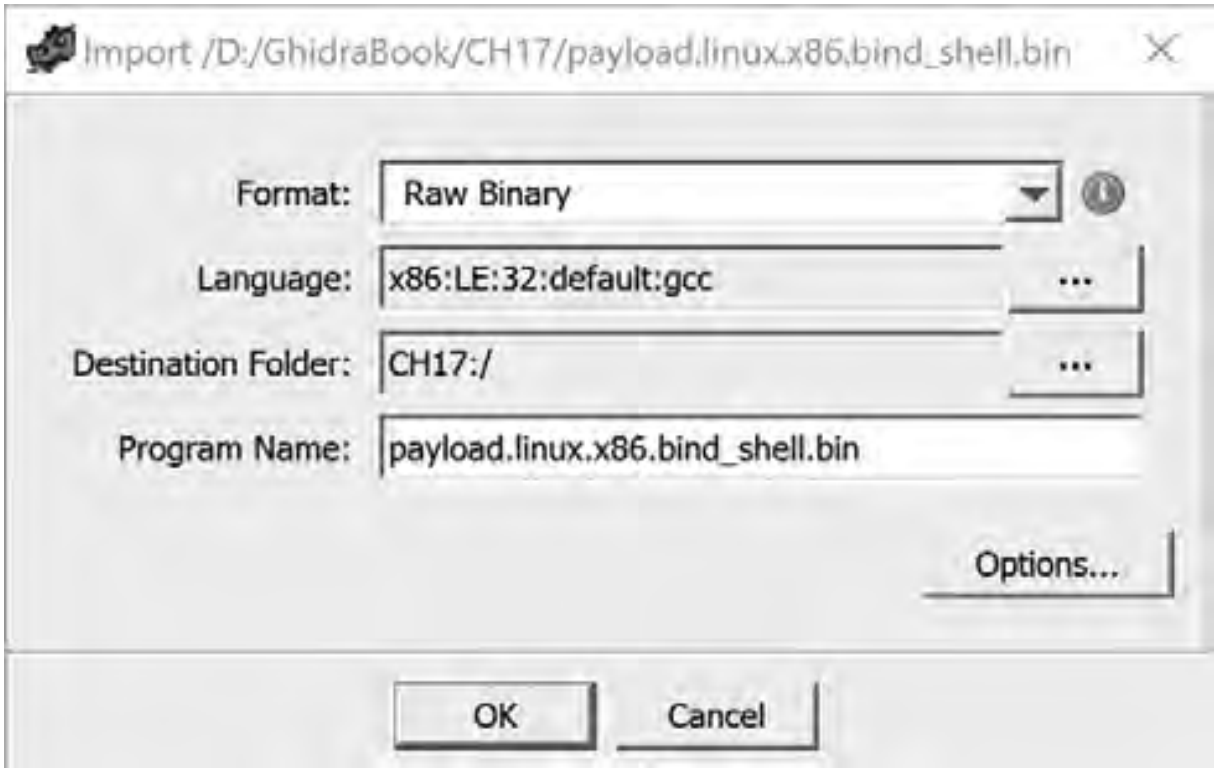


Рисунок 17-10. Диалог Import со спецификацией языка/компилятора

Нажимаем `OK` и получаем окно `Import Results Summary`, включающее содержимое, показанное на рисунке 17-11.

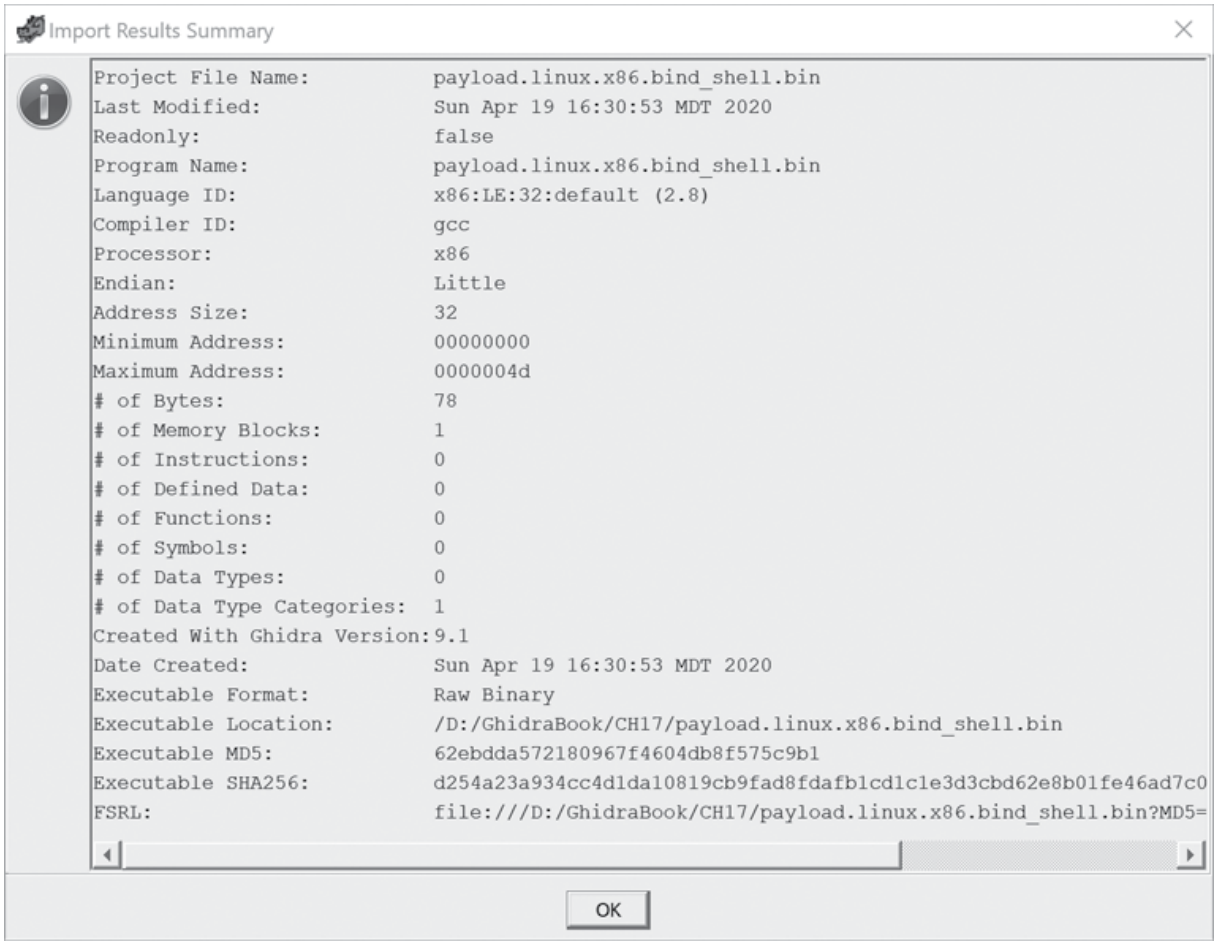


Рисунок 17-11. `Import Results Summary` *для файла шеллкода*

На основе содержимого увеличенного блока в сводке мы знаем, что в файле всего 78 байт в одном блоке памяти/данных, и это почти вся помощь, которую дает загрузчик `Raw Binary`. Если открыть файл в `CodeBrowser`, Ghidra предложит автоматически проанализировать файл. Независимо от того, выполняет ли Ghidra автоматический анализ, окно `Listing` в `CodeBrowser` отображает содержимое, показанное на рисунке 17-12. Обратите внимание, что в `Program Trees` есть только одна секция, `Symbol Tree` пуст, а `Data Type Manager` не имеет записей в папке, относящейся к файлу. Кроме того, окно `Decompiler` остается пустым, поскольку функции в файле не выявлены.

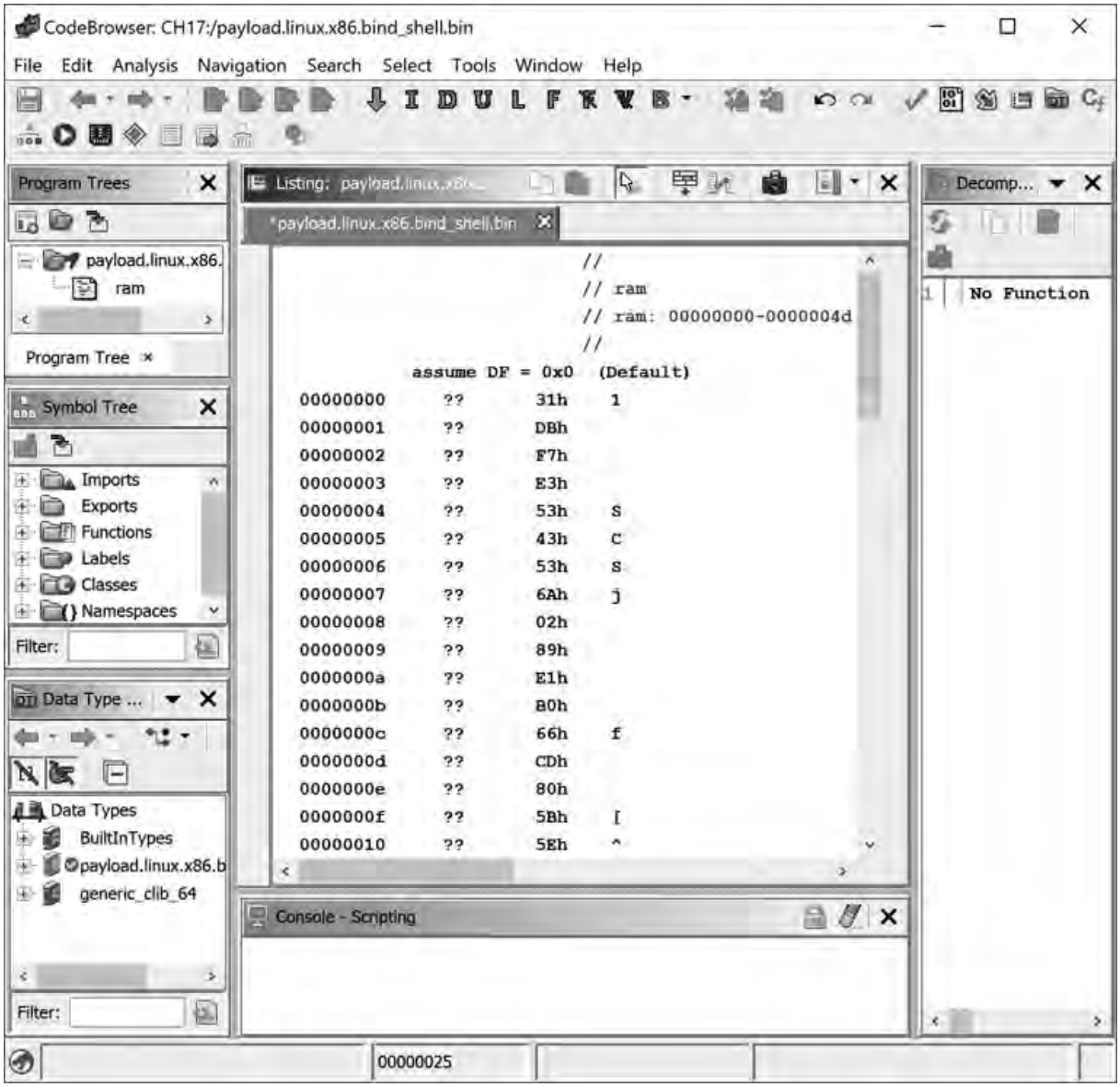


Рисунок 17-12. Окно `CodeBrowser` *после загрузки или анализа файла шеллкода*

Щелкните правой кнопкой мыши первый адрес в файле и выберите Disassemble (горячая клавиша `D`) из контекстного меню. В окне `Listing` теперь видно то, с чем можно работать, - список инструкций. Листинг 17-2 показывает инструкции после дизассемблирования и после того, как мы выполнили некоторый анализ файла. Комментарии конца строки документируют часть анализа этого короткого файла.

```
0000002b  INC    EBX
0000002c  MOV    AL,0x66        ; 0x66 is Linux sys_socketcall
0000002e  INT    0x80           ; передает поток ядру для
                                ; выполнения системного вызова
00000030  XCHG   EAX,EBX
00000031  POP    ECX
        LAB_00000032             XREF[1]:  00000038(j)
00000032  PUSH   0x3f           ; 0x3f is Linux sys_dup2
00000034  POP    EAX
00000035  INT    0x80           ; передает поток ядру для
                                ; выполнения системного вызова
00000037  DEC    ECX
00000038  JNS    LAB_00000
0000003a  PUSH   0x68732f2f     ; 0x68732f2f converts to "//sh"
0000003f  PUSH   0x6e69622f     ; 0x6e69622f converts to "/bin"
00000044  MOV    EBX,ESP
00000046  PUSH   EAX
```

```
00000047  PUSH    EBX
00000048  MOV     ECX,ESP
0000004a  MOV     AL,0xb        ; 0xb - это Linux sys_execve,
                                ; выполняющий заданную программу
0000004c  INT     0x80          ; передает поток ядру для
                                ; выполнения системного вызова
```

Листинг 17-2. Дизассемблированный 32-битный Linux-шеллкод

На основе нашего анализа шеллкод вызывает системный вызов Linux `execve` (по адресу `0000004c`) для запуска `/bin/sh`, который был помещен в стек по адресам `0000003a` и `000003f`. То, что эти инструкции имеют для нас смысл, указывает, что мы, вероятно, выбрали подходящий язык и начальную точку дизассемблирования.

Теперь мы знаем достаточно о процессе загрузки, чтобы определить наш загрузчик. У нас также достаточно информации для построения простого анализатора шеллкода, но это задача на другой день.

Шаг 1: определить проблему

Наша задача - спроектировать и разработать простой загрузчик, который будет загружать шеллкод в окно `Listing` и устанавливать точку входа, что упростит автоматический анализ. Загрузчик нужно добавить в Ghidra, и он должен быть доступен как вариант Ghidra Loader. Он также должен уметь отвечать на опрос Ghidra Importer подходящим образом: так же, как это делает загрузчик `Raw Binary`. Это сделает наш новый загрузчик вторым универсальным вариантом загрузчика.

Кстати, все примеры будут использовать `FlatProgramAPI`. Хотя `FlatProgramAPI` обычно не используется для построения расширений, его применение закрепит концепции скриптинга из главы 14, которые вы, вероятно, будете использовать при разработке скриптов Ghidra на Java.

Шаг 2: создать Eclipse Module

Как обсуждалось в главе 15, используйте GhidraDev -> New -> Ghidra Module Project, чтобы создать модуль `SimpleShellcode`, использующий шаблон Loader Module. Это создаст файл `SimpleShellcodeLoader.java` в папке `src/main/java` внутри модуля `SimpleShellcode`. Эта иерархия папок показана в контексте на рисунке 17-13.

Рисунок 17-13. Иерархия SimpleShellcode

Шаг 3: построить загрузчик

Частичное изображение шаблона загрузчика `SimpleShellcodeLoader.java` показано на рисунке 17-14. Функции свернуты, чтобы вы могли видеть все методы загрузчика, предоставленные в шаблоне загрузчика. Помните, что Eclipse будет рекомендовать импорты, если они понадобятся при разработке кода, так что можно сразу переходить к программированию и принимать рекомендованные операторы `import`, когда Eclipse обнаружит, что они вам нужны.

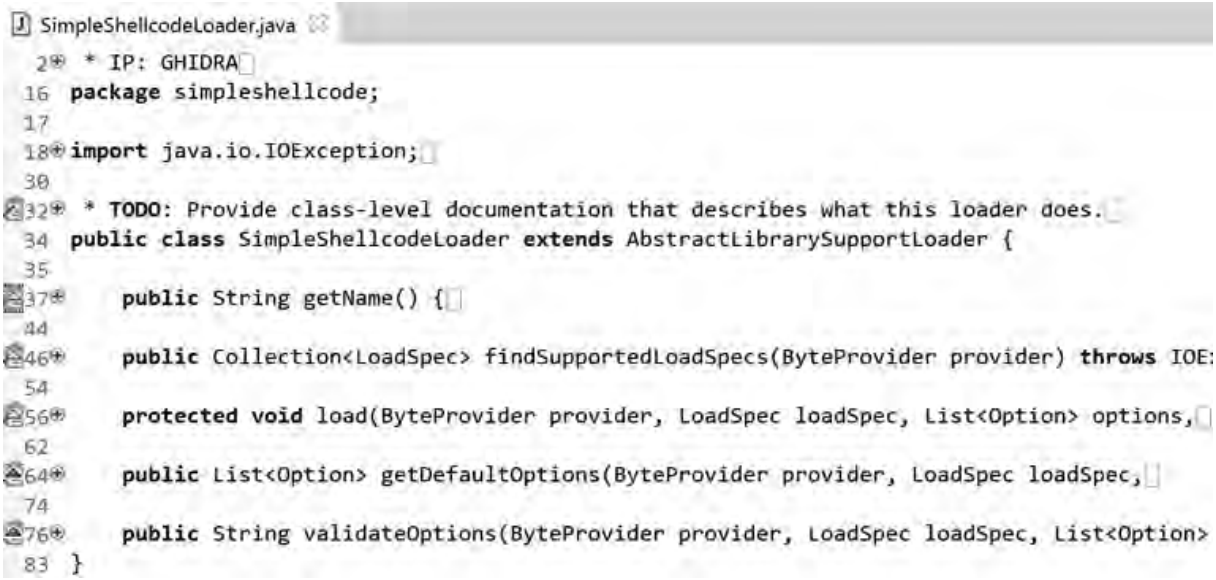


Рисунок 17-14. Шаблон SimpleShellcodeLoader

Внутри шаблона загрузчика на рисунке 17-14 есть шесть тегов задач слева от номеров строк, указывающих, где следует начать разработку. Мы раскроем каждый раздел по мере выполнения конкретных задач и включим содержимое до и после, связанное с каждой задачей, чтобы было понятно, как нужно изменить шаблон. Часть содержимого будет перенесена или переформатирована для удобства чтения, а комментарии будут сведены к минимуму ради экономии места. В отличие от модуля анализатора из главы 15, этот модуль не требует очевидных переменных-членов класса, поэтому можно сразу перейти к текущим задачам.

Шаг 3-1: документировать Class

При раскрытии первого тега задачи вы видите следующее описание задачи:

```
/**
 * TODO: Добавить документацию уровня класса, описывающую,
 * что делает этот загрузчик.
 */
```

Эта задача включает замену существующих комментариев TODO комментариями, описывающими, что делает загрузчик:

```
/*
 * Этот загрузчик загружает бинарные файлы шеллкода в Ghidra,
 * включая установку точки входа.
 */
```

Шаг 3-2: назвать и описать загрузчик

Раскрытие следующего тега задачи показывает комментарий TODO и строку, которую нужно отредактировать. Это упрощает понимание того, где начинать работу. Вторая задача содержит:

```
public String getName() {
      // TODO: Назвать загрузчик. Это имя должно совпадать с именем
      // загрузчика в файлах .opinion
     return "Мой загрузчик";
}
```

Измените строку на осмысленное значение. Не нужно беспокоиться о совпадении имени в файлах `.opinion`, поскольку они неприменимы к загрузчикам, принимающим любые файлы. Вы увидите файлы `.opinion` в третьем примере. Если игнорировать комментарий о файле `.opinion` в шаблоне, получается такой код:

```
public String getName() {
   return "Простой загрузчик шеллкода";
}
```

Шаг 3-3: определить, может ли загрузчик загрузить этот файл

Второй шаг процесса загрузки, описанного в начале главы, включал опрос загрузчиков Importer. Эта задача требует определить, может ли загрузчик загрузить файл, и предоставить ответ Importer через возвращаемое методом значение:

```
public Collection<LoadSpec> findSupportedLoadSpecs(ByteProvider provider)
                            throws IOException {
   List<LoadSpec> loadSpecs = new ArrayList<>();
   // TODO: Проверить байты в 'provider', чтобы определить, может ли
   // загрузчик загрузить файл. Если может, вернуть подходящие
   // спецификации загрузки.
   return loadSpecs;
}
```

Большинство загрузчиков делает это, просматривая содержимое файла в поисках магического числа или структуры заголовка. Входной параметр `ByteProvider` - предоставленная Ghidra обертка только для чтения вокруг входного потока файла. Мы упростим задачу и примем список `LoadSpec`, который использует загрузчик `Raw Binary`: он игнорирует содержимое файла и просто перечисляет все возможные LoadSpec. Затем мы дадим загрузчику более низкий приоритет, чем у загрузчика `Raw Binary`, чтобы если существует более специализированный загрузчик, он автоматически имел более высокий приоритет в диалоге Import Ghidra.

```
public Collection<LoadSpec> findSupportedLoadSpecs(ByteProvider provider)
                                                   throws IOException {
   // Список спецификаций загрузки, поддерживаемых этим загрузчиком
   List<LoadSpec> loadSpecs = new ArrayList<>();
   List<LanguageDescription> languageDescriptions =
        getLanguageService().getLanguageDescriptions(false);
   for (LanguageDescription languageDescription : languageDescriptions) {
      Collection<CompilerSpecDescription> compilerSpecDescriptions =
         languageDescription.getCompatibleCompilerSpecDescriptions();
      for (CompilerSpecDescription compilerSpecDescription :
```

```
            compilerSpecDescriptions) {
          LanguageCompilerSpecPair lcs =
             new LanguageCompilerSpecPair(languageDescription.getLanguageID(),
             compilerSpecDescription.getCompilerSpecID());
          loadSpecs.add(new LoadSpec(this, 0, lcs, false));
       }
    }
    return loadSpecs;
 }
```

Каждый загрузчик имеет связанный уровень и приоритет уровня. Ghidra определяет четыре уровня загрузчиков, от очень специализированных (уровень 0) до независимых от формата (уровень 3). Когда несколько загрузчиков готовы принять файл, Ghidra сортирует список загрузчиков, показываемый пользователю, по возрастанию уровня. Загрузчики внутри одного уровня дополнительно сортируются по возрастанию приоритета уровня, то есть приоритет уровня 10 выводится раньше приоритета уровня 20.

Например, загрузчик PE и загрузчик `Raw Binary` оба готовы загружать PE-файлы, но загрузчик PE - лучший выбор для этого формата (уровень 1), поэтому он появляется раньше загрузчика `Raw Binary` (уровень 3, приоритет уровня 100). Мы устанавливаем уровень простого загрузчика шеллкода в 3 (`LoaderTier.UNTARGETED_LOADER`) и приоритет в 101, чтобы Importer назначал ему самый низкий приоритет при заполнении окна Import кандидатными загрузчиками. Для этого добавьте в загрузчик следующие два метода:

```
@Override
public LoaderTier getTier() {
   return LoaderTier.UNTARGETED_LOADER;
}
@Override
public int getTierPriority() {
   return 101;
}
```

Шаг 3-4: загрузить байты

Следующий метод, показанный до и после редактирования, выполняет основную работу по загрузке содержимого из импортируемого файла в проект Ghidra, в этом случае загружает шеллкод:

```
protected void load(ByteProvider provider, LoadSpec loadSpec,
               List<Option> options, Program program, TaskMonitor monitor,
               MessageLog log) throws CancelledException, IOException {
   // TODO: Загрузить байты из 'provider' в 'program'.
}
```

```
protected void load(ByteProvider provider, LoadSpec loadSpec,
               List<Option> options, Program program, TaskMonitor monitor,
               MessageLog log) throws CancelledException, IOException {
   FlatProgramAPI flatAPI = new FlatProgramAPI(program);
   try {
      monitor.setMessage("Простой шеллкод: начало загрузки");
     // создать блок памяти, в который будет загружен шеллкод
      Address start_addr = flatAPI.toAddr(0x0);
      MemoryBlock block = flatAPI.createMemoryBlock("SHELLCODE",
      start_addr, provider.readBytes(0, provider.length()), false);
     // сделать этот блок памяти доступным на чтение/выполнение, но не на запись
      block.setRead(true);
      block.setWrite(false);
      block.setExecute(true);
     // установить точку входа для шеллкода на начальный адрес
      flatAPI.addEntryPoint(start_addr);
      monitor.setMessage( "Простой шеллкод: загрузка завершена" );
   } catch (Exception e) {
      e.printStackTrace();
      throw new IOException("Не удалось загрузить шеллкод");
```

```
    }
}
```

Обратите внимание: в отличие от скриптов в главах 14 и 15, которые наследуются от `GhidraScript` и в итоге от `FlatProgramAPI`, наш класс загрузчика не имеет прямого доступа к Flat API. Поэтому, чтобы упростить доступ к некоторым часто используемым классам API, мы создаем собственный объект `FlatProgramAPI`. Затем мы создаем `MemoryBlock` с именем `SHELLCODE` по нулевому адресу и заполняем его всем содержимым входного файла. Мы тратим время на установку разумных разрешений для новой области памяти, прежде чем добавить точку входа, которая сообщает Ghidra, где следует начинать дизассемблирование.

Добавление точки входа - очень важный шаг для загрузчика. Наличие точек входа является главным средством, с помощью которого Ghidra находит адреса, про которые известно, что они содержат код, а не данные. При разборе входного файла загрузчик идеально подходит для обнаружения любых точек входа и передачи их Ghidra.

Шаг 3-5: зарегистрировать пользовательские параметры загрузчика

Некоторые загрузчики предлагают пользователям возможность изменять различные параметры, связанные с процессом загрузки. Можно переопределить функцию `getDefaultOptions`, чтобы предоставить Ghidra список пользовательских параметров, доступных для вашего загрузчика:

```
public List<Option> getDefaultOptions(ByteProvider provider, LoadSpec
       loadSpec,DomainObject domainObject, boolean isLoadIntoProgram) {
   List<Option> list = super.getDefaultOptions(provider, loadSpec,
                       domainObject, isLoadIntoProgram);
   // TODO: Если у загрузчика есть пользовательские параметры, добавить их в
    список
   list.add(new Option("Здесь находится имя параметра",
                       Здесь находится значение параметра по умолчанию));
   return list;
}
```

Поскольку этот загрузчик предназначен только для демонстрационных целей, мы не будем добавлять никаких параметров. Параметры загрузчика могут включать установку смещения внутри файла, с которого следует начать чтение, и установку базового адреса, по которому следует загрузить бинарный файл. Чтобы просмотреть параметры, связанные с любым загрузчиком, нажмите кнопку `Options...` в правом нижнем углу диалога Import (см. рисунок 17-1).

```
public List<Option> getDefaultOptions(ByteProvider provider, LoadSpec
       loadSpec,DomainObject domainObject, boolean isLoadIntoProgram) {
   // параметров нет
   List<Option> list = new ArrayList<Option>();
   return list;
}
```

Шаг 3-6: проверить параметры

Следующая задача - проверить параметры:

```
public String validateOptions(ByteProvider provider, LoadSpec loadSpec,
                              List<Option> options, Program program) {
   // TODO: Если у загрузчика есть пользовательские параметры, проверить их здесь.
   // Не все параметры требуют проверки.
   return super.validateOptions(provider, loadSpec, options, program);
}
```

Так как у нас нет параметров, мы просто возвращаем `null`:

```
public String validateOptions(ByteProvider provider, LoadSpec loadSpec,
                              List<Option> options, Program program) {
   // Параметров нет, поэтому проверять нечего
   return null;
}
```

Тестирование модулей из Eclipse

Если вы из тех программистов, у кого код не всегда получается полностью правильным с первой попытки, можно избежать многократных циклов "экспортировать, запустить Ghidra, импортировать расширение, добавить расширение в список импорта, выбрать расширение, перезапустить Ghidra, протестировать расширение", запуская новый код из Eclipse. Если выбрать Run -> Run As в меню Eclipse, появится вариант запуска как Ghidra или Ghidra Headless. Это запустит Ghidra, и вы сможете импортировать файл в текущий проект. Ваш загрузчик будет включен в параметры импорта, а вся обратная связь из консоли будет предоставлена в консоли Eclipse. С файлом можно взаимодействовать в Ghidra как с любым другим файлом. Затем можно выйти из проекта Ghidra без сохранения и либо (1) скорректировать код, либо (2) выполнить длинный цикл "экспортировать, запустить Ghidra, импортировать расширение, добавить расширение в список импорта, выбрать расширение, перезапустить Ghidra и протестировать расширение" всего один раз.

Шаг 4: добавить загрузчик в нашу установку Ghidra

После подтверждения, что модуль работает правильно, экспортируйте расширение модуля Ghidra из Eclipse, а затем установите расширение в Ghidra, как мы делали с модулем `SimpleROPAnalyzer` в главе 15. Выберите GhidraDev -> Export -> Ghidra Module Extension, выберите модуль `SimpleShellcode` и выполните тот же пошаговый процесс с щелчками, что и в главе 15.

Чтобы импортировать расширение в Ghidra, выберите File -> Install Extensions в окне `Ghidra Project`. Добавьте новый загрузчик в список и выберите его. После перезапуска Ghidra новый загрузчик должен быть доступен как вариант, но это нужно проверить.

Шаг 5: протестировать загрузчик внутри Ghidra

Наш упрощенный план тестирования нужен только для демонстрации функциональности. `SimpleShellcode` прошел приемочный тест со следующими критериями:

1. (Пройдено) SimpleShellcode появляется как вариант загрузчика с более низким приоритетом, чем `Raw Binary`.
2. (Пройдено) SimpleShellcode загружает файл и устанавливает точку входа.

Тестовый случай 1 пройден, как показано на рисунке 17-15. Второе подтверждение показано на рисунке 17-16, где загружается PE-файл, анализировавшийся ранее в этой главе. В обоих случаях видно, что вариант простого загрузчика шеллкода имеет самый низкий приоритет в списке `Format`.

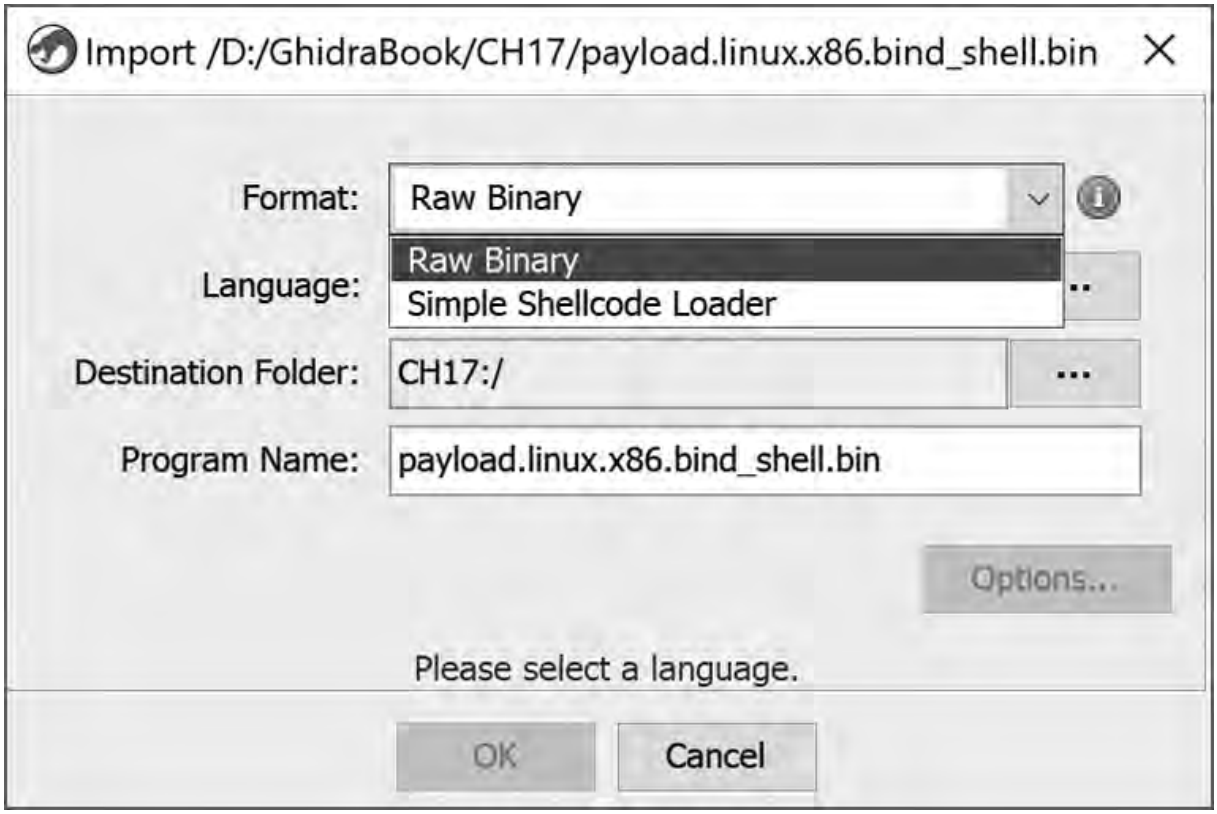


Рисунок 17-15. Окно Import с нашим новым загрузчиком в списке вариантов

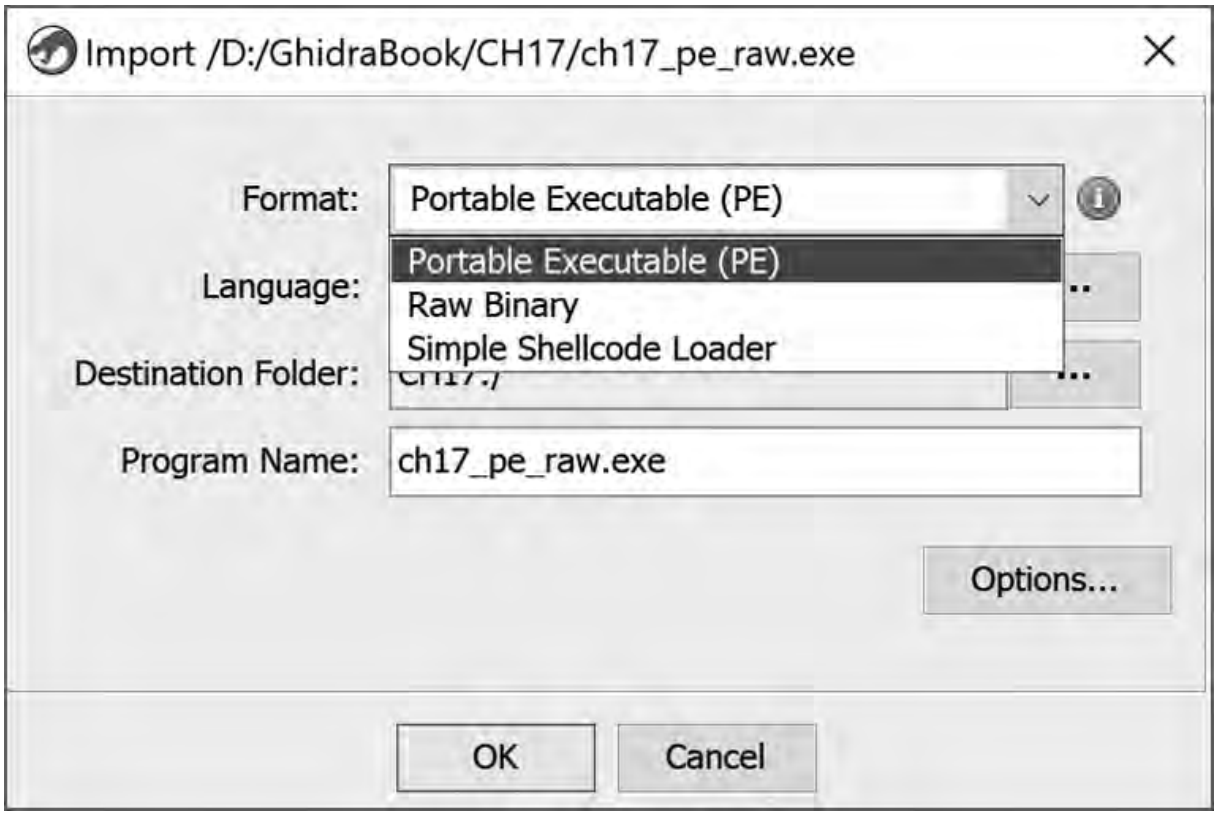


Рисунок 17-16. Окно Import с нашим новым загрузчиком как вариантом для PE-файла

Выберите спецификацию языка на основе доступной информации о бинарном файле и о том, как он был получен. Предположим, шеллкод был захвачен из пакетов, направлявшихся к x86-машине. В таком случае выбор `x86:LE:32:default:gcc` в качестве спецификации языка/компилятора, вероятно, будет хорошей отправной точкой.

После выбора языка и нажатия OK для файла на рисунке 17-15 бинарный файл будет импортирован в наш проект Ghidra. Затем можно открыть программу в CodeBrowser, и Ghidra предоставит вариант проанализировать файл. Если мы примем анализ, увидим следующий листинг:

```
      undefined FUN_00000000()
         undefined  AL:1 <RETURN>
         undefined4 Stack[-0x10]:4 local_10  XREF[1]: 00000022(W)
      FUN_00000000                           XREF[1]: Entry Point(*)
00000000 31 db          XOR    EBX,EBX
00000002 f7 e3          MUL    EBX
00000004 53             PUSH   EBX
00000005 43             INC    EBX
00000006 53             PUSH   EBX
00000007 6a 02          PUSH   0x2
00000009 89 e1          MOV    ECX,ESP
0000000b b0 66          MOV    AL,0x66
0000000d cd 80          INT    0x80
0000000f 5b             POP    EBX
00000010 5e             POP    ESI
00000011 52             PUSH   EDX
00000012 68 02 00 11 5c PUSH   0x5c110002
```

Точка входа выявлена, поэтому Ghidra может предоставить дизассемблирование, с которого начнется наш анализ.

SimpleShellcodeLoader был тривиальным примером, поскольку шеллкод обычно встречается встроенным в какие-то другие данные. Для демонстрационных целей мы используем наш модуль загрузчика как основу для создания модуля загрузчика, который извлекает шеллкод из исходных файлов C и загружает шеллкод для анализа. Это может, например, позволить нам построить сигнатуры шеллкода, которые Ghidra сможет распознавать в других бинарных файлах. Мы не будем глубоко разбирать каждый шаг, поскольку лишь расширяем возможности нашего существующего загрузчика шеллкода.

Пример 2: загрузчик исходного шеллкода Simple Shellcode Source Loader

Поскольку модули предоставляют способ организовать код, а созданный вами модуль SimpleShellcode имеет все необходимое для создания загрузчика, вам не нужно создавать новый модуль. Просто выберите File -> New -> File в меню Eclipse и добавьте новый файл (SimpleShellcodeSourceLoader.java) в папку SimpleShellcode src/main/java. Благодаря этому все ваши новые загрузчики будут включены в новое расширение Ghidra.

Чтобы упростить себе жизнь, вставьте содержимое существующего SimpleShellcodeLoader.java в этот новый файл и обновите комментарии о том, что делает загрузчик. Следующие шаги выделяют части существующего загрузчика, которые нужно изменить, чтобы новый загрузчик работал должным образом. В основном вы будете дополнять существующий код.

Обновление 1: изменить ответ на опрос Importer

Простой загрузчик исходного кода будет принимать решение строго на основе расширения файла. Если файл не заканчивается на `.c`, загрузчик вернет пустой список `loadSpecs`. Если файл заканчивается на `.c`, он вернет тот же список `loadSpecs`, что и предыдущий загрузчик. Чтобы это работало, добавьте следующую проверку в метод `findSupportLoadSpecs`:

```
// Список спецификаций загрузки, поддерживаемых этим загрузчиком
List<LoadSpec> loadSpecs = new ArrayList<>();
// Активировать загрузчик, если имя файла заканчивается расширением .c
if (!provider.getName().endsWith(".c")) {
   return loadSpecs;
}
```

Мы также решили, что наш загрузчик заслуживает более высокого приоритета, чем загрузчик `Raw Binary`, потому что наш загрузчик определяет конкретный тип файлов для приема и лучше подходит для этого типа файлов. Это делается путем возврата более высокого приоритета (меньшего значения) из метода `getTierPriority`:

```
public int getTierPriority() {
   // приоритет этого загрузчика
   return 99;
}
```

Обновление 2: найти шеллкод в исходном коде

Вспомните, что шеллкод - это просто сырой машинный код, который делает для нас что-то полезное. Отдельные байты в шеллкоде лежат в диапазоне `0..255`, и многие из этих значений находятся вне диапазона печатных символов ASCII. Поэтому, когда шеллкод встроен в исходный файл, значительная его часть должна быть представлена с помощью шестнадцатеричных escape-последовательностей, таких как `\xFF`. Строки такого рода довольно уникальны, и мы можем построить регулярное выражение, которое поможет нашему загрузчику их выявлять. Следующее объявление переменной экземпляра описывает регулярное выражение, которое все функции нашего загрузчика могут использовать для поиска байтов шеллкода внутри выбранного C-файла:

```
private String pattern = "\\\\x[0-9a-fA-F]{1,2}";
```

В методе `load` загрузчик разбирает файл в поисках шаблонов, соответствующих регулярному выражению, чтобы помочь вычислить объем памяти, необходимый при загрузке файла в Ghidra. Поскольку шеллкод часто не является непрерывным, загрузчик должен разобрать весь файл, ища области шеллкода для загрузки из файла.

```
// настроить сопоставитель регулярного выражения
CharSequence provider_char_seq =
      new String(provider.readBytes(0, provider.length()), "UTF-8");
Pattern p = Pattern.compile(pattern);
Matcher m = p.matcher(provider_char_seq);
// Определить, сколько совпадений (байтов шеллкода) найдено, чтобы
// правильно задать размер области памяти, затем сбросить сопоставитель
int match_count = 0;
while (m.find()) {
   match_count++;
}
m.reset();
```

После загрузки всего содержимого входного файла мы подсчитываем все совпадения с регулярным выражением.

Обновление 3: преобразовать шеллкод в байтовые значения

Затем метод `load()` должен преобразовать шестнадцатеричные escape-последовательности в байтовые значения и поместить их в массив байтов:

```
byte[] shellcode = new byte[match_count];
// преобразовать шестнадцатеричное представление байтов в исходном коде
// в фактические значения байтов бинарного файла, создаваемого в Ghidra
int ii = 0;
while (m.find()) {
   // отбросить \x
   String hex_digits = m.group().replaceAll("[^0-9a-fA-F]+", "");
   // разобрать оставшееся как целое число и привести его к byte,
   // затем записать это значение в текущий байт массива
   shellcode[ii++] = (byte)Integer.parseInt(hex_digits, 16);
}
```

Шестнадцатеричные цифры извлекаются из каждой совпавшей строки и преобразуются в байтовые значения, которые добавляются в массив `shellcode`.

Обновление 4: загрузить преобразованный массив байтов

Наконец, поскольку шеллкод находится в массиве байтов, метод `load()` должен скопировать его из массива байтов в память программы. Это собственно шаг загрузки и последний обязательный шаг для загрузчика:

```
// создать блок памяти и заполнить его шеллкодом
Address start_addr = flatAPI.toAddr(0x0);
MemoryBlock block =
      flatAPI.createMemoryBlock("SHELLCODE", start_addr, shellcode, false);
```

Результаты

Для проверки нового загрузчика мы создаем исходный файл C, содержащий следующее экранированное представление x86-шеллкода:

```
unsigned char buf[] =
   "\x31\xdb\xf7\xe3\x53\x43\x53\x6a\x02\x89\xe1\xb0\x66\xcd\x80"
   "\x5b\x5e\x52\x68\x02\x00\x11\x5c\x6a\x10\x51\x50\x89\xe1\x6a"
   "\x66\x58\xcd\x80\x89\x41\x04\xb3\x04\xb0\x66\xcd\x80\x43\xb0"
   "\x66\xcd\x80\x93\x59\x6a\x3f\x58\xcd\x80\x49\x79\xf8\x68\x2f"
   "\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\xb0"
   "\x0b\xcd\x80";
```

Поскольку имя исходного файла заканчивается на `.c`, загрузчик появляется в списке как верхний вариант с более высоким приоритетом, чем загрузчики `Raw Binary` и Simple Shellcode, как показано на рисунке 17-17.

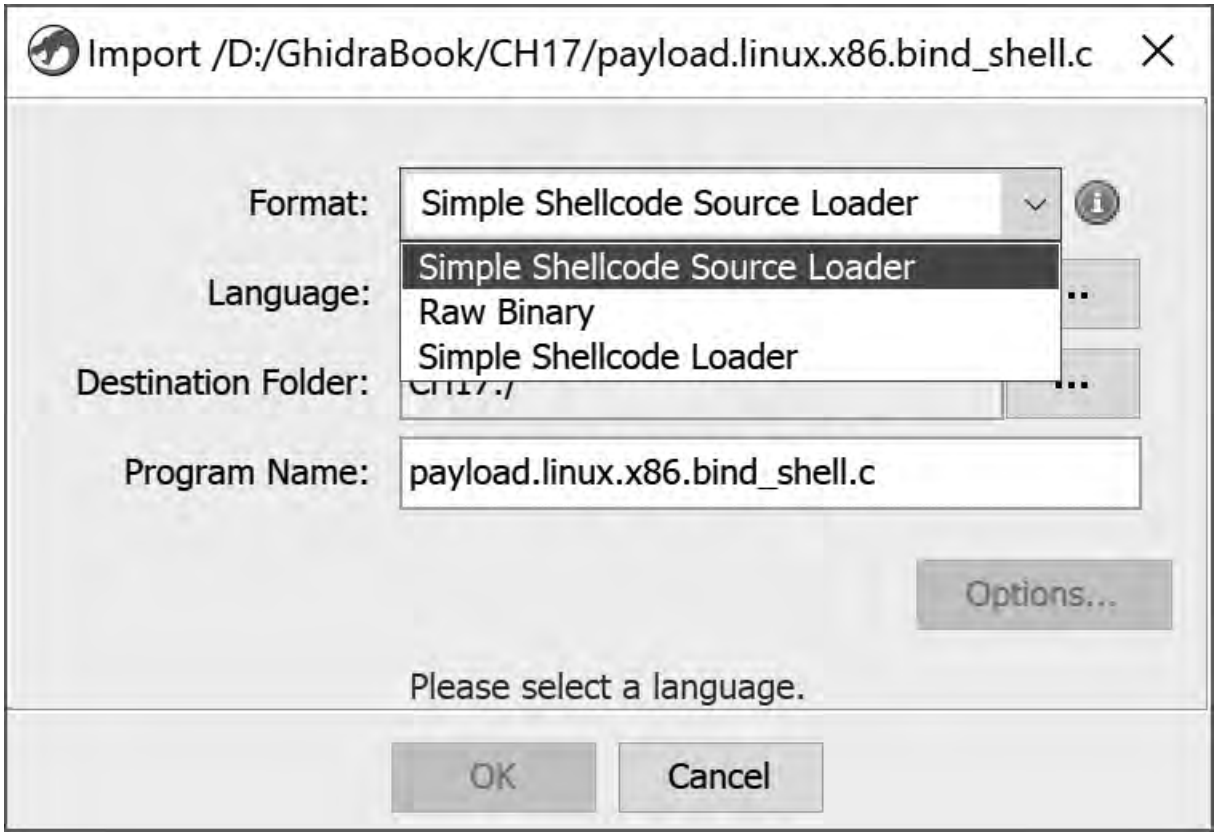


Рисунок 17-17. Диалог Import для исходного файла шеллкода

Выбор этого загрузчика, использование той же спецификации компилятора/языка по умолчанию, что и в предыдущем примере (`x86:LE:32:default:gcc`), и разрешение Ghidra автоматически проанализировать файл дают следующую функцию в дизассемблерном листинге:

```
     *************************************************************
     *                          FUNCTION                         *
     *************************************************************
     undefined FUN_00000000()
        undefined AL:1 <RETURN>
        undefined4 Stack[-0x10]:4 local_10
     FUN_00000000                    XREF[1]: Entry Point(*)
00000000  XOR    EBX,EBX
00000002  MUL    EBX
00000004  PUSH   EBX
00000005  INC    EBX
00000006  PUSH   EBX
```

Прокрутка вниз по листингу приводит к знакомому содержимому (см. листинг 17-2), показанному здесь с комментариями, добавленными для ясности:

```
        LAB_00000032
00000032  PUSH   0x3f
00000034  POP    EAX
00000035  INT    0x80
00000037  DEC    ECX
00000038  JNS    LAB_00000
0000003a  PUSH   0x68732f2f          ; 0x68732f2f converts to "//sh"
0000003f  PUSH   0x6e69622f          ; 0x6e69622f converts to "/bin"
```

Большинство усилий по обратной разработке сосредоточено на бинарных файлах. В этом случае мы вышли за эти рамки и использовали Ghidra для загрузки шеллкода для анализа, а также для извлечения шеллкода из исходных файлов C. Наша цель состояла в том, чтобы продемонстрировать гибкость и простоту создания загрузчиков для Ghidra. Теперь вернемся в эти рамки и создадим загрузчик для структурированного формата файлов.

Предположим, наш целевой шеллкод содержится внутри ELF-бинарного файла и, ради этого примера, Ghidra не распознает ELF-бинарные файлы. Более того, никто из нас никогда не слышал об ELF-бинарных файлах. Пусть приключение начнется.

Пример 3: простой ELF-загрузчик шеллкода

Поздравляем! Теперь вы местный эксперт по обратной разработке шеллкода, и коллеги сообщают о том, что, как они подозревают, является шеллкодом, содержащимся в бинарных файлах, тогда как Ghidra направляет их к загрузчику `Raw Binary`. Поскольку это не похоже на разовую проблему и вы считаете, что есть хороший шанс увидеть больше бинарных файлов с похожими характеристиками, вы решаете построить загрузчик, который будет обрабатывать этот новый тип файлов. Как обсуждалось в главе 13, можно использовать инструменты внутри Ghidra или вне ее, чтобы собрать информацию о файле. Если снова обратиться к командной строке, `file` предоставляет полезную информацию для начала построения загрузчика:

```
$ file elf_shellcode_min
  elf_shellcode_min: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV),
  statically linked, corrupted section header size
$
```

Команда `file` сообщает информацию о формате, о котором вы никогда раньше не слышали, - ELF. Первый шаг - исследование, чтобы посмотреть, можно ли найти какую-либо информацию об этом типе бинарных файлов. Google с радостью укажет на несколько справочных материалов о формате ELF, которые можно использовать, чтобы найти информацию, необходимую для построения загрузчика.[5] Подойдет любой источник, предоставляющий достаточно точной информации для решения задачи.

Поскольку это более сложная задача, чем два предыдущих примера загрузчиков, мы разобьем ее на разделы, связанные с отдельными файлами внутри модуля Eclipse SimpleShellcode, которые вам нужно создать, изменить или удалить, чтобы завершить новый `SimpleELFShellcodeLoader`. Начнем с простой подготовки.

Подготовительные действия

Первый шаг - создать файл `SimpleELFShellcodeLoader.java` внутри модуля `SimpleShellcode` в Eclipse. Поскольку не хочется начинать с нуля, используйте `Save As` с `SimpleShellcodeLoader.java`, чтобы создать этот новый файл. После этого нужно внести несколько небольших изменений в новый файл, прежде чем сосредоточиться на новой задаче:

- Измените имя класса на `SimpleELFShellcodeLoader`.
- Измените возвращаемое значение метода `getTier` с `UNTARGETED_LOADER` на `GENERIC_TARGET_LOADER`.
- Удалите метод `getTierPriority`.
- Измените метод `getName`, чтобы он возвращал `"Простой ELF-загрузчик шеллкода"`.

После завершения подготовительных задач применим информацию, полученную при исследовании нового формата заголовка.

Формат заголовка ELF

Исследуя новый формат, вы обнаруживаете, что формат ELF содержит три типа заголовков: заголовок файла (или заголовок ELF), программные заголовки и заголовки секций. Можно начать с заголовка ELF. С каждым полем в заголовке ELF связано смещение, а также другая информация о поле. Поскольку вам нужно обращаться лишь к нескольким из этих полей и вы не будете изменять смещения, объявите следующие константы как переменные экземпляра внутри класса загрузчика, чтобы помочь загрузчику правильно разобрать этот новый формат заголовка:

```
private final byte[] ELF_MAGIC            = {0x7f, 0x45, 0x4c, 0x46};
private final long EH_MAGIC_OFFSET        = 0x00;
private final long EH_MAGIC_LEN           = 4;
private final long EH_CLASS_OFFSET        = 0x04;
private final byte EH_CLASS_32BIT         = 0x01;
private final long EH_DATA_OFFSET         = 0x05;
private final byte EH_DATA_LITTLE_ENDIAN  = 0x01;
private final long EH_ETYPE_OFFSET        = 0x10;
private final long EH_ETYPE_LEN           = 0x02;
private final short EH_ETYPE_EXEC         = 0x02;
private final long EH_EMACHINE_OFFSET     = 0x12;
private final long EH_EMACHINE_LEN        = 0x02;
private final short EH_EMACHINE_X86       = 0x03;
private final long EH_EFLAGS_OFFSET       = 0x24;
private final long EN_EFLAGS_LEN          = 4;
private final long EH_EEHSIZE_OFFSET      = 0x28;
private final long EH_PHENTSIZE_OFFSET    = 0x2A;
private final long EH_PHNUM_OFFSET        = 0x2C;
```

Имея описание заголовка ELF, следующий шаг - определить, как отвечать на опрос Importer, чтобы убедиться, что новый загрузчик ELF способен загружать только файлы, соответствующие формату ELF. В предыдущих двух примерах загрузчики шеллкода не смотрели на содержимое файла, чтобы определить, могут ли они загрузить файл. Это значительно упростило программирование этих примеров. Теперь все немного сложнее. К счастью, документация ELF предоставляет важные подсказки для определения подходящих спецификаций загрузчика.

Найти поддерживаемые спецификации загрузки

Загрузчик не может загрузить ничего, что не находится в правильном формате, и может отклонить любой файл, вернув пустой список `loadSpecs`. Внутри метода `findSupportedLoadSpecs()` сразу исключите все бинарные файлы без ожидаемого магического числа:

```
byte[] magic = provider.readBytes(EH_MAGIC_OFFSET, EH_MAGIC_LEN);
if (!Arrays.equals(magic, ELF_MAGIC)) {
   // бинарный файл не является ELF
   return loadSpecs;
}
```

После исключения неподходящих файлов загрузчик может проверить разрядность и порядок байтов, чтобы понять, разумна ли архитектура для ELF-бинарного файла. Для этой демонстрации дополнительно ограничим типы принимаемых бинарных файлов 32-битными с прямым порядком байтов:

```
byte ei_class = provider.readByte(EH_CLASS_OFFSET);
byte ei_data = provider.readByte(EH_DATA_OFFSET);
if ((ei_class != EH_CLASS_32BIT) || (ei_data != EH_DATA_LITTLE_ENDIAN)) {
   // это не тот ELF, который нужно принять
   return loadSpecs;
}
```

Чтобы завершить процесс проверки, следующий код проверяет, является ли это исполняемым ELF-файлом, а не разделяемой библиотекой, для архитектуры x86:

```
byte[] etyp = provider.readBytes(EH_ETYPE_OFFSET, EH_ETYPE_LEN);
short e_type =
      ByteBuffer.wrap(etyp).order(ByteOrder.LITTLE_ENDIAN).getShort();
byte[] emach = provider.readBytes(EH_EMACHINE_OFFSET, EH_EMACHINE_LEN);
short e_machine =
      ByteBuffer.wrap(emach).order(ByteOrder.LITTLE_ENDIAN).getShort();
if ((e_type != EH_ETYPE_EXEC) || (e_machine != EH_EMACHINE_X86)) {
   // это не тот ELF, который нужно принять
   return loadSpecs;
}
```

Теперь, когда вы ограничили типы файлов, можно запросить службу выбора на предмет совпадающих спецификаций языка и компилятора. Концептуально вы запрашиваете службы выбора со значениями, извлеченными из загружаемого файла, например из поля `e_machine` заголовка ELF, и в ответ получаете список спецификаций языка/компилятора, которые ваш загрузчик готов принять. Действия "за кулисами", происходящие при запросе службы выбора, описаны подробнее в следующих разделах.

```
byte[] eflag = provider.readBytes(EH_EFLAGS_OFFSET, EN_EFLAGS_LEN);
int e_flags = ByteBuffer.wrap(eflag).order(ByteOrder.LITTLE_ENDIAN).getInt();
List<QueryResult> results =
     QueryOpinionService.query(getName(), Short.toString(e_machine),
                               Integer.toString(e_flags));
```

Предположим, что служба выбора, вероятно, выдаст больше результатов, чем вы хотите обрабатывать этим загрузчиком. Можно дополнительно сократить список, исключая результаты на основе атрибутов, указанных в связанных спецификациях языка/компилятора. Следующий код отфильтровывает компилятор и вариант процессора:

```
for (QueryResult result : results) {
   CompilerSpecID cspec = result.pair.getCompilerSpec().getCompilerSpecID();
   if (cspec.toString().equals("borlanddelphi")) {
      // игнорировать все, что создано Delphi
      continue;
   }
   String variant = result.pair.getLanguageDescription().getVariant();
   if (variant.equals("System Management Mode")) {
       // игнорировать все, что относится к режиму управления системой
       continue;
   }
   // допустимая спецификация загрузки, добавить ее в список
   loadSpecs.add(new LoadSpec(this, 0, result));
}
return loadSpecs;
```

Примеры выше специально исключают компилятор Delphi и режим управления системой x86. При желании можно исключить и другие. Все результаты, которые вы не исключили, нужно добавить в список `loadSpecs`.

Загрузить содержимое файла в Ghidra

Метод `load()` нашего упрощенного загрузчика предполагает, что файл состоит из минимального заголовка ELF и короткого программного заголовка, за которыми следует шеллкод в секции `.text`. Нужно определить общую длину заголовка, чтобы выделить для него правильный объем пространства. Следующий код определяет требуемый размер, используя поля `EH_EEHSIZE_OFFSET`, `EH_PHENTSIZE_OFFSET` и `EH_PHNUM_OFFSET` из заголовка ELF:

```
// Получить из заголовка значения, необходимые для процесса загрузки
//
// Каков размер заголовка ELF?
byte[] ehsz = provider.readBytes(EH_EEHSIZE_OFFSET, 2);
e_ehsize = ByteBuffer.wrap(ehsz).order(ByteOrder.LITTLE_ENDIAN).getShort();
// Каков размер одного программного заголовка?
byte[] phsz = provider.readBytes(EH_PHENTSIZE_OFFSET, 2);
e_phentsize =
      ByteBuffer.wrap(phsz).order(ByteOrder.LITTLE_ENDIAN).getShort();
// Сколько здесь программных заголовков?
byte[] phnum = provider.readBytes(EH_PHNUM_OFFSET, 2);
e_phnum = ByteBuffer.wrap(phunm).order(ByteOrder.LITTLE_ENDIAN).getShort();
// Каков общий размер заголовков для нашего упрощенного формата ELF
// (Сюда входит заголовок ELF и программные заголовки.)
long hdr_size = e_ehsize + e_phentsize * e_phnum;
```

Теперь, когда размер известен, создайте и заполните блоки памяти для секции заголовка ELF и секции text:

```
// Создать блок памяти для заголовка ELF
long LOAD_BASE = 0x10000000;
Address hdr_start_adr = flatAPI.toAddr(LOAD_BASE);
MemoryBlock hdr_block =
      flatAPI.createMemoryBlock(".elf_header", hdr_start_adr,
                                provider.readBytes(0, hdr_size), false);
// Сделать этот блок памяти доступным только для чтения
hdr_block.setRead(true);
hdr_block.setWrite(false);
hdr_block.setExecute(false);
// Создать блок памяти для секции text из упрощенного ELF-бинарника
Address txt_start_adr = flatAPI.toAddr(LOAD_BASE + hdr_size);
MemoryBlock txt_block =
      flatAPI.createMemoryBlock(".text", txt_start_adr,
             provider.readBytes(hdr_size, provider.length() - hdr_size),
             false);
// Сделать этот блок памяти доступным на чтение и выполнение
txt_block.setRead(true);
txt_block.setWrite(false);
txt_block.setExecute(true);
```

Форматировать байты данных и добавить точку входа

Еще несколько шагов, и все будет готово. Загрузчики часто применяют типы данных и создают перекрестные ссылки для информации, полученной из файловых заголовков. Кроме того, задача загрузчика - выявлять точки входа в бинарном файле. Создание списка точек входа во время загрузки предоставляет дизассемблеру список мест, которые он должен считать кодом. Наш загрузчик следует этим практикам:

```
// Добавить структуру к заголовку ELF
flatAPI.createData(hdr_start_adr, new ElfDataType());
// Добавить метку и точку входа в начало шеллкода
flatAPI.createLabel(txt_start_adr, "shellcode", true);
flatAPI.addEntryPoint(txt_start_adr);
// Добавить перекрестную ссылку из заголовка ELF на точку входа
Data d = flatAPI.getDataAt(hdr_start_adr).getComponent(0).getComponent(9);
flatAPI.createMemoryReference(d, txt_start_adr, RefType.DATA);
```

Сначала тип данных заголовка ELF из Ghidra применяется в начале заголовков ELF.[6] Затем для шеллкода создаются метка и точка входа. Наконец, создается перекрестная ссылка между полем точки входа в заголовке ELF и началом шеллкода.

Поздравляем: Java-код для загрузчика написан, но нужно рассмотреть пару вопросов, чтобы вы понимали все зависимости между новым загрузчиком и некоторыми важными связанными файлами, необходимыми для ожидаемой работы загрузчика.

Этот пример использует существующую архитектуру процессора (x86), и часть работы, которая помогла этому загрузчику работать правильно, была выполнена за кулисами. Вспомните, что Importer опросил загрузчики и волшебным образом выдал приемлемые спецификации языка/компилятора. Следующие два файла предоставили информацию, критически важную для загрузчика. Первый из них - файл определения языка x86 `x86.ldefs`, компонент процессорного модуля x86.

Файлы определения языка

Каждый процессор имеет связанный файл определения языка. Это XML-форматированный файл, включающий всю информацию, необходимую для генерации спецификаций языка/компилятора для процессора. Определения языка из файла `x86.ldefs`, отвечающие требованиям 32-битного ELF-бинарного файла, показаны в следующем листинге:

```
<language processor="x86"
          endian="little"
          size="32"
          variant="default"
          version="2.8"
          slafile="x86.sla"
          processorspec="x86.pspec"
          manualindexfile="../manuals/x86.idx"
          id="x86:LE:32:default">
   <description>Intel/AMD 32-bit x86</description>
   <compiler name="Visual Studio" spec="x86win.cspec" id="windows"/>
   <compiler name="gcc" spec="x86gcc.cspec" id="gcc"/>
   <compiler name="Borland C++" spec="x86borland.cspec" id="borlandcpp"/>
   <compiler name="Delphi" spec="x86delphi.cspec" id="borlanddelphi"/>
</language>
<language processor="x86"
          endian="little"
          size="32"
          variant="System Management Mode"
          version="2.8"
          slafile="x86.sla"
          processorspec="x86-16.pspec"
          manualindexfile="../manuals/x86.idx"
          id="x86:LE:32:System Management Mode">
   <description>Intel/AMD 32-bit x86 System Management Mode</description>
   <compiler name="default" spec="x86-16.cspec" id="default"/>
</language>
```

Этот файл используется для заполнения рекомендуемых спецификаций языка/компилятора, представляемых как параметры импорта. В этом случае есть пять рекомендуемых спецификаций, каждая начинается с тега compiler, которые будут возвращены на основе информации, связанной с ELF-бинарным файлом, но наш загрузчик исключает две из рассмотрения на основе компилятора и варианта.

Файлы .opinion

Другой тип вспомогательного файла - файл `.opinion`. Это XML-форматированный файл, содержащий ограничения, связанные с вашим загрузчиком. Чтобы быть распознанным службой запросов выбора, каждый загрузчик должен иметь запись в файле `.opinion`. Следующий листинг показывает подходящую запись файла `.opinion` для загрузчика, который вы только что построили:

```
<opinions>
   <constraint loader="Простой ELF-загрузчик шеллкода" compilerSpecID="gcc">
      <constraint primary="3" processor="x86"  endian="little" size="32" />
      <constraint primary="62" processor="x86"  endian="little" size="64" />
   </constraint>
</opinions>
```

Все в записи должно быть знакомо, за возможным исключением поля `primary`. Это первичный ключ поиска, который идентифицирует машину так, как она определена в заголовке ELF. В заголовке ELF значение `0x03` в поле `e_machine` означает x86, а `0x3E` означает amd64. Тег `<constraint>` определяет связь между первичным ключом (`"3"`/x86) и остальными атрибутами тега `<constraint>`. Эта информация используется службой запросов, чтобы найти подходящие записи в файлах определения языка.

Оставшаяся задача - поместить данные выбора в подходящее место, где Ghidra их найдет. Единственные файлы `.opinion`, поставляемые с Ghidra, находятся в подкаталоге `data/languages` процессорного модуля Ghidra. Хотя можно вставить свои данные выбора в существующий файл `.opinion`, лучше избегать изменения любых процессорных файлов `.opinion`, поскольку ваши изменения придется повторно применять при каждом обновлении установки Ghidra.

Вместо этого создайте новый файл `.opinion`, содержащий наши данные выбора. Имя файла может быть любым, но `SimpleShellcode.opinion` выглядит разумно. Наш шаблон Eclipse Loader Module содержит собственный подкаталог `data`. Сохраните файл `.opinion` в этом месте, чтобы он был связан с вашим модулем загрузчика. Ghidra найдет его при поиске файлов `.opinion`, и любые обновления установки Ghidra не должны затронуть ваш модуль.

Теперь, когда вы понимаете, что происходит за кулисами, пора протестировать загрузчик и посмотреть, ведет ли он себя ожидаемым образом.

Результаты

Чтобы продемонстрировать успех нового упрощенного ELF-загрузчика (один программный заголовок и без секций), пройдем процесс загрузки и понаблюдаем, как загрузчик работает на каждом шаге процесса.

Из окна `Ghidra Project` импортируйте файл. Importer опросит все загрузчики Ghidra, включая ваш, чтобы посмотреть, какие из них готовы загрузить этот файл. Вспомните, что ваш загрузчик ожидает файл, соответствующий следующему профилю:

• Магическое число ELF в начале файла
• 32-битный, с прямым порядком байтов
• Исполняемый ELF-файл для архитектуры x86
• Не может быть скомпилирован Delphi
• Не может иметь вариант "System Management Mode"

Если загрузить файл, соответствующий этому профилю, вы должны увидеть диалог Import, похожий на рисунок 17-18, который отображает приоритетный список загрузчиков, готовых обработать этот файл.

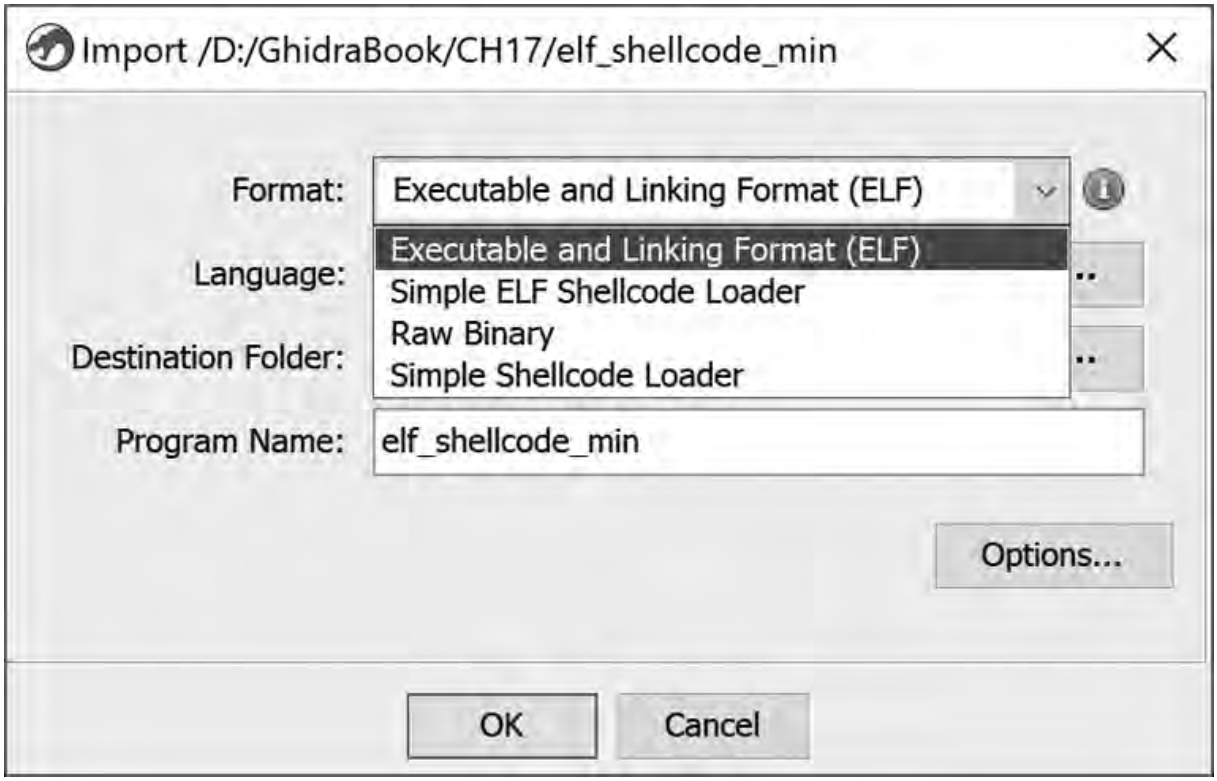


Рисунок 17-18. Import options для `elf_shellcode_min`

Загрузчик с самым высоким приоритетом - штатный ELF-загрузчик Ghidra. Сравните спецификации языка/компилятора, которые он принимает (верхняя часть рисунка 17-19), с теми, которые принимает ваш новый загрузчик (нижняя часть рисунка).

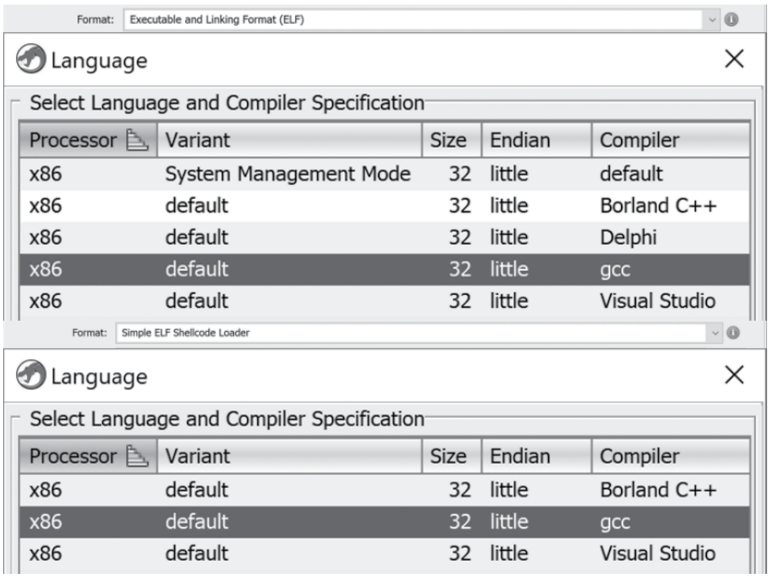


Figure 17-19: Acceptable language/compiler specifications for two different loaders

The Delphi compiler and the System Management Mode variant are accepted by the stock ELF loader but not by your loader, as they have been filtered out. When you select your loader for the file *elf_shellcode_min*, you should see a summary similar to Figure 17-20.

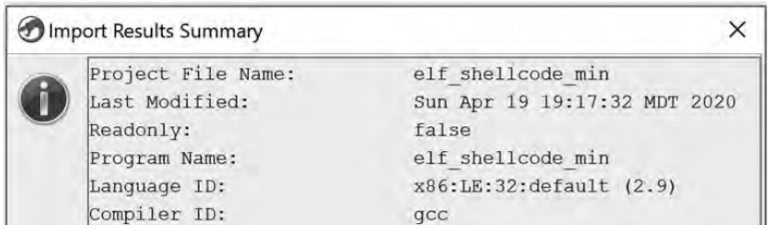


Рисунок 17-19. Допустимые спецификации языка/компилятора для двух разных загрузчиков

Компилятор Delphi и вариант System Management Mode принимаются штатным ELF-загрузчиком, но не вашим загрузчиком, поскольку они были отфильтрованы. Когда вы выберете свой загрузчик для файла `elf_shellcode_min`, вы должны увидеть сводку, похожую на рисунок 17-20.

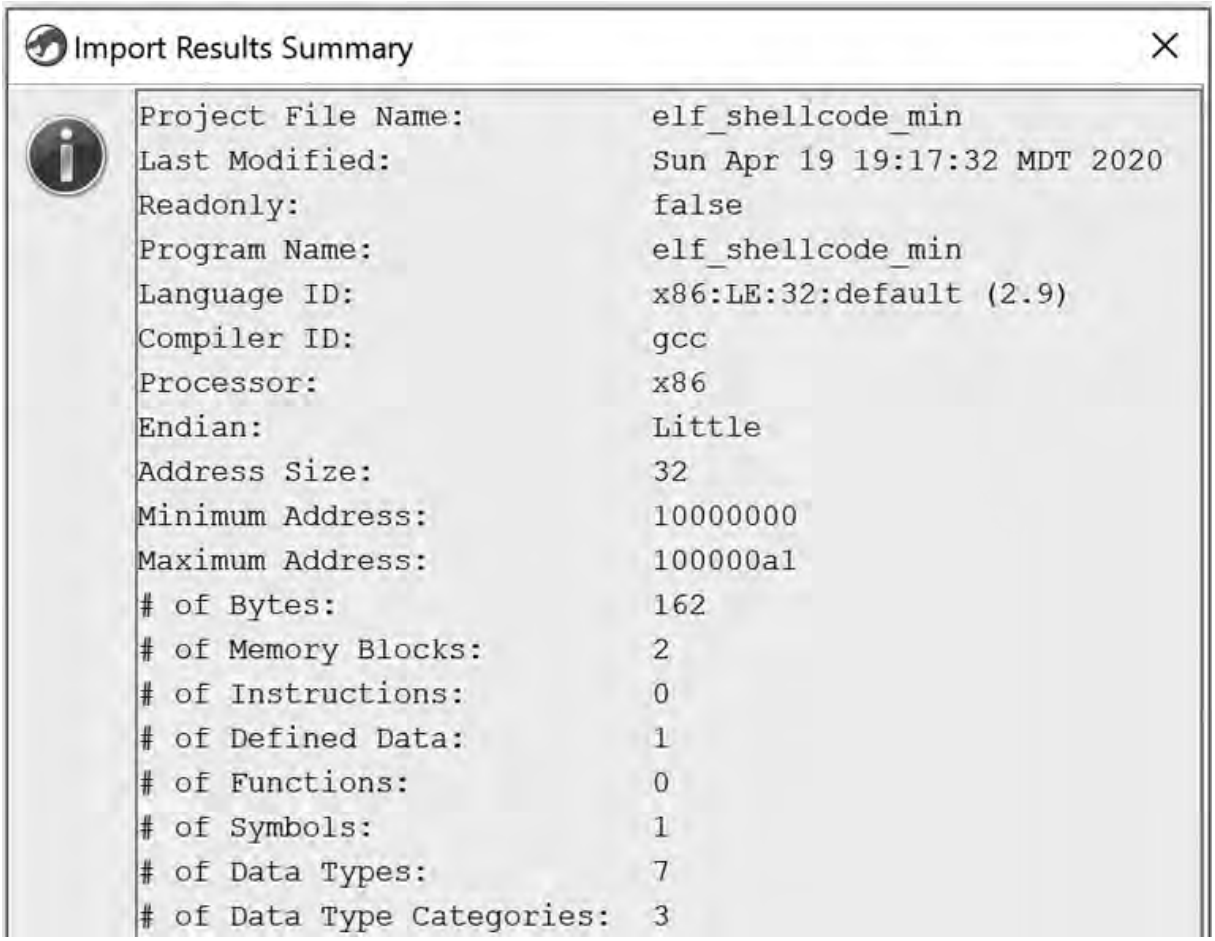


Рисунок 17-20. `Import Results Summary` *для нового ELF Shellcode Loader*

Если открыть файл в `CodeBrowser` и разрешить Ghidra автоматически проанализировать файл, в верхней части файла вы должны увидеть следующее определение заголовка ELF:

```
10000000 7f             db      7Fh          e_ident_magic_num
10000001 45 4c 46       ds      "ELF"        e_ident_magic_str
10000004 01             db      1h           e_ident_class
10000005 01             db      1h           e_ident_data
10000006 01             db      1h           e_ident_version
10000007 00 00 00 00 00 db[9]                e_ident_pad
         00 00 00 00
10000010 02 00          dw      2h           e_type
10000012 03 00          dw      3h           e_machine
10000014 01 00 00 00    ddw     1h           e_version
10000018 54 00 00 10    ddw     shellcode    e_entry
1000001c 34 00 00 00    ddw     34h          e_phoff
10000020 00 00 00 00    ddw     0h           e_shoff
10000024 00 00 00 00    ddw     0h           e_flags
10000028 34 00          dw      34h          e_ehsize
```

Внутри листинга метка `shellcode` явно связана с точкой входа. Двойной щелчок по метке `shellcode` переносит вас к функции с именем `shellcode`, содержащей тот же шеллкод, который мы видели в предыдущих двух примерах, включая следующее:

```
1000008c  JNS    LAB_10000086
1000008e  PUSH   "//sh"
10000093  PUSH   "/bin"
10000098  MOV    EBX,ESP
1000009a  PUSH   EAX
```

Теперь, подтвердив, что ваш новый загрузчик работает, можно добавить его как расширение в установку Ghidra и поделиться им с коллегами, которые с нетерпением ждут этой функциональности.

Резюме

В этой главе мы сосредоточились на трудностях, связанных с работой с нераспознанными бинарными файлами. Мы прошли примеры процессов загрузки и анализа, которые можно использовать внутри Ghidra для помощи в сложных сценариях обратной разработки. Наконец, мы расширили наши возможности создания модулей до мира загрузчиков Ghidra.

Хотя построенные нами примеры были тривиальными, они заложили основу и представили все компоненты, необходимые для написания более сложных модулей загрузчиков в Ghidra. В следующей главе мы завершим обсуждение модулей Ghidra введением в процессорные модули - компоненты, которые в наибольшей степени отвечают за общее форматирование дизассемблированного бинарного файла.

Сноски

[1] [назад] Если выбрать Visual Studio как компилятор, в спецификацию языка/компилятора вставляется `windows`. Для большинства других компиляторов имя выбора ближе совпадает с отображаемым именем.
[2] [назад] См. https://docs.microsoft.com/en-us/windows/win32/debug/pe-format.
[3] [назад] Выравнивание описывает начальный адрес или смещение блока данных. Адрес или смещение должны быть четным кратным значения выравнивания. Например, когда данные выровнены по границе `200h` (512) байт, они должны начинаться с адреса или смещения, которое делится на `200h`.
[4] [назад] Тривиальные числа обычно имеют очень мало значащих цифр и включают -1, 0 и другие малые целые. Интересные числа обычно имеют много значащих цифр, как правило порядка разрядности архитектуры, и потому с большей вероятностью дают более релевантные результаты поиска.
[5] [назад] Хотя страницы man обычно являются авторитетным ресурсом, в этом случае Wikipedia также предоставляет нужную информацию. Используйте доступные ресурсы, чтобы отвечать на имеющиеся вопросы.
[6] [назад] Если бы это был действительно новый формат, вам, вероятно, пришлось бы создать эту структуру внутри Ghidra на основе своего исследования. Для этого примера используйте ту, которая есть в окне `Data Type Manager` Ghidra.

Глава 18. Процессоры Ghidra

Процессорные модули, самый сложный тип модулей Ghidra, отвечают за все операции дизассемблирования, выполняемые внутри Ghidra. Помимо очевидного преобразования опкодов машинного языка в их эквиваленты на языке ассемблера, процессорные модули также поддерживают создание функций, перекрестных ссылок и стековых фреймов.

Хотя количество процессоров, поддерживаемых Ghidra, впечатляет и растет с каждым крупным выпуском, в некоторых обстоятельствах требуется разработка нового процессорного модуля Ghidra. Очевидный случай для разработки процессорного модуля - обратная разработка бинарного файла, для которого в Ghidra не существует процессорного модуля. Среди прочего, такой бинарный файл может представлять образ прошивки для встроенного микроконтроллера или исполняемый образ, извлеченный из портативных устройств или устройств Internet of Things (IoT). Менее очевидное применение процессорного модуля - дизассемблирование инструкций пользовательской виртуальной машины, встроенной в обфусцированный исполняемый файл x86. В таких случаях существующий процессорный модуль Ghidra для x86 поможет понять только саму виртуальную машину, но не лежащий в ее основе байт-код виртуальной машины.

Если вы возьметесь за эту трудную задачу, мы хотим убедиться, что у вас есть надежная опора для этого начинания. Каждый из предыдущих примеров модулей (анализатор и загрузчик) требовал изменить один файл Java. Если вы создавали эти модули в среде Eclipse GhidraDev, вам предоставляли шаблон модуля и теги задач внутри каждого шаблона, чтобы помочь выполнить задачу. Процессорные модули сложнее, и для правильной работы процессорного модуля нужно поддерживать связи между разными файлами. Хотя в этой главе мы не будем строить процессорный модуль с нуля, мы дадим прочную основу для понимания процессорных модулей Ghidra и продемонстрируем создание и изменение компонентов внутри этих модулей.

Кто, вероятно, будет расширять Ghidra?

На основе совершенно ненаучного исследования мы сильно подозреваем, что существуют следующие категории:

Категория 1. Малый процент людей, использующих Ghidra, изменит или напишет скрипт, чтобы настроить или автоматизировать какую-либо функциональность, связанную с Ghidra.

Категория 2. Из людей в категории 1 малый процент решит изменить или разработать плагин, чтобы настроить какую-либо функциональность, связанную с Ghidra.

Категория 3. Из людей в категории 2 еще меньший процент решит изменить или написать анализатор, чтобы расширить возможности анализа Ghidra.

Категория 4. Из людей в категории 3 малый процент решит изменить или написать загрузчик для нового формата файлов.

Категория 5. Очень малый процент людей в категории 4 решит изменить или написать процессорный модуль Ghidra, потому что количество наборов инструкций, требующих декодирования, намного меньше количества форматов файлов, использующих эти наборы инструкций. Таким образом, спрос на новые процессорные модули сравнительно низок.

Чем глубже вы продвигаетесь по списку категорий, тем более специализированным становится характер связанных задач. Однако если вы сейчас не представляете себя автором процессорного модуля Ghidra, это не означает, что нет пользы в изучении того, как они устроены. Процессорные модули образуют основу, на которой построены возможности Ghidra по дизассемблированию,

ассемблированию и декомпиляции, а некоторое понимание их внутренней работы вполне может повысить вас до статуса мастера Ghidra в глазах коллег.

Понимание процессорных модулей Ghidra

Создание процессорного модуля для реальной архитектуры - крайне специализированная и трудоемкая работа, выходящая за рамки этой книги. Однако фундаментальное понимание того, как процессоры и связанные с ними наборы инструкций представлены в Ghidra, поможет понять, где искать, чтобы нужные ресурсы были под рукой, когда вам понадобится информация о процессорном модуле Ghidra.

Eclipse Processor Modules

Начнем с уже отчасти знакомой территории. Когда вы используете Eclipse -> GhidraDev для создания процессорного модуля, получившаяся структура папок в основном такая же, как у любого другого типа модуля, но процессорный модуль не предоставляет исходный файл Java с комментариями, тегами задач и списком TODO в папке `src/main/java`, как видно на рисунке 18-1.

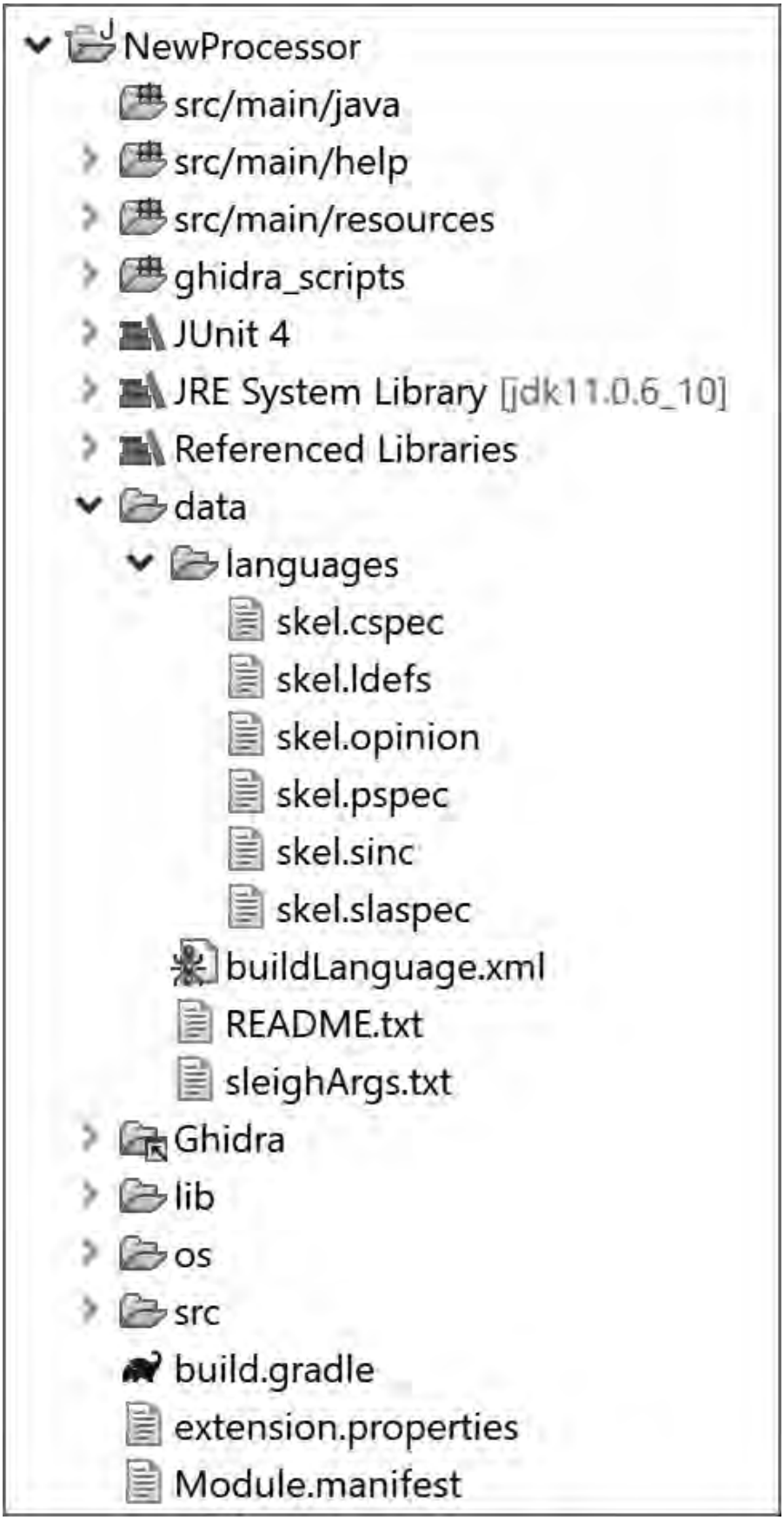


Рисунок 18-1. Содержимое процессорного модуля

Вместо этого папка `data`, развернутая на рисунке, содержит гораздо больше, чем краткий `README.txt`, предоставляемый в папке `data` для других типов модулей. Кратко познакомимся с

девятью файлами, содержащимися в папке `data`, сосредоточившись на их расширениях. Префикс `skel` сообщает нам, что мы работаем со скелетом.

skel.cspec. XML-форматированный, поначалу ошеломляющий файл спецификации компилятора.

skel.ldefs. XML-форматированный файл определения языка. В скелете есть закомментированный шаблон для определения языка.

skel.opinion. XML-форматированный файл `.opinion` импортера. В скелете есть закомментированный шаблон для определения спецификации языка/компилятора.

skel.pspec. XML-форматированный файл спецификации процессора.

skel.sinc. Обычно файл SLEIGH для инструкций языка.

skel.slaspec. Файл спецификации SLEIGH.

buildLanguage.xml. XML-файл, описывающий процесс сборки для файлов в каталоге `data/languages`.

README.txt. Такой же файл есть во всех модулях, но внутри этого модуля он наконец обретает смысл, поскольку сосредоточен на содержимом каталога `data/`.

sleighArgs.txt. Файл, содержащий параметры компилятора SLEIGH.

Файлы `.ldefs` и `.opinion` использовались в главе 17 при построении загрузчика ELF-шеллкода. Другие расширения файлов будут показаны в контексте по мере прохождения примеров. Вы узнаете, как работать с этими файлами для изменения процессорного модуля, но сначала обсудим новый термин, специфичный для процессорных модулей, - SLEIGH.[1]

SLEIGH

SLEIGH - язык, специфичный для Ghidra, который описывает наборы инструкций микропроцессоров для поддержки процессов дизассемблирования и декомпиляции Ghidra. Файлы внутри каталога `languages` (см. рисунок 18-1) либо написаны на SLEIGH, либо представлены в формате XML, поэтому вам определенно нужно немного изучить SLEIGH, чтобы создать или изменить процессорный модуль.

Спецификация того, как инструкции кодируются и как они интерпретируются процессором, содержится в файле `.slaspec`, что отчасти аналогично роли файла `.c`. Когда семейство процессоров имеет несколько отдельных вариантов, у каждого варианта может быть собственный файл `.slaspec`, тогда как общее поведение разных вариантов может быть вынесено в отдельные файлы `.sinc`, подобно роли файлов `.h`, которые могут включаться во многие файлы `.slaspec`. Процессор ARM в Ghidra - отличный пример: у него более десятка файлов `.slaspec`, каждый из которых ссылается на один или несколько из пяти файлов `.sinc`. Эти файлы составляют исходный код SLEIGH для процессорного модуля, и задача компилятора SLEIGH - скомпилировать их в файл `.sla`, пригодный для использования Ghidra.[2]

Вместо глубокого погружения в SLEIGH с теоретической точки зрения мы будем вводить различные компоненты языка SLEIGH по мере того, как будем встречать их и нуждаться в них в примерах. Но сначала посмотрим, какую информацию файл SLEIGH содержит об инструкциях.

Чтобы увидеть дополнительную информацию, связанную с инструкцией в листинге `CodeBrowser`, щелкните правой кнопкой мыши и выберите Instruction Info из контекстного меню. Отображаемая информация получена из спецификаций файла SLEIGH для выбранной инструкции. Рисунок 18-2 показывает окно Instruction Info для инструкции `PUSH` x86-64.

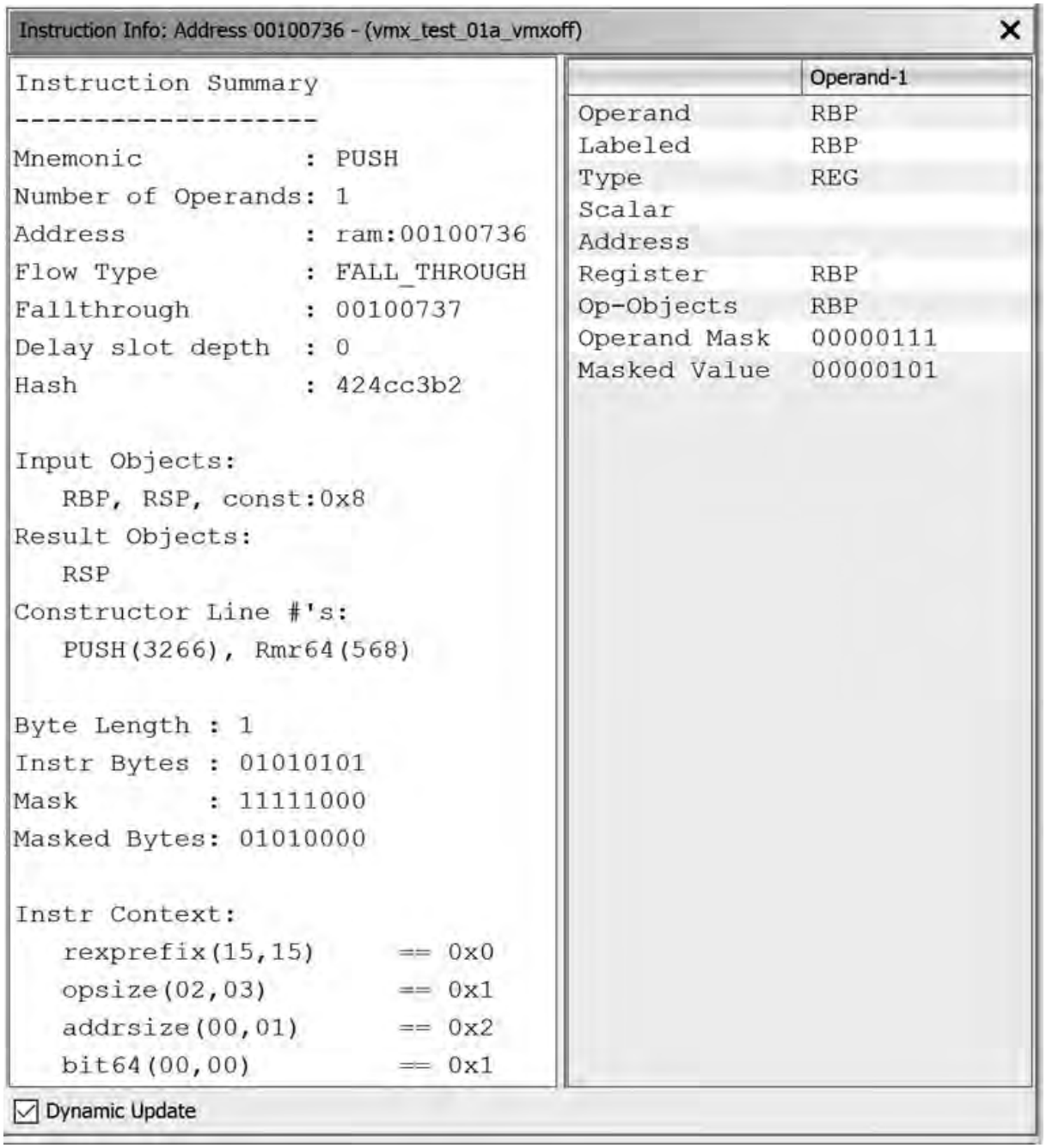


Рисунок 18-2. Окно Instruction Info для инструкции PUSH x86-64

Окно Instruction Info объединяет информацию об инструкции `PUSH` из файла SLEIGH с подробностями о конкретном использовании `PUSH` по адресу `00100736`. Позднее в главе мы будем работать с определениями инструкций внутри файла SLEIGH и вернемся к этому окну в контексте инструкций, с которыми работаем.

Руководства процессоров

Документация, предоставляемая производителем процессора, - важный ресурс для получения информации о наборе инструкций. Хотя эти материалы, защищенные авторским правом, не могут быть включены в дистрибутив Ghidra, их легко подключить через пункт контекстного меню правой кнопки мыши в окне `Listing`. Если щелкнуть любую инструкцию правой кнопкой мыши и выбрать `Processor Manual`, вы, вероятно, увидите сообщение, похожее на показанное на рисунке 18-3, сообщающее, что руководство для текущего процессора недоступно в ожидаемом месте.

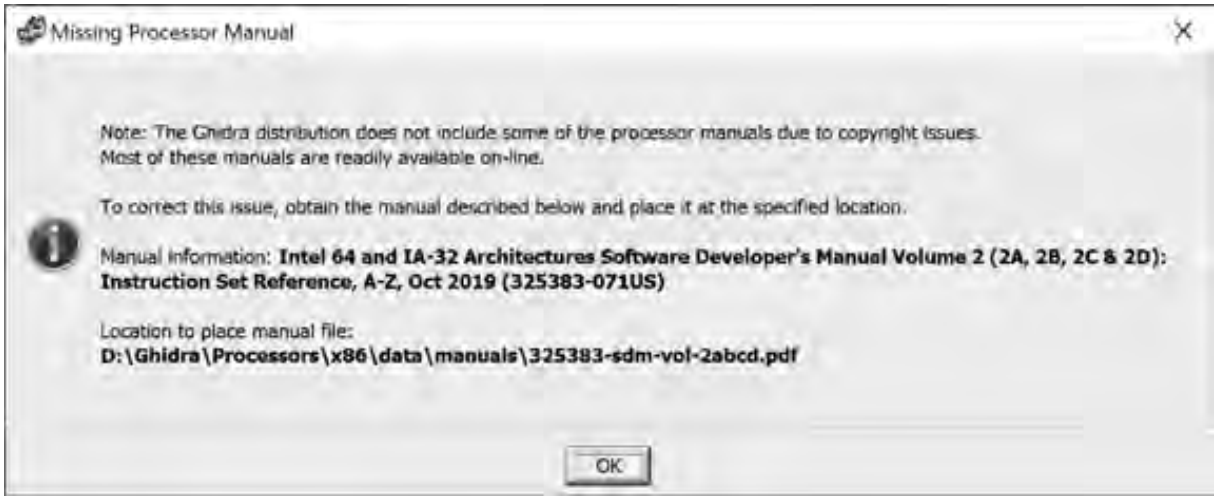


Рисунок 18-3. Диалог Missing Processor Manual

Здесь Ghidra предоставляет информацию, необходимую для разрешения ситуации с отсутствующим руководством. В этом конкретном примере сначала нужно найти руководство x86 онлайн, а затем сохранить его с указанным именем и в указанном месте.

Примечание. С x86 связано много руководств процессоров. Найдите правильное руководство онлайн, выполнив поиск по идентификатору, указанному в конце информации о руководстве: `325383-060US`.

После правильной установки руководства выбор `Processor Manual` отобразит руководство. Поскольку руководства процессоров обычно велики (это конкретное руководство процессора x86 почти 2200 страниц), Ghidra предусмотрительно включает возможность обрабатывать индексные файлы, сопоставляющие инструкцию с конкретной страницей руководства. К счастью, индекс для этого конкретного руководства x86 уже создан для вас.

Руководства процессоров следует помещать в каталог `Ghidra/Processors/<proc>/data/manuals`, соответствующий вашему процессору. Индексные файлы должны находиться в том же каталоге, что и связанное с ними руководство. Формат индексного файла относительно прост. Первые несколько строк файла `x86.idx` из Ghidra показаны ниже:

```
@Intel64_IA32_SoftwareDevelopersManual.pdf [Intel 64 and IA-32 Architectures
     Software Developer's Manual Volume 2 (2A, 2B, 2C & 2D): Instruction Set
     Reference, A-Z, Sep 2016 (325383-060US)]
AAA, 120
AAD, 122
BLENDPS, 123
AAM, 124
```

Первая строка в файле (в этом листинге перенесенная на три строки) связывает локальное имя файла руководства с описательным текстом, отображаемым пользователю, когда руководство отсутствует в системе. Формат строки:

```
@FilenameInGhidraManualDirectory [Description of manual file]
```

Каждая дополнительная строка имеет форму `INSTRUCTION, page`. Инструкция должна быть записана прописными буквами, а номер страницы отсчитывается от первой страницы файла `.pdf`. Это не обязательно номер страницы, который отображается на любой конкретной странице документа.

В одном файле `.idx` можно ссылаться на несколько руководств. Просто используйте дополнительные директивы `@`, чтобы отделить карту инструкций каждого дополнительного руководства. Дополнительную информацию об индексных файлах руководств процессоров можно найти в `docs/languages/manual_index.txt` в каталоге установки Ghidra.

После того как руководство сохранено и проиндексировано, выбор `Processor Manual` для любой инструкции в окне `Listing` должен перенести вас на соответствующую страницу внутри руководства. Если руководство не появляется, возможно, нужно выбрать `Edit -> Tool Options -> Processor Manuals`, чтобы настроить подходящее приложение просмотра для руководства. Пример настройки средства просмотра для открытия руководства с помощью Firefox показан на рисунке 18-4.

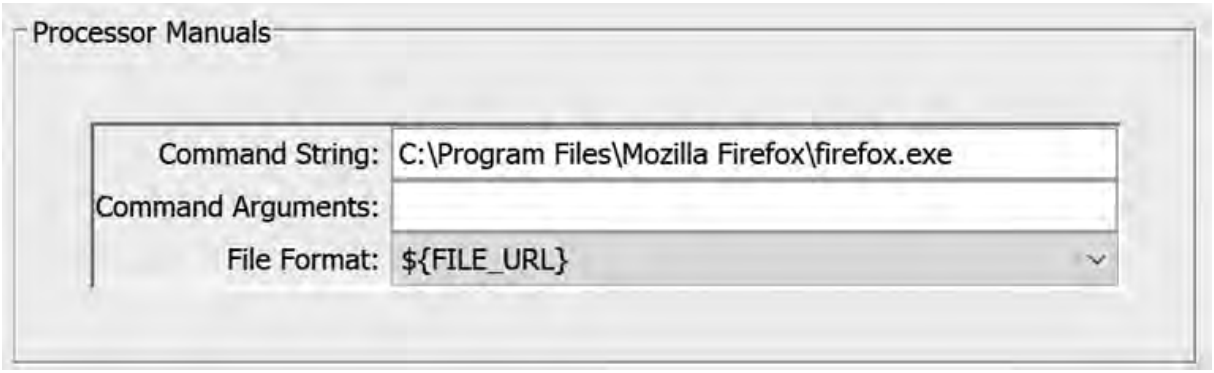


Рисунок 18-4. Параметры инструмента `Processor Manuals`

Теперь, когда у вас есть базовая терминология процессорных модулей, пора погрузиться во внутреннее устройство реализации процессорного модуля.

Изменение процессорного модуля Ghidra

Построение процессорного модуля с нуля - значительное предприятие. Вместо того чтобы бросаться в него с головой, мы начнем, как и в предыдущих примерах, с изменения существующего модуля. Поскольку мы хотим продемонстрировать концепции, связанные с реальными проблемами, начнем с выявления гипотетической проблемы, касающейся процессорного модуля x86 в Ghidra. Мы пройдем примеры, решающие эту проблему, а затем используем изученный материал, чтобы создать общую картину того, как разные компоненты работают вместе и образуют полный процессорный модуль Ghidra.

Постановка проблемы

Быстрый поиск в каталоге `Ghidra/Processors` локальной установки показывает, что процессорный модуль x86 включает множество инструкций, но, похоже, в нем отсутствует гипотетическая инструкция управления расширением виртуальной машины (VMX) для архитектур IA32 и IA64.[3] Эта инструкция, придуманная для данного примера, называется `VMXPLODE`. Ее поведение похоже на инструкцию `VMXOFF`, которую Ghidra поддерживает. В то время как существующая инструкция `VMXOFF` заставляет процессор покинуть режим VMX, `VMXPLODE` делает это с размахом. Мы пройдем добавление этой очень важной инструкции в существующий процессорный модуль Ghidra для x86, чтобы представить концепции, связанные с построением и изменением процессорного модуля.

SLEIGH Editor в Ghidra

Чтобы помочь в изменении и построении процессорных модулей, Ghidra включает редактор SLEIGH, который легко интегрируется в среду Eclipse. Инструкции по установке редактора являются частью файла README SLEIGH, упомянутого ранее, и занимают всего несколько шагов. Специальная функциональность, поддерживаемая редактором, включает следующее:

Syntax highlight. Раскрашивает содержимое, имеющее специальный смысл, например комментарии, токены, строки, переменные и так далее.

Validation. Помечает многие синтаксические ошибки и генерирует предупреждения для ошибок, которые иначе остались бы необнаруженными до компиляции.

QuickFix. Предоставляет рекомендации по устранению проблем, обнаруженных редактором. Это похоже на варианты QuickFix для операторов импорта из главы 15.

Hover. Предоставляет дополнительную информацию для многих конструкций, когда вы наводите указатель на конструкцию.

Navigation. Предоставляет функциональность навигации, специфичную для SLEIGH, например по подконструкторам, токенам, регистрам, pcodeops и так далее.

Find references. Быстро находит все использования переменной.

Renaming. Вместо традиционного строкового поиска и замены переименовывает фактическую переменную в файле и других связанных файлах `.sinc` и `.slaspec`.

Code formatting. Переформатирует файлы в соответствии со структурой языка SLEIGH, например выравнивает конструкторы по ключевым словам и записи внутри `attach`. Эту функциональность можно применить ко всему файлу или к выбранному разделу.

Хотя мы рекомендуем использовать этот редактор, особенно ради полезной ранней проверки синтаксиса, разработка примеров в этой главе не привязана к этому редактору.

Пример 1: добавление инструкции в процессорный модуль

Наша первая цель - найти файлы, которые нужно изменить для поддержки инструкции `VMXPLODE`. Каталог `Ghidra/Processors` содержит подкаталоги для всех процессоров, поддерживаемых Ghidra, включая x86. Можно открыть процессорный модуль x86 или любой другой процессорный модуль прямо в Eclipse, используя `File -> Open Projects from File System or Archive` и указав путь к папке процессора (`Ghidra/Processors/x86`). Это свяжет ваш экземпляр Eclipse с процессорным модулем Ghidra для x86, а значит, изменения, сделанные в Eclipse, будут напрямую отражаться в процессорном модуле Ghidra.

Частично развернутая версия модуля x86 в Eclipse, точно отражающая связанную структуру каталогов Ghidra, показана на рисунке 18-5. Загруженное вами руководство процессора присутствует вместе с индексным файлом x86.

Рисунок 18-5. Процессорный модуль x86 в Eclipse Package Explorer

Папка x86 содержит папку `data`, похожую на ту, которую вы видели в процессорном модуле, созданном с помощью Eclipse -> GhidraDev. Внутри этой папки находится папка `languages`, содержащая более 40 файлов, включая 19 файлов `.sinc`, определяющих инструкции языка. Поскольку набор инструкций x86 довольно велик, он разбит на файлы, группирующие похожие инструкции. Вместо создания нового файла `.sinc` для нашей инструкции мы добавим ее в существующий файл `.sinc` x86. Если бы мы добавляли в Ghidra новую группу инструкций, например набор инструкций x86 SGX, мы могли бы создать новый файл `.sinc`, чтобы сгруппировать их вместе. Фактически инструкции SGX сгруппированы в общем файле `sgx.sinc`; на него и приходится один из множества файлов `.sinc`.

Поиском по файлам `.sinc` мы обнаруживаем, что `ia.sinc` содержит определения существующего набора инструкций VMX. Мы используем определение `VMXOFF` в `ia.sinc` как модель для определения `VMXPLODE`. `VMXOFF` упоминается в двух разных разделах внутри `ia.sinc`. Первый раздел - это определения инструкций аппаратно-ассистированной виртуализации Intel IA:

```
# MFL: определения инструкций аппаратной виртуализации Intel IA
define pcodeop invept;   # Аннулировать трансляции, полученные из расширенных
                         # таблиц страниц (EPT); опкод 66 0f 38 80
# -----СОДЕРЖИМОЕ ЗДЕСЬ ОПУЩЕНО-----
define pcodeop vmread;   # Читать поле из управляющей структуры виртуальной
    машины;
                         # опкод 0f 78
define pcodeop vmwrite;  # Записать поле в управляющую структуру виртуальной
    машины;
                         # opcode 0f 79
define pcodeop vmxoff;   # Выйти из режима VMX; опкод 0f 01 c4
define pcodeop vmxon;    # Войти в режим VMX; опкод f3 0f C7 /6
```

Каждая запись в разделе определений определяет `pcodeop`, новую микрокодовую операцию для архитектуры x86. Определение включает имя и, в этом случае, комментарий с описанием и опкодом. Нам нужно заполнить комментарий для нашей новой команды. Быстрый веб-поиск в альтернативной реальности с небольшим тестированием подтверждает, что опкод `0f 01 c5` давно зарезервирован для `VMXPLODE`. Теперь у нас есть информация, необходимая для добавления новой инструкции в файл:

```
define pcodeop vmxoff;   # Выйти из режима VMX; опкод 0f 01 c4
define pcodeop vmxplode; # Взорвать фиктивную операцию VMX; опкод 0f 01 c5
define pcodeop vmxon;    # Войти в режим VMX; опкод f3 0f C7 /6
```

Второе место, где мы встречаем `VMXOFF` внутри `ia.sinc` и куда вставим новую инструкцию, - раздел определения опкодов. В содержимом ниже некоторые строки опущены для ясности, а некоторые строки определения инструкций перенесены ради удобства чтения:

```
# Опкоды аппаратной виртуализации Intel
# -----СОДЕРЖИМОЕ ЗДЕСЬ ОПУЩЕНО-----
# TODO: вызывает функцию VM, заданную в EAX
:VMFUNC EAX     is vexMode=0 & byte=0x0f; byte=0x01; byte=0xd4 & EAX     { vmfunc(
    EAX); }
# TODO: запускает VM, управляемую текущим VMCS. Как VMCS
#       выражается для эмулятора? Для анализа Ghidra?
:VMLAUNCH       is vexMode=0 & byte=0x0f; byte=0x01; byte=0xc2           {
    vmlaunch(); }
# TODO: возобновляет VM, управляемую текущим VMCS. Как VMCS
#       выражается для эмулятора? Для анализа Ghidra?
:VMRESUME       is vexMode=0 & byte=0x0f; byte=0x01; byte=0xc3           {
    vmresume(); }
# -----СОДЕРЖИМОЕ ЗДЕСЬ ОПУЩЕНО-----
:VMWRITE Reg32, rm32 is vexMode=0 & opsize=1 & byte=0x0f; byte=0x79;
         rm32 & Reg32 ... & check_Reg32_dest ... { vmwrite(rm32,Reg32); build
    check_Reg32_dest; }
@ifdef IA64
:VMWRITE Reg64, rm64 is vexMode=0 & opsize=2 & byte=0x0f;  byte=0x79;
         rm64 & Reg64 ...    { vmwrite(rm64,Reg64); }
@endif
:VMXOFF         is vexMode=0 & byte=0x0f; byte=0x01; byte=0xc4         { vmxoff();
     }
:VMXPLODE       is vexMode=0 & byte=0x0f; byte=0x01; byte=0xc5         { vmxplode(
    ); }
# -----СОДЕРЖИМОЕ ЗДЕСЬ ОПУЩЕНО-----
# КОНЕЦ изменений для опкодов VMX
```

Комментарии TODO, встречающиеся во многих файлах Ghidra, обозначают задачи, которые еще предстоит выполнить. Поиск задач TODO в файлах Ghidra - отличный способ найти возможности внести вклад в этот проект с открытым исходным кодом.

Далее мы видим инструкцию VMWRITE для 32-битной и 64-битной архитектур. 64-битная инструкция окружена проверкой, чтобы гарантировать, что она включается только в 64-битный файл .sla. Хотя 32-битные инструкции допустимы в 64-битном мире (например, EAX - это 32 младших бита RAX), обратное неверно. Условный оператор гарантирует, что инструкции, работающие с 64-битными регистрами, включаются только для 64-битных сборок.

Инструкция VMXOFF напрямую не задействует регистры, поэтому нет необходимости различать 32- и 64-битные версии инструкции. Конструктор для нашей новой инструкции VMXPLODE, вместе с ее новым опкодом, очень похож на конструктор для VMXOFF. Разберем строку на компоненты:

```
:VMXPLODE
```

Это определяемая инструкция, которая отображается в дизассемблерном листинге.

```
is vexMode=0 & byte=0x0f; byte=0x01; byte=0xc5
```

Это битовые шаблоны, связанные с инструкцией, которые задают ограничение для инструкции.[4] & представляет логическое И. Точки с запятой выполняют двойную роль конкатенации и логического И. Эта часть говорит: если мы не находимся в режиме VEX и опкод состоит из этих трех байтов в таком порядке, то это ограничение выполнено.

```
{ vmxplode(); }
```

Фигурные скобки заключают раздел семантических действий инструкции. Компилятор SLEIGH переводит эти действия во внутреннюю форму Ghidra, известную как p-code, которая обсуждается далее. Определение инструкции требует понимания операторов и синтаксиса SLEIGH. Эта часть конструктора, где выполняется реальная работа, связанная с большинством инструкций, может быстро превратиться в сложную последовательность операторов, разделенных точками с запятой. В этом случае, поскольку VMXPLODE определена как новая операция p-code (`define pcodeop vmxplode;`), мы можем вызвать инструкцию здесь. В будущих примерах мы добавим в этот раздел дополнительные семантические действия SLEIGH.

Самый большой файл .sinc для x86 - это ia.sinc, потому что внутри него определено много инструкций, включая нашу новую VMXPLODE, а также значительный объем содержимого для определения атрибутов процессора x86, например порядка байтов, регистров, контекстов, токенов и переменных. Большая часть специфичного для x86 содержимого внутри ia.sinc не дублируется в других файлах .sinc, поскольку все файлы .sinc, в свою очередь, включаются в файл спецификации SLEIGH (.slaspec).

Два файла .slaspec для x86, x86.slaspec и x86-64.slaspec, каждый содержат операторы включения для требуемых файлов .sinc. Можно отказаться от использования файлов .sinc и напрямую включить содержимое в .slaspec, что может иметь смысл для процессора с небольшим набором инструкций. Содержимое x86-64.slaspec:

```
@define IA64 "IA64"         # Только в x86-64.slaspec
@include "ia.sinc"
@include "avx.sinc"
@include "avx_manual.sinc"
@include "avx2.sinc"
@include "avx2_manual.sinc"
@include "rdrand.sinc"      # Только в x86-64.slaspec
@include "rdseed.sinc"      # Только в x86-64.slaspec
@include "sgx.sinc"         # Только в x86-64.slaspec
@include "adx.sinc"
@include "clwb.sinc"
@include "pclmulqdq.sinc"
@include "mpx.sinc"
@include "lzcnt.sinc"
@include "bmi1.sinc"
@include "bmi2.sinc"
@include "sha.sinc"
@include "smx.sinc"
```

```
@include "cet.sinc"
@include "fma.sinc"         # Только в x86-64.slaspec
```

Комментарии конца строки обозначают содержимое, уникальное для `x86-64.slaspec`. Файл `x86.slaspec` является подмножеством `x86-64.slaspec`. Среди включенных файлов есть `ia.sinc`, где мы определили `VMXPLODE`, поэтому ничего добавлять не нужно. Если вы создаете новый файл `.sinc`, нужно добавить оператор включения и в `x86.slaspec`, и в `x86-64.slaspec`, чтобы инструкция распознавалась и в 32-, и в 64-битных бинарных файлах.

Чтобы проверить, может ли Ghidra распознать новую инструкцию при ее использовании в бинарном файле, мы создаем тестовый файл. Файл сначала проверяет, что `VMXOFF` по-прежнему распознается, а затем проверяет, что `VMXPLODE` успешно добавлена. Исходный файл C для тестирования `VMXOFF` содержит следующее:

```
#include <stdio.h>
// Следующая функция объявляет ассемблерный блок и сообщает
// компилятору, что код нужно выполнить без перемещения или изменения.
void do_vmx(int v) {
   asm volatile (
      "vmxon %0;"       // Включить работу гипервизора
      "vmxoff;"         // Отключить работу гипервизора
      "nop;"            // Короткая nop-площадка для примеров
      "nop;"
      "nop;"
      "nop;"
      "nop;"
      "nop;"
      "nop;"
      "vmxoff;"         // Отключить работу гипервизора
      :
      :"m"(v)           // Содержит входную переменную
      :
   );
}
int main() {
   int x;
   printf("Введите целое число: ");
   scanf("%d", &x);
   printf("После ввода x=%d\n", x);
   do_vmx(x);
   printf("После do_vmx x=%d\n", x);
   return 0;
}
```

Когда скомпилированный бинарный файл загружен в Ghidra, в окне `Listing` мы видим следующее тело функции `do_vmx`:

```
0010071a 55             PUSH     RBP
0010071b 48 89 e5       MOV      RBP,RSP
0010071e 89 7d fc       MOV      dword ptr [RBP + local_c],EDI
00100721 f3 0f c7       VMXON    qword ptr [RBP + local_c]
         75 fc
00100726 0f 01 c4       VMXOFF
00100729 90             NOP
0010072a 90             NOP
0010072b 90             NOP
0010072c 90             NOP
0010072d 90             NOP
0010072e 90             NOP
0010072f 90             NOP
00100730 0f 01 c4       VMXOFF
00100733 90             NOP
00100734 5d             POP RBP
00100735 c3             RET
```

Байты, отображаемые для опкода (`0f 01 c4`) в двух вызовах `VMXOFF`, совпадают с опкодом, который мы наблюдали в `ia.sinc` для этой команды. Следующий листинг из окна `Decompiler` согласуется с тем, что мы знаем об исходном коде и связанном дизассемблировании:

```
void do_vmx(undefined4 param_1)
{
   undefined4 unaff_EBP;
   vmxon(CONCAT44(unaff_EBP,param_1));
   vmxoff();
   vmxoff();
   return;
}
```

Чтобы проверить, что Ghidra обнаруживает инструкцию `VMXPLODE`, замените первое вхождение `VMXOFF` в тестовой функции `do_vmx` на `VMXPLODE`. Однако `VMXPLODE` отсутствует не только в процессорном определении Ghidra, но и в базе знаний компилятора. Чтобы ассемблер принял код, мы вручную собрали инструкцию, используя объявление данных вместо непосредственного использования мнемоники инструкции:

```
//"vmxoff;"                 // заменить эту строку
".byte 0x0f, 0x01, 0xc5;"   // этой вручную собранной строкой
```

Когда обновленный бинарный файл загружен в Ghidra, окно `Listing` показывает:

```
0010071a 55 PUSH RBP
0010071b 48 89 e5 MOV RBP,RSP
0010071e 89 7d fc MOV dword ptr [RBP + local_c],EDI
00100721 f3 0f c7 VMXON qword ptr [RBP + local_c]
         75 fc
00100726 0f 01 c5 VMXPLODE
00100729 90 NOP
0010072a 90 NOP
0010072b 90 NOP
0010072c 90 NOP
0010072d 90 NOP
0010072e 90 NOP
0010072f 90 NOP
00100730 0f 01 c4 VMXOFF
00100733 90 NOP
00100734 5d POP RBP
00100735 c3 RET
```

Новая инструкция появляется вместе с назначенным ей опкодом (`0f 01 c5`). Окно `Decompiler` также показывает новую инструкцию:

```
void do_vmx(undefined4 param_1)
{
   undefined4 unaff_EBP;
   vmxon(CONCAT44(unaff_EBP,param_1));
   vmxplode();
   vmxoff();
   return;
}
```

Какую работу Ghidra выполнила в фоновом режиме, чтобы добавить новую инструкцию в набор инструкций процессора x86? Когда Ghidra перезапускается, что необходимо для вступления этих изменений в силу, она обнаруживает, что базовый файл `.sinc` изменился, и при необходимости генерирует новый файл `.sla`.

В этом примере при загрузке исходного скомпилированного 64-битного бинарного файла Ghidra обнаружила изменение в `ia.sinc` и показала окно на рисунке 18-6, пока перекомпилировала `ia.sinc`. Она перекомпилирует только при необходимости, а не автоматически при перезапуске. Поскольку мы загрузили 64-битный файл, был обновлен только `x86-64.sla`, а не `x86.sla`. Позднее, когда мы загрузили обновленный файл с `VMXPLODE`, Ghidra не выполняла перекомпиляцию, так как с предыдущей загрузки никаких изменений в базовые исходные файлы SLEIGH внесено не было.

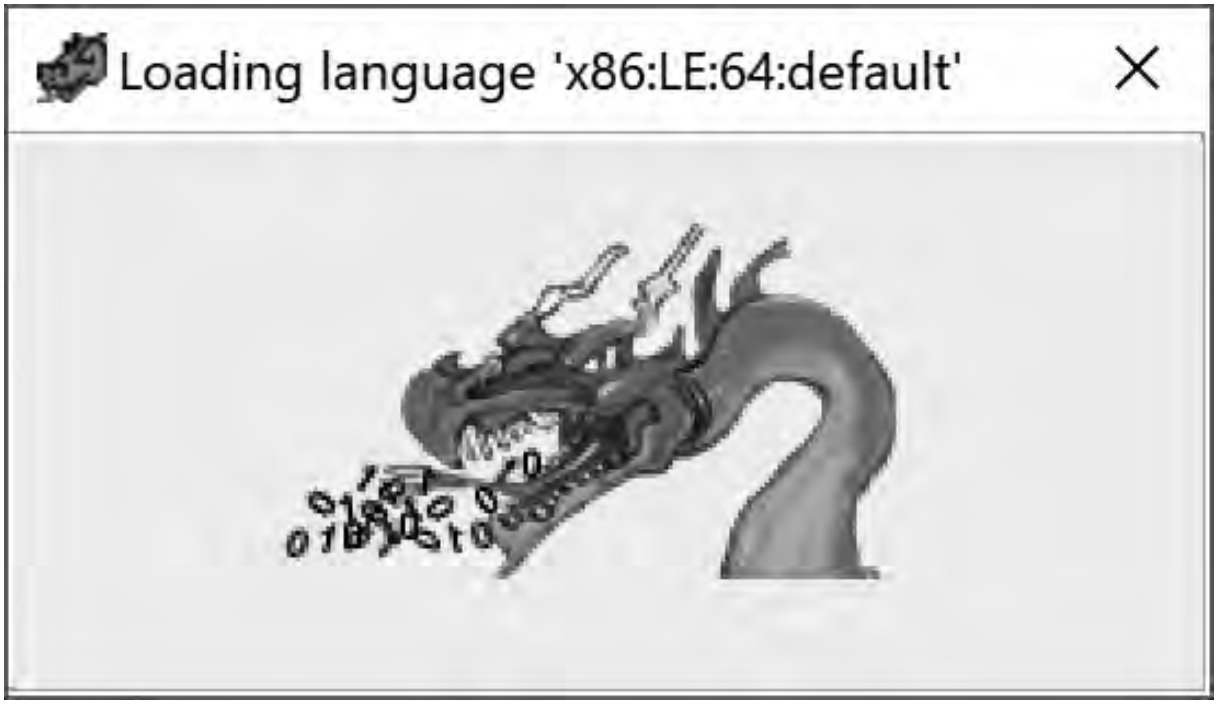


Рисунок 18-6. Окно Ghidra при перекомпиляции файла языка

Вот краткий список шагов для добавления новой инструкции в процессорный модуль:

1. Найти каталог `languages` для целевого процессора, например `Ghidra/Processor/<<targetprocessor>>/data/languages`.
2. Добавить инструкцию в выбранный файл процессора `.sinc` или создать новый файл `.sinc`, например `Ghidra/Processor/<targetprocessor>/data/languages/<targetprocessor>.sinc`.
3. Если вы создали новый файл `.sinc`, убедиться, что он включен в файл `.slaspec`, например `Ghidra/Processor/<targetprocessor>/data/languages/<targetprocessor>.slaspec`.

Пример 2: изменение инструкции в процессорном модуле

Мы успешно добавили инструкцию в процессорный модуль Ghidra для x86, но еще не достигли цели заставить `VMXPLODE` выходить с эффектным жестом. Сейчас она просто выходит без какого-либо волнения. Хотя заставить инструкцию языка ассемблера сделать что-либо, что можно было бы счесть эффектным жестом, непросто, мы можем заставить нашу инструкцию dab при выходе. В этом примере мы пройдем три варианта, как заставить `VMXPLODE` сделать dab. Первый вариант: выйти после установки `EAX` в жестко заданное значение `0xDAB`.

Вариант 1: установить EAX в постоянное значение

Чтобы инструкция `VMXPLODE` перед выходом устанавливала значение `EAX` в `0xDAB`, требуется лишь небольшое изменение одной инструкции в том же файле (`ia.sinc`). Следующий листинг показывает инструкции `VMXOFF` и `VMXPLODE` в том виде, в каком мы оставили их после примера 1:

```
:VMXOFF         is vexMode=0 & byte=0x0f; byte=0x01; byte=0xc4      { vmxoff(); }
:VMXPLODE       is vexMode=0 & byte=0x0f; byte=0x01; byte=0xc5      { vmxplode();
    }
```

Внутри содержимого инструкции добавьте присваивание EAX непосредственно перед действием vmxplode, как показано в следующем листинге:

```
:VMXOFF         is vexMode=0 & byte=0x0f; byte=0x01; byte=0xc4      { vmxoff(); }
:VMXPLODE       is vexMode=0 & byte=0x0f; byte=0x01; byte=0xc5      { EAX=0xDAB;
    vmxplode(); }
```

Когда мы снова открываем Ghidra и загружаем тестовый файл, Ghidra снова отображает окно с рисунка 18-6, сообщая, что обнаружила изменение в связанном файле языка и регенерирует x86-64.sla. Окно Listing после автоматического анализа не показывает никаких изменений, но разница очевидна в окне Decompiler:

```
undefined4 do_vmx(undefined4 param_1)
{
   undefined4 unaff_EBP;
   vmxon(CONCAT44(unaff_EBP,param_1));
   vmxplode();
   vmxoff();
   return 0xdab;
}
```

В окне Decompiler оператор return теперь возвращает содержимое EAX (0xDAB). Это интересно, потому что мы знаем, что это функция void и она не имеет возвращаемого значения. Запись окна Listing для новой инструкции не показывает, что команда VMXPLODE как-либо изменилась:

```
00100726 0f 01 c5       VMXPLODE
```

Важное различие между декомпиляторами и дизассемблерами состоит в том, что декомпиляторы понимают и включают полное семантическое поведение каждой инструкции как часть анализа, тогда как дизассемблеры в основном сосредоточены на правильном синтаксическом представлении каждой инструкции. В этом примере VMXPLODE не принимает операндов и правильно отображается дизассемблером, не давая визуальной подсказки о том, что EAX изменился. При чтении дизассемблирования полностью ваша ответственность - понимать семантическое поведение каждой инструкции. Этот пример также демонстрирует ценность декомпилятора: понимая полную семантику VMXPLODE, он способен распознать, что EAX изменяется как побочный эффект инструкции. Декомпилятор также распознает, что EAX не используется в оставшейся части функции, и предполагает, что значение предназначено для возврата вызывающей функции.

Ghidra дает возможность глубже погрузиться в то, как работают инструкции, и выявлять и тестировать тонкие различия. Сначала посмотрим на часть информации об инструкции, связанной с VMXPLODE, показанную на рисунке 18-7.

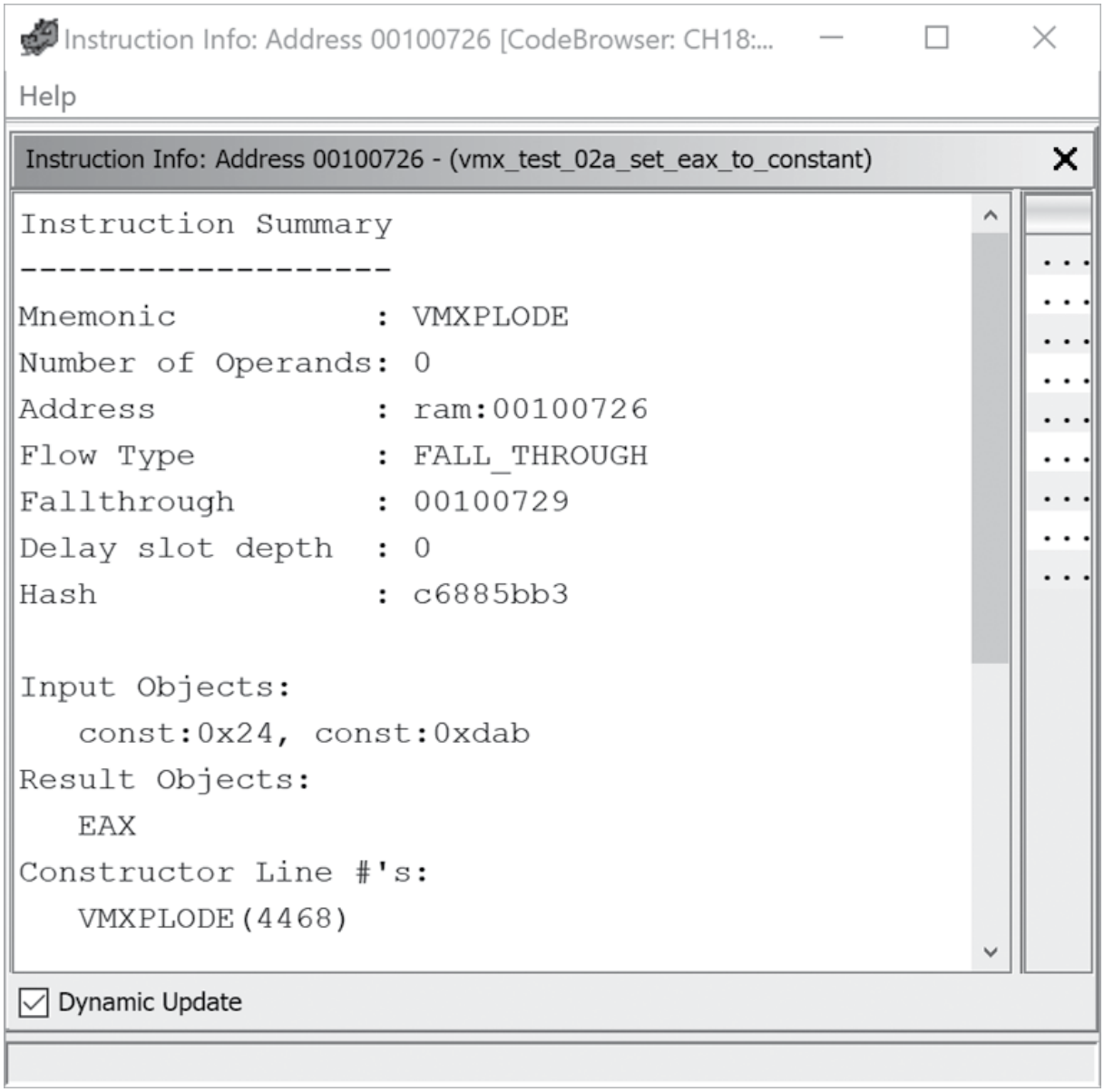


Рисунок 18-7. Информация об инструкции VMXPLODE

Слева находится исходная инструкция `VMXPLODE`, а справа - измененная версия, где `0xdab` указан в разделе Input Objects, а `EAX` - в Result Objects. Дополнительное понимание любой инструкции можно получить, посмотрев на базовую информацию, называемую p-code, которую мы ранее не рассматривали.[5] P-code, связанный с инструкцией, может быть очень информативен относительно того, что именно делает инструкция.

P-code: насколько низко можно спуститься?

Документация Ghidra описывает p-code как "язык регистровых передач, предназначенный для приложений обратной разработки". Язык регистровых передач (RTL) - архитектурно-независимый язык, похожий на язык ассемблера, который часто используется как промежуточное представление (IR, или IL) между высокоуровневым языком, таким как C, и целевым языком ассемблера, таким как x86 или ARM. Компиляторы часто состоят из фронтенда, специфичного для языка, который переводит исходный код в IR, и бэкенда, специфичного для архитектуры, который переводит IR в конкретный язык ассемблера. Такая модульность позволяет объединить фронтенд C с бэкендом x86, чтобы создать C-компилятор, производящий x86-код, и дает гибкость заменить бэкенд на модуль ARM, чтобы сразу получить C-компилятор, генерирующий ARM-код. Замените фронтенд C на фронтенд FORTRAN, и теперь у вас есть компилятор FORTRAN для ARM.

Работа на уровне IR позволяет строить инструменты, работающие с IR, вместо того чтобы поддерживать набор C-специфичных или ARM-специфичных инструментов, бесполезных для других языков или архитектур. Например, как только у нас есть оптимизатор, работающий с IR, его можно повторно использовать с любой комбинацией фронтенда и бэкенда, не переписывая оптимизатор каждый раз.

Цепочка инструментов обратной разработки, что неудивительно, работает в направлении, противоположном традиционной цепочке сборки ПО. RE-фронтенд должен переводить машинный код в IR; этот процесс часто называется лифтингом, тогда как RE-бэкенд переводит IR в высокоуровневый язык, например C. Чистый дизассемблер не считается фронтендом по этому определению, потому что он доводит нас только от машинного кода до языка ассемблера. Декомпилятор Ghidra - это бэкенд из IR в C. Процессорные модули Ghidra - это фронтенды из машинного кода в IR.

Когда вы строите или изменяете процессорный модуль Ghidra в SLEIGH, одно из первых действий - сообщить компилятору SLEIGH о новых операциях p-code, которые нужно ввести для описания семантических действий любых новых или измененных инструкций. Например, определение операции

```
define pcodeop vmxplode
```

которое мы добавили в `ia.sinc`, сообщает компилятору SLEIGH, что `vmxplode` является допустимым семантическим действием, доступным для описания поведения любой инструкции в нашей архитектуре. Одна из самых сложных задач, с которыми вы столкнетесь, - описать каждую новую или измененную инструкцию с помощью последовательности синтаксически корректных операторов SLEIGH, которые правильно описывают действия, связанные с инструкцией. Вся эта информация фиксируется в `.slaspec` и включенных файлах `.sinc`, из которых состоит процессор. Если вы выполните достаточно хорошую работу, Ghidra бесплатно выдаст вам бэкенд декомпилятора.

Чтобы просмотреть p-code внутри окна `Listing`, откройте Browser Field Formatter, выберите вкладку Instruction/Data, щелкните правой кнопкой мыши полосу P-code и включите поле. После того как окно `Listing` начнет отображать p-code, связанный с каждой инструкцией, можно сравнить два предыдущих листинга и увидеть различия. С включенным p-code первая реализация `VMXPLODE` выглядит следующим образом:

```
0010071b 48 89 e5       MOV      RBP,RSP
                                            RBP = COPY RSP
                                            $U620:8 = INT_ADD RBP, -4:8
                                            $U1fd0:4 = COPY EDI
                                            STORE ram($U620), $U1fd0
00100721 f3 0f c7 75 fc VMXON    qword ptr [RBP + local_c]
                                            $U620:8 = INT_ADD RBP, -4:8
                                            $Ua50:8 = LOAD ram($U620)
                                            CALLOTHER "vmxon", $Ua50
00100726 0f 01 c5       VMXPLODE
                                            CALLOTHER "vmxplode"
00100729 90             NOP
```

А вот измененная `VMXPLODE`:

```
00100726 0f 01 c5       VMXPLODE
                                            EAX = COPY 0xdab:4
                                            CALLOTHER "vmxplode"
```

Связанный p-code теперь показывает, что постоянное значение (`0xdab`) перемещается в `EAX`.

Вариант 2: установить регистр, определенный операндом, в постоянное значение

Наборы инструкций обычно состоят из смеси инструкций, работающих с нулем или большим количеством операндов. По мере увеличения количества и типов операндов, связанных с инструкцией, растет и сложность описания семантики инструкции. В этом примере мы расширим поведение `VMXPLODE`, чтобы она требовала один регистровый операнд, который будет выполнять dab. Для этого понадобится посетить разделы `ia.sinc`, с которыми мы раньше не сталкивались. На этот раз начнем с измененной версии инструкции и пойдем назад:

```
:VMXPLODE   Reg32 is vexMode=0 & byte=0x0f; byte=0x01; byte=0xc5; Reg32
        {  Reg32=0xDAB; vmxplode(); }
```

Здесь `Reg32` объявлен как локальный идентификатор, а затем конкатенируется с опкодом, становясь частью ограничений, связанных с инструкцией. Вместо присваивания `0xDAB` непосредственно в `EAX`, как мы делали ранее, инструкция теперь присваивает значение в `Reg32`. Чтобы достичь цели, нужно определить способ связать значение в `Reg32` с выбранным регистром x86. Исследуем другие компоненты внутри `ia.sinc`.

Вблизи начала `ia.sinc` мы видим все определения, которые понадобятся всей спецификации, как показано в листинге 18-1.

```
# Файл спецификации SLA для Intel x86
@ifdef IA64
@define SIZE     "8"
@define STACKPTR "RSP"
@else
@define SIZE     "4"
@define STACKPTR "ESP"
@endif
define endian=little;
define space ram type=ram_space size=$(SIZE) default;
define space register type=register_space size=4;
# Регистры общего назначения
@ifdef IA64
```

```
define register offset=0 size=8 [ RAX    RCX    RDX    RBX    RSP    RBP    RSI
     RDI ];
define register offset=0 size=4    [ EAX _  ECX _  EDX _  EBX _  ESP _  EBP _
    ESI _  EDI ];
define register offset=0 size=2    [ AX _ _ _ CX _ _ _ DX _ _ _ BX];       #
    усечено
define register offset=0 size=1    [ AL AH _ _ _ _ _ _ CL CH _ _ _ _ _ _]; #
    усечено
define register offset=0x80 size=8 [ R8    R9    R10    R11    R12    R13    R14
      R15 ];
define register offset=0x80 size=4 [ R8D _ R9D _ R10D _ R11D _ R12D _ R13D _ R14D
    _ R15D ];
define register offset=0x80 size=2 [ R8W _ _ _ R9W _ _ _ R10W _ _ _ R11W ];  #
    усечено
define register offset=0x80 size=1 [ R8B _ _ _ _ _ _ _ R9B _ _ _ _ _ _ _ ];  #
    усечено
@else
define register offset=0 size=4    [  EAX  ECX  EDX  EBX  ESP  EBP  ESI  EDI ];
define register offset=0 size=2    [  AX _ CX _ DX _ BX _ SP _ BP _ SI _ DI ];
define register offset=0 size=1    [  AL AH _ _ CL CH _ _ DL DH _ _ BL BH ];
@endif
```

Листинг 18-1. Частичная спецификация SLEIGH для регистров x86, адаптированная из `ia.sinc`

В верхней части файла мы видим имя и размер указателя стека для 32- и 64-битных сборок, а также порядок байтов для x86. Комментарий вводит начало определений регистров общего назначения. Как и для всех остальных компонентов, в SLEIGH есть специальное соглашение об именовании и определении регистров: регистры находятся в специальном адресном пространстве с именем `register`, и каждому регистру, который может занимать один или больше байтов, назначается смещение внутри адресного пространства. Определение регистра SLEIGH указывает смещение, с которого список регистров начинается внутри адресного пространства регистров. Все регистры в списке регистров непрерывны, если только символ подчеркивания не используется для создания промежутка между ними. Раскладка адресного пространства 64-битных регистров `RAX` и `RCX` подробнее показана на рисунке 18-8.

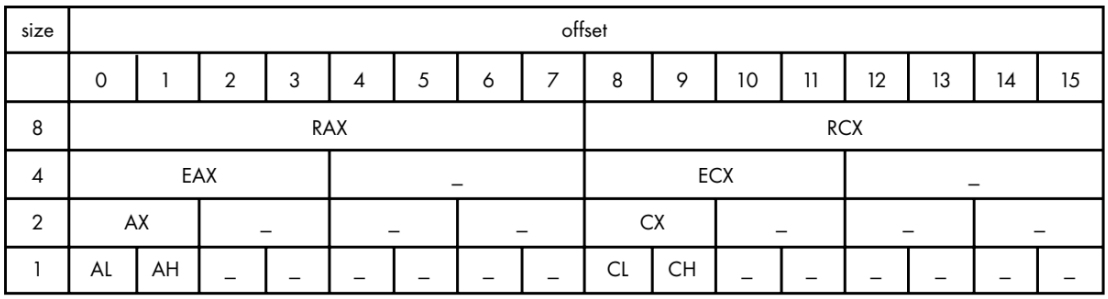


Figure 18-8: Register layout for x86-64 RAX and RCX registers

> The register named AL occupies exactly the same location as the least significant byte of RAX, EAX, and AX (since x86 is a little-endian). Similarly, EAX occupies the low 4 bytes of RAX. An underscore indicates that no name is associated with a given range of bytes for the given size. In this case, there is no name for the 4-byte block at offsets four to seven, although these bytes are synonymous with the upper half of the RAX register. Listing 18-1 describes a

Рисунок 18-8. Раскладка регистров RAX и RCX для x86-64

Регистр с именем `AL` занимает ровно то же место, что и младший значащий байт `RAX`, `EAX` и `AX` (поскольку x86 использует прямой порядок байтов). Аналогично, `EAX` занимает младшие 4 байта `RAX`. Символ подчеркивания указывает, что с данным диапазоном байтов для данного размера не связано никакое имя. В этом случае нет имени для 4-байтового блока со смещений четыре-семь, хотя эти байты являются синонимом верхней половины `RAX`. Листинг 18-1 описывает отдельный блок регистров, начинающийся с `R8` по смещению `0x80`. 1-байтовый регистр по смещению `0x80`

называется R8B, а 1-байтовый регистр по смещению 0x88 называется R9B. Сходство между текстовым определением регистров и табличным представлением должно быть очевидно: определения регистров в файле SLEIGH - это просто текстовое представление адресного пространства регистров архитектуры.

Если вы пишете описание SLEIGH для архитектуры, полностью не поддерживаемой Ghidra, ваша задача - разложить адресное пространство регистров для этой архитектуры, убедившись, что регистры не перекрываются, если только архитектура этого не требует, как RAX, EAX, AX, AH, AL в x86-64.

Теперь, когда представление регистров в SLEIGH понятно, вернемся к нашей цели выбора регистра для dab. Чтобы инструкция работала правильно, ей нужно сопоставить идентификатор Reg32 с регистром общего назначения. Для выполнения этой задачи можно использовать существующее определение в ia.sinc:

```
define token modrm (8)
    mod            = (6,7)
    reg_opcode     = (3,5)
    reg_opcode_hb = (5,5)
    r_m            = (0,2)
    row            = (4,7)
    col            = (0,2)
    page           = (3,3)
    cond           = (0,3)
    reg8           = (3,5)
    reg16          = (3,5)
    reg32          = (3,5)
    reg64          = (3,5)
    reg8_x0        = (3,5)
```

Оператор define объявляет 8-битный токен с именем modrm.[6] Токен SLEIGH - это синтаксический элемент, используемый для представления байтовых компонентов, из которых состоят моделируемые инструкции. SLEIGH позволяет определять битовые поля (диапазон из одного или нескольких смежных битов) внутри токена. При определении инструкций в SLEIGH эти битовые поля предоставляют удобное символическое средство задания связанных операндов. В этом листинге битовое поле с именем reg32 охватывает биты с 3 по 5 в modrm. Это 3-битное поле может принимать значения от 0 до 7 и использоваться для выбора одного из восьми 32-битных регистров x86.

Если перейти к следующей ссылке на reg32 в файле, мы увидим следующие интересные строки кода:

```
# attach variables fieldlist registerlist;
  attach variables [ r32   reg32   base   index ]   [ EAX  ECX  EDX  EBX  ESP
    EBP  ESI  EDI ];
#                                                     0    1    2    3    4    5
       6    7
```

Первая и последняя строки листинга содержат комментарии, показывающие синтаксис SLEIGH для этого оператора и порядковые значения для каждого регистра. Оператор attach variables связывает поле со списком, в данном случае со списком регистров общего назначения x86. Грубая интерпретация строки кода с учетом предыдущего определения modrm такова: значение reg32 определяется просмотром битов 3-5 токена modrm. Получившееся значение (0-7) затем используется как индекс для выбора регистра из списка.

Теперь у нас есть способ определить регистры общего назначения, в которые нужно поместить 0xDAB. Следующая встреча с Reg32 внутри файла обнаруживает следующий код, содержащий конструктор для Reg32 как для 32-, так и для 64-битных регистров, и теперь мы можем увидеть связь между reg32 и Reg32:[7]

```
Reg32:    reg32  is rexRprefix=0 & reg32   { export reg32; } #64-bit Reg32
Reg32:    reg32  is reg32                  { export reg32; } #32-bit Reg32
```

Вернемся к команде, с которой началось это небольшое приключение:

```
:VMXPLODE Reg32 is vexMode=0 & byte=0x0f; byte=0x01; byte=0xc5; Reg32
                                        { Reg32=0xDAB; vmxplode(); }
```

Мы включим операнд в вызов `VMXPLODE`, который будет определять, какой регистр получит значение `0xDAB`. Мы дополнительно обновим тестовый бинарный файл, удалив первый `NOP` и добавив значение `0x08` к вручную собранной инструкции. Первые 3 байта - это опкод (`0f 01 c5`), а следующий байт (`08`) будет операндом, задающим используемый регистр:

```
".byte 0x0f, 0x01, 0xc5, 0x08;"            // вручную собрано с операндом
```

Рисунок 18-9 демонстрирует пошаговый перевод от операнда к определению регистра на основе информации в файле `ia.sinc`.

Figure 18-9 demonstrates the step-by-step translation from the operand through to the determination of the register based on the information in the *ia.sinc* file.

❶	Operand	08						
❷	Value	0	0	0	0	1	0	0	0
❸	modrm bits	7	6	5	4	3	2	1	0
❹	Reg32	001						
❺	Ordinals	0	1	2	3	4	5	6	7
❻	Registers	EAX	ECX	EDX	EBX	ESP	EBP	ESI	EDI

Figure 18-9: Translation path from operand to register

The original operand value, shown in the first row, is `0x08` ❶. The value is decoded into its binary ❷ form and overlaid with the fields of the `modrm` token ❸. Bits 3 to 5 are extracted, yielding the `Reg32` value `001` ❹. This value is used to index the ordinal map ❺ to select the `ECX` register ❻. Therefore, the operand `0x08` specifies that `ECX` will get the value `0xDAB`.

When we save the updated *ia.sinc* file, restart Ghidra, and then load and analyze the file, the following listing is generated, showing the use of our new instruction. As expected, `ECX` is the register selected to hold `0xDAB`:

Рисунок 18-9. Путь перевода от операнда к регистру

Исходное значение операнда - `0x08`. Значение декодируется в двоичную форму и накладывается на поля токена `modrm`. Биты 3-5 извлекаются, давая значение `Reg32` `001`. Это значение используется для индексирования порядковой карты и выбора регистра `ECX`. Поэтому операнд `0x08` задает, что `ECX` получит значение `0xDAB`.

Когда обновленный `ia.sinc` сохранен, Ghidra перезапущена, а файл загружен и проанализирован, генерируется следующий листинг. Как и ожидалось, для хранения `0xDAB` выбран `ECX`:

```
00100721 f3 0f c7 75 fc VMXON    qword ptr [RBP + local_c]
                                    $U620:8 = INT_ADD RBP, -4:8
                                    $Ua50:8 = LOAD ram($U620)
                                    CALLOTHER "vmxon", $Ua50
00100726 0f 01 c5 08    VMXPLODE ECX
                                    ECX = COPY 0xdab:4
                                    CALLOTHER "vmxplode"
0010072a 90 NOP
```

Значение `0xDAB` больше не появляется в окне `Decompiler`, потому что декомпилятор предполагает, что возвращаемое значение находится в `EAX`. В этом случае мы используем `ECX`, поэтому

декомпилятор не выявляет возвращаемого значения.

Теперь, когда выбранный регистр может выполнить dab, добавим 32-битное непосредственное значение как второй операнд. Это удваивает праздничный потенциал.

Вариант 3: операнды регистра и значения

Чтобы расширить синтаксис нашей инструкции так, чтобы она принимала два операнда (регистр назначения и исходную константу), обновите определение `VMXPLODE`:

```
:VMXPLODE Reg32,imm32 is vexMode=0 & byte=0x0f; byte=0x01; byte=0xc5;
          Reg32; imm32                    { Reg32=imm32; vmxplode(); }
```

Добавление непосредственной 32-битной константы к инструкции требует 4 дополнительных байтов для кодирования. Соответственно, замените следующие четыре `NOP` значениями, которые правильно кодируют наш `imm32` в прямом порядке байтов:

```
".byte 0x0f, 0x01, 0xc5, 0x08, 0xb8, 0xdb, 0xee, 0x0f;"
"nop;"
"nop;"
```

Когда файл перезагружен, `VMXPLODE` выходит с еще одним эффектным жестом. Как показано в следующем листинге с отображаемым p-code, `ECX` теперь имеет значение `0xFEEDBB8`, что может быть более привлекательным выходным жестом для поклонников научной фантастики:

```
00100726 0f 01 c5       VMXPLODE ECX,0xfeedbb8
         08 b8 db
         ee 0f
                                          ECX = COPY 0xfeedbb8:4
                                          CALLOTHER "vmxplode"
```

Пример 3: добавление регистра в процессорный модуль

Мы завершаем примеры процессорных модулей расширением архитектуры двумя совершенно новыми регистрами. Вспомните определение 32-битных регистров общего назначения из более раннего места главы:[8]

```
define register offset=0  size=4  [EAX ECX EDX EBX ESP EBP ESI EDI];
```

Определение регистра требует смещения, размера и списка регистров. Мы выбрали начальное смещение в адресном пространстве памяти регистров после просмотра уже выделенных смещений и поиска пространства, необходимого для двух 4-байтовых регистров. Мы можем использовать эту информацию, чтобы определить в файле `ia.sinc` два новых 32-битных регистра с именами `VMID` и `VMVER`:

```
# Определить VMID и VMVER
define register offset=0x1500 size=4 [ VMID VMVER ];
```

Нашим инструкциям нужно средство для определения того, с каким новым регистром (VMID или VMVER) они работают. В предыдущем примере мы использовали 3-битное поле для выбора одного из восьми регистров. Для выбора между двумя новыми регистрами требуется только один бит. Следующий оператор определяет 1-битное поле внутри токена modrm и связывает поле с vmreg:

```
# Связать vmreg с одним битом в токене modrm.
vmreg = (3, 3)
```

Следующий оператор привязывает vmreg к порядковому набору, содержащему два регистра, где 0 представляет VMID, а 1 представляет VMVER:

```
attach variables [ vmreg ]   [ VMID  VMVER ];
```

Определения инструкций могут ссылаться на vmreg, когда любой из привязанных регистров допустим внутри инструкции, тогда как программисты на языке ассемблера могут ссылаться на VMID и VMVER как на операнды в любой инструкции, допускающей операнд vmreg. Сравните следующие два определения VMXPLODE. Первое взято из предыдущего примера, где регистр выбирался среди регистров общего назначения; второе выбирает один из двух наших регистров:

```
:VMXPLODE Reg32,imm32 is vexMode=0 & byte=0x0f; byte=0x01; byte=0xc5;
          Reg32, imm32                    { Reg32=imm32; vmxplode(); }
:VMXPLODE vmreg,imm32 is vexMode=0 & byte=0x0f; byte=0x01; byte=0xc5;
          vmreg, imm32                    { vmreg=imm32; vmxplode(); }
```

Во втором листинге Reg32 заменен на vmreg. Если использовать тот же входной файл с тестовой инструкцией vmxplode 0x08,0xFEEDBB8, непосредственный операнд 0xFEEDBB8 будет загружен в VMVER, поскольку входное значение 0x08 сопоставляется с порядковым значением 1 (потому что бит 3 установлен), как показано на рисунке 18-10, а VMVER является регистром 1 в vmreg. После загрузки тестового файла (после сохранения ia.sinc и перезапуска Ghidra) мы видим, что p-code в окне Listing показывает загрузку непосредственного операнда в VMVER:

```
00100726 0f 01 c5       VMXPLODE   VMVER,0xfeedbb8
         08 b8 db
         ee 0f
                                        VMVER = COPY 0xfeedbb8:4
                                        CALLOTHER "vmxplode"
```

Связанная информация об инструкции, показанная на рисунке 18-10, также подтверждает изменение.

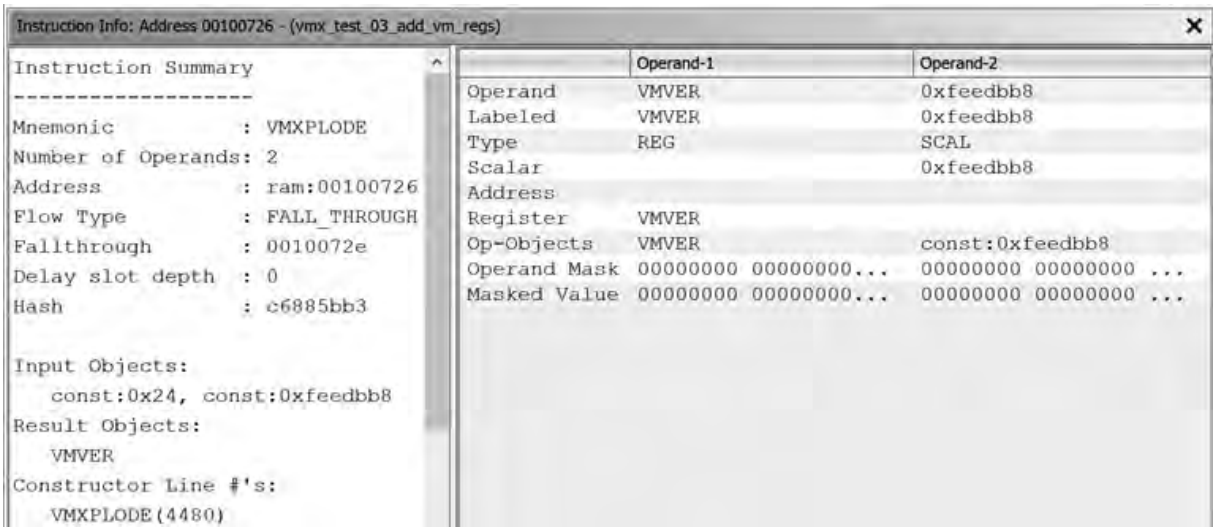


Рисунок 18-10. Instruction Info для VMXPLODE с выбранным новым регистром VMVER

Резюме

Хотя в этой главе мы представили лишь малую часть содержимого файлов процессора x86, мы рассмотрели основные компоненты процессорного модуля, включая определения инструкций, определения регистров и токены, а также то, как специфичный для Ghidra язык SLEIGH можно использовать для построения, изменения и расширения процессорных модулей Ghidra. Если у вас есть желание или необходимость добавить новый процессор в Ghidra, мы настоятельно рекомендуем посмотреть на некоторые из более новых процессоров, добавленных в Ghidra. Файл `SuperH4.sinc` особенно хорошо документирован, а процессор значительно менее сложен, чем процессор x86.

Невозможно достаточно подчеркнуть роль терпения и экспериментов в любой ситуации разработки процессора. Тяжелая работа с лихвой окупается, когда вы можете повторно использовать свой процессорный модуль с каждым новым собранным бинарным файлом и потенциально внести модуль обратно в проект Ghidra на пользу другим специалистам по обратной разработке.

В следующей главе мы глубоко погрузимся в функциональность, связанную с Ghidra `Decompiler`.

Сноски

[1] [назад] Для больших наборов инструкций, например набора инструкций x86, файл `.sinc` может быть разбит на несколько файлов `.sinc`. В таком случае часть файлов может использоваться как заголовочные файлы с определениями и операторами включения.
[2] [назад] Подробную информацию о языке SLEIGH можно найти в установке Ghidra в `docs/languages/html/sleigh.html`.
[3] [назад] Раздел 30-1 следующего документа описывает существующие инструкции обслуживания VMCS: https://www.intel.com/content/dam/www/public/us/en/documents/manuals/64-ia-32-architectures-software-developer-vol-3c-part-3-manual.pdf.
[4] [назад] Схема кодирования VEX описана в разделе 2.3 документа https://www.intel.com/content/dam/www/public/us/en/documents/manuals/64-ia-32-architectures-software-developer-instruction-set-reference-manual-325383.pdf.
[5] [назад] В `Ghidra Help` p-code появляется без дефиса (`pcode`), а в большинстве других мест, включая документацию p-code, - с дефисом. Если у вас возникают трудности с поиском информации о p-code внутри Ghidra, попробуйте искать `pcode` без дефиса.
[6] [назад] Эта концепция подробно описана в разделе `Tokens and Fields` (6) документации SLEIGH.
[7] [назад] Эта концепция подробно описана в разделе Constructors (7) документации SLEIGH.
[8] [назад] Эта концепция подробно описана, начиная с раздела Naming Registers (4.4) документации SLEIGH.

Глава 19. Декомпилятор Ghidra

До сих пор мы сосредоточивали наш анализ обратной разработки на окне `Listing` и представляли возможности Ghidra через призму дизассемблерного листинга. В этой главе мы переносим внимание на окно `Decompiler` и исследуем, как можно выполнять знакомые задачи анализа (и некоторые новые) с помощью `Decompiler` и связанной с ним функциональности.

Мы начнем с краткого обзора процесса декомпиляции, а затем перейдем к функциональности, доступной в окне `Decompiler`. После этого мы разберем несколько примеров, которые помогут вам увидеть, как окно `Decompiler` можно использовать для улучшения процесса обратной разработки.

Анализ Decompiler

Логично предположить, что содержимое окна `Decompiler` получается из окна `Listing`, но, что удивительно, содержимое окон `Listing` и `Decompiler` выводится независимо. Именно поэтому они иногда расходятся, и именно поэтому оба представления следует оценивать в контексте, когда вы пытаетесь установить истинное положение дел. Основная функция `Decompiler` в Ghidra - преобразовать инструкции машинного языка в p-code (см. главу 18), затем преобразовать p-code в C и представить его в окне `Decompiler`.

В упрощенном виде процесс декомпиляции включает три отдельных фазы. В первой фазе `Decompiler` использует файл спецификации SLEIGH, чтобы создать черновой p-code и вывести связанные базовые блоки и потоки. Вторая фаза сосредоточена на упрощении: ненужное содержимое, например недостижимый код, удаляется, а затем потоки управления корректируются и настраиваются с учетом изменений. На завершающей фазе добавляются последние штрихи, выполняются финальные проверки, и итоговые результаты проходят через алгоритм красивой печати перед отображением в окне `Decompiler`. Конечно, это сильно упрощает очень сложный процесс, но главные выводы таковы:[1]

• `Decompiler` является анализатором.
• Он начинает работу с бинарного файла и создает p-code.
• Он преобразует p-code в C.
• C-код и все связанные сообщения отображаются в окне `Decompiler`.

Мы обсудим некоторые из этих шагов подробнее, пока будем перемещаться по функциональности декомпиляции Ghidra. Начнем исследование с процесса анализа и основных возможностей, которые он открывает.

Параметры анализа

Во время процесса автоматического анализа существует несколько анализаторов, относящихся к окну `Decompiler`. Параметры анализа `Decompiler` управляются через меню `Edit -> Tool Options`, показанное на рисунке 19-1 с выбранными настройками по умолчанию.

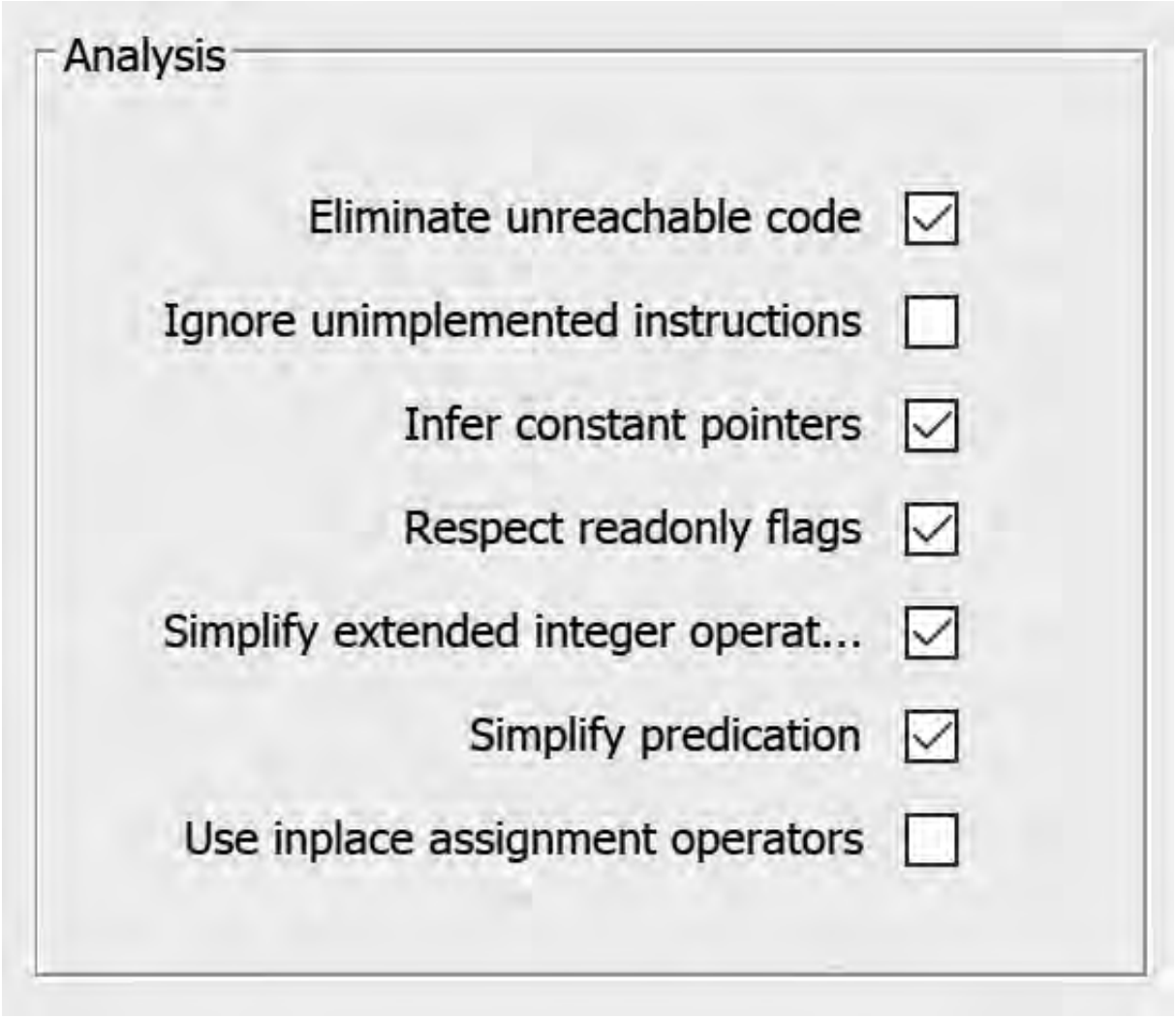


Рисунок 19-1. Параметры анализа `Decompiler` *Ghidra с выбранными настройками по умолчанию*

Далее мы обсудим две из этих опций: `Eliminate unreachable code` и `Simplify predication`. С результатами остальных опций вы можете поэкспериментировать самостоятельно или обратиться к `Ghidra Help`.

Eliminate Unreachable Code

Опция `Eliminate unreachable code` исключает недостижимый код из листинга `Decompiler`. Например, в следующей C-функции есть два условия, которые никогда не могут выполниться, из-за чего соответствующие условные блоки становятся недостижимыми:

```
int demo_unreachable(volatile int a) {
    volatile int b = a ^ a;
    if (b) {
        printf("Это недостижимо\n");
        a += 1;
    }
    if (a - a > 0) {
        printf("Это тоже должно быть недостижимо\n");
        a += 1;
    } else {
        printf("Это должно быть видно всегда\n");
        a += 2;
    }
    printf("Конец demo_unreachable()\n");
```

```
    return a;
}
```

Переменная b инициализируется нулем, хотя и не самым очевидным способом. Когда b проверяется, ее значение никогда не может быть ненулевым, и тело соответствующего оператора `if` никогда не будет выполнено. Аналогично, `a - a` никогда не может быть больше нуля, поэтому условие во втором операторе `if` также никогда не может стать истинным. Когда опция `Eliminate unreachable code` выбрана, окно `Decompiler` отображает предупреждающие сообщения, сообщающие, что недостижимый код был удален.

```
/* WARNING: Removing unreachable block (ram,0x00100777) */
/* WARNING: Removing unreachable block (ram,0x0010079a) */
ulong demo_unreachable(int param_1)
{
  puts("Это должно быть видно всегда");
  puts("Конец demo_unreachable()");
  return (ulong)(param_1 + 2);
}
```

Simplify Predication

Эта опция оптимизирует блоки `if`/`else`, объединяя блоки, которые имеют одно и то же условие. В следующем листинге первые два оператора `if` имеют одно и то же условие:

```
int demo_simppred(int a) {
    if (a > 0) {
          printf("A is > 0\n");
    }
    if (a > 0) {
          printf("Да, A точно > 0!\n");
    }
    if (a > 2) {
          printf("A > 2\n");
    }
    return a * 10;
}
```

С включенной Simplify predication получившийся листинг `Decompiler` показывает объединенные блоки:

```
ulong demo_simppred(int param_1)
{
  if (0 < param_1) {
    puts("A is > 0");
    puts("Да, A точно > 0!");
  }
  if (2 < param_1) {
    puts("A > 2");
  }
  return (ulong)(uint)(param_1 * 10);
}
```

Окно Decompiler

Теперь, когда вы понимаете, как `Decompiler Analysis Engine` заполняет окно `Decompiler`, посмотрим, как можно использовать это окно для упрощения анализа. Навигация в окне `Decompiler` относительно проста, потому что оно отображает только одну функцию за раз. Чтобы переходить между функциями или видеть функцию в контексте, полезно сопоставлять его с окном `Listing`. Поскольку окна `Decompiler` и `Listing` по умолчанию связаны, вы можете перемещаться по ним обоим с помощью доступных вариантов на панели инструментов `CodeBrowser`.

Функция, отображаемая в окне `Decompiler`, помогает анализу, но сначала ее может быть не так легко читать. При отсутствии информации о типах данных, используемых функциями, которые он декомпилирует, Ghidra приходится выводить эти типы данных самостоятельно. В результате декомпилятор может чрезмерно использовать приведения типов, как видно в следующих примерах операторов:

```
printf("a=%d, b=%d, c=%d, d=%d, e=%d, f=%d, g=%d\n", (ulong)param_1,
       (ulong)param_2,(ulong)uVar1,(ulong)uVar2,(ulong)(uVar1 + param_1),
       (ulong)(uVar2 * 100),(ulong)uVar4);
uStack44 = *(undefined4 *)**(undefined4 **)(iStack24 + 0x10);
```

По мере того как вы предоставляете более точную информацию о типах с помощью параметров редактирования `Decompiler`, вы заметите, что `Decompiler` все меньше полагается на приведения типов, а сгенерированный C-код становится легче читать. В следующих примерах мы обсудим некоторые из самых полезных возможностей окна `Decompiler` для очистки сгенерированного исходного кода. Конечная цель - читаемый исходный код, который проще понять, что сокращает время, необходимое для понимания поведения кода.

Пример 1: Редактирование в окне Decompiler

Рассмотрим программу, которая принимает от пользователя два целочисленных значения, а затем вызывает следующую функцию:

```
int do_math(int a, int b) {
    int c, d, e, f, g;
    srand(time(0));
    c = rand();
    printf("c=%d\n", c);
    d = a + b + c;
    printf("d=%d\n", d);
    e = a + c;
    printf("e=%d\n", e);
    f = d * 100;
    printf("f=%d\n", f);
    g = rand() - e;
    printf("g=%d\n", g);
    printf("a=%d, b=%d, c=%d, d=%d, e=%d, f=%d, g=%d\n", a, b, c, d, e, f, g);
    return g;
}
```

Функция использует два целочисленных параметра и пять локальных переменных для формирования своего вывода. Взаимозависимости можно кратко описать так:

- Переменная `c` зависит от возвращаемого значения `rand()`, напрямую влияет на `d` и `e` и косвенно влияет на `f` и `g`.
- Переменная `d` зависит от `a`, `b` и `c` и напрямую влияет на `f`.
- Переменная `e` зависит от `a` и `c` и напрямую влияет на `g`.
- Переменная `f` напрямую зависит от `d`, косвенно зависит от `a`, `b` и `c` и ни на что не влияет.
- Переменная `g` напрямую зависит от `e`, косвенно зависит от `a` и `c` и ни на что не влияет.

Когда связанный бинарный файл загружается в Ghidra и функция анализируется, вы видите следующее представление функции `do_math` в окне `Decompiler`:

```
ulong do_math(uint param_1,uint param_2)
{
  uint uVar1;
  uint uVar2;
  int iVar3;
  uint uVar4;
  time_t tVar5;
  tVar5 = time((time_t *)0x0);
  srand((uint)tVar5);
  uVar1 = rand();
  printf("c=%d\n");
  uVar2 = uVar1 + param_1 + param_2;
  printf("d=%d\n");
  printf("e=%d\n");
  printf("f=%d\n");
  iVar3 = rand();
  uVar4 = iVar3 - (uVar1 + param_1);
  printf("g=%d\n");
  printf("a=%d, b=%d, c=%d, d=%d, e=%d, f=%d, g=%d\n", (ulong)param_1,
         (ulong)param_2,(ulong)uVar1,(ulong)uVar2,(ulong)(uVar1 + param_1),
         (ulong)(uVar2 * 100),(ulong)uVar4);
  return (ulong)uVar4;
}
```

Если вы хотите выполнять анализ с помощью `Decompiler`, вам нужно убедиться, что код, который `Decompiler` генерирует, настолько точен, насколько возможно. Обычно это делается путем предоставления как можно большего количества информации о типах данных и прототипах функций. Функции, принимающие переменное число аргументов, например `printf`, особенно сложны для `Decompiler`, потому что `Decompiler` должен был бы полностью понимать семантику требуемых аргументов, чтобы оценить количество переданных необязательных аргументов.

Переопределение сигнатур функций

Вы можете видеть несколько операторов `printf`, которые выглядят не совсем правильно. У каждого есть строка формата, но нет дополнительных аргументов. Поскольку `printf` принимает переменное число аргументов, можно переопределить сигнатуру функции в каждой точке вызова и, основываясь на строке формата, указать, что оператор `printf` должен принимать один целочисленный аргумент.[2] Чтобы внести это изменение, щелкните правой кнопкой мыши оператор `printf` и выберите Override Signature в контекстном меню, чтобы открыть диалог, показанный на рисунке 19-2.

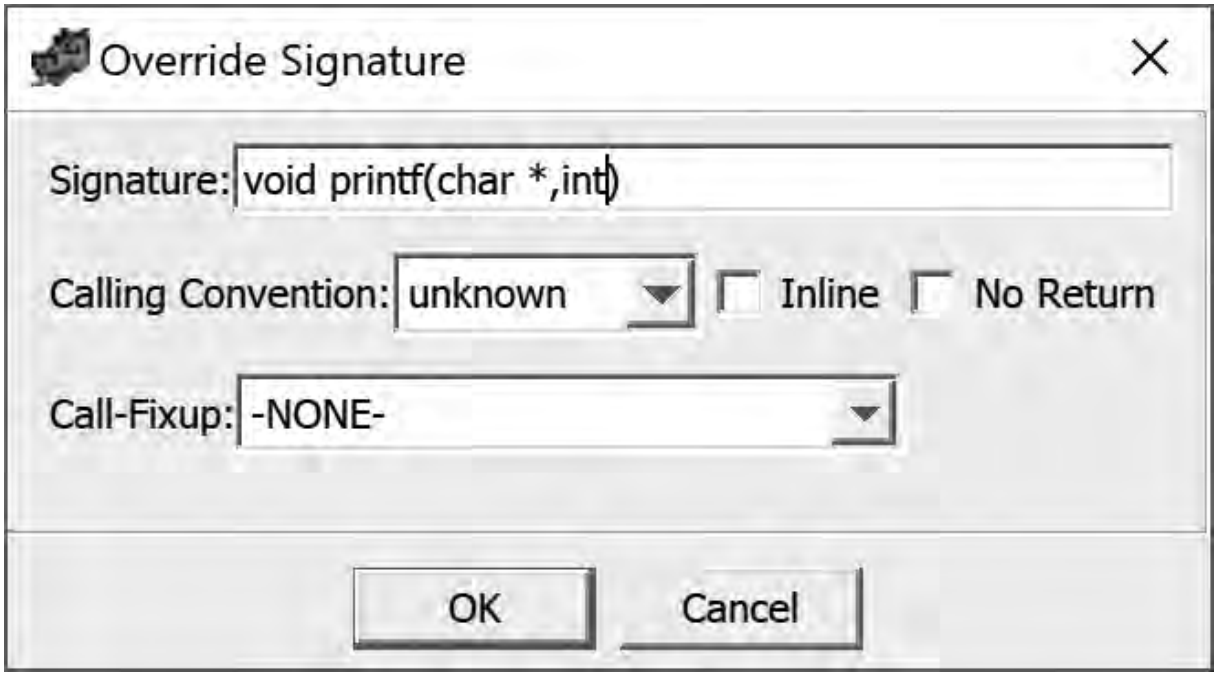


Рисунок 19-2. Диалог Override Signature

Добавление второго типа параметра, `int`, в сигнатуру (как показано на рисунке) для каждого из операторов `printf` приводит к следующему листингу:

```
ulong do_math(uint param_1,uint param_2)
{
  uint uVar1;
  uint uVar2;
  uint uVar3;
  int iVar4;
  uint uVar5;
  time_t tVar6;
  tVar6 = time((time_t *)0x0);
  srand((uint)tVar6);
  uVar1 = rand();
  printf("c=%d\n",uVar1);
  uVar2 = uVar1 + param_1 + param_2;
  printf("d=%d\n",uVar2);
  uVar3 = uVar1 + param_1;
  printf("e=%d\n",uVar3);
  printf("f=%d\n",uVar2 * 100);
  iVar4 = rand();
  uVar5 = iVar4 - uVar3;
  printf("g=%d\n",uVar5);
  printf("a=%d, b=%d, c=%d, d=%d, e=%d, f=%d, g=%d\n", (ulong)param_1,
         (ulong)param_2,(ulong)uVar1,(ulong)uVar2,(ulong)(uVar1 + param_1),
         (ulong)(uVar2 * 100),(ulong)uVar4);
  return (ulong)uVar4;
}
```

Помимо обновленных вызовов `printf` с правильными аргументами, в листинг `Decompiler` были добавлены две новые строки в результате переопределения функции `printf`. Раньше эти операторы не включались, потому что Ghidra считала, что результаты не используются. Как только `Decompiler` понимает, что результаты используются в каждом `printf`, эти операторы становятся осмысленными и отображаются в окне `Decompiler`.

Редактирование типов и имен переменных

После исправления вызовов функций можно продолжить очистку листинга, переименовав (горячая клавиша L) и изменив тип (горячая клавиша ctrl-L) параметров и переменных на основе имен, найденных в строках формата. К слову, строки формата являются чрезвычайно ценным источником информации о типе и назначении переменных в любой программе.

После выполнения этих изменений последний оператор `printf` все еще остается немного громоздким:

```
printf("a=%d, b=%d, c=%d, d=%d, e=%d, f=%d, g=%d\n", (ulong)a,
       (ulong)(uint)b, (ulong)(uint)c, (ulong)(uint)d, (ulong)(uint)e,
       (ulong)(uint)(d * 100),(ulong)(uint)g);
```

Щелчок правой кнопкой мыши по этому оператору позволяет переопределить сигнатуру функции. Первый аргумент в этом операторе `printf` - строка формата, и его изменять не нужно. Изменение остальных аргументов на тип `int` приводит к более чистому коду (листинг 19-1) в окне `Decompiler`.

```
int do_math(int a, int b)
{
  int c;
  int d;
  int e;
  int g;
  time_t tVar1;
  tVar1 = time((time_t *)0x0);
  srand((uint)tVar1);
  c = rand();
  printf("c=%d\n",c);
  d = c + a + b;
  printf("d=%d\n",d);
  e = c + a;
  printf("e=%d\n",e);
  printf("f=%d\n",d * 100);
  g = rand();
  g = g - e;
  printf("g=%d\n",g);
  printf("a=%d, b=%d, c=%d, d=%d, e=%d, f=%d, g=%d\n",a,b,c,d,e,d * 100,g);
  return g;
}
```

Листинг 19-1. Декомпилированная функция с обновленными сигнатурами

Это очень похоже на исходный код и намного легче читается, чем первоначальный листинг `Decompiler`, поскольку изменения аргументов функции были распространены по всему листингу. Одно отличие между листингом `Decompiler` и нашим исходным кодом состоит в том, что переменная `f` была заменена эквивалентным выражением.

Подсветка срезов

Теперь, когда окно `Decompiler` стало более понятным, можно начать дальнейший анализ. Предположим, вы хотите узнать, как отдельные переменные влияют на другие переменные и как другие переменные влияют на них. Срез программы - это набор операторов, которые участвуют в формировании значения переменной (обратный срез) или затрагиваются значением переменной (прямой срез). В сценариях анализа уязвимостей это может проявляться как вопрос: "Я контролирую эту переменную; где используется ее значение?"

Ghidra предоставляет пять вариантов в своем контекстном меню правой кнопки мыши для подсветки связей между переменными и инструкциями внутри функции. Если щелкнуть переменную в окне `Decompiler` правой кнопкой мыши, можно выбрать следующие варианты:

Highlight Def-use. Эта опция подсвечивает все использования переменной внутри функции. Тот же эффект можно получить средним щелчком мыши.

Highlight Forward Slice. Эта опция подсвечивает все, на что влияет значение выбранной переменной. Например, если выбрать переменную `b` в листинге 19-1 и выбрать эту опцию, все вхождения `b` и `d` будут подсвечены в листинге, потому что изменение значения `b` также может привести к изменению значения `d`.

Highlight Backward Slice. Это обратная опция к предыдущей; она подсвечивает все переменные, которые вносят вклад в конкретное значение. Если щелкнуть правой кнопкой мыши переменную `e` в последнем операторе `printf` в листинге 19-1 и выбрать эту опцию, будут подсвечены все переменные, влияющие на значение `e` (в данном случае `e`, `a` и `c`). Изменение `a` или `c` также может изменить значение `e`.

`Highlight Forward Inst Slice`. Эта опция подсвечивает весь оператор, связанный с опцией `Highlight Forward Slice`. В листинге 19-1, если использовать эту опцию при выбранной переменной `b`, будут подсвечены все операторы, в которых встречаются `b` или `d`.

`Highlight Backward Inst Slice`. Эта опция подсвечивает весь оператор, связанный с опцией `Highlight Backward Slice`. В листинге 19-1 выбор этой опции при подсвеченной переменной `e` в последнем операторе `printf` приведет к подсветке всех операторов, в которых встречаются `a`, `c` или `e`.

Теперь, когда у нас есть общее понимание некоторых подходов к работе с окном `Decompiler` и его использованию в анализе, рассмотрим более конкретный пример.

Пример 2: невозвращающие функции

В общем случае Ghidra может безопасно предполагать, что вызовы функций возвращают управление, и поэтому обрабатывает вызовы функций так, будто они демонстрируют последовательный поток внутри базовых блоков. Однако некоторые функции, например помеченные ключевым словом `noreturn` в исходном коде или завершающиеся обфусцированной инструкцией перехода в вредоносном ПО, не возвращают управление, и Ghidra может генерировать неточный дизассемблированный или декомпилированный код. Ghidra предлагает три подхода к работе с невозвращающими функциями: два анализатора невозвращающих функций и возможность вручную редактировать сигнатуры функций.

Ghidra может идентифицировать невозвращающие функции на основе списка известных функций `noreturn`, например `exit` и `abort`, используя анализатор `Non-Returning Functions-Known`. Этот анализатор выбран по умолчанию как часть автоматического анализа, и его задача проста: если имя функции присутствует в его списке, он помечает функцию как невозвращающую и делает все возможное, чтобы исправить связанные проблемы (например, установить связанные вызовы как невозвращающие, найти потоки, которые могут требовать исправления, и так далее).

Анализатор `Non-Returning Functions-Discovered` ищет подсказки, которые могут указывать, что функция не возвращает управление (например, данные или плохие инструкции сразу после вызова). То, что он делает с этой информацией, в значительной степени контролируется тремя опциями, связанными с анализатором, как показано на рисунке 19-3.

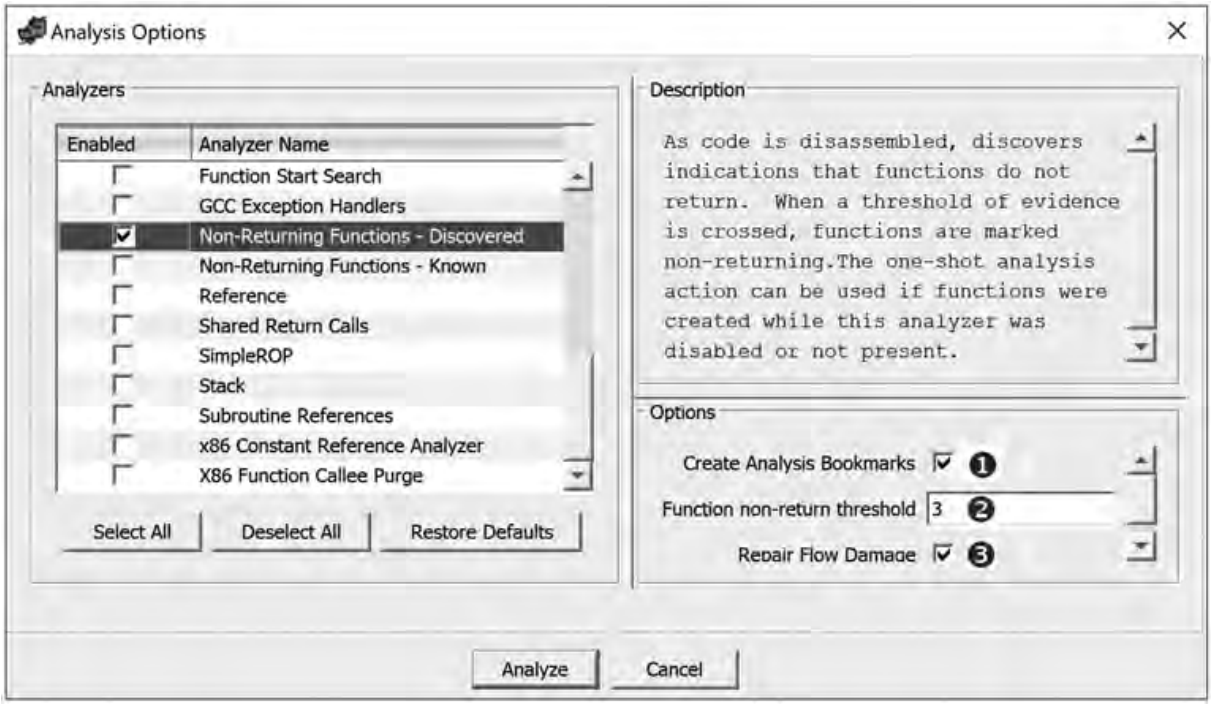


Рисунок 19-3. Параметры анализа для `Non-Returning Functions-Discovered`

Первая опция позволяет автоматически создавать закладки анализа, которые появляются на панели закладок окна `Listing`. Вторая опция позволяет указать порог, определяющий, следует ли назначать функцию как невозвращающую на основе серии проверок характеристик, которые, вероятно, указывают на невозвращающую функцию. Наконец, есть флажок для исправления связанного повреждения потока.

Когда Ghidra не может идентифицировать невозвращающую функцию, у вас есть возможность самостоятельно отредактировать сигнатуру функции. Если вы завершили анализ и у вас есть закладки ошибок, которые используются для отметки плохих инструкций, это хороший признак, что с собственным анализом Ghidra что-то не совсем правильно. Если плохая инструкция следует за `CALL`, как здесь:

```
00100839                CALL          noReturnA
0010083e                ??            FFh
```

то в окне `Decompiler` вы, скорее всего, увидите связанный пост-комментарий, предупреждающий о ситуации, например:

```
noReturnA(1);
/* WARNING: Bad instruction - Truncating control flow here */
halt_baddata();
```

Если щелкнуть имя функции (`noReturnA` в этом случае) в окне `Decompiler` и затем выбрать Edit Function Signature, у вас будет возможность изменить атрибуты, связанные с функцией, как показано на рисунке 19-4.

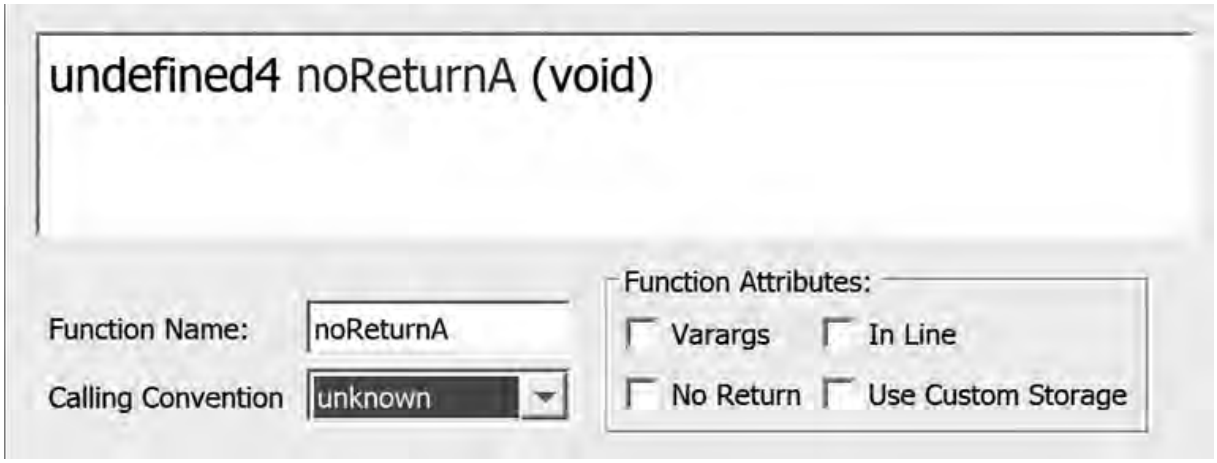


Рисунок 19-4. Редактирование атрибутов функции

Установите флажок No Return, чтобы пометить функцию как невозвращающую. После этого Ghidra вставляет предварительный комментарий, показанный далее, в окно `Decompiler`, а также пост-комментарий в окно `Listing`:

```
/* WARNING: Subroutine does not return */
noReturnA(1);
```

После исправления этой ошибки можно перейти к другим проблемам.

Пример 3: автоматическое создание структур

При анализе декомпилированного исходного кода C вы, вероятно, встретите операторы, которые, как кажется, содержат ссылки на поля структуры. Ghidra помогает создать структуру и заполнить ее на основе связанных ссылок, обнаруженных `Decompiler`. Разберем пример, начав с исходного кода и первоначальной декомпиляции этого кода в Ghidra.

Предположим, у вас есть исходный код, который определяет два типа `struct`, а затем создает глобальный экземпляр каждого из них:

```
struct s1 {
    int a;
    int b;
    int c;
};
typedef struct s2 {
    int x;
    char y;
    float z;
} s2_type;
struct s1 GLOBAL_S1;
s2_type GLOBAL_S2;
```

Одна структура содержит однородные элементы, а другая - неоднородную коллекцию типов. Исходный код также содержит три функции, одна из которых (`do_struct_demo`) объявляет локальный экземпляр каждого типа структуры:

```
void display_s1(struct s1* s) {
    printf("Поля в s1 = %d, %d и %d\n", s->a, s->b, s->c);
}
void update_s2(s2_type* s, int v) {
    s->x = v;
    s->y = (char)('A' + v);
    s->z = v * 2.0;
}
void do_struct_demo() {
    s2_type local_s2;
    struct s1 local_s1;
    printf("Введите шесть целых чисел: ");
    scanf("%d %d %d %d %d %d", (int *)&local_s1, &local_s1.b, &local_s1.c,
          &GLOBAL_S1.a, &GLOBAL_S1.b, &GLOBAL_S1.c);
    printf("Вы ввели: %d и %d\n", local_s1.a, GLOBAL_S1.a);
    display_s1(&local_s1);
    display_s1(&GLOBAL_S1);
    update_s2(&local_s2, local_s1.a);
}
```

Декомпилированная версия `do_struct_demo` показана в листинге 19-2.

```
void do_struct_demo(void)
{
  undefined8 uVar1;
  uint local_20;
  undefined local_1c [4];
  undefined local_18 [4];
  undefined local_14 [12];
  uVar1 = 0x100735;
  printf("Введите шесть целых чисел: ");
  __isoc99_scanf("%d %d %d %d %d %d", &local_20, local_1c, local_18,
                 GLOBAL_S1,0x30101c,0x301020,uVar1);
  printf("Вы ввели: %d и %d\n",(ulong)local_20,(ulong)GLOBAL_S1._0_4_);
  display_s1(&local_20);
  display_s1(GLOBAL_S1);
  update_s2(local_14,(ulong)local_20,(ulong)local_20);
  return;
}
```

Листинг 19-2. Первоначальная декомпиляция функции `do_struct_demo`

Переход к функции `display_s1` из любого вызова функции двойным щелчком по нему в окне `Decompiler` дает следующее:

```
void display_s1(uint *param_1)
{
  printf("Поля в s1 = %d, %d и %d\n", (ulong)*param_1,
         (ulong)param_1[1],(ulong)param_1[2]);
  return;
}
```

Поскольку вы подозреваете, что аргумент `display_s1` может быть указателем на структуру, можно попросить Ghidra автоматизировать процесс создания `struct`: щелкните правой кнопкой мыши `param_1` в списке аргументов функции и выберите Auto Create Structure в контекстном меню. В ответ Ghidra отслеживает все использования `param_1`, рассматривает всю арифметику, выполняемую над указателем, как ссылку на член структуры и автоматически создает новый тип `struct`, содержащий поля по каждому смещению, на которое есть ссылка. Это меняет несколько вещей в листинге `Decompiler`:

```
void display_s1(astruct *param_1)
{
  printf("Поля в s1 = %d, %d и %d\n",(ulong)param_1->field_0x0,
         (ulong)param_1->field_0x4,(ulong)param_1->field_0x8);
  return;
}
```

Тип параметра изменился и теперь равен `astruct*`, а вызов `printf` теперь содержит ссылки на поля. Новый тип также был добавлен в `Data Type Manager`, а наведение курсора на имя структуры показывает определения полей, как на рисунке 19-5.

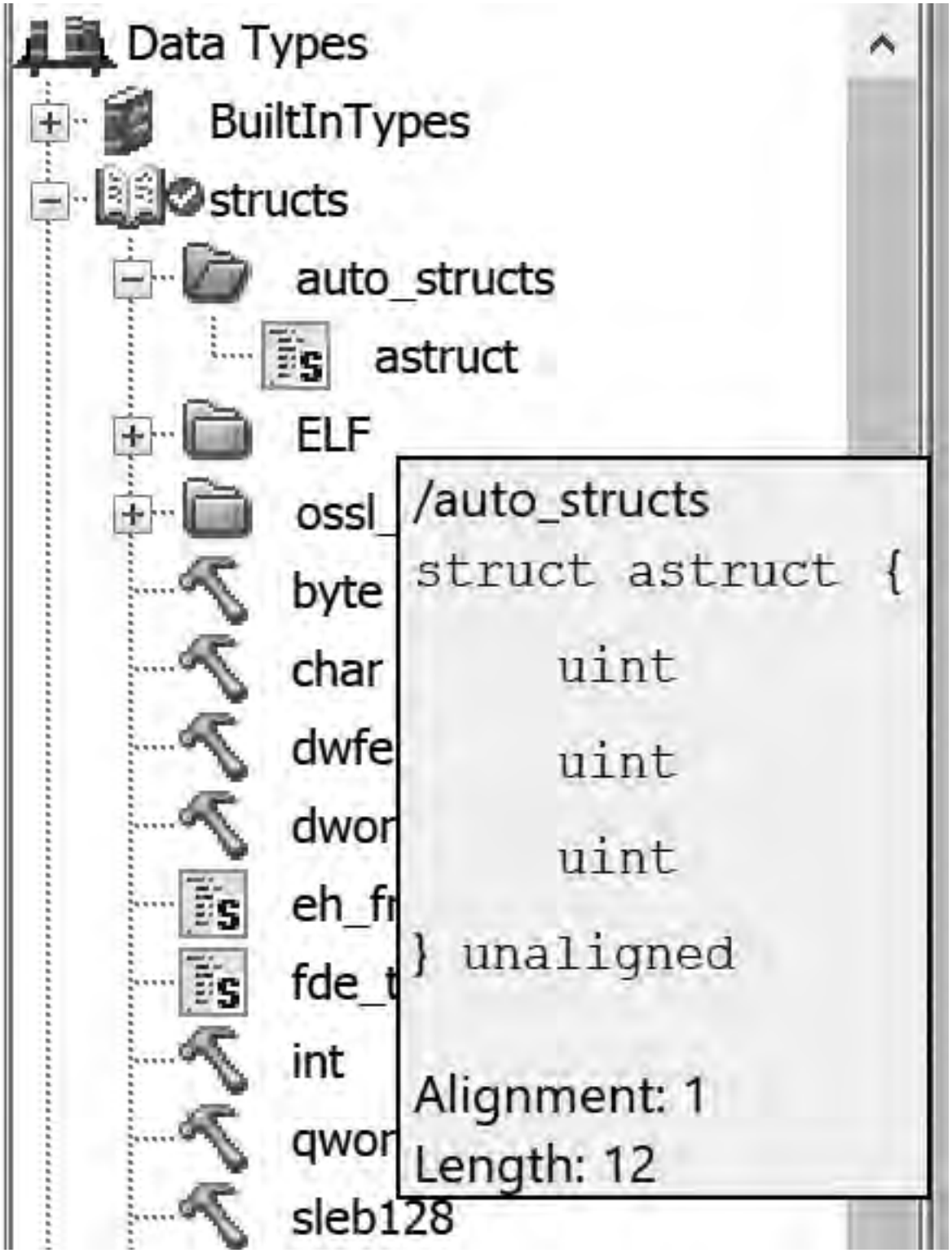


Рисунок 19-5. Автоматические структуры в `Data Type Manager`

Можно обновить тип для `local_20` и `GLOBAL_S1` до `astruct`, используя опцию Retype Variable из контекстного меню правой кнопки мыши. Результаты показаны в следующем листинге:

```
void do_struct_demo(void)
{
  undefined8 uVar1;
  astruct local_20;
  undefined local_14 [12];
  uVar1 = 0x100735;
  printf("Введите шесть целых чисел: ");
  __isoc99_scanf("%d %d %d %d %d %d", &local_20, &local_20.field_0x4,
                 &local_20.field_0x8, &GLOBAL_S1, 0x30101c, 0x301020, uVar1);
  printf("Вы ввели: %d и  %d\n", (ulong)local_20.field_0x0,
         (ulong)GLOBAL_S1.field_0x0);
  display_s1(&local_20);
  display_s1(&GLOBAL_S1);
  update_s2(local_14,(ulong)local_20.field_0x0,(ulong)local_20.field_0x0);
  return;
}
```

Сравнение с листингом 19-2 показывает изменение типа для `local_20` и добавление ссылок на поля как для `local_20`, так и для `GLOBAL_S1`.

Перенесем внимание на декомпиляцию третьей функции, `update_s2`, показанную в листинге 19-3.

```
void update_s2(int *param_1,int param_2)
{
  *param_1 = param_2;
  *(char *)(param_1 + 1) = (char)param_2 + 'A';
  *(float *)(param_1 + 2) = (float)param_2 + (float)param_2;
  return;
}
```

Листинг 19-3. Первоначальная декомпиляция функции `update_s2`

Вы можете использовать предыдущий подход, чтобы автоматически создать структуру для `param_1`. Просто щелкните правой кнопкой мыши `param_1` в функции и выберите Auto Create Structure в контекстном меню.

```
void update_s2(astruct_1 *param_1,int param_2)
{
  param_1->field_0x0 = param_2;
  param_1->field_0x4 = (char)param_2 + 'A';
  param_1->field_0x8 = (float)param_2 + (float)param_2;
  return;
}
```

Теперь `Data Type Manager` имеет второе определение `struct`, связанное с этим файлом, как показано на рисунке 19-6.

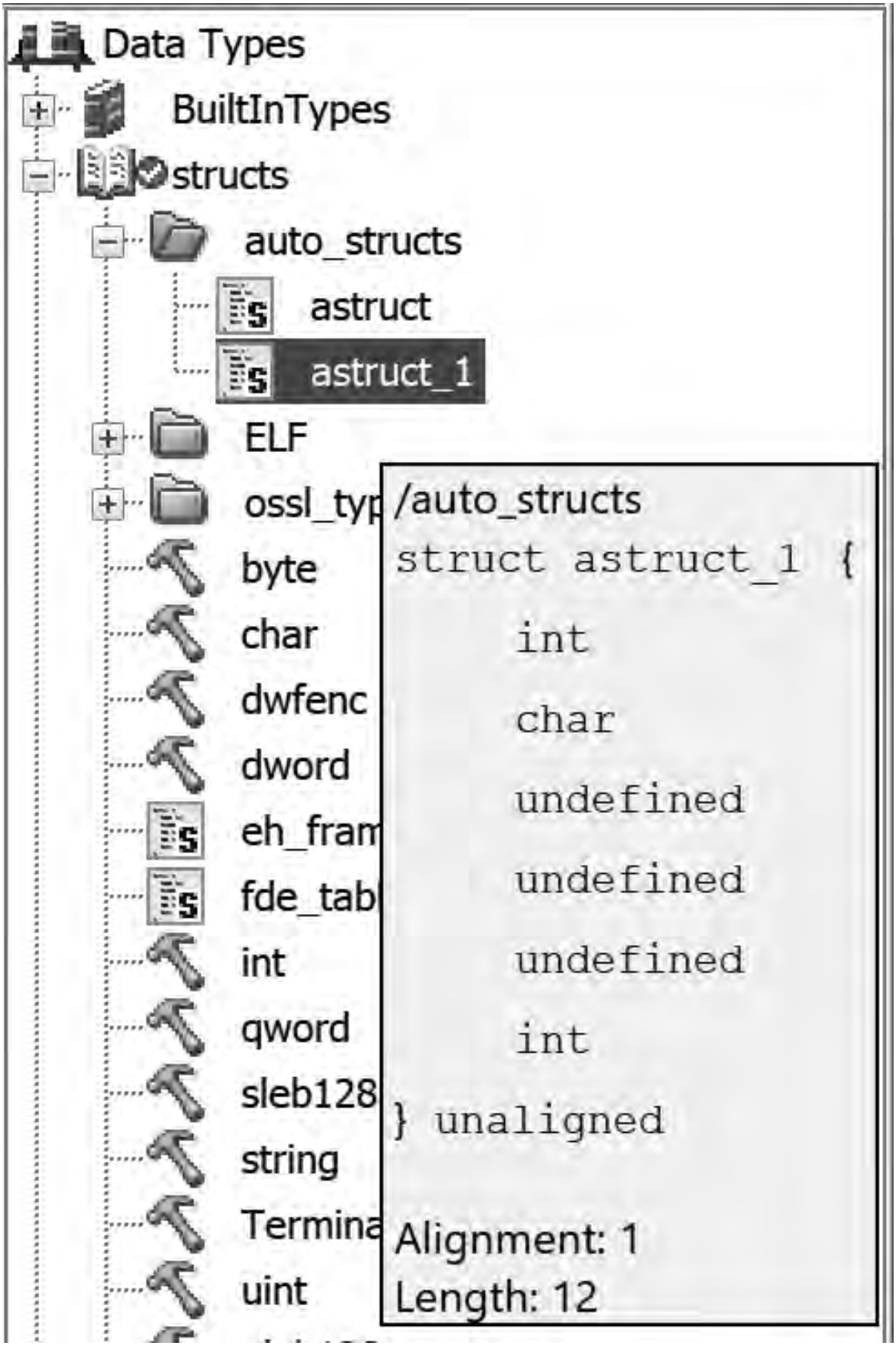


Рисунок 19-6. Дополнительные автоматические структуры в окне `Data Type Manager`

Эта структура содержит `int`, `char`, три неопределенных байта (вероятно, заполнение, вставленное компилятором) и `float`. Чтобы отредактировать структуру, щелкните правой кнопкой мыши `astruct_1` и выберите Edit в контекстном меню, что откроет окно Structure Editor. Если мы решим назвать поле `int` как `x`, поле `char` как `y`, а поле `float` как `z`, а затем сохраним изменения, новые имена полей отразятся в листинге `Decompiler`:

```
void update_s2(astruct_1 *param_1,int param_2)
{
  param_1->x = param_2;
  param_1->y = (char)param_2 + 'A';
  param_1->z = (float)param_2 + (float)param_2;
  return;
}
```

Этот листинг намного легче читать и понимать, чем первоначальную декомпиляцию в листинге 19-3.

Резюме

Окно `Decompiler`, как и окно `Listing`, дает вам взгляд внутрь бинарного файла, и у каждого из этих окон есть свои сильные и слабые стороны. `Decompiler` предоставляет более высокоуровневое представление, которое может помочь быстрее понять общую структуру и функциональность одной функции, чем просмотр дизассемблирования, особенно тем, у кого нет многолетнего опыта чтения дизассемблерных листингов. Окно `Listing` предоставляет более низкоуровневое представление всего бинарного файла со всеми доступными подробностями, но из-за этого может быть труднее увидеть общую картину.

`Decompiler` Ghidra можно эффективно использовать вместе с окном `Listing` и всеми другими инструментами, которые мы представили на протяжении книги, чтобы помогать вам в процессе обратной разработки. В конечном итоге роль специалиста по обратной разработке состоит в том, чтобы определить лучший подход к решению текущей проблемы.

Эта глава была сосредоточена на окне `Decompiler` и вопросах, связанных с декомпиляцией. Многие из этих трудностей можно отнести к огромному разнообразию компиляторов и связанных параметров компилятора, которые напрямую влияют на получающийся бинарный файл. В следующей главе мы рассмотрим некоторые поведения, специфичные для компилятора, и параметры сборки компилятора, чтобы лучше понимать получающиеся бинарные файлы.

Сноски

[1] [назад] Рабочий процесс декомпиляции Ghidra разбит на 15 шагов, включающих подкомпоненты. Полную внутреннюю документацию для Ghidra `Decompiler` можно извлечь с помощью Doxygen.
[2] [назад] Для функций, которые не имеют переменного числа аргументов, следует изменять сигнатуру функции, а не переопределять ее в месте вызова.

Глава 20. Вариации компиляторов

На этом этапе, если мы хорошо сделали свою работу, у вас уже есть основные навыки, чтобы эффективно использовать Ghidra и, что важнее, подчинять ее своей воле. Следующий шаг - научиться адаптироваться к проблемам, которые будут создавать для вас бинарные файлы (а не Ghidra). В зависимости от того, зачем вы смотрите на язык ассемблера, вы можете либо очень хорошо знать то, что видите, либо никогда заранее не знать, с чем столкнетесь. Если вы все время изучаете код, скомпилированный с помощью `gcc` на платформе Linux, вы хорошо привыкнете к стилю кода, который он генерирует, но отладочная версия программы, скомпилированная с помощью компилятора Microsoft C/C++, может поставить вас в тупик. Если вы аналитик вредоносного ПО, за один и тот же день вы можете увидеть код, созданный с помощью `gcc`, `clang`, компилятора Microsoft C++, Delphi и других.

Как и вы, Ghidra лучше знакома с выводом одних компиляторов, чем других, и знакомство с кодом, сгенерированным одним компилятором, никак не гарантирует, что вы распознаете высокоуровневые конструкции, скомпилированные с помощью совершенно другого компилятора (или даже другой версии того же семейства компиляторов). Вместо того чтобы полностью полагаться на аналитические возможности Ghidra при распознавании часто используемых конструкций кода и данных, вы всегда должны быть готовы задействовать собственные навыки: знакомство с конкретным языком ассемблера, знания о компиляторах и исследовательские навыки, необходимые для правильной интерпретации дизассемблирования.

В этой главе мы рассмотрим некоторые способы, которыми различия компиляторов проявляются в дизассемблерных листингах. В примерах мы в основном используем скомпилированный C-код, потому что изменчивость C-компиляторов и целевых платформ дает фундаментальные концепции, которые можно распространить на другие компилируемые языки.

Высокоуровневые конструкции

В некоторых случаях различия между компиляторами могут быть просто косметическими, но в других они гораздо значительнее. В этом разделе мы рассмотрим конструкции высокоуровневого языка и покажем, как разные компиляторы и параметры компилятора могут существенно влиять на получающийся дизассемблерный листинг. Мы начнем с операторов `switch` и двух механизмов, наиболее часто используемых для выбора варианта `switch`. Затем рассмотрим, как параметры компилятора влияют на генерацию кода для распространенных выражений, прежде чем перейти к обсуждению того, как разные компиляторы реализуют конструкции, специфичные для C++, и обрабатывают запуск программы.

Операторы switch

C-оператор `switch` часто является целью оптимизаций компилятора. Цель этих оптимизаций - сопоставить переменную `switch` с допустимой меткой `case` как можно эффективнее, но распределение меток `case` в `switch` ограничивает тип поиска, который можно использовать. Поскольку эффективность поиска измеряется количеством сравнений, необходимых для нахождения правильного варианта, мы можем проследить логику, которую компилятор может использовать, чтобы определить лучший способ представления таблицы `switch`. Алгоритм с постоянным временем, например поиск по таблице, является самым эффективным.[1] На другом конце континуума находится линейный поиск, который в худшем случае требует сравнить переменную `switch` с каждой меткой `case`, прежде чем найти совпадение или перейти к варианту по умолчанию, и потому является наименее эффективным.[2] Эффективность бинарного поиска в среднем намного лучше, чем линейного поиска, но он вводит дополнительные ограничения, поскольку требует отсортированного списка.[3]

Чтобы выбрать наиболее эффективную реализацию для конкретного оператора `switch`, полезно понимать, как распределение меток `case` влияет на процесс принятия решений компилятором. Когда метки `case` сгруппированы близко, как в исходном коде в листинге 20-1, компиляторы обычно разрешают переменную `switch`, выполняя поиск по таблице, чтобы сопоставить переменную `switch` с адресом связанного варианта, конкретно - используя таблицу переходов.

```
switch (a) {
/** ПРИМЕЧАНИЕ: тела case опущены для краткости **/
    case 1:  /*...*/ break;
    case 2:  /*...*/ break;
    case 3:  /*...*/ break;
    case 4:  /*...*/ break;
    case 5:  /*...*/ break;
    case 6:  /*...*/ break;
    case 7:  /*...*/ break;
    case 8:  /*...*/ break;
    case 9:  /*...*/ break;
    case 10: /*...*/ break;
    case 11: /*...*/ break;
    case 12: /*...*/ break;
}
```

Листинг 20-1. Оператор `switch` *с последовательными метками* `case`

Таблица переходов - это массив указателей, где каждый указатель в массиве ссылается на возможную цель перехода. Во время выполнения динамический индекс в таблице выбирает один из множества потенциальных переходов каждый раз, когда используется таблица переходов. Таблицы переходов хорошо работают, когда метки `case` в `switch` расположены близко друг к другу (плотно), и большинство вариантов попадает в последовательный числовой ряд. Компиляторы учитывают это, когда решают, использовать ли таблицу переходов. Для любого оператора `switch` можно вычислить минимальное количество записей, которое будет содержать связанная таблица переходов:

```
num_entries = max_case_value - min_case_value + 1
```

Плотность, или коэффициент использования, таблиц переходов можно затем вычислить так:

```
density = num_cases / num_entries
```

Полностью непрерывный список, где представлено каждое значение, будет иметь значение плотности 100 процентов (1.0). Наконец, общий объем пространства, необходимый для хранения таблицы переходов, таков:

```
table_size = num_entries * sizeof(void*)
```

Оператор `switch` со 100-процентной плотностью будет реализован с использованием таблицы переходов. Набор вариантов с плотностью 30 процентов может не быть реализован с помощью таблицы переходов, поскольку записи таблицы переходов все равно придется выделять для отсутствующих вариантов, которые составят 70 процентов таблицы переходов. Если `num_entries` равно 30, таблица переходов будет содержать записи для 21 метки `case`, на которую нет ссылок. На 64-битной системе это 168 из 240 байт, выделенных для таблицы, что не очень много накладных расходов, но если `num_entries` подскакивает до 300, накладные расходы становятся 1680 байт, и этот компромисс может не стоить того ради 90 возможных вариантов. Компилятор, оптимизирующий скорость, может предпочесть реализации через таблицу переходов, тогда как компилятор, оптимизирующий размер, может выбрать альтернативную реализацию с меньшими накладными расходами памяти: бинарный поиск.

Бинарный поиск эффективен, когда метки `case` сильно разбросаны (низкая плотность), как видно в листинге 20-2 (плотность 0.0008).[4] Поскольку бинарный поиск работает только с отсортированными списками, компилятор должен убедиться, что метки `case` упорядочены, прежде чем начать поиск со среднего значения. Это может привести к переупорядочиванию блоков `case` при просмотре дизассемблирования по сравнению с порядком, в котором они появляются в соответствующем исходном коде.[5]

```
switch (a) {
/** ПРИМЕЧАНИЕ: тела case опущены для краткости **/
    case 1:     /*...*/ break;
    case 211:   /*...*/ break;
    case 295:   /*...*/ break;
    case 462:   /*...*/ break;
    case 528:   /*...*/ break;
    case 719:   /*...*/ break;
    case 995:   /*...*/ break;
    case 1024:  /*...*/ break;
    case 8000:  /*...*/ break;
    case 13531: /*...*/ break;
    case 13532: /*...*/ break;
    case 15027: /*...*/ break;
}
```

Листинг 20-2. Пример оператора `switch` *с непоследовательными метками* `case`

Листинг 20-3 показывает схему неитеративного бинарного поиска по фиксированному числу постоянных значений. Это приблизительный каркас, который компилятор использует для реализации `switch` из листинга 20-2.

```
if (value < median) {
    // значение находится в процентиле [0-50)
    if (value < lower_half_median) {
        // значение находится в процентиле [0-25)
        // ... продолжать последовательное деление пополам до определения значения
    } else {
        // значение находится в процентиле [25-50)
        // ... продолжать последовательное деление пополам до определения значения
   }
} else {
    // значение находится в процентиле [50-100)
    if (value < upper_half_median) {
        // значение находится в процентиле [50-75)
```

```
        // ... продолжать последовательное деление пополам до определения значения
    } else {
        // значение находится в процентиле [75-100)
        // ... продолжать последовательное деление пополам до определения значения
    }
}
```

Листинг 20-3. Неитеративный бинарный поиск по фиксированному числу постоянных значений

Компиляторы также способны выполнять более тонкие оптимизации по диапазону меток `case`. Например, столкнувшись с метками `case`

```
label_set = [1, 2, 3, 4, 5, 6, 7, 8, 50, 80, 200, 500, 1000, 5000, 10000]
```

менее агрессивный компилятор может увидеть здесь плотность 0.0015 и сгенерировать бинарный поиск по всем 15 вариантам. Более агрессивный компилятор может выдать таблицу переходов для разрешения вариантов с 1 по 8 и бинарный поиск для оставшихся вариантов, достигая оптимальной производительности более чем для половины вариантов.

Прежде чем смотреть на дизассемблированные версии листингов 20-1 и 20-2, посмотрим на окна Ghidra Function Graph, соответствующие этим листингам и показанные рядом на рисунке 20-1.

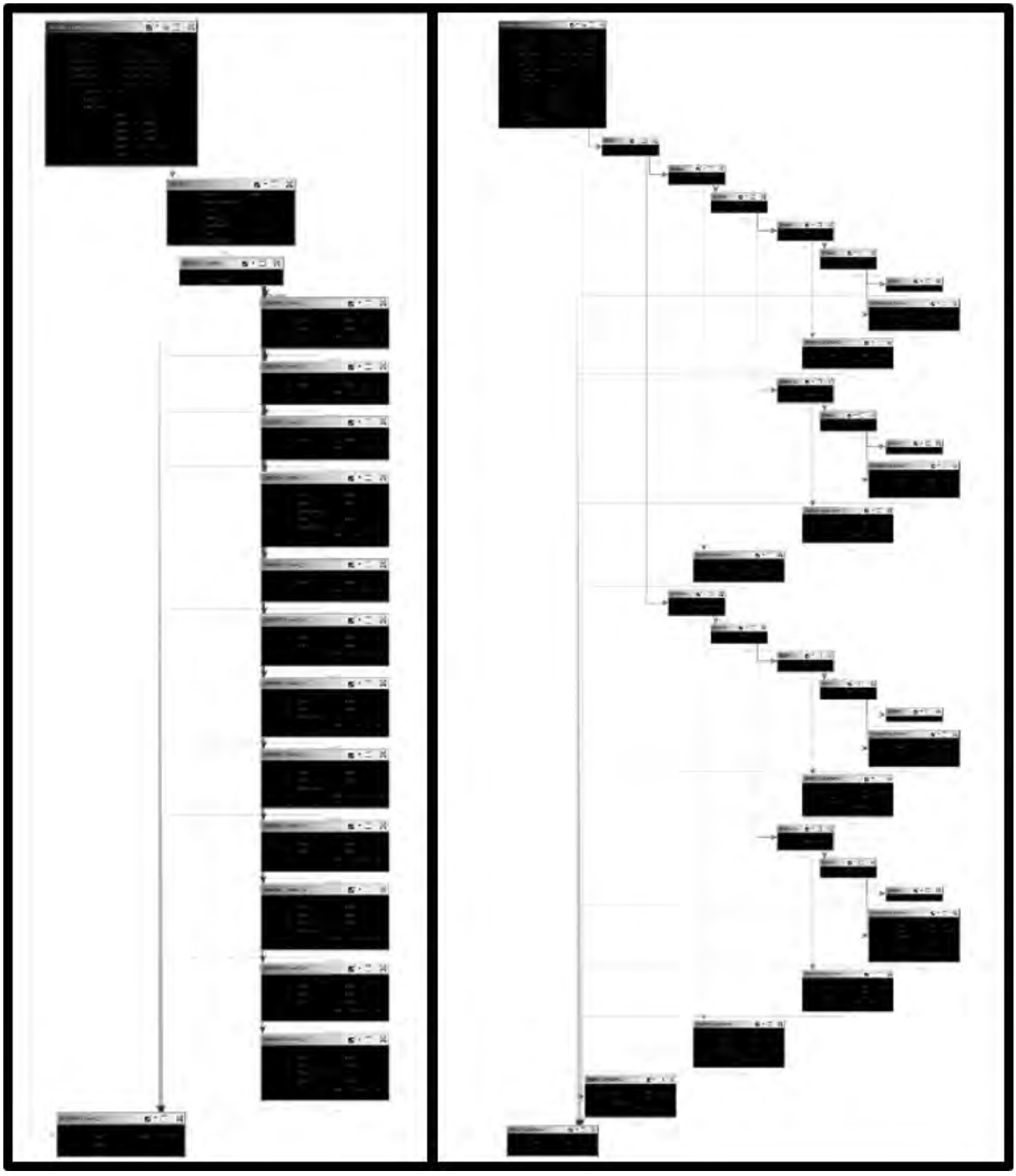


Рисунок 20-1. Примеры операторов switch в Ghidra Function Graph

Слева граф для листинга 20-1 показывает аккуратный вертикальный стек вариантов. Каждый уложенный в стек блок кода находится на той же глубине вложенности, что верно для вариантов в операторе `switch`. Стек подсказывает, что мы можем использовать индекс для быстрого выбора одного блока из многих (подумайте о доступе к массиву). Именно так работает разрешение через таблицу переходов, и левый граф дает визуальную подсказку об этом еще до того, как мы посмотрели хотя бы одну строку дизассемблирования.

Правый граф - это результат Ghidra, основанный исключительно на ее понимании дизассемблирования листинга 20-2. Отсутствие таблицы переходов значительно усложняет идентификацию этого кода как оператора `switch`. То, что вы видите, - визуальное представление оператора `switch` с использованием компоновки `Ghidra Nested Code Layout`. Это компоновка по умолчанию для графов функций в Ghidra, предназначенная для представления структур потока в

программе. Горизонтальное ветвление в этом графе подсказывает условное выполнение (`if`/`else`) с ветвлением к взаимоисключающим альтернативам. Вертикальная симметрия говорит о том, что альтернативные пути выполнения были очень тщательно сбалансированы, чтобы поместить равное количество блоков в каждую вертикальную половину графа. Наконец, расстояние, на которое граф проходит по горизонтали, является индикатором глубины, достигнутой поиском, которая, в свою очередь, определяется общим количеством меток `case` в `switch`. Для бинарного поиска эта глубина всегда будет порядка `log2(num_cases)`. Сходство между отступами графического представления и алгоритмом, описанным в листинге 20-3, легко наблюдаемо.

Перейдем к окну `Decompiler`. Рисунок 20-2 показывает частичную декомпиляцию функций, отображенных на рисунке 20-1. Слева находится декомпилированная версия листинга 20-1. Как и в графе, присутствие таблицы переходов в бинарном файле помогает Ghidra распознать код как оператор `switch`.

Справа находится декомпилированная версия листинга 20-2. `Decompiler` представил оператор `switch` как вложенную структуру `if`/`else`, согласующуюся с бинарным поиском и похожую по структуре на листинг 20-3. Можно видеть, что первое сравнение выполняется с 719, средним значением в списке, а последующие сравнения продолжают делить пространство поиска пополам. Обращаясь к рисунку 20-1 (а также к листингу 20-3), мы снова можем наблюдать, что графические представления каждой функции тесно соответствуют шаблонам отступов, наблюдаемым в окне `Decompiler`.

Теперь, когда у вас есть представление, что происходит на высоком уровне, заглянем внутрь бинарных файлов и исследуем, что происходит на низком уровне. Поскольку наша цель в этой главе - наблюдать различия между компиляторами, мы представим этот пример как серию сравнений между двумя компиляторами, `gcc` и Microsoft C/C++.[6]

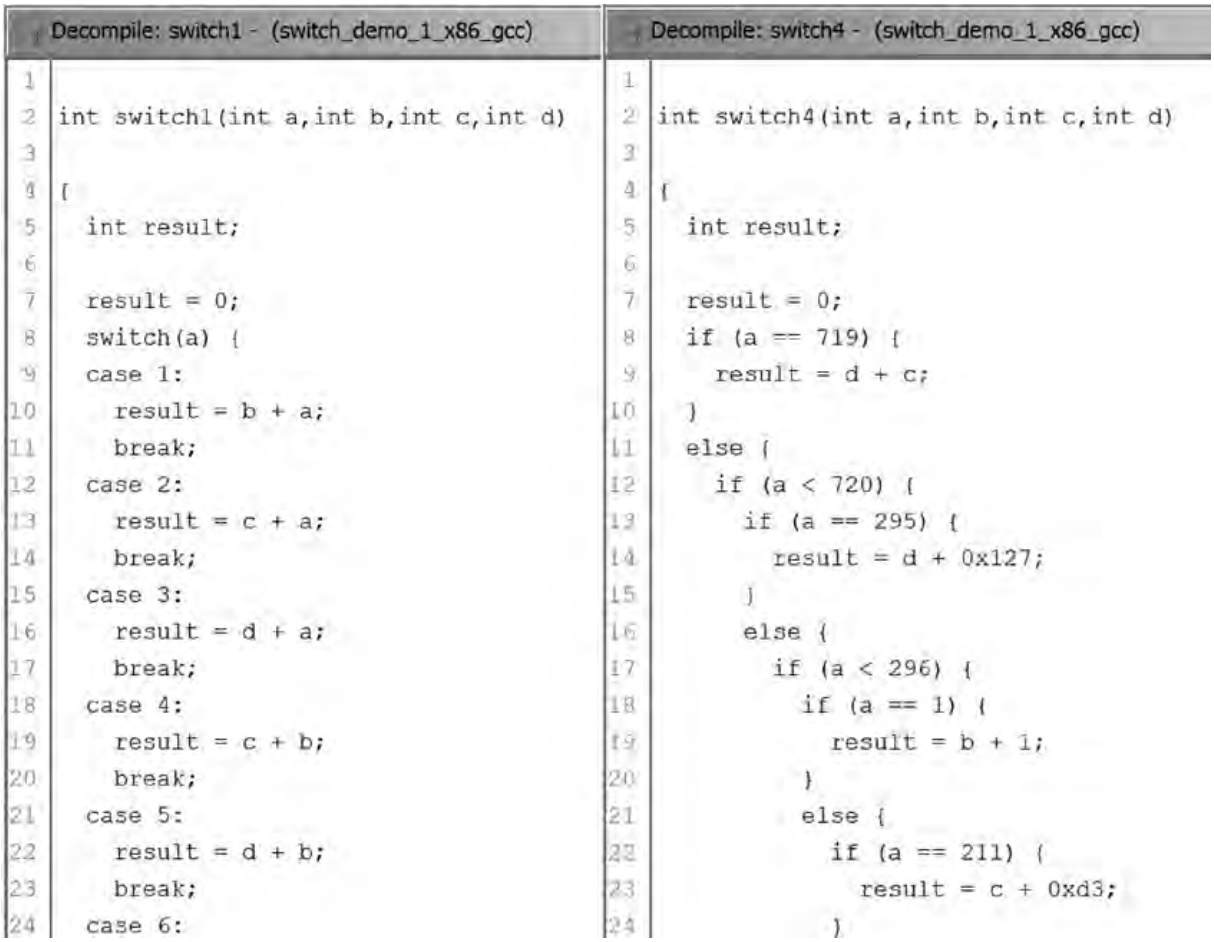


Рисунок 20-2. Декомпилированные примеры операторов switch в Ghidra

Пример: сравнение gcc с компилятором Microsoft C/C++

В этом примере мы сравним два 32-битных x86-бинарных файла, сгенерированных для листинга 20-1 двумя разными компиляторами. Мы попытаемся выявить компоненты оператора `switch` в каждом бинарном файле, найти связанную таблицу переходов в каждом бинарном файле и указать значимые различия между двумя бинарными файлами. Начнем с просмотра компонентов, связанных со `switch`, для листинга 20-1 в бинарном файле, собранном с помощью `gcc`:

```
0001075a  CMP    dword ptr [EBP + value],12
0001075e  JA     switchD_00010771::caseD_0
00010764  MOV    EAX,dword ptr [EBP + a]
00010767  SHL    EAX,0x2
0001076a  ADD    EAX,switchD_00010771::switchdataD_00010ee0       = 00010805
0001076f  MOV    EAX,dword ptr [EAX]=>->switchD_00010771::caseD_0 = 00010805
        switchD_00010771::switchD
00010771  JMP    EAX
        switchD_00010771::caseD_1             XREF[2]:      00010771(j), 00010ee4(
    *)
00010773  MOV    EDX,dword ptr [EBP + a]
00010776  MOV    EAX,dword ptr [EBP + b]
00010779  ADD    EAX,EDX
0001077b  MOV    dword ptr [EBP + result],EAX
0001077e  JMP    switchD_00010771::caseD_0
;--содержимое остальных вариантов опущено--
        switchD_00010771::switchdataD_00010ee0  XREF[2]: switch_version_1:
    0001076a(*),
                                                         switch_version_1:
    0001076f(R)
00010ee0  addr    switchD_00010771::caseD_0
```

```
00010ee4  addr    switchD_00010771::caseD_1
00010ee8  addr    switchD_00010771::caseD_2
00010eec  addr    switchD_00010771::caseD_3
00010ef0  addr    switchD_00010771::caseD_4
00010ef4  addr    switchD_00010771::caseD_5
00010ef8  addr    switchD_00010771::caseD_6
00010efc  addr    switchD_00010771::caseD_7
00010f00  addr    switchD_00010771::caseD_8
00010f04  addr    switchD_00010771::caseD_9
00010f08  addr    switchD_00010771::caseD_a
00010f0c  addr    switchD_00010771::caseD_b
00010f10  addr    switchD_00010771::caseD_c
```

Ghidra распознает проверку границ `switch`, таблицу переходов и отдельные блоки `case` по значению, например блок по адресу `switchD_00010771::caseD_1`. Компилятор сгенерировал таблицу переходов с 13 записями, хотя листинг 20-1 содержал только 12 вариантов. Дополнительный вариант, case 0 (первая запись в таблице переходов), разделяет целевой адрес со всеми значениями вне диапазона 1-12. Иными словами, case 0 является частью варианта по умолчанию. Хотя может показаться, что отрицательные числа исключены из варианта по умолчанию, последовательность `CMP`, `JA` работает как сравнение беззнаковых значений; поэтому -1 (`0xFFFFFFFF`) будет рассматриваться как 4294967295, что намного больше 12, и потому будет исключено из допустимого диапазона для индексирования таблицы переходов. Инструкция `JA` направляет все такие варианты в расположение по умолчанию: `switchD_00010771::caseD_0`.

Теперь, когда мы понимаем базовые компоненты кода, сгенерированного компилятором `gcc`, перенесем внимание на те же компоненты в коде, сгенерированном компилятором Microsoft C/C++ в режиме отладки:

```
00411e88  MOV    ECX,dword ptr [EBP + local_d4]
00411e8e  SUB    ECX,0x1
00411e91  MOV    dword ptr [EBP + local_d4],ECX
00411e97  CMP    dword ptr [EBP + local_d4],11
00411e9e  JA     switchD_00411eaa::caseD_c
00411ea4  MOV    EDX,dword ptr [EBP + local_d4]
        switchD_00411eaa::switchD
00411eaa  JMP    dword ptr [EDX*0x4 + ->switchD_00411eaa::caseD      = 00411eb1
        switchD_00411eaa::caseD_1                XREF[2]: 00411eaa(j), 00411f4c(*)
00411eb1  MOV    EAX,dword ptr [EBP + param_1]
00411eb4  ADD    EAX,dword ptr [EBP + param_2]
00411eb7  MOV    dword ptr [EBP + local_c],EAX
00411eba  JMP    switchD_00411eaa::caseD_c
;--содержимое остальных вариантов опущено--
        switchD_00411eaa::switchdataD_00411f4c   XREF[1]: switch_version_1:
    00411eaa(R)
00411f4c  addr   switchD_00411eaa::caseD_1
00411f50  addr   switchD_00411eaa::caseD_2
00411f54  addr   switchD_00411eaa::caseD_3
00411f58  addr   switchD_00411eaa::caseD_4
00411f5c  addr   switchD_00411eaa::caseD_5
00411f60  addr   switchD_00411eaa::caseD_6
00411f64  addr   switchD_00411eaa::caseD_7
00411f68  addr   switchD_00411eaa::caseD_8
00411f6c  addr   switchD_00411eaa::caseD_9
00411f70  addr   switchD_00411eaa::caseD_a
00411f74  addr   switchD_00411eaa::caseD_b
00411f78  addr   switchD_00411eaa::caseD_c
```

Здесь переменная `switch` (`local_d4` в этом случае) уменьшается, чтобы сдвинуть диапазон допустимых значений к 0-11, устраняя необходимость в фиктивной записи таблицы для значения 0. В результате первая запись (или запись с индексом 0) в таблице переходов фактически ссылается на код для варианта `switch` case 1.

Другое, возможно более тонкое различие между двумя листингами - расположение таблицы переходов внутри файла. Компилятор `gcc` помещает таблицы переходов `switch` в секцию данных только для чтения (`.rodata`) бинарного файла, обеспечивая логическое разделение между кодом,

связанным с оператором `switch`, и данными, требуемыми для реализации таблицы переходов. Компилятор Microsoft C/C++, напротив, вставляет таблицы переходов в секцию `.text`, сразу после функции, содержащей связанный оператор `switch`. Такое размещение таблицы переходов почти не влияет на поведение программы. В этом примере Ghidra способна распознать операторы `switch` для обоих компиляторов и использует термин `switch` в связанных метках.

Один из ключевых моментов здесь: не существует единственного правильного способа скомпилировать исходный код в ассемблер. Поэтому нельзя считать, что что-то не является оператором `switch` только потому, что Ghidra не смогла пометить это как `switch`. Понимание характеристик оператора `switch`, влияющих на реализацию компилятора, может помочь сделать более точный вывод об исходном коде.

Параметры сборки компилятора

Компилятор преобразует высокоуровневый код, решающий конкретную задачу, в низкоуровневый код, решающий ту же задачу. Несколько компиляторов могут решать одну и ту же проблему весьма разными способами. Кроме того, один компилятор может решать задачу очень по-разному в зависимости от связанных параметров компилятора. В этом разделе мы посмотрим на код языка ассемблера, получающийся при использовании разных компиляторов и разных параметров командной строки. Некоторые различия будут иметь ясное объяснение; другие - нет.

Microsoft Visual Studio может собирать либо отладочные, либо выпускные версии бинарных файлов программ.[7] Чтобы увидеть, чем эти две версии отличаются, сравните параметры сборки, указанные для каждой. Выпускные версии обычно оптимизированы, тогда как отладочные версии - нет; отладочные версии связываются с дополнительной символьной информацией и отладочными версиями библиотеки времени выполнения, тогда как выпускные версии - нет.[8] Символы, связанные с отладкой, позволяют отладчикам сопоставлять операторы языка ассемблера с их аналогами в исходном коде и определять имена локальных переменных (такая информация иначе теряется во время процесса компиляции). Отладочные версии библиотек времени выполнения Microsoft также скомпилированы с включенными отладочными символами, отключенными оптимизациями и дополнительными проверками безопасности, чтобы проверять допустимость некоторых параметров функций.

При дизассемблировании с помощью Ghidra отладочные сборки проектов Visual Studio выглядят значительно иначе, чем выпускные сборки. Это результат параметров компилятора и компоновщика, указанных только в отладочных сборках, например базовых проверок времени выполнения (`/RTCx`), которые вводят дополнительный код в получающийся бинарный файл.[9] Давайте сразу посмотрим на некоторые из этих различий в дизассемблировании.

Пример 1: оператор остатка

Начнем примеры с простой математической операции, взятия остатка. Следующий листинг содержит исходный код программы, единственная цель которой - принять целочисленное значение от пользователя и продемонстрировать целочисленное деление и оператор остатка:

```
int main(int argc, char **argv) {
    int x;
    printf("Введите целое число: ");
    scanf("%d", &x);
    printf("%d %% 10 = %d\n", x, x % 10);
}
```

Исследуем, как дизассемблирование различается между компиляторами для оператора остатка в этом примере.

Остаток с Microsoft C/C++ Win x64 Debug

Следующий листинг показывает код, который Visual Studio генерирует, когда настроена сборка отладочной версии бинарного файла:

```
1400119c6 MOV    EAX,dword ptr [RBP + local_f4]
1400119c9 CDQ
1400119ca MOV    ECX,0xa
1400119cf IDIV   ECX
1400119d1 MOV    EAX,EDX
1400119d3 MOV    R8D,EAX
1400119d6 MOV    EDX,dword ptr [RBP + local_f4]
1400119d9 LEA    RCX,[s_%d_%%_10_=_%d_140019d60]
1400119e0 CALL   printf
```

Прямая инструкция x86 `IDIV` оставляет частное в `EAX`, а остаток от деления - в `EDX`. Затем результат перемещается в младшие 32 бита `R8` (`R8D`), который является третьим аргументом в вызове `printf`.

Остаток с Microsoft C/C++ Win x64 Release

Выпускные сборки оптимизируют ПО по скорости и размеру, чтобы повысить производительность и минимизировать требования к хранению. При оптимизации по скорости авторы компиляторов могут прибегать к неочевидным реализациям распространенных операций. Следующий листинг показывает, как Visual Studio генерирует ту же операцию остатка в выпускном бинарном файле:

```
140001136 MOV    ECX,dword ptr [RSP + local_18]
14000113a MOV    EAX,0x66666667
14000113f IMUL   ECX
140001141 MOV    R8D,ECX
140001144 SAR    EDX,0x2
140001147 MOV    EAX,EDX
140001149 SHR    EAX,0x1f
14000114c ADD    EDX,EAX
14000114e LEA    EAX,[RDX + RDX*0x4]
140001151 MOV    EDX,ECX
140001153 ADD    EAX,EAX
140001155 LEA    RCX,[s_%d_%%_10_=_%d_140002238]
14000115c SUB    R8D,EAX
14000115f CALL   printf
```

В этом случае используется умножение, а не деление, и после длинной последовательности арифметических операций то, что должно быть результатом операции остатка, оказывается в `R8D` (снова третий аргумент в вызове `printf`). Интуитивно, правда? Объяснение этого кода следует за следующим примером.

Остаток с gcc для Linux x64

Мы увидели, как по-разному может вести себя один компилятор просто при изменении параметров времени компиляции, используемых для генерации бинарного файла. Можно было бы ожидать, что совершенно несвязанный компилятор снова сгенерирует совсем другой код. Следующее дизассемблирование показывает версию той же операции остатка от `gcc`, и оказывается, что она выглядит отчасти знакомо:

```
00100708  MOV    ECX,dword ptr [RBP + x]
0010070b  MOV    EDX,0x66666667
00100710  MOV    EAX,ECX
00100712  IMUL   EDX
00100714  SAR    EDX,0x2
00100717  MOV    EAX,ECX
00100719  SAR    EAX,0x1f
0010071c  SUB    EDX,EAX
0010071e  MOV    EAX,EDX
00100720  SHL    EAX,0x2
00100723  ADD    EAX,EDX
00100725  ADD    EAX,EAX
00100727  SUB    ECX,EAX
00100729  MOV    EDX,ECX
```

Код очень похож на ассемблер, созданный выпускной версией Visual Studio. Мы снова видим умножение, а не деление, за которым следует последовательность арифметических операций, в итоге оставляющая результат в `EDX`, где он затем используется как третий аргумент для `printf`.

Код использует мультипликативную обратную величину, чтобы выполнять деление через умножение, потому что аппаратное умножение быстрее аппаратного деления. Вы также можете увидеть умножение, реализованное с помощью серии сложений и арифметических сдвигов, поскольку каждая из этих операций в аппаратуре значительно быстрее умножения.

Ваша способность распознать этот код как остаток от деления на 10 зависит от опыта, терпения и творческого подхода. Если вы раньше видели похожие последовательности кода, вероятно, вам проще распознать, что здесь происходит. Если такого опыта нет, можно вручную пройти код с примерными значениями, надеясь распознать шаблон в результатах. Можно даже извлечь язык ассемблера, обернуть его в тестовый каркас на C и выполнить высокоскоростную генерацию данных в помощь себе. `Decompiler` Ghidra может быть еще одним полезным ресурсом для сведения сложных или необычных последовательностей кода к более узнаваемым C-эквивалентам.

В качестве последнего средства или первого средства (не стыдитесь) можно обратиться за ответами в интернет. Но что искать? Обычно уникальные, конкретные запросы дают наиболее релевантные результаты, и самая уникальная особенность в последовательности кода - целочисленная константа `0x66666667`. При поиске этой константы три первых результата оказались полезны, но один особенно стоил закладки: http://flaviojslab.blogspot.com/2008/02/integer-division.html. Уникальные константы также довольно часто используются в криптографических алгоритмах, и быстрый поиск в интернете может быть всем, что нужно, чтобы точно определить, на какую криптографическую процедуру вы смотрите.

Пример 2: тернарный оператор

Тернарный оператор вычисляет выражение, а затем выдает один из двух возможных результатов в зависимости от булева значения этого выражения. Концептуально тернарный оператор можно рассматривать как оператор `if`/`else` (и даже заменить оператором `if`/`else`). Следующий намеренно неоптимизированный исходный код демонстрирует использование этого оператора:

```
int main() {
    volatile int x = 3;
    volatile int y = x * 13;
    volatile int z = y == 30 ? 0 : -1;
}
```

Примечание. Ключевое слово `volatile` просит компилятор не оптимизировать код, затрагивающий связанные переменные. Без его использования здесь некоторые компиляторы оптимизируют все тело этой функции, поскольку ни один оператор не влияет на результат функции. Это одна из трудностей, с которыми вы можете столкнуться, когда пишете примеры для себя или для других.

Что касается поведения неоптимизированного кода, присваивание в переменную `z` можно заменить следующим оператором `if`/`else` без изменения семантики программы:

```
    if (y == 30) {
        z = 0;
    } else {
        z = -1;
    }
```

Посмотрим, как код тернарного оператора обрабатывается разными компиляторами и разными параметрами компилятора.

Тернарный оператор с gcc в Linux x64

`gcc` без параметров сгенерировал следующий ассемблер для инициализации `z`:

```
00100616  MOV    EAX,dword ptr [RBP + y]
00100619  CMP    EAX,0x1e
0010061c  JNZ    LAB_00100625
0010061e  MOV    EAX,0x0
00100623  JMP    LAB_0010062a
        LAB_00100625
00100625  MOV    EAX,0xffffffff
        LAB_0010062a
0010062a  MOV    dword ptr [RBP + z],EAX
```

Этот код использует реализацию `if`/`else`. Локальная переменная y сравнивается с 30, чтобы решить, установить ли EAX в 0 или `0xffffffff` в противоположных ветвях `if`/`else`, перед присваиванием результата в `z`.

Тернарный оператор с Microsoft C/C++ Win x64 Release

Visual Studio дает совсем другую реализацию оператора, содержащего тернарный оператор. Здесь компилятор распознает, что одну инструкцию можно использовать, чтобы условно сгенерировать либо 0, либо -1 (и никакое другое возможное значение), и использует эту инструкцию вместо конструкции `if`/`else`, которую мы видели раньше:

```
140001013 MOV    EAX,dword ptr [RSP + local_res8]
140001017 SUB    EAX,0x1e
14000101a NEG    EAX
14000101c SBB    EAX,EAX
14000101e MOV    dword ptr [RSP + local_res8],EAX
```

Инструкция `SBB` (вычитание с заемом) вычитает второй операнд из первого, а затем вычитает флаг переноса `CF`, который может быть только 0 или 1. Эквивалентное арифметическое выражение для `SBB EAX,EAX` - `EAX - EAX - CF`, что сводится к `0 - CF`. В свою очередь это может дать только 0 (когда `CF == 0`) или -1 (когда `CF == 1`). Чтобы этот трюк сработал, компилятор должен правильно установить перенос до выполнения инструкции `SBB`. Это достигается сравнением `EAX` с константой `0x1e` (30) с помощью вычитания, которое оставляет `EAX` равным 0 только тогда, когда `EAX` изначально был `0x1e`. Инструкция `NEG` затем устанавливает флаг переноса для следующей инструкции `SBB`.[10]

Тернарный оператор с gcc в Linux x64 (оптимизированный)

Когда мы просим `gcc` постараться немного сильнее, оптимизируя код (`-O2`), результат напоминает код Visual Studio из предыдущего примера:

```
00100506  MOV    EAX,dword ptr [RSP + y]
0010050a  CMP    EAX,0x1e
0010050d  SETNZ  AL
00100510  MOVZX  EAX,AL
00100513  NEG    EAX
00100515  MOV    dword ptr [RSP + z],EAX
```

В этом случае `gcc` использует `SETNZ`, чтобы условно установить регистр `AL` либо в 0, либо в 1 на основе состояния флага нуля, получившегося от предыдущего сравнения. Затем результат меняет знак, чтобы стать либо 0, либо -1 перед присваиванием в переменную `z`.

Пример 3: встраивание функций

Когда программист помечает функцию как `inline`, он подсказывает компилятору, что любые вызовы функции должны быть заменены копией всего тела функции. Цель - ускорить вызов функции, устранив настройку и снятие параметров и стекового фрейма. Компромисс состоит в том, что множество копий встроенной функции увеличивают бинарный файл. Встроенные функции могут быть очень трудны для распознавания в бинарных файлах, потому что характерная инструкция `call` устраняется.

Даже когда ключевое слово `inline` не использовалось, компиляторы могут решить встроить функцию по собственной инициативе. В третьем примере мы выполняем вызов следующей функции:

```
int maybe_inline() {
    return 0x12abcdef;
}
int main() {
    int v = maybe_inline();
    printf("after maybe_inline: v = %08x\n", v);return 0;
}
```

Вызов функции с gcc в Linux x86

После сборки Linux x86-бинарного файла с помощью `gcc` без оптимизаций мы дизассемблируем его и видим следующий листинг:

```
00010775  PUSH   EBP
00010776  MOV    EBP,ESP
00010778  PUSH   ECX
00010779  SUB    ESP,0x14
0001077c  CALL   maybe_inline
00010781  MOV    dword ptr [EBP + local_14],EAX
00010784  SUB    ESP,0x8
00010787  PUSH   dword ptr [EBP + local_14]
0001078a  PUSH   s_after_maybe_inline:_v_=_%08x_000108e2
0001078f  CALL   printf
```

Мы ясно видим вызов функции `maybe_inline` в этом дизассемблировании, хотя это всего одна строка кода, возвращающая постоянное значение.

Оптимизированный вызов функции с gcc в Linux x86

Затем посмотрим на оптимизированную (`-O2`) версию того же исходного кода:

```
0001058a  PUSH   EBP
0001058b  MOV    EBP,ESP
0001058d  PUSH   ECX
0001058e  SUB    ESP,0x8
00010591  PUSH   0x12abcdef
00010596  PUSH   s_after_maybe_inline:_v_=_%08x_000108c2
0001059b  PUSH   0x1
0001059d  CALL   __printf_chk
```

Сопоставляя этот код с неоптимизированным кодом, мы видим, что вызов `maybe_inline` устранен, а постоянное значение, возвращаемое `maybe_inline`, помещается прямо в стек для использования как аргумент вызова `printf`. Эта оптимизированная версия вызова функции идентична тому, что вы увидели бы, если бы функция была обозначена как `inline`.

Рассмотрев некоторые способы, которыми оптимизации могут влиять на код, генерируемый компиляторами, обратимся к разным способам, которыми разработчики компиляторов выбирают реализацию языково-специфичных возможностей, когда разработчики языков оставляют детали реализации авторам компиляторов.

Специфичная для компилятора реализация C++

Языки программирования проектируются программистами для программистов. Когда пыль процесса проектирования улеглась, авторы компиляторов должны построить инструменты, которые добросовестно переводят программы, написанные на новом высокоуровневом языке, в семантически эквивалентные программы на машинном языке. Когда язык позволяет программисту делать A, B и C, авторы компиляторов должны найти способ сделать это возможным.

C++ дает нам три отличных примера поведения, требуемого языком, но детали реализации которого оставлены автору компилятора:

- Внутри нестатической функции-члена класса программисты могут ссылаться на переменную с именем `this`, которая нигде явно не объявляется. Обработку этого компиляторами см. в главах 6 и 8.
- Перегрузка функций разрешена. Программисты свободны повторно использовать имена функций так часто, как им угодно, с учетом ограничений на списки параметров.
- Интроспекция типов поддерживается с помощью операторов `dynamic_cast` и `typeid`.

Перегрузка функций

Перегрузка функций в C++ позволяет программистам называть функции одинаково с оговоркой, что любые две функции с общим именем должны иметь разные последовательности параметров. Искажение имен, представленное в главе 8, - внутренний механизм, который позволяет перегрузке работать, гарантируя, что к моменту, когда компоновщик должен выполнить свою работу, никакие два символа не имеют одного и того же имени.

Часто один из самых ранних признаков того, что вы работаете с C++-бинарным файлом, - присутствие искаженных имен. Две наиболее популярные схемы искажения имен - Microsoft и Intel Itanium ABI.[11] Стандарт Intel был широко принят другими Unix-компиляторами, такими как `g++` и `clang`. Ниже показано имя C++-функции и искаженная версия этого имени в схемах Microsoft и Intel:

```
Функция          void SubClass::vfunc1()
Схема Microsoft  ?vfunc1@SubClass@@UAEXXZ
Схема Intel      _ZN8SubClass6vfunc1Ev
```

Большинство языков, допускающих перегрузку, включая Objective-C, Swift и Rust, включают некоторую форму искажения имен на уровне реализации. Поверхностное знакомство со стилями искажения имен может дать подсказки об исходном языке программы, а также о компиляторе, использованном для сборки программы.

Реализации RTTI

В главе 8 мы обсуждали C++ Runtime Type Identification (RTTI) и отсутствие стандарта реализации RTTI компилятором. Фактически идентификация типов времени выполнения вообще не упоминается в стандарте C++, поэтому неудивительно, что реализации различаются. Чтобы поддержать оператор `dynamic_cast`, структуры данных RTTI записывают не только имя класса, но и всю его иерархию наследования, включая любые отношения множественного наследования. Поиск структур данных RTTI может быть чрезвычайно полезен при восстановлении объектной модели программы. Автоматическое распознавание конструкций, связанных с RTTI, внутри бинарного файла - еще одна область, где возможности Ghidra различаются между компиляторами.

Программы Microsoft C++ не содержат встроенной символьной информации, но структуры данных RTTI Microsoft хорошо изучены, и Ghidra найдет их при наличии. Любая информация, связанная с RTTI, которую Ghidra найдет, будет обобщена в папке `Classes` дерева `Symbol Tree`, где будет запись для каждого класса, найденного Ghidra с помощью ее анализатора RTTI.

Программы, собранные с помощью g++, включают информацию таблицы символов, если из них не были удалены символы. Для g++-бинарных файлов с неудаленными символами Ghidra полагается исключительно на искаженные имена, которые находит в бинарном файле, и использует эти имена для идентификации структур данных, связанных с RTTI, и классов, связанных с ними. Как и с бинарными файлами Microsoft, любая информация, связанная с RTTI, будет включена в папку `Classes` дерева `Symbol Tree`.

Одна стратегия понимания того, как конкретный компилятор встраивает информацию о типах для C++-классов, - написать простую программу, использующую классы с виртуальными функциями. После компиляции программы можно загрузить получившийся исполняемый файл в Ghidra и поискать экземпляры строк, содержащих имена классов, использованных в программе. Независимо от компилятора, использованного для сборки бинарного файла, одна общая черта структур данных RTTI состоит в том, что все они каким-либо образом ссылаются на строку, содержащую искаженное имя представляемого класса. Используя извлеченные строки и перекрестные ссылки данных, должно быть возможно найти внутри бинарного файла кандидатные структуры данных, связанные с RTTI. Последний шаг - связать кандидатную структуру RTTI обратно с vftable связанного класса; лучше всего это сделать, следуя по перекрестным ссылкам данных назад от кандидатной структуры RTTI, пока не будет достигнута таблица указателей функций (vftable). Разберем пример, использующий этот метод.

Пример: поиск информации RTTI в Linux x86-64 g++-бинарном файле

Чтобы продемонстрировать эти концепции, мы создали небольшую программу с `BaseClass`, `SubClass`, `SubSubClass` и набором виртуальных функций, уникальных для каждого класса. Следующий листинг показывает часть основной программы, которую мы использовали для ссылки на наши классы и функции:

```
BaseClass *bc_ptr_2;
srand(time(0));
if (rand() % 2) {
    bc_ptr_2 = dynamic_cast<SubClass*>(new SubClass());
}
else {
    bc_ptr_2 = dynamic_cast<SubClass*>(new SubSubClass());
}
```

Мы скомпилировали программу с помощью g++, чтобы построить 64-битный Linux-бинарный файл с символами. После анализа программы дерево `Symbol Tree` предоставляет информацию, показанную на рисунке 20-3.

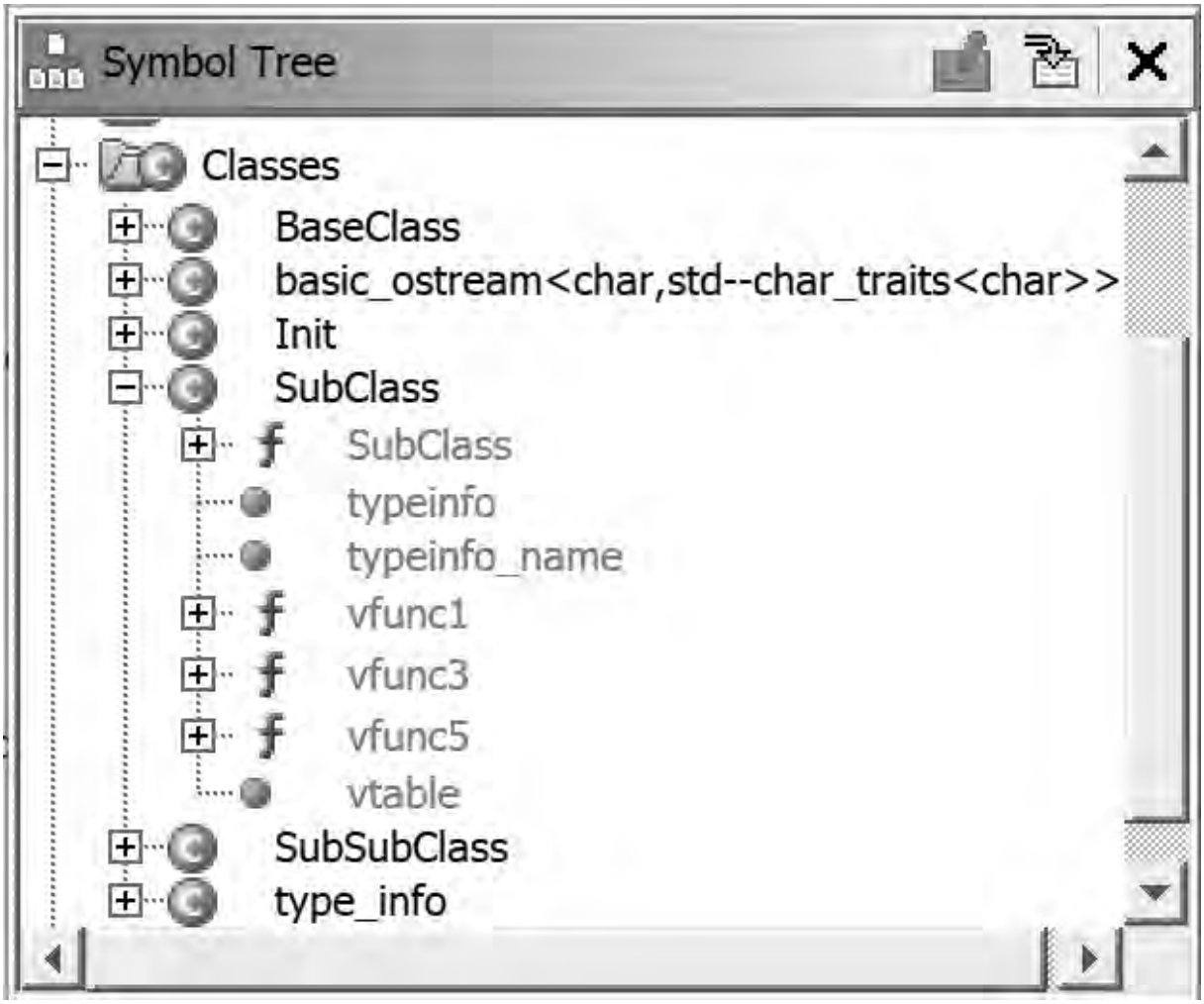


Рисунок 20-3. `Classes` *в* `Symbol Tree` *для бинарного файла с неудаленными символами*

Папка `Classes` содержит записи для всех трех наших классов. Развернутая запись SubClass раскрывает дополнительную информацию, обнаруженную Ghidra. Версия того же бинарного файла с удаленными символами содержит гораздо меньше информации, как показано на рисунке 20-4.

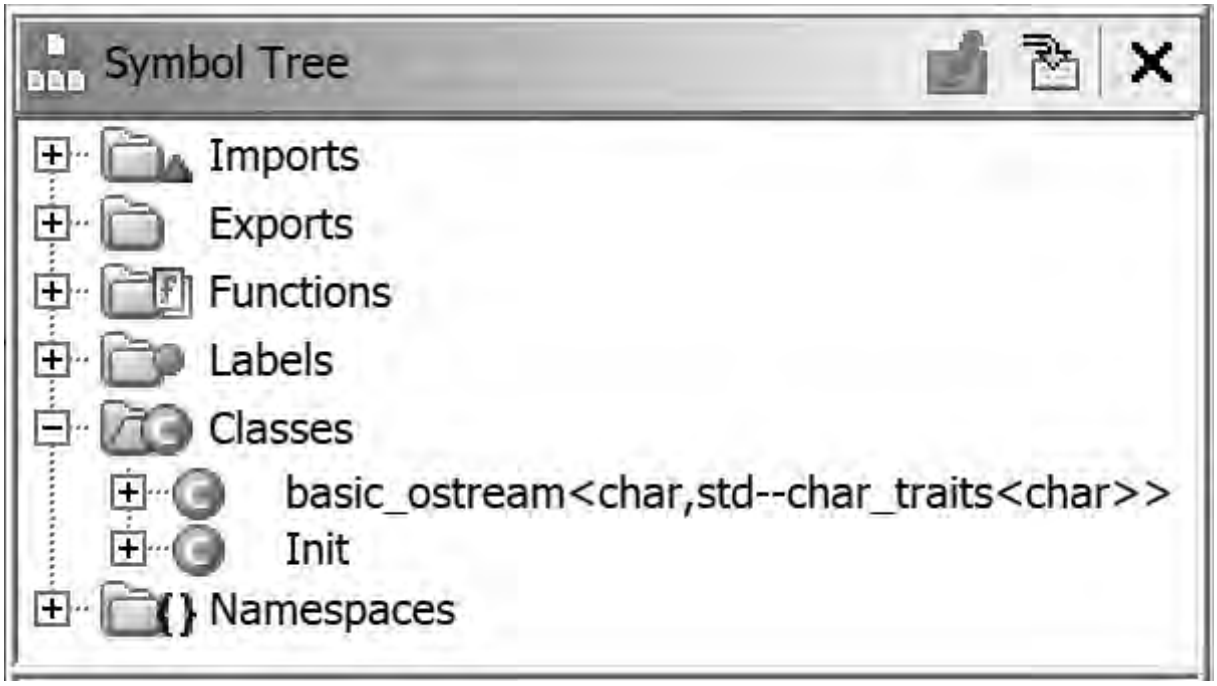


Рисунок 20-4. `Classes` *в* `Symbol Tree` *для бинарного файла с удаленными символами*

В этом случае мы могли бы ошибочно предположить, что бинарный файл не содержит интересующих C++-классов, хотя это, вероятно, C++-бинарный файл, судя по ссылке на базовый C++-класс (`basic_ostream`). Поскольку удаление символов удаляет только символьную информацию, мы все еще можем найти информацию RTTI, поискав имена классов в строках программы и пройдя назад к любой структуре данных RTTI. Поиск строк дает результаты, показанные на рисунке 20-5.

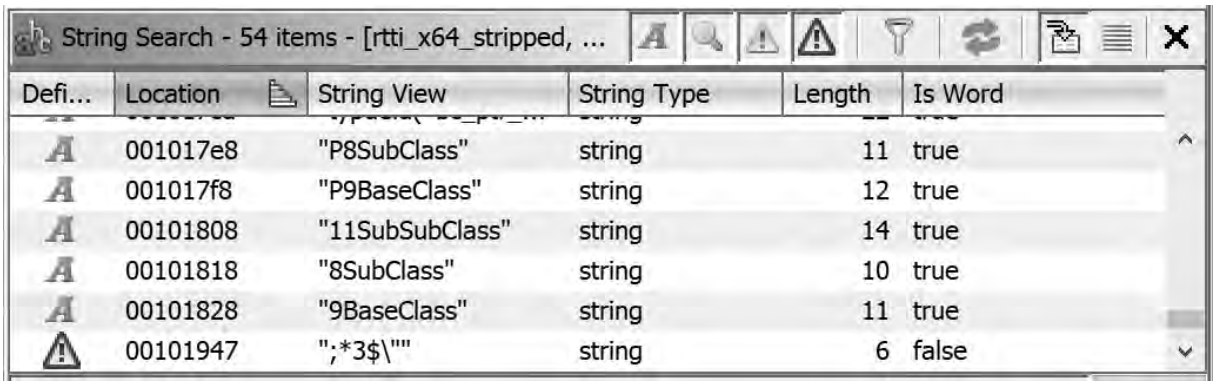


Рисунок 20-5. Результаты String Search, раскрывающие имена классов

Если щелкнуть строку `"8SubClass"`, мы попадем в эту часть окна `Listing`:

```
        s_8SubClass_00101818                XREF[1]:   00301d20(*)
00101818  ds "8SubClass"
```

В g++-бинарных файлах структуры, связанные с RTTI, содержат ссылки на соответствующую строку имени класса. Если пройти по перекрестной ссылке в первой строке к ее источнику, мы приходим к следующему фрагменту дизассемблированного листинга:

```
         PTR___gxx_personality_v0_00301d18  XREF[2]: FUN_00101241:00101316(*),
                                                     00301d10(*)
00301d18  addr   __gxx_personality_v0      = ??
00301d20  addr   s_8SubClass_00101818      = "8SubClass"
00301d28  addr   PTR_time_00301d30         = 00303028
```

Источник перекрестной ссылки - второе поле внутри структуры `typeinfo` класса `SubClass`, которая начинается по адресу `00301d18`. К сожалению, если вы не готовы погружаться в исходный код g++, такие раскладки структур приходится изучать по опыту. Последняя оставшаяся задача - найти `vftable` класса `SubClass`. В этом примере, если пройти по единственной перекрестной ссылке на структуру `typeinfo`, которая исходит из области данных (другая перекрестная ссылка исходит из функции и никак не может быть `vftable`), мы попадаем в тупик. Небольшая арифметика показывает, что перекрестная ссылка исходит из расположения непосредственно перед структурой `typeinfo` (`00301d18 - 8 == 00301d10`). В обычных обстоятельствах перекрестная ссылка существовала бы от `vftable` к структуре `typeinfo`; однако из-за отсутствия символов Ghidra не создает эту ссылку. Поскольку мы знаем, что где-то должен существовать еще один указатель на нашу структуру `typeinfo`, можно попросить Ghidra помочь. Когда курсор расположен в начале структуры, можно использовать пункт меню Search -> For Direct References, который просит Ghidra найти для нас текущий адрес в памяти. Результаты показаны на рисунке 20-6.

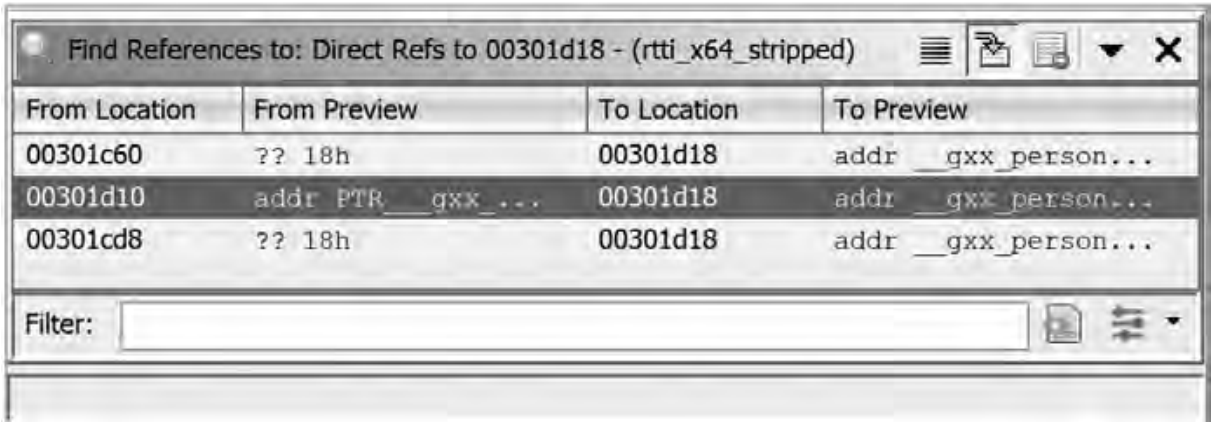


Рисунок 20-6. Результаты поиска прямых ссылок

Ghidra нашла две дополнительные ссылки на эту структуру `typeinfo`. Исследование каждой из них наконец приводит нас к `vftable`:

```
00301c60       ??      18h                ?v -> 00301d18
00301c61       ??      1Dh
00301c62       ??      30h                0
00301c63       ??      00h
00301c64       ??      00h
00301c65       ??      00h
00301c66       ??      00h
00301c67       ??      00h
        PTR_FUN_00301c68                XREF[2]: FUN_00101098:001010b0(*),
                                                 FUN_00101098:001010bb(*)
00301c68  addr   FUN_001010ea
00301c70  addr   FUN_00100ff0
00301c78  addr   FUN_00101122
00301c80  addr   FUN_00101060
00301c88  addr   FUN_0010115a
```

Ghidra не отформатировала источник перекрестной ссылки на `typeinfo` как указатель (что объясняет отсутствие перекрестной ссылки), но она предоставляет комментарий в конце строки, намекающий, что это указатель. Сама `vftable` начинается на 8 байтов позже и содержит пять

указателей на виртуальные функции, принадлежащие `SubClass`. Таблица не содержит искаженных имен, потому что из бинарного файла были удалены символы.

В следующем разделе мы применим этот прием анализа "следовать по хлебным крошкам", чтобы помочь идентифицировать функцию `main` в C-бинарных файлах, сгенерированных несколькими компиляторами.

Поиск функции main

С точки зрения программиста выполнение программы обычно начинается с функции `main`, поэтому начинать анализ бинарного файла с `main` - неплохая стратегия. Однако компиляторы и компоновщики (а также использование библиотек) добавляют код, который выполняется до того, как управление достигнет `main`. Поэтому часто неточно предполагать, что точка входа бинарного файла соответствует функции `main`, написанной автором программы. Фактически представление, что у всех программ есть функция `main`, - это соглашение C/C++-компиляторов, а не жесткое правило написания программ. Если вы когда-либо писали Windows GUI-приложение, вам может быть знаком вариант `WinMain` вместо `main`. Как только вы отходите от C/C++, можно обнаружить, что другие языки используют другие имена для своей основной функции точки входа. Мы будем обобщенно называть эту функцию функцией `main`.

Если в бинарном файле есть символ с именем `main`, можно просто попросить Ghidra перенести вас туда, но если вы анализируете бинарный файл с удаленными символами, вы окажетесь у заголовка файла и будете искать `main` самостоятельно. При небольшом понимании того, как работают исполняемые файлы, и некотором опыте это не должно оказаться слишком сложной задачей.

Все исполняемые файлы должны назначить адрес внутри бинарного файла как первую инструкцию, которую нужно выполнить после отображения файла в память. Ghidra называет этот адрес `entry` или `_start`, в зависимости от типа файла и доступности символов. Большинство форматов исполняемых файлов указывает этот адрес в области заголовка файла, и загрузчики Ghidra точно знают, как его найти. В ELF-файле адрес точки входа указан в поле с именем `e_entry`, тогда как PE-файлы содержат поле с именем `AddressOfEntryPoint`. Скомпилированная C-программа, независимо от платформы, на которой выполняется исполняемый файл, имеет в точке входа код, вставленный компилятором, чтобы выполнить переход от совершенно нового процесса к работающей C-программе. Часть этого перехода состоит в том, чтобы гарантировать, что аргументы и переменные окружения, предоставленные ядру при создании процесса, собраны и переданы в `main` с использованием соглашения о вызовах C.

Примечание. Ядро вашей операционной системы не знает и не интересуется тем, на каком языке был написан исполняемый файл. Ядро знает ровно один способ передавать параметры новому процессу, и этот способ может быть несовместим с функцией входа вашей программы. Задача компилятора - преодолеть этот разрыв.

Теперь, когда мы знаем, что выполнение начинается с опубликованной точки входа и в итоге достигает функции `main`, посмотрим на компиляторно-специфичный код, выполняющий этот переход.

Пример 1: от _start к main с gcc в Linux x86-64

Исследуя стартовый код в исполняемом файле с неудаленными символами, мы можем узнать, как именно достигается `main` для данного компилятора в данной операционной системе. Linux `gcc` предлагает один из более простых подходов:

```
      _start
004003b0  XOR    EBP,EBP
004003b2  MOV    R9,RDX
004003b5  POP    RSI
004003b6  MOV    RDX,RSP
004003b9  AND    RSP,-0x10
004003bd  PUSH   RAX
004003be  PUSH   RSP=>local_10
004003bf  MOV    R8=>__libc_csu_fini,__libc_csu_fini
004003c6  MOV    RCX=>__libc_csu_init,__libc_csu_init
004003cd  MOV    RDI=>main,main
004003d4  CALL   qword ptr [->__libc_start_main]
```

Адрес `main` загружается в `RDI` непосредственно перед вызовом библиотечной функции с именем `__libc_start_main`, что означает, что адрес `main` передается как первый аргумент в `__libc_start_main`. Вооружившись этим знанием, мы можем легко найти `main` в бинарном файле с удаленными символами. Следующий листинг показывает подготовку к вызову `__libc_start_main` в бинарном файле с удаленными символами:

```
004003bf  MOV    R8=>FUN_004008a0,FUN_004008a0
004003c6  MOV    RCX=>FUN_00400830,FUN_00400830
004003cd  MOV    RDI=>FUN_0040080a,FUN_0040080a
004003d4  CALL   qword ptr [->__libc_start_main]
```

Хотя код содержит ссылки на три обобщенно названные функции, мы заключаем, что `FUN_0040080a` должна быть `main`, потому что она передается как первый аргумент в `__libc_start_main`.

Пример 2: от _start к main с clang во FreeBSD x86-64

В текущих версиях FreeBSD `clang` является C-компилятором по умолчанию, а функция `_start` несколько более объемна и сложнее для отслеживания, чем простой Linux-заглушка `_start`. Чтобы упростить задачу, мы используем декомпилятор Ghidra для просмотра хвостовой части `_start`.

```
    //~40 строк кода опущено для краткости
    atexit((__func *)cleanup);
    handle_static_init(argc,ap,env);
    argc = main((ulong)pcVar2 & 0xffffffff,ap,env);
                    /* WARNING: Subroutine does not return */
    exit(argc);
}
```

В этом случае `main` - предпоследняя функция, вызываемая в `_start`, а возвращаемое значение из `main` немедленно передается в `exit`, чтобы завершить программу. Использование декомпилятора Ghidra на версии того же бинарного файла с удаленными символами дает следующий листинг:

```
    // 40 строк кода опущено для краткости
    atexit(param_2);
    FUN_00201120(uVar2 & 0xffffffff,ppcVar5,puVar4);
    __status = FUN_00201a80(uVar2 & 0xffffffff,ppcVar5,puVar4);
                    /* WARNING: Subroutine does not return */
    exit(__status);
}
```

И снова мы можем выделить `main` среди остальных даже тогда, когда из бинарного файла были удалены символы. Если вам интересно, почему этот листинг показывает два имени функций, которые не были удалены при удалении символов, причина в том, что этот конкретный бинарный файл скомпонован динамически. Функции `atexit` и `exit` не являются символами в бинарном файле; это внешние зависимости. Эти внешние зависимости остаются даже после удаления символов и продолжают быть видимыми в декомпилированном коде. Соответствующий код для статически скомпонованной версии этого бинарного файла с удаленными символами показан здесь:

```
    FUN_0021cc70();
    FUN_0021c120(uVar2 & 0xffffffff,ppcVar13,puVar11);
    uVar7 = FUN_0021caa0(uVar2 & 0xffffffff,ppcVar13,puVar11);
                    /* WARNING: Subroutine does not return */
    FUN_00266d30((ulong)uVar7);
}
```

Пример 3: от _start к main с компилятором Microsoft C/C++

Стартовая заглушка компилятора Microsoft C/C++ немного сложнее, потому что основной интерфейс к ядру Windows идет через `kernel32.dll` (а не через `libc`, как в большинстве Unix-систем), которая не предоставляет функций C-библиотеки. В результате компилятор часто статически компонует многие функции C-библиотеки прямо в исполняемые файлы. Стартовая заглушка использует эти и другие функции для взаимодействия с ядром, чтобы настроить среду выполнения вашей C-программы.

Однако в конечном счете стартовая заглушка все равно должна вызвать `main`, а после ее возврата - `exit`. Поиск `main` среди всего стартового кода обычно сводится к выявлению функции с тремя аргументами (`main`), возвращаемое значение которой передается в функцию с одним аргументом (`exit`). Следующий фрагмент из бинарного файла такого типа содержит вызовы двух функций, которые мы ищем:

```
140001272 CALL   _amsg_exit
140001277 MOV    R8,qword ptr [DAT_14000d310]
14000127e MOV    qword ptr [DAT_14000d318],R8
140001285 MOV    RDX,qword ptr [DAT_14000d300]
14000128c MOV    ECX,dword ptr [DAT_14000d2fc]
140001292 CALL   FUN_140001060
140001297 MOV    EDI,EAX
140001299 MOV    dword ptr [RSP + Stack[-0x18]],EAX
14000129d TEST   EBX,EBX
14000129f JNZ    LAB_1400012a8
1400012a1 MOV    ECX,EAX
1400012a3 CALL   FUN_140002b30
```

Здесь `FUN_140001060` - функция с тремя аргументами, которая оказывается `main`, а `FUN_140002b30` - одноаргументная `exit`. Обратите внимание, что Ghidra смогла восстановить имя одной из статически скомпонованных функций, вызываемых стартовой заглушкой, потому что функция совпадает с записью FidDb. Мы можем использовать подсказки, предоставляемые любыми идентифицированными символами, чтобы сэкономить время при поиске `main`.

Резюме

Сам объем компиляторно-специфичного поведения слишком велик, чтобы охватить его в одной главе (или даже в одной книге, если на то пошло). Среди прочего компиляторы различаются алгоритмами, которые они выбирают для реализации различных высокоуровневых конструкций, и способом оптимизации сгенерированного кода. Поскольку на поведение компилятора сильно влияют аргументы, переданные компилятору в процессе сборки, один компилятор может генерировать радикально разные бинарные файлы из одного и того же исходного кода при выборе разных параметров сборки.

К сожалению, справляться со всеми этими вариациями получается только с опытом, и часто очень трудно искать помощь по конкретным конструкциям языка ассемблера, поскольку очень трудно составить поисковые выражения, которые дадут результаты, применимые именно к вашему случаю. Когда это происходит, лучшим ресурсом обычно оказывается форум, посвященный реверс-инжинирингу, где можно опубликовать код и воспользоваться знаниями других людей, сталкивавшихся с похожими ситуациями.

Сноски

[1] [назад] Для любителей анализа алгоритмов: использование табличного поиска позволяет найти целевой вариант за постоянное число операций независимо от размера пространства поиска, что, как вы, возможно, помните из курса алгоритмов, также называется постоянным временем, или O(1).
[2] [назад] Алгоритмы линейного времени имеют сложность O(n) и, к счастью, не используются в операторах `switch`.
[3] [назад] Двоичный поиск имеет сложность O(log n).
[4] [назад] Для тех, кто анализирует алгоритмы дома: это означает, что переменная `switch` сопоставляется максимум после log2N сравнений, где N - число вариантов, содержащихся в операторе `switch`. Это O(log n).
[5] [назад] Хотя сложность сортировки очень высока по сравнению со сложностью поиска, важно отметить, что сортировка происходит только один раз во время компиляции, тогда как поиск выполняется каждый раз, когда оператор `switch` используется во время выполнения.
[6] [назад] `gcc` принимает большое число аргументов командной строки, и каждый из них может влиять на результирующий сгенерированный код. В качестве отправной точки для компиляции примера использовалась следующая команда `gcc`: `gcc switch_demo_1.c -m32 -fno-pie -fno-pic -fno-stack-protector -o switch_demo_1_x86`. Пример Microsoft C/C++ - неизмененная отладочная x86-сборка. Дополнительные параметры вводятся в последующих примерах.
[7] [назад] Другие компиляторы, такие как `gcc`, также предлагают возможность вставлять отладочные символы в процессе компиляции.
[8] [назад] Оптимизация обычно включает устранение избыточности в коде или выбор более быстрых, но потенциально более крупных последовательностей кода, чтобы удовлетворить желание разработчика создать более быстрые или более маленькие исполняемые файлы. Оптимизированный код может быть не так прямолинеен для анализа, как неоптимизированный, и поэтому может считаться плохим выбором для этапов разработки и отладки программы.
[9] [назад] См. https://docs.microsoft.com/en-us/cpp/build/reference/rtc-run-time-error-checks.
[10] [назад] Инструкция `NEG` очищает флаг переноса (`CF`), когда ее операнд равен нулю, и устанавливает флаг переноса во всех остальных случаях.
[11] [назад] См. https://docs.microsoft.com/en-us/cpp/build/reference/decorated-names для Microsoft и https://itanium-cxx-abi.github.io/cxx-abi/abi.html#mangling для Intel.

Глава 21. Анализ обфусцированного кода

Даже при идеальных обстоятельствах понимание дизассемблированного листинга - трудная задача. Высококачественные дизассемблирования необходимы каждому, кто пытается понять внутреннюю работу бинарного файла, и именно поэтому последние 20 глав мы обсуждали Ghidra и связанные с ней возможности. Можно утверждать, что Ghidra настолько эффективна в том, что делает, что снизила порог входа в область анализа бинарных файлов. Хотя, конечно, это нельзя приписывать одной только Ghidra, недавние достижения в реверс-инжиниринге бинарных файлов не остаются незамеченными теми, кто не хочет, чтобы их ПО анализировали. Поэтому за последние несколько лет между программистами, желающими сохранить свой код в секрете, и реверс-инженерами происходит своего рода гонка вооружений.

В этой главе мы рассмотрим роль Ghidra в этой гонке вооружений и обсудим некоторые меры, предпринятые для защиты кода, а также подходы к преодолению этих мер. В конце главы мы представим класс `Emulator` Ghidra и приведем примеры того, как сценарии эмуляции могут дать нам преимущество в этой гонке.

Анти-реверс-инжиниринг

Анти-реверс-инжиниринг - зонтичная тема, охватывающая все приемы, которые разработчики ПО могут применять, чтобы усложнить реверс-инжиниринг своих продуктов. Существует много инструментов и приемов, помогающих разработчикам достичь этой цели, и каждый день появляются новые. Экосистема RE/anti-RE похожа на нарастающую динамику, разворачивающуюся между авторами вредоносного ПО и поставщиками антивирусов.

Как реверс-инженер вы, скорее всего, столкнетесь с приемами, диапазон которых простирается от тривиальных до почти невозможных для преодоления. Подходы, которые вам потребуется использовать, также будут различаться в зависимости от природы приемов противодействия реверс-инжинирингу, с которыми вы столкнулись, и могут требовать некоторой уверенности как в статическом, так и в динамическом анализе. В следующих разделах мы обсудим некоторые из более распространенных приемов противодействия реверс-инжинирингу, почему они применяются и как их преодолевать.

Обфускация

Различные словарные определения скажут вам, что обфускация - это действие, делающее нечто неясным, запутанным, сбивающим с толку или озадачивающим, чтобы помешать другим понять обфусцируемый объект. В контексте этой книги и использования Ghidra обфусцируемые объекты - это бинарные исполняемые файлы (а не исходные файлы или кремниевые кристаллы, например).

Обфускация сама по себе слишком широка, чтобы считаться приемом анти-реверс-инжиниринга. Кроме того, она не охватывает все известные приемы анти-реверс-инжиниринга. Конкретные отдельные приемы часто можно описать как обфусцирующие или необфусцирующие, и там, где это применимо, мы отмечаем это в следующих разделах. Важно понимать, что единственного правильного способа классифицировать приемы нет, поскольку общие категории часто пересекаются в своих описаниях. Кроме того, новые приемы анти-реверс-инжиниринга постоянно разрабатываются, и невозможно дать единый всеобъемлющий список.

Поскольку Ghidra прежде всего является инструментом статического анализа, нам полезнее разделить обсуждение приемов на две широкие категории: противодействие статическому анализу и противодействие динамическому анализу. Обе категории могут содержать обфусцирующие приемы, но первая с большей вероятностью сбивает с толку статические инструменты, а вторая обычно нацелена на отладчики и другие инструменты анализа времени выполнения.

Приемы противодействия статическому анализу

Приемы противодействия статическому анализу стремятся помешать аналитику понять природу программы без фактического запуска программы. Это ровно те приемы, которые нацелены на дизассемблеры вроде Ghidra, и потому они представляют наибольший интерес каждый раз, когда вы используете Ghidra для реверс-инжиниринга бинарных файлов. Далее обсуждаются несколько типов приемов противодействия статическому анализу.

Десинхронизация дизассемблирования

Один из более старых приемов, предназначенных для затруднения процесса дизассемблирования, предполагает творческое использование инструкций и данных, чтобы помешать дизассемблеру найти правильный начальный адрес для одной или нескольких инструкций. Принуждение дизассемблера потерять синхронизацию обычно приводит к неудачному или, как минимум, неправильному дизассемблированному листингу. Листинг 21-1 показывает попытку Ghidra дизассемблировать часть инструмента анти-реверс-инжиниринга Shiva.[1]

```
0a04b0d1 e8 01 00 00 00 CALL   FUN_0a04b0d7
0a04b0d6 c7             ??     C7h
     ************************************************************
     *                          FUNCTION                        *
     ************************************************************
     undefined FUN_0a04b0d7()
        undefined AL:1 <RETURN>
     FUN_0a04b0d7                            XREF[1]: FUN_0a04b0c4:0a04b0d1(c)
0a04b0d7 58             POP    EAX
0a04b0d8 8d 40 0a       LEA    EAX,[EAX + 0xa]
        LAB_0a04b0db+1                       XREF[0,1]: 0a04b0db(j)
0a04b0db eb ff          JMP    LAB_0a04b0db+1
0a04b0dd e0             ??     E0h
```

Листинг 21-1. Фрагмент начального дизассемблирования Shiva

Этот пример выполняет `CALL`, сразу после которого следует `POP`. Такая последовательность нередко встречается в самомодифицирующемся коде и используется кодом, чтобы выяснить, где он выполняется в памяти. Возвращаемый адрес для инструкции `CALL` равен `0a04b0d6` и находится на вершине стека, когда выполнение приходит к инструкции `POP`. Инструкция `POP` удаляет возвращаемый адрес из стека и загружает его в `EAX`, а следующая `LEA` немедленно добавляет к `EAX` значение `0xa` (10), так что `EAX` теперь содержит `0a04b0e0` (держите это значение под рукой, оно понадобится через несколько мгновений).

Вызванная функция вряд ли когда-либо вернется в исходную точку вызова, поскольку исходный возвращаемый адрес больше не находится на вершине стека (его пришлось бы заменить, чтобы `RET` попал в исходное место возврата), а Ghidra не может сформировать инструкцию по возвращаемому адресу, потому что `C7h` не является допустимым начальным байтом инструкции.

Пока что код может быть немного необычным или трудным для отслеживания, но Ghidra показывает правильное дизассемблирование. Все меняется, когда достигается инструкция `JMP`. Эта

инструкция перехода имеет длину 2 байта, ее адрес - `0a04b0db`, а цель перехода - `LAB_0a04b0db+1`. Суффикс `+1` в метке для нас новый. Адресная часть метки такая же, как адрес самого перехода. `+1` сообщает, что цель перехода находится на 1 байт после `LAB_0a04b0db`. Иными словами, переход попадает прямо в середину 2-байтовой инструкции перехода. Хотя процессор не волнует эта необычная ситуация (он спокойно выбирает то, на что указывает указатель инструкции), Ghidra не может заставить это работать. У Ghidra нет способа одновременно отображать байт по адресу `0a04b0db` (`ff`) и как второй байт перехода, и как первый байт другой инструкции. В результате Ghidra внезапно не может продолжить дизассемблирование, на что указывает неопределенное значение данных по адресу `0a04b0dd`. (Такое поведение не ограничивается Ghidra: практически все дизассемблеры, использующие как алгоритм рекурсивного спуска, так и алгоритм линейного прохода, становятся жертвами этого приема.)

Ghidra отмечает любые проблемы, встреченные во время дизассемблирования, создавая закладки ошибок в дизассемблировании. Рисунок 21-1 показывает две такие закладки (значок X слева от проблемных адресов) в левом поле окна `Listing`. Наведение курсора на закладку ошибки отображает связанное подробное сообщение. Кроме того, можно открыть список всех закладок в текущем бинарном файле, используя `Window -> Bookmarks`.

Сообщение Ghidra для первой ошибки: `Unable to resolve constructor at 0a04b0d6 (flow from 0a04b0d1)`, что примерно означает: "Я думаю, что по адресу 0a04b0d6 должна существовать инструкция, но я не смогла ее создать". Сообщение Ghidra для второй ошибки: `Failed to disassemble at 0a04b0dc due to conflicting instruction at 0a04b0db (flow from 0a04b0db)`, что примерно означает: "Я не могу дизассемблировать инструкцию внутри существующей инструкции".

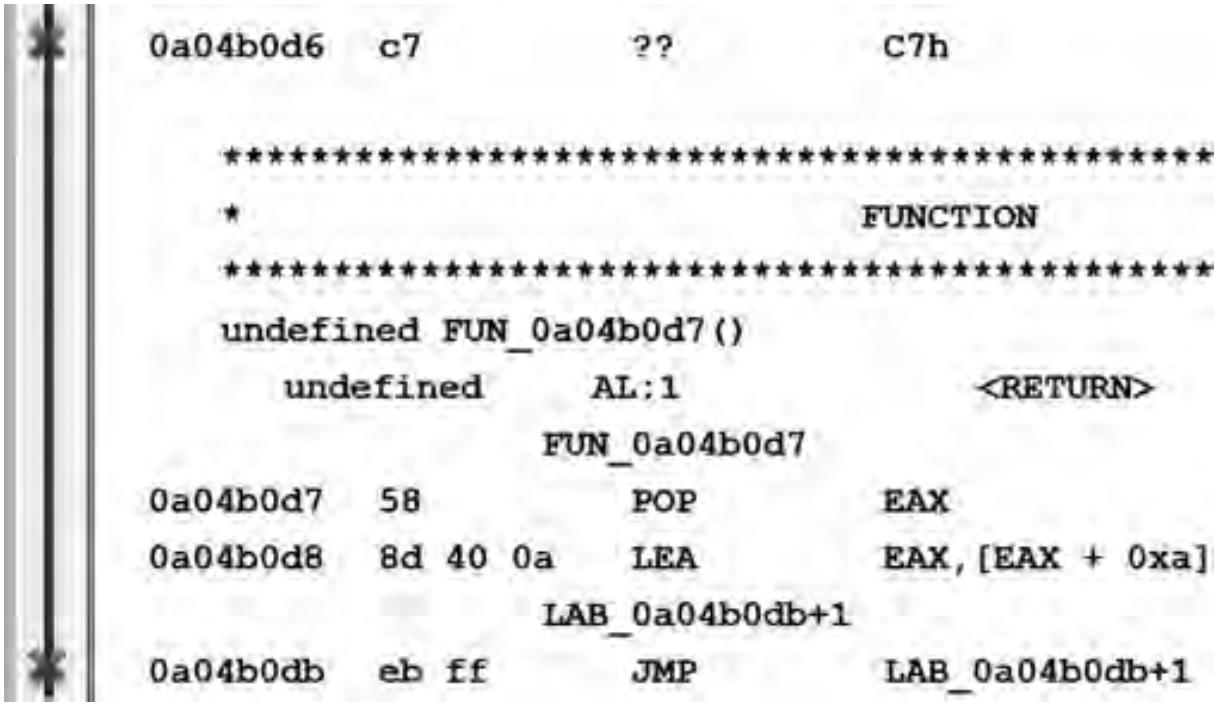


Рисунок 21-1. Закладки ошибок Ghidra

Как пользователь Ghidra, вы не имеете решения для первой ошибки. Последовательность байтов либо является допустимой инструкцией, либо нет. С некоторым усилием с вашей стороны можно разобраться со второй ошибкой. Правильный способ справиться с такой ситуацией - снять определение инструкции, содержащей байты, которые являются целью вызова, а затем определить инструкцию по целевому адресу вызова, пытаясь повторно синхронизировать дизассемблирование. Вы потеряете исходную инструкцию, но можете оставить себе комментарий, чтобы помнить, какой была исходная инструкция. Следующая часть предыдущего листинга содержит ошибку перекрывающейся инструкции:

```
         LAB_0a04b0db+1                          XREF[0,1]:   0a04b0db(j)
0a04b0db eb ff     JMP    LAB_0a04b0db+1
0a04b0dd e0        ??     E0h
```

Щелчок правой кнопкой по инструкции `JMP` и выбор `Clear Code Bytes` (горячая клавиша C) в контекстном меню дает следующий листинг неопределенных байтов:

```
0a04b0db eb        ??     EBh
0a04b0dc ff        ??     FFh
0a04b0dd e0        ??     E0h
```

Байт, который является целью `JMP`, теперь доступен для переформатирования. Сырые байты превращаются в код щелчком правой кнопкой по начальному байту инструкции и выбором Disassemble (горячая клавиша D). Теперь листинг обновлен до следующего вида:

```
0a04b0dc ff e0     JMP    EAX
0a04b0de 90        ??     90h
0a04b0df c7        ??     C7h
```

Целевая инструкция перехода оказывается еще одной инструкцией перехода. В этом случае, однако, переход невозможно отследить дизассемблеру (и потенциально трудно отследить человеку-аналитику), потому что цель перехода содержится в регистре (`EAX`) и вычисляется во время выполнения. Это пример еще одного типа приема противодействия статическому анализу, обсуждаемого в следующем разделе, "Динамически вычисляемые целевые адреса". Мы ранее определили, что к моменту достижения этого перехода `EAX` содержит значение `0a04b0e0`, и именно по этому адресу нужно возобновить процесс дизассемблирования. Намылить, смыть, повторить.

Возвращаясь к листингу 21-1: вместо ручного перехода к адресу `0a04b0e0`, чтобы возобновить дизассемблирование, можно установить значение `EAX` в известное значение, щелкнув адрес правой кнопкой и выбрав Set Register Values. Затем Ghidra добавит вокруг инструкции специальную разметку, называемую переходом регистра, чтобы указать предполагаемое значение цели `JMP`, `EAX`. Последующая очистка (горячая клавиша C) и дизассемблирование (горячая клавиша D) из этого места перезапускают процесс дизассемблирования рекурсивным спуском от `JMP` к цели, `0a04b0e0`, и далее (включая создание XREF между этими блоками кода).

Преимущество этого подхода в том, что код снабжен аннотацией, показывающей цель `JMP`, что позволяет другим аналитикам легко проследить фактический поток управления через этот участок. (Это еще яснее в сочетании с переопределением перехода по fallthrough для инструкции `LEA` по адресу `0a04b0d8` в листинге 21-1.) Этот альтернативный подход приводит к следующему листингу:

```
0a04b0d7 58        POP    EAX
0a04b0d8 8d 40 0a  LEA    EAX,[EAX + 0xa]
                      -- Fallthrough Override: 0a04b0dc
0a04b0db eb        ??     EBh
             assume EAX = 0xa04b0e0
         LAB_0a04b0dc                            XREF[1]:     0a04b0d8
0a04b0dc ff e0     JMP    EAX=>LAB_0a04b0e0
             assume EAX = <UNKNOWN>
0a04b0de 90        ??     90h
0a04b0df c7        ??     C7h
         LAB_0a04b0e0                            XREF[1]:     0a04b0dc(j)
0a04b0e0 58        POP    EAX 0a04b0e0 POP EAX
```

Другой пример десинхронизации, взятый из другого бинарного файла, демонстрирует, как флаги процессора можно использовать для превращения условных переходов в безусловные. Следующее дизассемблирование демонстрирует использование флага x86 `Z` именно для этой цели:

```
00401000  XOR    EAX,EAX
00401002  JZ     LAB_00401009+1
00401004  MOV    EBX,dword ptr [EAX]
00401006  MOV    dword ptr [param_1 + -0x4],EBX
        LAB_00401009+1                      XREF[0,1]: 00401002(j)
00401009  CALL   SUB_adfeffc6
0040100e  FICOM  word ptr [EAX + 0x59]
```

Здесь инструкция `XOR` используется, чтобы обнулить регистр `EAX` и установить флаг x86 `Z`. Программист, зная, что флаг `Z` установлен, использует инструкцию перехода при нуле (`JZ`), которая всегда будет выполнена, чтобы получить эффект безусловного перехода. В результате инструкции между переходом и целью перехода никогда не будут выполнены и служат только для того, чтобы запутать любого аналитика, который не осознает этот факт. Этот пример также скрывает фактическую цель перехода, выполняя переход в середину инструкции `CALL` по адресу `00401009`. При правильном дизассемблировании код должен выглядеть следующим образом:

```
00401000  XOR    EAX,EAX
00401002  JZ     LAB_0040100a
00401004  MOV    EBX,dword ptr [EAX]
00401006  MOV    dword ptr [param_1 + -0x4],EBX
00401009  ??    E8h
        LAB_0040100a                        XREF[1]: 00401002(j)
0040100a  MOV    EAX,0xdeadbeef
0040100f  PUSH   EAX
00401010  POP    param_1
```

Фактическая цель перехода раскрыта, как и лишний байт, который изначально вызвал десинхронизацию. Конечно, можно использовать гораздо более обходные способы установки и проверки флагов перед выполнением условного перехода. Уровень сложности анализа такого кода растет вместе с числом операций, которые могут повлиять на биты флагов процессора перед проверкой их значения.

Динамически вычисляемые целевые адреса

Фраза "динамически вычисляемый" просто означает, что адрес, к которому перейдет выполнение, вычисляется во время выполнения. В этом разделе мы обсудим несколько способов, которыми может быть получен такой адрес. Цель таких приемов - скрыть (обфусцировать) фактический путь потока управления, которому будет следовать бинарный файл, от любопытных глаз процесса статического анализа.

Один пример этого приема был показан в предыдущем разделе. В примере использовалась инструкция вызова, чтобы поместить возвращаемый адрес в стек. Возвращаемый адрес извлекался прямо из стека в регистр, а к регистру добавлялось постоянное значение, чтобы получить конечный целевой адрес, который в итоге достигался путем выполнения перехода к месту, заданному содержимым регистра.

Можно разработать бесконечное число похожих последовательностей кода для получения целевого адреса и передачи управления по этому адресу. Следующий код, также использованный в Shiva, демонстрирует альтернативный метод динамического вычисления целевых адресов:

```
0a04b3be  MOV    ECX,0x7f131760             ; ECX = 7F131760
0a04b3c3  XOR    EDI,EDI                    ; EDI = 00000000
0a04b3c5  MOV    DI,0x1156                  ; EDI = 00001156
0a04b3c9  ADD    EDI,0x133ac000             ; EDI = 133AD156
0a04b3cf  XOR    ECX,EDI                    ; ECX = 6C29C636
0a04b3d1  SUB    ECX,0x622545ce             ; ECX = 0A048068
0a04b3d7  MOV    EDI,ECX                    ; EDI = 0A048068
0a04b3d9  POP    EAX
0a04b3da  POP    ESI
0a04b3db  POP    EBX
0a04b3dc  POP    EDX
0a04b3dd  POP    ECX
0a04b3de  XCHG   dword ptr [ESP],EDI        ; TOS =   0A048068
0a04b3e1  RET                               ; возврат к 0A048068
```

Комментарии справа от точек с запятой документируют изменения, выполняемые с различными регистрами процессора при каждой инструкции. Процесс завершается перемещением полученного значения в верхнюю позицию стека (`TOS`), что заставляет инструкцию возврата передать управление в вычисленное место (`0A048068` в этом случае). Аналитику, по сути, приходится выполнить код вручную, чтобы определить фактический путь потока управления, пройденный в программе.

Обфусцированный поток управления

За последние годы были разработаны и использованы гораздо более сложные методы скрытия потока управления. В самых сложных случаях программа будет использовать несколько потоков или дочерних процессов, чтобы вычислять информацию о потоке управления и получать эту информацию через некоторую форму межпроцессного взаимодействия (для дочерних процессов) или примитивы синхронизации (для нескольких потоков).

В таких случаях статический анализ может стать чрезвычайно трудным, потому что необходимо понимать не только поведение нескольких исполняемых сущностей, но и точный способ, которым эти сущности обмениваются информацией. Например, один поток может ждать на общем объекте семафора, тогда как второй поток вычисляет значения или модифицирует код, который первый поток будет использовать после того, как второй поток просигнализирует о завершении через семафор.[2]

Еще один прием, часто используемый во вредоносном ПО для Windows, включает настройку обработчика исключений,[3] намеренное возбуждение исключения, а затем манипулирование состоянием регистров процесса во время обработки исключения. Следующий пример используется инструментом анти-реверс-инжиниринга `tElock`, чтобы скрыть фактический поток управления программы:

```
0041d07a  CALL   LAB_0041d07f
        LAB_0041d07f                        XREF[1]: 0041d07a(j)
0041d07f  POP    EBP
0041d080  LEA    EAX,[EBP + 0x46]
0041d083  PUSH   EAX
0041d084  XOR    EAX,EAX
0041d086  PUSH   dword ptr FS:[EAX]
0041d089  MOV    dword ptr FS:[EAX],ESP
0041d08c  INT    3
0041d08d  NOP
0041d08e  MOV    EAX,EAX
0041d090  STC
0041d091  NOP
```

```
0041d092  LEA    EAX,[EBX*0x2 + 0x1234]
0041d099  CLC
0041d09a  NOP
0041d09b  SHR    EBX,0x5
0041d09e  CLD
0041d09f  NOP
0041d0a0  ROL    EAX,0x7
0041d0a3  NOP
0041d0a4  NOP
0041d0a5  XOR    EBX,EBX
0041d0a7  DIV    EBX
0041d0a9  POP    dword ptr FS:[0x0]
```

Последовательность начинается с использования `CALL` к следующей инструкции; инструкция `CALL` помещает `0041d07f` в стек как возвращаемый адрес, который сразу извлекается из стека в `EBP`. Затем регистр `EAX` устанавливается в сумму `EBP` и `46h`, то есть `0041d0c5`, и этот адрес помещается в стек как адрес функции обработчика исключений. Остальная настройка обработчика исключений происходит путем связывания нового обработчика исключений с существующей цепочкой обработчиков исключений, на которую ссылается `FS:[0]`.[4]

Следующий шаг - намеренно сгенерировать исключение, в данном случае `INT 3`, которое является программной ловушкой (прерыванием) для отладчика. (В x86-программах инструкция `INT 3` используется отладчиками для реализации программной точки останова.) Обычно в этот момент подключенный отладчик получил бы управление, потому что отладчикам предоставляется первая возможность обработать исключение.

В этом случае программа полностью ожидает обработать исключение, поэтому любой подключенный отладчик должен быть настроен передать исключение программе. Если не позволить программе обработать исключение, программа может работать неправильно или аварийно завершиться. Без понимания того, как обрабатывается исключение `INT 3`, невозможно знать, что может произойти дальше в этой программе. Если предположить, что выполнение просто возобновится после `INT 3`, то кажется, что инструкции `XOR EBX,EBX` и `DIV EBX` в итоге вызовут исключение деления на ноль.

Декомпилированная версия обработчика исключений, связанного с предыдущим кодом, начинается по адресу `0041d0c5`. Первая часть этой функции показана здесь:

```
int FUN_0041d0c5(EXCEPTION_RECORD *param_1,void *frame,CONTEXT *ctx) {
  DWORD code;
  ctx->Eip = ctx->Eip + 1;
  code = param_1->ExceptionCode;
  if (code == EXCEPTION_INT_DIVIDE_BY_ZERO) {
    ctx->Eip = ctx->Eip + 1;
    ctx->Dr0 = 0;
    ctx->Dr1 = 0;
    ctx->Dr2 = 0;
    ctx->Dr3 = 0;
    ctx->Dr6 = ctx->Dr6 & 0xffff0ff0;
    ctx->Dr7 = ctx->Dr7 & 0xdc00;
  }
```

Третий аргумент функции обработчика исключений - указатель на структуру Windows `CONTEXT` (определенную в заголовочном файле Windows API `winnt.h`). Структура `CONTEXT` инициализируется содержимым всех регистров процессора в том виде, в каком они существовали в момент исключения. Обработчик исключений имеет возможность просмотреть и, при желании, изменить содержимое структуры `CONTEXT`. Если обработчик исключений считает, что он исправил проблему, приведшую к исключению, он может уведомить операционную систему, что проблемному потоку следует разрешить продолжить работу. В этот момент операционная система перезагружает регистры процессора для потока из структуры `CONTEXT`, предоставленной обработчику исключений, и выполнение потока возобновляется так, будто ничего не произошло.

В предыдущем примере обработчик исключений начинает с доступа к CONTEXT потока, чтобы увеличить указатель инструкции и разрешить возобновление выполнения на инструкции, следующей за той, которая сгенерировала исключение. Затем извлекается код типа исключения (поле внутри предоставленной структуры EXCEPTION_RECORD), чтобы определить природу исключения. Эта часть обработчика исключений обрабатывает ошибку деления на ноль, сгенерированную в предыдущем примере, обнуляя все аппаратные отладочные регистры x86 и отключая аппаратные точки останова.[5]

Без изучения остального кода tElock не сразу очевидно, зачем очищаются отладочные регистры. В этом случае tElock очищает значения от предыдущей операции, в которой он использовал отладочные регистры, чтобы установить четыре точки останова в дополнение к INT 3, виденной ранее. Помимо обфускации истинного потока программы, очистка или изменение отладочных регистров x86 может сильно нарушить работу программных отладчиков, таких как OllyDbg или GDB. Такие антиотладочные приемы обсуждаются в разделе "Приемы противодействия динамическому анализу" на странице 487.

Обфускация опкодов

Хотя описанные до сих пор приемы могут затруднять - и фактически предназначены затруднять - понимание потока управления программы, ни один из них не мешает вам наблюдать правильную дизассемблированную форму анализируемой программы. Десинхронизация оказала наибольшее влияние на дизассемблирование, но ее легко удалось преодолеть, переформатировав дизассемблирование так, чтобы оно отражало правильный поток инструкций.

Более эффективный прием для предотвращения правильного дизассемблирования - кодировать или шифровать фактические инструкции при создании исполняемого файла. Обфусцированные инструкции должны быть деобфусцированы обратно в исходную форму до того, как процессор выберет их для выполнения. Следовательно, по крайней мере некоторая часть программы должна остаться незашифрованной, чтобы служить стартовой процедурой, которая в случае обфусцированной программы обычно отвечает за деобфускацию некоторой части или всего остатка программы. Очень общий обзор процесса обфускации показан на рисунке 21-2.

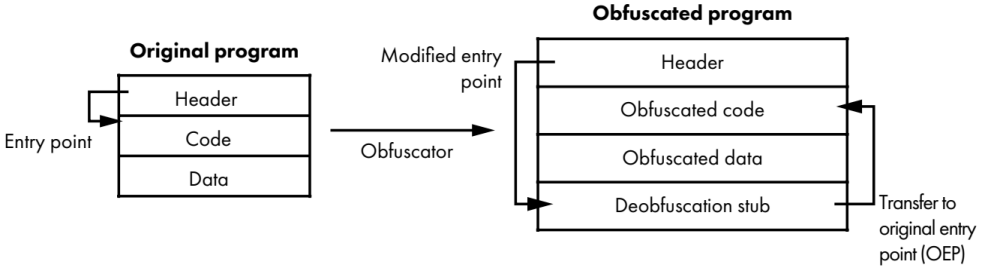


Figure 21-2: Generic obfuscation process

Рисунок 21-2. Общий процесс обфускации

Как показано, входом процесса является программа, которую нужно обфусцировать. Во многих случаях входная программа написана с использованием стандартных языков программирования и инструментов сборки (редакторов, компилятора и тому подобного), почти без учета будущей обфускации. Получившийся исполняемый файл передается утилите обфускации, которая преобразует бинарный файл в функционально эквивалентный, но обфусцированный бинарный файл. Как изображено, утилита обфускации отвечает за обфускацию секций кода и данных исходной программы и добавление дополнительного кода (заглушки деобфускации), выполняющего задачу деобфускации кода и данных до того, как исходная функциональность станет доступна во время выполнения. Утилита обфускации также изменяет заголовки программы,

чтобы перенаправить точку входа программы на заглушку деобфускации, гарантируя, что выполнение начнется с процесса деобфускации. После деобфускации выполнение обычно передается точке входа исходной программы, которая начинает выполняться так, будто она вообще никогда не была обфусцирована.

Этот чрезмерно упрощенный процесс сильно различается в зависимости от утилиты обфускации, использованной для создания обфусцированного бинарного файла. Для выполнения процесса обфускации доступно постоянно растущее число утилит. Такие утилиты предлагают возможности от сжатия до приемов антидизассемблирования и антиотладки. Примеры включают программы вроде UPX (упаковщик, также работает с ELF; https://upx.github.io/), ASPack (упаковщик; http://www.aspack.com/), ASProtect (анти-реверс-инжиниринг от создателей ASPack) и `tElock` (сжатие и анти-реверс-инжиниринг; http://www.softpedia.com/get/Programming/Packers-Crypters-Protectors/Telock.shtml) для Windows PE-файлов. Возможности утилит обфускации продвинулись до такой степени, что некоторые инструменты анти-реверс-инжиниринга, такие как VMProtect, интегрируются со всем процессом сборки, позволяя программистам внедрять возможности анти-реверс-инжиниринга на каждом этапе разработки, от исходного кода до постобработки скомпилированного бинарного файла (https://vmpsoft.com/).

Песочницы

Назначение песочницы для реверс-инжиниринга - позволить вам выполнять программы таким образом, чтобы наблюдать поведение программы, не позволяя этому поведению негативно влиять на критически важные компоненты вашей платформы реверс-инжиниринга или на что-либо, к чему она подключена. Песочницы обычно строятся с использованием ПО виртуализации платформ, но могут быть построены и на выделенных системах, способных возвращаться в заведомо исправное состояние после выполнения любого вредоносного ПО.

Системы-песочницы обычно сильно инструментированы, чтобы наблюдать и собирать информацию о поведении программ, запущенных внутри песочницы. Собранные данные могут включать информацию о файловой активности программы, активности реестра (Windows-) программы и сведения о любой сетевой активности, сгенерированной программой. Один пример полноценной песочницы - Cuckoo (https://cuckoosandbox.org/), популярная песочница с открытым исходным кодом, специально ориентированная на анализ вредоносного ПО.

Как и с любой наступательной технологией, для противодействия многим инструментам анти-реверс-инжиниринга были разработаны защитные меры. В большинстве случаев цель таких инструментов - восстановить исходный, незащищенный исполняемый файл (или приемлемое подобие), который затем можно анализировать с помощью более традиционных инструментов, таких как дизассемблеры и отладчики.

Один такой инструмент, предназначенный для деобфускации исполняемых файлов Windows, называется QuickUnpack (http://qunpack.ahteam.org/?p=458; сайт на русском). QuickUnpack, как и многие другие автоматизированные распаковщики, работает как отладчик: он позволяет обфусцированному бинарному файлу выполнить фазу деобфускации, а затем захватывает образ процесса из памяти. Помните, что инструменты такого типа фактически запускают потенциально вредоносные программы в надежде перехватить выполнение этих программ после того, как они распаковали или деобфусцировали себя, но до того, как успели сделать что-либо вредоносное. Поэтому такие программы всегда следует выполнять в окружении типа песочницы.

Использовать чисто статическое окружение анализа для анализа обфусцированного кода - сложная задача. Не имея возможности выполнить заглушку деобфускации, обфусцированные части бинарного файла нужно распаковать или расшифровать до начала дизассемблирования. Панель обзора Ghidra Address Type справа на рисунке 21-3 показывает компоновку исполняемого файла, упакованного с помощью упаковщика UPX. Ghidra раскрашивает содержимое на панели обзора, чтобы дать представление о соответствующем содержимом бинарного файла. Общие категории для панели обзора включают следующее:

• Function
• Uninitialized
• External Reference
• Instruction
• Data
• Undefined

Сосредоточившись на панели обзора на рисунке, мы можем увидеть предварительную оценку Ghidra для различных частей бинарного файла. Наведение курсора на любые секции панели обзора предоставит дополнительную информацию о соответствующей области бинарного файла. Необычный вид именно этой навигационной панели - подсказка, что бинарный файл каким-то образом обфусцирован. Давайте подробнее рассмотрим некоторые секции панели обзора.

Ghidra идентифицировала секцию данных в начале файла. Изучение этого содержимого раскрывает заголовки файла вместе с информативным содержимым, указывающим на тип обфускации, использованный в этом файле:

```
Этот файл упакован исполняемым упаковщиком UPX http://upx.tsx.org
UPX 1.07 Copyright (C) 1996-2001 команда UPX. Все права защищены.
/oneinv
/twoinv
/threeinv
/fourinv
/fiveinv
```

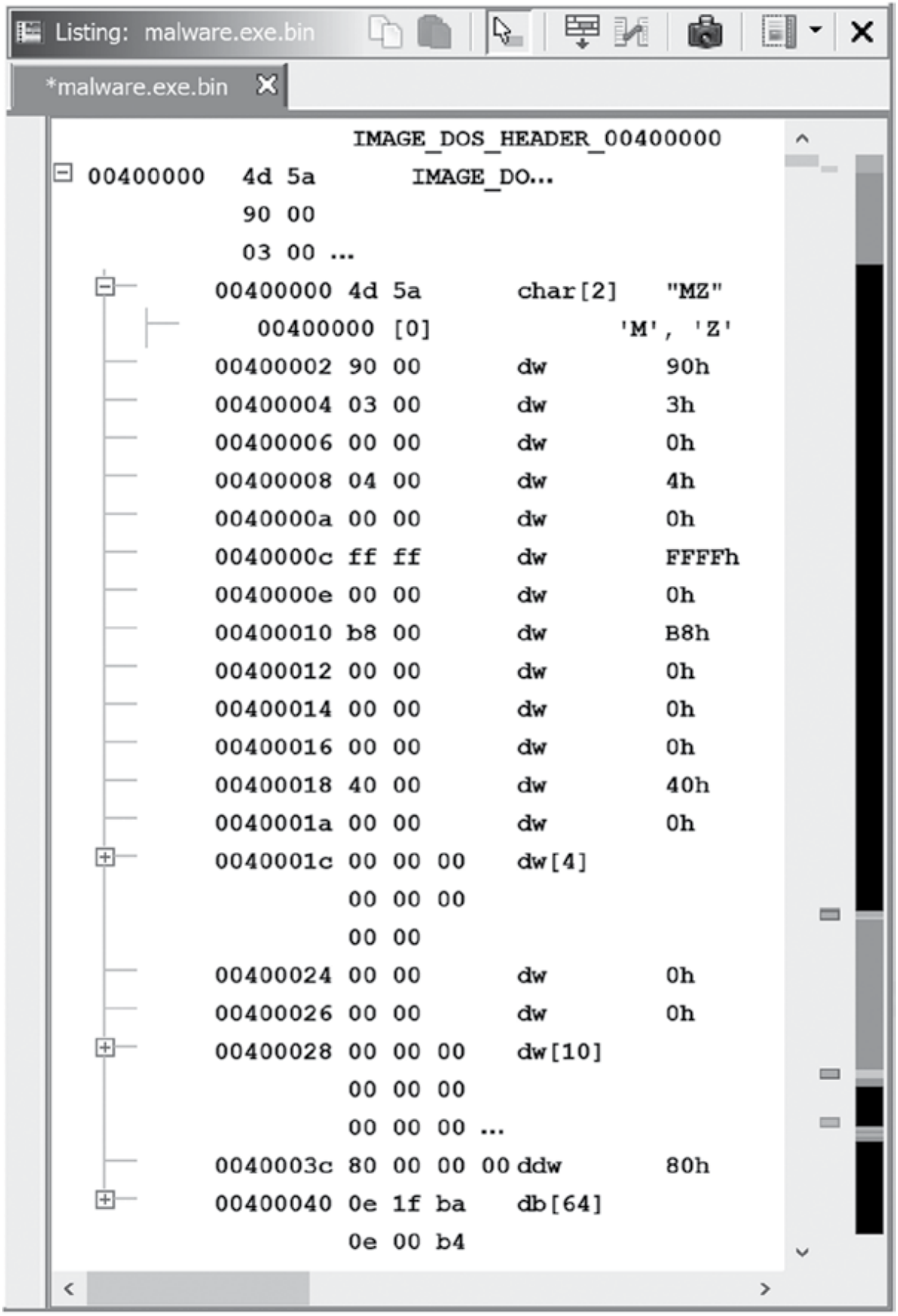


Рисунок 21-3. Окно `Listing` *Ghidra и панель обзора Address Type для бинарного файла, упакованного с помощью UPX*

За этой секцией следует блок неопределенного содержимого, похожий на следующий, который появляется в окне `Listing`:

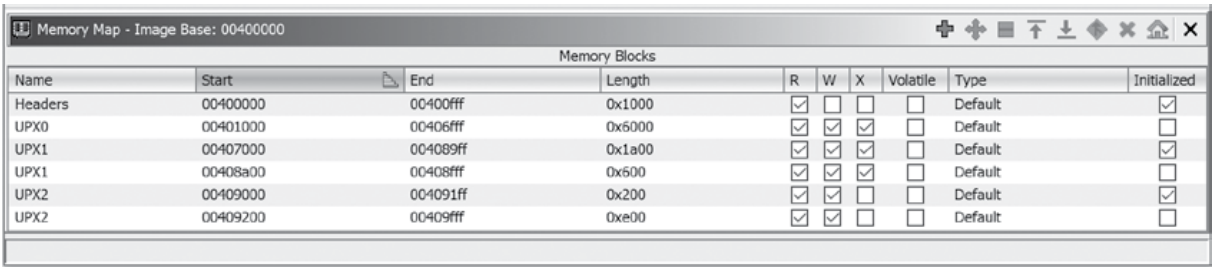
```
004008a3 72        ??     72h    r
004008a4 85        ??     85h
004008a5 6c        ??     6Ch    l
```

Самая большая секция содержит неинициализированные данные, которые в окне `Listing` выглядят следующим образом:

```
004034e3  ??       ??
004034e4  ??       ??
```

Немного дальше в файле Ghidra идентифицировала еще один блок неопределенного содержимого. В конце этих данных находится область, которую Ghidra идентифицировала как функцию. Эта функция легко распознается как заглушка распаковки UPX, которую Ghidra идентифицировала как точку входа бинарного файла, как показано в окне `Listing` слева на рисунке 21-3. Наблюдаемые нами сегменты неопределенного содержимого являются результатом процесса сжатия UPX. Задача заглушки распаковки - распаковать эти данные в неинициализированную область перед окончательной передачей управления распакованному коду.

Информацию, представленную панелью обзора Address Type, можно сопоставить со свойствами каждого сегмента внутри бинарного файла, чтобы определить, согласованы ли сведения в каждом отображении. Карта памяти для этого бинарного файла показана на рисунке 21-4.



Name	Start	End	Length	R	W	X	Volatile	Type	Initialized
Headers	00400000	00400fff	0x1000	☑	☐	☐	☐	Default	☑
UPX0	00401000	00406fff	0x6000	☑	☑	☑	☐	Default	☐
UPX1	00407000	004089ff	0x1a00	☑	☑	☑	☐	Default	☑
UPX1	00408a00	00408fff	0x600	☑	☑	☑	☐	Default	☐
UPX2	00409000	004091ff	0x200	☑	☑	☐	☐	Default	☑
UPX2	00409200	00409fff	0xe00	☑	☑	☐	☐	Default	☐

Рисунок 21-4. Карта памяти бинарного файла, упакованного UPX

В этом конкретном бинарном файле весь диапазон адресов, содержащийся в сегменте `UPX0` и сегменте `UPX1` (`00401000-00408fff`), помечен как исполняемый (флаг X установлен). Учитывая этот факт, мы должны ожидать, что вся панель обзора Address Type будет раскрашена как представляющая функции. Тот факт, что этого не происходит, в сочетании с тем, что весь диапазон `UPX0` неинициализирован и доступен для записи, следует считать крайне подозрительным; он дает ценные подсказки о бинарном файле и о том, как можно продолжить анализ.

Приемы использования Ghidra для выполнения операции распаковки в статическом контексте (без фактического выполнения бинарного файла) на файлах вроде этого обсуждаются в разделе "Статическая деобфускация бинарных файлов с помощью Ghidra" на странице 491.

Обфускация импортируемых функций

Приемы противодействия статическому анализу также могут скрывать, какие разделяемые библиотеки и библиотечные функции использует бинарный файл, чтобы избежать утечки информации о потенциальных действиях, которые он может выполнять. В большинстве случаев можно сделать инструменты вроде `dumpbin`, `ldd` и `objdump` бесполезными для вывода списка библиотечных зависимостей.

Влияние такой обфускации на Ghidra наиболее очевидно в дереве `Symbol Tree`. Все дерево `Symbol Tree` для нашего предыдущего примера `tElock` показано на рисунке 21-5.

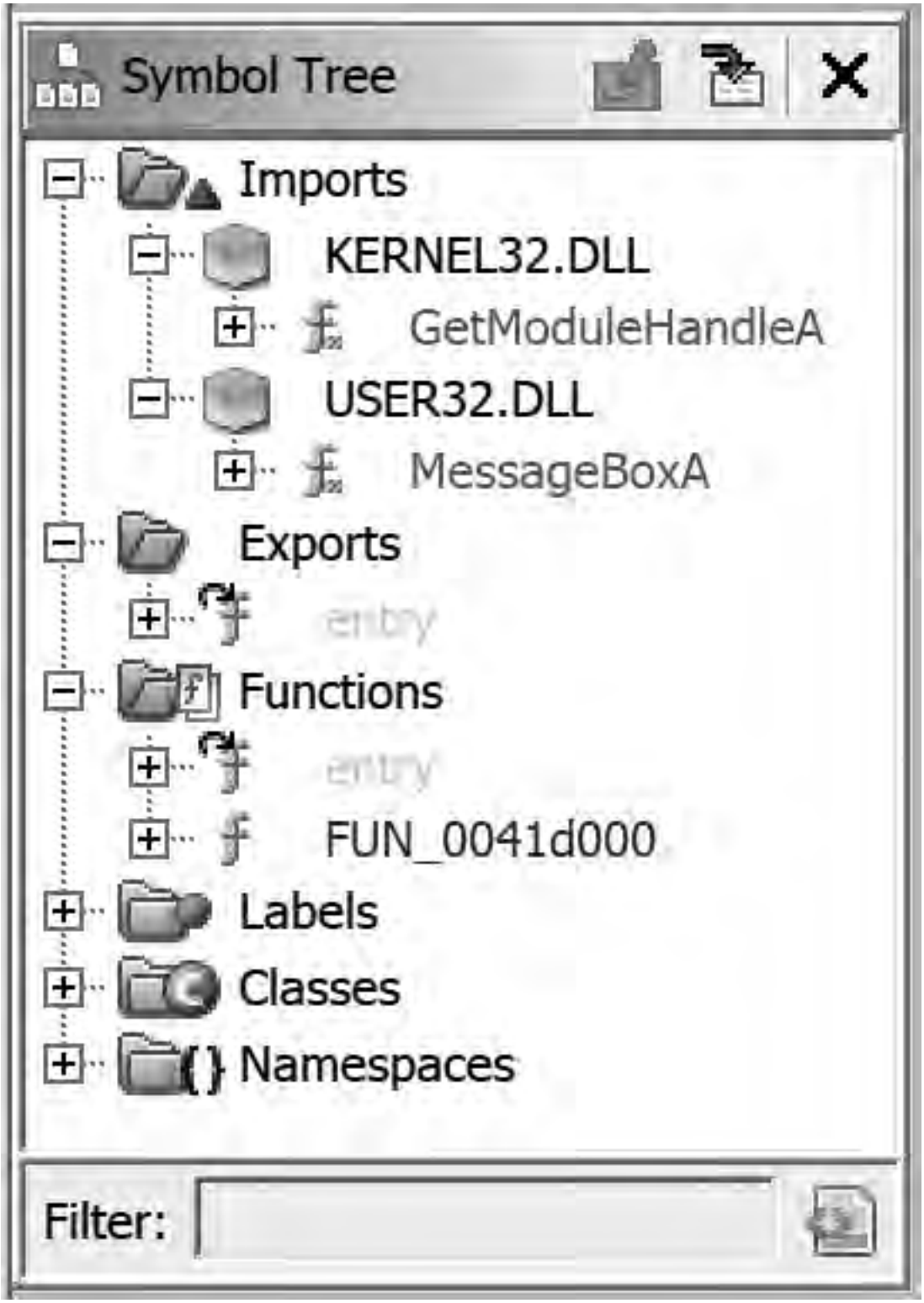


Рисунок 21-5. `Symbol Tree` *для обфусцированного бинарного файла*

Упоминаются только две импортируемые функции: `GetModulehandleA` (из `kernel32.dll`) и `MessageBoxA` (из `user32.dll`). Из этого короткого списка практически ничего нельзя вывести о поведении программы. И здесь приемы различаются, но по сути сводятся к тому, что сама программа должна загрузить все дополнительные библиотеки, от которых зависит, а после загрузки библиотек программа должна найти внутри них все требуемые функции. В большинстве случаев эти задачи выполняются заглушкой деобфускации до передачи управления деобфусцированной программе. Конечная цель состоит в том, чтобы таблица импортов программы была правильно

инициализирована так, как если бы этот процесс был выполнен собственным загрузчиком операционной системы.

Для Windows-бинарных файлов простой подход - использовать функцию `LoadLibrary`, чтобы загрузить требуемые библиотеки по имени, а затем функцию `GetProcAddress`, чтобы выполнять поиск адресов функций внутри каждой библиотеки. Чтобы использовать эти функции, программа должна быть либо явно скомпонована с ними, либо иметь альтернативные средства их поиска. `Symbol Tree` для примера `tElock` не включает ни одну из этих функций, тогда как `Symbol Tree` для примера UPX, показанное на рисунке 21-6, включает обе.

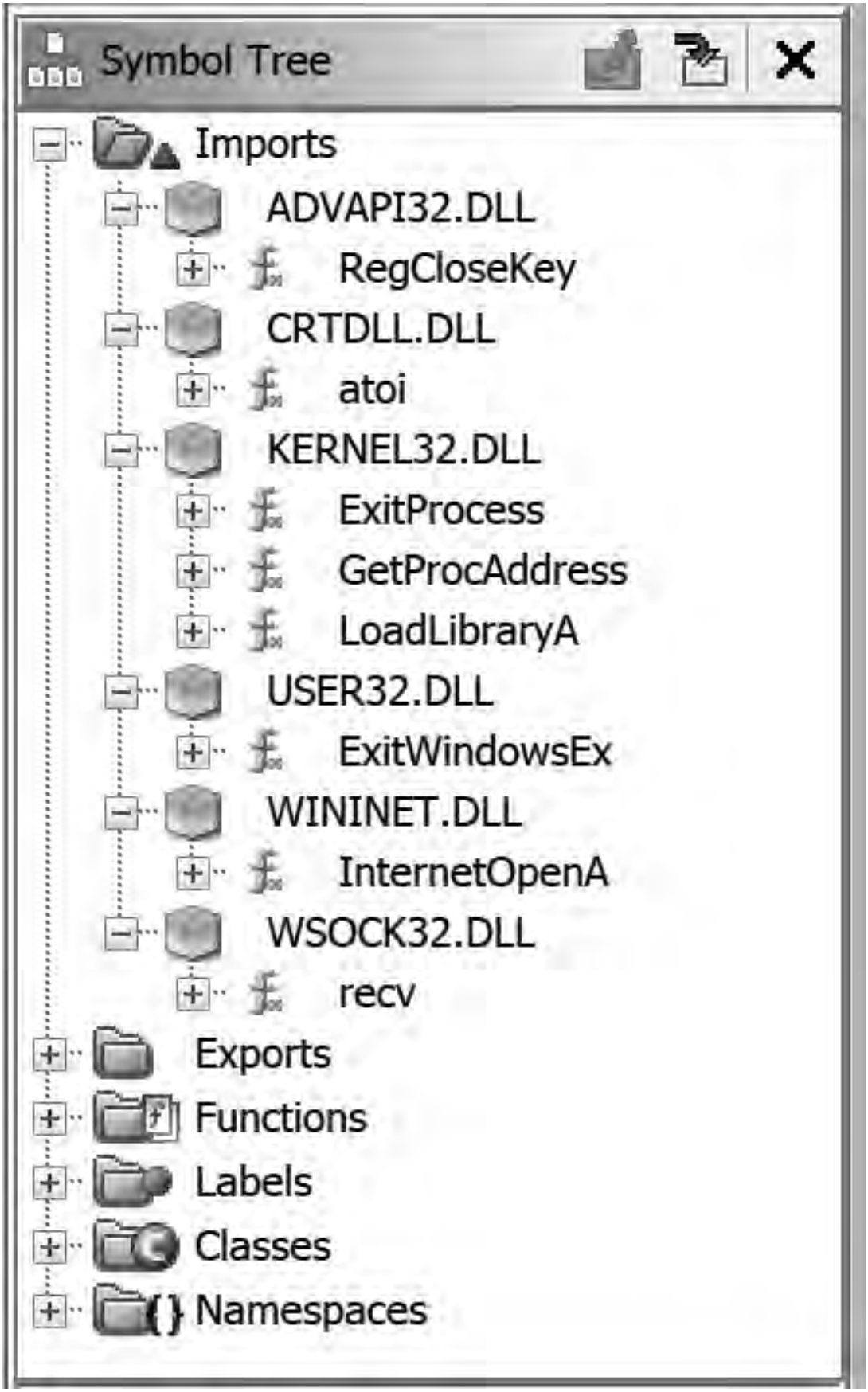


Рисунок 21-6. `Symbol Tree` *для примера UPX*

Фактический код UPX, отвечающий за перестроение таблицы импортов, показан в листинге 21-2.

```
LAB_0040886c                    XREF[1]: 0040888e(j)
```

```
0040886c  MOV    EAX,dword ptr [EDI]
0040886e  OR     EAX,EAX
00408870  JZ     LAB_004088ae
00408872  MOV    EBX,dword ptr [EDI + 0x4]
00408875  LEA    EAX,[EAX + ESI*0x1 + 0x8000]
0040887c  ADD    EBX,ESI
0040887e  PUSH   EAX
0040887f  ADD    EDI,0x8
00408882  CALL   dword ptr [ESI + 0x808c]=>KERNEL32.DLL::LoadLibraryA
00408888  XCHG   EAX,EBP
        LAB_00408889                          XREF[1]: 004088a6(j)
00408889  MOV    AL,byte ptr [EDI]
0040888b  INC    EDI
0040888c  OR     AL,AL
0040888e  JZ     LAB_0040886c
00408890  MOV    ECX,EDI
00408892  PUSH   EDI
00408893  DEC    EAX
00408894  SCASB.REPNE ES:EDI
00408896  PUSH    EBP
00408897  CALL   dword ptr [ESI + 0x8090]=>KERNEL32.DLL::GetProcAddress
0040889d  OR     EAX,EAX
0040889f  JZ     LAB_004088a8
004088a1  MOV    dword ptr [EBX],EAX ; сохранить в таблицу импорта
004088a3  ADD    EBX,0x4
004088a6  JMP    LAB_00408889
```

Листинг 21-2. Восстановление таблицы импортов в UPX

Этот пример содержит внешний цикл, отвечающий за вызов `LoadLibrary`, и внутренний цикл, отвечающий за вызов `GetProcAddress`. После каждого успешного вызова `GetProcAddress` заново полученный адрес функции сохраняется в восстановленной таблице импортов.

Эти циклы выполняются как последняя часть заглушки деобфускации UPX, потому что каждая функция принимает параметры-указатели на строки, указывающие либо на имя библиотеки, либо на имя функции, а связанные строки хранятся внутри области сжатых данных, чтобы избежать обнаружения утилитой `strings`. В результате загрузка библиотек в UPX не может происходить, пока требуемые строки не будут распакованы.

Возвращаясь к примеру `tElock`, мы сталкиваемся с другой проблемой. Имея только две импортируемые функции, ни одна из которых не является `LoadLibrary` или `GetProcAddress`, как утилита `tElock` может выполнять задачи разрешения функций, которые выполнялись UPX? Все процессы Windows зависят от `kernel32.dll`, а значит, она присутствует в памяти всех процессов. Если программа может найти `kernel32.dll`, то относительно прямолинейный процесс можно использовать для поиска любой функции внутри DLL, включая `LoadLibrary` и `GetProcAddress`. Как было показано ранее, имея эти две функции, можно загрузить любые дополнительные библиотеки, требуемые процессом, и найти все требуемые функции внутри этих библиотек.

В своей статье "Понимание шеллкода Windows" Skape обсуждает приемы именно для этого.[6] Хотя `tElock` не использует в точности приемы, подробно описанные Skape, параллелей много, а итоговый эффект состоит в сокрытии деталей процесса загрузки и связывания. Без тщательного трассирования инструкций программы чрезвычайно легко пропустить загрузку библиотеки или поиск адреса функции. Следующий небольшой фрагмент кода иллюстрирует, каким образом `tElock` пытается найти адрес `LoadLibrary`:

```
0041d1e4  CMP    dword ptr [EAX],0x64616f4c
0041d1ea  JNZ    LAB_0041d226
0041d1ec  CMP    dword ptr [EAX + 0x4],0x7262694c
0041d1f3  JNZ    LAB_0041d226
0041d1f5  CMP    dword ptr [EAX + 0x8],0x41797261
0041d1fc  JNZ    LAB_0041d226
```

Сразу очевидно, что несколько сравнений выполняются в быстрой последовательности. Что может быть не сразу ясно, так это цель этих сравнений. Переформатирование операндов (щелкните правой кнопкой и выберите Convert -> Char Sequence), используемых в каждом сравнении, проливает немного света на код, как видно в следующем листинге.

```
0041d1e4  CMP    dword ptr [EAX],"Load"
0041d1ea  JNZ    LAB_0041d226
0041d1ec  CMP    dword ptr [EAX + 0x4],"Libr"
0041d1f3  JNZ    LAB_0041d226
0041d1f5  CMP    dword ptr [EAX + 0x8],"aryA"
0041d1fc  JNZ    LAB_0041d226
```

Каждая шестнадцатеричная константа фактически является последовательностью из четырех символов ASCII, которые Ghidra способна отображать как ASCII в кавычках; вместе они образуют `LoadLibraryA`.[7] Если три сравнения успешны, `tElock` нашел запись таблицы экспортов для `LoadLibraryA` и через несколько коротких операций получит адрес этой функции для загрузки дополнительных библиотек. Подход `tElock` к поиску функций отчасти устойчив к анализу строк, потому что 4-байтовые константы, встроенные прямо в инструкции программы, не выглядят как стандартные строки с нулевым завершением и потому не включаются в список строк, сгенерированный Ghidra, если вы не измените значения по умолчанию (например, сняв флажок Require Null Termination во время поиска строк).

Ручное восстановление таблицы импортов программы через тщательный анализ кода программы упрощается в случае UPX и `tElock`, потому что в конечном счете оба содержат символьные данные ASCII, которые можно использовать, чтобы точно определить, на какие библиотеки и функции есть ссылки. В статье Skape подробно описан процесс разрешения функций, при котором в коде вообще не появляются строки. Основная идея, обсуждаемая в статье, состоит в предварительном вычислении уникального хеш-значения для имени каждой функции, которую нужно разрешить.[8] Чтобы разрешить каждую функцию, можно пройти по таблице экспортируемых имен библиотеки. Каждое имя в таблице хешируется, и полученный хеш можно сравнить с предварительно вычисленным хеш-значением для нужной функции. Если хеши совпадают, вы нашли нужную функцию и можете легко найти ее адрес в таблице адресов экспорта библиотеки.

Чтобы статически анализировать бинарные файлы, обфусцированные таким образом, нужно понять алгоритм хеширования, используемый для каждого имени функции, и применить этот алгоритм ко всем именам, экспортируемым библиотекой, в которой ищет программа. Имея полную таблицу хешей, можно выполнить простой поиск каждого хеша, встреченного в программе, чтобы определить, на какую функцию ссылается хеш. Часть такой таблицы, сгенерированной для `kernel32.dll`, может выглядеть так:

```
GetProcAddress : 8A0FB5E2
GetProcessAffinityMask : B9756EFE
GetProcessHandleCount : B50EB87C
GetProcessHeap : C246DA44
GetProcessHeaps : A18AAB23
GetProcessId : BE05ED07
```

Обратите внимание, что хеш-значения специфичны для хеш-функции, используемой внутри конкретного бинарного файла, и, вероятно, будут различаться от одного бинарного файла к другому. Используя эту конкретную таблицу, если бы хеш-значение `8A0FB5E2` встретилось внутри программы, вы могли бы быстро определить, что программа пытается найти адрес функции `GetProcAddress`.

Использование Skape хеш-значений для разрешения имен функций изначально было разработано и задокументировано для применения в полезных нагрузках эксплойтов для уязвимостей Windows; однако этот подход также был взят на вооружение в обфусцированных программах.

Приемы противодействия динамическому анализу

Ни один из приемов противодействия статическому анализу, рассмотренных в нескольких предыдущих разделах, никак не влияет на то, будет ли программа фактически выполняться. На самом деле, хотя приемы противодействия статическому анализу могут затруднить понимание истинного поведения программы с использованием только статического анализа, они не могут предотвратить выполнение программы, иначе сделали бы программу бесполезной с самого начала и тем самым устранили бы саму необходимость анализировать программу.

Поскольку программа должна выполняться, чтобы делать любую полезную работу, динамический анализ стремится наблюдать поведение программы в движении (пока она выполняется), а не наблюдать программу в покое (используя статический анализ, пока программа не выполняется). В этом разделе мы кратко суммируем некоторые из более распространенных приемов противодействия динамическому анализу. В большинстве случаев эти приемы мало влияют на инструменты статического анализа; однако там, где есть пересечение, мы укажем на него.

Обнаружение виртуализации

Песочницы обычно используют ПО виртуализации, такое как VMware, чтобы предоставить среду выполнения для вредоносного ПО (или любого другого интересующего ПО). Преимущество таких сред состоит в том, что они обычно предлагают возможности контрольных точек и отката, упрощающие быстрое восстановление песочницы до заведомо чистого состояния. Главный недостаток состоит в том, что вредоносное ПО может обнаружить песочницу. Исходя из предположения, что виртуализация равна наблюдению, многие программы, желающие остаться незамеченными, просто завершают работу, когда определяют, что выполняются внутри виртуальной машины. С учетом широкого использования виртуализации в производственных целях это предположение сегодня менее справедливо, чем было исторически.

Следующий список описывает несколько приемов, которые использовались программами, работающими в виртуализированных средах, чтобы определить, что они выполняются внутри виртуальной машины, а не на физическом оборудовании:

Обнаружение ПО, специфичного для виртуализации. Пользователи часто устанавливают вспомогательные приложения внутри виртуальных машин, чтобы облегчить взаимодействие между виртуальной машиной и операционной системой хоста или просто повысить производительность внутри виртуальной машины. Набор VMware Tools - один пример такого ПО. Наличие такого ПО легко обнаруживается программами, работающими внутри виртуальной машины. Например, когда VMware Tools устанавливается в виртуальную машину Microsoft Windows, он создает ключи реестра Windows, которые может прочитать любая программа. Вредоносное ПО, обнаружившее эти ключи, может решить завершить работу до проявления какого-либо заметного поведения. С другой стороны, виртуализация сегодня используется настолько широко, что образ VMware без установленного VMware Tools может казаться вредоносному ПО столь же подозрительным.

Обнаружение оборудования, специфичного для виртуализации. Виртуальные машины используют слои абстракции виртуального оборудования, чтобы предоставить интерфейс между виртуальной машиной и физическим оборудованием хост-компьютера. Характеристики виртуального оборудования часто легко обнаруживаются ПО, работающим внутри виртуальной машины. Например, VMware были назначены собственные организационно уникальные идентификаторы (OUI) для ее виртуализированных сетевых адаптеров.[9] Наблюдение OUI, специфичного для VMware, является хорошим признаком того, что программа выполняется внутри виртуальной машины. ПО, завершающее работу по этой причине, можно склонить к выполнению, изменив MAC-адрес, назначенный любым виртуальным сетевым адаптерам, связанным с виртуальной машиной.

Обнаружение специфичных для процессора изменений поведения. Идеальной виртуализации трудно достичь. В идеале программа не должна уметь обнаруживать никакой разницы между виртуализированной средой и физическим оборудованием. Однако так бывает редко. Joanna Rutkowska разработала свой прием обнаружения VMware Red Pill после наблюдения поведенческих различий между работой инструкции x86 `sidt` на физическом оборудовании и той же инструкцией, выполненной внутри среды виртуальной машины.[10]

Обнаружение инструментирования

После создания песочницы и перед выполнением любой программы, которую вы хотите наблюдать, нужно убедиться, что инструментирование развернуто для правильного сбора и записи информации о поведении анализируемой программы. Для выполнения таких задач мониторинга существует широкий набор инструментов. Два широко используемых примера - Process Monitor от группы Sysinternals в Microsoft и Wireshark.[11] Process Monitor - утилита, способная отслеживать определенную активность, связанную с любым выполняющимся процессом Windows, включая обращения к реестру Windows и активность файловой системы. Wireshark - инструмент захвата и анализа сетевых пакетов, часто используемый для анализа сетевого трафика, сгенерированного вредоносным ПО.

Авторы вредоносного ПО с достаточным уровнем паранойи могут запрограммировать свое ПО на поиск запущенных экземпляров таких программ мониторинга. Используемые приемы варьируются от сканирования списка активных процессов на предмет имен процессов, связанных с таким ПО мониторинга, до сканирования текста заголовков всех активных приложений Windows в поисках известных строк. Могут выполняться и более глубокие поиски, причем некоторое ПО доходит до поиска конкретных характеристик, связанных с GUI-компонентами Windows, используемыми внутри определенного ПО инструментирования.

Обнаружение отладчиков

Выходя за пределы простого наблюдения за программой, отладчик позволяет аналитику полностью контролировать выполнение программы, требующей анализа. Отладчики часто используются, чтобы запустить обфусцированную программу ровно настолько, чтобы завершить любые задачи распаковки или расшифрования, а затем возможности отладчика по доступу к памяти используются для извлечения деобфусцированного образа процесса из памяти. В большинстве случаев стандартные инструменты и приемы статического анализа можно использовать для завершения анализа извлеченного образа процесса.

Авторы утилит обфускации хорошо знают о таких приемах деобфускации с помощью отладчика и поэтому разработали меры, пытающиеся сорвать использование отладчиков для выполнения их обфусцированных программ. Программы, обнаруживающие присутствие отладчика, часто предпочитают завершиться, а не продолжать какие-либо операции, которые могли бы позволить аналитику определить поведение программы.

Приемы обнаружения присутствия отладчиков варьируются от простых запросов к операционной системе через хорошо известные API-функции, такие как функция Windows `IsDebuggerPresent`, до низкоуровневых проверок артефактов памяти или процессора, возникающих из-за использования отладчика. Один пример последнего - обнаружение того, что установлен флаг трассировки процессора (пошагового выполнения).

Если вы знаете, что искать, в попытках обнаружить отладчик нет ничего особенно хитрого, и такие попытки легко наблюдаются во время статического анализа (если одновременно не применяются приемы противодействия статическому анализу). Дополнительную информацию об обнаружении отладчиков см. в статье "Методы обнаружения отладчика с примерами", которая дает всесторонний обзор антиотладочных приемов Windows.[12]

Предотвращение отладки

Даже необнаруживаемому отладчику можно помешать с помощью дополнительных приемов, пытающихся запутать отладчик путем внесения ложных точек останова, очистки аппаратных точек останова, затруднения дизассемблирования для усложнения выбора подходящих адресов точек останова или предотвращения подключения отладчика к процессу с самого начала. Многие приемы, обсуждаемые в ранее упомянутой статье об антиотладке, направлены на предотвращение корректной работы отладчиков.

Намеренное генерирование исключений - один из способов, которым программа может пытаться затруднить отладку. В большинстве случаев подключенный отладчик перехватит исключение, а пользователь отладчика должен проанализировать, почему произошло исключение и следует ли передать исключение отлаживаемой программе. В случае программной точки останова, такой как x86 `INT 3`, может быть трудно отличить программное прерывание, сгенерированное базовой программой, от прерывания, возникшего из-за настоящей точки останова отладчика. Именно этого замешательства и добивается создатель обфусцированной программы. В таких случаях возможно, хотя и труднее, понять истинный поток программы с помощью тщательного анализа дизассемблированного листинга.

Кодирование частей программы имеет двойной эффект: оно затрудняет статический анализ, потому что дизассемблирование невозможно, и затрудняет отладку, потому что размещение точек останова сложно. Даже если начало каждой инструкции известно, программные точки останова нельзя разместить до фактического декодирования инструкций, поскольку изменение инструкций

путем вставки программной точки останова, вероятно, приведет к неудачному расшифрованию обфусцированного кода, а затем к аварийному завершению программы, когда выполнение достигнет предполагаемой точки останова.

Инструмент обфускации ELF Shiva для Linux использует прием, называемый взаимным `ptrace`, как средство предотвращения использования отладчика при анализе поведения Shiva.

Трассировка процессов

API `ptrace`, или трассировка процессов, доступный во многих Unix-подобных системах, предоставляет механизм, с помощью которого один процесс может отслеживать и контролировать выполнение другого процесса. Отладчик GNU (`gdb`) - одно из более известных приложений, использующих API `ptrace`. Используя API `ptrace`, родительский процесс `ptrace` может подключиться к дочернему процессу `ptrace` и контролировать его выполнение.

Чтобы начать контролировать процесс, родительский процесс сначала должен подключиться к дочернему процессу, которым он хочет управлять. После подключения родительского процесса дочерний процесс останавливается всякий раз, когда получает сигнал, а родитель уведомляется об этом через POSIX-функцию `wait`; в этот момент родитель может изменить или изучить состояние дочернего процесса, прежде чем поручить ему продолжить выполнение. После того как родительский процесс подключился к дочернему, никакой другой процесс не может подключиться к тому же дочернему процессу, пока трассирующий родитель не решит отключиться от дочернего процесса.

Shiva использует тот факт, что к процессу в любой момент времени может быть подключен только один другой процесс. В начале выполнения процесс Shiva выполняет fork, чтобы создать копию самого себя. Исходный процесс Shiva немедленно выполняет операцию подключения `ptrace` к только что порожденному дочернему процессу. Только что порожденный дочерний процесс, в свою очередь, немедленно подключается к своему родительскому процессу. Если любая из операций подключения завершается неудачно, Shiva завершается, предполагая, что для наблюдения за процессом Shiva используется отладчик. Если обе операции успешны, никакой другой отладчик не может подключиться к работающей паре Shiva, и Shiva может продолжать работу, не опасаясь наблюдения. Работая таким образом, любой из процессов Shiva может изменять состояние другого, из-за чего трудно определить с помощью статического анализа, какой именно путь потока управления проходит через бинарный файл Shiva.

Статическая деобфускация бинарных файлов с помощью Ghidra

В этот момент вы можете задаваться вопросом, как, учитывая все доступные приемы анти-реверс-инжиниринга, вообще возможно анализировать ПО, которое программист намерен сохранить в секрете. Учитывая, что эти приемы нацелены и на инструменты статического анализа, и на инструменты динамического анализа, какой подход лучше всего выбрать для раскрытия скрытого поведения программы? К сожалению, нет единого решения, одинаково хорошо подходящего для всех случаев.

В большинстве случаев решение зависит от ваших навыков и доступных инструментов. Если ваш инструмент анализа по выбору - отладчик, вам нужно будет разработать стратегии обхода защит, обнаруживающих и предотвращающих отладку. Если ваш предпочитаемый инструмент анализа - дизассемблер, вам нужно будет разработать стратегии получения точного дизассемблирования, а в случаях, когда встречается самомодифицирующийся код, - имитации поведения этого кода,

чтобы правильно обновлять дизассемблированные листинги.

В этом разделе мы обсудим два приема работы с самомодифицирующимся кодом в среде статического анализа (то есть без выполнения кода). Статический анализ может быть вашим единственным вариантом, когда вы не хотите (из-за враждебного кода) или не можете (из-за отсутствия оборудования) анализировать программу, контролируя ее отладчиком. Если эти концепции заставляют вас чувствовать, что вы уходите в кроличью нору, не отчаивайтесь. У Ghidra есть секретное (или не такое уж секретное) оружие, которое мы можем использовать в гонке статической деобфускации.

Деобфускация, ориентированная на сценарии

Поскольку Ghidra можно использовать для дизассемблирования бинарных файлов для постоянно растущего набора процессоров, нередко приходится анализировать бинарный файл, разработанный для совершенно другой платформы, чем та, на которой вы запускаете Ghidra. Например, вас могут попросить проанализировать Linux x86-бинарный файл, хотя вы запускаете Ghidra на macOS, или вас могут попросить проанализировать MIPS- или ARM-бинарный файл, хотя вы запускаете Ghidra на x86-платформе.

В таких случаях у вас может не быть доступа к подходящим инструментам, таким как отладчики, для динамического анализа бинарного файла. Когда такой бинарный файл был обфусцирован путем кодирования частей программы, у вас может не быть другого варианта, кроме создания сценария Ghidra, который будет имитировать этап деобфускации программы, чтобы правильно декодировать программу и дизассемблировать декодированные инструкции и данные.

Это может казаться пугающей задачей; однако во многих случаях этапы декодирования обфусцированной программы используют лишь небольшое подмножество набора инструкций процессора, поэтому знакомство с необходимыми операциями может не требовать понимания всего набора инструкций целевого процессора.

В главе 14 был представлен алгоритм разработки сценариев, эмулирующих поведение частей программы. В следующем примере мы используем эти шаги для разработки простого сценария Ghidra, декодирующего программу, зашифрованную инструментом шифрования ELF Burneye. В нашей примерной программе выполнение начинается с инструкций в листинге 21-3.

```
05371035  PUSH   dword ptr [DAT_05371008]
0537103b  PUSHFD
0537103c  PUSHAD
0537103d  MOV    ECX,dword ptr [DAT_05371000]
05371043  JMP    LAB_05371082
...
        LAB_05371082                          XREF[1]:     05371043(j)
05371082  CALL   FUN_05371048
05371087  SHL    byte ptr [EBX + -0x2b],1
0537108a  PUSHFD
0537108b  XCHG   byte ptr [EDX + -0x11],AL
0537108e  POP    SS
0537108f  XCHG   EAX,ESP
05371090  CWDE
05371091  AAD    0x8e
05371093  PUSH   ECX
05371094  OUT    DX,EAX
05371095  ADD    byte ptr [EDX + 0xa81bee60],BH
0537109b  PUSH   SS
0537109c  RCR    dword ptr [ESI + 0xc],CL
0537109f  PUSH   CS
053710a0  SUB    AL,0x70
053710a2  CMP    CH,byte ptr [EAX + 0x6e]
053710a5  CMP    dword ptr [DAT_cbd35372],0x9c38a8bc
053710af  AND    AL,0xf4
```

```
053710b1  SBB    EBP,ESP
053710b4  POP    DS
053710b5  ??     C6h
```

Листинг 21-3. Стартовая последовательность Burneye и обфусцированный код

Программа начинает с помещения содержимого ячейки памяти `05371008h` в стек, а затем помещает в стек флаги процессора и все регистры процессора. Цель этих инструкций не сразу ясна, поэтому мы просто отложим эту информацию на потом. Затем регистр `ECX` загружается содержимым ячейки памяти `05371000h`. Согласно алгоритму, представленному в главе 14, в этот момент нужно объявить переменную с именем `ECX` и инициализировать ее из памяти с помощью функции Ghidra `getInt`, как показано здесь:

```
int ECX = getInt(toAddr(0x5371000));    // из инструкции 0537103d
```

После абсолютного перехода программа вызывает функцию `FUN_05371048`, которая помещает адрес `05371087h` (возвращаемый адрес) в стек. Дизассемблированные инструкции, следующие за инструкцией `CALL`, начинают иметь все меньше и меньше смысла. Инструкция `OUT` обычно не встречается в коде пользовательского пространства, а Ghidra не может дизассемблировать инструкцию по адресу `053710B5h`. И то и другое указывает, что с этим бинарным файлом что-то не так (как и тот факт, что `Symbol Tree` перечисляет только две функции: `entry` и `FUN_05371048`).

В этот момент анализ нужно продолжить с вызова функции `FUN_05371048`, показанной в листинге 21-4.

```
FUN_05371048                                XREF[1]:     entry:05371082(c)
05371048  POP    ESI
05371049  MOV    EDI,ESI
0537104b  MOV    EBX,dword ptr [DAT_05371004] = C09657B0h
05371051  OR     EBX,EBX
05371053  JZ     LAB_0537107f
05371059  XOR    EDX,EDX
        LAB_0537105b                        XREF[1]:  0537107d(j)
0537105b  MOV    EAX,0x8
        LAB_05371060                        XREF[1]:  05371073(j)
05371060  SHRD   EDX,EBX,0x1
05371064  SHR    EBX,1
05371066  JNC    LAB_05371072
0537106c  XOR    EBX,0xc0000057
        LAB_05371072                        XREF[1]:  05371066(j)
05371072  DEC    EAX
05371073  JNZ    LAB_05371060
05371075  SHR    EDX,0x18
05371078  LODSB  ESI
05371079  XOR    AL,DL
0537107b  STOSB  ES:EDI
0537107c  DEC    ECX
0537107d  JNZ    LAB_0537105b
        LAB_0537107f                        XREF[1]:  05371053(j)
0537107f  POPAD
05371080  POPFD
05371081  RET
```

Листинг 21-4. Основная функция декодирования Burneye

Это нетипичная функция: она начинается с немедленного извлечения возвращаемого адреса из стека в регистр ESI. Вспоминая, что сохраненный возвращаемый адрес был 05371087h, и учитывая инициализацию EDI, EBX и EDX, наш сценарий вырастает до следующего:

```
int ECX = getInt(toAddr(0x5371000));   // из инструкции 0537103D
int ESI = 0x05371087;                  // из инструкции 05371048
int EDI = ESI;                         // из инструкции 05371049
int EBX = getInt(toAddr(0x5371004));   // из инструкции 0537104B
int EDX = 0;                           // из инструкции 05371059
```

После этих инициализаций функция выполняет проверку значения, содержащегося в регистре EBX, перед входом во внешний и внутренний циклы. Оставшаяся логика функции отражена в следующем завершенном сценарии. Внутри сценария комментарии используются, чтобы связать действия сценария с соответствующими действиями в предыдущем дизассемблированном листинге.

```
public void run() throws Exception {
   int ECX = getInt(toAddr(0x5371000));   // из инструкции 0537103D
   int ESI = 0x05371087;                  // из инструкции 05371048
   int EDI = ESI;                         // из инструкции 05371049
   int EBX = getInt(toAddr(0x5371004));   // из инструкции 0537104B
   if (EBX != 0) {                        // из инструкций 05371051
                                          //   и 05371053
      int EDX = 0;                        // из инструкции 05371059
      do {
         int EAX = 8;                     // из инструкции 0537105B
         do {
                                          // имитировать инструкцию x86 shrd
                                          //   с помощью нескольких операций
            EDX = EDX >>> 1;              // беззнаковый сдвиг вправо на один бит
            int CF = EBX & 1;             // запомнить младший бит EBX
            if (CF == 1) {                // CF представляет флаг переноса x86
               EDX = EDX | 0x80000000;    // вдвинуть младший бит EBX, если он
    равен 1
            }
            EBX = EBX >>> 1;              // беззнаковый сдвиг вправо на один бит
            if (CF == 1) {                // из инструкции 05371066
               EBX = EBX ^ 0xC0000057;    // из инструкции 0537106C
            }
            EAX--;                        // из инструкции 05371072
         } while (EAX != 0);              // из инструкции 05371073
         EDX = EDX >>> 24;                // беззнаковый сдвиг вправо на 24 бита
         EAX = getByte(toAddr(ESI));      // из инструкции 05371078
         ESI++;
         EAX = EAX ^ EDX;                 // из инструкции 05371079
         clearListing(toAddr(EDI));       // очистить байт, чтобы его можно было
    изменить
         setByte(toAddr(EDI), (byte)EAX); // из инструкции 0537107B
         EDI++;
         ECX--;                           // из инструкции 0537107C
      } while (ECX != 0);                 // из инструкции 0537107D
   }
}
```

Когда вы пытаетесь эмулировать инструкцию, следует уделять особое внимание размерам данных и псевдонимам регистров. В этом примере нужно выбрать подходящий размер данных и переменную, чтобы правильно реализовать инструкции x86 `LODSB` (загрузить байт строки) и `STOSB` (сохранить байт строки). Эти инструкции записывают в (`LODSB`) и читают из (`STOSB`) младшие 8 бит регистра `EAX`,[13] оставляя старшие 24 бита неизменными. В Java нет способа разделить переменную на битовые части иначе, кроме как с помощью различных побитовых операций для маскирования и повторного объединения частей переменной. Конкретно в случае инструкции `LODSB` более точная эмуляция выглядела бы так:

```
EAX = (EAX & 0xFFFFFF00) | (getByte(toAddr(ESI)) & 0xFF);
```

Этот пример сначала очищает младшие 8 бит переменной `EAX`, а затем объединяет новое значение для младших 8 бит с помощью операции OR. В примере декодирования Burneye весь регистр `EAX` устанавливается в 8 в начале каждого внешнего цикла, что приводит к обнулению старших 24 бит `EAX`. В результате мы решили упростить нашу реализацию `LODSB`, игнорируя влияние присваивания на старшие 24 бита `EAX`. О нашей реализации `STOSB` думать не нужно, поскольку функция `setByte` требует привести второй аргумент к байту.

После выполнения сценария декодирования Burneye наше дизассемблирование отразит все изменения, которые обычно были бы ненаблюдаемы до выполнения обфусцированной программы в системе Linux. Если процесс деобфускации был выполнен правильно, мы, скорее всего, увидим гораздо больше читаемых строк в параметре Ghidra Search -> "For Strings...". Чтобы увидеть это, может потребоваться выбрать значок Refresh в окне String Search.

Оставшиеся задачи включают: (1) определить, куда вернется функция декодирования, учитывая, что она извлекла свой возвращаемый адрес самой первой инструкцией функции; и (2) заставить Ghidra правильно отобразить декодированные значения байтов как инструкции или данные, в зависимости от ситуации. Функция декодирования Burneye завершается следующими тремя инструкциями:

```
0537107f  POPAD
05371080  POPFD
05371081  RET
```

Вспомним, что функция началась с извлечения собственного возвращаемого адреса, а значит, оставшиеся значения стека были подготовлены вызывающей стороной. Используемые здесь инструкции `POPAD` и `POPFD` являются парными к инструкциям `PUSHAD` и `PUSHFD`, использованным в начале стартовой процедуры Burneye, как показано здесь:

```
      entry
05371035  PUSH   dword ptr [DAT_05371008]
0537103b  PUSHFD
0537103c  PUSHAD
```

Итоговый результат состоит в том, что единственное значение, оставшееся в стеке, - это значение, помещенное в первой строке `entry`. Именно в это место возвращается процедура декодирования Burneye, и именно с этого места должен продолжаться дальнейший анализ бинарного файла, защищенного Burneye.

Предыдущий пример может создать впечатление, что написать сценарий для декодирования или распаковки обфусцированного бинарного файла относительно легко. Это верно в случае Burneye, который не использует слишком сложный начальный алгоритм обфускации. Заглушка деобфускации более сложных утилит, таких как ASPack и `tElock`, потребовала бы несколько больших усилий для реализации с помощью Ghidra.

Преимущества деобфускации на основе сценариев включают то, что анализируемый бинарный файл никогда не нужно выполнять, и то, что можно создать функциональный сценарий, так и не разработав полного понимания точного алгоритма, использованного для деобфускации бинарного

файла. Последнее утверждение может казаться нелогичным, поскольку кажется, что перед эмуляцией алгоритма с помощью сценария нужно полностью понять деобфускацию. Однако при использовании процесса разработки, описанного здесь и в главе 14, все, что вам действительно нужно, - полное понимание каждой инструкции процессора, участвующей в процессе деобфускации. Добросовестно реализуя каждое действие процессора с помощью Ghidra и правильно упорядочивая каждое действие согласно дизассемблированному листингу, вы получите сценарий, имитирующий действия программы, даже если не полностью понимаете высокоуровневый алгоритм, который эти действия в совокупности реализуют.

Недостатки подхода на основе сценариев включают то, что сценарии довольно хрупки. Если алгоритм деобфускации меняется в результате обновления инструмента деобфускации или из-за использования альтернативных параметров командной строки, переданных инструменту обфускации, сценарий, который был эффективен против этого инструмента, вероятно, придется соответствующим образом изменить. Например, можно разработать универсальные сценарии распаковки для использования с бинарными файлами, упакованными UPX, но такие сценарии требуют постоянной настройки по мере развития UPX.

Наконец, сценарная деобфускация страдает от отсутствия универсального решения для деобфускации. Не существует мегасценария, способного деобфусцировать все бинарные файлы. В каком-то смысле сценарная деобфускация страдает от многих тех же недостатков, что и системы обнаружения вторжений и антивирусные системы на основе сигнатур. Для каждого нового типа упаковщика нужно разработать новый сценарий, а тонкие изменения в существующих упаковщиках, вероятно, сломают существующие сценарии. Перенесем внимание и рассмотрим более общий подход к деобфускации.

Деобфускация, ориентированная на эмуляцию

Повторяющаяся тема, встречающаяся при создании сценариев для выполнения задач деобфускации, - необходимость эмулировать набор инструкций процессора, чтобы вести себя идентично деобфусцируемой программе. Эмуляторы инструкций позволяют перенести часть или всю работу, выполняемую этими сценариями, на эмулятор и резко сократить время, требуемое Ghidra для деобфускации. Эмуляторы могут заполнить пустоту между сценариями и отладчиками и могут быть гибче отладчиков. Эмулятор может, например, эмулировать MIPS-бинарный файл на x86-платформе или эмулировать инструкции из Linux ELF-бинарного файла на Windows-платформе.

Эмуляторы различаются по возможностям. Как минимум эмулятору требуется поток байтов инструкций и достаточно памяти для операций со стеком и регистрами процессора. Более сложные эмуляторы могут предоставлять доступ к эмулируемым аппаратным устройствам и службам операционной системы.

Класс Emulator в Ghidra

К счастью, Ghidra предоставляет богатый класс `Emulator`, а также `EmulatorHelper`, который предоставляет более высокоуровневую абстракцию общей функциональности эмулятора и облегчает быстрое и простое создание сценариев эмуляции. В главе 18 мы представили p-code как промежуточное представление базового ассемблера и описали, как это позволяет декомпилятору работать с множеством целевых архитектур. Аналогично p-code поддерживает и функциональность эмулятора, а класс Ghidra `ghidra.pcode.emulate.Emulate` предоставляет возможность эмулировать одну инструкцию p-code.

Мы можем использовать связанные с эмулятором классы Ghidra для построения эмуляторов, позволяющих эмулировать широкий спектр процессоров. Как и другие пакеты и классы Ghidra, эта функциональность документирована в Javadoc, поставляемой с Ghidra, и может быть вызвана как справка щелчком по красному инструменту с плюсом в окне `Script Manager`. Если вас интересует написание эмуляторов, советуем посмотреть Javadoc, связанный с методами эмулятора, используемыми в следующем примере.

Crackme: взломай себя сам

Crackme - это головоломка, созданная реверс-инженерами для реверс-инженеров. Название происходит от взлома ПО для обхода ограничений копирования или использования - одного из более сомнительных применений навыков реверс-инжиниринга. Crackme предоставляют законный способ практиковать эти навыки, а также дают и автору crackme, и человеку, анализирующему crackme, шанс продемонстрировать свой талант.

Распространенный стиль crackme принимает пользовательский ввод, некоторым образом преобразует этот ввод, а затем сравнивает результат преобразования с предварительно вычисленным выводом. Когда вы пытаетесь решить crackme, вам обычно дают только скомпилированный исполняемый файл, содержащий и код, выполняющий преобразование, и конечный вывод для неизвестного ввода. Crackme считается решенным, когда вы выводите ввод, использованный для генерации вывода, содержащегося в бинарном файле; обычно для этого требуется понять преобразование настолько хорошо, чтобы вывести обратную функцию преобразования.

Пример: SimpleEmulator

Предположим, что у нас есть бинарный файл, связанный со следующей задачей crackme, включая некоторое закодированное содержимое в начале файла, которое в итоге служит телом функции. В этом примере мы строим сценарий эмулятора, чтобы автоматизировать процесс декодирования информации, необходимой для решения crackme:

```
unsigned char check_access[] = {
    0xf0, 0xed, 0x2c, 0x40, 0x2c, 0xd8, 0x59, 0x26, 0xd8,
    0x59, 0xc1, 0xaa, 0x31, 0x65, 0xaa, 0x13, 0x65, 0xf8, 0x66
};
unsigned char key = 0xa5;
void unpack() {
    for (int ii = 0; ii < sizeof(check_access); ii++) {
        check_access[ii] ^= key;
    }
}
void do_challenge() {
    int guess;
    int access_allowed;
    int (*check_access_func)(int);
    unpack();
    printf("Введите правильное целое число: ");
    scanf("%d", &guess);
    check_access_func = (int (*)(int))check_access;
    access_allowed = check_access_func(guess);
    if (access_allowed) {
        printf("Доступ разрешен!\n");
    } else {
        printf("Доступ запрещен!\n");
    }
}
int main() {
```

```
    do_challenge();
    return 0;
}
```

Даже при наличии исходного кода это crackme потребовало бы некоторых усилий для решения из-за закодированного содержимого. Декомпилятор Ghidra часто оказывается отличным партнером при решении задач crackme, но у этой задачи есть интересные характеристики, усложняющие процесс. Ghidra видит только закодированное тело функции, а нам нужно знать фактическое назначение функции, прежде чем мы сможем решить задачу. Во время выполнения вызов функции `unpack` приводит к декодированию функции `check_access` до вызова `check_access`. Ответ на это crackme обфусцирован, и мы можем построить сценарий эмулятора в Ghidra, чтобы помочь себе атаковать эту задачу. В отличие от предыдущего примера, этот эмулятор не просто решит проблему для данного конкретного случая, но будет способен эмулировать несколько произвольный код.

Шаг 1: определение задачи

Наша задача - спроектировать и разработать простой эмулятор, который позволит выбрать область дизассемблирования и эмулировать инструкции в этой области. Эмулятор нужно добавить в Ghidra и сделать доступным как сценарий. Например, если мы выберем функцию `unpack` для задачи crackme и запустим сценарий, наш эмулятор должен использовать ключ, чтобы распаковать массив `check_access` и сообщить нам решение задачи crackme. Сценарий запишет распакованные байты кода в память программы в Ghidra.

Шаг 2: создание проекта сценария Eclipse

Мы можем создать проект с именем SimpleEmulator, используя GhidraDev -> New -> Ghidra Script Project. Это дает нам папку SimpleEmulator в Eclipse с папкой Home scripts (см. рисунок 15-16 на странице 325), ожидающей наш новый сценарий. Нам все еще нужно создать фактический сценарий и ввести связанные метаданные, чтобы гарантировать, что наш сценарий документирован и может быть каталогизирован. Метаданные, собранные в диалоге создания сценария, включаются в файл, и, как показывает рисунок 21-7, нам остается сделать только одно: добавить сюда код сценария.

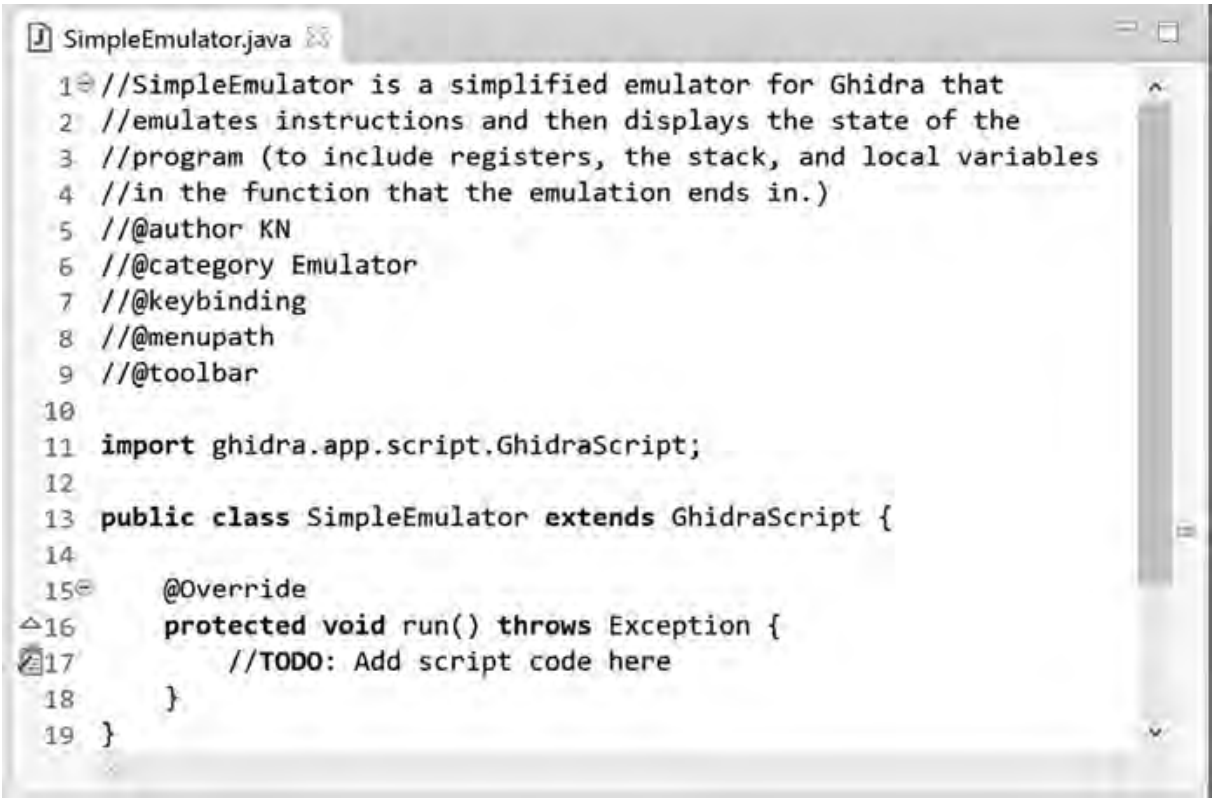


Рисунок 21-7. Шаблон сценария для SimpleEmulator

Шаг 3: построение эмулятора

Мы знаем, что Eclipse предложит импорты, если они понадобятся во время разработки кода, поэтому можно сразу перейти к задачам кодирования, которые нужно выполнить, и добавить рекомендуемые операторы `import`, когда Eclipse обнаружит, что они нужны. Для функциональности мы будем полагаться на следующие объявления переменных экземпляра во всем классе `SimpleEmulator`:

```
private EmulatorHelper emuHelper;   // Объект переменной-члена EmulatorHelper
private Address executionAddress;   // Изначально начало выделения
private Address endAddress;         // Конец выделенной области
```

Комментарии, связанные с каждым объявлением, описывают назначение каждой переменной. `executionAddress` изначально будет установлен в начало выбранного диапазона, но также будет использоваться для продвижения по выделению.

Шаг 3-1: настройка эмулятора

Первое, что мы сделаем в методе `run` сценария, - создадим экземпляр вспомогательного объекта эмулятора и активируем отслеживание любой памяти, записанной в эмуляторе, чтобы можно было записать обновленные значения обратно в текущую программу. Создание экземпляра действует как блокировка, похожая на блокировку, которую `CodeBrowser` устанавливает на открытый бинарный файл:

```
emuHelper = new EmulatorHelper(currentProgram);
emuHelper.enableMemoryWriteTracking(true);
```

Шаг 3-2: выбор диапазона адресов для эмуляции

Поскольку мы хотим, чтобы пользователь выбрал участок кода для эмуляции, нужно убедиться, что он что-то выделил в окне `Listing`. В противном случае мы сгенерируем сообщение об ошибке.

```
if (currentSelection != null) {
    executionAddress = currentSelection.getMinAddress();
    endAddress = currentSelection.getMaxAddress().next();
} else {
    println("Ничего не выбрано");
    return;
}
```

Шаг 3-3: подготовка к эмуляции

Внутри выделения нужно убедиться, что мы смотрим на инструкцию, чтобы установить начальный контекст процессора, инициализировать указатель стека и установить точку останова в конце выбранной области. Флаг `continuing` указывает, только ли мы начинаем эмуляцию или продолжаем ее, и определяет, какая версия `emuHelper.run` вызывается на шаге 3-4:

```
Instruction executionInstr = getInstructionAt(executionAddress);
if (executionInstr == null) {
    printerr("Инструкция не найдена по адресу: " + executionAddress);
    return;
}
long stackOffset = (executionInstr.getAddress().getAddressSpace().
                    getMaxAddress().getOffset() >>> 1) - 0x7fff;
emuHelper.writeRegister(emuHelper.getStackPointerRegister(), stackOffset);
// Установить точку останова на конечном адресе
emuHelper.setBreakpoint(endAddress);
// Установить continuing в false, поскольку эмуляция только начинается
boolean continuing = false;;
```

Шаг 3-4: выполнение эмуляции

В этом разделе вы должны узнать использование некоторых функций API Ghidra, представленных в главе 14 (таких как `monitor.isCancelled`). Нам нужен цикл, который будет вести эмуляцию до достижения заданного нами условия завершения:

```
while (!monitor.isCancelled() &&
       !emuHelper.getExecutionAddress().equals(endAddress)) {
    if (continuing) {
        emuHelper.run(monitor);
    } else {
        emuHelper.run(executionAddress, executionInstr, monitor);
    }
    executionAddress = emuHelper.getExecutionAddress();
    // определить, почему остановился эмулятор, и обработать каждую возможную
    причину
    if (emuHelper.getEmulateExecutionState() ==
        EmulateExecutionState.BREAKPOINT) {
        continuing = true;
    } else if (monitor.isCancelled()) {
        println("Эмуляция отменена по адресу 0x" + executionAddress);
        continuing = false;
    } else {
        println("Ошибка эмуляции по адресу 0x" + executionAddress +
                ": " + emuHelper.getLastError());
        continuing = false;
    }
    writeBackMemory();
    if (!continuing) {
```

```
            break;
        }
    }
```

В этом примере эмуляция продолжается, пока монитор не обнаружил отмену пользователем, пока мы не достигли конца выбранного диапазона инструкций и пока не сработало условие ошибки. Когда эмулятор останавливается, нужно обновить текущий адрес выполнения, а затем соответствующим образом обработать условие остановки. Последний шаг - вызвать метод `writeBackMemory()`.

Шаг 3-5: запись памяти обратно в программу

Реализация `writeBackMemory()` показана здесь. Этот эмулятор будет тестироваться на процедуре `unpack`, которая в итоге изменяет байты в памяти. Изменения памяти, сделанные эмулятором, существуют только в его рабочей памяти. Содержимое нужно записать обратно в бинарный файл, чтобы листинг и другие пользовательские интерфейсы точно отражали изменения, возникающие в результате выполнения инструкций в процедуре `unpack`. Ghidra предоставляет функциональность внутри своего `emulatorHelper`, чтобы облегчить этот процесс.

```
private void writeBackMemory() {
    AddressSetView memWrites = emuHelper.getTrackedMemoryWriteSet();
    AddressIterator aIter = memWrites.getAddresses(true);
    Memory mem = currentProgram.getMemory();
    while (aIter.hasNext()) {
        Address a = aIter.next();
        MemoryBlock mb = getMemoryBlock(a);
        if (mb == null) {
            continue;
        }
        if (!mb.isInitialized()) {
            // инициализировать память
            try {
                mem.convertToInitialized(mb, (byte)0x00);
            } catch (Exception e) {
                println(e.toString());
            }
        }
        try {
            mem.setByte(a, emuHelper.readMemoryByte(a));
        } catch (Exception e) {
            println(e.toString());
        }
    }
}
```

Шаг 3-6: очистка ресурсов

На этом шаге нужно очистить ресурсы и освободить блокировку, которую мы держим на текущей программе. И то и другое можно выполнить одним простым оператором:

```
emuHelper.dispose();
```

Поскольку этот эмулятор предназначен только для демонстрационных целей, мы позволили себе некоторые вольности в том, что было включено в сценарий. Чтобы сэкономить место, мы минимизировали комментарии, функциональность, проверку ошибок и обработку ошибок, которые обычно включили бы в производственный сценарий. Остается только подтвердить, что наш сценарий эмулятора способен выполнить нашу цель.

Шаг 4: добавление сценария в нашу установку Ghidra

Добавление сценария в нашу установку Ghidra требует лишь поместить его туда, где Ghidra его найдет. Если вы настроили проект сценария как связанный проект, Ghidra уже знает, где его искать. Если вы не связали проект сценария (или создали сценарий эмулятора в другом редакторе), нужно сохранить его в одном из каталогов сценариев Ghidra, как обсуждалось в главе 14.

Шаг 5: тестирование сценария внутри Ghidra

Чтобы протестировать сценарий, мы загрузим бинарный файл, связанный с исходным кодом задачи crackme. Когда мы загружаем бинарный файл и переходим к функции `unpack`, мы замечаем, что она содержит ссылки на метку `check_access`:

```
0010077d 48 8d 05 8c 08 20 00  LEA    RAX,[check_access]
```

Код в окне `Decompiler` содержит следующее, что никак не приближает нас к решению crackme:

```
check_access[(int)local_c] = check_access[(int)local_c] ^ key;
```

Двойной щелчок по `check_access` в окне `Listing` приводит нас к адресу `00301010`, который не выглядит как инструкции внутри функции.

```
00301010 f0 ed 2c 40 2c d8 59  undefined1[19]
         26 d8 59 c1 aa 31 65
         aa 13 65 f8 66
```

Если бы мы решили дизассемблировать это содержимое, мы получили бы в Ghidra ошибку плохих данных. Окно `Decompiler` также ничем не помогает для этого места. Поэтому используем наш сценарий и посмотрим, можем ли мы эмулировать функцию `unpack`. Мы выбираем инструкции, составляющие функцию `unpack`, открываем `Script Manager` и запускаем наш сценарий. Мы не видим наблюдаемых изменений в функции `unpack` или в окне `Decompiler`. Но если перейти к `check_access` (`00301010`), содержимое изменилось!

```
00301010 55 48 89 e5 89 7d     undefined1[19]
         fc 83 7d fc 64 0f
         94 c0 0f b6 c0 5d c3
```

Мы можем очистить эти байты кода (горячая клавиша C), а затем дизассемблировать (горячая клавиша D) и получить следующие результаты:

```
     check_access
00301010 55                    PUSH   RBP
00301011 48 89 e5              MOV    RBP,RSP
00301014 89 7d fc              MOV    dword ptr [RBP + -0x4],EDI
00301017 83 7d fc 64           CMP    dword ptr [RBP + -0x4],100
0030101b 0f 94 c0              SETZ   AL
0030101e 0f b6 c0              MOVZX  EAX,AL
00301021 5d                    POP    RBP
00301022 c3                    RET
```

Вот соответствующий код в окне `Decompiler`:

```
ulong UndefinedFunction_00301010(int param_1)
{
  return (ulong)(param_1 == 100);
}
```

Это был всего лишь экспериментальный сценарий, демонстрирующий использование эмуляторов для помощи в деобфускации кода, но он показывает, как с помощью классов поддержки эмулятора Ghidra можно построить относительно универсальный эмулятор. Есть и другие ситуации, где разработка и использование эмуляторов являются подходящим курсом действий. Непосредственное преимущество эмуляции перед отладкой состоит в том, что потенциально вредоносный код никогда фактически не выполняется эмулятором, тогда как деобфускация с помощью отладчика должна разрешить выполнение хотя бы некоторой части вредоносной программы, чтобы получить деобфусцированную версию программы.

Резюме

В наши дни обфусцированные программы являются скорее правилом, чем исключением, когда речь идет о вредоносном ПО. Любые попытки изучить внутреннюю работу образца вредоносного ПО почти наверняка потребуют какого-либо типа деобфускации. Используете ли вы динамический подход к деобфускации с помощью отладчика или предпочитаете не запускать потенциально вредоносный код и вместо этого выбираете сценарии или эмуляцию для деобфускации бинарных файлов, конечная цель - получить деобфусцированный бинарный файл, который можно полностью дизассемблировать и правильно проанализировать.

В большинстве случаев этот финальный анализ будет выполняться с помощью инструмента вроде Ghidra. С учетом этой конечной цели (использовать Ghidra для анализа) имеет смысл попытаться использовать Ghidra от начала до конца. Приемы, представленные в этой главе, предназначены для демонстрации того, что Ghidra способна на гораздо большее, чем простая генерация дизассемблированных листингов, и мы будем развивать это в следующей главе, когда посмотрим, как можно использовать Ghidra для исправления наших дизассемблированных листингов.

Сноски

[1] [назад] За эти годы было сделано несколько презентаций, связанных с Shiva, начиная с этой: http://cansecwest.com/core03/shiva.ppt.
[2] [назад] Представьте семафор как жетон, который должен быть у вас, прежде чем вы сможете войти в комнату для выполнения действия. Пока вы держите жетон, никто другой не может войти в комнату. Когда вы завершили свою задачу в комнате, вы можете выйти и передать жетон кому-то еще, кто затем сможет войти в комнату и воспользоваться проделанной вами работой (без вашего ведома, потому что вас больше нет в комнате). Семафоры часто используются для обеспечения блокировок взаимного исключения вокруг кода или данных в программе.
[3] [назад] Дополнительную информацию о структурированной обработке исключений Windows (SEH) см. в http://bytepointer.com/resources/pietrek_crash_course_depths_of_win32_seh.htm.
[4] [назад] Windows настраивает регистр `FS`, чтобы он указывал на базовый адрес блока окружения текущего потока (TEB). Первое поле в TEB - это голова связанного списка указателей на функции обработчиков исключений, которые вызываются по мере необходимости, когда в процессе возбуждается исключение.
[5] [назад] В архитектуре x86 отладочные регистры 0-7 (`DR0-DR7`) используются для управления использованием аппаратно поддерживаемых точек останова. `DR0-DR3` используются для задания адресов точек останова, тогда как `DR6` и `DR7` используются для включения и отключения конкретных аппаратных точек останова.
[6] [назад] См. http://www.hick.org/code/skape/papers/win32-shellcode.pdf, в частности главу 3, "Shellcode Basics", и раздел 3.3, "Resolving Symbol Addresses".
[7] [назад] Многие функции Windows, принимающие строковые аргументы, существуют в двух версиях: одна принимает строки ASCII, а другая принимает строки Unicode. ASCII-версии этих функций имеют суффикс `A`, тогда как Unicode-версии имеют суффикс `W`.

[8] [назад] Хеш-функция - это математический процесс, способный получить результат фиксированного размера (например, 4 байта) из входа произвольного размера (например, строки).
[9] [назад] OUI составляет первые 3 байта назначенного производителем MAC-адреса сетевого адаптера.
[10] [назад] See https://web.archive.org/web/20041130172213/http://invisiblethings.org/papers/redpill.html.
[11] [назад] См. https://docs.microsoft.com/en-us/sysinternals/downloads/procmon для Process Monitor и http://www.wireshark.org/ для Wireshark.
[12] [назад] См. https://www.apriorit.com/dev-blog/367-anti-reverse-engineering-protection-techniques-to-use-before-releasing-software.
[13] [назад] Младшие 8 бит регистра `EAX` также называются регистром `AL`.

Глава 22. Исправление бинарных файлов

Иногда при реверс-инжиниринге бинарного файла вы можете решить, что хотите изменить поведение исходного бинарного файла. Изменение поведения обычно достигается исправлением бинарного файла, чтобы вставить, удалить или изменить существующие инструкции. Существует много мотивов для таких изменений - некоторые более спорные, чем другие, - включая следующие:

- Изменение образца вредоносного ПО, чтобы устранить антиотладочные приемы, мешающие изучать вредоносное ПО
- Исправление уязвимостей в ПО, для которого у вас нет исходного кода
- Настройка заставки приложения или строкового содержимого
- Изменение игровой логики для целей читерства
- Разблокировка скрытых возможностей
- Обход проверок лицензирования или других защит от пиратства

В этой главе мы не собираемся учить вас делать что-либо неэтичное, но обсудим высокоуровневые трудности изменения бинарного файла так, чтобы отразить любые изменения, внесенные вами в Ghidra. Глава 14 представила API-функцию `setByte`, а глава 21 показала, как разные стили сценариев эмуляции смогли изменять содержимое программы, загруженной в Ghidra. Эти приемы изменяют содержимое, импортированное в Ghidra, и никак не влияют на исходный бинарный файл, который Ghidra обработала во время процесса импорта. Чтобы завершить процесс исправления, вы узнаете, как заставить Ghidra записать изменения обратно в файл на диске. Мы также обсудим трудности, которые могут создавать разные типы патчей.

Планирование патча

Процесс исправления обычно включает следующие шаги:

1. Определите тип патча, который намерены сделать. Это часто определяется вашей причиной для исправления, как обсуждалось выше.
2. Определите точное место или места программы, которые нужно исправить. Обычно это требует некоторого исследования и анализа исправляемой программы.
3. Спланируйте содержимое патча. Изменения содержимого могут требовать новых данных, нового машинного кода или и того и другого. В любом случае ваши изменения должны быть хорошо продуманы, чтобы предотвратить непреднамеренное поведение программы.
4. Используйте Ghidra, чтобы заменить существующее содержимое программы (данные или код) вашим заменяющим содержимым.
5. Используйте Ghidra, чтобы проверить, что изменения выглядят корректно реализованными.
6. Используйте Ghidra, чтобы экспортировать изменения в новый бинарный файл.
7. Проверьте, что новый бинарный файл ведет себя как задумано, при необходимости повторяя с шага 2.

В некоторых сценариях исправления многие из этих шагов будут почти тривиальными; в других они будут гораздо сложнее. В следующих разделах мы рассмотрим те шаги, с которыми может помочь Ghidra, и обсудим ситуации, которые могут довести вас или Ghidra до предела возможностей. Начнем с шага 2 и рассмотрим некоторые способы, которыми Ghidra помогает находить интересующие элементы в контексте исправления.

Поиск того, что нужно изменить

Точная природа вашего патча будет определять, что нужно исправлять. Настройка заставок или строк требует найти исходные данные, которые нужно изменить. Изменение логики программы требует модификации или вставки кода, чтобы изменить поведение программы. В этом случае может потребоваться значительный объем реверс-инжиниринга уже просто для поиска мест программы, которые нужно изменить. Многие возможности Ghidra, облегчающие эти действия, были рассмотрены в предыдущих главах. Рассмотрим некоторые возможности, полезные для исправления.

Поиск в памяти

Когда ваш патч включает изменение данных программы, основным средством определения места применения патчей будет та или иная форма поиска в памяти. Самый общий поиск в памяти - пункт меню `Search -> Memory` в `CodeBrowser` (горячая клавиша S), показанный на рисунке 22-1 (с раскрытыми параметрами Advanced). Диалог Search Memory ранее обсуждался в главе 6.

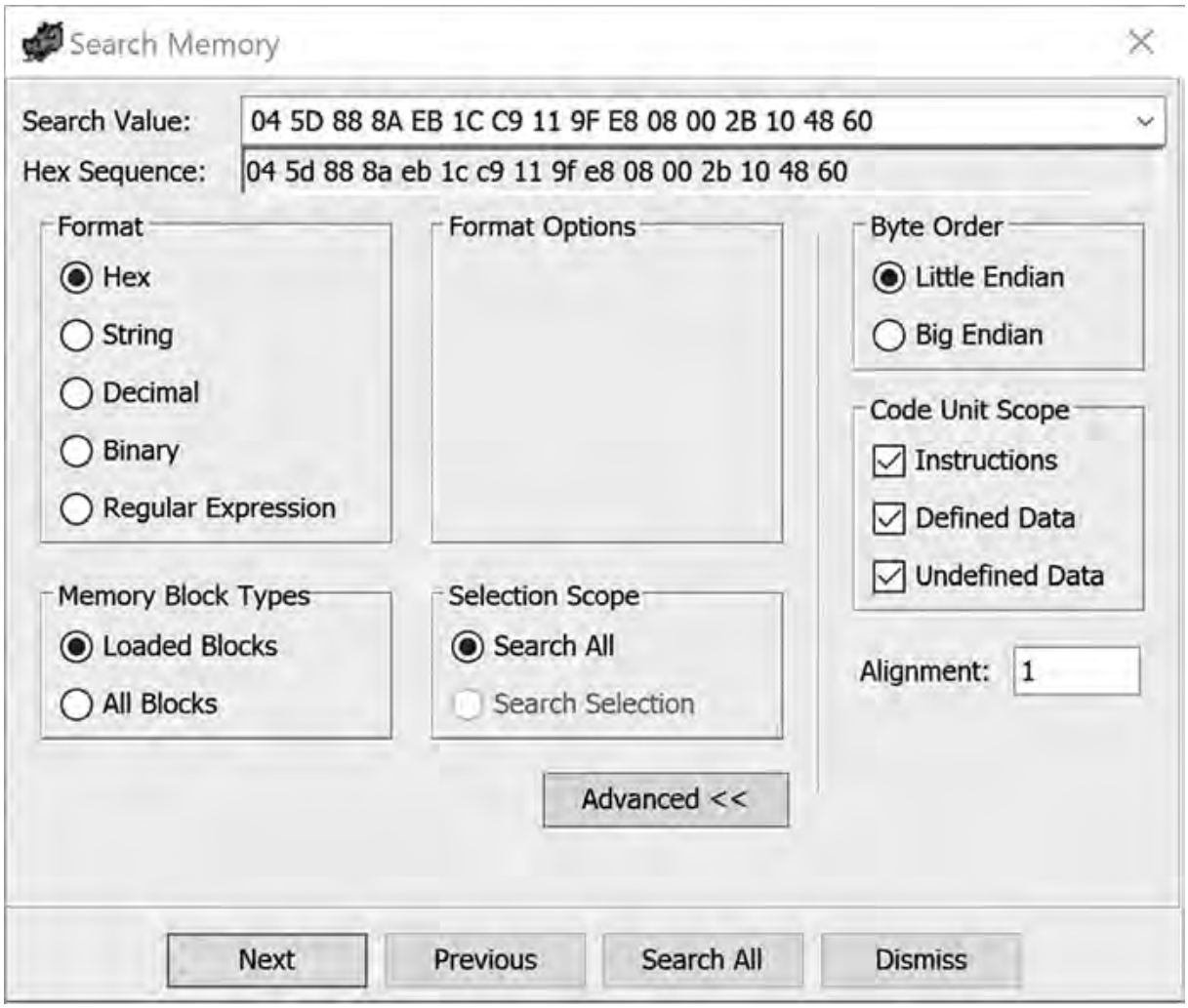


Рисунок 22-1. Диалог Search Memory

Диалог Search Memory наиболее полезен в контексте исправления, когда вы ищете конкретные известные данные внутри бинарного файла, например известные строки или шестнадцатеричные последовательности. Успешный поиск переместит все связанные отображения к местоположению

совпавших байтов или, в случае Search All, откроет новый диалог со списком всех адресов, по которым найдено совпадающее содержимое. Для очень больших бинарных файлов может быть полезно ограничить область поиска конкретными регионами (Instructions, Defined Data, Undefined Data и т. д.) внутри программы, которые, вероятно, содержат совпадение, сняв выбор с неинтересующих типов единиц кода.

Примечание. Хотя Search -> Memory предоставляет наиболее настраиваемую универсальную возможность поиска в Ghidra, это поиск по сырому байтовому содержимому базы данных, и другие типы поиска могут лучше подходить для типа данных, которые вы ищете. Например, Search -> Memory - неправильный выбор, если вы хотите искать внутри текста комментариев, которые внесли в программу. Дополнительную информацию о поиске внутри самого дизассемблированного листинга см. в разделе "Search Program Text" на странице 115.

Поиск прямых ссылок

В главе 20 мы использовали Search -> For Direct References, чтобы просканировать бинарное содержимое программы на все вхождения конкретного адреса. Наиболее распространенное использование этого типа поиска - нахождение указателей на интересные данные, когда Ghidra не смогла создать перекрестную ссылку на данные. В контексте исправления это чаще всего используется для полного понимания и обновления всех ссылок на место данных или кода, чтобы сохранить правильные отношения между кодом и данными в исправленном бинарном файле.

Поиск шаблонов инструкций

Возможность Ghidra Search -> For Instruction Patterns находит конкретную последовательность инструкций, сопоставляя шаблон. При определении шаблона инструкций нужно найти тонкий баланс между слишком конкретными и слишком общими шаблонами. Рассмотрим пример, иллюстрирующий эту идею. Предположим, у нас есть листинг, включающий функцию `cleanup_and_exit`, которая завершает программу:

```
int test_even(int v) {
    return (v % 2 == 0);
}
int test_multiple_10(int v) {
    return (v % 10 == 0);
}
int test_lt_100(int v) {
    return v < 100;
}
int test_gte_20(int v) {
    return v >= 20;
}
void cleanup_and_exit(int rv, char* s) {
    printf("Result: %s\n", s);
    exit(rv);
}
void do_testing() {
    int v;
    srand(time(0));
    v = rand() % 150;
    printf("Проверка %d\n", v);
    if (!test_even(v)) {
        cleanup_and_exit(-1, "проверка четности не пройдена");
    }
    if (test_multiple_10(v)) {
        cleanup_and_exit(-2, "проверка на некратность 10 не пройдена");
    }
    if (!test_lt_100(v)) {
```

```
        cleanup_and_exit(-3, "проверка <100 не пройдена");
    }
    if (!test_gte_20(v)) {
        cleanup_and_exit(-4, "проверка > 20 не пройдена");
    }
    // все проверки пройдены, здесь выполняется полезная работа
    system("/bin/sh");
    cleanup_and_exit(0, "успех!");
}
int main() {
    do_testing();
    return 0;
}
```

Функция `do_testing` выполняет серию проверок. Если какая-либо проверка завершается неудачно, вызывается функция `cleanup_and_exit`, и выполнение заканчивается. Если все проверки успешны, будет выполнен некоторый очень интересный код. Наша задача исправления - определить, где нужно внести патч, чтобы гарантировать прохождение всех проверок и добраться до интересного кода.

Если загрузить бинарный файл в Ghidra, можно найти все вызовы `cleanup_and_exit`, чтобы определить, что нужно исправить для прохождения всех проверок независимо от их числа. Нужно рассмотреть несколько вариантов:

- Можно просто перейти к этой функции и исправить ее так, чтобы она возвращалась, чтобы неудачная проверка не завершала программу, а продолжала выполнение. Это не оптимальное решение, потому что функция также используется для законного выхода в конце программы после выполнения интересной работы.
- Можно использовать функциональность поиска или XREF к `cleanup_and_exit`. Это даст нам все вызовы, но мы хотим исправить только некоторые из них.
- Можно определить общий для вызовов шаблон инструкций и использовать Search -> For Instruction Patterns, чтобы найти правильные вызовы для исправления.

Чтобы использовать эту функциональность поиска, нужно определить полезный шаблон. Каждая проверка, которую мы пытаемся пройти, принимает в окне `Listing` следующую форму:

```
001008af  CALL   test_even
001008b4  TEST   EAX,EAX
001008b6  JNZ    LAB_001008c9
001008b8  LEA    RSI,[s_failed_even_test_00100a00]
001008bf  MOV    EDI,0xffffffff
001008c4  CALL   cleanup_and_exit
```

Попробуем найти эту последовательность, выбрав последовательность инструкций и Search -> For Instruction Patterns. Это автоматически заполнит диалог Instruction Pattern Search, как показано на рисунке 22-2.

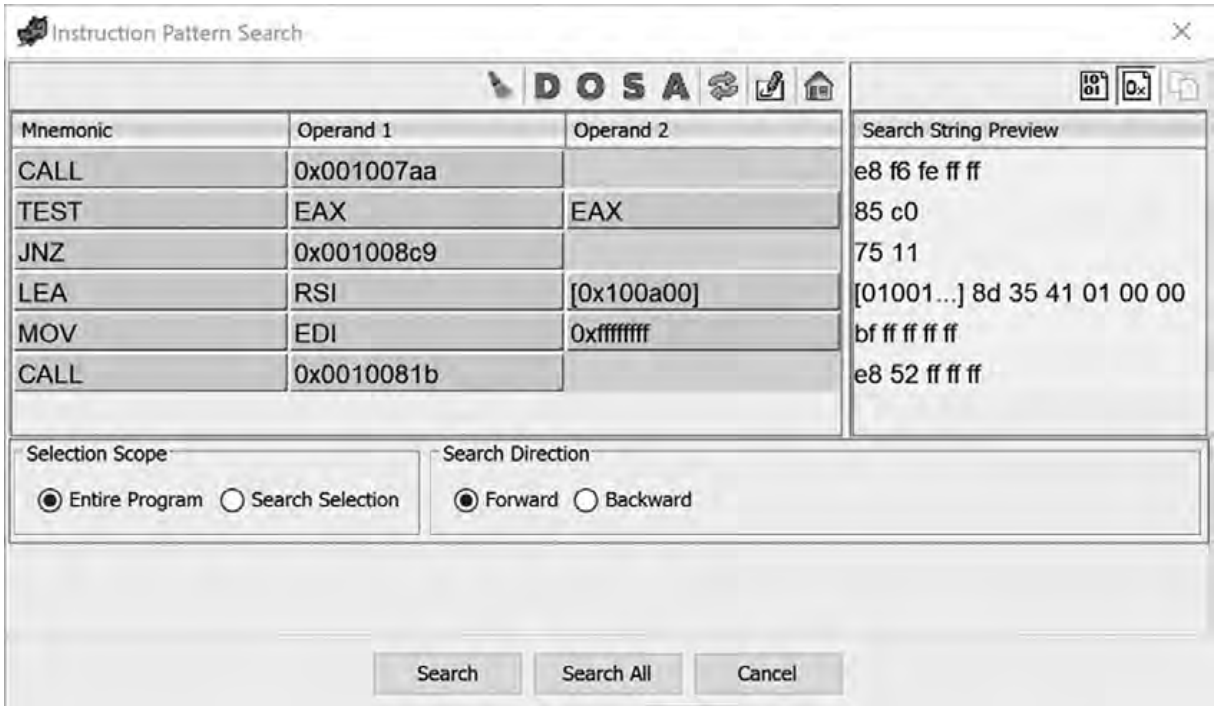


Рисунок 22-2. Диалог Instruction Pattern Search со всеми выбранными полями

Если щелкнуть Search All, мы увидим только один результат (конкретное место, выбранное при запуске поиска), как показано на рисунке 22-3.

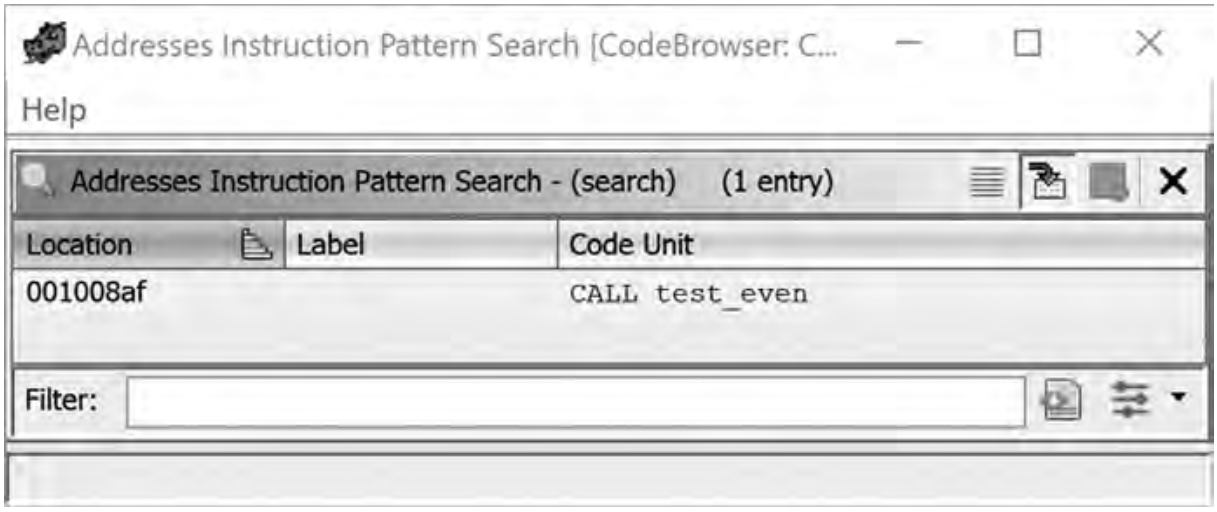


Рисунок 22-3. Результаты поиска шаблона инструкций со всеми выбранными полями

Проблема в том, что мы включаем операнды, которые не остаются постоянными между тестовыми случаями. Например, операнд первого вызова - адрес конкретной тестовой функции. Можно снять выбор с отдельных компонентов (мнемоник и операндов) любой инструкции в шаблоне, чтобы сделать его более общим, как показано на рисунке 22-4. Все, с чего снят выбор, рассматривается как подстановочный шаблон в последующих поисках.

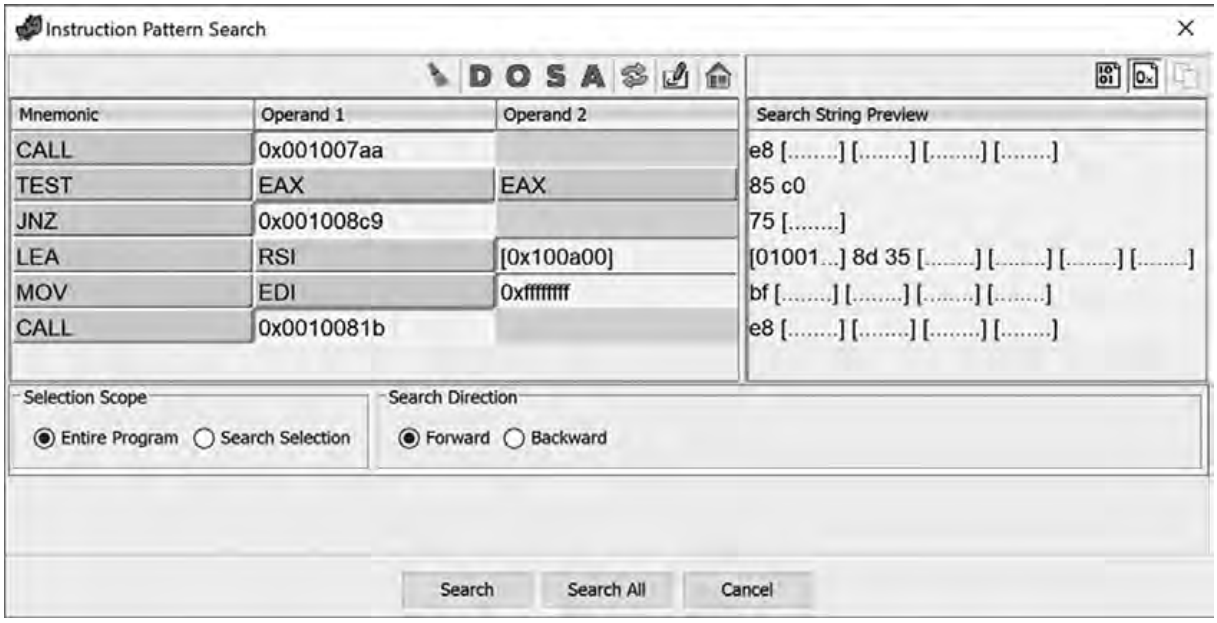


Рисунок 22-4. Диалог Instruction Pattern Search с отключенными некоторыми операндами

Если щелкнуть Search All с отключенными полями операндов, мы увидим три результата, показанные на рисунке 22-5.

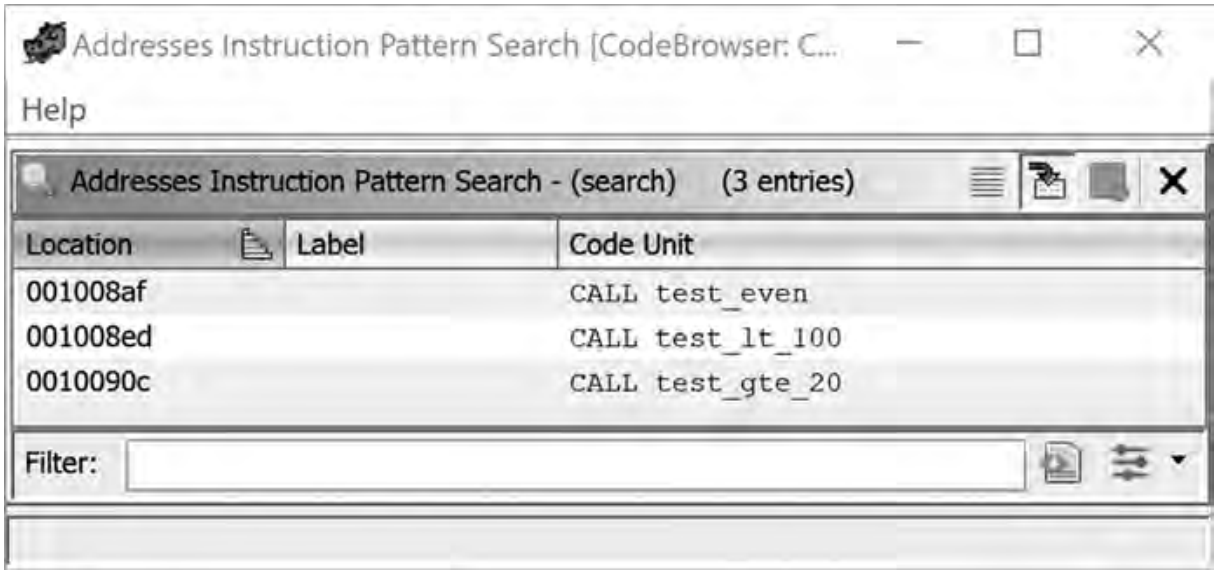


Рисунок 22-5. Результаты поиска шаблона инструкций с отключенными некоторыми операндами

Поиск все еще не может идентифицировать вызов `test_multiple_10`, который использует инструкцию `JZ`, а не `JNZ`. Снятие выбора с поля мнемоники для инструкции `JNZ` и повторный запуск поиска дают результаты, показанные на рисунке 22-6: они включают четыре вызова, которые мы хотим исправить, и не включают финальный вызов `cleanup_and_exit`, который мы исправлять не хотим.

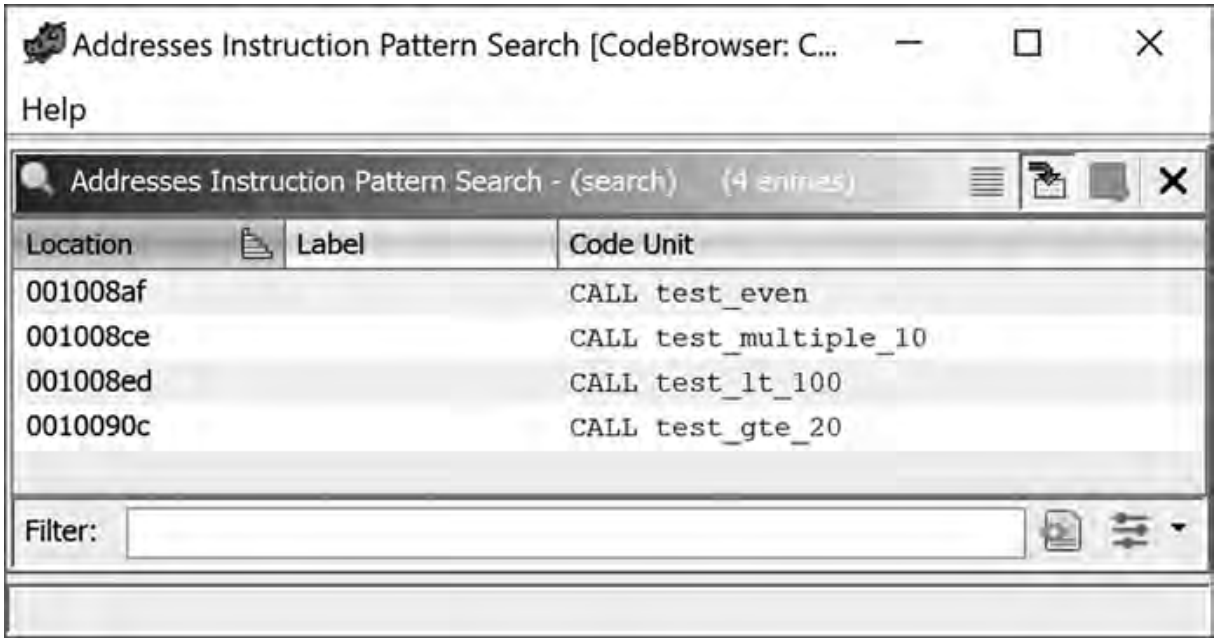


Рисунок 22-6. Результаты поиска шаблона инструкций с отключенными JNZ и некоторыми операндами

Эта функциональность поиска имеет ряд применений помимо нахождения кандидатных шаблонов инструкций для исправления. Ее можно использовать для анализа уязвимостей, поиска конкретной функциональности и других поисков, идентифицирующих шаблоны инструкций, важные для реверс-инженеров.

Поиск конкретного поведения

Поведение программы определяется инструкциями, которые она выполняет, в сочетании с данными, над которыми она выполняется. Когда задача исправления включает изменение поведения программы, найти точное поведение, которое вы хотите изменить, обычно гораздо труднее, чем найти данные, которые вы хотите изменить. Поскольку мы никогда не можем предсказать точную последовательность инструкций, которую компилятор может сгенерировать для любого исходного кода, трудно использовать автоматизированные возможности поиска Ghidra, чтобы точно указать место применения патча кода. Поиск конкретного поведения сводится к обычному анализу функций программы с использованием приемов, рассмотренных во всей этой книге.

Помимо тщательного анализа всех функций в бинарном файле или аккуратного обхода дерева вызовов, начиная с хорошо известной функции вроде `main`, два наиболее распространенных приема идентификации интересующих функций - опора на имя функции (если предположить, что в бинарном файле есть символы) и использование перекрестных ссылок от "интересных" данных для обратного движения к потенциально интересным функциям. Например, если нас интересует нахождение функций, связанных с аутентификацией, внутри бинарного файла, можно искать распространенные строки, связанные с аутентификацией, такие как `"Please enter your password:"` и `"Authentication failed"`. Строки, похожие на эти, часто обрамляют процесс аутентификации, и нахождение функций, ссылающихся на эти строки, может значительно сократить пространство поиска других функций, связанных с аутентификацией.

И здесь природа данных, которые могут привести вас к интересным функциям, будет зависеть от конкретного сценария исправления. Независимо от подходов, используемых для поиска функции-кандидата для патча, всегда следует проверять, что функция действительно реализует

поведение, которое вы хотите изменить. В частности, всегда следует с осторожностью относиться к именам, которые программисты назначают функциям, поскольку нет требования, чтобы поведение функции соответствовало ее имени.

Применение патча

Наконец ваши усердная работа и настойчивость окупились, и вы нашли код или данные, которые хотите изменить. Что теперь? Предположим, вы уже разработали заменяющее содержимое, которое намерены внести в бинарный файл, и точно знаете, куда хотите его поместить. Значит, пора задействовать возможности Ghidra, изменяющие программу.

Первое, что нужно учесть, - размер нового содержимого относительно содержимого, которое вы заменяете. Если размер нового содержимого меньше или равен размеру исходного содержимого, вы в хорошем положении, потому что патч поместится в область памяти исходного содержимого. Однако все становится немного сложнее, когда патч больше исходного содержимого, и вскоре мы уделим этому случаю отдельное внимание.

Внесение базовых изменений

Есть ли у вас на руках набор байтов или вам нужна помощь ассемблера, в итоге вам нужно будет поместить содержимое в Ghidra. Для коротких последовательностей байтов может быть проще использовать встроенный байтовый редактор или ассемблер Ghidra. Для более длинных последовательностей, вероятно, захочется автоматизации. Следующие несколько разделов описывают некоторые возможности Ghidra для редактирования на уровне байтов.

Byte Viewer

Ghidra `Byte Viewer` (`Window -> Bytes`), показанный на рисунке 22-7, предоставляет стандартное представление шестнадцатеричного дампа сырого байтового содержимого в текущем месте листинга, синхронизированное со всеми другими связанными окнами.

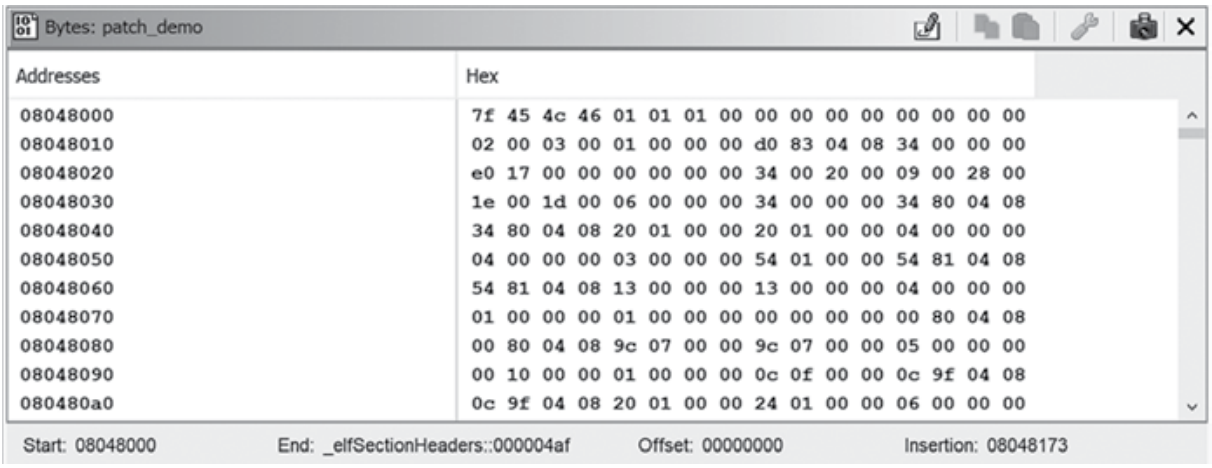


Рисунок 22-7. Ghidra `Byte Viewer`

`Byte Viewer` также может служить hex-редактором при переключении инструмента Edit Mode, и это удобный вариант, когда нужно изменить несколько байтов за раз.

К сожалению, Ghidra не позволит редактировать байты, являющиеся частью существующей инструкции. Обход этого ограничения - очистить связанную инструкцию в окне `Listing` (щелкните правой кнопкой `Clear Code Bytes` или нажмите горячую клавишу C). Инструмент `Byte Viewer Options` используется для открытия диалога, показанного на рисунке 22-8, который позволяет настроить отображение `Byte Viewer`.

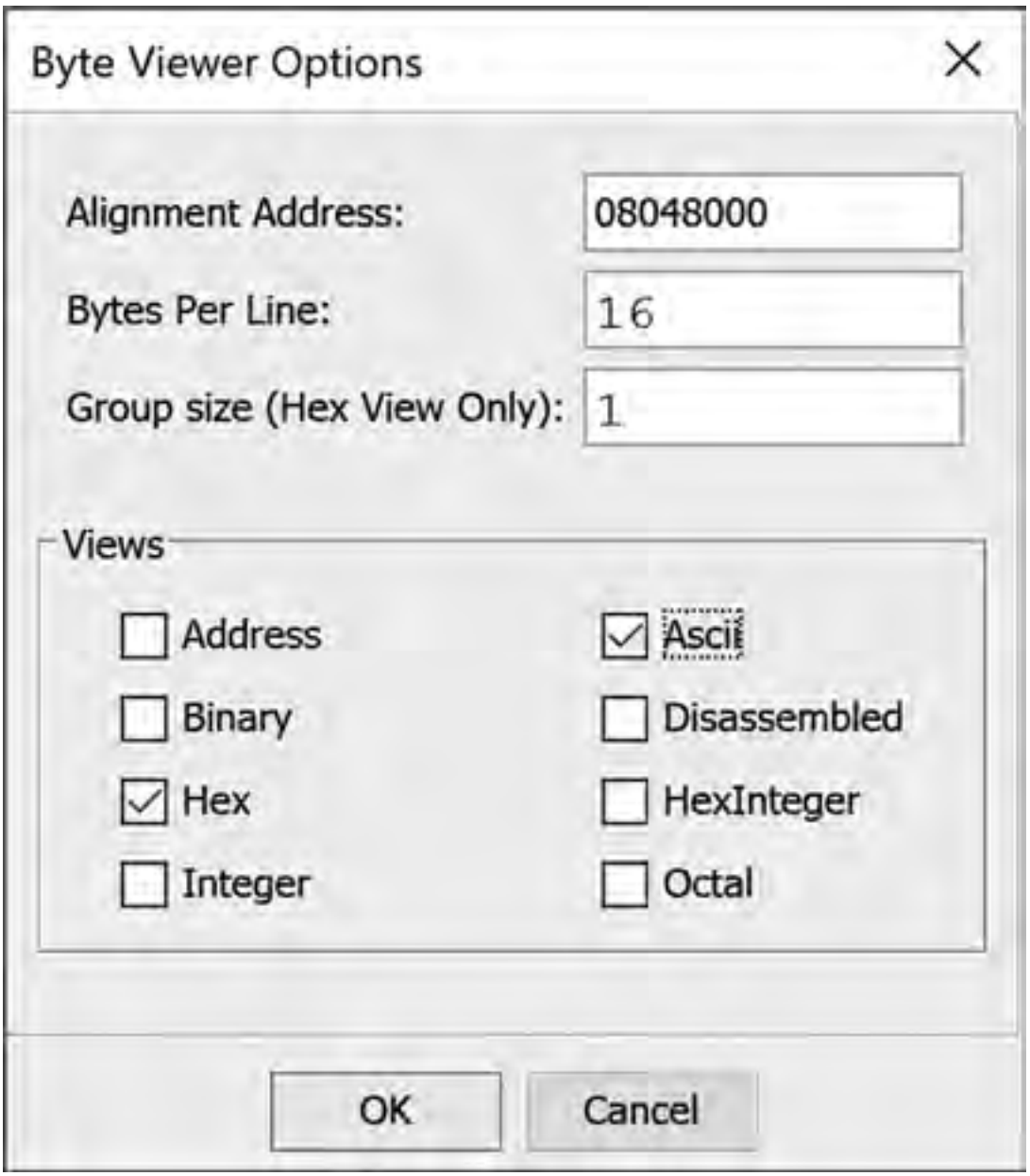


Рисунок 22-8. Диалог `Byte Viewer Options`

Выбор параметра Ascii добавляет ASCII dump в `Byte Viewer` (см. рисунок 22-9), который затем служит ASCII-редактором в режиме редактирования.

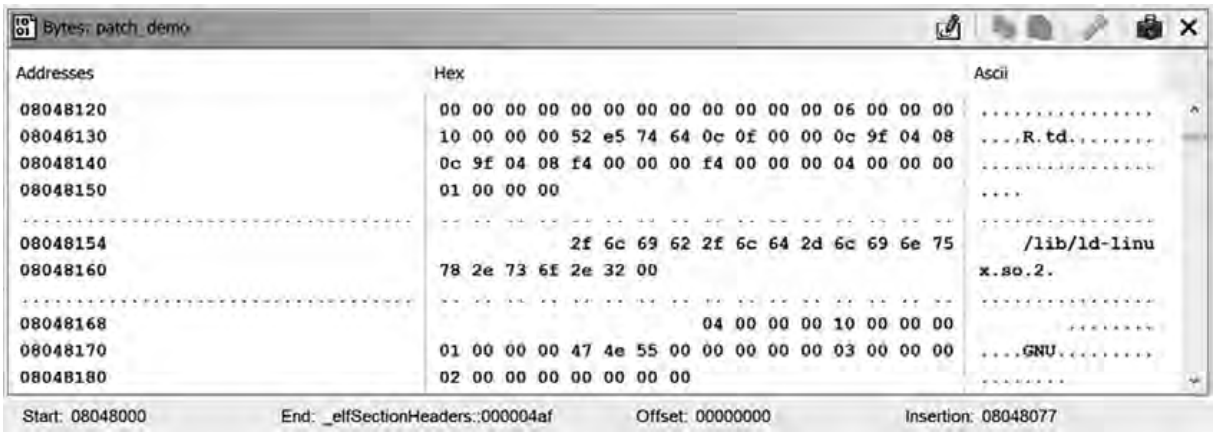


Рисунок 22-9. `Byte Viewer` *с включенным ASCII dump*

Закончив ввод новых значений, следует выйти из Edit Mode и вернуться в окно `Listing`, чтобы проверить правильность изменений.

Сценарное внесение изменений

Если патч не очень короткий, наиболее эффективное средство изменения исходных байтов в Ghidra - поручить это сценарию. При наличии патча в форме массива байтов и начального адреса патча следующая функция применяет патч внутри Ghidra:

```
public void patchBytes(Address start, byte[] patch) throws Exception {
    Address end = start.add(patch.length);
    clearListing(start, end);
    setBytes(start, patch);
}
```

Можно включить эту функцию в сценарий, который создает массив байтов патча из выбранного вами источника (например, объявляя инициализированный массив или загружая содержимое файла). Вызов `clearListing` необходим, поскольку Ghidra не позволит изменять байты, являющиеся частью существующей инструкции или элемента данных. После завершения сценария нужно вручную отформатировать исправленные байты как код или данные и проверить правильность патча.

Использование ассемблера

Когда вы хотите исправить код в бинарном файле, вы, скорее всего, будете мыслить в терминах замены одной инструкции языка ассемблера другой (например, замены `CALL _exit` на `NOP`), что не обязательно неверно, но склонно сглаживать некоторые сложности, связанные с исправлением кода. Когда придет время фактически применить патч к программе, нельзя вставить заменяющие операторы языка ассемблера; вместо этого нужно вставить соответствующие байты машинного кода, а значит, вероятно, понадобится ассемблер для генерации машинно-кодовых версий всех заменяющих инструкций.

Один подход - использовать внешний редактор, чтобы написать заменяющие ассемблерные операторы, собрать их внешним ассемблером (например, `nasm` или `as`), извлечь сырой машинный код[1] и наконец внести его в программу, возможно с помощью сценария, как обсуждалось ранее. Альтернативный подход - использовать встроенную возможность ассемблера Ghidra, доступную через щелчок правой кнопкой по любой инструкции и выбор пункта меню `Patch Instruction`.

Подобно тому как спецификации SLEIGH говорят Ghidra, как переводить машинный код в язык ассемблера, они также позволяют Ghidra выполнять перевод из ассемблера в машинный код, то есть действовать как ассемблер. При первом выборе параметра `Patch Instruction` для данной архитектуры Ghidra построит ассемблер на основе спецификации SLEIGH этой архитектуры. Сначала вам будет показано сообщение, похожее на приведенное на рисунке 22-10.

Рисунок 22-10. Диалог Assembler Rating

Разработчики Ghidra провели тесты точности ассемблеров, сгенерированных Ghidra. Если ассемблер процессора был протестирован, ему назначается один из следующих рейтингов (в порядке убывания точности): `platinum`, `gold`, `silver`, `bronze` и `poor`. Все непроверенные ассемблеры помечаются как `unrated`. Дополнительную информацию о рейтингах ассемблеров Ghidra, а также текущий рейтинг всех доступных ассемблеров можно найти в `Ghidra Help`.

После закрытия диалога Assembler Rating Ghidra строит требуемую возможность ассемблера из текущей спецификации SLEIGH процессора. Пока вы ждете построения ассемблера, Ghidra отображает информативный диалог ожидания, похожий на показанный на рисунке 22-11.

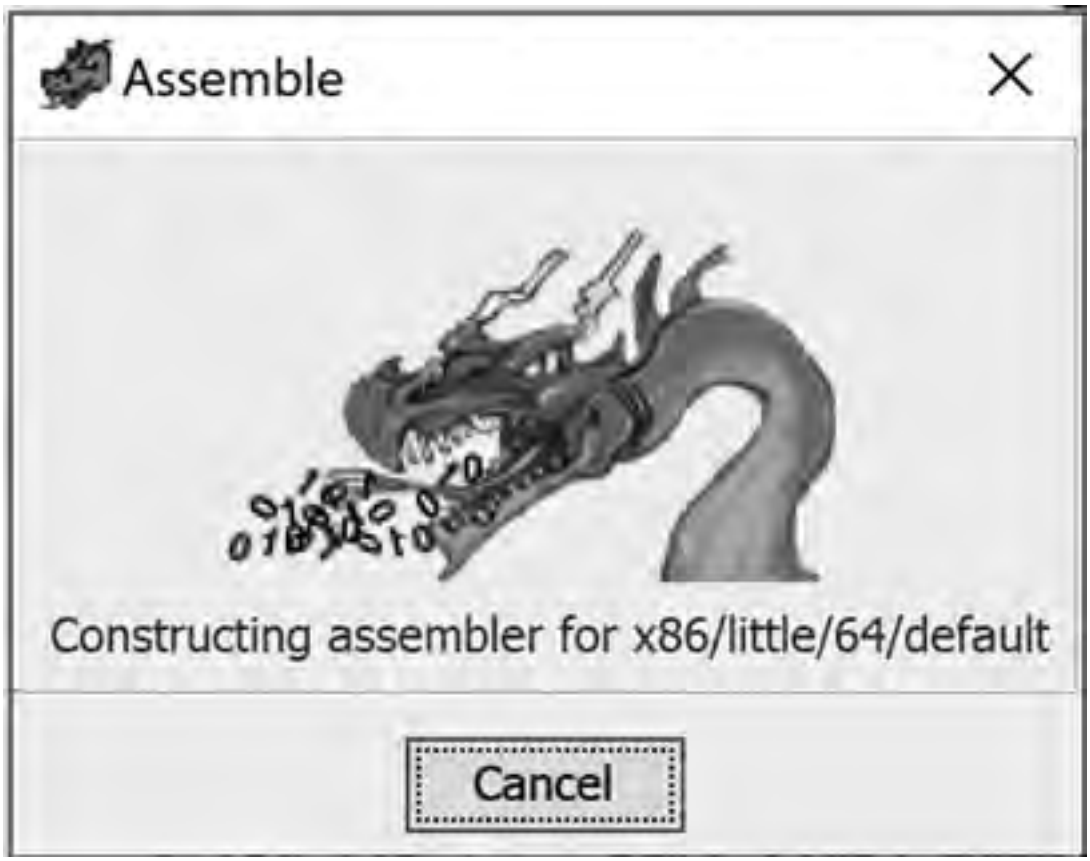


Рисунок 22-11. Диалог ожидания Assemble

Когда ассемблер построен, Ghidra заменяет выбранную инструкцию в окне `Listing` двумя полями ввода текста (см. рисунок 22-12), позволяющими редактировать мнемонику и операнды инструкции. Клавиша `esc` отбрасывает изменения до их сборки, тогда как клавиша `enter` собирает новую инструкцию и заменяет байты машинного кода старой инструкции байтами новой инструкции.

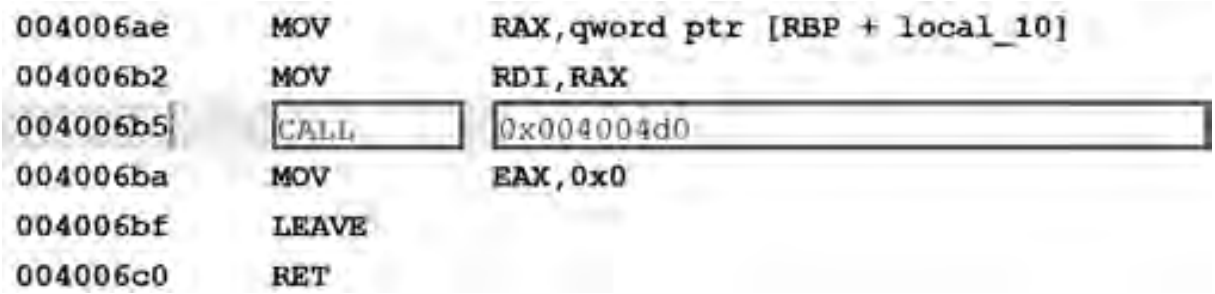


Рисунок 22-12. Сборка новой инструкции

Поскольку они происходят из той же спецификации, что и соответствующий дизассемблер, ассемблеры Ghidra распознают тот же синтаксис ассемблера, который используется в окне Ghidra `Listing`. Ассемблеры Ghidra чувствительны к регистру и предоставляют варианты автодополнения при вводе новых инструкций. После ввода инструкции Ghidra возвращает вас к обычному виду окна `Listing`, и можно снова выбрать `Patch Instruction`, если есть дополнительные инструкции для изменения. Для коротких патчей ассемблер Ghidra предлагает удобный способ одновременно собрать инструкции и изменить программу.

Ловушки замены инструкций

Хотя ассемблер Ghidra быстро справляется с изменением одной инструкции, новая заменяющая инструкция может быть короче, длиннее или того же размера, что и старая инструкция. Третий случай, в котором заменяющая и исходная инструкции имеют одинаковый размер, неинтересен. (Первые две проблемы могут возникнуть только на архитектурах без фиксированного размера инструкции, таких как x86.)

Рассмотрим первый случай, в котором заменяющая инструкция короче исходной, как отражено в следующем листинге:

```
;BEFORE:
0804851b  83 45 f4 01   ADD    dword ptr [EBP + local_10],0x1
0804851f  83 45 f0 01   ADD    dword ptr [EBP + local_14],0x1
;AFTER
0804851b  66 90         NOP
0804851d  f4            ??     F4h
0804851e  01            ??     01h
0804851f  83 45 f0 01   ADD    dword ptr [EBP + local_14],0x1
;FIXED:
0804851b  66 90         NOP
0804851d  90            NOP
0804851e  90            NOP
0804851f  83 45 f0 01   ADD    dword ptr [EBP + local_14],0x1
```

В этом случае 4-байтовая инструкция `ADD` заменяется 2-байтовой `NOP`. Ассемблер Ghidra сделал все возможное, чтобы заполнить доступное пространство, вставив байт префикса x86 (`66`) перед x86-опкодом для `NOP` (`90`). К сожалению, заменяющая инструкция все еще слишком коротка, чтобы учесть два оставшихся байта исходной инструкции, один из которых переводится в `HLT` (используйте горячую клавишу D, чтобы дизассемблировать его), а другой Ghidra дизассемблировать не может, что указывает, что он не представляет допустимую инструкцию. Если бы вы так исправили исходный бинарный файл и запустили его, он почти наверняка аварийно завершился бы при достижении этого места.

Ghidra не дает никаких подсказок - кроме символов `??`, появляющихся в листинге, - что проблема может существовать, поскольку Ghidra не понимает вашу мотивацию для такого изменения, а "правильное" решение зависит от конкретного варианта использования. Если вы изменяете инструкции внутри листинга без намерения экспортировать результат, можно использовать параметр Ghidra для переопределения сквозного перехода из контекстного меню правой кнопки, чтобы обойти ненужные байты.[2] Альтернативно можно попросить Ghidra дизассемблировать неопределенные байты, но крайне маловероятно, что они дизассемблируются в инструкцию, которую вы сочтете полезной. Наиболее распространенное решение в такой ситуации - заменить все лишние байты исходной инструкции на `NOP`, чтобы заполнить пространство до начала следующей инструкции.

Когда заменяющая инструкция длиннее исходной, это создает новый набор трудностей, как показано здесь:

```
;BEFORE:
08048502 6a 01          PUSH   0x1
08048504 ff 75 f0       PUSH   dword ptr [EBP + local_14]
08048507 ff 75 08       PUSH   dword ptr [EBP + param_1]
0804850a e8 51 fe ff ff CALL   read
;AFTER:
08048502 68 00 01 00 00 PUSH   0x100
08048507 ff 75 08       PUSH   dword ptr [EBP + param_1]
0804850a e8 51 fe ff ff CALL   read
```

В этом случае цель патча - читать 256 (`0x100`) байтов, а не 1 байт. Исходная 2-байтовая инструкция `PUSH`, помещающая третий аргумент для `read` (аргумент длины) в стек, заменяется 5-байтовой `PUSH`, помещающей более крупную константу. Дополнительные байты в заменяющей инструкции полностью перезаписывают инструкцию, которая отвечала за второй аргумент для `read` (буфер чтения).

Получившийся код не только не предоставляет `read` достаточно аргументов, но и передает целое число там, где ожидается указатель. Как и в предыдущем примере, это почти наверняка приведет исправленную программу к аварийному завершению. Потенциальные решения этой конкретной проблемы исправления нетривиальны и обсуждаются в следующем разделе.

Внесение нетривиальных изменений

Как только размер патча становится больше инструкции (или инструкций) либо данных, которые вы заменяете, жизнь усложняется. В большинстве случаев это не означает, что патч невозможно реализовать, но для правильной реализации потребуется значительно больше обдумывания и усилий. В этом разделе мы обсудим несколько подходов к решению проблемы "патч слишком велик", в зависимости от того, содержит патч код или данные.

Слишком большие патчи кода

Когда патч слишком велик, чтобы поместиться поверх инструкций, которые вы хотите изменить, у вас нет выбора, кроме как найти или создать неиспользуемую область достаточного размера, поместить код патча в эту пустую область, а затем вставить переход (известный как перехват) в исходное место патча, чтобы передать управление фактическому патчу. В большинстве случаев также потребуется добавить переход к заменяющему коду, чтобы передать управление обратно в подходящее место в перехваченной функции. Рисунок 22-13 показывает условный поток исправленной функции с установленным перехватывающим переходом.

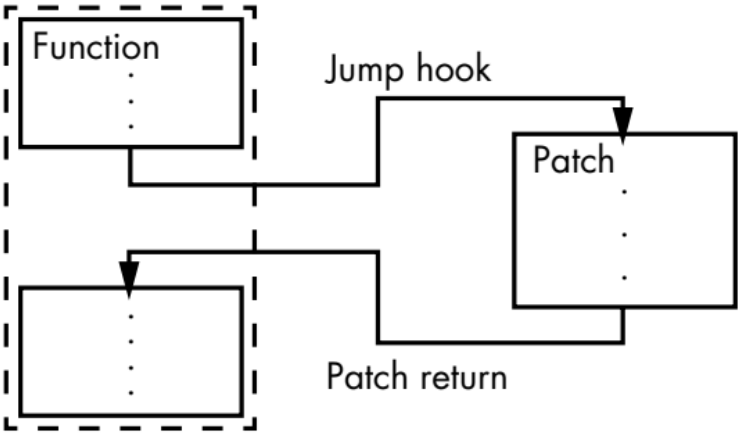


Figure 22-13: Function with installed patch

Рисунок 22-13. Функция с установленным патчем

Доступное неиспользуемое пространство для слишком большого патча кода должно:

- Быть не меньше вашего патча
- Находиться по адресу, который будет исполняемым во время выполнения
- Быть инициализированным из содержимого файла; иначе патч не будет загружен во время выполнения

Самое простое место для начала поиска крупных неиспользуемых исполняемых блоков байтов - любые "пещеры кода", которые могут присутствовать в бинарном файле. Пещера кода существует, когда исполняемая секция в бинарном файле, например секция `.text`, дополняется, чтобы соответствовать требованиям выравнивания секций, заданным форматом исполняемого файла. Пещеры кода очень распространены в Windows PE-бинарных файлах, поскольку они часто требуют, чтобы каждая секция бинарного файла была кратна 512 байтам по размеру.

Первое место для поиска пещеры кода обычно находится в конце секции `.text`. Можно легко перейти к концу секции `.text` (или любой другой секции), дважды щелкнув имя секции в окне `Program Trees` `CodeBrowser`, а затем прокрутив окно `Listing` до конца.

В нашем примерном PE-бинарном файле окно `Listing` показывает следующее в конце секции `.text`:

```
140012df8 ??     00h
140012df9 ??     00h
140012dfa ??     00h
140012dfb ??     00h
140012dfc ??     00h
140012dfd ??     00h
140012dfe ??     00h
140012dff ??     00h
```

Листинг сообщает нам следующее:

- Байты не классифицированы Ghidra (`??`).
- Байты инициализированы значением `00h`.
- Секция `.text` заканчивается по адресу `140012dff`, что удовлетворяет требованию выравнивания файла, согласно которому размер секции кратен 512 байтам (`140012e00` кратен `0x200`).

Переходя к предыдущей инструкции прокруткой вверх (или выбирая инструмент I в `CodeBrowser` с направлением поиска Up), мы приходим к следующему:

```
140012cbd POP    RBP
140012cbe RET
140012cbf ??     CCh
140012cc0 ??     00h
```

`RET` - последняя осмысленная инструкция в этом конкретном бинарном файле, и теперь можно вычислить размер пещеры кода этого бинарного файла как `0x140012e00 - 0x140012cbf = 0x141` (или 321 байт). Это означает, что мы легко можем внести в этот бинарный файл до 321 байта нового кода. Предположим, что мы внесли новый код по адресу `0x140012cbf`; тогда нужно было бы где-то в существующем коде бинарного файла вставить переход к `0x140012cbf`, чтобы гарантировать, что поток выполнения в итоге достигнет патча.

Когда пещеру кода не удается найти или она недостаточно велика для патча, придется проявить немного изобретательности, чтобы найти достаточно места. В зависимости от параметров компилятора, использованных для сборки бинарного файла, можно распределить патч по пространству, собранному из межфункциональных зазоров выравнивания. Межфункциональные зазоры выравнивания существуют, когда компиляторы выбирают выравнивать начало каждой функции по адресу, кратному 2 (часто 16). Когда выравнивание функций принудительно, между каждой функцией в бинарном файле будет вставлено в среднем `align / 2` байтов и вплоть до `align - 1` байтов заполнения. Следующий листинг показывает оптимальный (с точки зрения исправления) зазор выравнивания (`align = 16`) между двумя соседними функциями:

```
1400010a0 RET
1400010a1 ??     CCh
1400010a2 ??     CCh
1400010a3 ??     CCh
1400010a4 ??     CCh
1400010a5 ??     CCh
1400010a6 ??     CCh
1400010a7 ??     CCh
1400010a8 ??     CCh
1400010a9 ??     CCh
1400010aa ??     CCh
1400010ab ??     CCh
1400010ac ??     CCh
1400010ad ??     CCh
1400010ae ??     CCh
1400010af ??     CCh
     *************************************************************
     *                     FUNCTION                              *
     *************************************************************
```

Все байты от `1400010a1` до `1400010af` можно безопасно перезаписать кодом патча.

Существуют дополнительные методы втиснуть код патча в бинарный файл; некоторые предполагают расширение существующих секций программы или внедрение совершенно новых. Любой прием, который таким образом манипулирует секциями, также требует обновить заголовки секций бинарного файла, чтобы убедиться, что они остаются согласованными с внесенными изменениями. Соответственно, эти приемы очень специфичны для формата файла и требуют подробного понимания структур данных заголовка файла.

Слишком большие патчи данных

Исправлять данные в некоторых отношениях проще, чем код, а в других - сложнее. Для структурированных типов данных основные заботы - правильный размер и порядок байтов каждого члена структуры, и поскольку размер структуры определяется во время компиляции, вам не нужно беспокоиться о слишком больших заменяющих структурах. При исправлении строковых данных рекомендуется, чтобы любые заменяющие данные полностью помещались в область исходной строки. Если новая строка больше исходной, вам может повезти найти несколько байтов заполнения между концом строки и следующим элементом данных, но нужно быть осторожным, чтобы не повредить данные, от которых зависит программа. Если данные просто не помещаются в область памяти исходных данных, придется найти для них новое место, но правильно перемещать данные может быть трудно.

На все глобальные элементы данных ссылаются по их смещениям от секций кода или данных программы. Чтобы переместить элемент данных, помимо поиска достаточного неиспользуемого пространства, нужно найти каждую ссылку на исходный элемент данных и исправить ее так, чтобы она ссылалась на новый элемент данных. Перекрестные ссылки Ghidra сильно помогают идентифицировать каждую ссылку на глобальный объект, но не смогут идентифицировать производные указатели (указатели, полученные в результате арифметики указателей).

Когда все патчи внесены в Ghidra и вы довольны получившимся листингом программы, вы захотите перенести изменения в исходный бинарный файл, чтобы проверить, что патчи ведут себя как ожидается.

Экспорт файлов

Чтобы подтвердить, что любые ваши изменения окажут желаемое влияние на поведение бинарного файла, нужно обновить исходный бинарный файл, чтобы отразить изменения. В этом разделе мы обсудим некоторые возможности экспорта Ghidra применительно к исправлению.

Пункт меню Ghidra File -> Export Program предлагает возможность экспортировать информацию программы в любом из нескольких форматов. Получившийся диалог Export показан на рисунке 22-14.

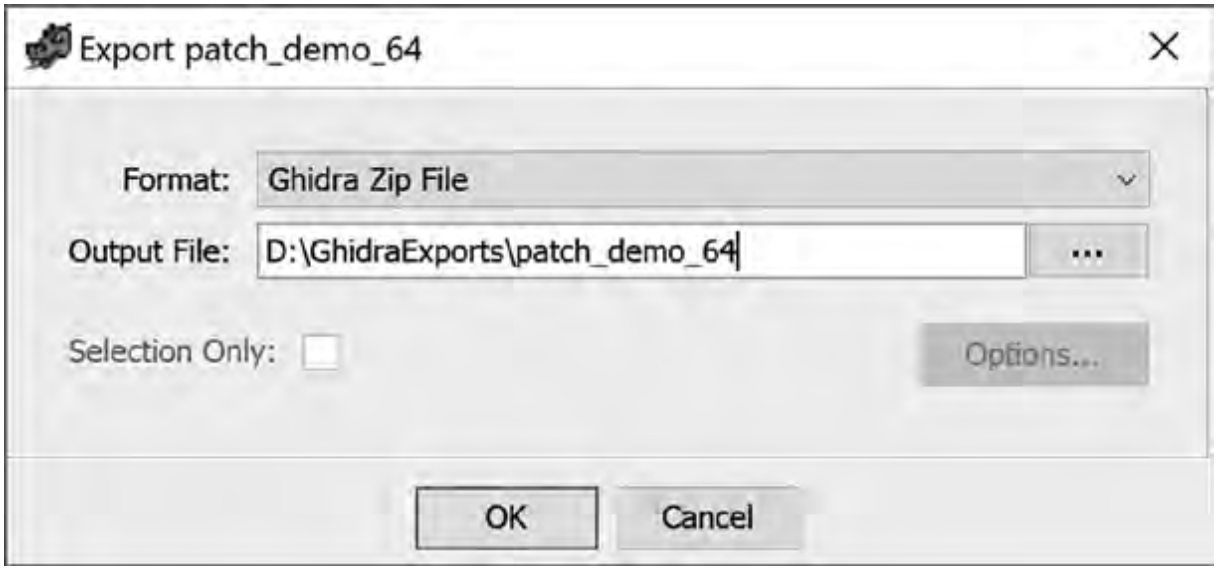


Рисунок 22-14. Диалог Ghidra Export

Диалог Export также доступен из Project Manager: щелкните правой кнопкой файл, который хотите экспортировать, и выберите Export в контекстном меню. В диалоге вам предлагается указать формат экспорта и место выходного файла, а также отметить, хотите ли вы ограничить область экспорта диапазоном, выбранным в CodeBrowser. Некоторые форматы экспорта предлагают дополнительные параметры для более тонкого управления процессом экспорта.

Форматы экспорта Ghidra

Ghidra поддерживает следующие форматы экспорта, хотя только один (binary) особенно полезен для исправления бинарных файлов:

Ascii. Формат экспорта Ascii можно использовать для сохранения текстового представления программы, похожего на то, что отображается в окне Listing, с параметрами выбора полей для включения в выходной файл.

Binary. Формат экспорта Binary, создающий бинарный файл, наиболее полезен для исправления приложений и обсуждается в отдельном разделе.

C/C++. Формат экспорта C/C++ используется для сохранения сгенерированного Decompiler исходного представления программы вместе с объявлениями всех типов, известных Data Type Manager. Этот параметр также доступен из окна Decompiler.

Ghidra Zip File. ZIP-файл Ghidra - это сериализованное Java-объектное представление вашей программы, пригодное для импорта в другие экземпляры Ghidra.

HTML. Формат экспорта HTML генерирует HTML-представление листинга программы. Параметры, похожие на доступные в экспортере Ascii, позволяют выбрать, какие поля включать в выходной файл. Метки и перекрестные ссылки представлены как гиперссылки, чтобы обеспечить базовую навигацию по сгенерированному выводу.

Intel Hex. Формат Intel Hex определяет ASCII-представление для бинарных данных, обычно используемое для программирования EEPROM.

XML. Экспортер XML выводит содержимое программы в структурированном XML-формате с параметрами выбора программных конструкций, которые должны быть включены в сгенерированный файл. Эта функциональность также доступна для отдельных функций в окне Decompiler, чтобы облегчить отладку декомпиляции функций. Хотя Ghidra включает соответствующий загрузчик XML, этот экспортер содержит предупреждение о том, что XML является форматом с потерями и предназначен только для передачи данных внешним инструментам, а рекомендуемым форматом для сохранения и совместного использования данных программы является GZF.

Формат экспорта Binary

Экспорт Binary в Ghidra используется для записи базового бинарного содержимого программы в файл. Все инициализированные блоки памяти программы (см. Window -> Memory Map) конкатенируются, образуя выходной файл. Будет ли выходной файл идентичен исходному импортированному файлу, зависит от модуля загрузчика, использованного для импорта файла. Загрузчик Raw Binary гарантированно воссоздает исходный входной файл, потому что загружает каждый байт исходного файла в единый блок памяти. Другие загрузчики могут загружать или не загружать каждый байт файла (например, PE-загрузчик загружает, а ELF-загрузчик - нет).

Когда приходит время применить любые изменения, внесенные с помощью Ghidra, нужно убедиться, что сгенерированный файл содержит ваши патчи и будет выполняться. Если вы исправляете PE-файл, экспорт Binary сгенерирует исправленную версию исходного бинарного файла. Аналогично, если вы импортировали программу с помощью загрузчика `Raw Binary`, экспорт Binary сгенерирует исправленную версию исходного бинарного файла. Конечно, как обсуждалось в главе 17, при использовании загрузчика `Raw Binary` вам, возможно, придется вручную выполнить большую часть компоновки памяти для программы, так что здесь есть компромисс. К счастью, можно написать сценарное решение, работающее с любым загрузчиком.

Экспорт с помощью сценария

Вместо выполнения исчерпывающей проверки каждого загрузчика Ghidra, чтобы понять, охватывают ли созданные загрузчиком блоки памяти весь диапазон байтов файла, можно создать сценарий Ghidra, который сохранит исправленную версию программы за нас. Этот сценарий предоставляет независимую от загрузчика возможность генерации исправленных файлов с помощью Ghidra. Он всегда будет обрабатывать весь диапазон байтов исходного файла независимо от текущей компоновки карты памяти, известной Ghidra.

```
public void run() throws Exception {
    Memory mem = currentProgram.getMemory();
    java.util.List<FileBytes> fbytes = mem.getAllFileBytes();
    if (fbytes.size() != 1) {
        return;
    }
    FileBytes fb = fbytes.get(0);
    File of = askFile("Выберите выходной файл", "Сохранить");
    FileOutputStream fos = new FileOutputStream(of, false);
    writePatchFile(fb, fos);
    fos.close();
}
```

Сценарий начинается с получения списка программы `FileBytes`. Объект `FileBytes` инкапсулирует все байты из импортированного файла программы и отслеживает как исходное, так и измененное значение каждого байта в файле. Поскольку Ghidra позволяет импортировать несколько файлов в одну программу, этот сценарий обрабатывает только байты из первого файла, импортированного в программу (первый диапазон байтов файла).

После запроса выходного файла объект `FileBytes` и открытый `OutputStream` передаются нашей функции `writePatchFile`, чтобы обработать более тонкие детали генерации исправленного исполняемого файла.

Чтобы представить отображенный вид памяти программы, загрузчики Ghidra могут обрабатывать записи таблицы релокаций программы способом, похожим на загрузчик времени выполнения. Результат этой обработки состоит в том, что места программы, помеченные для fixup (места, имеющие записи таблицы релокаций), изменяются Ghidra с исходных файловых значений на правильные релокированные значения. Когда мы генерируем исправленную версию бинарного файла, мы не хотим включать байты, измененные Ghidra для целей релокации.

Функция `writePatchFile`, показанная далее, начинается с генерации набора адресов, которые исправляются во время выполнения (и Ghidra) в соответствии с таблицей релокаций программы:

```
public void writePatchFile(FileBytes fb, OutputStream os) throws Exception {
    Memory mem = currentProgram.getMemory();
    Iterator<Relocation> relocs;
    relocs = currentProgram.getRelocationTable().getRelocations();
    HashSet<Long> exclusions = new HashSet<Long>();
    while (relocs.hasNext()) {
        Relocation r = relocs.next();
        AddressSourceInfo info = mem.getAddressSourceInfo(r.getAddress());
        for (long offset = 0; offset < r.getBytes().length; offset++) {
            exclusions.add(info.getFileOffset() + offset);
        }
    }
    saveBytes(fb, os, exclusions);
}
```

После получения итератора по таблице релокаций программы извлекается `AddressSourceInfo` для каждой записи релокации.

Объект `AddressSourceInfo` предоставляет отображение адреса программы на дисковый файл и смещение внутри этого файла, из которого был загружен соответствующий байт программы. Файловое смещение каждого байта релокации добавляется в набор смещений, которые нужно игнорировать при генерации окончательного исправленного файла. Функция завершается вызовом функции `saveBytes` для записи окончательной исправленной версии текущего файла программы. Функция `saveBytes` показана в следующем листинге:

```
public void saveBytes(FileBytes fb, OutputStream os, Set<Long> exclusions)
                    throws Exception {
    long begin = fb.getFileOffset();
    long end = begin + fb.getSize();
    for (long offset = begin; offset < end; offset++) {
        int orig = fb.getOriginalByte(offset) & 0xff;
        int mod = fb.getModifiedByte(offset) & 0xff;
        if (!exclusions.contains(offset) && orig != mod) {
            os.write(mod);
        }
        else {
            os.write(orig);
        }
    }
}
```

Функция итерирует по всему диапазону байтов файла, чтобы определить, сохранить ли исходный или измененный байт в выходной файл.

При каждом файловом смещении методы класса `FileBytes` используются для получения исходного значения байта, загруженного из импортированного файла, и текущего значения байта, которое могло быть изменено Ghidra или пользователем Ghidra. Если исходное значение отличается от текущего и байт не связан с записью релокации, измененный байт записывается в выходной файл; иначе в выходной файл записывается исходный байт.

В завершение этого раздела рассмотрим пример исправления бинарного файла и подтверждения того, что патч работает так, как мы ожидаем.

Пример: исправление бинарного файла

Рассмотрим пример, демонстрирующий исправление в контексте. Предположим, у вас есть фрагмент вредоносного ПО, который проверяет наличие отладчика и, если он присутствует, завершает работу, не позволяя изучить свое поведение. Следующий исходный код очерчивает эту функциональность в тривиальной программе:

```
int is_debugger_present() {
    return ptrace(PTRACE_TRACEME, 0, 0, 0) == -1;
}
void do_work() {
    if (is_debugger_present()) {
        printf("Отладка запрещена - выход!\n\n");
        exit(-1);
    }
    // здесь выполняются полезные действия
    printf("Подтверждено, что отладчика нет, поэтому делаем\n"
           "интересные вещи, которые мы не хотим\n"
           "показывать аналитикам!\n\n");
}
int main() {
    do_work();
    return 0;
}
```

Код проверяет наличие отладчика и завершается, если находит его. В противном случае он приступает к своим сомнительным делам. Ниже показан вывод программы, запущенной самостоятельно (без отладчика):

```
# ./debug_check_x64
  Подтверждено, что отладчика нет, поэтому делаем
  интересные вещи, которые мы не хотим
  показывать аналитикам!
```

Когда программа выполняется под отладчиком, мы видим другой ответ:

```
# gdb ./debug_check_x64
  Reading symbols from ./debug_check_x64...(no debugging symbols found)...done.
  (gdb) run
  Starting program: /ghidrabook/CH22/debug_check_x64
  Отладка запрещена - выход!
  [Inferior 1 (process 434) exited with code 0377]
  (gdb)
```

Если загрузить бинарный файл в Ghidra, в окне `Listing` мы увидим следующее:

```
     undefined do_work()
        undefined  AL:1 <RETURN>
001006f8  PUSH   RBP
001006f9  MOV    RBP,RSP
001006fc  MOV    EAX,0x0
00100701  CALL   is_debugger_present
00100706  TEST   EAX,EAX
00100708  JZ     LAB_00100720
0010070a  LEA    RDI,[s_No_debugging_allowed_-_exiting!_001007d8]
00100711  CALL   puts
00100716  MOV    EDI,0xffffffff
0010071b  CALL   exit
   -- Flow Override: CALL_RETURN (CALL_TERMINATOR)
        LAB_00100720
00100720  LEA    RDI,[s_Confirmed_that_there_is_no_debug_001008
00100727  CALL   puts
0010072c  NOP
0010072d  POP    RBP
0010072e  RET
```

Окно Decompiler предоставляет следующий соответствующий код:

```
void do_work(void)
{
  int iVar1;
  iVar1 = is_debugger_present();
  if (iVar1 != 0) {
    puts("Отладка запрещена - выход!\n");
                    /* WARNING: Subroutine does not return */
    exit(-1);
  }
  puts("Подтверждено, что отладчика нет, поэтому делаем\n"
       "интересные вещи, которые мы не хотим\n"
       "показывать аналитикам!\n"
      );
  return;
}
```

Чтобы исправить этот бинарный файл и обойти проверку, можно заменить вызов функции `is_debugger_present` на `NOP`, изменить условие проверки или изменить содержимое функции `is_debugger_present`. Если использовать параметр `Patch Instruction` из контекстного меню правой кнопки, легко заменить `JZ` на `JNZ` (фактически перевернув условие так, чтобы код выполнялся только при отладке), как показано на рисунке 22-15.

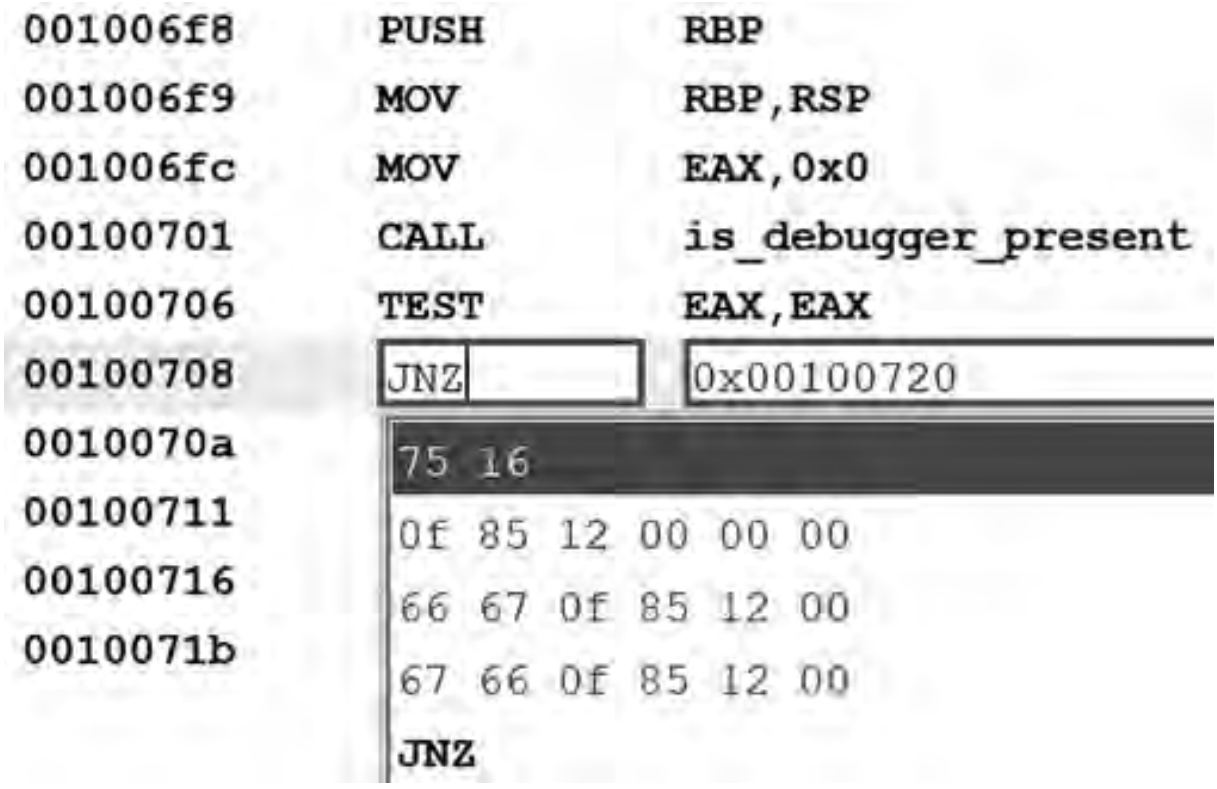


Рисунок 22-15. Опция Patch Instruction после замены JZ на JNZ

Это приведет к следующему коду в окне `Decompiler`:

```
void do_work(void)
{
  int iVar1;
  iVar1 = is_debugger_present();
  if (iVar1 == 0) {
    puts("Отладка запрещена - выход!\n");
                    /* WARNING: Subroutine does not return */
    exit(-1);
  }
  puts("Подтверждено, что отладчика нет, поэтому делаем\n"
       "интересные вещи, которые мы не хотим\n"
       "показывать аналитикам!\n"
      );
  return;
}
```

Если затем экспортировать файл как бинарный с помощью нашего сценария экспорта и снова запустить его, мы увидим следующие два листинга, которые демонстрируют поведение, которого мы надеялись добиться своим патчем:

```
# ./debug_check_x64.patched
  Отладка запрещена - выход!
# gdb ./debug_check_x64.patched
  Reading symbols from ./debug_check_x64.patched...(no debugging symbols found)...
    done.
  (gdb) run
  Starting program: /ghidrabook/CH22/debug_check_x64.patched
  Подтверждено, что отладчика нет, поэтому делаем
  интересные вещи, которые мы не хотим
  показывать аналитикам!
  [Inferior 1 (process 445) exited normally]
  (gdb)
```

Хотя существует много внешних инструментов (например, VBinDiff), доступных для подтверждения того, что в файле для этого примера был изменен только 1 байт, можно также использовать внутренние инструменты Ghidra, чтобы прийти к тому же выводу. Следующая глава сосредоточена на методах достижения этой цели.

Резюме

Независимо от вашей конкретной мотивации для исправления бинарного файла, патч потребует тщательного планирования и внедрения. Ghidra предоставляет все необходимое, чтобы спланировать патч; подготовить патч с помощью hex-редактирования, встроенного ассемблера Ghidra или сценариев; увидеть эффекты каждого изменения; и, возможно, откатить изменения с помощью Undo перед генерацией исправленной версии исходного бинарного файла. Следующая глава демонстрирует, как можно использовать Ghidra для сравнения неисправленных и исправленных версий бинарных файлов, а также обсуждает возможности Ghidra для более продвинутого сравнения бинарных файлов и отслеживания версий.

Сноски

[1] [назад] При использовании `nasm` параметр `-f bin` предписывает `nasm` выдавать сырой машинный код без заголовков файла. При использовании `as` требуется вторая утилита, такая как `objcopy`, чтобы извлечь сырые байты из получившегося объектного файла.
[2] [назад] Переход fall-through также имеет дополнительный параметр, позволяющий обходить части дизассемблирования путем указания начального и конечного адресов обхода.

Глава 23. Сравнение бинарных файлов и Version Tracking

В предыдущих главах мы знакомили вас со способами, которыми Ghidra может помогать вашим усилиям по анализу в реверс-инжиниринге. На всем этом пути мы представили множество способов преобразовывать и аннотировать вашу работу, чтобы документировать и облегчать понимание бинарного файла.

В этой главе мы представляем сравнение бинарных файлов и инструмент Ghidra `Version Tracking`, чтобы помочь выявлять сходства и различия между файлами и функциями и облегчать применение результатов предыдущего анализа к новым файлам. Мы также обсуждаем различия файлов с трех точек зрения: сравнение бинарных файлов, сравнение функций и отслеживание версий.

Сравнение бинарных файлов

В предыдущей главе мы исправили бинарный файл, чтобы изменить поток функции и обойти вызов `exit`, изменив один байт в одной инструкции: `JZ` (`74`) на `JNZ` (`75`). Чтобы подтвердить изменение и точно задокументировать, что было изменено, можно использовать внешний инструмент, такой как VBinDiff или WinDiff, для сравнения двух файлов на уровне байтов. Однако для сравнения файлов на уровне инструкций нужен гораздо более сложный инструмент: `Program Diff`, доступный в окне `Listing` Ghidra. После вычисления различий их можно просматривать с помощью специальных отображений, предназначенных для подчеркивания различий, облегчения понимания каждого изменения и предоставления возможностей действовать в зависимости от типа различия.

Чтобы сравнить два файла, импортированных в проект и находящихся в одном состоянии (например, оба проанализированы или ни один не проанализирован), откройте один из файлов в `CodeBrowser`, затем выберите `Tools -> Program Differences` и выберите другой файл в текущем проекте для сравнения. Альтернативно можно использовать значок инструмента окна `Listing`, показанный на рисунке 23-1. Этот значок служит переключателем для открытия или закрытия инструмента `Program Diff`.

Рисунок 23-1. Значок-переключатель `CodeBrowser` `Diff View`

Для этого примера начнем с открытия неисправленной версии файла, выбора значка `Diff View` и выбора исправленной версии файла. Это открывает диалог `Determine Program Differences`, показанный на рисунке 23-2.

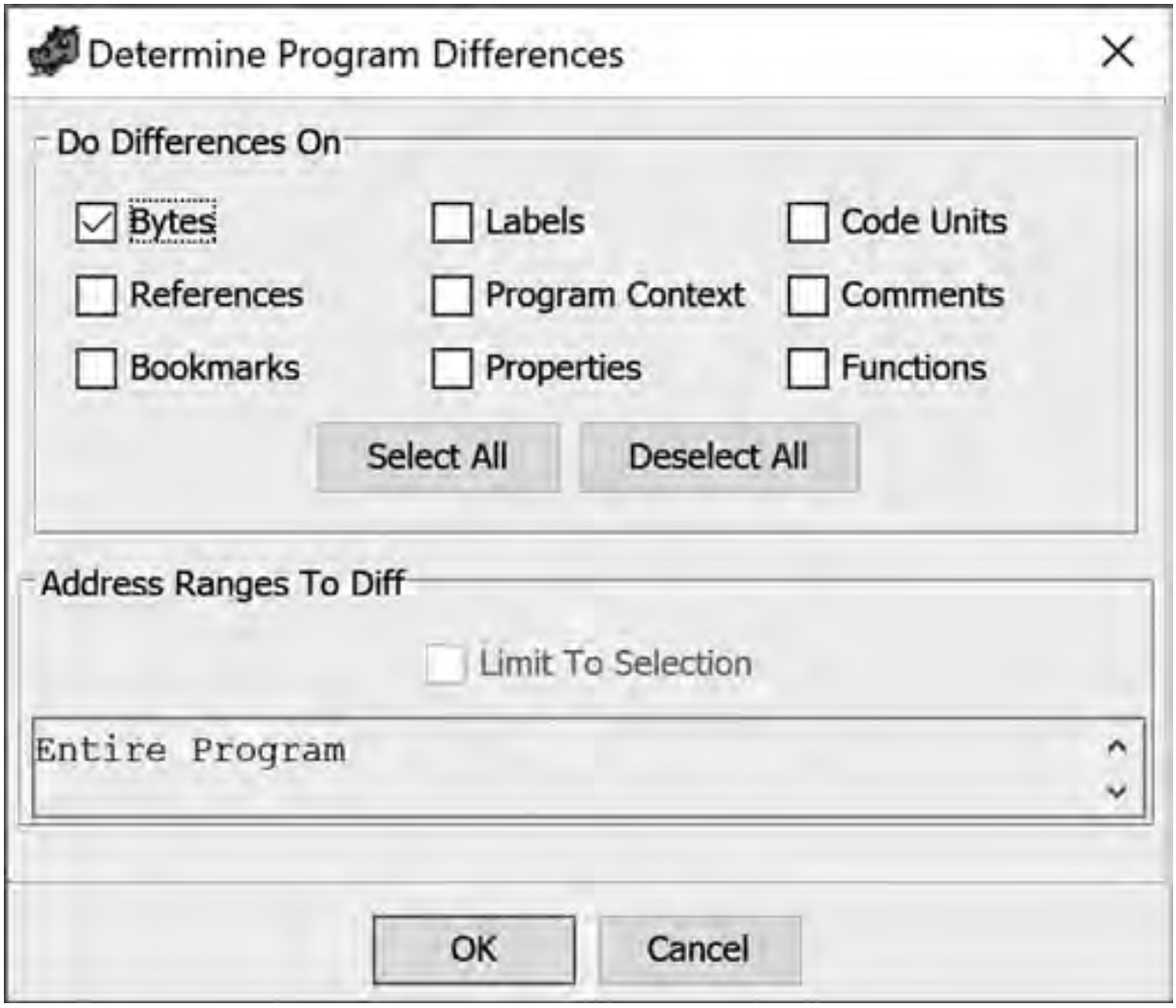


Рисунок 23-2. Диалог `Determine Program Differences` *инструмента Diff*

Хотя по умолчанию будут выбраны все доступные поля различий, в этом случае `Bytes` - подходящий выбор для подтверждения того, что патч сработал корректно, поэтому выбран только этот параметр различий. Когда вы щелкнете `OK`, в окне `Listing` можно увидеть два бинарных файла; окно будет разделено на расположенные рядом листинги, по одному файлу в каждом. По умолчанию листинги синхронизированы, поэтому навигация в одном также перемещает другой. В этом инструменте есть несколько способов перемещаться по обнаруженным различиям; мы представим их далее в главе.

Когда вы открываете два файла для сравнения, Ghidra сначала помещает вид `Listing` в начало каждого файла. Инструмент со стрелкой вниз на панели инструментов окна `Listing` (или `ctrl-alt-N`) можно использовать для перехода к первому различию между двумя файлами. Чтобы привлечь внимание к отличающемуся коду, изменения обозначаются строками дизассемблирования с цветовой подсветкой в окне `Listing` каждого файла, а также в окне `Decompiler`, если различие находится внутри функции. (Окно `Decompiler` синхронизировано с первым из двух файлов.) Переход к первому обнаруженному различию раскрывает единственный байт: `JZ` (`74`) в исходном листинге и `JNZ` (`75`) во втором листинге.

Чтобы просмотреть больше деталей, выберите `Window -> Diff -> Diff Details`. В окне Diff Details внизу окна `CodeBrowser` вы увидите следующий отчет:

```
Diff address range 1 of 1.
Difference details for address range: [ 00100708 - 00100709 ]
Byte Diffs :
    Address   Program1  Program2
    00100708    0x74       0x75
Code Unit Diffs :
    Program1 CH23:/DiffDemo/debug_check_x64 :
            00100708 - 00100709    JZ 0x00100720
                                   Instruction Prototype hash = 16af243b
    Program2 CH23:/DiffDemo/debug_check_x64.patched :
            00100708 - 00100709    JNZ 0x00100720
                                   Instruction Prototype hash = 176d4e0c
```

Первая строка указывает число диапазонов адресов, содержащих различия. В этом случае только один диапазон в файле содержит различия, поэтому можно быть уверенным, что две программы отличаются всего одним байтом. Этот простой пример едва затрагивает поверхность возможностей, связанных с инструментом Ghidra `Program Diff`, поэтому уделим немного времени изучению других возможностей, которые предоставляет этот инструмент.

Инструмент Program Diff

Девять параметров в верхней части рисунка 23-2 образуют основу сравнения, и можно выбрать любой из них или все сразу. По умолчанию инструмент `Program Diff` работает со всей программой для каждого файла. Если вы хотите ограничить сравнение конкретным диапазоном адресов, нужно выделить диапазон в первом файле до открытия инструмента. После того как вы сделали выбор и щелкнули `OK`, разделенное окно `Listing`, которое вы видите, называется `Program Diff`.

Program Diff

Представление `Program Diff` позволяет просматривать оба файла одновременно. По сути, окно `Listing` теперь содержит два листинга: один слева, другой справа. Когда вы открываете окно Diff Details, оно открывается внизу окна `CodeBrowser`. В окне Diff Details левый файл считается Program1 (файл, который вы изначально открыли), тогда как правый файл считается Program2 (файл, который вы выбрали для сравнения с Program1). Окно `Decompiler` отражает содержимое Program1. Когда вы сравниваете два файла, Ghidra может вычислять различия в любом направлении. Вам самим нужно помнить, какой файл какой, пока вы используете инструмент `Program Diff`.

Распространенный рабочий процесс - начать анализ файла, а затем понять, что часть или весь код выглядит знакомо; это может побудить вас открыть ранее проанализированный файл, чтобы начать сравнение. К счастью, `Program Diff` поддерживает выравнивание между двумя файлами, вставляя пустые строки при необходимости. Различия подсвечиваются, а панель инструментов `Program Diff` предоставляет возможности навигации и средства определения того, как вы хотите обработать различия.

Панель инструментов Program Diff

Панель инструментов `Program Diff` расширяет параметры панели инструментов окна `Listing`, добавляя инструменты, показанные на рисунке 23-3.

is considered Program2 (the file you chose to compare to Program1). The Decompiler window reflects the contents of Program1. When you are comparing two files, Ghidra can compute the differences in either direction. It is up to you to remember which file is which as you use the Program Diff tool.

A common workflow is to begin analyzing a file and then realize that some or all of the code looks familiar, which may prompt you to open a file you have previously analyzed in order to begin diffing. Fortunately, Program Diff maintains alignment between the two files by inserting blank lines when necessary. Differences are highlighted, and the Program Diff toolbar provides you navigational capability and the means to determine how you wish to handle the differences.

Program Diff Toolbar

The Program Diff toolbar extends the Listing window toolbar options by adding the tools shown in Figure 23-3.

Apply Differences	Applies the selected settings and remains in the same location
Apply Differences and Move	Applies the selected settings and moves you to the next highlighted difference
Ignore Differences and Move	Ignores the selected settings and moves you to the next highlighted difference
Show Details	Opens the Diff Details window and provides information about the selected difference
Go to Next	Moves you to the next highlighted difference
Go to Previous	Moves you to the previous highlighted difference
Display Diff Apply settings	Opens the Diff Apply Settings window and allows you to modify the settings.
Determine Program	Reopens the Determine Program Differences dialog to allow you

Рисунок 23-3. Параметры панели инструментов `Program Diff`

Diff Apply Settings

Настройки `Diff Apply Settings` определяют действия, которые вы хотите выполнить при наличии различия между двумя файлами. Выбор параметра `Display Diff Apply Settings` отображает окно, показанное на рисунке 23-4.

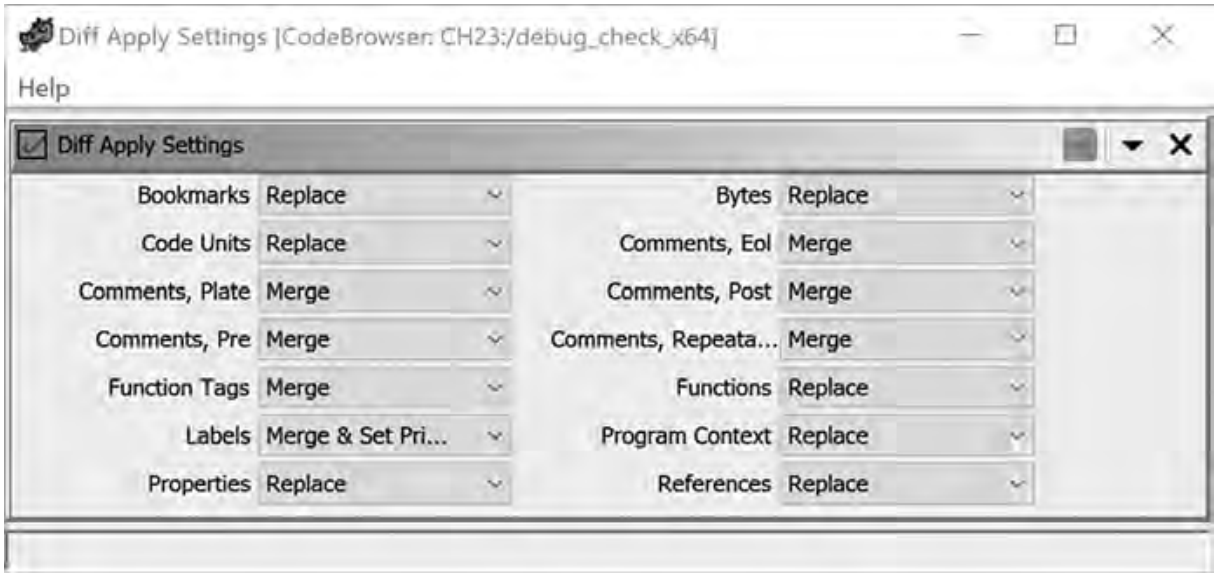


Рисунок 23-4. Окно `Diff Apply Settings`

Каждый параметр задает действие по умолчанию, которое вы хотите применить из второй программы к первой открытой программе, и способ применения этого параметра. В каждом раскрывающемся списке доступны следующие четыре варианта:

Ignore. Не изменять первую программу (доступно во всех случаях).

Replace. Изменить содержимое первой программы так, чтобы оно соответствовало содержимому второй программы (доступно во всех случаях).

Merge. Добавить различие из второй программы в первую программу. Если применяется к метке, это не изменит, какая метка установлена как основная (доступно только для Comments и Labels).

Merge & Set Primary. Действие такое же, как Merge, но основная метка устанавливается в метку второй программы, если это возможно (доступно только для Labels).

В верхней части рисунка 23-4 находятся два значка панели инструментов. Значок `Save as Default` сохраняет текущие `Diff Apply Settings`. Стрелка открывает меню, позволяющее изменить все параметры сразу, выбрав один из вариантов, показанных на рисунке 23-5.

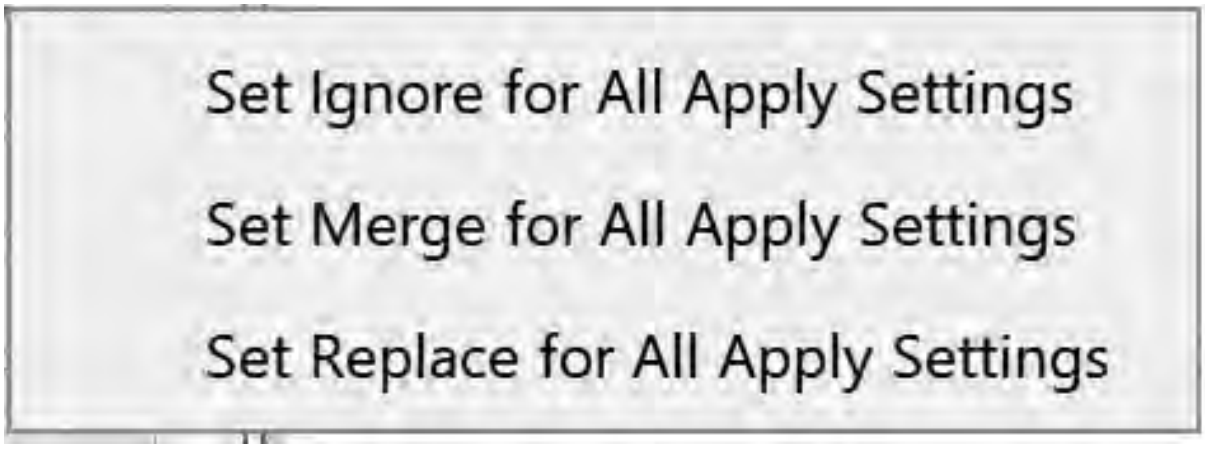


Рисунок 23-5. Раскрывающееся меню `Diff Apply Settings`

Если выбрать параметр Set Merge, а Merge не является допустимым выбором для конкретной настройки, она будет изменена на Set Replace. Для меток она будет изменена на Merge & Set Primary.

Выберите Apply Differences на панели инструментов, если хотите применить все изменения по умолчанию. Когда вы закончите работу с инструментом `Program Diff`, переключите значок `Diff View` в окне `Listing`, и вы увидите диалог, показанный на рисунке 23-6.

Рисунок 23-6. Диалог подтверждения Close Diff Session

Подтверждение закрытия текущего сеанса сравнения закроет отображение второго файла и вернет вас к обычному окну `Listing` с первым файлом (и всеми изменениями, выбранными вами из анализа различий).

Инструмент `Program Diff` был разработан для двух основных сценариев: во-первых, для сравнения файлов, проанализированных двумя разными пользователями, которые не используют общий экземпляр Ghidra Server; во-вторых, для сравнения кода, сгенерированного из разных версий одной и той же базы исходного кода (например, неисправленной и исправленной версий разделяемой библиотеки). В следующем примере мы пройдем процесс использования этого инструмента для согласования двух копий одного бинарного файла, каждая из которых была проанализирована независимо.

Пример: слияние двух проанализированных файлов

Предположим, вы анализируете бинарный файл, содержащий криптографическую процедуру. Коллега упоминает, что находится на середине анализа бинарного файла, который тоже, похоже, имеет криптографическую процедуру и, вероятно, относится к тому же семейству вредоносного ПО. Она соглашается предоставить вам свой проект, чтобы вы могли сравнить два файла. Когда вы смотрите на файлы в `Diff View`, вы сразу замечаете, что вы оба, похоже, анализируете один и тот же бинарный файл.

Проблема в том, что каждый из вас продвинулся и изменил содержимое файла на основе собственного анализа. Нужно объединить два проанализированных файла, чтобы каждый мог воспользоваться анализом другого. Вы согласились взять на себя эту ответственность, открыли свой бинарный файл в `CodeBrowser` и инициировали сеанс `Program Diff`, добавив бинарный файл коллеги для сравнения.

Выбор стрелки вниз на панели инструментов `Program Diff` переносит вас к первому различию в этом файле. В этот момент можно открыть окно Diff Details, выбрав параметр на панели инструментов `Program Diff` (или горячую клавишу F5). Это дает следующий листинг (разбитый на две секции для удобства обсуждения). В первой секции вверху Diff Details вы видите следующее:

```
Diff address range 1 of 4.
Difference details for address: 0010075a
Function Diffs :
  Program1 CH23:/Crypto/diff_sample1 :
    Signature: void encrypt_rot13(char * inbuffer, char * outbuffer)
    Thunk? : no
    Stack Frame:
       Parameters:
         DataType    Storage        FirstUse Name      Size Source
         /char *     RDI:8          0x0      inbuffer  8    USER_DEFINED
         /char *     RSI:8          0x0      outbuffer 8    USER_DEFINED
       Local Variables:
         DataType    Storage        FirstUse Name      Size Source
         /int        EAX:4          0xc0     length    4    USER_DEFINED
         /int        Stack[-0x1c]:4 0x0      idx       4    USER_DEFINED
         /char       Stack[-0x1d]:1 0x0      curr_char 1    USER_DEFINED
  Program2 CH23:/Crypto/diff_sample1a :
    Signature: void encrypt(char * param_1, long param_2)
    Thunk? : no
    Stack Frame:
       Parameters:
         DataType    Storage        FirstUse Name      Size Source
         /char *     RDI:8          0x0      param_1   8    DEFAULT
         /long       RSI:8          0x0      param_2   8    DEFAULT
       Local Variables:
         DataType    Storage        FirstUse Name      Size Source
         /undefined4 Stack[-0x1c]:4 0x0      local_1c  4    DEFAULT
         /undefined1 Stack[-0x1d]:1 0x0      local_1d  1    DEFAULT
```

Отображается первый из четырех идентифицированных диапазонов адресов различий в этом файле, связанный с текущим адресом `0010075a`. Листинг начинается с подробного описания различия в заголовках функций двух бинарных файлов. Для своего бинарного файла вы задали осмысленное имя функции и параметров в сигнатуре функции. Кроме того, каждый параметр имеет правильно определенный тип. Аналогично локальным переменным даны осмысленные имена и типы. Во второй программе аналитик не внес никаких изменений в стандартный заголовок Ghidra для соответствующей функции.

Вы хотите сохранить свою версию изменений для определения функции и локальных переменных. Можно использовать значок панели инструментов, чтобы отклонить изменение, но это отклонит все различия, связанные с этим адресом. Поскольку вы еще не просмотрели все различия, просто прокрутите вниз к следующему различию в окне Diff Details.

Следующая секция различий, связанных с первым диапазоном адресов, содержит различия меток и комментариев.

```
Label Diffs :
  Program1 CH23:/Crypto/diff_sample1 at 0010075a :
    0010075a is an External Entry Point.
      Name           Type           Primary  Source        Namespace
      encrypt_rot13  Function       yes      USER_DEFINED  Global
  Program2 CH23:/Crypto/diff_sample1a at 0010075a :
    0010075a is an External Entry Point.
      Name           Type           Primary  Source        Namespace
      encrypt        Function       yes      USER_DEFINED  Global
Plate-Comment Diffs :
  Program1 CH23:/Crypto/diff_sample1 at 0010075a :
    *************************************************************
    *                      FUNCTION                             *
    * Это криптографическая функция, исходно названная cryptor.    *
    * Переименована в наш стандартный формат encrypt_rot13.        *
    * Параметры функции изменены на char *. Добавлены осмысленные  *
```

```
    * имена переменных. Функция впервые замечена в fileC13d, Ken H *
    ****************************************************************
  Program2 CH23:/Crypto/diff_sample1a at 0010075a :
    No Plate-Comment.
EOL-Comment Diffs :
  Program1 CH23:/Crypto/diff_sample1 at 0010075a :
    No EOL-Comment.
  Program2 CH23:/Crypto/diff_sample1a at 0010075a :
    Похоже на процедуру шифрования. TODO: Проанализировать подробнее.
```

Среди различий меток единственное отличие - имя функции, которое уже обсуждалось. В секции Plate-Comment ваш файл имеет подробный комментарий, но другой файл не имеет plate-комментариев. В секции EOL-Comment есть краткий комментарий другого аналитика, но в вашем файле его нет. Изучив комментарий, вы видите, что это TODO-задача, которую вы уже выполнили в своем файле.

После оценки всех различий между двумя файлами ваше решение - сохранить свое содержимое и не принимать новое содержимое из другого бинарного файла. Для этого вы выбираете значок `Ignore Differences and Move`. Это переносит вас к следующему различию. Поскольку окно `Diff Details` уже открыто, его содержимое обновляется сразу после перехода, и вы видите следующее:

```
Diff address range 1 of 3.
Difference details for address range: [ 0010081a - 0010081e ]
Reference Diffs :
  Program1 CH23:/Crypto/diff_sample1 at 0010081a :
    Reference Type: WRITE  From: 0010081a  Mnemonic  To: register:
      RAX  USER_DEFINED  Primary
  Program2 CH23:/Crypto/diff_sample1a at 0010081a :
    No unmatched references.
```

Вы уменьшили число диапазонов, содержащих различия, отклонив предыдущее различие. И снова ваш файл содержит больше информации, чем второй файл. На этот раз вы перейдете к следующему различию, щелкнув стрелку вниз. Это приводит вас к следующему:

```
Diff address range 2 of 3.
Difference details for address: 00100830
Function Diffs :
  Program1 CH23:/Crypto/diff_sample1 :
    Signature: undefined display_message()
    Thunk? : no
    Calling Convention: unknown
    Return Value :
         DataType    Storage        FirstUse Name      Size Source
         /undefined  AL:1           0x0      <RETURN>  1    IMPORTED
      Parameters:
        No parameters.
  Program2 CH23:/Crypto/diff_sample1a :
    Signature: void display_message(char * message)
    Thunk? : no
    Calling Convention: __stdcall
    Return Value :
         DataType    Storage        FirstUse Name      Size Source
         /void       <VOID>         0x0      <RETURN>  0    IMPORTED
      Parameters:
         DataType    Storage        FirstUse Name      Size Source
         /char *     RDI:8          0x0      message   8    USER_DEFINED
```

Обратите внимание, что число диапазонов не изменилось. Вас просто переместили к следующему диапазону различий без влияния на общее число диапазонов различий. Оценка этого нового различия показывает, что второй файл содержит информацию, предоставленную другим аналитиком, которой нет в вашем файле. Сигнатура функции имеет тип возврата, и в сигнатуру функции добавлен параметр. Можно включить это в свой бинарный файл, щелкнув правой кнопкой различие в правом окне `Listing` и выбрав Apply Selection (горячая клавиша F3) или щелкнув значок Apply Differences на панели инструментов.

Переходя к следующему различию, вы видите следующие детали:

```
Diff address range 2 of 2.
Difference details for address range: [ 00100848 - 0010084c ]
Pre-Comment Diffs :
  Program1 CH23:/Crypto/diff_sample1 at 00100848 :
    No Pre-Comment.
  Program2 CH23:/Crypto/diff_sample1a at 00100848 :
    Это потенциальная уязвимость. Параметр передается в printf
    как первый/единственный параметр, что может привести к
    уязвимости форматной строки.
```

Число диапазонов различий уменьшилось, потому что вы применили различия в предыдущем диапазоне. В этом последнем различии вы видите интересную запись в секции Pre-Comment в другом файле. Аналитик обнаружил потенциальную уязвимость. Чтобы убедиться, что эта информация включена в ваш файл, вы выбираете Apply Differences.

Теперь, когда вы завершили сравнение двух файлов, можно щелкнуть значок `Diff View` и подтвердить, что вы хотите закрыть текущий сеанс `Program Diff`. Ваш вид листинга теперь отражает объединенный анализ из обоих бинарных файлов, и можно сохранить и закрыть файл.

Инструмент Ghidra `Program Diff` предоставляет возможность исследовать различия между двумя версиями одного файла. Хотя он попытается сравнить два несвязанных файла, любые результаты, вероятно, будут отражать только случайные сходства. Перенесем внимание на другой инструмент, который облегчает сравнение выбранных функций внутри одной или разных программ.

Сравнение функций

Если вы видите функцию, напоминающую функцию, которую анализировали в прошлом, полезно напрямую сравнить две функции, чтобы результаты первоначального анализа можно было при необходимости применить к текущей функции. Ghidra предоставляет эту возможность через окно `Function Comparison`, позволяющее просматривать две функции одновременно, как показано на рисунке 23-7.

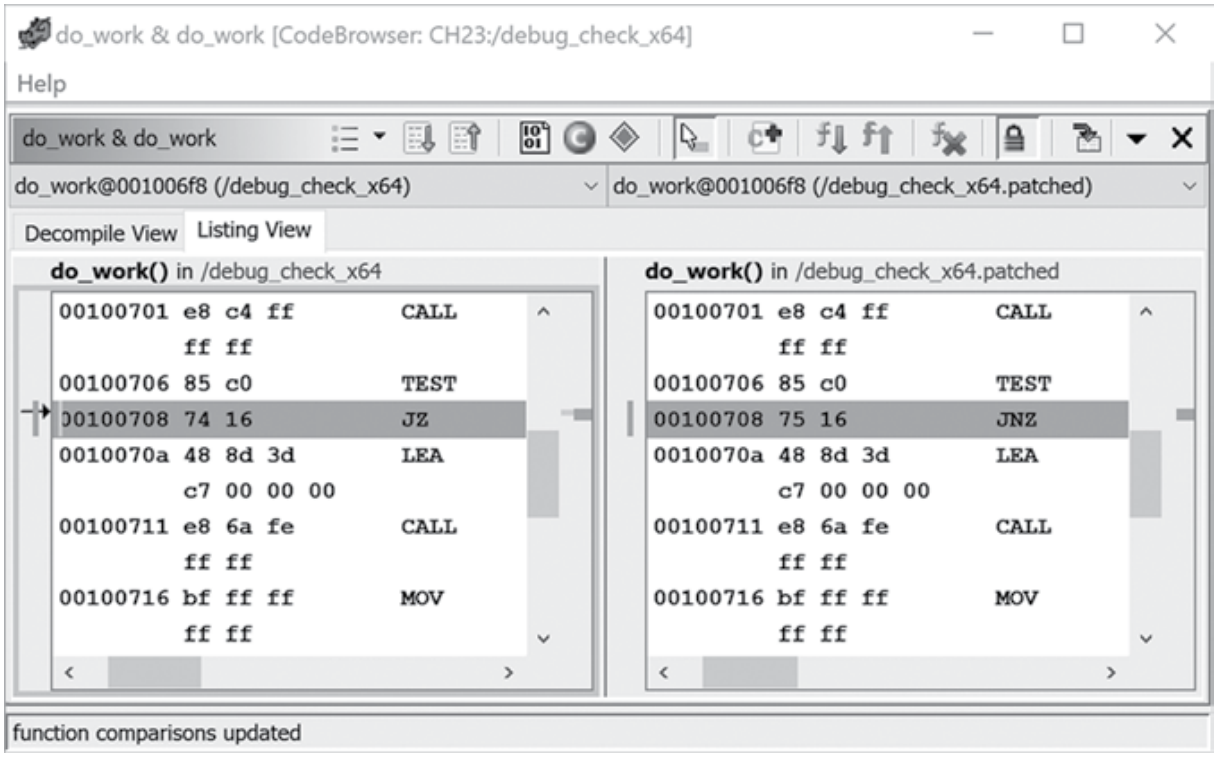


Рисунок 23-7. Вид `Listing` *в окне* `Function Comparison`

Окно Function Comparison

Чтобы использовать окно `Function Comparison`, откройте один или несколько бинарных файлов, содержащих функции, в `CodeBrowser`, загрузите начальную функцию, выделив функцию в активной вкладке `CodeBrowser`, и выберите `Compare Selected Functions` (горячая клавиша shift-C) в контекстном меню правой кнопки. Окно `Function Comparison` показывает две функции рядом друг с другом с подсветкой потенциальных различий, как показано на рисунке 23-7. (Если выбрана только одна функция, она будет отображаться в обоих окнах, пока вы не загрузите больше функций.)

Чтобы добавить дополнительные функции для сравнения, выберите значок `Add Functions`. Это отобразит список всех функций активной программы в `CodeBrowser`. Можно выбрать функцию из списка или переключиться в окно `CodeBrowser`, чтобы изменить активную программу, выбрав другую вкладку программы в окне `Listing`.

Слева от активного листинга (обозначенного рамкой вокруг листинга) находится стрелка курсора. Если функции совпадают, стрелка также появится в том же месте в другом окне. На рисунке 23-7 инструкция в основном окне не совпадает с инструкцией в другом окне, поэтому стрелка курсора не показана в обоих окнах.

Окно `Function Comparison` предоставляет возможность загрузить больше двух функций из более чем двух бинарных файлов. При необходимости можно добавлять и удалять функции из каждой панели. Полезное раскрывающееся меню позволяет выбрать функцию, отображаемую в связанном окне.

Это окно позволяет легко переключаться между представлениями Decompile и `Listing` для двух функций и изменять функцию, отображаемую в любом окне. Представление Decompile для этого примера показано на рисунке 23-8.

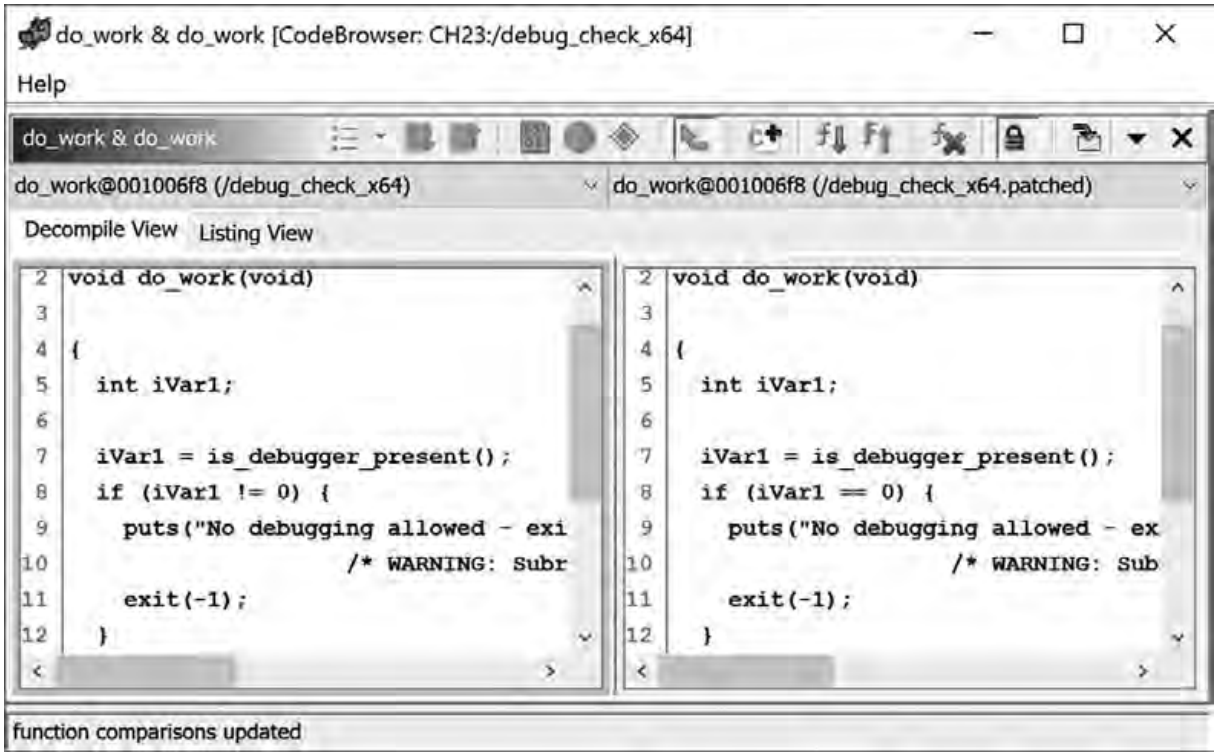
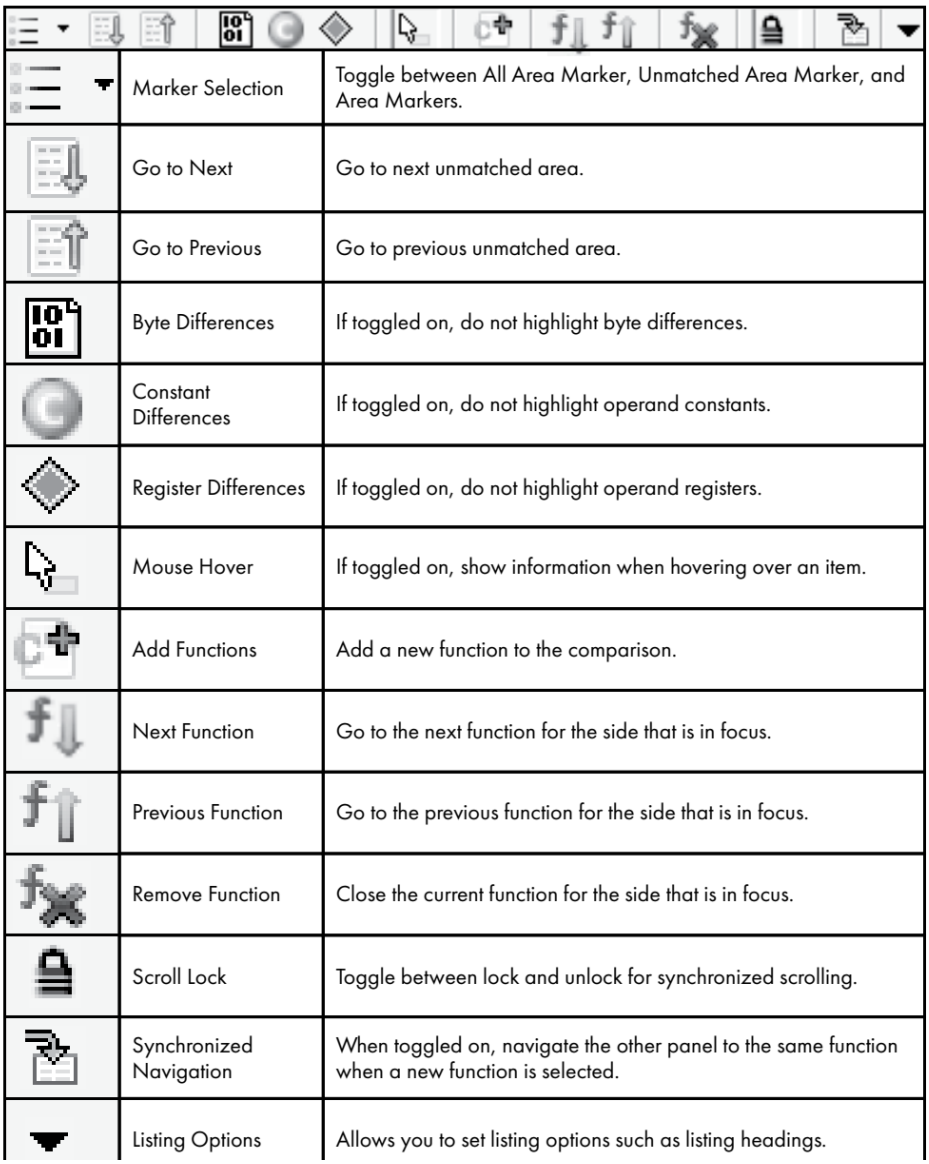


Рисунок 23-8. Вид Decompile в окне `Function Comparison`

Исследовательские возможности, доступные в этом окне, существенно пересекаются с инструментом `Program Diff`, за исключением того, что вы сравниваете только две функции за раз и можете легко переключаться между декомпилированным кодом и листингом. Меню панели инструментов этого окна показано на рисунке 23-9.

The exploratory capabilities available in this window overlap significantly with the Program Diff tool, except you are comparing only two functions at a time and you can easily switch between the decompiled code and the listing. The toolbar menu for this window is shown in Figure 23-9.



Marker Selection	Toggle between All Area Marker, Unmatched Area Marker, and Area Markers.
Go to Next	Go to next unmatched area.
Go to Previous	Go to previous unmatched area.
Byte Differences	If toggled on, do not highlight byte differences.
Constant Differences	If toggled on, do not highlight operand constants.
Register Differences	If toggled on, do not highlight operand registers.
Mouse Hover	If toggled on, show information when hovering over an item.
Add Functions	Add a new function to the comparison.
Next Function	Go to the next function for the side that is in focus.
Previous Function	Go to the previous function for the side that is in focus.
Remove Function	Close the current function for the side that is in focus.
Scroll Lock	Toggle between lock and unlock for synchronized scrolling.
Synchronized Navigation	When toggled on, navigate the other panel to the same function when a new function is selected.
Listing Options	Allows you to set listing options such as listing headings.

Рисунок 23-9. Параметры панели инструментов `Function Comparison`

Пройдем пример, демонстрирующий некоторые дополнительные возможности инструмента `Function Comparison`.

Пример: сравнение криптографических процедур

Поздравляем с повышением! На основе вашего успешного анализа и использования инструмента `Program Diff` для криптографических процедур вас теперь считают криптоэкспертом в вашей группе. Каждый раз, когда один из коллег подозревает, что у него есть криптографическая процедура, он отправляет вам бинарный файл, чтобы выяснить, является ли это процедурой, которую вы узнаете.

Теперь у вас есть новый файл от коллеги, и вы хотите определить, является ли криптографическая процедура, использованная в этом файле, чем-то новым или процедурой, которую вы уже

идентифицировали раньше. Вместо загрузки и сравнения каждой криптографической процедуры с новой функцией вы настроили специальный проект Ghidra, содержащий все ранее проанализированные и задокументированные криптографические процедуры. Ваша цель - загрузить свои криптографические процедуры на одну сторону окна `Function Comparison`, а затем импортировать новый файл на другую сторону для сравнения с существующими криптографическими процедурами. (Чтобы упростить пример, сейчас в вашей коллекции есть только одна проанализированная криптографическая процедура: процедура ROT13, которую вы объединили в предыдущем примере.)

После загрузки полной коллекции проанализированных криптофайлов в `CodeBrowser` и загрузки вашей функции `encrypt_rot13` в окно `Function Comparison` нужно загрузить новый файл в тот же экземпляр `CodeBrowser` (`File -> Open`) и сделать его активным файлом. В этот момент можно исследовать файл, но это не обязательно. Всегда можно вернуться в окно `CodeBrowser`, если вы не можете найти нужную функцию. В этом случае, выбрав параметр `Add Functions` на панели инструментов `Function Comparison`, вы видите полный список функций в новом бинарном файле, и примерно в середине списка находится функция с интригующим именем `encrypt`, как показано на рисунке 23-10.

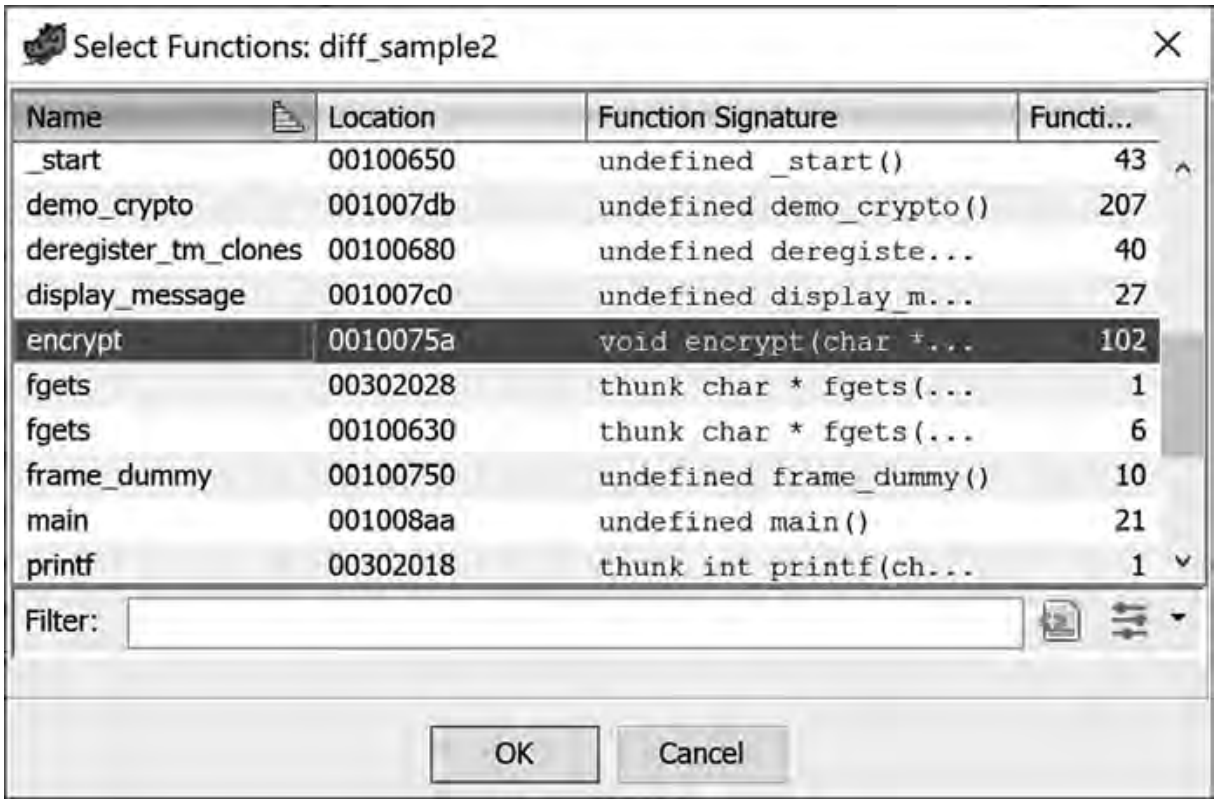


Рисунок 23-10. Окно `Select Functions` *с выбранной функцией encrypt*

Беглый взгляд на загруженные файлы в `Decompile View` функций, показанном на рисунке 23-11, подсказывает, что эти две функции довольно сильно различаются.

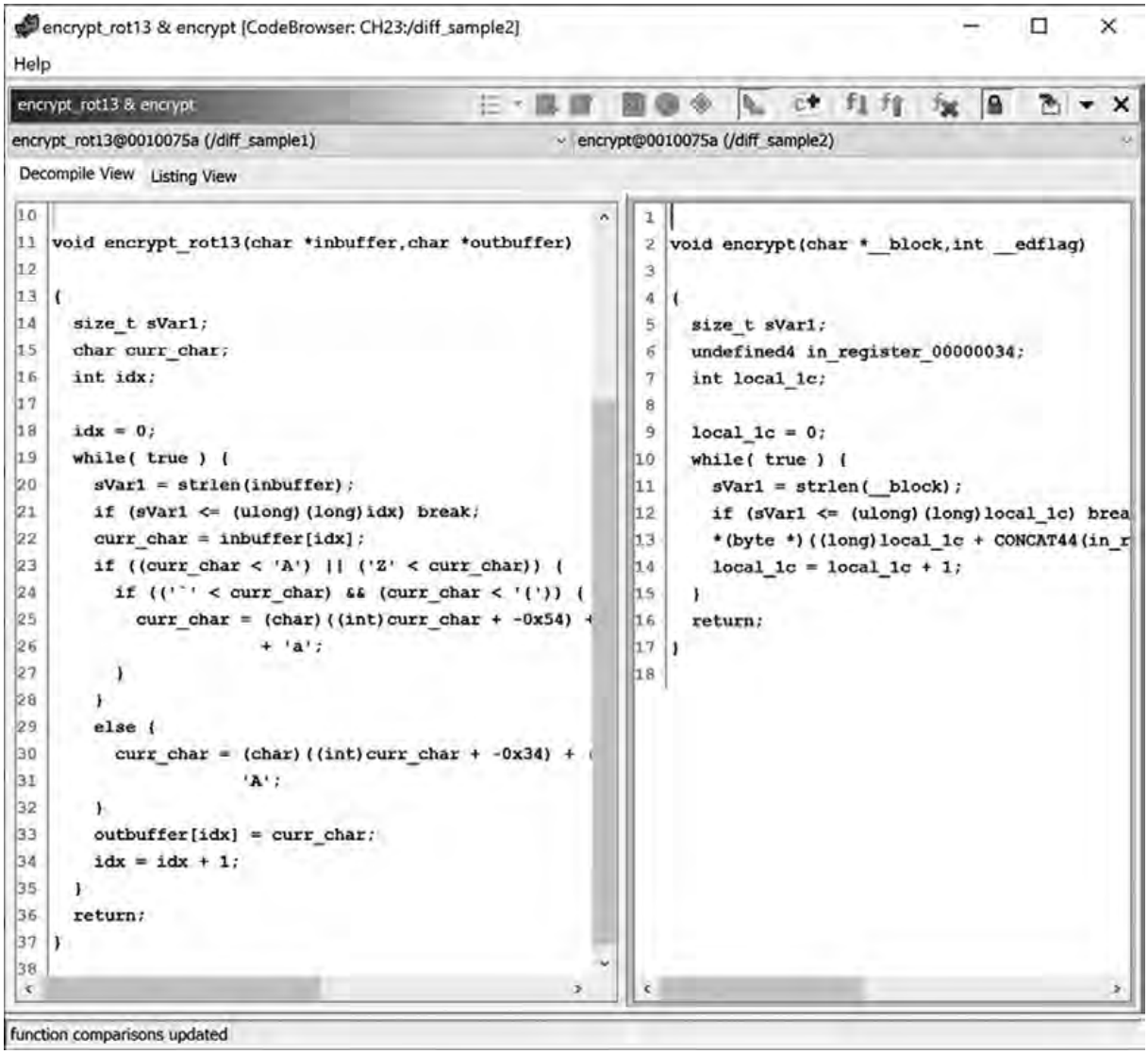


Рисунок 23-11. Представление Decompile окна `Function Comparison` *для двух криптографических процедур*

Представление `Listing`, показанное на рисунке 23-12, подтверждает, что эти две функции имеют существенные различия.

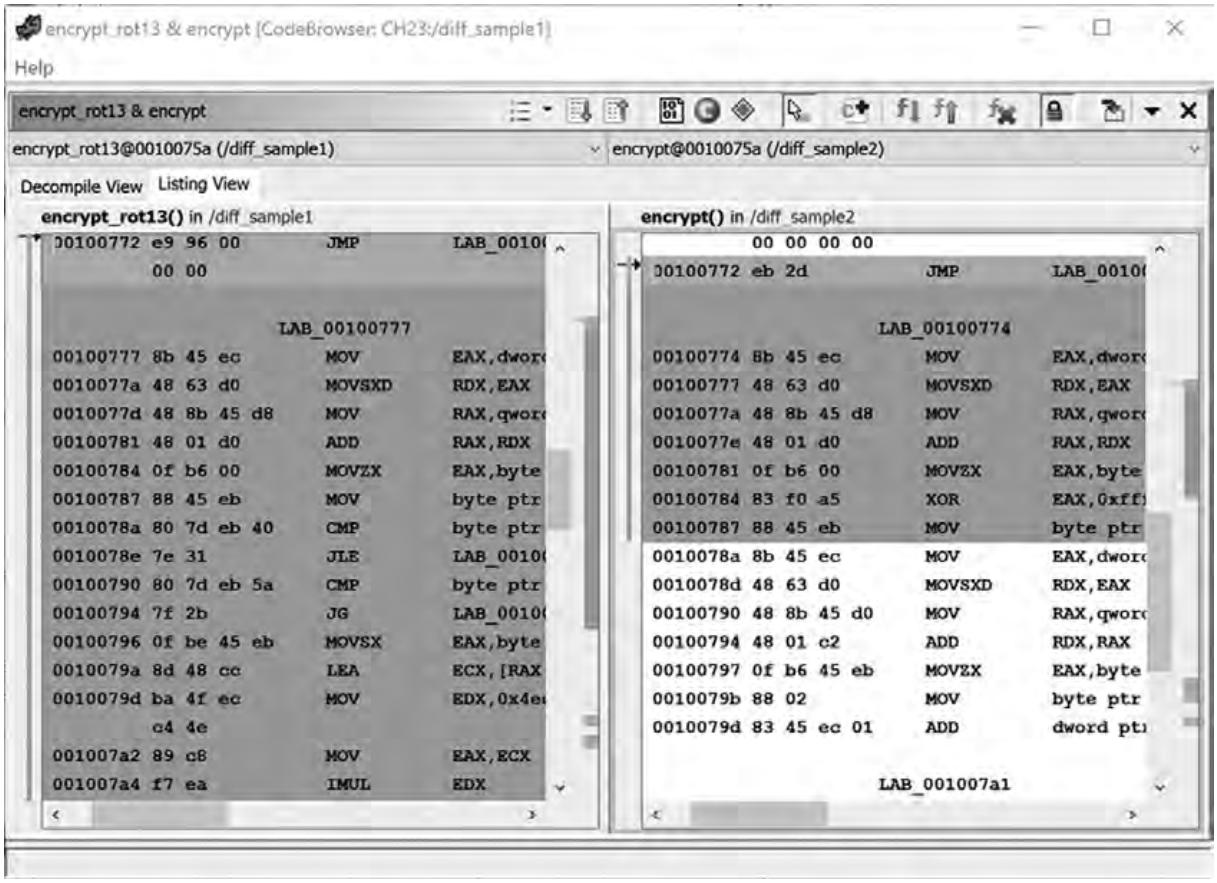


Рисунок 23-12. Представление `Listing` *окна* `Function Comparison` *для двух криптографических процедур с подсвеченными различиями*

При дальнейшем анализе вы обнаруживаете, что новая процедура выполняет XOR каждого байта с постоянным значением `0xa5`. Это определенно отличается от текущей имеющейся криптографической процедуры, поэтому вы называете и документируете эту новую функцию и добавляете ее в свою коллекцию (в которой теперь будет два элемента!). Вернувшись в `CodeBrowser`, вы обновляете сигнатуру функции и добавляете комментарии, документирующие новую криптографическую процедуру. Внесенные изменения также отражаются в окне `Function Comparison`.

Во время документирования вы замечаете, что в новом бинарном файле есть функция с именем `display_message`, как и в бинарном файле, с которым вы его сравниваете. Вы вспоминаете, что эта функция была идентифицирована как имеющая уязвимость в текущем бинарном файле, поэтому решаете сравнить эти две функции. Вы загружаете их в окно `Function Comparison`, чтобы посмотреть, есть ли у них сходства помимо общего имени. Они выглядят разными как в представлении Decompile, так и в представлении `Listing`, как показано на рисунке 23-13.

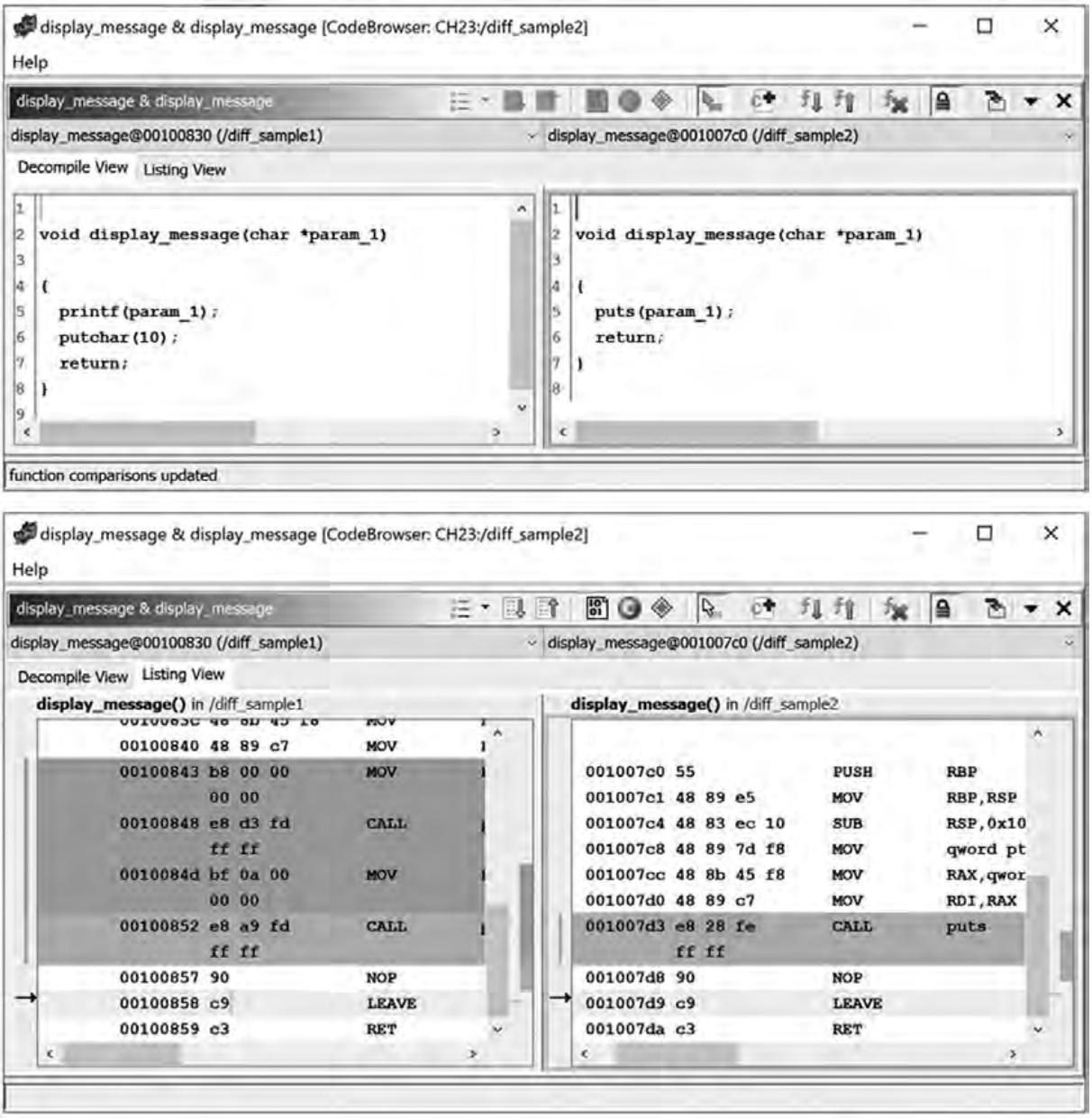


Рисунок 23-13. Представления Decompile и `Listing` *для функций display_message*

Во втором примере `param_1` передается в `puts` для вывода, что исправляет уязвимость.

Теперь, когда вы задокументировали эту криптографическую процедуру, вы видите, что получили от коллег еще один бинарный файл. Чтобы сбросить состояние к началу процесса сравнения криптографических процедур, можно использовать значок панели инструментов `Function Comparison`, чтобы удалить функции `display_message` из окна, оставив коллекцию криптографических процедур, которая теперь имеет два разных элемента: `encrypt_rot13` и `encrypt_XOR_a5`.

Первичное исследование этого нового файла показывает, что три функции, похоже, связаны с шифрованием: `encrypt`, `encrypt_strong` и `encrypt_super_strong`. Вы загружаете их в окно `Function Comparison`, чтобы сравнить с существующими криптографическими процедурами. После сравнения `encrypt_rot13` с каждой из новых функций вы замечаете следующее:

encrypt_rot13 vs. encrypt. Почти полностью отличается. Процедура `encrypt` - это просто проверка, которая может вызвать одну из двух других процедур шифрования.

encrypt_rot13 vs. encrypt_strong. Почти полностью совпадает.

encrypt_rot13 vs. encrypt_super_strong. Очень сильно отличается. Более внимательное изучение различий между этими двумя функциями приводит вас к выводу, что это не одна и та же функция.

Более внимательное изучение различий показывает, что инструкции в `encrypt_rot13` и `encrypt_strong` идентичны; различия в основном состоят из адресных меток, как показано на рисунке 23-14.

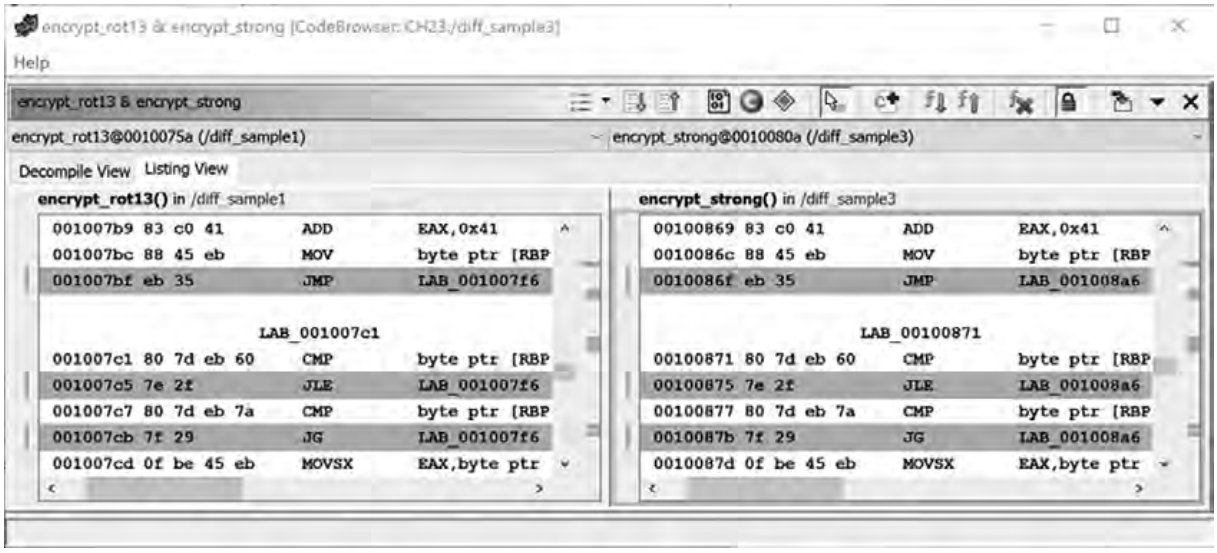


Рисунок 23-14. Окно `Function Comparison` *с различиями в адресных метках*

В этом случае не стоит ожидать идеального совпадения адресных меток, поскольку расположения функций внутри бинарных файлов различаются. Расположения согласованы относительно текущего адреса, поэтому, вероятно, мы имеем дело с той же функцией. Единственное другое различие - 1 байт, связанный с вызовом `strlen`, как показано на рисунке 23-15. Это похожая проблема, и ее можно объяснить различием относительных положений функции шифрования и `strlen` в каждом бинарном файле.

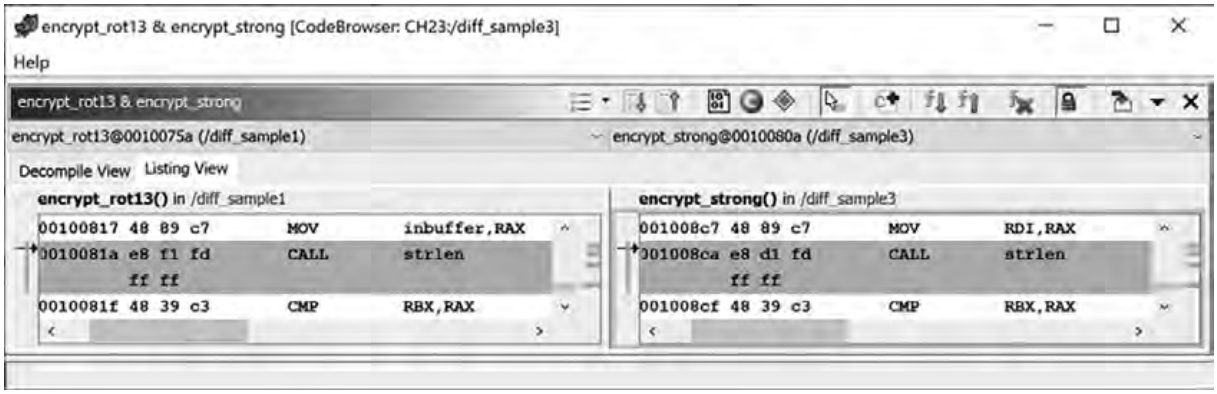


Рисунок 23-15. Окно `Function Comparison` *с байтовым различием в вызове strlen*

Определив, что это одна и та же функция, можно щелкнуть правой кнопкой ранее проанализированную функцию и выбрать `Apply Function Signature To Other Side` в контекстном меню правой кнопки. Это обновит сигнатуру функции во всех нужных местах, включая окно `Listing` и `Symbol Tree`. Обратите внимание, что окно `Function Comparison` не предоставляет всех возможностей, доступных в `Diff View`. Чтобы скопировать дополнительную информацию (например, подробные комментарии, связанные с функцией), используйте инструмент `Program Diff`.

Завершив сравнительный анализ `encrypt_rot13`, вы переключаете внимание на `encrypt_XOR_a5` и наблюдаете следующие отношения с каждой из новых функций:

encrypt_XOR_a5 vs. encrypt. Почти полностью отличается.

encrypt_XOR_a5 vs. encrypt_strong. Очень сильно отличается. Более внимательное изучение различий между этими двумя функциями также приводит вас к выводу, что это не одна и та же функция.

encrypt_XOR_a5 vs. encrypt_super_strong. Почти полностью совпадает.

Идентифицированные различия между `encrypt_XOR_a5` и `encrypt_super_strong` также являются лишь адресными метками и некоторыми байтами в вызове `strlen`. Эту ситуацию можно обработать тем же способом, что и предыдущие совпадающие функции.

Хотя это тривиальный пример (и он вряд ли соответствует реальным криптографическим процедурам, которые можно встретить в дикой природе), он демонстрирует, как `Function Comparison` можно использовать для минимизации дублирования аналитических усилий, когда вы встречаете знакомые процедуры в новых бинарных файлах.

Последний инструмент для исследования двух файлов - самый сложный: `Version Tracking`.

Version Tracking

Представьте, что вы потратили месяцы на анализ очень большого бинарного файла. В нем сотни или тысячи функций и нет символов. В рамках своей работы вы дали осмысленные имена большинству функций, переименовали данные, локальные переменные и параметры функций, а также добавили гору комментариев, на повторное создание которых ушли бы дни или больше.

Теперь представьте, что выпущена новая версия бинарного файла, и мир перестает использовать версию, о которой вы знаете так много. Можно продолжить анализировать старую версию, чтобы узнать о ней больше, предполагая, что новая версия ведет себя похоже, но тогда вы не узнаете о новых или измененных вариантах поведения в обновленном бинарном файле. Вместо этого вы решаете начать работу с более новой версией бинарного файла и быстро понимаете, что тратите значительное время на чтение разметки в старом бинарном файле, чтобы ориентироваться в новом.

Постоянное переключение между двумя окнами `CodeBrowser` - не оптимальное использование времени. Пора переключиться с `CodeBrowser` на другой стандартный инструмент, который Ghidra предоставляет в `Project Tool Chest`, как показано на рисунке 23-16.

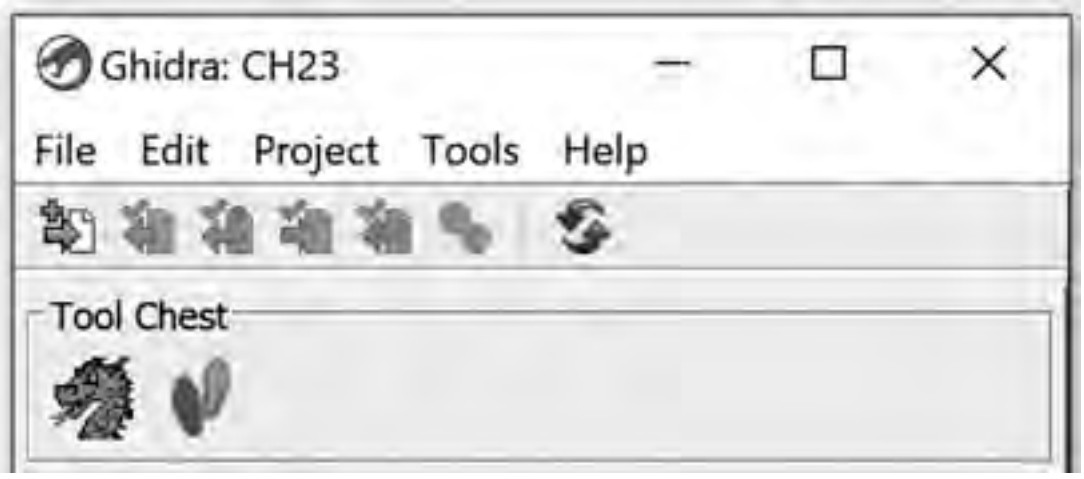


Рисунок 23-16. Инструмент `Version Tracking` *(следы) в* `Project Tool Chest`

Инструмент Ghidra `Version Tracking` предназначен именно для помощи в такой ситуации. С помощью различных коррелляторов Ghidra пытается сопоставить элементы, такие как функции или данные, в исходном бинарном файле с их соответствующими версиями в целевом бинарном файле. После сопоставления функций между двумя бинарными файлами Ghidra может автоматически перенести информацию, включая метки и комментарии, из исходного бинарного файла в целевой. Помимо быстрого переноса существующего анализа, инструмент `Version Tracking` упрощает определение того, что не изменилось, что имеет лишь небольшие изменения (обнаруженные сравнением) и что является совершенно новым.

Инструмент `Version Tracking` - один из наиболее настраиваемых инструментов Ghidra, что позволяет легко адаптировать его к конкретной линии исследования. Это также сложный инструмент для полного представления. В следующих разделах мы проведем вас через процесс отслеживания версий на очень высоком уровне и укажем на ресурсы, которые помогут найти правильные настройки и компоненты для использования `Version Tracking` при выявлении отношений между двумя файлами.

Концепции Version Tracking

В то время как инструменты `Function Comparison` и `Program Diff` отвечают на конкретные вопросы об атомарных различиях между двумя файлами или функциями, инструмент `Version Tracking` предоставляет функциональность для ответа на более целостный вопрос: насколько похожи эти два бинарных файла, и можете ли вы подсветить и объяснить сходства между ними? Базовая рабочая единица называется сеансом, и каждый сеанс настроен для идентификации и обработки корреляций между двумя файлами.

Корреляторы

На высоком уровне инструмент `Version Tracking` ищет корреляции между двумя файлами. Есть семь типов коррелляторов, генерирующих совпадения между двумя бинарными файлами:

- Коррелляторы Data Match
- Коррелляторы Function Match
- Коррелляторы Legacy Import
- Коррелляторы Implied
- Коррелляторы Manual Match
- Коррелляторы Symbol Name Match
- Коррелляторы Reference

Вместо простого подсчета и составления списка конкретных различий в каждой категории инструмент `Version Tracking` расширяет корреляции между двумя файлами, чтобы идентифицировать совпадения с разными уровнями точности:

Точные совпадения. Это однозначные совпадения между двумя файлами, которые могут сопоставлять данные, байты функций, инструкции функций или мнемоники функций (например, когда два бинарных файла содержат точно такую же функцию).

Дублирующее совпадение данных. Это точные совпадения, которые не являются однозначными (например, когда строка появляется один раз в одном файле и семь раз в другом).

Похожие совпадения. Это совпадения, которые проходят управляемый пользователем порог сходства. Сопоставление похоже на подход, использованный для моделей слов, описанных в главе 13, но использует 4-граммы наряду с триграммами.

Благодаря возможности вводить пороги, принимать и отклонять совпадения этот инструмент предлагает мощную возможность переноса предыдущего анализа в новые версии бинарного файла. Кроме того, информация, связанная с каждым сеансом, предоставляет эффективный аудиторский след анализа, который может помочь фиксировать постепенные изменения бинарного файла или эволюцию семейства вредоносного ПО.

Сеансы

Хотя глубокое пошаговое прохождение полного сеанса заняло бы значительное время, базовый сеанс отслеживания версий может включать следующие шаги:

1. Откройте инструмент Ghidra `Version Tracking`.
2. Создайте новый сеанс, выбрав исходный файл и целевой файл.
3. Для всех подходящих коррелляторов: добавьте их в существующий сеанс, выберите коррелятор, выберите все получившиеся совпадения, примите все совпадения и примените их элементы разметки.
4. Сохраните сеанс.
5. Закройте сеанс.

Предыдущий рабочий процесс дает очень общий обзор, а комбинаторный потенциал, связанный с шагом коррелляторов, обширен. Потенциал и нюансы, связанные с этим инструментом, невозможно охватить в одной главе. Команда Ghidra предоставила примерные рабочие процессы (а также значительную документацию об инструменте `Version Tracking`) в `Ghidra Help`. Вам предстоит определить, как лучше всего применять возможности этого инструмента в своем рабочем процессе реверс-инжиниринга.

Резюме

В этой главе мы отошли от одного бинарного файла и начали рассматривать способы выявления различий и сходств между бинарными файлами с помощью инструментов `Program Diff`, `Function Comparison` и `Version Tracking`. Эти инструменты ценно экономят время при переносе существующей работы в новые бинарные файлы, слиянии аннотаций от коллег и быстром точном выявлении того, что изменилось между двумя версиями одной программы.

Завершая обзор обширного ландшафта возможностей Ghidra, помните, что мы лишь слегка коснулись поверхности возможностей Ghidra. Теперь у вас должно быть более глубокое понимание Ghidra и того, как ее можно применять к задачам реверс-инжиниринга, с которыми вы сталкиваетесь. Когда у вас возникают вопросы, сообщество Ghidra готово помочь через такие ресурсы, как GitHub, Stack Exchange, Reddit, YouTube и многие другие форумы.

Что важнее, теперь вы должны быть в состоянии внести вклад, отвечая на вопросы и помогая другим. Ghidra - ПО, поддерживаемое сообществом, и оно постоянно развивается. Мы надеемся, что вы будете участвовать: публиковать руководства, писать и публиковать сценарии и модули Ghidra, выявлять и решать проблемы или, возможно, даже разрабатывать новую функциональность для самой Ghidra. Будущее Ghidra будет определяться сообществом, и теперь оно включает вас. Добро пожаловать и удачного реверсинга!

Приложение. Ghidra для пользователей IDA

Если вы опытный пользователь IDA Pro и хотите попробовать Ghidra - из любопытства или ради более постоянного перехода, - многие понятия, представленные в этой книге, могут быть вам уже знакомы. Это приложение сопоставляет терминологию и приемы работы IDA с похожей функциональностью Ghidra, но не обучает использованию самой Ghidra. Если вам нужно конкретное описание какой-либо возможности Ghidra, упомянутой здесь, обратитесь к соответствующим главам этой книги, где эти возможности рассматриваются гораздо подробнее.

Мы не пытаемся сравнивать производительность двух инструментов и не утверждаем, что один из них лучше другого. Выбор может быть продиктован ценой или конкретной возможностью, которая есть в одном продукте и отсутствует в другом. Далее следует быстрый обзор тем книги с точки зрения пользователя IDA.

Основы

В начале пути полезно держать под рукой краткий справочник, который поможет выучить совершенно новый набор горячих клавиш. Ghidra Cheat Sheet (https://ghidra-sre.org/CheatSheet.html) - удобная памятка, где перечислены распространенные действия пользователя и соответствующие им горячие клавиши и/или кнопки инструментов. Чуть позже мы рассмотрим, как переназначать горячие клавиши, если вам не хватает привычных сочетаний IDA.

Создание базы данных

IDA импортирует один двоичный файл в одну базу данных и по своей природе рассчитана на одного пользователя. Ghidra, напротив, ориентирована на проекты, может содержать несколько файлов в одном проекте и поддерживает совместную обратную разработку, когда несколько пользователей работают над одним проектом. Понятие базы данных IDA ближе всего соответствует отдельной программе внутри проекта Ghidra. Пользовательский интерфейс Ghidra разделен на два основных компонента: `Project` и `CodeBrowser`.

Первое взаимодействие с Ghidra состоит в создании проектов (общих или необщих) и импортировании "программ" (двоичных файлов) в эти проекты через окно `Project`. Когда вы используете IDA, чтобы открыть новый двоичный файл и в итоге создать новую базу данных, вы и IDA выполняете следующие действия:

1. (IDA) Опросить все доступные загрузчики, чтобы узнать, какие из них распознают выбранный файл.
2. (IDA) Показать диалог загрузки файла со списком подходящих загрузчиков, процессорных модулей и параметров анализа.
3. (Пользователь) Выбрать модуль загрузчика, который должен загрузить содержимое файла в новую базу данных, или принять выбор IDA по умолчанию.
4. (Пользователь) Выбрать процессорный модуль, который должен использоваться при дизассемблировании содержимого базы данных, или принять выбор IDA по умолчанию; этот выбор может определяться модулем загрузчика.
5. (Пользователь) Выбрать параметры анализа, которые должны использоваться при создании начальной базы данных, или принять настройки IDA по умолчанию. На этом этапе также можно полностью отключить анализ.

6. (Пользователь) Подтвердить выбор, нажав `OK`.
7. (IDA) Выбранный модуль загрузчика заполняет базу данных байтовым содержимым, взятым из исходного файла. Загрузчики IDA обычно не загружают весь файл в базу данных, и, как правило, исходный файл невозможно воссоздать из содержимого, доступного в новой базе данных.
8. (IDA) Если анализ включен, выбранный процессорный модуль используется для дизассемблирования кода, который был найден загрузчиком и выбранными анализаторами; IDA называет анализаторы параметрами ядра.
9. (IDA) Полученная база данных отображается в пользовательском интерфейсе IDA.

В Ghidra есть аналоги каждого из перечисленных шагов, однако процесс разбит на две отдельные фазы: импорт и анализ. Импорт в Ghidra обычно запускается из окна `Project` и включает следующие действия:

1. (Ghidra) Опросить все доступные загрузчики, чтобы узнать, какие из них распознают выбранный файл.
2. (Ghidra) Показать диалог импорта со списком подходящих форматов (примерно соответствуют загрузчикам) и языков (примерно соответствуют процессорным модулям).
3. (Пользователь) Выбрать формат для импорта файла в текущий проект или принять выбор Ghidra по умолчанию.
4. (Пользователь) Выбрать язык для дизассемблирования содержимого программы или принять выбор Ghidra по умолчанию.
5. (Пользователь) Подтвердить выбор, нажав `OK`.
6. (Ghidra) Загрузчик, связанный с выбранным форматом, загружает байтовое содержимое исходного файла в новую "программу" текущего проекта. Загрузчик создает секции программы и обрабатывает таблицы символов, импортов и экспортов двоичного файла, но не выполняет анализ, связанный с дизассемблированием. Загрузчики Ghidra обычно загружают весь файл в проект Ghidra, хотя некоторые части файла могут не отображаться в `CodeBrowser`.

Хотя этот процесс похож на создание базы данных в IDA, некоторые шаги отсутствуют. В Ghidra анализ выполняется в `CodeBrowser`. После успешного импорта файла двойной щелчок по нему в представлении `Project` открывает файл в инструменте Ghidra `CodeBrowser`. Когда вы впервые открываете программу, Ghidra выполняет следующие шаги:

1. (Ghidra) Открыть `CodeBrowser`, показать результаты импорта и спросить, хотите ли вы проанализировать файл.
2. (Пользователь) Решить, анализировать ли файл. Если вы решите не выполнять анализ, вы попадете в `CodeBrowser`, где сможете прокручивать байтовое содержимое, но дизассемблирования там не будет. В этом случае вы можете в любой момент выбрать `Analysis -> Auto Analyze`, чтобы выполнить анализ файла. Когда вы решаете анализировать файл, Ghidra показывает список анализаторов, совместимых с текущим форматом файла и настройкой языка. Вы можете выбрать анализаторы для запуска и изменить любые параметры, которые использует анализатор, прежде чем разрешить Ghidra выполнить начальный анализ.
3. (Ghidra) Выполнить все выбранные анализаторы и поместить пользователя в `CodeBrowser` для начала работы с полностью проанализированной программой.

Более подробные сведения о стадиях импорта и анализа см. в соответствующих главах этой книги. У IDA нет аналога представления `Project`, а также нет возможностей совместной обратной разработки, кроме общей базы Lumina. Представление `Project` представлено в главе 4. Общие проекты и поддержка совместной обратной разработки обсуждаются в главе 11. `CodeBrowser` представлен в главе 4; более подробное рассмотрение начинается в главе 5 и продолжается до конца книги.

CodeBrowser - это инструмент Ghidra и ваш основной интерфейс для анализа программ. Поэтому именно этот компонент Ghidra больше всего похож на пользовательский интерфейс IDA, и дальше мы сопоставим элементы интерфейса IDA с их эквивалентами в CodeBrowser.

Базовые окна и навигация

В конфигурации по умолчанию CodeBrowser является контейнером для нескольких специализированных окон, которые показывают информацию о свойствах программы. Подробное обсуждение CodeBrowser начинается в главе 5 и продолжается, включая связанные представления данных, до главы 10.

Представление Listing

В центре CodeBrowser находится окно Ghidra Listing, которое дает классическое дизассемблированное представление, похожее на IDA View в текстовом режиме. Для настройки формата листингов компонент Browser Field Formatter позволяет изменять, переставлять и удалять отдельные элементы листинга. Как и в IDA, навигация в окнах Listing в основном выполняется двойным щелчком по меткам (именам в IDA), чтобы перейти к адресу, связанному с меткой. Контекстные меню, вызываемые правой кнопкой мыши, дают доступ к распространенным операциям с метками, включая переименование и изменение типа.

Как и в IDA, каждая функция в листинге имеет заголовочный комментарий, где перечислен прототип функции, приведена сводка локальных переменных функции и показаны перекрестные ссылки, указывающие на функцию. Эквивалент представления IDA Stack view в Ghidra доступен только через правый щелчок в заголовке функции и выбор Function -> Edit Stack Frame.

Если вам нравится, что IDA подсвечивает все вхождения строки, по которой вы щелкнули (например, имя регистра или мнемонику инструкции), вас может разочаровать, что в Ghidra это не является поведением по умолчанию. Чтобы включить его, откройте Edit -> Tool Options -> Listing Fields -> Cursor Text Highlight и измените Mouse Button to Activate с MIDDLE на LEFT. Другая возможность, которую вы можете любить или ненавидеть, - Markup Register Variable References: она заставляет Ghidra автоматически переименовывать регистры, используемые для хранения входных параметров функции. Чтобы отключить это поведение и заставить Ghidra использовать имена регистров как операнды инструкций, перейдите в Edit -> Tool Options -> Listing Fields -> Operands Fields и снимите флажок Markup Register Variable References.

Наконец, если вы хотите, чтобы Ghidra "делала правильное" тогда, когда мышечная память заставляет вас вводить любимые последовательности горячих клавиш из IDA, стоит потратить время на Edit -> Tool Options -> Key Bindings и переназначить горячие клавиши Ghidra по умолчанию на те, которыми вы пользуетесь в IDA. Это настолько частая задача для пользователей IDA, что существуют сторонние файлы привязок клавиш, автоматизирующие переназначение всех любимых сочетаний клавиш.[1]

Представление Graph

Окно Ghidra Listing - это только текстовое представление. Если вы предпочитаете работать в графовом представлении IDA, в Ghidra нужно открыть отдельное окно Function Graph. Как и графовое представление IDA, окно Function Graph в Ghidra может показывать только одну функцию за раз, а элементы в Function Graph можно изменять так же, как в окне Listing.

По умолчанию алгоритм раскладки графа в Ghidra может проводить ребра за узлами базовых блоков, из-за чего их сложнее прослеживать. Это поведение можно отключить: откройте Edit -> Tool Options -> Function Graph -> Nested Code Layout и установите флажок Route Edges Around Vertices.

Декомпилятор

Ghidra включает возможность декомпиляции для всех поддерживаемых процессоров. По умолчанию окно Decompiler появляется справа от окна Listing и показывает декомпилированный исходный код C каждый раз, когда курсор находится внутри функции в представлении Listing. Если вы хотите добавлять и видеть комментарии конца строки в сгенерированном исходном коде C, включите их через Edit -> Tool Options -> Decompiler -> Display, установив Display EOL comments. На той же вкладке параметров есть Disable printing of type casts; в некоторых случаях этот параметр улучшает читаемость, заметно разгружая сгенерированный код.

Decompiler также склонен агрессивно оптимизировать генерируемый код. Если вы читаете дизассемблированную версию функции и вам кажется, что в декомпилированной версии отсутствует часть поведения, возможно, декомпилятор удалил то, что считает мертвым кодом внутри функции. Чтобы показать этот код в окне Decompiler, перейдите в Edit -> Tool Options -> Decompiler -> Analysis и снимите флажок Eliminate dead code. Decompiler подробнее обсуждается в главе 19.

Symbol Tree

Окно Symbol Tree в CodeBrowser дает иерархическое представление всех символов, содержащихся в программе. Symbol Tree содержит шесть папок верхнего уровня, представляющих шесть классов символов, которые могут существовать в программе. Щелчок по имени в любой папке Symbol Tree переносит окно Listing к соответствующему адресу:

Imports. Папка Imports важна для динамически скомпонованных бинарных файлов и содержит список внешних функций и библиотек, на которые ссылается программа. Это ближе всего соответствует вкладке Imports в IDA.

Exports. Папка Exports перечисляет символы в программе, которые публично видимы за пределами программы. Символы в этой папке часто похожи на вывод утилиты nm.

Functions. Эта папка содержит запись для каждой функции в листинге программы.

Labels. Эта папка содержит записи для любых дополнительных нелокальных меток внутри программы.

Classes. Эта папка содержит имена любых классов C++, для которых Ghidra обнаружила Runtime Type Identification (RTTI).

Namespaces. Эта папка содержит запись для каждого пространства имен, созданного Ghidra во время анализа программы. Подробнее о пространствах имен Ghidra см. Ghidra Help.

Data Type Manager

`Data Type Manager` хранит все знания Ghidra о структурах данных и прототипах функций. Каждая папка в `Data Type Manager` является приблизительным эквивалентом библиотеки типов (`.til`) в IDA. `Data Type Manager` выполняет роль окон Structures, Enums, Local Types и Type Libraries в IDA и подробно обсуждается в главе 8.

Сценарии

Ghidra реализована на Java, и ее естественный язык сценариев - Java. Помимо обычных сценариев, основные расширения Java для Ghidra включают анализаторы, плагины и загрузчики. Анализаторы и плагины Ghidra вместе выполняют роль, которую в IDA выполняют плагины, а загрузчики Ghidra по сути играют ту же роль, что загрузчики IDA. Ghidra поддерживает понятие процессорных модулей; однако процессоры Ghidra определяются с помощью языка спецификаций под названием SLEIGH.

Ghidra включает базовый редактор сценариев для повседневных задач написания сценариев, а также плагин Eclipse, облегчающий создание более сложных сценариев и расширений Ghidra. Python поддерживается через Jython. Ghidra API реализован как иерархия классов, представляющая свойства двоичного файла в виде объектов Java; кроме того, предоставляются вспомогательные классы для простого доступа к некоторым наиболее часто используемым классам API. Сценарии Ghidra обсуждаются в главах 14 и 15, а расширения - в главах 15, 17 и 18.

Итоги

Возможности Ghidra явно похожи на возможности IDA. В одних случаях отображения Ghidra настолько похожи на IDA, что замедлять вас будут только новые горячие клавиши, кнопки инструментов и меню. В других случаях информация представлена иначе, чем в IDA, и кривая обучения будет круче. В любом случае, используете ли вы возможности настройки Ghidra, чтобы она управлялась как IDA, или тратите время на изучение нового способа работы, вы, вероятно, обнаружите, что Ghidra покрывает большую часть ваших потребностей в обратной разработке, а иногда открывает совершенно новые способы выполнять работу.

Сноски

[1] [назад] Попробуйте https://github.com/enovella/ida2ghidra-kb/ или https://github.com/JeremyBlackthorne/Ghidra-Keybindings/.